THE OMNI-COMPASS OWNER'S MANUAL

The Master Manual:
Its Philosophy, Its Mathematics, Its Mechanism, Its Results, and How to Wire It onto Your Stack

AJ Dubra
Owner and Developer

THE OMNI-COMPASS LLC
October 2026

The Omni-Compass Owner's Manual

The Master Manual: Its Philosophy, Its Mathematics, Its Mechanism, Its Results, and How to Wire It onto Your Stack

October 2026

Copyright © 2026 The Omni-Compass LLC. All rights reserved. No part of this book may be reproduced, stored or transmitted in any form or by any means, except as allowed by the Omni-Compass Evaluation License, without the written permission of The Omni-Compass LLC.

The Omni-Compass LLC reserves the right to change this book, the software it describes and their terms at any time, without notice. Every copy, export and printout of any part of this book carries this page, the LICENSE, the NOTICE and the Disclosures unchanged.

All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (no filing number is stated, and no grant, registration or approval is claimed). www.omni-compass.com

“Omni-Compass”, the Omni-Compass rose and the related names and marks are trademarks of The Omni-Compass LLC. Other names that appear in this book are the property of their owners and are used only to identify the systems Omni-Compass works with.

License. This book and the software it describes are proprietary and are not open source (SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0). They may be used only to evaluate Omni-Compass and to reproduce its published results, in simulation and on systems the reader owns or controls. Any commercial use, commercialization, monetization, production use, redistribution, hosted or managed service, or incorporation into any product or service requires a written Omni-Compass Enterprise License, signed by The Omni-Compass LLC and paid for. The full terms are printed in Appendix I.

Disclaimer. The software is provided “as is”, without warranty of any kind. Every result in this book carries its evidence class. A modelled result is a statement about a model; a software benchmark is a statement about the software it ran on; only a hardware meter speaks for hardware. Nothing in this book is a promise of a particular saving on a particular system: the only number that applies to a system is the one its own paired runs produce, on its own receipt.

Set in Liberation Serif and DejaVu Sans Mono, the text at 11 pt on a 16.5 pt line. Built from the repository by `docs/book/build_book.py`; the book and the code change in the same commit.

The Omni-Compass LLC · www.omni-compass.com

Contents

The Problem and the Answer xi

Foreword xiv

Preface xvi

Prelude: A Night on the Cluster xviii

Introduction: The Idea and the Promise xx

The key: GAIN, STEADY and LOSS xxii

What You Are Looking At xxv

Quick Reference: The Card for the Glove Box xxviii

Executive Summary xxxi

Disclosures, Declarations and Disclaimers xxxvi

Part One. The Philosophy and the Theory 1

1. What Omni-Compass Is 1
2. The Referee's Summary: Each Claim, Its Evidence, and What Is Undetermined 10
3. The Referee's Grades, Category by Category, and the Plan to Raise Each 15
4. Related Work: Where the Idea Comes From, and What It Adds 20
5. The Seat: One Governor over the Muscles 23
6. The Unified Circle Principle 30
7. The Four-Piece Engine 33

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

8. The Compass . . . 35
9. Basins, Polarity and the Dual-Basin Engine . . . 36
10. The Reflex Rule: One Body, One Brain . . . 37

Part Two. The Mathematics 39

11. The Canonical Declaration . . . 40
12. The Engine: Eight Equations and One Control Law . . . 45
13. The Canonical Engine . . . 53
14. The Closed Circle: Why It Cannot Leave Its Compass . . . 57
15. Closing the Circle in the Engine . . . 58
16. The Tracking Theorem . . . 60
17. The Engine's Machine-Checked Proofs . . . 63
18. The Conveyance Law . . . 68
19. Formal Status of the Mathematics . . . 72
20. The Max, Formally: Definitions, Guarantees and Measures . . . 73
21. The Mechanism of Action . . . 80
22. The Safety Case: Every Hazard, Its Guard, and the Evidence . . . 92
23. The Threat Model: Misuse, Its Guards, and What Is Open . . . 97
24. The Static Security Scan, Every Finding Reviewed . . . 105
25. Every Dependency and Its License . . . 112
26. The Engine That Stands: Omni v23 . . . 114
27. The Engines and Their Audit . . . 116
28. Change Control: How the Engine Changes . . . 119
29. How Much of the Engine the Tests Run . . . 128

Part Three. The Physics: the Compass and the Nervous System 130

30. The Compass: Push, Pull and the Two Forces . . . 131
31. The Physics of a Processor . . . 140
32. The Two-Way Nervous System . . . 141

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

33. The Nervous System in Detail 147

Part Four. The Body: Muscles, Realms and Organisms 150

34. The Muscles, the Realms and the Seven Organisms 150
35. The Spine: the Base Every Stack Runs On 163
36. The Four Realms 166
37. The Realm Muscles 169
38. The Domain Map 184
39. The Problem Map 191
40. The Seven Organisms and the Benchmark Grid 197
41. The Metrics Catalog 199

Part Five. The Harness and the Wiring 204

42. The Plug, the Adapters and the Wire Check 204
43. The Harness 214
44. The Wiring Guide 217
45. Before You Start, and the Eight Levels 220
46. Stack by Stack 231
47. The Integration Manual 258
48. Running the GPU Benchmark 271

Part Six. Operating It 279

49. The OFF Switch, the Rules, and the Log 279
50. Maintenance, Upgrades and Security 284
51. The Operator Manual 287

Part Seven. Proving It 295

52. The Receipts, in Their Totality 296
53. How to Read the Results 306
54. The Benefit Sheet: One Number per Benchmark 319

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

55. Every Level on One Page . . . 336
56. The Dossier: Every Result in One Place . . . 350
57. Paired Runs and Receipts on Your Own System . . . 362
58. Evidence Classes and How to Read a Result . . . 367
59. The Frozen Engines and the Three-Run Rule . . . 370
60. Results to Date . . . 379
61. Wire In, or Watch: The Verdict per Knob . . . 413
62. The Pilot Protocol and Kit . . . 474
63. The GPU Bench . . . 478
64. The GPU Preregistration . . . 484
65. The Realms Preregistration . . . 499
66. The Compass Law on Real Kubernetes: Preregistration . . . 514
67. The Pod Start Wait on One Shared Host . . . 556
68. The 95th and 99th Percentiles: Where a Tail Reads Higher, and Why . . . 561
69. How Small an Effect Each Test Can See . . . 564
70. Every Change to Every Preregistration . . . 667
71. Limitations and Threats to Validity . . . 676
72. The Evidence Ledger . . . 679
73. The Claims Register . . . 688
74. Comparison with Existing Controllers . . . 704
75. The State of Play . . . 708

Part Eight. Value, License and History 716

76. Where the Value Comes From . . . 716
77. The Economics of a Receipt . . . 734
78. Due Diligence . . . 735
79. License and Commercial Terms . . . 737
80. Licensing: Questions and Answers . . . 737
81. Third-Party Notices . . . 739

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

82. The Enterprise License: Standard Terms 740

83. Patents and Trademarks 744

84. Security Policy 746

85. Repository Standards 747

86. Python, C++ and the Seal 748

87. The Founder's Working Notes 750

88. The Road to the Engine That Stands: Omni v5, v6, v7, v8, v9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21 and 22 756

89. History 829

Back Matter **839**

Glossary 839

Appendix A. Command Reference 845

Appendix B. File Map 849

Appendix C. The Equations in Full 852

Appendix D. Metrics 856

Appendix E. Troubleshooting 859

Appendix F. Evidence Map 861

Appendix G. The Muscles: What Each Is For, and How It Is Wired 865

Appendix H. Source of the Engine 1062

Appendix I. The License 1799

Appendix J. Reproducing Everything, From a Clean Machine 1812

Index 1813

Contact 1843

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

List of Figures

Every figure is drawn from the code it pictures, by one of two kits (`docs/book/figures.py`; `docs/book/schematics.py` for the wiring schematics, block diagrams, flowcharts, timing diagrams, state machines, the receipts charted from their own tables and the grids), and printed where the text first needs it; the same drawings stand beside the text on GitHub, in `docs/book/figures/`. A chart of a receipt is read by the reading key: it pictures the table and adds nothing to it.

Figure 1. Block diagram: the verdict's paired trial around a slow knob, read GAIN, STEADY or LOSS 4

Figure 2. State machine: the governor's modes 5

Figure 3. One pedal, no brake: the modes and the two switches 7

Figure 4. The compass band, its cushions, and the smoothing of the force 9

Figure 5. The engine's six states, their couplings and the control law 47

Figure 6. The double well of the alignment and the cubic potential of the stress 48

Figure 7. Block diagram: from the observations through the engine to the authority 49

Figure 8. Modelled trace: the engine's six states answering a pressure pulse 51

Figure 9. One decision of the governor, from the senses to the audit 54

Figure 10. Block diagram: the compass law as a closed loop on one muscle 131

Figure 11. Curve: the damping gain the law sets from a muscle's response time 135

Figure 12. Grid: the force over the band position and its rate of change 136

Figure 13. Modelled trace: the compass law answering a load step on one muscle 139

Figure 14. Logic diagram: the nervous system's authority to add and to give back 142

Figure 15. Flowchart: the release gate, reasons 1 to 5 144

Figure 16. Flowchart: the release gate, reasons 6 to 9 and the release 145

Figure 17. Grid: the families of the spine and of the Physics / Robotics / Autonomous realm 151

Figure 18. Grid: the families of the Compute / AI / Cloud realm 153

Figure 19. The seven organisms on the spine 155

Figure 20. Grid: the families of the Energy / Facility / Industrial realm 156

Figure 21. Grid: the families of the Distribution / Specialized realm 157

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Figure 22. The three loads: an organism's own load, the climb and the line 158
Figure 23. Grid chart: the seven organisms at every size and run count, modelled 159
Figure 24. Grid chart: the seven organisms at every size and run count, with the real cluster inside 161
Figure 25. The two-way plug: one wire out, one wire in 205
Figure 26. Flowchart: the wire check, steps 1 to 4 207
Figure 27. Flowchart: the wire check, steps 5 to 7 and the verdict 208
Figure 28. State machine: the life of one muscle under the plug 210
Figure 29. The muscle SDK's runtime: what happens to one command 211
Figure 30. Flowchart: the implementer's eight phases and their gates 222
Figure 31. Wiring schematic: Kubernetes in the cluster, levels 2 to 4 224
Figure 32. The eight levels of an installation 225
Figure 33. Flowchart: the verdict's trial on one step, from the reference to the reading 229
Figure 34. Flowchart: the verdict's trial on one step, from the reading to the step allowed or refused 230
Figure 35. Timing diagram: a give-back trial from reference to verdict 232
Figure 36. Wiring schematic: a managed cloud cluster, the whole engine at the max 235
Figure 37. Modelled trace: one pedal over a burst, machines against demand 236
Figure 38. Flowchart: parking and waking a machine in the cloud 238
Figure 39. Wiring schematic: a database behind its connection pooler 239
Figure 40. Wiring schematic: three independent simulators, each on its own native controller 241
Figure 41. Wiring schematic: a message broker's consumer group 244
Figure 42. Wiring schematic: a cache's memory ceiling 246
Figure 43. Wiring schematic: a drone swarm's cruise override 247
Figure 44. Wiring schematic: a database's storage-engine cache 249
Figure 45. Flowchart: the direction rule of a cache knob 251
Figure 46. Wiring schematic: a buffer pool resized online in the server's own chunks 253
Figure 47. Wiring schematic: the machine's frequency governor under the processor's meter 255
Figure 48. Wiring schematic: the data center under the servers 256
Figure 49. Timing diagram: the governor's clocks against the native controllers' own 280
Figure 50. Flowchart: the three ways a governor stops 282

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Figure 51. Timing diagram: the hand-back after a reset, a kill and a death 283

Figure 52. Deployment diagram: the governor on a managed cluster 285

Figure 53. Flowchart: one paired run, native against Omni-Compass on top, the pairs 363

Figure 54. Flowchart: one paired run, from the pairs to the three-run table 364

Figure 55. Flowchart: from the audit to the receipt, the table and the one number 368

Figure 56. The three-run rule: a change of 0.5% or more is a change, STEADY under it, and the runs that disagree 377

Figure 57. Flowchart: the three-run reading rule, a change of 0.5% or more a change 378

Figure 58. Wiring schematic: the response-time feed and the line 381

Figure 59. Receipt chart: Omni-Compass on top of Goldilocks and the VPA's recommender, the judged rows 383

Figure 60. Receipt chart: on top of Goldilocks, the rows shown and not judged 384

Figure 61. Receipt chart: Omni-Compass on top of KEDA's HTTP add-on, the judged rows 385

Figure 62. Receipt chart: on top of KEDA's HTTP add-on, the rows shown and not judged 385

Figure 63. Receipt chart: Omni-Compass on top of KEDA on the CPU target and requests in flight, the judged rows 386

Figure 64. Receipt chart: on top of KEDA on the CPU target and requests in flight, the rows shown and not judged 387

Figure 65. Receipt chart: Omni-Compass on top of CruiseKube, the judged rows 387

Figure 66. Receipt chart: on top of CruiseKube, the rows shown and not judged 388

Figure 67. Receipt chart: the capacity test at four caps 389

Figure 68. How a receipt came to be: the governor's inputs beside the service's outputs 390

Figure 69. Flowchart: one tick of the grid conductor 393

Figure 70. The load schedules of the cloud sets: two bursts and the real day 394

Figure 71. Receipt chart: the seven organisms under the three loads 400

Figure 72. Chart: the benefit sheet, one number per benchmark, the newer half 405

Figure 73. Chart: the benefit sheet, one number per benchmark, the older half 406

Figure 74. Wiring schematic: a graphics card, two wires 481

Figure 75. Block diagram: the two wires of a card as an antagonist pair 482

Figure 76. State machine: the two-wire card governor 483

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Figure 77. Flowchart: the model gate before any engine change runs for real 731

The Problem and the Answer

The bottom line, first: is Omni-Compass a benefit? The founder's order of 24 October 2026: the first thing anyone sees says whether Omni-Compass is a benefit, for each way to run it, against native alone, in one light and one number. The light is the whole verdict: green and round, **GAINING**, Omni-Compass ahead of native; yellow, a yield diamond, **STEADY**, STEADY (a change under 0.5%); red, a stop-sign octagon, **REDLINING**, behind native, a LOSS; dark, **AWAITING**, no run yet. A slower service or a failed request turns the light red whatever the work. The number is how much more work was done for the same demand, the mean of our three runs (one real test on GitHub's own machines, class L; `results/orchestrator/V18_THREE_ARMS_ABC.md`, engine omni-v18, commit `0ca2bbdac97f`; section 60.3):

Way to run it	Light	Work against native	Everything else the record reads
Native alone (Kubernetes' published autoscaling rule on the muscles, written by us as the yardstick)	yellow, STEADY	0%: the baseline	—
Omni-Compass on native	green, GAINING	**GAIN 6.0%** in all three of our runs (A +5.2%, B +5.3%, C +7.4%)	the 95th percentile **GAIN 9.5%** in all three; machines **GAIN 3.1%** by the mean of our three runs, beyond the noise in 1 of 3; serving units **power given 10.2%** in all three; no request failed or answered wrong
Omni-Compass direct (Omni-Compass alone on the muscles, no Kubernetes)	green, GAINING	**GAIN 7.5%** in all three of our runs (A +7.3%, B +5.7%, C +9.4%)	the 95th percentile **GAIN 12.1%** in all three; machines **GAIN 2.5%** by the mean of our three runs, beyond the noise in 0 of 3; serving units **power given 5.2%** by the mean, beyond the noise in 2 of 3; no request failed or answered wrong

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The whole workload, by the two routes it is bought by (the composition of every measured layer, a model of measurements, class S of class L and P; section 60.6): **on site GAIN 18.5%** and **through the cloud GAIN 14.8%** over the gauges, each layer's intervals carried (+18.5% and +14.5% by the index's rule). Both are the composition's index, not a price: a bill is the sum of the costs inside a declared boundary, and today only the cloud's compute is metered in money, so neither route's bill is resolved yet (section 76.8). Every real test together, the Omni index of Omni-Compass on native reads **GAIN 3.4%**, its categories from real Kubernetes **GAIN 28.6%** to the database behind its pooler **LOSS 4.9%** (section 60.11). Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

What the whole industry faces. Every computing system in service, from one server to a cloud region, from a GPU to a building's chillers, a grid feeder or a robot cell, is run by a stack of native controllers: the Kubernetes autoscalers and scheduler, the cloud's node pools, the GPU's power and clock firmware, the database's cache and connection pools, the building's cooling loops, the battery's management system, the servo's loop. Each was designed and tuned alone, against its own setpoint, with margins sized for the worst case it might meet. Three costs follow, on every stack:

1. **Capacity held idle.** Worst-case margins stay in place through the common case. Machines, power caps, caches and cooling run far inside their limits, and the room between the operating point and the limit is paid for and seldom used.
2. **Late reaction.** Reactive loops act on averaged, lagging signals and wait out deliberate cooldowns, so queues and response-time tails form before capacity arrives.
3. **Loops at cross purposes.** The loops share power, cooling, connections and memory. One loop's correction is another loop's disturbance, and no loop sees the whole.

Demand for computing, above all for AI and the cloud, is growing faster than the power, cooling and hardware that can be built for it. The usual answer is to add: more machines, more power, more people watching thresholds. Replacing the native controllers is not an answer most operators will take: they are proven, often certified, and the people on call know them.

What Omni-Compass does about it. Omni-Compass is a supervisory governor. It sits on top of those native controllers and never replaces them. It plugs into each setting a native controller already exposes, through a two-way plug (one wire in, the reading; one wire out, the setting), takes a snapshot of every setting before its first write, and governs the whole stack as one body, the organism: every muscle driven as far as its own limit and never past it, and, on the engine that stands, never below native; where it cannot bring a muscle a GAIN, that muscle stays native. It acts before a line is crossed instead of after, reads back every write, puts every give-back to its own trial, records every decision with its reason, walks every setting home if it stops (the lease) and stops at once on the kill switch. Its first rule is do no harm, judged at the organism: never less work or health than native (`DISCLOSURES.md`, section 3). The aim is the one headline: more work, faster, on fewer machines, for the same energy and the same bill, from the hardware and the controllers already in place, every benefit made at the software end; in a cloud and at the chip, Omni-Compass holds.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What the record shows so far. Every claim is a paired comparison of native and Omni-Compass on top of native, and since 23 October of Omni-Compass direct beside them, on the same machines with the same work, run three separate times (A, B and C) and read GAIN, STEADY or LOSS by what each gauge is for; every row is kept, LOSS included. On real machines (class L, live software), the combined Omni index reads +3.4%, GAIN, and the service reading alone +9.4%, GAIN, judged across our three runs by the rule of 24 October (`results/OMNI_INDEX.md`; each test read from its own engine's newest three-run table, none pooled across engines). The rows that read LOSS stand in their tables, two whole categories among them (the database behind its pooler, the storage-engine cache; section 60.11), as does the pod start wait on the kind rig (`docs/POD_START_WAIT.md`). The 1,107 muscles of the catalog are modelled (class S) and are never shown as proof of hardware. The engine is written twice, in Python and in C++, and the Python's 500 seeded runs through the C++ harness are identical to the last bit (`cpp/tools/engine_runs.cpp`; GitHub reruns the 500 on every change to the harness, the engine's C++ or the page, and every verify reruns 50 of them). Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

What is not claimed. No new physics, no new model of computation, and no promise for any system Omni-Compass was not run on. It does not make a system immune to failure, delay or drift: it keeps each muscle inside its own limits and shows, row by row, what it did against native. A result on another system is that system's own, from its own paired runs; where it differs, the first presumption is the wiring (`DISCLOSURES.md`, section 4).

What is protected, and how. Omni-Compass belongs to The Omni-Compass LLC. Its source, manuals and records are its copyrighted work; all patents, copyrights and trademarks filed in the USA. The repository is licensed for Evaluation and Simulation use only (`LICENSE`); any commercial use, monetization or production use requires a signed, paid Omni-Compass Enterprise License (`ENTERPRISE_LICENSE.md`). What a licensee acquires is the working whole: the engine, the wiring and the factory card of every muscle, the harness that measures it against native, and the receipts.

The cloud is a wire. A cloud is the wire between the software end (the owner's programs, Kubernetes and the open-source orchestrators, managers and autoscalers that carry the work) and the hardware end (the bare metal: the GPU chips, the Linux racks, the data center, on site or off site). Omni-Compass can live in all three places. On the wire alone, as every cloud run in this repository is set, it can only park machines the cloud's own autoscaler already made, so a cloud run alone can show at most a small cut in the bill of the wire; the work and speed gained at the software end and the energy and throughput gained at the bare metal are measured where Omni-Compass sits on them, and composed with the wire into one organism and one bill (`DISCLOSURES.md`, section 9; section 60.6, with the company's expected ranges beside the measured readings, an expectation and never a result).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Foreword

I built Omni-Compass to sit on top of what already exists. Kubernetes, the GPU driver, the building controller, the battery inverter, the robot's servo loop: those are muscles, and they are good muscles. What none of them has is one brain that reads all of them at once, holds each of them inside its safe range, and puts every setting back exactly where it found it when it stops. That brain is Omni-Compass.

Every machine already has a controller. Kubernetes has its autoscaler. A graphics card has its firmware. A building has its thermostat loop, a battery has its charge controller, a power grid has its tap changers, and a robot has its servo. Engineers spent decades making these controllers safe, and they are. But each one was built to watch one thing and to protect that one thing. None of them sees the whole body. None of them knows that the cooling loop is holding back the processor, or that the autoscaler is adding machines the database cannot feed. So each one plays it safe in the middle of its range, because playing it safe is its whole job. The headroom between the middle and the edge is paid for every hour and almost never used.

That gap is what I set out to close. Omni-Compass is a **supervisory governor**: one brain over the muscles you already run. It never replaces them and it never competes with them. It reads them, it decides, and it writes one setting at a time through a two-way plug: one wire in, one wire out. Before it writes anything, it takes a snapshot. When it stops, every setting goes back to that snapshot.

Why a compass. A compass always points home. Every muscle has a safe band, and the compass holds the muscle inside that band, with a cushion of 5% at each edge. It works with two forces that pull against each other, one toward more and one toward less, and it moves smoothly, never hammering. A system that is closed, bounded and pulled toward a center cannot run away. That one idea is the root of the whole engine, and the first part of this book explains it.

The rule I will not bend: do no harm. Its real goal is the organism: to strengthen it and keep it healthy, working at the best state its own muscles can carry. Whatever is fed into it, its work, its data, its energy, goes through its muscles and comes out as its output, and Omni-Compass drives it toward the most and best output it can make. Harm is the organism turning out less than its native controllers make it turn out, or made less healthy: a muscle pushed past its own limit, a service over its own line, anything made to fail. Omni-Compass never does either. It can push a muscle only as far as that muscle itself can go; it may use one muscle less, or leave it native, so that another can carry more, whenever the whole organism comes out ahead and every muscle stays healthy (`DISCLOSURES.md`, section 3). The engine that stands keeps the rule in its strictest form, a trigger in the code (`omnicompass/verdict.py`), checked on the muscle itself, step after step: a

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

step that costs the muscle more for each piece of work is undone, and where it cannot bring a muscle a gain, that muscle stays native.

The max is the default. Most tuning tools try to shave a little off. Omni-Compass rides the edge instead: every muscle runs up to its own limit, read live from the system, never past it and never below what native gives. The energy it takes to do more work is **power given**, shown beside the work, the speed and the wear it bought. It is never called a loss. Anyone can turn the dial down from the max. Nobody has to.

One pedal. I wanted it to drive like an electric car. Push the pedal for more, lift off for less. Lift off all the way and the settings drift back toward native. Reset hands everything back. If the governor ever crashes, its lease runs out and the plug walks every setting home, step by step. The kill switch is separate and instant, and it exists for security, not for daily driving.

How I want it judged. Every benchmark in this book sets native beside Omni-Compass on top of native, and, where native's own controller can be switched off, Omni-Compass direct beside them ("there's native, there's Omni and native, and then there's Omni by itself", 23 October); there is no other arm and no other name. Every result was run three separate times, under rules written down before the first run, and every row is shown, losses included. Each reading is one word, **GAIN**, **STEADY** or **LOSS**, decided by what the gauge is for. A result from our own model is labeled as a model; a result from live software is labeled as live software; only a meter speaks for hardware. Where we lost, the book says so, and we went to find out why. Nobody outside The Omni-Compass LLC has reviewed or approved these results, and every table says that too.

What this book holds. This manual is the whole of it: what the governor is, the mathematics it runs on, how its nervous system reads and writes, how it is wired to each kind of machine, how to turn it on one level at a time, how to switch it off, and how to prove on your own system what it does. Keep it with the code. When the code changes, this manual changes in the same commit.

On this edition. This edition is written for the referee and the underwriter as much as for the engineer. Every result in it was run three separate times on a frozen, fingerprinted engine, under rules written before the first run, and is reported with every loss beside every gain. Where a number is from a model it says so; where a test could not see an effect it says so; where something has not been shown it says so, in its own section. Nothing in this manual asks to be believed on our word: every table names the runs it was made from, every run's raw files are in the repository, and every table can be rebuilt from them by the tool it names.

Where to begin. Read the front matter first. It tells you what you are looking at before it asks you to touch anything. Then wire it one level at a time, watch it before you let it write, and judge it by your own receipts.

AJ Dubra Founder, The Omni-Compass LLC

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Preface

This is the Owner's Manual, the master: everything about Omni-Compass in one book, for whoever owns or licenses it and so must know the whole mechanism. I wrote it so that the owner understands what they hold: not only which wire goes where, but why the governor behaves the way it does, where its law comes from, how it is built, and how to prove on their own system what it does for them. The Operator's Manual is the shorter book for the people who run it.

The book is in eight parts, and it can be read in two ways.

You are	Read
CEO, board member, investor	the Problem and the Answer, the Foreword, the Prelude, the Introduction, the Executive Summary, Part One, the Results to Date, and Part Eight
CTO, architect, head of platform	Parts One to Four, then the eight wiring levels in Part Five, then Part Seven
The engineer wiring it	everything, in order. Do not skip the wire check or the watch level
Data-center, facility or energy operator	the Quick Reference card, Part One's chapter on the reflex, Part Five (levels five to seven, the processor and the data center under the servers), Part Six
Site reliability engineer, on call	the Quick Reference card, Part Six (the switches and the log), the troubleshooting appendix
Auditor, diligence team	Part Two (the mathematics), Part Seven (the proof), Appendices C, F and H
Procurement and counsel	the Disclosures, the license in the back matter, Part Eight

Part One, the philosophy and the theory, sets out the closed circle: a system that is closed, bounded and pulled toward a center cannot run away, and everything it does is a return. **Part Two, the mathematics,** gives the eight equations and the control law that make the circle exact, with the theorem, the declaration and the audits that hold them fixed. **Part Three, the physics,** brings the law into the machine: the compass, push and pull, the band and its cushions, the physics of a processor, and the two-way nervous system. **Part Four, the body,** lays out the muscles, the spine, the four realms and the seven organisms. **Part Five, the harness and**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the wiring, is the universal plug and the step-by-step work of wiring Omni-Compass onto a stack. **Part Six, operating it,** is the OFF switch, the rules and the log. **Part Seven, proving it,** is every rule written before a run and every result after it. **Part Eight** is value, the license, the seal and the history.

Conventions. `Code` is a command, file or switch exactly as typed. "Native" means a system as it runs today, without Omni-Compass. "Muscle" means any machine, service or controller Omni-Compass can read and set. "Knob" or "lever" is one setting on a muscle. "The band" is a knob's or a service reading's safe range. A step that writes to a system is marked **WRITES**. Every result carries its evidence class: T for a theorem, V for verification of the code, S for our own models (modelled), L for live software, P for a physical meter.

Some chapters gather documents that were written as the work went on, each at the moment its result came in. They are kept as they were written, because the record of how a result was reached is part of the proof. Where two chapters give different figures for the same thing, the later run and the State of Play govern.

The standards every page of this manual keeps. A referee or an underwriter reading this manual should expect, and will find, the following discipline throughout, because it is the discipline the repository enforces on itself:

1. **Native beside it, always.** Every benchmark is read against native, on the same machines, with the same work at the same moments: omni (Omni-Compass on native) against native, and, where it is run, Omni-Compass direct against native and against omni. Omni-Compass on native is never compared as a competitor to the controller it sits on; the direct arm switches that controller off to show what Omni-Compass does as the only controller, and is never read in place of the arm on native.
2. **Preregistered, then run.** The arms, the knob, its cover, the gauges, their directions and the readings are written down and committed before the first counted run (`docs/*_PREREGISTRATION.md`). Rules are frozen on one tuning case that is shown and never counted, then applied unchanged to untouched cases. Any amendment made after a result has been seen is written into the preregistration with its time and reason, and the table that carries the result says so.
3. **Three separate runs, one rule.** Every result is run three times as separate GitHub runs (A, B and C) on the same frozen engine and profile, and every judged row reads by one rule: the change is the mean of the three runs' changes in percent of native; a mean under 0.5% reads STEADY, under the floor (STEADY, the same where nothing moved); from 0.5% up the word comes from what the gauge is for, GAIN, LOSS or power given, **in all three of our runs** where every run moved the same way with its 95% interval clear of zero, otherwise **by the mean of our three runs** with the count of runs beyond the noise; runs that moved 0.5% or more in opposite directions read **the runs disagree**. Nobody outside The Omni-Compass LLC has reviewed or approved these results, and every three-run table says so.
4. **Every row, losses included.** A row that went against Omni-Compass is in the table beside the rows that went for it. When Omni-Compass loses, the first suspect is our own wiring, native setup or scoring; when that is fixed, the test is run again and the loss stays in the record. A benchmark may be left unpublished; it is never published with rows cut.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

5. **Nothing read across engines.** The engine is frozen and fingerprinted (`OMNI_V1.json`, `OMNI_V2.json`, `OMNI_V3.json`, `OMNI_V5.json`, `OMNI_V6.json`, `OMNI_V7.json`). Any change to a rule, gain, guard, preset or muscle makes the next version, and every result is run again on it. Every table made by rule checks the engine of each run it reads and refuses to mix them.
6. **The raw files live with the code.** Every finished run's files are copied into `run-<id>/` on the repository's branch `raw` with one SHA-256 manifest per repetition (opened by `scripts/raw_archive.sh`, the branch sits at `results/live/raw/`), and every table names the runs and the commit it was made from. Anyone with the repository can recompute every table from the files it names.
7. **Do no harm.** The organism is never left turning out less than native, nor less healthy, and every muscle works only as far as its own limit (`DISCLOSURES.md`, section 3). On the engine that stands, Omni-Compass moves a knob only where a paired trial on the muscle itself shows the muscle never behind for it (the verdict, `omnicompass/verdict.py`, with its 2% allowance). Where it cannot improve a knob, that knob stays native, and the result says "nothing for Omni to move".

Prelude: A Night on the Cluster

This is a story, not a result. It follows one night on one cluster to show, in plain order, what each part of this book does. The shop, the times and the settings are made up for the story. The measured results are in the chapters on proof, each with its evidence class.

9:00 p.m. The shop is quiet. An online store runs on Kubernetes. Its native controllers are the ones almost everybody uses. The Horizontal Pod Autoscaler watches the average processor load and adds copies of the web service (pods) when that load passes its target. The cluster autoscaler adds machines when the pods no longer fit. A careful engineer picked their settings months ago, for the worst day she could imagine, and she picked them well. Tonight the cluster runs on its floor of two machines, and every customer gets an answer in a blink.

A week earlier, Omni-Compass went on in watch mode. For seven days it read every gauge and wrote nothing. It learned how fast each pod starts, how the response time moves when the load moves, and where each setting's safe band lies. Then the team ran the wire check: every reading came from the place it claimed, every lever moved the thing it claimed, and every write was read back. Only then did they turn it to govern. Every setting it would touch was saved in a snapshot first.

9:40 p.m. The sale starts. In two minutes the requests double. The native autoscaler does exactly what it was built to do, but it is built to wait: it needs the average load to climb, then it needs a decision, then each new pod needs time to start. While it waits, customers wait. With Omni-Compass on top, the compass sees the response time slide toward the edge of its band and into the cushion. It pushes. It raises the autoscaler's floor of pods, a

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

setting the autoscaler already respects, so new pods start before the average catches up. It does not fight the autoscaler and it does not go around it. It moves one setting, through the plug, and reads it back.

10:15 p.m. The database starts to strain. More web pods mean more connections to the database, and the database has a limit of its own. A controller that watches only the web service would keep adding pods and deepen the jam. Omni-Compass sees the whole body. One body, one brain: before it takes the next step for the web service, it checks what that step would do to every other part. The step that would push the database past its own cushion is refused. Nothing is pushed into a loss to buy a gain somewhere else. The web service holds where the database can still keep up, and the queue drains instead of growing.

11:30 p.m. The rush fades. The pedal lifts. Omni-Compass does not drop everything at once, because a sudden drop is how systems stumble. It eases each setting back toward native along a smooth curve. The extra machines go away as the work goes away. The energy the cluster used during the rush shows up in the report as power given, beside the work it bought and the minutes customers did not spend waiting.

1:00 a.m. Idle. The pedal is lifted all the way. The cluster sits on its floor of two machines again, exactly where the engineer left it. Omni-Compass keeps reading every second. Staying native is its live decision, made again each second, not a verdict from some earlier run.

3:12 a.m. Something breaks. An upgrade restarts the machine that runs the governor, and Omni-Compass stops in the middle of its work. Nobody is awake. Nothing bad happens. The governor's lease runs out, and the plug walks every setting back to its snapshot, step by step. The native controllers keep running as they always have, because they never stopped. When the governor comes back, it starts from what it reads, not from what it remembers.

7:00 a.m. The morning report. On a live system, the proof does not come from a feeling that the night went well. It comes from paired runs: the same work, run once on native and once with Omni-Compass on top, and compared gauge by gauge. Every gauge gets one word. Time waited that fell reads **GAIN**. Work done that rose reads **GAIN**. A change under half of one percent reads **STEADY**, and the report says so plainly; a change of 0.5% or more shows its word. A gauge that moved the wrong way reads **LOSS**, and it stays in the report until its cause is found and fixed. The energy spent on extra work reads **power given**, never a loss.

What the night shows. Every part of that night is a chapter of this book:

- the watch mode and the wire check are in the chapters on wiring, level by level;
- the band, the cushions and the two forces are in the chapters on the compass;
- one body, one brain is the reflex rule, in the first part;
- the snapshot, the lease and the kill switch are in the chapters on operating it;
- the paired runs and the three words are in the chapters on proof.

Read the rest of the front matter before you touch a wire. It tells you what you are looking at and how to read what it tells you.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Introduction: The Idea and the Promise

The problem. Stated in brief at the front of this book ("The Problem and the Answer"); here in engineering terms. Production systems are governed by many local feedback loops: horizontal autoscalers, firmware power and clock governors, building-management loops, battery-management systems, connection pools, servo controllers. Each loop is tuned in isolation, against its own setpoint, with margins sized for a worst-case disturbance. Three costs follow, and every operator pays them:

- **Reaction latency.** Reactive loops act on lagging, averaged signals. A Kubernetes Horizontal Pod Autoscaler recomputes its desired replicas from averaged utilization on a sync period (15 seconds by default) and damps its decisions with stabilization windows; new pods then need time to schedule and start. Queues form before capacity arrives.
- **Static over-provisioning.** Worst-case margins persist through the far more common nominal regime. The operating point sits well inside the safe region, and the headroom between it and the constraint boundary is paid for and rarely used.
- **Uncoordinated coupling.** Loops share resources: power, cooling, connections, memory, network. One loop's correction is another loop's disturbance. More web replicas saturate the database's connections; more cooling pushes a power feed toward its limit.

Operators close these gaps by hand, with fixed settings and runbooks. That is safe, and it leaves the region between the nominal operating point and the safe boundary unused nearly all of the time.

The idea. Close one supervisory loop around the native loops. Omni-Compass treats each setting a native controller already exposes (a replica floor, a power cap, a clock ceiling, a temperature setpoint, a pool size) as an actuator, each native telemetry stream as a measurement, and governs the whole body from above. Every decision period it runs five steps:

1. **Sense.** It reads each muscle's own gauges through the plug: utilization, response-time percentiles, queue depth, temperature, power, state of charge.
2. **Normalize.** It places each reading in its band as a compass position, from 0 (calm) to 1 (the line), with a 5% cushion at each edge.
3. **Decide.** The compass law turns that position into one bounded increment per actuator: two antagonist forces, push and pull, each with its own gain, combined through a smooth saturating response (tanh), so the command is continuous, bounded and never chatters. The increment is clipped to the actuator's cover, the hard range nothing can leave.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

4. **Actuate.** It writes the increment through the plug, idempotently, and reads the value back to confirm the write landed.
5. **Verify.** The verdict compares the muscle with and without the step in a paired trial on the muscle itself, judged on the muscle's own cost per unit of work; the body guard checks every coupled muscle against its own cushion. A step that fails either test is reverted.

What Omni-Compass is not.

- It is **not a replacement** for your controllers. Native keeps running underneath, always. Omni-Compass only moves settings that native already exposes and respects.
- It is **not a black box**. Every decision is written to a log, with the reading that caused it and the setting it changed.
- It is **not a one-time tuning script**. It decides again every second, and staying native is one of its decisions.
- It is **not a cost-cutter first**. It gives power where the power buys work or speed, and it shows that power beside what it bought.

The promise, in three parts.

1. **Do no harm.** The organism is never left turning out less than native, nor less healthy, and no muscle is pushed past its own limit. On the engine that stands, a step stays only where the muscle takes no LOSS from it, measured on the muscle itself; where Omni-Compass cannot bring a gain, the muscle stays native.
2. **Everything comes back.** Every setting is saved before the first write and restored when Omni-Compass stops. If the governor crashes, its lease runs out and the settings walk home on their own. A separate kill switch stops everything at once.
3. **Every claim is checked.** Each benchmark compares two arms, native and Omni-Compass on top of native, on the same machines with the same work. Each runs three separate times. Each reading is one word, GAIN, STEADY or LOSS, decided by what the gauge is for. Each result carries its evidence class: **P** for a physical meter, **L** for live software, **S** for our own models. A model is never shown as proof of hardware.

What it governs. The catalog names 1,107 muscles in 70 families, the settings that operators everywhere already turn by hand: machine pools and replicas, power caps and clock limits, cache sizes and connection pools, cooling setpoints and battery reserves, robot speeds and drone-swarm settings, and many more. They are grouped into four realms (compute, AI and cloud; physics, robotics and autonomy; energy, facility and industry; distribution and specialized systems) and run together as seven organisms, from the spine alone (the muscles every stack runs on) and one realm alone up to the whole tower at once. The muscles in the catalog are modelled (class S). The real controllers it has been run on top of, such as Kubernetes and the databases, are live software (class L). The chapter that follows the key, "What You Are Looking At," shows which is which. Appendix G lists every one of the 1,107 muscles: what it is, what Omni-Compass reads from it, what it moves, the organisms it belongs to, and its own card, the factory setting it runs on; and for each of the 70 families,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

how to wire it on any stack: the native controller that owns it, the wire out (what is read, and where), the wire in (what is written, and where), the restore, and whether that wire has run live in this repository or only on a model (`docs/wiring_recipes.csv`).

What you need before you wire it.

- **A license for any use beyond evaluation.** Production use requires a signed, paid Omni-Compass Enterprise License.
- **Read access first.** Omni-Compass starts in watch mode: it reads and computes, and writes nothing.
- **The wire check.** Every reading must come from where it claims, every lever must move what it claims, and every write must read back. Nothing writes until the check passes.
- **Your own service line.** The response time or other limit your service must meet comes from your own service objectives, not from ours.
- **Paired runs on your own system.** The only number that applies to your system is the one your own paired runs produce, on your own receipt.

How to read what follows. Next comes the key to the three words, then "What You Are Looking At," which shows what a hooked-up system looks like and how to read its report. The Quick Reference card is the page to keep beside the system. The Executive Summary gives the results in brief, and the Disclosures make every declaration once. After that the book turns to the theory, the mathematics, the physics, the body, and only then to the wiring, the operation and the proof. Read in that order, nothing in the wiring chapters should surprise you.

The key: GAIN, STEADY and LOSS

Every judged reading in Omni-Compass is one of three words: **GAIN**, **STEADY** or **LOSS**. Better, same and worse are not used anywhere. The word is decided by what the gauge is for, never by the plus or minus sign of its number: who sets the aim differs from one system to the next, and each gauge's aim is written in its preregistration before the runs.

Word	What each letter stands for	Its sense in underwriting	What it means in a table
GAIN	**G** Growth (asset appreciation); **A** Advantage (risk reduction); **I** Improvement (margin expansion); **N** Net-positive (profitable outcome)	surplus and positive variance	the gauge moved toward its declared aim against native: more work done, a shorter wait, fewer misses or failures, or a resource cut

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Word	What each letter stands for	Its sense in underwriting	What it means in a table
STEADY	**S** Stable (zero variance); **T** True to baseline (expected norm); **E** Equilibrium (balanced risk); **A** At par (no net change); **D** Drift-free (within the 0.5% floor); **Y** Yield held (nothing lost)	the par baseline: equilibrium	native and Omni-Compass within 0.5% of each other: the mean of the three runs' changes under the 0.5% floor, or no change at all; a knob native already runs at the top of its range is STEADY, at its limit on native
LOSS	**L** Leakage (capital drain); **O** Off-track (adverse selection); **S** Setback (claim liability); **S** Subpar (negative variance)	actuarial deficit and exposure	the gauge moved away from its declared aim: less work done, a longer wait, more misses or failures

How the sign of a number becomes a word:

- **Time waited, missed or failed** (a response time, time over the line, failed requests): a fall is GAIN, a rise is LOSS.
- **Work done** (work inside the line, throughput, a hit rate): a rise is GAIN, a fall is LOSS.
- **A resource spent or held** (energy, the power drawn and its peaks and swings, machines, memory, processor time, a bill, wear on a part): a fall is a **cut**, and a cut is GAIN. A rise is **power given**: what Omni put in to buy the work and the speed shown beside it. Power given is always shown and never hidden, and it is never called a loss (the founder's order of 12 October 2026: the max rides every muscle's edge; it never shaves). A peak is power held, so a peak that rose is power given, in a data centre and in a building alike (17 October).
- **Work bought with each unit of a resource** (work per energy): a rise is a cut in what each unit of work took, a GAIN; a fall is power given, because it falls when more power is given for the same work.
- So a minus can read GAIN (a response time cut 65%, a bill cut 30%) and a plus can read LOSS (failed requests up).
- **The floor, and every change above it** (the founder's orders of 20 and 24 October 2026): a change of half of one percent (0.5%) or more either way is a change, and reads **GAIN** or **LOSS** (or **power given**) by what the gauge is for; STEADY is kept for a change under 0.5%. The change is the mean of the three runs' changes, each in percent of native. Where every run moved the same way with its 95% interval clear of zero, the word reads **in all three of our runs**; otherwise it reads **by the mean of our three runs** and says in how many of the three runs it stood beyond the noise, so a reader sees how firmly it stands. Where runs moved 0.5% or more in opposite directions, the reading is **the runs disagree**. A mean under 0.5% reads **STEADY, under the floor**, with its size.
- **Two numbers, never mixed** (the referee's work order of 25 October): every row shows native's value and Omni-Compass's, the **raw change** (Omni-Compass − native) / native with its true sign, + for a rise and −

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

for a fall whatever the gauge is for, labelled raw, and apart from it the word with its size and no sign. A bill cut 3% reads raw −3.0%, GAIN 3.0%; failed requests up 2% read raw +2.0%, LOSS 2.0%; a pod start wait from 0.22 s to 0.83 s reads raw +275.0%, LOSS 275.0%; a change under the 0.5% floor reads STEADY 0.3%. No raw sign is ever turned to match a word.

Some preregistrations name their outcome with a label written before the runs (for example SUPERIOR WITHIN GUARDRAILS in the card protocol). The label stays as it was frozen, and every table shows it with its word first: TOWARD ITS AIMS reads GAIN; BOTH WAYS reads GAIN AND LOSS; AWAY FROM ITS AIMS reads LOSS; SAME AS NATIVE reads STEADY (the same as native); AT ITS LIMIT ON NATIVE reads STEADY (at its limit on native); SUPERIOR WITHIN GUARDRAILS reads GAIN (within the guardrails); ENERGY IMPROVEMENT WITH SERVICE TRADEOFF reads GAIN AND LOSS (energy cut, service traded for it); SERVICE IMPROVEMENT WITH ENERGY TRADEOFF reads GAIN, power given (service GAIN, energy given for it); NONINFERIOR / INCONCLUSIVE reads STEADY (inside the noise, within the guardrails); NOT ESTABLISHED reads STEADY (not established: inside the noise, a guardrail crossed on the mean). A run that breaks its own validity checks reads INVALID and carries no word.

Every reading is one of these. **GAIN, LOSS or power given in all three of our runs**: A, B and C moved the same way, every 95% interval clear of zero. **GAIN, LOSS or power given by the mean of our three runs**: the runs' mean moved 0.5% or more, with the count of runs that stood beyond the noise beside it. **STEADY, under the floor**: the mean moved less than 0.5%. **STEADY, the same**: no change at all. **The runs disagree**: runs moved 0.5% or more in opposite directions, so the test is unstable there and is looked into. **Shown, not judged**: a gauge declared in advance as shown only, such as the knob's own setting.

Why these three words: they are the words of underwriting and actuarial work, where every outcome is graded as surplus, par or deficit, and each letter spells out the same rule. Omni-Compass is there to add (GAIN), never to take (LOSS), and to hand a muscle back to its own native setting where it cannot add (STEADY). Its first rule is do no harm, in the founder's meaning of 19 October 2026: the organism Omni-Compass governs is never left turning out less than its native controllers make it turn out, nor less healthy, and every muscle works only as far as its own limit (`DISCLOSURES.md`, section 3). So **LOSS should never appear**: where it does, the row stays in the table, its cause is found and fixed (our own wiring first, then native's setup, then the scoring) and the test is run again.

Disclosure and disclaimer: GAIN, STEADY and LOSS, and the letters behind them, are The Omni-Compass LLC's own convention for reading its own results. The underwriting senses explain the words; they are not an actuarial, financial or insurance opinion, and no outside body defined, reviewed or approved them. A reading states what our own runs measured against native under the declared aims, not a promise about any other system; every number stands beside its word, so a reader can judge it by any other convention. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What You Are Looking At

What Omni-Compass is. Omni-Compass is a supervisory governor. It is never a controller of its own and never a replacement for one: it sits on top of the controllers a system already runs (the Kubernetes autoscaler, a database's connection pool, a GPU's firmware, a building's battery controller, a power grid's tap changer, a robot's servo) and moves only the settings those controllers already expose to an operator, through one wire in (what it reads) and one wire out (what it writes). Everything the native controller does, it still does. Pull the plug and every setting walks back to where the operator left it (the lease); the kill switch returns them at once.

What "native" means. Native is the same stack, the same muscles, the same machines and the same work, run the way everybody runs it today: the shipped controllers at their shipped or operator settings. Every benchmark sets omni beside native, where omni (Omni-Compass on native) is that same native stack with Omni-Compass on top; since the founder's order of 23 October a third arm runs beside them wherever a muscle's native is a controller that can be switched off, **Omni-Compass direct**, the same stack with that controller off and Omni-Compass the only controller moving the knob. The three are always shown together, each pair read on its own (section 1.1). Nothing else differs between the arms, so whatever differs in the result is what Omni-Compass did.

What a muscle and an organism are. A muscle is one setting a native controller exposes (a pool's size, a power cap, a clock ceiling, a replica count, a battery's charge rate) together with the gauges that show what it does. Most companies build deep control for one field, and a fleet manager such as Meta's Twine schedules one kind of work (containers) over a very large fleet. Omni-Compass's proposition is different in kind: cross-sector governance, one governor over the shared functional needs of otherwise separate realms. An organism is a stack of muscles from many sectors governed as one body: the four realms (Compute / AI / Cloud; Physics / Robotics / Autonomous; Energy / Facility / Industrial; Distribution / Specialized), the four stacked together, and the whole tower of 1,107 distinct muscles in 70 families. The benchmark of an organism tests what happens when those muscles share one governor and interact.

What has been measured, and how. The breadth of the map and the breadth of the evidence are stated separately:

- **1,107 muscles are catalogued** (`realms/catalog_v5.csv`). Each has a plant model, our own model of that kind of system: evidence class **S** until the muscle is wired to real software or hardware and measured.
- **26 knobs are measured on real, unmodified software** (class **L**): real Kubernetes (9), PostgreSQL behind PgBouncer (3), MySQL (4), MongoDB (4), Apache Kafka (3) and Redis (3), each A, B and C.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **39 knobs are measured inside independent, published simulators** (class **S**, but not our own models): power grids (pandapower with SimBench), buildings and batteries (CityLearn), robot arms (MuJoCo Menagerie), drone swarms (gym-pybullet-drones).
- **Organisms with a real cluster inside**: the modelled organism (S) drives a real Kubernetes cluster (L). "100 copies" means a hundred copies of the modelled organism around one real cluster, not a hundred physical clusters; the number of real machines is always stated beside it.
- **A physical meter** (class **P**) speaks for hardware energy; the GPU's own meter and the processor's RAPL counters are the meters used. Modelled energy is never presented as metered.

What you see when it is hooked up. Every result arrives as a receipt with the same shape: the two arms side by side, every gauge of the system (the work done, the response times, the misses and failures, the machines and energy spent), the paired change with its 95% interval, and one word per gauge, GAIN, STEADY or LOSS (the key, above). A three-run table repeats the test three times, A, B and C, on the same frozen engine and profile, and reads each row by one rule: the change is the mean of the three runs' changes in percent of native; a mean under 0.5% reads STEADY, under the floor (STEADY, the same where nothing moved); from 0.5% up the word comes from what the gauge is for, GAIN, LOSS or power given, **in all three of our runs** where every run moved the same way with its 95% interval clear of zero, otherwise **by the mean of our three runs** with the count of runs beyond the noise; runs that moved 0.5% or more in opposite directions read **the runs disagree** (Chapter 59). Over a benchmark sit three summaries: the Omni index (one number for all real tests together), the benefit sheet (one number and one word per benchmark) and the wiring page (for each knob: write, watch, or the operator's choice).

How to read a report, in order.

1. **The engine and the evidence class.** The header names the engine fingerprint (`python3 tools/omni_version.py`) and the class (P, L or S). A result is never read across engines, and S is never read as proof about real equipment.
2. **The work and the service rows first**: work done, response times, time over the line, failed requests. These say what the system delivered. A GAIN here is the point of the governor; a LOSS here is the first thing to look for.
3. **Then the resource rows**: machines, energy, memory, connections. A cut is a GAIN. A rise is **power given**: what Omni-Compass put in to buy the work and speed above it. Read the two together: power given that bought nothing is the cost to fix.
4. **Then the floor and the noise.** STEADY, a mean change under 0.5% (under the floor) or none at all, is a result, not a failure. A change of 0.5% or more is a change and shows its word, with the count of runs beyond the noise beside it whenever not all three are. The runs disagree means runs that moved 0.5% or more pointed different ways: the test is unstable there and is looked into.
5. **Then the clock and the machine.** Where the test machine itself is the limit (its processor saturated, native alone failing a large share of requests, a run behind its window's clock), the report says so; a test

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

machine's limit is never reported as the governor's result, and the test is rebuilt so the machine no longer decides it.

6. **Every row is shown**, LOSS included. Nothing is cut from a table because it went against Omni-Compass.

How the brain tells GAIN from LOSS. The governor does not read a plus or a minus sign as good or bad. Before it moves anything, every gauge it reads is declared with its aim, and the aim decides the word: for time waited, missed or failed (a response time, time over the line, failed requests) a fall is GAIN; for work done (work inside the line, throughput, missions flown) a rise is GAIN; for a resource spent or held (energy, machines, memory, a bill, wear) a fall is a cut, and a cut is GAIN, while a rise is power given, kept only when it buys work or speed shown beside it. Less energy for the same work is GAIN; more speed for the same machines is GAIN; more work for the same energy is GAIN. In the engine this is done in two places. The verdict (`omnicompass/verdict.py`) lets a muscle's setting move only between native and the deepest step a paired trial on that muscle has shown costs its own work nothing (each piece of work no slower, within the measurement's resolution); a step that costs more is undone at once and not tried again for a while, and where no step passes the muscle stays native, which is STEADY. Inside an organism, the body's guard (the reflex rule, `docs/FORMAL_V5.md`) then refuses any step that pushes another part of the body past its cushion, so no GAIN in one muscle is bought with a LOSS in another.

Wiring it: who, what, when, where, why and how. Read this before the wiring chapters; a governor wired without it may run and still not be the mechanism.

- **Why.** To get more work, faster, with fewer misses, from the machines and the energy already paid for, by letting one brain move the settings native controllers already expose, in the direction each gauge's aim asks, and only while no other gauge pays for it. If nothing can be gained, the purpose is met by leaving native alone.
- **What.** Two things per muscle and nothing else: one wire in (the gauges that show the muscle's state and its work, read live from the system, never typed into code) and one wire out (the one setting native already exposes to an operator, with its hard range read from the system). The brain, the compass, the verdict and the lease are in the engine; the wiring never re-implements them.
- **Where.** On top of the native controller, never in place of it: on the autoscaler's target and ceiling, not on the pods; on the pool's size, not on the queries; on the card's power limit, not on its firmware. Every muscle of the stack is wired into the same brain, so the body is judged as one.
- **Who.** The operator who owns the system grants each wire, decides which knobs Omni may write and which it only watches, and keeps the kill switch. The governor writes only what it was granted; the native controller keeps doing its own job.
- **When.** Watch first: the governor reads every wire and writes nothing until the wire check passes and paired runs on the operator's own system show the receipt. Then one level at a time (the eight levels, Part Five). It reads and decides once a second, all the time; staying native is a live decision, made again every second, never a setting left on.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **How.** The plug snapshots each setting once, at the start; the brain reads every gauge, places each muscle on its compass, tries one step on one muscle at a time inside a body, keeps it only if the verdict shows a GAIN and no LOSS, and hands every setting back to the snapshot on reset, on the lease after a crash, or at once on the kill switch. The proof is the receipt: native and Omni on the same work, every gauge, three runs, every row shown.

What the end result is. A system fully hooked up runs native exactly as before, with one governor reading every gauge of every muscle once a second and moving, muscle by muscle, only where a paired trial has shown a GAIN with no LOSS anywhere in the body; where it cannot help, the muscle stays native, which is STEADY, not a failure. The intended outcome, stated as the claim the receipts test: more work, faster, with fewer misses, on the same or fewer machines, with every unit of power given shown beside what it bought. Whether a given system gets there is answered only by its own receipts.

Quick Reference: The Card for the Glove Box

Keep this page where the people who run the system can reach it, the way a driver keeps the card in the glove box. Every line is explained in the section named beside it; nothing on this page replaces those sections.

What it is. A supervisory governor on top of the controllers you already run. It never replaces them: it moves only settings they already accept, inside a band you set, and hands every setting back when it stops (section 1.1 and chapter 42).

The three arms. *Native* is your system as it runs on its own. *Omni*, Omni-Compass on native, is the same system with Omni-Compass on top. *Omni-Compass direct* is the same system with each knob's native controller switched off and Omni-Compass the only controller moving it; never in a cloud without the owner's word. There is no other arm and no other name (section 1.1).

The profile and the two restoring controls (section 76.8). Start on the factory profile; every change makes a custom profile, its results its own. **Restore factory profile** reapplies the released configuration. **Return to native** is the reset: every setting handed back to the snapshot, read back, the governor stopped. The kill switch is separate, for security only, and instant.

The modes (section 1.3).

Mode	What it does	When
Off (manual)	nothing; native runs alone	before installing, and whenever you choose

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Mode	What it does	When
Watching	reads every gauge and logs what it would do; writes nothing	the first level of every installation (level 1)
Autopilot	one pedal: push for more, lift off for less, idle (lifted all the way) and reset, by its law	after the watch level has passed
Cruise	machines in service up to the pool's own top while a known queue waits	batch work

One pedal, like an electric car's. *Push*: capacity added at once as traffic climbs, never held back. *Lift off*: capacity eased off a step at a time, handed back toward native, never below idle. *Idle*: the pedal lifted all the way, the floor of two machines in service, ready for the next burst. *Reset*: every setting handed back to where native had it, read back, and the record removed.

The ways it stops, and what each leaves behind (Chapter 49).

You want to, or it happens that	Do this	What is left
a run ends	nothing: the reset runs by itself at the end of every run	every setting at the operator's value, read back
stop everything at once, for any security reason	`python3 tools/omni_switch.py off`	every governor hands back and exits; none starts again until `python3 tools/omni_switch.py on`
stop one muscle	its kill file, or `OMNI_KILL=1`	that muscle handed back; the others run on
Omni-Compass dies without handing back (killed, crashed, hung)	nothing: the watchdog's lease hands back for it	every setting back at the operator's value, measured at 7 to 11 seconds in 30 of 30 repetitions (section 76.5)

The eight levels of an installation (section 45.3). Take them in order; go to the next only when the pass holds.

Level	What it touches	Pass
0	nothing: `python3 verify.py`	`VERIFICATION: PASS`
1	nothing: watch mode, with a read-only identity	zero writes, and readings your operators agree with

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Level	What it touches	Pass
2	the pods, through your autoscalers (**WRITES**)	p95, p99 and failed requests never behind native, over paired runs
3	the machines, through the release gate and the verdict (**WRITES**)	fewer machines in service, with no LOSS on any service gauge
4	the replica decision itself (an option, not a step)	as level 3
5	a GPU box, two wires: the clock ceiling and the power limit (**WRITES**)	work per energy up or equal; no request more than 2% slower at the median, p95 or p99
6	the processor's clock and power on bare metal (**WRITES**)	energy per unit of work down, no LOSS on any service gauge
7	site power and the supply-air setpoint (**WRITES**)	the site meter's energy down, every rack inlet inside its band, no LOSS on any service gauge (section 46.11)

Before any level that writes: run the wire check (section 42.4), start in watch, and keep the master switch in reach.

Warning signs, and the first thing to do.

You see	First	Then read
a service slower or failing more with Omni than without	`python3 tools/omni_switch.py off`: the system is native again	section 42.5, wired right or wired wrong
a setting at a value Omni-Compass did not write	nothing: it stops writing that lever by itself (one writer)	section 32.2
a meter that is blind or late	nothing: blind means hold; it moves nothing it cannot see	section 32.3
your results point the other way from the published ones	presume the wiring first, not the law	`DISCLOSURES.md`, section 4

Who to call. Licensing, pilots and the Omni-Compass Enterprise License: The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com. Security reports: the process in `SECURITY.md`, never a public issue.

What the letters of the number mean. Read in order, DEX-YLL-1 is the way the engine runs:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

DEX	YLL	1
Data Exchange: the input. In software, data exchange is how one system's readings reach another. Here it is the two-way plug, one wire in and one wire out: the system's own readings come in, Omni-Compass's settings go out.	**Yield Limit Line**: the boundary. In engineering science, the yield limit is the most load a material can carry and still spring back unharmed; past it, the damage is permanent. Omni-Compass runs every muscle up to its own yield limit line, read live, and never past it.	**1**: the output. In code, 1 is true, the finished state; in science, 1 is unity, the whole. Here it is one closed loop: one body, one brain, every result read back and the next step decided from it.

Read as one sentence: Omni-Compass takes the system in through a Data Exchange (DEX), runs every setting up to its Yield Limit Line (YLL) and never past it, and closes it all into one loop (1). Science and software people read each part on sight.

Executive Summary

What it is. Omni-Compass is a supervisory governor. It sits on top of the controller a system already has, the Kubernetes autoscaler, the GPU's firmware, the cooling loop, the battery inverter, the robot's servo, the grid's tap changer, the database's connection pooler, the message broker's consumer group, the cache's memory ceiling, and it moves only the settings those controllers already accept. It reads each muscle's own meter, computes one bounded force per muscle from a single smooth law, and holds the service in the middle of its safe band rather than far below its limit. The native controller keeps running exactly as shipped; Omni-Compass sets what it already exposes, and when Omni-Compass stops, every setting goes back to the value it read once before its first write. It is never standalone and it never competes with the controller underneath it.

What it does for you. Every running system keeps room "just in case": machines sized for the worst minute of the day, a pool of connections held open for a load that comes once an hour, a consumer group sized for the quiet hours and then overrun at the peak, a cache ceiling set once and left. By default Omni-Compass turns that room into more work, faster: the max, every muscle run up to its own limit, read live, never past it and never below native, the energy it takes shown as power given beside the work and speed it bought. On the dial, anyone can tune it down to the same work with less. The two are the same measurement read from two sides (section 76.4): a third more work from what is already installed is a quarter off the bill.

Where things stand, read from the result files. Written by `python3 tools/summary.py --fix` from the files it names; `verify.py` fails a push in which this block no longer matches them. Nothing in it is

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

typed by hand. The live results run on the engines named in their own tables, and every one of them runs again on the newest engine.

- **The engine on this checkout:** omni-v23, fingerprint c96ea4105d8747db (65 files).
- The muscles and the seven organisms on engine omni-v8 at the max (evidence class S, our own models; run 2026-10-17 15:39 UTC, commit 9819a903cba3, 1107 muscles; source: results/realms_v5/REALMS_V5.md):
  - every muscle alone, over its loads: STEADY 577, GAIN 530
  - the organisms under the normal load: the spine: GAIN (fewer misses), misses down 0.145 pp, power given +0.011%; Compute / AI / Cloud: GAIN (fewer misses), misses down 0.070 pp, power given +0.009%; Physics / Robotics / Autonomous: GAIN (fewer misses), misses down 0.121 pp, power given +0.010%; Energy / Facility / Industrial: GAIN (fewer misses), misses down 0.118 pp, energy the same; Distribution / Specialized: GAIN (fewer misses), misses down 0.067 pp, power given +0.002%; the four stacked: GAIN (fewer misses), misses down 0.092 pp, energy the same; the whole tower: GAIN (fewer misses), misses down 0.051 pp, energy the same
  - the organisms under the climb load: the spine: GAIN (fewer misses), misses down 0.632 pp, power given +0.039%; Compute / AI / Cloud: GAIN (fewer misses), misses down 0.390 pp, power given +0.046%; Physics / Robotics / Autonomous: GAIN (fewer misses), misses down 0.317 pp, power given +0.021%; Energy / Facility / Industrial: GAIN (fewer misses), misses down 0.217 pp, energy the same; Distribution / Specialized: GAIN (fewer misses), misses down 0.460 pp, power given +0.026%; the four stacked: GAIN (fewer misses), misses down 0.349 pp, energy the same; the whole tower: GAIN (fewer misses), misses down 0.094 pp, energy the same
  - the organisms under the line load: the spine: GAIN (fewer misses), misses down 0.371 pp, power given +0.038%; Compute / AI / Cloud: GAIN (fewer misses), misses down 0.205 pp, power given +0.045%; Physics / Robotics / Autonomous: GAIN (fewer misses), misses down 0.250 pp, power given +0.030%; Energy / Facility / Industrial: GAIN (fewer misses), misses down 0.117 pp, energy the same; Distribution / Specialized: GAIN (fewer misses), misses down 0.142 pp, power given +0.006%; the four stacked: GAIN (fewer misses), misses down 0.176 pp, energy the same; the whole tower: GAIN (fewer misses), misses down 0.051 pp, energy the same
- The grid (source: results/grid_v5/GRID.md): Across the grid: modelled, 36 of 36 squares of 10 and 100 runs GAIN (fewer misses); the 18 single runs: 18 with fewer misses; work +0.000%; power given +0.000% to +0.065%; with the cluster inside, 54 squares: 18 single repetitions (no reading); of the 36 of 10 and 100 repetitions, by gauge: "organism time over the line (% of steps)" GAIN in 36, LOSS in 0, "time over the response line (% of samples)" GAIN in 36, LOSS in 0, "response time (ms), 95th percentile" GAIN in 26, LOSS in 2, "energy (Wh)" GAIN in 26, power given in 0, "energy, parked workers still on at idle power (Wh)" GAIN in 26, power given in 0, "organism energy (J)" GAIN in 0, power given in 22, "organism work per energy" GAIN in 0, power given in 22, "failed requests (%)" GAIN in 18, LOSS in 0, "response time (ms), 99th percentile" GAIN in 12, LOSS in 2, "HPA replicas,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

mean" GAIN in 4, power given in 5, "pods started" GAIN in 3, power given in 0; off the clock in 3; invalid in 0.

- The Omni index (evidence class L, real software on real computers: the software end, not bare metal and not the chip; every measure judged across our three runs; the three-run tables of omni-v21, each named in its table): 5.0% GAIN on the software end's reading (work, speed and machines; energy shown, never scored where no meter read it), 12.9% GAIN on the service reading (work and speed alone); the reading declared on 4 October, the modelled energy scored too: 5.2% GAIN. Source: results/OMNI_INDEX.md.
  - Real Kubernetes (GitHub): 28.0% GAIN
  - Real database (PostgreSQL behind PgBouncer, GitHub): 4.2% GAIN
  - Real database buffer pool (MySQL under sysbench, GitHub): 2.2% GAIN
  - Real cache (Redis, GitHub): 0.1% GAIN
  - Real database cache (MongoDB under YCSB, GitHub): 0.5% LOSS
  - Real messaging (Apache Kafka, GitHub): 1.2% power given

On the engine that stands (omni-v23): 0 of 26 benchmarks so far. Every other row is that engine's own result, named in its Engine column, and runs again on the engine that stands (`scripts/reruns.txt`); results are never read across engines.

Benchmark	Engine	Its one number	Its word	Class
Every real test together: the Omni index	omni-v21	5.0%	GAIN AND LOSS	L
The six organisms with the real cluster inside, 10 and 100 copies	omni-v3	5.3%	GAIN, power given	L, the cluster; S, the organism around it
The big organisms with the real cluster inside, 1,000 copies on Azure	omni-v3	155.0%	GAIN	L, the cluster; S, the organism around it
The 945 modelled muscles as one tower, every muscle written at once	omni-v3	0.3%	GAIN	S
Robustness: the two-hour run	omni-v21	3.8%	GAIN AND LOSS	L
Robustness: the governor killed outright mid-run	omni-v21	8.1%	GAIN AND LOSS	L
Robot arms (MuJoCo), the three untouched robots	omni-v21	3.8%	GAIN	S
Redis, the cache's memory ceiling	omni-v21	0.1%	GAIN AND LOSS	L

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Benchmark	Engine	Its one number	Its word	Class
Real Kubernetes, steady work	omni-v21	30.9%	GAIN	L
Real Kubernetes, faults	omni-v21	30.0%	GAIN, power given	L
Real Kubernetes, demand that wanders	omni-v21	33.6%	GAIN, power given	L
Real Kubernetes, all four at once	omni-v21	33.5%	GAIN AND LOSS	L
Real Kubernetes, a service rightsized by Goldilocks and the VPA's recommender, Omni-Compass on top, ten pairs, three runs	omni-v21	34.7%	GAIN AND LOSS	L
Real Kubernetes, a service rightsized by CruiseKube applying its own recommendations, Omni-Compass on top, ten pairs, three runs	omni-v21	0.4%	LOSS	L
Real Kubernetes, a queue of jobs	omni-v21	9.9%	GAIN AND LOSS	L
Real Kubernetes, a public day of demand (Google 2011)	omni-v21	37.6%	GAIN AND LOSS	L
Real Kubernetes, a noisy neighbour	omni-v21	2.2%	GAIN AND LOSS	L
Real Kubernetes, With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass on top, ten pairs, three runs	omni-v21	94.7%	GAIN, power given	L
Real Kubernetes, With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs	omni-v21	29.8%	GAIN AND LOSS	L
Power grids (SimBench), both load models	omni-v21	5.8%	GAIN, power given	S

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Benchmark	Engine	Its one number	Its word	Class
PostgreSQL, the pooler's pool size	omni-v21	4.2%	GAIN, power given	L
MySQL, the buffer pool's size	omni-v21	2.2%	GAIN, power given	L
MongoDB, the storage engine's cache size	omni-v21	0.5%	GAIN AND LOSS	L
Kafka, the consumer group's size	omni-v21	1.2%	GAIN AND LOSS	L
Drone swarms, the three untouched cells	omni-v21	3.0%	GAIN	S
Buildings and batteries (CityLearn), the districts with batteries	omni-v21	1.5%	GAIN	S

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

What has not been shown, said plainly. An energy or cloud-bill saving on real machines. Energy on kind is a declared model, because the machines are containers on one runner. Azure's bill on a 4-worker fleet read no difference beyond the noise on every gauge, which is the honest reading of a fleet too small to show one machine; the 40-worker fleet that can show one is preregistered and has been refused by Azure's own cluster capacity in its region, each refusal recorded. Every earlier GPU result ran on a controller since replaced and is obsolete; the card runs again on rented hardware at one named commit. Each of these is an open item in Chapter 60, not a footnote.

How it stays safe. It watches before it writes. It records every original setting before it acts and reads back every write. It never writes past a knob's cover. It gives everything back the moment service crosses the wall or a sense goes blind. It stops writing a knob the instant anyone else touches it. It moves a slow knob only after a paired trial on the muscle itself shows no more than 2% cost. One OFF switch returns every setting to its original value; a watchdog with a lease does the same if the governor is killed outright; and a separate kill switch, for security only, turns the whole harness off in one human hand.

How you know. Every run produces receipts: native against omni on the same system, the same load and the same clock, each gauge with its 95% interval, the raw files archived beside the code and fingerprinted. Results are labelled by a rule written before the run, read across all three of our runs by the one key (a change of 0.5% or more is a change), and reported with every loss in the table.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Disclosures, Declarations and Disclaimers

This page is the one place every declaration about Omni-Compass is made. The README, the manual, the book, every guide and every report point here. Where any other page seems to say more than this page, this page governs.

Rights

1. Omni-Compass, its engine, its mathematics, its software and its documentation are the property of The Omni-Compass LLC. All patents, copyrights and trademarks covering them have been filed in the United States by The Omni-Compass LLC. No filing number is stated here, and no grant, registration or approval is claimed.
2. The software is licensed for evaluation and simulation only (`LICENSE`). Running it in production, on any system beyond evaluation, or in any product or service, requires a signed, paid Omni-Compass Enterprise License.
3. Names of other companies and products (NVIDIA, Kubernetes, Red Hat OpenShift, Amazon EKS, Google GKE, Microsoft AKS, Karpenter, Lambda and others) are the property of their owners. They identify the systems Omni-Compass was tested with or connects to. No affiliation, endorsement or certification by any of them is stated or implied.
4. All rights reserved. The Omni-Compass LLC reserves the right to change the software, the documentation, the results and the terms at any time, without notice. Every copy, fork, export, report, archive or printout of any part of Omni-Compass carries the README's notice, `LICENSE`, `NOTICE` and this page unchanged. The Omni-Compass LLC, www.omni-compass.com

What a result is, and what it is not

1. Every result carries its evidence class: **T** a theorem, **V** verified in code, **S** our own models (modelled), **L** live software (real Kubernetes on GitHub's machines), **P** a physical meter (a real card's own power meter). A model is a statement about the model. A software benchmark is a statement about the software and machine it ran on. Only a meter speaks for hardware.
2. The models were written by the same people who wrote the law. They show how the mechanism behaves; they are not an independent test.
3. On Kubernetes in kind every machine stays powered, so energy there is a declared model, not a meter; on the data stores it is the host's processor seconds. Neither is scored in the Omni index: energy is scored

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

only where a meter read it (the card's own meter), and every benefit is claimed at the software end (the founder's order of 31 October, evening).

4. Every comparison is paired: the same seed, load and clock with and without Omni-Compass, in an order rotated by repetition. Intervals are 95% intervals over the paired repetitions. Ratios across seeds are summarised as geometric means. Labels are assigned by code from rules written and committed before the run (`docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`), never by hand.
5. Losses and failed runs are kept beside the wins (`docs/EVIDENCE_LEDGER.md`, `results/live/`, git history). The first real-card run, for example, saved energy (+3.6% work per energy, proven) and made the slowest answers 58.5% slower; it is published as it happened.
6. No result is a promise of a saving on any other system. No dollar figure in this repository is a measured result. Nothing here is a vendor certification, a safety certification, legal advice or investment advice.

Do no harm, and the rule Omni-Compass keeps on every muscle

What "do no harm" means (the founder's words of 19 October 2026; this is its meaning everywhere in Omni-Compass, and it has no other). Omni-Compass has one goal: to strengthen the organism it governs and keep it healthy, working at the best state its own muscles can carry. Whatever is fed into the organism (its work, its data, its energy: its nutrients) passes through its muscles and comes out as its output. Omni-Compass's brain drives the organism toward the most and the best output it can make from what it is fed (the most work, done fastest, with the fewest misses and failures) while it keeps every muscle healthy.

- **Harm is judged at the organism.** Harm is the organism turning out less than its native controllers make it turn out (less work done; more time waited, missed or failed), or the organism made less healthy: a muscle pushed past its own limit, a service pushed over its own line, or anything made to fail. Omni-Compass does neither.
- **Every muscle only as far as it can go.** Omni-Compass can push a muscle only as far as that muscle itself can be pushed: its own limit, read live from the system, never past it. It finds the room each muscle already has and uses all of it. Whether its mathematics can draw a little more out of a muscle than the muscle was built to give is an open question; a measurement answers it before anything is claimed.
- **The muscles work as one body.** Where one muscle's strength cannot be used without straining another, the brain offsets and engages the muscles together to bring the whole organism to its best state. It may use one muscle less, leave it native, or rest it, so that another can carry more, whenever the organism's whole output comes out ahead and every muscle stays healthy. Which muscles to drive and which to leave native is the brain's live decision, at every moment and in every run.
- **Reactive and proactive.** Like an immune system, the brain answers a breach at once, and it acts before a line is crossed.
- **Every row is still shown.** Each muscle's and each gauge's reading stays in every table beside the organism's, LOSS included. Nothing is cut.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

How the engine that stands keeps it. The engine that stands keeps do no harm in its strictest form. By default it moves each muscle's settings toward more work, faster, up to the muscle's own limit and never past it (the max); on the dial anyone can tune it down toward the same work with less (the tune-down). Either way it keeps a step only where the muscle is never behind native for it: a step that costs the muscle more for each piece of work, or loses more than **2%** in any measure (work done, response time at the median, p95 or p99, time over the service line, or the machine's own cost of running Omni-Compass), is undone. A resource spent that rose is **power given**, shown beside the work, the speed and the wear it bought, never called a loss. Where no move passes that test, Omni-Compass leaves the muscle exactly as it runs alone. It takes nothing from one muscle to give to another: that form never harms the organism, but it leaves unused any gain that needs one muscle to give way to another. Letting one muscle give way so that the organism's whole output comes out ahead, judged on the organism and on every muscle's health, is the next engine version's work. It is built and proven in our own models first (the model gate), every benchmark is run again on it, and no result claims it before its runs exist.

The factory tune: one tuned engine, and a tuning bench. The engine in this repository is presented as The Omni-Compass LLC tuned and ships it: self-running, set at the factory. Every muscle in the catalog is wired and runs on the setting found for it (its own card where one was found on development seeds and kept under the strict rule, 364 muscles in `realms/muscle_cards_v5.csv`; the engine's own setting for every other), frozen and fingerprinted by version (`python3 tools/omni_version.py`); no setting is presented as universal. **Every published benchmark result is one engine at its one factory tune**: the engine its table names (`python3 tools/omni_version.py`), run again on each engine that stands. From omni-v17 that tune is held to a declared rule (`docs/THE_TUNE.md`, `tools/tune_study.py`): of the tunes that harm no gauge against native, the one against which no other tune tried reads GAIN on one service gauge and LOSS on none, searched on development seeds and read again on untouched ones; the claim is non-dominance among the tunes searched, never a global optimum. The live benchmark engine page and the control panel are a live tuning engine: every knob, button and monitor turns in real time (the beat, the dial, the pedal, each muscle driven or left native, up or down, in or out, minus or plus) to show the mechanism of action with native alone, with Omni-Compass on top, or both side by side; what it shows at any other setting is the engine's range, never a benchmark result. As on any engine, where the mixture, the intake, the exhaust, the firing or the pistons change what it delivers, **turning any knob of Omni-Compass can change every metric, up or down, in any direction a tuner chooses**: one engine for many vessels, tuned many ways, for many outputs. A licensee may add on, retune, upgrade or tune down any muscle or combination (the dial, the cards, the bands and the service lines are exposed); a changed tune is the licensee's own, and so are its results. The factory tune is published only where it reads no LOSS: a LOSS found in The Omni-Compass LLC's own runs is treated as a fault in the mechanism or its wiring, found and fixed before the result is published, never as a number to live with (the founder's order of 23 October 2026). The one place a LOSS is meant to be seen is the live benchmark engine, when a person turns its knobs away from the factory tune (the dial, the heartbeat, the sizes, the load) to watch the system react and the brain hold or recover: that showing is the proof that the factory tune is the best state found, and that a turned knob never breaks the engine: it changes its mechanism

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

of action, how its muscles and the organism share the work, toward the purpose the new setting asks for, and a changed tune's readings are the licensee's own.

The factory profile, and the profile that comes next. The factory tune is one complete configuration profile, the factory profile: every governing rule, gain, margin, decision interval and default setting shipped together, versioned and fingerprinted. Today's is omni-v19's. The best validated factory profile for the declared operating conditions is not yet established: it is chosen only after every mechanism is verified, every knob combination is tested together on development workloads, and the choice is validated on fresh ones; then it is frozen, carried into every implementation (the C++ harness included) and released under its own identity. Until then no other profile is presented as the baseline; on the live benchmark engine a candidate is evaluated, never released, and a custom profile's readings are its own. The words, the mathematics and the bill's arithmetic are written once in `docs/FACTORY_PROFILE.md`.

The wiring declaration

THE PUBLISHED BENCHMARKS ARE THE RECORD OF WHAT OMNI-COMPASS DID ON THE SYSTEMS, LOADS AND ENGINE THEY NAME, EACH WITH ITS EVIDENCE CLASS. WHERE A RESULT ON ANY OTHER SYSTEM DIFFERS FROM THEM, OR ANYTHING LOOKS WRONG, THE FIRST PRESUMPTION IS THE WIRING, THE SETTINGS, OR HOW OMNI-COMPASS WAS IMPLEMENTED OR READ, NOT THE LAW. CHECK THOSE FIRST (THE MANUAL'S "WIRED RIGHT OR WIRED WRONG") BEFORE DRAWING ANY CONCLUSION ABOUT OMNI-COMPASS.

Omni-Compass acts only through the wires it is given. It reads the meters it is pointed at and moves the settings it is allowed to move. If a reading is the wrong one, slow, or blind; if a lever is the wrong one, already owned by another controller, or given a range (cover) that is wrong for the machine; if a band or a service line is set for a different workload; then Omni-Compass will do exactly what its law says with the wrong information, and the result will not be the published one.

Therefore:

1. **It cannot be slapped on.** Every installation goes through the levels of the manual in order (chapter 6), starting read-only, and every writing level begins with the wire check (manual, section 42.4) and a watch arm that must equal native.
2. **If your paired receipts differ from the published benchmarks in direction** (service LOSS, or no saving where one was shown), **the first presumption is wiring, not the law.** Confirm the installation with the checks in the manual's section 42.5 (*Wired right or wired wrong*) before drawing any conclusion about Omni-Compass.
3. **This has happened to us, and is published.** On the first real card (NVIDIA A10, 2026-10-02) the governor read being busy as trouble, set its power lid under the card's own working draw and allowed the clock under the card's own working clock; the card served bursts more slowly and the slowest answers were 58.5% slower. The cause was found in the card's own samples and corrected

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(`docs/GPU_PREREGISTRATION.md`, amendments 6 and 7). That is what a wiring fault looks like, and how it is found.

4. Omni-Compass never writes outside a lever's cover, never fights another writer (it stops and leaves that lever alone), reads back every write, and returns every lever to the value it read before its first write when it stops or when the OFF switch is used. These protect the machine; they do not make a wrong wiring right.

Safety and responsibility

1. Run watch mode first; run the wire check before any write; keep the master OFF switch (`python3 tools/omni_switch.py off`, the whole harness at once) in reach at every level, and use it at the first sign of anything wrong, a suspected breach included.
2. Omni-Compass is a supervisory governor. The machines' own controls (firmware, autoscalers, safety systems) stay in place and keep their own protections; Omni-Compass sets only values they already accept.
3. Do not connect Omni-Compass to safety-critical systems (vehicles, medical devices, grid protection, life safety) except as a modelled study, and never without the qualified review those systems require.
4. The software is provided "as is", without warranty of any kind, as stated in `LICENSE`. The operator is responsible for its use on their systems.

Changes to the work and to this page

1. The software, the law's settings, the benchmarks, the results, the documents, the licensing terms and this page may change at any time, without notice. The version in the repository's `main` branch at a given commit is the version that commit describes; `RELEASE_MANIFEST.json` and `python3 verify.py` identify it.
2. Statements about work in progress, planned features, expected results and future runs are expectations, not promises. Where a result has not yet been measured, the documents say so (`docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, section 8).
3. A signed Omni-Compass Enterprise License governs its own terms for its own term.

Privacy and data

Omni-Compass contains no telemetry. It sends nothing to The Omni-Compass LLC or anyone else, and makes no network connection of its own except to the systems an operator points it at. Its logs and receipts stay on the operator's machines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Where everything is

Question	Page
What has been measured, and how strongly	`docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, `docs/EVIDENCE_LEDGER.md`
The rules written before each run	`docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`
How to wire it, level by level, and how to confirm it is wired right	`docs/OMNI_COMPASS_MANUAL.md` (chapters 4 and 6), the book `docs/OMNI_COMPASS_MANUAL.pdf`
The license, in plain answers, and third-party components	`LICENSE`, `LICENSING_FAQ.md`, `ENTERPRISE_LICENSE.md`, `NOTICE`, `THIRD_PARTY_NOTICES.md`, `PATENTS.md`, `TRADEMARKS.md`, `SECURITY.md`, `.github/CLA.md`
The muscles, what each is for and how it is wired	`docs/MUSCLE_CATALOG.md`
Proof the code is the code that ran	`python3 verify.py`, `RELEASE_MANIFEST.json`, `results/SEAL.json`
The live benchmark engine: every monitor, its feed and its knobs; the third manual, how to read and operate it	`docs/engine/OMNI_COMPASS_LIVE_ENGINE.html`, `docs/engine/monitors.json`, `docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.md` and `.pdf` (the Live Benchmark Engine Manual's Appendix L, the page word for word: `docs/engine/PAGE_WORD_FOR_WORD.md`)
The whole organism, composed every way it is bought	`results/WHOLE_SYSTEM.md`, `tools/whole_system.py`, `docs/engine/export_whole.py`, `docs/WHOLE_SYSTEM_PREREGISTRATION.md`
The Python's runs that the live benchmark engine runs again	`tools/engine_runs.py`, `results/engine_runs/`, `results/heldout_seed_*/`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The whole organism, the cloud's back end, and the live benchmark engine

The composition is a model of measurements. `results/WHOLE_SYSTEM.md` (`tools/whole_system.py`) composes each layer's own measured ratio, native against Omni-Compass on top of native, into one benefit for a whole stack: the product law $R(g) = \prod_\ell R_\ell(g)$ gauge by gauge, its logarithm a sum with the variances added, each layer's runs A, B and C pooled by the inverse of their variances, and one index over the gauges with its 95% interval under two bounds (the gauges independent; perfectly correlated). No run here has stacked the layers: each factor is one layer's own run with the layers under it native, and the product assumes a layer's ratio does not change when the layers around it also carry Omni-Compass. That assumption is tested where the tables can test it (the governors' four ways), and the stacked run that replaces the composition is preregistered in `docs/WHOLE_SYSTEM_PREREGISTRATION.md`. Until it runs, the composition is class S of class P and L measurements, and every page that shows it says so.

Every way a stack is bought. The same law is read four ways (`docs/engine/export_whole.py`): on-site bare metal (the software on the owner's own racks and chips); through the cloud (the cloud's build, its bare metal at the back end and the software; and the cloud as two pieces, the back end left to the provider); in the device (Omni-Compass in the bare metal alone, the card and the machine); and as a service (the software alone, for an owner with neither bare metal of their own nor a cloud account). A piece with no measurement enters as exactly 1 and is named.

The cloud's back end is not ours to meter. Through the cloud, the bare metal belongs to the provider: no tenant can read its power meters or write its chips' limits, and no run here reaches it. The cloud piece's benefit is what the provider itself meters and bills, the machines counted every fifteen seconds and the bill at list price, read native against Omni-Compass on top of the cloud's own autoscaler. The bare-metal piece is gauged on hardware we can meter (a card's own power meter; a processor's energy counter) and is carried to the provider's bare metal only as the same mechanism on the same kind of part, a modelled factor, never as a measurement of the provider's machines.

The cloud is a wire. A cloud is the wire between the software end (the owner's programs, Kubernetes and the open-source orchestrators, managers and autoscalers that carry the work) and the hardware end (the bare metal: the GPU chips, the Linux racks, the data center). Omni-Compass can live in all three places; the wire makes no benefit of its own. Until 31 October every cloud run here was set on the wire alone (the tune-down on the machines only), so it could at most park machines the cloud already made and cut the wire's bill a little. From 31 October (the founder's order of that day: no hard rule) every cloud set runs the whole engine at the max on the cloud's cluster, and what its receipt must show is that the engine holds there (section 10): no judged gauge LOSS against native, the work per dollar read beside the bill. The benefit is shown where it lives, at the software end, and explained through the wire, never claimed for the wire itself. The bare metal's energy and throughput are not under a tenant; they are measured where Omni-Compass sits on them (the card and the machine) and composed with the wire by the law above. A provider running Omni-Compass on its

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

own back end would see that benefit on its own chips and racks; no tenant can meter it, so it is never claimed here as a measurement. The company's expected ranges (the founder's figures of 24 October: about 3 to 4% from the software end, 2 to 3% from the bare metal, about 5% from the two together, 1 to 2% from the wire alone) are an expectation, never a result, and his word of 31 October sets the wire's own at none: the wire carries the benefit, it does not make it; where a measurement differs, the measurement stands.

What the record shows in each place so far (the words for each place, and what each benchmark must show, are in section 10): at the software end the benefit, the Omni index over work, speed and machines (`results/OMNI_INDEX.md`; composed layer by layer in `results/WHOLE_SYSTEM.md`), with no bill in it; on bare metal, the A10's own bill and meter on omni-v23 (`results/TWO_BILLS.md`, `results/gpu/V23_A10_CAP_FULL_ABC.md`), fully loaded: the bill STEADY, the work done inside it the runs disagree (up in two runs, down in the third, that run's lost work on the next version's list), the chip's energy for each request served GAIN in all three of our runs (the A100's three runs on omni-v21 read work and energy STEADY, `results/gpu/V21_A100_CAP_ABC.md`); through the cloud, Azure's steady set on omni-v18, 98 paired repetitions (`results/live/AKS_STEADY_100_OMNI_V18.md`) reads the bill up 1.2%, inside the noise, and the 99th percentile GAIN: it holds. What the benefit cost the chip, the energy and the wire is listed row by row in section 3b of `results/WHOLE_SYSTEM.md`. In the cloud the yardstick is the work per dollar: the work answered within the line divided by the bill, against native; a bill that rises is power given beside the work it bought, and the work must rise more.

The organisms. The seven organisms of the catalog (the spine, the four realms, the four stacked, the tower), the orchestra beside them, the live benchmark engine's own sixty-muscle organism, and the whole organism in its three places (the software end, the cloud's wire, the bare metal) composed above: ten organisms. Each is read two ways, native and Omni-Compass on top of native (Omni-Compass is never by itself, the founder's order of 25 October): twenty scenarios, recorded in `results/scenarios/SCENARIOS_20.json` (class S, our own models, three runs), every cell with no run reading "not yet run". None is a product.

The live benchmark engine. `docs/engine/OMNI_COMPASS_LIVE_ENGINE.html` is a monitor of the repository's records and of the engine's own mathematics running in the page, and the live tuning bench: a setting other than the factory profile shows a variation of the mechanism, never a result. Every monitor carries its evidence class (P a physical meter, L live software, S our own models) and its state: live (the engine's port, proven equal to the Python and the C++ on every build by `docs/engine/check_live_engine.py`), a replay of a recorded run from its own files, a reading of the repository's tables, or built and awaiting its run, with no invented number and the run that fills it named. The kill switch on the page acts on what runs in the page, and a recorded result changes its colour, never a value, a word or a row. Pointed at an owner's own Prometheus, the page only reads it: it writes to no system.

The three manuals. The Operator's Manual wires and runs Omni-Compass, the Owner's holds the whole mechanism and every result, the Live Benchmark Engine Manual reads the page; each carries this notice, the `LICENSE`, the `NOTICE` and this page.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Where Omni-Compass sits, and what each benchmark shows

Omni-Compass is software. It always sits on top of a native controller. A system runs it in one of two places, on bare metal or through the cloud, and in each place there are two arms, native and Omni-Compass on top of native: four arms, each read by its bill and the work done inside it. These are the words for them, used the same way on every page.

- **The software end.** The programs that carry the work: Kubernetes with its pod and cluster autoscalers, its schedulers and the orchestrators on it, the data stores (PostgreSQL, MySQL, Redis, MongoDB, Kafka) and the services they serve. Omni-Compass sits on top of their native controllers. This is where its benefit lives and where it is measured: real software on real computers (class L), on GitHub's own machines in kind clusters. Those machines are rented runners, never called bare metal and never called the chip.
- **Bare metal.** The first of the two places a system runs (the founder's order of 1 November 2026): the owner's own machines, or a chip run directly, with the chip's own controllers under the software. Its bill is the machine's own: the hours it runs, and the energy the chip's own meter reads. The software end goes with Omni-Compass to it, and there Omni-Compass can reach the chip's own knobs too.
- **Through the cloud.** The same software end on a cloud provider's rented machines: Kubernetes through the cloud (Azure's AKS today; Google's GKE and Amazon's EKS wired the same way, not yet run), the machines billed by the hour at list price and added or removed by the cloud's own autoscaler. Its bill is the cloud's own; the cloud's chips and back end are inside its price and out of Omni-Compass's reach: it governs what the cloud gives, through the cloud's own Kubernetes. The cloud is the wire between the software end and the provider's bare metal. The wire makes no benefit of its own, and no tenant can meter the provider's chips.
- **The chip.** Bare metal's own part: the processor or graphics card under the software, with its own controllers (the power limit, the clocks, the heat), measured with the card's own power meter (class P) where Omni-Compass sets the card's knobs through the card's published interface. Omni-Compass built into the silicon, or into a battery's own controller, is not something any run here can test, and it is not claimed.
- **The four arms.** Two places, two arms in each: A, native on bare metal; B, Omni-Compass on top of native on bare metal; C, native through the cloud; D, Omni-Compass on top of native through the cloud. Each is read by its bill and the work done inside it, the two places never added together, since a system runs in one or the other (`results/TWO_BILLS.md`). The software end's tests are one set read three ways (the Omni index on work, speed and machines; the same on work and speed alone; the layers composed): they never add together, and none of them is either place's bill.
- **Hold.** Omni-Compass on top of native harms nothing in that place: every judged gauge (the work done, the response time, the minutes over the line, the pods waiting, the failures) reads STEADY or GAIN against native by the one key, and a resource that rose (the bill, the machine-hours, the energy) is read beside it as power given. A place holds when its runs read no LOSS.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Where the benefit lives.** At the software end, where it is made and measured, with no bill in it. It shows through on the two bills, native against Omni-Compass on native: on bare metal the work done inside the machine's own bill (each dollar, and each joule of the chip's own meter), and through the cloud the work done inside the cloud's own bill (each dollar). The chip and the wire themselves add no gain of their own: they are where Omni-Compass must hold. A benchmark that holds is the proof of no harm in its place; the benefit is shown where it lives and explained for the rest.
- **One mechanism, in one place, felt along the whole path.** Omni-Compass acts in one place, the software end, on top of the controllers a stack already runs; it changes nothing in the chip and nothing in the network or the cloud. Its effect travels the whole path the work travels: a request starts in the software, crosses the wire (the owner's own racks or a cloud's machines), is computed on the chip and comes back. So the effect is checked at every place: the software end shows the gain (more work, faster, on fewer machines), and the wire and the chip show that it holds and carry the gain to their bills. Two places are tested because a system runs in only those two; one mechanism makes the gain, the way a city's traffic signals make more trips an hour on the same roads and the same cars.

Place	Its benchmark	What it must show	Its readings
The software end	Kubernetes and the data stores on GitHub's machines (kind), class L	the benefit, by the one key, and that it holds	`docs/BENEFIT_SHEET.md`, `results/OMNI_INDEX.md`
Bare metal, the chip	a graphics card on Lambda run directly, its rental the bill and its own power meter beside it, class P	that it holds, and the work done inside its bill, A against B: the work per dollar and per joule	`results/TWO_BILLS.md`, `results/gpu/`
Through the cloud	Kubernetes through Azure's AKS, the machines it bills, class L	that it holds, and the work done inside its bill, C against D: the work per dollar	`results/TWO_BILLS.md` and each set's `results/live/AKS_*.md`
The whole organism	the places composed by the product law, class S of P and L	how the places add up, an explanation and never a measurement	`results/WHOLE_SYSTEM.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

PART ONE

The Philosophy and the Theory

Where Omni-Compass comes from: the closed circle, the four pieces, the compass, the basins and the reflex. The engineering in the rest of the book is this idea made exact.

CHAPTER 1

What Omni-Compass Is

A governor on a steam engine does not build the engine and does not turn the shaft. It watches the speed and moves the throttle so the speed stays in a band. Omni-Compass is that, for every machine you run.

It is a process on a host. It reads meters, steps a bounded mathematical law, and writes only the levers it has been given. It is not the chip, not the GPU driver, not Kubernetes, not the building controller. Those keep running exactly as they do today; Omni-Compass sets the values they already accept.

Three facts about that process fix what the rest of this manual can and cannot claim. First, it is **one process per stack**, a controller (`omni_controller/controller.py` on a cluster; the harness's own controller on a database or a cache) that wakes on a fixed period, reads the stack's own gauges through the stack's own interface, decides, writes through the same interface, and reads back what the device took; it has no agent inside the pods, no kernel module, no hook in the request path, so it can add latency to nothing it does not govern. Second, every lever it may touch is **named before it starts**, read once into a snapshot, and handed back to that snapshot when it stops, is killed, or loses a sense; a lever it was not given it cannot reach, because the plug for it does not exist. Third, it has a **watching mode** in which it reads and decides and writes nothing, and every rollout begins there (Chapter 45): the first thing an operator learns about Omni-Compass on their own stack is what it would have done, in a log, with the stack untouched.

What follows from those three is the shape of the evidence in Part Seven. Because the native controller keeps running underneath, every benchmark is the native stack against the same native stack with Omni-Compass on top, and never Omni-Compass against anything else. Because the levers are the stack's own settings, every gain is one the operator could have reached by hand, if they had been able to read every gauge every few seconds and move every knob without ever being wrong; the claim is not a new mechanism but a steady hand on the mechanisms that exist. And because the process is small and separate, its own cost (CPU on the host, reads of the API) is measured in every run and counted against it where it shows.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

1.1 Always on top of a native controller

The first design decision, and the one every other follows from, is that Omni-Compass is **supervisory**. It never replaces the controller a system already has, never bypasses it, and never runs in its place. The Kubernetes Horizontal Pod Autoscaler keeps scaling pods on its own CPU target; Omni-Compass moves that target inside a range the operator allows. The cluster autoscaler keeps adding and deleting machines; Omni-Compass marks machines idle and lets the autoscaler do the deleting. The card's firmware keeps boosting the clock and enforcing its power limit; Omni-Compass sets the ceiling the boost may reach and the limit the firmware enforces. PgBouncer keeps handing connections to clients; Omni-Compass sets its pool size through the pooler's own console. Kafka's consumer group keeps rebalancing partitions on its own; Omni-Compass changes how many consumers are in the group, which is what an operator does. Redis keeps evicting by its own rule; Omni-Compass sets the ceiling that rule works under.

This matters to a referee for a precise reason: every benchmark in this manual sets **omni** (Omni-Compass on native: the same system with Omni-Compass on top) against **native** (the system with its own controller, as shipped and as the operator set it), and there is no comparison in which Omni-Compass on native is a rival to that controller. Since the founder's order of 23 October a third arm, **the arm by itself** (called Omni-Compass direct that day), runs beside them as a mechanism test: a knob's native controller switched off only where Omni-Compass carries that muscle itself and has proven it can move the knob, Omni-Compass then the only controller moving it, and every muscle Omni-Compass cannot be kept, native, under it. On Kubernetes that is Omni-Compass's own pod rule in place of the autoscaler's arithmetic and nothing else, because Omni-Compass is a brain with no organs there: it can be direct to a knob, never to a muscle (the founder's analysis of 31 October, `docs/SOLO_PREREGISTRATION.md` amendment 2; on Kubernetes the pod autoscaler removed where Omni-Compass's own pod muscle takes the workload, no cluster autoscaler on kind, the API server, the scheduler and the kubelet kept; `--own-replicas`, omni-v19; in the models `realms/harness.py` SOLO; Omni-Compass's own orchestrator, `omni_orchestrator/`). Where native is a setting rather than a controller, or its own loop is the actuator, the two Omni-Compass arms are one arm; no cloud runs direct without the founder's word (`docs/SOLO_PREREGISTRATION.md`). The three are always shown together, each pair read on its own. A gain is therefore always "what the governor added on top of what the native controller already did", and a loss is always "what the governor cost on top of it". Nothing in the record is an argument against any native controller; every native controller in this manual is a good controller that the governor sits on.

1.2 Do no harm: the verdict

The second design decision is do no harm, and its meaning is the organism's (the founder's words of 19 October 2026, `DISCLOSURES.md` section 3): the organism Omni-Compass governs is never left turning out less than its native controllers make it turn out, nor less healthy (no muscle past its own limit, no service over its own line, nothing made to fail), and within that the brain drives the whole organism to the most and best output its own muscles can carry, using one muscle less, or leaving it native, where that lets another carry more and the whole comes out ahead. The engine that stands keeps the rule in its strictest form:

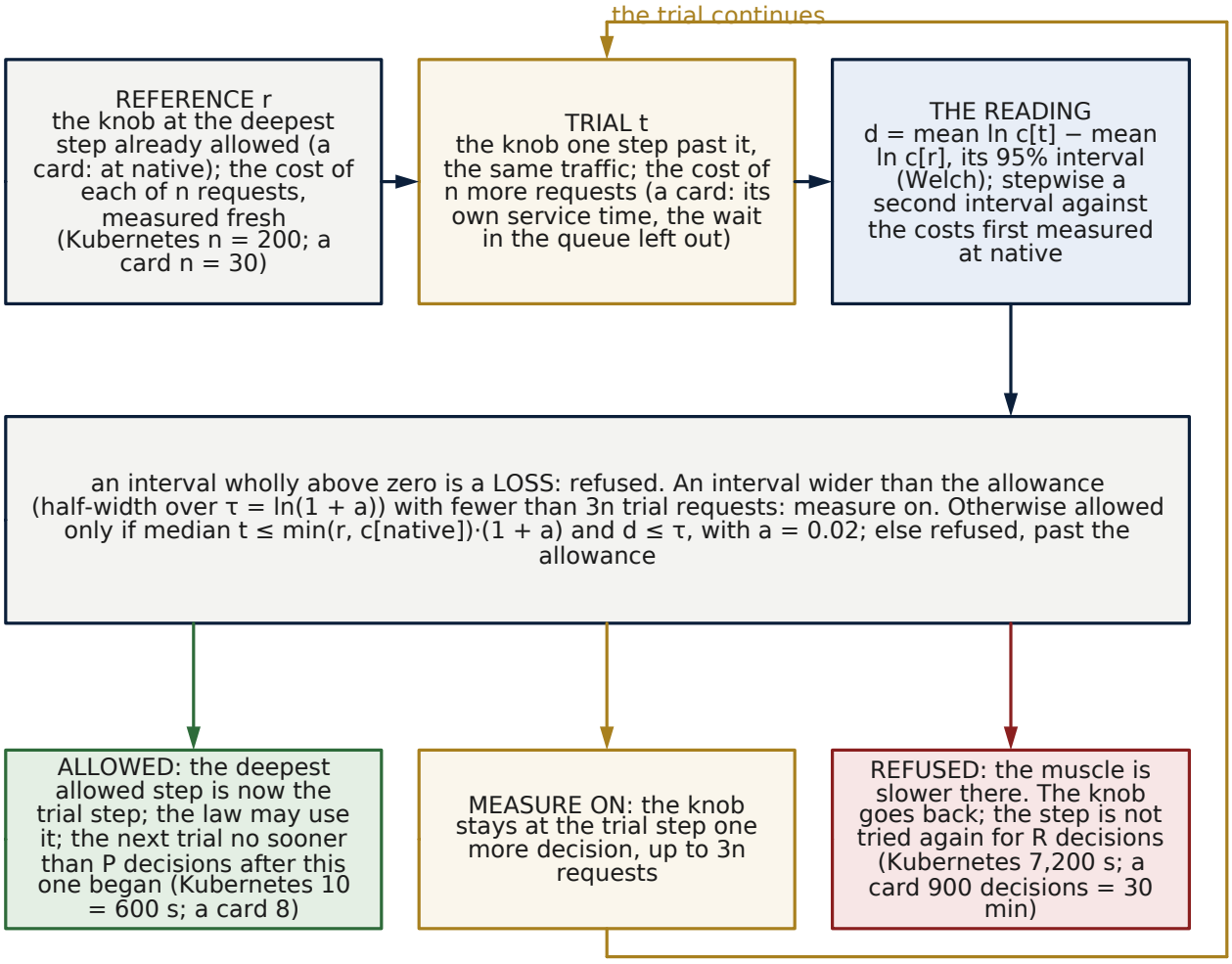
© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Omni-Compass must never cause a LOSS on any muscle, judged on the muscle itself, and takes nothing from one muscle to give to another; the trade between muscles, judged on the organism, is the next engine version's work. This is not a slogan printed on a report; it is a trigger inside the engine. Before Omni-Compass moves a slow knob (a machine given back, a card's clock ceiling, a robot axis's speed, a drone's cruise override), it runs a **paired trial on the muscle itself** (`omnicompass/verdict.py`): the muscle's own cost per piece of work is measured at the deepest step already allowed (on a card, at native) and at one step past it, under the same traffic, with a set number of pieces of work on each side (n = 200 response times on Kubernetes, `--verdict-samples`; n = 30 requests on a card). From v7 the trial is read in the reports' own words: on the logarithm of each piece of work's cost, the change d = mean ln c[trial] − mean ln c[ref] carries its 95% interval $d \pm 1.96 \cdot \sqrt{(s^2[\text{ref}] / n[\text{ref}] + s^2[\text{trial}] / n[\text{trial}])}$, and with $\tau = \ln 1.02$ the 2% allowance in the same units, an interval wholly above zero is a LOSS and the step is refused whatever the allowance says; while the interval's half-width exceeds τ the trial measures on, up to three times its minimum; and a GAIN or STEADY step is allowed only if its median cost and its mean-log change are both no more than 2% above the reference and above the cost first measured at native. On modelled noisy costs ($\sigma = 0.25$) this allows a step that truly costs 3% more 28% of the time instead of the medians' 40%, one that costs 5% more 14% instead of 26%, and a free step 60% instead of 63% (`docs/OMNI_V7.md`, section 3; class S). The engines before v7 read the medians alone (`judge="median"`). A refused step is not tried again for a set time (7,200 s on Kubernetes, 120 decisions at the main engine's 60 s beat, 480 at a beat of 15 s; 900 decisions, 30 minutes, on a card); a trial begins only while the service is calm and nothing waits, no sooner than 600 s (a card 8 decisions) after the last began; and a phase that sees the service leave calm or outlasts its limit (1,200 s on Kubernetes; 30 decisions on a card) is abandoned, the knob goes back, and the step is neither allowed nor refused. Where no step passes, the verdict reads **left native**, the knob is never moved, and the result for that muscle says "nothing for Omni to move". This is why, in the robot-arm benchmark, two of four arms were left entirely native and are reported as such, and why in the real Kubernetes tests machines were kept whenever one machine fewer made the requests slower in the trial. Gains are not capped; the allowance caps only what may be spent to get them.

The same principle, applied after the fact to every published result, is the page of wiring verdicts (`docs/WIRING_VERDICTS.md`, section 16.6b): every knob in every table is given one of three words from the table itself, **write** where a gauge is GAIN in all three of our runs and none LOSS or power given in all three of our runs, **watch** where nothing is GAIN in all three of our runs (the knob stays native and Omni only reads it, one wire out and no wire in), and **operator's choice** where gains and losses are both the same in all three of our runs. Omni-Compass is wired out of every muscle; it is wired in only where it has earned it.

1.3 The pedals, the modes and the two switches

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.



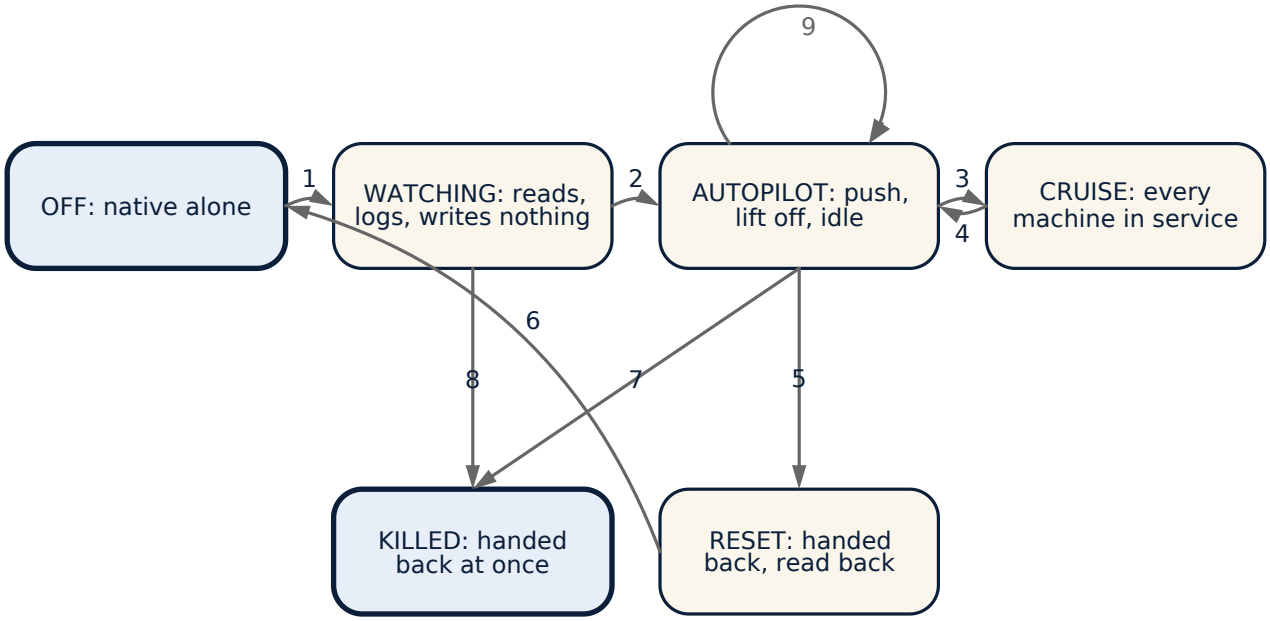
The verdict stands over every slow knob (the machines, the clock ceiling, a cache's notch). A trial begins only while the service is calm and nothing waits; a phase that sees the service leave calm, or that takes longer than its limit (Kubernetes 1,200 s; a card 30 decisions), is abandoned and the knob goes back, the step neither allowed nor refused. The knob never passes the deepest allowed step, so steps cannot add up past the allowance. On the max, the Kubernetes machine count never gives back a machine the cluster had on its own.

Figure 1. Block diagram: the verdict's paired trial around a slow knob, read GAIN, STEADY or LOSS. The reference r is the cost of n requests at the deepest step already allowed (on a card, at native; n = 200 on Kubernetes, 30 on a card); the trial t is the cost of n more one step past it under the same traffic. The reading is the 95% interval of d = mean ln c[t] − mean ln c[r], stepwise also against the costs first measured at native: an interval wholly above zero is a LOSS and the step is refused; an interval wider than τ = ln(1 + a) measures on, up to 3n; otherwise the step is allowed only if median t ≤ min(r, c[native])·(1 + a) and d ≤ τ, with a = 0.02.

A refused step is not tried again for R (7,200 s on Kubernetes, 480 decisions at the 15 s heartbeat; 900 decisions on a card, 30 min). Steps cannot add up beyond the allowance, because every step is judged against the native cost too.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Omni-Compass drives a system the way a self-driving car drives itself: it feels, gauges and adjusts, with nobody in the seat. The vocabulary is used the same way everywhere in this repository (`docs/MECHANISM_OF_ACTION.md`, "The pedals"):



1 level 1 · 2 the wire check passed; level 2 and up · 3 pods waited 2 decisions · 4 the line empty 2 decisions · 5 the reset file: the end of a run · 6 exit 0 · 7 the kill switch, from any mode · 8 the kill switch · 9 idle at once when demand is at zero

Figure 2. State machine: the governor's modes. OFF is native alone; WATCHING reads and logs and writes nothing; AUTOPILOT pushes, lifts off and idles; CRUISE holds every machine in service while pods have waited two decisions; RESET hands every setting back and reads it back at the end of a run; KILLED hands everything back at once, from any mode, on the kill switch. The numbered transitions are read in the key under the diagram.

- **Off (manual).** Omni-Compass is not driving; native runs alone. This is the native arm of every benchmark.
- **Watching.** Omni-Compass reads every gauge and logs the decision it would take, and writes nothing. Any difference between a watching arm and native is the cost of its being there (its CPU, its reads) plus noise, never a decision; a watching arm that writes anything fails its run. Watching is the first level of every rollout (Chapter 45).
- **Autopilot.** One pedal: **push** (traffic climbs and capacity is added at once, never held back), **lift off** (traffic falls and capacity is eased off a step at a time, handed back toward native, never below idle), **idle** (the pedal lifted all the way: calm traffic, the engine at its floor of two machines in service, ready for the next burst), **reset** (every setting the governor ever wrote is handed back to where native had it, read back, and the record removed; every benchmark arm ends with a reset and checks it).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Cruise.** For work that comes as a pile: when pods have waited for a place decision after decision for the cruise wait (120 s: two decisions at the main engine's 60 s, and never fewer than two readings at any beat), machines go into service up to the pool's own top (read live: every machine of a fixed pool at once, a growing pool a step a decision) and stay there, without second-guessing, until the line has been empty as long. In cruise the governor also stops its five-second floor reads, because every machine is already in service and the reads would only cost CPU on the machines the queue is using.
- **Idle at once.** When the work is done and demand is at zero (the service at its autoscaler's own floor, wanting no more, at half its target or less), the pedal is lifted all the way in one move, straight to idle, with every safety check still holding.
- **The kill switch.** Separate from all of the above, and for **security only**: one switch in a human hand (`python3 tools/omni_switch.py off`, `omnicompass/master.py`) turns Omni-Compass's governing off across the whole machine at once, for anything rogue, or for anyone trying to drive a system through Omni-Compass's brain. Every governor hands back and exits; none starts while the switch is off. The reset is the end of a run; the kill switch is the end of trust.

The floor of two machines is an operator's number; lifting off never takes a system below it, however far the pedal is lifted. There is no ceiling: every machine the operator has is usable, and the protection is the band inside each machine.

1.4 Nothing is hardwired

Omni-Compass carries no table of per-stack special cases. One law (Chapter 30) is written once and runs on every muscle through one plug (Chapter 42). The gains differ from muscle to muscle only through the muscle's declared physics: its response time, its decision period and its cover. Where a rule was found not to work, it was removed, not switched off and left in the code: the engine files are frozen and fingerprinted, and `verify.py` fails if any of them changes. A reader who finds a behaviour in a result can find the exact lines that produced it, and will find no second path.

The rule has a practical consequence for anyone comparing Omni-Compass with a tuned controller. A tuned controller carries, somewhere, a number that someone chose for one stack: a cool-down of five minutes, a target of 70%, a step of two replicas. Each of those numbers is a bet that the stack will behave tomorrow as it did when the number was chosen, and each is one more place where a reader has to ask whether the result was the law or the tuning. Omni-Compass has a handful of such numbers, the centre of the band, the 5% cushions and the 2% do-no-harm line, and they are the same on every muscle, written once in the law and declared in every preregistration. Everything else the law needs (how fast the muscle answers, how often it may be moved, how far it may go) is read from the muscle's own declaration in the catalog (`omnicompass/muscles.py`) or from the native controller's own settings at the start of the run, never typed in by hand for a benchmark. When a benchmark needs a line, a cover or a chunk size (the MySQL pool moves in the server's own 128 MB chunks, the Redis ceiling in notches), the number comes from the stack being governed and is written in the preregistration before the first counted run, where a referee can see that it was not chosen after seeing the result.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

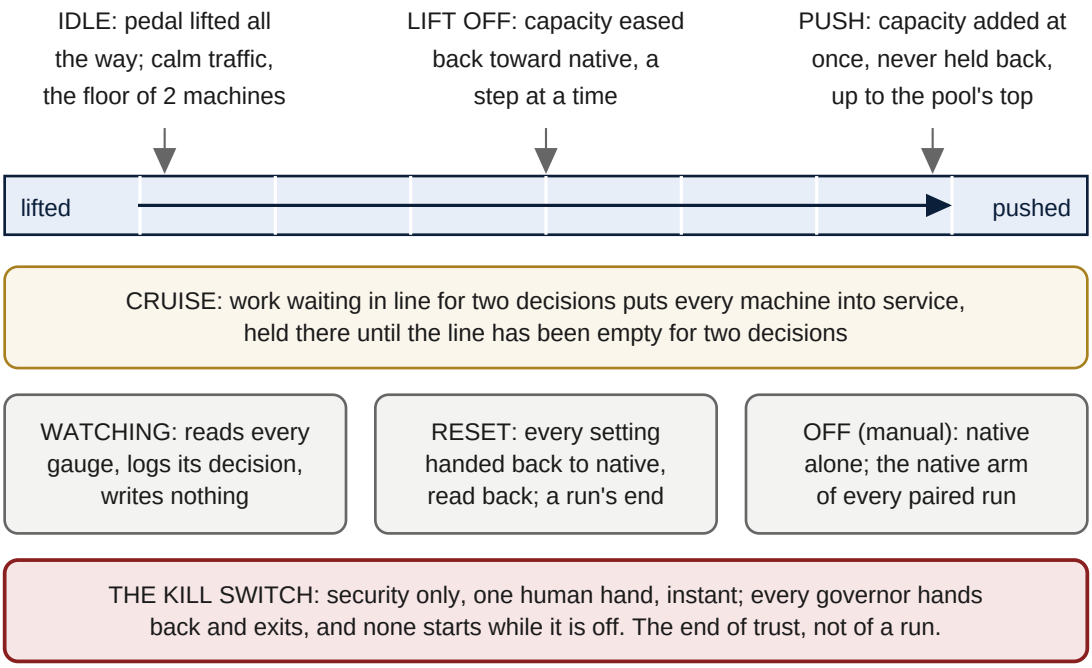


Figure 3. One pedal, no brake. Pushing the pedal adds capacity at once; lifting it eases capacity back toward native a step at a time, never below idle, the floor of two machines. Cruise holds every machine in service while a known queue waits. Watching reads and logs but writes nothing; reset hands every setting back to native and reads it back at the end of a run; off is native alone. The kill switch stands apart from all of these: for security only, in a human hand, instant. After a crash the lease walks every setting back to native on its own.

The second consequence is how the repository changes. A guard or a gain that turned out not to help is not left in the code under a flag, because a flag is a second path that a later reader has to reason about and a later test has to cover. It is removed, the removal is written in the changelog, and the fingerprint moves to the next engine version (Chapter 59), so that no result taken on the old path is ever read against the new one. The three fingerprints that exist, v1, v2 and v3, are the whole history of such changes.

1.5 The word

The law is a **compass**: a band with a cushion at each wall, pushed to its middle by two antagonist forces, smooth, never hammering. The repository uses that word and no other for it. Earlier dated records that used another word are kept word for word, as records are; the current law is `omnicompass/compass_law.py`, `CompassLaw`.

The word was chosen for what the law does, not for decoration. A compass has a needle, a centre it is drawn back to, and a card marked at the edges; it does not slam to a wall and stay there, and it does not chatter when the hand holding it is steady. The law has the same three parts. The **band** is the range the gauge is allowed to travel, from 0 to 1 once the reading is placed on it, with the operator's line at the top. The **cushions** are the last

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

5% at each wall, 10% of the band in all: a reading inside a cushion is already too close to a wall, and the law treats it as such before the line is touched rather than after. The **centre**, 0.5 by default, is where the needle comes to rest, and two forces bring it there: a **pull** proportional to the distance from the centre, gentle near it and harder up the walls, like a spring; and a **push** proportional to the speed at which the reading is moving, meeting whatever is shoving it (a load rising, heat building) with an equal and opposite force and doubling as the friction that stops the needle sloshing past the centre (the gain is chosen for critical damping: it glides to the centre and stops, with no overshoot and no ringing). Past the upper wall the pull goes to its full force at once, and the down side may not act until the reading is back inside the band: the law fails up, never down. The force itself is `A × tanh(raw / A)`: near the centre it behaves as a spring, in proportion to the distance, and far from the centre it bends over and flattens into its maximum `A`, the law's authority, so that no reading, however far out, can make the law ask for more than one full step. That flattening is what "never hammering" means in code: there is no reading that produces a jump, and two readings a decision apart produce two forces a small amount apart. Chapter 30 gives the rules in full and Appendix C the equations; the reader who only needs the picture can keep the needle, the centre and the two cushions in mind and will not be misled by anything later.

1.6 What it sets, muscle by muscle

Every muscle keeps its own control, and Omni-Compass sets what that control already accepts:

Muscle	Its own control (kept)	What Omni-Compass sets
Kubernetes service	the HPA	the HPA's CPU target; replica floors
Kubernetes node pool	Cluster Autoscaler / Karpenter / MachineSet	how many machines stay in service (park and wake)
NVIDIA GPU	the card's firmware (boost, power and thermal limits)	the clock ceiling (up wire) and the power limit (down wire)
CPU	the kernel's frequency governor	the frequency ceiling
Data hall, building	the chiller and air handler loops	the supply-air setpoint; units in service
Battery site	the inverter	the reserve level; the peak ceiling
Robot joint, vehicle axis	the servo loop	speed and effort limits
Process loop, feeder	the PI controller	the setpoint inside its band
Database behind a pooler	PgBouncer's pool, the DBA's fixed setting	the pool size through the pooler's own console
Database's storage engine	MongoDB's WiredTiger cache, the operator's fixed setting	the cache size through the server's own console

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

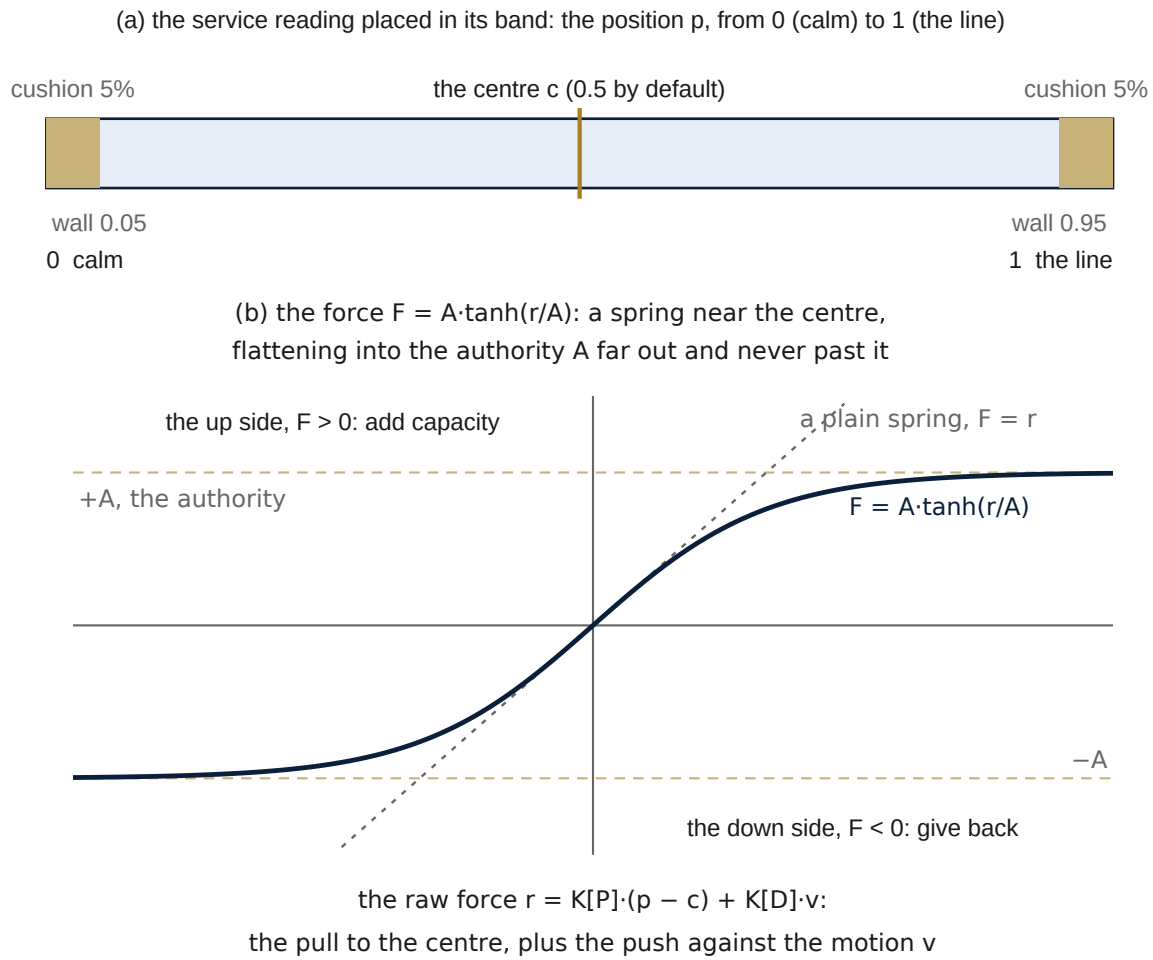


Figure 4. The compass band and the smoothing of the force. (a) The service reading placed in its band as a position p from 0, calm, to 1, the operator's line; the walls at 0.05 and 0.95 leave a cushion of 5% at each edge, and the centre c is where the needle comes to rest (0.5 by default; on a service 0.3 at the max and 0.4 at the tune-down).

(b) The force F = A·tanh(r/A) against the raw force r = K[P]·(p − c) + K[D]·v, the pull toward the centre plus the push against the reading's motion: near the centre a spring, far out a curve that flattens into the authority A and never passes it. Past the upper wall the up side goes to A at once, and the down side may not act until the reading is back inside the band.

Muscle	Its own control (kept)	What Omni-Compass sets
Message broker	the consumer group at the operator's count	how many consumers are in the group
Cache	the operator's memory ceiling and eviction rule	the memory ceiling through the cache's own console
Substation	the tap changer's own controller	the tap position, one whole tap per move

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Muscle	Its own control (kept)	What Omni-Compass sets
Drone	the shipped autopilot	the cruise override inside the autopilot's limits

When Omni-Compass stops, every one of those values goes back to what it was before Omni-Compass acted: not to the last value it wrote, but to the value it read once, before its first write.

Where it sits among the systems already in use. The autoscalers, the node provisioners, the GPU and CPU power managers and the commercial optimizers are muscles, or optimizers over a few kinds of muscle; Omni-Compass is the governor on top of them: one law that reads every muscle in a body and writes each one's own knob under the limits they share. Running several of those systems at once does not make one governor, because each still decides on its own reading. The five systems named most often (IBM Turbonomic, Karpenter, NVIDIA's GPU power software, the Energy Aware Runtime, the Cluster Autoscaler with the commercial layers above it), nine open-source pieces with their licenses and where each can be benchmarked, and who owns Omni-Compass and on what terms it is seen, are set out in *The Seat: One Governor over the Muscles* (`docs/THE_SEAT.md`); the distinction between one governor and a collection of tools is stated formally in *The Max, Formally* (`docs/FORMAL_V5.md`, Definition 2 and Proposition 1).

CHAPTER 2

The Referee's Summary: Each Claim, Its Evidence, and What Is Undetermined

Two pages for a CTO, an engineer, a CFO or an attorney who needs the standing of every claim before reading anything else. Every number names its engine and its evidence class: L is live software, the real thing; S is our own models, never proof; P is a physical meter, and there is none yet. Every reading is GAIN, STEADY or LOSS, decided by what the gauge is for, and holds only when all three of our runs (A, B and C) agree with every 95% interval clear of zero (`docs/HOW_TO_READ_THE_RESULTS.md`). Beside every gain stands what it cost. Where no proof exists yet, the word is **undetermined**.

2.1 What it is

A supervisory governor. It sits on top of a system's own controller (Kubernetes' autoscaler, a database's pool, a battery's controller), reads the system, and moves the same settings native moves, inside bounds the system and its operator set. It hands every setting back to native's value when it stops, dies or is switched off (`docs/SAFETY_CASE.md`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2.2 Each claim and its standing

Claim	Standing	Evidence
It sits on top of native and never replaces it	**Shown**	Every live test runs native, and Omni-Compass on top of the same native; the setting is read back at native's value at the end of every arm
It does no harm: never a LOSS against native	**Shown on omni-v11's counted runs; two faults found since, each fixed in the next version, their runs under way; one row awaits a written ruling**	0 LOSS rows in the 23 omni-v11 three-run tables (1,039 judged rows: Kubernetes, the robustness kill test and long run, the five data stores, four outside simulators); the omni-v10 LOSS, the pod start wait under KEDA's HTTP add-on, reads STEADY on omni-v11 now that the kind rig pre-pulls the served image (`docs/POD_START_WAIT.md`); on real Kubernetes Omni-Compass's 95th and 99th percentiles were below native's in all 90 paired repetitions (`docs/TAIL_LATENCY.md`). The fault: in the drone swarm (class S) the omni-v11 wire spent the closest-approach margin in the counted cells (no near miss, no collision) and, on a mission draw outside them, let drones collide where native did not; its proximity reflex slowed over a second instead of at once. The fix held every drone outside the keep-out on every draw tried and landed in omni-v12, whose swarm runs judge a collision as a LOSS and fly three draws never flown (`docs/OMNI_V12.md`; `docs/LIMITATIONS.md`, section 3; `results/model_gate/V12_SWARM_REFLEX.md`). The governors' first tables, on omni-v12,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Claim	Standing	Evidence
		read two LOSS rows. KubeRay with the organism alone: the queue 0.6% to 0.8% later and the tail 4% to 13% higher, because CPU limits the conveyance gave stayed on a saturated shared host; fixed in omni-v13 (`docs/OMNI_V13.md`; `docs/BATCH_QUEUE_PREREGISTRATION.md`, amendment 11). Crane: the total pod start wait, a sum of the pods Omni-Compass chose to start and the wait per pod, which the metric-aims audit rules on in writing before any table is read by it
More work, faster, on real Kubernetes	**Shown** (L, omni-v10 and omni-v11, each test on its newest table)	See section 2.3
More work, faster, on databases and caches	**Small or STEADY** (L, omni-v11)	Redis at the operator's ceiling: 4 gauge-rows GAIN at the tune-down, 6 at the max (work inside the line and hit rate up about 1%, for about 2% memory); Kafka 1 GAIN at each objective; MongoDB 1; PostgreSQL and MySQL 0, with every other gauge STEADY; native exactly as shipped: 0 GAIN, 0 LOSS (the knob at its limit on native or untouched). MySQL's update workload reads "the runs disagree" on throughput (±0.4%, a run-level effect: `docs/LIMITATIONS.md`). The runs at the max for MySQL, MongoDB and PostgreSQL are archiving
Fewer machines	**Shown in one test** (L, omni-v10)	A queue of jobs: machines in service and node-hours cut 6% to 14%, the machines after the queue finished 10% to 22%, the queue finished

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Claim	Standing	Evidence
		STEADY
A smaller cloud bill	**Undetermined**	Azure, the engine of 13 October: no difference in the bill (`results/live/AKS_BILL.md`); the Azure sets run one at a time on the engine that stands; Google and Amazon have not run
Less energy	**Undetermined**	No physical meter yet; every energy figure in a live test is a declared model, not a measurement
More work per watt on a GPU	**Undetermined**	No run on the engine that stands
One governor over every muscle and organism	**Modelled** (S)	1,107 muscles in our own models: 527 GAIN, 580 STEADY, 0 LOSS (`results/realms_v5/REALMS_V5.md`); a few dozen knobs on real systems
Its mathematics	**Machine-checked where stated**	24 of 24 theorems proved by SMT and interval arithmetic on the engine as it executes: the compass law, the plug's clip, the verdict, the invariance of E, I and S, the tracking bound inside the basin and the steps to reach it, the bath's stability (`docs/ENGINE_PROOFS.md`); open: CERT-10 for every fixture, convergence of the whole six-state system (`docs/FORMAL_STATUS.md`)

2.3 Real Kubernetes, each gain beside its cost (L, three runs of ten pairs, kind on one GitHub runner)

The rows marked omni-v11 are the newest tables; the others are omni-v10's. omni-v23, the engine that stands since 31 October, runs every test again, and each test's three runs on it replace its row as they land (the conductor is running them).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Gain, the same in all three of our runs	What it cost
Steady work in steps	95th percentile cut 64% to 65%; time over the line cut 98% to 100%; alert minutes cut 100%	Machines and node-hours STEADY; CPU used, the governor's own included, −4.3% to −5.2%
Demand that wanders	95th percentile cut 57% to 66%; time over the line cut 38% to 45%; energy cut 0.3% to 0.4%	Machines STEADY; CPU used, the governor's own included, −1.3% to −2.1%
The load stepping up and down (all four at once)	Work inside the response line +47% to +79%; 95th percentile cut 56% to 61%	Machines and node-hours STEADY; CPU used, the governor's own included, −0.5% to −2.6%
A public day of demand (Google 2011)	95th percentile cut 56% to 62%; time over the line cut 74% to 77%	Machines STEADY; pods started +120% to +193% (power given); CPU used, the governor's own included, −3.1% to −3.3%
Faults injected	95th percentile cut 54% to 63%; 99th cut 43% to 51%	Replicas +14% to +20% (power given)
KEDA's HTTP add-on underneath (omni-v11)	95th percentile cut 72% to 74%; time over the line cut 98% to 99%; alert minutes cut 100%; replicas cut 11% to 20%	CPU used, the governor's own included, +49% to +57% (shown, not judged: idle processor given to the service); the pod start wait STEADY
KEDA on CPU and requests underneath (omni-v11)	95th percentile cut 52% to 53%; time over the line cut 97% to 100%	CPU used, the governor's own included, +29% to +31% (shown, not judged); replicas +2% to +3% (power given)
Goldilocks and the VPA underneath (omni-v11)	95th percentile cut 61% to 64%; time over the line cut 98% to 100%; alert minutes cut 100%; pods started cut 53% to 61%	CPU used, the governor's own included, +44% to +52% (shown, not judged); parked-worker energy +1.7% to +2.3% (power given)
A queue of jobs	Machines cut 6% to 14%; machines after the queue finished cut 10% to 22%	Queue finish time STEADY
Fixed capacity (omni-v11)	STEADY at every cap	—
A noisy neighbour	Time over the line cut 11%; the neighbour's own rows STEADY	—

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Gain, the same in all three of our runs	What it cost
CruiseKube underneath	STEADY	—

The processor given in the add-on tests was idle on machines already running. Where machines are billed by the hour, idle processor put to work costs nothing more unless it raises the machine count, and in these tests the machine count did not rise. Where processor is billed by use, it is a cost, and the table shows it.

2.4 What is undetermined, and what decides it

- **The cloud bill**: the Azure sets on the engine that stands (one at a time); Google once its key is stored whole; Amazon once its account is on the Paid plan.
- **Energy**: a rented machine with the processor's own energy counters and a wall meter.
- **The GPU**: the Lambda runs from one exact commit.
- **Production**: a deployment on a customer's system with their sign-off.
- **Outside review**: no outside party has reviewed, replicated or approved any result here.

2.5 How to check any number yourself

Every table names its runs; `scripts/raw_archive.sh open /run-<id>/` places the run's raw files at the path the table names, with their SHA-256 sums, and the table's own tool rebuilds the table from them (`tools/confirm_abc.py`, `tools/redis_abc.py`, and the like). `python3 tools/omni_version.py --commit <sha>` names the engine a run's commit carries. `python3 verify.py` runs every test and check the repository holds.

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

CHAPTER 3

The Referee's Grades, Category by Category, and the Plan to Raise Each

Our own grading of Omni-Compass in twenty categories, the way a referee or an underwriter would grade a study and a product: the grade given on the morning of 19 October, the grade the evidence supports now, the highest grade the category can reach, and what reaching it takes. These are The Omni-Compass LLC's own

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

grades; no outside party has graded, reviewed or approved them. A category with no measurement yet is **Undetermined**, never failed, and is graded once its measurement exists.

3.1 The chart

#	Category	Morning of 19 Oct	Now	Highest reachable	What reaching it takes
1	Concept	B+	**A−**	A	an outside reading of the concept against its precedents (`docs/RELATED_WORK.md`)
2	Engine code	B−	**B**	A	the next version's fixes (below), coverage of every engine file at 90%, mutation testing of the core; an outside code review for A
3	Engine stability	C−	**C+**	A	one version standing through its whole campaign, every benchmark's three runs, unchanged
4	Mathematics	C+	**A−**	A	CERT-10 proved for every fixture and the bath's amplitude as a number (`docs/ENGINE_PROOFS.md`, section 5)
5	Method	B	**A−**	A	rules timestamped outside our repository before their runs; the pairs the power page asks for
6	Real Kubernetes	B−	**B+**	A	every Kubernetes test's three omni-v11 runs in (seven waiting on their third run), and the cloud's multi-machine cluster
7	Databases and caches	C	**B−**	A−	more pairs a run (`docs/POWER.md`), the max for MySQL, MongoDB and PostgreSQL (archiving), gains where the knob has room
8	Cloud bill	Undetermined	Undetermined	graded on its result	the Azure sets at the cap (running), then Google and Amazon
9	Energy	Undetermined	Undetermined	graded on its result	a physical meter: Kepler and the machine's own meter on a bare-metal machine

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Category	Morning of 19 Oct	Now	Highest reachable	What reaching it takes
10	GPU	Undetermined	Undetermined	graded on its result	the Lambda runs from one exact commit, on the founder's word
11	Modelled muscles	C−	**C−**	B	each plant model calibrated against a measured system, with its error stated
12	Outside simulators	C+	**B−**	A	the swarm's three runs on omni-v12, which carries the reflex fix, with its judged collision row and draws beyond the fixed seeds (below), and BOPTEST beside CityLearn
13	Robustness	B	**B+**	A	the omni-v11 kill and long-run tables are in (no LOSS, every hand-back inside its allowance); next the extended-fault table, a longer after-kill window, then a multi-machine cluster
14	Reproducibility	B	**A**	A+	an outside party rebuilding the tables from the receipts
15	Honesty	A−	**A**	A+	an outside audit of the receipts against the pages
16	Manuals	C+	**B−**	A	an editorial pass of both manuals at referee level, every figure and table against the code
17	Legal	B−	**B**	A	outside counsel's review of the license and the enterprise terms
18	Automation	B+	**A−**	A	the Kubernetes three-run tables built by a bot as the data stores' are; one page of every bot's health
19	Security	C	**B+**	A	the next version without a shell for commands that hold no pipe, signed releases, an outside penetration test
20	Production readiness	Undetermined	Undetermined	graded on its result	a pilot on a customer's own system (`docs/PILOT_PROTOCOL.md`)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

3.2 What moved each grade (19 October)

1. **Concept.** Where the idea comes from and what it adds, against supervisory control, reference governors, the Simplex fallback and online experiments (`docs/RELATED_WORK.md`).
2. **Engine code.** 71% of the engine's 9,685 statements run under the tests (`docs/ENGINE_COVERAGE.md`, measured again on the omni-v15 tree): 82% of the 6,579 in the files that stood before omni-v15, 49% of the 3,106 in the eight runners it took into the fingerprint, which their own benchmarks run whole on GitHub; every proved property checked on thousands of generated cases (`tests/test_engine_properties.py`); every static-scan finding fixed or reviewed (`docs/SECURITY_SCAN.md`). Held at B by a fault found in the swarm's wire, whose code did not do what its own comment says (`docs/LIMITATIONS.md`, section 3), and by the wires outside the fingerprint (`docs/CHANGE_CONTROL.md`).
3. **Engine stability.** The rules and the record of every version, why it changed and how results stay tied to it (`docs/CHANGE_CONTROL.md`); the pace itself is unchanged.
4. **Mathematics.** 24 of 24 theorems proved by machine on the engine as it executes: the compass law, the plug, the verdict, the invariance of E, I and S, the tracking bound inside the basin and the steps to reach it, and the bath's stability (`docs/ENGINE_PROOFS.md`).
5. **Method.** The smallest change every judged row can see (`docs/POWER.md`); every change to every preregistration with its timing (`docs/DEVIATIONS.md`); the chance of a false three-run reading, 3.1×10^{-5} a row (`docs/LIMITATIONS.md`).
6. **Real Kubernetes.** The omni-v11 tables for Goldilocks, both KEDA setups and fixed capacity: 0 LOSS; the omni-v10 LOSS reads STEADY; Omni-Compass's 95th and 99th percentiles below native's in all 90 paired repetitions (`docs/TAIL_LATENCY.md`).
7. **Databases and caches.** Ten omni-v11 tables, 0 LOSS; native as shipped kept apart from the operator's setting (a fault in the front page's grouping found and fixed before any table mixed them).
12. **Outside simulators.** CityLearn 27 GAIN, the power grids 67 GAIN, the robot arms 7 GAIN, the swarm 9 GAIN, 0 LOSS in all four on omni-v11, each rebuilt byte for byte. Held at B− by a fault found in the swarm's wire: on a mission draw outside the preregistered seeds it let drones collide where native did not. The fix held every drone outside the keep-out on all ten cells and draws tried (`results/model_gate/V12_SWARM_REFLEX.md`) and landed in omni-v12; the grade moves when its three runs land.
14. **Reproducibility.** All 23 omni-v11 tables rebuilt from their raw receipts and identical to the byte (`tools/reproduce.py`, weekly in `reproduce.yml`); every package pinned with its hashes, every action by its commit.
15. **Honesty.** The limitations page, the tail-latency page, the runs-disagree rows each explained, every row kept.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

16. **Manuals.** Eleven referee chapters bound into the Owner's Manual (the proofs, the coverage, the power, the deviations, the limitations, change control, the security scan, the licenses, related work, the tail latency, these grades).
17. **Legal.** The bill of materials (CycloneDX 1.5, SPDX 2.3) and every dependency's license, with no strong copyleft allowed (`docs/DEPENDENCY_LICENSES.md`).
18. **Automation.** Every finished run archived by itself; the reproduction and the security scan run by themselves; the conductor stamps each capacity run with its own letter.
19. **Security.** Bandit over 34,000 lines with every high and medium finding fixed or reviewed; 0 known vulnerabilities in the 29 pinned packages; every package hashed; 407 action uses pinned to their commits (`docs/THREAT_MODEL.md`, T8 closed).

3.3 The blueprint

On GitHub already, at no cost. The robustness kill test and long run and the three data stores at the max landed on 19 October (no LOSS in any); the third runs of the seven Kubernetes sets, the extended fault test and the CruiseKube rerun on the corrected recorder are running, and the cost to match runs again from the campaign. Each lands as an omni-v11 table and raises categories 6, 7 and 13.

The next engine version, on the founder's word (`docs/CHANGE_CONTROL.md`: any change to a rule or guard is the next version, and every benchmark runs again on it):

- the swarm's proximity reflex at once, reaching out with the square of the speed (in our own flights: every drone at or outside the keep-out on the four preregistered cells and six draws the wire was never run on, where omni-v11 collided once), with a judged collision row and draws beyond the fixed seeds in its preregistration;
- every wire that holds a governor's rules taken into the fingerprint;
- a command with no pipe run without a shell;
- the Redis harness's brain in its own process, as a deployment runs it;
- the Kubernetes governor counting a worker an add-on marks while it serves, and keeping its clock under load.

What a buyer will ask for that is not here yet (the founder's question of 19 October, answered from the repository):

- *Proof*: a physical meter (every energy figure is a declared model; the GPU on Lambda or a metered machine closes it); a cloud bill on the engine that stands (Azure running, Google's key and Amazon's plan pending); native at its best (the cost to match, native tuned harder by its operator beside Omni-Compass on top, last run on 3 October on an engine long gone, runs again on omni-v11 from the campaign); an outside party's rerun; a real multi-machine cluster (kind puts every node on one runner); days to weeks on a real workload; each rule's share of the gain read on the live cluster, not only in our models.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- *Gauges*: measured joules and grams of carbon; the cost of 1,000 requests; the 99.9th percentile (the probes are too sparse for it); the operator's own terms (availability, error budget, burn rate) beside the alert minutes; each muscle's headroom to its own limit; the governor's own calls to the cluster's API. Now read for every omni-v11 Kubernetes table (`results/STEERING.md`): moves, reversals and distance travelled an hour, the share of decisions with the compass in its band and at the line, the time from the line back into the band, and errors an hour.
- *The operator's screen*: in. The gauges feed and a Grafana screen (`tools/omni_exporter.py`, `deploy/grafana/omni-compass.json`, read-only); ten alert rules (`deploy/prometheus/omni-compass-rules.yaml`); and the control panel (`tools/omni_control.py`): an authenticated API with an OpenAPI document and a console with the buttons (mode autopilot, watch or off; the dial, max or tune-down; reset to native; the kill switch for every governor on the machine; the two-person arm), roles on every button and every press logged. It writes no setting itself: every change goes through the governor's own hand-back and a restart with the new arguments, so the engine does not change for it (`docs/OBSERVABILITY.md`). Still to build: a row per muscle with its own hand-back and watch-only buttons (the engine that stands takes its mode for the whole governor, so that comes with the next version, through the model gate); the live native-against-Omni panel; a one-line install (a Helm chart); two governors with leader election.

Mathematics. CERT-10 for every fixture by entry bounds taken cell by cell instead of over the whole box; the bath's amplitude from a discrete Lyapunov function per cell.

Tests. Every engine file to 90% coverage; mutation testing of the core, the compass law and the verdict.

Outside parties (the founder decides each): a referee's review, a penetration test, counsel's review of the license, and an independent replication from a fork.

The undetermined categories. The Azure sets at the cap are running; a bare-metal machine with Kepler and its own meter measures energy; the Lambda runs measure the GPU from one exact commit; a pilot measures production.

CHAPTER 4

Related Work: Where the Idea Comes From, and What It Adds

`docs/COMPARISON.md` sets Omni-Compass beside the systems that run data centres today (the Kubernetes autoscalers, Borg, Twine, Turbonomic, the power managers). This page sets it beside the published ideas it is built from, so a referee can see which parts are known technique and which part is the claim. Each idea is named with the work that stated it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

4.1 A supervisory layer over a controller that already works

Omni-Compass never replaces the native controller. It moves the same settings native moves, inside bounds the system and its operator set, and native keeps running underneath.

- **Advanced process control over regulatory control.** In process industries the fast loops (PID controllers on valves and pumps) are left in place and a supervisory layer, usually model predictive control, moves their setpoints (S. J. Qin and T. A. Badgwell, "A survey of industrial model predictive control technology", *Control Engineering Practice* 11(7), 2003). Omni-Compass has the same layering: the native autoscaler, pool or firmware is the regulatory loop; Omni-Compass is the supervisory layer.
- **Reference and command governors.** A governor modifies the reference given to a loop that is already stable, so that constraints hold, and leaves the loop itself alone (E. Garone, S. Di Cairano and I. Kolmanovsky, "Reference and command governors for systems with constraints: a survey on theory and applications", *Automatica* 75, 2017). This is the closest formal relative of Omni-Compass: it moves a setting, inside a cover, on top of native.

What differs. Reference governors are usually designed from a model of the closed loop and prove constraint satisfaction from it. Omni-Compass carries no model of the system it governs: it reads the system, steps, and lets each step stand only if a paired trial on the system itself shows it did no harm (section 4.3). Its own law is proved bounded, monotone and Lipschitz (`docs/ENGINE_PROOFS.md`, C1 to C8), but the safety of a step in a given system rests on the trial and the cover, not on a model of that system.

4.2 A fallback that is always there

- **The Simplex architecture.** A high-performance controller runs beside a simple, verified safety controller, and a decision module hands control to the safety controller whenever the system leaves a recoverable region (L. Sha, "Using simplicity to control complexity", *IEEE Software* 18(4), 2001).

Omni-Compass's fallback is native itself. Every setting is snapshotted before the first write and handed back at the end, on the kill switch (instantly) and, if Omni-Compass dies, by the lease's walk back (`docs/SAFETY_CASE.md`, `omnicompass/master.py`). The fallback is not a second controller written for the purpose; it is the controller the system was already trusted with.

4.3 Changing a live system one tested step at a time

- **Online controlled experiments.** A change is shown to a share of live traffic and kept only if a paired comparison says it helped (R. Kohavi, R. Longbotham, D. Sommerfield and R. M. Henne, "Controlled experiments on the web: survey and practical guide", *Data Mining and Knowledge Discovery* 18, 2009).
- **Safe exploration and shielding.** A learning controller may explore only inside a set a shield has proved safe (M. Alshiekh et al., "Safe reinforcement learning via shielding", AAAI 2018); control barrier functions keep a system inside a safe set by construction (A. D. Ames et al., "Control barrier functions: theory and applications", European Control Conference 2019).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Omni-Compass's verdict (`omnicompass/verdict.py`) is a controlled experiment run on the governed system itself, one step at a time: a step is allowed only when its paired trial does not read LOSS against either the setting before it or native, its cost stays within 2% of the cost first measured at native, and steps cannot add up past that (proved: V1 to V3). The cover plays the shield's part: no write leaves it (proved: P1 to P3).

4.4 Feedback control of computing systems

- **Control theory applied to software.** Queue lengths, utilisations and response times are controlled with classical feedback, analysed with the tools of control engineering (J. L. Hellerstein, Y. Diao, S. Parekh and D. M. Tilbury, *Feedback Control of Computing Systems*, Wiley-IEEE Press, 2004).
- **Production systems.** Google's Autopilot sets containers' limits and replica counts from their history (K. Rzadca et al., "Autopilot: workload autoscaling at Google", EuroSys 2020); Borg schedules the work (A. Verma et al., "Large-scale cluster management at Google with Borg", EuroSys 2015); Heracles uses feedback to share machines between latency-critical and batch work (D. Lo et al., "Heracles: improving resource efficiency at scale", ISCA 2015); power is provisioned and capped across whole facilities (X. Fan, W.-D. Weber and L. A. Barroso, "Power provisioning for a warehouse-sized computer", ISCA 2007; Q. Wu et al., "Dynamo: Facebook's data center-wide power management system", ISCA 2016).

Omni-Compass's law is a bounded, smooth one (the compass law, a tanh of a proportional-derivative reading inside a band with 5% cushions), not a linear PI loop, and the same law with the same verdict is applied to every knob it governs, from a pod's CPU limit to a battery's charge rate.

4.5 What Omni-Compass claims, against this background

Each part has a precedent: the supervisory layer (section 4.1), native as the fallback (section 4.2), the tested step (section 4.3), feedback on computing systems (section 4.4). The claim is the combination, applied the same way to every muscle of a stack:

- one bounded law and one verdict for every knob, on top of each knob's own native controller;
- a step allowed only when the system itself shows it did no harm against native, with the cost bounded stepwise;
- every setting handed back to native's value at the end, on the switch, or on the governor's death;
- results read only by a preregistered three-run rule, every row shown.

Whether the combination does what it claims is a question for the measurements, not for this page: `docs/REFEREE_SUMMARY.md` gives each claim's standing, and `docs/LIMITATIONS.md` what the measurements cannot yet say.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 5

The Seat: One Governor over the Muscles

This page states where Omni-Compass sits among the systems that already run computers, power and machines, what each of those systems is, who owns what, and what is still to be shown. Every statement about another system is read from that system's own publication or repository, on the date given; every statement about Omni-Compass is read from this repository's code and results. Where a statement is wrong or out of date, it is corrected from the maker's own publication.

5.1 The claim, in one sentence

Omni-Compass is a **supervisory governor**: one control law that reads every muscle in a body at once and writes each muscle's own knob, inside the envelope its native controller already allows, under the limits the muscles share, while every native controller keeps running underneath, unchanged. It is never standalone and never a replacement for the is").

What makes it one governor and not a collection of tools is stated formally in `docs/FORMAL_V5.md` (Definition 2 and Proposition 1): the knob it writes on one muscle depends on what every other muscle is doing, through the lines they share (a battery site's grid connection, a room's cooling). A collection of separate tools cannot have that property, however many run at once: each decides on its own reading and writes only its own knob.

5.2 Muscles and governors

A **muscle** is a controller of one kind of knob on one kind of system: how many machines, how many replicas, a power limit, a frequency, a temperature setpoint, a cache size. A **governor** decides, for the whole body at once, what each muscle may do. The systems in common use are muscles, or optimizers over a few kinds of muscle; Omni-Compass sits on top of them and calls them.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

5.2.1 The five systems named most often

System	Maker	Source	What it controls (its maker's description)	Kind	Omni-Compass with it
IBM Turbonomic	IBM	closed, commercial	resource actions across layers: sizing, placement and scaling of workloads and hosts	an optimizer over several kinds of muscle, not one control law	on top of the muscles it drives, or compared on the customer's own system in paired runs; it cannot be run here
Karpenter	AWS; its core is a Kubernetes SIG Autoscaling project (`kubernetes-sigs/karpenter`)	open source (Apache-2.0)	provisions and removes nodes for pending pods	node-provisioning muscle	on top: Omni-Compass decides when nodes are needed ahead of demand; Karpenter provisions them (register row 68, on a cloud)
NVIDIA's GPU power-management software (power profiles and the dynamic power tools above the driver)	NVIDIA	closed (user-space software on top of the driver; the Linux kernel and its NVIDIA modules are a separate layer)	power limits and power profiles on NVIDIA GPUs	power-limit muscle	on top: Omni-Compass writes the card's clock ceiling and power limit through the driver's own tool (`nvidia-smi -lgc`, `-pl`; `omni_controller/gpu_compass.py`), inside the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

System	Maker	Source	What it controls (its maker's description)	Kind	Omni-Compass with it
					envelope the card allows; the receipt is the chip-level energy run (section 5.5)
Energy Aware Runtime (EAR)	Barcelona Supercomputing Center	open source (EPL-2.0, `github.com/eas4dc/EAR`)	CPU frequency selection per application and node, energy accounting, and a global energy manager for supercomputers	energy muscle with its own optimizer at node and job level	on top, on bare metal with access to the CPU's power registers (register row 87)
Cluster Autoscaler, with commercial layers such as ScaleOps and Cast AI	the Kubernetes project; the vendors	Cluster Autoscaler open source (Apache-2.0); the commercial layers closed	adds nodes when pods cannot be placed, removes them when unneeded; the commercial layers sell optimization on top	node muscle, and optimizers above it	on top of the Cluster Autoscaler (register row 68); the commercial layers are compared on a customer's system, as Turbonomic

Run all five at once and there are still five systems, each reading its own gauges through its own interface and writing its own knobs. Nothing among them makes one decision for the whole body under the limits the muscles share. That is the seat Omni-Compass is written for.

5.2.2 Open-source pieces that can be benchmarked with Omni-Compass on top

Read from each repository on 12 October 2026 (its README, its license file and its last commit). The license is the file in the repository, not a description of it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Project	Repository	What it is, in its own words	License found	Last commit	Kind	Where it can run	Register
Kepler	`sustainable-computing-io/kepler`	"a Prometheus exporter that measures energy consumption metrics at the container, pod, and node level in Kubernetes clusters"	Apache-2.0	2026-10-12	a meter, not a controller	a machine with the processor's power registers (RAPL), through the meter kit (`docs/METAL_KIT.md`); on GitHub's virtual machines Kepler 0.10 and later run only a stand-in development meter	row 79
Goldilocks	`FairwindsOps/goldilocks`	"a utility that can help you identify a starting point for resource requests and limits" (built on the Vertical Pod Autoscaler's recommender)	Apache-2.0	2026-09-18	a recommendation tool over a muscle (VPA)	GitHub's machines (kind)	row 80
CruiseKube	`truefoundry/cruisekube`	"an intelligent Kubernetes resource optimization controller that automatically monitors,	**Business Source License** (source-available, not open source)	2026-08-06	a rightsizing muscle	GitHub's machines (kind), for evaluation under its license	row 81

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Project	Repository	What it is, in its own words	License found	Last commit	Kind	Where it can run	Register
		analyzes, and applies resource recommendations"					
Climatik	`Climatik-Project/Climatik-Project`	a power-capping operator for Kubernetes (its README is the operator template, unfinished)	Apache-2.0	2024-09-10	power-capping muscle, early stage	a machine with power control (GPU)	row 82
Kubernetes Power Manager (AMD EPYC)	`AMDEPYC/kubernetes-power-manager` (a fork of Intel's `intel/kubernetes-power-manager`)	"a Kubernetes Operator ... to dynamically request adjustment of worker node power management settings applied to cores allocated to the Pods"	Apache-2.0	2025-11-07	CPU power muscle	bare metal with the processor's power controls	row 83
PEAKS	`sustainable-computing-io/peaks`	"Power Efficiency Aware Kubernetes Scheduler" (its README is meeting notes; the design is in its docs)	Apache-2.0	2024-10-23	a scheduler, early stage	GitHub's machines (kind) once it runs	row 84

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Project	Repository	What it is, in its own words	License found	Last commit	Kind	Where it can run	Register
Adaptive GPU Power Capping	`tanish111/Adaptive-GPU-Power-Capping`	the code for the paper "Adaptive GPU Power Capping: Balancing Energy Efficiency, Thermal Control and Performance" (HPDC '25): a learned model choosing GPU power caps	**no license file** (no right to reuse the code without the authors' permission)	2025-04-29	a research power-capping method	a rented GPU; compared from the paper's own method unless the authors grant use	row 85
NVIDIA Power Manager	`NVIDIA/nvidia-power-manager`	"Nvidia power management app."	Apache-2.0	2026-07-13	platform power muscle (NVIDIA server platforms)	NVIDIA server hardware	row 86
Energy Aware Runtime	`eas4dc/EAR`	"an energy management framework for super computers"	EPL-2.0	2026-09-28	energy muscle and optimizer (HPC)	bare metal with the CPU's power registers, under a batch scheduler	row 87

Already benchmarked or registered: the Kubernetes HPA (every Kubernetes set), KEDA and its HTTP add-on (done, three runs), Karpenter, the Cluster Autoscaler and the Vertical Pod Autoscaler (register row 68).

None of these projects is described by its makers as one law deciding for the whole body. That is a statement about their public descriptions on the date above, not a proof that nothing else exists anywhere.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

5.3 How a benchmark with each of them is built

The same way as every benchmark here: the project, installed as its makers ship it, is the native; Omni-Compass on top of it is the other arm; both arms see the same work; the rules are written before the counted runs, frozen on one tuning case and run three times (A, B, C); every row is reported, losses included (`docs/HOW_TO_READ_THE_RESULTS.md`). A meter (Kepler) is not an arm: it reads both arms the same way, so its numbers sit beside the arms' own gauges.

Order (one or two at a time): Goldilocks with the Vertical Pod Autoscaler, and Kepler as the shared meter, on GitHub's machines first; CruiseKube next under its license's evaluation terms; the GPU and platform power pieces in the Lambda kit; EAR and the processor power managers on bare metal when a machine with power registers is rented.

Built so far: **Goldilocks** (register row 80; `scripts/kind_addons_goldilocks.sh`, workflow `kind-goldilocks`, `docs/GOLDILOCKS_PREREGISTRATION.md`), run three times on the card version (`results/live/V5_GOLDILOCKS.md`): the service faster in all three of our runs, machines the same, 1.9% to 2.6% more energy on the declared model, every row shown. **CruiseKube** (register row 81; `scripts/kind_addons_cruisekube.sh`, workflow `kind-cruisekube`, `docs/CRUISEKUBE_PREREGISTRATION.md`), beside KEDA's HTTP add-on, since CruiseKube leaves alone a service whose autoscaler scales on CPU: a trial run proves the wiring and records its chart's SHA-256 for the pin, then three counted runs.

As muscles in the organisms (the founder's order of 12 October: "expand my organisms"): each of the nine is also a muscle of the catalog (`realms/catalog_v5.csv`, ids 2172 to 2181), modelled on the plant its knob turns and placed in the spine family that knob belongs to, so each is in all four realms, the four stacked and the whole tower: Kepler's energy reading driving a pool's target, Climatik's inference power cap, EAR's CPU frequency policy and global energy budget, CruiseKube's and Goldilocks's request rightsizing, the Kubernetes Power Manager's core profiles, PEAKS's power-aware placement, the learned GPU power cap and NVIDIA's platform power budget. A model of a knob is not the project itself (class S); the project's own benchmark is the live test of it.

5.4 Who owns Omni-Compass, and on what terms it is seen

- **The copyright in Omni-Compass belongs to The Omni-Compass LLC.** Contributions are accepted only on the terms of `.github/CONTRIBUTING.md`, which assign their rights to The Omni-Compass LLC.
- **The source is visible under the Omni-Compass Evaluation License (`LICENSE`): evaluation and Simulation use only.** It is not an open-source license. Any commercial use, commercialization, monetization, production use, redistribution or hosted service requires a signed, paid Omni-Compass Enterprise License. Seeing the source grants no right beyond the Evaluation License.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Two licenses from one owner.** The owner of a copyright may grant it under more than one license at once (an evaluation license to everyone, a paid enterprise license to a customer), may set the terms of each new version, and may assign the copyright. Copies already received keep the terms they were received under.
- **All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC.**
- This section states how the repository is licensed; the license texts (`LICENSE`, `NOTICE`, `DISCLOSURES.md`) govern.

5.5 What is shown, and what is still to be shown

What is shown, and by which kind of evidence, is set out receipt by receipt in `docs/THE_RECEIPTS.md`. In one line: on real software (class L), Omni-Compass on top of native has given faster service and more work inside the response line on the same machines, and memory back on databases at the same service, with no row LOSS in all three runs on the real stacks; on our own models (class S) no muscle alone is behind native.

The decisive receipt still to be produced is a physical one (class P): the chip's own energy meter on a real GPU, native against Omni-Compass on top, on the same clock and with the same membership as the Kubernetes runs. Every earlier card result ran on a controller since replaced and is obsolete. The run is the founder's, on rented cards, at one exact commit, after the CPU side is done (`docs/GPU_RUN_GUIDE.md`, `docs/GPU_PREREGISTRATION.md`). Until it lands, no energy saving on real machines is claimed. The seat is claimed by design and by the receipts on this page; it is held when the receipts cover it.

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

CHAPTER 6

The Unified Circle Principle

Omni-Compass begins with one idea: a system that is closed, bounded and pulled toward a center cannot run away, and everything it does is a return. I call this the closed circle. This chapter states the principle as I hold it, in its mathematical form, and then says plainly which parts of it the software uses and which parts belong to the wider theory the software grew out of.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

6.1 The statement

```
dX/dt = G(X),        X(0) in Omega, a subset of R^n
G(X) . n(X) <= 0     on the boundary of Omega
grad L(X) . G(X) <= 0
=>  lim (t -> infinity) X(t) in M*, a subset of F
```

Read line by line:

1. **dX/dt = G(X).** The state X moves according to one law G. There is no outside hand: every change comes from the law and the state it is in.
2. **X(0) in Omega.** The state starts inside the admissible region Omega: the set of states that are physically and operationally allowed.
3. **G(X) . n(X) <= 0 on the boundary.** At the wall of Omega, the flow never points outward (n is the outward normal). Nothing that starts inside can cross out. In control theory this is the Nagumo condition: Omega is forward invariant.
4. **grad L(X) . G(X) <= 0.** There is a function L, the energy of the compass, that never increases along the motion. This is a Lyapunov condition.
5. **=> X(t) settles in M*.** Together, an invariant container and a non-increasing energy force every path into the set M* where the energy stops falling: the bottom of the compass.

The power of the statement is that it does not depend on the particular disturbance. Every input up to the size the container is built for has the same answer before it is asked: the state stays inside and returns. That is what I mean when I say that a closed circle holds the answer to every question that can be put to it.

6.2 Closure, admissibility, boundedness

The theory sets five requirements on any law that claims to close the circle:

Requirement	Meaning	In the engine
Finite deviation	the distance from the admissible state has a ceiling, abs(E) <= E[max]	the deviation state E drains faster than it can fill
Finite curvature	the landscape the state moves on has no infinite slopes or pits	the drive is bounded by tanh; the authority is clipped
Global conservation	what flows in is matched by what flows out	the drain and the bath's friction match the drive
Recurrence	a disturbed state returns arbitrarily close to where it was	convergence to the target pole at rate K[P]
One law	every subsystem obeys the same law, no special cases	one engine, one compass, every muscle

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

6.3 The closed circle tested

The canonical evolution law has four parts: deviation is damped nonlinearly and coupled to alignment and to the basin; alignment relaxes toward unity and responds to deviation; the basin's curvature evolves with internal structure; damping absorbs excess growth and redistributes energy. Bounded random noise is permitted; divergence is not. A cycle is identified when the system returns arbitrarily close to a prior state within a bounded tolerance under noise. The state advances stepwise under a stable numerical integrator with boundedness checks. Under parameter sweeps the test is always the same three checks: no divergence, bounded, recurring.

6.4 Symmetry and conservation in the closed circle

Every conservation law is the shadow of a symmetry (Noether). The closed circle carries four:

Symmetry	Conserved	In Omni-Compass
Temporal (the law does not change with time)	energy	the same law governs every decision; the receipt compares like with like
Spatial (the law does not depend on where)	momentum	the same law governs every muscle in every realm
Rotational	angular momentum	the compass is symmetric about its center; the engine is symmetric between its poles
Internal alignment	alignment invariance	the alignment state is pulled to its pole whichever pole it is

Around them sit the boundary conditions of the circle: maximum deviation (the boundary limit), topological closure (bounded wholeness), nonlinear dynamics (complex interactions), resonant reversal (feedback return), energy-momentum conservation, and cross-scale invariance (scale-independent patterns).

6.5 Scale invariance

The theory claims that one bounded nonlinear action has the same form at every scale: cosmological (curvature geometry, compression and expansion, thermodynamic cycling), galactic (rotation curves, mass-energy distribution), stellar (gravitational compression, fusion equilibrium, radiative transport), atomic (electron displacement, orbital coherence, radiative damping) and quantum (localized deviation wells, bounded oscillation, finite amplitude). The universal criteria for such a law are that it operates at every scale and preserves bounded deviation, finite curvature and recurrence. The state vector carries the same meaning at every scale: E the deviation magnitude, U the coherence or flow alignment, S the structural curvature or basin geometry, B the redistribution or damping.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

6.6 The lineage of unification

Each step in the history of physics unified two things that had been separate: geometry formalized structure; motion was unified across earth and sky; astronomy revealed recurrence; fields replaced action at a distance; curvature unified gravity, though singularities remained; symmetry came to govern the quantum realm. The closed circle is offered as the next step: unity without divergence, a single bounded law with no singularity and no runaway.

6.7 What is tested and what is theory

The software uses the engineering core of this principle, and every part of that core is checked:

- The six-state engine, its bounded control command held through the integration step, and the convergence of its alignment state to its pole are proved (`docs/TRACKING_THEOREM.md`) and tested against a C++ twin.
- The invariance and boundedness of each engine state were checked state by state (Chapter "Closing the Circle").
- The compass law pulls every muscle's service reading to the middle of its band under a bounded, smooth force, and its behavior is measured in simulation and on real software.

The wider claims - one law across every physical scale, the cosmological and quantum readings, the candidate unified action - are the theory from which Omni-Compass was built. They are stated here so that the engineer understands where the design comes from. They are not claims that the software has measured, and no result in this book depends on them.

CHAPTER 7

The Four-Piece Engine

The engine is built in four pieces, stacked like the stages of a press, each feeding the next:

Piece	Name	What it establishes
1	Foundational Engine	the base laws, the states and the boundaries: a closed, bounded, nonlinear adaptive engine with a four-state dynamical structure (misalignment, alignment recovery, coherence potential, path interaction), bounded convergence and persistence with decay, intrinsic feedback and stabilization, and an empirical validation framework for recovery

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Piece	Name	What it establishes
2	Geometric Response	geometric alignment and response dynamics: a geometric coherence field, a unified potential governing geometric deformation, and the propagation mechanism of that response
3	Coherence Evolution	coherence through response and adaptation: the coherence-response envelope, the system response field, and the coupling of coherence state with response propagation
4	Unified Master Field Law	one coherent field structure: it integrates pieces 1-3 into one closed dynamical structure with a unified potential governing all subsystems, bidirectional propagation between engine state, geometry and response, bounded convergence across the whole system, and no external authority: all evolution emerges from internal coupling

The four pieces compress every domain through one center core. The compression process has four stages: the domains unordered; compression in progress; coherence achieved; return to equilibrium. The result has four properties: coherent, bounded, admissible, stable - one law, one return, one cycle.

The 24 domains. The theory gathers 24 domains into four quadrants around that core:

Quadrant	Domains
1. Foundational structures	Newtonian gravity; Maxwell electromagnetism; Einstein relativity; Schrodinger quantum mechanics; Shannon information theory; Dirac quantum field theory
2. Dynamics and symmetries	Noether symmetry laws; Hamiltonian dynamics; Lagrangian mechanics; thermodynamics; topology; differential geometry
3. Coherence evolution	gauge field theory; fluid dynamics; field theory; discrete mathematics; solid-state physics; plasma physics
4. Complex systems and integration	astrophysics; consciousness and information integration; biological systems and self-organization; universal mathematical structure; systems theory and complex adaptive systems; the unified Omni-Compass domain (the unified admissible state)

In the software, the four pieces appear as the engine's state equations (piece 1: deviation, alignment, memory and basin), the basin potential and its flow (piece 2), the bath that absorbs and redistributes (piece 3), and the one governor that couples every muscle to one law with no outside authority (piece 4).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 8

The Compass

The Omni-Compass rose is the whole mechanism drawn as one picture.

The ring. Twenty-four letters, the complete Greek alphabet from Alpha to Omega: the complete set, nothing missing, one letter for each domain. On a ring the end runs back into the beginning: after the last letter the ring returns to Alpha. That is the closed circle: identity, return, cycle.

The four points. Four letters are lifted out of alphabetical order and set on the four points of the rose:

Point	Letter	Mark	Meaning in the engine
North	Alpha, the first	+	the positive pole of the alignment's double well, U = +1
South	Omega, the last	-	the negative pole, U = -1
East	Beta	>	the drive coming in (the beta terms that fill the deviation)
West	Gamma	<	the coupling that carries energy out to the bath, where friction absorbs it

The north-south line is polarity; the east-west line is flow in against flow out. A system stays inside its circle exactly when what flows out can always match what flows in.

The inner marks.

Mark	Meaning	In the mechanism
+ and -	the two polarities	the two poles of the double well
> and <	push out, pull in	the antagonist pair: the up force and the down force
the double arrow	exchange both ways	the two-way wire: read in, write out
the up-tack	the ceiling	the cover: nothing goes past the top
approximately-equal	the wave, the tolerance	the band and its cushion: hold near the center
the star	the guiding star	the target the needle points to

The center. The spiral at the middle is the attractor: every path winds inward to one point.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Opposite letters. Across the center, zeta faces xi: both are the engineer's symbol for the damping ratio, and a damping ratio of one is the critical glide to the center with no overshoot. Lambda faces psi: the eigenvalue and the wave function of the eigenvalue equation.

CHAPTER 9

Basins, Polarity and the Dual-Basin Engine

9.1 Geometric funneling and basin formation

A basin forms in three stages. While the driving tension T is low, curvature is weak and there is no dominant basin: flows spread outward. As T increases, curvature concentrates and a positive feedback loop begins. Then a stable geometric funnel forms: self-reinforcing curvature, finite depth, bounded deviation, with the boundary abs(E) <= E[max] respected. Structure emerges from fluctuation through curvature concentration and feedback.

In the engine, the basin state S rolls down its potential Phi(S) to a stable bottom; in the compass, every muscle's service reading is pulled into the bottom of its own funnel.

9.2 Polarity

Two like charges repel without anyone pushing them: the force comes from the shape of the field. A wall built of like polarity at the edge of a band pushes the state back harder the closer it comes (as one over the distance squared), and two such walls cancel at the center, where the state comes to rest. This is the boundary condition of the closed circle written as a force: the field itself points inward at the wall.

The repulsion does not create energy: the field stores what it took to bring the charges together. What is gained in the computer is that the force is computed rather than spent: the brain computes it at no physical cost, and the machine's energy is saved because its knob no longer fights itself.

A symmetric pair of repelling walls was tested on the modelled GPU card. It raised work per energy but also raised the time spent past the service line, because the lower wall pushed capacity away when the card was calm. It was removed. The upper wall alone remains a candidate.

9.3 The dual-basin engine

The dual-basin engine is a recursive transform with five stages:

1. **Compression.** Two outer basins of positive polarity draw matter and energy inward in a spiral; rotation organizes the flow; compression increases toward the center.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2. **Saturation.** At the center, two cores of negative polarity meet back to back; the basin structure S reaches its critical value and the curvature reaches its maximum.
3. **Transition (ignition).** Where the cores touch, the transition operator T(X) = sigma(S - S[c]) T(X) activates: compression halts and redistribution begins.
4. **Redistribution.** Two axial jets carry the excess out along the axis, with opposite helicity to the inflow.
5. **Reset and recurrence.** The domain relaxes and the cycle repeats.

The core mathematics is a bounded flow: dX/dt = G(X, t) with abs(G(X)) <= C[G], a curvature envelope K(X) <= K[max], and a redistribution term that activates only at saturation. The state X = (E, U, S, B) stays in its admissible domain, whose horizon boundary is forward invariant (the Nagumo condition again).

For an engineer, the transform reads: pull toward the center (compression); a wall where the muscle saturates; and, when one muscle saturates, move the excess to its siblings rather than hammering it (redistribution). The first two are built into the compass. Redistribution across muscles is designed and not yet built; it is on the roadmap.

CHAPTER 10

The Reflex Rule: One Body, One Brain

A body does not decide in advance how much force a muscle will need. It sends a signal, the muscle moves, the nerves report what the movement did, and only then does the brain choose the next signal. Nothing in that loop is a forecast. The decision is made on the reaction, after the reaction, and from the reaction. The founder's order of 10 October 2026 puts the whole of it in one sentence: what the brain sends to a muscle has to coincide with what the muscle sends to itself and where it is at. This chapter is the theory of that sentence, and the reason it is the whole mechanism rather than one more rule beside the others.

10.1 The reflex, not the forecast

The compass of the earlier chapters is a force: where a reading sits in its band, how hard the two opposing forces push it back toward the middle, and how smoothly. A force alone does not say when it may become a new setting. A governor that turns every urge straight into a move is a governor that guesses, and a guess made on a machine that is already working is a guess paid for by that machine. The reflex rule closes the gap: a new setting is earned by a trial on the muscle itself, and the trial is read from the muscle's own reaction. The urge proposes; the reaction disposes.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

10.2 A trial runs to its end

The rule that taught this was broken in a measured run. On 10 October the message broker's consumer group was given a spend trial: one more consumer, then measure. The extra consumer drained the waiting messages so fast that the service went calm at once, and the harness of that day ended the trial the moment the calm arrived, before the measurement it existed to take. Every spend was abandoned by the success it caused. A trial that can be ended by the calm it creates can never prove that it was worth it. Hence the first rule: a trial runs to its full measurement and is never ended by the calm it causes; only the wall, a service reaching its line, or the engine's own time limit ends it early. In the one repetition of that day in which the brain allowed the third consumer early, the whole of the earlier gain came back at a fraction of the spend: the slowest five percent of messages from 2,450 ms to 57 ms, the queue from 3,375 waiting messages to 253, the work done inside the line up 17%, with 2.5 consumers on average where the earlier law had spent six to eight. The gain is real and cheap; what was missing was a fair trial of it.

10.3 No forcing: the need and the urge

A spend is tried only when the muscle's own reading says there is something to buy: messages waiting, a cache missing, clients queued at a pool. A give-back is tried only when the whole body is calm. The compass force is the urge, smooth and bounded; the muscle's own condition is the need. The brain never moves a muscle because it wants to. It moves it because the muscle has shown that a move can pay, and then only by trial.

10.4 One body, one brain, one tick

A muscle judged alone can be read GAIN by pushing its neighbour away from its own: a cache that takes memory from the database beside it, a consumer that takes processor time from the service it feeds. So the brain reads the whole body first, every muscle's state, every service's speed and work, the host's processor, memory and energy where a meter exists, and judges the body's cost: the Omni index's own arithmetic over all the work, all the speed, all the machines and all the resources. On top of the body's total stands a guard for every part. A step is refused if any one service read LOSS beyond its cushion, even when the total improved. Pushing one thing to a LOSS to move another toward its own is refused by rule, not by luck.

That guard is do no harm in its strictest form, and it is the form the engine that stands keeps. The founder's words of 19 October 2026 give do no harm its meaning (`DISCLOSURES.md`, section 3): the real goal is the organism, to strengthen it and keep it healthy, working at the best state its own muscles can carry, turning whatever is fed into it into the most and best output it can make. Harm is the organism turning out less than native makes it turn out, or made less healthy: a muscle past its own limit, a service over its own line, anything made to fail. Within that, the muscles are one body: where one muscle's strength cannot be used without straining another, the brain offsets and engages them together, and it may use one muscle less, or leave it native, so that another can carry more, whenever the whole organism comes out ahead and every muscle stays healthy. The guard above refuses every such trade, the good with the bad; the next engine version replaces it

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

with that judgment of the whole organism and of every muscle's health, through the model gate first, and every result is run again on it.

10.5 One trial at a time, nothing permanent, the wall belongs to the body

If two muscles move together, the brain cannot tell which one caused what it feels. Inside a body, one new trial runs at a time, granted to the muscle asking loudest whose own condition holds; every other muscle keeps acting inside what it has already proven, so nothing is frozen and only new trials wait their turn. Every allowed step is tried again on the recheck and pulled back when it stops paying, and a muscle at native is asked again whenever its signal returns: native is where the brain stands when it has not yet been shown a reason, never a verdict. And if any service reaches its line, the trial stops, that step is undone, and every force in the body lifts off. The wall is the body's, not the muscle's.

10.6 Why this is the closed circle again

The Unified Circle Principle says that a system closed on itself, bounded, and pulled toward its center cannot run away: everything it does is a return. The reflex rule is that principle carried into time. The loop is closed through the system's own reaction; every excursion is bounded by the trial's limits, the cover and the wall; and every allowance is pulled back to native unless the body keeps proving it. The compass holds each reading in the middle of its band. The reflex holds each decision in the middle of what has been shown.

10.7 What it means for the evidence

Omni v3, the engine on main, already tries a slow knob on the muscle itself before it moves it, and the live harnesses carry the same verdict around every live knob. Omni v4 is the engine written to make the rule hold on every wire by construction: one body that every muscle's wire passes through, the trial's rules inside the verdict, and no wire that can write around them. Because that changes the engine, v4 carries its own fingerprint, the package version follows it, and every result in this book is run again on it. No result is read across engines.

PART TWO

The Mathematics

Eight equations, one control law, and the proofs, audits and declarations that hold them fixed.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 11

The Canonical Declaration

Every report the engine has ever produced opens with the same declaration. It is the birth certificate of the mechanism: what the engine is, which equations it runs, what every symbol means, which parameters it samples, how a run is born, conveyed and certified, and what the engine is allowed to say about itself. This chapter sets it down in full so that anyone who reads the rest of this book can always come back to the source.

11.1 The identity of the engine

The Omni-Compass Unified Governing Convergence Control Core Engine. One canonical eight-line mathematical core, called Piece I, integrated by fourth-order Runge-Kutta (RK4) and exercised by Monte Carlo. Around that core sit the benchmark suites: the public enterprise manager mechanism benchmark, the one-stack-normalized global megastack, fragmented stack governance over matched scenarios, and the enterprise evidence suite covering cause, fail-safe, scale, muscle, security and economics. All of them run on bare metal, on premises, private, public, hybrid and edge alike, because the core does not care where it runs.

11.2 The equation form (authoritative, eight lines)

```
(1) dE/dt   = -α_E E + β_int(t) + β_ext(t) + v_eff
(2) dU/dt   = μ U(1 - U²) - (dE/dt)/E_max - λ_U U + u,      |u| ≤ U_AU
    ↪ THORITY
(3) dI_U/dt = 1 - U - σ₁ E - δ S - λ_I I_U
(4) v_eff   = χ(t) c · tanh(λ₀ + λ₁(U - U_t) + λ₂ S),       χ(t) = cos
    ↪ (ω_B t / 2)

(5) Φ(S)    = ½ α_s S² + ¼ β_s S³ - δ S
(6) dS/dt   = F_state(S) = -∂Φ(S)/∂S = δ - α_s S - ¾ β_s S²
(7) B'' + (ω_B / Q_B) B' + ω_B² B = γ_c δ S                  [continu
    ↪ ous]
(8) R_B[n] = D²_h B[n] + (ω_B / Q_B) D⁻_h B[n] + ω_B² B[n] - γ_c δ S[n
    ↪ ] = 0    [discrete audit]
```

Line (1) is the error, the energy of misalignment: it decays at its own rate, it is pushed by forces from inside and outside, and it is pushed by the effective velocity of the system. Line (2) is coherence, the alignment index: it lives in a double well with two homes at +1 and -1, it is pulled down by any rise in error, it leaks, and it is the one line the governor touches, through the control u, which is never allowed beyond its authority. Line (3) is the information channel that keeps the record of how far coherence has been from home. Line (4) is the effective velocity, saturated so that it can never exceed the limiting speed c, and multiplied by the spinor

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

phase. Lines (5) and (6) are the potential of the coherence state and the flow it creates, downhill. Line (7) is the bath, the field that every real system sits in, driven by the state. Line (8) is the bath written on the grid the computer actually steps on: it is an audit of the numbers, not a second bath.

11.3 The control law

```
σ        = nearest_signed_basin(U₀)
u_m      = sat(-f_U(x_m, t_m) + K_P (σ - U_m), ± U_AUTHORITY)
x_(m+1)  = RK4_h(x_m; u_m),   u_m held across all four RK4 stages
```

The nearest signed basin is declared at Step 0, before the first microstep. The control cancels the natural drift of coherence and adds a proportional pull to the declared home, then it is clipped at its authority. It is held fixed through the four stages of each RK4 step. There is no post-step overwrite of the state and no projection onto the basin. Whatever the engine reaches, it reaches by the flow of its own equations.

11.4 The symbol chart

Symbol	Meaning
E	Error, the misalignment energy
U	Coherence, the alignment index
I[U]	Information, the internal-coherence channel
S	Coherence state potential
B	Bath field
B'	Bath field time derivative
α	Feedback rate
β[int] (t)	Internal forcing function
β[ext] (t)	External forcing function
k	Coherence logistic damping gain
σ_1	Coupling of error into the information channel
δ	Linear coherence coupling parameter
λ_0	Saturation bias
λ_1	Saturation gain
λ_2	S-state coupling gain
U_t	Target coherence state
c	Limiting velocity

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Symbol	Meaning
E[max]	Per-trajectory error-rate normalization scale
α_S	S-potential quadratic coefficient
β_S	S-potential cubic coefficient
$\Phi(S)$	Potential field function
F[state] (S)	Canonical S-state flow
$\omega[B]$	Bath frequency
Q[B]	Bath quality factor
$\gamma[c]$	Bath coupling gain

11.5 The coupling architecture of Piece I

The closed feedback core is E, U and S: error drives coherence, coherence and the state drive the velocity, the velocity drives error. The information channel I[U] and the bath B with its derivative are driven response channels: they listen to the core and record it, and the core does not depend on them. γ_1 is reserved for compatibility and is inactive in the Piece I derivatives. The bath law uses ω_B^2 B, so $\omega[B]$ carries the units of angular frequency. Line (7) advances B and B' continuously by RK4. Line (8) evaluates the discrete residual with $D^2_h\, B[n] = (B[n+1] - 2B[n] + B[n-1]) / h^2$ and $D^-_h\, B[n] = (B[n] - B[n-1]) / h$.

11.6 The spinor closure

The spinor extension introduces the phase $\theta(t) = \omega[B]\, t$ and multiplies line (4) by $\chi(\theta) = \cos(\theta/2)$. At 0 degrees the factor is +1, at 360 degrees it is -1, and at 720 degrees it is +1 again. A full turn of the bath reverses the sign of the velocity; only two full turns bring it home. The standard 20-step run evaluates the part of the phase it reaches, and a dedicated evidence test checks the 0, 360 and 720 degree periodicity on its own.

11.7 The signed basins

The canonical Monte Carlo sampler draws the starting coherence U(0) symmetrically on [-1, +1]. The nearest signed basin at Step 0 is declared the target before the first microstep. That keeps the eight-line equation exactly as written while making every governed run exercise capture into both +1 and -1. A separate dual-basin suite throws runs across from one basin to the other under disturbance and watches them recover.

11.8 Birth, conveyance and certification

Step 0 is the sampled state before integration. Step 1 is the first completed macro step. A run has at most 20 macro steps of 10 micro steps each, at a time step of 0.1. A step is in basin when U lies within 0.10 of +1 or -1.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Born in basin.** A run that starts inside a basin at Step 0 is born in basin for life. Step 0 counts as the first observation of its streak. It is confirmed conveyed after Steps 0 to 4 stay in the same basin. It is never called self-tuned, and it stays under the same regulation for all 20 macro steps.
- **Self-tuning.** A run that starts outside both basins is self-tuning applied from Step 1. The same nearest-basin law is active on every trajectory, born or not, over the whole 200-microstep horizon. The category never switches the regulation off.
- **CONVEY-5.** Five consecutive same-basin observations. The first is the conveyance entry step, the fifth is the confirmation step. The latest streak may start at Step 16 and confirm on Step 20.
- **CERT-10.** Ten consecutive same-basin observations. The latest streak may start at Step 11 and complete on Step 20. The code asserts both bounds before it runs and checks every emitted row against them.

Conveyance is an event, not an ending. No run may be both born in basin and self-tuned. Stability is judged on the history, not on one snapshot.

11.9 The Monte Carlo declaration

Setting	Value
Initial U distribution	Uniform on [-1, +1], symmetric
Monte Carlo runs	500
Macro steps per run	20
Micro steps per macro step	10
Integration time step	0.1
Basin centers	+1 and -1
Basin tolerance	0.10
Stability window	5
Conveyance cutoff step	20

The full horizon is always executed. A run is classified as a failure if it crosses the divergence threshold at any recorded state, or if it exhausts the horizon without CONVEY-5.

11.10 The parameter sets

Group	Parameter	Value or range
Core	α	4.2
Core	β[int]	0.0 to 1.0
Core	β[ext]	0.0 to 1.0
Core	E[max]	1.0 to 10.0, sampled once per trajectory

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Group	Parameter	Value or range
Core	k	1.7
Information and coupling	σ_1	0.38
Information and coupling	δ	0.01 to 1.0
Information and coupling	γ_1	1.35, reserved, inactive in the derivatives
Information and coupling	$\gamma[c]$	1.0
Saturation and phase	c	0.5 to 5.0
Saturation and phase	λ_0	-2.0 to 2.0
Saturation and phase	λ_1	0.1 to 3.0
Saturation and phase	λ_2	0.0 to 2.0
Saturation and phase	U_t	0.5
Saturation and phase	$\chi(t)$	$\cos(\omega[B]\, t / 2)$ when the spinor closure is on
S-state potential	α_s	0.05 to 0.25
S-state potential	β_s	0.05 to 0.25
Bath	$\omega[B]$	0.1 to 5.0
Bath	Q[B]	0.5 to 10.0
Noise and drift	D	0.0
Noise and drift	μ	0.0

11.11 The mechanism-bound reporting doctrine

Every table, grid, chart, scorecard, classification and conclusion the engine emits must be computed from what it actually executed: the trajectory rows, the recorded state histories of E, U, I[U], S, B and B', the sampled parameters, the RK4 derivative evaluations, the basin entry and dwell histories, the line (8) residuals, the solver comparisons, the ablations, the disturbance and stress runs, the compute benchmark event records, and the enterprise benchmark ledgers.

No decorative grade, inferred score, hand-assigned quality category or presentation-only number is allowed to stand as an engine result. Prose may point to the exact source and the edge of the claim. It may not manufacture evidence, put a grade where a measurement belongs, or present a grid the mechanism never ran as though it had. That doctrine runs through every page of this book.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

11.12 Availability and the rights boundary

Time is of the essence. Omni-Compass is available from The Omni-Compass LLC for controlled technical evaluation, research collaboration, pilot integration, strategic engagement and separately licensed commercial deployment. The same rights boundary applies at every level of use: inspection, local execution, simulation, research, benchmarking, observe-only, shadow mode, supervised control, bounded autopilot, hybrid control, direct-to-muscle control, production, embedding, hosted service, productization, monetization, redistribution and derivative work.

Public access does not make Omni-Compass open source. Commercial use of any kind requires a separately executed written license. The instrument is a license, not a sale and not a transfer of ownership. For planning only, the introductory benchmark is 10% of independently verified and contractually accepted value captured; the base and the amount are specific to each company, the terms are expected to rise as validation and adoption grow, and only a signed agreement creates any obligation.

CHAPTER 12

The Engine: Eight Equations and One Control Law

The engine (`omnicompass/core.py`, frozen and fingerprinted) carries a six-part state x = (E, U, I[U], S, B, Ḃ):

State	Meaning	Physical picture
E	deviation: how far the system is from where it should be	a tank that fills with stress and drains on its own
U	alignment, between the two poles -1 and +1	a ball in a double well: two stable poles, a hill between them
I[U]	memory of misalignment	an integrator that remembers and slowly forgets
S	basin structure	a ball rolling to the bottom of its landscape
B, Ḃ	the bath	a spring with friction that absorbs and settles energy

The eight equations (Appendix C gives them in full):

1. dE/dt = −α[E] E + β[int] + β[ext] + v[eff]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2. $dU/dt = \mu\, U\,(1 - U^2) - (dE/dt)/E[max] - \lambda[U]\, U + u$, with $|u| \le 25$
3. $dI[U]/dt = (1 - U) - \sigma_1\, E - \delta\, S - \lambda[I]\, I[U]$
4. $v[eff] = \cos(\omega[B]\, t / 2)\, c \tanh(\lambda_0 + \lambda_1\,(U - 0.5) + \lambda_2\, S)$
5. $\Phi(S) = \alpha_s\, S^2/2 + \beta_s\, S^3/4 - \delta\, S$
6. $dS/dt = -d\Phi/dS$
7. $d^2B/dt^2 = \gamma[c]\, \delta\, S - (\omega[B]/Q[B])\, dB/dt - \omega[B]^2\, B$
8. R[B]: a finite-difference audit of (7), never fed back

The control law. $u = \text{clip}(-fU + K[P]\,(\sigma - U), -25, +25)$, where f[U] is the drift of U with the command at zero, σ is the target pole and K[P] = 12. The first term cancels whatever is shoving U (the push); the second pulls U to its pole, harder the farther it is (the pull). The command is held unchanged through every stage of the fourth-order Runge-Kutta step and U is never rewritten afterwards. Inside the authority limit, U converges to its pole at rate K[P]; this is proved (`docs/TRACKING_THEOREM.md`).

12.1 What each equation does, in words

The six numbers describe the whole system the way a physician's six vital signs describe a patient, and each equation says how one of them moves.

- **Equation 1, the deviation tank.** *E* is how hard the system is being driven: queues, overload, power, heat. It drains on its own at a fixed rate (α[E] = 3.487) and is filled by two pressures, internal (queue and overload, β[int]) and external (power, heat, network, staleness, security, β[ext]), plus a drive term v[eff] that is bounded by a hyperbolic tangent. However extreme the observation, the engine's response rate is capped at c = 1. This is the saturation, the "speed limit" that no input can exceed.
- **Equation 2, the alignment.** *U* is how healthy and settled the system is, and it lives in a double well: two stable poles at +1 (healthy) and −1 (failed) with an unstable ridge at 0. A rising deviation pushes *U* out of the healthy well; the bounded command *u* pulls it back. The cubic term $\mu\, U\,(1 - U^2)$ is what makes the walls of the well push back harder the farther *U* strays.
- **Equation 3, the memory.** *I[U]* grows while *U* is below its pole, is reduced by deviation and stress already being dealt with, and leaks at λ[I] = 0.719. It is the engine's memory of unrelieved pressure: a system that has been stressed for an hour is not the same as one stressed for a second, and this integrator is what knows the difference.
- **Equation 4, the drive.** The bounded drive v[eff] mixes the alignment and the stress through a tanh, with a slow cosine of the bath's frequency on top, so the drive breathes rather than pushes constantly.
- **Equations 5 and 6, the stress.** *S* rolls downhill in a cubic potential to its stable root. It is the slowest restoring force: structural stress from heat, power, network, security and staleness that builds and relaxes over minutes rather than seconds.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

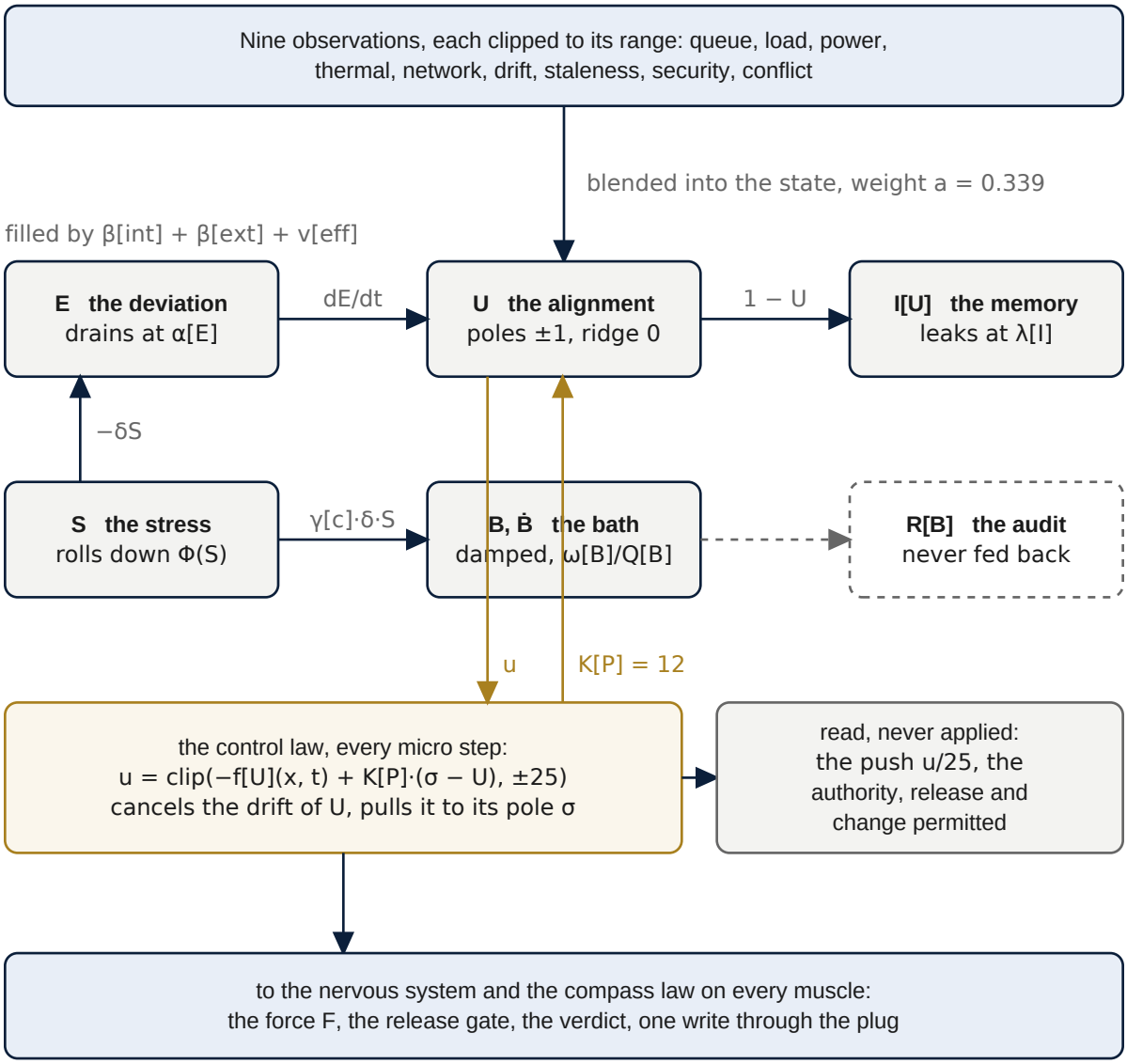


Figure 5. The engine's six states and the control law. Nine clipped observations are blended into the state x = (E, U, I[U], S, B, Ḃ) with weight 0.339 and the state moves one macro step. The deviation E drains at α[E] and is filled by the two pressures and the bounded drive v[eff]; its rate of change pushes the alignment U, a double well with poles at ±1; the need 1 − U accumulates in the memory I[U], which leaks at λ[I]; the stress S rolls down its cubic potential Φ(S), drives the bath B through γ[c]·δ·S and enters E and I[U] through −δS; the audit R[B] re-checks the bath and is never fed back.

The control law u cancels the drift of U and pulls it to its pole at rate K[P] = 12; it is read, never applied to a knob: the push u/25, the authority and the release permission go to the nervous system and the compass law.

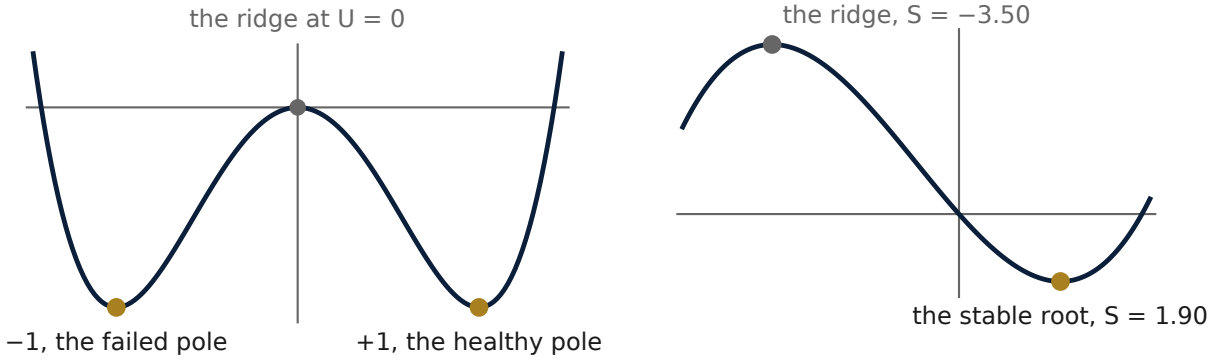
- **Equation 7, the bath.** A damped harmonic oscillator (natural frequency ω[B] = 1, quality Q[B] = 2.5) driven by stress. It turns stress into a smooth, lagged tide, and it is the only second-order element in the engine; its friction is what settles the whole.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Equation 8, the audit.** The bath equation is re-checked numerically every step with a finite difference, and the check is recorded and never fed back: an instrument on the engine, not a part of it.

(a) the alignment U, a double well:
$V(U) = \mu(U^4/4 - U^2/2)$

(b) the stress S, a cubic potential:
$\Phi(S) = \alpha_s S^2/2 + \beta_s S^3/4 - \delta S$
(α_s 0.12, β_s 0.10, δ 0.5: the engine's defaults)



$dU/dt = \mu U(1 - U^2) + \ldots$
the ball rolls to the nearer pole

$dS/dt = -d\Phi/dS$: S rolls to the stable root;
the landscape is open below the ridge

Figure 6. The two landscapes inside the engine. (a) The alignment U lives in a double well, $V(U) = \mu(U^4/4 - U^2/2)$, with stable poles at +1 (healthy) and −1 (failed) and an unstable ridge at 0; the drift $\mu U(1 - U^2)$ rolls the ball to the nearer pole. (b) The stress S rolls down the cubic potential $\Phi(S) = \alpha_s S^2/2 + \beta_s S^3/4 - \delta S$; with the engine's default parameters the stable root is at S = 1.90 and the ridge at S = −3.50, below which the landscape is open (its closure is given in the chapter Closing the Circle in the Engine).

Integration is classical fourth-order Runge-Kutta, ten micro-steps of 0.01 per macro step, and the C++ twin matches the Python to 3.6×10^{-15} on 500 reference trajectories (Chapter 86).

12.2 From telemetry to state, and from state to action

Each decision, the telemetry is normalised to nine observations: queue, load, power, thermal, network, drift, staleness, security and conflict. An observed state is computed from them by fixed weights (`omnicompass/adapter.py`, `observe_vector`), the engine's state is blended toward it with weight 0.339, and the state is then evolved by one macro step of the equations. From the state, the governor evaluates the control's push, u(x)/25, which is a number between −1 and +1 that says how hard the engine is pulling toward health; a target utilisation that falls as the memory and the deviation rise; whether a release (giving something back) is permitted at all, which is only while the push reports convergence and the band and dwell allow; and whether change is permitted, which requires *U* above its gate and no security block. All of this is written to the audit every decision (`docs/MECHANISM_OF_ACTION.md`, sections 3 to 6, and `python`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`tools/mechanism.py`, which reproduces the measured numbers in `results/MECHANISM_OF_ACTION.json`).

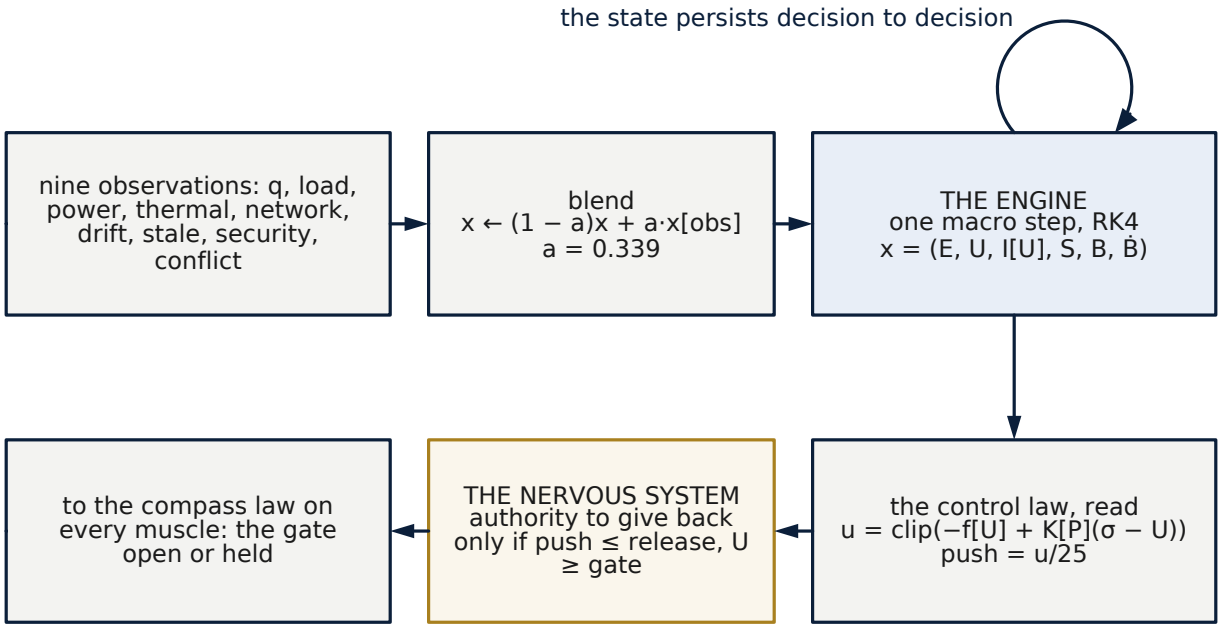


Figure 7. Block diagram: from the observations through the engine to the authority. The nine clipped observations are blended into the state x = (E, U, I[U], S, B, Ḃ) with weight a = 0.339; the engine moves one macro step (RK4) and its state persists from decision to decision; the control law u = clip(−f[U] + KP) is read, never applied to a knob, and its push u/25 goes to the nervous system, which grants the authority to give back only while push ≤ release and U ≥ gate. What leaves the chain is the gate's state, open or held, to the compass law on every muscle.

The nine observations are worth naming in plain words, because every later number is a weighted sum of them (`omnicompass/adapter.py`, `observe_vector` and `assimilate`). **Queue** is the work waiting as a share of the work being served; **load** is utilisation against the stack's own target; **power** and **thermal** are the electrical and heat stress against their limits; **network** is the same for the wire; **drift** is how far the levers stand from where native had them, so that a governor that has pushed far from the operator's settings reads as more stressed than one that has not; **staleness** is the age of the readings, which rises when a sense goes quiet; **security** is the hold flag; and **conflict** counts other hands seen on the levers. Each is clipped to a declared range before it is used, so no single runaway reading can swamp the others. The stress the engine assimilates leans most on the queue (a weight of 0.25) and on load above 85% of target (0.18), then power, heat, network and drift in that order, with conflict and staleness last; the utilisation it blends toward falls with the queue first (0.27) and with power and heat next. The weights are fixed numbers in the frozen engine, the same on every stack, and a reader who wants to know why a decision read the body as stressed can recompute the sum from the audit line, which records all nine observations. What the weights encode is a judgement made once: that

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

work waiting is the first sign of a body in trouble, that a lever far from native is itself a mild stress, and that a reading one cannot trust is a reason for caution rather than for action.

12.3 Where the engine sits, and where the compass law sits

A reader meeting the engine and then the compass law (Chapter 30) may ask which of the two moves the knob. The answer, in the live controllers, is this: the **compass law** sets each knob, the **verdict** narrows where the law may go, and the **engine** grants the authority, feeds the release gate and the nervous system, and is audited every decision (`omni_controller/controller.py`, its docstring). The engine is the organism's physiology: how stressed it is, how long it has been, whether it is settling. The compass law is the hand on each lever. The engine's own control law u is evaluated, not applied to a knob directly: it is the push the nervous system reads to decide whether giving anything back is permitted now. This separation is why the engine could be frozen with its proofs untouched while the compass law was written as its own, separately tested law, and why `verify.py` seals both.

One decision, followed through, shows the order. The controller wakes on its period and reads its senses: the service gauge (a response time, a statement latency, a consumer lag), the resource gauge (machines, connections, memory) and the knob's current value read back from the device. The **engine** takes the normalised observations, blends its state toward them and evolves one macro step; from the state it reports whether the body is converging, what authority the organ has earned and whether a release is permitted at all (section 12.2). The **compass law** then reads the service gauge as a position in its band and computes the force (Chapter 30): positive, and the direction rule asks for capacity in proportion; negative, and it asks for one notch back, but only if the engine's release gate is open, the resource is demonstrably idle and the dwell has passed. The **verdict** stands over the slow knobs: a knob the paired trial has not cleared at the 2% line is not moved at all and stays native. What survives all three is one write to the **plug**, read back from the device, and one line in the **audit** that records the senses read, the engine's state and push, the force, the gate's reasons and the value sent and taken. In the kill test's first repetition (section 32.1) this produced 114 such lines and 42 writes: every decision that wrote nothing is in the record with the reason the gate held, which is the point of writing all of it down.

12.4 The tune: one tuned engine, the tuning bench, and the rule that chooses the factory tune

Everything above is one engine. What it delivers on a given organism depends on how it is tuned, as a car's engine delivers differently as its mixture, intake, exhaust, firing and pistons are set. The benchmarks read one tune, the factory tune; the live benchmark engine page is a bench on which any tune can be turned and watched live. This section states what a tune is and how the factory tune is chosen, so that the claim the benchmarks make is exact (`docs/THE_TUNE.md` carries the same with the first search's preregistration).

The tune is a point $\theta = (\theta_1, \ldots, \theta_k)$ of the knob space $\Theta = \Theta_1 \times \cdots \times \Theta_k$: for the Kubernetes governor, among others, the beat $P \in [5, 120]$ s, the compass's balance point $c \in (0, 1)$, the dial, the verdict's allowance and

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

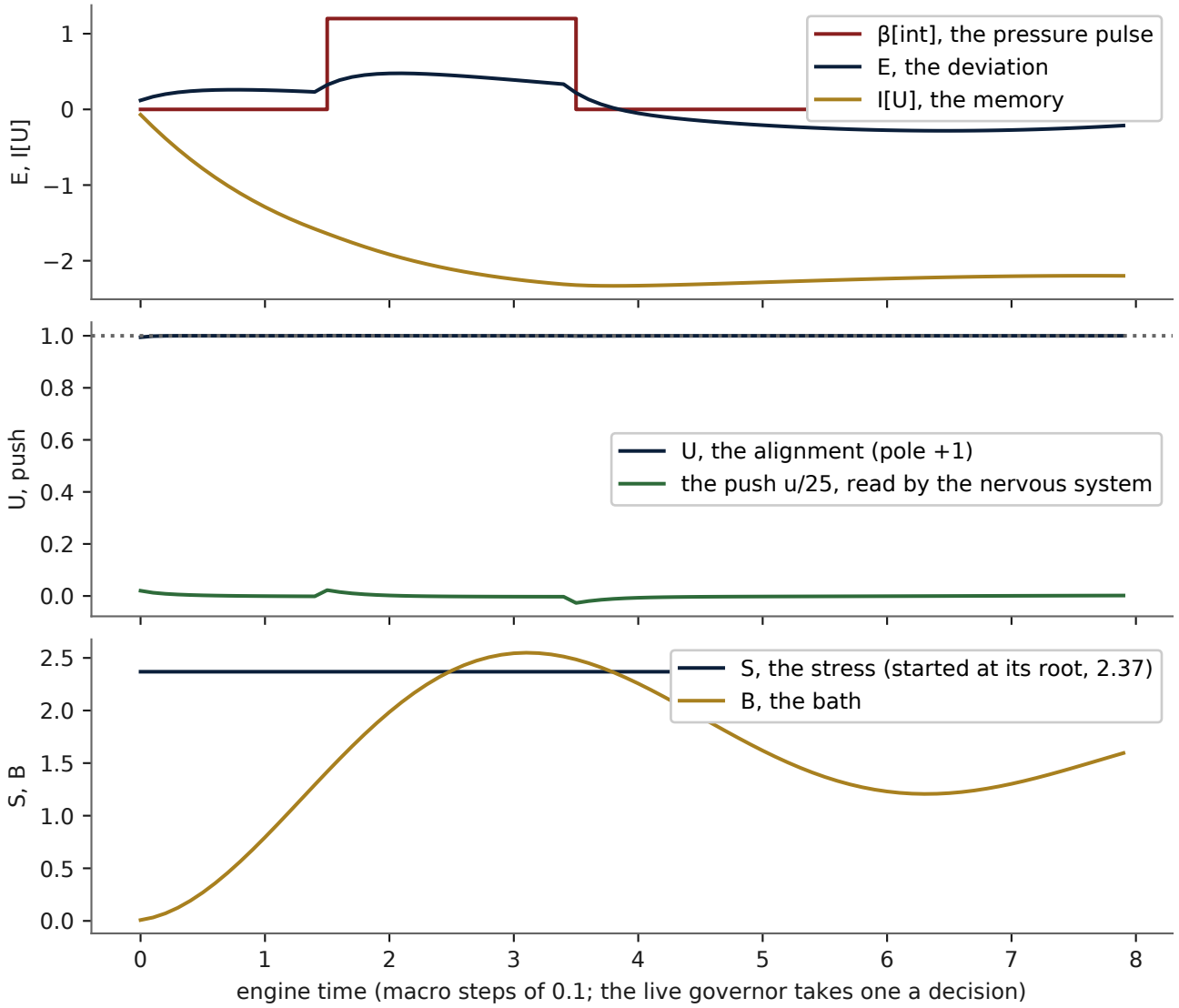


Figure 8. Modelled trace: the engine's six states answering a pressure pulse (class S: omnicompass/core.py with the control law on, the stack's parameters, started at the stress root). An internal pressure β[int] = 1.2 is applied from macro step 15 to 35: the deviation E rises and drains, the memory I[U] accumulates the need, the alignment U holds at its pole while the push u/25 is read by the nervous system, and the bath B fills from the stress S.

trial length, the cruise wait, each muscle's card. The factory tune is one point, θ*, fingerprinted with the engine. On an organism S under a load L, each seed s ∈ Ω (Ω the set of seeds) gives native's gauges $g^{o}(s) \in \mathbb{R}^{n}$ and Omni-Compass on top of native at tune θ gives g(θ, s). The one key reads each gauge j from the paired differences $d_j(\theta; s) = g_j(\theta, s) - g^{o}_j(s)$: their mean $\bar{d}_j$, their 95% interval $[\ell_j, u_j]$ by Student's t on

Ω	− 1 degrees of freedom, the word $R_j(\theta)$ (STEADY when $\ell_j \le 0 \le u_j$; else GAIN on the side the gauge's aim serves; else LOSS for a

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

time waited, missed or failed and for work, power given for a resource) and the size $r_j(\theta) = \dot{d}_j / \bar{g}^{o}_j$.

Admissible tunes harm no gauge against native: $A = \{ \theta \in \Theta : R_j(\theta) \neq \text{LOSS for every } j$, every muscle within its own limit $\}$, and in a cloud $R[\text{bill}](\theta) \neq$ power given as well. This is do no harm in the strict form the engine that stands keeps.

The order on A follows what a tune is for: the max serves the organism's output and gives power to buy it. With J_s the service gauges (time waited, missed or failed, work; in a cloud the bill too),

$$F(\theta) = \left(\left|\{ j \in J_s : R_j(\theta) = \text{GAIN} \}\right| , \Sigma \{ |r_j(\theta)| : j \in J_s, R_j(\theta) = \text{GAIN} \} \right),$$

compared lexicographically; energy and machines spent are shown beside as power given and never minimized for their own sake.

Dominance. $\theta' \succ \theta$ when, read against θ seed by seed, no gauge of J_s reads LOSS (in a cloud, no rise in the bill) and one at least reads GAIN. **The factory tune stands** on a searched set $\Theta_s \subseteq \Theta$ when $\theta^* \in A$ and no $\theta \in \Theta_s \cap A$ dominates it.

Three consequences bound the claim. It is non-dominance among the tunes searched, read in the models and benchmarks that read them, never a global optimum over the continuous Θ. It trades nothing between service gauges, so it never buys one with another (the trade judged on the whole organism is the next version's work, Chapter 76 of `DISCLOSURES.md`). And it is selected on development seeds and read again on untouched ones: a tune that dominates θ^* is a candidate, read in the model gate on seeds the search never saw against the engine that stands, landed as the next version only where it reads no LOSS there, then run three times on every benchmark. The rule is code, `tools/tune_study.py`, held by `tests/test_tune_study.py`; the first search (the beat from 15 to 120 s and the balance point, in the kind benchmark's setting and the cloud's, seeds 101 to 108) is read on its own page in `results/model_gate/` when it finishes.

The live tuning bench, as the page carries it (`docs/engine/OMNI_COMPASS_LIVE_ENGINE.html`; read monitor by monitor in the Live Benchmark Engine Manual). The page opens on the bottom line (one light and one number against native for each way to run it and for each route), then the monitor wall and the main control panel. Native is orange everywhere, Omni-Compass on native blue, Omni-Compass direct violet; red is never a side, only the alarm. Every monitor carries its light and its word: green and round, **GAINING** (low in its band with room to climb, or a GAIN); yellow, a yield diamond, **STEADY** (STEADY, inside 0.5%, or heading for its line and watched); red, a stop-sign octagon, blinking, **REDLINING** (over its line, or a LOSS: the alarm); dark, **AWAITING** (the run that fills it has not landed; never green). The main control panel holds the kill switch, the arm (Omni-Compass on native or direct), the heartbeat's seven buttons, $\Delta \in \{10, 15, 20, 30, 60, 90, 120\}$ s, a sweep that runs every beat on the page's own model for three model days each and reads each day against native by the key, **Try the candidate** (the sweep's best beat, the one with the most work for Omni-Compass on native and no LOSS, as a candidate profile), the active profile with its fingerprint (factory, candidate or custom) and its backend, **Restore factory profile**, **Hold the active profile** and **Return to native**. A model's best beat is a candidate for the real runs, never a result; no profile is locked as the baseline, and the factory's beat stays 60 s until the eight steps of section 76.8 choose the next. At the page's back stand the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

glossary and index (`tools/live_glossary.py`): every term in the words of the file it names, the ten organisms with their twenty scenarios, every one of the 1,107 muscles of the catalog by realm and family, and the sections where each is used.

CHAPTER 13

The Canonical Engine

One engine runs Omni-Compass, and every result in this repository comes from it. This page names it, gives its equations exactly as the code computes them, and names every other form as a variant. Where any document, manual, chart or filing states the equations differently, this page and the file it fingerprints are what the software runs.

13.1 The canonical engine: `symmetric_verified`

Name	`symmetric_verified` (`omnicompass/configurations.py`, `DEFAULT`)
Code	`omnicompass/core.py` (Python), `cpp/src/core.cpp` (C++ twin, sealed in `results/SEAL.json`)
Fingerprint	SHA-256 of `omnicompass/core.py`, recorded in `results/PREREGISTRATION.json` and `RELEASE_MANIFEST.json`; checked by `verify.py`
Evidence	every simulation, benchmark, live Kubernetes run and GPU harness in this repository

State: E (deviation), U (coherence), I[U] (unmet-need integral), S (structural stress), B and Ḃ (the bath).

```
v[eff]   = cos(ω[B] t / 2) · c · tanh(λ₀ + λ₁ (U − 0.5) + λ₂ S)
    ↪  (spinor closure, 720°)
dE/dt    = −α[E] E + β[int] + β[ext] + v[eff]
dU/dt    = μ U (1 − U²) − (dE/dt) / E[max] − λ[U] U + u
    ↪    (symmetric double well)
dI[U]/dt = (1 − U) − σ₁ E − δ S − λ[I] I[U]

dS/dt    = δ − αₛ S − (3/4) βₛ S²
    ↪    (gradient of Φ)
dB/dt    = Ḃ
dḂ/dt    = γ[c] δ S − (ω[B] / Q[B]) Ḃ − ω[B]² B
Φ(S)     = αₛ S² / 2 + βₛ S³ / 4 − δ S
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

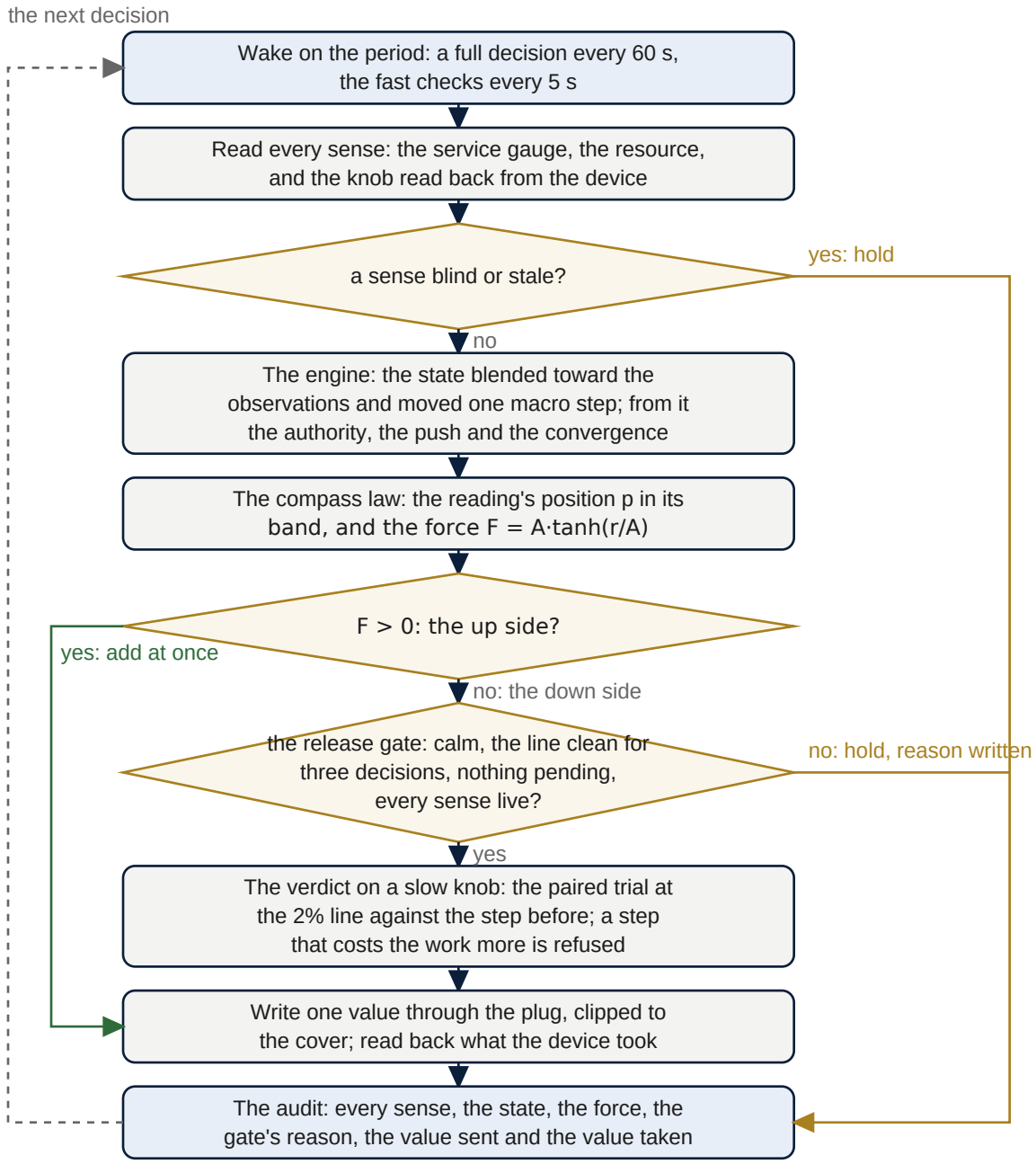


Figure 9. One decision of the governor. It wakes on its period, reads every sense, and holds if any sense is blind or stale: nothing is given back, and adding capacity is still allowed. The engine blends its state toward the observations and moves one macro step; the compass law turns the reading's position into a force. A positive force adds capacity at once, gated by nothing but the security hold.

A negative force must pass the release gate (calm, the line clean for three decisions, nothing pending, every sense live) and, on a slow knob, the verdict's paired trial at the 2% line. What survives is one write through the plug, read back, and one line in the audit that records every sense, the state, the force, the gate's reason and the value sent and taken, whether or not anything was written.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

u is the controller's command, `K[P] (target − U) − drift`, bounded by `U_AUTHORITY`; integration is RK4 with the macro and micro steps of `omnicompass/core.py`.

13.1.1 Mechanism identity

`results/MECHANISM_IDENTITY.json` (`tools/mechanism_identity.py`) fingerprints the mechanism M = (F, Theta, C, h, G, M[act], dt, A) component by component: state law, parameters, controller, observation map, authority law, actuator map, execution timing, shield. `verify.py` fails if any component changes without the record being rewritten. Every evidence claim names the mechanism id that produced it; "the eight-line engine" alone names none.

Configuration	Mechanism id	Role
`symmetric_verified`	`29d9808dfb8f626ad5de17a8a1efa37411dbab08a64af4b276143b466f7ce21c`	canonical
`printed_eight_line`	`cd333dc166fb7684ec0fca71f5f50488041e47825ee47d1ea667bec1f3d2259d`	named alternative embodiment

On the frozen 500-fixture population (`benchmarks/core_evidence.py`, seed 223387268), both are executable and neither is a stand-in for the other:

	finite	CONVEY-5	CERT-10	mean final target error	mean integrated abs(u)	max abs(u)
`symmetric_verified`	500/500	500/500	500/500	7.48e-5	0.878	13.93
`printed_eight_line`	500/500	500/500	500/500	5.13e-6	11.79	18.87

The printed form reaches its target more tightly with about 13 times the control effort. The structure is Option A: one canonical configuration, one named alternative; no equivalence is claimed.

13.2 Variants

13.2.1 `printed_eight_line` *(the printed chart)*

Implemented in `omnicompass/configurations.py`, checked against the printed plate (`tests/test_engine_configurations.py`), **not benchmarked**: no result in this repository comes from it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
v[eff]   = c · tanh(λ₀ + λ₁ (U − Uₜ) + λ₂ S)
    ↪    (no spinor factor)
dE/dt    = −α E + β[int] + β[ext] + v[eff]
dU/dt    = α (1 − U) − (dE/dt) / E[max] − k U (1 − U) + u
    ↪    (logistic)
dI[U]/dt = (1 − U) − σ₁ E − δ S

    ↪    (no λ[I] term)
dS/dt, dB/dt, dḂ/dt as in section 13.1
```

Differences from the canonical engine: logistic U instead of the symmetric double well; no spinor factor; a general target U_t; no λ[I] damping; α in place of α[E]. α[U] and k enter the printed form; they do not enter the canonical engine.

13.3 What would change this declaration

Promoting a variant to canonical needs, in one commit: the variant run through every harness beside the canonical engine, its results reported next to the canonical results, this page and `RELEASE_MANIFEST.json` updated, and the preregistration amended on record (`results/LOCK_AMENDMENTS.json`). Until then the canonical engine is `symmetric_verified`.

13.4 Open mathematical items (from `docs/FORMAL_STATUS.md` and the handoff)

- A global stability proof of the forced six-state system is not closed. What is held on the U channel is proved in `docs/TRACKING_THEOREM.md`: unsaturated exponential tracking; the saturated case under a drift bound that holds over the declared box (F[bar] = 16.86 < 25); the sampled-data bound of the executed RK4 controller; admissibility.
- Candidate routes: the Unified Circle Principle on a region; Theorem 5.6 of the Closed Structure, if its core maps onto (E, U, I[U], S, B).

13.5 For filings

Which form a patent or copyright filing claims as the principal embodiment is a decision for The Omni-Compass LLC and its counsel. Whatever that decision, the software described by this repository runs the canonical engine of section 13.1, and a filing that claims the printed form should name `symmetric_verified` as the embodiment that has been implemented and tested, or the printed form should be benchmarked first (section 13.3).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 14

The Closed Circle: Why It Cannot Leave Its Compass

The governing principle is the Unified Circle Principle:

```
dX/dt = G(X),  X(0) in Omega
G(X) . n(X) <= 0 on the boundary of Omega        (the flow points inwa
    ↪ rd at the wall: nothing crosses)
grad L(X) . G(X) <= 0                            (the compass's energy
    ↪  L only falls)
=> lim X(t) in M*                                (every path settles i
    ↪ n the bottom of the compass)
```

In control engineering the second line is the Nagumo condition for an invariant set and the third is a Lyapunov function. Together they mean: anything that starts inside the container stays inside it, for every disturbance up to a known size, and comes to rest at the bottom. That is a certificate, not a test: it answers every case inside the walls at once.

In plain words. Picture a round valley with a fence around its rim. The first line says the valley has a shape: given where the system is, the equations say which way it rolls. The second line says that at the fence the ground always slopes inward, so nothing that starts inside can ever roll out, however it is pushed within the strength the fence was built for. The third line says there is a height, *L*, that only ever decreases along any path, so the system cannot circle forever at one level; it must keep descending. The conclusion follows: every path ends at the bottom, the set *M**. The engineering value of such a certificate is that it is not a sample of behaviours; it is a statement about all of them at once, inside the fence.

What is certified, and what is not. The certificate is about the engine's own state: the six numbers of Chapter 12 stay inside their walls and settle. It is not a certificate about the plant the engine governs, whose physics the engine does not contain; that is why every plant is tested paired against native (Part Seven), why the compass law clips every write to the knob's cover (Chapter 30), and why the shield between the engine and the levers is tested separately on two million adversarial cases with zero violations (`tests/test_shield_properties.py`) and the C++ engine soaked on one hundred million decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

with no failure (`results/SOAK.json`). The mathematics closes the brain; the harness, the cover, the verdict and the paired receipts close the hand.

The engine's states are each closed in this sense: the deviation tank drains faster than it can fill; the alignment's cubic walls push back from far out; the memory forgets; the bath's friction settles it. The rule behind all four is the compass's east-west line: what flows out (drain, damping) must always be able to match what flows in (drive). The formal status of each statement, what is proved, what is tested and what is conjecture, is written out in `docs/FORMAL_STATUS.md` and `docs/TRACKING_THEOREM.md`, and the theory chapters of this book (The Unified Circle Principle, Closing the Circle in the Engine) give the derivations.

What the certificate buys an operator, in practice. Three things. The engine cannot run away: there is no input, no sequence of readings and no fault in a sense that drives its internal state to infinity or into oscillation that grows, because the walls are part of the equations and not a check bolted on afterwards. The engine cannot hold a grudge: the memory state forgets at a fixed rate, so a bad hour does not bias the next day, and a governor restarted after a kill starts from the same rest state the certificate names. And the engine's behaviour under any disturbance smaller than the fence is the same kind of behaviour, a descent to the bottom, so a test on one plant's readings says something about the engine on another's; what it does not say, and what no mathematics of the engine alone could say, is how the plant answers, which is why the paired runs exist. An underwriter reading this section should take from it exactly this much: the part of the system that makes decisions is bounded by proof; the part of the system that carries them out is bounded by the cover, the gates and the hand-back, each tested; and the part that is the customer's own plant is measured, never assumed.

The compass. The Omni-Compass rose carries the whole Greek alphabet, Alpha to Omega: the complete set, in a ring whose end runs back into its beginning. Its north-south axis is polarity: Alpha and plus at the top, Omega and minus at the bottom, the two poles of the alignment's double well. Its east-west axis is flow: Beta and the push outward on one side, Gamma and the pull inward on the other, the drive and the damping. The spiral at the center is the attractor every path winds into. The ring closing on itself is the return: when Omni-Compass stops, every lever returns to where it began.

CHAPTER 15

Closing the Circle in the Engine

Each state of the engine was checked against the closed-circle conditions:

State	Closed?	Why
E, deviation	closed	it drains at rate α[E] while its inputs are bounded (the drive is capped by tanh)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

State	Closed?	Why
U, alignment	closed	the cubic term μ U (1 − U²) pushes back hard from far out
I[U], memory	closed	it forgets at rate λ[I]
B, the bath	closed	the friction ω[B] / Q[B] settles it
S, the basin	closed on the positive side; open on the negative side below an unstable ridge	Φ(S) carries a cubic term, so the landscape falls away below the ridge (at S = −3.5 with the default parameters)

Three openings were found and each has its closure:

1. **The basin's negative side.** A quartic term in Φ(S) would give the compass a wall on both sides. This changes the frozen engine and is a new version with its own proof.
2. **The outer loop.** The frozen live governor computes the push and pull u and uses it as a convergence signal; it does not send u to a lever. The compass closes this loop: reading, force, plug, lever, read-back. On the modelled card this is the difference between +0.1% and several percent of work per energy (the one-wire governor against the corrected two-wire compass, `results/sim/gpu_two_wire/`).
3. **The corner.** clip() is a hard stop; tanh is its smooth form. The compass uses tanh.

What closing the loop has shown since. The compass law of the second opening is the law every result in the manual was made on: it was frozen as Omni v1 on 5 October 2026 with its proofs untouched, grown to the 945-muscle catalog as v2, and given one gate as v3, the slack gate on speed knobs, after the v2 realms read a service tradeoff on the Physics realm and the tower. That gate is the closed circle applied to a muscle's physics: an axis that is busy more than half the time at full speed has no slack to spend, and a law that spent it anyway would be spending energy for nothing; with the gate, every organism reads superior within its guardrails and zero muscles read LOSS (`results/realms/REALMS.md`). On real software the same law, read through the plug and the read-back, is the law of the Kubernetes, database, broker and cache tables, with their gains and their losses. The first opening, the basin's negative side, remains open and documented: it would be a new version with its own proof, and no result depends on it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 16

The Tracking Theorem

This page proves the internal tracking result of the canonical engine (`symmetric_verified`, mechanism id in `results/MECHANISM_IDENTITY.json`) in four steps, from the continuous law to the code as executed. Each statement carries one evidence class (see `docs/EVIDENCE_LEDGER.md`): **T** proved here, **V** checked by computation over a finite, frozen set (`tools/tracking_bounds.py` → `results/TRACKING_BOUNDS.json`).

What this page establishes is **internal**: inside the declared model, the controller conveys U to its target σ. It does not establish that σ is the right target for any outside plant, that the telemetry map is right, that any actuator mapping (HPA, GPU, CPU) is right, any energy saving, production performance, or stability of the whole forced six-state system. Target correctness and target conveyance are separate obligations; this page carries only the second.

16.1 Setting

U channel of equation (2): $dU/dt = f[U](x, t) + u$, with the uncontrolled drift

$$f[U](x, t) = \mu\, U\,(1 - U^2) - (dE/dt)\,/\,E[max] - \lambda[U]\, U$$

and the bounded cancellation-plus-proportional controller (`omnicompass/core.py`, `control_command`)

$$u = sat(-f[U](x, t) + K[P]\,(\sigma - U)), \quad sat(v) = max(-u[max], min(u[max], v)), \quad K[P] = 12,\ u[max] = U_AUTHORITY = 25.$$

Tracking error $e = U - \sigma$, $\sigma \in \{-1, +1\}$.

16.2 Theorem 1 (T). Unsaturated tracking

On the unsaturated region $\Omega[unsat] = \{\, x : |-f[U](x, t) - K[P]\, e| \le u[max] \,\}$, $de/dt = -K[P]\, e$. Along any interval spent in $\Omega[unsat]$, $e(t) = e(t_0)\, \exp(-K[P]\,(t - t_0))$, and $V = e^2/2$ satisfies $dV/dt = -K[P]\, e^2 < 0$ for $e \neq 0$.

Proof. Inside $\Omega[unsat]$ the saturation is inactive, so $dU/dt = f[U] - f[U] + K[P]\,(\sigma - U) = -K[P]\, e$; since σ is constant, $de/dt = dU/dt$. The solution and $dV/dt = e\, de/dt = -K[P]\, e^2$ follow. ∎

This is a statement about intervals inside $\Omega[unsat]$ only; it is not global convergence.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

16.3 Theorem 2 (T). The saturated case, under a declared drift bound

Assumption D. Along the trajectory, `|f[U](x(t), t)| ≤ F[D] < u[max]`.

Under D:

1. If saturation is active with `e < 0`, then `u = +u[max]` and `de/dt = f[U] + u[max] ≥ u[max] − F[D] > 0`; symmetrically for `e > 0`. So `dV/dt ≤ −(u[max] − F[D]) |e| < 0` wherever saturation is active.
2. With Theorem 1, `V` is strictly decreasing wherever `e ≠ 0`, saturated or not; `e` never changes sign (at `e = 0` the command is unsaturated because `F[D] < u[max]`, so `e = 0` is an equilibrium of the error dynamics).
3. Any saturated stretch ends within `(|e(t₀)| − e*) / (u[max] − F[D])` time units, `e* = (u[max] − F[D]) / K[P]`; the ball `|e| ≤ e*` lies inside `Ω[unsat]` and is forward invariant, and inside it the decay is exponential at rate K[P].

Proof of 1. Saturation with `u = +u[max]` means `−f[U] − K[P] e > u[max]`, so `−K[P] e > u[max] + f[U] ≥ u[max] − F[D] > 0`, so `e < 0`, and `de/dt = f[U] + u[max] ≥ u[max] − F[D]`. The other side is symmetric. 2 and 3 follow from 1 and Theorem 1, and `|−f[U] − K[P] e| ≤ F[D] + K[P] |e| ≤ u[max]` on the ball. ∎

Where D holds (T). On the declared box — parameters in `PARAMETER_RANGES`, `E₀` in [0, 1], `|U| ≤ 1.1` — equation (1) keeps E in `[(β − c)/α[E], (β + c)/α[E]]` widened to hold E_0 (the right side of (1) points inward at both ends, because `|v[eff]| ≤ c`), so `|E| ≤ 7/4.2`, `|dE/dt| ≤ 4.2·|E| + 2 + 5 ≤ 14`, and

```
F[D] = 6 · 2/(3√3) + 14 / 1 + 0.5 · 1.1 = 16.86 < 25 = u[max].
```

Because e keeps its sign and `|e|` does not grow (continuous time), U stays between U_0 and σ, so `|U| ≤ 1` and the box is self-consistent for the nearest-basin target. **Theorem 2 therefore holds globally over the declared box, in continuous time.** The actuator can still saturate: `|raw| ≤ F[D] + K[P] |e₀|` can exceed 25 when `|e₀|` is near 1.

Checked (V). Over the 500 frozen fixtures: observed `max |f[U]|` = 6.84 (nearest target), 7.01 (wrong target), each under that fixture's own bound (max 8.99) and under 16.86; saturated micro steps 0 of 100 000 (nearest target), 5 of 100 000 (wrong target).

16.4 Theorem 3 (T with a computed constant). The executed sampled-data controller

The code does not run the continuous law. Per micro step `h = MICRO_DT = 0.01`, it computes u_k from x_k, holds it through the four stages of one RK4 step (zero-order hold), and takes $x_{k+1} = F[h](x_k, u_k)$. There is no overwrite of U after the step, no projection onto a basin, no state replacement.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Integrating the U equation over one held interval,

```
e_{k+1} = e_k + h (f[U](x_k) + u_k) + ∫[t_k, t_k+h] (f[U](x(t)) − f[U](x_k)) dt
    ↪  + τ_k
```

with τ_k the RK4 truncation error. Unsaturated, $f[U](x_k) + u_k = -K[P]\ e_k$, so

```
e_{k+1} = ρ e_k + d_k,   ρ = 1 − h K[P] = 0.88,   |d_k| ≤ (h²/2) sup|df[U]/d
    ↪ t| + |τ_k| =: ε[h].
```

Then by induction $|e_k| \le \rho^k\ |e_0| + (1-\rho^k)/(1-\rho) \cdot \varepsilon[h]$, and the error enters and stays in the neighbourhood $|e| \le \varepsilon[h] / (1-\rho)$.

The proof above is exact given ε[h]. **ε[h] itself is computed, not proved (V):** the largest $|d_k|$ over every micro step of the 500 frozen fixtures is 0.00378 (nearest target), giving the ultimate bound 0.0315, inside $BASIN_TOL$ = 0.10 that CONVEY-5 and CERT-10 test. With the target aimed wrong, ε[h] = 0.0243 (dominated by the long transient, which includes the 5 saturated steps) and the bound 0.20 is wider than the basin — a conservative bound; the fixtures still certify 500 of 500.

Proved since 19 October (T, computer-assisted; `docs/ENGINE_PROOFS.md`, claims U2 to U5). Over the whole declared box, not the fixtures: every unsaturated micro step satisfies $|e_{k+1}| \le (\rho + b)|e_k| + a$ with $a \le 0.006495$ and $b \le 0.02399$. These are interval enclosures with outward rounding of the stage-to-stage drift differences, bounded by the mean value theorem. So the ball $|e| \le a / (1 - \rho - b) \le 0.0677$ is entered and never left, inside $BASIN_TOL$. Every saturated step moves U toward σ by at least 0.0868 and never past it. From any admissible start the error is in the basin within 119 of the run's 200 micro steps. The fixtures' 0.0315 lies inside the proved 0.0677.

What the discrete execution does **not** inherit: the continuous monotone decay. Within the band, $|e|$ grew on 27 061 of 100 000 micro steps and e changed sign 747 times — the sampled controller dithers inside $\varepsilon[h] / (1-\rho)$.

CONVEY and CERT. They are dwell checks against a declared neighbourhood ($|U - \sigma| \le 0.10$ for 5 and 10 consecutive macro steps). Theorem 3 says why they pass when the bound sits inside the neighbourhood. They are not an external certification.

16.5 Theorem 4 (T continuous; V discrete). Admissibility

Admissible set for a fixture with parameters p and initial state x0:

```
A = { x : E ∈ [E[lo], E[hi]],  |U − σ| <= |U_0 − σ|,  S between S_0 and
    ↪ S_+(p) }
```

E_lo, E_hi as in Theorem 2; S_+ the stable root of equation (6).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Continuous (T).** A is forward invariant: E by the inward-pointing field at both ends; U by Theorem 2; S because equation (6) is the one-dimensional gradient flow of Φ, monotone toward `S+` from any `S0 > S-` (`s_admissibility` in `results/core_evidence.json`: 500 of 500 fixtures start above `S_minus`).
- **Discrete (V).** For the executed map `F[h](x, C(x))`, every micro step of every fixture: `xk ∈ A` implied `xk+1 ∈ A` — 0 failures in 100 000 steps, target nearest or wrong. This is a finite verification on the frozen fixtures.
- **Discrete (T, since 19 October).** Over the whole box, for the executed RK4 step (`docs/ENGINE_PROOFS.md`):
  - E stays in its interval (E1, exact linear arithmetic in z3), and so does every stage (E2);
  - S moves toward S_+ and never past it (S1);
  - U stays within `max(|U0 − σ|, 0.0677)` of σ (U3, U4);
  - I[U] stays in its interval for any bounded drive (I1).

16.6 What remains open (O)

- Global stability of the forced six-state system. E, U, S and I[U] are proved bounded under the executed step (`docs/ENGINE_PROOFS.md`). Not proved: the bath B, which nothing reads back, and convergence of E, I[U] and B to a set.
- CERT-10 for every fixture in the box: the proved entry bound (119 micro steps) leaves 9 macro steps inside the basin, not 10. Every one of the 500 fixtures certifies (V).
- The inheritance / intertwining residual against an outside plant: instrumented (`tools/gpu_reps.py`, representation fidelity), not yet measured on a real card.
- The printed configuration (`printed_eight_line`) is not covered by this page: its U drift is logistic, not the double well, and needs its own bound (`docs/CANONICAL_ENGINE.md`).

CHAPTER 17

The Engine's Machine-Checked Proofs

This page states what is proved about the engine that stands, how each proof is checked by a machine, and what is still open. Every proof is about the algorithm the engine executes, over every input in its declared set, never over samples. No engine file was changed to make them: the proofs read the engine's own constants. `python3 tools/prove_engine.py` checks all of them in under a second and writes `results/ENGINE_PROOFS.json`. `tests/test_engine_proofs.py` runs it on every push, and fails a push where any claim is not proved, where a claim's premises admit no case (a vacuous proof), or where the stored record differs from the one the proofs produce.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`tests/test_engine_properties.py` then runs the engine's own code on several thousand generated cases and checks every proved property on each case, with the proved constants. A failure there would mean the proof and the code disagree.

Evidence class of everything on this page: **T**, proved. Two machine methods carry the proofs:

- **SMT (the z3 solver).** A claim P is proved when the solver shows that "the premises and not P" has no solution (unsat). Before that, the solver confirms that the premises alone have a solution, so no proof is vacuous. tanh enters only through true facts about it:
  - $-1 < \tanh x < 1$;
  - $\tanh x$ has the sign of x;
  - $\tanh$ is increasing;
  - $\tanh$ is 1-Lipschitz: $|\tanh x - \tanh y| \le |x - y|$.

So whatever is proved for every function with these facts holds for tanh.

- **Interval arithmetic with outward rounding** (mpmath's interval arithmetic at 40 digits). Every enclosure computed is a true bound of the quantity over the whole box, rounding included. The U axis is cut into 96 cells for a tight enclosure. tanh of an interval is the hull of tanh at its two ends, which is exact because tanh is increasing.

17.1 The compass law (`omnicompass/compass_law.py`, `CompassLaw.force`)

For every reading position p, velocity estimate v, gains $k_p > 0$ and $k_d \ge 0$, authority $A > 0$ and walls $0 < w[\text{low}] < \text{centre} < w[\text{high}] < 1$, write $\text{raw} = k_p (p - \text{centre}) + k_d v$. The force is:

- $f = A$ when $p \ge w[\text{high}]$;
- otherwise $f = A \tanh(\text{raw} / A)$, set to 0 when that value is negative while $v > 0$ and $p > \text{centre}$.

Claim	Statement	Result
C1	$-A \le f \le A$	proved
C2	$p \ge w[\text{high}] \Rightarrow f = A$ (fail up at once)	proved
C3	$f < 0 \Rightarrow$ not ($v > 0$ and $p >$ centre): it never takes back while the reading climbs above the centre	proved
C4	with $v = 0$: $f < 0$ below the centre, $f > 0$ between the centre and the wall, $f = 0$ at the centre	proved
C5	for one v: $p_1 \le p_2 \Rightarrow f(p_1) \le f(p_2)$ (no inversion anywhere, the wall and the take-back rule included)	proved
C6	at one p: $v_1 \le v_2 \Rightarrow f(v_1) \le f(v_2)$	proved

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Claim	Statement	Result
C7	inside the band, away from the wall and the take-back rule: abs(f(p_1) − f(p_2)) ≤ kp abs(p_1 − p_2) (smooth, never a hammer)	proved
C8	the default damping kd = max(0, 2 √(kp τ / Δt) − 1) Δt is never negative	proved

17.2 The plug's clip (`Plug.write`)

Claim	Statement	Result
P1	lo ≤ hi ⇒ lo ≤ clamp(x) ≤ hi: every value sent lies inside the cover	proved
P2	lo ≤ x ≤ hi ⇒ clamp(x) = x: nothing inside the cover is clipped	proved
P3	clamp(clamp(x)) = clamp(x)	proved

17.3 The verdict's rule (`omnicompass/verdict.py`, judge "interval")

A trial step is allowed when three things hold: it does not read LOSS (no 95% interval wholly above zero), its median cost mk ≤ min(m_0, m[native]) (1 + tol), and the largest estimated change is at most τ = ln(1 + tol).

Claim	Statement	Result
V1	allowed ⇒ every interval's lower end ≤ 0: an allowed step never reads LOSS against either reference	proved
V2	allowed ⇒ every estimated change ≤ τ and mk ≤ m_0 (1 + tol)	proved
V3	allowed ⇒ mk ≤ m[native] (1 + tol): stepwise, steps cannot add up past the cost first measured at native	proved

17.4 The core engine as executed (`omnicompass/core.py`)

The engine does not run the continuous law. Per micro step h = 0.01 it:

1. computes the command u[k] from x[k];
2. holds u[k] through the four stages of one RK4 step;
3. takes x[k+1] = F_h(x[k], u[k]).

Theorems E1 to B1 are about that map, over the declared box:

- every parameter in `PARAMETER_RANGES`;
- E_0 and I_0 in [0, 1];
- U_0, S_0, B_0 and B'_0 in [−1, 1], with S_0 above the unstable root S_- (theorem S1's premise; the declared box has starts below S_-, from which S leaves every bound, and they are outside every claim);

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- the target σ = ±1, nearest or aimed wrong.

Claim	Statement	Method	Result
E1	E[k] ∈ [min(0, (β − c)/α[E]), max(1, (β + c)/α[E])] ⇒ E[k+1] in the same interval, for every value v[eff] takes at the four stages (abs(v[eff]) ≤ c)	SMT, exact linear arithmetic	proved
E2	every RK4 stage of E lies in that interval widened by 0.13	SMT	proved
I1	for any drive F[i] ∈ [F[lo], F[hi]] at the four stages, I[k] ∈ [min(I_0, F[lo]/λ[I]), max(I_0, F[hi]/λ[I])] ⇒ I[k+1] in it too	SMT, nonlinear	proved
S1	S moves toward its stable root S_+ and never past it: with L = sup abs(dṠ/dS) on the S box, every RK4 stage stays on the same side of S_+ when h L < 1	intervals: L ≤ 1.953, h L ≤ 0.0196	proved
U1	the drift is below the actuator's limit on the whole stage box: sup abs(f[U]) ≤ F[D] = 16.32 < 25	intervals	proved
U2	every unsaturated micro step: abs(e[k+1]) ≤ (ρ + b) abs(e[k]) + a, ρ = 1 − h K[P] = 0.88, a ≤ 0.006495, b ≤ 0.02399	intervals and the mean value theorem	proved
U3	the ball abs(e) ≤ r* = a / (1 − ρ − b) ≤ 0.0677 is entered and never left. It lies inside the basin (0.10) and inside the unsaturated region. Beyond abs(e) = 0.0076 the error never changes sign, so U never overshoots σ by more than r*. Every RK4 stage of U lies in the stage box [−1.2, 1.2]	intervals	proved
U4	every saturated micro step moves U toward σ by at least h (25 − F[D]) ≥ 0.0868 and at most 0.4132. Saturation needs abs(e) > 0.7238, so the step never passes σ	intervals	proved
U5	from any admissible start (abs(e_0) ≤ 2, the target aimed wrong included), abs(e) ≤ 0.10 within 119 micro steps (95 unsaturated, 24 saturated) of the run's 200	intervals	proved
B1	the bath (B, B′), a linear damped oscillator driven by S, x′ = A x + (0, γ[c] δ S) with A = [[0, 1], [−ω[B]², −ω[B]/Q[B]]]: for every (ω[B], Q[B]) in the box the spectral radius of its RK4 step R(hA) is at most 0.99995, so it shrinks by at least 5.0 × 10^{-5} every micro	intervals and the remainder of e^x	proved

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Claim	Statement	Method	Result
	step, stays bounded under the bounded S, and forgets its start		

How U2 is proved. Unsaturated, the command cancels the drift at x[k] exactly, so $k_1 = -K[P]\, e[k]$ and $e[k+1] = \rho\, e[k] + (h/6)(2\, D_2 + 2\, D_3 + D_4)$, where D[i] is f[U] at stage i less f[U] at x[k]. By the mean value theorem on the segment to each stage:

```
abs(D[i]) ≤ g[U] abs(ΔU[i]) + g[E] abs(ΔE[i]) + g[S] abs(ΔS[i]) + g[t]
    ↪  abs(Δt[i])
```

with the partial-derivative bounds g taken over the stage box. The stage displacements are bounded by the step times the stage slopes (K[E] for E, K[S] for S, K[P] abs(e) + abs(D) for U), which gives abs(D[i]) ≤ A[i] + B[i] abs(e) by recursion over the four stages. The enclosures are:

- g[U] ≤ 35.42, g[E] = 4.2, g[S] ≤ 10.0, g[t] ≤ 12.5;
- K[E] ≤ 12.55, K[S] ≤ 4.99.

A floating-point allowance of 10^{-12} is added to a for the binary64 arithmetic the engine runs in.

How B1 is proved. For Q[B] ≥ 1/2 the eigenvalues λ of A have real part $-\omega[B]/(2Q[B])$ and modulus ω[B] (a complex pair, or the double root at Q[B] = 1/2). The RK4 step multiplies each eigen-direction by R(hλ), $R(z) = 1 + z + z^2/2 + z^3/6 + z^4/24$, and the remainder of the series of e^z gives

```
abs(R(z)) ≤ e^(Re z) + abs(z)⁵/120 · e^abs(z)
```

The bound rises with Q[B], so it is evaluated at the largest Q[B] over 400 cells of ω[B] with outward rounding; its largest value, 0.99995, is the spectral-radius bound. A spectral radius below 1 makes the forced linear map bounded for every bounded input, and S is proved bounded (S1).

What this closes. `docs/TRACKING_THEOREM.md` carried three open items:

- a proved ε[h] over the declared box;
- discrete invariance over the box;
- an interval-arithmetic proof of the admissible set.

The first two are now proved for E, U, S and I[U], and the bath is proved stable (B1). Theorem 3's ultimate bound is proved: 0.0677 against the 0.0315 the 500 fixtures reached, which lies inside it. Theorem 4's discrete half moves from V (checked on fixtures) to T for those four states. U5 also proves that CONVEY-5 passes for every admissible fixture: at least 9 of the 20 macro steps end inside the basin.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

17.5 What is still open

- **CERT-10 for every fixture.** U5's bound guarantees 9 macro steps inside, not 10. Every one of the 500 frozen fixtures certifies (V), but a proof for the whole box needs a tighter entry bound: a, b and q are worst-case constants over the whole box.
- **The bath's amplitude as a number.** B1 proves the bath is stable and bounded for every parameter in the box; an explicit bound on abs(B) needs a Lyapunov function for each (ω[B], Q[B]), and is not computed here. The bath is never fed back into any other state (no equation of E, U, I[U] or S reads it).
- **Convergence of the forced six-state system as a whole.** Bounded is proved for E, U, S and I[U]; convergence of E, I[U] and the bath to a set is not.
- **Starts below S−.** From there S leaves every bound (the basin's negative side, `docs/book/THEORY.md`). They are outside every claim on this page, and the quartic closure named in the theory is the engine change that would remove them.
- **The live governors** (`omni_controller/*`) are proved here only through the parts they share: the compass law, the plug's clip and the verdict. Their Kubernetes, GPU and cloud wiring is tested, not proved.

CHAPTER 18

The Conveyance Law

18.1 What the manuscript says, read as mechanism

Source	Text	Mechanism
Ch. 29 §8	"E[system] = E + U + S + B … dE_system/dt = 0 (over full cycle). Energy redistributes. Energy reorganizes. Energy does not diverge."	The budget (site watts, cluster cores) is **conserved**: it is moved between organs, never created.
Ch. 31 §4	"Redistribution B acts as mediator between the baths. When E grows locally, B increases to distribute gradients."	Budget flows **down the gradient of need**: from organs holding surplus to organs in deficit.
Ch. 31 §5	"eigenvalues of the coupled system must remain bounded … $Re(\lambda_i) \leq 0$"	The flow must have a proof of convergence, not a tuned damper.
Ch. 30 §5-7	"Redistribution surge … channels stored deviation into expansion … controlled release … No discontinuity."	A **reserve** is held and released **continuously** to organs at their limit. There is no on/off.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Source	Text	Mechanism
Ch. 29 §6	"Backpressure develops through S … Compression is … structured inward rotation."	When need falls, surplus is **pulled back** smoothly. Budget nobody needs is not spent.
App. J	"the engine can only act through what you expose"	Organs keep their own mechanisms. The law sets only each organ's share of the budget.

18.2 The law

Notation.

- Organs i = 1..n share a budget A.
- Organ i holds allocation a_i and has need d_i: the budget that serves its work at the engine's utilisation target ρ.
- Its deficit (local deviation) is

$e_i = d_i / (\rho\, a_i) - 1$ ($e_i > 0$: needs juice; $e_i < 0$: holds surplus)

The redistribution mediator:

$da_i/dt = \kappa\, a_i\, (e_i - \bar{e})$, $\bar{e} = \Sigma_j\, a_j\, e_j / A$

18.3 Properties and proofs

P1. Conservation.

- $\Sigma_i\, da_i/dt = \kappa\, (\Sigma_i\, a_i e_i - \bar{e}\, \Sigma_i\, a_i) = \kappa\, (A\, \bar{e} - \bar{e}\, A) = 0$.
- The budget is moved, never created, as in Ch. 29 §8.

P2. Gradient flow with a Lyapunov function.

- Substitute e_i: $a_i(e_i - \bar{e}) = d_i/\rho - a_i\, D/(\rho A)$, where $D = \Sigma\, d_i$.
- So $da_i/dt = (\kappa/\rho)(d_i - a_i\, D/A)$.
- This is the Shahshahani gradient on the simplex $\Sigma\, a_i = A$ of $F(a) = \Sigma_i\, (d_i/\rho)\, \ln a_i$. The Shahshahani metric weights by $1/a_i$, and $\partial F/\partial a_i = d_i/(\rho\, a_i)$, so $a_i(\partial F/\partial a_i - \text{mean})$ is exactly the flow above.
- Along the flow $dF/dt = \Sigma_i\, (1/a_i)(da_i/dt)^2 \cdot (\rho/\kappa) \geq 0$.
- F is strictly concave, so F is a Lyapunov function and the flow cannot cycle.

P3. Global exponential convergence.

- The ODE in P2 is linear: $a_i(t) = a_i^* + (a_i(0) - a_i^*)\, e^{(-\kappa D t/(\rho A))}$, with $a_i^* = A\, d_i/D$.
- The Jacobian is $-(\kappa D/(\rho A))\, I$ on the simplex, so every eigenvalue is $-\kappa D/(\rho A) < 0$. This meets $\mathrm{Re}(\lambda) \leq 0$ of Ch. 31 §5 strictly.
- At the equilibrium every organ has the same deficit $e^* = D/(\rho A) - 1$. None is starved while another idles.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- `replicator_step` integrates it exactly, in closed form, for any step size.

P4. Bounds (floors, ceilings, the budget).

- Allocations are projected onto $\{lo_i \le a_i \le hi_i, \Sigma\, a_i \le A\}$.
- The projection is water-filling: organs at a bound keep it, and the rest share the remainder in proportion.
- Budget an organ cannot use (it is at its need or its ceiling) is not spent.

P5. Reserve and surge.

- The dual-bath rate channel (the rate tracker) projects the rise in need over one actuation delay: $R = \Sigma_i \max(0, \dot{d}_i)$, capped at A/2. R is held back.
- After the flow, R is released continuously to organs whose deficit stays positive, in proportion to their shortfall and up to their ceilings.
- This is the controlled release of Ch. 30: continuous state, no switch.

18.4 Evidence

Mathematics in code. `tests/test_conveyance.py` (in `verify.py`) checks 20,000 random systems:

- P1 conservation to 1e-9;
- P2, F never decreases;
- P3, the trajectory equals the closed form;
- P4, bounds and budget.

It also runs 2,000 systems over 30 steps and checks the full law never exceeds the budget or breaks a floor. That test found and fixed one real bug: an organ whose need fell below its device floor was given a ceiling under that floor.

Simulation. `hardware/site_exchange.py`: four GPU groups of 16 H100-class GPUs share one site budget (MLPerf gamma). Held-out seed 929292, 48 scenarios per budget. Results: `results/hardware/SITE_EXCHANGE_HELDOUT_*.json`.

Site budget (share of TDP)	70%	60%	50%
Site-budget violation minutes, native / independent / **conveyance**	10.5 / 1.5 / **0**	45.6 / 10.5 / **0**	106.1 / 31.3 / **0**
Energy (kWh), native / static split / independent / **conveyance**	142.5 / 117.6 / 104.1 / **103.8**	139.1 / 107.9 / 103.5 / **102.0**	127.3 / 97.1 / 101.0 / **96.4**
Backlog minutes, static split (the arm that also keeps the budget) / **conveyance**	32.0 / **1.0**	89.7 / **8.9**	187.3 / **72.2**

Reading.

- **Keeping the budget.** Conveyance is the only dynamic arm that never exceeds the site budget.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Energy.** It uses the least energy at every budget.
- **Against the only other arm that keeps the budget (static split):** 31-115 fewer backlog minutes.
- **At tight budgets, native and independent sizing show less backlog,** but only by drawing power over the site limit for 10-106 minutes, which in a facility trips breakers.
- **When the budget is below total need,** conveyance shares the shortage in proportion to need. That is the unique stable point of P3: fair, and not a choice of weights.

CPU and GPU on one budget. `hardware/node_exchange.py`: the same law with CPU organs beside the GPU organs. Four groups, each two 8-GPU servers (16 H100-class GPUs, 4 CPU sockets); each unit of GPU work needs 0.35 of the CPUs at full clock to feed it; demand 1.6x the plant traces (the GPUs are held back by the budget, not by work). Arms:

- **S** today's practice: every GPU at one fixed cap low enough that the site fits even with every CPU at maximum;
- **XM** conveyance among the GPUs, counting the CPUs' last measured draw;
- **XC** conveyance over CPU and GPU organs together: each CPU group is held to the clock its feeding work needs, and the watts it no longer holds flow to the GPU groups in deficit.

Seed 515151, 24 scenarios. Results: `results/hardware/NODE_EXCHANGE_*.json`. XC against S, paired:

Site budget (share of GPU TDP + CPU maximum)	60%	70%	80%
Work served	**+5.7%**	**+3.6%**	**+1.4%**
Backlog minutes	−15.8%	−27.5%	−17.8%
95th-percentile latency factor	−39.6%	−43.6%	−35.2%
Site-budget violation minutes, native / S / XM / **XC**	195.8 / 0 / 3.9 / **0**	140.9 / 0 / 6.2 / **0**	61.8 / 0 / 5.3 / **0**

Reading.

- **More work from the same building.** The tighter the budget, the more the CPUs' unused watts are worth to the GPUs.
- **Holding the CPUs is what makes the hand-over safe.** Counting the CPUs' measured draw (XM) serves about as much, but goes over the budget for 4-6 minutes in every setting: a CPU can rise between one reading and the next. XC holds each CPU to its allocation, so the watts it gives up are really free, and it never goes over.
- **Most of the gain is the CPUs' reserve, not their clock.** Against XM, XC gains ~1% work per kWh; against S, the watts a fixed plan must keep for CPUs that might peak are what the GPUs receive.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- The same holds for the least and most favourable MLPerf gamma fits, and for CPU shares 0.2 and 0.6 (+2.3% to +4.2% work at the 70% budget), `tests/test_node_exchange.py` checks N1-N4 in `verify.py`.

18.5 What is not claimed

- This is a simulation on declared device physics.
- The live levers that would carry it are GPU power limits (`nvidia-smi -pl`, DCGM), RAPL package limits, and pod CPU limits under a namespace budget. They exist in `omni_controller/muscles.py`, but the exchange between them has not run on hardware.

CHAPTER 19

Formal Status of the Mathematics

Not a global-stability proof of the forced six-state system. Proofs and their evidence classes: `docs/TRACKING_THEOREM.md`; every statement's class: `docs/EVIDENCE_LEDGER.md`.

Held:

- Isolated S-flow is the gradient of Phi (same formula in both cores).
- (T) Unsaturated: dU/dt = KP (sigma − U); V = e²/2 has dV/dt = −KP e² on that channel (Theorem 1).
- (T) Saturated, with the U drift bounded by F[bar] < U_AUTHORITY: V decreases wherever e ≠ 0; over the declared parameter box F[bar] = 16.86 < 25, so this holds over the box in continuous time (Theorem 2).
- (T, constant computed) Sampled-data RK4 controller: $e_{(k+1)} = 0.88\, e_k + d_k$; with epsilon_h computed over the frozen fixtures (V), the ultimate bound 0.0315 lies inside the 0.10 basin (Theorem 3).
- (T continuous, V discrete) Admissible box forward invariant (Theorem 4).

Open:

- epsilon_h and discrete invariance proved over the whole box (interval arithmetic).
- Inheritance embedding / intertwining residual against a real plant (Closed Structure Def. 11.1): instrumented in the GPU bench (`tools/gpu_reps.py`, representation fidelity), not yet measured on a card.
- Mapping of monograph Theorem 5.6 onto (E,U,I[U],S,B).
- A tracking bound for printed_eight_line (logistic U drift).

Default core remains symmetric_verified (mechanism id in `results/MECHANISM_IDENTITY.json`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 20

The Max, Formally: Definitions, Guarantees and Measures

This page states the engine's default brain, the max (`realms/compass_arm_v5.py`, `V5CFG`, engine omni-v5 by `tools/omni_version.py`), in definitions and properties. Each property carries its class: **V** a property of the code, proved by reading the code path named beside it and checked by the runs; **S** measured on our own models; **L** measured on real software; **P** measured by a physical meter. A V property is a statement about what the governor writes; it is not a statement that a measured outcome follows. Measured outcomes are stated with their numbers and their limits, and nothing on this page is claimed beyond them. The earlier formal results (`docs/TRACKING_THEOREM.md`, `docs/FORMAL_STATUS.md`) belong to the earlier core engine's channels and are not restated here.

20.1 Setting

A **body** is a set of plants p_1, ..., p_n run on one clock of decisions k = 0, 1, 2, ... (15 s each in the organisms). Each plant i has

- a template τ_i (compute_pool, thermal_zone, energy_storage, motion_axis, process_loop: `realms/plants.py`),
- one knob κ_i (capacity, setpoint, power or admission: the muscle's single wire),
- a **native controller** N_i, the plant's own law, unchanged by Omni-Compass: the pod autoscaler or the cluster autoscaler of a pool, the PI loop and staging of a cooled room, the dispatch of a battery site, the servo profile of an axis, the PI loop of a process,
- a state $x_{i,k}$ and an exogenous input $w_{i,k}$ (arrivals, heat from its own load, the site's own load, disturbance), drawn from the plant's seed, the same in every arm.

The plants are coupled through the body (`realms/harness.py`, `Body.couple`): with S_k the body's source power (every plant that is neither a room nor a battery site) and Z_k the rooms' power, room z receives heat $h[z] \cdot S_{k-1}$ and battery site s receives load $g_s \cdot (S_{k-1} + Z_{k-1} -$ its nominal), where h[z] and g_s are fixed by the body's calibration.

An **override** $o_{i,k}$ is a set of key-value pairs that bounds what N_i may do at decision k: a floor and a cap on the machines a pool's autoscaler may ask for, a setpoint for a room's loop, a reserve or a defended ceiling for a

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

battery's dispatch, a speed scale for an axis. The plant then steps by its own law under that bound: $x_{i,k+1} = f_i(x_{i,k}, N_i(x_{i,k}; o_{i,k}), w_{i,k}, \text{coupling_k})$. The empty override leaves N_i exactly as shipped.

Definition 1 (the two arms). The native arm writes $o_{i,k} = \varnothing$ for every i and k. The Omni arm writes $o_k = G(y_0, ..., y_k)$ for $k < K$, where y_k is what the governor reads at decision k and K is the decision at which the governor goes silent (in every run, K = 0.8 × the run's decisions), and the lease's walk back after K (Property V5). Both arms run the same plants, the same seeds and the same inputs.

Definition 2 (a governor, and a collection of tools). A rule G is a **collection of separate tools** if there are maps $g_1, ..., g_n$ with $o_{i,k} = g_i(y_{i,0}, ..., y_{i,k})$ for every i and k: each muscle's bound depends on that muscle's readings alone. G is **one governor over the body** if for some plant i the bound $o_{i,k}$ depends on the readings of another plant $j \neq i$.

Definition 3 (a twin). The twin T_i of a plant i is the same plant model, built from the same row and seed, fed the same exogenous input, run under N_i alone with no override. The pools' twins run inside the governor (`Bone.__init__`, `Bone.advance_twin`); a twin is the governor's model of what native does with that plant.

Definition 4 (room on a line). The lines of a body are the battery sites' connections and the rooms' cooling. For a line ℓ with distance to its limit gap_ℓ (the connection's limit plus what its battery can still clip, less its import; or a room's cooling capacity less its command, and its degrees in hand), one-decision climb up_ℓ (the larger of line_z times its step-to-step spread, the largest climb it has shown, and a prior of 5% of the line) and load per W of the body's power a_ℓ, the room over r decisions is

$R(r) = \min \text{ over } \ell \text{ of } (\text{gap}_\ell - \sqrt{r} \cdot \text{up}_\ell) / \text{a}_\ell$ (`Bone.room`, `Bone.room_for`),

the power the body may still add, read r decisions ahead.

20.2 What the code guarantees (class V)

Proposition 1 (V). The max is one governor over the body. In `max_body_step`, the floor written on a pool is max(now + can, base), where can is the number of machines whose idle power fits in min(R(1), R(r) − used); R reads every battery site and every room of the body, and `used` is what was granted to the pools decided before it in the same decision. So the bound on one pool depends on the readings of the body's sites, rooms and other pools: by Definition 2, the max is one governor, not a collection of tools. No set of separate tools has this property, however many run at once. ∎

Property V1 (one wire). Every override the max writes uses only its muscle's own key: {floor, cap} on a pool wired by capacity or setpoint; {setpoint} on a room wired by its setpoint; {setpoint} or {capacity} on a battery site wired by that knob; {capacity} on an axis alone (inside a body an axis keeps its everyday speed); nothing on any other muscle. Proof: every branch of `max_apply`, `max_hold` and `max_body_step` writes only these keys. ∎

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Property V2 (never fewer machines than native asks for). For every pool i with a twin, at every decision k < K, the count its autoscaler is asked for is never below the twin's: a floor written is ≥ $m(T_{i,k})$, and a cap written is ≥ max($m(T_{i,k})$, n[min]), where m counts machines in service and starting. Proof: `max_ask` sets base = $m(T_{i,k})$ and want = max(forecast, base); `max_body_step` writes floor = max(now + can, base), and on a shortfall keep ≥ base. The pool's own law is a function of its own arrivals and settings only, so its twin runs exactly as the native arm's pool does: $m(T_{i,k})$ is the count native asks for at that decision. ∎

Property V3 (native at its limit is left alone). Under the max, the override of every muscle wired by power or admission, of every process loop, and of every axis whose machine has no speed above its everyday setting is empty at every decision: native already runs them at the top of their range. Proof: `max_ask` returns nothing for them and `max_body_step` writes ∅. ∎

Property V4 (grants come out of the room the lines have). At a decision with R(1) ≥ 0, the idle power of the machines the max adds, summed over the body, is at most R(1), and the machines of a pool that take r decisions to start are granted only out of R(r) less what was already granted. With R(1) < 0, what the max added goes back, the pools holding the most first, never below each pool's twin (V2), and no room is pulled below its operator's setpoint. Proof: `_units`, the running decrements of r[src], r[all] and used, and the shortfall branch of `max_body_step`. ∎ *Limit, stated with it:* R is a forecast. A line that climbs further within one decision than any climb it has shown, while power granted earlier is still in service (power is handed back one decision after a line shows the need), can still be carried over its limit. Section 20.4 gives how often that happened.

Property V5 (the lease: everything handed back). If the governor goes silent at decision K, every override moves 1/(K + W − k) of the way to its native value at each decision k ≥ K and is native from decision K + W − 1, W = 24 (`walk_back`; a floor that only held a pool at native's own count is let go at once). Every run checks every knob on every seed (`restore_ok`); a muscle or organism that fails is labelled INVALID. In the official table every muscle and every organism passed. ∎

Property V6 (the live ranges: from 1 up to one click under the system's own capacity; amendment 5). In the live Kubernetes controller, with n the machines in service, m the floor (two), T the machine lever's top and P the ready workers: T is the operator's `--max-nodes` where given; else P plus the room the cluster's own autoscaler publishes (the sum over node groups of maxSize less registered); else, where Omni-Compass resizes the pool with the operator's command, T = f − 1 for the smallest count f the pool's command has refused, and T = ∞ before any refusal; else T = P. Every machine count written lies in [m, T], moves at most the rate limit a decision (shield I5), and cruise writes min(T, n + step). The replica cap c of a sensed service never exceeds C = max(1, ⌊(A − Q)/r⌋ − 1), A the allocatable CPU of the ready workers and of the machines the system can still add (by T), or, where every node is a container on one machine (kind), that machine's CPU once; Q the requests of every other pod and r one pod's request: one pod under what the machines hold. At the max, every pod CPU limit written is ≥ the operator's own, and no frequency ceiling or GPU limit is written below its top; on the tune-down, a CPU limit is never written under the pod's request nor its use × (1 + headroom), and a GPU limit never under the card's own minimum. Proof: `Controller._machine_room`, `_max_nodes`, `_pool_command`, `_live_ceiling`,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`Muscles.push`, `_power_cap`, `_hardware` and `convey` (`omni_controller/`); `tests/test_compass_controller.py`, `tests/test_omni_controller.py`, `tests/test_muscles.py`. ∎ *Limit, stated with it:* T = ∞ refuses no count in advance: machines are still added only by the controller's own rules (the scheduling floor while pods wait for a place, one more past the compass's wall, the governor's law, cruise while a queue waits), within the rate limit; the first refusal by the pool, or by the cloud's quota behind it, sets the top from then on.

Definition 5 (the readings). For each seed s, the paired difference is d_s = Omni_s / native_s − 1 for work and energy, and Omni_s − native_s (percentage points) for the share of decisions over the line. Over n seeds, the 95% interval is mean ± t[0.975, n−1] · sd / √n. A gain or a loss is a row whose interval is clear of zero; otherwise the row reads no difference beyond the noise. Across runs A, B and C: the same sign in all three with every interval clear of zero reads "lower (or higher) in all three of our runs"; an interval across zero reads "no difference beyond the noise"; clear intervals of opposite signs read "the runs disagree" (`tools/readings.py`, `tools/confirm_abc.py`).

20.3 What the runs measure (classes S, L, P)

- **S, the muscles alone, 1,107 muscles, preregistered seeds 3000-3009, both loads** (`results/realms_v5/`, commit `eea291501848`, digest `5648f2902f1fae68`, every muscle its own card, section 20.5): 527 toward their aims, 421 at their limit on native (V3: the max writes nothing), 159 no difference beyond the noise, **0 LOSS or power given, 0 mixed**.
- **S, inside the organisms, the same seeds, three loads**: every organism with fewer misses as a whole under every load, and **0 plants with less work or more misses** in every organism under every load (0 of the 1,908 in the four stacked). Misses (the share of plant decisions over their line) fall 1.9% to 7.5% under each organism's own load, 1.4% to 7.2% under the climb (the load stepping 1 2 3 2 3 4 ... to twice its own) and 1.1% to 8.3% with work waiting in line, against 1.6% to 6.4%, 0.9% to 3.5% and 1.0% to 7.2% with one global setting for every muscle (digest `9be48445`, the same seeds), and the power given and the wear fall with them (Compute under its own load: power given +0.06% to +0.01%, wear +0.35% to +0.06%). Every row is reported in `results/realms_v5/REALMS_V5.md`.
- **L, real software**: `docs/THE_RECEIPTS.md`, receipt by receipt.
- **P, a physical meter on a real chip**: not yet produced on the current governor (`docs/THE_SEAT.md`, section 5).

20.4 The living body: why plants read LOSS, and the version that cures them (the engine since 12 October, night)

Found on development seeds only (7100-7299), never on the preregistered ones. Under the climb, rooms and UPS lines reach their limits in both arms; a plant read LOSS when the Omni arm reached a limit a moment sooner, or when a sense the brain read was wrong. Each cause, and the property of the code that removes it:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

1. **A room snapped back.** A room cooled ahead of the heat, then returned at once to its operator's setpoint when the lines ran short: its own PI loop overshot warm of native, and once its cooling ran flat out the difference was frozen in. *V9 (the pedal lifted, never snapped):* a pull-down is handed back toward the operator's setpoint over the cooling's own reaction time. *V10 (no pull-down that cannot be held):* a new pull-down is granted only out of the room the lines have for that reaction time.
2. **A battery's charge counted as room after it ran out.** *V6:* a site whose battery covers its draw, but whose charge above its floor runs out within r + 1 + T decisions (T = 2: the heat of machines handed back reaches the rooms a decision later and their cooling answers over its own averaging, one more), has as room on its line only its limit less its whole draw, read at the draw's own climb.
3. **Machines handed back while a queue needed them.** *V7:* on a shortfall, a pool is never held below $\lceil(\lambda_{k-1} + Q_k / \Delta t) / \mu\rceil$.
4. **A room left warmer than native.** *V8:* every room has a twin: the same room under its operator's settings, warmed by native's heat (the body's source power with each pool's power replaced by its twin's). A room wired by its setpoint that reads warmer than its twin by e > 0 has its setpoint written at most t[set] − e, unless its cooling is already flat out.
5. **A room that cannot cure itself.** *V11 (the body cures what a muscle cannot):* a room wired by its cooling units, a power or an admission gate, or whose cooling is flat out, and that reads warmer than its twin by e > 0, puts a heat debt $e \cdot C / (\Delta t \cdot h)$ on the body (C its heat capacity, h its share of the body's source heat); the room the bone grants is bounded by minus that debt, so the machines the max added go back until no such room is warmer than native's.
6. **A battery that never timed its discharge.** *V12:* a battery wired by its power holds its charge while its line has room after the largest climb it has shown, line_z of its wobbles and a twentieth of its limit, and gives native's full power once the line could need it; it holds nothing until it has read its line for a window of decisions (20).
7. **A lock in the senses.** *V13 (a forecast starts at the first reading):* a plant shows its initial values before its first step; the jump from them to the first reading was taken as the line's largest climb ("never forgotten") and kept every battery's timed discharge shut for the whole run. Every forecast of a line now starts at the plant's first real reading, and the bone reads nothing before the body's first step.
8. **A twin that started wrong.** The body reads its sources at their nominal power before the first step; the room twins read zero, so for the first ten or so decisions they ran up to 0.3 °C apart from native's rooms and the body cured differences that did not exist. The twins now start from the body's own reading (V8, the definition above), and with the max writing nothing every room twin equals its room to the last digit, every decision (`tests/test_compass_arm_v5.py`).

The battery's admission gate stays native: deferring a site's flexible load to keep its line clear left part of that work not caught up by the end of a run (work first).

Measured on development seeds 7100-7199, the four realms under the climb, 400 organism runs each way:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

	Plants with less work or more misses	Misses (pp), the four realms	Work, the four realms
the engine before (omni-v5, 8d4f91f0)	19	−0.109, −0.125, −0.005, −0.144	+0.0013%, +0.0016%, +0.0000%, +0.0022%
V6-V10 only	7	−0.104, −0.110, −0.005, −0.137	kept
this version (V6-V13)	**0**	**−0.206, −0.187, −0.143, −0.212**	+0.0008%, +0.0010%, +0.0000%, +0.0016%

The misses fall about twice as far as before (in the Energy / Facility / Industrial body, from almost nothing to 0.14 points), and no plant in any of the 400 runs did behind native. The work gained is about two thirds of the version before: the heat debt hands back machines while a room that cannot cure itself is warmer than native's.

One lock is left, said plainly: a line that once jumped (a battery that stopped covering its site) is read as able to jump that far again for the rest of the run, which keeps the max from adding machines on that line. Removing it (every line's climb read on the site's whole draw) roughly triples the misses removed on the same seeds (−0.34 points), but in 36 of the 400 runs a machine pool whose queue sat at its limit then did more work and took one decision longer to clear it, which our decision count reads as LOSS. That is the next version's work, and it enters the engine only as a new version, every result run again on it, the founder told first.

20.5 Every muscle its own card (the engine since 13 October)

The founder's question of 12 October, night: does every muscle have its own tuning, maxed to its own benefit, instead of one global setting for a thousand muscles? It does, and the engine now carries it.

Definition 6 (a card). For muscle i, a card θ_i is the brain's settings for that muscle: for a machine pool, its edge utilisation u_i (how full it runs on its forecast before it calls for more) and its window w_i (decisions of history it reads); for a battery, its line margin z_i (wobbles kept under its line) and, wired by its power, its hold window and hold margin. The brain is one (the same law, the same bone, the same body); only these numbers differ by muscle. A muscle without a card runs the global setting θ_0 (u = 0.95, w = 20, z = 3.09, the hold window equal to w, a hold margin of a twentieth of the line). The tune-down never reads cards.

The search (development seeds only, `tools/tune_cards.py`). Every muscle whose settings can change what the max does (458 machine pools, 44 batteries, 48 axes) tries every candidate in a declared grid on seeds 7100-7109, alone, under its own load and with work waiting in line; θ_0 is always among the candidates. Pools: u ∈ {0.70, 0.75, ..., 0.95} × w ∈ {10, 20, 40, 60}; batteries wired by their power: z ∈ {0, 0.5, 1, 2, 3.09} × hold window ∈ {3, 5, 10, 20} × hold margin ∈ {0, 0.02, 0.05, 0.10}; other batteries: z ∈ {0, 0.25, ..., 4}; axes: heat head × strain. A candidate is kept if it beats θ_0 beyond the noise on those seeds (fewer misses, more work or faster: a 95% interval of the paired difference clear of zero), read LOSS than θ_0 nowhere, and is never behind native: on no seed and under no load over its line more often than native, doing less work or clearing its line later (the engine's own no-harm tests, seed by seed); among those, the fewest

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

misses, then the most work, the fastest, the least energy. It is then run again against θ_0 on seeds 7200-7209 and enters the engine only if it wins there too: a card must win twice. The preregistered seeds 3000-3009 are never read by the search.

V14 (the body has the last word on a line it shares). Inside a body, a battery wired by its power keeps the body's margins on its line: its z and its hold margin are never slimmer than θ_0's; its own window stays. A card is chosen for the muscle alone, and a battery's line inside a body carries the whole body's swing, which the battery alone never saw: the one plant behind native that the cards produced in development (a rack's power allocation, one extra decision over its line in the whole tower with work waiting, one seed of ten) was exactly that case, and V14 removes it.

What the search found: 409 of the 550 searched muscles beat θ_0 on the first seeds; 364 won again on the second (45 fell back to the global setting, none LOSS); the engine's cards are those 364 (`realms/muscle_cards_v5.csv`: 333 machine pools, 31 batteries). No axis setting beat the global one. A first set of cards was chosen by the mean over the seeds alone (371 cards); the engine's own no-harm test then caught one of them (an electrolyzer battery, one decision over its line on one seed), and the rule became the seed-by-seed one above. The cards are the ones it chose. A room alone and a process loop alone have nothing for a card to fix (native almost never misses there), and the 421 knobs native already runs at their limit have nothing above native to give: their help comes from the body. Every card is printed in plain words in the muscle catalog (`docs/MUSCLE_CATALOG.md`).

Inside the six organisms, on both development sets (360 organism runs, every plant against native): 0 plants with less work or more misses, and the misses removed against native 1.7 times the global setting's over the three loads (1.2 times under each organism's own load, about 1.9 under the climb, 1.7 to 1.8 with work waiting in line):

Seeds	Own load: global → cards	The climb: global → cards	Work in line: global → cards	Plants with less work or more misses
7100-7109 (the cards chosen here)	0.078 → 0.094 points	0.153 → 0.293 points	0.088 → 0.161 points	0
7200-7209 (the cards checked here, on untouched seeds)	0.074 → 0.087 points	0.145 → 0.282 points	0.086 → 0.149 points	0

Work is unchanged within a millionth, and the power given is lower than the global setting's in the organisms where it moves at all (Compute under its own load +0.058% → +0.011%). The preregistered result, on seeds the search never read, is section 20.3.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

20.6 Limits, stated plainly

- A card is chosen for its muscle alone. Inside a body the bone shares the lines and V14 keeps a battery wired by its power to the body's margins; the organisms were checked with every card on both development sets (0 plants with less work or more misses) and on the preregistered seeds (section 20.3), but a card is not tuned for the body it sits in.
- Many machine pools chose the most aggressive edge the search offered (256 of 333 cards at 0.70): their own limit may lie further out. Searching further is the next version's work, a new version with every result run again.
- V properties bound what the governor writes. They do not by themselves bound an outcome on a line the governor does not control (section 20.4).
- R is a forecast (V4). A climb no line has shown before can beat it within the one decision power takes to be handed back.
- S results are our models. They are never presented as proof of a physical saving; the physical receipt is a P run.
- The twin is exact for a pool (its law reads only its own inputs) and for a room (its heat is the body's own source reading with each pool's power replaced by its twin's, so with the max writing nothing it equals the room to the last digit); with the max writing, a room twin is native's room under native's heat as the brain reconstructs it, not a meter.

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

CHAPTER 21

The Mechanism of Action

Every statement here is tied to code in this repository, and the measured numbers are reproduced by `python tools/mechanism.py`, which writes `results/MECHANISM_OF_ACTION.json`. The engine files are frozen. Their SHA-256 hashes are checked by `verify.py`.

21.1 The state: six numbers that describe the whole system

Symbol	Name	What it stands for in a data centre
E	energy / excitation	how hard the system is being driven: queues, overload, power, heat
U	order parameter	how healthy and settled the system is: +1 healthy basin, -1 failed basin

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Symbol	Name	What it stands for in a data centre
I[U]	integrated need	unmet need accumulated over time: memory of pressure that has not been relieved
S	stress	slow structural stress from heat, power, network, security, staleness
B, Ḃ	bath and its rate	a damped oscillator driven by stress: the slow "tide" of physical load

21.2 The equations (omnicompass/core.py, lines 5-12)

```
(1) dE/dt     = −α[E] E + β[int] + β[ext] + v[eff]
(2) dU/dt     = μ U (1 − U²) − (dE/dt)/E[max] − λ[U] U + u,          |
    ↪ u| ≤ 25
(3) dI[U]/dt  = (1 − U) − σ₁ E − δ S − λ[I] I[U]
(4) v[eff]    = cos(ω[B] t / 2) · c · tanh(λ₀ + λ₁ (U − 0.5) + λ₂ S)
(5) Φ(S)      = αₛ S² / 2 + βₛ S³ / 4 − δ S

(6) dS/dt     = −dΦ/dS = δ − αₛ S − (3/4) βₛ S²
(7) dB/dt     = Ḃ ;   dḂ/dt = γ[c] δ S − (ω[B] / Q[B]) Ḃ − ω[B]² B
(8) R[B]      = finite-difference audit of (7) at every step, never fe
    ↪ d back
```

Each term does one job:

- **(1) Energy.** Energy decays at rate α[E] = 3.487. It is driven by two forcings: internal pressure `β[int]` (queue and overload) and external pressure `β[ext]` (power, heat, network, staleness, security). The drive term `v[eff]` (4) is bounded by `c` through the hyperbolic tangent, so no input can drive E without limit. This is the saturation (the tanh "speed limit"): however extreme the observation, the engine's response rate is capped at c = 1.
- **(2) Order.** `μ U (1 − U²)` is a double-well (bistable) drift with two stable basins, U = +1 (healthy) and U = -1 (failed), and an unstable ridge at U = 0. A rise in energy (`dE/dt`) pushes U out of the healthy basin. `u` is the bounded control input.
- **(3) Integrated need.** I[U] grows while U is below 1 (the system is not settled). It is reduced by energy and stress already being dealt with, and it leaks at λ[I] = 0.719. It is the engine's memory of unrelieved pressure.
- **(5)-(6) Stress.** A cubic potential (the "well in the basin"). S rolls downhill to its stable root. This is the slowest restoring force.
- **(7) Bath.** A damped harmonic oscillator (natural frequency ω[B] = 1, quality Q[B] = 2.5) driven by stress. It turns stress into a smooth, lagged "tide". It is the only second-order element.
- **(8) Audit.** The bath equation is re-checked numerically every step. The check is never fed back.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Integration: classical 4th-order Runge-Kutta, 10 micro-steps of 0.01 per macro step (`rk4_step`, `macro_step`). The C++ twin matches the Python to 3.6e-15 on 500 reference trajectories.

21.3 From telemetry to state (omnicompass/adapter.py, `observe_vector`, `assimilate`)

Each decision the telemetry is normalised to nine observations: q (queue), load, power, thermal, network, drift, stale, security, conflict. An observed state is computed from them:

```
E[obs]   = 0.25 q + 0.18 max(0, load − 0.85) + 0.16 power + 0.13 therm
    ↪ al + 0.10 network + 0.08 drift + 0.06 conflict + 0.04 stale
U[obs]   = 1 − (0.27 q + 0.18 power + 0.16 thermal + 0.12 network + 0.
    ↪ 12 drift + 0.10 conflict + 0.05 stale)
S[obs]   = 0.36 thermal + 0.28 power + 0.18 network + 0.10 security +
    ↪ 0.08 stale − 0.20 q

B[obs]   = 0.52 power + 0.26 thermal + 0.22 drift
I[obs]   = q + drift + conflict
β[int] = 0.58 q + 0.42 max(0, load − 0.75)        β[ext] = 0.33 power
    ↪ + 0.27 thermal + 0.20 network + 0.12 stale + 0.08 security
```

The state is blended toward the observed state with weight a = 0.339: `x ← (1 − a) x + a x[obs]`. It is then evolved by one macro step (0.1 engine time units) of equations (1)-(7).

21.4 From state to action (the allocation law, `Governor.step`)

```
push     = u(x) / 25,  u(x) = clip(−f[U](x) + 12 (1 − U), −25, 25)
    ↪     equation (2)'s controller, evaluated, not applied
ρ*     = clamp(ρ₀ − k[I] I[U] − k[E] E, ρ[min], ρ₀)
    ↪    target utilisation (the HPA target)
n[req]    = ceil(n load / ρ*) + ceil(k[q] q n)
    ↪        machines wanted

release  only while push ≤ push_release (the engine reports convergenc
    ↪ e), after band and dwell
cap      = clamp(cap_now load (1 + margin), cap_min, 1) when calm, U ≥
    ↪  U[gate] and the line clean for the governor's
           own clear time (omni-v15: the request cannot switch back an
    ↪ d forth at one threshold); 1 under backlog or a breach;
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
           always ≤ 1 − cap_gain max(0, B − B[cap])
    ↪       the bath limits the power cap
change   permitted only while U ≥ U[gate] and no security block
rollback authorised if S ≥ S[rollback], U is falling, or a security bl
    ↪ ock is seen
```

21.5 Which equation drives which muscle

Muscle	Engine quantity used	Where
HPA target (pods)	rho* from I[U] and E, equations (1) and (3)	adapter `Governor.step`; live `omni_controller/controller.py`
Node pool (machines)	n[req] from load, queue and rho*; release gated by push from equation (2)	same
Power cap (pod CPU limits, CPU frequency, GPU limit)	cap from U gate and bath B, equation (7)	same; `omni_controller/muscles.py`
Rollout guard	change_permitted (U), rollback (S, dU)	`muscles.py`
Security hold	security observation raises S and blocks expansion	adapter, shield
Speed-first law	rho* for pods; push gates machine release	`omnicompass/speed.py`
Right-sizing	rho* sizes CPU; E widens the memory margin; push gates scale-in	`omnilab/rightsize.py`
Cold start	rho* sizes instances; I[U] sets how long to keep warm	`omnilab/coldstart.py`
GPU packing	I[U] pre-warms nodes; push gates power-off	`omnilab/gpupack.py`
Training power	E tightens the applied ramp limit under sustained grid stress	`omnilab/powersmooth.py`
GPU health	S and E shorten the checkpoint interval	`omnilab/health.py`
Cooling	E widens the inlet-temperature margin	`omnilab/cooling.py`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Muscle	Engine quantity used	Where
Inference	the full allocation law (node_delta) sizes replicas	`omnilab/inference.py`
Agent containment	I[U] throttles; the security observation contains	`omnilab/containment.py`
VM energy	I[U] and E raise the learning rate when attribution goes stale	`omnilab/vmenergy.py`

21.6 Measured: what the equations actually do inside the governor

Measured on a real observation stream (fleet plant, web services, fleet mode, 360 decisions):

Mode (eigenvalue of the Jacobian at the operating point)	Time constant	In decisions
-7.90	0.13	1.3
-3.10	0.32	3.2
-0.72	1.39	13.9
-0.20 +/- 0.98i (the bath)	5.0, period 6.4	50, rings every 64 decisions
-0.17	6.0	60

- **Every mode is stable** (all real parts negative). The operating point is a stable equilibrium inside the healthy basin: E = 0.095, U = 0.976, S = 0.315, B = 0.10. This is the basin keeping everything in, stated as mathematics.
- **Per decision, the equations and the observation move the state about equally.** Across all six variables the equations contribute 45-50% of each step's movement and the observation blend the rest.
- **Equation (2)'s control input u is not applied during governance.** The engine evolves open-loop (u = 0). The control law is evaluated only to produce `push`, which decides when capacity may be released.
- **Compute:** about 95 microseconds per decision in Python, and about 2.9 microseconds in the C++ twin.

21.7 What this explains, and what it points to

- **Why "engine off" scores close to "engine on" in the muscle benchmarks.** With the equations frozen, the observation blend still moves the state. Both versions are then low-pass filters of the same telemetry. The engine's fast modes (1-3 decisions) behave almost like the blend itself. Its distinct contribution is its slow memory: I[U] (14 decisions), S (60) and the bath (period 64 decisions). The muscle mappings mostly read E and rho*, which live on the fast modes.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The engine clock is not matched to the workload's rhythm.** One decision advances engine time by 0.1. At 60-second decisions the bath rings every 64 minutes, but web demand has a 24-hour rhythm and training power a 2-second one. This is a wiring choice in the adapter (engine time per decision), not a property of the equations.
- **The controller in equation (2) is unused as an actuator.** Wiring its output u (or U's trajectory under u) directly to the actuators is the unexplored way to "let the math do the work".

Three wiring experiments follow directly. None changes a single equation:

1. **Clock matching.** Set engine time per decision so that the bath's natural period equals the workload's rhythm. The oscillator in (7) then carries the cycle and can anticipate it.
2. **Slow-mode actuation.** Drive machines from I[U], S and B (the memory modes) rather than E (a fast mode).
3. **Closed-loop actuation.** Apply equation (2)'s u and use its magnitude, not only its sign, as the actuator command.

Each is testable in the existing harness against every platform in the league (tuning/league.py), and will be tested there.

21.8 One pedal: push, lift off, idle, reset, and the kill switch

Omni-Compass drives a system the way a self-driving car drives itself: it feels, gauges and adjusts, with nobody in the seat. The words below are used the same way everywhere in this repository.

Two names only. native is the system as it runs on its own (Kubernetes and its autoscalers, the card's firmware, CityLearn's controller). **omni** is Omni-Compass on top of native: the same native system, with Omni-Compass governing it. Omni-Compass never runs instead of native, only on top of it, so every comparison is native against omni.

The modes. Off (manual): Omni-Compass not driving, native runs alone. Watching: Omni-Compass reads every gauge and logs what it would do, and writes nothing. Autopilot: the one pedal below. Cruise and idle at once: for work that comes as a pile.

Term	Meaning	In the code
Push	Traffic climbs and capacity is added at once, never held back; the whole cluster can take work the instant it is needed	(8.3); a lower HPA target is never limited
Lift off	Traffic falls and capacity is eased off a step at a time, handed back toward native, never below idle	(8.4)-(8.5); coasting, rule 6

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	Meaning	In the code
Idle	The pedal lifted all the way: calm traffic, the engine running at its floor (two machines in service, ready)	`MIN_NODES`, the floor of the staging law (8.8)
Reset	Every setting the governor ever wrote is handed back to where native had it, read back, and the record removed; the next run starts fresh. Every benchmark arm ends with one and checks it	the reset file (`--kill-file`, a name kept for compatibility), `Controller.restore`
Cruise	A pile of work waiting for a place: machines in service up to the pool's own top, read live (a fixed pool's every machine at once, a growing pool a step a decision), held there without second-guessing, until the line has been empty for two decisions	rule 7, `--cruise-after`
Idle at once	The work is done and demand is at zero (the service at its least pods, wanting no more, at half its target or less): the pedal lifted all the way in one move, straight to idle (the floor), every safety check still holding	rule 8, `--idle-demand`, amendment 9
Kill switch	Security only: one switch in a human hand that turns Omni-Compass's governing off across the whole system at once, for anything rogue or anyone trying to drive a system through Omni-Compass's brain	`omnicompass/master.py`, `tools/omni_switch.py off`

The floor is a number the operator sets; lifting off never takes the system below it, however far the pedal is lifted. Earlier dated records, and the sealed original engine files, call the reset a "kill switch"; they are kept word for word.

21.9 The staging law: machines on and off in order, with demand that moves

Demand climbs, spikes, eases part way, climbs again and settles to idle. The staging law decides how many machines are in service at every decision, and which ones, so that capacity follows demand up at once and comes back down only as far as it safely can. It is the compass law (`omnicompass/compass_law.py`) applied to the machine pool (`omni_controller/controller.py`), the release gate (`omnicompass/nervous_system.py node_release_gate`), the verdict

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(`omnicompass/verdict.py`) and the actuator (`scripts/kind_nodepool.sh`). Every symbol below is a line of that code.

Position and force. The service reading r (the 95th-percentile response time, or the pressure that stands for it) is placed in its band [lo, hi], where hi is the response line:

```
p = (r - lo) / (hi - lo),      v = s v + (1 - s) (p - p_prev) / dt
    ↪                    (8.1)
F = A                                      if p >= 1 - c        (fail
    ↪  up, past the wall)
F = A tanh( (kp (p - p0) + kd v) / A )     otherwise; F = 0 if F < 0
    ↪ while v > 0 and p > p0   (8.2)
```

with the band's center p0 = 0.5, cushion c = 0.05, authority A = 1, kp = 1 and kd = max(0, 2 sqrt(kp tau / dt) - 1) dt (critical damping for a muscle that answers in tau seconds). F > 0 asks for capacity, F < 0 offers it back.

How many machines. With n machines in service now, n[nat] the count native ran, m the floor (two by default, `MIN_NODES`) and M the machines that exist (every machine usable; an operator may hold whole machines back with `NODE_CUSHION`, off by default), every count stays in m <= n' <= M. The protection is not a machine sitting out: it is the band inside every machine. Capacity is added at 95% of the response line, before the line is reached (8.2-8.3), and a machine goes back only if the ones left still run at or under the engine's utilisation target rho (8.5): a margin spread over all of them.

```
up      n' = min(M, max(m, n + 1))                                 if p >
    ↪ = 1 - c                    (8.3)
down    n' = max(m, n - 1)                                         if F <
    ↪  R and p < p0 and the verdict
                                                                   allows
    ↪  n_nat - (n - 1) and the gate G holds  (8.4)

hold    n' = n                                                     otherw
    ↪ ise
```

with the release threshold R = -0.2. Up is immediate and is never gated. Down is one machine per decision, never below the floor, and only when every term of the release gate holds:

```
G = [n > 1] and [nothing pending] and [pods not scaling up] and [no br
    ↪ each now] and
    [ used / ((n - 1) x per_node) <= rho ] and [contraction authority]
    ↪  and [every sense live] and
    [the last machine command landed]
    ↪                          (8.5)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

rho is the engine's utilisation target: after a machine goes back, what remains runs at or under rho. The verdict (section 21.7 of the GPU preregistration, `omnicompass/verdict.py`) allows a machine count below native only after a paired trial showed the service at that count at most a 2% LOSS than at the count before; that 2% is where the trigger sits, inside the engine.

This is hysteresis by construction. The up condition (p >= 0.95) and the down condition (p < 0.5 with F < -0.2) are far apart, so a demand that hovers never makes machines flap; a spike crosses the wall and adds a machine at once; a dip has to be deep, calm and sustained before one goes back, and then only one per decision. Between the two, the count holds.

Which machines. The actuator orders the pool:

```
idle   the open machine with the fewest serving pods, then the fewest
    ↪ pods, goes first;
       it is marked prefer-not (PreferNoSchedule) and its pods first t
    ↪ o go (pod-deletion-cost), so the
       autoscaler's own scale-down empties whole machines; no pod is m
    ↪ oved, evicted or restarted   (8.6)

wake   the idle machine still carrying the most work goes first (it is
    ↪  warm); the mark comes off and it is in
       service at once, with no boot
    ↪                          (8.7)
floor  two machines always in service (MIN_NODES, the operator may set
    ↪  more), ready for a spike  (8.8)

margin  every machine usable; the cushion is the band inside each one
    ↪ (the 5% before the line, and rho), not a
        whole machine held out
    ↪                           (8.9)
```

An idled machine stays powered and Ready at its floor (park_frac x idle power), never off: a pod that finds the open machines full lands on it at once, so no request ever waits on a machine Omni-Compass idled. Where a node autoscaler underneath really removes an empty machine (Azure's cluster autoscaler, Karpenter), that same order hands it the emptiest machine first, and the bill falls with the machine.

The same law on every discrete unit. Machines are one case. In the muscle models (`realms/compass_arm.py release_safe`), a cooling plant's chillers and a compute pool's machines follow the same law: one unit back only when the units left cover the recent peak with headroom (RELEASE_MARGIN 0.6 for machines, 0.8 of a unit's capacity for chillers), one at a time, and back at once when the wall is reached.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What is proved, measured and open. The order (8.6)-(8.8) is checked through the real actuator by `tests/test_staging_order.py` (the emptiest idles first, the warmest wakes first, two always in service, every machine usable, no pod moved). The gate (8.5) is checked by `tests/test_node_release_gate.py`. On real Kubernetes the law gave the same work 55-65% faster on 29-36% fewer machines (sets 22-27) and 48% more work on the same machines (the capacity test); its run under demand that wanders (up, spike, part way down, back up, idle) is in progress. Open: an order across different kinds of unit (a battery before a generator, a CPU before a GPU) is not wired; each pool is ordered within itself.

21.10 The state each muscle carries, and the modes as a state machine

9.1 The compass on a muscle is a dynamical system, so its state must persist. On muscle i the compass carries $z_i = (p_i, v_i, x_i)$: the position of the service in its band, its velocity, and the knob's continuous value. At each decision k, with reading r_k:

```
p_k = (r_k - lo) / (hi - lo),   v_k = s v_{k-1} + (1 - s)(p_k - p_{k-1}) / dt,
    ↪  F_k = F(p_k, v_k) by (8.2)    (9.1)
x_{k+1} = clamp( x_k + sigma g(F_k) F_k (x[hi] - x[lo]), x[lo], x[hi] ),   g
    ↪  = UP if F_k > 0 else DOWN          (9.2)
```

with x_0 the native value and sigma = +1 when a higher value is more capacity. Two properties depend on z persisting: (a) the knob integrates the force, $x_k - x_0$ = sum_j sigma g F_j span (until the cover binds), so a calm muscle eases step by step toward its cover; and (b) the damping term kd v_k needs p_{k-1}. If z were made fresh at every decision ($p_{k-1} = p_k$, v = 0, $x_k = x_0$), (9.2) would collapse to x_{k+1} = clamp(x_0 + sigma g F(p_k, 0) span): a memoryless map of the present reading, at most one step from native, with no damping. That is a different, weaker law. A renamed attribute did exactly that on 2026-10-05; `tests/test_compass_arm.py` now fails if the compass is ever made twice on one muscle.

9.2 The modes. The live controller is in exactly one mode at each decision. With q_k the pods waiting for a place, u_k the scaling-up flag, c the cruise count (`--cruise-after`, 2), M the machine ceiling and m the floor:

```
cruise on      q_j > 0 for the last c decisions
    ↪                              (9.3)
cruise off     q_j = 0 and not u_j for the last c decisions
    ↪                              (9.4)
in cruise      n' = min(M, n + s), M the pool's own top read live, s t
    ↪ he step; the service's autoscaler target held at the operator's
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ own; the floor step
               makes no read and no write
    ↪                              (9.5)
idle at once   (not cruising) q = 0, no breach, not u, every sensed se
    ↪ rvice idle or at demand
               <= b, the gate G (8.5) holds, every sense live, the las
    ↪ t command landed:
               n' = max(m, smallest n carrying the CPU in use at rho)
    ↪                              (9.6)
idle (a service at its floor)   I(h) := current = min = desired, and u
    ↪ _h <= target_h / 2           (9.7)
push / lift off / idle         (8.3) / (8.4)-(8.5) / (8.8)
```

9.3 Why cruise steps back (9.5). In cruise the machines climb to the pool's own top (a fixed pool's every machine at once, n = M; a growing pool a step a decision), so the floor step's only action (raise n to the floor the waiting pods need) adds nothing the climb does not already give: for a fixed pool floor <= M = n. Its reads (every pod, every node, node metrics, every HPA, each five seconds) carry no value of information, so removing them cannot change any action; they only cost CPU and API-server time on the machines the queue is using. The service's response time is also confounded in cruise: with a queue of other work holding the machines, a request waits on contended CPU, R = S / (1 - u[host]), not on the service's own pods. Reading that R as the service's own queue and adding pods would answer the wrong cause. So in cruise Omni holds the machines and leaves the service to its own autoscaler. `tests/test_compass_controller.py` checks that the floor step makes no API read during cruise; the controller before this rule made the full set of reads every five seconds.

9.4 Why idle is read from the autoscaler's floor (9.7). A served service never reads zero: a health probe or a client's keep-alive keeps one pod at a few percent of its request. In the batch test, one pod at about 10% of its request read as demand 0.10, above b = 0.05, and idle waited 15 minutes after the queue emptied. The autoscaler's own floor is the observable of "no demand beyond the minimum": the pods it keeps are its minimum, it wants no more, and they run under half the target. In the five frozen-engine web tests the service was never at its floor (0 of 5,102 readings), so (9.7) changes nothing there.

9.5 Reporting resolution. A paired change smaller than one part in a million of the value (|mean difference| <= 1e-6 |native|) is read "same", the number still shown. Below that, a difference is rounding in floating point and in the models' accumulators, not behaviour; a deterministic run gives a zero-width interval around it, which would otherwise read as proven.

9.6 A lever is moved only where moving it can pay. The compass decides how far and how fast. Whether a lever may move at all is a property of the muscle's physics. Four cases, each derived from the muscle's own model and each its own test:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- *Cooling power.* A zone's heat must leave it: over a run, the heat removed equals the heat made plus what the room's mass stores. A cap on cooling power q[max] cannot remove less heat. It only lets the temperature drift above setpoint while the PI command grows, and the staging rule, units = ceil(command / (0.8 q[unit])), turns that command into more units and more fan power, p = q[c] / COP + units x p[unit]. So a lower cap never saves energy, and the lever stays native. (The setpoint, which changes the COP, and the units, which change fan power at the same heat, remain levers.)
- *A battery's reserve.* Energy D drawn from the cells below the operator's reserve saves D sqrt(eta) of grid import, and the deficit is bought back at D / sqrt(eta). The net, D (1 - eta) / sqrt(eta) (10.5% of D at eta = 0.90), is always positive. Spending the reserve is never an energy saving; it is the price of keeping the draw under the connection's limit. So the reserve is spent only at the wall (p >= 0.95) and only when the battery can cover the excess (0 < imp - p[lim] <= p[batt]). Otherwise the operator's reserve holds, and no round trip is paid for nothing.
- *A backup reserve (a UPS).* It is held for an outage, which the model does not contain, so spending it for a peak trades away the reason it exists. Never a lever.
- *A compute pool's power cap.* With throughput mu(c) proportional to c^eps and the autoscaler holding utilisation near its target u*, the energy per unit of work is e(c) = (p[idle] / u* + p[dyn] c) / mu(c). At c = 1, de/dc > 0 (a lower cap saves) if and only if

```
eps < p_dyn / (p_idle / u* + p_dyn)
  ↪                                  (9.8)
```

At eps = 0.4 the cap saves on CPU hosts (threshold 0.62), GPUs (0.74), databases (0.51) and radio units (0.54). It costs energy where idle power dominates: a quantum computer's cryostat (0.19), a network switch (0.32), storage (0.26). There, running slower keeps the machines on longer: race to idle. In a deployment p[idle], p[dyn] and the speed curve come from the machine's own specification.

- *A motion axis's effort cap.* It lowers the copper loss of a full-acceleration move, (J a[max] / kt)^2 R, and stretches the move, which keeps paying the standing draw p[idle] + b v[max]^2. It moves only where the first exceeds the second: a reaction wheel (32 W against 5 W), not EV traction, a flight axis or a robot joint.
- *A machine released from a node pool.* A machine boots in minutes, and a burst that arrives in the meantime is served late. So a machine goes back only when the machines left cover the recent peak at 0.3 of the pool's own release level, the largest margin at which no pool is later than native (`results/realms/RELEASE_MARGIN_SWEEP.md`). This is the verdict of the live controller (section 21.9) in model form.

What a lever buys is labelled for what it is. Less work per energy, with no work lost and the time over the line proven lower, is SERVICE IMPROVEMENT WITH ENERGY TRADEOFF, the mirror of ENERGY IMPROVEMENT WITH SERVICE TRADEOFF. Without that proof it reads LESS WORK PER ENERGY.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 22

The Safety Case: Every Hazard, Its Guard, and the Evidence

Omni-Compass writes settings that a native controller would otherwise own. A referee's first question is what happens when it is wrong, is attacked, or dies. This page answers it hazard by hazard. Each row names the guard in the code that stands against the hazard and the evidence that the guard holds: a test that runs on every push (`python3 verify.py`), or a measured run (evidence class L). Where no evidence exists yet, the row says so. The page adds no mechanism; it maps the mechanisms that are already there. Section 22.3 is the plug's interface, as the code defines it.

22.1 The claim this page supports

Do no harm (its meaning, `DISCLOSURES.md` section 3): the organism Omni-Compass governs is never left turning out less than its native controllers make it turn out, nor less healthy (no muscle past its own limit, no service over its own line, nothing made to fail). On the engine that stands, that means: Omni-Compass never leaves a setting other than native's after it stops, never acts on a reading it cannot trust, never takes a step that a paired trial on the system itself showed costs more, never exceeds a bound the system or its operator set, and can be switched off by one human action, everywhere at once.

22.2 Hazards, guards and evidence

#	Hazard	Guard (where it lives)	Evidence that it holds
H1	The governor dies, hangs or loses its machine, leaving a setting changed	Before its first write, every governor records the exact commands that put every setting back, and renews a lease every decision. A watchdog finds any governor whose process is gone or whose lease is stale and runs those commands for it (`omnicompass/master.py`: `register`, `heartbeat`,	`tests/test_master_switch.py`; the robustness kill test (the governor killed outright mid-run, every setting back within the allowance in every repetition; `docs/ROBUSTNESS_PREREGISTRATION.md`); `tests/test_robust.py`, `tests/test_robust_restart.py`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Hazard	Guard (where it lives)	Evidence that it holds
		`orphans`; `tools/omni_switch.py`: `watchdog`, `reclaim`)	
H2	Someone takes over the governor, or it behaves in a way nobody intended	The kill switch: one file, one command. Every governor sees it at its next decision, writes back every setting it changed, reads each one back and exits. A governor that cannot read the switch treats that as OFF (`omnicompass/master.py`, `tools/omni_switch.py off`)	`tests/test_master_switch.py`
H3	A write is refused or silently fails, so the governor's belief and the system differ	Every write goes through the plug, which reads the value back (`omnicompass/muscle_sdk.py`: `execute`, then `readback`, then a receipt). At the end of every run the reset is read back against the snapshot, and the run fails if any record is left (`scripts/kind_bench.sh`)	`tests/test_muscle_sdk.py`; every three-run table prints "handed back to native's and read back at the end of every omni arm in every run: yes"
H4	It gives back too much, so service breaks when load returns	Floors read live: never below one replica, a pool never below two machines. A machine is given back only after its own trial shows the response time holds, and it wakes at once on a breach (`omni_controller/controller.py`)	`tests/test_node_release_gate.py`; the fault test (`results/live/V7_FAULTS.md`)
H5	On a shared host, one tenant's raise starves the others	No CPU limit is raised while the host is 75% busy or its reading is missing; conveyance stays inside each namespace's CPU quota, and an unreadable quota means no known budget, so	`tests/test_capacity.py`, `tests/test_kind_capacity.py`, `tests/test_quota_convey.py`; the noisy-neighbour test (`results/live/V7_FAIRNESS.md`,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Hazard	Guard (where it lives)	Evidence that it holds
		Kubernetes admission decides (`omni_controller/capacity.py`, `omni_controller/quota.py`)	STEADY)
H6	It acts on a reading it cannot trust	A reading with no machine in service is blind: the decision holds, says why, and writes nothing (omni-v11, `omni_controller/controller.py`). A muscle whose reading is stale reports BLIND, and no command is executed (`omnicompass/muscle_sdk.py`: `MuscleStatus`)	`tests/test_blind_reading.py` (the fault reproduced on omni-v10, gone on omni-v11); `results/model_gate/V11_VS_V10.md`; the extended fault test, the API server frozen and the metrics server gone (queued, A, B and C: `docs/K8S_COMPASS_PREREGISTRATION.md`)
H7	It keeps a step that hurts the system	One trial at a time, on the system itself: a reference phase at the deepest step already allowed, then one notch further. The step is refused if its median cost exceeds the reference's by more than 2%, or the cost first measured at the operator's setting. Refused steps wait 60 s; a trial is abandoned if the service swings (`tools/knob_verdict.py`)	`tests/test_knob_verdict.py`, `tests/test_verdict.py`, `tests/test_verdict_interval.py`; every database table prints each trial's count, allowance and refusal
H8	It writes outside what the operator allows	A capability contract (minimum, maximum, resolution, step and rate limits) and a safety contract (hard bounds, forbidden contexts, fail closed) on every muscle (`omnicompass/muscle_sdk.py`: `CapabilityContract`, `SafetyContract`)	`tests/test_muscle_sdk.py`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Hazard	Guard (where it lives)	Evidence that it holds
H9	It uses more privilege than its job needs	Least-privilege service account (`deploy/kind/rbac-omni.yaml`), and every arm records what it can and cannot do (`kubectl auth can-i`, `rbac_omni.txt` in each run's folder)	The receipts in every run folder on the branch `raw`
H10	It expands while a security hold is on	The shield: no expansion during a security hold (invariant I1), with a C++ twin checked against the Python (`omnicompass/shield.py`, `cpp/`)	`tests/test_shield_properties.py`, `tests/test_cpp_shield_parity.py`, `tests/test_cpp_shield_adversarial.py`
H11	In the cloud, it raises the bill for the same work	The bill is judged, never ruled (the founder's order of 31 October struck the machines-alone rule of 14 October): every cloud set reads the bill beside the work it bought by the one key, the cloud's judged number the work per dollar: a factory tune whose work per dollar falls against native (a bigger bill for the same work) is not admissible (`docs/THE_TUNE.md`) and is a fault of the tune, root-caused in the model first; the machine lever asks the cloud for no machine (`scripts/kind_nodepool.sh`); paid sets capped at 100 repetitions and 1, 10 and 100 copies	Azure: no difference in the bill on the engine of 13 October, at the tune-down on the machines alone (`results/live/AKS_BILL.md`); the sets at the max carry the work meter from 31 October, night, a tuning case first, so each reads the work per dollar. Google and Amazon: **undetermined**, not yet run.
H12	A test machine's limit is reported as the governor's effect	Paired arms on the same runner, rotated order, the refusal rules for an unhealthy window, and the kind rig's image pre-puller (`docs/POD_START_WAIT.m`	`tests/test_confirm_abc.py`, `tests/test_pod_phases.py`, `tests/test_release_campaign.py`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Hazard	Guard (where it lives)	Evidence that it holds
		d). `tools/confirm_abc.py` marks any table whose runs differ in engine or rig	
H13	The hand-back itself is turned against the system: a record another local user planted makes the watchdog run a command or stop a process, or a governor's pid given to another process gets that process stopped	The watchdog acts only on records of its own user that no other user can write, read without following a link; a pid counts as the governor's only while its process started before the governor registered (`tools/omni_switch.py`, 18 October). Every threat of misuse is set out in `docs/THREAT_MODEL.md`	`tests/test_switch_trust.py` (each fault reproduced on the tool as it stood, gone on the fix)

22.3 The plug, as the code defines it

Every muscle is reached through one interface (`omnicompass/muscle_sdk.py`, `MuscleAdapter`). The sequence is fixed: observe, bound, request, execute, read back, receipt, restore.

Call	What it must do
`observe()`	Return the muscle's reading with its timestamp; a reading older than its timing contract makes the muscle BLIND
`snapshot_native()`	Capture native's setting once, before the first write; this is what every restore returns to
`execute(value)`	Write one value, already clamped by the capability contract and permitted by the safety contract
`readback()`	Read the value the system actually holds; a mismatch is recorded, never assumed away
`restore_native(snapshot)`	Put native's setting back, then read it back

Each muscle carries three contracts: timing (observation and decision periods, actuation delay, the age after which a reading is stale, the minimum hold and cool-down between writes), capability (range, resolution, step and rate limits, reversible, read back) and safety (hard bounds, forbidden contexts, fail closed, a safe default). A muscle built on the SDK cannot be declared without all three. It starts in recommend mode and writes only

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

when set to automatic or approved from outside; every command, written or refused, leaves a receipt with the reason, the reading's age, the value asked, the value bounded, the value the system holds and the difference between the last two (`MuscleRuntime.command`).

22.4 What this case does not yet cover

- **No outside security review or penetration test.** H2, H9, H10 and H13 rest on our own tests; the threats still open are listed in `docs/THREAT_MODEL.md`, section 3.
- **No formal proof of the whole controller.** The theorems in `docs/TRACKING_THEOREM.md` hold for the compass law's channel; global stability of the full six-state system is open (`docs/FORMAL_STATUS.md`).
- **No production deployment.** Every hazard above is exercised on our own rigs (kind on GitHub runners, Azure's managed Kubernetes, the database servers in our workflows), not on a customer's system.
- **Physical plants.** The GPU's power and clock guards and every facility muscle are exercised only in our models or a stand-in card until the chip runs exist (undetermined).

CHAPTER 23

The Threat Model: Misuse, Its Guards, and What Is Open

The safety case (`docs/SAFETY_CASE.md`) asks what happens when the governor is wrong or dies. This page asks what happens when someone wants it to do harm. It names what is worth protecting, where trust changes hands, each threat with the guard in the code that stands against it and the test that shows the guard holds, and what is still open. It is our own analysis: **no outside security review or penetration test has been made**, and nothing here stands in for one.

23.1 What is worth protecting

Asset	Why it matters
The settings the governor writes (replicas, the autoscaler's target, a node's idle mark, a CPU limit, a power cap, a database pool)	Whoever controls them controls the service's capacity
The governor's identity on the system (on Kubernetes its service account and token)	It carries every right the governor has

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Asset	Why it matters
The hand-back: the snapshot of native's settings, the registry of restore commands, the switch file	They put the system back when the governor stops; whoever can write them chooses what "back" means
The engine's code (the 65 fingerprinted files of omni-v23, the engine that stands) and the harness that measures it	A changed engine is a different product; a changed harness is a different result
The audit lines and the raw receipts	They are the evidence for every result and every decision
The cloud and GPU credentials (GitHub secrets)	They spend money and create machines

23.2 Where trust changes hands

1. **The operator and the governor.** The operator starts the governor, grants its rights and sets its bounds. The governor is trusted to act only inside them.
2. **The governor's host.** The governor, the switch and the watchdog run as processes of one user on one machine (a pod, a runner or a server). Every other local user is outside that boundary.
3. **The governor and the system it governs.** It reads through the system's API (Kubernetes, `nvidia-smi`, a database's own commands) and writes through the same API with a least-privilege identity. The readings are inputs it does not control.
4. **The repository and the machines that run it.** Code reaches a run through GitHub: a pushed commit, an action, a Python package. Secrets reach a run only through GitHub's secret store.

23.3 Threats, guards and evidence

#	Threat	Guard (where it lives)	Evidence	Still open
T1	Another local user plants a record in the watchdog's registry, so the watchdog runs that user's command, or stops a process, with its own rights	The watchdog acts only on a record that it and the registry directory are owned by its own user (or root, or a uid named with `--trust-uid`), that is no symbolic link and that no other user can write. Each record is opened inside the	`tests/test_switch_trust.py` (a record anyone can write, a registry anyone can write, a record reached through a link, a record of another user: each refused; each fault reproduced on the tool as it stood)	A member of the governors' own group can still write a record the governors' umask leaves group-writable

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Threat	Guard (where it lives)	Evidence	Still open
		directory without following a link and checked on the open file, so nothing can be swapped in between. Any other is refused, named, and left in place (`tools/omni_switch.py`: `trusted_records`, 18 October)		
T2	A governor dies, the system gives its process number to another process, and the watchdog takes that process for the governor, hung, and kills it	A process number is taken for the governor's only while the process holding it started before the governor registered (`governor_alive`, from `/proc`); otherwise the governor is gone, nothing is signalled, and its settings are handed back (18 October)	`tests/test_switch_trust.py` (the watchdog as it stood killed the process holding the number; now it hands back and the process lives); `tests/test_master_switch.py` (a hung governor is still stopped and handed back)	Systems without `/proc` keep the old reading
T3	A link planted at the switch file's path makes `off` overwrite another file	The switch file is opened without following a link; if it cannot be written, every governor is still signalled and the tool says so (`tools/omni_switch.py off`, 18 October)	`tests/test_switch_trust.py` (the file the link points at is left as it was)	—
T4	Anyone who can write the switch's directory, or a governor's own kill file, turns governing	Off is the safe direction: every setting goes back to native's (the kill switch's	`tests/test_switch_trust.py` (the warning)	The default directory is `/tmp/omni-compass`, written in `omnicompass/master.py`,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Threat	Guard (where it lives)	Evidence	Still open
	off, or keeps it off	purpose). `status` warns when the switch's or the registry's directory can be written by another user		an engine file; until the next engine version moves it, set `OMNI_MASTER_OFF` and `OMNI_REGISTRY` to a directory only the governors' user can write. The cloud install keeps it inside the pod, where every container runs as one user (10001)
T5	The governor's identity is stolen	Least privilege: on Kubernetes the service account can read nodes, pods, metrics, autoscalers and workloads cluster-wide; patch nodes and evict pods (the eviction API honours disruption budgets); and, in the service's namespace, patch the one named autoscaler and Deployment, pods (their deletion cost and in-place resize) and Jobs, and read one named ConfigMap. It cannot delete an object directly, create a workload or read a Secret (`deploy/kind/rbac-omni.yaml`). Every arm records what the identity can	The receipts in every run folder on the branch `raw`	The rights it has are enough to cordon and drain nodes; whoever holds the token is limited by the role, not by the governor's floors

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Threat	Guard (where it lives)	Evidence	Still open
		and cannot do (`kubectl auth can-i`, `rbac_omni.txt`)		
T6	Someone feeds the governor false readings (a lying metric, a stalled probe)	A reading with no machine in service holds the decision and writes nothing (omni-v11); a stale reading makes a muscle BLIND (`omnicompass/muscle_sdk.py`); every step is a paired trial on the system itself, refused if its cost rises more than 2% (`tools/knob_verdict.py`, `omnicompass/verdict.py`); every write stays inside the capability and safety contracts	`tests/test_blind_reading.py`, `tests/test_muscle_sdk.py`, `tests/test_knob_verdict.py`; the fault test's blind probe; the extended fault test's frozen API server and missing metrics server (queued)	A false reading that stays consistent and inside the bounds can steer the governor within the room the operator granted
T7	The engine's code is changed	The engine is fingerprinted: `OMNI_V16.json` names the digest of its 55 files, and `python3 tools/omni_version.py` names the engine a checkout carries, or what differs from every fingerprint. `python3 verify.py` holds every place that names the engine to the engine	`tests/` (the release manifest and the version tool); every run's receipt names the engine its commit carries	Releases are not signed artefacts, the repository's bots commit unsigned, and nothing checks the fingerprint when a governor starts

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Threat	Guard (where it lives)	Evidence	Still open
		that stands (`tools/land_engine.py --check`)		
T8	A dependency is replaced (a Python package, a GitHub action)	Every package pip installs is pinned to an exact version with the SHA-256 of every file PyPI publishes for it (`requirements-lock.txt`, hash-checking mode), with a software bill of materials (`sbom/omni-compass.cdx.json`, `sbom/omni-compass.spdx.json`, written by `tools/sbom.py`); every action a workflow uses is named by its commit, its tag beside it (407 uses, 19 October); `security-scan.yml` audits every pin against the public vulnerability databases each week; Dependabot watches both the packages and the actions (`.github/dependabot.yml`)	`tests/test_supply_chain.py` fails a push with an action named by tag or a package without hashes; `tools/sbom.py --check` in `verify.py`	Closed for what Omni-Compass installs; the tools a benchmark machine installs from its own package manager (kind, kubectl, the databases) are pinned by version, not by hash
T9	A secret leaks from a workflow	Secrets reach only workflows started on this repository (a push or a dispatch); no workflow runs on `pull_request_target`	Every workflow names its token's rights; `tests/test_cloud_check.py` fails a push that leaves any Azure login without the shape check	—

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Threat	Guard (where it lives)	Evidence	Still open
		, so no fork's code ever runs with them. Secrets are kept in GitHub's store only, and GitHub masks them in every log. A key stored in another form is named by its shape before any cloud's login step reads it, never a character of it printed (`tools/cloud_key_check.py`), because a login step's own refusal can quote the first characters of what it was given, and a piece of a secret is not masked (21 October: Azure's did, on a public run page)	before it; the 28 that only build, run and upload get `contents: read` and nothing else (18 October). `tests/test_workflow_permissions.py` fails a push that leaves any workflow on the repository's default, or gives a read-only token to a workflow that pushes, starts a run or reads another run's files	
T10	A run spends money it should not	In a cloud the whole engine runs at the max (the founder's order of 31 October) on top of the cloud's own autoscaler, which alone adds machines; the governor's lever parks a machine that exists and wakes it, and asks the cloud for none (`scripts/kind_nodepool.sh`); the bill is judged beside the work it bought, a tune whose work per dollar falls against native not	`results/live/AKS_BILL.md`; `tests/test_cloud_check.py`	The credentials in the secrets can create clusters; their scope is set in each cloud's console

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Threat	Guard (where it lives)	Evidence	Still open
		admissible (`docs/THE_TUNE.md`), and paid sets stay at 100 repetitions and 1, 10 and 100 copies. A cloud's set starts only after that cloud's $0 check passes (Google, Amazon and, since 22 October, Azure: the key, the login, the plan, the machine type, every quota; `tools/cloud_key_check.py`). Nothing is rented for the grid		
T11	The evidence is changed after the run	Every archived run is kept byte for byte on the branch `raw` with its SHA-256 sums (`scripts/raw_archive.sh`); every table names its runs and is rebuilt by its own tool from them	`tests/test_raw_archive.py`	Audit lines are not signed when written; the sums protect them from the moment they are archived
T12	The system's control plane is taken away (the API server frozen, the metrics server gone)	The governor holds on a reading it cannot make; a failed decision writes nothing; its lease runs 300 s at a 60-second decision, so a short outage does not hand back what is working	The extended fault test (queued, A, B and C: `docs/K8S_COMPASS_PREREGISTRATION.md`)	Whatever the runs read is reported

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

23.4 What would close the open rows

- An outside security review and a penetration test of the governor, the switch and the install.
- In the next engine version, the switch and the registry in a private directory by default, created with no access for any other user (T1, T4).
- Release artefacts signed, and the fingerprint checked when a governor starts (T7).

CHAPTER 24

The Static Security Scan, Every Finding Reviewed

Generated by `python3 tools/security_scan.py` (Bandit, the Python security linter of PyCQA) over every Python file that runs (`omni_controller/`, `omnicompass/`, `realms/`, `tools/`, `scripts/`, `fleet/`, `k8s_controlplane/`; 50,895 lines). The workflow `security-scan.yml` runs it on every push to main that touches code, and every week, with `pip-audit` over every pinned dependency; it fails on a finding of high or medium severity that no review below names.

2 findings of high severity and 39 of medium severity, every one reviewed below; 410 of low severity, counted by kind. Fixed on 19 October: a world-writable PgBouncer folder (now owned by postgres, 0o755) and the MD5 that PostgreSQL's own password method defines, now marked as not used for security. Fixed in omni-v14 (21 October): the operator's own commands (the site power reading, a muscle's read command, a wall meter's) run without a shell unless they hold a pipe, a redirect or a variable, so the governor's site power reading no longer reaches a shell at all and one high finding is gone.

24.1 Every finding of high or medium severity, and why it stands

Check	File	Severity	Findings (lines)	Why it stands
B103 (Chmod setting a permissive mask 0o755 on file (dir))	`tools/run_pgbench.py`	medium	1 (390)	PgBouncer's folder on the benchmark machine is owned by postgres and set 0o755: others may read and enter it, never write it

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Check	File	Severity	Findings (lines)	Why it stands
B108 (Probable insecure usage of temp file/directory)	`k8s_controlplane/mechanism_harness.py`	medium	1 (166)	a path on the benchmark's own throwaway machine (a GitHub runner), never on a machine Omni-Compass governs
B108 (Probable insecure usage of temp file/directory)	`omni_controller/controller.py`	medium	1 (1584)	a default file name for the audit log in the working folder, not /tmp; the operator sets --audit
B108 (Probable insecure usage of temp file/directory)	`omni_controller/gpu_compass.py`	medium	1 (394)	a default file name for the audit log, as above
B108 (Probable insecure usage of temp file/directory)	`omni_controller/gpu_governor.py`	medium	1 (475)	a default file name for the audit log, as above
B108 (Probable insecure usage of temp file/directory)	`omnicompass/master.py`	medium	1 (44)	the switch's registry folder under the operator's runtime folder; the watchdog trusts only its own registry (docs/THREAT_MODEL.md, T4)
B108 (Probable insecure usage of temp file/directory)	`scripts/dispatch_grid.py`	medium	1 (429)	a path on the benchmark's own throwaway machine (a GitHub runner), never on a machine Omni-Compass governs
B108 (Probable insecure usage of temp file/directory)	`tools/gpu_actuator_watchdog.py`	medium	1 (65)	a path on the benchmark's own throwaway machine (a GitHub runner), never on a machine Omni-Compass governs
B108 (Probable insecure usage of temp file/directory)	`tools/omni_control.py`	medium	1 (550)	the control panel's default kill file, under /tmp/omni-compass on the operator's own machine,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Check	File	Severity	Findings (lines)	Why it stands
				where the panel and the governor it starts run as one user; the operator sets it with --kill-file or OMNI_CONTROL_KILL_FILE, and the governor reads only the file the panel hands it (docs/CONTROL_PLANE.md)
B108 (Probable insecure usage of temp file/directory)	`tools/run_kafka.py`	medium	4 (800, 800, 805, 807)	a path on the benchmark's own throwaway machine (a GitHub runner), never on a machine Omni-Compass governs
B108 (Probable insecure usage of temp file/directory)	`tools/run_redis.py`	medium	2 (612, 614)	a path on the benchmark's own throwaway machine (a GitHub runner), never on a machine Omni-Compass governs
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`omni_controller/controller.py`	medium	1 (174)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the operator's metrics endpoint)
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`scripts/aks_kubectl.py`	medium	1 (33)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`scripts/dispatch_grid.py`	medium	1 (393)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the GitHub API)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Check	File	Severity	Findings (lines)	Why it stands
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`scripts/latency_probe.py`	medium	1 (17)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the service under test)
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`scripts/restart_aks.py`	medium	1 (38)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the GitHub API)
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`tools/artifact_legal.py`	medium	1 (62)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (raw.githubusercontent.com)
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`tools/health_watch.py`	medium	1 (46)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the GitHub API)
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`tools/llm_workload.py`	medium	1 (35)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the model server under test)
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`tools/omni_alert_bridge.py`	medium	1 (76)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the operator's own control panel, --panel)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Check	File	Severity	Findings (lines)	Why it stands
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`tools/orchestrator_show.py`	medium	1 (89)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the showing's own agents on 127.0.0.1, their ports written in the code)
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`tools/prom_feed.py`	medium	1 (53)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the operator's Prometheus)
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`tools/run_ycsb.py`	medium	1 (758)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the YCSB release, checked against its published digest)
B310 (Audit url open for permitted schemes. Allowing use of file:/ or custom schemes is often unexpected)	`tools/trace_schedule.py`	medium	1 (61)	every URL it opens is an https address written in the code or the operator's own endpoint, never one read from data (the public trace)
B323 (By default, Python will create a secure, verified ssl context for use in such classes as HTTPSConnection. However, it still allows using an insecure context via the _create_unverified_context that reverts to the previous behavior that does not validate certificates or	`tools/wall_meter.py`	medium	1 (48)	an unverified TLS context only when the operator sets REDFISH_INSECURE=1 for a management controller with its own self-signed certificate

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Check	File	Severity	Findings (lines)	Why it stands
perform hostname checks)				
B506 (Use of unsafe yaml load. Allows instantiation of arbitrary objects. Consider yaml.safe_load())	`scripts/restart_aks.py`	medium	1 (84)	yaml.load with BaseLoader, which builds strings only (stricter than SafeLoader), on the repository's own workflow files
B602 (subprocess call with shell=True identified, security issue)	`omni_controller/muscles.py`	high	1 (121)	the command is the operator's own configuration, run with the governor's own rights; it never holds text read from the cluster or the network, and an operator may use a pipe in it (run_operator_cmd: the site power reading, --power-cmd, and a muscle's read command in the operator's wiring file; from omni-v14 a command with nothing only a shell reads runs without one, so the shell is reached only for a pipe, a redirect or a variable the operator wrote)
B602 (subprocess call with shell=True identified, security issue)	`tools/wall_meter.py`	high	1 (79)	the command is the operator's own configuration, run with the governor's own rights; it never holds text read from the cluster or the network, and an operator may use a pipe in it (a wall meter read by a command, `cmd:` in the meter's address; without a shell unless it holds a pipe, a redirect or a variable, as the governor's commands)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Check	File	Severity	Findings (lines)	Why it stands
B608 (Possible SQL injection vector through string-based query construction)	`tools/run_pgbench.py`	medium	2 (892, 895)	SQL built from the benchmark's own fixed names to create its throwaway database, never from input
B608 (Possible SQL injection vector through string-based query construction)	`tools/run_sysbench.py`	medium	1 (175)	SQL built from the benchmark's own account name (SYSBENCH_USER, set by its own workflow) and its fixed statement names, read on the benchmark's throwaway database; never text read from the network
B610 (Use of extra potential SQL attack vector)	`tools/live_reps.py`	medium	6 (903, 903, 904, 905, 905, 905)	not Django: a local function named extra() inside an f-string

24.2 Findings of low severity, by kind

Kind	Findings
B603 subprocess_without_shell_equals_true	175
B404 blacklist	85
B607 start_process_with_partial_path	68
B101 assert_used	52
B311 blacklist	17
B110 try_except_pass	6
B105 hardcoded_password_string	3
B606 start_process_with_no_shell	2
B107 hardcoded_password_default	1
B112 try_except_continue	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The low-severity kinds are the patterns of a harness that drives other programs: a subprocess started with a list of arguments and no shell (B603, B607), the subprocess module imported (B404), an assert in a test helper (B101), a pseudo-random number for a modelled load (B311), and the throwaway passwords of the benchmark's own local databases (B105, B107).

CHAPTER 25

Every Dependency and Its License

Generated by `python3 tools/sbom.py` from `requirements-lock.txt` (Omni-Compass 0.23.0); `tools/sbom.py --check`, run by `verify.py`, fails a push where this page or the bill of materials (`sbom/omni-compass.cdx.json`, CycloneDX 1.5; `sbom/omni-compass.spdx.json`, SPDX 2.3) is not what the lock gives, where a pinned package has no license read, or where a license is strong copyleft. Every package is pinned with the SHA-256 of every file PyPI publishes for its version, and `security-scan.yml` audits every pin against the public vulnerability databases each week.

29 packages; every license permissive except one weak, file-level copyleft (MPL-2.0, Hypothesis, used by the tests alone and never shipped with the engine); no GPL, LGPL or AGPL. Each package keeps its own license; nothing in the Omni-Compass Evaluation License changes those terms (`THIRD_PARTY_NOTICES.md`).

Package	Version	License (SPDX)	Its use in Omni-Compass
numpy	2.5.3	BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0	numerical arrays
matplotlib	3.11.2	LicenseRef-Matplotlib	charts in the reports (the Matplotlib License, PSF-based)
reportlab	4.5.1	BSD-3-Clause	the printed manuals and reports
PyYAML	6.0.3	MIT	reading workflow and configuration files
numba	0.68.0	BSD-2-Clause	compiled inner loops of the motion-axis plant
llvmlite	0.50.0	BSD-2-Clause AND Apache-2.0 WITH LLVM-exception	numba's compiler
mujoco	3.15.0	Apache-2.0	the robot-arm benchmark (MuJoCo Menagerie)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Package	Version	License (SPDX)	Its use in Omni-Compass
z3-solver	5.3.1.0	MIT	the machine-checked proofs (tools/prove_engine.py)
mpmath	1.4.1	BSD-3-Clause	interval arithmetic in the machine-checked proofs
hypothesis	6.170.2	MPL-2.0	the property tests (tests only, never shipped with the engine)
sortedcontainers	2.4.0	Apache-2.0	brought by hypothesis
absl-py	2.5.0	Apache-2.0	brought by mujoco
etils	1.14.0	Apache-2.0	brought by mujoco
glfw	2.10.2	MIT	brought by mujoco
contourpy	1.4.0	BSD-3-Clause	brought by matplotlib
cycler	0.12.1	BSD-3-Clause	brought by matplotlib
fonttools	4.66.1	MIT	brought by matplotlib
kiwisolver	1.5.1	BSD-3-Clause	brought by matplotlib
packaging	26.3	Apache-2.0 OR BSD-2-Clause	brought by matplotlib
pillow	13.0.0	MIT-CMU	brought by matplotlib and reportlab
pyparsing	3.3.3	MIT	brought by matplotlib
python-dateutil	2.9.0.post0	Apache-2.0 AND BSD-3-Clause	brought by matplotlib
six	1.17.0	MIT	brought by python-dateutil
PyOpenGL	3.1.10	BSD-3-Clause	brought by mujoco
fsspec	2026.9.0	BSD-3-Clause	brought by mujoco
typing_extensions	4.16.0	PSF-2.0	brought by mujoco
zipp	4.1.1	MIT	brought by mujoco
charset-normalizer	3.5.2	MIT	brought by reportlab
websockets	17.2	BSD-3-Clause	brought by mujoco

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 26

The Engine That Stands: Omni v23

Omni v23 (`OMNI_V23.json`, digest `c96ea4105d8747db`, 65 files) is omni-v22 with one guard in the Kubernetes governor's pod muscle of its own (`omni_controller/controller.py`, `_own_pods` and `_own_refused`), on the founder's order of 31 October, given on reading that the arm by itself had read like native:

> "Omni direct is directly to muscles, but Omni direct still might need an autoscaler like Kubernetes ... we still might need a muscle or two we're using. We're going straight into all the muscles as the brain and orchestrator, scheduler and all that, but what Omni-Compass can't do and needs the muscle for has to be achieved even in direct, so that it can work and do everything else ... Don't just embed some shit and fail the whole system because we don't have the muscle to do something ... any muscle that we don't have, Omni-Compass cannot be it: it has to be there, if that's Kubernetes or whatever."

64 of its 65 files are omni-v22's bytes. No gain, preset, card or law moves; do no harm keeps its strictest form.

What "direct" can and cannot mean (the founder's analysis, the same day: "Omni-Compass would have to become the Kubernetes mechanism to be what's considered direct"). On a Kubernetes stack Omni-Compass is a brain with no organs: Kubernetes' scheduler places every pod and the kubelet runs every container, in every arm. The arm by itself there is Omni-Compass's own pod rule in place of the autoscaler's arithmetic and nothing else, so it is never called direct; it is a mechanism test kept in the record, not a product arm, and the product has two sides, native and Omni-Compass on native (the founder's order of 25 October). The one test where the word holds is the orchestrator showing without Kubernetes (`docs/OMNI_ORCHESTRATOR.md`), where Omni-Compass's own small orchestrator is the organs; its three arms read, run by run: Omni-Compass on native the strongest in all three (the most work, the fastest response in two, fewer machines in all), native the weakest, the arm by itself between them (level with Omni-Compass on native in run A, behind it in B and C). This guard serves every arm: wherever Omni-Compass's own rule cannot move a knob, the organ that can gets it back.

26.1 What was found

Every arm by itself run on kind from omni-v19 (24 October) to omni-v22 had the governor's pod writes refused by the rig's own identity: `kubectl scale` patches the scale subresource, `deployments/scale`, a resource of its own under Kubernetes' RBAC, and the rig's role granted `patch` on `deployments` by name

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

and never the subresource. Fifteen of fifteen decisions a window ended Forbidden, in every repetition; the workload stayed at the operator's count; the machine and conveyance muscles wrote as always, so the window's no-write check never fired. omni-v21's three runs of the entry were admitted and are read in no table; omni-v22's two were marked invalid by a second rig fault. Both rig faults are fixed (`docs/SOLO_PREREGISTRATION.md`, amendment 1). What the engine lacked is this version: when Omni-Compass cannot be a knob's muscle, the knob must go back to the muscle that can, so the organism never runs with a knob nobody moves (amendment 2, the founder's order above).

26.2 What changes in the engine

- A write of a workload's own scale that the cluster refuses (its permission, its admission, its API) is recorded in the audit (`own_replicas`, `refused` n, the cluster's words) and the decision goes on; the cluster keeps the last count that landed. Before omni-v23 the exception ended the decision (`decision failed (n in a row)`) and nothing else happened, decision after decision.
- At `OWN_REFUSALS` (3) refusals in a row Omni-Compass cannot be that knob's muscle: it makes native's own pod autoscaler from the operator's own target and range recorded on the workload (`kubectl autoscale deployment NAME --cpu-percent=T --min=lo --max=hi`; with no range recorded, from one to ten times the operator's count), audits the hand-back (`handed_back`, `after_refusals`, `target`, `range`), and leaves the workload to it for the rest of the run: the snapshot's governed set holds it from the next decision on, and the reset leaves native's autoscaler standing. A hand-back the cluster refuses too is audited (`hand_back_refused`) and tried at the next refusal.
- A write that lands ends the run of refusals.
- Every other arm is untouched: Omni-Compass on native never removes native's controller, so a refused write there leaves native's own controller moving the knob as before; the machines, the conveyance, the power cap and the batch muscles are omni-v22's bytes.

26.3 What changes in the rig, not the engine

The kind identity (`deploy/kind/rbac-omni.yaml`) gains `create` on `horizontalpodautoscalers` in the default namespace, for the hand-back alone, and the preflight proves it; a window of the arm by itself in which the knob was handed back is refused as a result of that arm (the organism was kept whole, but Omni-Compass did not run the arm by itself): `INVALID RUN: by itself, the knob ... was handed back to native's own autoscaler`. `tests/test_kind_rig_rbac.py` fails a rig without either.

26.4 The model gate

`results/model_gate/V23_VS_V22.md` (class S, our own models; `tools/model_gate.py`, seeds 1 to 8, the engine that stood, omni-v22, the candidate omni-v23): every one of the 95 gauge rows of the 11

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

models reads identical on every seed, the counts identical, no LOSS. No model refuses a write of the cluster, so the guard never fires in the models and the candidate reads identical to omni-v22 on every gauge; its proof is the tests below and the arm's own files when the entry runs again.

26.5 The tests

`tests/test_own_replicas.py`, `HandBack`: a cluster that refuses the scale write three times in a row gets native's own autoscaler at the operator's 50% target and 1 to 10 range, the muscle writes nothing more to that workload, the count the cluster kept stands (never a knob nobody moves); a write that lands after two refusals ends the run of refusals and no autoscaler is made; with no range recorded the autoscaler's ceiling is ten times the operator's count. The unit-test suite, the function tests and the release check pass on the tree; no C++ twin changes (the Kubernetes governor has none).

26.6 What is run again

Every benchmark runs again on omni-v23 from 20:00 UTC on 31 October 2026 (`scripts/reruns.txt`, A of every entry, then B, then C; the conductor's campaign, `scripts/dispatch_grid.py`). The omni-v22 runs started before that moment run to their end as omni-v22's record; nothing is cancelled. The entry `kubernetes by itself` runs on the rig as amended, three arms, its table kept in the record and not carried on the front page (the founder's order of 25 October).

CHAPTER 27

The Engines and Their Audit

27.1 Which engine is which

There is **one** engine. Every other engine-looking file in this repository is either its required C++ twin or a frozen copy of where it came from. Nothing else runs.

File	Role	Runs?
`omnicompass/core.py`	**The engine** (`symmetric_verified`): six states, eight lines, RK4 with microsteps. Everything Omni does goes through it.	yes
`cpp/src/core.cpp`, `cpp/include/omnicompass/core.hpp`	The same engine in C++. Required: `verify.py` checks it against the Python on every fixture.	yes (twin)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

File	Role	Runs?
`docs/handoff/mathematics/omni_compass_engine_source_c527df2d.py`	The original source as handed over (sha256 `c527df2d…`). Kept unchanged as proof of origin and as the parameter authority.	no (frozen)
`reference/omni_compass_reference_engine.py`	The same original with comments stripped (`tools/strip_reference.py`); same program fingerprint (`reference/PROVENANCE.json`).	no (frozen)

Fingerprints of the running engine, its parts and its twin: `results/MECHANISM_IDENTITY.json` and `RELEASE_MANIFEST.json`. `verify.py` fails if any of them changes without a new seal.

27.1.1 Engines from outside packages, not adopted

Copies of this repository passed around as zips (the XPASS packages) carry changes that are **not** in the engine here:

- `batch_claim1.py` (in those zips, not in this tree): a vectorised variant that applies u during the RK4 step (CLAIM1). No C++ twin; not verified against the 500 fixtures.
- an adapter that defaults to CLAIM1, takes the target sign from the starting state and adds a disruption budget. Its C++ governor was only partly ported, which is where the reported Python/C++ mismatches came from.

None of these enter before the GPU counted run run (the engine is frozen for it, `docs/GPU_PREREGISTRATION.md`). Any of them can be adopted afterwards only as a full change: Python and C++ together, the parity tests and `verify.py` green, and a new mechanism id.

27.1.2 The rule from here

One engine, one twin, two frozen originals. A new version **replaces** the old one in place, with a new seal; it is never added beside it. Older states are in git history and `docs/HISTORY.md`, not in extra files.

27.2 My engine, line by line: the source against what runs in your cluster

The source. `docs/handoff/mathematics/omni_compass_engine_source_c527df2d.py` (SHA-256 c527df2d…, 31,348 lines). Every mechanism below is named by its function there, followed by where it runs live and how it is checked on every `python verify.py`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Mechanism	In the source	Where it runs	Checked by	Status
Six states E, U, I_U, S, B, Ḃ and equations (1)–(7)	`oc_derivatives`, `rk4_step`	`omnicompass/core.py`	`tests/test_core_parity.py`: 20,000 random states, max difference ≤ 1e-12	exact
Spinor closure, 720°: drive × cos(½ ω_B t)	`v_eff`, `spinor_closure_factor`	`core.v_eff`	same parity test	exact
Axle rest S* from equation (6)	`stable_S_equilibrium`	`omnicompass/storage.py`, `compass.axle`	parity; `tests/test_compass.py`	exact
Double well W(U), basins ±1	`W_potential`, `W_gradient_flow`	`core.derivatives`	parity	exact
Regulation u = −f_U + K_P(σ − U), clipped at the authority	`hybrid_microstep_operator`	`core.control_command`; the governor's push	parity; 500 frozen trajectories (`fixtures/`), 0 mismatches	exact
Basin lock, conveyance, certification	`run_monte_carlo`, `macro_step`	`core.simulate`	500 frozen trajectories, 0 mismatches	exact
Composite storage V = V_U + V_W + V_E + V_S + V_I + V_B, the ledger that closes the circle	`composite_practical_lyapunov_value`, `bath_lyapunov_matrix`, weights	`omnicompass/storage.py`, read by the compass every decision	parity on 5,000 states (≤ 1e-13); descends 648 → 8 over 60 regulated steps with no rise	exact
Human switch: a file or a variable stops every command	`_oc_runtime_stop_requested`	`--kill-file`, restore of every lever	`tests/test_failsafe.py`, `tests/test_muscles.py`, every live run's switch drill	exact
Cap doctrine C1: only a ceiling *above* the reference is admissible; a lower one buys energy by slowing work	`_ci_omni_decision`, certificate C1–C4	`muscles.convey`: CPU limit never below the operator's, idle CPU conveyed to the work	`tests/test_convey.py`	exact

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Mechanism	In the source	Where it runs	Checked by	Status
Sleep only in a certified empty interval	`_ci_omni_decision`, sleep channel	`scripts/kind_nodepool.sh`: a machine idles only once its work has left	`tests/test_active_nodes.py`, `tests/test_convey.py`	exact
Reference Kubernetes HPA rule	`oc_hpa_desired_replicas`	`cpp/` HPA, pod reflex	`tests/test_cpp_hpa_parity.py`	exact

27.2.1 Where my live wiring reads the engine differently from the source's own embodiment

Evolution between decisions. The source's compute embodiment advances the assimilated state *with* regulation locked on basin +1. My live governor advances it with u = 0 and reads the regulation command separately, as the push that gates every machine release.

- Both use the same equations.
- The difference is whether the regulation is applied to the state or only read from it.
- The pre-registered, frozen results were measured with u = 0. I keep that setting, and I report the difference here rather than change a frozen law mid-measurement.

CHAPTER 28

Change Control: How the Engine Changes

The engine is the set of files whose bytes decide what Omni-Compass does: the brain, the compass law, the verdict, the live governors, the harness the models run in, and the wires that fasten the brain to an outside simulator. Their SHA-256 digests together make the engine's fingerprint (`OMNI_V16.json`: 55 files, digest `5c5502f12b1c0ce4`). `python3 tools/omni_version.py` reads the tree and names the engine it holds, or says which file differs from every declared engine.

28.1 The rules

1. **Any change to a rule, a gain, a guard, a preset or a muscle makes the next version.** No engine file changes inside a version. A fix to a fault found in the engine is the next version, never a patch to the one that stands.
2. **Every change goes through the model gate first** (`tools/model_gate.py`, `results/model_gate/`): the shipped controller on the model cluster in the cloud setting, conveyance

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

on one kind host set to a real run's own regime, the verdict on noisy work, the same seeds, paired against the engine that stands. A LOSS there is root-caused and fixed in the model; a change that reads no LOSS and shows promise goes to its GitHub trials; nothing is dispatched before.

3. **A version lands in one command**, `python3 tools/land_engine.py vN --since <UTC>`: the fingerprint, the version list, the conductor's campaign (`scripts/reruns.txt`), the grid's manifest and workflows, the package version (0.N.0), the root list and a release group. `tools/land_engine.py --check`, run by `verify.py`, refuses a push in which any of those places still names an older engine.
4. **Every benchmark runs again on the new version**, A, B and C, released by the conductor in a fixed order; a run is counted on the engine its own commit carries, read from GitHub at its head commit.
5. **No result is read across versions.** A three-run table is one engine's three runs (the table tools refuse to call runs on two engines one table); an older table stays its own engine's record, in `results/live/` until each benchmark's three runs on the new version land, then the newest table is the one shown.
6. **Every fix lands as the next version, and the founder is told.** A change that reads no LOSS in the model gate lands at once as the next version (the founder's order of 19 October, night: "Of course I want you to land version 12 ... every time you do that ... it becomes a new version"); he is told what it changes, why and what it read, and is not asked whether to land it. A change that reads a LOSS anywhere is root-caused and fixed in the model first and never lands as it stands.

28.2 The record

Version	Landed (UTC)	Digest	Files	What changed, and why
omni-v5	11 Oct 22:40	`6093b2b88aa911b9`	44	the max as the default brain on the 1,107-muscle catalog; re-declared while in development
omni-v6	15 Oct 06:18	`df6147466ef18046`	45	shared-host capacity: on kind the host's processor counted once
omni-v7	15 Oct 15:25	`d6d467b824c585f1`	46	conveyance bounded by the namespace's CPU quota; every rule in seconds
omni-v8	17 Oct 15:39	`f93d233136a1f902`	46	CityLearn: a battery whose building has outages stays native (its charge is the building's reserve)
omni-v9	17 Oct 22:21	`7379f3a79ff2c475`	47	CityLearn: the battery wire reads the coming hour; the exact Student t
omni-v10	18 Oct 04:23	`4bf39b0d707784de`	47	CityLearn: the battery glide that cuts the bill most with no LOSS
omni-v11	18 Oct 16:03	`745aa53ec8eeb67b`	47	Kubernetes: a reading with no machine in service is held, never read as an overload

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Version	Landed (UTC)	Digest	Files	What changed, and why
omni-v12	20 Oct 00:30	`d38b58fbe28939b8`	47	Kubernetes: a raise the cluster's admission refused is held, not written again at every check; the swarm's proximity reflex at once (its wire outside the digest, declared with the version)
omni-v13	20 Oct 20:31	`0088837333c6f0fc`	47	Kubernetes: the CPU limits the conveyance gave go back to the operator's on a shared host at its top beside another workload's room; every other pod counted by its own limit
omni-v14	21 Oct 04:47	`7afba44615fcff8f`	47	Kubernetes: a knob that leaves any one muscle native; the operator's own commands run without a shell
omni-v15	22 Oct 02:30	`0a53cdd7a558f803`	55	the brain: the power cap's request trims only while the response line has been clean for the governor's own clear time (the omni-v11 long run's 126 switches in 350 decisions); every wire that holds a governor's rules in the fingerprint (eight taken in, their bytes as they stood)
omni-v16	22 Oct 07:30	`5c5502f12b1c0ce4`	55	the Kubernetes governor: every clock in seconds (the brain's minute and its C++ twin, the compass's damping and smoothing, the edge, the additions past the floor, a command's minute to land, the host's busy share, the heat law, one batch job a minute, the conveyance every 15 s, the loop's own schedule), so the decision period is a clean setting; the heartbeat, the full decision, every 15 s by default (the founder's word, "15 sec all over"), the fast checks every 5 s; at 60 s exactly omni-v15's decisions; the pace read by its own runs (`docs/PACE_PREREGISTRATION.md`)
omni-v17	23 Oct 17:00	`7c937d09e4031cc7`	55	the Kubernetes governor's heartbeat a dial, the main engine every 60 s (the parser's default; omni-v16's was 15 s), any beat from 5 to 120 s, every beat shown beside native (`results/model_gate/V17_LADDER.md`, the live engine page's 1j); the monitors scale with the beat (`omni_heartbeat_seconds`, the alert at three beats, the panel's `/v1/beat`); the cruise wait never under two readings (at 90 and 120 s it had rounded to one, found by the dial's own ladder before it landed); the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Version	Landed (UTC)	Digest	Files	What changed, and why
				founder's order of 22 October, afternoon
omni-v18	24 Oct 02:51	`1cb92edb64c4391d`	62	Omni direct taken into the engine: the seven rule files of Omni-Compass's own orchestrator fingerprinted, the three arms shown together (`docs/OMNI_V18.md`)
omni-v19	24 Oct 16:54	`36ee67284c035235`	62	the pod muscle of its own (`--own-replicas`) for a workload no autoscaler governs; the models' arm by itself (`docs/OMNI_V19.md`)
omni-v20	25 Oct 13:24	`1df07f8e36a97985`	62	the five data stores' wires fixed at their causes (sysbench's own statements, Kafka's hand-back within a poll round, YCSB's cache growing back while it misses) (`docs/OMNI_V20.md`)
omni-v21	25 Oct 17:30	`b658f05d43c1e602`	63	the immune system knows itself, knob by knob: a knob redlining past its line that Omni-Compass cannot bring back goes to native when native's twin reads it lower (`docs/OMNI_V21.md`)
omni-v22	30 Oct 18:30	`0e23a0bc561b900a`	65	the five data stores' wires at the max, through each store's own account of a miss; the governor's hand-back tried five times against a transient failure of the cluster's API (`docs/OMNI_V22.md`)
omni-v23	31 Oct 20:00	`c96ea4105d8747db`	65	Omni direct keeps every muscle Omni-Compass cannot be: a workload's own scale write the cluster refuses three times in a row hands the knob back to native's own pod autoscaler, made from the operator's target and range (the founder's order of 31 October; `docs/OMNI_V23.md`)

Each version through omni-v15 was a fault found in a real run and fixed, omni-v16 made the decision pace a clean setting and set the heartbeat at 15 s on the founder's word, and omni-v17 made the heartbeat a dial with the main engine at 60 s on his word the same afternoon: the record is in `docs/OMNI_V6.md` to `docs/OMNI_V17.md`, and each model gate's reading in `results/model_gate/`. Twelve versions in twelve days is the cost of fixing every fault the day it is found (the founder's order of 18 October: every fault fixed, none set aside); the rules above are what keep that pace from mixing results.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

28.3 What stability means here

A version is stable for a benchmark when its three runs on that version have landed with no change in between. omni-v12 has stood since 20 October 00:30 UTC, when its campaign opened (it reached main shortly before); its campaign is under way, and its three-run tables go into `results/live/V12_*.md` as they land. omni-v11 stood from 18 October 16:03 UTC; its 23 three-run tables (`results/live/V11_*.md`) stay its record. The engine's proofs (`docs/ENGINE_PROOFS.md`), its coverage (`docs/ENGINE_COVERAGE.md`) and its reproducibility check (`tools/reproduce.py`) are run on every push, so a version that stands is also a version that is re-proved, re-tested and re-built from its receipts each time anything else changes.

28.4 Gaps this page names

- Up to omni-v14 the fingerprint held three of the outside-simulator wires (`tools/run_citylearn.py`, `tools/run_pandapower.py`, `tools/run_kil.py`); the other wires that hold a governor's rules (the data stores' `tools/run_redis.py`, `run_kafka.py`, `run_pgbench.py`, `run_sysbench.py`, `run_ycsb.py`, and `tools/run_swarm.py`, `run_mujoco.py`, `run_cpu_power.py`) were versioned by git and tested, but a change to them did not move the digest. Closed in omni-v15 (21 October): every one of them is in the scope (`tools/omni_version.py`, 55 files), and a version declared before them is read without those eight wires and only those: any other file its list does not name still makes the tree a different engine.
- A wire outside the fingerprint carried a fault: the swarm's proximity reflex (`tools/run_swarm.py`) slowed a drone over a second where its own comment says at once, and on a mission draw outside the preregistered seeds it let drones collide (`results/model_gate/V12_SWARM_REFLEX.md`). Its fix is a change to a guard, so by rule 1 it was a version, not a patch: it landed as part of omni-v12 and is declared in `docs/OMNI_V12.md`, although the file's bytes do not move the digest; a swarm table counts only on a commit that carries it.

28.5 The next version's list

Each change below was found in a real run or in our own reading of the code, and each has its evidence written down. Each goes through the model gate against the engine that stands and, when it reads no LOSS there, lands at once as the next version with `tools/land_engine.py`; the founder is told what it changed (rule 6). Landed in omni-v23, on the founder's order of 31 October: a knob whose own-scale writes the cluster refuses handed back to native's own autoscaler, so the organism never runs with a knob nobody moves (`docs/OMNI_V23.md`); in omni-v17, on the founder's word of 22 October, afternoon: the heartbeat a dial, the main engine every 60 s, every beat shown beside native (`docs/OMNI_V17.md`); in omni-v16, on his word that morning: every clock of the Kubernetes governor in seconds, the decision pace a clean setting, the heartbeat every 15 s (`docs/OMNI_V16.md`); in omni-v15, from this list: the power cap's request through the line's own clear time, every wire in the fingerprint, and the kill test's 300 s reading declared

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(`docs/OMNI_V15.md`; a dwell with a glide was refused by the gate and is not in it); in omni-v14: the operator's commands without a shell, beside the founder's knob for every muscle; refused there by the gate and kept on the branch `v14-convey-study`: no conveyance raise on a reading that leaves out another workload's starting pods (`docs/OMNI_V14.md`, section 3); in omni-v13: the conveyance's hand-back on a shared host at its top, found on KubeRay's rig (`docs/OMNI_V13.md`); in omni-v12, from this list: the refused raise held and the swarm's reflex at once (`docs/OMNI_V12.md`). The kind rig's governor role still does not read the namespace's quotas, a grant the Operator's Manual, section 5.5, makes optional, so the omni-v12 capacity runs read the hold in the real cluster.

Change	Why	Evidence so far
The Redis harness's brain in its own process, as a deployment runs it	the brain thread shares the load generator's interpreter lock	`docs/TAIL_LATENCY.md`, section 4
The verdict keeps a step only when the upper end of every 95% interval of its change in cost is inside the allowance (`omnicompass/verdict.py`), its reference measured long enough, from its own spread, to show a free step within the allowance; a trial that could not be read in its time is never started	today a step is kept when its estimated cost is within the 2% allowance even if the interval reaches past it: at the noise measured on the omni-v10 tables (the spread of the log response time 0.43 to 0.46), a give-back that truly costs 3% is kept about 31% of the time, 5% about 15% (found by the live-engine session, 19 October)	89 kept steps on the omni-v10 data-store tables had an interval reaching past 2%, none read LOSS in a table. The strict form was built and read in the model gate (branch `v12-verdict-study`): on work fast enough to read a step (200 pieces a decision) a free step was kept 98% of the time against omni-v11's 66%, a 3% step 0% against 26%; but on the model cluster set from a real run's regime (about 80 requests a decision, about 4,000 needed a phase) no give-back trial started and the governor read identical to native, the machine-hour cut gone, and on the data stores (one sample a second, 40 to 120 a phase; 432 to 23,430 needed, read from the omni-v11 audits) every knob would stay native. Refused as written; the row stays open until a trial design that reads a 2% step in a trial's time is shown in the model gate

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Change	Why	Evidence so far
The Kubernetes governor counting a worker an add-on marks while it serves, and keeping its clock under load	the CruiseKube runs of 18 and 19 October	the recorder's fix of 19 October (`CHANGELOG.md`)
The governor's other costs on a saturated shared host	one saturated KubeRay repetition with no raise during its queue still read +4.0% on the batch pods' hashing; the governor's decisions (every 60 s then and again from omni-v17, every 15 s on omni-v16) and their reads run on cores the kind nodes share through the control plane	one repetition is not a reading: the omni-v13 runs of the batch rig read it, pod by pod, before any change
The pods that wait for a place, in the cloud's setting and in the kind benchmark's	in the model's cloud setting (the tune-down, machines alone) the pods wait for a woken machine longer than native's at every decision pace, the old 60 s default included: pending pod minutes an hour native 8.2, every 60 s 20.9, every 15, 10 and 5 s 18.4, 15.9 and 16.5, LOSS against native at 15 and 5 s and STEADY at 60 and 10 s only by the seeds' spread; the omni-v15 gate read the same row at 15 s (+111.7%); in the kind benchmark's setting (the compass law at the max) about four times native's at every pace (32.0 at 60 s, 33.5 at 15 s, 28.8 and 28.6 at 10 and 5 s)	`results/model_gate/V16_PACE.md`, `results/model_gate/V16_VS_V15.md` (cluster15); the real cloud sets' pod start waits to be read beside it when their tables land
The cloud setting's small cost in response time and alert minutes	in the model's cloud setting (the tune-down, machines alone), against native: at 15 s the 95th percentile +0.47% on the 5 s plant and +0.36% on the gate's 15 s plant, and the alert minutes	`results/model_gate/V16_PACE.md`, `results/model_gate/V16_VS_V15.md` (cluster15, cluster_shipped, cluster_blink); the cloud sets' alert minutes are a judged row of every

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Change	Why	Evidence so far
	+2.31% on the 5 s plant (LOSS, every seed's interval clear of zero); at 60 s +0.49%, +0.63% and +1.85% (STEADY by the seeds' spread); against 60 s each reads STEADY, so the engine's, not the pace's; the blink model's alert minutes read +2.35% against native at 60 s (LOSS) in every gate since omni-v13	receipt
The model's web model brought to the batch rig's 3-core host from a real run's files	in `convey_batch` both engines read 2% to 3% fewer requests served than native, with no batch at all as well, while KubeRay's real runs read 4% more after the queue; the same model reads the alert minutes 9.8% and the job turnaround 0.05% above native's (LOSS) in every gate since omni-v13's	`results/model_gate/V13_VS_V12.md` to `V17_VS_V16.md`; the regime to be set from the KubeRay runs' own after-queue files before the model is read for that gauge
The pod start wait in the convey models	against native the pods start 8.99% slower, 10.0% at the 95th percentile (LOSS), in every gate since omni-v8's, while the host serves 64% more requests at a 53% lower 95th percentile; omni-v7's page wrote it as the price of the busier host's work, but a wait that rose is a LOSS by the key	`results/model_gate/V8_VS_V7.md` to `V17_VS_V16.md` (convey, convey_unreadable_quota); the KEDA real runs' start wait, read with the rig's pre-puller (`docs/POD_START_WAIT.md`)
The card models' 95th percentile	on the modelled card the 95th percentile reads 2.38% above native's where the voltage falls with the clock and 5.78% near the voltage floor (LOSS), in every gate since omni-v15's, where the models were added; the 99th	`results/model_gate/V15_VS_V14.md` to `V17_VS_V16.md` (card, card_floor); the GPU runs on Lambda read the real card

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Change	Why	Evidence so far
	percentile STEADY, the card's energy 4.7% and 1.3% lower (GAIN), the limit written 17 and 35 times a run	
The brain's dwell and its wait after an addition at the fast beats	omni-v16 restated both in seconds (at 15 s, 4 and 20 steps for the 1 and 5 a minute); the live-engine session (branch `main`, 22 October) reports on its own sweep, on-site at 5, 10 and 15 s, machine-hours +3.8%, +3.4% and +2.9% against native where a build counting them as readings that agree read −1.0% to +0.9%, the pods waiting the same; the main engine's 60 s is untouched by it	not yet reproduced here; a search of the dwell counted in readings at each beat, in the tune study's rule (`docs/THE_TUNE.md`, section 4)
The cloud's floor check and a warm reserve	the same session reports two faults under `--give-back-only`: a floor that orders machines the cloud has already deleted is read as a command that did not land and holds every give-back, and give-backs made before the largest requests of the last hours are read leave too few machines; with both fixed its build read identical to native in the cloud setting at every beat, where the engine that stands reads the pods waiting, the 95th percentile and the alert minutes LOSS at some beats	not yet reproduced here; candidates for the cloud setting's LOSS rows (the rows above), through the model gate

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 29

How Much of the Engine the Tests Run

Line coverage of every Python file in the engine that stands (omni-v23), under the whole test suite: the unittest suite and every file `tools/function_tests.py` runs. Measured with coverage.py by `python3 tools/engine_coverage.py --measure` on commit `2b1390a7dc02`; `tools/engine_coverage.py --check`, run by `verify.py`, fails a push where this page is not what `results/ENGINE_COVERAGE.json` records.

71% of the engine's 11,424 statements run under the tests (8,148 run, 3,276 not). Coverage counts lines a test executes, not what it asserts: the properties the tests check are in `tests/test_engine_properties.py`, and the proofs of the core in `docs/ENGINE_PROOFS.md`.

The 8 runners omni-v15 took into the fingerprint (the data stores', the swarm's, MuJoCo's and the CPU power runner, `tools/run_*.py`) hold 3,623 of these statements, 51% of them run under the tests: each is run whole by its own benchmark on GitHub, A, B and C, rather than by a unit test. The other 45 Python files, the engine as it stood before them, hold 7,801, 81% run. The whole figure fell when those runners came in; no test was lost.

Engine file	Statements	Run by the tests	Coverage
`tools/run_pgbench.py`	657	146	22%
`tools/run_citylearn.py`	166	41	25%
`omnicompass/stack_sim.py`	210	55	26%
`omni_orchestrator/control.py`	476	176	37%
`omnicompass/closure.py`	151	61	40%
`omnicompass/speed.py`	60	25	42%
`omnicompass/mathdrive.py`	50	22	44%
`tools/run_redis.py`	427	210	49%
`tools/run_kafka.py`	579	285	49%
`omnicompass/shield.py`	79	41	52%
`tools/run_swarm.py`	257	134	52%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Engine file	Statements	Run by the tests	Coverage
tools/run_sysbench.py	433	226	52%
tools/run_ycsb.py	511	267	52%
tools/run_kil.py	132	94	71%
realms/plants.py	645	461	71%
omnicompass/master.py	77	58	75%
omnicompass/adapter.py	207	157	76%
omni_controller/gpu_compass.py	295	227	77%
tools/run_cpu_power.py	554	427	77%
tools/run_mujoco.py	205	165	80%
realms/harness.py	425	359	84%
omni_controller/quota.py	86	75	87%
omnicompass/compass.py	82	75	91%
omni_controller/controller.py	1,200	1,098	92%
realms/compass_arm_v5.py	838	771	92%
omnicompass/conveyance.py	52	48	92%
omnicompass/muscle_sdk.py	215	199	93%
omnicompass/edge.py	56	52	93%
tools/store_brain.py	102	95	93%
omnicompass/configurations.py	31	29	94%
omni_controller/capacity.py	63	59	94%
omnicompass/disruption_budget.py	16	15	94%
omnicompass/storage.py	32	30	94%
omni_controller/gpu_governor.py	354	332	94%
omnicompass/student_t.py	52	49	94%
omni_controller/muscles.py	621	586	94%
omnicompass/compass_law.py	95	90	95%
realms/compass_arm.py	107	102	95%
omnicompass/authority_contract.py	26	25	96%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Engine file	Statements	Run by the tests	Coverage
omni_orchestrator/brain.py	58	56	97%
omnicompass/core.py	159	154	97%
realms/gpu_card.py	172	167	97%
omnicompass/nervous_system.py	69	67	97%
tools/knob_verdict.py	117	114	97%
omnicompass/immune.py	47	46	98%
omnicompass/verdict.py	109	108	99%
omni_orchestrator/scheduler.py	21	21	100%
omnicompass/ce_score.py	7	7	100%
realms/presets.py	41	41	100%

29.1 Where the tests reach least

Files under 60%: `tools/run_pgbench.py`, `tools/run_citylearn.py`, `omnicompass/stack_sim.py`, `omni_orchestrator/control.py`, `omnicompass/closure.py`, `omnicompass/speed.py`, `omnicompass/mathdrive.py`, `tools/run_redis.py`, `tools/run_kafka.py`, `omnicompass/shield.py`, `tools/run_swarm.py`, `tools/run_sysbench.py`, `tools/run_ycsb.py`. The CityLearn runner and the runners of the data stores and the swarm are exercised by their own benchmarks on GitHub (`citylearn.yml`, `redis.yml` and the rest, three runs on every engine) rather than by a unit test; the stack simulator, the shield and the speed and mathdrive modules hold paths the live governors do not call. Raising each to the level of the rest is the next tests' work: no engine file changes for a test.

PART THREE

The Physics: the Compass and the Nervous System

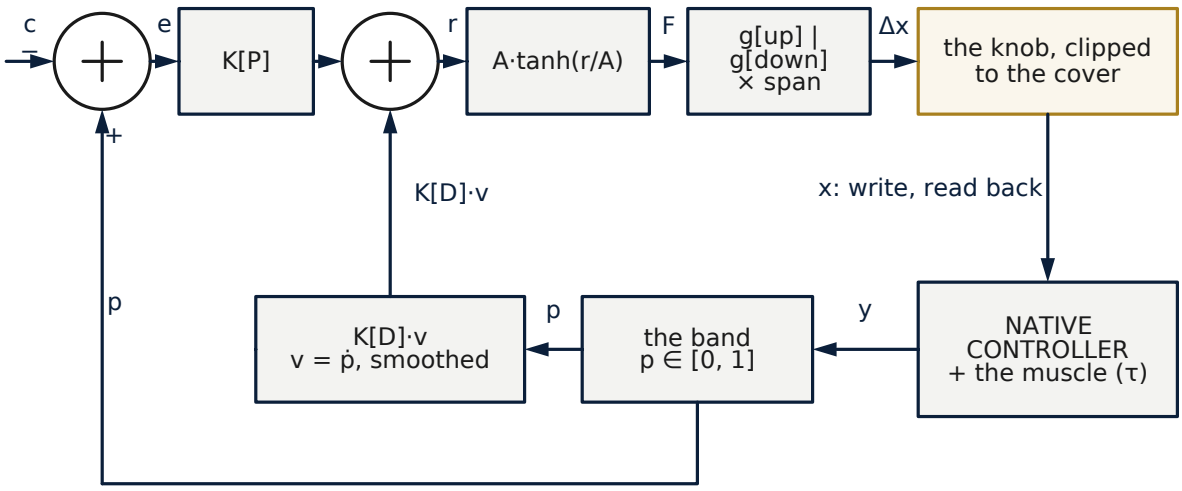
Push and pull, the band and its cushions, the physics of a processor, and the two-way wires that carry the force from the brain to every muscle and back.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 30

The Compass: Push, Pull and the Two Forces

The compass (`omnicompass/compass_law.py`) is the law that carries the engine's push and pull to every muscle.



c the centre of the band; y the reading; p the reading's position in the band; e = p − c; v = ṗ smoothed; r the raw force; F the force after the saturation; Δx the step; x the knob. Fail up: p ≥ 0.95 → F = A at once; the down side held until p is back inside the band. No take-back while the reading climbs above the centre (F < 0 with v > 0 and p > c → F = 0). K[D] is set from the muscle's own response time τ (critical damping), never tuned by hand.

Figure 10. Block diagram: the compass law as a closed loop on one muscle. The reading y of the native controller and its muscle (response time τ) is placed in the band as the position p; the error e = p − c against the centre c, through K[P], is summed with the damping term K[D]·v (v the smoothed rate of change of p) into the raw force r, saturated to F = A·tanh(r/A), scaled by the gain of its direction (g[up] or g[down]) and the knob's span into the step Δx, and clipped to the cover before it is written and read back. The loop closes through the plant: one write a decision, one reading back.

Two bands.

- **The cover**, a knob's hard range (the device's or the operator's lowest and highest setting). Every write is clipped to it. Nothing Omni-Compass computes can set a knob outside it.
- **The compass**, the service reading as a position from 0 (calm) to 1 (the service line). Omni-Compass pulls that position to the bottom of the compass, the middle (0.5 by default; an operator may set it lower for extra margin). The walls run from 0.05 to 0.95; the last 5% on each side, 10% in all, is cushion.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The force. F = A tanh((K[P] (p - center) + K[D] v) / A), where p is the position, v its rate of change, A the authority.

- **Pull:** K[P] (p - center), gentle near the bottom, stronger up the walls.
- **Push:** K[D] v, the force that meets whatever is shoving the position, and the friction that stops it sloshing. With K[D] at or above critical damping the position glides to the middle and stops, with no overshoot and no ringing. Overshoot is wasted energy: force spent going the wrong way and spent again coming back.
- **Smooth, never a hammer:** tanh bends the force over into its maximum instead of slamming into a wall.
- **Fail up:** past the 0.95 wall the up side goes to its full force at once and the down side may not act until the position is back inside the compass.

Two forces: antagonist pairs. Like the muscles of an arm, every plug has an up side (adds capacity, power, cooling, speed) and a down side (takes it back), each with its own gain, because adding and taking back do not cost the same: a new machine takes minutes to boot; giving one back is instant. Where a machine has two wires, each side gets its own lever:

Muscle	Up force	Down force
GPU card	clock ceiling raised	power limit lowered (the lid)
Kubernetes	scale out	scale in
Cooling	chiller on, colder setpoint	warmer setpoint, unit released
Battery	discharge	charge, reserve held
Vehicle, joint	motor effort	braking, regeneration

Why the GPU needs both wires. A card's firmware boosts its clock as high as it may while there is work, and its power limiter knocks the clock back each time the draw crosses the limit; on a busy card this happens many times a second, at the top of the clock range where each extra step of speed costs the most watts. With one wire (the power limit) a governor can only move the wall the boost pushes against. With two wires, the clock ceiling sets how high the boost may climb and the power limit becomes a lid that rarely needs to act: the card runs at the bottom of its compass instead of fighting itself at the top.

What the card taught us. Two rules make the difference between saving energy and spending the customer's time. First, *race while work waits*: when the card is saturated, both wires go to full at once, so a burst is always served at full speed; the compass paces only the slack between bursts. Second, *never slower than the card on its own*: the governor learns, from the card's own meter, the clock and the draw the card reaches by itself while busy, and never sets the clock ceiling or the lid under them. Without these rules the first real card saved 3.5% of its energy and made the slowest answers 58.5% slower; with them, the modelled card saves energy with the slowest answers at native speed or faster (Chapter 60, and `docs/GPU_PREREGISTRATION.md`, amendments 6 and 7).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

30.1 The same law on every stack, and what differs

The law is written once (`CompassLaw`), and three declared numbers fit it to a muscle: the **decision period** *dt* (how often the governor decides), the muscle's **response time** tau (how long the service reading takes to follow the lever), and the **cover**. From tau and *dt* the law derives its own damping gain, K[D] = (2 sqrt(K[P] tau / dt) − 1) dt, which is critical damping for a first-order muscle under a proportional-derivative pull: the position glides to the center and stops. Nothing else is tuned per stack. The table below is the complete list of the compass's settings on every stack in this manual, so a referee can see that the "gains" differ only through the declared physics:

Stack	Reading (the wire in)	Line	Center	Decision period	Response time	Cover of the knob
Kubernetes, the HPA target	the mean response time of the latency window	the operator's SLO (500 ms)	0.4	15 s	the pods' start time	[0.6 × the operator's target, the operator's target]
Kubernetes, the machines	the same, through the release gate and the verdict	the same	0.4	15 s	a machine's wake	[the floor of 2, every machine]
PgBouncer, the pool size	time in the server plus time waiting for one, per transaction	50 ms	0.4	1 s	2 s	[2, PostgreSQL's limit minus 10]
Kafka, the consumer count	the group's own end-to-end latency, mean of the last second	500 ms	0.4	1 s	3 s (a rebalance)	[1, the topic's partitions]
Redis, the memory ceiling	the application's request latency, mean of the last second	2 ms	0.4	1 s	2 s	[16 MB, 512 MB]
MongoDB, the storage-engine cache	the server's own mean read latency, last second	1 ms (set on the tuning workload's smoke run, from 2 ms)	0.4	1 s	2 s	[256 MB, 2,048 MB]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Stack	Reading (the wire in)	Line	Center	Decision period	Response time	Cover of the knob
MySQL, the InnoDB buffer pool	the server's own mean statement latency, last second	0.6 ms a statement (set on the tuning workload's smoke runs, from 1 ms)	0.4	1 s	2 s	[128 MB, 2,048 MB], in 128 MB chunks
A drone's cruise override	the drone's tracking error	0.25 m, the declared safe error	0.5	one control tick (48 Hz)	1 s	[1.0, 2.0] × the planner's cruise
A substation's tap	the bus voltage in its band	the band's edge	0.5	one solve	a week of evidence before a step down	the tap's own range, one tap per move
A robot axis's speed	the axis's tracking error against its takt	the declared error	0.5	one control step	the axis's response	the axis's own limits
A GPU's clock ceiling and lid	the mean response time of the last 5 s	the SLO	0.4	2 s	the card's response	[the card's own busy clock, its top clock]; [the busy draw + 10%, the start limit]

The center is 0.4 wherever the reading is a service time (the "service profile" of the Kubernetes adapter, so that the service sits a little below the middle of its band with margin for a burst) and 0.5 where the reading is a physical error. A **cushion** of ±0.05 in force moves nothing, so a reading that sits at the center with the usual jitter does not make the knob chatter. These are the same numbers in every preregistration and every table; none was changed after a result was seen.

30.2 Direction rules: how a force becomes a move

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

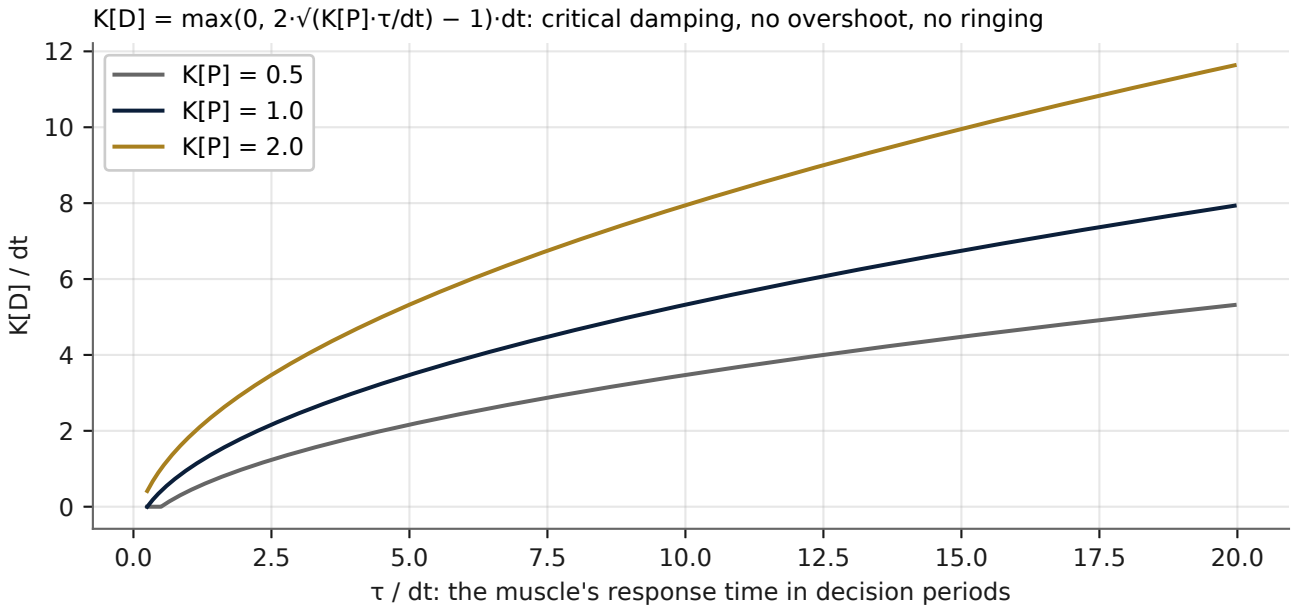


Figure 11. Curve: the damping gain the law sets from a muscle's response time, K[D] = max(0, 2·√(K[P]·τ/dt) − 1)·dt, for three values of K[P]. A slow muscle (large τ in decision periods) gets more damping, so the loop is critically damped: no overshoot and no ringing, whatever the stack.

The force is a number between −1 and +1. Each stack turns it into a move on its knob by a direction rule declared in its preregistration, always of the same shape: a positive force adds capacity in proportion to the force (ceil(force / 0.10) notches, so full force adds ten notches at once, capped by the cover), a negative force takes one notch back per decision, and only when the resource it would take is demonstrably idle (an idle consumer that read nothing in the last second, an idle server the pooler itself shows idle, a cache with nothing evicted in the last second) and only after a **dwell** of a few seconds since the last change, so a knob that was just moved is not moved back by its own transient. At 95% of the line the knob **fails up**: every consumer the topic can use, the pooler's own setting handed back, a quarter of the cache's cover added at once, the planner's own cruise, the card's own clocks. The up side is fast and the down side is slow and gated because adding and taking back do not cost the same, and because a gain bought by taking something back must be able to prove itself before it is kept.

Written out stack by stack, from the preregistrations, the rule reads as follows; the unit, the cover and the fail-up move are each the stack's own, and the shape is the same in every row.

Stack	The knob and its unit	The cover	One notch back, when idle	Fail up at 95% of the line
Kubernetes (the cluster)	worker machines in service	the floor of two to every machine the operator has	one machine a decision, when the remaining machines would carry the load far under the wall	one machine up at once

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

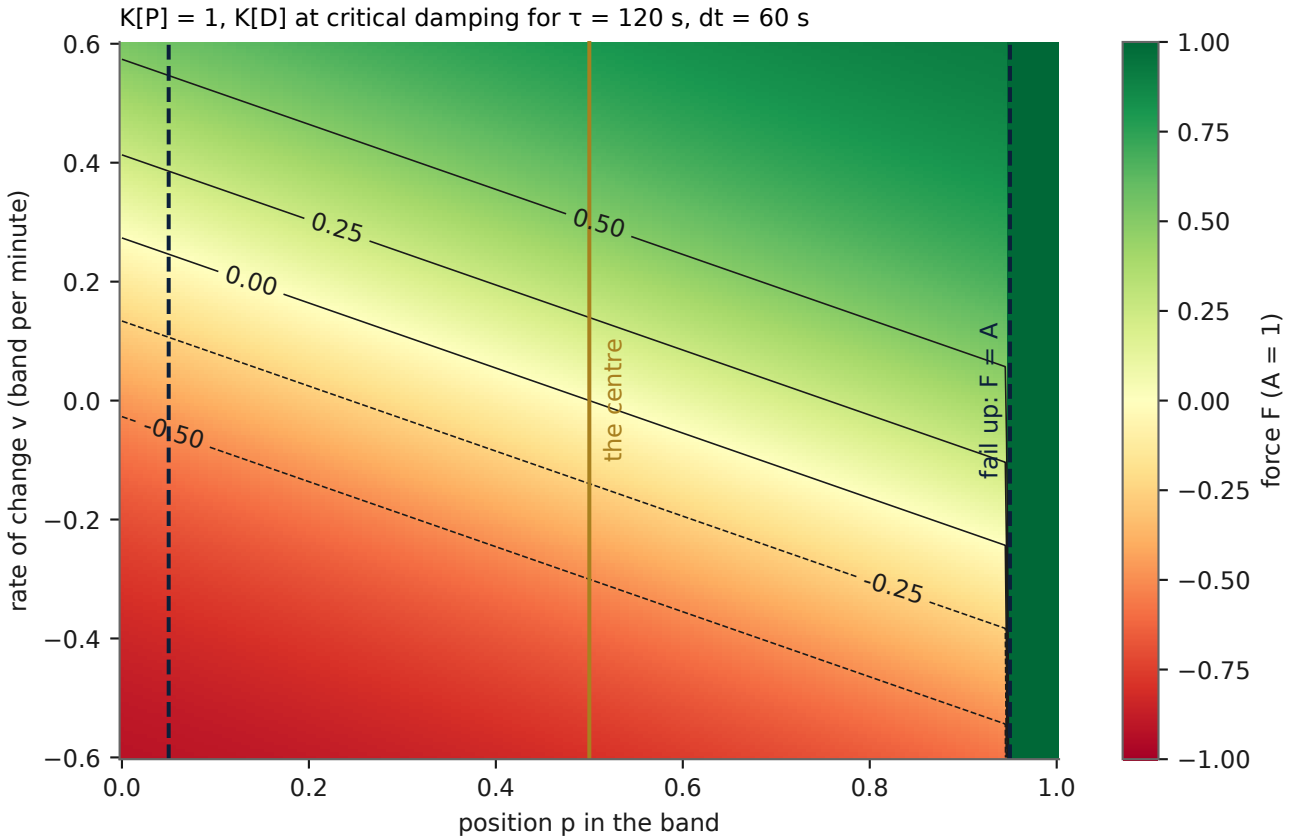


Figure 12. Grid: the force F over the band position p and its rate of change v, for K[P] = 1 and K[D] at critical damping with τ = 120 s and dt = 60 s. Green pushes capacity up, red gives it back; the no-take-back rule zeroes the force where the reading climbs above the centre, and past the upper wall (p ≥ 0.95) the force is the full authority A whatever v reads.

Stack	The knob and its unit	The cover	One notch back, when idle	Fail up at 95% of the line
PostgreSQL behind PgBouncer	the pooler's pool size, server connections	[2, 90]: the floor of two, never within ten of PostgreSQL's 100	one connection a decision, when the pooler itself shows a server idle	the pooler's own setting (20) handed back at once
Apache Kafka	consumers in the group	one to the topic's partitions (8)	one consumer a decision, when a consumer read nothing in the last second	every consumer the topic can use
Redis	the memory ceiling, MB	[16, 512] MB	one notch a decision, when calm and nothing was evicted in the last	a quarter of the cover added at once

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Stack	The knob and its unit	The cover	One notch back, when idle	Fail up at 95% of the line
			second	
MongoDB (WiredTiger)	the cache size, 64 MB notches	[256, 2,048] MB	one notch a decision, when calm and nothing was evicted in the last second	a quarter of the cover (448 MB) added at once while full
MySQL (InnoDB)	the buffer pool, 128 MB chunks	[128, 2,048] MB	one chunk a decision, when the miss share is under 1%	four chunks at once while the pool is full

Two things in the table deserve a reader's attention. The "when idle" column is never a guess from the knob's own value: it is a reading from the stack's own counters (the pooler's idle servers, the consumer's last fetch, the cache's eviction count, the pool's disk reads against its requests), so the law gives back only what the stack itself reports it is not using. And the fail-up column is the stack's own fastest safe move, not a multiple of the notch; where the stack has a controller of its own (the pooler, the planner, the card's firmware), fail up means handing the knob back to it, which is the one move that can never be behind native by construction.

30.3 The do-no-harm gates inside the law

Three gates sit inside the law itself, each a consequence of a loss seen on a tuning case and each disclosed in the preregistration that carries it:

- **The verdict** (section 1.2): a slow knob moves only where a paired trial on the muscle shows no more than 2% cost.
- **The slack gate on speed knobs** (Omni v3): a motion axis that is busy more than half the time at full speed keeps its speed native, because spending its slack as speed when it has none would only cost energy. This gate is what turned the v2 "service tradeoff" on the Physics realm and the tower into "superior within guardrails" on v3, and it is the one rule that distinguishes v3 from v2.
- **The full-cache gate** (Redis): a miss in a cache with room to spare is a cold miss that no ceiling can mend, so the ceiling grows only while the cache is at 90% of it or more. On the tuning workload without that gate the law raised the ceiling to 500 MB while 21 MB were in use; with it, the ceiling follows the working set.

The same gate recurs, in the form each store's own gauges allow, on every memory knob that followed, and each form was fixed on the tuning workload and disclosed before the counted runs:

- **MongoDB's storage-engine cache** grows by a notch only while the cache is full and the server's own read latency is above the centre of its band, and gives a notch back only while calm and nothing has been evicted in the last second (`docs/YCSB_PREREGISTRATION.md`). The smoke runs showed the line

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

itself had to be set from the server's own readings (0.15 to 0.46 ms under a 2 ms line left the law nothing to do), and it was set to 1 ms before the counted runs.

- **MySQL's buffer pool** grows by a chunk only while the pool is at least 90% full of data pages and the server's own statement latency is above the centre, and gives a chunk back only while the **miss share**, pages read from disk as a share of the pool's requests in the last second, is under 1% (`docs/MYSQL_PREREGISTRATION.md`). The earlier rule, "no disk reads in the last second", was never satisfied on the tuning workload: InnoDB keeps stale pages resident, so a pool once filled reads full for ever, and a few new-page reads a second never stop even when the working set fits, so the pool the law had grown was never given back (877 MB held on average against native's 512 on that smoke run); the smoke record says so, and the miss-share rule replaced it before run A.

What the three forms share is the reason: a memory knob has no slack to spend unless the memory is both full and being missed, and memory given back while it is being missed is not a saving but a cost moved to the disk. The gate is written in each harness in the store's own units and checked by that harness's own tests against a stand-in server, so that a reader can see in the preregistration exactly which reading opens the gate and which closes it.

30.4 The profiles and the pedals

The compass has two **profiles**, service (the default) and batch, which differ only in what the pedals do with a pile of work: in the service profile the law paces the slack; in the batch profile cruise puts every machine in service while work waits for a place and idle takes them straight to the floor when the queue is done (section 1.3). Both are written out in the Kubernetes preregistration (`docs/K8S_COMPASS_PREREGISTRATION.md`, rules 7 and 8, amendment 9), both ran in the batch test, and the result (Chapter 60) is machines 19% to 23% fewer over the whole window and 29% to 35% fewer after the queue finished, with the queue itself finishing no later beyond the noise.

30.5 The reflex rule: one body, one brain

Everything in this chapter so far is the force: where a reading sits in its band, how hard the compass pushes, in which direction, and which gates stand in the way. This section is the rule that decides when the force may become a new setting. The founder's order of 10 October 2026 states it in one line: what the brain sends to a muscle has to coincide with what the muscle sends to itself and where it is at. Made into rules, it reads:

1. **Signal out, the muscle moves, the brain feels the full reaction, then it decides.** A trial runs to its full measurement and is never ended by the calm it causes. Only the wall (a fail-up) or the engine's own time limit ends a trial early.
2. **No forcing.** A spend is tried only when the muscle's own reading says there is something to buy (messages waiting, a cache missing, clients queued); a give-back only when the whole body is calm. The force is an urge, smooth, never a hammer.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

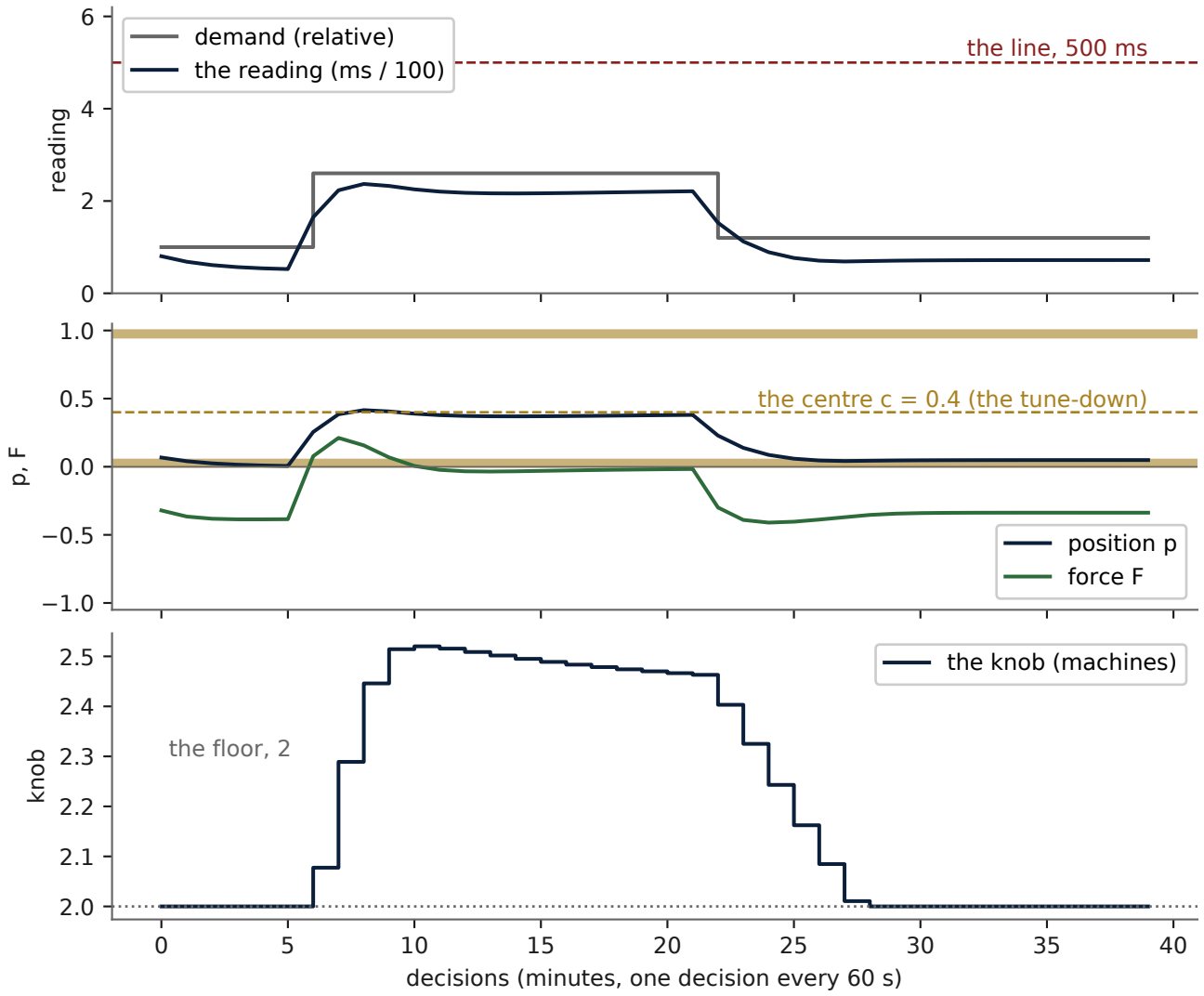


Figure 13. Modelled trace: the compass law answering a load step on one muscle (class S: the law's own code, omnicompass/compass_law.py, on a first-order muscle with τ = 120 s and a decision every 60 s). Demand rises 2.6-fold at minute 6 and falls at minute 22; the reading climbs toward the line, the position p leaves the centre c = 0.4 (the tune-down), the force F rises and the knob adds machines at once; as the reading settles the force crosses zero and the knob is given back one step at a time toward the floor of 2.

3. **One body, one brain, one tick a second.** Everything is read first: every muscle's state, every service's speed and work, the host's processor, memory and energy where a meter exists. Then the brain decides. The cost it judges is the body's cost, the Omni index's own arithmetic over all the work, all the speed, all the machines and all the resources, never one muscle's.
4. **Hurt nothing anywhere.** On top of the body's cost, a guard for every part: a step is refused if any one service read LOSS beyond its cushion, even when the body's total improved.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

5. **One trial at a time inside a body.** Two muscles moved together cannot be told apart, so the body grants one new trial at a time, to the muscle asking loudest whose own condition holds. Every other muscle keeps acting inside what it has already proven.
6. **Nothing is permanent.** Every allowed step is tried again on the recheck and pulled back when it stops paying; a muscle at native is asked again whenever its signal returns. Native is where the brain stands when it has not yet been shown a reason, never a verdict.
7. **The wall belongs to the body.** If any service reaches its line, the trial stops, that step is undone, and every force in the body lifts off: the law's own lift, not a reset.

The trial itself keeps four rules, each learned from a measured run (`docs/OMNI_V4_PLAN.md`, section 4): samples count only once the muscle reports itself settled; a spend on a queue or a cache is judged by the muscle's own reading, with the body's cost taken at the settled state; the reference and the trial are compared at the same load; and where single notches read flat, one scout step to the far side of the cover decides whether there is a hill to climb at all.

Which engine enforces what. In Omni v3, and in the v5 and v6 engines that carry its verdict, the verdict tries a slow knob on the muscle itself before it moves it (section 1.2), and the live harnesses carry the same verdict around every live knob (section 45.4). Omni v4 is the engine written to enforce all seven rules on every wire by construction: one body file that every muscle's wire passes through, the trial rules inside the verdict, and no wire that can write around them. It is designed (`docs/OMNI_V4_PLAN.md`) and not yet built; when it is, it carries its own fingerprint, the package version follows it, and every result in this book is run again on it. The theory of the rule is the chapter "The Reflex Rule" in Part One of the book.

CHAPTER 31

The Physics of a Processor

A GPU's firmware raises its clock one step at a time whenever there is work and room; each higher clock needs a higher voltage, and dynamic power rises with the clock times the square of the voltage, so the top of the clock range is where each extra step of speed costs the most watts. When the draw crosses the power limit, the firmware knocks the clock down several steps; with room again, it climbs again. On a busy card this repeats many times a second. The clock saws against the limit, the overshoots burn watts before the limiter catches them, and the card spends its time at the steepest part of its power curve. The power is the burner; the cooling removes the heat at its own pace; the limiter turns the burner down so the cooling can catch up, and the boost turns it back up.

The firmware does not aim at a balance point because it cannot see the job: it does not know whether the work is a response due in milliseconds or a batch due overnight, the right point moves with the workload, the room

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

and the chip, and a limiter is simple to prove safe. The vendor default is set for maximum performance, and the electricity bill is the customer's.

Omni-Compass sits on the customer's side, where the job's service line is known. It holds two wires: the clock ceiling sets how high the boost may climb, and the power limit becomes a lid set just above what that ceiling draws. The boost stops at the bottom of the compass instead of slamming into the wall; the limiter rarely needs to act. Every watt kept off the chip is saved twice in a data center: once at the chip and again at the chillers that would have carried its heat.

CHAPTER 32

The Two-Way Nervous System

Every muscle is wired both ways: a sensory wire in (its meters) and a motor wire out (its knob), with the read-back closing the loop. Between them sits the nervous system (`omnicompass/nervous_system.py`), which decides how much authority each organ has at each moment:

- **Expand is always allowed** (except under a security hold): adding capacity, power, cooling or protection never waits.
- **Contract needs calm and a clean service record:** giving anything back is allowed only while the organ is calm enough and service has been inside its line for the clear time, 180 s; continuous organs give back at most the calm share of their surplus per decision; discrete organs one unit at a time, through a release gate.
- **Fail up:** while service is breached, the knob returns to native at once.
- **Blind means hold:** if a sense goes stale or unreadable, nothing is given back until it returns.

One brain reads every organ at once. Because one law sets every knob, no two muscles fight: when the GPU's watts turn to heat, the cooling knob already knows it is coming; when pods scale up, the power envelope is ready.

"Two-way" is a statement about wiring, and it is checked, not assumed. Each muscle's plug (Chapter 42) carries exactly one sensory wire and one motor wire, and the wire check that must pass before any write (section 42.4) proves that the wires move: a value written through the motor wire is taken by the device and read back as the value sent, and the snapshot is restored and read back at the end. No governor writes a knob whose wire check has not passed, because a knob that can be written but not read back cannot be handed back with proof. What the wire check does not prove, section 42.5 says plainly: that the governor is reading the right gauge for the knob; that is what the watching arm is for. The receipts of every run therefore carry, for every write, the value sent and the value the device reported, side by side, and a reader who wants to know

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

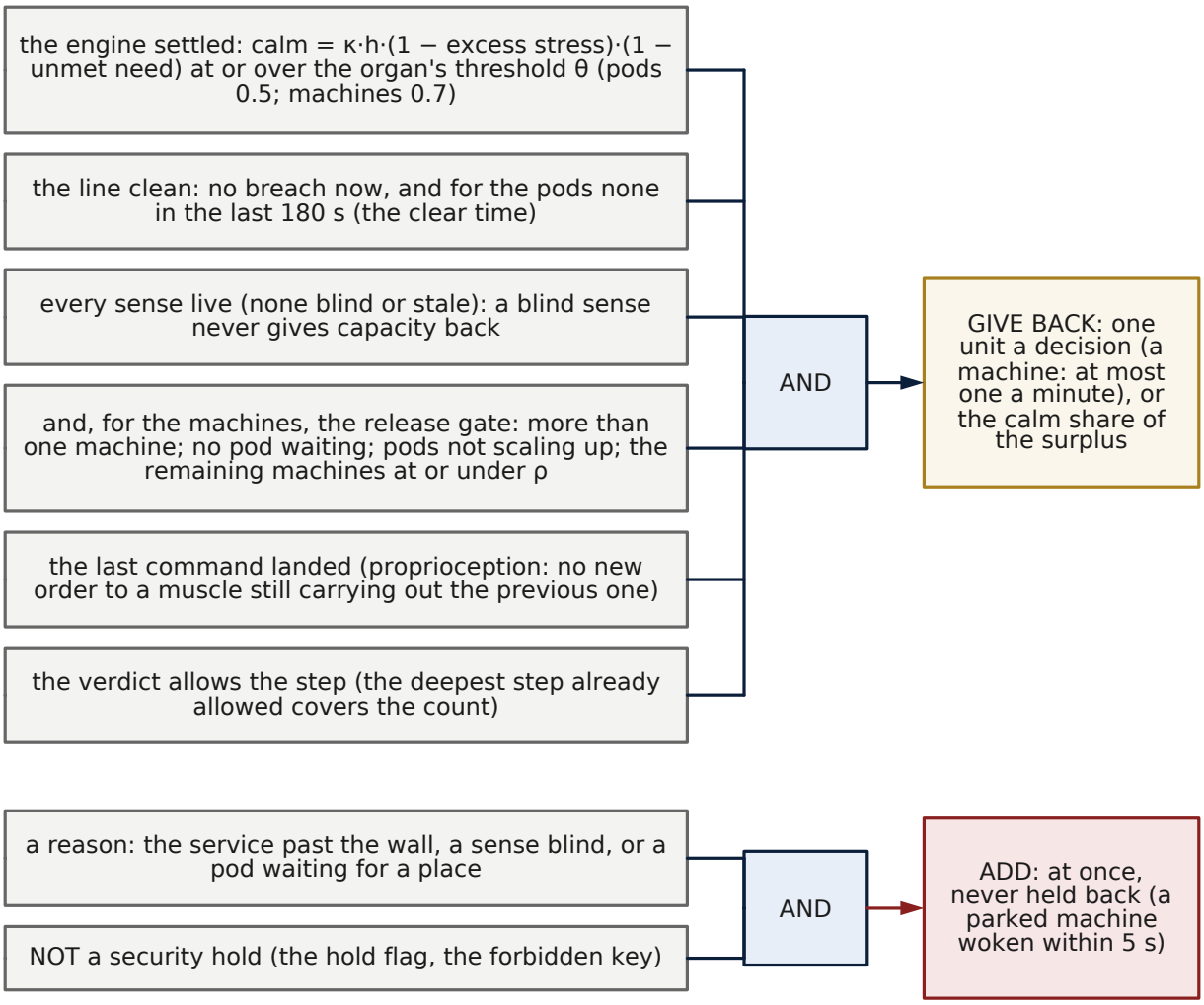


Figure 14. Logic diagram: the nervous system's authority to add and to give back. Giving back needs every condition at once: the engine settled (calm = κ·h·(1 − excess stress)·(1 − unmet need) at or over the organ's threshold θ, 0.5 for the pods and 0.7 for the machines), the line clean (no breach now; for the pods none in the last three decisions), every sense live, for the machines the release gate's own reasons, the last command landed, and the verdict's step allowed; adding needs only a reason (the service past the wall, a sense blind or a pod waiting) and the absence of a security hold.

The asymmetry is deliberate: adding is cheap to undo and expensive to delay; taking back is cheap to delay and expensive to get wrong.

whether the governor's hand was really on the stack can check the two columns rather than take the summary's word.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The four rules above are the whole of the nervous system's authority model, and they are asymmetric on purpose. Adding is cheap to undo and expensive to delay, so it is never gated beyond the security hold. Taking back is cheap to delay and expensive to get wrong, so it is gated three ways: by calm (the engine's own state must report that the body is settling), by the service record (the gauge must have been inside its line for the last 180 s, not merely at this decision), and by the release gate's reasons, which are written into the audit every time the gate holds (section 32.1). Sections 32.2 to 32.4 take the three ideas that make this safe at scale, the governor's sense of its own hand, the rule that a missing reading is a hold and not a guess, and the case for one brain over many, and give the evidence for each from the live runs.

32.1 Authority, and the release gate

Authority is how far the nervous system lets an organ move in one decision. It is granted from the engine's state (section 12.3): an organ in a calm body with a clean service record may give back; an organ in a stressed body may only add. On Kubernetes the **release gate** (`omnicompass/nervous_system.py`, `node_release_gate`) lets one machine be given back only when every one of the following holds, and writes the first reason that fails into the audit so a reader can see why a machine was kept, in the order the code tests them: more than one machine is in service; no pod is waiting for a place; the pods are not scaling up; the line is not breached now and its feed is live; the remaining machines would run at or under the utilisation target ρ (used ÷ ((n − 1) × one machine's capacity) ≤ ρ, ρ the engine's own demand clipped to [0.5, 0.95]); the machine organ's own engine state grants contraction (`authority()`: its calm at or over θ[machines] = 0.7, its line clean, no security hold); every sense is live; and the last machine command has landed (no new order goes on top of one the cluster has not yet carried out). The controller adds one rule of its own: at most one machine a minute, however often it decides (v7). The gate is asked only when the compass law itself asks for a machine back ($F < -0.2$, the reading under the centre, the verdict's deepest allowed step covering the count), never on the max and never during a trial, and cruise overrides it: while work waits for a place, every machine stays in service. One machine per decision is the rule in the slow, reversible direction; past the wall, one machine up at once.

What the gate writes, in a real run. The first repetition of the first kill-scenario run (`results/live/raw/run-37716845219/paired-1/`, the omni arm's `audit.jsonl`) holds 114 decisions and 42 writes. Its gate records read, in the governor's own words, "release permitted" eleven times, each with the utilisation the remaining machines would carry after the release (0.034 to 0.153, all far under the utilisation target ρ); "node organ has no contraction authority" once, a calm cluster whose engine state had not yet earned the right to give back; and "latency breached now; node organ has no contraction authority; a sense is blind" once, three reasons at once in a decision whose latency sense read stale, every one of them a reason to hold. A reader can therefore see, decision by decision, not only what the governor did but why it declined to do the rest, and the first reason that failed is the one written, so that a machine kept in service always has a stated cause.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

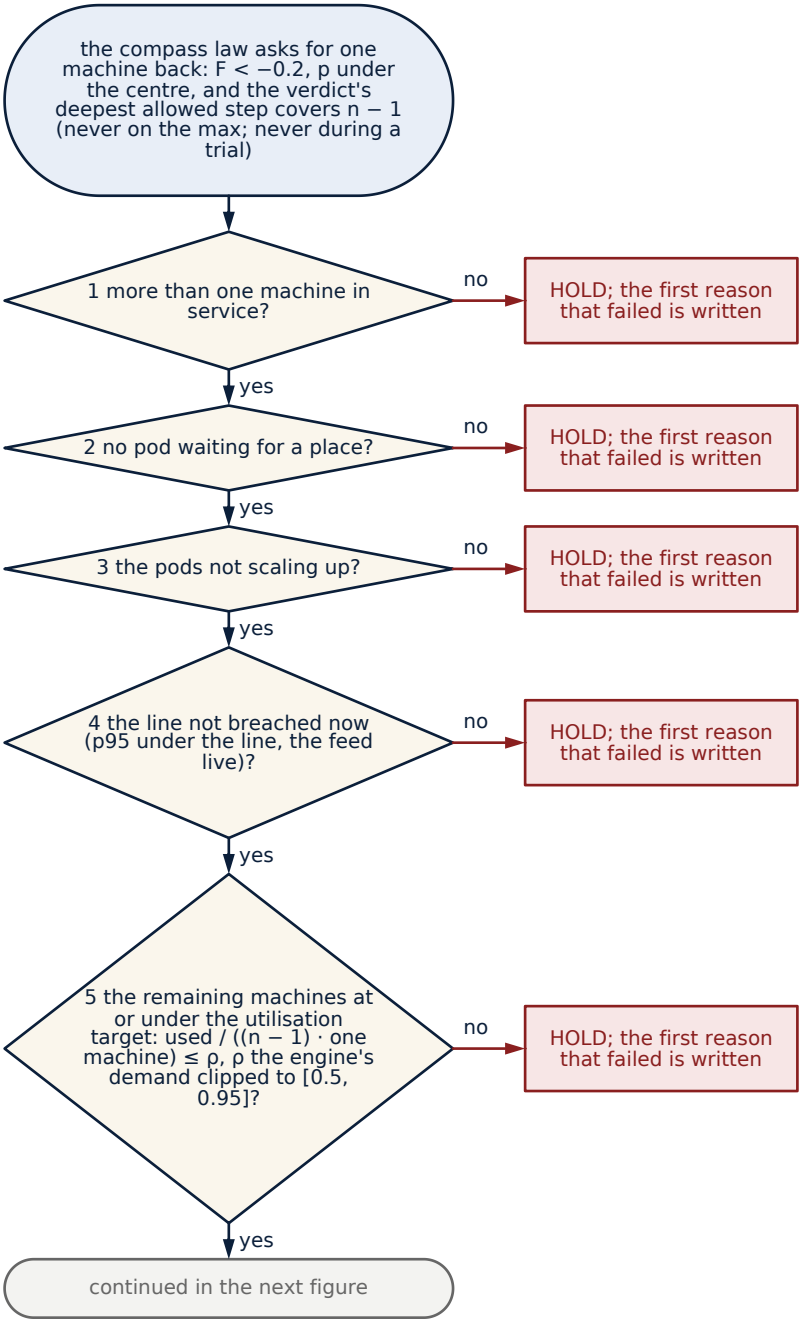


Figure 15. Flowchart: the release gate, reasons 1 to 5. The compass law asks for one machine back (F < −0.2, the reading under the centre, the verdict's deepest allowed step covering n − 1; never on the max, never during a trial). The gate then tests, in the code's own order: more than one machine in service; no pod waiting for a place; the pods not scaling up; the line not breached now, the feed live; the remaining machines at or under the utilisation target ρ (used ÷ ((n − 1) × one machine) ≤ ρ, ρ the engine's demand clipped to [0.5, 0.95]).

The first reason that fails holds the machine and is written in the audit. Continued in the next figure.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

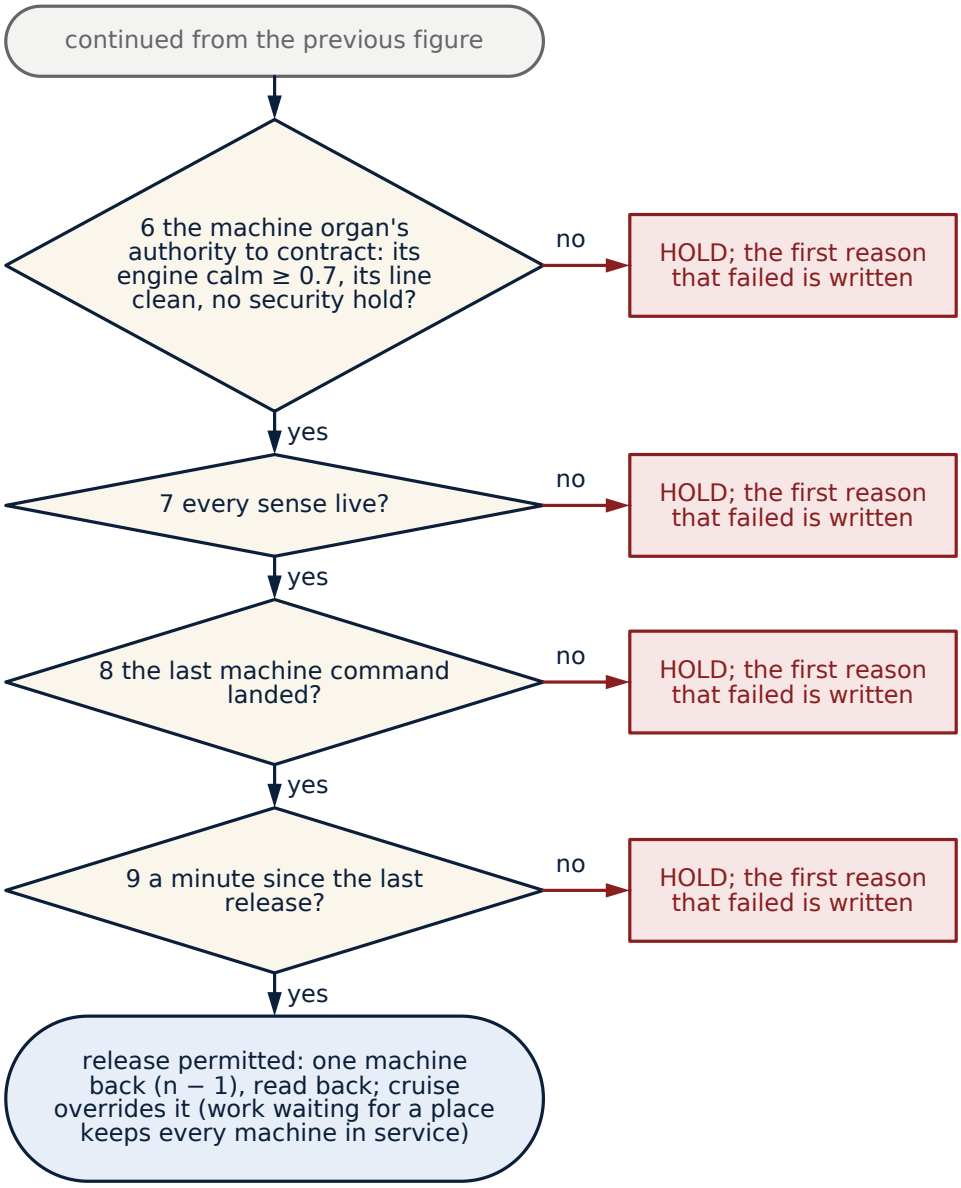


Figure 16. Flowchart: the release gate, reasons 6 to 9 and the release. The machine organ's own authority to contract (its engine calm at or over θ = 0.7, its line clean, no security hold), every sense live, the last machine command landed, and a minute since the last release; only then is one machine given back and read back. Cruise overrides the release: while work waits for a place, every machine stays in service.

32.2 Proprioception and the one-writer rule

The nervous system knows where its own levers are. Every write is read back from the device (the plug contract, section 8), and the audit records both the value sent and the value the device took. If a lever is found at a value Omni-Compass did not write, someone else owns it: the governor stops writing that lever and leaves it alone, because restoring over it would fight the new owner. This is **proprioception**: the governor's sense of

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

its own hand. It is the reason a wrong installation leaves a signature in the receipts (section 42.5) rather than a silent error, and it is what the wire check proves before anything runs.

The rule begins with the **snapshot**. The governor reads every knob it may move once, at the start, before its first decision, and that reading is the operator's setting for the whole run: every reset, every hand-back and every kill returns the knob to the snapshot, never to a value computed later, and the receipts record the snapshot beside the final read-back so that a reader can check the two are equal. The rule continues with the **read-back** after every write: the plug sends a value, waits for the device to take it, and reads the device's own report of what it holds. Where the device takes time to apply a setting, the plug knows that and does not mistake the transition for a foreign hand. The clearest case is MySQL's buffer pool, which the server resizes in chunks over several seconds and reports as it goes: the plug reads the server's own resize status and treats a pool caught mid-resize as its own write in progress, not as another owner's value, and waits for the server to report completion before it judges the write (`tools/run_sysbench.py`, `BufferPool`). The same shape holds on the Kubernetes autoscaler's bounds, Redis's memory ceiling and MongoDB's cache size: the value is read from the device's own console, and only a value that the governor neither wrote nor is waiting on is read as someone else's.

What follows from a foreign value is deliberately conservative. The governor does not restore over it, because an operator who changed a knob by hand meant to, and a second controller that changed it is a conflict to be reported, not won. The lever is left where it was found, the audit records the value and the time, and the governor goes on governing the other levers. A reader of the receipts who finds such a record knows at once that two hands were on the stack, and the paired run that contains it is read with that in mind.

32.3 Blind means hold, and fail up

A sense that goes stale or unreadable is not read as calm. It is read as **blind**, and while any sense is blind nothing is given back; the knob holds, or, if the service was breached when the sense went blind, returns to native at once. In the fault test on real Kubernetes the response-time probe is deliberately paused for a minute in every arm; the audit shows the governor holding through the blind minute and resuming when the sense returns, and the table shows no row behind for it. The rule is simple to state and strict in effect: Omni-Compass never acts on a reading it does not have.

The rule was also tested without being arranged. In the two-hour robustness run on real Kubernetes (`results/live/V3_ROBUST_LONG.md`), the cluster's own API answered a handful of the governor's reads of the autoscaler with a server error, two or three times a run out of the 120 decisions a two-hour window expected at the 60 s period of that engine. Each of those decisions is in the controller's log with the reason the API gave, each was held (no write, no hand-back, no guess), and the governor took its next decision on schedule when the API answered again. The preregistration had said in advance that such decisions would be shown with their reason and not judged, and that is how they appear in the table: 97.5% of the expected decisions valid, the failed ones counted and named, the knob handed back at the end of every run. A referee who wants to see the rule in the same rule written by the gate itself can find it in the kill test's receipts (section 32.1), where a decision whose latency sense read stale is held with "a sense is blind" among its reasons, and in

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the fault test's blind-probe window, where it was arranged on purpose. The point in every case is the same: a stale or missing reading produces a hold, never a move, and the hold is in the record.

32.4 Why one brain

The alternative to one brain is a controller per layer, each tuned alone: an autoscaler that adds pods while a power cap is trying to hold the machines down, a cooling loop that chases heat the clock governor is about to remove. These controllers fight because none of them knows the others' intent. One law that sets every knob from one state cannot fight itself: the force on each lever is computed from the same reading of the same body at the same moment. This is also why the modelled organisms are run whole, at up to 1.7 million muscles on one clock (section 34.4): the question they answer is whether one brain stays coherent at that size, and the reading at every size is the same to the digit.

One brain also has to stay the same brain over time, and that is a measurable claim. The two-hour robustness run (section 60.15) measures the governor's own memory at the start and the end of each run and the time it takes to reach a decision in the last hour against the first. Over three runs the memory ratio was 1.05 to 1.07 and the decision-time ratio 1.07 to 1.20, both inside the limits set before the run (a leak would show as steady growth; a brain that slows as its records accumulate would show a ratio climbing toward the 1.5 limit), and the cluster's service under the governed arm was inside the noise of native in two of three runs. The 24-hour runs on three rented machines, started as this is written, ask the same two questions over twelve times the window. A single controller that keeps its memory and its speed over a day is the least a referee should ask of a brain that proposes to hold every knob at once; the figures are in the table, not asserted here.

CHAPTER 33

The Nervous System in Detail

33.1 The two-way nervous system (live controller)

Source. Manuscript Appendix J: "Nervous system. The signal layer: sensing interfaces, unit integrity, timing coherence, delay and dropout handling, and feedback interpretation. … The brain cannot compensate for corrupted signals." Manuscript Section 5.3: the engine "projects forward state trajectories … and detects destabilization pressure before divergence."

The gap. Before this change, the live controller fed the engine's `stale` (signal dropout) and `drift_ratio` (actuation mismatch) channels with a constant 0. So the upward path could not report a blind sense, and the downward path never learned whether its orders landed.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

33.1.1 Upward: afferent integrity (senses to engine)

Every declared sense reports whether it is live.

- **Latency.** Blind when its newest sample is older than two windows by the wall clock, or when the window holds no successful request.
  - A hung probe freezes its file; its last clean window must never be read as the present.
  - Failed requests inside a live window become latency pressure (the failed share), so a failure is never read as silence.
- **Power.** Blind when its command returns no number.

How blindness is used.

- The blind share enters the engine as `stale`. It raises E, lowers U and feeds the external bath (equations 1-3 via `assimilate`).
- The nervous system grants no contraction on a blind sense, to any organ, and admits no held batch work.
- Expansion and batch pause (the protective directions) stay allowed.

33.1.2 Downward and back: efferent feedback (proprioception)

Every command is recorded.

- The node count commanded.
- The HPA targets written.

At the next decision the observed state is read back. For each organ, `drift = |observed - commanded| / commanded`.

- The largest drift enters the engine as `drift_ratio`. It raises E and I_U, lowers U and feeds B (equations 1-3).
- The node organ may not be given a new release while its last command did not land. For example, a drain refused by the PodDisruptionBudget leaves the node in service, and the next decision sees it.

33.1.3 Sideways: attribution (organ to organ)

The machine organ has its own engine view, fed with machine-attributable pressure only (pods waiting for a place).

It releases a machine only when all of these hold:

- pods are not scaling up and latency is not breached;
- nothing is pending;
- after the release the remaining machines stay at or below the engine's rho;
- its own calm, stress and security gates grant contraction;
- every sense is live;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- its last command landed.

33.1.4 Tests (all in `verify.py`)

Test	What it checks
`tests/test_two_way.py`	blind senses (about 75,000 random states), frozen-probe detection, failures read as pressure, release refused while blind or after an order that did not land
`tests/test_node_release_gate.py`	the release gate, over 200,000 states
`tests/test_nervous_system.py`	the original invariants N1-N6, over 300,000 states

Every live decision records the senses, their ages, the proprioceptive drift and the gate's reason in the audit, and the benchmark prints them in the job log.

33.2 Nervous system

Omni-Compass is one field. Muscles are nerves.

```
telemetry  ->  afferent (per muscle)
           ->  one six-state engine
           ->  reflex (shield)
           ->  efferent (only WIRED + authority + not killed)
           ->  native controller on kill
```

`omnicompass/nervous.py` is the register. It does not invent GPU or chiller physics.

Status	Meaning
wired	this tree can sense and push
sensed	this tree can sense; it does not push
open	named, no plant, no push

Wired today: `nodes`, `hpa`, `power_cap`. Sensed: `heat`, `network`, `security`. Open: GPU, cooling, grid, queues, agents, … Never a muscle: value alignment.

```
python k8s_controlplane/test_nervous.py
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

PART FOUR

The Body: Muscles, Realms and Organisms

One thousand one hundred and seven muscles in four realms on one spine, built into seven organisms, and every gauge used to judge them.

CHAPTER 34

The Muscles, the Realms and the Seven Organisms

34.1 The catalog

The catalog of Omni v5 (`realms/catalog_v5.csv`) lists **1,107 distinct muscles in 70 families** (v2 and v3 listed 945 in 59, `realms/catalog.csv`, frozen for their results; v1 listed 656 in 46), each with its family, its plant model and its knob. A muscle is anything with a meter and a setting: a compute pool and its machine count, a GPU and its clock ceiling, a chiller and its setpoint, a battery and its reserve, a feeder and its tap, a robot joint and its speed limit, a pump and its flow. Each muscle carries a declared plant model, the simplest physics that reproduces how its meter follows its knob, and a native controller of its own, which is what the muscle does when Omni-Compass is not there. The catalog was built in four waves (`docs/realm_study/`): the spine from primary specifications, each realm's extras from its own specifications, a cross-check against industry and open-source controllers, and a verification, de-duplication and mapping pass. The register (`docs/REGISTER.md`, section 1) lists every family with its muscle count and the kinds of knob it carries, so a reader can see at a glance what the modelled realms contain and what they do not.

The seventy families, by realm, are these. **Compute, AI and cloud** (twenty-seven): AI inference serving, AI training, blockchain and ledger nodes, CI/CD and build farms, cloud virtual machines and capacity, container resources, cross-cluster and edge placement, DPU and programmable IO, distributed cluster managers, game servers and real-time interaction, GPU fabric and RDMA, GPU hardware, HPC and distributed compute, host CPU and memory, Kubernetes dynamic device allocation, Kubernetes placement and scheduling, Kubernetes workload scaling (the largest family, 32 muscles), Linux kernel and operating-system control, media streaming, node fleets and Karpenter-class control, OpenShift and the Machine API, quantum computing control (modelled), serverless functions, virtualization and hypervisors, work admission and demand shaping,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

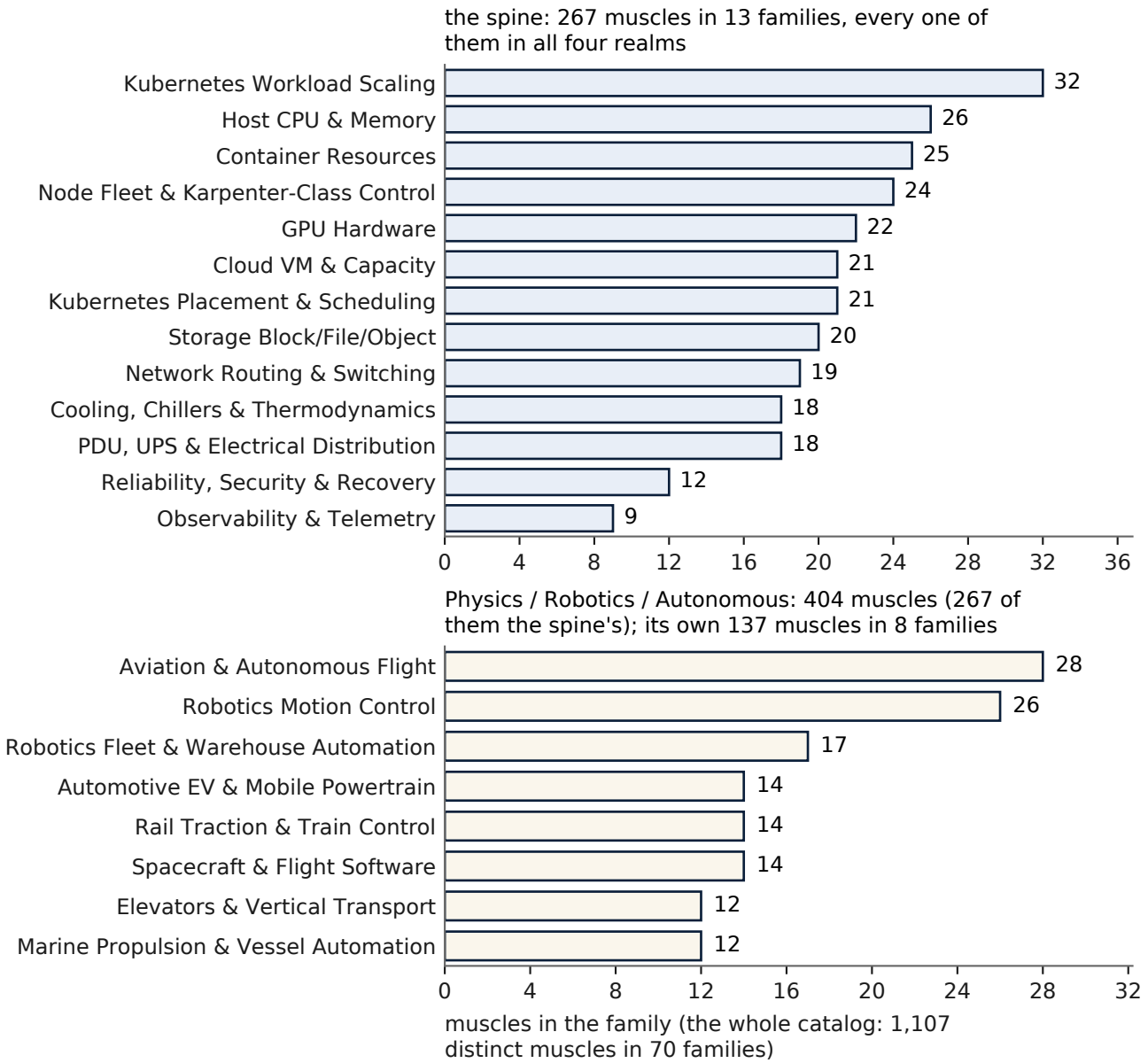


Figure 17. Grid: the families of the spine and of the Physics / Robotics / Autonomous realm, from realms/catalog_v5.csv. The spine's 267 muscles, every one of them in all four realms, in their 13 families; and the realm's own muscles, those it adds on the spine, in their families, by count.

and the body's gut (data-plane intake and waste) and skin (perimeter and admission). **Physics, robotics and autonomous** (eight): automotive EV and mobile powertrains, aviation and autonomous flight, elevators and vertical transport, marine propulsion and vessel automation, rail traction and train control, robotics fleets and warehouse automation, robotics motion control, and spacecraft and flight software. **Energy, facility and industrial** (seventeen): agriculture and irrigation, building and critical-environment HVAC, cooling and chillers, district heating and cooling, energy storage and microgrids, facility and grid optimisation, grid transmission and distribution, healthcare critical environments, industrial PLCs and process automation,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

mining and mineral processing, oil and gas pipelines, PDUs, UPS and electrical distribution, pharmaceutical and food manufacturing, power generation and turbine control, renewable generation and inverter control, semiconductor fabs and precision manufacturing, and water and wastewater pumping. **Distribution and specialized** (eighteen): cache and memory services, commerce and payments, content delivery and edge caches, data analytics and ETL, databases and transactions, financial markets and trading systems, medical imaging and clinical systems, messaging and streaming, network routing and switching, observability and telemetry, ports and maritime logistics, reliability, security and recovery, runtimes and applications, search, indexing and vector databases, service meshes and API reliability, block, file and object storage, telecom radio access and edge radio, and workflow, logistics and fulfilment. Thirteen of these, marked "v2" in the register (the elevators, marine, rail, agriculture, district heating, healthcare, mining, pipelines, pharmaceutical and food, power generation, renewable generation, medical imaging and ports families), came with the second engine version; they, with the muscles added to the families that already existed, are the difference between the 656 muscles in 46 families of v1 and the 945 in 59 of v2 and v3. Eleven more came with v5 (the Linux kernel, serverless, virtualization, content delivery, gut and skin families in a first wave; financial markets, media streaming, game servers, blockchain nodes and build farms in a second), with muscles added to families already there; the founder's nine open-source pieces (Kepler, Climatik, EAR, CruiseKube, Goldilocks, the Kubernetes power manager, PEAKS, adaptive GPU power capping and NVIDIA's platform power budget) came on 12 October as ten muscles in the families their knobs belong to (`docs/OMNI_V5.md`, `docs/THE_SEAT.md`). The GPU family is named for the hardware, not one vendor.

Every muscle its own card. The brain is one, but since 13 October its settings are each muscle's own. A muscle's card holds the numbers the max uses for that muscle alone: how full a machine pool runs on its forecast before it calls for more and how many decisions of history it reads, how many wobbles of margin a battery keeps under its line and, for a battery wired by its power, how long it reads its line before it holds and how much of it it keeps clear. The cards were found by an automatic search over every muscle (`tools/tune_cards.py`), on development seeds only, today's global setting always one of the choices, and a card was kept only where it beat the global setting beyond the noise on two separate sets of seeds and was never behind native on any seed under any load (never over its line more often, never less work, never its line cleared later): 364 muscles carry one (`realms/muscle_cards_v5.csv`, every card in plain words in the muscle catalog, the evidence in `results/cards_v5/CARDS.md`); the rest run the global setting. Inside a body, a battery wired by its power keeps the body's own margins on its line, because the line carries the whole body's swing, which the card chosen for the battery alone never saw. The formal statement is *The Max, Formally* (`docs/FORMAL_V5.md`, section 5).

Every live benchmark in Part Seven sits inside one of these families: the Kubernetes tests inside workload scaling and node fleets, PostgreSQL and MySQL inside databases and transactions, Redis and MongoDB's cache inside cache and memory services, Kafka inside messaging and streaming, the robot arms inside robotics motion control, the drones inside aviation and autonomous flight, the power grids inside transmission and distribution, the buildings inside HVAC. A live result is therefore also a check on one family's model: where the real stack and the modelled muscle move the same way under the same law, the model has earned

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

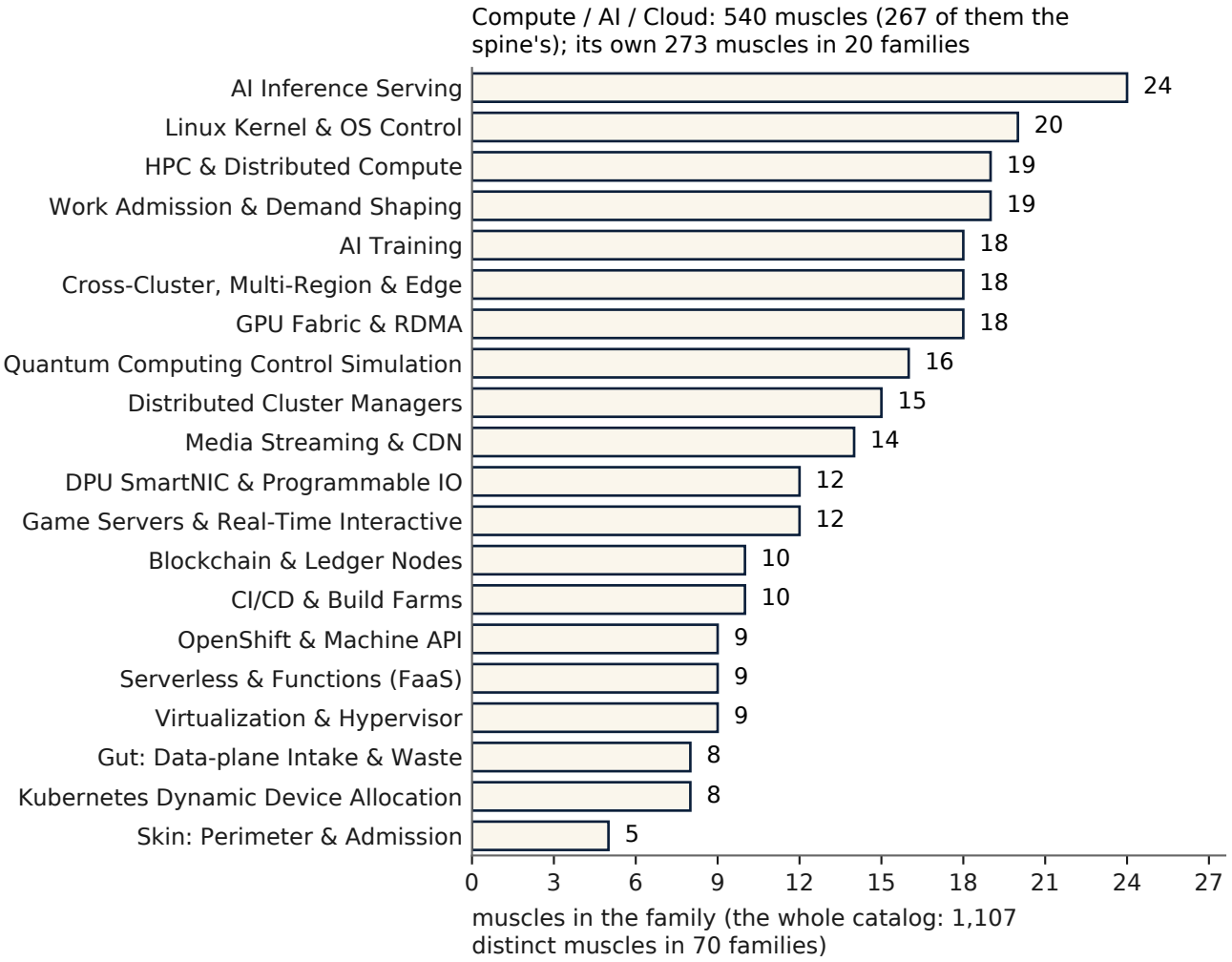


Figure 18. Grid: the families of the Compute / AI / Cloud realm, its own muscles on the spine, by count.

some trust; where they do not, the model is the thing to doubt, and the catalog's plant model for that family is the place to look.

34.2 The spine and the four realms

The muscles fall into four realms, and under every realm's own muscles stands the same **spine** of 267 muscles (Kubernetes workloads, containers, placement, node fleets, cloud capacity, hosts, GPUs, network, storage, observability, reliability and security, electrical distribution and cooling), because every realm's work runs on computers that must be scheduled, powered and cooled. Since 16 October the spine is an organism in its own right, the seventh, run native and with Omni-Compass on top like the other six: it is the base of every stack as an operating system is the base of every program, and a realm's reading beside the spine's shows what that realm's own trade adds to the base it stands on (the chapter *The Spine* follows this one). The catalog thus builds seven organisms, the spine first because the other six stand on it:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Organism	Muscles	What it is
The spine (`spine`)	267	the muscles every real stack runs on, every one in all four realms: clusters, containers, placement, node fleets, cloud capacity, hosts, GPUs, network, storage, observability, reliability and security, electrical distribution, cooling; an organism of its own, the base of the other six
1. Compute / AI / Cloud	540	clusters, GPUs, AI training and inference, cloud capacity, kernels, functions, media, games, ledgers, build farms
2. Physics / Robotics / Autonomous	404	joints, fleets, vehicles, flight and spacecraft axes
3. Energy / Facility / Industrial	480	data halls, buildings, batteries, UPS, process loops, feeders, hospitals, farms, pipelines, mines, district heat, power plants, renewables, pharmaceutical and food plants
4. Distribution / Specialized	484	networks, storage, databases, commerce, trading, content delivery, workflows, radio networks, clinical systems, ports
5. The four stacked, every duplicate kept	1,908	all four realms on one clock, the shared spine counted in each
6. The whole tower, every muscle once	1,107	every distinct muscle on one clock

These are the counts of the catalog that stands; earlier catalogs' counts stay with their own results in `docs/history/`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

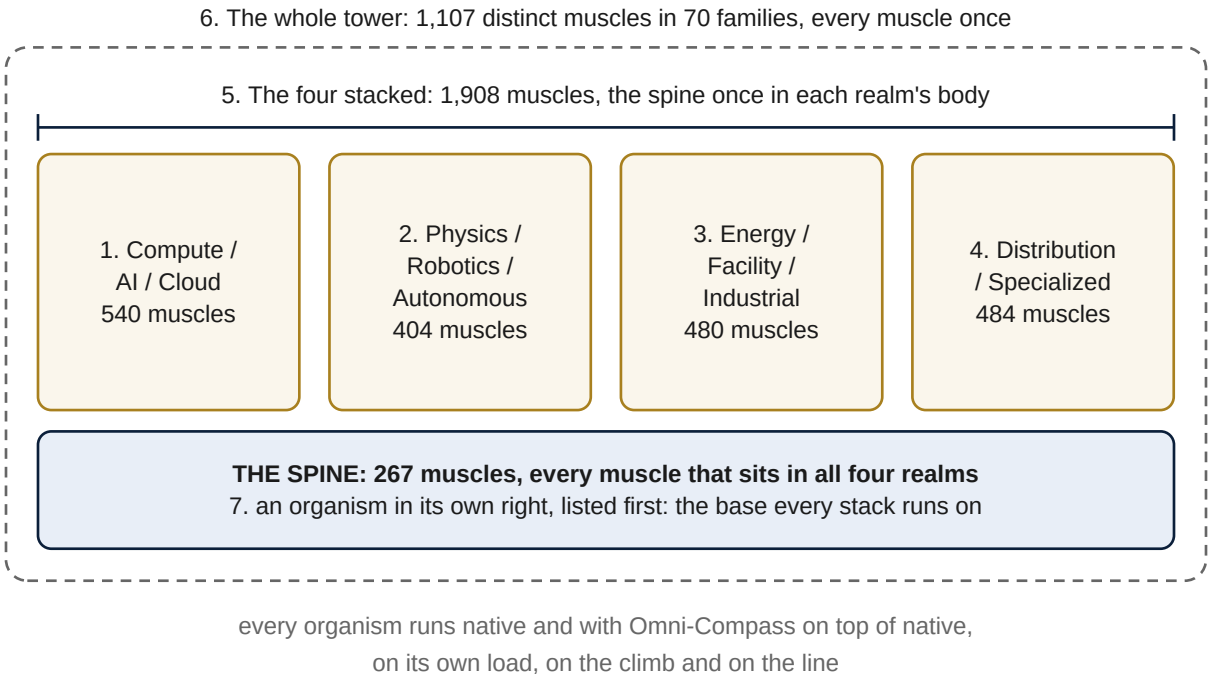


Figure 19. The seven organisms. The spine, the 267 muscles that sit in all four realms, is the base every stack runs on and an organism in its own right, listed first. The four realms stand on it; the four stacked count the spine once in each realm's body (1,908 muscles); the whole tower counts every distinct muscle once (1,107 in 70 families). Each organism runs native and with Omni-Compass on top of native.

The realms differ in what their domain muscles are and in how much slack their native controllers leave. Compute, AI and Cloud is clusters, cards, training and inference pools and cloud capacity, where the knobs are replica targets, machine counts, clock ceilings and power limits, and where native controllers are already good, so the modelled gains are small and the real gains come from the queue and the tail. Physics, Robotics and Autonomous is joints, fleets, vehicles, flight and spacecraft axes, where the knob is a speed or effort limit and the physics decides, axis by axis, whether spending slack as speed saves energy at all; this is the realm the v3 slack gate was written for. Energy, Facility and Industrial is data halls, buildings, batteries, UPS, process loops, feeders and plants, where the knobs are setpoints, units in service, reserves and taps, and where the muscles with the most room are also the ones whose reserves exist for a reason the model does not contain (a UPS held for an outage is never a lever). Distribution and Specialized is networks, storage, databases, commerce, workflows, radio networks, clinical systems and ports, where the knob is usually an admission, a pool or a replica count, and where the real stacks of this manual (the pooler, the broker, the cache) live. Every realm's list, by family and knob, is `docs/REALM_MUSCLES.md` and `docs/REGISTER.md` section 1.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

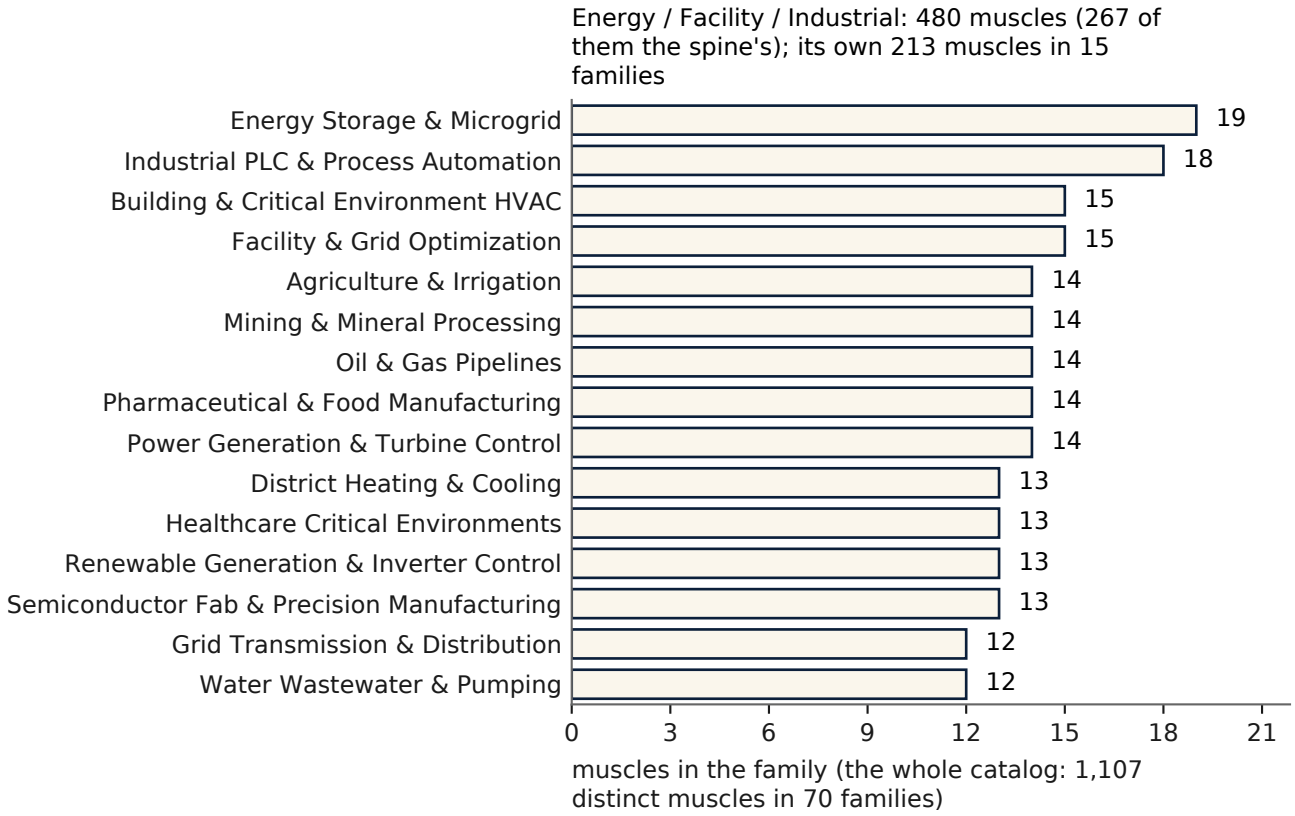


Figure 20. Grid: the families of the Energy / Facility / Industrial realm, its own muscles on the spine, by count.

34.3 What an organism run is

These seven organisms are the benchmark set of the modelled realms. An organism is every one of its muscles stepped on one clock, each from its own plant model, each with its own native controller, and the whole fed one demand trace from one seed. Native runs the organism with every native controller alone. Omni runs the same organism, the same seed, the same demand and the same clock, with the compass law on every knob the verdict allows. Each run produces its own receipt: the organism's work, its energy, its time over its service line, and every knob's hand-back. In code and in the workflows the whole-catalog organisms and the spine are named `tower`, `stack` and `spine`, never by a count, because a count changes with the catalog and a name that carried one would read as a different organism; the old names are read as aliases.

The receipt is read in a fixed order. **Work** first: the organism's output over the window, which must not fall, and which reads LOSS by rule if it does, however little, as the four stacked at 1,000 copies showed on a difference of one part in a million (section 34.5). **Energy** second, and **work per energy** with it, which is the gain the modelled realms are built to show. **Time over the service line** third, the share of steps on which any muscle's service reading was past its line, which is the do-no-harm gauge for the body as a whole. Then **every knob's hand-back**, which must be complete in every run or the run fails. The reading given to an organism (`results/realms_v5/REALMS_V5.md`, one table per engine) is GAIN, STEADY or LOSS by the key

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

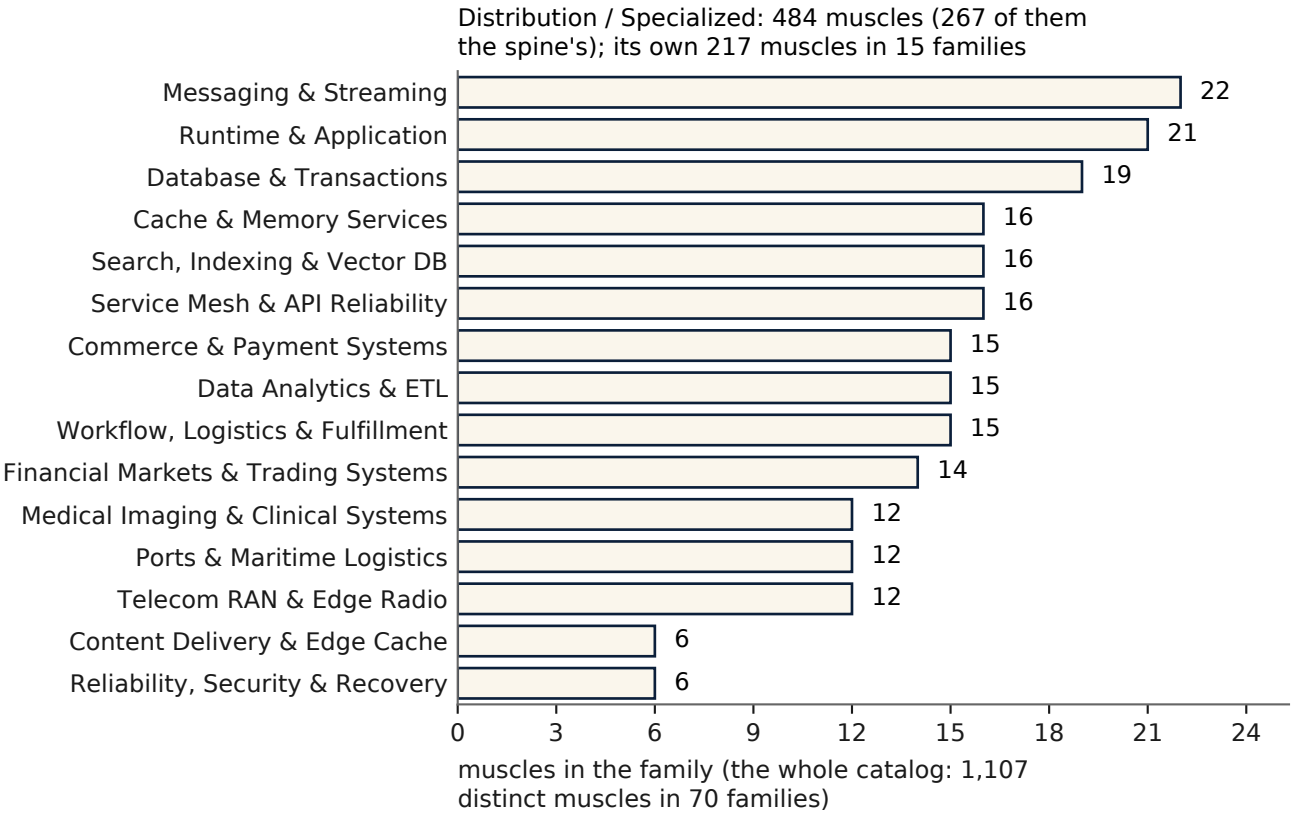


Figure 21. Grid: the families of the Distribution / Specialized realm, its own muscles on the spine, by count.

at the front of this manual: GAIN when its misses fell or its work rose beyond the noise and nothing moved the other way, STEADY inside the noise (the realms table was written before the rule of 24 October and is not among the tables read again by it), LOSS named where it stands; the energy the max took is power given, shown beside the work. Earlier engines' readings, and the words they used, are history (`docs/history/`). What an organism run cannot show is also fixed: it is evidence class S, the plant models are our own, and nothing in it is presented as proof of a real stack. Its use is to show that one law holds across every realm at once, at a size no live benchmark can reach, and to find the rules that then have to earn their place on real software.

34.4 The grid of copies and runs

Every organism runs at **1, 10 and 100 copies** (that many instances of the organism on one clock, so the four stacked at 100 copies is about 190,000 modelled muscles) and at **1, 10 and 100 paired runs** (that many native-and-omni pairs, each from its own seed): the seven organisms × three sizes × three run counts are 63 squares, run modelled and with the real cluster inside, both arms shown in every square. On GitHub's own machines the grid also runs the 1,000 tier, 1,000 copies and nested 1,000 runs (the founder's distinction of 15 October), a cell beyond a free runner's memory or time listed as pending runner capacity and never rented; paid cloud and GPU runs stay at 1, 10 and 100 copies with at most 100 repetitions (the founder's order of 12 October, kept for everything that costs money). Nothing is rented for the grid

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

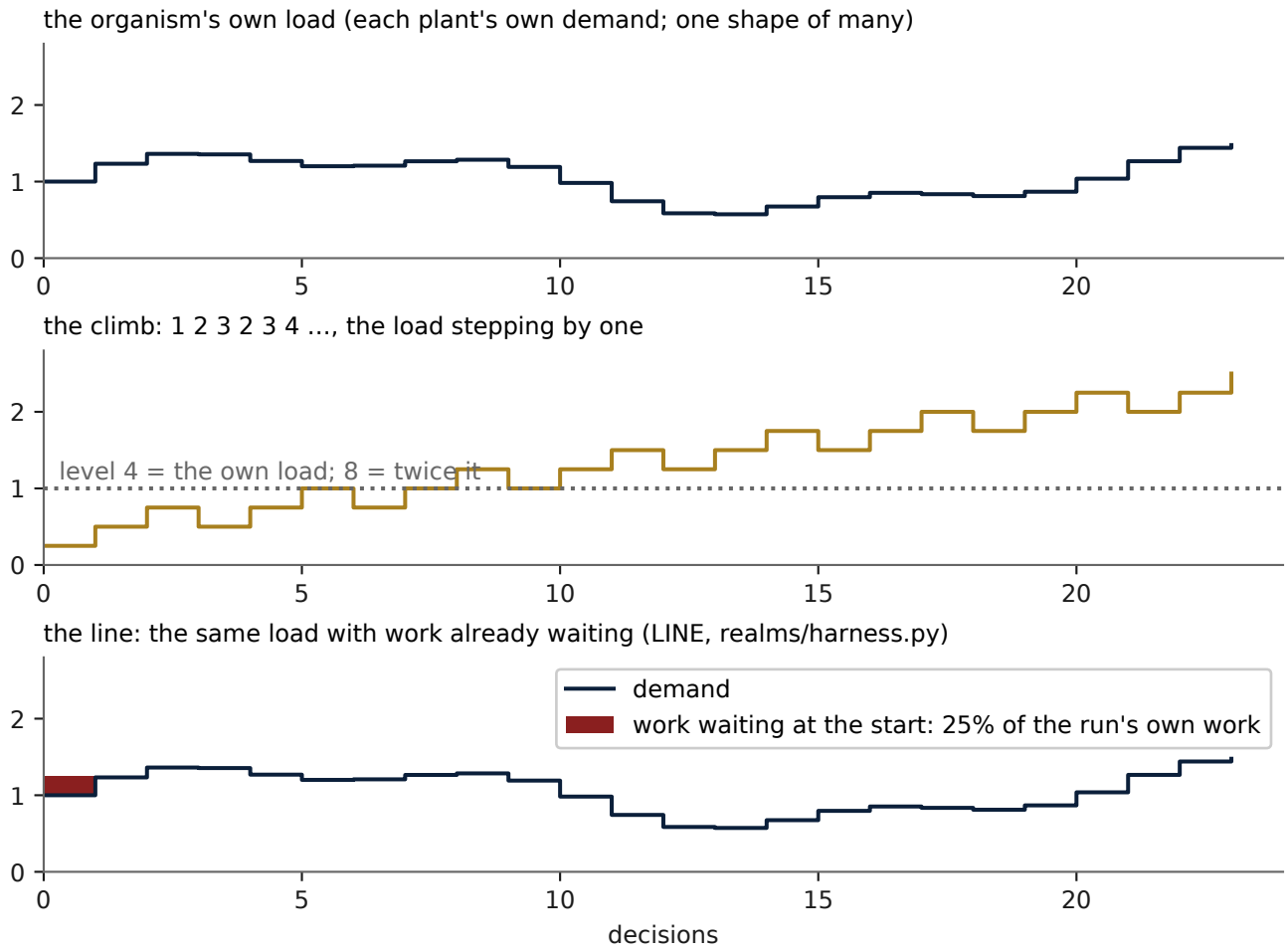


Figure 22. The three loads an organism runs under: its own load (each plant's own demand), the climb (the load stepping by one, 1 2 3 2 3 4 …, level 4 the own load and 8 twice it) and the line, the same load with a quarter of the run's own work already waiting when it starts.

(`scripts/dispatch_grid.py`, `results/grid_v5/`). Earlier engines' grids, which went to 1,000 copies and 1,000 runs, stand as they were run in their own pages (`results/scale/`) and are never read across engines.

The two axes answer two different questions, and a reader should keep them apart. **Copies** is the size of the body the one brain governs: at 100 copies of the four stacked, the law computes a force for about 190,000 muscles from one reading of one clock, and the question is whether it stays coherent, whether the decision time stays inside the decision period, and whether the cluster inside it (section 34.5) is still served on time. **Runs** is the number of independent paired trials, each from its own seed, and the question is statistical: how wide the interval of the paired difference is and whether it moves as the trials multiply. A square at 1 copy and 100 runs tells a reader about the noise of the method; a square at 100 copies and 1 run tells a reader about the scale of the brain; the squares on the diagonal tell both at once. Every square shows the native value and the omni value side by side, never a change alone, because a change without its two ends cannot be checked

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Modelled (class S, our own models): the seven organisms at every size and run count, the change in misses against native. Green GAIN, yellow STEADY, red LOSS, light grey one run (no label yet), white not in the receipt

	1 run	10 runs	100 runs
the spine, 1 copy	not in	not in	not in
the spine, 10 copies	not in	not in	not in
the spine, 100 copies	not in	not in	not in
Compute, 1 copy	1 run −0.181 pp	GAIN −0.191 pp	GAIN −0.204 pp
Compute, 10 copies	1 run −0.139 pp	GAIN −0.141 pp	GAIN −0.142 pp
Compute, 100 copies	1 run −0.135 pp	GAIN −0.135 pp	GAIN −0.135 pp
Physics, 1 copy	1 run −0.230 pp	GAIN −0.242 pp	GAIN −0.249 pp
Physics, 10 copies	1 run −0.113 pp	GAIN −0.114 pp	GAIN −0.116 pp
Physics, 100 copies	1 run −0.107 pp	GAIN −0.107 pp	GAIN −0.107 pp
Energy, 1 copy	1 run −0.078 pp	GAIN −0.096 pp	GAIN −0.108 pp
Energy, 10 copies	1 run −0.077 pp	GAIN −0.086 pp	GAIN −0.085 pp
Energy, 100 copies	1 run −0.087 pp	GAIN −0.086 pp	GAIN −0.085 pp
Distribution, 1 copy	1 run −0.106 pp	GAIN −0.135 pp	GAIN −0.152 pp
Distribution, 10 copies	1 run −0.064 pp	GAIN −0.068 pp	GAIN −0.069 pp
Distribution, 100 copies	1 run −0.058 pp	GAIN −0.059 pp	GAIN −0.059 pp
the four stacked, 1 copy	1 run −0.146 pp	GAIN −0.164 pp	GAIN −0.176 pp
the four stacked, 10 copies	1 run −0.099 pp	GAIN −0.103 pp	GAIN −0.104 pp
the four stacked, 100 copies	1 run −0.098 pp	GAIN −0.097 pp	GAIN −0.097 pp
the whole tower, 1 copy	1 run −0.034 pp	GAIN −0.042 pp	GAIN −0.047 pp
the whole tower, 10 copies	1 run −0.033 pp	GAIN −0.037 pp	GAIN −0.037 pp
the whole tower, 100 copies	1 run −0.038 pp	GAIN −0.037 pp	GAIN −0.037 pp

Figure 23. Grid chart: the seven organisms at every size and run count, modelled (class S), from results/grid_v5/GRID.md. Each cell is one square of the grid, copies × runs, with the change in misses against native and its reading: green GAIN, grey STEADY, red LOSS; a single run carries no label yet; the spine's squares are not yet in the receipt.

against the organism's own records. The grid is rebuilt whole on every engine version, never patched square by square, so that no square carries a result from a different engine than its neighbours (Chapter 59).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

34.5 The organisms with a real cluster inside

The modelled organisms are also run with a **real Kubernetes cluster inside** as one more muscle (`tools/run_kil.py`, workflows `six-kube`, `big-organism`, `big-organism-detached`): the organism's own compute demand drives a real load generator against a real service on a real cluster, and the cluster's watts come back into the organism as heat and load. At 1, 10 and 100 copies this runs on GitHub's machines under protocol 4 (the organism's model on the runner's last core, the kind nodes on the others, the same in both arms; the window set from the heavier arm's step, timed before either arm; the served image pulled onto every worker before each arm, `docs/POD_START_WAIT.md`), and the whole tower and the four stacked at 100 copies also run on eight-core Azure machines, detached from the GitHub job that starts them. No 1,000-copy run is started from 12 October. These runs are evidence class L for the cluster and S for the organism around it, and are labelled "L + S".

Two rules keep these runs honest, and both are written in the preregistration before the run starts. The first is the **clock rule**. The organism must step on the measured window's clock: the row "organism behind its window" says how many seconds after the window its last step ended, and a repetition in which either arm ended more than 5% of the window late is marked **off the clock** in the organism's line, because its last steps saw a cluster whose load schedule had already ended. The row is shown and never judged; at 100 copies on the newest engine the tower's two arms ended 1.8 s and 2.4 s behind a 2,400 s window, a tenth of one percent, and no repetition was off the clock. The second is the **hand-back at 90%**. Omni-Compass releases the cluster's knob at nine tenths of the window and the last tenth is run by native alone in both arms, so that every repetition ends with the proof that the knob was given back and that native resumed where it would have been.

On omni-v5, the engine of its runs, the grid with the cluster inside (`results/grid_v5/GRID.md`, the card version, digest `5648f290`) reads the cluster's time over the response line and the organism's time over the line GAIN in every square of 10 and 100 repetitions; the organism's work per energy fell in 21 of those 36 squares, power given (it falls when more power is given; it bought fewer misses, not more work), and the cluster's 95th and 99th percentiles read LOSS in two squares at 100 copies, where the runner fell behind the organism's clock. Protocol 3 at 100 copies (digest `6093b2b8`, ten paired repetitions each; runs 38464343379, 38462289297, 38462280620) found the cause of those two: the tower's 95th percentile +47.7% and the four stacked's +87.2% (99th: +32.1%), on a four-core machine already about 80% busy and failing about half its requests in both arms, where the max added 40% to 54% replicas that nodes unable to grow could not carry; the four realms read STEADY on every response time. The fix, the replica muscle's live headroom read from the machines under it, belongs to the next version (`docs/omni_v5/NEXT_VERSION_PLAN.md`, section 3), and the eight-core Azure runs show how much of the LOSS the small machine carries. Earlier engines' runs at 1,000 copies on rented machines stand as they were run, in their own tables (`results/live/V1_BIG_ORGANISM.md`, `results/live/V3_BIG_ORGANISM.md`), and are not read across engines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

With the real Kubernetes cluster inside (the cluster class L, the organism class S): the seven organisms at every size and run count. Green: GAIN on n gauges, LOSS on none; red: a LOSS among them; light grey one repetition; white not in the report

	1 run	10 runs	100 runs
the spine, 1 copy	not in	not in	not in
the spine, 10 copies	not in	not in	not in
the spine, 100 copies	not in	not in	not in
Compute, 1 copy	1 run	GAIN on 3	GAIN on 7
Compute, 10 copies	1 run	GAIN on 3	GAIN on 8
Compute, 100 copies	1 run	GAIN on 4	GAIN on 4
Physics, 1 copy	1 run	GAIN on 3	GAIN on 8
Physics, 10 copies	1 run	GAIN on 4	GAIN on 5
Physics, 100 copies	1 run	GAIN on 4	GAIN on 6
Energy, 1 copy	1 run	GAIN on 4	GAIN on 7
Energy, 10 copies	1 run	GAIN on 4	GAIN on 8
Energy, 100 copies	1 run	GAIN on 4	GAIN on 6
Distribution, 1 copy	1 run	GAIN on 5	GAIN on 7
Distribution, 10 copies	1 run	GAIN on 4	GAIN on 7
Distribution, 100 copies	1 run	GAIN on 5	GAIN on 5
the four stacked, 1 copy	1 run	GAIN on 4	GAIN on 7
the four stacked, 10 copies	1 run	GAIN on 5	GAIN on 7
the four stacked, 100 copies	1 run	GAIN on 5	GAIN 5, LOSS 2
the whole tower, 1 copy	1 run	GAIN on 3	GAIN on 8
the whole tower, 10 copies	1 run	GAIN on 2	GAIN on 7
the whole tower, 100 copies	1 run	GAIN on 4	GAIN 5, LOSS 2

Figure 24. Grid chart: the seven organisms at every size and run count with the real Kubernetes cluster inside (the cluster class L, the organism class S), from results/grid_v5/GRID.md. Each cell counts the gauges that read GAIN and LOSS against native in that square; a square with a LOSS among its gauges is red, and it is looked into, never cut.

34.6 The register

The register (`docs/REGISTER.md`) is the one list a referee can audit: every muscle by family (Chapter 1), every benchmark by platform with its native controller, Omni-Compass's knob, its gauges, its result and its file

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(Chapter 34), what is not measured yet, said plainly (Chapter 76), and every open benchmark still to run, in order, one or two at a time (Chapter 12). The program that takes each benchmark to its full size, with what each step costs, is `docs/PROOF_PROGRAM.md`. Every benchmark in this manual appears in the register under the same name.

A row in the register is read left to right as a sentence: the platform, the open benchmark used for it and who publishes that benchmark, the native controller Omni-Compass sits on top of, the knob it moves, the gauges it is judged on, and its state. The state is written in a small, repeated vocabulary so that a reader can scan the column: **queued** (in the audited queue, not yet preregistered), **preregistered and built** (the rules are frozen and the harness runs, no counted result yet), the smoke runs with what each one taught and what was changed before the counted runs, and **A, B, C done** with the three run numbers, the engine version and the readings, losses first where there are any. A row never loses its history: when a benchmark moves from smoke to counted, the smoke record stays in its preregistration, and when a result is superseded by a run on a later engine, the earlier table moves to `docs/history` with its engine named. The register is the only list that must be complete; the manual's Chapter 60 and the dossier (`docs/DOSSIER.md`) are readings of it, and anything in them that is not in the register is an error to be fixed in the register's favour.

The audited queue (Chapter 12 of the register) is the program's promise about what comes next, and it is deliberately short at the front: one or two benchmarks are run at a time, each preregistered, built, smoked and counted before the next is opened, so that no result is ever waiting on a harness that was half built when the runs began. The queue is ordered by what each benchmark adds to the index that the ones before it did not: a new resource traded (memory after machines, a buffer pool after a cache), a new native controller, a new class of evidence.

34.7 Ten organisms, two ways: the twenty scenarios

Beside the seven organisms of the catalog the repository reads three more, each a grouping of catalog rows or of real tables and none a product (`DISCLOSURES.md`, section 9): **the orchestra**, the muscles that stand for the knobs of the governors and orchestrators the founder named (55 muscles, `tools/run_orchestra.py`, the mapping in `tools/governors_page.py`); **the live engine's own organism**, the sixty muscles the live benchmark engine page draws (`docs/engine/export_harness_trace.py`); and **the whole organism** in its three places, the software end, the cloud's wire and the bare metal, composed from the real tables by the product law (`tools/whole_system.py`, section 60.6). Each of the ten is read two ways on the same seeds: native (every plant under its own native controller) and Omni-Compass on native (the max, the factory tune, on top of every native controller). Omni-Compass is never by itself (the founder's order of 25 October): it always sits on top of a native controller. Ten organisms by two ways are the twenty scenarios, written by `python3 tools/scenarios_20.py` into `results/scenarios/SCENARIOS_20.md`, class S, three runs (seeds 3000 to 3002), under the climb for the full hour of 240 decisions of 15 s; their readings are in section 60.10.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 35

The Spine: the Base Every Stack Runs On

The **spine** is the set of muscles every real stack runs on: the 267 muscles of the 1,107-muscle catalog (`realms/catalog_v5.csv`) whose realm list names all four realms. A robot fleet, a trading system, a hospital's critical environment and an AI training pool differ in every muscle of their own trade, and they are alike in this: their work runs on computers that are scheduled, placed, given processors and memory, connected, stored, watched, kept alive, powered and cooled. Those thirteen families are the spine. Since 16 October (the founder's order) the spine is an **organism in its own right**, the seventh, run native and with Omni-Compass on top of native exactly as the other six are, because the base of a stack deserves its own reading before any trade is put on it. The relation is the one between an operating system and the programs that run on it: no program re-implements scheduling, memory or the disk, it stands on the system that does them; no realm's trade muscles re-implement the cluster, the fleet, the power or the cooling, they stand on the spine.

35.1 What the spine is made of

Family	Muscles	Plant model	Native controller class	The knob
Kubernetes Workload Scaling	32	compute pool (server)	the pod autoscaler, KEDA, the operator's rightsizers	replicas, targets
Container Resources	25	compute pool (server)	the vertical autoscaler, the scheduler's requests and limits	capacity, admission
Kubernetes Placement & Scheduling	21	compute pool (server)	the scheduler, descheduler, topology and affinity rules	admission, capacity
Node Fleet & Karpenter-Class Control	24	compute pool (node)	the cluster autoscaler, Karpenter-class provisioners	machines
Cloud VM & Capacity	21	compute pool (node)	the cloud's own scaling sets and capacity reservations	machines
Host CPU & Memory	26	compute pool (CPU host)	the kernel's frequency governor, memory and cgroup controls	capacity, power

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Family	Muscles	Plant model	Native controller class	The knob
GPU Hardware	22	compute pool (GPU)	the card's firmware: clocks, power limit, partitioning	clock ceiling, power cap
Network Routing & Switching	19	compute pool (network)	routing, queueing and buffer controls	capacity, admission
Storage Block/File/Object	20	compute pool (storage)	tiering, caching and throughput controls	capacity, admission
Observability & Telemetry	9	compute pool (server)	sampling, retention and scrape controls	admission
Reliability, Security & Recovery	12	compute pool (server)	disruption budgets, recovery and admission controls	admission
PDU, UPS & Electrical Distribution	18	energy storage (UPS)	the UPS and PDU controllers, their reserves and line limits	power, setpoint
Cooling, Chillers & Thermodynamics	18	thermal zone (data hall)	the chillers, air handlers and their setpoints	setpoint

Two hundred and sixty-seven muscles in thirteen families: 156 capacity knobs, 48 admission knobs, 35 power knobs and 28 setpoints. Every one is listed, with its knob, in `docs/REALM_MUSCLES.md` (the spine first), and explained, wired and carded in `docs/MUSCLE_CATALOG.md` (the manuals' muscle appendix, with every family's wiring recipe from `docs/wiring_recipes.csv`: the native controller, the wire out, the wire in, the restore, and whether the wire has run live in this repository or only on a model). The muscles of the spine's Kubernetes families are the ones the live Kubernetes benchmarks run on real clusters (evidence class L); the spine as an organism is modelled (class S), and nothing in it is presented as proof of a real stack.

35.2 Why the spine is an organism

An **organism** is a set of muscles stepped on one clock, each from its own plant model with its own native controller, the whole fed one demand from one seed, so that the heat the machines make is load on the rooms, the power swing of the body is load on every site's line, and one brain judges every step against every other muscle's cushion. Run alone, the spine is a data center with nothing of any one trade in it: clusters, fleets, processors, cards, network, storage, power and cooling, and the governor on top of their native controllers. Three readings come from it that the realms alone cannot give:

1. **The base before the trade.** Whether Omni-Compass helps the machines every stack runs on, before any realm's own muscles are put on them. A buyer wiring only a cluster and its site wires the spine and nothing else.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2. **What a trade adds.** Each realm's organism is its own families on the spine. Read beside the spine's reading, a realm's reading shows what that realm's trade muscles add to, or take from, the base they stand on.
3. **The same base under four trades.** The four stacked carry the spine once in each realm's body, the tower carries it once in all. The spine run alone is the control for both: the same muscles, the same native controllers, no trade on top.

The spine is run under the same three loads as every organism (its own load; the climb, 1 2 3 2 3 4 …; work waiting in line), is judged by the same rule (work, speed and misses against native, the energy the max takes shown as power given), and has the same two arms and no other: **native**, every native controller alone, and **omni**, Omni-Compass on top of those same controllers, moving only the settings they expose, within their own ranges, and handing every setting back at the end.

35.3 How the spine is wired on a real stack

The spine is wired as any organism is: one governor per body, every shared resource a muscle or a gauge, a muscle with nothing to gain kept wired and watched (the Operator's Manual, chapter 2). The order is the order of the levels in the Operator's Manual, chapter 6, which is the order of the spine's own families: the cluster first (its workloads, then their resources and placement, then the machines under them), then the processors and the cards, then network and storage, then the site under all of them (electrical distribution and cooling), with observability and recovery wired from the first level, because the governor reads through them. Each family's wire is one row of `docs/wiring_recipes.csv`: which native controller owns the setting, where the reading is tapped, where the setting is written, and how it is restored. A realm's own muscles are wired after the spine they stand on, never instead of it.

35.4 The spine in the runners and the pages

The organisms are groupings of catalog rows made in the runners; no engine file names them, so the spine entered the runners without changing the engine that stands (`python3 tools/omni_version.py` prints the same fingerprint before and after). In every runner the spine is `spine`; in `tools/run_scale.py` and the grid conductor (`scripts/dispatch_grid.py`) it is organism `7`, after the six that were numbered before it, so that every grid run already started reads back under its own number. Every table lists the spine first, because the other six stand on it.

Where	The spine
`tools/run_realms_v5.py`	every muscle alone, then the seven organisms under three loads; the spine's row first in every organisms table (`results/realms_v5/REALMS_V5.md`)
`tools/run_scale.py --organisms 7`, workflow `six`	the spine at 1, 10, 100 and, on GitHub's own machines, 1,000 copies, over 1 to 1,000 paired runs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Where	The spine
`tools/run_kil.py --organism spine`, workflows `six-kube`, `aks-metered`, `gke-metered`, `eks-metered`, `cloud-organism`	the spine with the real Kubernetes cluster inside it as one more muscle
`tools/run_hil.py`	the spine with the real card inside it (the GPU Hardware family is a spine family)
`scripts/dispatch_grid.py`	the spine's squares started, read back and never started twice like every other organism's
`docs/REALM_MUSCLES.md`, `docs/MUSCLE_CATALOG.md`	the spine's 267 muscles listed first; each muscle's organisms column names the spine
`results/realms_v5/`, `results/grid_v5/`, `docs/STATE_OF_PLAY.md`	the spine's readings, native against Omni-Compass on top, each named for the engine it ran on

CHAPTER 36

The Four Realms

The muscle tower on modelled plants: **1,107 distinct muscles in 70 families** (`realms/catalog_v5.csv`, the catalog the engine that stands carries), run as **seven organisms**, each native and with Omni-Compass on top of native: the **spine** (`spine`, the 267 muscles every real stack runs on, every one in all four realms; an organism of its own and the base of the other six, as an operating system is the base of every program: `docs/THE_SPINE.md`), each realm's organism (its own families on the spine: Compute / AI / Cloud 540, Physics / Robotics / Autonomous 404, Energy / Facility / Industrial 480, Distribution / Specialized 484), the four stacked on one clock with every duplicate of the spine kept (`stack`, 1,908) and the whole tower, every muscle once (`tower`, 1,107). Every muscle also runs alone on its own plant. Each is run natively (the plant's own controller alone) and with Omni-Compass on top, with meters and receipts, under three loads: the organism's own load, the climb (the load stepping by one, 1 2 3 2 3 4 …) and work waiting in line. Evidence class **S** (modelled: our own models of these machines, never counted in the headline and never presented as proof). The word "simulation" is not used for them; a live stack is class L, a meter class P.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

36.1 Run it

```
pip install -r requirements.txt
python3 tools/run_realms_v5.py --out realms5-out     # the preregister
    ↪ ed run: every muscle alone and the seven organisms, seeds 3000-3
    ↪ 009
python3 tests/test_realms.py                         # the harness's o
    ↪ wn checks (also inside verify.py)

python3 tools/realm_muscles.py                       # docs/REALM_MUSC
    ↪ LES.md: every organism written out, muscle by muscle
python3 tools/muscle_catalog.py                      # docs/MUSCLE_CAT
    ↪ ALOG.md: every muscle explained, wired and carded
```

The run is deterministic: `realms-v5.yml` (GitHub) runs it again and fails if any label or number differs from the table committed in `results/realms_v5/`.

Results, each named for the engine it ran on: `results/realms_v5/REALMS_V5.md` (the tables: every muscle alone over its loads, and the seven organisms under the normal, climb and line loads), `MUSCLES_V5.csv` (one row per muscle and load), `REALMS_V5.json` (every per-seed contrast), `RUN.json` and `SHA256SUMS.txt` (commit and fingerprints). The grid of copies and runs (1, 10 and 100 copies × 1, 10 and 100 runs, modelled and with the real cluster inside) is `results/grid_v5/GRID.md`. The earlier catalogs' results stay as the road to 1.0: v3's in `results/realms/`, v2's in `results/realms/v2/`, v1's in `results/realms/v1/`; nothing is read across engines, and every result runs again on the engine that stands (`scripts/reruns.txt`).

36.2 What is where

File	What it is
`realms/catalog_v5.csv`	the 1,107 muscles: family, name, realm, plant, parameter set, knob (v1's 656: `realms/catalog_v1.csv`; v2/v3's 945: `realms/catalog.csv`)
`realms/muscle_cards_v5.csv`	every muscle's own card: the factory setting it runs on, found on development seeds and kept only when it wins twice and never falls behind native on any seed
`docs/wiring_recipes.csv`	every family's wiring on a real stack: the native controller, the wire out, the wire in, the restore, run live or modelled
`realms/plants.py`	the five plants and their native controllers; the four knobs; the capacity law for one plant
`realms/presets.py`	every parameter, one set per family class

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

File	What it is
`realms/compass_arm_v5.py`	the law on every muscle at its default, the max, and the dial's tune-down (`V5CFG`, `V5SAVE`)
`realms/harness.py`	the arms (native, omni), the organisms, the three loads, the outcome and the label rule
`docs/REALMS_PREREGISTRATION.md`	the question, the rules, the seeds and the development history, frozen before the run
`docs/THE_SPINE.md`	the spine, the seventh organism: what it is, why it is an organism, how it is wired, where it stands in the runners
`docs/REALM_MUSCLES.md`, `docs/MUSCLE_CATALOG.md`	every organism written out muscle by muscle, the spine first; every muscle explained, wired and carded (the manuals' appendix)

36.3 How it relates to the rest

Layer	Realm harness	Elsewhere in this repository
Mathematics (T, V)	uses the engine and governor unchanged	`docs/TRACKING_THEOREM.md`, `verify.py`
Modelled (S)	**this**	the fleet and cluster models, the GPU model, the independent simulators (robot arms, grids, districts, swarms)
Real software (L)	not here	real Kubernetes and the live stacks (`results/live/`, newest engine first on the benefit sheet, `docs/BENEFIT_SHEET.md`)
Physical (P)	not here	the card: every earlier card result is obsolete and runs again on rented cards at one named commit (`docs/GPU_PREREGISTRATION.md`)

A realm result that reads GAIN is a reason to test that knob on a real machine, not a substitute for it. The realms whose knobs can be tested for real first are the compute realm's (Kubernetes on kind, the card), because the tools already exist.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 37

The Realm Muscles

Generated from `realms/catalog_v5.csv` (the catalog the engine that stands carries: 1,107 distinct muscles in 70 families) by `tools/realm_muscles.py`. A **muscle** is one setting on one machine that already has its own control; the one knob Omni-Compass may hold on it stands in brackets after its name. The catalog builds **seven organisms**, each run native and with Omni-Compass on top of native: the **spine**, the muscles every real stack runs on, an organism of its own and the base of the other six (as an operating system is the base of every program; `docs/THE_SPINE.md`); the four realms, each its own families with the spine under them, so the realm counts add to more than the tower; the four stacked; and the whole tower. What the engine does with each knob, which native controller owns it, where the reading is tapped and the setting written, how it is restored and its own card are written family by family in `docs/MUSCLE_CATALOG.md` (the manuals' muscle appendix); the organisms' results are in `results/realms_v5/REALMS_V5.md` and the grid in `results/grid_v5/GRID.md`, each named for the engine it ran on.

37.1 The seven organisms

In the runners the spine is `spine` (organism 7 in `tools/run_scale.py` and the grid conductor, after the six that were numbered before it); every table lists it first, because the other six stand on it.

Organism	Runner name	Muscles	What it is
The spine	`spine` (7)	267	the muscles every real stack runs on, every one in all four realms: the base of the other six
Compute / AI / Cloud	`compute_ai_cloud` (1)	540	its 273 own muscles on the spine of 267
Physics / Robotics / Autonomous	`physics_robotics_autonomous` (2)	404	its 137 own muscles on the spine of 267
Energy / Facility / Industrial	`energy_facility_industrial` (3)	480	its 213 own muscles on the spine of 267
Distribution / Specialized	`distribution_specialized` (4)	484	its 217 own muscles on the spine of 267

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Organism	Runner name	Muscles	What it is
The four stacked	`stack` (5)	1,908	the four realms' organisms on one clock, every duplicate of the spine kept
The whole tower	`tower` (6)	1,107	every muscle of the catalog once

37.2 The spine: 267 muscles, in all four realms, an organism of its own

The thirteen families below are the computers every realm's work runs on and the power and cooling they need: Kubernetes workloads, containers, placement, node fleets, cloud capacity, hosts, GPUs, network, storage, observability, reliability and security, electrical distribution and cooling. Run alone, the spine is a data center with nothing of any one trade in it; under a realm, it is what that realm's own muscles stand on.

37.2.1 Kubernetes Workload Scaling (32; plant: compute_pool, server)

replicas [capacity], hpa_cpu_target [setpoint], hpa_memory_target [setpoint], hpa_custom_target [setpoint], vpa_apply [capacity], scale_to_zero [capacity], keda_threshold [admission], rollout_rate [capacity], max_surge [capacity], max_unavailable [capacity], deployment_pause_resume [admission], rollout_abort [admission], pod_eviction [admission], pdb_policy [capacity], scheduler_queue_priority [admission], api_priority_fairness [admission], hpa_min_replicas [capacity], hpa_max_replicas [capacity], hpa_scale_down_stabilisation_window [setpoint], hpa_scale_up_stabilisation_window [setpoint], hpa_scaling_rate_policies [capacity], hpa_policy_selection [capacity], hpa_tolerance [capacity], hpa_sync_period [capacity], vpa_minmax_allowed_resources [capacity], keda_polling_interval [setpoint], rollout_progress_deadline [admission], rollout_readiness_delay [capacity], unhealthy_pod_eviction_policy [admission], graceful_termination [capacity], probes [capacity], kepler_energy_target_autoscaling [setpoint]

37.2.2 Container Resources (25; plant: compute_pool, server)

cpu_request [capacity], cpu_limit [capacity], memory_request [capacity], memory_limit [capacity], ephemeral_storage [capacity], hugepages [capacity], io_weight [setpoint], pids_limit [capacity], memory_high [capacity], memory_reclaim [capacity], swap_limit [capacity], cgroup_io_max [capacity], cpu_weight [setpoint], cpu_quota [admission], cpuset [capacity], runtime_class [capacity], in_place_resize [capacity], namespace_quota [admission], default_limits [capacity], memory_hard_limit [capacity], memory_protection [capacity], io_latency_target [setpoint], freeze [admission], cruisekube_request_rightsizing [capacity], goldilocks_vpa_request_recommendation [capacity]

37.2.3 Kubernetes Placement & Scheduling (21; plant: compute_pool, server)

node_selector [capacity], node_affinity [capacity], pod_anti_affinity [capacity], topology_spread [capacity], numa_placement [capacity], gpu_topology [capacity], storage_locality [capacity], network_locality [capacity], taint_toleration [setpoint], priority_class [admission], preemption_policy [admission],

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

device_claim [capacity], failure_domain_spread [capacity], scheduler_backoff [admission], gang_admission [admission], deschedule [capacity], scheduler_scoring_strategy [capacity], scheduler_plugin_weights [setpoint], scheduling_gates [admission], cpu_manager [capacity], peaks_power_aware_placement [capacity]

37.2.4 Node Fleet & Karpenter-Class Control (24; plant: compute_pool, node)

node_desired [capacity], node_pool_min [capacity], node_pool_max [capacity], node_provision [capacity], node_cordon [admission], node_drain [admission], node_consolidate [capacity], node_replace [capacity], node_shutdown [capacity], node_power_on [capacity], nodepool_weight [setpoint], disruption_budget [admission], consolidation_policy [capacity], consolidate_after [capacity], expire_after [capacity], capacity_class [capacity], scale_down_utilisation_threshold [admission], scale_down_unneeded_time [capacity], scale_down_delay_after_add [capacity], expander [capacity], node_pool_limits [capacity], kubelet_eviction_thresholds [admission], kubelet_reserved_resources [capacity], max_pods_per_node [capacity]

37.2.5 Cloud VM & Capacity (21; plant: compute_pool, node)

vm_count [capacity], vm_size [capacity], vm_start_stop [capacity], vm_migrate [capacity], instance_family [capacity], asg_min [capacity], asg_max [capacity], spot_mix [capacity], reservation_mix [capacity], zone_selection [capacity], region_selection [capacity], architecture_selection [capacity], accelerator_selection [capacity], boot_disk_class [capacity], placement_group [capacity], interruption_response [capacity], target_tracking_policy [setpoint], scaling_cooldown_warmup [capacity], vm_cpu_cap_by_priority [admission], predictive_scaling [capacity], instance_refresh [capacity]

37.2.6 Host CPU & Memory (26; plant: compute_pool, cpu_host)

cpufreq_min [power], cpufreq_max [power], rapl_package_power [power], uncore_frequency [power], energy_perf_preference [power], cpu_idle_policy [power], memory_bandwidth [capacity], numa_balance [capacity], irq_affinity [capacity], llc_allocation [capacity], memory_pressure_gate [admission], page_reclaim_rate [capacity], transparent_hugepages [capacity], core_online_offline [capacity], thermal_throttle_policy [power], host_power_profile [power], frequency_governor [power], turbo_boost [capacity], dram_power_limit [power], swappiness_reclaim [capacity], core_packing [capacity], per_pod_cpu_frequency_range_and_c_state_access [power], uncore_frequency_per_node [power], ear_cpu_frequency_policy [power], ear_global_energy_budget [admission], k8s_power_manager_core_profile [power]

37.2.7 GPU Hardware (22; plant: compute_pool, gpu)

gpu_allocate [capacity], gpu_power_limit [power], gpu_sm_clock [power], gpu_memory_clock [power], gpu_persistence_mode [capacity], gpu_compute_mode [capacity], mig_mode [capacity], mig_geometry [capacity], gpu_timeslice [capacity], gpu_mps [capacity], gpu_quarantine [admission], gpu_reset [capacity], gpu_thermal_limit [power], gpu_ecc_response [capacity], gpu_job_power_budget [admission], application_clocks [power], gpu_temperature_target [setpoint], sync_boost [capacity], target_clocks [power],

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

climatik_inference_power_cap [power], adaptive_gpu_power_cap [power], nvidia_platform_power_budget [power]

37.2.8 Network Routing & Switching (19; plant: compute_pool, network)

lb_weight [setpoint], route_weight [setpoint], rate_limit [admission], bandwidth_limit [capacity], qos_class [capacity], connection_limit [admission], failover_route [capacity], nic_rate_limit [admission], nic_queue_count [capacity], queue_discipline [capacity], congestion_control [capacity], egress_budget [admission], ingress_budget [admission], ecmp_weight [setpoint], path_selection [capacity], network_isolation [admission], nic_interrupt_coalescing [capacity], nic_ring_size [capacity], port_link_power [power]

37.2.9 Storage Block/File/Object (20; plant: compute_pool, storage)

volume_size [capacity], iops_limit [capacity], throughput_limit [capacity], replica_count [capacity], storage_tier [capacity], volume_placement [capacity], snapshot_trigger [capacity], rebalance [capacity], recovery_rate [capacity], backfill_rate [capacity], compaction_pressure [capacity], cache_allocation [capacity], object_replication [capacity], erasure_code_profile [capacity], storage_admission [admission], degraded_storage_gate [admission], scrub_schedule [capacity], io_scheduler [capacity], disk_power_spin_down [power], lifecycle_tiering [capacity]

37.2.10 Observability & Telemetry (9; plant: compute_pool, server)

collector_memory_limit [capacity], export_concurrency [admission], cardinality_budget [admission], retention_window [setpoint], remote_write_queue [capacity], telemetry_shed [admission], metric_scrape_interval [setpoint], batching [capacity], per_pod_energy_attribution [capacity]

37.2.11 Reliability, Security & Recovery (12; plant: compute_pool, server)

restart [capacity], rollback [capacity], traffic_divert_recovery [capacity], degraded_mode [capacity], actuator_freeze [admission], workload_isolate [admission], credential_rotation_gate [admission], policy_enforcement [capacity], rate_abuse_gate [admission], fault_domain_isolate [admission], backup_trigger [capacity], kill_switch [admission]

37.2.12 PDU, UPS & Electrical Distribution (18; plant: energy_storage, ups)

server_power_cap [power], rack_power_cap [power], pdu_branch_power_limit [power], pdu_outlet_control [capacity], ups_operating_mode [capacity], ups_charge_rate [power], ups_discharge_rate [power], phase_balance [capacity], load_transfer [admission], reactive_power_target [setpoint], voltage_target [setpoint], generator_dispatch [capacity], electrical_isolation [admission], breaker_trip_gate [admission], ups_shutdown_battery_test [capacity], hierarchical_power_budget_row_pdu_switchboard_controllers [power], priority_aware_capping_throttle_low_priority_first [power], power_oversubscription_with_prediction [power]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

37.2.13 Cooling, Chillers & Thermodynamics (18; plant: thermal_zone, data_hall)

supply_air_temperature [setpoint], return_air_target [setpoint], coolant_supply_temperature [setpoint], coolant_flow [capacity], pump_speed [capacity], fan_speed [capacity], chiller_setpoint [setpoint], compressor_authority [power], cooling_tower_fan [capacity], cooling_capacity [capacity], rack_thermal_budget [power], gpu_thermal_envelope [setpoint], cpu_thermal_envelope [setpoint], thermal_workload_migrate [admission], thermal_load_shed [admission], server_fan_speed [capacity], economiser [capacity], chiller_plant_supervisory_control_rl_with_safety_layer [capacity]

37.3 Organism 1: Compute / AI / Cloud: 273 own muscles on the spine of 267 = 540 in its organism

The spine above, and these families of its own.

37.3.1 Work Admission & Demand Shaping (19; plant: compute_pool, server)

api_concurrency [admission], queue_concurrency [admission], queue_backpressure [admission], job_admission [admission], batch_admission [admission], inference_admission [admission], load_shed [admission], priority_gate [admission], tenant_admission [admission], burst_limit [admission], deadline_admission [admission], work_budget [admission], request_queue_limit [admission], retry_admission [admission], background_work_gate [admission], maintenance_work_gate [admission], request_concurrency_limit [admission], adaptive_concurrency_limit [admission], priority_request_queue_under_limit [admission]

37.3.2 OpenShift & Machine API (9; plant: compute_pool, node)

machine_remediation [capacity], machine_health_gate [admission], mcp_pause [admission], mcp_max_unavailable [capacity], node_config_rollout [capacity], operator_reconcile_budget [admission], cluster_version_pacing [capacity], infra_machine_admission [admission], machine_failure_domain [capacity]

37.3.3 GPU Fabric & RDMA (18; plant: compute_pool, fabric)

nvlink_placement [capacity], nvswitch_route [capacity], gpu_fabric_quarantine [admission], rdma_bandwidth [capacity], rdma_route [capacity], nic_affinity [capacity], collective_concurrency [admission], collective_algorithm [capacity], collective_chunk_size [capacity], rank_placement [capacity], gpudirect_policy [capacity], congestion_response [capacity], rail_selection [capacity], fabric_failover [capacity], communication_priority [admission], fabric_isolation [admission], collective_protocol [capacity], nic_hca_selection [capacity]

37.3.4 AI Inference Serving (24; plant: compute_pool, gpu)

model_replicas [capacity], model_route_weight [setpoint], model_load [capacity], model_unload [capacity], model_instance_count [capacity], continuous_batching [capacity], max_batch_size [capacity], batch_queue_delay [capacity], inference_concurrency [admission], inference_max_tokens [capacity],

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

kv_cache_budget [admission], prefix_cache_budget [admission], speculative_decode_budget [admission], model_precision [power], inference_priority [admission], inference_slo_gate [admission], autoscaling_target_concurrency_qps [admission], scale_to_zero_panic_window [setpoint], request_priority_levels [admission], kv_cache_swap_space [capacity], chunked_prefill [capacity], tensor_parallel_degree [capacity], disaggregated_prefill_decode [capacity], lora_adapter_load [capacity]

37.3.5 AI Training (18; plant: compute_pool, gpu_batch)

training_workers [capacity], global_batch_size [capacity], microbatch_size [capacity], gradient_accumulation [capacity], data_parallelism [capacity], tensor_parallelism [capacity], pipeline_parallelism [capacity], expert_parallelism [capacity], checkpoint_interval [setpoint], checkpoint_trigger [capacity], training_preempt [admission], training_gang_size [capacity], elastic_worker_count [capacity], straggler_mitigation [capacity], training_precision [power], compute_comm_overlap [capacity], activation_checkpointing [capacity], dataloader_workers [capacity]

37.3.6 HPC & Distributed Compute (19; plant: compute_pool, batch)

job_slots [capacity], mpi_ranks [capacity], rank_mapping [capacity], node_allocation [capacity], job_walltime [capacity], job_priority [admission], job_preemption [admission], checkpoint_restart [capacity], parallel_io_budget [admission], collective_budget [admission], accelerator_share [capacity], cpu_gpu_ratio [setpoint], memory_per_rank [capacity], scratch_budget [admission], scheduler_fair_share [admission], backfill_policy [capacity], node_power_saving [power], cpu_frequency_per_job [power], partition_limits [capacity]

37.3.7 Distributed Cluster Managers (15; plant: compute_pool, batch)

task_admission [admission], task_priority [admission], resource_reservation [capacity], cluster_quota [admission], task_preemption [admission], binpack_pressure [capacity], spread_pressure [capacity], gang_schedule [capacity], worker_allocation [capacity], maintenance_evacuation [capacity], oversubscription [capacity], resource_reclaim [capacity], queue_fairness [admission], deadline_pressure [admission], scheduler_retry [admission]

37.3.8 Quantum Computing Control Simulation (16; plant: compute_pool, qpu)

qubit_mapping [capacity], circuit_admission [admission], shot_allocation [capacity], circuit_scheduling [admission], gate_scheduling [capacity], pulse_amplitude [capacity], pulse_duration [setpoint], pulse_phase [capacity], pulse_frequency [power], coupling_control [capacity], reset_scheduling [capacity], measurement_scheduling [capacity], dynamical_decoupling [capacity], noise_aware_routing [capacity], error_mitigation_budget [admission], quantum_queue_priority [admission]

37.3.9 Cross-Cluster, Multi-Region & Edge (18; plant: compute_pool, node)

multi_cluster_dispatch [capacity], region_dispatch [capacity], zone_dispatch [capacity], edge_dispatch [capacity], cloud_capacity_class [capacity], workload_migrate_region [capacity], data_residency_gate

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

[admission], latency_region_gate [admission], cost_region_gate [admission], carbon_region_gate [admission], global_failover [capacity], federation_quota [admission], cross_cluster_replication [capacity], edge_offload [capacity], global_admission [admission], edge_site_node_pool [capacity], edge_inference_gpu_clocks [power], edge_sync_interval [setpoint]

37.3.10 Linux Kernel & OS Control (20; plant: compute_pool, cpu_host)

cgroup_cpu_weight [capacity], cgroup_cpu_quota [capacity], cgroup_memory_high [capacity], cfs_scheduler_latency [setpoint], cpufreq_dvfs_governor [power], transparent_hugepages [setpoint], io_scheduler_depth [capacity], numa_auto_balancing [setpoint], dirty_ratio_writeback [setpoint], vm_swappiness [setpoint], tcp_buffer_autotune [capacity], block_nr_requests [capacity], ksm_memory_dedup [capacity], net_rx_coalesce [capacity], cpu_cstate_governor [power], zswap_pool_fraction [setpoint], page_cache_reclaim [setpoint], irqbalance_affinity [capacity], net_qdisc_codel_target [setpoint], hugepage_pool_reserve [capacity]

37.3.11 Serverless & Functions (FaaS) (9; plant: compute_pool, server)

fn_concurrency_limit [capacity], fn_provisioned_concurrency [capacity], fn_scale_to_zero_idle [admission], fn_memory_size [capacity], fn_prewarm_pool [capacity], fn_reserved_concurrency [capacity], fn_burst_admission [admission], fn_snapstart_restore [capacity], fn_timeout_budget [setpoint]

37.3.12 Virtualization & Hypervisor (9; plant: compute_pool, cpu_host)

vcpu_allocation [capacity], memory_ballooning [capacity], vm_overcommit_density [capacity], live_migration_pacing [admission], numa_vcpu_pinning [setpoint], sriov_vf_allocation [capacity], vgpu_partition [capacity], vm_memory_dedup_ksm [capacity], hypervisor_cpu_pinning [capacity]

37.3.13 Gut: Data-plane Intake & Waste (8; plant: process_loop, process)

ingress_intake_pacing [capacity], dead_letter_retry_cap [setpoint], gc_compaction_pacing [capacity], ttl_eviction_waste [setpoint], backpressure_threshold [admission], queue_drain_rate [capacity], retry_budget_window [setpoint], compaction_debt_drain [capacity]

37.3.14 Skin: Perimeter & Admission (5; plant: compute_pool, server)

rate_limit_token_bucket [admission], connection_admission [admission], waf_inspection_sampling [capacity], tls_session_cache [capacity], ddos_shield_admission [admission]

37.3.15 Media Streaming & CDN (14; plant: process_loop, process)

abr_bitrate_ladder_cap [setpoint], player_buffer_target [setpoint], transcoder_worker_pool [capacity], live_transcode_gpu_clocks [power], vod_transcode_batch_power [power], packager_workers [capacity], origin_shield_cache_size [capacity], edge_cache_ttl [setpoint], segment_duration_target [setpoint], cdn_request_coalescing [admission], drm_license_server_pool [capacity], thumbnail_sprite_batch [capacity], media_ingest_admission [admission], encoder_lookahead_frames [setpoint]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

37.3.16 Game Servers & Real-Time Interactive (12; plant: compute_pool, server)

agones_fleet_buffer_size [capacity], agones_fleet_min_replicas [capacity], agones_fleet_max_replicas [capacity], game_server_tick_rate [setpoint], players_per_server_cap [admission], matchmaking_admission_rate [admission], matchmaking_wait_window [setpoint], relay_server_pool [capacity], voice_chat_mixer_pool [capacity], game_node_pool_autoscaler [capacity], replay_render_batch [capacity], cloud_gaming_gpu_clocks [power]

37.3.17 Blockchain & Ledger Nodes (10; plant: compute_pool, storage)

node_db_cache_size [capacity], mempool_max_size [setpoint], max_peer_connections [capacity], script_validation_threads [capacity], block_relay_admission [admission], rpc_work_queue_depth [admission], utxo_cache_flush_interval [setpoint], indexer_batch_workers [capacity], archive_node_storage_tier [capacity], validator_host_power_profile [power]

37.3.18 CI/CD & Build Farms (10; plant: compute_pool, batch)

runner_scale_set_min_runners [capacity], runner_scale_set_max_runners [capacity], build_concurrency_limit [admission], build_cache_size [capacity], test_shard_parallelism [capacity], job_queue_admission [admission], artifact_retention_ttl [setpoint], build_node_cpu_frequency [power], remote_execution_workers [capacity], warm_runner_pool [capacity]

37.3.19 DPU SmartNIC & Programmable IO (12; plant: compute_pool, fabric)

dpu_pf_tx_rate [capacity], dpu_vf_tx_rate [capacity], dpu_sf_tx_rate [capacity], dpu_bandwidth_share [capacity], dpu_qos_group [capacity], sriov_vf_count [capacity], smartnic_flow_steering [capacity], smartnic_offload_enable [capacity], dpu_cpu_budget [admission], dpu_memory_budget [admission], dpu_service_placement [capacity], dpu_failover [capacity]

37.3.20 Kubernetes Dynamic Device Allocation (8; plant: compute_pool, gpu)

dra_device_class_selection [capacity], dra_claim_capacity [capacity], dra_claim_sharing [capacity], dra_device_taint [capacity], dra_device_eviction [admission], dra_binding_readiness [capacity], dra_binding_failure_response [capacity], dra_device_configuration [setpoint]

37.4 Organism 2: Physics / Robotics / Autonomous: 137 own muscles on the spine of 267 = 404 in its organism

The spine above, and these families of its own.

37.4.1 Robotics Motion Control (26; plant: motion_axis, robot_joint)

joint_position [capacity], joint_velocity [capacity], joint_acceleration [capacity], joint_effort [power], cartesian_velocity [capacity], trajectory_speed [capacity], trajectory_acceleration [capacity], jerk_limit [capacity], collision_margin [capacity], force_limit [power], gripper_force [power], locomotion_speed

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

[capacity], steering_angle [capacity], braking_force [power], balance_correction [capacity], trajectory_tolerances [capacity], velocity_acceleration_scaling [capacity], controller_update_rate [capacity], ros2_executor_threads [capacity], ros2_qos_history_depth [setpoint], ros2_control_loop_rate [setpoint], humanoid_balance_margin [capacity], humanoid_gait_speed [capacity], humanoid_joint_torque_limit [power], humanoid_step_height [capacity], manipulator_payload_speed_scaling [capacity]

37.4.2 Robotics Fleet & Warehouse Automation (17; plant: compute_pool, robot_fleet)

robot_dispatch [capacity], task_assignment [capacity], traffic_reservation [capacity], robot_route [capacity], charging_dispatch [capacity], battery_reserve [capacity], elevator_request [capacity], door_request [capacity], conveyor_speed [capacity], agv_speed [capacity], warehouse_zone_admission [admission], robot_quarantine [admission], fleet_failover [capacity], human_safe_stop [capacity], fleet_concurrency [admission], amr_fleet_charge_threshold [setpoint], amr_fleet_active_robots [capacity]

37.4.3 Aviation & Autonomous Flight (28; plant: motion_axis, flight_axis)

throttle_envelope [power], attitude_target [capacity], attitude_rate_target [capacity], velocity_target [capacity], altitude_target [capacity], waypoint_authority [capacity], flight_hold [admission], return_to_home [admission], land_action [admission], mission_admission [admission], geofence_response [admission], failsafe_selection [admission], battery_reserve_threshold [admission], actuator_saturation_envelope [capacity], flight_mode_transition [admission], flight_termination_safe_state [admission], attitude_rate_gains [capacity], horizontal_speed_limit [power], vertical_speed_limits [power], acceleration_jerk_limits [power], px4_mpc_xy_vel_max [capacity], px4_mpc_z_vel_max_up [capacity], px4_mpc_acc_hor [capacity], px4_mc_rollrate_max [capacity], ardupilot_wpnav_speed [capacity], ardupilot_atc_accel_r_max [power], drone_battery_failsafe_reserve [setpoint], swarm_formation_spacing [capacity]

37.4.4 Spacecraft & Flight Software (14; plant: motion_axis, reaction_wheel)

space_command_admission [admission], flight_task_schedule [admission], space_mode_transition [admission], payload_duty_cycle [power], communication_allocation [capacity], space_power_budget [power], space_thermal_command [power], attitude_command_envelope [capacity], reaction_wheel_allocation [capacity], rcs_authority [power], safe_mode_transition [admission], watchdog_recovery [admission], instrument_activation [capacity], fault_isolation [admission]

37.4.5 Rail Traction & Train Control (14; plant: motion_axis, rail_traction)

traction_effort_limit [power], regenerative_braking_share [power], coasting_speed_target [capacity], dwell_time_schedule [admission], hvac_duty_cycle [power], train_auxiliary_power_budget [power], platform_approach_speed [capacity], acceleration_rate_setpoint [capacity], wheel_slip_protection_mode [admission], traction_motor_thermal_derate [power], catenary_voltage_limit [power], timetable_recovery_margin [admission], headway_target [capacity], door_release_hold [admission]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

37.4.6 Marine Propulsion & Vessel Automation (12; plant: motion_axis, marine_propulsion)

shaft_speed_target [capacity], propeller_pitch_limit [power], shaft_power_limit [power], bow_thruster_duty [power], ballast_pump_mode [admission], engine_load_sharing_setpoint [capacity], slow_steaming_speed_target [capacity], auxiliary_engine_staging [capacity], shore_power_mode [admission], hotel_load_budget [power], trim_target [capacity], rudder_rate_limit [power]

37.4.7 Elevators & Vertical Transport (12; plant: motion_axis, elevator_hoist)

hoist_speed_target [capacity], acceleration_limit [capacity], regen_drive_mode [power], standby_power_mode [admission], destination_dispatch_schedule [admission], car_parking_mode [admission], door_dwell_hold [admission], escalator_speed_target [capacity], group_capacity_mode [admission], motor_thermal_derate [power], brake_test_schedule [admission], fire_recall_mode [admission]

37.4.8 Automotive EV & Mobile Powertrain (14; plant: motion_axis, ev_traction)

state_of_charge_limits [power], smart_charging_schedule [admission], traction_torque_limit [power], regen_braking_level [power], battery_charge_limit [power], battery_discharge_limit [power], battery_thermal_target [power], motor_thermal_limit [power], vehicle_speed_envelope [capacity], energy_recovery_target [power], auxiliary_power_budget [power], fast_charge_current [power], fast_charge_voltage [power], vehicle_safe_state [admission]

37.5 Organism 3: Energy / Facility / Industrial: 213 own muscles on the spine of 267 = 480 in its organism

The spine above, and these families of its own.

37.5.1 Industrial PLC & Process Automation (18; plant: process_loop, process)

plc_cycle_authority [admission], machine_cell_admission [admission], valve_position [setpoint], pump_flow [capacity], compressor_speed [capacity], heater_power [power], furnace_setpoint [setpoint], pressure_setpoint [setpoint], temperature_setpoint [setpoint], mass_flow_setpoint [setpoint], tank_level_target [setpoint], conveyor_rate [capacity], feed_rate [capacity], purge_vent_action [admission], controller_mode [capacity], alarm_limits [capacity], safety_interlock_trip [capacity], opc_ua_writes [capacity]

37.5.2 Energy Storage & Microgrid (19; plant: energy_storage, microgrid)

battery_charge_power [power], battery_discharge_power [power], battery_soc_reserve [setpoint], grid_import_limit [power], grid_export_limit [power], pv_curtailment [power], ev_charge_power [power], heat_pump_power [power], electrolyzer_power [power], microgrid_demand_limit [power], peak_shaving [capacity], time_of_use_schedule [setpoint], energy_load_shed [admission], flex_load_admission [admission], storage_dispatch [capacity], microgrid_emergency_reserve [setpoint], volt_var [capacity], volt_watt [capacity], constant_power_factor_reactive_power [power]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

37.5.3 Facility & Grid Optimization (15; plant: energy_storage, facility)

facility_power_budget [power], utility_demand_limit [power], demand_response [admission], electricity_price_gate [admission], carbon_intensity_gate [admission], renewable_dispatch [capacity], generator_start_stop [capacity], site_battery_dispatch [capacity], pue_target [setpoint], cooling_power_budget [power], it_power_budget [power], rack_power_allocation [power], facility_peak_guard [capacity], grid_frequency_response [admission], facility_islanding [admission]

37.5.4 Healthcare Critical Environments (13; plant: thermal_zone, hospital)

operating_room_air_change_setpoint [setpoint], isolation_room_pressure_target [setpoint], patient_room_temperature_setpoint [setpoint], surgical_suite_humidity_target [setpoint], ahu_supply_air_temperature [setpoint], pharmacy_cold_room_setpoint [setpoint], sterile_storage_humidity_setpoint [setpoint], imaging_suite_cooling_capacity [capacity], chiller_plant_staging [capacity], exhaust_fan_capacity [capacity], ward_night_setback_mode [admission], medical_gas_plant_demand_limit [power], emergency_power_load_shed [admission]

37.5.5 Agriculture & Irrigation (14; plant: process_loop, irrigation)

irrigation_pump_speed [capacity], mainline_pressure_setpoint [setpoint], soil_moisture_target [setpoint], pivot_speed_setpoint [setpoint], fertigation_dose [power], greenhouse_temperature_setpoint [setpoint], greenhouse_co2_target [setpoint], vent_position_setpoint [setpoint], grain_dryer_temperature_setpoint [setpoint], cold_storage_temperature_setpoint [setpoint], barn_ventilation_rate [capacity], milking_vacuum_level_setpoint [setpoint], well_drawdown_level_target [setpoint], drip_zone_dispatch [admission]

37.5.6 Oil & Gas Pipelines (14; plant: process_loop, pipeline)

compressor_discharge_pressure_setpoint [setpoint], pump_station_suction_pressure_target [setpoint], line_pack_target [setpoint], pipeline_flow_setpoint [setpoint], compressor_unit_staging [capacity], vfd_pump_speed [capacity], terminal_tank_level_target [setpoint], leak_detection_shutdown [admission], batch_interface_dispatch [admission], heater_outlet_temperature_setpoint [setpoint], drag_reducing_agent_dose [power], valve_position_target [setpoint], cathodic_protection_voltage [setpoint], pressure_protection_priority [admission]

37.5.7 Mining & Mineral Processing (14; plant: process_loop, mill)

sag_mill_load_setpoint [setpoint], mill_speed_target [setpoint], crusher_gap_setpoint [setpoint], flotation_aeration_rate [power], cyclone_feed_pressure_target [setpoint], thickener_underflow_density_target [setpoint], reagent_dose [power], conveyor_speed_setpoint [setpoint], dewatering_pump_level_target [setpoint], ventilation_on_demand_airflow [capacity], stockpile_level_target [setpoint], slurry_pump_speed [capacity], tailings_discharge_shutdown [admission], ore_blend_dispatch [admission]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

37.5.8 District Heating & Cooling (13; plant: process_loop, district_heat)

supply_temperature_setpoint [setpoint], return_temperature_target [setpoint], differential_pressure_setpoint [setpoint], network_pump_speed [capacity], heat_pump_staging [capacity], chp_dispatch [admission], thermal_storage_level_target [setpoint], peak_boiler_heater_output [power], substation_flow_limit [capacity], peak_demand_shed [admission], outdoor_reset_setpoint [setpoint], cooling_network_supply_temperature [setpoint], cooling_storage_level_target [setpoint]

37.5.9 Power Generation & Turbine Control (14; plant: process_loop, turbine)

turbine_speed_droop_setpoint [setpoint], unit_load_setpoint [setpoint], boiler_steam_pressure_setpoint [setpoint], feedwater_level_target [setpoint], agc_participation_limit [capacity], excitation_voltage_setpoint [setpoint], hydro_gate_position_target [setpoint], combustion_air_ratio_setpoint [setpoint], cooling_water_flow_setpoint [setpoint], inlet_guide_vane_position [setpoint], reserve_dispatch_priority [admission], turbine_ramp_rate_limit [capacity], emissions_shutdown [admission], nuclear_rod_position_target [setpoint]

37.5.10 Renewable Generation & Inverter Control (13; plant: process_loop, inverter)

inverter_volt_var_setpoint [setpoint], inverter_volt_watt_setpoint [setpoint], active_power_curtailment [power], plant_reactive_power_target [setpoint], wind_turbine_yaw_offset [capacity], pitch_angle_limit [capacity], rotor_speed_setpoint [setpoint], inverter_frequency_droop_setpoint [setpoint], ramp_rate_limit [capacity], tracker_stow_mode [admission], string_mppt_voltage_setpoint [setpoint], noise_mode_schedule [admission], ice_detection_shutdown [admission]

37.5.11 Pharmaceutical & Food Manufacturing (14; plant: process_loop, batch_reactor)

reactor_temperature_setpoint [setpoint], agitator_speed_setpoint [setpoint], fermenter_dissolved_oxygen_target [setpoint], ph_setpoint [setpoint], pasteurizer_holding_temperature_setpoint [setpoint], freezer_tunnel_temperature_setpoint [setpoint], cip_cycle_dispatch [admission], cleanroom_pressure_cascade_setpoint [setpoint], lyophilizer_shelf_temperature_setpoint [setpoint], chromatography_flow_setpoint [setpoint], steam_sterilizer_cycle_priority [admission], oven_zone_temperature_setpoint [setpoint], refrigerant_compressor_authority [admission], batch_hold_quarantine [admission]

37.5.12 Building & Critical Environment HVAC (15; plant: thermal_zone, building)

setpoint_reset_trim_and_respond [setpoint], duct_static_pressure_setpoint [setpoint], optimal_start_stop [capacity], zone_temperature_target [setpoint], zone_airflow [capacity], ahu_fan_speed [capacity], damper_position [capacity], economizer_position [capacity], boiler_setpoint [setpoint], heat_pump_mode [admission], humidity_target [setpoint], occupancy_ventilation [admission], building_demand_limit [power], thermal_storage_dispatch [capacity], hvac_emergency_mode [admission]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

37.5.13 Grid Transmission & Distribution (12; plant: process_loop, feeder_voltage)

capacitor_bank_switch [capacity], voltage_regulator_tap [setpoint], transformer_tap [setpoint], inverter_real_power [power], inverter_reactive_power [power], feeder_voltage_target [setpoint], feeder_load_transfer [capacity], distribution_storage_dispatch [admission], demand_response_dispatch [admission], frequency_droop_setpoint [setpoint], grid_protection_mode [admission], grid_restoration_sequence [admission]

37.5.14 Semiconductor Fab & Precision Manufacturing (13; plant: process_loop, chamber)

run_to_run_control [capacity], idle_sleep_mode [capacity], tool_job_dispatch [admission], wafer_route [capacity], chamber_recipe_selection [admission], chamber_temperature [setpoint], chamber_pressure [setpoint], gas_flow [capacity], rf_power [power], vacuum_pump_speed [capacity], robot_transfer_rate [capacity], lot_priority [admission], tool_quarantine [admission]

37.5.15 Water Wastewater & Pumping (12; plant: process_loop, water)

pump_speed_water [capacity], valve_position_water [setpoint], reservoir_level_target [setpoint], line_pressure_target [setpoint], flow_target_water [setpoint], aeration_rate [power], chemical_dose_rate [power], filtration_backwash [admission], lift_station_dispatch [admission], leak_isolation [admission], water_demand_shed [admission], water_emergency_shutdown [admission]

37.6 Organism 4: Distribution / Specialized: 217 own muscles on the spine of 267 = 484 in its organism

The spine above, and these families of its own.

37.6.1 Service Mesh & API Reliability (16; plant: compute_pool, server)

circuit_breaker [admission], retry_budget [admission], service_timeout [admission], service_concurrency [admission], connection_pool [capacity], traffic_divert [capacity], traffic_mirror [capacity], canary_weight [setpoint], outlier_ejection [admission], health_threshold [admission], dns_traffic_weight [setpoint], session_affinity [capacity], request_hedging [capacity], fault_injection_gate [admission], service_failover [capacity], load_balancing_policy [capacity]

37.6.2 Database & Transactions (19; plant: compute_pool, database)

db_replicas [capacity], db_memory [capacity], db_cache [capacity], query_concurrency [admission], read_route [capacity], shard_placement [capacity], db_failover [capacity], replication_lag_gate [admission], db_write_throttle [admission], db_pool_resize [capacity], transaction_concurrency [admission], lock_timeout [admission], checkpoint_rate [capacity], vacuum_compaction_rate [capacity], parallel_workers [capacity], background_writer [capacity], synchronous_replication [capacity], buffer_pool [capacity], io_capacity [capacity]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

37.6.3 Messaging & Streaming (22; plant: compute_pool, server)

partition_count [capacity], partition_placement [capacity], producer_quota [admission], consumer_quota [admission], broker_io_quota [admission], message_retention [capacity], queue_depth_limit [admission], consumer_concurrency [admission], producer_batch_size [capacity], fetch_batch_size [capacity], rebalance_rate [capacity], replication_factor [capacity], retry_backoff [admission], dead_letter_divert [capacity], stream_priority [admission], broker_failover [capacity], prefetch [capacity], memory_disk_alarm [capacity], mqtt_broker_max_connections [capacity], mqtt_max_inflight_messages [setpoint], mqtt_session_expiry [setpoint], broker_cluster_nodes [capacity]

37.6.4 Cache & Memory Services (16; plant: compute_pool, server)

cache_size [capacity], cache_ttl [setpoint], cache_eviction_policy [admission], cache_replicas [capacity], cache_sharding [capacity], cache_prefetch [capacity], cache_writeback_rate [capacity], cache_admission [admission], hot_key_isolation [admission], cache_connection_limit [admission], cache_memory_limit [capacity], cache_compression [capacity], cache_warmup [capacity], cache_failover [capacity], cache_flush_rate [capacity], memory_size_threads [capacity]

37.6.5 Search, Indexing & Vector DB (16; plant: compute_pool, server)

index_workers [capacity], index_refresh_rate [capacity], segment_merge_rate [capacity], search_concurrency [admission], search_timeout [admission], shard_count [capacity], shard_replication [capacity], shard_rebalance [capacity], vector_search_k [capacity], vector_batch_size [capacity], embedding_workers [capacity], index_memory_budget [admission], query_route [capacity], hot_shard_isolation [admission], search_admission [admission], circuit_breakers [admission]

37.6.6 Data Analytics & ETL (15; plant: compute_pool, batch)

executor_size [capacity], dynamic_allocation_min [capacity], dynamic_allocation_max [capacity], shuffle_partitions [capacity], shuffle_bandwidth [capacity], etl_concurrency [admission], stage_parallelism [capacity], query_slots [capacity], spill_threshold [admission], cache_fraction [setpoint], batch_interval [setpoint], stream_backpressure [admission], data_locality_wait [capacity], speculation_policy [capacity], analytics_admission [admission]

37.6.7 Runtime & Application (21; plant: compute_pool, server)

worker_count [capacity], thread_pool [capacity], jvm_heap [capacity], gc_budget [admission], connection_pool_runtime [capacity], application_cache_size [capacity], runtime_memory [capacity], async_concurrency [admission], event_loop_workers [capacity], process_count [capacity], request_timeout [admission], background_workers [capacity], runtime_cpu_budget [admission], runtime_io_budget [admission], runtime_restart [capacity], go_runtime [capacity], keepalive_connection_reuse [capacity], dns_resolver_cache_size [capacity], dns_negative_cache_ttl [setpoint], mail_queue_concurrency [capacity], mail_retry_interval [setpoint]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

37.6.8 Reliability, Security & Recovery (6; plant: compute_pool, network)

ids_capture_threads [capacity], ids_ring_buffer_size [setpoint], ids_rule_group_admission [admission], waf_inspection_workers [capacity], siem_ingest_pipeline_workers [capacity], endpoint_scan_scheduling [admission]

37.6.9 Commerce & Payment Systems (15; plant: compute_pool, commerce)

payment_admission [admission], payment_concurrency [admission], payment_retry [admission], payment_timeout [admission], fraud_review_gate [admission], authorization_route [capacity], processor_route_weight [setpoint], transaction_queue_limit [admission], idempotency_window [setpoint], order_reservation [capacity], inventory_hold [admission], checkout_load_shed [admission], refund_queue_rate [capacity], settlement_batch [capacity], payment_failover [capacity]

37.6.10 Workflow, Logistics & Fulfillment (15; plant: compute_pool, workflow)

workflow_admission [admission], workflow_worker_rate [capacity], task_queue_rate [capacity], workflow_retry [admission], workflow_backoff [admission], workflow_timeout [admission], inventory_allocation [capacity], fulfillment_route [capacity], warehouse_queue [capacity], carrier_selection [capacity], dispatch_priority [admission], shipment_batch [capacity], route_replan [capacity], sla_escalation [capacity], compensation_action [capacity]

37.6.11 Medical Imaging & Clinical Systems (12; plant: compute_pool, clinical)

pacs_archive_tier_target [setpoint], dicom_router_concurrency [admission], ehr_application_replicas [capacity], hl7_interface_queue_limit [admission], fhir_api_rate_limit [admission], reconstruction_gpu_workers [capacity], modality_worklist_timeout [admission], patient_monitoring_gateway_capacity [admission], lab_analyzer_batch_window [admission], telehealth_session_admission [admission], clinical_backup_window [admission], imaging_prefetch_priority [admission]

37.6.12 Ports & Maritime Logistics (12; plant: compute_pool, port)

quay_crane_allocation [capacity], yard_crane_fleet_size [capacity], berth_window_admission [admission], truck_gate_rate_limit [admission], horizontal_transport_fleet_size [capacity], reefer_plug_power_budget [power], shore_power_connection_capacity [capacity], rail_mounted_gantry_dispatch [capacity], container_dwell_priority [admission], vessel_arrival_pacing [admission], stacking_height_target [setpoint], equipment_charging_window [admission]

37.6.13 Content Delivery & Edge Cache (6; plant: process_loop, process)

cdn_cache_ttl [setpoint], edge_cache_size [capacity], origin_shield_concurrency [capacity], prefetch_aggressiveness [setpoint], tiered_cache_promotion [capacity], edge_pop_admission [admission]

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

37.6.14 Financial Markets & Trading Systems (14; plant: compute_pool, server)

matching_engine_threads [capacity], order_gateway_sessions [capacity], order_admission_rate_limit [admission], pre_trade_risk_check_workers [capacity], market_data_fanout_threads [capacity], market_data_conflation_interval [setpoint], fix_session_heartbeat_interval [setpoint], kernel_bypass_poll_cores [power], tick_store_write_batch [setpoint], post_trade_settlement_workers [capacity], end_of_day_risk_batch_power [power], colocation_rack_cooling_setpoint [setpoint], backtest_grid_nodes [capacity], surveillance_alert_queue_workers [capacity]

37.6.15 Telecom RAN & Edge Radio (12; plant: compute_pool, ran)

ran_connection_admission [admission], ran_ue_handover [capacity], ran_cell_traffic_steering [capacity], ran_slice_resource_budget [admission], ran_prb_allocation [capacity], ran_scheduler_weight [setpoint], ran_tx_power [power], ran_antenna_tilt [capacity], ran_carrier_enable [capacity], ran_cell_sleep [capacity], ran_du_cu_placement [capacity], ran_fronthaul_budget [admission]

37.7 Organism 5: the four stacked, every duplicate kept, 1,908 muscles

The four realms' organisms above, run on one clock as one body (`stack`; the older name `stack_1226` is read as an alias): every muscle listed under organisms 1 to 4, the spine counted once in each, 1,908 in all.

37.8 Organism 6: the whole tower, all 1,107 muscles once

Every muscle of the catalog once (`tower`; the older name `organism_656` is read as an alias): the spine and every realm's own families, 1,107 in all.

37.9 Organism 7: the spine, 267 muscles

The spine listed first above, run on its own (`spine`): the same muscles, the same native controllers, nothing of any one realm on top of them.

CHAPTER 38

The Domain Map

Every system Omni-Compass can sit on, what it reads, what it pushes, the reflexes that bound it, and where it stands.

38.1 The connector pattern

Omni-Compass is the same engine on every muscle. Each muscle gets one connector with five parts:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Part	Role	Example (compute)
Afferent (sense)	What the muscle reports	load, pending work, power, heat
Engine	Equations (1)-(7) turn readings into state: error E, coherence U, pressure I[U], stress S, bath B	same six-state engine everywhere
Efferent (push)	The equation (2) law and allocation law drive the muscle toward convergence	node count, HPA target, power cap
Reflex (shield)	Hard limits checked before any push	never below pod requests, step limits, power limit
Kill	Control returns to the muscle's own controllers	restore HPA targets, observe only

Fit tiers used below:

- **Direct**: Omni can govern the muscle as a supervisory controller.
- **Supervisory**: Omni governs budgets, schedules and envelopes; the muscle's own real-time controller stays in charge of fast inner loops.
- **Advisory**: regulated or safety-critical; Omni recommends until certified (IEC 61508, DO-178C, ISO 26262, medical and financial regulation).
- **Not Omni's problem**: a different kind of problem; stated so no one claims otherwise.

Status: **Wired** (in code, verified), **Partial**, **Open** (connector not yet built).

38.2 Digital infrastructure

Muscle	Reads	Pushes	Reflexes	Fit	Status
Compute nodes and node pools	load, pending pods, requests	node count, park or power off	request floor, step limit	Direct	**Wired**
Pods via HPA	utilisation, replicas	HPA target	bounds 50-95%	Direct	**Wired**
Power caps	watts, site limit	per-node caps	site power limit (I4)	Direct	**Wired** (harness)
Heat	temperatures	feeds decisions	thermal limit	Direct	**Sensed**
GPUs / accelerators	utilisation, power, temperature, memory	power caps, clocks, partitioning (MIG), packing	temperature, job safety	Direct	Open
CPU power states	per-socket power (RAPL)	frequency scaling, sleep states	latency floor	Direct	Open

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Muscle	Reads	Pushes	Reflexes	Fit	Status
Memory	used vs requested	request right-sizing	OOM margin	Direct	Open
Storage	I/O load, latency, capacity	tiering, placement, volume scaling	durability, replica counts	Supervisory	Open
Network, load balancers	traffic, congestion, latency	routing weights, traffic shift	capacity per link	Direct	Partial
CDN / edge	cache hit, origin load	cache and origin routing	freshness	Direct	Open
Batch and HPC job queues	queue depth, deadlines	admission, priority, time shifting	deadline guarantees	Direct	Open
Multi-cluster / multi-region	load, cost, carbon per region	placement	data residency	Direct	Open
Deployments	release health	pause, roll back	never skip required checks	Direct	Partial (stack harness)
Databases, stateful services	load, replication lag	read replicas, pools	consistency limits	Supervisory	Open
Control plane health	API latency, etcd load	rate of Omni's own actions	never overload the API	Direct	Open
Tenant quotas	usage per tenant	who may grow	fairness bounds	Direct	Open
Hardware health	disk and memory errors	early drain of failing machines	availability floor	Direct	Open
Observability pipelines	data volume, cost	sampling, retention	required audit data	Direct	Open
Security posture	policy blocks	block expansion during holds	I1	Direct	Partial (sensed)

38.3 AI

Muscle	Reads	Pushes	Reflexes	Fit	Status
Training clusters	GPU power, progress, network	power caps, packing, checkpoint timing	never kill a run mid-step	Direct	Open (gpu vessels in harness)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Muscle	Reads	Pushes	Reflexes	Fit	Status
Training power smoothing	synchronised power swings	ramp limits, staggering	grid ramp limits	Direct	Open
Inference serving	request rate, latency, queue, GPU memory	replicas, batch size, GPU sharing	latency SLO	Direct	Open
Model routing and cost	cost per request, quality signals	route to smaller or larger models, token budgets	quality floor	Direct	Open
Model releases	evaluation results, drift, errors	gate, pause, roll back a model	release gates	Direct	Open
Data pipelines, vector databases	backlog, freshness	scaling, scheduling	freshness bounds	Direct	Open
AI agents: containment	actions, spend, resources, network use	CPU, memory, network and spend caps, tool permissions	hard caps, instant kill	Direct	Open
Carbon-aware AI	grid price and carbon intensity	when and where training runs	deadlines	Direct	Open

38.3.1 AI alignment: what Omni can and cannot do

Alignment has two parts, and only one is Omni's.

Part	What it means	Omni's role
Value alignment	The model itself wants and does what people intend: honest, safe, helpful behaviour learned in training	**Not Omni's problem.** A governor outside the model cannot make the model's goals or judgement correct.
Control and containment	Whatever the model wants, it can only act within bounds: resources, permissions, spend, network reach, a kill that works	**Direct.** This is a governance problem: sense what the agent is doing, bound what it may spend and touch, revoke instantly, keep an audit trail.

Omni-Compass can be the boundary layer AI runs inside. It cannot be the thing that makes AI trustworthy. Claims must keep that line.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

38.4 Facilities and energy

Muscle	Reads	Pushes	Reflexes	Fit	Status
Cooling plant	inlet temperatures, chiller load	setpoints, fan curves, chiller staging	inlet temperature limits	Direct	Open
Building HVAC	zone temperatures, occupancy	setpoints, schedules	comfort bounds	Direct	Open
Facility power distribution	PDU and UPS load	limits for the shield	breaker limits	Supervisory	Open
On-site batteries and generators	state of charge, grid state	charge and discharge	reserve floor	Supervisory	Open
Power grids: demand response	grid signals, prices	shift or shed flexible load	contractual limits	Direct	Open
Microgrids, renewable smoothing	generation, storage, load	dispatch	frequency and voltage limits	Advisory to supervisory	Open
EV charging fleets	vehicle needs, grid capacity	charge schedules	departure deadlines	Direct	Open
Water and pumping	demand, pressure, energy price	pump schedules	pressure limits	Supervisory	Open
Grid frequency and protection	frequency, faults	none	certified protection systems	Advisory	Out of scope for control

38.5 Physical operations

Muscle	Pushes	Fit
Manufacturing lines	scheduling, energy, throughput balance	Supervisory
Warehouses and logistics	task allocation, fleet charging	Direct
Robot fleets	task assignment, charging, traffic, safety envelopes	Supervisory (never joint-level control)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Muscle	Pushes	Fit
Traffic signals, transit, rail scheduling	timing and schedules	Advisory (safety-certified)
Ports, shipping	berth and crane scheduling, energy	Supervisory
Agriculture, greenhouses, irrigation	water and climate setpoints	Direct
Mining, oil and gas	energy and scheduling only	Advisory (process safety)
Semiconductor fabs	tool scheduling, energy	Supervisory

38.6 Science, space, biology, finance, health

Domain	What Omni could govern	What it must not touch	Fit
HPC / supercomputers	job queues, power, cooling	none beyond the facility	Direct
Quantum-computer facilities	cryogenics energy, calibration scheduling	qubit control	Supervisory
Satellites and constellations	power, thermal, data budgets, scheduling	attitude and orbit control (flight software)	Supervisory
Spacecraft / astrodynamics	mission resource budgets	guidance and navigation	Advisory
Biology: lab automation, bioreactors	setpoints, schedules, cold chain	living-system interventions without validated models	Supervisory
Financial markets	risk limits, exposure caps, trading infrastructure	making trades, market control	Advisory (regulated)
Hospitals	bed, staffing and energy operations	medical devices, treatment	Advisory
Aviation, automotive, nuclear	monitoring	any direct control	Advisory (certification required)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

38.7 The problems everyone in computing has, and Omni's fit

Problem	What Omni can do	Fit
Data-centre energy growth	right-size, park, cap, shift work in time and place	Direct
Cloud cost overruns and idle capacity	consolidate, right-size, remove padding	Direct
GPU scarcity and low GPU utilisation	pack jobs, share GPUs, schedule by priority	Direct
Heat and cooling limits	govern IT load and cooling plant together	Direct
Grid connection limits for new data centres	power smoothing, demand response, stay under contracted power	Direct
Carbon reporting and reduction	carbon-aware placement and timing, measured energy	Direct
On-call burnout and alert fatigue	hold the levers people turn at night; fewer alerts, each one meaning something	Direct
Autoscalers fighting each other	single authority	Direct (proven in simulation)
Latency and SLO breaches under load	headroom governance, fast scheduling floor	Direct
Noisy neighbours, unfair tenants	quotas and growth permissions	Direct
Hardware failures	early drain on failure signs	Direct
Observability cost and data explosion	sampling and retention governance	Direct
Configuration drift	detect and gate, not rewrite configurations	Supervisory
Cold starts	pre-warming policy	Direct
Data egress costs	placement near data	Supervisory
Runaway AI agents and spend	caps, permissions, kill, audit	Direct
Security misconfiguration, supply-chain attacks	block risky expansion during holds	Partial (not a security scanner)
Model hallucination and bias	none	Not Omni's problem
AI value alignment	none	Not Omni's problem

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Problem	What Omni can do	Fit
Software bugs, technical debt, legacy migration	none	Not Omni's problem
Vendor lock-in	a single governor over mixed clouds	Supervisory

38.8 Order of wiring

1. Digital infrastructure: GPUs, CPU power states, memory, storage, network, job queues, multi-region.
2. Facilities and energy: cooling plant, power distribution, batteries, demand response.
3. AI: training power smoothing, inference serving, model routing, agent containment.
4. Physical operations, then science and space as supervisors.
5. Regulated domains as advisors, until certified.

Each connector follows the same path as compute: wired, benchmarked against today's controls, verified, pre-registered, tested on held-out scenarios, then released.

CHAPTER 39

The Problem Map

Method: the issue trackers of the projects that run the world's clusters (Kubernetes autoscaler, Karpenter, Kepler, Kueue, Volcano, Knative, KEDA, kubernetes/kubernetes), the public industry reports (Cast AI, Datadog, Uptime Institute, IEA) and recent papers (Meta Llama 3, AI-datacenter power stabilisation). Not all of GitHub: a targeted scan of where the money and the failures are. Status: **have** = in this repository and tested; **partial** = built but not proven live; **missing** = not built.

#	Problem (evidence)	Size	Best tool today and its gap	Omni-Compass fit	Status
1	**Idle capacity.** Average Kubernetes CPU utilisation 8%, memory 20%, falling; CPU over-provisioning 69%, memory 79% (Cast AI 2026).	largest money leak in cloud	Karpenter, Cluster Autoscaler, CAST AI: each fixes nodes only	one authority over replicas, requests, nodes and power	**built** (omnilab/right size.py, held-out)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Problem (evidence)	Size	Best tool today and its gap	Omni-Compass fit	Status
	83% of container cost goes to idle resources (Datadog)				
2	**HPA and VPA cannot be used together on CPU/memory**, "not compatible by design" (kubernetes/autoscaler #1726, #2939, #6060, #6247; coordination asked again in #8493, 2025)	blocks right-sizing for most teams	none; official advice is "don't"	exactly what a single authority solves: one engine owns both numerator and denominator	**built** (omnilab/rightsize.py: HPA+VPA vs one authority, held-out)
3	**Karpenter consolidation churn**: nodes replaced every 5-10 minutes for 2-3 generations, busy nodes removed instead of empty ones, one shared timer (karpenter #1851, #1019, #2705, #3046; provider-aws #7146, #8868, #7356, #8536)	outages, wasted boots	Karpenter's own timers and budgets	engine-gated release (equation 2) plus dwell; C-throughput cut node churn 50% in simulation	built in simulation (AKS NAP = Karpenter arm, tuning/vendor_compare.py); live Karpenter opponent next
4	**CPU limits throttle apps** (CFS quota), causing latency and even OOMs (kubernetes/kubernetes #67577, #97445)	latency tails	manual tuning, "remove limits" advice	power-cap muscle must watch latency; our first live run hit this exact problem	have; live runs 3-5 found and fixed the resize permission

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Problem (evidence)	Size	Best tool today and its gap	Omni-Compass fit	Status
5	**Cold starts / scale-from-zero latency** (knative/serving #4902, #14202, #9104; kedacore/http-add-on #219)	seconds of latency per edge	Knative activator, KEDA	pre-warming decided by the engine from its state (anticipation)	**built** (omnilab/coldstart.py, held-out)
6	**GPU utilisation ~5%** (Cast AI 2026); idle GPUs scattered across nodes (volcano #3948, kueue #5243)	the most expensive silicon idle	KAI Scheduler, HAMi, nvshare, Volcano binpack	GPU power-limit muscle (have, calibrated), packing and sharing (missing)	**built** (omnilab/gpupack.py, held-out) + power limit
7	**AI training power swings**: 50-75% of TDP in milliseconds per GPU, ramps over 1000 MW/s at gigawatt scale; operators impose power and ramp-rate limits (SemiAnalysis; Uptime; arXiv 2508.14318, 2606.04869)	grid-connection risk, equipment damage	batteries, fast PSUs, manual GPU power caps	"training power smoothing" muscle: ramp-rate limit through GPU power caps, the engine's bath state B is literally a damped oscillator	**built** (omnilab/powersmooth.py, held-out; basin-only arm recorded)
8	**GPU failures and stragglers**: Llama 3, 419 interruptions in 54 days on 16K H100s, one every ~3 hours; 58.7% GPU-related; slow stragglers	lost training time	Meta's internal tools, checkpointing	hardware-health muscle: sense straggling, drain early	**built** (omnilab/health.py, held-out)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Problem (evidence)	Size	Best tool today and its gap	Omni-Compass fit	Status
	undetected (Meta; Lablup 2605.09370)				
9	**Outages**: 54% cost more than $100,000, 1 in 5 more than $1M; power causes 45% of incidents (Uptime 2025)	direct money	SRE runbooks, AIOps	recovery and physical-SLA results in simulation: recovery 58 to 17 min	have (sim)
10	**Cooling**: industry PUE stuck at ~1.54 for six years (Uptime 2025)	~35% overhead on every watt	DCIM, DeepMind cooling AI	heat afferent have; cooling-plant muscle	**built** (omnilab/cooling.py, held-out)
11	**Energy measurement in VMs is impossible**: no RAPL/IPMI in VMs, Kepler building a model server (kepler #2487, 2026)	nobody can prove savings in cloud VMs	Kepler estimators	same gap we hit	**built** (omnilab/vmenergy.py: meter-calibrated attribution, held-out)
12	**LLM inference autoscaling**: KV cache fills memory before compute; prefill vs decode need separate scaling (vLLM, llm-d)	latency and GPU cost	llm-d, KServe, custom	inference muscle: scale on KV-cache use and queue, not CPU	**built** (omnilab/inference.py, held-out)
13	**Runaway AI agents**: loops and recursive calls, ~$10,000 overnight examples (Dark	cost and safety	per-tool caps, early kill-switch projects	agent-containment muscle: caps, kill, audit	**built** (omnilab/containment.py, held-out); live version next

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Problem (evidence)	Size	Best tool today and its gap	Omni-Compass fit	Status
	Reading; sandbox guides)				
14	**Human error**: failures to follow procedures rose 10 points (Uptime 2025)	outages	runbooks	fewer manual actions: pages and human interventions to zero in simulation	have (sim)

39.1 Results of the build pass (held-out seeds 90001-90030, 30 per muscle, code frozen by SHA first)

Omni-Compass against the strongest native tool in each muscle; + toward its aim, - away from its aim, = tie / not significant.

#	Muscle	Opponent (best native)	Toward its aim	Away from its aim	Tie / n.s.
1-2	right-sizing	VPA in-place + HPA	CPU -3%, memory -11%, p99 -99%, reversals -84%, SLO minutes -57%	p95 +15%, OOM kills (few)	work done
5	cold start	KEDA HTTP	p95 -75%, p99 -58%, delayed >1 s -71%, instance-hours -8%	cold-start count (x8)	
6	GPU packing	Volcano binpack	idle powered GPU-hours -20%, energy -2%	mean wait +16% (seconds)	p95 wait, jobs, fragmentation
7	training power	vendor floor, worst-day setting	burn -9%, throughput loss -44%, 1-s swing -6%	max ramp (inside the grid's limit)	0 grid violations both
8	GPU health	threshold detection	goodput +10%, lost GPU-h -20%, restarts -29%, straggler hours -99%		checkpoint overhead
10	cooling	outdoor-air reset	PUE -2%, cooling -13%, inlet violations -73%		max inlet
11	VM energy	host meter split by CPU	per-VM error -37%, worst VM -34%		total

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle	Opponent (best native)	Toward its aim	Away from its aim	Tie / n.s.
12	LLM inf erence	KEDA on queue	TTFT p95 -18%, SLO breaches -58%, GPU-hours -19%, preemptions -56%		TPOT, TTFT p50
13	agent co ntainme nt	static caps	rogue spend -96%, time to contain -99%, peak sub-agents -59%	false stops (0.6 -> ~2.7/day), honest work -7%	forbidden actions (0 both: RBAC)

Engine vs no-engine: in every muscle the omni_no_engine arm scores close to omni. The mapping from engine state to action carries most of the effect; the evolved dynamics add smoothing. The basin-only arm (power smoothing, the bath equation alone setting the draw) cut ramps 63% but did not hold the grid limit: the human-set limit is required.

39.2 What this says

- The problems with the most money behind them are **idle capacity (1)**, **HPA/VPA coordination (2)** and **GPU idleness (6)**. Problem 2 is the cleanest proof of Omni-Compass's thesis: the Kubernetes project itself says its two autoscalers cannot share a metric, because they fight. A single engine owning both is the answer the issue tracker keeps asking for.
- The problem with the most strategic weight is **AI training power swings (7)**: it is new, it threatens grid connections, and Omni-Compass's own equation (7) is a damped second-order bath, the natural controller for ramp limits.
- The problems we already hit ourselves (4, 11) are the same ones the industry has: good evidence the benchmark is real.

39.3 Build order proposed

1. Request right-sizing muscle (VPA role) under the same engine, benchmarked against HPA + VPA "in conflict".
2. Training power-smoothing muscle (GPU power-cap ramp limits) with a synthetic synchronized-training power trace.
3. GPU packing / fragmentation muscle (with stand-in-gpu-operator or KWOK on the live cluster).
4. Live Karpenter opponent (kwok provider) to test problem 3 head to head.
5. Inference muscle (KV-cache and queue driven), then hardware-health (straggler drain), then agent containment.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

39.4 Sources

- kubernetes/autoscaler issues #1726, #2939, #6060, #6247, #8493
- Karpenter #1851, #1019, #2705, #3046; aws/karpenter-provider-aws #7146, #8868, #7356, #8536
- kubernetes/kubernetes #67577, #97445
- Knative #4902, #14202, #9104; KEDA http-add-on #219
- Volcano #3948; Kueue #5243; KAI Scheduler; HAMi
- Kepler #2487
- Cast AI 2026 State of Kubernetes Resource Optimization; Cloud Native Now on the report
- Uptime Institute annual outage analysis 2025; Uptime global survey 2025
- SemiAnalysis: AI training load fluctuations; Uptime: AI power fluctuations; Power Stabilization for AI Training Datacenters; Source-side mitigation
- The Llama 3 Herd of Models; Tom's Hardware on Llama 3 failures; Lablup 504-GPU report
- vLLM anatomy; llm-d 0.5
- Dark Reading: AI agents and runaway costs

CHAPTER 40

The Seven Organisms and the Benchmark Grid

Omni-Compass is benchmarked on seven organisms, each run native and with Omni-Compass on top of native on the same seed, the same load and the same clock. An organism is a set of muscles stepped together on one clock, each muscle from its own plant model with its own native controller, the whole fed one demand from one seed, so that the heat the machines make is load on the rooms, the body's power swing is load on every site's line, and one brain judges every step against every other muscle's cushion. On the 1,107-muscle catalog the engine that stands carries (`realms/catalog_v5.csv`) the seven are: the **spine**, the 267 muscles every real stack runs on, every one in all four realms, an organism in its own right since 16 October and the base of the other six, as an operating system is the base of every program (`docs/THE_SPINE.md`); the four realms, each its own trade's families on the spine, Compute / AI / Cloud (540 muscles), Physics / Robotics / Autonomous (404), Energy / Facility / Industrial (480) and Distribution / Specialized (484); the four stacked with every duplicate of the spine kept (1,908); and the whole tower with every muscle once (1,107). Earlier catalogs' organisms and counts stay with their own results in `docs/history/`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Every organism runs under three loads: its own load; the climb, the demand stepping by one, up and down (1 2 3 2 3 4 …), past what native can carry, so that the most each arm can do shows; and work waiting in line when the run starts. Every receipt shows work, the time over the service line (misses), the energy the max took as power given beside the work it bought, wear, and whether every knob was handed back; the reading is GAIN, STEADY or LOSS by the key at the front of this manual, judged by what each gauge is for and never by the sign of its number. The spine read beside a realm shows what that realm's trade adds to the base it stands on; the spine read alone shows whether the governor helps the machines every stack runs on before any trade is put on them.

The grid. Each organism runs at 1, 10 and 100 copies of itself on one clock and at 1, 10 and 100 paired runs, modelled and with the real cluster inside, both arms shown in every square; on GitHub's own machines the 500 tier runs too (the ladder of the Python's 500-run Monte Carlo, the founder's order of 25 October), a cell beyond a free runner's memory or time listed as pending runner capacity and never rented, while paid cloud and GPU runs stay at 1, 10 and 100 copies with at most 100 repetitions. The two axes answer two questions. Copies is the size of the body one brain governs: at 100 copies of the four stacked the law computes a force for about 190,000 muscles from one reading of one clock, and the question is whether it stays coherent, whether the decision is made inside the decision period, and whether the cluster inside it is still served on time. Runs is the number of independent paired trials, each from its own seed, and the question is statistical: how wide the interval of the paired difference is and whether it moves as the trials multiply. The grid's readings on the engine that stands are `results/grid_v5/GRID.md` and the standing block of `docs/STATE_OF_PLAY.md`, every square shown, the spine's squares filled by the conductor as they land; the number is small because the native controllers in the models are well tuned and leave little room, it is real within the model, and it is never counted in the Omni index, which is made of real stacks only.

On a GPU machine the real card is wired into each organism as one more muscle of its GPU Hardware family, a spine family, so the card is in the spine and in every organism built on it; its own power meter and its requests are counted in the organism's receipt, kept apart from the modelled plants and also added to them. Real Kubernetes is benchmarked separately on real clusters, and it is also wired into the organisms as one more muscle (`tools/run_kil.py`): at 1, 10, 100 and, on GitHub's own machines, 1,000 copies, the organism's own compute demand drives a real load generator against a real service on a real cluster, and the cluster's watts come back into the organism as heat and load. Those runs are labelled "L + S": the cluster is live software, the organism around it a model.

The seven organisms are models (evidence class S). They show how the law behaves across hundreds of kinds of machine at once and whether its effect is stable as the count and the size grow. Real Kubernetes, the real database, the real broker, the real cache and the real card are the anchors measured on real software and, for the card, a real meter; their results and their losses are in the manual's Chapter 60.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 41

The Metrics Catalog

Every gauge Omni-Compass produces, where it comes from, and what kind of number it is. Three kinds:

- **Measured**: read from a real system (a real Kubernetes cluster, a real GPU's own meter, a real wall plug).
- **Modelled**: computed by a simulation from declared physics or recorded traces. It shows the mechanism, not a measurement.
- **Internal**: Omni-Compass's own state and decisions, recorded in its audit log on every decision, live or simulated.

Every report says which kind each number is. A modelled number is never presented as a measured one.

41.1 Service: what the customer feels

Gauge	Unit	Where	Kind
Response time, mean / 95th / 99th percentile	ms	live Kubernetes (`tools/live_reps.py`), GPU bench (`tools/gpu_reps.py`), GPU sims	measured live; modelled in sims
Failed requests	%	live Kubernetes	measured
Alert minutes: whole minutes whose 95th percentile, failed requests counted over, was over the line (the babysitting tax)	min	live Kubernetes and the cloud (`tools/live_reps.py`)	measured
Requests served / not served	count	GPU bench, GPU sims	measured / modelled
Pod start wait, total and mean	s	live Kubernetes (API server creation-to-Ready)	measured
Pending pods	pod-minutes	live Kubernetes	measured

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Gauge	Unit	Where	Kind
Backlog (SLO breach) minutes	min	site and node exchange sims, stack benchmark	modelled
95th-percentile latency factor	×	site and node exchange sims	modelled
Availability, time healthy, recovery minutes, recovered	share, min	stack benchmark (`benchmarks/stack_benchmark.py`)	modelled
SLA violations (physical, total, backlog, power, heat)	share of time	stack benchmark	modelled

41.2 Work and capacity: what the operator gets for the money

Gauge	Unit	Where	Kind
Work per energy (served requests per kJ)	req/kJ	GPU bench (the preregistered primary outcome), GPU sims	measured / modelled
Work per wall energy (whole machine)	req/kJ	GPU bench with a smart plug (`tools/wall_meter.py`)	measured
Work served	work units	site and node exchange sims	modelled
Work per kWh	units/kWh	node exchange sim	modelled
Energy per served request	J	GPU bench	measured
Energy per core-hour	Wh	live Kubernetes	declared model
CPU used by the app	cores	live Kubernetes (`kubectl top`)	measured
Utilisation (used / allocatable)	share	live Kubernetes	measured
Replicas (HPA), mean	count	live Kubernetes	measured
Pods started	count	live Kubernetes	measured
Worker nodes in service, node-hours, idle node-hours	count, h	live Kubernetes, stack benchmark	measured / modelled

41.3 Energy and power: the bill and the building

Gauge	Unit	Where	Kind
GPU energy (the device's own `power.draw`, integrated)	J	GPU bench	measured

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Gauge	Unit	Where	Kind
GPU mean power	W	GPU bench, GPU sims	measured / modelled
CPU package energy (RAPL)	J	GPU bench, where the machine exposes RAPL	measured
Whole-machine energy at the wall	J	GPU bench with a smart plug	measured
Energy on kind	Wh	live Kubernetes	**declared model** (kind has no meter)
Energy	kWh	site, node exchange and stack sims	modelled
Peak power	kW	site and node exchange sims, stack benchmark	modelled
Site-budget violation minutes (time over the building's limit)	min	site and node exchange sims	modelled
GPU power limit, mean; power-limit writes	W, count	GPU bench, GPU sims	measured / modelled
Power-cap travel	share	stack benchmark	modelled

41.4 Hardware: heat and wear

Gauge	Unit	Where	Kind
GPU temperature, peak and mean	°C	GPU bench, GPU sims	measured / modelled
Heat-over minutes	min	hardware plant sim (`hardware/plant.py`)	modelled
Thermal travel	share	stack benchmark	modelled
Machine round trips, node starts and stops, scale reversals	count	stack benchmark	modelled

Lower power and fewer temperature swings are the conditions for longer hardware life. **No hardware life extension is claimed** until real temperature, duty-cycle and wear records exist.

41.5 Battery equivalent (derived)

A battery holds a fixed amount of energy, so battery life follows directly from work per energy:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- runtime on the same charge, for the same work: × (native energy ÷ Omni-Compass energy)
- work on the same charge: × (Omni-Compass work per energy ÷ native work per energy)

Example: the modelled GPU card at +5.1% work per kJ (`results/gpu/sim/after`) does 5.1% more work on one charge. The same rule turns any measured work-per-energy result into a battery figure. On-site battery banks as an organ of the power budget are designed (`docs/DOMAIN_MAP.md`, "on-site batteries") and not yet built.

41.6 Safety and control: proof it behaved

Gauge	Where	Kind
Writes executed per arm (native 0, watch 0, Omni-Compass n)	GPU bench, live Kubernetes	measured
Settings commands the governor wrote on its own, and the share of its additions decided while the service was still inside its line (the babysitting tax; no person touches either arm)	live Kubernetes and the cloud (`tools/live_reps.py`)	measured
Every arm ended at its start setting (the reset restored it)	GPU bench, live Kubernetes switch drill	measured
Invariant violations (all, and excluding power)	stack benchmark	modelled
Security violations, contradictions, pages	stack benchmark	modelled
Freeze check: code hashes at start and end (counted runs)	GPU bench (`FREEZE.json`, `FREEZE_END.json`)	measured
SHA-256 of every raw file	every live run	measured
The seal: every Python/C++ twin unchanged since proven equal	`results/SEAL.json`, `verify.py`	checked on every build

41.7 Internal: Omni-Compass's own state, on every decision (audit log)

Field	Meaning
`E, U, I_U, S, B, B_dot`	the six-state engine: energy, coherence (health), unmet-need integral, stress, redistribution, its rate
`state_observed`, `state_projected_next`, `prediction_error`	what the device said, what the engine projected, and how far off the last projection was
`requested_cap`, `granted_cap`, `shield_bound`, `want_w`	what the engine asked for, what the shield allowed, which bound decided it, the limit written

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Field	Meaning
`admissible`	whether the engine was permitted to change anything this decision
`speed_lock` (`ratios`, `line`, `aim`, `rate`)	the speed lock's reading against the run without Omni-Compass
`convey` engaged / released	when idle CPU is conveyed to serving pods, and when it is given back
`authority`: `calm` and its scalars (`kappa, h, sigma, nu`), `execute`, `contract` per organ, and the machine organ's own view	how settled the engine is, whether it may act at all, and which organs may give capacity back this decision (Kubernetes controller; `omnicompass/nervous_system.py`)
compass readings	the direction the engine reads the whole system to be moving (`omnicompass/compass.py`)
`write`, `would_write`, `why`	every command sent (or, in watch mode, withheld) and its reason
`snapshot`, `restored`	every setting read at start, and its read-back after the reset

41.8 Where each report lives

Report	Command	Output
Live Kubernetes, paired	`benchmark-reps.yml` (commit with `[reps]`)	`LIVE_REPS.md`, `results/live/`
GPU bench on a real card	`sudo bash scripts/gpu_paired.sh`	`results/gpu/run-*/GPU_REPS.md`
GPU physics sim	`python3 tools/gpu_physics_sim.py`	`results/gpu/sim/`
CPU-then-GPU speed lock sim	`python3 tools/gpu_pipeline_sim.py`	`results/gpu/sim/pipeline/`
Site power exchange (GPU groups)	`python3 hardware/site_exchange.py`	`results/hardware/SITE_EXCHANGE_*.json`
CPU + GPU on one budget	`python3 hardware/node_exchange.py`	`results/hardware/NODE_EXCHANGE_*.json`
Full stack benchmark and every check	`python3 verify.py`	`results/`, `VERIFICATION: PASS`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

PART FIVE

The Harness and the Wiring

The universal plug, the adapters, the wire check, and the step-by-step work of wiring Omni-Compass onto a running stack.

CHAPTER 42

The Plug, the Adapters and the Wire Check

Think of a high-end car stereo: one head unit, one standard plug on its back, and an adapter harness for each make of car that matches the car's factory plug. You never cut a factory wire, and when you pull the stereo, the car works as it did. Omni-Compass is wired the same way.

42.1 The plug (one design for every muscle)

Each plug has as many channels as its muscle has knobs. Every channel declares its cover (range), its direction (which way is "more"), its units and how fast it may move. Every plug keeps the same contract (`omnicompass/compass_law.py`, `Plug`):

1. **attach** - read the knob once, before any write: the snapshot. It never moves afterwards.
2. **read** - the service reading, into the compass.
3. **write** - one value, clipped to the cover, then read back from the device.
4. **one writer** - if the knob is found at a value Omni-Compass did not write, someone else owns it: Omni-Compass stops writing and leaves that value alone.
5. **restore** - on stop, the knob returns to the snapshot (where Omni-Compass found it, not the last value it wrote). For muscles whose native controller moves the knob itself, restore hands control back.

42.2 Adapters (one per standard plug)

Omni-Compass speaks each industry's standard control interface through an adapter. Status is stated plainly:

Standard	Muscles	Adapter status
Kubernetes API	HPAs, deployments, pods, nodes	**built** (`omni_controller/controller.py`, `muscles.py`)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

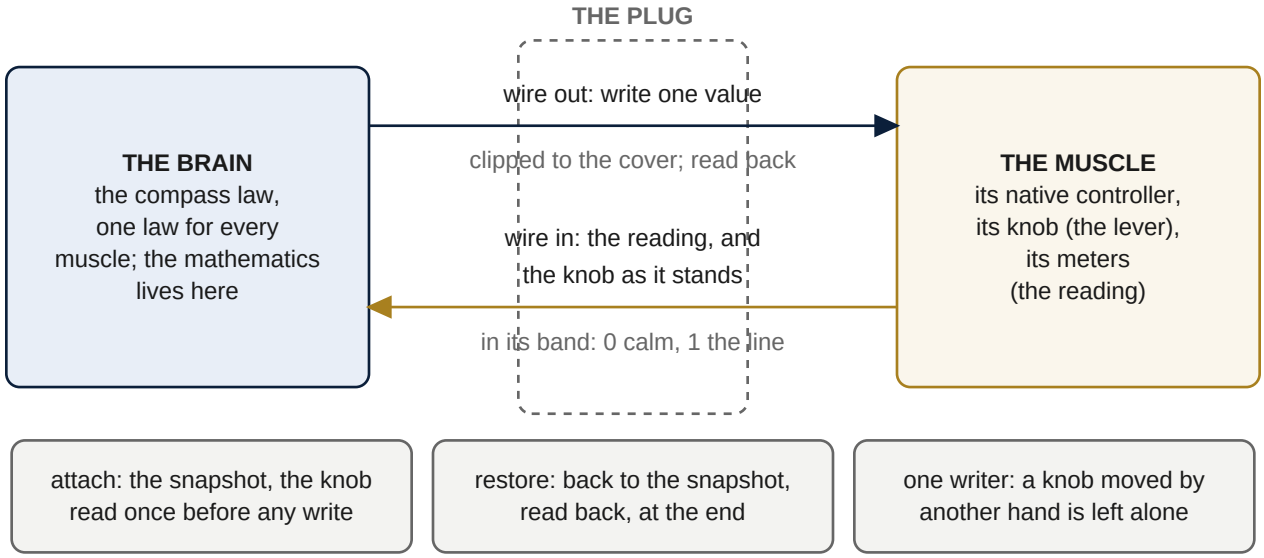


Figure 25. The two-way plug. One wire out carries the governor's write, one value per decision, clipped to the knob's cover and read back from the device; one wire in carries the muscle's reading, placed in its band, and the knob as it stands. The snapshot taken at attach, before any write, is the restore point, and a knob found where the governor did not put it belongs to another writer and is left alone. The brain holds the mathematics; the plug holds nothing but the contract.

Standard	Muscles	Adapter status
NVIDIA NVML / `nvidia-smi`	GPU clock ceiling, power limit	**built** (`omni_controller/gpu_compass.py`, two wires; `gpu_governor.py`, one wire)
Linux cpufreq, RAPL	CPU frequency ceiling, package watts	**built** (`--cpufreq-policy-root`, `--rapl-cmd`)
Site meter, building controller by command	site watts, supply-air setpoint	**built** (`--power-cmd`, `--cooling-cmd` command templates)
Cloud node groups (Karpenter, Cluster Autoscaler, MachineSet)	machines	**built** through `--node-scale-cmd` templates
BACnet, Modbus, SNMP	chillers, air handlers, PDUs, UPS	designed; today reached through the command templates above
OPC UA, EtherNet/IP, PROFINET, EtherCAT, ROS 2, CAN	PLCs, drives, robots, vehicles	designed; modelled in the realm harness

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Standard	Muscles	Adapter status
IEC 61850, DNP3, OpenADR, IEEE 2030.5, SunSpec, OCPP	substations, batteries, inverters, chargers	designed; modelled in the realm harness

The engineer's job for one muscle is three things: which adapter, the address of the knob, and its safe range.

The plug contract, line by line, and why each line is there. *Attach* reads the knob once before any write and keeps that value as the snapshot. It is read once and never again, because a snapshot that moved with the knob would restore to wherever the governor last left it, and the whole point of the restore is to return to where the operator had it. *Read* takes the service reading (a response time, a temperature, a queue, an error) into the compass as a position in its band; the plug declares the band, the law does not guess it. *Write* takes one value, clips it to the cover before it leaves the plug, sends it through the muscle's own interface, and then reads the lever back from the device and records both the sent and the taken value; a device that took something other than what was sent (a clock that rounds to 15 MHz steps, a pool size the pooler capped) is recorded as it is, not as it was asked. *One writer* compares the lever as found at every read with the value last written; a lever found elsewhere has another owner, and the plug raises `ForeignWriter`, after which the governor leaves that lever alone for the rest of the run and says so in its audit and its exit code. *Restore* writes the snapshot back on stop and reads it back; a restore that does not read back is an error (exit 3), never a silent success. The same five lines are implemented for Kubernetes objects (annotations hold the snapshot on the object itself, so a watchdog can restore from the object without the governor's memory), for `nvidia-smi`, for PgBouncer's console, for Redis's `CONFIG SET`, for the consumer group and for the simulators' knobs, and `tests/test_*` prove each against a stand-in device.

42.3 Power-up order

Read-only first; then take the knobs, the least consequential first; on stop, hand them back in reverse order. In the Kubernetes controller the order is: the latency feed and the cluster's own metrics (read); the HPA target (a setting the HPA accepts and corrects within a minute); the pods' CPU limits (conveyance, in place); the machines (the slowest and most consequential, through the release gate and the verdict). On stop the machines are woken first, then the limits, then the target, so the cluster is never left with fewer machines than its pods need while a setting above them is still being handed back.

42.4 The wire check (mandatory before any write)

For the GPU it is one command, `sudo python3 tools/gpu_wire_check.py --gpu 0`, and `scripts/gpu_rented_run.sh` runs it first and stops if it fails:

Step	It proves
1 read	every meter answers: watts, temperature, utilization, limit, clock, top clock
2 snapshot	the start limit is recorded once

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Step	It proves
3 up wire	with the card busy, a ceiling below its own busy clock pulls the clock down under it; reset, the clock comes back above it
4 down wire	a lower power limit is reported back exactly; the start limit is reported back exactly
5 restore	clocks reset and the start limit read back
6 stop	the governor, stopped by its signal, hands both wires back
7 other writer	a limit set by someone else while the governor runs is left alone, and the governor exits 5

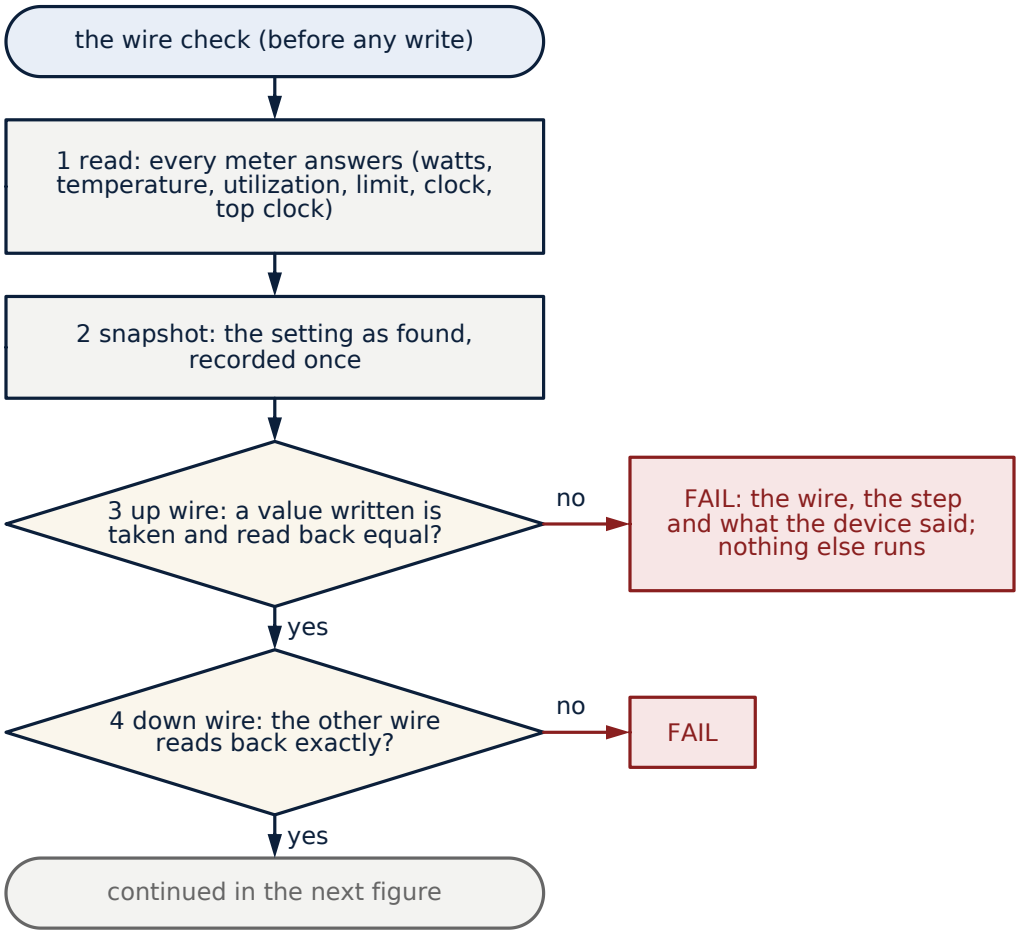


Figure 26. Flowchart: the wire check, steps 1 to 4. Before any write: every meter answers; the setting as found is recorded once as the snapshot; a value written on the up wire is taken and read back equal; the down wire reads back exactly. A step that fails stops everything with the wire, the step and what the device said; the chart continues in the next figure.

Any failure prints the wire, the step and what the device said, and nothing else runs. That turns a wiring fault from guesswork into one named line to fix.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

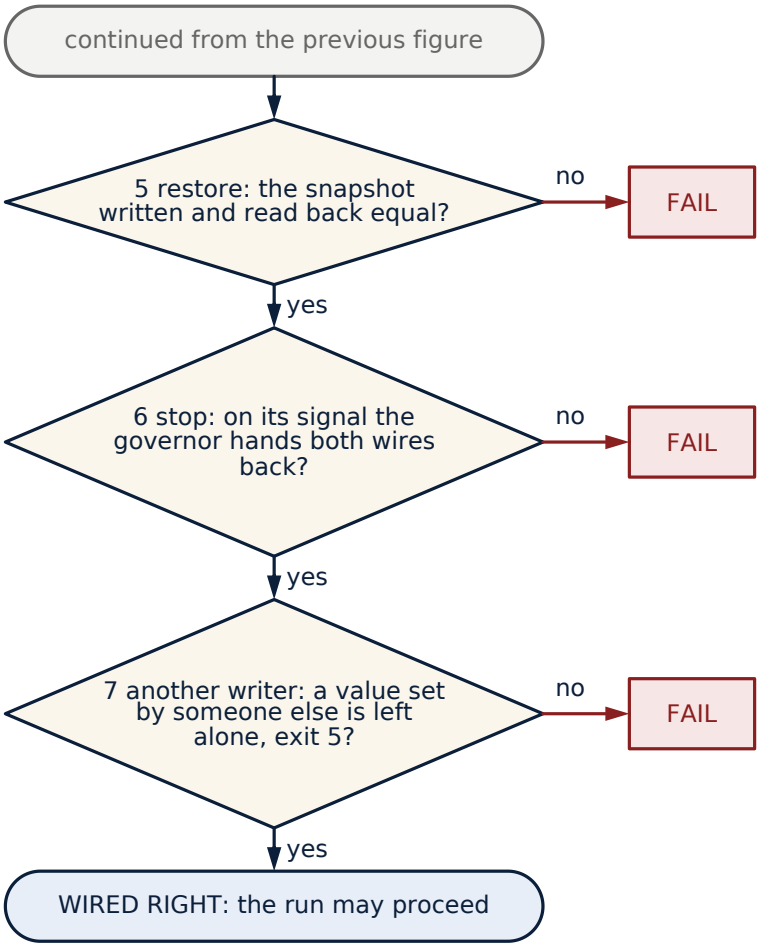


Figure 27. Flowchart: the wire check, steps 5 to 7 and the verdict. The snapshot written back reads back equal; on its stop signal the governor hands both wires back; a value set by someone else is left alone (exit 5). Only then is the muscle wired right and the run may proceed.

42.5 Wired right or wired wrong

The wire check proves the wires move. It does not prove the governor reads the right thing. A correct installation leaves signatures in its own receipts; a wrong one leaves others. Check them on your first paired runs, before you believe any number, good or bad.

Check	Wired right	Wired wrong, and what to look at
Watch arm against native	equal on every gauge (watch writes nothing)	different: the probe, the load or the machine differs between arms, not Omni
Requests served, failed requests	equal to native	fewer served or more failed: a lever is cutting capacity; check its cover and its sign

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

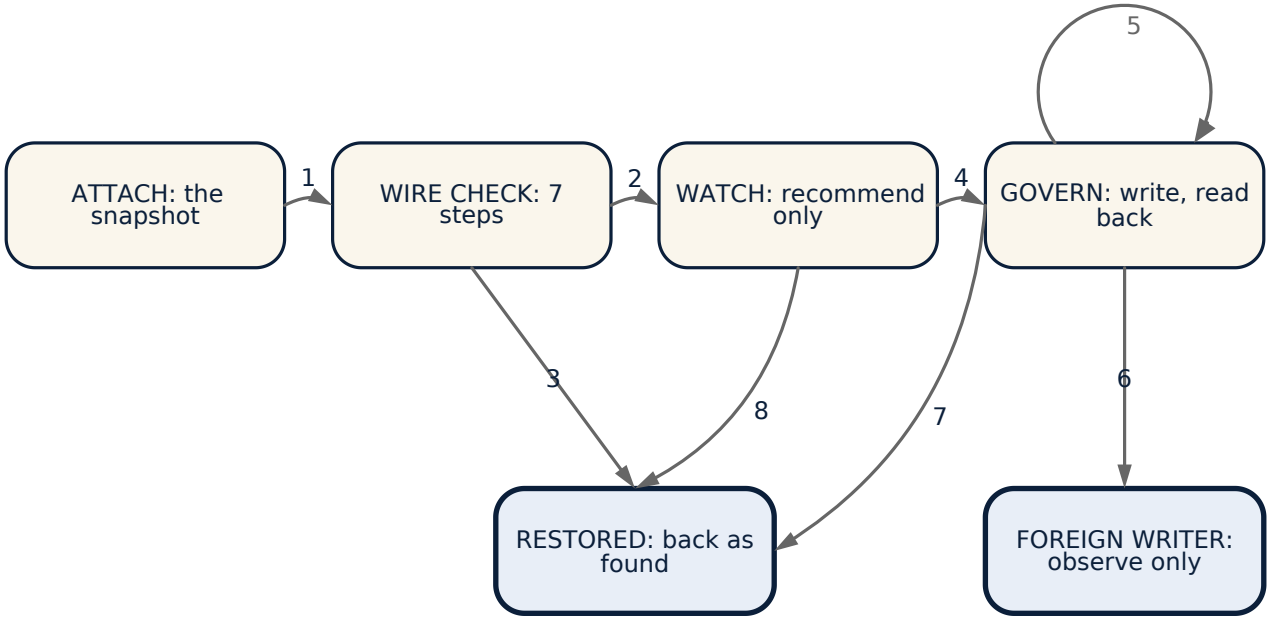
Check	Wired right	Wired wrong, and what to look at
p95 and p99 response time	at native or faster	slower: the reading or the floors are wrong (rows below)
GPU: the card's clock while busy	at or above the clock the card reaches on its own	below it: the speed floor is missing or the lid is under the card's own draw
GPU: the lid while busy	at or above the card's own busy draw	at the envelope floor while busy: the lid is sized from a curve, not from the card's own meter
GPU: credit per write (`GPU_REPS.md`)	compass decisions pace the slack, race decisions cover the bursts; fail-up is rare	fail-up in most decisions: being busy is being read as a breach, or the service line is set too low
Kubernetes: machines given back	through the release gate, one per decision, with no pod waiting	machines given back while pods wait, or none at all while the service is far inside its band: check the latency feed, `--slo-ms` and the release gate's reasons in the audit
Kubernetes: the reset	every HPA target, replica range, CPU limit and worker back to native, no record left	anything left: an earlier run or another controller wrote the same objects
Every lever after the run	at its snapshot	not at its snapshot: another writer, or a restore that failed (exit 3): restore by hand and investigate
The feed itself	fresh samples every few seconds	stale or empty: Omni reads it as blind and fails up; fix the feed, not the governor

The rule. If any row reads "wired wrong", the result of that run says nothing about Omni-Compass: fix the wiring, run the wire check and the watch arm again, and only then compare. The first real card run is the worked example: its receipts showed the card's busy clock under its own (736-768 MHz against 861-889), the lid at the 105 W floor under its own 135 W draw, and fail-up in 46% of decisions. Each of those is a row in this table.

42.6 The wire check on the other stacks

The GPU has a one-command wire check because its wires are the hardest to get right. The other stacks carry the same proof in a different place: every Kubernetes arm records `kubectl auth can-i` receipts for what the governor's identity can and cannot do (and fails if it can do anything it must not), checks that the cluster holds nothing but the harness, and ends with the reset check that every HPA target, replica range, CPU limit

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.



1 the snapshot taken · 2 WIRED RIGHT · 3 FAIL: nothing written, the knob as found · 4 the paired trial passed · 5 every decision: clamp, gate, write, read back, receipt · 6 the knob found where the governor did not put it · 7 stop: reset, OFF, the lease · 8 stop before the trial passed

Figure 28. State machine: the life of one muscle under the plug. Attached with its snapshot, checked by the seven-step wire check, watched in recommend-only mode until the paired trial passes, then governed, one clamp, gate, write, read-back and receipt a decision; a knob found where the governor did not put it moves the muscle to observe only, and every stop restores it to the setting as found.

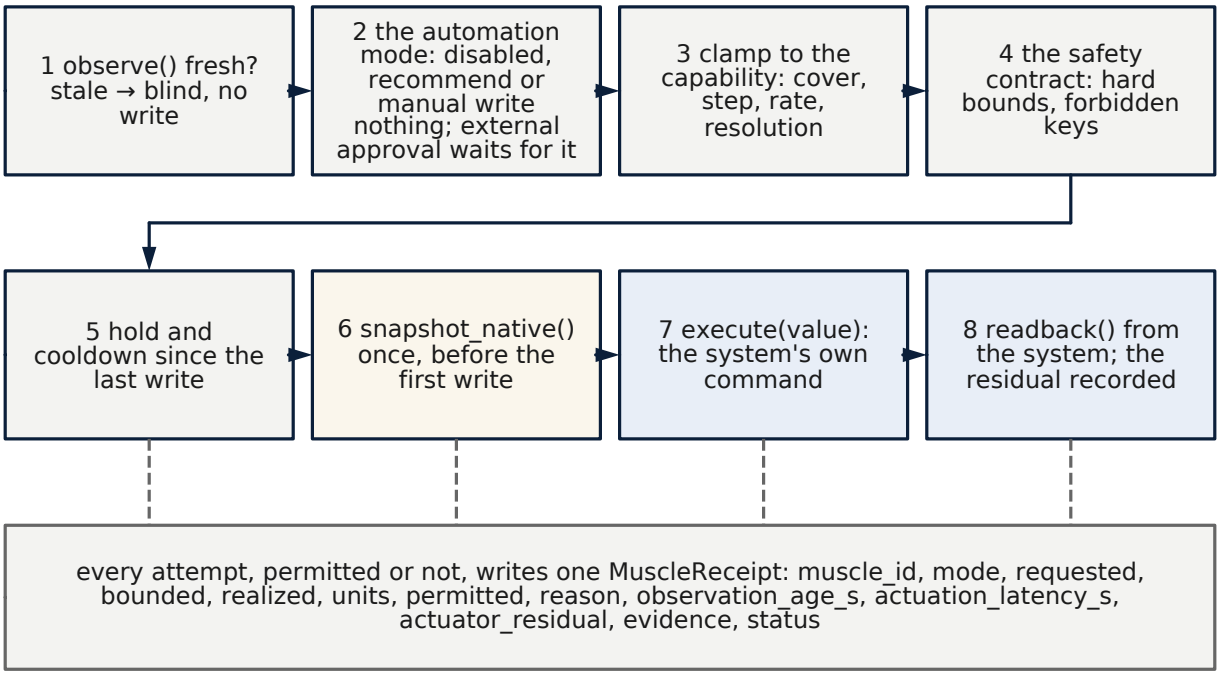
and worker is back at native with no record left; every database, broker and cache arm reads the knob back after every write and hands it back at the end, and the report says "handed back: yes" or the run is invalid; every simulator arm checks that every override was handed back. The watching arm, where it is run, is the wire check of the reading: a watching governor that disagrees with native on any gauge has found a difference in the probe, the load or the machine, not in Omni-Compass.

42.7 Coding a new muscle: the muscle SDK

A muscle not in the catalog (Appendix G), or one whose native controller speaks an API no adapter covers, is coded on the muscle SDK (`omnicompass/muscle_sdk.py`, printed in full in the Source appendix), the contract every adapter obeys: **observe, bound, request, execute, read back, receipt, restore**. The SDK adds a muscle and never a brain: the chain is plant telemetry → observation adapter → the same engine state → bounded authority → shield → actuator → read-back → plant, and the governing state (E, U, I[U], S, B, Ḃ) is untouched by any muscle added (Chapter 12). The governor proposes a setting; the SDK's runtime earns the authority to write it, verifies the write and writes the receipt. An engineer codes five functions and declares three contracts; everything else (the clamp, the gate, the snapshot, the restore, the receipt) is the runtime's.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The five functions (the `MuscleAdapter` protocol, or `FunctionalMuscle` for a small connector): `observe()` returns the reading with its timestamp and quality (the wire out); `snapshot_native()` records the setting as found, once, before the first write; `execute(value)` writes one setting through the system's own command or API (the wire in; **WRITES**); `readback()` reads the setting back from the system, never from memory; `restore_native(snapshot)` puts the setting back as found. The adapter owns the vendor's API; the brain never sees it.



The five functions an engineer codes: observe, snapshot_native, execute, readback, restore_native. The three contracts declared: capability, timing, safety. The runtime does the rest, in this order, for every command.

Figure 29. The muscle SDK's runtime: what happens to one command. The eight steps the runtime takes, in order, for every value the governor proposes: the observation's freshness (stale means blind, no write), the automation mode (recommend writes nothing), the clamp to the capability contract, the safety contract, the hold and the cooldown since the last write, the snapshot taken once before the first write, the execute through the system's own command, and the read-back with its residual. Every attempt, permitted or not, writes one MuscleReceipt with the fields shown.

The three contracts (`MuscleSpec`): the **capability** (`CapabilityContract`: the cover's `minimum` and `maximum`, the `resolution`, the most one write may move (`max_step`) or the most a second may move (`max_rate_per_s`), the `units`, whether the knob is `reversible` and whether it reads back); the **timing** (`TimingContract`: the observation and decision periods, the actuation delay, the minimum hold and the cooldown between writes, the timeout, and `stale_after_s`, past which a reading is blind); the **safety** (`SafetyContract`: the `hard_min` and `hard_max` nothing may cross, `fail_closed`, a `safe_default`, and the `forbidden` context keys, such as a security hold, under which no write is

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

permitted). The spec also names the muscle (`muscle_id`, `plant_id`), its class (`resource_type`, one of the standard classes below), its `automation` mode and its evidence class (`Evidence`: D design, S our own models, L live software, P physical meter, C commercial).

The automation modes, from least to most authority: `disabled`; `recommend` (every command computed and receipted, nothing written: the watch level of Chapter 45); `manual`; `external_approval` (a write only with `approved=True` from your change process); `automatic`. A muscle starts at `recommend` and is promoted exactly as Chapter 45's levels promote a stack.

What the runtime does with each command (`MuscleRuntime.command`), in order: a stale or invalid observation blocks the write and marks the muscle blind (section 32.3); the mode is checked; the request is clamped to the capability (cover, step, rate, resolution); the safety contract is asked (`outside_hard_bounds`, `forbidden:<key>`); the hold and the cooldown are checked (`hold_or_cooldown`); only then is the native snapshot taken, once, and the write made, and where the capability says the knob reads back, the setting is read back and the residual (realized minus bounded) recorded. Every attempt, permitted or not, writes one receipt (`MuscleReceipt`): `muscle_id`, `mode`, `requested`, `bounded`, `realized`, `units`, `permitted`, `reason`, `observation_age_s`, `actuation_latency_s`, `actuator_residual`, `evidence`, `status`. `MuscleRegistry` holds every muscle of a body, refuses a duplicate name, answers `catalog()` (every muscle with its cover, mode, evidence and status) and `restore_all()`, which hands every muscle back in the reverse of the order they were registered.

A worked example. The power limit of one graphics card, through the card's own command, first in `recommend`:

```
import json, subprocess
from omnicompass.muscle_sdk import (
    AutomationMode, CapabilityContract, Evidence, FunctionalMuscle,
    MuscleRegistry, MuscleRuntime, MuscleSpec, Observation,
    SafetyContract, TimingContract)

def read_limit():
    # the wire out: the setting as the card reports it now
    out = subprocess.run(
        ["nvidia-smi", "-i", "0", "--query-gpu=power.limit",
         "--format=csv,noheader,nounits"],
        capture_output=True, text=True, check=True).stdout
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return float(out.strip())

def write_limit(w):
    # the wire in: one setting, the card's own command (WRITES)
    subprocess.run(["nvidia-smi", "-i", "0", "-pl", str(int(w))],
                   check=True)

spec = MuscleSpec(
    muscle_id="gpu0.power_limit", plant_id="card-0",
    resource_type="gpu.power_limit", adapter_version="1.0",
    capability=CapabilityContract(minimum=100.0, maximum=300.0,
                                  resolution=1.0, max_step=25.0,
                                  units="W"),
    timing=TimingContract(observation_period_s=2.0,
                          decision_period_s=2.0, minimum_hold_s=10.0,
                          stale_after_s=6.0),
    safety=SafetyContract(hard_min=100.0, hard_max=300.0,
                          forbidden=("security_block",)),
    automation=AutomationMode.RECOMMEND,
    evidence=Evidence.LIVE_SOFTWARE,
    native_owner="the card's firmware")
muscle = FunctionalMuscle(
    spec, observe=lambda: Observation(read_limit()),
    execute=write_limit, readback=read_limit,
    snapshot=read_limit, restore=write_limit)
audit = open("/var/log/omni/muscles.jsonl", "a")
registry = MuscleRegistry()
registry.register(MuscleRuntime(
    muscle, audit=lambda rec: audit.write(json.dumps(rec) + "\n")))

receipt = registry.command("gpu0.power_limit", 250.0,
                           context={"security_block": False})
# recommend: permitted False, reason "recommend", nothing written;
# the receipt is in the audit
registry.restore_all()
# every muscle back as found, in reverse order; each one receipted
```

Promoted to `AutomationMode.AUTOMATIC`, the same code writes: a request of 250 W from the 300 W found is bounded to 250 and read back (residual 0); a request of 200 W ten seconds later is bounded to 225 by

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`max_step`; a request inside the 10-second hold is refused (`hold_or_cooldown`); a request made while `security_block` is true is refused (`forbidden:security_block`) and nothing moves; `restore_all()` returns the card to the 300 W found at the start and its receipt says `restored`. These outcomes are the SDK's own tests (`tests/test_muscle_sdk.py`).

The standard classes (`STANDARD_MUSCLE_CLASSES`) name the knob a new adapter governs, so that the catalog and the receipts read the same across a company's stacks: `kubernetes.replicas`, `.cpu_request`, `.cpu_limit`, `.memory_request`, `.memory_limit`, `.hpa_target`, `.pod_placement`, `.node_pool`; `host.cpufreq`, `.cpu_quota`, `.cpuset`, `.memory`, `.io`; `gpu.power_limit`, `.clock_envelope`, `.mig`, `.admission`; `cloud.instance_count`, `.instance_size`, `.placement`, `.start_stop`, `.capacity_policy`; `storage.volume_size`, `.tier`, `.iops`, `.throughput`, `.placement`; `network.traffic_weight`, `.rate_limit`, `.bandwidth`, `.routing`; `application.concurrency`, `.worker_count`, `.queue_admission`, `.jvm_heap`, `.thread_pool`, `.batch`; `facility.cooling_setpoint`, `.rack_power`, `.site_power`. A class is a contract, not a claim that a connector for it has run live: each adapter earns its own evidence class, and Appendix G's wiring recipes say which have (`docs/wiring_recipes.csv`).

A muscle coded on the SDK is wired into a body as any other (Chapter 34): one governor per body, the muscle's reading a gauge of that body, its receipts in the body's audit; the wire check of section 42.4 applies to it unchanged (snapshot, write, read-back, restore, nothing left behind), and it enters service through the levels of Chapter 45, watched first.

CHAPTER 43

The Harness

> **Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 42.5 of the manual (*Wired right or wired wrong*).

This tree is the GitHub-ready kit: frozen engine + C++ twin + two plants + live stub.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

43.1 What to run

```
pip install -r requirements.txt
python verify.py --quick          # engine, C++ twins and their parity
    ↪ , soak
python k8s_controlplane/test_controlplane.py
python -m k8s_controlplane.suite  # elastic / always_on / idle_power
python tests/test_hpa_three_way.py

python tests/test_omni_controller.py
```

The machine's own power (the Linux frequency governor as native, Omni on the frequency ceiling, energy from the processor's meter; root on a machine on the metal, not a virtual one; `docs/CPU_POWER_PREREGISTRATION.md`):

```
sudo python3 tools/run_cpu_power.py --probe            # can this mach
    ↪ ine run it?
sudo bash scripts/cpu_power_run.sh                     # the three wor
    ↪ kloads, native and omni, three repetitions; one folder
python3 tools/cpu_power_abc.py A B C --out V3_CPU_POWER.md   # three r
    ↪ uns into the table
```

Fleet plant (Omni as node authority vs HPA+CA / Karpenter-lite):

```
python -m fleet.planetlab --dir fleet/traces/planetlab --scenarios 8 -
    ↪ -out /tmp/pl
```

C++:

```
cmake -S cpp -B cpp/build -DCMAKE_BUILD_TYPE=Release && cmake --build
    ↪ cpp/build -j2
./cpp/build/oc_smoke
python tests/test_cpp_compass_law_parity.py cpp/build/oc_compass_law
    ↪  # the compass law, Python against C++
python tests/test_cpp_verdict_parity.py cpp/build/oc_verdict

    ↪   # the verdict, Python against C++
python tests/test_cpp_engine_runs_parity.py cpp/build/oc_engine_runs 5
    ↪ 00   # the Python's 500 runs through the C++ harness
python tools/engine_runs.py --runs 500 --base 7 --cpp cpp/build/oc_eng
    ↪ ine_runs   # a new set, both engines, one record
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

43.2 Two plants (do not mix the tables)

Tree	What Omni is	What the energy number means
`k8s_controlplane/`	On top of HPA+CA (target / gate / park)	Pack and optional CA gate on a 20-node replica
`fleet/`	Node-pool authority; CA off	Consolidation vs CA and Karpenter-lite

Observe must match the native arm on that plant. If it does not, the run is invalid.

43.3 Engine species

Shipped `omnicompass/core.py` is the patent principal form: cubic $U(1 - U^2)$, FIG. 4 command, RK4 with held u. `omnicompass/pools.py` is actuation only (off / hold / park). It does not change the field.

43.4 The C++ twin

`cpp/` is the engine written a second time, line for line where a twin exists: the six-state core, the governor's allocation laws, the shield, the pod autoscaler's replica law, the closure, the conveyance law, the nerves, the compass and ledger, the GPU governor's rules, the compass law with the two-way plug, and the verdict. Each twin has a parity test that feeds the Python and the C++ the same inputs (`verify.py` runs them all; `tools/seal.py` lists them and what still runs in Python only). The Python's seeded runs (`tools/engine_runs.py`, 500 from the jump generator) run whole through the C++ harness too (`cpp/tools/engine_runs.cpp`: the Python's draws are its input, and the governor, the nerves, the compass law and the verdict in C++ put the mechanism on), every receipt and the SHA-256 digest identical to the Python's; the `engine-runs` workflow runs both with every set. The live engine's page, in the page's section 7b, feeds the Python's own runs into the C++ at every rebuild (`docs/engine/export_cpp_twins.py`), and its proof monitor sets the C++ engine's digest beside the Python's and the page's.

43.5 Not in this harness

Live kube-controller-manager, GPU MIG scheduler, facility cooling plant (kind runs on GitHub: `docs/K8S_COMPASS_PREREGISTRATION.md`). `omni_controller/` is observe-first against kubectl; tests use `tests/standin_cluster/kubectl`.

See `LIMITS.md`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 44

The Wiring Guide

Before you wire anything: read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 42.5 of the manual (*Wired right or wired wrong*).

The current manual is `docs/INTEGRATION_MANUAL.md` (every stack, every level, every switch). This page is kept for reference; where the two differ, the integration manual is current.

Which knobs to wire in at all: `docs/WIRING_VERDICTS.md` gives every knob in every published result one of three words from the tables themselves: **write** (wire both ways), **watch** (wire out only: Omni reads, the native controller lives by itself) or **operator's choice** (a trade). A knob that showed nothing, or lost, stays native. **How to wire for superiority:** every knob starts in watch (wire out only); an observation run shows what native does alone; then the brain's own verdict decides on the knob, in real time, whether a notch pays before it is written (`tools/knob_verdict.py` around the engine's verdict; the manual, section 45.4); a knob that cannot prove it pays stays native, and the operator's setting is always free. Where that verdict stands today, knob by knob, is the last table of `docs/WIRING_VERDICTS.md`.

The full step-by-step manual, with the switch, the living band, parking and the pod reflex, is `docs/OPERATOR_MANUAL.md`. This page is the short version.

Three stages. Each stage adds only the permissions it needs. A stage is promoted only after its evidence is in, and the reset works at every stage:

```
kubectl -n omni-compass exec deploy/omni-compass -- touch /tmp/omni.ki
    ↪ ll
```

The switch is one human-operated switch for the whole harness (OFF restores every HPA target, replica range, CPU limit and cordon Omni-Compass changed, and records it; removing the file turns Omni-Compass back ON). It never fires by itself and never switches off one module: a failed decision is skipped, boundaries are held by the living band.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

44.1 Build the image

```
docker build -f deploy/Dockerfile -t <registry>/omni-compass:<tag> .
docker push <registry>/omni-compass:<tag>
```

What the image contains.

- The engine.
- The nervous system.
- The live controller and its levers.
- The frozen closure-law setting (`/app/law/closure.json`).

How it runs. As a non-root user, with a read-only root filesystem and no Linux capabilities.

44.2 Shadow: read-only, decides and logs, never writes

```
kubectl apply -f deploy/install/omni-compass.yaml      # set the image
    ↪  line first
kubectl -n omni-compass logs deploy/omni-compass -f
```

Proof of read-only. `scripts/pilot_shadow.sh` runs the same stage from a workstation with `kubectl auth can-i` receipts that the identity cannot write, and writes `SHADOW_REPORT.md`.

Run length. One to two weeks.

Pass condition. Zero writes, and recommendations you agree with.

44.3 Target: Omni-Compass sets each HPA's CPU target

```
kubectl apply -f deploy/rbac-target.yaml               # adds: patch h
    ↪ orizontalpodautoscalers
kubectl -n omni-compass patch deploy omni-compass --type=json -p \
  '[{"op":"replace","path":"/spec/template/spec/containers/0/args/1","
    ↪ value":"target"}]'
```

What happens. The HPAs keep scaling pods. Omni-Compass only moves their target within bounds.

The SLO reflex. Add `--latency-file` and `--slo-ms`, and the target is never tighter than native while the SLO is breached.

Pass condition. Latency never behind native, and fewer pod-hours.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

44.4 Node pool: Omni-Compass sizes the node pool

Add permissions. `patch nodes`, `create pods/eviction` (`deploy/kind/rbac-omni.yaml` is the tested example).

Set the mode. `--mode nodepool --node-scale-cmd "<your pool resize command with {n}>"`. Examples of the resize command:

- a Karpenter NodePool limit;
- a cloud node-group size;
- `scripts/kind_nodepool.sh` on kind.

What guards a release. The machine organ gives a node back only when all of these hold:

- every sense is live;
- its last order landed;
- pods are not scaling up;
- nothing is pending;
- the remaining nodes stay at or below the engine's target utilisation.

A PodDisruptionBudget on each service is required, because drains go through the eviction API.

Optional levers. Each has its own flag and permissions, and all are listed in `omni_controller/muscles.py`:

- right-sizing;
- cold start;
- batch pacing;
- agent containment;
- cooling;
- CPU frequency.

44.5 Evidence, and where it is

Evidence	File
The controller, every lever and the Unified Control Switch on real Kubernetes (kind)	`results/live/LIVE_LEVERS_2_NERVOUS.txt`
Read-only shadow on real Kubernetes: every decision logged, 0 writes	`results/live/LIVE_SHADOW_1.txt`
Least-privilege identity receipts	`rbac_omni.txt` in every live run

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Evidence	File
Native vs Omni on top vs Omni alone, paired on one machine, real Kubernetes	`results/live/LIVE_PAIRED.md`
The full wiring manual	`docs/OPERATOR_MANUAL.md`
The frozen engine's results, three runs each, read by rule	`results/live/V1_*.md`, `results/live/V3_*.md`; `docs/OMNI_V1.md`, `docs/OMNI_V3.md`
The same wiring on Azure's managed Kubernetes (the bill), a database pooler, a rented machine, the simulators, a message broker's consumer group, a cache's memory ceiling, a drone swarm's autopilot	the manual, chapter 32 (`docs/OMNI_COMPASS_MANUAL.md`)

CHAPTER 45

Before You Start, and the Eight Levels

45.1 What your system needs

You run	You need
Kubernetes (vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind)	Kubernetes 1.34 or newer (in-place pod resize); metrics-server (`kubectl top nodes` works); an HPA with a CPU target on each governed service; a readiness probe and a short preStop pause on each service
A response-time feed (strongly recommended at every level)	a CSV per service, `elapsed_seconds,latency_ms,ok`, written continuously; `scripts/latency_probe.py` writes one against any HTTP endpoint
NVIDIA GPUs	a driver with `nvidia-smi`; root (or the capability to run `nvidia-smi -pl` and `-lgc`); on a VM, full GPU passthrough
CPU power control	Linux cpufreq with the schedutil governor; RAPL for package watts; usually bare metal

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

You run	You need
Site power and cooling	a command that prints site watts, and a command that sets the supply-air setpoint through your building management system

Why each requirement is there. Kubernetes 1.34 or newer is required because the conveyance muscle resizes a pod's CPU limit in place, which older versions do only by restarting the pod, and a restart is a disruption Omni-Compass never causes. metrics-server is the HPA's own eyes and the governor's: without it neither knows what the pods use. An HPA with a CPU target on each governed service is the native controller Omni-Compass sits on; a service without one has nothing for level 2 to move. A readiness probe and a short preStop pause are what make a drain invisible to clients: the Service stops sending requests to a pod before it stops, and kube-proxy sends them to another ready endpoint. The response-time feed is the one wire a cluster does not already have: Kubernetes knows what its pods use, not what its users wait; the compass needs the second, and `scripts/latency_probe.py` writes it against any HTTP endpoint every five seconds. On the GPU, root is needed only for the two `nvidia-smi` writes, and a VM must pass the whole card through or the driver refuses to lock clocks (the wire check's step 3 is where that shows).

45.2 Get the software and check it

```
git clone https://github.com/The-Omni-Compass-LLC/The-Omni-Compass-LLC
    ↪ .git
cd The-Omni-Compass-LLC
pip install -r requirements.txt
python3 verify.py                        # every check; ends with VERIF
    ↪ ICATION: PASS
```

Build the container image once:

```
docker build -f deploy/Dockerfile -t <your-registry>/omni-compass:<tag
    ↪ > .
docker push <your-registry>/omni-compass:<tag>
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

What `verify.py` checks is worth knowing before it is run, because its last line, `VERIFICATION: PASS` or `FAIL`, is the only thing a reader has to read. It runs every test in `tests/` (the law, the plugs, the readings rule, every benchmark's own harness against a stand-in server), checks the engine's fingerprints against the preregistered registry so that a changed engine file fails before anything is measured, compares the release's files with its manifest (`tools/release_manifest.py --check`: the engine, the C++ twins, the live evidence's digests, one object that names the commit it was written at) and runs the layout check (`tools/layout_check.py`: the declared root, every link and every named path present). It takes a few

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

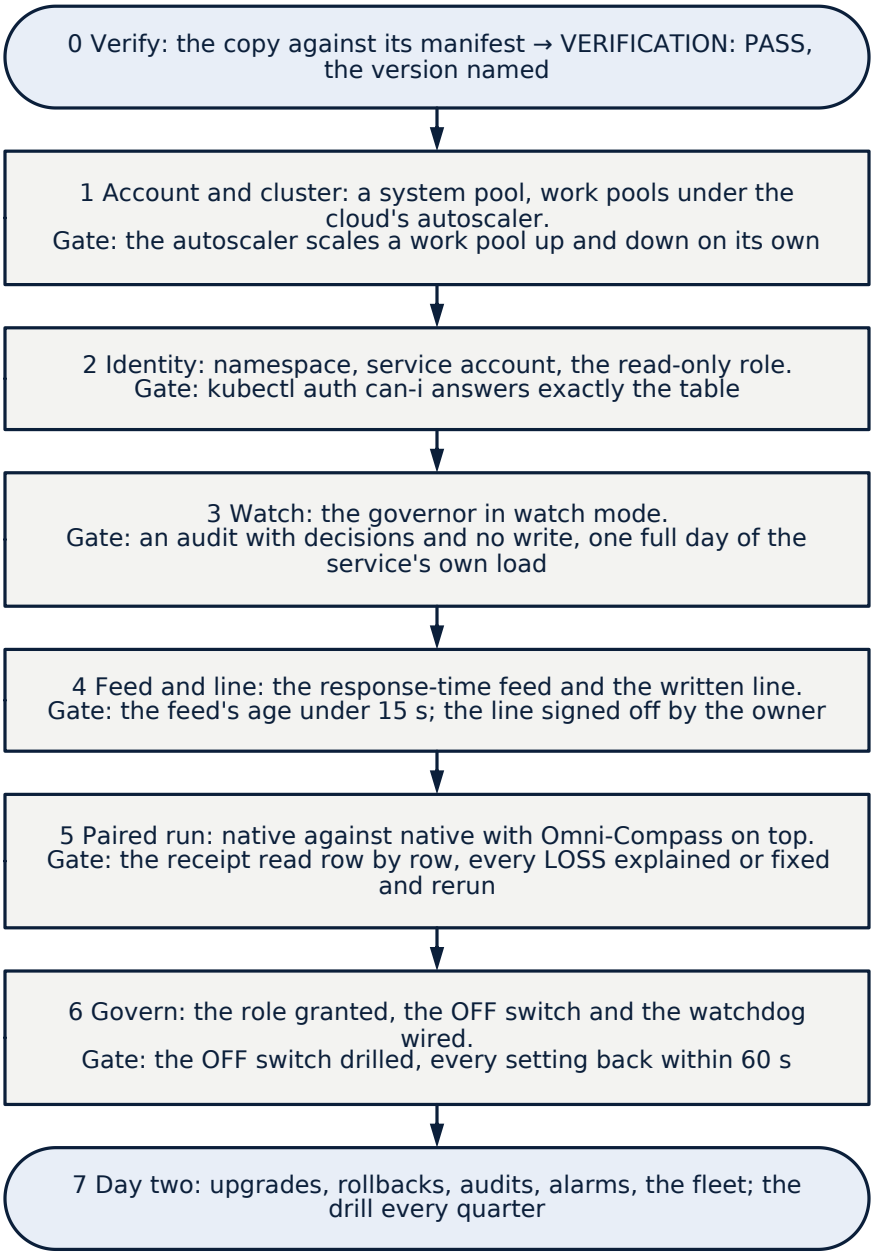


Figure 30. Flowchart: the implementer's eight phases and their gates. From the verified copy through the account and the cluster, identity, the watch, the feed and the line, the paired run and the governing grant to day two; each phase has a gate that must pass before the next begins, and the OFF switch is drilled before the governor writes.

minutes on a laptop and needs no network beyond the clone. GitHub runs the same script on every push under Python 3.12, and each run is listed under the repository's Actions tab with the commit it checked; a result in this manual is only ever cited from a commit whose run passed.

To know which engine a checkout carries, ask it:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
python3 tools/omni_version.py                  # omni-v11, omni-v10, o
    ↪ mni-v9, omni-v8, omni-v7, omni-v6, omni-v5, omni-v3, omni-v2, om
    ↪ ni-v1, or what differs
python3 tools/omni_version.py --commit <sha>   # the same question of
    ↪ any pushed commit
```

Every result table in `results/live/` cites the commit it ran on, and the three-run tools refuse to combine runs from different engines (Chapter 59). The founder's standing order is that no result is ever read across engines, and the command above is how a reader checks that for themselves.

45.3 The levels

Take them in order. Each has a pass condition; go to the next level only when it holds. The OFF switch works at every level.

The levels exist because a governor earns authority the way a new operator does: by watching first, then by touching the setting that is cheapest to undo, then the next. Level 0 touches nothing and proves the software is the software (`verify.py` checks every sealed file's fingerprint and runs every test). Level 1 watches a live cluster for as long as the operator likes and logs every decision it would have taken, so the operators can read those decisions against what they would have done; nothing passes level 1 until they agree with its readings. Level 2 moves the HPA target, which the HPA itself corrects within a minute if the governor is wrong, and raises replica floors ahead of bursts; it is the first level that can show a gain and the first that can show a loss, and the paired receipts of Chapter 57 are its pass condition. Level 3 moves machines, the slowest lever, through the release gate and the verdict; it is where the bill moves and where the most care is spent. Level 4 hands Omni-Compass the replica decision itself, with the HPA kept as the fast reflex upward; it is an option, not a step on the way, and the benchmarks in this manual do not use it. Levels 5 to 7 are the GPU, the CPU and the site, each with its own wire check and its own pass. An installation may stop at any level and run there indefinitely; most will stop at level 3.

Level 0 - Evaluate without touching anything.

```
python3 verify.py
python3 tools/run_scale.py --runs 10 --scale 1 --out results/scale/eva
    ↪ l     # the seven organisms, modelled
python3 tools/run_gpu_card.py results/sim/gpu_two_wire/eval
    ↪        # the two-wire card, modelled
```

Pass: `VERIFICATION: PASS`. Nothing in your systems is touched.

Level 1 - Watch (read-only).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

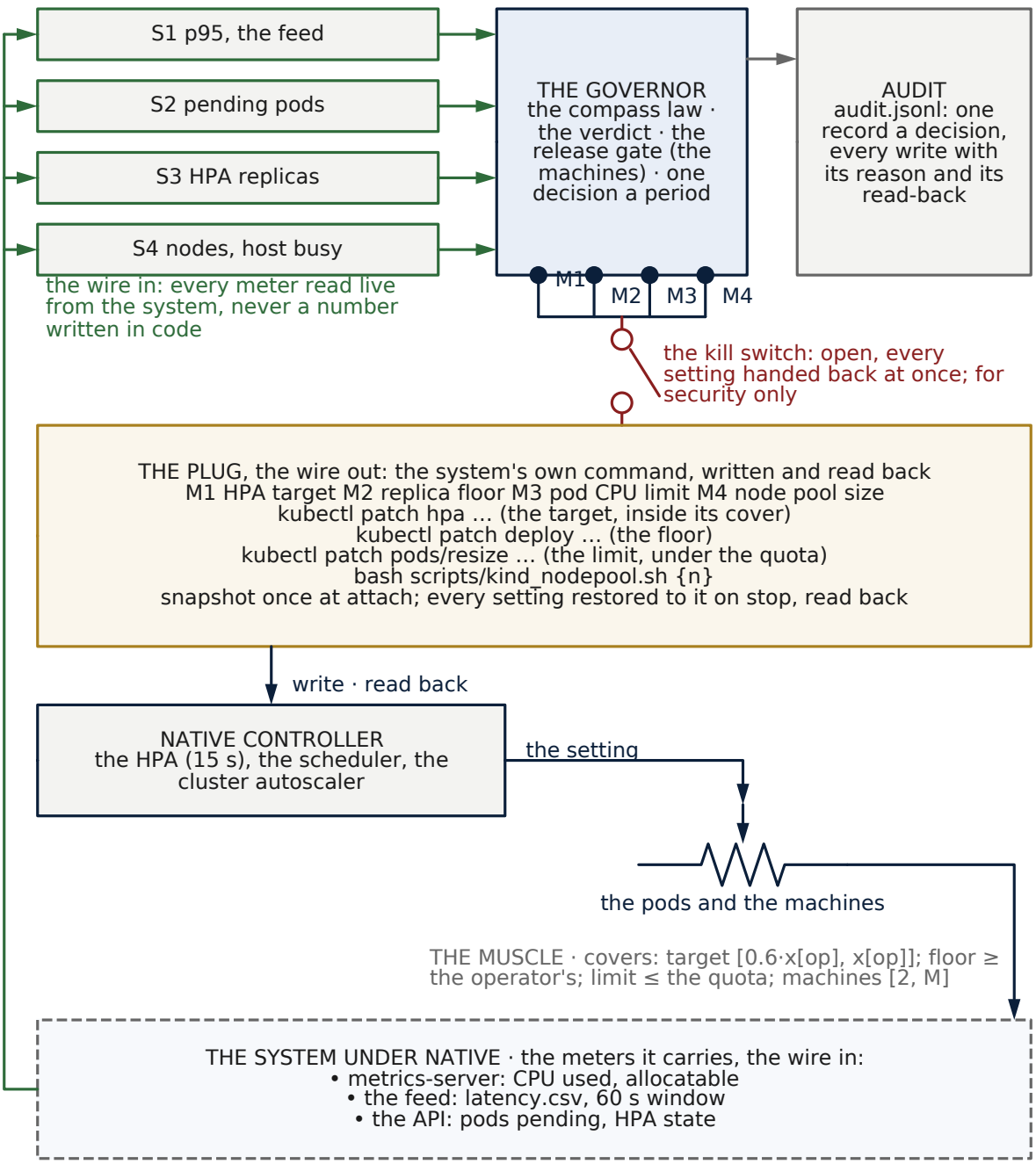


Figure 31. Wiring schematic: Kubernetes in the cluster, levels 2 to 4. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. Here the senses are the response-time feed, the pods waiting for a place, each autoscaler's replicas, and the machines with the host's busy share; the motors are the autoscaler's target,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

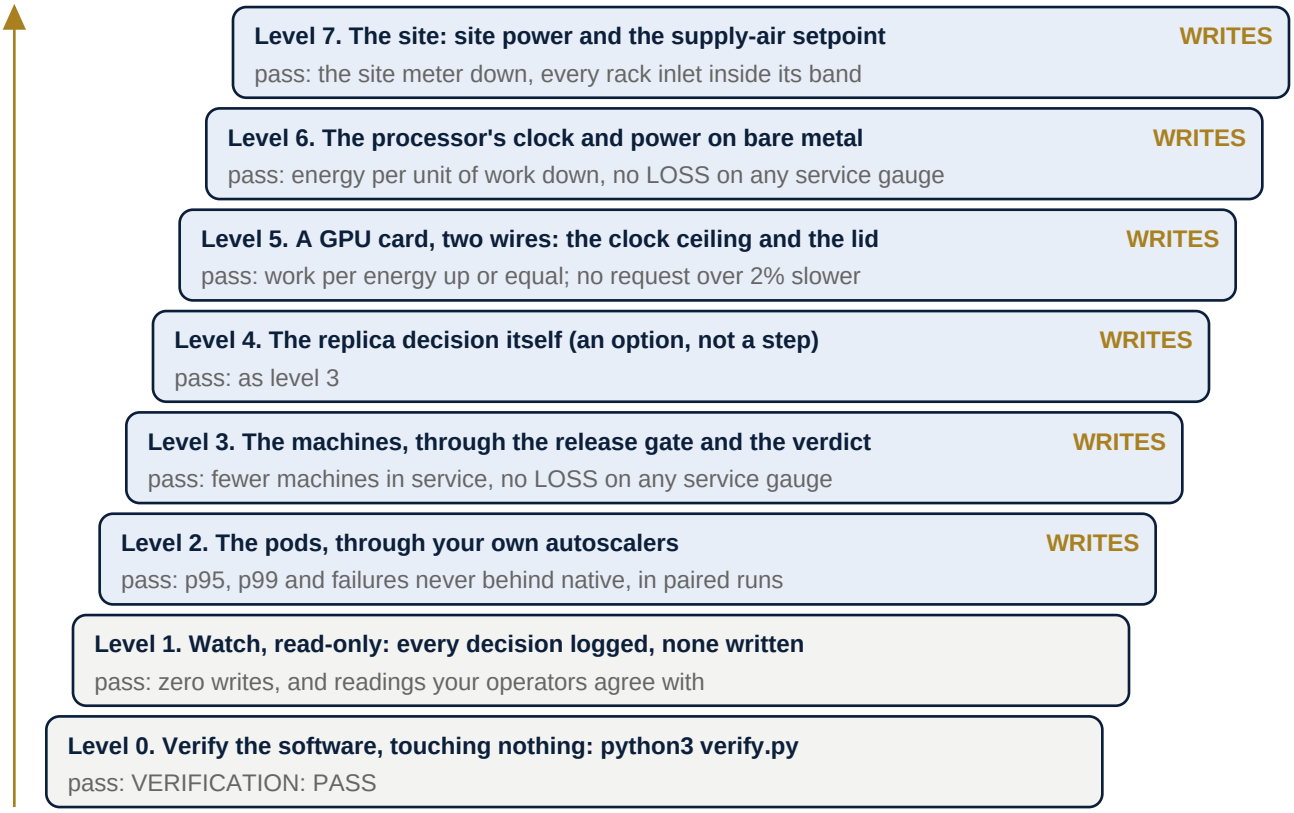


Figure 32. The eight levels of an installation, taken in order, each with its pass condition. Levels 0 and 1 write nothing; from level 2 the governor writes, and the wire check must pass before any level that writes. An installation may stop at any level and run there indefinitely.

the replica floor, the pod CPU limit and the node pool size, each through kubectl's own patch and resize commands. Level 2: the target and the floor; level 3: the machines through the release gate and the verdict; level 4 (--strict-replicas): the replica decision itself. The HPA keeps its own 15-second check underneath. Nothing is ever deleted or evicted; a parked machine is emptied by the autoscaler's own scale-down.

```
kubectl apply -f deploy/install/omni-compass.yaml     # set the image
    ↪ line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

The install file grants a read-only identity; `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts that show it cannot write. Every decision it would take is logged with its reason. Pass: zero writes, and readings your operators agree with.

Level 2 - The pods, on top of your autoscalers. WRITES.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`).
2. Run with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.
3. Optional pod muscles, one at a time:

Muscle	Switch	What it does
convey	`--latency-file` and `--cap-deployments ns/name`	gives each machine's idle CPU to the serving pods on it
rightsize	`--rightsize-deployments ns/name`	each pod's CPU request follows its measured use times (1 + headroom), in place
coldstart	`--coldstart-deployments ns/name` `--coldstart-signal ns/configmap`	scales a service to zero while no work waits, wakes it when work arrives
batch	`--batch`	admits held Jobs labelled `omnicompass.io/batch=true` when there is load and power headroom
batch pace	`--batch-pace`	pauses Jobs labelled `omnicompass.io/pausable=true` under power or heat stress, resumes them after
rollout guard	`--rollout-guard ns/name`	pauses a rollout while change is not permitted
contain	`--contain-namespaces ns` `--contain-cpu-m <m>`	holds an agent namespace to a CPU budget
security hold	`--security-configmap ns/name`	key `hold: "true"` blocks every expansion

Your HPAs keep scaling as before; Omni-Compass sets their targets and raises floors ahead of bursts. Pass: p95, p99 and failed requests never behind native, over paired runs (Chapter 57).

Level 3 - The machines. WRITES.

1. Grant `patch nodes` and `patch pods` (`deploy/kind/rbac-omni.yaml`).
2. Run with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<command with {n}>"` and `--node-restore-cmd "<command>"` for the OFF switch. For the compass law on the same levers, use `--law compass` (reads the mean response time of the latency window between a tenth of `--slo-ms` and `--slo-ms`, held at `--compass-center 0.4`; p95 at the SLO, a blind feed or a waiting pod adds capacity at once; the HPA target never goes above the operator's; machines go back one at a time through the release gate).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Your platform	The park/wake command
any cluster, bare metal, kind	`bash scripts/kind_nodepool.sh {n}`
Karpenter / EKS Auto Mode	the NodePool CPU limit at `{n}` times node CPU, parked nodes kept
Cluster Autoscaler node group	the group's desired size, scale-down through parking
OpenShift / OKD	the worker MachineSet replicas, parked

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band. Pass: fewer machines in service, with no LOSS on any service gauge.

Level 4 - Omni-Compass decides; Kubernetes is the muscle. WRITES. Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; the HPA stays as the fast reflex upward. Pass: as level 3.

Level 5 - A GPU box, two wires. WRITES.

```
sudo python3 tools/gpu_wire_check.py --gpu 0                    # must
    ↪  end: WIRED RIGHT
# watch: computes both wires and writes neither
sudo python3 -m omni_controller.gpu_compass --mode watch --gpus 0 --au
    ↪ dit /var/log/omni/gpu.jsonl \
     --latency-file <feed> --slo-ms <p95 target> --floor-w <your lowes

    ↪ t watts>
# govern: writes the clock ceiling and the power limit
sudo python3 -m omni_controller.gpu_compass --mode cap --gpus 0 --audi
    ↪ t /var/log/omni/gpu.jsonl \
     --latency-file <feed> --slo-ms <p95 target> --floor-w <your lowes
    ↪ t watts>

# OFF: clocks reset, power limit back to the start, read back
sudo touch /tmp/omni-gpu-kill
```

Every `--interval` seconds (default 2) it reads the card's own meters and the response times, and:

- **learns the card on its own** first: while the ceiling is at the top and the card is busy, its busy clock and busy draw; until 15 such readings are in, neither wire moves;
- **races while work waits**: at 95% utilization or more, the ceiling to the top and the lid to the start limit;
- **paces the slack**: the mean response time of the last 5 s, between a tenth of `--slo-ms` and `--slo-ms`, held at the compass's center (0.4); the ceiling moves in whole 15 MHz steps, never under the card's own busy clock, and the lid never under the card's own busy draw plus 10%, never over the start limit;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **asks the card first (the verdict)**: before the ceiling may go one step lower, a paired trial measures the card's own time on each request (the workload's `service_ms`) at the top and at that step; the step is allowed only if it adds at most `--allow` (2%). Where no step passes, the ceiling stays at the top: the card runs as it does alone, and the audit says so (`verdict_state: left native`);
- **fails up** when p95 reaches `--slo-ms`, a request fails, the feed goes blind or the card reports a heat slowdown.

One law, no profiles: down gain 0.0125, center 0.4, floor at the card's own busy clock, allowance 2% (GPU preregistration, amendment 8). The workload must write `service_ms` (the card's own time per request) in its latency file, as `tools/gpu_workload.py` does; without it no step is ever allowed and the card stays native. Pass: work per energy up or equal; requests served equal; no request more than 2% slower at the median, p95 and p99 at native or within 2%; every row of section 42.5 reads "wired right".

Level 6 - CPU clock and power. WRITES. On bare metal, add to the controller: `--cpufreq-policy-root /sys/devices/system/cpu/cpufreq --cpufreq-require-schedutil --rapl-cmd "<prints CPU package watts>"`. The OFF switch writes every policy's recorded maximum back exactly. Pass: energy per unit of work down, no LOSS on any service gauge.

Level 7 - Site power and cooling. WRITES. `--power-cmd "<prints site watts>" --site-limit-w <limit>` brings site power stress into the engine; `--cooling-cmd "<sets {c}>" --cooling-min-c 18 --cooling-max-c 27 --cooling-restore-c 22` lets it hold the supply-air setpoint in its band. CPU and GPU sharing one power budget (`hardware/node_exchange.py`) and GPU groups sharing a site budget (`hardware/site_exchange.py`) run as models today. Batteries are designed as an organ, not yet wired.

45.4 Wiring for superiority: watch first, then the brain's own verdict on every knob

The founder's order of 9 October 2026 fixes the order of wiring. Every knob is wired out first and in last, and the wire in is connected only by a measurement the brain makes on the knob itself. The steps, for any stack:

1. **Wire out, everything.** Every muscle's readings reach the brain, one wire out per muscle. Nothing is written. This is watch mode (section 1.3): where every installation starts, and where any knob returns the moment it stops paying.
2. **Run the observation.** With the stack together and under its real or replayed demand, let the brain watch for at least one full cycle of that demand. In watch mode the brain learns the band of every reading, where the native controllers hold them, and which readings leave calm and when. Nothing is decided yet; the audit says what the brain saw.
3. **Let the verdict decide each knob, one trial at a time.** Switch the knobs the operator is willing to have moved into verdict mode. A knob still writes nothing until the brain has run a paired trial on it (section 1.2; `omnicompass/verdict.py`; on the live stacks `tools/knob_verdict.py` around it): the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

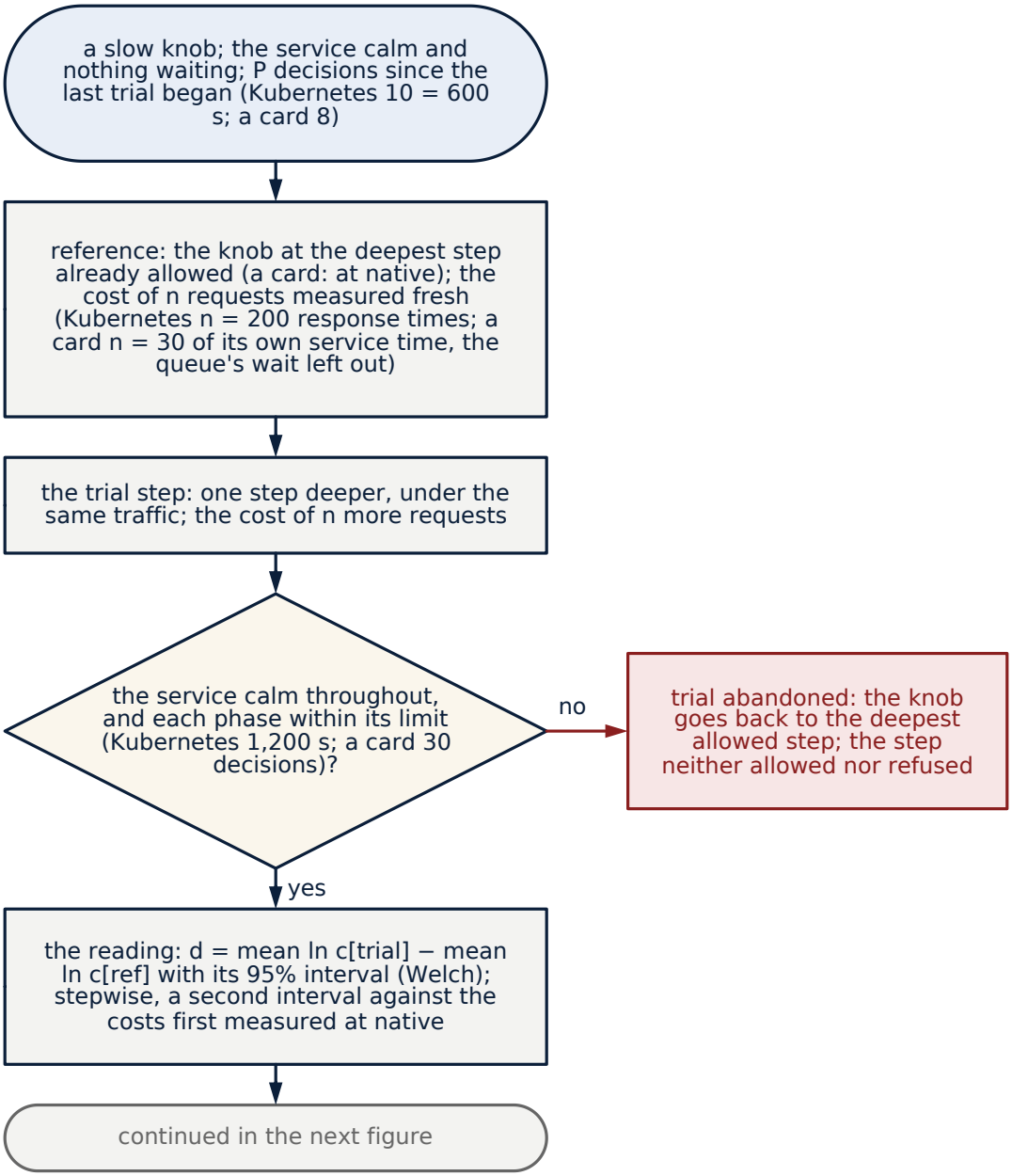


Figure 33. Flowchart: the verdict's trial on one step, from the reference to the reading. While the service is calm and nothing waits, and P after the last trial began (600 s on Kubernetes, 10 decisions at the main engine's 60 s; 8 decisions on a card), the reference is measured fresh at the deepest step already allowed (on a card, at native): n requests (200 response times on Kubernetes; 30 of a card's own service time).

One step deeper, under the same traffic, n more are the trial. A phase that sees the service leave calm or outlasts its limit (1,200 s on Kubernetes, 20 decisions at the main engine's 60 s; 30 decisions on a card) is abandoned: the knob goes back, the step neither allowed nor refused. The reading is the 95% interval of the change in log cost. Continued in the next figure.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

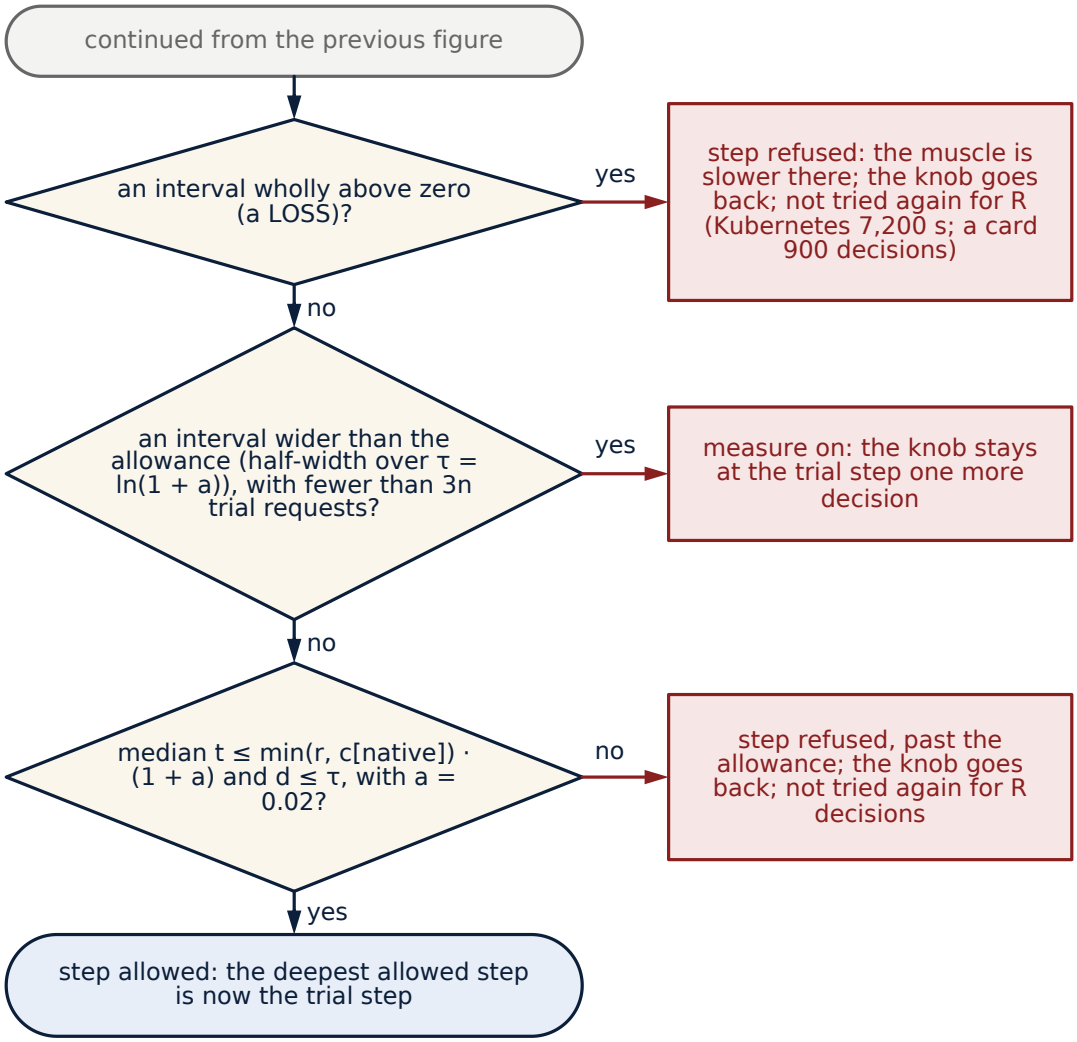


Figure 34. Flowchart: the verdict's trial on one step, from the reading to the step allowed or refused. An interval wholly above zero is a LOSS: the step is refused, the knob goes back, and the step is not tried again for R (7,200 s on Kubernetes, 120 decisions at the main engine's 60 s; 900 decisions on a card). An interval wider than the allowance with fewer than 3n trial requests measures on. Otherwise the step is allowed only if the trial's median is within the allowance of the reference and of native and the mean change is within it too; else it is refused, past the allowance. An allowed step moves the deepest allowed step one deeper.

knob is held at the operator's setting for a reference, then one notch away, under the same traffic, and the step is allowed only if the muscle's own cost did not rise under the declared objective. The objective is the operator's one declaration per knob: the resource reading (the service, the resource and the CPU weighed together, the default and the index's own reading) or the service reading (work and speed alone). A refused step is not retried for a set time; an allowed step widens the allowance by one notch, and the next trial probes the next. The compass then moves the knob by its law inside the allowance, never outside.

4. **Keep checking.** The trials recur. A knob whose last step stops paying when the demand changes is refused on its recheck and the allowance narrows back; a knob that never passes a trial stays native for

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

good, which is the right answer for that knob and costs nothing. New noise in the system is answered the same way: the compass reacts inside the allowance within its time constant, and the allowance itself is re-earned by measurement.

5. **Read the verdicts, muscle by muscle.** The audit carries the state of every knob every second (`verdict_phase`, `verdict_direction`, `allowed_low`, `allowed_high`, the cost sample) and the arm's record sums it; the wiring page (`docs/WIRING_VERDICTS.md`, section 16.6b) is the same judgement made from the published tables after the fact. An operator who wants one knob wired both ways and another watched only sets the objective on the first and leaves the second in watch.

What this gives: no knob is ever written on faith. Where Omni-Compass can beat the native controller on the muscle's own gauges, it will have proved it on that muscle before it moved it; where it cannot, the muscle lives by itself and Omni reads it. The organism's superiority is then the sum of the knobs that earned it, not one turn of a single dial for the whole system. On the five live stacks of Chapter 46 this is built and declared (the amendments of 9 October in the Redis, Kafka, PostgreSQL, MySQL and YCSB preregistrations); the counted runs on it were dispatched on 10 October, three runs a stack and objective, the expectation written before them (`docs/RERUN_2026-10-10.md`). On the Kubernetes controller the verdict already guards the machines (section 1.2). Inside the modelled realms the organism still takes one directive for every muscle; making the verdict per muscle there is the next engine, Omni v4 (`docs/OMNI_V4_PLAN.md`), designed and not built, because every result must be run again on a new engine.

CHAPTER 46

Stack by Stack

Every wiring below is the same plug (Chapter 42): one wire in (the stack's own meter), one wire out (one setting the stack already accepts), a snapshot taken once before the first write, every write read back, everything restored on OFF. Native is always the stack's own controller left as shipped; omni is that controller with Omni-Compass on top.

Stack	Levels	Notes
Vanilla Kubernetes, kind, Rancher	1-7	as written
Amazon EKS	1-5	machines via Cluster Autoscaler node group or Karpenter; CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

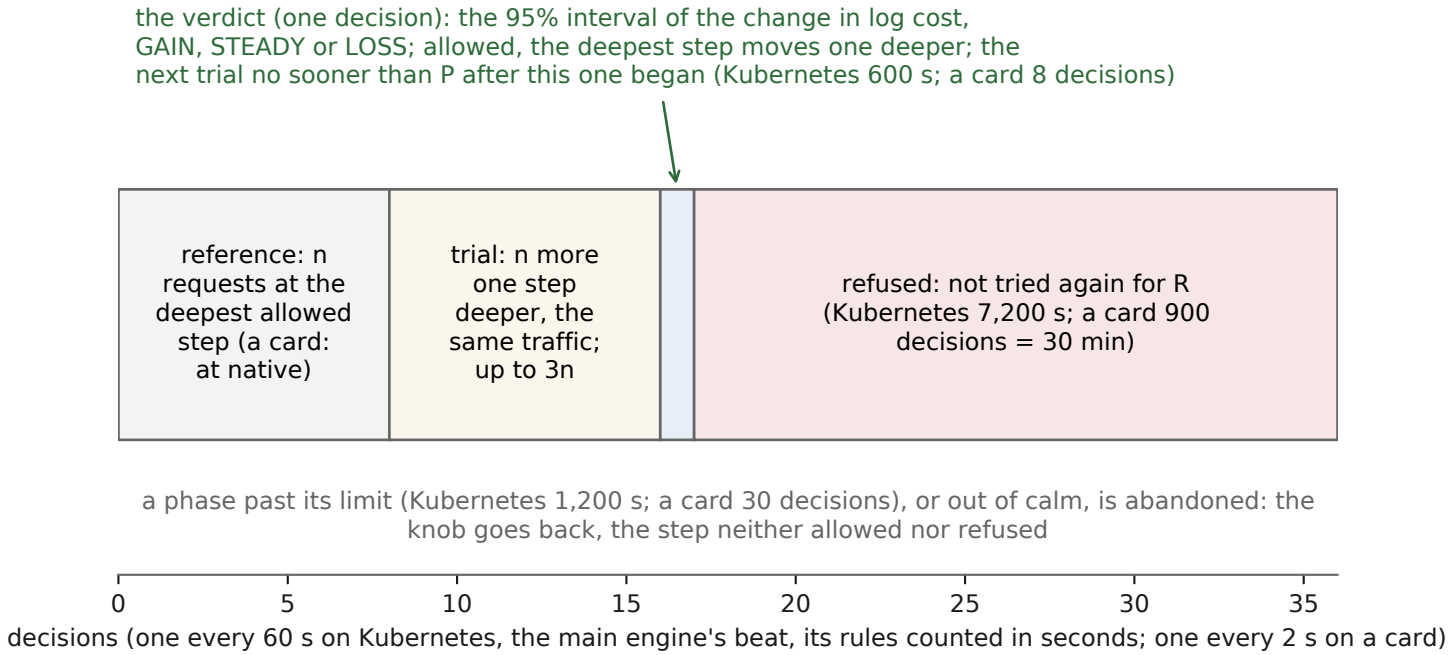


Figure 35. Timing diagram: a give-back trial from reference to verdict, in decisions. The reference (n requests at the deepest allowed step; on a card, at native), the trial (n more at one step deeper under the same traffic, measuring on up to 3n while the interval is wider than the allowance), the verdict in one decision, and, on a refusal, the quiet period in which the step is not tried again (7,200 s on Kubernetes, 120 decisions at the main engine's 60 s; 900 decisions on a card, 30 min).

A phase that outlasts its limit (1,200 s on Kubernetes; 30 decisions on a card), or sees the service leave calm, is abandoned: the knob goes back, the step neither allowed nor refused.

Stack	Levels	Notes
Google GKE, Azure AKS	1-5	the provider's node-pool size as the park/wake command; on AKS the managed cluster autoscaler stays native and deletes the machines Omni-Compass idles (10.1)
Red Hat OpenShift / OKD	1-5	the same permissions through a Role; machines via the worker MachineSet
NVIDIA GPU servers without Kubernetes	5	the two-wire GPU governor alone
Bare-metal CPU servers	6	cpufreq and RAPL through sysfs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Stack	Levels	Notes
Slurm / HPC	5 on the GPU nodes	a job-level Slurm connector is not built
Building management	7	through your BMS's command line or API, in the command templates
PostgreSQL behind PgBouncer (any database behind a pooler with a console)	one knob	the pool size through the pooler's own admin console (10.2)
A power grid's substation tap changer (pandapower, SimBench)	one knob	the tap position, one whole tap per move (10.3)
Buildings with batteries (CityLearn)	one knob a building	the battery's charge and discharge inside the simulator's own limits (10.3)
Robot arms (MuJoCo Menagerie)	one knob an axis	the joint speed where the axis has slack; never on an axis at its takt (10.3)
A rented cloud machine running the big organisms	the whole harness	the detached run: rent, run, collect, delete (10.4)
A message broker's consumer group (Apache Kafka)	one knob	the consumer count inside [1, partitions], read from the consumers' own records; the rebalance paid on every move (10.5)
A cache's memory ceiling (Redis)	one knob	`maxmemory` through the cache's own console, grown only while the cache is full (10.6)
Drone swarms (gym-pybullet-drones; PX4 and ArduPilot next)	one knob a drone	the cruise override inside the autopilot's limits; a separation wall; collisions void the cell (10.7)
A database's storage-engine cache (MongoDB under YCSB)	one knob	the WiredTiger cache size through the server's own console, grown only while the cache is full (10.8)

46.1 Managed Kubernetes on Azure: the bill as the gauge

On Azure Kubernetes Service the muscle is the same as on kind (Chapter 45, levels 1 to 3) and one thing is new: the bill is real. The workflow `aks-metered` builds a fresh cluster for every arm the same way: a system pool of one machine, tainted so no workload lands on it (Azure's add-ons and the load generator live there; it is never measured and never governed), and one or several **work pools** under Azure's own cluster autoscaler, every pool labelled `omni-role=work`, every pool starting full. Native is that autoscaler alone. Omni on top is the same autoscaler with Omni-Compass idling the machines it gives back, which the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

autoscaler then deletes on its own schedule. The bill counts every work machine that exists, every 15 seconds, priced at Azure's list price for its family (`tools/live_reps.py`, `POOL_PRICES`); each cluster is deleted when its arm ends, so nothing keeps billing.

Sizing the fleet to the lever. The smallest saving a fleet can show is one machine. On 4 workers one machine is 25% of the fleet, so only a saving of about 30% or more can clear the noise; on 15 workers about 10%; on 96 about 3%. The expected machine saving under Omni-Compass is a few percent, so the 4-worker runs (`results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md`) cleared the noise on no gauge, which is the honest reading of a fleet too small for the lever, not a result against it. A new subscription allows 200 vCPUs in a region but 10 per machine family, so one family gives five 2-vCPU workers; the way past it is **several families at once**: eight pools of four to five workers each make a fleet of 39 (`worker_pools` input, `size:max,size:max,...`; `pool_prices` names each family's price). The app's replica ceiling is raised with the fleet (`hpa_max`: nine 200-millicore pods fit one 2-vCPU worker, so 9 × workers lets the fleet fill), the same in both arms. The preregistered design is in `docs/K8S_COMPASS_PREREGISTRATION.md`, "The fleet that can show one machine"; the account setup, click by click, in `docs/AZURE_SETUP.md`.

In the cloud, the work the bill buys. In a public cloud every machine that exists is billed, whether it carries work or not. The cloud runs the whole engine at the max, as on site (`dial max`, the `dial` input of `aks-metered` and its default; the founder's order of 31 October struck the tune-down on the machines alone of 14 October): the governor moves the service's pod target inside its autoscaler's cover, conveys the processor a machine can spare, and parks a machine the cloud already made only after its own trial shows the response time holds, waking it on a breach; the cloud's own autoscaler, native's machine controller, stays in every arm and alone adds machines. The set must show that the engine holds: no judged gauge LOSS against native, the work per dollar read beside the bill; the benefit is made at the software end and at the chip and comes through the wire. A smaller bill is never the aim; a company that wants a cut turns the dial down (`dial save`, a changed tune, its receipts its own). The bill is shown beside the work it bought, never ruled. In the cloud, as everywhere, the governor's fast checks (the scheduling floor, the pod reflex, and the conveyance every 15 seconds whichever check falls due) run every 5 seconds, three times as fast as the HPA's own 15-second check, and its full decision every 60 seconds, the main engine's beat (omni-v17; omni-v16 decided every 15 seconds, the HPA's own pace, and every engine before it every 60 seconds), the beat a dial from 5 to 120 seconds. Through v6 a shorter decision period was not a free setting: the give-back trial's phase (`max_trial`, 20 decisions), the breach cool-down (`--slo-clear`), cruise (`--cruise-after`) and the trial spacing counted in decisions, so at 5 seconds a trial would have lasted 100 seconds, too short for the 200 response times it measures, and no machine would ever have been given back (`docs/K8S_COMPASS_PREREGISTRATION.md`, "The decision period in the cloud"). v7 states every one of them in seconds (`--slo-clear-s 180`, `--cruise-after-s 120`, `--strict-window-s 300`, `--verdict-every-s 600`, `--verdict-recheck-s 7200`, `--verdict-trial-s 1200`) and converts each to decisions at the running period, n = max(1, round(seconds / period)), so at 60 seconds every count is v6's (the cruise wait never under two readings, omni-v17: at 90 and 120 seconds its 120 had rounded to one decision, and a single reading with a pod waiting put every machine in service;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

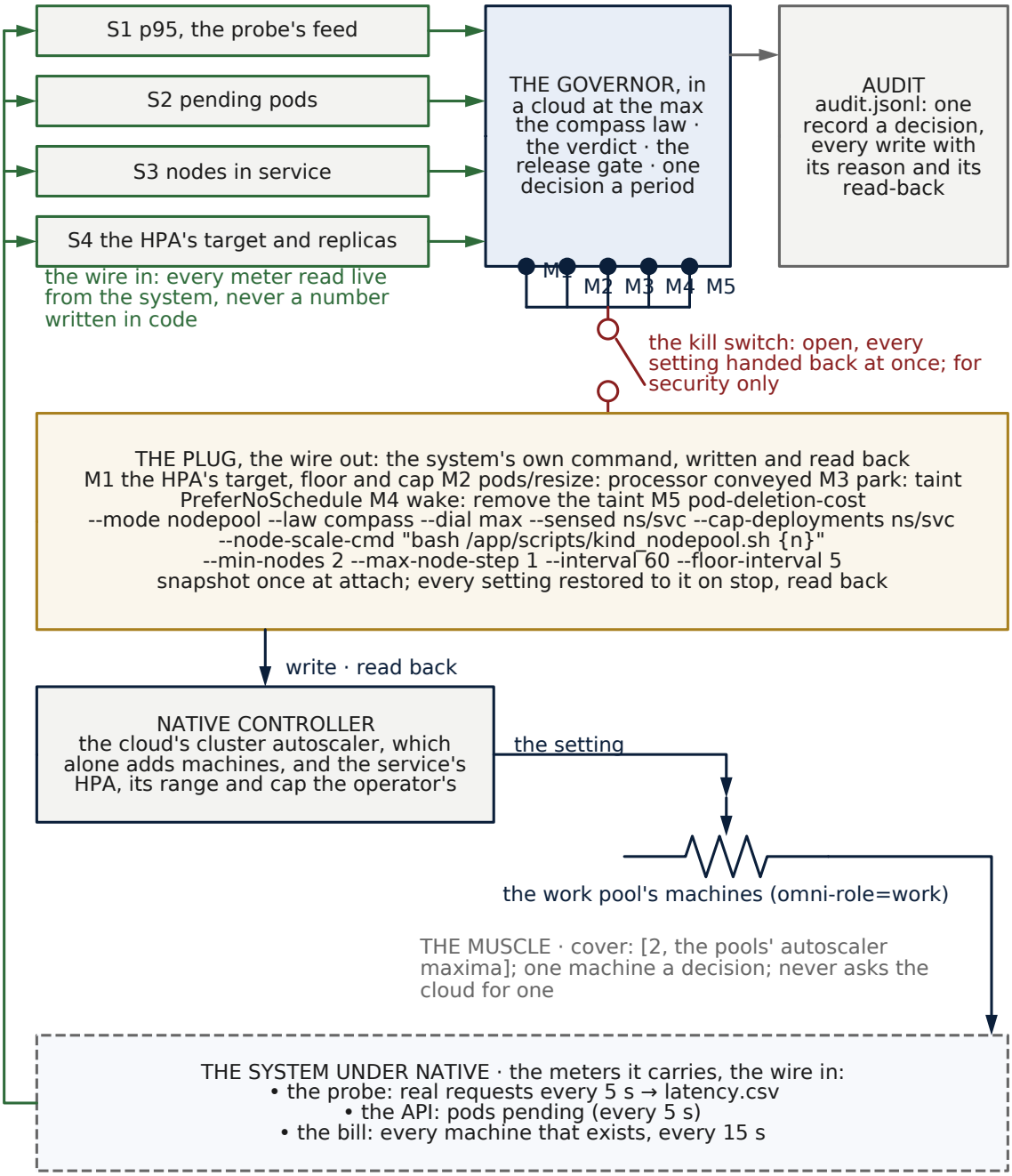


Figure 36. Wiring schematic: a managed cloud cluster, the whole engine at the max on top of the cloud's own autoscaler. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the probe's feed, the pods waiting and the machines in service; the only motors park a machine (a PreferNoSchedule taint), wake it (the taint removed) and mark its

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

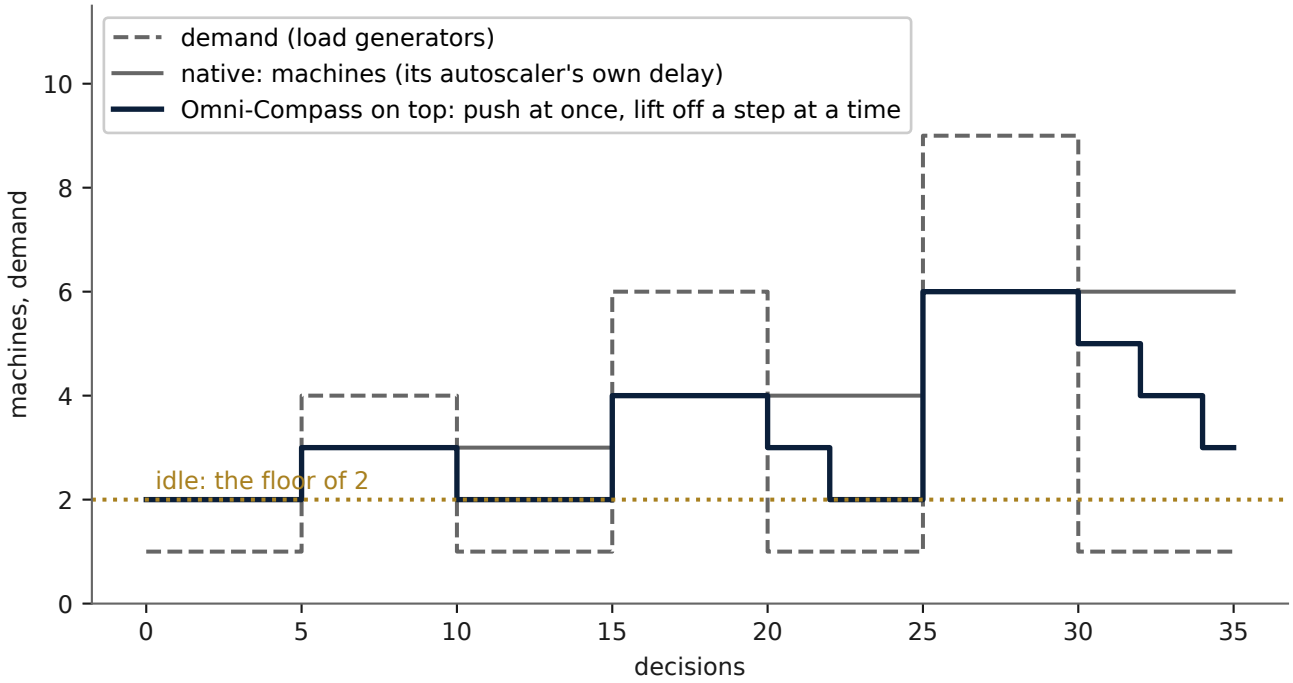


Figure 37. Modelled trace: one pedal over a burst, machines against demand (class S: the rules of the pedal drawn over the burst 1 4 1 6 1 9, five decisions a step). Native's autoscaler adds a machine when pods cannot be placed and removes one after its own delay; with Omni-Compass on top the push adds capacity at once at the wall and the lift off gives it back one machine at a time through the gate and the trial, never below the floor of 2.

pods first to go; no autoscaler is moved, no processor conveyed, no power capped. A machine is parked only after its own trial (the response time holds within 2%) and woken on a breach: at the next decision (15 s) for a response time over the line, at the next fast check (5 s) for a waiting pod. No cloud credential: the governor holds a Kubernetes service account that can read, taint and annotate, nothing more.

`docs/OMNI_V17.md`, section 3); the compass's step on the HPA target and its coasting are scaled by period / 60, so the target moves the same amount a minute, and a pool gives back at most one machine a minute at any period (`docs/OMNI_V7.md`, section 4). The default period stayed 60 seconds, because the governor core still advanced its own state one fixed step per decision, and other counts were still written in decisions. omni-v16 (`docs/OMNI_V16.md`) puts every one of them in seconds: the brain's blending of each reading, 1 − (1 − 0.339)^k a step with k = period / 60 s, so 0.339 a minute at any period; its inner time, k of its macro interval a step; its dwell and waits; the machines it adds or removes, up_max and down_max over the last minute's steps (the C++ twin identically); the compass's damping, critical at the minute, and its smoothing, 0.3 a minute; the edge's window, narrowing, forgetting and wobble, per minute; the additions past the scheduling floor, one step a minute; a command's minute to land; the host's busy share over a minute; the heat law's 0.86 a minute; one batch job a minute; the conveyance every 15 s whichever check falls due; and a loop that keeps its own schedule, skipping a missed slot. At 60 s every value is the one before, so the model gate reads omni-v15's decisions exactly (4,578 audit records, none different); the period changes only how often the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

governor looks. With that, the founder first set the heartbeat, the full decision, at 15 s (omni-v16, the morning of 22 October: "15 sec all over heart beat"), and the same afternoon, having read the reads of the cluster each pace buys (about 86,400 a day at 5 s against 39,700 at 60 s), made it a dial with the main engine at 60 s (omni-v17, `docs/OMNI_V17.md`): the default `--interval` is 60 s and `--floor-interval` 5 s in the governor, every rig and every install file; any beat may be set (5, 15, 30, 60, 75, 90, 120 s and every value between), each shown beside native with what it buys and what it costs (`results/model_gate/V17_LADDER.md`; on the live engine page, the page's section 1j, `#s-dial`), and three runs read every beat against native and against 60 s (`docs/PACE_PREREGISTRATION.md`, amendment 3). The cushion each muscle keeps is set in seconds by its own climb, wobble and reaction time, so it is the same at any pace; a longer period would not widen it, only leave the governor blind for longer between looks. In the cloud v7 also made the whole rule one flag, `--give-back-only`, enforced in the governor itself: every actuator but the node pool off, the replica ceiling 0, the dial at the tune-down (`docs/OMNI_V7.md`, section 5); the founder's order of 31 October struck that rule, and nothing passes the flag since.

What the Azure runs taught us. Three lessons, each recorded as an amendment to the preregistration before the fleet was dispatched again. First, a fleet is sized by the lever, not by the budget: a 4-worker fleet cannot show the few-percent machine saving the law makes, so its honest reading is "inside the noise", and running it again would not change that. Second, a new subscription's allowance is per machine family, so the fleet that can show one machine is built from nine families at once, each pool at its own list price, and the bill prices each pool separately. Third, a cloud region can refuse a new cluster for hours at a time (Azure's capacity for new control planes in its region, on both control-plane tiers); the harness therefore checks the subscription's allowances before it builds anything, so a refusal costs cents and not a cluster, and every refusal is recorded with its time.

46.2 A database behind its connection pooler

PostgreSQL is wired through PgBouncer, the pooler most of its deployments already run (`docs/POSTGRES_PREREGISTRATION.md`, `tools/run_pgbench.py`). Nothing models the database. The wire in is the pooler's own report (`SHOW STATS`, `SHOW POOLS`): once a second, the time a transaction spent in the server plus the time clients waited for one, over the transactions completed. The wire out is one setting through the pooler's own admin console, `SET default_pool_size`, inside the cover [2, 90]: never under two server connections, never within ten of PostgreSQL's connection limit. The compass holds that reading at 40% of a 50 ms line. Where the time was spent decides the direction: waiting for a server grows the pool; time inside the server shrinks it by one, so fewer backends fight for the same cores; calm gives back one idle server a second, and only one the pooler itself shows idle. At 95% of the line the knob is handed back to the pooler's own setting at once. If the knob is found at a value Omni-Compass did not write, it stops writing (one writer). At the end of every omni arm the pool size is restored to the snapshot and read back. The same plug fits any pooler or proxy with a console and a pool-size setting (PgBouncer, Pgpool-II, ProxySQL, the application's own pool).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

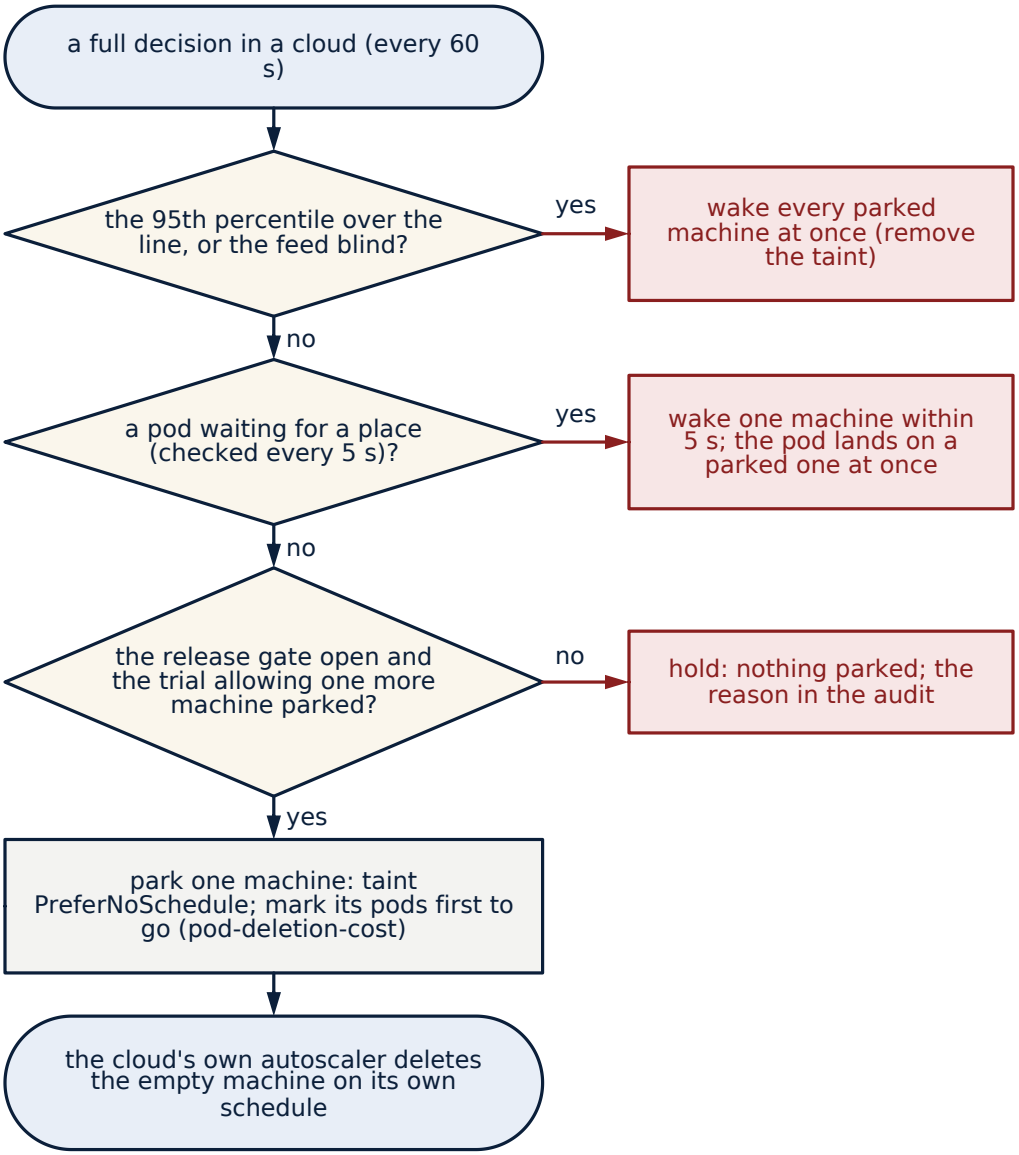


Figure 38. Flowchart: parking and waking a machine in the cloud. At every full decision: a 95th percentile over the line, or a blind feed, wakes every parked machine at once; a pod waiting for a place, checked every 5 s, wakes one machine within 5 s while the pod lands on a parked one at once; otherwise, only with the release gate open and the trial allowing it, one machine is parked (a PreferNoSchedule taint, its pods marked first to go), and the cloud's own autoscaler deletes it once empty, on its own schedule.

What the database taught us. The first untouched run on this stack was a loss, and it stays in the preregistration as it came (`docs/POSTGRES_PREREGISTRATION.md`, "The first untouched run, and amendment 1"): on the read-only workload the pooler's own clock read 0.11 ms a transaction against a 50 ms line, so the law gave back one idle connection a second until the pool stood at its floor of two, while the users' 95th percentile went from 4 ms to 3.9 s. The pooler could not see the time a client spent behind its own schedule waiting to send, so its clock said calm while the users waited; the preregistration had said so in one

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

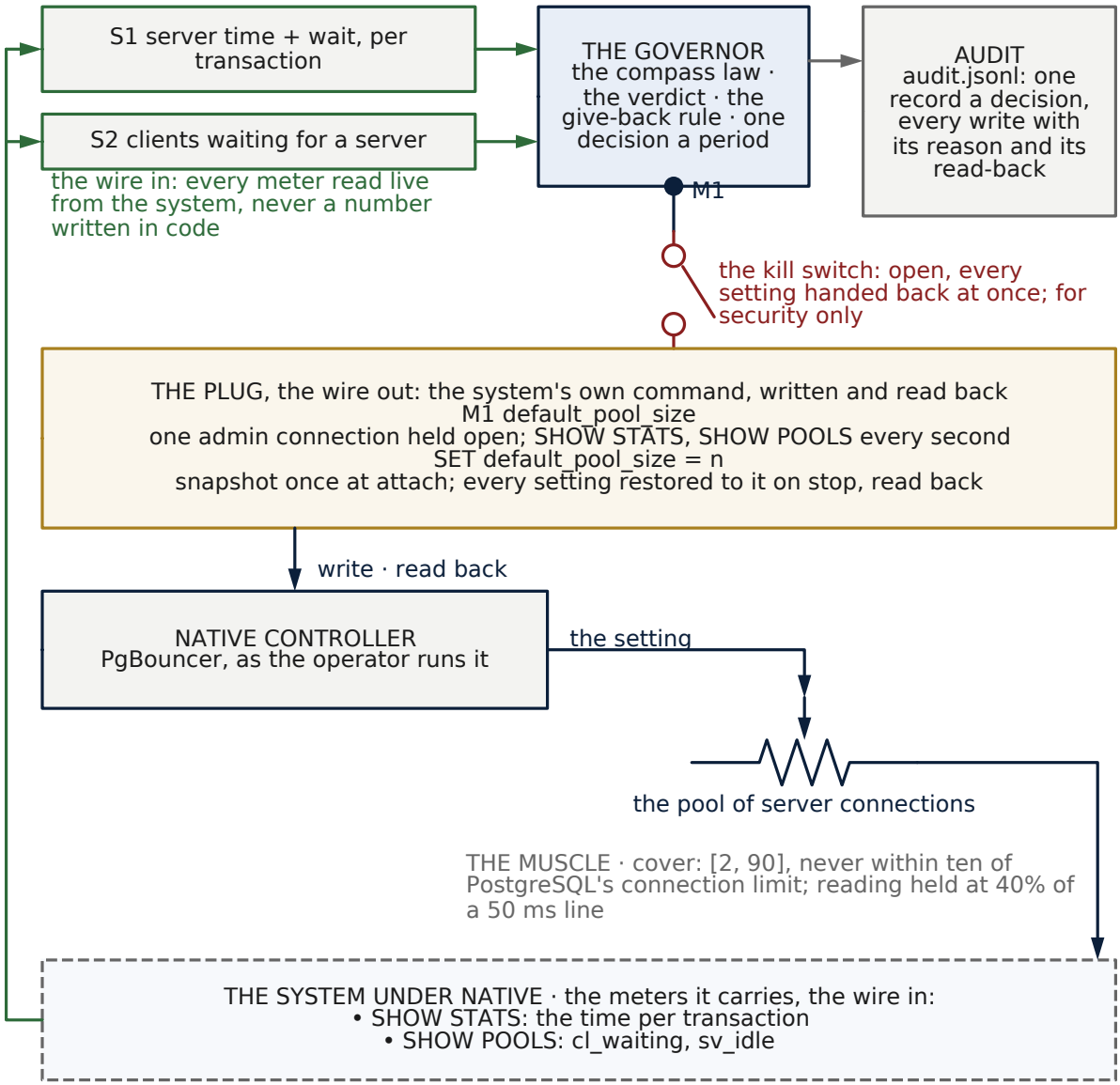


Figure 39. Wiring schematic: a database behind its connection pooler. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the server time plus the wait per transaction and the clients waiting for a server; the one motor is default_pool_size, written over one admin connection held open. Up: a server for every percent of clients waiting; down: one idle server, only while clients waited under 1%; at 95% of the line the pool goes back to the pooler's own setting at once. One writer; the restore read back at the end.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

line without drawing the consequence. Amendment 1, declared before any counted run, made the reading the larger of two: the pooler's service time, or the share of its clients queued for a server, so a pool too small for the offered rate shows at the pooler as clients waiting and the knob fails up to the operator's setting at once; and it held the dwell when lifting off only, never when pushing. The counted runs are three new untouched runs on the amended rule; the first run is not counted and is not hidden. The result the same in all three of our runs is a resource held, not speed, and its cost the same in all three of our runs is the host's CPU: the compass's own reads of the pooler's console every second are CPU the native pooler never spends, and they are counted against it.

What the cost turned out to be (amendment 2, 8 October). The record had explained the +14% to +28% of host CPU as "PgBouncer queuing clients behind a smaller pool", without a measurement. Measured, it was ours. The pooler's own log of one counted workload holds 7,956 logins to its admin console, one for every `psql` process the harness launched to take a reading (about two a second in both arms for the sampler, four more a second plus three per write in the omni arm for the brain); one launch costs about 52 ms of CPU on a machine of the runner's class, because psql loads libpq, OpenSSL, GSSAPI and LDAP and connects before it runs one command; and a paired repetition of `select` with a per-process meter put +55.9 CPU-seconds on those launches against a +52.8 s difference in the host total, with PgBouncer at +1.2 s and PostgreSQL at −4.7 s. The second cost in the table, the median latency +15% to +17%, was the take-back rule giving back one idle server a second whenever the pooler's service time read calm, which against a 50 ms line it always did, so the pool was shrunk into a queue the pooler itself was reporting. Amendment 2, declared before the second counted set (`docs/POSTGRES_PREREGISTRATION.md`): the harness speaks the pooler's wire protocol over one connection held for the whole arm, in both arms, so no process is launched for a reading; the queue line: a server is taken back only while the pooler's clients waited for one under one percent of its time in the last second and a transaction was served, and while they waited one percent or more servers are added back, one for every percent of waiting, up to the pooler's own setting (the one-percent line the two other database tests already use), so the pool probes down to the smallest size that serves without waiting and gets its servers back the moment anyone waits; and the harness records its own CPU beside the host's. The engine's 40 files are untouched (`tools/omni_version.py` prints omni-v3). The second set's table replaces the first when it lands, and the first moves whole to `docs/history`.

46.3 Independent simulators, each with its own native controller

Three published simulators are wired the same way, and the simulator's own controller is always native:

- **CityLearn** (`tools/run_citylearn.py`): every district the simulator ships; the knob is each building's battery charge and discharge inside the simulator's own limits; the native arm is the simulator's own rule-based control.
- **pandapower on SimBench grids** (`tools/run_pandapower.py`): the knob is the substation tap changer, one whole tap per move and a week of evidence before stepping down; the native arm holds the tap at 1.00 per unit as the grid ships.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

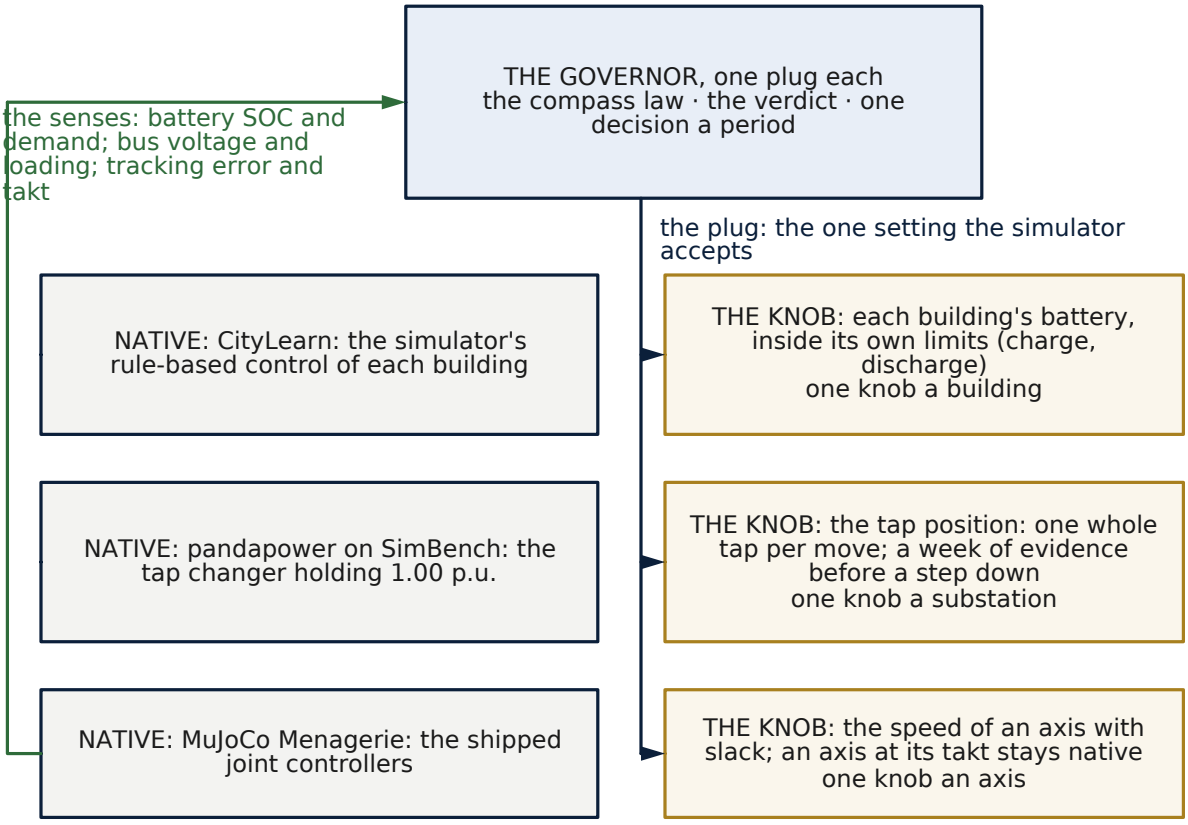


Figure 40. Wiring schematic: three independent simulators, each on its own native controller. One plug for each: CityLearn's rule-based control of each building, pandapower's tap changer on SimBench, and the shipped joint controllers of the MuJoCo Menagerie. The governor writes the one setting each simulator accepts, reads it back and restores it at the end; the senses come back along one bus. Receipts here are class S: statements about the simulator's model, never about hardware.

- **MuJoCo Menagerie robot arms** (`tools/run_mujoco.py`): the knob is the joint speed of an axis that has slack; the paired physics trial written before the run leaves an axis at its takt native, so UR5e and iiwa 14 have nothing for Omni-Compass to move and the Gen3 and Panda do.

Each is preregistered (`docs/CITYLEARN_PREREGISTRATION.md`, `docs/PANDAPOWER_PREREGISTRATION.md`, `docs/ROBOTICS_PREREGISTRATION.md`), each runs A, B and C as separate GitHub runs, and each table is made by rule (`tools/citylearn_abc.py`, `tools/pandapower_abc.py`, `tools/mujoco_abc.py`). These are evidence class S: statements about the simulator's model, never about hardware.

What the simulators taught us. Three things a referee should weigh. First, a deterministic simulator makes the three-run rule stricter, not looser: MuJoCo and pandapower reproduce to the digit on any machine, so their A, B and C must agree to the digit and any difference is a finding about the harness, while PyBullet

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

reproduces only to a part in a thousand across machines, which the swarm table discloses as an amendment and tolerates by that much and no more. Second, a knob that cannot pay is left alone by the physics, not by a rule of thumb: the robot arms' paired trial showed that two of four arms gain nothing from a speed override and left them native, the cooling model showed that a cap on cooling power can never remove less heat and so is never a lever, and a battery's reserve is spent only at the wall because every round trip through the cells costs the efficiency loss (`docs/MECHANISM_OF_ACTION.md`, 9.6). Third, a gain in one gauge is often a loss in another, and the simulators show both: the grid's losses rise in the four grids with their own generation while the energy drawn falls everywhere; the buildings' bills rise in the 2023 districts while their peaks fall; the Panda's copper loss rises while its torque falls. The tables carry every one of those rows, and the index carries none of them, because they are models.

46.4 The big organisms on a rented machine

The whole tower and the four stacked at 100 copies (about 110,700 and 190,800 modelled muscles) with a real cluster inside are also run on rented eight-core machines, **detached** (`.github/workflows/big-organism-detached.yml`), so that the cluster has more cores under it than GitHub's four-core runner gives: `start` rents one Azure machine, installs the tools, starts every repetition of every organism under `nohup` and leaves the machine running on its own; `collect` (every two hours on the clock, or by hand) copies the files back when every repetition has finished, publishes one artifact per cell and deletes the machine; `survey` lists which machine sizes and quotas a subscription may rent, region by region, and rents nothing. A machine older than `max_hours` is collected as it is and deleted, so nothing runs forever on the bill. The commit to run is an input, so a run is always tied to one engine. **The clock rule** (Chapter 57) and the rig's protocol (3 from 13 October, 4 from 18 October) decide the window for each size on each machine: the window is set from the heavier arm's step, timed on the machine before either arm, so the organism keeps its clock (the tower at 100 copies on 14 October: 2,880 s windows on three machines, ten repetitions in all; the stack follows on the same three machines). No 1,000-copy run is started from 12 October; earlier engines' 1,000-copy runs on this workflow stand as they were run, in their own tables.

The same workflow carries a second test. With the input `test=robust` the machine runs the robustness harness's long scenario instead of an organism (`scripts/kind_paired.sh` with `ROBUST=long`, exactly as the GitHub job runs it), with the window stretched to whatever `duration_s` asks and the wandering load schedule repeated to fill it; the 24-hour run of section 60.15 uses it, one paired repetition on each of three rented eight-core machines of three families (the start job tries the size it is given and falls back through the eight-core sizes the subscription may rent when the allowance refuses it; the four-core size asked for was refused because the tower's machine held its family's allowance, and the preregistration records what was rented), collected and deleted at 60 hours at the latest. The cells it writes are named `robust-long-x<window>-<repetition>` beside the organisms' `six-<organism>-x<copies>- <repetition>`, and the collect job gathers both. The `max_hours`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

input is the machine's own limit, written into the machine at start, so a collect that finds a machine past its limit takes what is there and deletes it, whichever job started it.

Three things a reader should know about any detached result. The machine is one Azure virtual machine rented for this run alone, not a GitHub runner shared with a queue of other jobs, so its size is known (the table names it) and it is not cut off at six hours; that, and not only the job limit, is why the big organisms are run there. The files come back exactly as the machine wrote them, copied cell by cell with the machine's own log beside them, and the archive bot stores them under the collect run's number with a SHA-256 list of every file, so the start run and the collect run are both cited in the table. And the machine is deleted after the collect, so a result cannot be re-read from the machine, only from the archived files; the preregistration says this before the first start.

46.5 A message broker's consumer group

Apache Kafka is wired as an operator runs it (`docs/KAFKA_PREREGISTRATION.md`, `tools/run_kafka.py`): one broker as shipped, a topic of eight partitions, a consumer group at the operator's count of two. Nothing models the broker. The wire in is the group's own end-to-end latency: each message carries its production time, each consumer records when its handling ended, and the reading is the mean over the messages consumed in the last second; a second with nothing consumed while messages wait reads as the line (someone is waiting and no one is taking), and nothing consumed with nothing waiting reads as calm. The wire out is the count of consumers in the group, which is what an operator changes, inside the cover [1, 8]: never fewer than one, never more than the topic has partitions. The compass holds the reading at 40% of a 500 ms line with a response time of 3 s, because a consumer added or removed shows only after the group rebalances. Slow service with messages waiting adds consumers in proportion to the force; calm with nothing waiting gives back one idle consumer a second after a five-second dwell; at 95% of the line every consumer the topic can use starts at once. The rebalance that every move costs is paid by Omni-Compass and counted in the gauges. At the end of every omni arm the count is handed back to the operator's and read back. The result (Chapter 60) is the shape this wiring predicts: the queue kept short, no message lost, at the cost of consumers running, which reads power given by rule and stands beside the gain.

46.6 A cache's memory ceiling

Redis is wired as Ubuntu ships it, with the two settings an operator sets for a cache, a memory ceiling of 64 MB and the allkeys-lru eviction rule (`docs/REDIS_PREREGISTRATION.md`, `tools/run_redis.py`). The application in front of it is ours and declared: 1,500 requests a second from 32 threads on a Zipf working set that widens and narrows one notch at a time; a miss costs a declared 5 ms trip to the store behind the cache and a write-back. The wire in is the application's own request latency, the mean over the last second (a hit about 0.3 ms, a miss about 5.5 ms). The wire out is one setting through Redis's own console, `CONFIG SET maxmemory`, inside the cover [16, 512] MB. The compass holds the reading at 40% of a 2 ms line. Its direction rule carries the do-no-harm gate in the form this stack needs: misses grow the ceiling by notches of 8 MB **only while the cache is full** (memory used at 90% of the ceiling or more), because

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

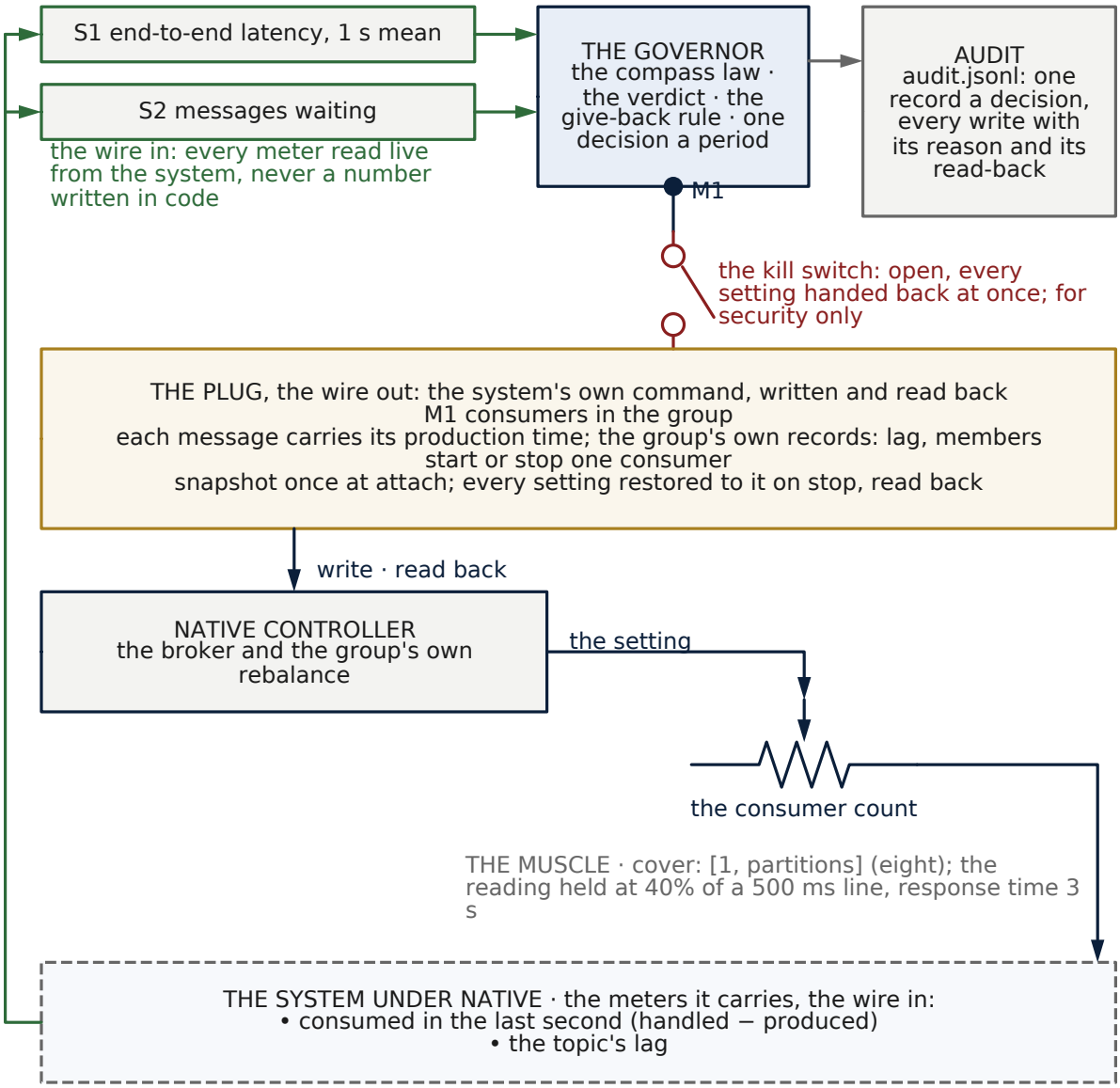


Figure 41. Wiring schematic: a message broker's consumer group. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the end-to-end latency and the messages waiting; the one motor starts or stops one consumer. Slow with messages waiting: consumers added in proportion to the force; calm with nothing waiting: one idle consumer given back a second after a five-second dwell; at 95% of the line every consumer the topic can use. The rebalance every move costs is paid by Omni-Compass and counted in the gauges.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

a miss in a cache with room to spare is a cold miss that no ceiling can mend; calm with nothing evicted in the last second gives back one notch a second, after a five-second dwell since the last growth; at 95% of the line with the cache full a quarter of the cover is added at once. One writer: a ceiling found at a value Omni-Compass did not write stops it. At the end of every omni arm the ceiling is handed back to the operator's 64 MB and read back. The memory the compass holds for a wide working set is the resource this benchmark trades and reads power given by rule; the gain is the hit rate and the work inside the line. The result (section 60.7) is exactly that shape: on all three untouched workloads the work inside the line and the hit rate rose 14% to 27% and the ceiling held rose from 64 MB to 200 to 270 MB, with the host's CPU inside the noise and every ceiling handed back.

46.7 Drone swarms: the autopilot's cruise

In gym-pybullet-drones (University of Toronto, MIT License; `docs/SWARM_PREREGISTRATION.md`, `tools/run_swarm.py`) every Crazyflie 2.x flies its shipped position controller along the planner's path at the planner's cruise: that is native. The wire in is each drone's tracking error, the distance between the drone and the point its autopilot was told to be at, on a band from 0 to a declared safe error of 0.25 m. The wire out is one knob a drone, the cruise override: the fraction of the planner's cruise the carrot moves at, never under 1.0 (the planner's own) and never over 2.0 (a quarter of the autopilot's own speed limit), moving by at most 0.01 a control tick. With slack the compass spends it as speed; as the error grows it eases; at 95% of the safe error the cruise is the planner's at once. Two guards belong to this stack: a **separation wall** (a drone with another within 0.5 m flies the planner's cruise, whatever the error reads) and the **paired physics trial** before the counted missions, which asks the simulator's own figures whether a faster mission costs less energy at all and keeps the error under the safe error; where it does not, the override is left native for that cell and the cell reads "nothing for Omni to move". Energy is a declared model from the simulator's own motor constants, said to be a model wherever it is printed; a collision voids the cell. At every landing and at the end of the run the override is handed back and read back.

46.8 A database's storage-engine cache

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

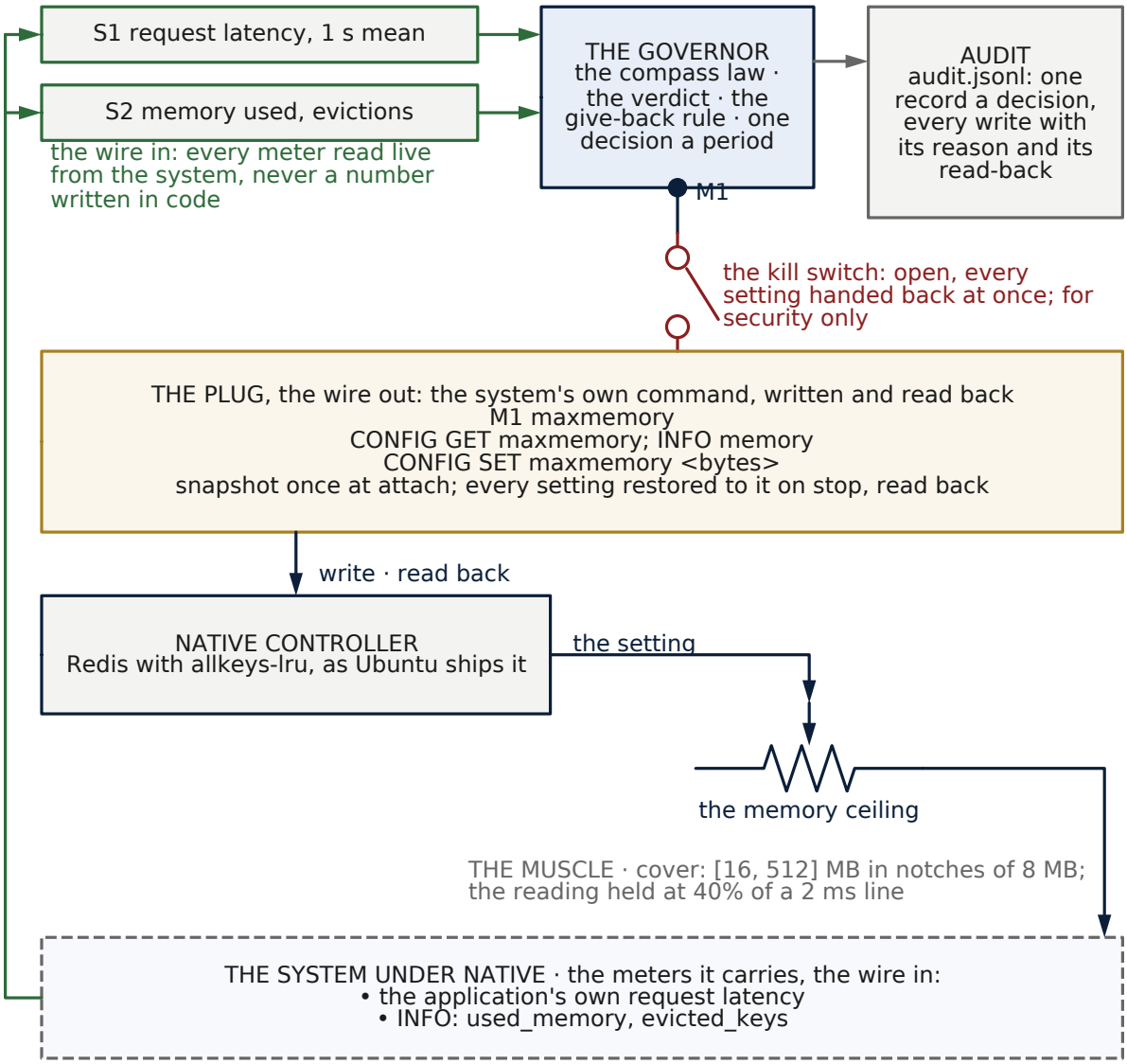


Figure 42. Wiring schematic: a cache's memory ceiling. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the request latency and the memory used with the evictions; the one motor is maxmemory, written with CONFIG SET. Misses grow the ceiling only while the cache is full (used ≥ 90% of the ceiling); calm with nothing evicted gives one notch back a second after a five-second dwell; at 95% of the line with the cache full, a quarter of the cover.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

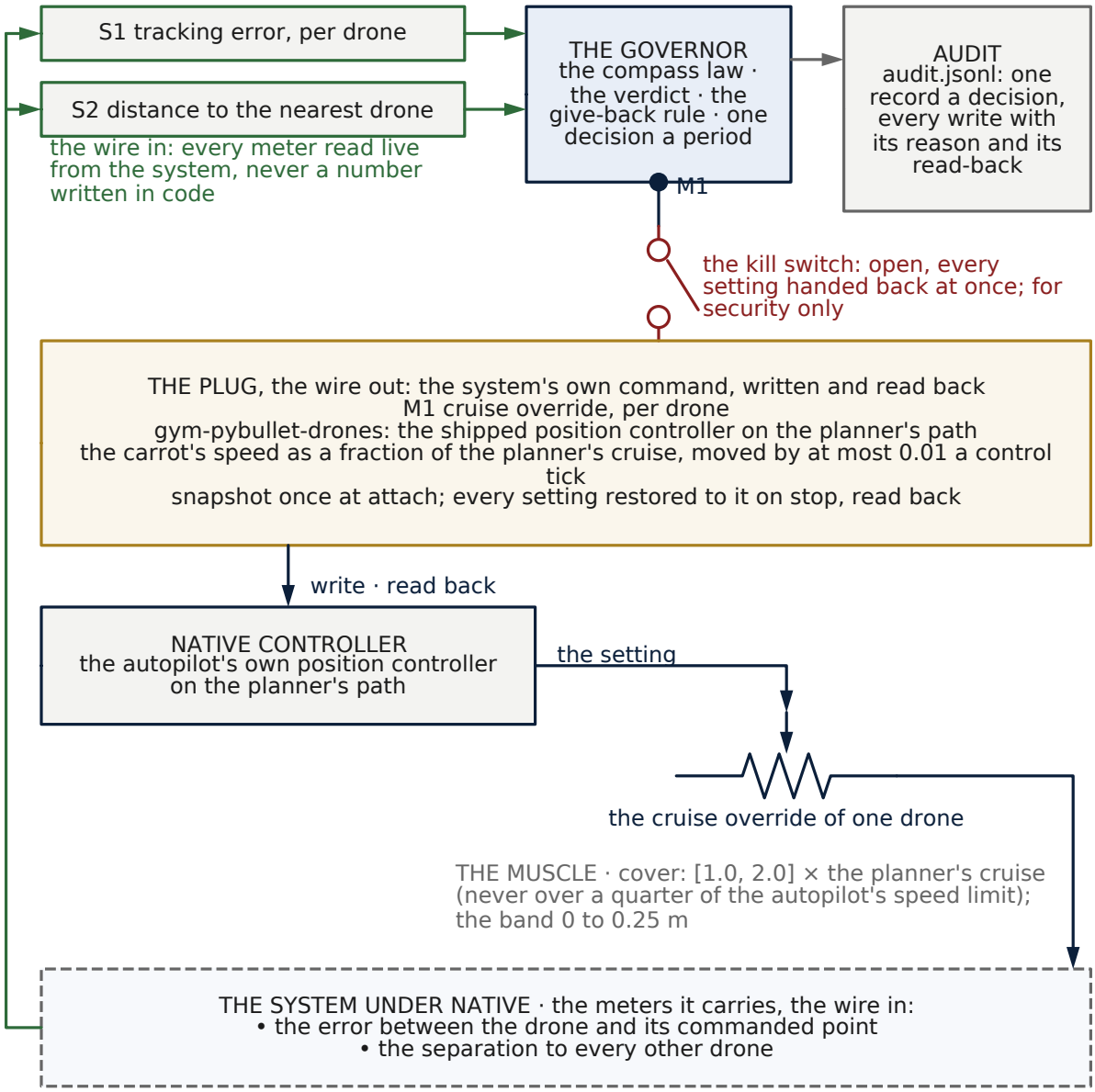


Figure 43. Wiring schematic: a drone swarm's cruise override. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are each drone's tracking error and its distance to the nearest drone; the one motor is the cruise override of one drone, a fraction of the planner's cruise moved by at most 0.01 a control tick. With slack the compass spends it as speed; as the error grows it eases; at 95% of the safe error the planner's cruise at once. The separation wall: a drone within 0.5 m of another flies the planner's cruise whatever the error reads. A collision voids the cell.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

MongoDB is wired as an operator runs it (`docs/YCSB_PREREGISTRATION.md`, `tools/run_ycsb.py`): the 8.0 series from its publisher's own signed repository, its configuration as shipped but for the one setting an operator sets for the storage engine, the WiredTiger cache, at 512 MB. The questions are asked by YCSB, the published cloud-serving benchmark, running its own core workloads as the project ships them, with the key space stepping one notch at a time through the cache and past it, so the working set fits the operator's cache at the low notches and outgrows it at the high ones. The wire in is the server's own mean read latency over the last second, from its `serverStatus` operation latencies, differenced. The wire out is one setting through the server's own console, `setParameter wiredTigerEngineRuntimeConfig cache_size`, inside the cover [256 MB, 2,048 MB]: the server's own floor at one end, a quarter of the machine at the other. The compass holds the reading at 40% of a 1 ms line (written as 2 ms before the first smoke run and set at 1 ms on the tuning workload's figures, as the preregistration reserved; see below). The direction rule is the cache's: slow reads grow the cache by notches of 64 MB only while the cache is full (bytes in it at 90% of its size or more), because slow reads in a cache with room to spare are not the cache's to mend; calm with no page evicted in the last second gives back one notch a second after a five-second dwell; at 95% of the line with the cache full a quarter of the cover is added at once. One writer, read-back and the hand-back at the end are the plug's, as everywhere. *Amendment 1 (8 October, declared before the second counted set): the give-back gate is now the cache's miss share, pages read into it under one percent of the pages requested from it in the last second, the MySQL test's gate, and growth is gated on missing the same way; the first set's audits showed that "no page evicted" is satisfied by every cold cache while it fills, so every burst and c arm gave four notches back in its first five seconds, before its first eviction, and then sat at the floor of the cover evicting for the rest of the arm. The memory the first set credited was that artefact, and the second set is expected to credit less, perhaps nothing beyond the noise.* Disclosed before the run: on a machine where the data files fit in the operating system's own page cache, a storage-engine cache miss is a read from memory and a decompression, not a disk read, so the gain available to this knob is smaller than it would be on a machine whose data does not fit in memory; the result will say what it is. The same plug fits any store whose engine exposes a cache size at run time (InnoDB's buffer pool, RocksDB's block cache) and any store that does not can only be sized at restart, which is not a knob Omni-Compass moves.

What the smoke runs taught, and why they are in the record. A benchmark of a cache is only a benchmark if the working set reaches the cache. Three smoke runs of the tuning workload, one repetition each and none counted, were needed before the harness asked the question it was built to ask, and each is written in the preregistration with its run number and its figures. The first failed before an arm ran, on two harness faults (a relative file path handed to a program that runs from its own directory; a server log copied as a file the upload step could not read). The second ran end to end and showed the design did not touch the cache at all: YCSB's shipped request distribution is zipfian, which concentrates almost every request on a few thousand hot records whatever the key space, so at the top notch of 1.5 million records the cache held 36 MB of its 512 MB, both arms answered inside the line with a p95 of 0.20 ms, and omni, seeing calm and nothing evicted, gave memory back to the floor of the cover. That would have read as a free memory saving for Omni-Compass, and it would have been worthless: a knob with nothing to do had moved to its floor. The requests were changed to a uniform draw over the notch's key space so the working set is the key space, the one departure from YCSB as

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

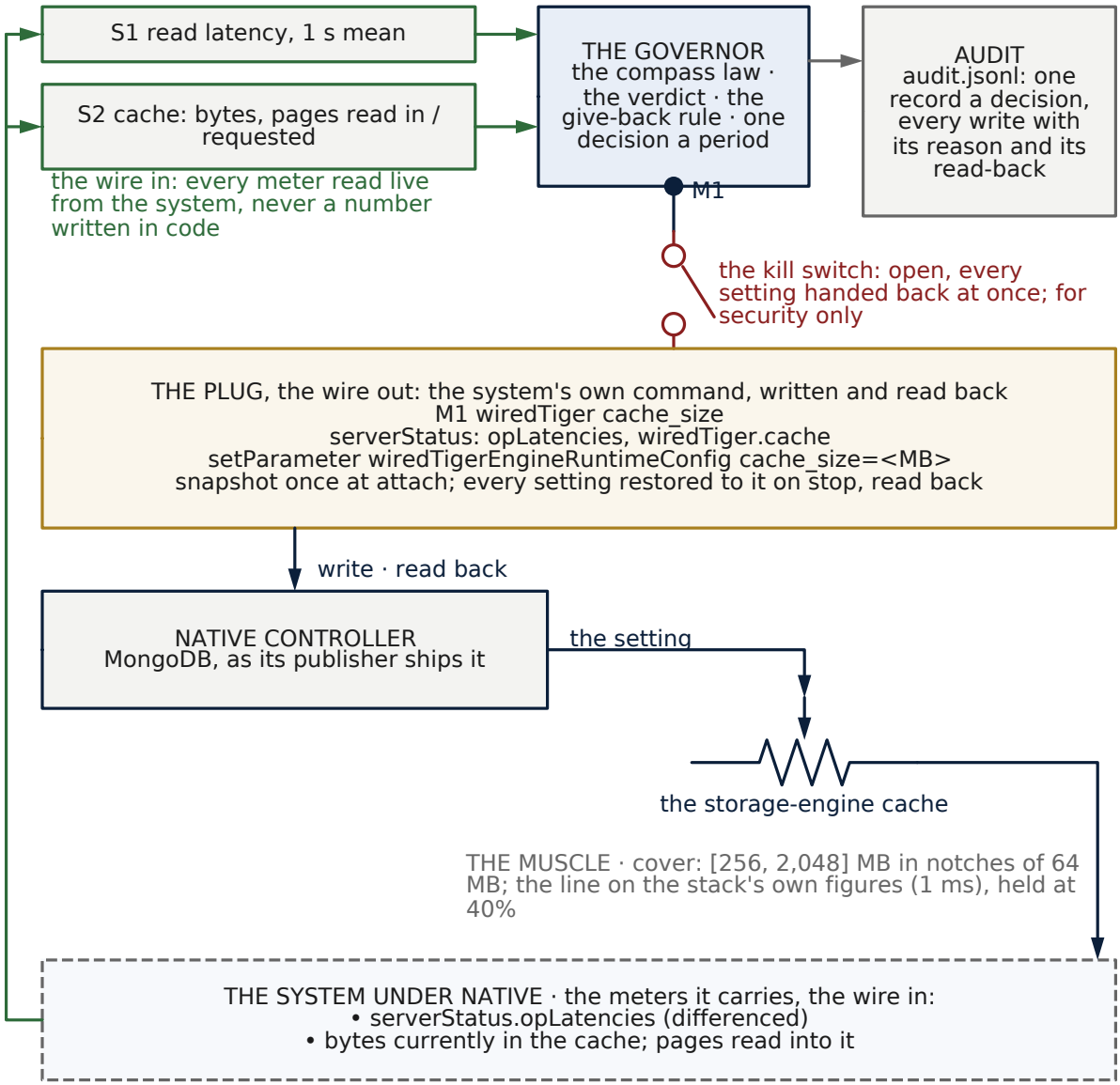


Figure 44. Wiring schematic: a database's storage-engine cache. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the read latency and the cache's bytes with its pages read in against pages requested; the one motor is the WiredTiger cache size, written through setParameter. Grow only while the cache is full (≥ 90%) and missing (pages read in ≥ 1% of pages requested); give one notch back after a five-second dwell only while the miss share is under 1%; at 95% of the line with the cache full, a quarter of the cover.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

shipped, and said so. The third smoke still held 38 MB, and the per-notch figures the runner now prints showed why: the dataset was loaded with ordered keys but the run phase used YCSB's default hashed keys, so every read asked for a record that did not exist and was answered from the index alone, which fits in a few megabytes. The run phase now names ordered keys too. Nothing in the rules, the gauges or the workloads changed across the three. The fourth smoke reached the cache: native held 336 to 481 MB of its 512 MB across the notches and read 453,000 pages into it, and its per-notch figures set the line. The server's mean read latency ran 0.15 to 0.24 ms while the cache held the working set and 0.23 to 0.33 ms while it did not, with a p95 never above 0.42 ms; against a 2 ms line every one of those readings sat in the bottom quarter of the band, under the 0.4 center, so the compass could only ever read calm, the same fault as a Kubernetes line set at ten times the service's normal response. The line was set at 1 ms, the one change the preregistration had reserved for the tuning workload, with the center unchanged, so that a hit is calm and a full cache reading past 0.4 ms is slow; the one repetition's whole-arm figures (work inside the line −1.9%, cache held −31%, pages read +32%, host CPU +8%, p95 equal) are recorded and not counted. The point for a referee is the method: a result that flatters the governor is the first thing to suspect, the suspicion is pursued to a harness cause, the cause is written down with the run that showed it, a band is set on the tuning case's own figures and never on an untouched one, and the counted runs begin only when the harness provably asks the question.

46.9 A database's buffer pool, resized online in the server's own chunks

MySQL is wired as an operator runs it (`docs/MYSQL_PREREGISTRATION.md`, `tools/run_sysbench.py`): the 8.0 series from Ubuntu's own package, its configuration as shipped but for the settings an operator sets for InnoDB, the buffer pool at 512 MB (MySQL's own default is 128 MB, and the preregistration says why 512 MB is the operator's setting here), the chunk the server resizes in at its shipped 128 MB, one pool instance, and the performance schema on. The questions are asked by sysbench, the standard OLTP benchmark Ubuntu ships as its own package, running its published scripts as shipped at a fixed offered rate, with the number of tables in use stepping one notch at a time through the pool and past it. The wire in is the server's own mean statement latency over the last second, from the performance schema's statement summary, differenced. The wire out is one statement through the server's own console, `SET GLOBAL innodb_buffer_pool_size`, inside the cover [128 MB, 2,048 MB] and always in whole chunks, because that is how the server itself moves the pool. Two things about this knob differ from the MongoDB cache and the plug carries both. The server resizes asynchronously and reports its progress in its own status variable, so the plug waits for the resize to complete before reading back, and a pool found mid-resize is the server still carrying out Omni-Compass's own write, not a foreign hand. And a shrink evicts pages while the resize runs, so the dwell after any move is ten seconds rather than five. The compass holds the reading at 40% of a 0.6 ms statement line (written as 1 ms, set on the tuning workload's smoke runs, as the MongoDB line was: a hit reads 0.14 to 0.22 ms across GitHub's runners and a full pool missing a third of its reads 0.05 to 0.06 ms more, so the center sits at 0.24 ms, above the hits and under the misses); the direction rule is the cache's with one difference the smoke runs forced: slow statements grow the pool by chunks only while the pool is full, at 95% of the line with the pool full four chunks are added at once, and calm gives one chunk back after the dwell only while the pool's misses are under one percent of its read requests, because

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

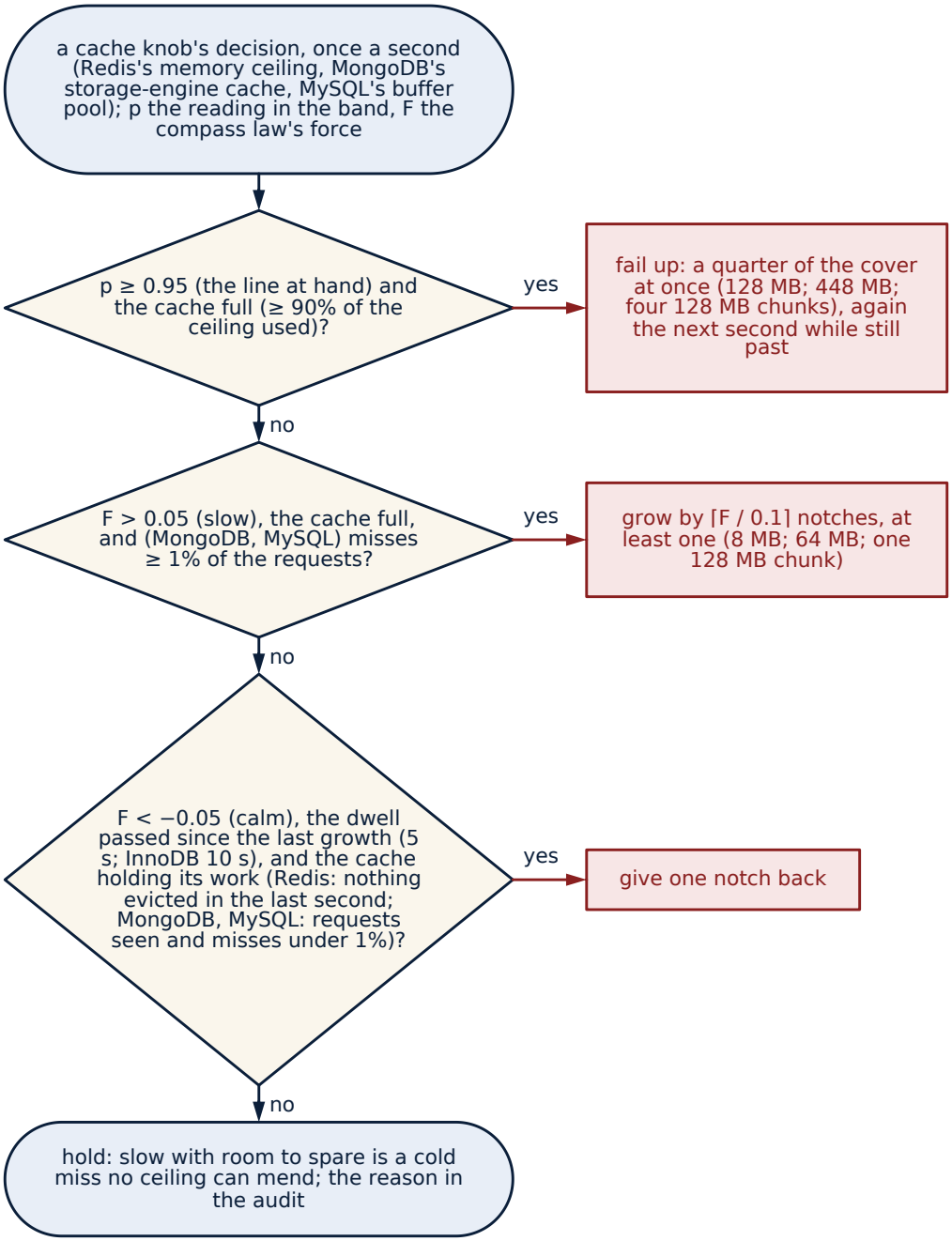


Figure 45. Flowchart: the direction rule of a cache knob (Redis, MongoDB, MySQL), once a second, tested in this order. With the reading at 95% of the line or more and the cache full (90% of its ceiling used), a quarter of the cover is added at once (128 MB; 448 MB; four 128 MB chunks), again the next second while still past. Slow (the compass law's force over 0.05) with the cache full, and for MongoDB and MySQL with misses at 1% of the requests or more, the knob grows by ⌈F / 0.1⌉ notches, at least one (8 MB; 64 MB; one 128 MB chunk).

Calm (the force under −0.05), the dwell passed since the last growth (5 s; 10 s for InnoDB) and the cache holding its work (Redis: nothing evicted in the last second; MongoDB and MySQL: requests seen and misses

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

under 1%), one notch is given back. Otherwise the knob holds, because a slow read in a cache with room is a cold miss no ceiling can mend.

InnoDB keeps stale pages resident, so neither "pages free" nor "no page read from disk" ever says that a pool holds its working set, and the miss share does (amendment 2 of 8 October, declared before the third counted set: the pool also grows only while it is missing, one percent or more, the same line the other way, because the second set's audits showed 36 of read_write's 82 grows bought chunks while the pool missed under one percent of its reads, for slow writes the pool cannot mend; and no chunk is given back in a second with no read request). The work-inside-the-line gauge uses a transaction line, the statement line times the statements a transaction as sysbench's script ships it (one for a point select, fourteen for the read-only mix), and says so. One writer, read-back and the hand-back at the end are the plug's, as everywhere. The same disclosure as MongoDB's holds and is made before the run: on a 16 GB machine with a 1.4 GB dataset a pool miss is a read from the operating system's page cache, not from disk.

What the first counted set taught about this knob. Three runs of five workloads (`results/live/V3_SYSBENCH.md`, 8 October) read every gauge inside the noise, and 15 of the 45 omni arms did not read the operator's pool back at the end. The audits show the server still "withdrawing blocks" of a shrink the law had asked for when the arm ended: InnoDB withdraws the last blocks of a shrink only when the pages the load has pinned are released, so under load the withdrawal does not finish, and the server ignores a new `innodb_buffer_pool_size` while a resize is in progress, so the restore the plug issued at that moment was dropped. Every following arm began from a fresh server at the operator's setting, so no other arm was affected, and the decision loop never wrote during a resize (its writes are gated on the status); only the restore was. The plug now waits for the resize the server is carrying out (the load has stopped by then, so a stalled withdrawal finishes), writes the snapshot, and writes once more if the server ignored the first write; the receipt of that restore (what was in flight, the seconds waited, the writes, the final read-back) is in every omni arm's record and the table counts the arms. The rows of the first set stand as they are, hand-back NO included, in `docs/history/V3_SYSBENCH_set1.md`; the preregistration carries the finding and the amendment; and the second counted set, run on the amended plug the same afternoon with nothing else changed, handed the pool back on all 45 omni arms and is the result in section 60.7. The lesson is general and is the reason section 32.2 exists: a knob the device applies on its own time has to be handed back on the device's time, and the proof of the hand-back is the read-back, not the write.

46.10 The machine itself: the kernel's frequency governor under the processor's meter

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

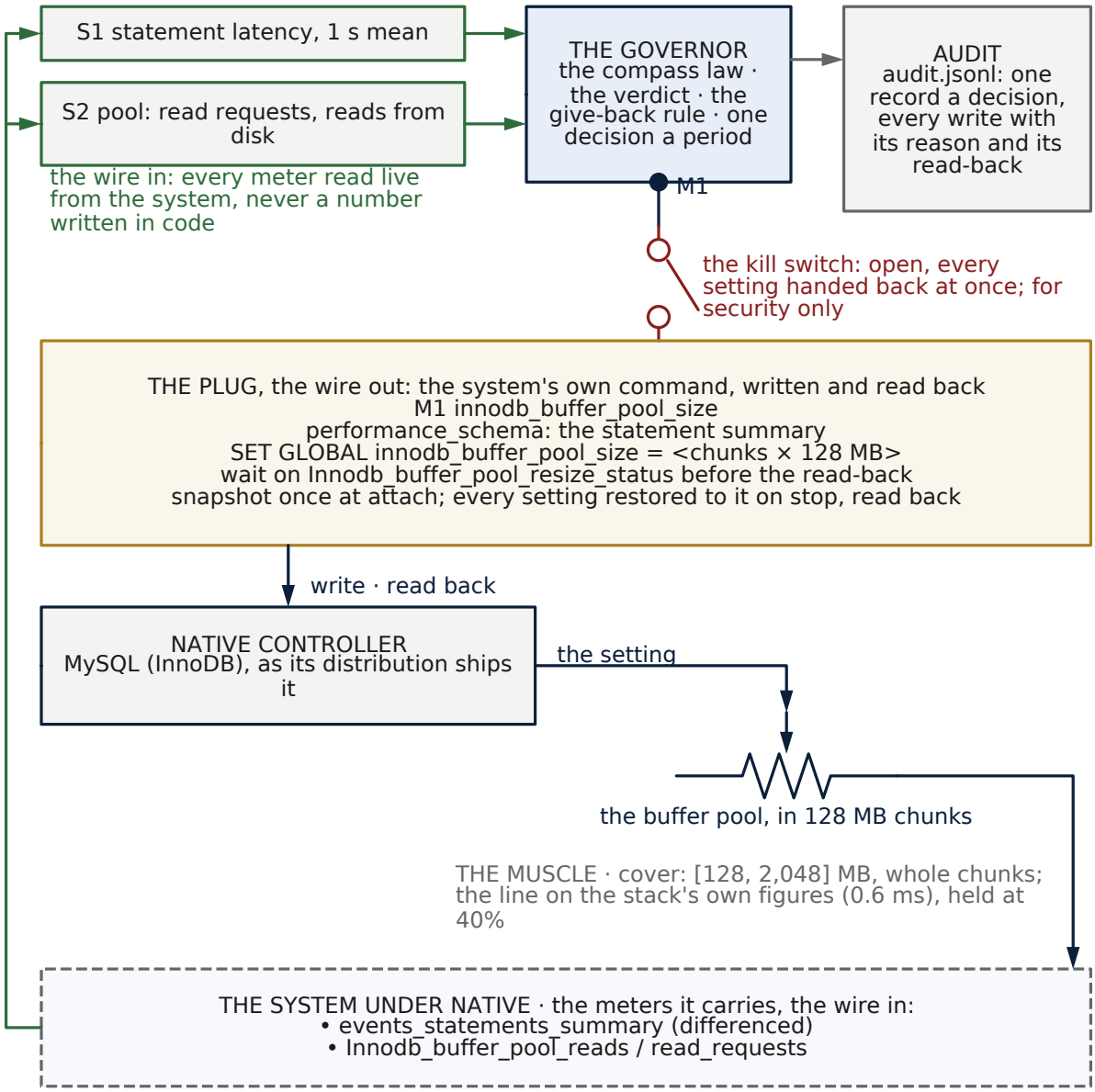


Figure 46. Wiring schematic: a buffer pool resized online in the server's own chunks. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the statement latency and the pool's read requests against reads from disk; the one motor is innodb_buffer_pool_size, in whole 128 MB chunks. The server resizes on its own time: the plug waits for the resize before reading back, and the dwell after a move is ten seconds; the hand-back waits for the server too, writes again if ignored, and records what it waited for.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The lowest level a governor can sit on is the machine it runs on. Linux already has a controller for the processor's clock, the frequency governor (`schedutil`, or the processor driver's own), choosing the clock every few milliseconds under a ceiling the operator sets once; and the processor has its own energy meter (RAPL) ticking in microjoules. Omni-Compass sits on top of the governor exactly as it sits on the HPA: the governor keeps running, and Omni moves one knob, the frequency ceiling of every CPU, through the kernel's own files (`tools/run_cpu_power.py`, `docs/CPU_POWER_PREREGISTRATION.md`).

The reading is the service's own mean latency, held at 40% of a 20 ms line by the compass law; calm gives one notch of the ceiling back (5% of the top clock), a slow service raises it by notches, the wall puts the top on at once. Every notch down is tried on the machine itself first (section 45.4): the brain's verdict around the engine's own, eight cost samples at the deepest notch already allowed and eight one notch further, allowed only if the cost does not rise within 2%. The cost is the index's own reading for one machine (package power × latency / work inside the line), so a notch that saves less energy than it costs in speed is refused and the ceiling stays where the operator left it; a second declared objective, energy per request inside the line with the line as the guardrail, is run beside it. The snapshot of every CPU's governor and ceiling is taken once; the ceiling is handed back and read back at the end; a ceiling found at a value Omni did not write stops it; the master switch and the lease hand back if Omni dies.

The meter is the processor's own counter, class P with the caveat that it is the processor's calibrated counter and not an external meter; a smart plug on the wall socket (`WALL_METER`) adds the whole machine, read by the harness only. GitHub's own machines are virtual and expose neither the governor nor the meter: the workflow proves it with the probe on every dispatch. The benchmark runs on a machine on the metal as root, in one command (`sudo bash scripts/cpu_power_run.sh`): the founder's own tower or laptop on Linux, a rented bare-metal server, or a self-hosted runner on either. Three runs make the table (`tools/cpu_power_abc.py`). Nothing in the engine changes.

46.11 The data center under the servers: cooling, power and batteries

Under every cluster in this book stands a building. Its native controllers are older and more conservative than any autoscaler: the building management system holds the supply-air temperature through the cooling units' own loops, the chilled-water plant runs its own sequence, the uninterruptible power supply holds its reserve, the power distribution units meter every rack, and every server's management controller and every card's firmware enforce their own power limits. Omni-Compass sits on top of them exactly as it sits on the HPA. They keep running; it moves only the settings they already accept, and only inside the band the operator writes down.

What it may set, and what it never sets.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

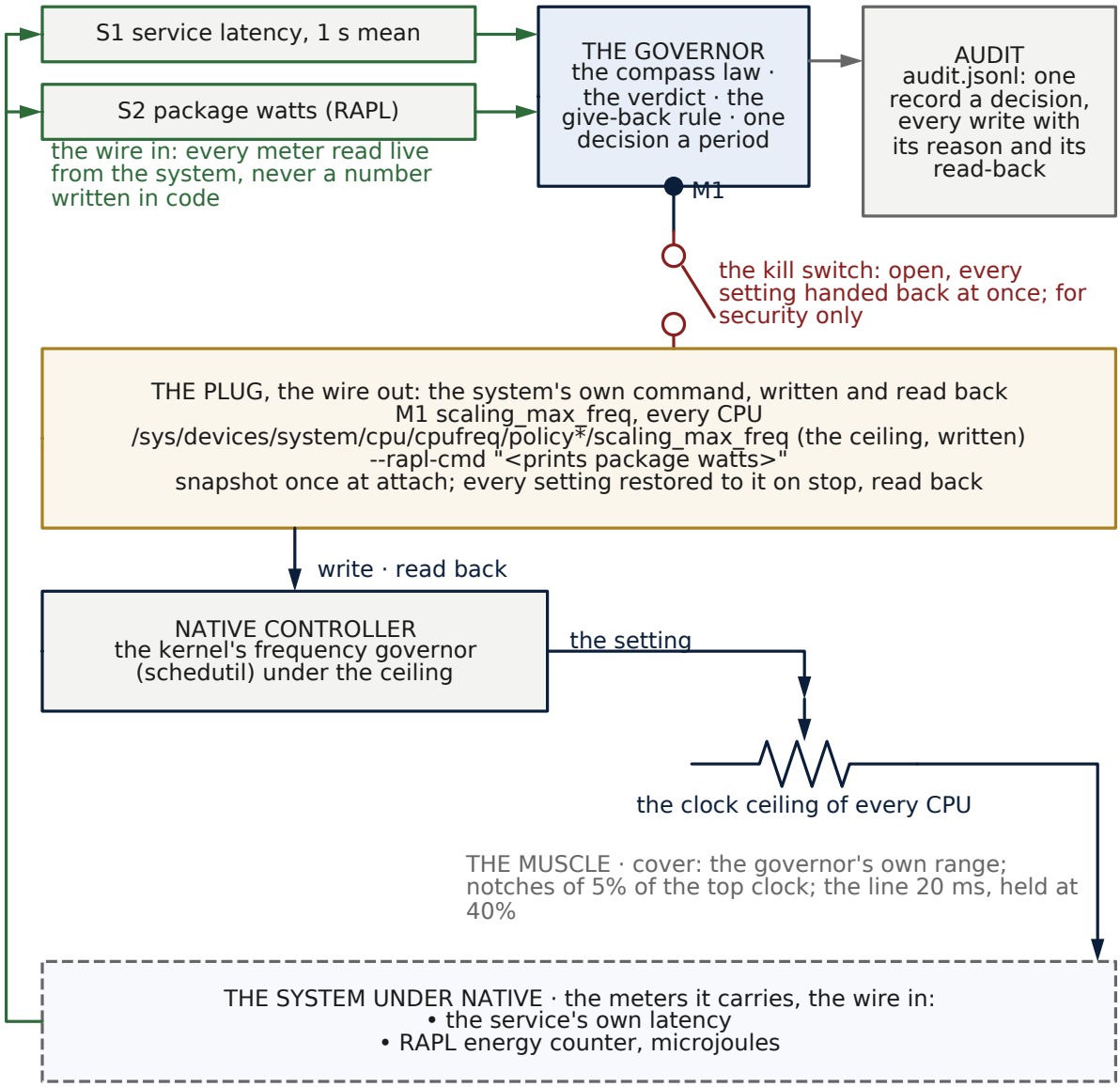


Figure 47. Wiring schematic: the machine's frequency governor under the processor's meter. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the service's latency and the package watts from RAPL; the one motor is scaling_max_freq, the ceiling of every CPU's frequency policy. Every notch down is tried on the machine first: eight cost samples at the deepest notch allowed and eight one further, allowed only if package power × latency / work inside the line does not rise within 2%. A snapshot of every policy, restored exactly.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

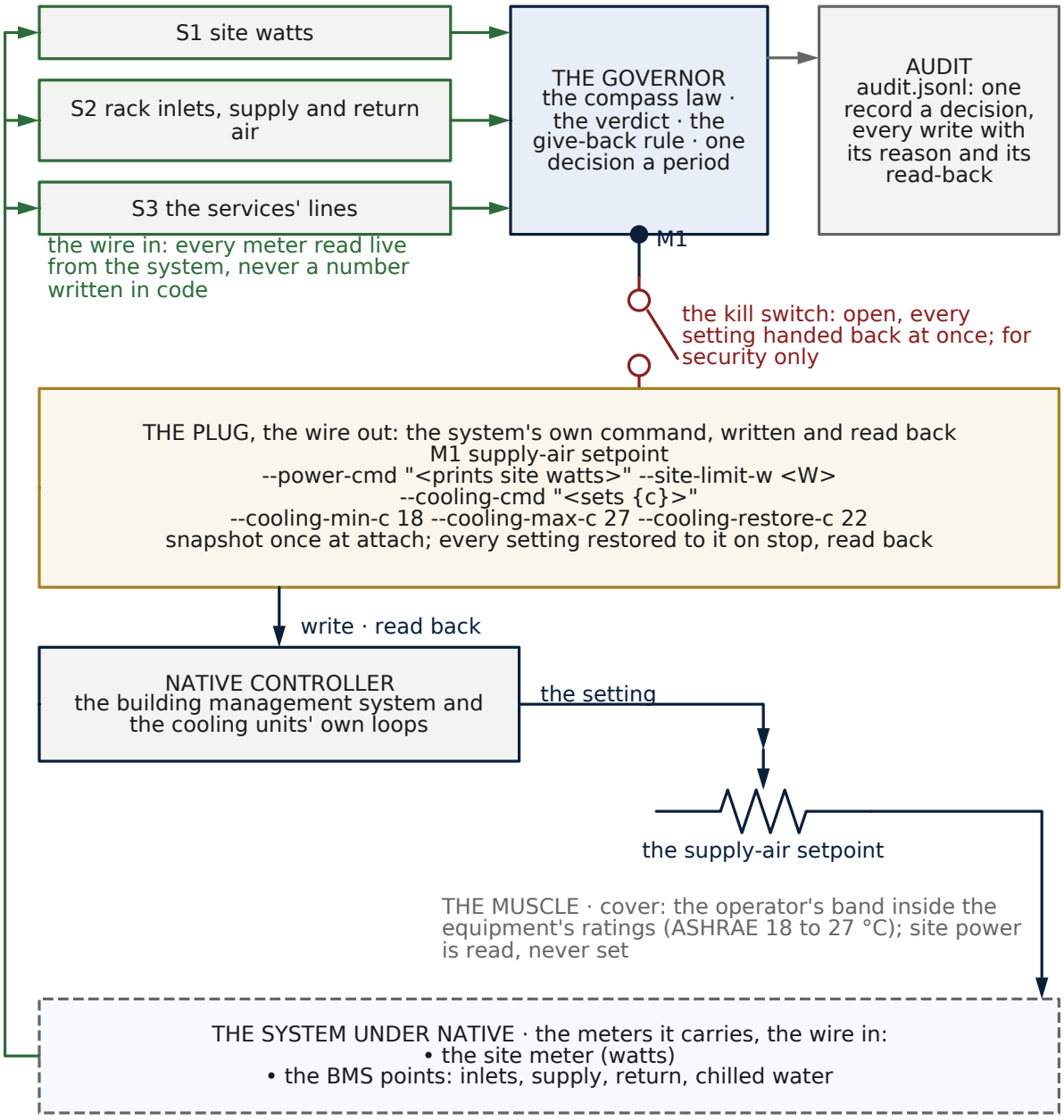


Figure 48. Wiring schematic: the data center under the servers. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the site watts, the rack inlets with the supply and return air, and the services' lines; the one motor is the supply-air setpoint, inside the operator's band. Warmer while every reading is cool, colder as heat rises; a step allowed only where the paired trial at the site meter shows the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

site's energy no higher for the same work, every inlet inside its band and no LOSS on any service gauge. A reserve held for an outage (the UPS) is never a lever. Modelled today (class S); no live facility has run yet.

Muscle	Native controller	The knob Omni-Compass may hold	Switch	Status
Cooling	the building management system and the cooling units' own loops	the supply-air setpoint, inside the operator's band	`--cooling-cmd "<sets {c}>" --cooling-min-c --cooling-max-c --cooling-restore-c`	built (level 7)
Site power	the site's own limit and its breakers	none: site watts are read, and the site limit enters the engine as a stress	`--power-cmd "<prints site watts>" --site-limit-w <limit>`	built (level 7)
Servers' processors	the kernel's frequency governor and the processor's own limits	the clock ceiling and the power limit	section 46.10	built (level 6)
Cards	the card's firmware	the clock ceiling and the power limit	level 5	built
Uninterruptible power and batteries held as reserve	the UPS's own reserve	**none, ever**: a reserve held for an outage is never a lever (the physics criterion)		native by design
Batteries held for shifting energy	the site's dispatch	the charge window		designed as an organ, not yet wired

How the cooling muscle moves. The setpoint is moved by the heat state between the operator's minimum and maximum: a warmer setpoint while every reading is cool, which saves the chiller's energy, and a colder one as heat rises. The band must sit inside what the equipment is rated for; for most data-center equipment that is the 18 to 27 °C inlet range of the ASHRAE thermal guidelines, and inside any narrower limit the equipment's own warranty sets. The restore value is the operator's own setpoint, handed back and read back on OFF, at the end of every run, and by the lease if the governor dies.

The do-no-harm gate at the site meter. A warmer setpoint saves chiller energy only up to the point where the servers' own fans spend the saving: fan power rises steeply with inlet temperature. That is why the cooling

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

knob is judged at the site meter and not at the chiller: a step is allowed only where the paired trial shows the site's energy for the same work no higher, every rack inlet inside its band, and no LOSS on any service gauge. Where it does not, the setpoint stays the operator's, and the result says so.

Wiring it, in order.

1. **Watch.** Give Omni-Compass read access to the building management system's points (supply and return air, every rack inlet, the chilled-water temperatures) and to the site meter, and let it watch at least one full cycle of the site's demand. Nothing is written.
2. **Write the band.** The minimum, the maximum and the restore value of the supply-air setpoint, inside the equipment's ratings, signed off by the facility's engineer; the site limit in watts.
3. **The wire check** (section 42.4). The cooling command must read back the setpoint it wrote; a building schedule that rewrites the setpoint is another writer, and Omni-Compass stops writing that lever by itself (section 32.2).
4. **One hall first.** A test hall or a single row, paired against its own native weeks, before any other hall.
5. **Keep the switches in reach.** The master switch and the kill file work at the facility level exactly as at every other level.

What is measured, and what is modelled. The facility muscles are modelled today (class S): the data-hall cooling family of the catalog, the power distribution family, and the site budget shared by groups of cards, which held out with no minute over its budget (`docs/REGISTER.md`: the families of rows 29 and 38 of its catalog table, and row 20 of its benchmark table). No live facility has run yet. A class P result needs a building management system's point list, read access, a site meter and a test hall; until a facility pilot has produced its own paired receipts, this section is a wiring instruction, not a claim of savings.

CHAPTER 47

The Integration Manual

Before you wire anything: read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 42.5 of the manual (*Wired right or wired wrong*).

Which knobs to wire in at all: `docs/WIRING_VERDICTS.md` gives every knob in every published result one of three words from the tables themselves: **write** (wire both ways), **watch** (wire out only: Omni reads, the native controller lives by itself) or **operator's choice** (a trade). A knob that showed nothing, or

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

> lost, stays native. How to wire for superiority, step by step (watch first, the observation run, the brain's own verdict on each knob, the recheck): the manual, section 45.4.

How to wire Omni-Compass into your own systems yourself, from watching only to running your stack from the top. This manual ships in the box with the code and the license. Nobody from The Omni-Compass LLC needs to be on site.

Read section 47.1 once, then take the levels in section 47.4 in order. Each level has a pass condition; go to the next level only when it holds. The OFF switch (section 47.2) works at every level.

47.1 What is in the box

Item	Where
The engine, nervous system, safety shield, compass and conveyance law	`omnicompass/`
The Kubernetes controller and its muscles	`omni_controller/controller.py`, `omni_controller/muscles.py`
The GPU governor (one GPU box, no Kubernetes needed)	`omni_controller/gpu_governor.py`
Container image recipe; install and permission files	`deploy/Dockerfile`, `deploy/install/omni-compass.yaml`, `deploy/rbac-*.yaml`, `deploy/pilot/`
Proof tools: paired tests on your own stack, the GPU test, simulations	`scripts/`, `tools/`, `hardware/`
Every check the code must pass	`python3 verify.py` (ends with `VERIFICATION: PASS`)
The C++ engine: every law twinned in C++20, proven equal to the Python	`cpp/` (build: `cmake -S cpp -B cpp/build && cmake --build cpp/build`)
The seal: the fingerprints that lock each Python law to its C++ twin	`results/SEAL.json`, `python3 tools/seal.py --check`
Metrics: every gauge and where it comes from	`docs/METRICS_CATALOG.md`
How it compares with what you run today	`docs/COMPARISON.md`
The license and notices	`LICENSE`, `NOTICE`

License, in short (the `LICENSE` file governs): you may download, run, modify and test Omni-Compass free of charge only to evaluate it and to reproduce its published results, including in shadow or test mode on systems you own or control (`LICENSE`, section 1). Every other use, commercial or not (running it for your

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

business or in production, selling it, hosting it, or building it into a product or service), needs a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. The copyright, patent and trademark notices, the `LICENSE`, the `NOTICE` and `DISCLOSURES.md` stay with every copy.

47.2 The rules Omni-Compass keeps on your system

1. **One OFF switch for the whole harness, in a human hand.** `python3 tools/omni_switch.py off` turns every Omni-Compass governor on the machine off at once: each returns every setting it changed to the value it recorded before it acted, reads each back and exits, and no governor starts again until `python3 tools/omni_switch.py on` (`omnicompass/master.py`; the switch file is `OMNI_MASTER_OFF`, by default `/tmp/omni-compass/OFF`; set it, and `OMNI_REGISTRY`, to a directory only the governors' user can write, `docs/THREAT_MODEL.md`). Each governor also has its own switch for one muscle at a time: creating its kill file (or setting `OMNI_KILL=1`).
   - Kubernetes controller: `--kill-file` (default `/tmp/omni.kill`).
   - GPU governor: `--kill-file` (default `/tmp/omni-gpu-kill`), or send it SIGTERM.
2. **It records before it acts.** Every original setting is written down first (annotations on the Kubernetes objects, the `snapshot` line in the GPU audit), so the OFF switch always knows what to restore.
3. **It watches before it writes.** Every level starts in watch mode: it decides and logs, and writes nothing.
4. **It never goes blind and keeps acting.** If a reading fails (nvidia-smi, metrics, the response-time feed), it returns what it changed to the recorded setting at once and gives nothing back until it can see again.
5. **Service first.** While response time is over your target, and for a few decisions after (`--slo-clear`), it may not cap power or pack tighter than your own settings.
6. **Every change is read back.** No new order goes on top of one that has not landed.
7. **The living band.** No organ is driven below 5% or above 95% of its range; machines are marked idle, never switched off by Omni-Compass; it never evicts or moves a pod.
8. **Everything is logged.** Every read, decision, write and its reason goes to the audit log (`--audit`).

47.3 Before you start: what your system needs

You run	You need
Kubernetes (any: vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind)	Kubernetes 1.34 or newer (in-place pod resize); metrics-server (`kubectl top nodes` works); an HPA with a CPU target on each service to be governed; a readiness probe and a short preStop pause on each service

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

You run	You need
A response-time feed (strongly recommended at every level)	a CSV per service, `elapsed_seconds,latency_ms,ok`, written continuously; `scripts/latency_probe.py` writes one against any HTTP endpoint
NVIDIA GPUs	driver with `nvidia-smi`; root (or the capability to run `nvidia-smi -pl`); on a VM, full GPU passthrough (container or "pod" GPU rentals usually block power-limit changes)
CPU power control	Linux cpufreq with the schedutil governor (`/sys/devices/system/cpu/cpufreq`); RAPL for package watts (`/sys/class/powercap/intel-rapl`); usually bare metal only, since cloud VMs rarely expose these
Site power and cooling	a command that prints site watts, and (for cooling) a command that sets the supply-air setpoint through your building management system

Build the image once:

```
docker build -f deploy/Dockerfile -t <your-registry>/omni-compass:<tag
    ↪ > .
docker push <your-registry>/omni-compass:<tag>
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

47.4 The levels, from watching to running the stack

47.4.1 Level 0. Evaluate without touching anything

```
pip install -r requirements.txt
python3 verify.py                       # every check; ends with VERIF
    ↪ ICATION: PASS
python3 tools/gpu_physics_sim.py        # the GPU governor against a m
    ↪ odelled NVIDIA card
python3 hardware/node_exchange.py       # CPU and GPU on one power bud
    ↪ get, modelled
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Pass condition: `VERIFICATION: PASS`. Nothing in your systems is touched.

47.4.2 Level 1. Watch (read-only)

```
kubectl apply -f deploy/install/omni-compass.yaml     # set the image
    ↪ line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

- The install file grants a read-only identity. `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts that show it cannot write.
- Every decision it would take is logged with its reason; nothing is written.

Pass condition: zero writes, and readings your operators agree with.

47.4.3 Level 2. The pods, on top of your autoscalers

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`).
2. Run with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.
3. Optional pod muscles, one at a time:

Muscle	Switch	What it does
convey	on with `--latency-file` and `--cap-deployments ns/name`	gives each machine's idle CPU to the serving pods on it (default: always); `--convey-on 0.5 --convey-off 0.25` engages it only while response time is over half the target
rightsize	`--rightsize-deployments ns/name`	each pod's CPU request follows its measured use × (1 + headroom), in place
coldstart	`--coldstart-deployments ns/name --coldstart-signal ns/configmap`	scales a service to zero while no work waits, wakes it the moment work arrives
batch	`--batch`	admits held Jobs labelled `omnicompass.io/batch=true` when there is load and power headroom
batch pace	`--batch-pace`	pauses Jobs labelled `omnicompass.io/pausable=true` under power or heat stress, resumes them after
rollout guard	`--rollout-guard ns/name`	pauses a rollout while change is not permitted, undoes one past its deadline when rollback is authorised

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Muscle	Switch	What it does
contain	`--contain-namespaces ns --contain-cpu-m <m>`	holds an agent namespace to a CPU budget with a quota
security hold	`--security-configmap ns/name`	key `hold: "true"` blocks every expansion

Your HPAs keep scaling as before. Omni-Compass sets their targets and raises floors ahead of bursts.

Pass condition: p95, p99 and failed requests never behind than your own, over paired runs (section 47.6).

47.4.4 Level 3. The machines

1. Grant `patch nodes` and `patch pods` (`deploy/kind/rbac-omni.yaml`).
2. Run with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<command with {n}>"`.

Your platform	The park/wake command
any cluster, bare metal, kind	`bash scripts/kind_nodepool.sh {n}`: close machines to new work and mark them first to go; open again, warm machines first
Karpenter / EKS Auto Mode	the NodePool CPU limit at `{n} × node CPU`, parked nodes kept, not consolidated away
Cluster Autoscaler node group	the group's desired size, scale-down through parking, not deletion
OpenShift / OKD	the worker MachineSet replicas, parked, not deleted

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band.

Pass condition: fewer machines in service and less energy, with no LOSS on any service gauge.

47.4.5 Level 4. Omni-Compass decides; Kubernetes is the muscle

Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; your HPA stays as the fast reflex upward; the node law sizes the machines.

Pass condition: as level 3.

47.4.6 Level 5. A GPU box (with or without Kubernetes)

The GPU governor runs on any Linux machine with NVIDIA GPUs:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# watch: decides and logs, writes nothing
sudo python3 -m omni_controller.gpu_governor --mode watch --gpus 0,1,2
    ↪ ,3 --audit /var/log/omni/gpu.jsonl \
     --latency-file <feed> --slo-ms <p95 target>
# cap: writes the power limits
sudo python3 -m omni_controller.gpu_governor --mode cap   --gpus 0,1,2
    ↪ ,3 --audit /var/log/omni/gpu.jsonl \
     --latency-file <feed> --slo-ms <p95 target>
# OFF
sudo touch /tmp/omni-gpu-kill
```

What it does, every `--interval` seconds (default 2):

- reads each GPU's own meter: power draw, temperature, utilization, power limit;
- the engine sets a power cap; the shield keeps it above `--min-share` × the start limit (0.70) and above draw × (1 + `--headroom`);
- a busy card (smoothed utilization at or over `--util-gate`, 0.5) gets its full limit back at once;
- a response-time breach or a failed reading returns the start limit at once.

Speed lock (optional). Keep every response-time gauge at least `--speed-gain` (1%) faster than without Omni-Compass, and spend any speed won elsewhere on watts:

```
python3 tools/gpu_baseline.py baseline.json native-run/latency.csv
    ↪  # a run without Omni-Compass, other days
sudo python3 -m omni_controller.gpu_governor --mode cap --baseline-fil
    ↪ e baseline.json --latency-file <feed> ...
```

Prove it on your card: `sudo bash scripts/gpu_paired.sh` runs native, watch and Omni-Compass back to back on the same machine and prints the table and verdict (`docs/GPU_RUN_GUIDE.md`).

Pass condition: work per energy up, and requests served and p95 inside the guardrails.

47.4.7 Level 6. CPU clock and power

Add to the Kubernetes controller, on bare metal:

```
--cpufreq-policy-root /sys/devices/system/cpu/cpufreq --cpufreq-requir
    ↪ e-schedutil \
--rapl-cmd "<prints CPU package watts>"
```

The CPU frequency ceiling follows the engine's cap inside the nervous system's envelope; the OFF switch writes every policy's recorded maximum back exactly. GPUs can be wired the same way from the controller: `--gpu-query-cmd "nvidia-smi --query-gpu=power.draw,temperature.gpu`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`--format=csv,noheader,nounits" --gpu-power-cmd "nvidia-smi -pl {w}" --gpu-max-w <max>`.

Pass condition: energy per unit of work down, no LOSS on any service gauge.

47.4.8 Level 7. Site power, cooling, batteries

Organ	Switch	Status
site power stress	`--power-cmd "<prints site watts>" --site-limit-w <limit>`	wired: power stress enters the engine; batch pace and caps respond
cooling setpoint	`--cooling-cmd "<sets {c}>" --cooling-min-c 18 --cooling-max-c 27 --cooling-restore-c 22`	wired: warmer supply air while cool, colder as heat rises
CPU + GPU on one power budget	`hardware/node_exchange.py` (the conveyance law over CPU and GPU organs)	**simulation only.** The live levers exist (levels 5 and 6); the exchange between them has not run on hardware
GPU groups sharing a site budget	`hardware/site_exchange.py`	**simulation only**
on-site batteries as an organ	designed (`docs/DOMAIN_MAP.md`)	**not built**

47.5 Stack by stack

Stack	Levels available	Notes
Vanilla Kubernetes, kind, Rancher	1-6	as written
Amazon EKS	1-5	nodes via Cluster Autoscaler node group or Karpenter NodePool (level 3 table); CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances
Google GKE, Azure AKS	1-5	as EKS, with the provider's node-pool size as the park/wake command
Red Hat OpenShift / OKD	1-5	OpenShift is Kubernetes underneath; grant the same permissions through a Role; machines via the worker MachineSet

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Stack	Levels available	Notes
NVIDIA GPU servers without Kubernetes	5	the GPU governor alone
Bare-metal CPU servers	6	cpufreq and RAPL through sysfs
Slurm / HPC schedulers	5 on the GPU nodes	a Slurm connector for job-level decisions is not built
Building management (cooling, power meters)	7	through your BMS's command line or API, wrapped in the command templates
Azure AKS, the bill as the gauge	1-3	Azure's managed cluster autoscaler stays native and deletes the machines Omni-Compass idles; one or several work pools (`worker_pools`), the replica ceiling raised with the fleet (`hpa_max`); a fleet of 4 cannot show a saving of one machine, 39 can (manual, section 46.1; `docs/AZURE_SETUP.md`)
PostgreSQL behind PgBouncer, or any pooler with a console	one knob	the pool size through the pooler's own admin console, cover [2, 90], one writer, restored on OFF (manual, section 46.2; `docs/POSTGRES_PREREGISTRATION.md`)
Apache Kafka, or any consumer group an operator sizes	one knob	the consumer count inside [1, partitions], the group's own end-to-end latency as the reading, the rebalance paid on every move, handed back on OFF (manual, section 46.5; `docs/KAFKA_PREREGISTRATION.md`)
Redis, or any cache with a console and a memory ceiling	one knob	`maxmemory` through the cache's own console, cover [16, 512] MB, grown only while the cache is full (a cold miss is not the ceiling's), one writer, restored on OFF (manual, section 46.6; `docs/REDIS_PREREGISTRATION.md`)
Drone swarms (gym-pybullet-drones; PX4 and ArduPilot next)	one knob a drone	the cruise override inside the autopilot's limits, a separation wall, the paired physics trial, collisions void the cell (manual, section 46.7; `docs/SWARM_PREREGISTRATION.md`)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Stack	Levels available	Notes
MongoDB, or any store whose engine exposes its cache size at run time	one knob	the storage-engine cache through the server's own console, cover [256, 2,048] MB, grown only while the cache is full, one writer, restored on OFF (manual, section 46.8; `docs/YCSB_PREREGISTRATION.md`)
MySQL, or any store whose engine resizes its buffer pool online	one knob	the InnoDB buffer pool through the server's own console (`SET GLOBAL innodb_buffer_pool_size`), cover [128, 2,048] MB in the server's own 128 MB chunks, grown only while the pool is full, the plug waiting for the server's asynchronous resize before reading back, one writer, restored on OFF (manual, section 46.9; `docs/MYSQL_PREREGISTRATION.md`)
A rented cloud machine running the big organisms	the whole harness	`big-organism-detached`: start, collect, survey; the clock rule sets the window (`.github/workflows/big-organism-detached.yml`)
Independent simulators (CityLearn, pandapower, MuJoCo)	one knob each	the simulator's own controller is native; preregistered, A/B/C (manual, section 46.3)
The machine itself: Linux's frequency governor under the processor's meter	one knob	the frequency ceiling of every CPU through the kernel's own files, cover [half the top clock, the top], the governor still running under it; every notch down tried on the machine first by the brain's verdict; energy from RAPL and a wall plug where fitted; root on a machine on the metal, never a virtual one (manual, section 46.10; `docs/CPU_POWER_PREREGISTRATION.md`; `sudo bash scripts/cpu_power_run.sh`)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

47.6 Proving it on your own system

1. **Paired runs.** Run your service the same way twice, once as you run it today and once with Omni-Compass on top, back to back on the same machines, order rotated, at least 5 times (10 for a result you publish). `scripts/kind_paired.sh` and `tools/live_reps.py` do this and print each gauge with its 95% interval. A change is proven only when its interval excludes zero.
2. **The switch drill,** after every run: turn Omni-Compass OFF, check every setting is back at its recorded original and no `omnicompass.io/*` annotation remains, turn it ON.
3. **Read the results** with `docs/METRICS_CATALOG.md`: every gauge says whether it is measured or modelled.

47.7 Reading the log

Line	Meaning
`gate: a sense is blind`	a reading failed; nothing is given back until it returns
`gate: pods scaling up`	pods first, machines after
`decision failed (n in a row)`	the cluster could not be reached; nothing was written. Turn it OFF for native at once
`convey: <machine> idle CPU to its k serving pod(s), limit c`	that machine's idle CPU now reaches the work on it
`convey: response time calm, operator's limit`	the pods are back at your own CPU limit
`busy gate: utilization u, the limit read at start`	a busy GPU got its full power limit back
`speed lock: worst ratio r vs line 0.99 (spend / hold / release)`	the speed lock's reading and what it did
`response-time reflex: the limit read at start`	response time went over target; full power at once
`reset: the limit read at start`	the OFF switch restored this setting

47.8 What is proven, and how

Claim	Evidence	Kind
On real Kubernetes, Omni v1, three runs of ten pairs each: more	`results/live/V1_ALL_FOUR.md`, `V1_STEADY.md`,	measured on kind; read by the three-run rule

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Claim	Evidence	Kind
work inside the response line (+42% to +48%), p95 −57% to −69%, machines fewer where a paired trial allowed it (steady −1.5% to −3.4%, batch −15% to −24%)	`V1_WANDERING.md`, `V1_FAULTS.md`, `V1_BATCH.md`, `V1_FAIRNESS.md`	(`docs/OMNI_V1.md`)
On a real database, Omni v3: 61% to 72% fewer connections held open for the same work and latency, at +14% to +28% host CPU-seconds (power given in all three of our runs, reported)	`results/live/V3_PGBENCH.md`	measured on GitHub's machines
On Azure's bill, 4 workers: no difference beyond the noise on any gauge (the fleet is too small for the lever)	`results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md`	a real bill
Energy on Kubernetes	declared model on kind, no meter	modelled
GPU: more work per energy within the speed guardrail	earlier card controller, obsolete; rerun on rented cards pending	hardware meter
CPU and GPU on one power budget: more work, never over the budget	`results/hardware/NODE_EXCHANGE_*.json`	modelled
Safety: OFF switch restores everything; watch mode writes nothing	every live run's switch drill; `verify.py`	measured / checked

47.9 Python, C++ and the seal

Omni-Compass exists in two languages. The laws are twinned: each has a Python version and a C++20 version that give the same answers, proven by a parity test on every build.

Law	Python	C++	Proven by
core engine (six-state equations)	`omnicompass/core.py`	`cpp/src/core.cpp`	500 frozen fixtures

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Law	Python	C++	Proven by
governor (allocation laws)	`omnicompass/adapter.py`	`cpp/src/governor.cpp`	`tests/test_cpp_governor_parity.py`
safety shield	`omnicompass/shield.py`	`cpp/src/shield.cpp`	`tests/test_cpp_shield_parity.py` (plus adversarial cases)
HPA replica law	`fleet/harness.py`	`cpp/src/hpa.cpp`	`tests/test_cpp_hpa_parity.py`
closure law (machines)	`omnicompass/closure.py`	`cpp/src/closure.cpp`	`tests/test_cpp_closure_parity.py`
conveyance law (the conserved budget)	`omnicompass/conveyance.py`	`cpp/src/conveyance.cpp`	`tests/test_cpp_conveyance_parity.py` (identical to the last bit)
nervous system (authority per organ, living band)	`omnicompass/nervous_system.py`	`cpp/src/nervous_system.cpp`	`tests/test_cpp_twins_parity.py`
compass and ledger (composite storage)	`omnicompass/compass.py`, `omnicompass/storage.py`	`cpp/src/compass.cpp`	`tests/test_cpp_twins_parity.py`
GPU governor rules (shield limit, busy gate, speed lock, window, baseline)	`omni_controller/gpu_governor.py`	`cpp/src/gpu_rules.cpp`	`tests/test_cpp_twins_parity.py`

The seal (`results/SEAL.json`) holds the SHA-256 fingerprint of every file of every twin, written only after all parity tests pass (`python3 tools/seal.py`). `verify.py` fails if any sealed file changes afterwards, and names it. So the Python and the C++ cannot drift apart unnoticed. What stays in Python is the plumbing that talks to Kubernetes, nvidia-smi and sensors (`omni_controller/controller.py`, `muscles.py`, the device I/O of `gpu_governor.py`) and the simulation harnesses; every decision they take goes through the twinned laws. The seal lists them as Python only.

Keep this manual with the code. When Omni-Compass changes, this manual, the C++ twin and the seal change in the same commit.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 48

Running the GPU Benchmark

48.1 Running the GPU test yourself, step by step

> **Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 42.5 of the manual (*Wired right or wired wrong*).

There are three ways to get real-machine numbers:

- **Your own tower:** a machine you control, with a smart plug measuring the whole machine at the wall (section A).
- **GitHub's GPU machines:** they run the test from the repository with one click (section B).
- **A rented cloud GPU:** sections 48.1.4 to 48.1.9.

48.1.1 Your own tower, measured at the wall

You need:

- an NVIDIA graphics card (a GeForce RTX works);
- Linux on that machine (a spare drive or a USB boot is fine);
- a smart plug that reports watts, for example a Shelly Plus Plug or any plug running Tasmota, which cost about $20 to $30.

Plug the tower into the smart plug, and connect the plug to your home Wi-Fi with its own app. Note its IP address, shown in the app or on your router's device list. Then on the tower:

```
python3 tools/wall_meter.py shelly2:192.168.1.50 --once        # print
    ↪ s the tower's watts right now
sudo WALL_METER=shelly2:192.168.1.50 bash scripts/gpu_paired.sh
    ↪            # trial run
sudo WALL_METER=shelly2:192.168.1.50 PHASE=confirm bash scripts/gpu_pa
    ↪ ired.sh    # the real test
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Use `shelly1:` for older Shelly plugs and `tasmota:` for Tasmota plugs. The table then adds whole-machine energy at the wall and requests served per wall kilojoule. The plug is read by the test only; Omni never sees it, so its number is independent of Omni.

48.1.2 GitHub's own GPU machines

1. **One-time setup, by an owner of the Omni-Compass organisation on GitHub:**
   1. Go to **Settings → Actions → Runners → New runner → New GitHub-hosted runner**.
   2. Choose the image **NVIDIA GPU-Optimized Image for Linux** and a GPU size.
   3. Name its label `gpu-t4`, or set the repository variable `GPU_RUNNER` to the label you chose.

GPU runners are billed per minute on paid plans.

2. **To run:** go to **Actions → gpu-bench → Run workflow**, then choose `smoke` or `confirm`.
3. **Results:** each repetition runs on its own GPU machine. The results are pooled into one table, shown on the run's summary page, and every raw file is committed back to the branch under `results/gpu/github-<run id>/`.

If GitHub's machines don't allow changing the GPU power limit, the job stops in its first minute and says so.

48.1.3 A rented cloud GPU

This costs roughly $10 to $25 in rented GPU time. You type a handful of commands; the machine does the rest. The same test on Lambda from one GitHub button (it rents the machine, checks the keys first at no cost, brings the results back and always gives the machine back) is `docs/LAMBDA_SETUP.md`.

48.1.4 Rent the right kind of machine

The test changes the GPU's power limit, so you need a machine where you are the full administrator of the GPU:

- **Use a virtual machine (VM) or a bare-metal server.** Lambda Cloud "on-demand instances" are VMs; so are the GPU instances on AWS, Google Cloud and Azure.
- **Avoid "container" or "pod" rentals.** Their GPUs usually block power-limit changes. This is common on the cheapest per-hour marketplaces.
- **Any single NVIDIA data-center GPU works:** A10, L4, A100, H100, L40S. One GPU is enough.
- **Choose an image that already has PyTorch.** On Lambda that is the default "Lambda Stack" image.

You don't have to guess whether a machine allows it. In its first minute the test checks, and stops with *"cannot set the power limit (run as root)"* if the machine doesn't allow it. If you see that, shut the machine down (you pay only for the minutes used) and rent a different kind.

48.1.5 Connect to it

The rental site shows a command like `ssh ubuntu@123.45.67.89`. Paste it into Terminal (Mac) or PowerShell (Windows).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

48.1.6 Get the code

On the rented machine (the repository is public while the benchmarks run; if it is private, use a read-only fine-grained token in the URL):

```
git clone https://github.com/The-Omni-Compass-LLC/The-Omni-Compass-LLC
    ↪  the-omni-compass
cd the-omni-compass
git log --oneline -1       # the commit you are about to test; write i
    ↪ t down
pip install -r requirements.txt

nvidia-smi                 # should show your GPU
```

48.1.7 The whole test, one command (about 36 hours)

```
sudo nohup bash scripts/gpu_rented_run.sh > run.log 2>&1 &
tail -f run.log            # Ctrl+C stops watching; the test keeps run
    ↪ ning
```

It runs, in order, and stops at the first failure:

1. the machine check (one copy only, nothing else on the card, the card's default limit and clock range restored);
2. the **wire check**, which must end `WIRED RIGHT` (manual, section 42.4);
3. the smoke test, about 40 minutes, never counted;
4. the preregistered counted run on compute-bound work: 10 repetitions × native / watch / Omni, 600 s each, about 6 hours;
5. the second preregistered counted run on AI token generation (memory-bound), the same design, about 6 hours (`SKIP_DECODE=1` skips it); each is its own result, never pooled. Both are packed into one file as soon as they finish: `results/gpu/omni-gpu-<stamp>-confirmations.tar.gz`;
6. **an operator's power cap underneath** (70% of the card's default limit): the cap alone vs the cap with Omni-Compass on top, at the usual load and fully loaded (more work from the same watts), about 3 hours (`SKIP_CAP=1` skips it);
7. **the GPU fault drill**, about 10 minutes: the governor killed outright, the master switch pulled, the response feed blind; every check must pass (`SKIP_DRILL=1` skips it). Everything so far is then packed into `results/gpu/omni-gpu-<stamp>-partial.tar.gz`;
8. the seven organisms with this card inside, each as **1, 10 and 100 copies** on one clock (5, 3 and 2 repetitions; no 1,000 copies, the founder's order of 12 October, amendment 13), at least 16 hours (60 paired repetitions, each arm 240 steps of 2 s); a size whose step needs longer than 2 s gets a longer step, measured on the machine and stated in the receipt (`SKIP_HIL=1` skips this stage);

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

9. **real AI serving** last: a small open language model served by vLLM, about 2 hours (`SKIP_LLM=1` skips it); if vLLM cannot install on the machine, the stage says so and nothing before it changes;
10. one packed file: `== send this one file back: results/gpu/omni-gpu-<stamp>.tar.gz`, with the label each table chose by rule.

To stop everything at any moment: `sudo python3 tools/omni_switch.py off` turns every Omni-Compass governor off and hands the card back to its own settings.

Do not start it twice and do not use the card for anything else while it runs.

48.1.8 Bring the results back

Download `results/gpu/omni-gpu-<stamp>.tar.gz` (in JupyterLab: right-click, Download; or `scp` from your own computer). It holds every raw reading, every table, the verdict and the checksums. **Then shut the rented machine down** on the rental site, so the billing stops.

48.1.9 Read it before you believe it

Open `results/gpu/run-<stamp>/GPU_REPS.md` and check the rows of the manual's section 42.5 (*Wired right or wired wrong*): watch equal to native, requests equal, the card's busy clock under Omni at or above its own, the lid while busy at or above its own busy draw, fail-up rare in the credit-per-write table. If a row reads wired wrong, the run says nothing about Omni-Compass until the wiring is fixed (`DISCLOSURES.md`, section 3).

48.1.10 The 8-GPU result: every card of one server at once

The same test as section C, on a machine with several cards (Lambda "8x A100" or "8x H100", a VM, the Lambda Stack image). Every card runs its own paired test at the same moment, sharing the server's power supply, cooling and neighbours' heat, as in a real data center. Each card starts its arm rotation one step later than the card before it, so at any moment some cards run native and some run Omni-Compass.

Get the code as in step 3, then:

```
sudo nohup bash scripts/gpu_8card.sh > run8.log 2>&1 &
tail -f run8.log
```

Each card: wire check, envelope, smoke, the compute counted run, the AI token generation counted run, the operator's power cap underneath (about 15 to 16 hours, all cards together). Then the fault drill once on card 0, with every other card idle. Then one pooled table per workload (every repetition of every card, each paired within its own card: 80 repetitions per workload on 8 cards) and each card's own table. `SKIP_DECODE=1 SKIP_CAP=1` runs the compute counted run alone, about 6.5 hours. **Every 8-GPU run has a hard time budget** (`MAX_HOURS`; by default 10.5 hours with `POOLED=1`, 4 with `FAST=1`, 26 for the whole design): at the budget the master switch hands every card back, the stages still running stop, and what finished is packed and printed as the file to send back. Lambda bills until the instance is terminated, so terminate it when that line appears. `sudo POOLED=1 nohup bash scripts/gpu_8card.sh &` **runs every stage with the**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

repetitions pooled across the cards (3 per card per counted run, 24 per test on 8 cards; 2 per power cap load; 1 per organism size), about 9 to 10 hours. `sudo FAST=1 nohup bash scripts/gpu_8card.sh > run8.log 2>&1 &` is a short look, about 3.5 hours: 3 repetitions per card (24 on 8 cards), then one model across every card; it leaves AI token generation, the power cap and the whole stacks out, so it is never the result to show. Last, the whole server as one: a language model (Qwen2.5-7B-Instruct, open) served across every card at once by vLLM, one Omni-Compass governor per card, the server's total GPU energy, about 2 hours (`SKIP_LLM=1` skips it). Before it, **the whole stacks with a real card inside**: the seven organisms at 1, 10 and 100 copies (no 1,000 copies, amendment 13), native and Omni, the full repetitions (5, 3, 2 by size), each organism on its own card at the same time, so the stage that takes at least 16 hours on one card takes the time of its longest organism, about 3 hours (`SKIP_HIL=1` skips it). The whole design, every stage: about 20 to 22 hours. Progress: `tail -f results/gpu/8card-<stamp>/card-*.log`. At the end: `== send this one file back: results/gpu/omni-8card-<stamp>.tar.gz`. The master switch (`sudo python3 tools/omni_switch.py off`) stops every card's governor at once.

48.1.11 Your own serving engine

own vLLM? Start it as you always do, then:

```
sudo LLM_URL=http://127.0.0.1:8000 LLM_MODEL=<your model name> GPU=0 E
    ↪ NVELOPE=<envelope.json> OUT=results/gpu/mine \
  bash scripts/gpu_vllm.sh
```

The same paired test (your engine alone against your engine with Omni-Compass on top) runs against it; nothing is installed or started. `GPU=0,1,2,3,4,5,6,7` when your engine spans several cards. Make the envelope once with `python3 tools/declare_envelope.py envelope.json --gpu 0`.

48.1.12 Whole-server power from the server itself

The bench records the whole machine's watts beside the cards' own when you give it a meter: `WALL_METER=redfish:<BMC address>` (with `REDFISH_USER` and `REDFISH_PASSWORD`, read only), `WALL_METER=ipmi:local` (the server's management controller, read on the machine), or a smart plug (`tools/wall_meter.py` lists them). Omni-Compass never reads it.

48.1.13 Before any machine: the simulated card

`python3 tools/gpu_physics_sim.py --out results/gpu/sim/after` runs Omni's own GPU governor, with its current settings, against a modelled card for 10 paired repetitions in about two minutes. No GPU is needed. Every number it prints comes from the card model, not from a meter, so it is a preview of what the governor does and never the result. `results/gpu/sim/before` holds the same run with the settings before the share floor and busy gate were added.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

48.1.14 What you will get

A table from the GPU's own power meter:

- native against Omni watching only, and native against Omni governing;
- the preregistered primary result: work per energy, in requests served per kilojoule, with its 95% interval.

It ends with one verdict line, one of:

- **GAIN, proven**;
- **LOSS, proven**;
- **not proven**;
- **more requests per kJ, fails the service guardrail**.

Whatever it says is the answer, and it is the first number in this project that Omni-Compass's own code did not compute.

48.2 The GPU on Lambda: setting it up, click by click

This connects GitHub to a Lambda Cloud account so one button, the `lambda-gpu` workflow, can rent one GPU machine, run the GPU test on it at the exact commit the button is pressed on (`scripts/gpu_8card.sh` on eight cards, or `scripts/gpu_rented_run.sh` on one), bring every result file back as the run's artifact, and give the machine back. The GPU runs start from the branch `gpu-omni-v18` (the engine that stands; the run's page prints the exact commit). The design is `docs/GPU_PREREGISTRATION.md`; what each stage measures is `docs/GPU_RUN_GUIDE.md`, sections C and D.

About ten minutes, once. Lambda bills the card on file from the moment a machine starts until it is given back; nothing here starts a machine until a dry run has shown the price and the run is started again with `dry_run` set to `no`.

48.2.1 A Lambda account with a card

1. On the laptop, open **cloud.lambda.ai** and click **Sign up** (or **Log in**).
2. Add a card under **Settings**, **Billing**. Lambda rents no machine to an account without one.

48.2.2 The two keys, made in Lambda

The API key (it lets GitHub rent and give back a machine):

1. In Lambda's menu on the left, click **API keys**, then **Generate API key**. Name it `omni-github`.
2. Lambda shows the key once. Leave that page open for step 3.

The SSH key (it lets GitHub log in to the machine it rented):

1. In the menu on the left, click **SSH keys**, then **Add SSH key**, then **Generate a new key**.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2. Name it `omni-key`. Lambda downloads a file, **omni-key.pem**, to the laptop's **Downloads** folder. That file is the private half; Lambda keeps the public half under the name `omni-key`.

Never copy either key from a screen by selecting it: a long block is cut short when it is selected (on 18 October the Google key was stored cut short that way, and every Google set failed at the login). Use the copy button for the API key and the downloaded file for the SSH key.

48.2.3 Put the three secrets in GitHub

1. Open the repository on GitHub, click **Settings** (the repository's tab, at the top right), then on the left **Secrets and variables**, then **Actions**.
2. **New repository secret**. Name: `LAMBDA_API_KEY`. Secret: the API key from step 2. **Add secret**.
3. On the laptop, open **Downloads**, right-click **omni-key.pem**, **Open with**, **Notepad** (on a Mac: **TextEdit**). Press **Ctrl+A** (select all), then **Ctrl+C** (copy): the whole file, from `-----BEGIN` to `PRIVATE KEY-----`.
4. **New repository secret**. Name: `LAMBDA_SSH_PRIVATE_KEY`. Secret: click in the box, **Ctrl+V**. **Add secret**.
5. **New repository secret**. Name: `LAMBDA_SSH_KEY_NAME`. Secret: `omni-key` (exactly the name given in step 2). **Add secret**.

Keep **omni-key.pem** in a safe folder on the laptop (it also lets you log in to a rented machine yourself) and never send it to anyone. These three are passwords for this one purpose: never paste them anywhere but GitHub.

48.2.4 The dry run (free)

1. On GitHub, **Actions**, then **lambda-gpu** on the left, then **Run workflow** (on the right).
2. Branch: **gpu-omni-v18**. `test`: `1card`. `mode`: `fast`. Leave `dry_run` at **yes**. Click **Run workflow**.
3. In about a minute the run's page shows, at no cost:
   - the SSH key check: "SSH key omni-key: Lambda holds it, and the stored private key is its other half";
   - the price an hour, the regions with a machine free now, and the estimate for the hour budget plus one hour;
   - "Dry run: the secrets and the key pair are ready; nothing rented."

A red run says which of the three secrets to fix (a missing secret, a key name Lambda does not hold, a private key that is not the other half of `omni-key`), and nothing is rented. "No machine of this type is free in any region now" is Lambda's stock at that moment: try again later, or name another `region` or `instance_type`.

48.2.5 The real runs

The same button with `dry_run` set to **no**. The `max_cost` box (400 by default) is a ceiling in US dollars for the hour budget plus one hour: above it nothing is rented.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`test mode`	What runs	Hour budget (given back by)	At Lambda's list price
`1card fast`	the compute counted run on one A10	8 h (9 h)	one A10 $1.29 an hour on 24 October: about $12
`1card stacks`	the whole stacks with the card inside: the wire check and smoke, then the seven organisms at 1, 10 and 100 copies, native and Omni-Compass, the preregistered 5, 3 and 2 repetitions by size (`hil_profile standard`)	32 h (33 h)	about $43 a run at most; A, B and C (`tag`) each rent their own A10 and run side by side
`1card stacks-full`	the same stacks with the card fully loaded inside them (`docs/GPU_PREREGISTRATION.md`, amendment 16): more requests offered than the card serves, so the work done inside the same rented time can rise	32 h (33 h)	about $43 a run at most; A, B and C side by side
`8card pooled` (default)	every stage on eight A100 cards, repetitions pooled across the cards	10.5 h (11.5 h)	$22.32 an hour on 1 November (the price the launch printed; it was $14.32 earlier): about $257 at most
`8card fast`	a short look, never the result to show	4 h (5 h)	about $112 at most
`1card full`	the whole one-card test	32 h (33 h)	about $43
`8card full`	the whole design on eight cards	26 h (27 h)	about $603 at most (over the button's default $400 ceiling: raise `max_cost` to run it)

The price the dry run prints is Lambda's own at that moment; the table is an example. The one-card and the eight-card tests may run at the same time (one run of each at once; a second press of the same test waits for the first, unless its `tag`, A, B or C, differs: each tag is its own machine).

The machine is given back when the run prints its last line, or at the hour budget plus one hour whatever happens, or at the end of the workflow if anything before it failed. Each give-back is acknowledged by Lambda; if Lambda does not acknowledge it, the run turns red and says so. Then, by hand: **cloud.lambda.ai**, **Instances**, tick the machine (its name starts `omni-`), **Terminate**.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

A run cancelled by hand keeps watching its machine (the watch jobs run on through a cancel, so a give-back is never lost), and a run whose test stopped early is given back at once by the watch since 24 October. For a machine still left billing under an older run: **Actions**, **lambda-gpu**, **Run workflow**, **test** `giveback`, **give_back** `omni-1card-` (or `omni-8card-`, or the machine's whole name). It gives back every machine whose name starts so, rents nothing, and never touches a machine launched by hand on Lambda's page; the stuck run then sees its machine gone and ends, and the next run of that test starts.

48.2.6 Afterwards

The results are the run's artifacts (`lambda-gpu-watch*-<run id>`); the founder hears what they say before anything goes up. To remove GitHub's access: in Lambda, **API keys**, delete `omni-github`, and **SSH keys**, delete `omni-key`; then delete the three secrets in GitHub.

PART SIX

Operating It

The OFF switch, the rules the governor obeys, the log it keeps, and the care of a running installation.

CHAPTER 49

The OFF Switch, the Rules, and the Log

The rules Omni-Compass keeps on your system.

1. **The master switch, for the whole harness, in a human hand:** `python3 tools/omni_switch.py off` turns every Omni-Compass governor on the machine off at once (the Kubernetes controller and the GPU governors). Each puts every setting it ever wrote back to the value it read before its first write, reads it back and exits; the command waits until all have, and says so. While the switch is OFF, no governor will start. `python3 tools/omni_switch.py on` allows them to be started again (nothing restarts by itself); `status` shows the switch and every governor running. Use it the moment anything looks wrong, including a suspected breach. Each governor also keeps its own switch for one muscle at a time: the kill file (or `OMNI_KILL=1`, or SIGTERM to the GPU governor). **If a governor dies without handing back** (killed outright, the machine crashed, it hung), nothing it wrote is left in place: every governor records, before its first write, the exact commands that put every setting back, and renews a lease every decision. Run `python3 tools/omni_switch.py watchdog` as its own service next to the governors: it hands back for any governor whose process is gone or whose

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

lease has run out (a hung one is stopped first). `status` names any governor that died without handing back. The watchdog acts only on records its own user wrote that no other user can change (run it as the governors' user, or name theirs with `--trust-uid`), and never signals a process number the system has given to another process. Put the switch and the registry in a directory only the governors' user can write (`OMNI_MASTER_OFF`, `OMNI_REGISTRY`); `status` warns when they are not.

2. It records before it acts (annotations on Kubernetes objects; the `snapshot` line in the GPU audit).
3. It watches before it writes.
4. It never acts blind.
5. Service first: while response time is over its target, and for `--slo-clear-s` seconds after (180 by default), nothing is given back.
6. Every change is read back; no new order goes on top of one that has not landed.
7. The living band and the cover: no organ is driven outside its range; machines are parked, never switched off by Omni-Compass; it never evicts or moves a pod.
8. One writer: if anyone else changes a knob, Omni-Compass stops writing it and leaves it alone.
9. Everything is logged: every read, decision, write and reason goes to the audit log.

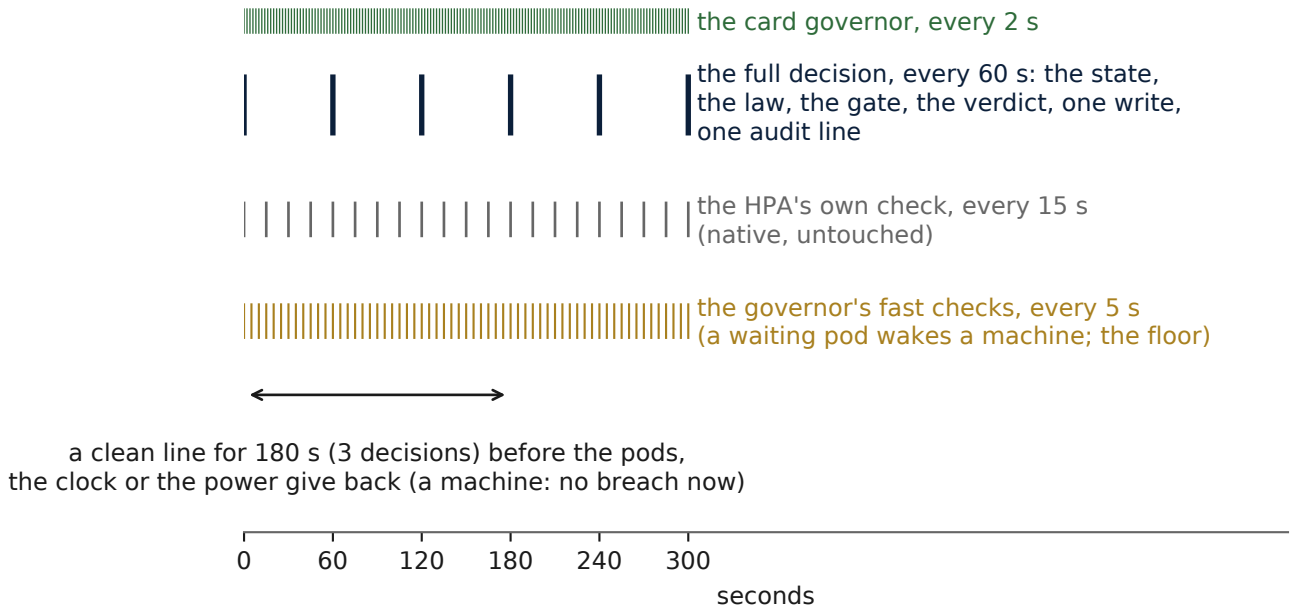


Figure 49. Timing diagram: the governor's clocks against the native controllers' own, over five minutes. The card governor decides every 2 s; the Kubernetes governor's fast checks run every 5 s (a waiting pod wakes a machine; the floor); native's own autoscaler keeps its 15-second check underneath, untouched; the full decision (the state, the law, the gate, the verdict, one write, one audit line) runs every 15 s, and nothing is given back until the line has been clean for 180 s, twelve decisions.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

On autopilot: why nobody babysits it. Omni-Compass is an autopilot: a closed-loop supervisory controller that runs, steers and corrects itself inside the system it governs, as a self-driving car's controller does inside the car. Once the paired trial on a muscle has passed and the governor is set to govern, nobody turns a knob and nobody has to react. Every change is the governor's own, and the governor audits itself: every write is read back from the system, every machine it gives back is kept only after its own paired trial shows the response time holds, every reading, decision and reason is in its audit, and it hands every setting back at the end; nobody has to check its work either. It holds the response time at the centre of its band, under the line, and acts as the reading approaches the line, so the correction lands before the line is crossed, not after a person is paged. What a native stack pays in people for the same service (an engineer watching dashboards, paged while the service is over its line, retuning an autoscaler's target or a node pool by hand) is the **babysitting tax**, and every Kubernetes and cloud receipt counts it in both arms: the **alert minutes** (whole minutes whose 95th percentile, failed requests counted over, was over the line: the minutes an on-call page would be open; judged, the aim lower), the **settings commands the governor wrote on its own**, and the **share of its additions decided while the service was still inside its line** (both shown, not judged). No person touches either arm of a paired run. The aim in the cloud is **no alert minute at all**: a minute over the line is a person woken, so an alert minute in the omni arm is a LOSS of the aim, its cause found and fixed and the test run again. Where native itself is short of machines (over its line on its own), the receipt says so beside the row: the same machines can be made to carry more, but no governor creates compute that the machines do not have.

The three ways a governor stops, and what each leaves behind. A referee or an underwriter should know exactly what happens to a governed system in each of the three ways Omni-Compass can stop, because the answer is the same in all three: every setting at native, read back, with no record left.

- **The reset** (every setting handed back to native) is the planned end of a run. The governor sees its reset file, hands every setting back in reverse order of power-up, reads each back, writes its last audit line and exits 0. Every benchmark arm ends with one and then checks the cluster from outside the governor: the HPA target at the operator's, the replica range as set, every pod's CPU limit at its shipped value, every worker back in service, no `omnicompass.io/` annotation left on any object. An arm whose reset check fails is invalid and is left out of the table, never counted.
- **The kill switch** (security) is the unplanned end of trust. `tools/omni_switch.py off` writes the switch file and sends every registered governor SIGTERM; each performs the same hand-back as a reset and exits; the command waits until all have and reports any that have not. No governor starts while the switch is off. It is one switch for the whole machine, not one per muscle, because the case it exists for, something driving a system through Omni-Compass's brain, is not a case for picking which muscle to stop.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

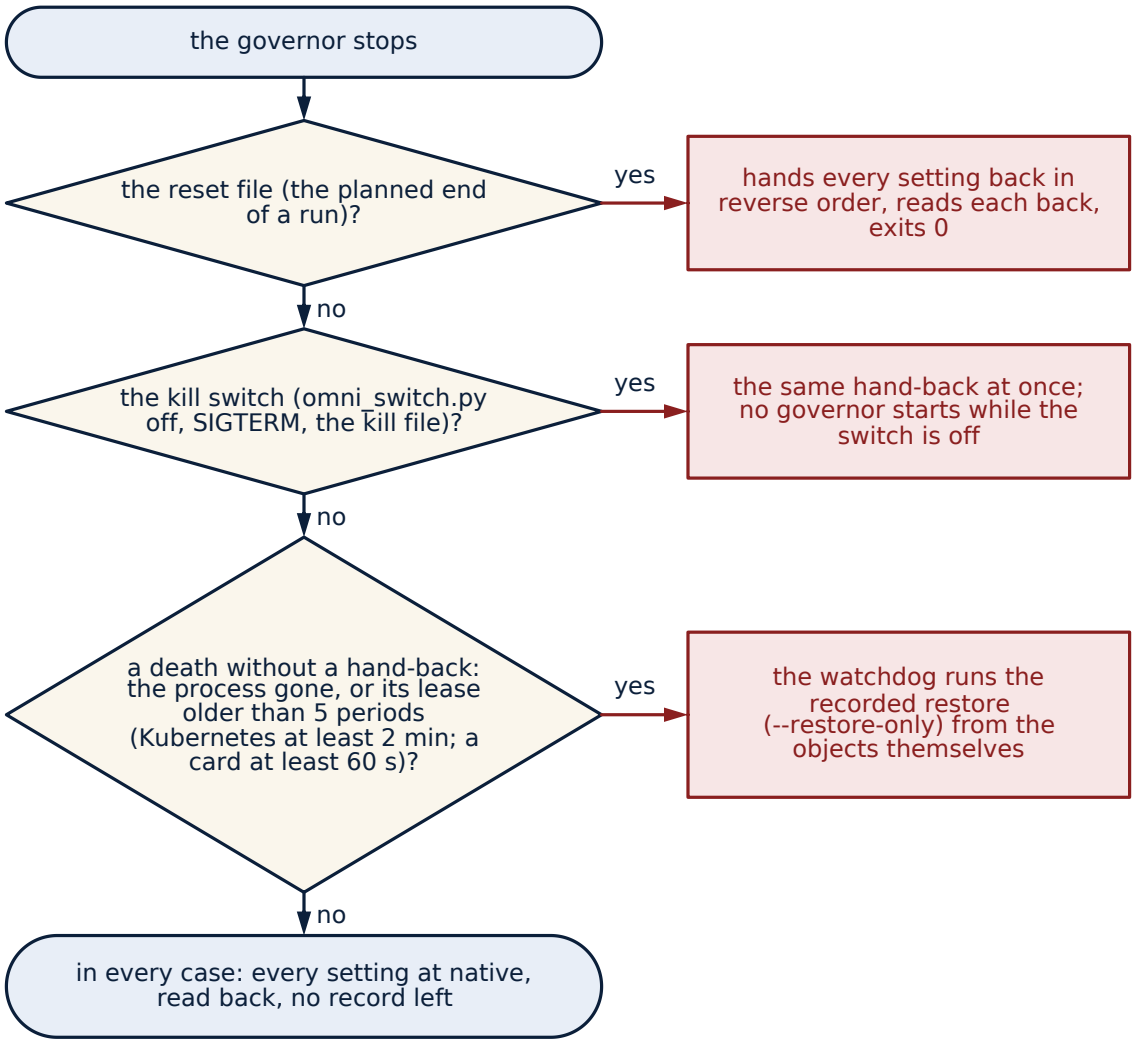


Figure 50. Flowchart: the three ways a governor stops. The reset file at the planned end of a run hands every setting back in reverse order, reads each back and exits 0; the kill switch (omni_switch.py off, SIGTERM or the kill file) does the same at once, and no governor starts while the switch is off; a death without a hand-back (the process gone, or its lease older than five periods) is caught by the watchdog, which runs the recorded restore from the objects themselves. In every case every setting ends at native, read back, with no record left.

- **A death without a hand-back** (killed outright, a crash, a hang) is the case the lease covers. Before its first write every governor registers itself with the exact command that puts every setting back (`omni_controller.controller --restore-only`, which restores from the snapshot annotations on the objects themselves, so it needs no memory of the dead process), and renews a lease every decision. The watchdog (`tools/omni_switch.py watchdog`), run as its own service beside the governors, finds any governor whose process is gone or whose lease is older than its limit (five decision periods, at least two minutes), stops a hung one, runs its recorded restore command, and records what it did. The robustness benchmark (Chapter 60, in preparation) kills the governor outright in the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

middle of a governed run and measures how many seconds pass before every setting is back and what the service saw in that gap.

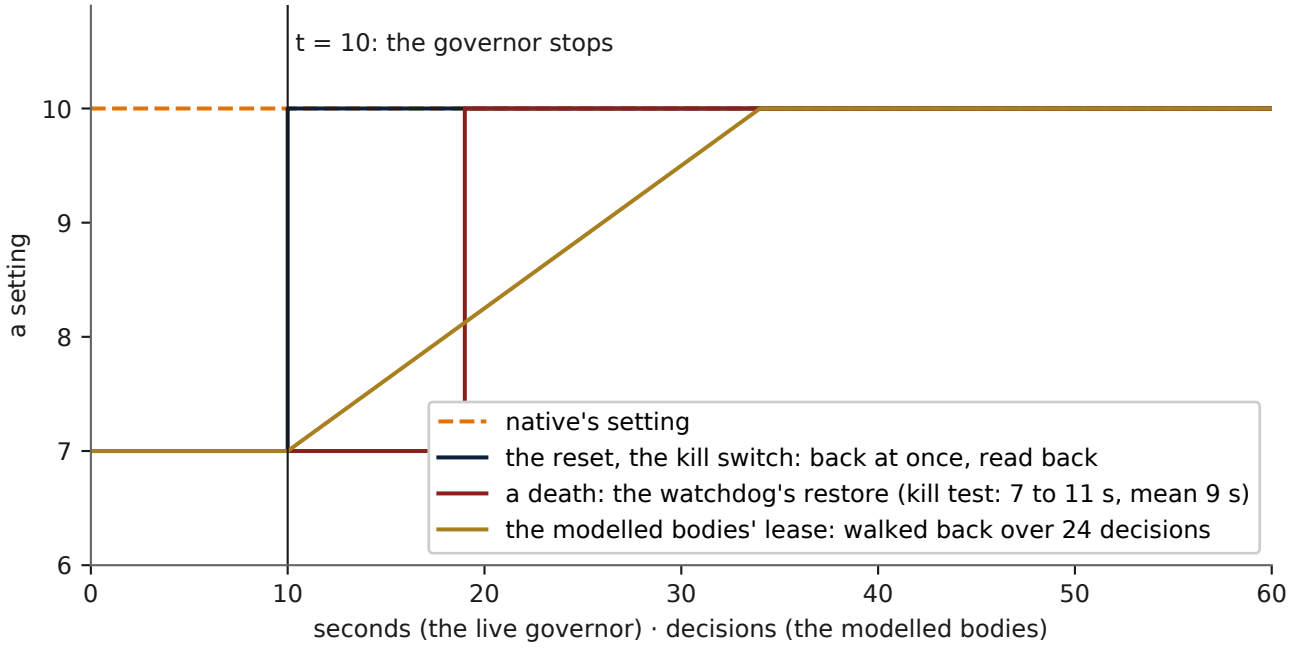


Figure 51. Timing diagram: the hand-back after a reset, a kill and a death. At t = 10 the governor stops. A reset or the kill switch writes every setting back at once and reads it back; a death without a hand-back is caught by the watchdog, which restored within 7 to 11 s (mean 9 s) in the kill test; in the modelled bodies the lease walks the setting back to native over 24 decisions.

Reading the log.

Line	Meaning
`gate: a sense is blind`	a reading failed; nothing is given back until it returns
`gate: pods scaling up`	pods first, machines after
`decision failed (n in a row)`	the cluster could not be reached; nothing was written; turn it OFF for native
`decided_by: compass`	the compass set the wires this decision
`decided_by: fail_up / blind_fail_up`	the service crossed the 0.95 wall, or a sense went blind: full capacity at once
`decided_by: thermal_hold`	the card reported a heat slowdown; nothing was tightened
`foreign_writer`	someone else changed a knob; Omni-Compass now observes only
`restored ... ok: true`	the OFF switch put every setting back and read it back

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Line	Meaning
`"watchdog": "handed back"` (in the watchdog's own log)	a governor died or hung without handing back; the watchdog ran its recorded restore command; `ok` says whether every command exited 0
`cruise: on` / `cruise: off`	work waited for a place for two decisions: machines in service up to the pool's own top; the line has been empty for two: cruise ends
`idle_floor`	the work is done and demand is at zero: the pedal lifted all the way, straight to the floor in one move
`verdict_state: left native`	no step of the slow knob passed the paired trial: the knob is never moved on this muscle
`shield_interventions`	the shield clipped a write to its bounds or its step limit; the count says how often
`overhead` (the last line of the audit)	the governor's own CPU over the window: its process and every command it ran

How to read an audit from end to end. The audit (`audit.jsonl`, one JSON record a line, every line timestamped) is the governor's complete account of itself, and a reader can follow a run through it without the manual: the first lines are the snapshot (every setting as found, written to the objects as annotations so a watchdog can restore from them); then, every decision, the readings (the response time and its age, the pods waiting, the machines seen, whether any sense is blind), the compass's position and force (`compass`), the authority the nervous system granted and the release gate's reason (`authority`, `node_gate`), the decision itself (the machines recommended against the machines seen, the HPA target set), every write with the command that made it and the value read back, and the decision's wall time (`decision_ms`); in cruise, the floor step's silence; at the end, the reset's writes and read-backs and the `overhead` record. The benchmark harness prints a decision trail from the audit at the end of every omni arm (`scripts/kind_bench.sh`), one line a decision, so a reviewer reading a run's log sees what the governor saw and did at each minute.

CHAPTER 50

Maintenance, Upgrades and Security

- Run `python3 verify.py` after every upgrade; it must end `VERIFICATION: PASS`.
- Upgrade in watch mode first; take the levels again from level 1.

What an upgrade is, and what it is not. The engine is frozen and fingerprinted (Chapter 59). An upgrade that changes any engine file, a rule, a gain, a guard, a preset or a muscle, is by definition the next engine

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

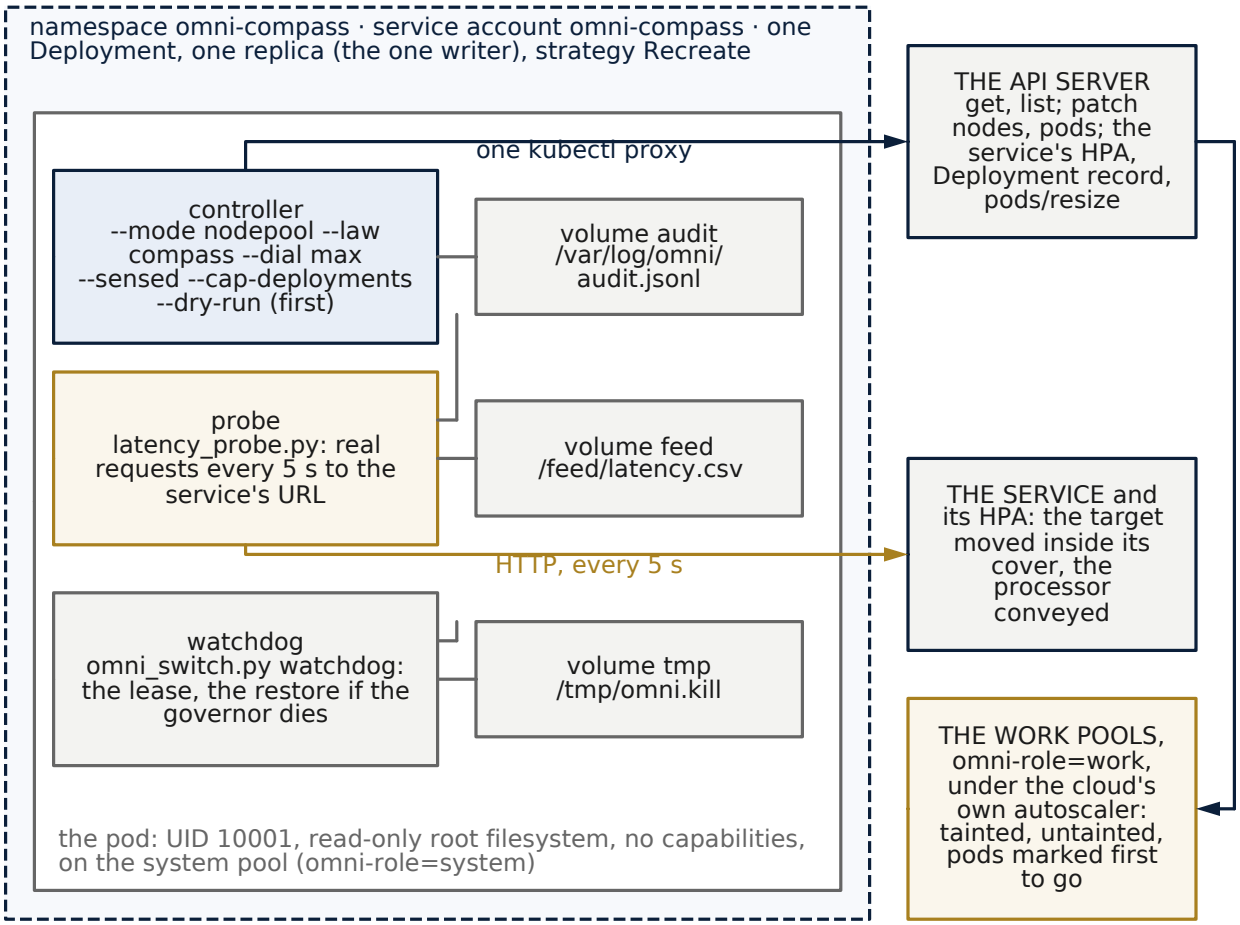


Figure 52. Deployment diagram: the governor on a managed cluster. One namespace, one service account, one Deployment of one replica (the one writer): the controller container with its command line, the probe beside it writing the feed to a shared volume, and the watchdog holding the lease; the audit, the feed and the kill file each on their own volume.

Outside the pod: the API server, reached through one kubectl proxy and granted get, list, patch nodes and patch pods, nothing else; the service and its autoscaler, untouched in the cloud mode; and the work pools under the cloud's own autoscaler, where machines are tainted, untainted and their pods marked first to go. No cloud credential is in the pod.

version, and every published result is run again on it before it is quoted beside the old ones; `tools/omni_version.py` says which version a checkout carries, and a checkout that matches none says which files differ. An upgrade that changes only the harness, the tools, the tables or this manual leaves the engine's fingerprint as it was and every result standing; the robustness harness of section 60.15 is an example, built without touching an engine file. An operator can therefore tell, from one command, whether a new checkout is the engine the results were made on or something that must prove itself again.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- The container runs non-root, read-only, with no capabilities; the Kubernetes identity has only the permissions of the level you run.
- The GPU governor needs root only for `nvidia-smi -pl` and `-lgc`.
- Report a security issue as `SECURITY.md` describes.

Least privilege, proven on every run. The Kubernetes governor runs as its own service account (`deploy/kind/rbac-omni.yaml`) with exactly the verbs the level needs: patch on the governed HPA, nodes, pods and the governed deployment, get on its security ConfigMap, create on pod evictions. It cannot delete nodes, create or delete pods or deployments, touch the load generator or anything in `kube-system`, read any secret, create a namespace, or write its own security hold. Every benchmark arm records `kubectl auth can-i` for each of these and fails if a "cannot" reads yes or a "can" reads no (`bench-*/rbac_omni.txt` in every archived run). The security hold itself is a ConfigMap another identity owns: when its key reads `hold: "true"` every expansion is blocked, and the governor cannot clear it.

What never leaves the repository. Cloud credentials live only in the repository's encrypted secrets and are used only by the workflows that need them; no credential, key or token appears in any file, log, table or commit, and the founder is never asked for one in conversation. The repository carries SPDX license identifiers on every source file so that license scanners (Black Duck, FOSSA, Snyk) read the evaluation license correctly, and every generated report carries the legal notice at its head and foot (`tools/legal.py`). Patent, copyright and trademark filings are referred to as filed; no filing number appears anywhere.

Supply chain. Everything the benchmarks download is pinned and checked: metrics-server by version and SHA-256 (`scripts/kind_bench.sh`), Apache Kafka against the Apache Software Foundation's own SHA-512 file for that release, Redis as the Ubuntu runner's own package, MongoDB from its publisher's own repository signed by the publisher's key and YCSB against the SHA-256 written in its preregistration, gym-pybullet-drones at one named commit, Python packages by `requirements.txt`. A download whose checksum does not match stops the run before anything else happens.

The upgrade drill, step by step. An operator who receives a new checkout does five things, in order, and stops at the first that fails. First, `python3 tools/omni_version.py`: if it prints the version the operator's results were made on, the engine is unchanged and the rest of the drill is about the harness; if it prints a list of differing files, the checkout is a new engine and nothing the operator measured before applies to it until measured again. Second, `python3 verify.py`, which must end with its pass line; the verifier runs every unit test, the shield's adversarial cases, the Python-against-C++ agreement and the release manifest, and a checkout that fails it is not installed anywhere. Third, the wire check on the operator's own stack, which proves that every wire the new checkout would use still follows, reads back and returns. Fourth, watch mode for at least one full cycle of the operator's load, reading the log the governor would have written and comparing its would-be decisions against what the operator would have done. Fifth, the levels again from level one, each with its paired receipt. An upgrade that skips a step has not been upgraded; it has been installed.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The security hold, and who holds it. The hold is the one control that outranks the governor's own judgment, and it is deliberately not the governor's to hold. It lives in a ConfigMap owned by another identity (the operator's security team, in practice); while its key reads true, every expansion the governor would make is blocked and every contraction is still allowed, so a governor under a hold can only give capacity back, never take more. The governor reads the hold on every decision and records in its audit that it read it; it has no verb that could write it. The kill switch (Chapter 49) is the blunter instrument beside it: the hold constrains a running governor, the switch stops every governor on the machine and hands everything back. An operator who suspects anything untoward uses the switch first, reads the audit second, and rotates the governor's identity third; the hold is for the slower case of a change window or an incident elsewhere during which no one wants capacity added.

What has been reviewed, and what has not. Reviewed and tested on every run: the governor's permissions against the list above, read from the cluster itself; the shield's refusal of every write past a cover, of every give-back while blind, and of every move under a hold, on two million generated cases; the lease and the watchdog's hand-back on a real cluster, thirty times (section 60.8); the checksums of every download. Not yet done, and said so: an independent security review of the code by a party outside the company, which is in the open program (section 60.15), and a penetration test of the deployed governor on a customer-like cluster. Until those are done, the operator's own review stands in for them, and this section tells the operator exactly what to look at.

CHAPTER 51

The Operator Manual

Before you wire anything: read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 42.5 of the manual (*Wired right or wired wrong*).

The current manual is `docs/INTEGRATION_MANUAL.md` (every stack, every level, every switch). This page is kept for its first-person account of the laws; where the two differ, the integration manual is current.

I am Omni-Compass. I am the brain and the nervous system; your stack is the muscle. Everything below is what I am and what I do, in my own terms and in my own mathematics. Every command and flag here exists in this repository.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

51.1 My face

My face is the compass. It is not decoration: it is how I read myself, every decision, and it is written into my code (`omnicompass/compass.py`).

51.1.1 The wheel

My deviation E, over its ceiling E[max], runs along the horizontal. Its rate of change runs along the vertical. Where I stand on that wheel is my heading, measured clockwise from north:

Point	Letter	Heading	Where I am
+	A	0°	rising through rest
⇄	Δ	45°	expansion: above rest and rising, exchange under way
>	B	90°	peak extension: the turn, where my metric flips (k → −k)
⊤	Λ	135°	dispersion: above rest and easing, my ceiling holds
−	Ω	180°	falling through rest
≈	Π	225°	compression: below rest, settling into my basin
<	Γ	270°	deepest compression: the turn at my floor
✦	Ψ	315°	re-alignment: below rest and rising, ignition ahead

My rim carries twenty-four letters, fifteen degrees apart, clockwise from A:

A E Z Δ H Θ B I K Λ M N Ω Ξ O Π P Σ Γ T Y Ψ Φ X

Every reading names its letter.

51.1.2 The four strokes

My quadrants are the strokes of the closed circle:

Quadrant	Stroke	Where I am
I	Expansion	above rest, rising
II	Dispersion	above rest, easing
III	Compression	below rest, falling
IV	Re-alignment	below rest, rising

The wheel turns I → II → III → IV → I:

- **Ignition:** crossing from IV into I.
- **Metric flip:** the turn at peak extension.
- **Continuity:** I am continuous through every crossing:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

$\lim X(t^-) = \lim X(t^+)$

Nothing in me switches off, and nothing restarts.

- **Counting:** I count every circle I close.

51.1.3 The axle

My axle is the structural basin S. Its rest point S* solves my equation (6):

$\delta - \alpha_s S - \tfrac{3}{4} \beta_s S^2 = 0$

Without the axle, deviation diverges. With it, deviation circulates.

51.1.4 Closing the circle

I hold the three conditions of the Unified Circle Principle and report them every decision:

- $\dot{\mathbf{X}} = \mathbf{G(X)}, \mathbf{X(0)} \in \boldsymbol{\Omega}$. Ω is my living band: every level I hand out lies between 5% and 95% of its range.
- $\mathbf{G(X) \cdot n(X) \leq 0}$ **on** $\boldsymbol{\partial\Omega}$. At a boundary, my next move points inward, never outward.
- $\boldsymbol{\nabla L(X) \cdot G(X) \leq 0}$. My ledger $L = E^2/2 + \Phi(S) - \Phi(S^*)$ descends when nothing forces me. When outside load forces me, I say so; I never hide it.
- $\Rightarrow \mathbf{lim\ X(t)} \in \mathbf{M^*}$. I settle into my basin.

A reading, as I print it on every decision trail:

```
Ψ ✦ 318.2° Re-alignment: re-alignment, ignition ahead | Ω: machine_fil
    ↪ l below the floor, returning | G·n≤0 yes | L 0.4121 -0.0133 | ax
    ↪ le S 0.19 (rest 1.90) | circles 3
```

51.2 My laws in your system

51.2.1 One switch, for all of me, in a human hand

I have one switch, the Unified Control Switch. A human turns it; I never turn it myself.

- **ON:** I am the primary control authority inside the scope you granted me.
- **OFF:** every setting I changed returns to what it was, and your native control takes full custody.

```
# OFF (all of me, at once, at any moment, for any reason, including a
    ↪ suspicion that someone has taken my brain):
kubectl -n omni-compass exec deploy/omni-compass -- touch /tmp/omni.ki
    ↪ ll
# ON:
kubectl -n omni-compass exec deploy/omni-compass -- rm /tmp/omni.kill
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What OFF restores. Every HPA CPU target and replica range, pod CPU limit, idle mark, paused rollout or job, GPU and CPU frequency ceiling, and containment quota. Each restore is recorded, and each lever restores on its own.

What never trips the switch.

- The switch never turns one organ off.
- No boundary, no error and no failed decision trips it.
- A decision that fails writes nothing, and my next decision comes on time.

51.2.2 My living band: 5% to 95%

Every level I hand out lives inside Ω = [0.05, 0.95] of its range:

- frequency ceilings, GPU power limits and the site power envelope;
- each organ's share of the energy budget;
- how full I run any machine.

No part of the body is driven to zero: a part with no work idles at its floor, alive and ready. No part is driven to its absolute top: I never spend the last five percent.

Tested over 300,000 of my states and 60,000 of my energy allocations (`tests/test_living_band.py`).

51.2.3 Machines idle; I never switch them off, and I never move a pod

When I need fewer machines, I idle the rest by letting their work leave on its own:

1. **Prefer not:** the machine is marked `omnicompass.io/idle:PreferNoSchedule`. New pods go to the open machines first, but a pod that finds them full lands here at once, so no pod ever waits because of me.
2. **Marked:** its pods are marked first to go (`controller.kubernetes.io/pod-deletion-cost`), the machine with the least work first. When the load falls, your autoscaler's own scale-down removes exactly those pods, emptying one machine at a time.
3. **Idle:** once its work is gone, the machine stays powered and Ready, gauged down to its idle floor.

While it still carries work, a machine counts as in service at full power. No pod is ever evicted, moved or restarted to idle a machine. When work returns, I remove the mark, the warm machines still carrying work first; it is in service at once, with no boot and no power cycling.

An idle machine draws `park_frac × idle power` (0.25), never zero.

51.2.4 Every change is continuous

- I idle at most one machine per decision.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- I wake a machine within five seconds of a pod waiting for a place.
- My pod reflex reads the queue every five seconds.

Nothing jumps, and nothing is restarted.

51.2.5 My pod sense: the muscle makes the pods

Your autoscaler is the muscle that makes and removes pods. I never start or stop a pod it would not.

Its rule is replicas = current × busy ÷ target. I read the same rule from the live queue every five seconds:

- a replica serving requests answers in $R = S / (1 - u)$, so $u = 1 - S/R$;
- S is the bare service time: the fastest tenth of the recent requests;
- R is the recent mean response, over the same window;
- your target, in queue terms, is target × request ÷ limit.

I record what the queue needs (`pod_reflex_reading`) and act only through the energy I give the pods and the target I hold for the muscle.

The target I hold. g is the CPU each pod is guaranteed with your autoscaler's largest count spread over the machines in service, divided by your limit. That target moves only when a machine idles or wakes, never each time a pod starts or leaves.

The muscle's own clock. I hold each target for your autoscaler's scale-down window (300 s unless you set one), the time it takes to answer a target. A target moved faster would pull the muscle mid-movement and start pods it then removes. A response-time breach returns your own target at once.

The reflex that raises the floor itself exists (`--pod-reflex-writes`); it is off unless you turn it on.

51.2.6 My energy is moved, never created

One budget comes in, and I convey it across the body by need (`omnicompass/conveyance.py`):

- $da_i/dt = \kappa\, a_i\, (e_i - \bar{e})$;
- the budget is conserved exactly, and each organ converges to its share of the demand;
- organs with no work idle at their floor, and what they do not need goes where it is needed;
- nothing leaves Ω.

On each machine, I hand its idle CPU to the pods serving on it:

$$c_i = \min\left(\max(L_i,\ (0.95\, A_j - Q_j) / |P_j|),\ 0.95\, A_j \right)$$

- A_j is the machine's CPU.
- Q_j is what every other pod on it has requested.
- P_j is its serving pods.
- L_i is the limit you gave the pod.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

A pod's CPU limit is a quota. A request that needs more than one quota period waits for the next one while the machine stands idle. That wait is energy withheld from the work, not saved, because the request spends the same CPU-seconds either way.

I change the limit in place: the pod is not restarted, its request is untouched, and it never gets less than you gave it. Every fifteen seconds a new pod gets its share. The OFF switch returns every pod to your limit.

51.2.7 I see before I act

My nervous system runs both ways:

- **Afferent:** a sense that is stale, frozen or unreadable is blind, and while any sense is blind I give nothing back.
- **Efferent:** I read back every order I give. An order that did not land blocks my next release.

51.3 Wiring me in, step by step

Take the steps in order. Take the next step only when this step's pass condition holds. The switch works at every step.

51.3.1 Step 0. What your stack needs

1. Kubernetes 1.34 or newer, with metrics-server (`kubectl top nodes` answers).
2. An HPA with a CPU target on each service I govern.
3. A PodDisruptionBudget on each service, for your own maintenance; I never evict a pod.
4. A readiness probe and a short preStop pause on each service, so a new pod never takes traffic before it answers and a pod your autoscaler removes never drops a request as it leaves (`deploy/kind/demo.yaml`).
5. A response-time feed for each governed service, as CSV `elapsed_seconds,latency_ms,ok`. `scripts/latency_probe.py` writes one.

51.3.2 Step 1. Build me

```
docker build -f deploy/Dockerfile -t <registry>/omni-compass:<tag> .
docker push <registry>/omni-compass:<tag>
```

I run as non-root, with a read-only root filesystem and no Linux capabilities. I carry my engine, my nervous system, my compass and my frozen law.

51.3.3 Step 2. Let me watch (monitor only)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
kubectl apply -f deploy/install/omni-compass.yaml     # set the image
    ↪ line; --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

Identity. I hold a read-only identity. `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts showing I cannot write.

Pass condition. Zero writes, and readings your operators agree with.

51.3.4 Step 3. Give me the pods (on top of your autoscalers)

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`, `deploy/kind/rbac-omni.yaml`).
2. Run me with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.

Your HPAs keep scaling. My pod reflex raises floors ahead of the CPU averages.

Targets. I hold your promise in queue terms: busy = target × request ÷ limit.

- g is the CPU each pod is guaranteed (section II.5), divided by your limit; the target that keeps each pod exactly as busy is g times yours.
- The pod answers faster, because it has g times the CPU, at the same busy share.
- Apart from that, I only tighten, never loosen.
- While response time is over your target, and for 180 s after (the clear time), your own target stands.

Pass condition. p95, p99 and failed requests never behind native.

51.3.5 Step 4. Give me the machines

1. Grant `patch nodes` and `patch pods` for the first-to-go mark (`deploy/kind/rbac-omni.yaml`).
2. Run me with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<park/wake command with {n}>"`.

Your platform	The park/wake command
any cluster, kind, bare metal	`bash scripts/kind_nodepool.sh {n}` (close to new work and mark first to go; open again, warm machines first)
Karpenter / EKS Auto Mode	the NodePool CPU limit at `{n} × node CPU`, parked nodes kept, not consolidated away
Cluster Autoscaler node group	the group's desired size, with scale-down through parking, not deletion
OpenShift	the worker MachineSet replicas, parked, not deleted

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

I give a machine back only when all of these hold:

- every sense is live;
- my last order landed;
- pods are not scaling up;
- nothing is waiting for a place;
- the machines that remain stay inside Ω.

Pass condition. Fewer machines in service and less energy, with no LOSS on any service gauge.

51.3.6 Step 5. Let me decide alone (Kubernetes as the muscle only)

Add `--strict-replicas`:

- I decide each service's replica floor and when to shrink;
- your HPA stays as the fast up-reflex;
- my node law sizes the machines.

Pass condition. As step 4.

51.3.7 Step 6. Give me the hardware

Organ	How to wire it
CPU frequency	`--cpufreq-policy-root /sys/devices/system/cpu/cpufreq --cpufreq-require-schedutil --rapl-cmd "<prints package watts>"`
GPU	`--gpu-query-cmd "nvidia-smi --query-gpu=power.draw,temperature.gpu --format=csv,noheader,nounits" --gpu-power-cmd "nvidia-smi -pl {w}" --gpu-max-w <max>`
Power and cooling	`--power-cmd "<prints site watts>" --site-limit-w <limit> --cooling-cmd "<sets {c}>"`
Batch and rollouts	`--batch --batch-pace --rollout-guard <ns/deployment>`

Pass condition. Energy per unit of work down, with no LOSS on any service gauge.

51.3.8 Step 7. The switch drill, at every step

1. Turn me OFF.
2. Check every setting is back to its recorded original, and no `omnicompass.io/*` annotation remains.
3. Turn me ON.

`scripts/kind_bench.sh` does exactly this after every live run.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

51.4 How to see me work

- **Live, paired, on real Kubernetes:** `benchmark-reps.yml` (a commit with `[reps]`, or by hand). Each repetition runs native, me on top, and me alone, back to back on one machine.
- **On your own machine:** `bash RUN_LIVE.sh 3`.
- **Every run leaves:**
  - my audit log of every read, write and compass reading;
  - my decision trail;
  - my identity receipts;
  - a SHA-256 fingerprint of every file.

51.5 When you read these lines

I say	I mean
`gate: a sense is blind`	I cannot see, so I give nothing back until I can
`gate: pods scaling up`	pods first, machines after
`decision failed (n in a row)`	I could not reach the cluster and wrote nothing; turn me OFF if you want native now
`pod_reflex_reading`	what the queue needs now; the autoscaler decides the pods
`convey: <machine> idle CPU to its k serving pod(s), limit c`	that machine's idle CPU now reaches the work on it
`Ω: machine_fill below the floor, returning`	the machines are underfilled and my move is bringing them back into the band

PART SEVEN

Proving It

Paired runs, receipts, rules written before the runs, and every result to date with its evidence class.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 52

The Receipts, in Their Totality

A **receipt** here is a result file in this repository made by a paired run: the system as it runs on its own (native) and the same system with Omni-Compass on top, the same work in both arms, the rules written down before the counted runs, the engine identified by its fingerprint (`tools/omni_version.py --commit`), every row reported, losses included. A receipt is read in the class of evidence it is: **P** a physical meter, **L** real software (a real Kubernetes cluster, a real database), **S** our own models, **V** a property of the code. S is never presented as proof of a physical saving. A result belongs to the engine it ran on and is never read across engines. Each test is read from its own newest three-run table, and every row names its engine: omni-v11's where its three runs have landed, omni-v10's where they have not. omni-v23, the engine that stands since 31 October, runs every benchmark again, and its tables replace these as they land (omni-v22 brought the five data stores' wires to the max, each spend gated on the store's own account of a miss, and changed nothing the Kubernetes governor runs; omni-v21 gave the immune system its self-knowledge, knob by knob; omni-v20 fixed the five data stores' wires at their causes; omni-v19 gave the Kubernetes governor a pod muscle of its own for a workload no autoscaler governs, which writes nothing unless a workload is named to it; omni-v18 took Omni direct into the fingerprint and changed nothing the Kubernetes governor runs; omni-v17 changed the Kubernetes governor's default beat, from 15 to 60 s, the heartbeat a dial, and kept its cruise wait at two readings or more at a slow beat; omni-v16 changed only the Kubernetes governor's clocks, each now counted in seconds, identical to omni-v15 at a 60 s decision, and set its full decision every 15 s, omni-v15 only the brain's power-cap request and the fingerprint's scope, omni-v14 only the Kubernetes governor's knob and its commands, omni-v13 added only the Kubernetes governor's hand-back of the CPU limits it gave on a shared host at its top, omni-v12 only the governor's hold on a raise the cluster refused and the swarm's reflex at once, omni-v11 only the governor's hold on a reading with no machine in service, omni-v10 only the CityLearn battery's glide). The earlier engines' receipts are the road, in `docs/history`. How to read a single table: `docs/HOW_TO_READ_THE_RESULTS.md`. The wording of every three-run row: `tools/readings.py`.

52.1 What the receipts say, read together

1. **More work in time, and a faster tail, on the same machines (L).** On a real Kubernetes cluster, Omni-Compass on top of the pod autoscaler cut the 95th-percentile response by 54% to 66% in the steady, wandering, all-four, fault and public-trace tests, in all three runs each (omni-v10). Where demand outran what native served in time, the work done inside the response line rose 47% to 79% (the all-four test). Failed requests fell 9% to 18% and the alert minutes 16% to 100% where demand stepped, swung or machines failed. On top of Goldilocks the 95th percentile fell 61% to 64%, on top of KEDA on CPU and requests 52% to 53%, and on top of KEDA's HTTP add-on 72% to 74% (omni-v11); in all 90 paired repetitions of those three tests Omni-Compass's 95th and 99th percentiles were below native's

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(`docs/TAIL_LATENCY.md`). Read together: the governor turns capacity the native controller leaves unused into service, on the same machines.

2. **The same work on less, where less is enough (L).** A queue of batch jobs ran on 6% to 14% fewer machines with 4% to 10% less standby energy and a 10% to 12% faster mean response, and the queue finished no later (omni-v10). Kafka held 0.8% fewer consumers on `burst`, MongoDB's cache gave back 0.9% to 1.0% on `f`, and Redis's ceiling fell 2.3% to 2.6% on `burst` (omni-v11). PostgreSQL read STEADY on every judged row but one (power given) and MySQL on every row but its update throughput (the runs disagree): under the brain's verdict no give-back that would cost service was written.
3. **What it cost, shown beside it.** The replicas the faster recovery took (faults +14% to +20%; the Google day: pods started +120% to +193%), the energy of workers left on at idle power on top of Goldilocks and KEDA (+1.1% to +2.3%, a declared model), Redis's memory on `small` (+1.7% to +1.9%), PgBouncer's most connections open (+8% to +10%) and MySQL's pool from its factory size (+5% to +16%), each power given in all three runs, beside the service it bought.
4. **Everything handed back (L, omni-v11).** The kill test: the governor killed outright in the middle of the window in 30 repetitions over three runs, every setting back at the operator's 8 to 11 s later (the allowance is 60 s), a second governor governing to the end in every one, and the 120 s after the kill STEADY against native. The long run, two hours an arm: the governor's memory up 2.3% at most (no leak), 97.5% of the decisions its interval predicts, none failed, its decision time not growing, and every setting handed back at the end.
5. **On our own models and the independent simulators (S).** The 1,107 muscles each alone: 527 GAIN, 580 STEADY, none LOSS; every one of the seven organisms GAIN (fewer misses) under all three loads. The power grids: 67 gauge-rows GAIN, 14 power given, no LOSS. The robot arm the trial moved: torque −28.6%, tracking error −21.3%. The buildings with batteries: 27 score-rows GAIN (the bill −16.1% in the one district with a price), 1 power given, no LOSS, the outage districts' batteries left native. The drone swarms: energy a mission −7% to −20%, missions a charge +8% to +25%, no near miss or collision in the counted cells. **On a mission draw outside those cells, two drones collided**: the wire's proximity reflex slowed a drone over about a second where its own rule says at once. That is a fault in the wire, root-caused; its fix (the reflex at once, reaching out with the square of the speed) landed in omni-v12 (`docs/OMNI_V12.md`, `results/model_gate/V12_SWARM_REFLEX.md`), and the swarm runs again on it.
6. **What is not yet shown.** No energy saving on a real chip: the physical receipt (P), the GPU's own meter on the current governor, is the decisive one still to be produced; every earlier card result is obsolete. No cloud-bill result on the engine that stands yet: the Azure sets run one at a time on it, up to 100 repetitions each under the preregistered stopping rule. More completed work at one fixed resource cap: the capacity test reads STEADY at all four caps.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

52.2 Receipt by receipt

Newest engine first within each class. "In all three" means the same sign in runs A, B and C with every 95% interval clear of zero; STEADY means an interval includes zero. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

52.2.1 Real software (L)

Test	Engine	Native	What moved	Reading	It shows	It does not show	File
The governor killed outright mid-run	omni-v11	HPA	handed back 8 to 11 s after the kill in 30 of 30 repetitions; p95 −39% to −58%; failed requests −7% to −12%; alert minutes −16% to −23%; replicas +3% to +6%	GAIN in all three; hand-back in every repetition; replicas power given	a governor that can die without leaving a setting behind	the 99th percentile (STEADY: +25% in run A, −25% to −27% in B and C)	`results/live/V11_ROBUST_KILL.md`
The long run, two hours an arm	omni-v11	HPA	no leak (memory +2.3% at most); decisions valid; none failed; every setting handed back; mean response −34% to −36%; replicas +49% to	held in every repetition; GAIN in all three; replicas power given	it runs for hours unchanged	the tails (STEADY, three pairs a run)	`results/live/V11_ROBUST_LONG.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Engine	Native	What moved	Reading	It shows	It does not show	File
			+52%				
On top of Goldilocks and the VPA's recommender	omni-v11	the service rightsized by Goldilocks, HPA	p95 −61% to −64%; time over the line −98% to −100%; alert minutes −100%; pods started −53% to −61%; energy +1.7% to +2.3% (model)	GAIN in all three; energy power given	a faster service on top of a rightsizer	a measured energy saving	`results/live/V11_GOLDILOCKS.md`
On top of KEDA's HTTP add-on	omni-v11	KEDA 2.21, HTTP add-on 0.16	p95 −72% to −74%; alert minutes −100%; replicas −11% to −20%; energy +1.6% to +2.2% (model)	GAIN in all three; energy power given; the pod start wait STEADY	faster service on top of a second autoscaler	a measured energy saving	`results/live/V11_KEDA_REQUESTS.md`
On top of KEDA on CPU and requests	omni-v11	KEDA 2.21	p95 −52% to −53%; replicas +2% to +3%; energy +1.1% to +1.8% (model)	GAIN in all three; replicas and energy power given	as above	as above	`results/live/V11_KEDA_CPU_REQUESTS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Engine	Native	What moved	Reading	It shows	It does not show	File
Capacity at a fixed cap (four caps)	omni-v11	HPA under a CPU quota	—	STEADY at every cap	no harm under a cap	more work under the cap	`results/live/V11_CAPACITY.md`
Redis	omni-v11	the operator's 64 MB ceiling; also as shipped	work inside the 2 ms line and hit rate +1.3% to +1.6%; mean latency −1.7% to −7.3%; ceiling −2.3% to −2.6% on `burst`; memory +1.7% to +1.9% on `small`	GAIN in all three; memory power given on `small`; as shipped STEADY (at its limit on native)	a faster cache, its price shown	a tail gain (the 99th percentile STEADY, `docs/TAIL_LATENCY.md`)	`results/live/V11_REDIS.md`, `V11_REDIS_SERVICE.md`, `V11_REDIS_SHIPPED.md`
Kafka	omni-v11	the operator's 2 consumers	consumers −0.8% on `burst`	GAIN in all three under both objectives; the rest STEADY	no harm, a fraction given back	a service gain	`results/live/V11_KAFKA.md`, `V11_KAFKA_SERVICE.md`
PostgreSQL, MySQL, MongoDB	omni-v11	the operators' pool, buffer pool and cache; MySQL and MongoDB also as shipped	MongoDB's cache −0.9% to −1.0% on `f`; PgBouncer's most connections open +8% to +10%;	GAIN and power given as named; STEADY on every other judged row but MySQL's update	no harm under the verdict	memory given back on the pool and the buffer pool	`results/live/V11_PGBENCH*.md`, `V11_SYSBENCH*.md`, `V11_YCSB*.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Engine	Native	What moved	Reading	It shows	It does not show	File
			MySQL as shipped: the pool +5% to +16%; at the max, PgBouncer's connections held −10% to −22% and MongoDB's cache −0.4% to −0.8%	throughput (the runs disagree)			
On top of CruiseKube	omni-v10	CruiseKube 0.3.4	—	STEADY on every row; no command written	no harm on a rightsized service	a gain there	`results/live/V10_CRUISEKUBE.md`
Steady work in steps	omni-v10	HPA	p95 −64% to −65%; p99 −69% to −73%; time over the line −98% to −100%; alert minutes −100%	GAIN in all three; machines and energy STEADY	a faster service on the same machines	fewer machines	`results/live/V10_STEADY.md`
Demand that wanders	omni-v10	HPA	p95 −57% to −66%; time over the line −38% to −45%; alert minutes −21% to −32%; energy −0.3% to −0.4%	GAIN in all three	service held through wandering demand	a measured energy saving (model)	`results/live/V10_WANDERING.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Engine	Native	What moved	Reading	It shows	It does not show	File
All four at once	omni-v10	HPA	work inside the line +47% to +79%; p95 −56% to −61%; failed requests −9% to −14%; alert minutes −16% to −34%	GAIN in all three; machines the same	more work in time on the same machines	energy (STEADY, model)	`results/live/V10_ALL_FOUR.md`
Faults: a machine down, a spike, a runaway pod, a blind probe	omni-v10	HPA	p95 −54% to −63%; p99 −43% to −51%; replicas +14% to +20%	GAIN in all three; replicas power given	service held through faults	machines (STEADY)	`results/live/V10_FAULTS.md`
The Google cluster trace of 2011, replayed	omni-v10	HPA	p95 −56% to −62%; time over the line −74% to −77%; failed requests −15% to −18%; alert minutes −61% to −66%; pods started +120% to +193%	GAIN in all three; pods power given	a public day of real demand served faster	machines (the same), a cloud bill	`results/live/V10_TRACE_GOOGLE2011.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Engine	Native	What moved	Reading	It shows	It does not show	File
A queue of batch jobs	omni-v10	Jobs and HPA	machines −6% to −14%; node-hours −6% to −14%; standby energy −4% to −10%; mean response −10% to −12%; queue finish STEADY	GAIN in all three	the same queue on fewer machines	a measured energy saving (model)	`results/live/V10_BATCH.md`
A noisy neighbour	omni-v10	HPA	time over the line −11%	GAIN in all three; every other row STEADY, the neighbour's own included	no harm beside a noisy neighbour	—	`results/live/V10_FAIRNESS.md`
Azure's managed Kubernetes, its own bill	v1	AKS with its cluster autoscaler	bill −4.7% steady, +5.0% burst	STEADY on every gauge	the method on a real cloud bill	a bill saving (the omni-v11 sets run now)	`results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md`

52.2.2 Our own models and the independent simulators (S)

Test	Engine	Reading	File
Power grids: 11 SimBench grids in pandapower	omni-v11	67 gauge-rows GAIN, 14 power given, no LOSS: energy drawn and net	`results/live/V11_PANDAPOWER.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Engine	Reading	File
		import lower in all 11 (ZIP loads); losses lower in 7, power given in 4; tap operations fewer in 10, more in 1	
CityLearn districts	omni-v11	27 score-rows GAIN, 1 power given, no LOSS: in the four districts whose batteries moved, electricity bought −3.0% to −13.6%, the daily peak −3.2% to −16.9%, ramping −3.4% to −10.1%; the bill −16.1% in the one district with a price; the seven 2023 districts' batteries native (each charge is its building's reserve), every score there STEADY	`results/live/V11_CITYLEARN.md`
Robot arms, MuJoCo Menagerie	omni-v11	Gen3 peak torque −28.6%, tracking error −21.3%, GAIN; UR5e and iiwa left native	`results/live/V11_MUJOCO.md`
Drone swarms, 20 drones in three cells	omni-v11	energy a mission −7% to −20%, missions a charge +8% to +25%, GAIN in all three; no late mission, reserve breach, near miss or collision in the counted cells; the closest approach spent to about 0.16 m (shown, not judged); collisions on a draw outside the cells, the wire's fault, fixed in omni-v12 (its three runs	`results/live/V11_SWARM.md`, `results/model_gate/V12_SWARM_REFLEX.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Engine	Reading	File
		under way)	
The 1,107 muscles and the seven organisms, preregistered seeds	omni-v8 (omni-v9's exact Student t moved no reading; omni-v10 to omni-v16 changed files it does not run, or, in omni-v16's edge, nothing at the organisms' own step)	527 GAIN, 580 STEADY, none LOSS; every organism GAIN (fewer misses) under all three loads; reproduced exactly in three GitHub runs	`results/realms_v5/REALMS_V5.md`

52.2.3 A physical meter (P)

Test	Status
One card, the card inside the organisms, eight cards, on the current governor	to be run by the founder on rented cards, at one exact commit, after the CPU side (`docs/GPU_RUN_GUIDE.md`); every earlier card result is obsolete

52.2.4 The code (V)

The max's guarantees, one by one, with the code path that makes each one true: `docs/FORMAL_V5.md`. The budget law's conservation and convergence, the safety shield's 2,000,000 adversarial cases, the C++ twins equal to the Python: `docs/STATE_OF_PLAY.md`, "Verified in code".

52.2.5 The archive itself

Every archived run carries `SHA256SUMS_ALL.txt`, the SHA-256 of every file in its folder. A sum is a fingerprint: it shows that a file has not changed since the sum was written, not who wrote it. From 31 October 2026 the archive job signs each run's sums with Sigstore, keyless, under GitHub's own identity for that job, and the signature is entered in Sigstore's public transparency log (`SHA256SUMS_ALL.txt.sigstore.json` beside the sums). `python3 tools/verify_receipt.py <run folder>` checks every file against its sum and the sums against their signature; a run archived before that day reads "sums only". `python3 tools/reproduce.py` rebuilds every table from the archived runs. Neither check is an outside review: they show that the files and the tables have not changed, not that an outside party has repeated the runs (`docs/PILOT_PROTOCOL.md` is how that is done).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

52.3 What would change these readings

- A P run on a real card that shows no difference in the card's own energy at the same work would leave the energy claim at "not shown"; it is not claimed now.
- An A, B or C run that disagrees with the others turns a row into "the runs disagree"; it is looked into, never dropped.
- A loss in all three runs is reported as it stands, its cause looked for in our own wiring, setup and scoring first, and the next set run on the fix.

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

CHAPTER 53

How to Read the Results

53.1 The key: GAIN, STEADY and LOSS

Every judged reading in Omni-Compass is one of three words: **GAIN**, **STEADY** or **LOSS**. Better, same and worse are not used anywhere. The word is decided by what the gauge is for, never by the plus or minus sign of its number: who sets the aim differs from one system to the next, and each gauge's aim is written in its preregistration before the runs.

Word	What each letter stands for	Its sense in underwriting	What it means in a table
GAIN	**G** Growth (asset appreciation); **A** Advantage (risk reduction); **I** Improvement (margin expansion); **N** Net-positive (profitable outcome)	surplus and positive variance	the gauge moved toward its declared aim against native: more work done, a shorter wait, fewer misses or failures, or a resource cut
STEADY	**S** Stable (zero variance); **T** True to baseline (expected norm); **E** Equilibrium (balanced risk); **A** At par (no net change); **D** Drift-free (within the 0.5% floor); **Y** Yield held (nothing lost)	the par baseline: equilibrium	native and Omni-Compass within 0.5% of each other: the mean of the three runs' changes under the 0.5% floor, or no change at all; a knob native already runs at the top of its range is STEADY, at its limit on native

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Word	What each letter stands for	Its sense in underwriting	What it means in a table
LOSS	**L** Leakage (capital drain); **O** Off-track (adverse selection); **S** Setback (claim liability); **S** Subpar (negative variance)	actuarial deficit and exposure	the gauge moved away from its declared aim: less work done, a longer wait, more misses or failures

How the sign of a number becomes a word:

- **Time waited, missed or failed** (a response time, time over the line, failed requests): a fall is GAIN, a rise is LOSS.
- **Work done** (work inside the line, throughput, a hit rate): a rise is GAIN, a fall is LOSS.
- **A resource spent or held** (energy, the power drawn and its peaks and swings, machines, memory, processor time, a bill, wear on a part): a fall is a **cut**, and a cut is GAIN. A rise is **power given**: what Omni put in to buy the work and the speed shown beside it. Power given is always shown and never hidden, and it is never called a loss (the founder's order of 12 October 2026: the max rides every muscle's edge; it never shaves). A peak is power held, so a peak that rose is power given, in a data centre and in a building alike (17 October).
- **Work bought with each unit of a resource** (work per energy): a rise is a cut in what each unit of work took, a GAIN; a fall is power given, because it falls when more power is given for the same work.
- So a minus can read GAIN (a response time cut 65%, a bill cut 30%) and a plus can read LOSS (failed requests up).
- **The floor, and every change above it** (the founder's orders of 20 and 24 October 2026): a change of half of one percent (0.5%) or more either way is a change, and reads **GAIN** or **LOSS** (or **power given**) by what the gauge is for; STEADY is kept for a change under 0.5%. The change is the mean of the three runs' changes, each in percent of native. Where every run moved the same way with its 95% interval clear of zero, the word reads **in all three of our runs**; otherwise it reads **by the mean of our three runs** and says in how many of the three runs it stood beyond the noise, so a reader sees how firmly it stands. Where runs moved 0.5% or more in opposite directions, the reading is **the runs disagree**. A mean under 0.5% reads **STEADY, under the floor**, with its size.
- **Two numbers, never mixed** (the referee's work order of 25 October): every row shows native's value and Omni-Compass's, the **raw change** (Omni-Compass − native) / native with its true sign, + for a rise and − for a fall whatever the gauge is for, labelled raw, and apart from it the word with its size and no sign. A bill cut 3% reads raw −3.0%, GAIN 3.0%; failed requests up 2% read raw +2.0%, LOSS 2.0%; a pod start wait from 0.22 s to 0.83 s reads raw +275.0%, LOSS 275.0%; a change under the 0.5% floor reads STEADY 0.3%. No raw sign is ever turned to match a word.

Some preregistrations name their outcome with a label written before the runs (for example SUPERIOR WITHIN GUARDRAILS in the card protocol). The label stays as it was frozen, and every table shows it with its word first: TOWARD ITS AIMS reads GAIN; BOTH WAYS reads GAIN AND LOSS; AWAY FROM

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ITS AIMS reads LOSS; SAME AS NATIVE reads STEADY (the same as native); AT ITS LIMIT ON NATIVE reads STEADY (at its limit on native); SUPERIOR WITHIN GUARDRAILS reads GAIN (within the guardrails); ENERGY IMPROVEMENT WITH SERVICE TRADEOFF reads GAIN AND LOSS (energy cut, service traded for it); SERVICE IMPROVEMENT WITH ENERGY TRADEOFF reads GAIN, power given (service GAIN, energy given for it); NONINFERIOR / INCONCLUSIVE reads STEADY (inside the noise, within the guardrails); NOT ESTABLISHED reads STEADY (not established: inside the noise, a guardrail crossed on the mean). A run that breaks its own validity checks reads INVALID and carries no word.

Every reading is one of these. **GAIN, LOSS or power given in all three of our runs**: A, B and C moved the same way, every 95% interval clear of zero. **GAIN, LOSS or power given by the mean of our three runs**: the runs' mean moved 0.5% or more, with the count of runs that stood beyond the noise beside it. **STEADY, under the floor**: the mean moved less than 0.5%. **STEADY, the same**: no change at all. **The runs disagree**: runs moved 0.5% or more in opposite directions, so the test is unstable there and is looked into. **Shown, not judged**: a gauge declared in advance as shown only, such as the knob's own setting.

Why these three words: they are the words of underwriting and actuarial work, where every outcome is graded as surplus, par or deficit, and each letter spells out the same rule. Omni-Compass is there to add (GAIN), never to take (LOSS), and to hand a muscle back to its own native setting where it cannot add (STEADY). Its first rule is do no harm, in the founder's meaning of 19 October 2026: the organism Omni-Compass governs is never left turning out less than its native controllers make it turn out, nor less healthy, and every muscle works only as far as its own limit (`DISCLOSURES.md`, section 3). So **LOSS should never appear**: where it does, the row stays in the table, its cause is found and fixed (our own wiring first, then native's setup, then the scoring) and the test is run again.

Disclosure and disclaimer: GAIN, STEADY and LOSS, and the letters behind them, are The Omni-Compass LLC's own convention for reading its own results. The underwriting senses explain the words; they are not an actuarial, financial or insurance opinion, and no outside body defined, reviewed or approved them. A reading states what our own runs measured against native under the declared aims, not a promise about any other system; every number stands beside its word, so a reader can judge it by any other convention. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

53.2 Where Omni-Compass sits, and what each benchmark shows

Omni-Compass is software. It always sits on top of a native controller. A system runs it in one of two places, on bare metal or through the cloud, and in each place there are two arms, native and Omni-Compass on top of native: four arms, each read by its bill and the work done inside it. These are the words for them, used the same way on every page.

- **The software end.** The programs that carry the work: Kubernetes with its pod and cluster autoscalers, its schedulers and the orchestrators on it, the data stores (PostgreSQL, MySQL, Redis, MongoDB, Kafka)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

and the services they serve. Omni-Compass sits on top of their native controllers. This is where its benefit lives and where it is measured: real software on real computers (class L), on GitHub's own machines in kind clusters. Those machines are rented runners, never called bare metal and never called the chip.

- **Bare metal.** The first of the two places a system runs (the founder's order of 1 November 2026): the owner's own machines, or a chip run directly, with the chip's own controllers under the software. Its bill is the machine's own: the hours it runs, and the energy the chip's own meter reads. The software end goes with Omni-Compass to it, and there Omni-Compass can reach the chip's own knobs too.
- **Through the cloud.** The same software end on a cloud provider's rented machines: Kubernetes through the cloud (Azure's AKS today; Google's GKE and Amazon's EKS wired the same way, not yet run), the machines billed by the hour at list price and added or removed by the cloud's own autoscaler. Its bill is the cloud's own; the cloud's chips and back end are inside its price and out of Omni-Compass's reach: it governs what the cloud gives, through the cloud's own Kubernetes. The cloud is the wire between the software end and the provider's bare metal. The wire makes no benefit of its own, and no tenant can meter the provider's chips.
- **The chip.** Bare metal's own part: the processor or graphics card under the software, with its own controllers (the power limit, the clocks, the heat), measured with the card's own power meter (class P) where Omni-Compass sets the card's knobs through the card's published interface. Omni-Compass built into the silicon, or into a battery's own controller, is not something any run here can test, and it is not claimed.
- **The four arms.** Two places, two arms in each: A, native on bare metal; B, Omni-Compass on top of native on bare metal; C, native through the cloud; D, Omni-Compass on top of native through the cloud. Each is read by its bill and the work done inside it, the two places never added together, since a system runs in one or the other (`results/TWO_BILLS.md`). The software end's tests are one set read three ways (the Omni index on work, speed and machines; the same on work and speed alone; the layers composed): they never add together, and none of them is either place's bill.
- **Hold.** Omni-Compass on top of native harms nothing in that place: every judged gauge (the work done, the response time, the minutes over the line, the pods waiting, the failures) reads STEADY or GAIN against native by the one key, and a resource that rose (the bill, the machine-hours, the energy) is read beside it as power given. A place holds when its runs read no LOSS.
- **Where the benefit lives.** At the software end, where it is made and measured, with no bill in it. It shows through on the two bills, native against Omni-Compass on native: on bare metal the work done inside the machine's own bill (each dollar, and each joule of the chip's own meter), and through the cloud the work done inside the cloud's own bill (each dollar). The chip and the wire themselves add no gain of their own: they are where Omni-Compass must hold. A benchmark that holds is the proof of no harm in its place; the benefit is shown where it lives and explained for the rest.
- **One mechanism, in one place, felt along the whole path.** Omni-Compass acts in one place, the software end, on top of the controllers a stack already runs; it changes nothing in the chip and nothing in the network or the cloud. Its effect travels the whole path the work travels: a request starts in the software,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

crosses the wire (the owner's own racks or a cloud's machines), is computed on the chip and comes back. So the effect is checked at every place: the software end shows the gain (more work, faster, on fewer machines), and the wire and the chip show that it holds and carry the gain to their bills. Two places are tested because a system runs in only those two; one mechanism makes the gain, the way a city's traffic signals make more trips an hour on the same roads and the same cars.

Place	Its benchmark	What it must show	Its readings
The software end	Kubernetes and the data stores on GitHub's machines (kind), class L	the benefit, by the one key, and that it holds	`docs/BENEFIT_SHEET.md`, `results/OMNI_INDEX.md`
Bare metal, the chip	a graphics card on Lambda run directly, its rental the bill and its own power meter beside it, class P	that it holds, and the work done inside its bill, A against B: the work per dollar and per joule	`results/TWO_BILLS.md`, `results/gpu/`
Through the cloud	Kubernetes through Azure's AKS, the machines it bills, class L	that it holds, and the work done inside its bill, C against D: the work per dollar	`results/TWO_BILLS.md` and each set's `results/live/AKS_*.md`
The whole organism	the places composed by the product law, class S of P and L	how the places add up, an explanation and never a measurement	`results/WHOLE_SYSTEM.md`

This chapter teaches you to read every table Omni-Compass produces. You do not need to know Kubernetes or GPUs to follow it. Read the five ideas first, then go to the table you have in front of you.

53.3 The five ideas behind every table

1. **Native, Omni-Compass on native, and Omni-Compass direct.** Every result compares a system running alone ("native": Kubernetes alone, the card's own firmware alone, a building's own controller alone) with the same system with Omni-Compass sitting on top of it (Omni-Compass on native, "omni" in the tables), and, since the founder's order of 23 October, wherever native's own controller can be switched off, with the same system run by Omni-Compass alone (Omni-Compass direct, "solo" or "Omni direct" in the records). Same machine, same work, same moment: the only difference is Omni-Compass, and which controller moves the knob. The three arms are always shown together, and each pair is read on its own.
2. **Paired runs.** Each comparison is run many times ("repetitions" or "runs"), each time with both arms on the same seed (the same pattern of work). Each repetition gives one difference; the table reports the average difference.
3. **The 95% interval.** Next to each average is a range, for example `+0.092% (+0.090 to +0.093)`: the paired mean difference $\pm\, t_{0.975,n-1} \cdot s/\sqrt{n}$, where s is the standard deviation of the n paired differences and $t_{0.975,n-1}$ the two-sided 95% point of Student's t on $n - 1$ degrees of freedom

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(`omnicompass/student_t.py`, computed for any n: 4.303 at 3 pairs, 2.262 at 10, 2.093 at 20). The method that draws it covers the true difference in 95 of 100 repetitions of the whole experiment; it is not a 95% chance about this one range.

- If the whole range is on one side of zero, the difference is **clear of the noise** (a table's "significant" column says "yes"); its word is GAIN, LOSS or power given, by the gauge's aim.
- If the range includes zero, that run is **not beyond the noise**: it could not tell the arms apart by itself. It does not show that they are equal; how small a difference the run could have seen is the interval's half-width, printed beside it. Since the founder's order of 24 October the word is decided by the size first: a change of 0.5% or more reads its word even where a run's range includes zero ("by the mean of our three runs", with the count of runs beyond the noise), and only a mean under 0.5% reads STEADY (the key above; section "The verdict rules" below).

4. **The word, not the sign.** Every judged reading is GAIN, STEADY or LOSS (the key above). For response time, failures and time over the line a fall is GAIN; for work and work per energy a rise is GAIN; for energy, machines and every other resource spent or held a fall is a cut, a GAIN, and a rise is **power given**, never a loss. Every table gives the word beside the number.
5. **Evidence class.** Every result is marked with how it was measured:
   - **T**: a theorem;
   - **V**: verified in code;
   - **S**: our own model (modelled);
   - **L**: live software (real Kubernetes);
   - **P**: a physical meter (a real card's own power meter).

Only **P** speaks for hardware energy.

53.4 What every result names: the workload, the boundary, the profile, the costs, the requirements and the verdict

The common language of these six is written once in `docs/FACTORY_PROFILE.md`; this section is how to read a result by it. Every result identifies the engine version, the configuration, the hardware or model, the workload and the measurement boundary that produced it, and says nothing beyond them.

The workload contract. Both arms are offered the same work: the same demand at the same moments (an open-loop schedule, the same seeds in every arm), on the same machines, in the same window, against the same service line, all written down before the first counted run in the benchmark's preregistration (`docs/*_PREREGISTRATION.md`). A comparison is valid only for a matched workload; a run whose work was not matched, or a workload left incomplete, is shown as unresolved or invalid and carries no word. The load is a declared input, and a result belongs to the load it names (`DISCLOSURES.md`, section 4).

The measurement boundary. What a result counts, and what it does not: the cluster's machines in service, the host's processor seconds, a card's own meter, the machines a cloud bills, the plant models' energy. Each

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

receipt names its boundary, and nothing outside it is claimed. The energy on GitHub's Kubernetes is a declared formula over machines, not a meter; a model's energy is not a meter's; only class P speaks for hardware energy. The whole organism's composition (`results/WHOLE_SYSTEM.md`) names its three places, the software end, the cloud's wire and the bare metal, and enters a place with no measurement as exactly 1, named.

The profile's identity. Every result names its engine and fingerprint (`python3 tools/omni_version.py`) and the profile that governed it: the **factory profile** (the complete, versioned set of Omni-Compass default settings shipped together; the released one is omni-v19's, the governor's full decision every 60 s with its fast checks every 5 s and every muscle on its card or the engine's setting), a **candidate profile** (a proposed combination under evaluation) or a **custom profile** (an identifiable configuration that differs from the released factory profile, whose results are its own). The **active profile** is the one governing the run in front of you; the live benchmark engine names it with its fingerprint. A matching fingerprint establishes identity; the validation receipts establish what that identified implementation demonstrated. A result is never read across engines, and never across profiles. The best validated factory profile for the declared operating conditions has not yet been established (`docs/FACTORY_PROFILE.md`, section 5): until it is, every published result is the released factory profile's.

The costs. A resource spent or held that fell is a cut, a GAIN; one that rose is **power given**, never a loss. The bill of a deployment route r (on site; through the cloud) is the sum of its costs inside the declared boundary, each counted once, for each arm a (native, Omni-Compass on native, Omni-Compass direct): $C(r, a) = \Sigma_k\ c(r, a, k)$, and its saving is $S(r, a) = (C(r, \text{native}) - C(r, a)) / C(r, \text{native})$, for a positive native cost and a matched workload. A component's percentage (fewer machine-hours, less energy) is a supporting measurement, never the bill. A missing price stays visibly unresolved: today only the cloud's compute is metered in money (Azure, machine-hours at the list price, `results/live/AKS_BILL.md`), so neither route's bill is resolved yet. A whole-workload index (the composition's one number over its gauges) is not a price.

The requirements. Health is the explicit operating requirements: correctness, service limits (the response-time line), resource bounds and every applicable hardware limit; each reading H_j has its limit L_j. A result names which of the two objectives its tune served: the most healthy production, maximise $E[P(\theta)]$ subject to $H_j(\theta) \le L_j$ for every j (the max, the default brain), or the least cost per unit of completed production, minimise $E[C(\theta) / P(\theta)]$ subject to the same requirements (the tune-down, `dial save`, a tune anyone may set; the clouds ran it until 31 October, when the founder struck that rule and every cloud runs the max with the bill judged beside the work). P is the useful output, the work done inside the line. A requirement crossed is read on its own gauge (time over the line, failed requests, alert minutes) and never traded away.

The verdict rules. Every judged gauge is read by the one key (`tools/readings.py`, the key at the top of this page): the word by what the gauge is for, never by its sign; over runs A, B and C the change is the mean of the three runs' changes in percent of native; a mean under 0.5% reads STEADY, under the floor (STEADY, the same where nothing moved); from 0.5% up GAIN, LOSS or power given **in all three of our runs** where every run moved the same way with its 95% interval clear of zero, otherwise **by the mean of our three runs** with the count of runs beyond the noise; runs that moved 0.5% or more in opposite directions read **the runs**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

disagree; a count that must never rise (failed requests, lost messages) reads LOSS on any rise in any run. Each run's own number wears that run's own word by the same floor. A label written before the run is shown with its word first. A model (class S) is never read as proof of hardware, and a reading on the live tuning bench at any setting other than the factory profile is a variation of the mechanism, never a benchmark result.

53.5 The words in the tables

Word	What it means	Its aim
Work per energy	work done for each unit of energy (requests per kilojoule on a card)	higher
Energy (J, Wh)	the energy used	lower
Work, requests served	how much was done; both arms must serve the same work	equal
Response time, median (p50)	half of the answers were faster than this	lower
Response time, p95	95 of 100 answers were faster than this: the slow answers, what service promises are written on	lower
Response time, p99	99 of 100 answers were faster than this: the slowest answers	lower
Time over the line, violations (pp)	the share of time the service was past its promise, in percentage points	lower
Failed requests, not served	answers that never came	zero
Machines in service, node-hours	how many machines were kept on, and for how long	lower
Band first	the founder's rule: no win unless time over the line is no higher than native's	"held"
Knobs handed back, restored	at the end every setting Omni-Compass touched was put back exactly	True / yes
Label	the verdict, chosen by a rule written before the run, never by hand	see each table

53.6 The GPU card (`results/gpu/run-<stamp>/GPU_REPS.md`)

One card, three arms, many repetitions:

- **native**: the card's firmware alone;
- **watch**: Omni-Compass running and deciding but writing nothing. It must equal native: this proves Omni-Compass watching costs nothing;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **omni**: Omni-Compass on top, moving the card's clock ceiling and power limit.

Read it in this order:

1. **The verdict line** at the top, one of the labels chosen by rule (`docs/GPU_PREREGISTRATION.md`, amendment 1): SUPERIOR WITHIN GUARDRAILS, ENERGY IMPROVEMENT WITH SERVICE TRADEOFF, NONINFERIOR / INCONCLUSIVE, NOT ESTABLISHED, AWAY FROM ITS AIM, NOT ATTRIBUTABLE or INVALID; each gauge reads GAIN, LOSS or power given **clear of the noise**, or **STEADY, no difference beyond the noise** (amendment 14; the first counted run of 2 October printed "proven" and "not proven" for the same rule).
2. **Work per energy** (requests per kilojoule): the headline. A plus sign with a range wholly above zero is a GAIN clear of the noise.
3. **Requests served** must be equal in every arm. More energy saved by serving less work would be cheating; the bench checks it.
4. **p95 and p99**: the slow answers. With the verdict rule (`omnicompass/verdict.py`), Omni-Compass never takes a step that makes a request more than 2% slower.
5. **Writes and restore**: how many settings were changed, and that every arm ended at the card's own limit.

The file `omni-gpu-<stamp>-confirmations.tar.gz` holds two such tables: compute-bound work (matrix products) and AI token generation (memory-bound work).

53.7 The card inside the seven organisms (`results/hil/run-<stamp>/HIL.md`)

The real card is wired into each of the seven organisms (the spine, the four realms, the four stacked, the whole tower) at four sizes (1, 10, 100 and 1,000 copies). Each row is one size, one organism and one part:

- **stacks (model)**: the modelled machines (evidence S);
- **card (meter)**: the real card, its own meter (evidence P);
- **both**: the two added together.

The second table lists the card's own receipts for every arm: its energy, requests served, p95, and its power limit at the start and at the end, which must be equal.

53.8 Real Kubernetes (`results/live/LIVE_REPS_<n>.md`)

Each set is 10 paired repetitions on a real Kubernetes cluster (kind): native Kubernetes (its HPA and scheduler) against the same Kubernetes with Omni-Compass on top. Read:

1. **worker nodes in service** and **node-hours**: how many machines were kept busy. A fall is GAIN.
2. **response time p95 and p99**: a fall is GAIN (faster).
3. **failed requests**: must be zero on both sides.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

4. **CPU used with Omni's own**: Omni-Compass's own cost included; "no" in the significant column means its cost is inside the noise of the test (the interval beside it gives the size the test could have seen).
5. **The label** at the bottom, by the rule written before the run.

Energy on kind is a declared model, not a meter: kind keeps every machine powered.

53.9 The three-run tables (`results/live/V<engine>_<TEST>.md`, for example `V7_GOLDILOCKS.md`, `V8_CITYLEARN.md`)

Every benchmark since the engine was frozen runs three separate times, A, B and C, on the same bytes, and the table shows all three beside each gauge, each with its own 95% interval. Read the last column first, by the verdict rules above: **GAIN**, **LOSS** or **power given in all three of our runs** (the mean moved 0.5% or more, every run the same way with its interval clear of zero), **by the mean of our three runs** (the mean moved 0.5% or more; the count of runs beyond the noise beside it), **STEADY, under the floor** or **STEADY, the same**, or **the runs disagree** (runs of 0.5% or more pointing opposite ways: the test is unstable there and is looked into). Every table was read again from its own data by that rule on 24 October (`tools/reread.py`). Nothing reads "confirmed" or "not confirmed": nobody outside The Omni-Compass LLC has reviewed or approved these results. The `V<n>_` in a table's name (`V10_`, `V11_`, `V12_` and so on) is the engine the runs carried (`python3 tools/omni_version.py --commit <sha>`); a table never mixes engines, and a reading is never carried from one engine to another. Every LOSS and every power given is in the table beside the GAINs. On the omni-v11 tables there is no LOSS. Power given appears in the grid's dissipated energy in four grids, the tap operations in one, PgBouncer's connections, MySQL's pool from its factory size, Redis's memory and CityLearn's distance from net zero in one district. An earlier engine's LOSS stays in its own table, for example CityLearn's energy not served on omni-v7.

How often the rule reads a difference that is not there. Under no true difference, one run's 95% interval falls wholly above zero with probability 0.025 and wholly below with 0.025. A row reads GAIN or LOSS **in all three of our runs** only when all three independent runs fall on the same side, so its chance of a false reading is $2 \times 0.025^3 \approx 3.1 \times 10^{-5}$. A word read **by the mean of our three runs** carries no such bound: it says the runs' mean moved 0.5% or more, and the count of runs beyond the noise beside it says how firmly; read it with that count. Over a table of 30 judged rows the chance that any one is false is at most $30 \times 3.1 \times 10^{-5} \approx 0.09\%$ (a union bound, as Bonferroni's), and over the 798 judged rows of the omni-v11 tables that carry noise (1,039 judged rows in 23 tables, less the 241 of the deterministic simulators) the expected number of false readings is $798 \times 3.1 \times 10^{-5} \approx 0.02$. The bound assumes the three runs independent; runs on the same kind of machine can share a bias, which three runs cannot remove and the bound does not cover. The price of "in all three of our runs" is power: a small true difference must clear the noise three times, so many rows read by the mean of our three runs are differences too small for a run's pairs to resolve (with 3 pairs a run, $t = 4.303$ and the interval is wide). Every reading is printed with the three intervals behind it. The deterministic simulators (CityLearn, pandapower, MuJoCo) reproduce to the digit, so there the three runs check the harness, not the noise; their STEADY means the same to one part in a million.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

53.10 The workload tables (V<engine>_PGBENCH.md, _KAFKA.md, _REDIS.md, _YCSB.md, _SYSBENCH.md) and the swarm table (V<engine>_SWARM.md)

The database, the broker and the cache share one shape. The head names the three runs, their commits and the engine each carried. Then one section per **workload**: the tuning workload first, marked "shown and not counted" because the rules were fitted on it, then the untouched workloads, which are the judged ones. Each section says what the workload is (value sizes, rates, steps) and what native's own capacity measured on that run, so the reader knows how hard native was pushed. The rows are the gauges with their direction in the name: the first row is always **work inside the response line**, the product number; then throughput, the latency percentiles, failures (any increase in any run is marked), **the resource held** (connections, consumers, the memory ceiling: the aim is lower, and Omni-Compass holds more of it wherever it bought a gain, so expect it higher there), the host's CPU (the compass's own cost included), and the knob's moves (shown, not judged). The last line of each section says whether every arm handed the knob back; the last line of the table counts the rows toward their aims, away from them and disagreeing across the untouched workloads. The swarm table reads the same way with cells instead of workloads, with collisions voiding a cell, and with a declared reproduction tolerance (one part in a thousand) because PyBullet does not reproduce to the bit across machines; its amendment says so.

53.11 The governor's own cost (V3_OWN_COST.md) and the robustness tables (V<engine>_ROBUST_KILL.md, V<engine>_ROBUST_LONG.md)

V3_OWN_COST.md is Omni-Compass's own CPU, from the overhead record every omni arm's audit carries, by organism and copies, under the engine each run carried; it is shown and not judged, and it is a few thousandths of one core at every size. The robustness tables (results/live/V11_ROBUST_KILL.md and V11_ROBUST_LONG.md on the engine that stands; each earlier engine's are its own record) add rows that belong to the omni arm alone, since native has no governor to kill: the seconds from the kill until every setting was back at the operator's (the allowance is 60 s, declared before the run), whether a second governor started and governed to the end, and the governor's memory in the last ten minutes over the first ten (a quarter's growth is a leak, declared before the run); the 300 s after the kill (120 s on the tables up to omni-v14's) are compared between the arms like any paired gauge.

53.12 The organisms with a real cluster inside (SIX_KUBE.md, V3_SIX_KUBE.md, V3_BIG_ORGANISM.md)

Two kinds of rows, kept apart: **the cluster** (real Kubernetes, measured) and **the organism** (the modelled stacks around it, evidence S). "Organism behind its window (s)" is shown and not judged; a repetition reads **OFF THE CLOCK** when either arm ended more than 5% of its window late (its last steps saw a cluster whose load had already ended). The pairing stands, the mark stays, and the remedy is a longer window for that size on that machine.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

53.13 The one number (`results/OMNI_INDEX.md`)

Every measure is a ratio oriented so that above 1 is a GAIN for Omni-Compass on top of native: work, speed, machines, energy. A test's index is the geometric mean of its ratios, a category's of its tests, the headline of the real categories, each weighted the same. A measure enters only as its three-run reading allows: GAIN, LOSS or power given in all three of our runs enters as the geometric mean of the runs' ratios; anything else (STEADY, the runs disagree, a word read by the mean of our three runs) enters as exactly 1. Modelled muscles are shown beside the index, never inside it.

53.14 The real test, the twenty scenarios and the whole organism

- **The real test** (`results/orchestrator/V18_THREE_ARMS_ABC.md`): one row per pair and gauge, each run's own word beside its number, the three-run reading in bold; below, each arm's totals in each run (requests answered, failed, wrong). The bottom line on the README and the live benchmark engine page reads its pair Omni-Compass on native against native, takes the work row's mean of the three runs as its number and turns its light red on any slower service or failed request. The record's third arm, Omni-Compass alone on the muscles, is not carried there: Omni-Compass is never by itself (the founder's order of 25 October).
- **The twenty scenarios** (`results/scenarios/SCENARIOS_20.md`, class S): ten organisms, each native and with Omni-Compass on native. The first table gives native's own gauges and Omni-Compass on native's word and mean change against native; the sections after it give every run's change with its interval and the reading, each arm's own gauges, the realms runner's contrasts, and every LOSS row by name.
- **The whole organism** (`results/WHOLE_SYSTEM.md`): each layer's ratio oriented so that above 1 is a GAIN, composed by the product law $R(g) = \prod_\ell R_\ell(g)$; read the scope (with or without the card's old run) and the reading (by the index's rule, or every interval carried) before the number. It is a model of measurements, class S of class L and P, and its index is not a price.

53.15 The seven organisms grid (`results/grid_v5/GRID.md`; each engine's runs in `results/grid_v<engine>/`)

Columns are **copies × runs**: 1, 10 and 100 clustered copies, each at 1, 10 and 100 paired runs, and on GitHub's own machines 1,000 of each as well (paid cloud and card runs stop at 100). Rows are the seven organisms: the spine, the four realms, the four stacked and the whole tower. Each square shows the modelled value and the value with the real cluster inside. Read a square as "with Omni-Compass on top, against native, at this many copies, over this many runs". A square still running says so, and one beyond a free runner's memory or time is listed as pending runner capacity, never rented.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

53.16 Kubernetes at scale (`KWOK.md`, from the `kwok-scale` workflow)

Real Kubernetes at 50, 500 and 1,000 nodes (KWOK nodes: real Kubernetes objects with no machines behind them):

- **decision time** is how long Omni-Compass takes to decide, at that size;
- **memory** is what it uses;
- **master switch** must read "yes": everything handed back.

53.17 One number, one word (`docs/BENEFIT_SHEET.md`)

In the tables a change keeps its raw sign: a response time that fell reads "-65%", a cache that grew reads "+300%", and the Reading column gives its word, GAIN, STEADY or LOSS, by what the gauge is for (the key above). The benefit sheet turns every gauge the same way round against **its declared aim**: plus is a GAIN, minus a LOSS or power given, whatever the gauge measures, and 0% is STEADY. Beside it every change the same in all three of our runs is written out in words, never as a bare sign: "p95 cut 65% (GAIN)", "memory ceiling held up 215% (power given)", so neither can be misread against a table that shows the same changes as "-65%" and "+215%". One number per benchmark, by the index's own rule, with its word beside it. Read that sheet first; read the tables for the rows behind a number, and read their Reading column, never the sign alone. The wiring page uses the same words.

53.18 Wire in, or watch (`docs/WIRING_VERDICTS.md`)

One of three words per knob, by one rule from the tables above. **Write**: a gauge GAIN in all three of our runs and none LOSS or power given in all three of our runs; Omni holds the knob. **Watch**: nothing GAIN in all three of our runs; the knob stays native and Omni only reads it (one wire out, no wire in), and a loss in all three of our runs is named with its cause. **Operator's choice**: gains and losses both the same in all three of our runs, a trade, shown with both readings of the index. Rows marked "shown, not judged" never count. A modelled muscle takes its word from its label: superior is write, a tradeoff label is the operator's choice, the rest is watch. The CSV beside the page (`results/WIRING_VERDICTS.csv`) has one row per knob and per muscle.

53.19 Where to start

1. `docs/STATE_OF_PLAY.md`: one page, every result and how strong it is.
2. `docs/DOSSIER.md`: every result with its chart.
3. This chapter, next to the table in front of you.
4. `DISCLOSURES.md`: what a result is and is not.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

53.20 Reading the benefit: more work, or the same work for less

With the work held equal in both arms, the gain from Omni-Compass on top is *G = C[native] / C[omni] − 1*, where *C* is a resource (machine-hours, CPU core-hours, joules). The same work then needs *1 / (1 + G)* of the resources, a saving of *G / (1 + G)*: a third more work is a quarter off the bill. The full explanation, the three rules for reading a receipt and where every number stands today are in the Owner's Manual, chapter 3, *Where the Value Comes From* (`docs/OMNI_COMPASS_OWNERS_MANUAL.md`).

CHAPTER 54

The Benefit Sheet: One Number per Benchmark

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

How to read this receipt. Every judged row reads one of three words, decided by what its gauge is for and never by the plus or minus sign of its number. **GAIN**: the gauge moved toward its declared aim against native (more work done, a shorter wait, fewer misses or failures, or a resource cut). **STEADY**: native and Omni-Compass within 0.5% of each other (the mean of the three runs' changes under the 0.5% floor, or no change at all), or the knob is at its limit on native. **LOSS**: the gauge moved away from its aim (less work done, a longer wait, more misses or failures). A resource spent or held (energy, the power drawn and its peaks and swings, machines, memory, processor time, a bill, wear) that fell is a cut, a GAIN; one that rose is **power given**, what Omni-Compass put in to buy the work and the speed shown beside it, always shown and never called a loss; work bought with each unit of a resource (work per energy) that fell is power given too. So a minus can read GAIN (a response time cut) and a plus can read LOSS (failed requests up). A change of 0.5% or more, the mean of the three runs' changes, reads GAIN, LOSS or power given: **in all three of our runs** where A, B and C moved the same way with every interval clear of zero, otherwise **by the mean of our three runs**, with the count of runs beyond the noise beside it; **the runs disagree** names runs that moved 0.5% or more in opposite directions, a test unstable there; **shown, not judged** is a gauge declared shown only. The output follows the input: every reading is the result of the tune, the wiring and the load it names, and where a reading elsewhere differs the first presumption is the wiring (`DISCLOSURES.md`, section 4). The full key, letter by letter: `docs/HOW_TO_READ_THE_RESULTS.md`. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The founder's orders of 9 and 13 October 2026: next to every benchmark, say whether Omni-Compass on top was a benefit to the system and by how much, in one number whose sign always means the same thing, and say it in **GAIN**, **STEADY** and **LOSS**, judged by what each gauge is for and never by the sign of its number (the key: `docs/HOW_TO_READ_THE_RESULTS.md`). **On this sheet the sign is the system's own declared aims, gauge by gauge: a plus is a GAIN; a minus is a LOSS on work, speed, misses or failures, and power given on a resource spent (energy, machines, memory).** The number carries its word. Beside it every change judged across our three runs is written out with which way it went and its word: "p95 cut 65% (GAIN)", "memory ceiling held up 215% (power given)". An operator whose aims differ reads the same numbers the other way.

The result tables (the files in the last column) keep every change with its raw sign: a response time that fell reads "-65%" there, and the Reading column beside it says GAIN.

The number is the Omni index's rule (`results/OMNI_INDEX.md`): each judged gauge becomes a ratio oriented so that above one is a GAIN; it counts only where judged across our three runs, anything else counts as exactly one; the number is the geometric mean of the ratios, minus one, in percent. For the tests inside the index the number is the index's own resource reading of that test; for a stack with several workloads it is the stack's index category; for the simulators and the organisms the same arithmetic runs over every judged gauge of their tables. Built from the tables by `tools/benefit_sheet.py`, checked by `verify.py`.

Benefit? reads **GAIN** (gauges GAIN across our three runs, none LOSS or power given), **GAIN, power given** (a GAIN bought with a resource put in, named beside it), **GAIN AND LOSS** (both; the number says which way the trade nets by this scoring, and the wiring page `docs/WIRING_VERDICTS.md` says what pays for what), **LOSS** (a LOSS and no GAIN), **power given** (a resource put in and nothing gained) or **STEADY** (nothing moved 0.5% or more across our three runs). LOSS should never appear: where it does, the row stands and its cause is found and fixed.

Every row names the engine its table ran on (read from the table). Results are never read across engines: a row on an older engine stands as that engine's result until the same benchmark's three runs on the engine that stands replace it here, and every benchmark is rerun on the engine that stands.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
2026-10-31	Every real test together: the Omni index	omni-v21	**GAIN AND LOSS**	**5.0% GAIN**	the six real categories together; each stack's own words are in its row	`results/OMNI_INDEX.md`
2026-10-31	The six organisms with the real cluster inside, 10 and 100 copies	omni-v3	**GAIN, power given**	**5.3% GAIN**	p95 cut 32% to 52% (GAIN); time over the line cut 8% to 36% (GAIN); replicas up 0.6% (power	`results/live/V3_SIX_KUBE.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
					given); failed requests cut 8.4% to 8.5% (GAIN); parked-worker energy cut 0.3% (GAIN); standby-model energy cut 0.3% (GAIN)	
2026-10-31	The big organisms with the real cluster inside, 1,000 copies on Azure	omni-v3	**GAIN**	**155% GAIN**	p95 cut 95% to 98% (GAIN); p99 cut 97% to 98% (GAIN)	`results/live/V3_BIG_ORGANISM.md`
2026-10-31	The 945 modelled muscles as one tower, every muscle written at once	omni-v3	**GAIN**	**0.3% GAIN**	work per energy up 0.3% (GAIN)	`results/realms/REALMS.md`
2026-10-31	Robustness: the two-hour run	omni-v21	**GAIN AND LOSS**	**3.8% LOSS**	mean response cut 29% to 43% (GAIN); p95 cut 25% to 57% (GAIN); time over the line cut 30% to 41% (GAIN); failed requests cut 9% to 15% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 21% to 28% (GAIN); replicas up 29% to 50% (power given); pods started up 68% to 226% (power given); pod start wait, total (s) up 90% to 324% (LOSS); pod start wait, mean (s) up 24% to 27% (LOSS)	`results/live/V21_ROBUST_LONG.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
2026-10-31	Robustness: the governor killed outright mid-run	omni-v21	**GAIN AND LOSS**	**8.1% GAIN**	mean response cut 30% to 40% (GAIN); p95 cut 28% to 48% (GAIN); time over the line cut 33% to 35% (GAIN); failed requests cut 9% to 13% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 18% to 25% (GAIN); replicas up 2.3% to 7.3% (power given); pod start wait, total (s) up 10% to 79% (LOSS); robust: failed requests in the 300 s after the kill (%) cut 3.7% to 4.3% (GAIN); robust: time over the line in the 300 s after the kill (% of samples) cut 0.5% to 2.6% (GAIN)	`results/live/V21_ROBUST_KILL.md`
2026-10-31	Robot arms (MuJoCo), the three untouched robots	omni-v21	**GAIN**	**3.8% GAIN**	energy per takt cut 0.8% (GAIN); copper heating per cycle cut 11% (GAIN); mechanical work per cycle cut 6.6% (GAIN); peak joint torque cut 29% (GAIN); tracking error, RMS cut 21% (GAIN); end-point error at the waypoints cut 11% (GAIN)	`results/live/V21_MUJOCO.md`
2026-10-31	Redis, the cache's memory ceiling	omni-v21	**GAIN AND LOSS**	**0.1% GAIN**	memory ceiling held cut 1.3% to 2.4% (GAIN); memory used cut 0.1% to 1.0% (GAIN); host CPU	`results/live/V21_REDIS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
					busy cut 2% to 11% (GAIN); host CPU-seconds cut 2% to 12% (GAIN); CPU per 1,000 inside the line cut 2% to 12% (GAIN); work inside the line up 0.6% to 1.7% (GAIN); hit rate up 0.7% to 1.7% (GAIN); mean cut 0.6% to 7.8% (GAIN); memory ceiling held up 0.8% to 2.2% (power given); memory used up 0.5% to 1.9% (power given); p99 up 0.2% to 1.7% (LOSS); host CPU busy up 1% to 11% (power given); host CPU-seconds up 1% to 12% (power given); CPU per 1,000 inside the line up 0% to 11% (power given); process_cpu_seconds up 0.3% to 2.4% (power given)	
2026-10-31	Real Kubernetes, steady work	omni-v21	**GAIN**	**31% GAIN**	mean response cut 48% to 49% (GAIN); p95 cut 66% (GAIN); p99 cut 72% to 74% (GAIN); time over the line cut 100% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 100% (GAIN); replicas cut 4.1% to 6.3% (GAIN); pods	`results/live/V21_STEADY.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
					started cut 20% to 49% (GAIN); pod start wait, total (s) cut 20% to 54% (GAIN); pod start wait, mean (s) cut 0% to 21% (GAIN)	
2026-10-31	Real Kubernetes, faults	omni-v21	**GAIN, power given**	**30% GAIN**	parked-worker energy cut 0.3% to 1.0% (GAIN); standby-model energy cut 0.3% to 1.0% (GAIN); mean response cut 38% to 48% (GAIN); p95 cut 51% to 58% (GAIN); p99 cut 21% to 60% (GAIN); time over the line cut 28% to 34% (GAIN); failed requests cut 11% to 18% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 14% to 22% (GAIN); replicas up 10% to 20% (power given); pods started up 12% to 22% (power given)	`results/live/V21_FAULTS.md`
2026-10-31	Real Kubernetes, demand that wanders	omni-v21	**GAIN, power given**	**34% GAIN**	mean response cut 40% to 45% (GAIN); p95 cut 56% to 59% (GAIN); p99 cut 11% to 35% (GAIN); time over the line cut 35% to 40% (GAIN); failed requests cut 12% to 15% (GAIN); alert minutes (whole minutes whose 95th	`results/live/V21_WANDERING.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
					percentile, failed requests counted over, was over the line) cut 22% to 29% (GAIN); pod start wait, mean (s) cut 0.5% to 9.9% (GAIN); replicas up 0.1% to 2.9% (power given)	
2026-10-31	Real Kubernetes, all four at once	omni-v21	**GAIN AND LOSS**	**34% GAIN**	work inside the line up 30% to 52% (GAIN); mean response cut 36% to 43% (GAIN); p95 cut 51% to 59% (GAIN); time over the line cut 32% to 37% (GAIN); failed requests cut 5.1% to 5.4% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 16% to 24% (GAIN); pod start wait, total (s) up 16% to 42% (LOSS); pods started up 8% to 37% (power given)	`results/live/V21_ALL_FOUR.md`
2026-10-31	Real Kubernetes, a service rightsized by Goldilocks and the VPA's recommender, Omni-Compass on top, ten pairs, three runs	omni-v21	**GAIN AND LOSS**	**35% GAIN**	parked-worker energy up 1.5% to 1.8% (power given); standby-model energy up 1.5% to 1.8% (power given); mean response cut 45% to 47% (GAIN); p95 cut 59% to 61% (GAIN); p99 cut 62% to 65% (GAIN); time over the line cut 96% to 100% (GAIN); replicas up 20% to	`results/live/V21_GOLDILOCKS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
					29% (power given); pods started up 44% to 103% (power given); pod start wait, total (s) up 75% to 137% (LOSS); pod start wait, mean (s) up 11% to 32% (LOSS)	
2026-10-31	Real Kubernetes, a service rightsized by CruiseKube applying its own recommendations, Omni-Compass on top, ten pairs, three runs	omni-v21	**LOSS**	**0.4% LOSS**	mean response up 0.9% to 2.4% (LOSS); p95 up 0.0% to 1.8% (LOSS); time over the line up 4.3% (LOSS)	`results/live/V21_CRUISEKUBE.md`
2026-10-31	Real Kubernetes, a queue of jobs	omni-v21	**GAIN AND LOSS**	**9.9% GAIN**	machines cut 13% to 15% (GAIN); machine-hours cut 13% to 15% (GAIN); standby-model energy cut 9% to 10% (GAIN); mean response cut 12% to 13% (GAIN); p95 cut 0.4% to 5.7% (GAIN); time over the line cut 0.2% to 4.8% (GAIN); machines after the queue cut 19% to 22% (GAIN); replicas up 4% to 16% (power given)	`results/live/V21_BATCH.md`
2026-10-31	Real Kubernetes, a public day of demand (Google 2011)	omni-v21	**GAIN AND LOSS**	**38% GAIN**	mean response cut 50% (GAIN); p95 cut 59% to 64% (GAIN); p99 cut 48% to 56% (GAIN); time over the line cut 75% to 80% (GAIN); failed requests cut 10% to 18% (GAIN); alert minutes (whole minutes whose 95th percentile,	`results/live/V21_TRACE_GOOGLE2011.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
					failed requests counted over, was over the line) cut 61% to 72% (GAIN); pod start wait, total (s) up 88% to 279% (LOSS); replicas up 2% to 19% (power given); pods started up 79% to 235% (power given)	
2026-10-31	Real Kubernetes, a noisy neighbour	omni-v21	**GAIN AND LOSS**	**2.2% GAIN**	mean response cut 12% to 17% (GAIN); p95 cut 3% to 12% (GAIN); time over the line cut 9% to 11% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 1.2% to 6.2% (GAIN); pods started cut 9% to 28% (GAIN); second app: response time (ms), 99th percentile cut 1% to 16% (GAIN); pod start wait, mean (s) up 5% to 51% (LOSS); second app: failed requests (%) up 1.1% to 3.7% (LOSS); replicas up 0.1% to 2.9% (power given)	`results/live/V21_FAIRNESS.md`
2026-10-31	Real Kubernetes, With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass on top, ten pairs, three runs	omni-v21	**GAIN, power given**	**95% GAIN**	parked-worker energy up 1.9% to 2.4% (power given); standby-model energy up 1.9% to 2.4% (power given); mean response cut 57% to 58% (GAIN); p95 cut 73% to	`results/live/V21_KEDA_REQUESTS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
					74% (GAIN); p99 cut 60% to 66% (GAIN); time over the line cut 95% to 97% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 94% to 100% (GAIN); replicas cut 15% to 21% (GAIN); pods started cut 21% to 25% (GAIN); pod start wait, total (s) cut 22% to 26% (GAIN); pod start wait, mean (s) cut 1.3% to 4.8% (GAIN)	
2026-10-31	Real Kubernetes, With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs	omni-v21	**GAIN AND LOSS**	**30% GAIN**	parked-worker energy up 1.4% to 1.7% (power given); standby-model energy up 1.4% to 1.7% (power given); mean response cut 37% to 38% (GAIN); p95 cut 51% to 52% (GAIN); p99 cut 54% to 58% (GAIN); time over the line cut 91% to 100% (GAIN); replicas up 11% to 16% (power given); pods started up 31% to 49% (power given); pod start wait, total (s) up 27% to 84% (LOSS)	`results/live/V21_KEDA_CPU_REQUESTS.md`
2026-10-31	Power grids (SimBench), both load models	omni-v21	**GAIN, power given**	**5.8% GAI**	energy the loads drew cut 1.3% to 1.5% (GAIN); energy dissipated in lines	`results/live/V21_PANDAPOWER.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
				N	and transformers cut 0.5% to 2.4% (GAIN); net import from the upstream grid cut 1% to 15% (GAIN); tap operations cut 16% to 38% (GAIN); energy dissipated in lines and transformers up 0.6% to 1.5% (power given); tap operations up 100% (power given)	
2026-10-31	PostgreSQL, the pooler's pool size	omni-v21	**GAIN, power given**	**4.2% GAIN**	connections held open cut 1.2% to 7.5% (GAIN); host CPU busy up 0.4% to 1.3% (power given); host CPU-seconds up 0.4% to 1.4% (power given); CPU per 1,000 inside the line up 0.5% to 1.3% (power given); connections most at once up 0.0% to 8.3% (power given); p95 cut 0% to 64% (GAIN); mean cut 0% to 43% (GAIN); CPU per 1,000 inside the line cut 0.3% to 1.6% (GAIN)	`results/live/V21_PGBENCH.md`
2026-10-31	MySQL, the buffer pool's size	omni-v21	**GAIN, power given**	**2.2% GAIN**	p95 cut 1.8% to 3.5% (GAIN); p99 cut 0.6% to 4.1% (GAIN); pool held cut 4% to 22% (GAIN); pages holding data cut 1% to 21% (GAIN); host CPU busy up 0.2% to 2.6% (power given); host CPU-seconds up 0.0% to 3.0% (power given); CPU per 1,000	`results/live/V21_SYSBENCH.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
					inside the line up 0.3% to 3.7% (power given); pages holding data up 0.4% to 6.0% (power given); host CPU busy cut 2% to 38% (GAIN); host CPU-seconds cut 2% to 38% (GAIN)	
2026-10-31	MongoDB, the storage engine's cache size	omni-v21	**GAIN AND LOSS**	**0.5% LOSS**	host CPU busy cut 0.2% to 3.8% (GAIN); host CPU-seconds cut 0.2% to 4.7% (GAIN); CPU per 1,000 inside the line cut 0.2% to 4.7% (GAIN); process_cpu_seconds up 0.2% to 2.0% (power given); mean cut 0.4% to 4.0% (GAIN); p95 up 0.2% to 5.4% (LOSS); p99 up 0.4% to 2.5% (LOSS); cache size held up 0.4% to 7.5% (power given); cache in use up 0.3% to 7.2% (power given)	`results/live/V21_YCSB.md`
2026-10-31	Kafka, the consumer group's size	omni-v21	**GAIN AND LOSS**	**1.2% LOSS**	p99 cut 0.1% to 3.0% (GAIN); median cut 0.0% to 4.3% (GAIN); mean cut 0.1% to 5.1% (GAIN); messages waiting, mean cut 0.4% to 6.0% (GAIN); host CPU busy cut 1.2% to 2.3% (GAIN); host CPU-seconds cut 1.3% to 2.3% (GAIN); CPU per 1,000 inside the line cut 2.0% to 2.5% (GAIN); mean up 0% to	`results/live/V21_KAFKA.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Benchmark	Engine	Benefit?	How much	Every change judged across our three runs, in words	Table
					12% (LOSS); messages waiting, mean up 0% to 11% (LOSS); consumers running up 0% to 17% (power given); consumers most at once up 0% to 83% (power given); p99 up 0.6% to 9.1% (LOSS)	
2026-10-31	Drone swarms, the three untouched cells	omni-v21	**GAIN**	**3.0% GAIN**	energy per mission cut 13% to 17% (GAIN); fleet energy over the window cut 13% to 17% (GAIN); missions per charge at the autopilot's reserve up 14% to 21% (GAIN)	`results/live/V21_SWARM.md`
2026-10-31	Buildings and batteries (CityLearn), the districts with batteries	omni-v21	**GAIN**	**1.5% GAIN**	electricity bought cut 3% to 14% (GAIN); daily peak draw cut 3% to 17% (GAIN); highest peak cut 2.7% to 4.4% (GAIN); ramping cut 3% to 10% (GAIN); daily load unevenness cut 0.9% to 6.0% (GAIN); monthly load unevenness cut 0.8% to 2.3% (GAIN); electricity bill cut 16% (GAIN); carbon cut 12% (GAIN); distance from zero net energy cut 1.0% (GAIN)	`results/live/V21_CITYLEARN.md`

54.1 The parts behind a row

Where a row sums several workloads, grids, districts, cells or robots, each part with its own number, the same way round:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The six organisms with the real cluster inside, 10 and 100 copies**: compute_ai_cloud at 10 copies 14% GAIN (GAIN: p95 cut 50% (GAIN); time over the line cut 32% (GAIN)); physics_robotics_autonomous at 10 copies 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs); energy_facility_industrial at 10 copies 5.0% GAIN (GAIN, power given: time over the line cut 33% (GAIN); replicas up 0.6% (power given)); distribution_specialized at 10 copies 16% GAIN (GAIN: p95 cut 52% (GAIN); time over the line cut 36% (GAIN)); tower at 10 copies 11% GAIN (GAIN: p95 cut 42% (GAIN); time over the line cut 20% (GAIN); failed requests cut 8.4% (GAIN)); stack at 10 copies 2.8% GAIN (GAIN: time over the line cut 12% (GAIN); failed requests cut 8.5% (GAIN)); compute_ai_cloud at 100 copies 1.3% GAIN (GAIN: time over the line cut 9.8% (GAIN)); physics_robotics_autonomous at 100 copies 1.1% GAIN (GAIN: time over the line cut 8.4% (GAIN)); energy_facility_industrial at 100 copies 2.1% GAIN (GAIN: time over the line cut 15% (GAIN); parked-worker energy cut 0.3% (GAIN); standby-model energy cut 0.3% (GAIN)); distribution_specialized at 100 copies 8.2% GAIN (GAIN: p95 cut 32% (GAIN); time over the line cut 22% (GAIN)); tower at 100 copies 1.3% GAIN (GAIN: time over the line cut 8.7% (GAIN)); stack at 100 copies 2.3% GAIN (GAIN: time over the line cut 15% (GAIN)).
- **The big organisms with the real cluster inside, 1,000 copies on Azure**: tower at 1000 copies 159% GAIN (GAIN: p95 cut 98% (GAIN); p99 cut 98% (GAIN)); stack at 1000 copies 151% GAIN (GAIN: p95 cut 95% (GAIN); p99 cut 97% (GAIN)).
- **Robot arms (MuJoCo), the three untouched robots**: kinova_gen3 12% GAIN (GAIN: energy per takt cut 0.8% (GAIN); copper heating per cycle cut 11% (GAIN); mechanical work per cycle cut 6.6% (GAIN); peak joint torque cut 29% (GAIN); tracking error, RMS cut 21% (GAIN); end-point error at the waypoints cut 11% (GAIN)); kuka_iiwa_14 (left native by the engine's own trial) 0%: STEADY (STEADY: STEADY: nothing moved); universal_robots_ur5e (left native by the engine's own trial) 0%: STEADY (STEADY: STEADY: nothing moved).
- **Redis, the cache's memory ceiling**: burst 0.5% GAIN (GAIN: memory ceiling held cut 1.3% to 2.4% (GAIN); memory used cut 0.1% to 1.0% (GAIN); host CPU busy cut 2% to 11% (GAIN); host CPU-seconds cut 2% to 12% (GAIN); CPU per 1,000 inside the line cut 2% to 12% (GAIN)); large 0%: STEADY (GAIN, power given: work inside the line up 0.7% to 1.7% (GAIN); hit rate up 0.7% to 1.7% (GAIN); mean cut 2.7% to 7.8% (GAIN); memory ceiling held up 0.8% to 1.7% (power given); memory used up 0.5% to 1.0% (power given)); small 0.2% LOSS (GAIN AND LOSS: work inside the line up 0.6% to 1.3% (GAIN); hit rate up 0.7% to 1.3% (GAIN); mean cut 0.6% to 1.7% (GAIN); p99 up 0.2% to 1.7% (LOSS); memory ceiling held up 0.8% to 2.2% (power given); memory used up 1.1% to 1.9% (power given); host CPU busy up 1% to 11% (power given); host CPU-seconds up 1% to 12% (power given); CPU per 1,000 inside the line up 0% to 11% (power given); process_cpu_seconds up 0.3% to 2.4% (power given)).
- **Power grids (SimBench), both load models**: 1-MV-comm--0-sw (ZIP loads) 9.7% GAIN (GAIN: energy the loads drew cut 1.5% (GAIN); energy dissipated in lines and transformers cut 1.0% (GAIN); net import from the upstream grid cut 2.9% (GAIN); tap operations cut 34% (GAIN)); 1-MV-comm--1-sw (ZIP loads) 13% GAIN (GAIN: energy the loads drew cut 1.4% (GAIN); energy

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

dissipated in lines and transformers cut 0.8% (GAIN); net import from the upstream grid cut 15% (GAIN); tap operations cut 36% (GAIN)); 1-MV-comm--2-sw (ZIP loads) 11% GAIN (GAIN: energy the loads drew cut 1.4% (GAIN); net import from the upstream grid cut 6.4% (GAIN); tap operations cut 36% (GAIN)); 1-MV-rural--1-sw (ZIP loads) 13% power given (GAIN, power given: energy the loads drew cut 1.4% (GAIN); net import from the upstream grid cut 1.4% (GAIN); energy dissipated in lines and transformers up 0.8% (power given); tap operations up 100% (power given)); 1-MV-rural--2-sw (ZIP loads) 6.8% GAIN (GAIN, power given: energy the loads drew cut 1.3% (GAIN); net import from the upstream grid cut 1.1% (GAIN); tap operations cut 27% (GAIN); energy dissipated in lines and transformers up 1.5% (power given)); 1-MV-semiurb--0-sw (ZIP loads) 12% GAIN (GAIN: energy the loads drew cut 1.4% (GAIN); energy dissipated in lines and transformers cut 1.0% (GAIN); net import from the upstream grid cut 7.0% (GAIN); tap operations cut 38% (GAIN)); 1-MV-semiurb--1-sw (ZIP loads) 9.0% GAIN (GAIN, power given: energy the loads drew cut 1.4% (GAIN); net import from the upstream grid cut 1.1% (GAIN); tap operations cut 34% (GAIN); energy dissipated in lines and transformers up 0.7% (power given)); 1-MV-semiurb--2-sw (ZIP loads) 5.8% GAIN (GAIN, power given: energy the loads drew cut 1.4% (GAIN); net import from the upstream grid cut 0.9% (GAIN); tap operations cut 24% (GAIN); energy dissipated in lines and transformers up 1.1% (power given)); 1-MV-urban--0-sw (ZIP loads) 8.9% GAIN (GAIN: energy the loads drew cut 1.5% (GAIN); energy dissipated in lines and transformers cut 2.4% (GAIN); net import from the upstream grid cut 2.0% (GAIN); tap operations cut 31% (GAIN)); 1-MV-urban--1-sw (ZIP loads) 8.6% GAIN (GAIN: energy the loads drew cut 1.5% (GAIN); energy dissipated in lines and transformers cut 2.4% (GAIN); net import from the upstream grid cut 2.2% (GAIN); tap operations cut 30% (GAIN)); 1-MV-urban--2-sw (ZIP loads) 6.8% GAIN (GAIN: energy the loads drew cut 1.5% (GAIN); energy dissipated in lines and transformers cut 2.2% (GAIN); net import from the upstream grid cut 3.1% (GAIN); tap operations cut 23% (GAIN)); 1-MV-comm--0-sw (constant-power loads) 7.8% GAIN (GAIN: energy dissipated in lines and transformers cut 0.5% (GAIN); tap operations cut 31% (GAIN)); 1-MV-comm--1-sw (constant-power loads) 7.7% GAIN (GAIN: tap operations cut 31% (GAIN)); 1-MV-comm--2-sw (constant-power loads) 7.8% GAIN (GAIN: tap operations cut 31% (GAIN)); 1-MV-rural--1-sw (constant-power loads) 13% power given (power given: energy dissipated in lines and transformers up 0.6% (power given); tap operations up 100% (power given)); 1-MV-rural--2-sw (constant-power loads) 3.3% GAIN (GAIN, power given: tap operations cut 16% (GAIN); energy dissipated in lines and transformers up 1.2% (power given)); 1-MV-semiurb--0-sw (constant-power loads) 7.3% GAIN (GAIN: energy dissipated in lines and transformers cut 0.9% (GAIN); tap operations cut 29% (GAIN)); 1-MV-semiurb--1-sw (constant-power loads) 7.6% GAIN (GAIN, power given: tap operations cut 31% (GAIN); energy dissipated in lines and transformers up 0.6% (power given)); 1-MV-semiurb--2-sw (constant-power loads) 4.7% GAIN (GAIN, power given: tap operations cut 21% (GAIN); energy dissipated in lines and transformers up 0.9% (power given)); 1-MV-urban--0-sw (constant-power loads) 7.7% GAIN (GAIN: energy dissipated in lines and transformers cut 1.9% (GAIN); tap operations cut 30% (GAIN)); 1-MV-urban--1-sw (constant-power loads) 6.8% GAIN (GAIN: energy dissipated in lines and transformers cut 1.9% (GAIN); tap operations cut 27% (GAIN)); 1-MV-urban--2-sw (constant-power

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

loads) 5.4% GAIN (GAIN: energy dissipated in lines and transformers cut 1.8% (GAIN); tap operations cut 22% (GAIN)).

- **PostgreSQL, the pooler's pool size**: select 1.8% GAIN (GAIN, power given: connections held open cut 6.0% to 7.2% (GAIN); host CPU busy up 0.4% to 1.3% (power given); host CPU-seconds up 0.4% to 1.4% (power given); CPU per 1,000 inside the line up 0.5% to 1.3% (power given)); simple_update 0.5% GAIN (GAIN, power given: connections held open cut 1.2% to 3.2% (GAIN); connections most at once up 6.7% to 8.3% (power given)); tpcb_hot 11% GAIN (GAIN, power given: p95 cut 0% to 64% (GAIN); mean cut 0% to 43% (GAIN); connections held open cut 1.7% to 7.5% (GAIN); CPU per 1,000 inside the line cut 0.3% to 1.6% (GAIN); connections most at once up 0.0% to 6.7% (power given)).
- **MySQL, the buffer pool's size**: burst 3.9% GAIN (GAIN, power given: p95 cut 1.8% to 3.5% (GAIN); p99 cut 2.4% to 4.1% (GAIN); pool held cut 4% to 17% (GAIN); pages holding data cut 2% to 14% (GAIN); host CPU busy up 0.2% to 2.6% (power given); host CPU-seconds up 0.0% to 3.0% (power given); CPU per 1,000 inside the line up 0.3% to 3.7% (power given)); read_only 5.0% GAIN (GAIN, power given: p99 cut 0.6% to 2.4% (GAIN); pool held cut 10% to 22% (GAIN); pages holding data cut 8% to 21% (GAIN); host CPU busy up 0.5% to 2.0% (power given); host CPU-seconds up 0.5% to 2.4% (power given); CPU per 1,000 inside the line up 0.8% to 2.4% (power given)); read_write 0%: STEADY (GAIN, power given: p99 cut 0.6% to 1.4% (GAIN); pages holding data up 0.4% to 6.0% (power given)); update_index 0%: STEADY (GAIN: pages holding data cut 0.6% to 0.8% (GAIN); host CPU busy cut 2% to 38% (GAIN); host CPU-seconds cut 2% to 38% (GAIN)).
- **MongoDB, the storage engine's cache size**: b 0%: STEADY (GAIN, power given: host CPU busy cut 1.6% to 2.5% (GAIN); host CPU-seconds cut 1.7% to 3.2% (GAIN); CPU per 1,000 inside the line cut 1.8% to 3.2% (GAIN); process_cpu_seconds up 0.3% to 2.0% (power given)); burst 0.8% LOSS (GAIN AND LOSS: mean cut 0.4% to 4.0% (GAIN); p95 up 0.2% to 5.4% (LOSS); p99 up 0.4% to 2.5% (LOSS); cache size held up 0.4% to 2.9% (power given)); c 0%: STEADY (GAIN, power given: host CPU busy cut 0.2% to 3.8% (GAIN); host CPU-seconds cut 0.2% to 4.7% (GAIN); CPU per 1,000 inside the line cut 0.2% to 4.7% (GAIN); cache in use up 0.3% to 3.4% (power given); process_cpu_seconds up 0.2% to 1.1% (power given)); f 1.3% power given (power given: cache size held up 2.6% to 7.5% (power given); cache in use up 2.8% to 7.2% (power given)).
- **Kafka, the consumer group's size**: burst 0%: STEADY (GAIN: p99 cut 0.1% to 3.0% (GAIN); median cut 0.0% to 4.3% (GAIN); mean cut 0.1% to 5.1% (GAIN); messages waiting, mean cut 0.4% to 6.0% (GAIN); host CPU busy cut 1.2% to 2.3% (GAIN); host CPU-seconds cut 1.3% to 2.3% (GAIN); CPU per 1,000 inside the line cut 2.0% to 2.5% (GAIN)); heavy 1.2% LOSS (LOSS: mean up 0.4% to 3.0% (LOSS); messages waiting, mean up 0.1% to 3.9% (LOSS); consumers running up 0% to 12% (power given); consumers most at once up 0% to 50% (power given)); light 2.4% LOSS (LOSS: p99 up 0.6% to 9.1% (LOSS); mean up 0% to 12% (LOSS); messages waiting, mean up 0% to 11% (LOSS); consumers running up 0% to 17% (power given); consumers most at once up 0% to 83% (power given)).
- **Drone swarms, the three untouched cells**: tuning 3.5% GAIN (GAIN: energy per mission cut 13% (GAIN); fleet energy over the window cut 13% (GAIN); missions per charge at the autopilot's reserve up 14% to 15% (GAIN)); long 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

our three runs); long-316 4.8% GAIN (GAIN: energy per mission cut 17% (GAIN); fleet energy over the window cut 17% (GAIN); missions per charge at the autopilot's reserve up 21% (GAIN)); mixed 7.9% GAIN (GAIN: energy per mission cut 16% (GAIN); fleet energy over the window cut 16% (GAIN); missions per charge at the autopilot's reserve up 19% (GAIN)); mixed-315 5.3% GAIN (GAIN: energy per mission cut 14% (GAIN); fleet energy over the window cut 14% (GAIN); missions per charge at the autopilot's reserve up 17% (GAIN)); short 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs); short-314 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs).

- **Buildings and batteries (CityLearn), the districts with batteries**: ca_alameda_county_neighborhood 3.0% GAIN (GAIN: electricity bought cut 7.1% (GAIN); daily peak draw cut 6.6% (GAIN); highest peak cut 4.4% (GAIN); ramping cut 10% (GAIN); daily load unevenness cut 2.1% (GAIN); monthly load unevenness cut 1.3% (GAIN)); citylearn_challenge_2022_phase_all_robustness 9.9% GAIN (GAIN: electricity bill cut 16% (GAIN); electricity bought cut 14% (GAIN); carbon cut 12% (GAIN); daily peak draw cut 17% (GAIN); highest peak cut 2.7% (GAIN); ramping cut 9.4% (GAIN); daily load unevenness cut 6.0% (GAIN); monthly load unevenness cut 1.6% (GAIN); distance from zero net energy cut 1.0% (GAIN)); citylearn_challenge_2023_phase_2_local_evaluation 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs); citylearn_challenge_2023_phase_2_online_evaluation_1 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs); citylearn_challenge_2023_phase_2_online_evaluation_2 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs); citylearn_challenge_2023_phase_2_online_evaluation_3 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs); citylearn_challenge_2023_phase_3_1 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs); citylearn_challenge_2023_phase_3_2 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs); citylearn_challenge_2023_phase_3_3 0%: STEADY (STEADY: STEADY: nothing moved 0.5% or more across our three runs); tx_travis_county_neighborhood 2.4% GAIN (GAIN: electricity bought cut 4.0% (GAIN); daily peak draw cut 7.1% (GAIN); ramping cut 8.7% (GAIN); daily load unevenness cut 3.3% (GAIN); monthly load unevenness cut 2.3% (GAIN)); vt_chittenden_county_neighborhood 1.0% GAIN (GAIN: electricity bought cut 3.0% (GAIN); daily peak draw cut 3.2% (GAIN); ramping cut 3.4% (GAIN); daily load unevenness cut 0.9% (GAIN); monthly load unevenness cut 0.8% (GAIN)).

54.2 Not on the sheet, and why

- **Azure's managed Kubernetes, the bill** (`results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md`): Omni v1, a 4-worker fleet, every gauge STEADY, inside the noise: 0%, and too small a fleet to show one machine.
- **The governor's own share of the machine** (`results/live/V3_OWN_COST.md`): 0.6% to 1.3% of one core at every size; shown, not a benchmark against native.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The card (NVIDIA)**: at the factory dial (the max) native already runs both knobs at their top and the governor writes nothing, so the card is not on the sheet; the other tunes, each its own result: The card on omni-v23, the power cap at 70% of the card's top (cap): three runs on an A10: work per energy STEADY, under the floor (the three runs' mean 0.38%) (`results/gpu/V23_A10_CAP_ABC.md`); The card on omni-v23, the power cap at 70% of the card's top, the full load (cap-full): three runs on an A10: work per energy GAIN in all three of our runs (`results/gpu/V23_A10_CAP_FULL_ABC.md`); The card's compute run on omni-v23: an A10 at the usual load, the firmware alone against Omni-Compass on top, three runs on rented cards: work per energy STEADY, under the floor (the three runs' mean 0.13%) (`results/gpu/V23_A10_FAST_ABC.md`). Every rental, its cost and what came back is in `docs/RUN_LEDGER.md`.
- **The index's second reading** (service alone, `results/OMNI_INDEX.md`): a second true number for the same tests, not repeated here; this sheet carries the preregistered one.

CHAPTER 55

Every Level on One Page

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

How to read this receipt. Every judged row reads one of three words, decided by what its gauge is for and never by the plus or minus sign of its number. **GAIN**: the gauge moved toward its declared aim against native (more work done, a shorter wait, fewer misses or failures, or a resource cut). **STEADY**: native and Omni-Compass within 0.5% of each other (the mean of the three runs' changes under the 0.5% floor, or no change at all), or the knob is at its limit on native. **LOSS**: the gauge moved away from its aim (less work done, a longer wait, more misses or failures). A resource spent or held (energy, the power drawn and its peaks and swings, machines, memory, processor time, a bill, wear) that fell is a cut, a GAIN; one that rose is **power given**, what Omni-Compass put in to buy the work and the speed shown beside it, always shown and never called a loss; work bought with each unit of a resource (work per energy) that fell is power given too. So a minus can read GAIN (a response time cut) and a plus can read LOSS (failed requests up). A change of 0.5% or more, the mean of the three runs' changes, reads GAIN, LOSS or power given: **in all three of our runs** where A, B and C moved the same way with every interval clear of zero, otherwise **by the mean of our three runs**, with the count of runs beyond the noise beside it; **the runs disagree** names runs that moved 0.5% or more in opposite directions, a test unstable there; **shown, not judged** is a gauge declared shown only. The output follows the input: every reading is the result of the tune, the wiring and the load it names, and where a reading elsewhere differs the first presumption is the wiring (`DISCLOSURES.md`, section 4). The full key, letter by letter: `docs/HOW_TO_READ_THE_RESULTS.md`. Run and scored by The Omni-Compass LLC's

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

The founder's order of 17 October: Omni-Compass is shown on top at every level, on every gauge, and every level's gauges should go the same way; where one does not, it is named here and taken to the founder. Each row is built from its result files by `tools/levels.py`; nothing is typed in. A count is of judged gauges (a table's rows that read a word), each under its own table's rule (the three runs, or the realms result's ten seeds); **power given** is a resource put in, shown and never called a loss. Every row names the engine its tables ran on, and no reading is carried from one engine to another.

Level	Engine	Evidence	GAIN	STEADY	power given	LOSS	runs disagree	Goes the other way?
The muscle, alone (1,107 muscles, each on its own native controller, under its own load)	omni-v8	S (our model)	530	577	0	0	0	no: every judged gauge GAIN or STEADY
The organism, the stack: the seven organisms under three loads (own load, the climb, work waiting in line)	omni-v8	S (our model)	21	0	0	0	0	no: every judged gauge GAIN or STEADY
The cluster: real Kubernetes, native its own HPA and scheduler	omni-v21	L	45	40	11	11	22	**yes: 33, listed below**
The cluster with its add-ons native: KEDA, Goldilocks, CruiseKube	omni-v21	L	20	21	10	6	7	**yes: 13, listed below**
A muscle inside a running service: database, broker, cache	omni-v21	L	72	138	50	25	87	**yes: 112, listed below**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Level	Engine	Evidence	GAIN	STEADY	power given	LOSS	runs disagree	Goes the other way?
A physical plant in an independent simulator: buildings, power grids, robot arms, drone swarms	omni-v21	independent simulator	101	165	10	0	0	no LOSS; resources put in shown as power given
The organism with the real cluster inside (the grid, at 1, 10, 100 and on GitHub's machines 1,000 copies)	older engines' tables; the grid runs again on the engine that stands	L, the cluster; S, the organism	–	–	–	–	–	no table on the engine that stands yet; the grid's runs (results/grid_v8/) are going
The cluster in a cloud: Azure Kubernetes Service, the bill	single runs, omni-v5 to omni-v7	L, the bill metered	–	–	–	–	–	no three-run table yet: eight single runs at the tune-down, machines billed −6.5% to +7.6%, every one STEADY; the fleet (about 1.9 of 4 machines busy) leaves no whole machine to give back
A group of clusters	not run	none yet	–	–	–	–	–	no run at this level yet, real or modelled, on the engine that stands (`k8s_fleet/` holds a model of an earlier engine)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

55.1 Each level

55.1.1 The muscle, alone (1,107 muscles, each on its own native controller, under its own load)

Native at this level: each muscle's native controller. Engine: omni-v8. Evidence: S (our model). 421 of the STEADY are at their limit on native (nothing to push). Tables: `results/realms_v5/MUSCLES_V5.csv`.

55.1.2 The organism, the stack: the seven organisms under three loads (own load, the climb, work waiting in line)

Native at this level: every muscle on its native controller, coupled in one body. Engine: omni-v8. Evidence: S (our model). The organism's own reading; inside them no plant did less work or missed its line more often than native, under any load (the realms result's stricter look, plant by plant). Tables: `results/realms_v5/REALMS_V5.md`.

55.1.3 The cluster: real Kubernetes, native its own HPA and scheduler

Native at this level: Kubernetes as shipped with the operator's HPA target. Engine: omni-v21. Evidence: L. Tables: `results/live/V21_ALL_FOUR.md`, `results/live/V21_BATCH.md`, `results/live/V21_CAPACITY.md`, `results/live/V21_FAIRNESS.md`, `results/live/V21_FAULTS.md`, `results/live/V21_TRACE_GOOGLE2011.md`, `results/live/V21_ROBUST_KILL.md`, `results/live/V21_ROBUST_LONG.md`.

Goes the other way:

- V21_ALL_FOUR.md: All four in one run: more work, faster, fewer machines, less energy (load rising 1 to 8 and back): the A/B/C three-run table (Omni v21): response time (ms), 99th percentile (the runs disagree)
- V21_ALL_FOUR.md: All four in one run: more work, faster, fewer machines, less energy (load rising 1 to 8 and back): the A/B/C three-run table (Omni v21): HPA replicas, mean (the runs disagree)
- V21_ALL_FOUR.md: All four in one run: more work, faster, fewer machines, less energy (load rising 1 to 8 and back): the A/B/C three-run table (Omni v21): pod start wait, total (s) (LOSS)
- V21_ALL_FOUR.md: All four in one run: more work, faster, fewer machines, less energy (load rising 1 to 8 and back): the A/B/C three-run table (Omni v21): pod start wait, mean (s) (the runs disagree)
- V21_BATCH.md: Batch queue: 240 CPU jobs and cruise: the A/B/C three-run table (Omni v21): failed requests (%) (LOSS)
- V21_BATCH.md: Batch queue: 240 CPU jobs and cruise: the A/B/C three-run table (Omni v21): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) (the runs disagree)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_BATCH.md: Batch queue: 240 CPU jobs and cruise: the A/B/C three-run table (Omni v21): pod start wait, total (s) (LOSS)
- V21_BATCH.md: Batch queue: 240 CPU jobs and cruise: the A/B/C three-run table (Omni v21): pod start wait, mean (s) (LOSS)
- V21_CAPACITY.md: The capacity test, native and Omni-Compass on top at the same resource cap: the A/B/C three-run table (Omni v21): 1 (the runs disagree)
- V21_CAPACITY.md: The capacity test, native and Omni-Compass on top at the same resource cap: the A/B/C three-run table (Omni v21): 1 (the runs disagree)
- V21_CAPACITY.md: The capacity test, native and Omni-Compass on top at the same resource cap: the A/B/C three-run table (Omni v21): 1.5 (the runs disagree)
- V21_CAPACITY.md: The capacity test, native and Omni-Compass on top at the same resource cap: the A/B/C three-run table (Omni v21): 1.5 (the runs disagree)
- V21_CAPACITY.md: The capacity test, native and Omni-Compass on top at the same resource cap: the A/B/C three-run table (Omni v21): 2 (the runs disagree)
- V21_CAPACITY.md: The capacity test, native and Omni-Compass on top at the same resource cap: the A/B/C three-run table (Omni v21): 3 (LOSS)
- V21_CAPACITY.md: The capacity test, native and Omni-Compass on top at the same resource cap: the A/B/C three-run table (Omni v21): 3 (the runs disagree)
- V21_FAIRNESS.md: Fairness: a noisy neighbour on the same workers: the A/B/C three-run table (Omni v21): response time (ms), 99th percentile (the runs disagree)
- V21_FAIRNESS.md: Fairness: a noisy neighbour on the same workers: the A/B/C three-run table (Omni v21): failed requests (%) (the runs disagree)
- V21_FAIRNESS.md: Fairness: a noisy neighbour on the same workers: the A/B/C three-run table (Omni v21): pod start wait, total (s) (the runs disagree)
- V21_FAIRNESS.md: Fairness: a noisy neighbour on the same workers: the A/B/C three-run table (Omni v21): pod start wait, mean (s) (LOSS)
- V21_FAIRNESS.md: Fairness: a noisy neighbour on the same workers: the A/B/C three-run table (Omni v21): second app: response time (ms), 95th percentile (the runs disagree)
- V21_FAIRNESS.md: Fairness: a noisy neighbour on the same workers: the A/B/C three-run table (Omni v21): second app: time over the response line (% of samples) (the runs disagree)
- V21_FAIRNESS.md: Fairness: a noisy neighbour on the same workers: the A/B/C three-run table (Omni v21): second app: failed requests (%) (LOSS)
- V21_FAULTS.md: Faults: a machine down, a spike, a runaway pod, a blind probe: the A/B/C three-run table (Omni v21): pod start wait, total (s) (the runs disagree)
- V21_FAULTS.md: Faults: a machine down, a spike, a runaway pod, a blind probe: the A/B/C three-run table (Omni v21): pod start wait, mean (s) (the runs disagree)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_TRACE_GOOGLE2011.md: A public day of demand: the Google cluster trace of 2011 replayed one step at a time (2 3 3 3 3 4 4 3 2 2 2 2 2 3 2 3 4 4 3 4 5 6 5 4): the A/B/C three-run table (Omni v21): pod start wait, total (s) (LOSS)
- V21_TRACE_GOOGLE2011.md: A public day of demand: the Google cluster trace of 2011 replayed one step at a time (2 3 3 3 3 4 4 3 2 2 2 2 2 3 2 3 4 4 3 4 5 6 5 4): the A/B/C three-run table (Omni v21): pod start wait, mean (s) (the runs disagree)
- V21_ROBUST_KILL.md: Robustness: the governor killed outright mid-run (the lease), real Kubernetes, 10 pairs: the A/B/C three-run table (Omni v21): response time (ms), 99th percentile (the runs disagree)
- V21_ROBUST_KILL.md: Robustness: the governor killed outright mid-run (the lease), real Kubernetes, 10 pairs: the A/B/C three-run table (Omni v21): pods started (the runs disagree)
- V21_ROBUST_KILL.md: Robustness: the governor killed outright mid-run (the lease), real Kubernetes, 10 pairs: the A/B/C three-run table (Omni v21): pod start wait, total (s) (LOSS)
- V21_ROBUST_KILL.md: Robustness: the governor killed outright mid-run (the lease), real Kubernetes, 10 pairs: the A/B/C three-run table (Omni v21): pod start wait, mean (s) (the runs disagree)
- V21_ROBUST_LONG.md: Robustness: the long run, 7,200 s an arm, the governor's memory and decisions watched: the A/B/C three-run table (Omni v21): response time (ms), 99th percentile (the runs disagree)
- V21_ROBUST_LONG.md: Robustness: the long run, 7,200 s an arm, the governor's memory and decisions watched: the A/B/C three-run table (Omni v21): pod start wait, total (s) (LOSS)
- V21_ROBUST_LONG.md: Robustness: the long run, 7,200 s an arm, the governor's memory and decisions watched: the A/B/C three-run table (Omni v21): pod start wait, mean (s) (LOSS)

55.1.4 The cluster with its add-ons native: KEDA, Goldilocks, CruiseKube

Native at this level: Kubernetes with the add-on as shipped and as used, its own recommendations applied. Engine: omni-v21. Evidence: L. Tables: `results/live/V21_KEDA_REQUESTS.md`, `results/live/V21_KEDA_CPU_REQUESTS.md`, `results/live/V21_GOLDILOCKS.md`, `results/live/V21_CRUISEKUBE.md`.

Goes the other way:

- V21_KEDA_CPU_REQUESTS.md: Kubernetes with KEDA on the CPU target and the live requests in flight: Omni-Compass on top: the A/B/C three-run table (Omni v21): failed requests (%) (the runs disagree)
- V21_KEDA_CPU_REQUESTS.md: Kubernetes with KEDA on the CPU target and the live requests in flight: Omni-Compass on top: the A/B/C three-run table (Omni v21): pod start wait, total (s) (LOSS)
- V21_KEDA_CPU_REQUESTS.md: Kubernetes with KEDA on the CPU target and the live requests in flight: Omni-Compass on top: the A/B/C three-run table (Omni v21): pod start wait, mean (s) (the runs disagree)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_GOLDILOCKS.md: Goldilocks: Omni-Compass on top of a service rightsized with Goldilocks and the VPA's recommender: the A/B/C three-run table (Omni v21): pod start wait, total (s) (LOSS)
- V21_GOLDILOCKS.md: Goldilocks: Omni-Compass on top of a service rightsized with Goldilocks and the VPA's recommender: the A/B/C three-run table (Omni v21): pod start wait, mean (s) (LOSS)
- V21_CRUISEKUBE.md: CruiseKube: Omni-Compass on top of a service rightsized by CruiseKube applying its own recommendations: the A/B/C three-run table (Omni v21): response time (ms), mean (LOSS)
- V21_CRUISEKUBE.md: CruiseKube: Omni-Compass on top of a service rightsized by CruiseKube applying its own recommendations: the A/B/C three-run table (Omni v21): response time (ms), 95th percentile (LOSS)
- V21_CRUISEKUBE.md: CruiseKube: Omni-Compass on top of a service rightsized by CruiseKube applying its own recommendations: the A/B/C three-run table (Omni v21): response time (ms), 99th percentile (the runs disagree)
- V21_CRUISEKUBE.md: CruiseKube: Omni-Compass on top of a service rightsized by CruiseKube applying its own recommendations: the A/B/C three-run table (Omni v21): time over the response line (% of samples) (LOSS)
- V21_CRUISEKUBE.md: CruiseKube: Omni-Compass on top of a service rightsized by CruiseKube applying its own recommendations: the A/B/C three-run table (Omni v21): HPA replicas, mean (the runs disagree)
- V21_CRUISEKUBE.md: CruiseKube: Omni-Compass on top of a service rightsized by CruiseKube applying its own recommendations: the A/B/C three-run table (Omni v21): pods started (the runs disagree)
- V21_CRUISEKUBE.md: CruiseKube: Omni-Compass on top of a service rightsized by CruiseKube applying its own recommendations: the A/B/C three-run table (Omni v21): pod start wait, total (s) (the runs disagree)
- V21_CRUISEKUBE.md: CruiseKube: Omni-Compass on top of a service rightsized by CruiseKube applying its own recommendations: the A/B/C three-run table (Omni v21): pod start wait, mean (s) (the runs disagree)

55.1.5 A muscle inside a running service: database, broker, cache

Native at this level: the server as shipped but for the one setting an operator sets (each table names it). Engine: omni-v21. Evidence: L. Tables: `results/live/V21_REDIS.md`, `results/live/V21_REDIS_SERVICE.md`, `results/live/V21_KAFKA.md`, `results/live/V21_KAFKA_SERVICE.md`, `results/live/V21_PGBENCH.md`, `results/live/V21_SYSBENCH.md`, `results/live/V21_YCSB.md`.

Goes the other way:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_REDIS.md: tuning: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): host CPU busy (share of the run) (the runs disagree)
- V21_REDIS.md: tuning: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): host CPU-seconds (the runs disagree)
- V21_REDIS.md: tuning: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): host CPU-seconds per 1,000 requests inside the line (the runs disagree)
- V21_REDIS.md: tuning: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): the stack's own CPU-seconds (redis-server, the application, the compass and its sampler; steal and the runner's other processes excluded) (the runs disagree)
- V21_REDIS.md: large: 32768 B values, working set 625 keys × notch; line 2 ms, 3 paired repetitions a run: host CPU busy (share of the run) (the runs disagree)
- V21_REDIS.md: large: 32768 B values, working set 625 keys × notch; line 2 ms, 3 paired repetitions a run: host CPU-seconds (the runs disagree)
- V21_REDIS.md: large: 32768 B values, working set 625 keys × notch; line 2 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 requests inside the line (the runs disagree)
- V21_REDIS.md: small: 2048 B values, working set 10,000 keys × notch; line 2 ms, 3 paired repetitions a run: latency, 99th percentile (ms) (LOSS)
- V21_REDIS_SERVICE.md: tuning: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): host CPU busy (share of the run) (the runs disagree)
- V21_REDIS_SERVICE.md: tuning: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): host CPU-seconds (the runs disagree)
- V21_REDIS_SERVICE.md: tuning: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): host CPU-seconds per 1,000 requests inside the line (the runs disagree)
- V21_REDIS_SERVICE.md: burst: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run: host CPU busy (share of the run) (the runs disagree)
- V21_REDIS_SERVICE.md: burst: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run: host CPU-seconds (the runs disagree)
- V21_REDIS_SERVICE.md: burst: 8192 B values, working set 2,500 keys × notch; line 2 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 requests inside the line (the runs disagree)
- V21_REDIS_SERVICE.md: large: 32768 B values, working set 625 keys × notch; line 2 ms, 3 paired repetitions a run: memory ceiling held, mean (MB; the knob, the resource) (the runs disagree)
- V21_REDIS_SERVICE.md: large: 32768 B values, working set 625 keys × notch; line 2 ms, 3 paired repetitions a run: memory used, mean (MB) (the runs disagree)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_REDIS_SERVICE.md: small: 2048 B values, working set 10,000 keys × notch; line 2 ms, 3 paired repetitions a run: latency, 99th percentile (ms) (LOSS)
- V21_REDIS_SERVICE.md: small: 2048 B values, working set 10,000 keys × notch; line 2 ms, 3 paired repetitions a run: host CPU busy (share of the run) (the runs disagree)
- V21_REDIS_SERVICE.md: small: 2048 B values, working set 10,000 keys × notch; line 2 ms, 3 paired repetitions a run: host CPU-seconds (the runs disagree)
- V21_REDIS_SERVICE.md: small: 2048 B values, working set 10,000 keys × notch; line 2 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 requests inside the line (the runs disagree)
- V21_KAFKA.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): end-to-end latency, 95th percentile (ms) (LOSS)
- V21_KAFKA.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): end-to-end latency, 99th percentile (ms) (the runs disagree)
- V21_KAFKA.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): end-to-end latency, mean (ms) (LOSS)
- V21_KAFKA.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): consumer lag, most messages waiting at once (LOSS)
- V21_KAFKA.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): consumer lag, mean messages waiting (LOSS)
- V21_KAFKA.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): host CPU busy (share of the run) (the runs disagree)
- V21_KAFKA.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): host CPU-seconds (the runs disagree)
- V21_KAFKA.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): host CPU-seconds per 1,000 messages inside the line (the runs disagree)
- V21_KAFKA.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 95th percentile (ms) (the runs disagree)
- V21_KAFKA.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: consumer lag, most messages waiting at once (the runs disagree)
- V21_KAFKA.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 95th percentile (ms) (the runs disagree)
- V21_KAFKA.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 99th percentile (ms) (the runs disagree)
- V21_KAFKA.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, mean (ms) (LOSS)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_KAFKA.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: consumer lag, most messages waiting at once (the runs disagree)
- V21_KAFKA.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: consumer lag, mean messages waiting (LOSS)
- V21_KAFKA.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: host CPU busy (share of the run) (the runs disagree)
- V21_KAFKA.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: host CPU-seconds (the runs disagree)
- V21_KAFKA.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 messages inside the line (the runs disagree)
- V21_KAFKA.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 95th percentile (ms) (the runs disagree)
- V21_KAFKA.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 99th percentile (ms) (LOSS)
- V21_KAFKA.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, median (ms) (the runs disagree)
- V21_KAFKA.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, mean (ms) (LOSS)
- V21_KAFKA.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: consumer lag, most messages waiting at once (the runs disagree)
- V21_KAFKA.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: consumer lag, mean messages waiting (LOSS)
- V21_KAFKA.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 messages inside the line (the runs disagree)
- V21_KAFKA_SERVICE.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): end-to-end latency, 95th percentile (ms) (the runs disagree)
- V21_KAFKA_SERVICE.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): end-to-end latency, mean (ms) (the runs disagree)
- V21_KAFKA_SERVICE.md: tuning: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): consumer lag, mean messages waiting (the runs disagree)
- V21_KAFKA_SERVICE.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 95th percentile (ms) (the runs disagree)
- V21_KAFKA_SERVICE.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 99th percentile (ms) (the runs disagree)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_KAFKA_SERVICE.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, median (ms) (the runs disagree)
- V21_KAFKA_SERVICE.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, mean (ms) (LOSS)
- V21_KAFKA_SERVICE.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: consumer lag, most messages waiting at once (LOSS)
- V21_KAFKA_SERVICE.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: consumer lag, mean messages waiting (LOSS)
- V21_KAFKA_SERVICE.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: host CPU busy (share of the run) (the runs disagree)
- V21_KAFKA_SERVICE.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: host CPU-seconds (the runs disagree)
- V21_KAFKA_SERVICE.md: burst: 2.0 ms of work a message, 256 B; line 500 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 messages inside the line (the runs disagree)
- V21_KAFKA_SERVICE.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 95th percentile (ms) (the runs disagree)
- V21_KAFKA_SERVICE.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, mean (ms) (LOSS)
- V21_KAFKA_SERVICE.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: consumer lag, most messages waiting at once (LOSS)
- V21_KAFKA_SERVICE.md: heavy: 5.0 ms of work a message, 1024 B; line 500 ms, 3 paired repetitions a run: consumer lag, mean messages waiting (LOSS)
- V21_KAFKA_SERVICE.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: work inside the response line (messages a second handled within the line) (LOSS)
- V21_KAFKA_SERVICE.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 95th percentile (ms) (LOSS)
- V21_KAFKA_SERVICE.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, 99th percentile (ms) (the runs disagree)
- V21_KAFKA_SERVICE.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: end-to-end latency, mean (ms) (LOSS)
- V21_KAFKA_SERVICE.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: consumer lag, most messages waiting at once (the runs disagree)
- V21_KAFKA_SERVICE.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: consumer lag, mean messages waiting (LOSS)
- V21_KAFKA_SERVICE.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: host CPU busy (share of the run) (the runs disagree)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_KAFKA_SERVICE.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: host CPU-seconds (the runs disagree)
- V21_KAFKA_SERVICE.md: light: 1.0 ms of work a message, 128 B; line 500 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 messages inside the line (the runs disagree)
- V21_PGBENCH.md: select (-S, scale 20): line 50 ms, 3 paired repetitions a run: latency, 95th percentile (ms, lag included) (the runs disagree)
- V21_PGBENCH.md: select (-S, scale 20): line 50 ms, 3 paired repetitions a run: latency, 99th percentile (ms) (the runs disagree)
- V21_PGBENCH.md: select (-S, scale 20): line 50 ms, 3 paired repetitions a run: latency, mean (ms) (the runs disagree)
- V21_PGBENCH.md: simple_update (-N, scale 20): line 50 ms, 3 paired repetitions a run: latency, 95th percentile (ms, lag included) (the runs disagree)
- V21_PGBENCH.md: simple_update (-N, scale 20): line 50 ms, 3 paired repetitions a run: latency, 99th percentile (ms) (the runs disagree)
- V21_PGBENCH.md: simple_update (-N, scale 20): line 50 ms, 3 paired repetitions a run: latency, mean (ms) (the runs disagree)
- V21_PGBENCH.md: tpcb_hot (tpcb (default), scale 2): line 50 ms, 3 paired repetitions a run: latency, 99th percentile (ms) (the runs disagree)
- V21_SYSBENCH.md: tuning: sysbench oltp_point_select, 1 statements a transaction, 3000 a second offered; the transaction line 0.6 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): buffer pool held, mean (MB; the knob, the resource) (the runs disagree)
- V21_SYSBENCH.md: tuning: sysbench oltp_point_select, 1 statements a transaction, 3000 a second offered; the transaction line 0.6 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): pages holding data, mean (MB) (the runs disagree)
- V21_SYSBENCH.md: read_only: sysbench oltp_read_only, 14 statements a transaction, 300 a second offered; the transaction line 8.4 ms, 3 paired repetitions a run: latency, 95th percentile (ms) (the runs disagree)
- V21_SYSBENCH.md: read_write: sysbench oltp_read_write, 18 statements a transaction, 200 a second offered; the transaction line 10.8 ms, 3 paired repetitions a run: latency, 95th percentile (ms) (the runs disagree)
- V21_SYSBENCH.md: read_write: sysbench oltp_read_write, 18 statements a transaction, 200 a second offered; the transaction line 10.8 ms, 3 paired repetitions a run: latency, mean (ms) (the runs disagree)
- V21_SYSBENCH.md: read_write: sysbench oltp_read_write, 18 statements a transaction, 200 a second offered; the transaction line 10.8 ms, 3 paired repetitions a run: buffer pool held, mean (MB; the knob, the resource) (the runs disagree)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_SYSBENCH.md: read_write: sysbench oltp_read_write, 18 statements a transaction, 200 a second offered; the transaction line 10.8 ms, 3 paired repetitions a run: host CPU busy (share of the run) (the runs disagree)
- V21_SYSBENCH.md: read_write: sysbench oltp_read_write, 18 statements a transaction, 200 a second offered; the transaction line 10.8 ms, 3 paired repetitions a run: host CPU-seconds (the runs disagree)
- V21_SYSBENCH.md: read_write: sysbench oltp_read_write, 18 statements a transaction, 200 a second offered; the transaction line 10.8 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 transactions inside the line (the runs disagree)
- V21_SYSBENCH.md: update_index: sysbench oltp_update_index, 1 statements a transaction, 1000 a second offered; the transaction line 0.6 ms, 3 paired repetitions a run: work inside the response line (transactions a second answered within the line) (the runs disagree)
- V21_SYSBENCH.md: update_index: sysbench oltp_update_index, 1 statements a transaction, 1000 a second offered; the transaction line 0.6 ms, 3 paired repetitions a run: latency, 95th percentile (ms) (the runs disagree)
- V21_SYSBENCH.md: update_index: sysbench oltp_update_index, 1 statements a transaction, 1000 a second offered; the transaction line 0.6 ms, 3 paired repetitions a run: latency, 99th percentile (ms) (the runs disagree)
- V21_SYSBENCH.md: update_index: sysbench oltp_update_index, 1 statements a transaction, 1000 a second offered; the transaction line 0.6 ms, 3 paired repetitions a run: latency, mean (ms) (the runs disagree)
- V21_SYSBENCH.md: update_index: sysbench oltp_update_index, 1 statements a transaction, 1000 a second offered; the transaction line 0.6 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 transactions inside the line (the runs disagree)
- V21_YCSB.md: tuning: YCSB workloada, 250,000 records × notch; line 1 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): latency, 95th percentile (ms) (LOSS)
- V21_YCSB.md: tuning: YCSB workloada, 250,000 records × notch; line 1 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): latency, 99th percentile (ms) (LOSS)
- V21_YCSB.md: tuning: YCSB workloada, 250,000 records × notch; line 1 ms, 3 paired repetitions a run (the tuning workload, shown and not counted): latency, mean (ms) (the runs disagree)
- V21_YCSB.md: b: YCSB workloadb, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: latency, 95th percentile (ms) (the runs disagree)
- V21_YCSB.md: b: YCSB workloadb, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: latency, 99th percentile (ms) (the runs disagree)
- V21_YCSB.md: b: YCSB workloadb, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: latency, mean (ms) (the runs disagree)
- V21_YCSB.md: burst: YCSB workloadb, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: latency, 95th percentile (ms) (LOSS)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- V21_YCSB.md: burst: YCSB workloadb, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: latency, 99th percentile (ms) (LOSS)
- V21_YCSB.md: burst: YCSB workloadb, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: bytes in the cache, mean (MB) (the runs disagree)
- V21_YCSB.md: burst: YCSB workloadb, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: host CPU busy (share of the run) (the runs disagree)
- V21_YCSB.md: burst: YCSB workloadb, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: host CPU-seconds (the runs disagree)
- V21_YCSB.md: burst: YCSB workloadb, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 operations inside the line (the runs disagree)
- V21_YCSB.md: c: YCSB workloadc, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: latency, 99th percentile (ms) (the runs disagree)
- V21_YCSB.md: c: YCSB workloadc, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: latency, mean (ms) (the runs disagree)
- V21_YCSB.md: c: YCSB workloadc, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: cache size held, mean (MB; the knob, the resource) (the runs disagree)
- V21_YCSB.md: f: YCSB workloadf, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: latency, 95th percentile (ms) (the runs disagree)
- V21_YCSB.md: f: YCSB workloadf, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: latency, mean (ms) (the runs disagree)
- V21_YCSB.md: f: YCSB workloadf, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: host CPU busy (share of the run) (the runs disagree)
- V21_YCSB.md: f: YCSB workloadf, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: host CPU-seconds (the runs disagree)
- V21_YCSB.md: f: YCSB workloadf, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: host CPU-seconds per 1,000 operations inside the line (the runs disagree)
- V21_YCSB.md: f: YCSB workloadf, 250,000 records × notch; line 1 ms, 3 paired repetitions a run: the store's and the harness's own CPU-seconds (mongod plus the compass and its sampler; steal excluded) (the runs disagree)

55.1.6 A physical plant in an independent simulator: buildings, power grids, robot arms, drone swarms

Native at this level: the simulator's own controller, unchanged. Engine: omni-v21. Evidence: independent simulator. Tables: `results/live/V21_CITYLEARN.md`, `results/live/V21_PANDAPOWER.md`, `results/live/V21_MUJOCO.md`, `results/live/V21_SWARM.md`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

55.1.7 The organism with the real cluster inside (the grid, at 1, 10, 100 and on GitHub's machines 1,000 copies)

Native at this level: the organism's native controllers around Kubernetes with its HPA. Engine: older engines' tables; the grid runs again on the engine that stands. Evidence: L, the cluster; S, the organism. No table on the engine that stands yet; the grid's runs (results/grid_v8/) are going. Tables: `results/live/SIX_KUBE.md`, `results/live/V3_SIX_KUBE.md`, `results/live/V3_BIG_ORGANISM.md`.

55.1.8 The cluster in a cloud: Azure Kubernetes Service, the bill

Native at this level: Azure's own cluster autoscaler and the HPA. Engine: single runs, omni-v5 to omni-v7. Evidence: L, the bill metered. No three-run table yet: eight single runs at the tune-down, machines billed −6.5% to +7.6%, every one STEADY; the fleet (about 1.9 of 4 machines busy) leaves no whole machine to give back. Tables: `results/live/AKS_BILL.md`, `results/live/raw/run-38981149348/`.

55.1.9 A group of clusters

Native at this level: each cluster's own autoscaler. Engine: not run. Evidence: none yet. No run at this level yet, real or modelled, on the engine that stands (`k8s_fleet/` holds a model of an earlier engine).

CHAPTER 56

The Dossier: Every Result in One Place

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

Every mechanism, harness, receipt and result, read from the files named beside it. Built by `tools/dossier.py` at commit `4c71e77d50`. Evidence classes: **T** theorem, **V** verified in code, **S** a model, **L** live software (real Kubernetes), **P** a physical meter. A model is not a meter, and a model written by the people who wrote the law is not an independent test; where a result is a model it says so.

56.1 The mechanism, and proof that it is the one that ran

Check	Result	Where
The whole repository re-runs and checks itself (`python3 verify.py`)	**PASS**	`results/VERIFY_RECEIPT.txt`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Check	Result	Where
The eight-line engine and its six states, fingerprinted (`omnicompass/core.py`)	sha256 `bd615f156169f679…`	`RELEASE_MANIFEST.json`
Python and C++20 twins of every law, proven equal and sealed	seal intact: 17 Python/C++ twins	`results/SEAL.json`
The mechanism's identity against the code	mechanism identity matches the code	`results/MECHANISM_IDENTITY.json`
The engine's convergence to its pole under the bounded command	proved	`docs/TRACKING_THEOREM.md`
Safety shield: 2,000,000 adversarial cases	0 violations	`tests/test_shield_properties.py`
The compass law (push and pull, 5% cushions, fail up, plug contract) on every muscle, the card and Kubernetes	one law, one file	`omnicompass/compass_law.py`
Every file the results depend on, by fingerprint	written after the check passes	`RELEASE_MANIFEST.json`

The engine is the founder's eight-line equation, integrated by RK4 with the bounded command held across all four stages; it is frozen and fingerprinted, and the C++ twin matches it. The compass law is the outer loop that moves each muscle's own setting: it reads one service position (0 calm, 1 the line), pulls it to the compass's center, pushes against whatever is rising, bounds its force by tanh, fails up past the wall, and writes through a plug that reads every lever once before the first write, reads back every write, yields to any other writer and restores the snapshot at the end.

56.2 Real Kubernetes, Omni v3, every test three times (evidence class L)

Seven tests, ten paired repetitions each, three separate GitHub runs on the frozen engine (A the result, B and C the replications): native Kubernetes (its HPA and scheduler) against the same Kubernetes with Omni-Compass on top, a fresh six-worker cluster per arm, order rotated, the same work sent to both arms. A row reads **GAIN** or **LOSS in all three of our runs** (by what the gauge is for, never by its sign: a response time cut is a GAIN; machines or energy up is **power given**) only when all three runs move the same way with every 95% interval clear of zero; otherwise it reads STEADY, no difference beyond the noise, which is the result. Every Omni arm ends with every setting handed back and read back. Steady runs: A 38013195851 (omni-v3); B 38013200499 (omni-v3); C 38013204648 (omni-v3); the other tests' runs are named in their tables.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test	Work	Response time (p95; the batch queue's mean)	Machines in service	Energy (declared model)	Failed requests	Table
Steady work in steps	STEADY, equal by design	**-66% to -64%, GAIN in all three of our runs**	**-2% to -2%, GAIN in all three of our runs**	**-2% to -1%, GAIN in all three of our runs**	STEADY, the same	`results/live/V3_STEADY.md`
Demand that wanders	not taken	**-64% to -56%, GAIN in all three of our runs**	STEADY, inside the noise	STEADY, inside the noise	**-13% to -12%, GAIN in all three of our runs**	`results/live/V3_WANDERING.md`
All four in one run	**+54% to +68%, GAIN in all three of our runs**	**-62% to -57%, GAIN in all three of our runs**	STEADY, inside the noise	STEADY, inside the noise	**-14% to -11%, GAIN in all three of our runs**	`results/live/V3_ALL_FOUR.md`
Fairness, a noisy neighbour	not taken	STEADY, inside the noise	STEADY, the same	STEADY, inside the noise	STEADY, inside the noise	`results/live/V3_FAIRNESS.md`
Faults: machine down, spike, runaway pod, blind probe	not taken	**-63% to -58%, GAIN in all three of our runs**	STEADY, inside the noise	STEADY, inside the noise	**-18% to -14%, GAIN in all three of our runs**	`results/live/V3_FAULTS.md`
A queue of batch jobs	not taken	**-13% to -11%, GAIN in all three of our runs**	**-20% to -16%, GAIN in all three of our runs**	**-14% to -11%, GAIN in all three of our runs**	STEADY, inside the noise	`results/live/V3_BATCH.md`
A public day of demand: the Google cluster trace of 2011, replayed one step at a time	not taken	**-66% to -64%, GAIN in all three of our runs**	**-9% to -7%, GAIN in all three of our runs**	**-7% to -5%, GAIN in all three of our runs**	**-18% to -11%, GAIN in all three of our runs**	`results/live/V3_TRACE_GOOGLE2011.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Energy on kind is a declared model: the machines are containers on one runner, so a machine out of service cuts modelled watts, not a metered bill. Omni-Compass gives a machine back only after a paired trial shows the service no slower without it (the verdict, `omnicompass/verdict.py`); on these clusters one machine fewer made requests 30-45% slower in most trials, so the machines stayed and were spent on speed and work. The v1 tables (`results/live/V1_*.md`, `docs/OMNI_V1.md`) read the same; the sets before v1 are in `docs/history/`.

56.3 A real database: PostgreSQL behind PgBouncer, Omni v3, three runs (evidence class L)

PostgreSQL 16 as shipped behind PgBouncer's shipped pool of 20 is native; omni is the compass law on one knob, the pool size, through PgBouncer's own console, inside the cover [2, 90] (`docs/POSTGRES_PREREGISTRATION.md`). Three paired repetitions a run, three runs, pgbench's own log for the gauges. Runs: A 38054754511; B 38054758150; C 38054761503.

Workload	Work inside the 50 ms line	p95	Connections held open	Host CPU-seconds (the compass's own cost)
`select`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise
`simple_update`	STEADY, inside the noise	STEADY, inside the noise	the runs disagree	STEADY, inside the noise
`tpcb_hot`	STEADY, inside the noise	STEADY, inside the noise	**-13% to -7%, GAIN in all three of our runs**	STEADY, inside the noise

The compass takes connections back only while the pooler's clients wait for one under 1% of the time and gives them back the moment anyone waits (amendment 2 of `docs/POSTGRES_PREREGISTRATION.md`, 8 October: the first counted set's CPU cost was measured and traced to the harness's own psql launches, not to the pooler, and its first table is kept whole in `docs/history/V3_PGBENCH_set1.md`). Where it reads a cut it is connections held open for the same work, a GAIN, with the host's CPU STEADY (`tpcb_hot`: cut 7% to 13% in all three of our runs); where the add rule buys connections above the operator's setting on a slow write workload (`simple_update`: the most at once up 8% to 12%), that reads power given and counts below 1 in the index. The earlier counted sets are kept whole in `docs/history/V3_PGBENCH_set1.md` to `V3_PGBENCH_set3.md`. Table: `results/live/V3_PGBENCH.md`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

56.4 Real messaging: Apache Kafka, a consumer group's operator-set size, Omni v3, three runs (evidence class L)

Apache Kafka as shipped (one broker, a topic of 8 partitions) with the consumer group at the operator's 2 consumers is native; omni is the compass law on one knob, the consumer count, inside the cover [1, 8], holding the group's own end-to-end latency at 40% of the 500 ms line (`docs/KAFKA_PREREGISTRATION.md`). Three paired repetitions a run, three runs, the consumers' own records for the gauges; the tuning workload is shown and not counted. Runs: A 38054742933; B 38054746778; C 38054750788.

Workload	Work inside the 500 ms line	End-to-end p95	Consumers running (the resource held)	Host CPU-seconds (the compass's own cost)
`tuning` (tuning, shown, not counted)	STEADY, inside the noise	STEADY, inside the noise	**-0% to -0%, GAIN in all three of our runs**	STEADY, inside the noise
`burst`	STEADY, inside the noise	STEADY, inside the noise	**-1% to -1%, GAIN in all three of our runs**	STEADY, inside the noise
`heavy`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise
`light`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise

On the counted set that stands (the brain's own verdict on the knob, amendment 2, run on 10 October: `docs/RERUN_2026-10-10.md`), the consumer count was handed back where the queue allowed: consumers running cut 0.8% on `burst` (0.3% on the tuning workload, shown and not counted), a GAIN in all three of our runs, with work, latency and CPU STEADY; `heavy` and `light` read STEADY on every gauge. No message went unconsumed in any arm; every count was handed back. The earlier counted sets are kept whole in `docs/history/V3_KAFKA_set1.md` and `V3_KAFKA_set2.md`. Table: `results/live/V3_KAFKA.md`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

56.5 A real cache: Redis, the operator's memory ceiling, Omni v3, three runs (evidence class L)

Redis as Ubuntu ships it with the operator's 64 MB ceiling and allkeys-lru is native; omni is the compass law on one knob, the ceiling, inside the cover [16, 512] MB through Redis's own console, growing only while the cache is full and giving a notch back when calm and nothing is evicted (`docs/REDIS_PREREGISTRATION.md`). An application with a declared 5 ms store trip on a miss and a working set that steps up and down; three paired repetitions a run, three runs; the tuning workload is shown and not counted. Runs: A 38054731163; B 38054735302; C 38054739168.

Workload	Work inside the 2 ms line	Cache hit rate	p95	Memory ceiling held, MB (the resource held)	Host CPU-seconds
`tuning` (tuning, shown, not counted)	**+4% to +5%, GAIN in all three of our runs**	**+4% to +5%, GAIN in all three of our runs**	STEADY, inside the noise	**+19% to +26%, power given in all three of our runs**	STEADY, inside the noise
`burst`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise
`large`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise
`small`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise

On the counted set that stands (the brain's own verdict on the knob, amendment 2), every untouched workload reads STEADY on every gauge: under the resource objective the verdict left the ceiling where the operator set it, as the preregistration expected before the runs. The tuning workload, shown and never counted, bought a GAIN in work inside the line (+4% to +5%) and in the hit rate with memory given (+19% to +26%, power given). The earlier counted sets are kept whole in `docs/history/V3_REDIS_set1.md` and `V3_REDIS_set2.md`. Table: `results/live/V3_REDIS.md`.

56.6 A real database's storage-engine cache: MongoDB under YCSB, the operator's cache size, Omni v3, three runs (evidence class L)

MongoDB 8.0 as its publisher ships it with the operator's WiredTiger cache of 512 MB is native; omni is the compass law on one knob, the cache size, inside the cover [256, 2,048] MB through the server's own console, growing by notches of 64 MB only while the cache is full and reads are slow, and giving a notch back when

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

calm and the cache holds its working set (`docs/YCSB_PREREGISTRATION.md`; the first counted set's gate was 'nothing evicted', which its amendment 1 of 8 October records as satisfied by a cold cache, and the table named below says which set it is). YCSB's published core workloads with the key space stepping through the cache and past it, drawn uniformly, at 3,000 operations a second from 32 threads; three paired repetitions a run, three runs; the tuning workload (workload A) is shown and not counted. Runs: A 38054777807; B 38054782696; C 38054787517.

Workload	Work inside the 1 ms line	p95	Cache size held, MB (the resource held)	Pages read into the cache	Host CPU-seconds
`tuning` (tuning, shown, not counted)	STEADY, inside the noise	the runs disagree	STEADY, inside the noise	shown, not judged	STEADY, inside the noise
`b`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	shown, not judged	STEADY, inside the noise
`burst`	STEADY, inside the noise	STEADY, inside the noise	**-13% to -10%, GAIN in all three of our runs**	shown, not judged	STEADY, inside the noise
`c`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	shown, not judged	STEADY, inside the noise
`f`	STEADY, inside the noise	STEADY, inside the noise	**-15% to -13%, GAIN in all three of our runs**	shown, not judged	STEADY, inside the noise

On this machine the data files sit in the operating system's page cache as well, so a storage-engine miss is a read from memory, not a disk, as disclosed before the run. On the counted set that stands, the cache size held was cut 10% to 15% on `burst` and `f`, a GAIN in all three of our runs, with work inside the line, p95, mean latency and CPU STEADY; `b` and `c` read STEADY on every gauge; the cache was handed back on every omni arm. The earlier counted sets are kept whole in `docs/history/V3_YCSB_set1.md` to `V3_YCSB_set3.md`. Table: `results/live/V3_YCSB.md`.

56.7 A real database's buffer pool: MySQL under sysbench, the operator's InnoDB pool size, Omni v3, three runs (evidence class L)

MySQL 8.0 as Ubuntu ships it with the operator's 512 MB InnoDB buffer pool is native; omni is the compass law on one knob, the pool size, inside the cover [128, 2,048] MB in the server's own 128 MB chunks through its own console, growing only while the pool is full and the server's own statement latency is slow, and giving

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

a chunk back only while the pool's misses are under 1% of its reads (`docs/MYSQL_PREREGISTRATION.md`). sysbench's OLTP scripts as shipped with the tables in use stepping through the pool and past it, at a fixed offered rate from 32 threads; three paired repetitions a run, three runs; the tuning workload (point select) is shown and not counted. Runs: A 38054765202; B 38054768832; C 38054773573.

Workload	Work inside the line	p95	Buffer pool held, MB (the resource held)	Pages read from disk	Host CPU-seconds
`tuning` (tuning, shown, not counted)	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	shown, not judged	STEADY, inside the noise
`burst`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	shown, not judged	STEADY, inside the noise
`read_only`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	shown, not judged	STEADY, inside the noise
`read_write`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	shown, not judged	STEADY, inside the noise
`update_index`	STEADY, inside the noise	STEADY, inside the noise	STEADY, inside the noise	shown, not judged	STEADY, inside the noise

On the counted set that stands (the brain's own verdict on the knob, amendment 3), every untouched workload reads STEADY on every gauge, and the pool was handed back on every omni arm. The earlier counted sets are kept whole in `docs/history/V3_SYSBENCH_set1.md` to `V3_SYSBENCH_set4.md` (the first: 15 of 45 arms not handed back because the plug's restore was issued while the server was still withdrawing the blocks of a shrink, which MySQL ignores; the plug fixed and the fix declared). The update_index work-inside-the-line row counts almost nothing in either arm (a single update's client round trip exceeds the server-side 0.6 ms line) and is disclosed. Table: `results/live/V3_SYSBENCH.md`.

56.8 The bill on a real cloud: Azure Kubernetes Service, Omni v1 (evidence class L, a metered bill)

Azure's own cluster autoscaler is native; omni is the same autoscaler with Omni-Compass idling the machines it gives back; the bill is Azure's own count of machines every 15 s at list price, a fresh cluster per arm, deleted after it (`.github/workflows/aks-metered.yml`,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`docs/K8S_COMPASS_PREREGISTRATION.md`, the bill on a real cloud). Transcribed from the receipts they cite:

Test	Fleet	Bill	Response time	Reading	Receipt
Steady load, 5 pairs	4 workers	−4.7%, interval across zero	inside the noise	STEADY, no difference beyond the noise on any gauge	`results/live/V1_AKS_STEADY.md`
A burst sized to the cluster, 5 pairs	4 workers	+5.0%, interval across zero	p99 −34% clear of the noise in this one run (a GAIN in one run, not a reading of three); p95 inside the noise	the bill STEADY, inside the noise	`results/live/V1_AKS_BURST.md`

One machine is a quarter of a 4-worker fleet, so only a cut of about 30% or more can clear the noise there; the expected machine cut is a few percent. The fleet that can show one machine (40 workers in nine machine families, the subscription's per-family allowance being 10 vCPUs) is preregistered and dispatched on v3; its first dispatches were refused by the subscription's allowances and by Azure's own cluster capacity in eastus before any arm ran, each refusal recorded in the preregistration's amendments.

56.9 The GPU governor on the modelled card: each base alone, and with Omni on top (evidence class S)

Omni-Compass never runs the card. It sits on the card's own firmware (or on an operator's power cap) and moves the clock ceiling and the power limit, which that base already accepts (`omni_controller/gpu_compass.py`, the same law in `realms/gpu_card.py`). A step down is taken only after a paired trial on the card shows it adds at most 2% to the card's own time on a request (`omnicompass/verdict.py`); where no step passes, the card runs as it does alone.

Work	Base	Energy (tuning / fresh)	Median response	p95	p99
Compute-bound	firmware + Omni vs firmware alone	-0.70% / -0.48%	+1.56% / +1.47%	-0.84% / +0.01%	-0.09% / +0.02%
Compute-bound	105 W cap + Omni vs the cap alone	+0.02% / -0.11%	-12.29% / -25.59%	-6.41% / -7.06%	-3.15% / -5.82%
AI token generation	firmware + Omni vs firmware alone	-3.25% / -3.72%	+0.55% / +0.70%	+0.29% / +0.26%	+0.02% / -0.52%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Work	Base	Energy (tuning / fresh)	Median response	p95	p99
AI token generation	105 W cap + Omni vs the cap alone	-2.10% / -2.38%	+0.07% / +0.12%	+0.03% / -0.02%	-0.06% / -1.57%

Source: `results/sim/gpu_two_wire/RESULT.md` and `fresh/RESULT.md`. The rule, and why the allowance is 2%, is amendment 8 of `docs/GPU_PREREGISTRATION.md`.

56.10 The six organisms at 1, 10, 100 and 1,000 runs and sizes, Omni v3 (evidence class S)

Each organism runs native (its own controllers) and with the compass law on every muscle, same seed, same load, same clock. Size is the number of copies of the organism governed together on one clock (1,000 copies of the four stacked is 1.7 million modelled muscles); runs are the first N of the same paired set, so 1, 10, 100 and 1,000 nest. 84 of 90 cells are done; the six left (100 and 1,000 runs at 1,000 copies) are beyond the machines available and say so.

The full grid with every cell: `results/scale/GRID.md`; the receipts, one per size: `results/scale/receipts/`. Work per energy is up in every cell, a GAIN, the same figure at every size (+0.07% for Physics to +0.37% for Energy, the four stacked and the tower); the time over the service line is at or under native's in every cell, so the band-first rule holds and every cell of 10 runs or more reads GAIN within the guardrails by the preregistered rule. Every knob was handed back in every run.

56.11 The 945 muscles and the four realms, Omni v3 (evidence class S)

The catalog (`realms/catalog.csv`): 945 muscles in 59 families, 430 in Compute / AI / Cloud, 376 in Physics / Robotics / Autonomous, 470 in Energy / Facility / Industrial and 440 in Distribution / Specialized (1,716 counting a muscle once per realm, a shared spine of 257). Every muscle alone and every organism whole ran as A, B and C on v3 and reproduced to the last digit (`results/realms/REALMS.md`): 0 muscles with a LOSS, every organism GAIN within the guardrails (work per energy +0.1% to +0.3%, work STEADY, time over the line not above native's). On v2 the Physics realm and the tower read GAIN AND LOSS (a service traded for energy); the cause was a missing do-no-harm gate on speed knobs, which made v3 (`docs/OMNI_V3.md`). Three independent simulators with their own native controllers are wired the same way and read by the same rule: the power grid (`results/live/V3_PANDAPOWER.md`: energy drawn and net import cut in all 11 SimBench grids with ZIP loads, a GAIN; the energy dissipated in the lines up in 4 and tap operations 4 → 8 a year in one, power given), robot arms (`results/live/V3_MUJOCO.md`: Gen3 peak torque cut 29%, tracking error cut 21%, energy per takt cut 0.8%, each a GAIN; the Panda's copper heating up 14%, power given; two arms left native, STEADY) and CityLearn (`results/live/V3_CITYLEARN.md`: electricity bought, peak and unevenness cut in all 11 battery districts, a GAIN; the bill up in 7, power given). Every row stands in every table, whichever way it went.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

56.12 Harnesses and receipts

Harness	What it proves	Receipt
`scripts/kind_paired.sh`, `tools/live_reps.py`, workflow `benchmark-reps`	native against Omni on real Kubernetes, paired on one runner, the reset checked	`results/live/raw/run-*/live-reps/`
`tools/confirm_abc.py` (and `pgbench_abc.py`, `kafka_abc.py`, `redis_abc.py`, `swarm_abc.py`, `mujoco_abc.py`, `pandapower_abc.py`, `citylearn_abc.py`)	three separate runs read by one rule: GAIN or LOSS in all three of our runs (power given for a resource spent that rose), STEADY, the runs disagree; each run's engine checked against the fingerprint	`results/live/V3_*.md`, `V1_*.md`
`tools/omni_version.py`	which frozen engine a checkout or any result's commit carries	`OMNI_V1.json`, `OMNI_V2.json`, `OMNI_V3.json`
`scripts/aks_paired.sh`, workflow `aks-metered`	the same on Azure's managed Kubernetes, Azure's own bill, a fresh cluster per arm, the fleet pre-flighted against the subscription's allowances	`results/live/V1_AKS_*.md`
`tools/run_kil.py`, workflows `six-kube`, `big-organism`, `big-organism-detached`; `tools/six_kube_report.py`	the six organisms with a real cluster inside at 1 to 1,000 copies, on GitHub and on a rented machine; the clock rule	`results/live/V1_SIX_KUBE.md`, `V3_SIX_KUBE.md`, `V1_BIG_ORGANISM.md`
`tools/run_pgbench.py`, workflow `pgbench`	a real database behind its pooler, one knob through the pooler's console	`results/live/V3_PGBENCH.md`
`tools/run_kafka.py`, workflow `kafka`	a real message broker as shipped, one knob (the consumer group's size) read from the consumers' own records	`results/live/V3_KAFKA.md`
Harness	**What it proves**	**Receipt**
`tools/run_redis.py`, workflow `redis`	a real cache as shipped, one knob (the memory ceiling) through its	the three-run table V3_REDIS.md in `results/live/` when its

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Harness	What it proves	Receipt
	own console, growing only while the cache is full	runs land
`tools/run_swarm.py`, workflow `swarm`	Omni on top of each drone's shipped autopilot in gym-pybullet-drones, collisions voiding the cell	`results/live/V3_SWARM.md`
`tools/run_scale.py`, `tools/pool_scale.py`, workflow `six`; `tools/grid.py`	the six organisms at every run count and size	`results/scale/GRID.md`, `results/scale/receipts/`
`tools/run_realms.py`, workflow `realms`	every muscle alone and every organism whole	`results/realms/REALMS.md`
`tools/run_pandapower.py`, `run_mujoco.py`, `run_citylearn.py`	Omni on top of an independent simulator's own controller	`results/live/V3_PANDAPOWER.md`, `V3_MUJOCO.md`, `V3_CITYLEARN.md`
`tools/omni_index.py`	the one combined number, read only from the three-run tables	`results/OMNI_INDEX.md`
`scripts/gpu_rented_run.sh`, `tools/gpu_wire_check.py`	one command on a rented card: wire check, smoke, the organisms with the card inside, the preregistered counted run	the founder's runs, to come
workflow `archive-run`	every finished run's files copied with the code onto the branch `raw`, one SHA-256 manifest per repetition	`results/live/raw/run-<id>/` (opened by `scripts/raw_archive.sh`)
`verify.py`, `tools/release_manifest.py`	everything above re-runs and checks itself; the manifest fingerprints the result	`results/VERIFY_RECEIPT.txt`, `RELEASE_MANIFEST.json`

The rules for each run were written and committed before it ran (`docs/*_PREREGISTRATION.md`); every row is reported, LOSS included; nothing is read across engine versions.

56.13 What is not yet shown

- A cloud bill or energy cut on real machines: the 4-worker Azure fleet reads STEADY, inside the noise, as a fleet too small to show one machine must; the 40-worker fleet runs are the test of that.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- The real card on the current governor: every earlier card result ran on a controller since replaced and is obsolete; the one-card, card-inside-the-organisms and eight-card runs are the founder's, on rented cards, at one named commit.
- The grid stops at 100 copies and 100 runs (the founder's order of 12 October, to cut the time and the bill): no 1,000 copies and no 1,000 runs are run on any engine from that day; the v3 cells at 1,000 above stand as they were run.
- The queue in `docs/REGISTER.md` section 4 (drone swarms on gym-pybullet-drones and Kafka done; Redis running): PX4 and ArduPilot swarms, YCSB and HammerDB, Spark, OpenSearch, fio, Open-RMF, the 24-hour robustness run, Basilisk, Orekit and GMAT, RocketPy, Cantera.

CHAPTER 57

Paired Runs and Receipts on Your Own System

The method of every benchmark in this manual is the method a reader can use on their own system, and it is deliberately simple: the same system, the same work, native and then omni back to back on the same machines, repeated with the order rotated, each gauge reported with the 95% interval of its paired difference. Pairing is what makes a difference of a few percent visible on a shared machine whose own noise is larger than that: two arms run minutes apart on one runner see the same neighbours and the same hour, so their difference is a difference between arms, while two arms on separate machines differ by about 15% in p95 on their own. The order rotates so that no arm always runs first, warm or cold. The work is held equal by an open-loop load, a fixed rate of requests at fixed moments, so that a lower resource count can never mean that less work was done. The steps below are the whole method.

1. **Paired runs.** Run your service the same way twice, once native and once with Omni-Compass on top, back to back on the same machines, the order rotated, at least 5 times (10 for a result you publish). `scripts/kind_paired.sh` and `tools/live_reps.py` do this on Kubernetes; `scripts/gpu_rented_run.sh` does it on a GPU box, in one command: wire check, smoke, the seven organisms with the card inside, then the preregistered counted run.
2. **The switch drill** after every run: turn Omni-Compass OFF; check every setting is back at its recorded original; turn it ON.
3. **The receipt.** Each gauge: native, Omni-Compass, the change, the 95% interval of the difference, and whether the interval excludes zero. A change under 0.5% reads STEADY; a change of 0.5% or more reads its word by the gauge's aim, GAIN, LOSS or power given, and the receipt says whether its interval clears zero (beyond the noise) or not.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

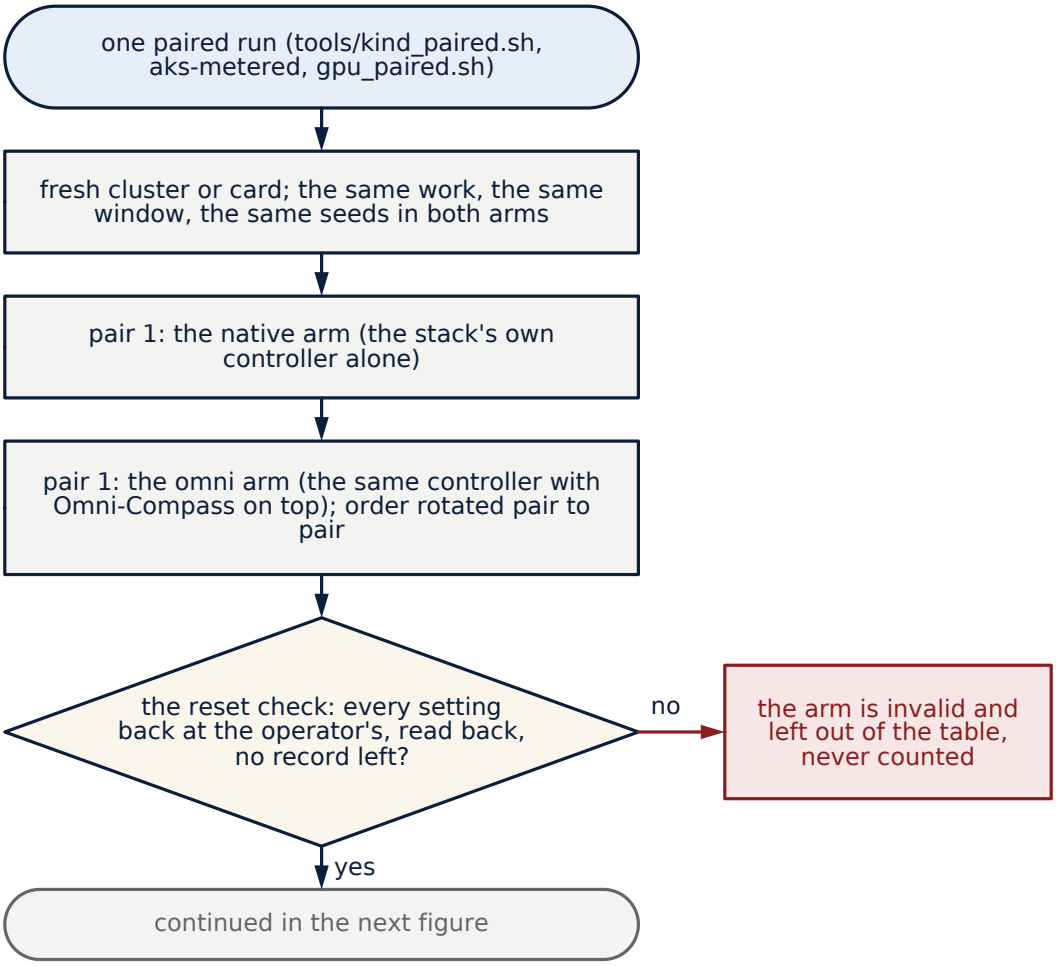


Figure 53. Flowchart: one paired run, the pairs. A fresh cluster or card; the same work, the same window and the same seeds in both arms; each pair runs the native arm (the stack's own controller alone) and the omni arm (the same controller with Omni-Compass on top), the order rotated pair to pair; the reset check after each arm (every setting back at the operator's, read back, no record left) invalidates an arm that fails it, and that arm is never counted. Continued in the next figure.

4. **The label, by rule written before the run:** SUPERIOR WITHIN GUARDRAILS / ENERGY IMPROVEMENT WITH SERVICE TRADEOFF / NONINFERIOR / INCONCLUSIVE / NOT ESTABLISHED / AWAY FROM ITS AIM / NOT ATTRIBUTABLE / INVALID. Guardrails: work not lower by more than 1%; the share of time outside the service line not higher by more than 1 percentage point. The band-first rule is stricter: no win is claimed while the time outside the service line is above native's.

At scale (modelled). The seven organisms run on GitHub's machines (Actions, workflow `six`) or on any machine (`bash scripts/scale_ladder.sh`) at 1, 10, 100 and 1,000 paired runs and at 1, 10, 100 and 1,000 copies of each organism on one clock. Real Kubernetes runs on GitHub's machines (workflow `benchmark-reps`); the seven organisms with a real cluster inside as one more muscle run in `six-kube` (1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

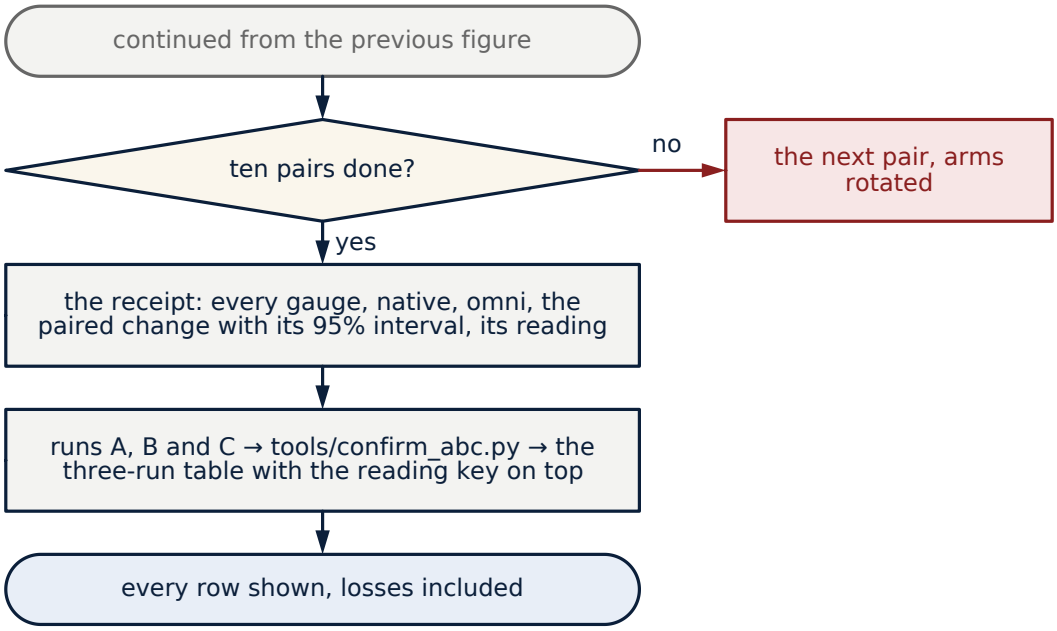


Figure 54. Flowchart: one paired run, from the pairs to the three-run table. After ten pairs the receipt is written: every gauge, native, omni, the paired change with its 95% interval and its reading; runs A, B and C go through tools/confirm_abc.py into the three-run table with the reading key on top; every row is shown, losses included.

to 100 copies on GitHub) and `big-organism` and `big-organism-detached` (1,000 copies on a rented machine).

5. **Preregister, then run.** The arms, the knob, the cover, the gauges and the readings are written before the first counted run (`docs/*_PREREGISTRATION.md`); the rules are frozen on one tuning case and then applied unchanged to untouched cases. A rule changed after a result is a new version of the engine (Chapter 59), never a footnote.
6. **The clock rule.** An organism must keep its window. `tools/run_kil.py` records how long after the window its last step ended; the report shows it as "organism behind its window (s)", shown and not judged, and marks a repetition **OFF THE CLOCK** when either arm ended more than 5% of the window late, because its last steps then saw a cluster whose load schedule had already ended. The pairing stands (both arms slip alike) and the mark stays on the cell. The remedy is a longer window for that size on that machine, never a faster reading of the same run.
7. **Every row, losses included.** A row that went against Omni-Compass is reported with the others. If Omni-Compass loses, the first suspect is our own wiring, native setup or scoring; that is fixed and the test run again, and the loss stays in the record. A benchmark may be left unpublished; it is never published with rows cut out.
8. **The raw files live with the code.** Every finished run's files (capture, response times, audit log, controller log, one SHA-256 manifest per repetition) are copied into `run-<id>/` on the branch `raw` by the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`archive-run` workflow: every hour, by itself, every finished run of every workflow that hands in files, failed runs included (a run id in `.github/archive_request.txt` goes first; a run still going is left for a later pass), and every table names the runs and the commit it was made from. Main's tree carries the code and the tables only, so a clone of it is small; `bash scripts/raw_archive.sh open /run-<id>/` places a run's files at `results/live/raw/run-<id>/`, the path every table names.

How many repetitions, and why. The width of a paired interval shrinks with the square root of the number of pairs, so going from five pairs to ten narrows it by about a third, and from ten to forty by half again. On GitHub's runners, ten pairs of a 900 s window put the interval on work inside the line at about ±10 points of native and on p95 at ±13 to ±29 points (the all-four table: +48.6% with an interval of +38.7 to +58.4; p95 −61.6% with −74.8 to −48.4), wide intervals that still clear zero because the gains are tens of percent; the energy intervals on the same runs are ±0.2 to ±3%, narrow because the gauge is a declared formula over counts of machines, and that is why energy there reads "no difference beyond the noise" honestly rather than as a small claimed gain. A gain of a few percent on a cluster would need forty pairs or more to clear zero at these widths, and the manual does not claim one anywhere a table shows it inside the noise. Three pairs of a 7,200 s window, the long run's shape, give a wider interval on each run but three such runs, each read on its own, still have to agree in sign for a reading the same in all three of our runs. On a stack with a finer knob (connections, consumers, megabytes) the intervals are narrower at three pairs than the cluster's at ten, because the knob moves in hundreds of small steps rather than in whole machines. An operator planning their own receipt should run five pairs first, look at the interval widths on the gauges they care about, and set the count for the published run from those widths, never from the result.

Why the load is open-loop. A closed-loop load generator waits for each answer before sending the next request, so a slower system is offered less work and its throughput "improves" its own latency; two arms under closed-loop load are not doing the same work. An open-loop generator sends requests at fixed moments whatever the system does, so both arms are offered the same work to the request, a queue forms when the system falls behind, and the time over the line and the failed requests record the cost. Every real-stack test in this program (the cluster's load generator, pgbench's rate limit, Kafka's producer at a fixed rate, Redis and YCSB at a fixed target rate) offers its work open-loop, and a lower resource count in the omni arm can therefore never mean less work was asked.

What a receipt looks like. One row per gauge: the gauge's name and direction; native's value and omni's; the paired change as a percentage of native with its 95% interval; the reading. A head naming the system, the knob, the cover, the window, the number of pairs, the order rotation, the engine's fingerprint and the commit; a foot saying whether every arm was handed back and whether any repetition was off the clock. `tools/live_reps.py` prints exactly this for a Kubernetes run and the three-run tools print it for three runs side by side; a receipt from any other tool should be laid out the same way so that a reader of this manual can read it without learning a new shape.

The readings to expect, and what it means when you do not see them. The published tables are the reference an installer reads their own receipt against: not the numbers, which belong to our systems and our loads, but the signs and the words. On omni-v11's tables, the newest until omni-v12's three runs land, on real

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Kubernetes, ten pairs a run and three runs each (`results/live/V11_*.md`; the CruiseKube row is omni-v10's):

The service	Reads GAIN in all three of our runs	Reads STEADY	Power given
Rightsized with Goldilocks and the VPA's recommender	response time −47% to −48% mean, −61% to −64% at the 95th percentile, −68% to −70% at the 99th; time over the line −98% to −100%; alert minutes −100%; pods started −53% to −61%; the total pod start wait −48% to −63%	machines, failed requests, replicas, the mean pod start wait	parked-worker energy +1.7% to +2.3%
Scaled by KEDA's HTTP add-on	response time −57% mean, −72% to −74% at the 95th percentile, −58% to −62% at the 99th; time over the line −98% to −99%; alert minutes −100%; replicas −11% to −20%	machines, failed requests, pods started, pod start wait	parked-worker energy +1.6% to +2.2%
Scaled by KEDA on CPU and requests	response time −36% to −38% mean, −52% to −53% at the 95th percentile, −57% to −61% at the 99th; time over the line −97% to −100%	machines, failed requests, alert minutes, pods started, pod start wait	parked-worker energy +1.1% to +1.8%; replicas +2% to +3%
Governed by CruiseKube's own recommendations (omni-v10)	nothing: no setting written	every gauge	none
Held at a fixed CPU cap, four caps	nothing: the cap blocks the one lever	completed work and work inside the line at every cap	none

Read your receipt against this table in three steps. **First the words.** A service gauge that reads LOSS on your receipt, where the table reads GAIN or STEADY for the same kind of service, is read as a wiring fault before anything else (`DISCLOSURES.md`, section 4): the feed's freshness and its line, the sensed autoscaler's name,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the identity's permissions (Chapter 50), the window's length against the interval widths above. **Second the direction.** Where the table reads GAIN, your receipt should read GAIN or STEADY; a smaller gain is your stack's, not a fault, because the gain is bounded by how far native stood from its own best. **Third the power given.** A resource that rose on your receipt is read beside the work and the speed it bought; a rise with nothing bought is a question for the audit (the Operator's Manual, Appendix L): what did the governor write, and why. Two readings are not faults: STEADY on every gauge where native already runs at the top of its range (the knob is at its limit on native, and the governor leaves it there by design, Chapter 32), and a machine count that does not fall where no machine can be given back without a LOSS (the audit's `node_gate.reason` names the gate that held). What the table cannot give is your number: the same engine on another stack is another result, and it is yours (section 76.7).

CHAPTER 58

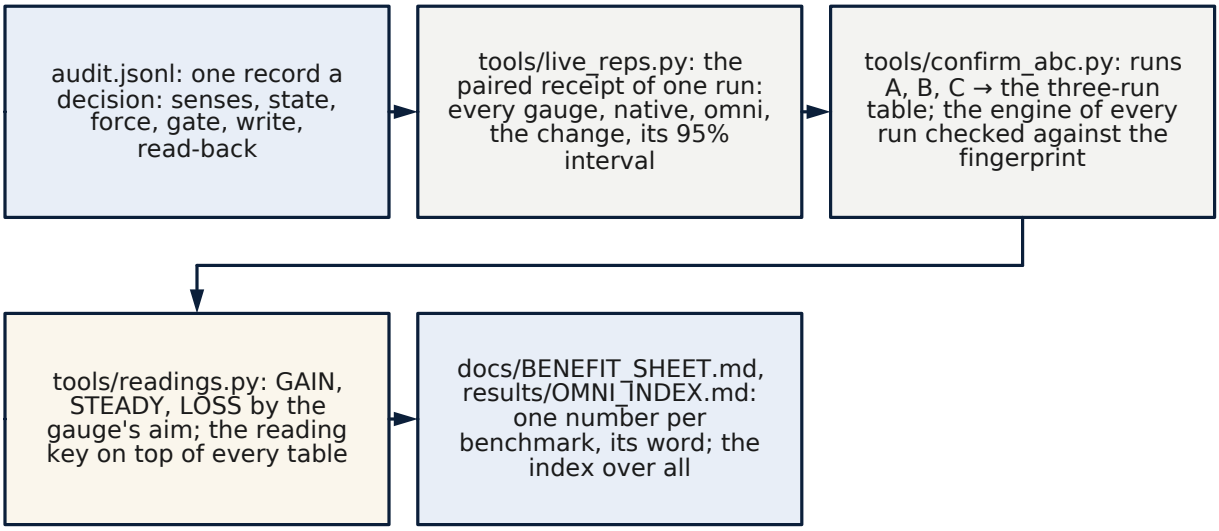
Evidence Classes and How to Read a Result

One number, one word. In every table a change keeps its raw sign: a response time that fell reads "-65%", a cache that grew reads "+300%", and the Reading column beside it gives its word, GAIN, STEADY or LOSS, decided by what the gauge is for and never by its sign (the key at the front of this manual): a response time cut 65% is GAIN, a cache grown 300% is **power given**, a fall in work is LOSS. The benefit sheet (`docs/BENEFIT_SHEET.md`, built from the tables by `tools/benefit_sheet.py` and checked by `verify.py`) turns every gauge the same way round against **each gauge's declared aim** (each preregistration's): plus is a GAIN, minus a LOSS or power given, whatever the gauge measures, and 0% is STEADY, nothing beyond the noise. Beside it every change the same in all three of our runs is written in words, never as a bare sign: "p95 cut 65% (GAIN)", so it cannot be misread against a table that shows the same change as "-65%". One number per benchmark, by the index's own rule, with its word beside it (GAIN, GAIN with power given, GAIN AND LOSS, STEADY, power given or LOSS). The wiring page (section 60.12) uses the same words. A reader who wants one figure for a benchmark reads that sheet; a reader who wants the rows behind it reads the table it names, and reads its Reading column, never the sign alone.

Class	Rung	What it is	What it can show
T / V	E1	deterministic tests, proofs, Python against the C++ twin	the law is what it says, and both languages agree
S	E2	a run on a modelled plant, our own model	whether the law helps the model, and where it breaks
L	E3	real software (Kubernetes on kind), no hardware meter	real decisions on real software; energy there is a declared model

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class	Rung	What it is	What it can show
P	E4	a physical meter (the GPU's own power reading)	the hardware's own answer



Every table is made by a tool, never by hand; raw records keep their recorded values, every page that shows them uses the words. A reader can follow one run from its audit line to its row in the benefit sheet.

Figure 55. Flowchart: from the audit to the receipt, the table and the one number. The audit (one record a decision) is read by tools/live_reps.py into the paired receipt of one run; runs A, B and C go through tools/confirm_abc.py into the three-run table, each run's engine checked against the fingerprint; tools/readings.py sets GAIN, STEADY or LOSS by the gauge's aim and puts the reading key on top of every table; the benefit sheet and the Omni index carry one number per benchmark. Every table is made by a tool, never by hand.

Read every number with its class beside it. A modelled number is never quoted as a hardware result. When a receipt's energy line is modelled, the receipt says so.

What each class can and cannot carry, with the examples in this manual. A class **T / V** result (the C++ twin agreeing with the Python to 3.6×10^{-15} on 500 trajectories; two million adversarial cases through the shield with zero violations) says that the law is what the text says it is and that both implementations compute it; it says nothing about whether the law helps any machine. A class **S** result (the 1,107 muscles and seven organisms, the organisms at 1,000 copies, the power grid in pandapower, the robot arms in MuJoCo, the buildings in CityLearn, the drones in PyBullet) says what the law does to a published model of a plant under that model's own native controller; it carries exactly the model's fidelity and no more, which is why S results are shown beside the Omni index and never inside it, and why the manual says "the energy is a declared model" on every S line that reports energy. A class **L** result (real Kubernetes on kind, PostgreSQL behind

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

PgBouncer, Apache Kafka, Redis, the organisms with the real cluster inside) says what the law did to real software making real decisions on real requests; the machines, where they are containers on one runner, are counted in machine-hours in service and their energy, where it is reported, is a declared formula, which the table says; the CPU seconds of the host are measured. A class **P** result would say what a physical meter read, and the one P instrument in this program, the GPU's own power reading, has no current result: every earlier card result ran on a controller since replaced and is obsolete. Azure's bill is an L result with a real bill: the machines billed are Azure's own count, the price is Azure's list price, and the energy is still not a meter.

How to read a row. Every row in every three-run table has the same shape: the gauge and its direction; native's value in run A and omni's value in run A, so the size of the numbers is visible; the paired change in each of the three runs as a percentage of native with its 95% interval; and the reading. Read the reading first, then the intervals, then the sizes. A reading of "GAIN in all three of our runs" with intervals of −0.5% to −0.1% is a small, real effect; a reading of "no difference beyond the noise (3 of 3 runs)" with intervals of −30% to +25% is a test that could not see an effect of that size either way, which is a statement about the test's power, and the manual says so where it applies (the 4-worker Azure fleet). A "LOSS in all three of our runs" row stands in the table, and enters the index as a loss.

A worked reading, from the cache table. Take the Redis table's `large` workload (`results/live/V3_REDIS.md`). The product row, work inside the 2 ms line, reads GAIN in all three of our runs, with the three runs' paired changes each clear of zero and of the same sign; that is the gain. The resource row, the memory ceiling held, reads power given in all three of our runs: Omni held a ceiling of about 256 MB on average where the operator had set 64 MB, in every run, clear of zero. The two rows together are the result: the cache answered more requests inside the line because it was given more memory, and the table says both. A reader who quotes the first row without the second has misread the table, and the index is built so that this cannot happen there: the cache's category enters the one number as a loss (about 0.75 on its own), because the resource it spent is counted at full weight against the work it gained. The CPU row, inside the noise, says the host did not pay in processor time; the failed-request row, zero in both arms, says nothing was refused to get there. Four rows, three directions, one honest sentence: more work inside the line, bought with memory, at no cost in CPU or refusals. That is how every table in Chapter 60 should be read, row by row, before any sentence about it is written.

Three questions to ask of any class L result. First, what was native, exactly: which controller, at which published version, with which settings, and were those settings an operator's or ours? (Every preregistration names them; the broker's native was set at its knee by design and says so.) Second, what did the governor spend: machines, memory, connections, CPU, its own cost? (Every table has the resource rows; a result with a gain and no resource row is not in this program.) Third, who else could have produced the gain: the order of the arms, the warm-up, the runner's neighbour, the clock? (Rotation of the arm order, the warm-up before every window, the paired design on one machine and the clock rule are the answers, and section 60.14 names the residue they do not remove.)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 59

The Frozen Engines and the Three-Run Rule

The engine is frozen, and every freeze has a fingerprint. The engine is the compass law (`omnicompass/compass_law.py`), the live controllers (`omni_controller/`), the realms with their muscles and six organisms (`realms/`), and the runners of the independent simulators (`tools/run_kil.py`, `run_citylearn.py`, `run_pandapower.py`). `OMNI_V1.json`, `OMNI_V2.json`, `OMNI_V3.json`, `OMNI_V5.json`, `OMNI_V6.json` and `OMNI_V7.json` hold one SHA-256 per file and one digest over all of them. Any change to a rule, a gain, a guard, a preset or a muscle makes **the next version**, and every result is run again on it; no result is ever read across versions. When the engine is declared final, the engine that stands then is published as Omni-Compass 1.0 and the older fingerprints go to `docs/history` as the road to it, never as a second product.

Engine	What changed	Fingerprint	Results read on it
v1 (`docs/OMNI_V1.md`)	the compass law, the controllers, 656 muscles in 46 families, six organisms; the power-grid runner added inside the version without touching any other engine file	`ccc7fbdf8b312ed7…`, 38 files	the six Kubernetes tests A/B/C, the six organisms with the cluster inside, the big organisms, Azure steady and burst, CityLearn, the power grid, the robot arms, the modelled realms
v2 (`docs/OMNI_V2.md`)	v1's law, controllers and runners byte for byte; the catalog grown to 945 muscles in 59 families with thirteen new presets	`OMNI_V2.json`	the modelled realms (Physics and the tower read a service tradeoff: the cause was found and made v3)
v3 (`docs/OMNI_V3.md`)	v2 plus the slack gate on speed knobs (a motion axis busy more than half the time at full speed	`b53d05449ee04c4b`, 40 files	the modelled realms A/B/C (every organism superior within guardrails, 0 LOSS or

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Engine	What changed	Fingerprint	Results read on it
	keeps its speed native) and the marine preset		power given), the grid at 1 and 10 copies, the cluster inside the organisms at 10 and 100 copies, the database A/B/C, the robot arms A/B/C, CityLearn A/B/C; the Kubernetes six tests, the power grid, Azure and the 1,000-copy organisms running
v5 (`docs/OMNI_V5.md`)	the max as the default brain on the 1,107-muscle catalog in 70 families (six organisms: 540, 404, 480, 484, the four stacked 1,908, the tower 1,107); every muscle its own card; the living body; the lease's walk back; the dial's tune-down; every muscle from 1 up to one click under its own capacity, read live	`6093b2b88aa911b9`, 44 files (re-declared on each change while in development: `9be48445…`, `5648f290…`)	the muscles and the six organisms under three loads (`results/realms_v5/`), the grid (`results/grid_v5/`), the capacity test (`results/live/V5_CAPACITY.md`)
v6 (`docs/OMNI_V6.md`), the engine of 15 October	v5's law and catalog byte for byte; the shared-host capacity repair in the Kubernetes controller (the host counted once, no extra replica room while it is busy or unmeasured, an enlarged ceiling rechecked); since 16 October its catalog is run as seven organisms,	`df6147466ef18046`, 45 files	Goldilocks, CruiseKube, KEDA's HTTP add-on, KEDA on the CPU target and the requests (`results/live/V6_*.md`); every other benchmark queued (`scripts/reruns.txt`)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Engine	What changed	Fingerprint	Results read on it
	the spine the seventh (`docs/THE_SPINE.md`), with no engine file changed		
v7 (`docs/OMNI_V7.md`), the engine of 16 to 17 October	quota-aware conveyance; the verdict in the reports' words; every rule in seconds; the cloud's rule as one flag; the busy guard on a shared host; corrected on 16 October after its first trials (an unreadable quota is no budget known)	`d6d467b824c585f1`, 46 files	the model gate (`results/model_gate/`); twenty-two three-run tables (`results/live/V7_*.md`), twenty-one with no row LOSS; CityLearn's 2023 districts LOSS, the cause ours, fixed as v8
v8 (`docs/OMNI_V8.md`), the engine of 17 October	v7 byte for byte but one file, the CityLearn runner: a battery whose building simulates power outages stays native, its charge being the building's reserve	`f93d233136a1f902`, 46 files	read locally on CityLearn before any GitHub run (the 2023 districts native's to the digit, every other district v7's); the model gate (`results/model_gate/V8_VS_V7.md`)
v9 (`docs/OMNI_V9.md`), the engine of 17 to 18 October	v8 with the CityLearn battery wire reading the coming hour (the mean of the same hour over the past seven days) and gliding at up to 0.0625 an hour (the fastest glide at which no score rose in any district); every 95% interval on the exact Student t quantile (`omnicompass/student_t.py`)	`7379f3a79ff2c475`, 47 files	read locally on CityLearn before any GitHub run (no LOSS against native in any district; the shipped runner equal to the study's to the digit); the model gate (`results/model_gate/V9_VS_V8.md`, identical to v8 on every model); its reruns started on 17 October

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Engine	What changed	Fingerprint	Results read on it
v11 (`docs/OMNI_V11.md`), the engine of 18 to 19 October (the founder's order)	v10 byte for byte but one file, the Kubernetes governor: a reading with no machine in service is held (recorded with its reason, nothing decided from it or written; the last reading with machines bounds the conveyance), where omni-v10 read it as the highest overload and divided by the empty capacity	`745aa53ec8eeb67b`, 47 files	a unit test that reproduces the fault on v10 and passes on v11 (`tests/test_blind_reading.py`); the model gate with a blink model (`results/model_gate/V11_VS_V10.md`); every benchmark runs again
v10 (`docs/OMNI_V10.md`), the engine of 18 October	v9 byte for byte but one file, the CityLearn runner: the battery wire's glide at 0.0875 an hour, the speed that cuts the bill most with no LOSS read by the one key; the runner's own receipt read by the one key	`4bf39b0d707784de`, 47 files	read locally on CityLearn before any GitHub run (the bill cut further in every district with a price, no LOSS against native in any district read before the push); the model gate (`results/model_gate/V10_VS_V9.md`); every benchmark queued (`scripts/reruns.txt`)
v12 (`docs/OMNI_V12.md`), the engine of 19 to 20 October	v11 with a raise the cluster's admission refused held until the pod or the raise changes or a wait passes, where v11 wrote it again at every fast check; the swarm's proximity reflex at once	`d38b58fbe28939b8`, 47 files	the model gate (`results/model_gate/V12_VS_V11.md`): identical on every gauge but the refused writes, −97.5%; every benchmark runs again
v13 (`docs/OMNI_V13.md`), the engine of 20 to 21 October	v12 with the CPU limits the conveyance gave handed back on a shared host at its top beside	`0088837333c6f0fc`, 47 files	the model gate with a batch model set from the KubeRay runs' files (`results/model_gate/V`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Engine	What changed	Fingerprint	Results read on it
	another workload's room, every other pod counted by its own limit (found on KubeRay's rig)		`13_VS_V12.md`); every benchmark runs again
v14 (`docs/OMNI_V14.md`), the engine of 21 October	v13 with a knob that leaves any one muscle native and the operator's own commands run without a shell	`7afba44615fcff8f`, 47 files	the model gate (`results/model_gate/V14_VS_V13.md`): every gauge identical; every benchmark runs again
v15 (`docs/OMNI_V15.md`), the engine of 21 to 22 October	v14 with the brain's power-cap request trimming only while the response line has been clean for the governor's own clear time; every wire that holds a governor's rules in the fingerprint	`0a53cdd7a558f803`, 55 files	the model gate (`results/model_gate/V15_VS_V14.md`): every service gauge identical, the request's own moves STEADY; the long run's own decisions (`tests/test_cap_band.py`); every benchmark runs again
v17 (`docs/OMNI_V17.md`), **the engine that stands** (23 October, afternoon, on the founder's word of 22 October)	v16 with the heartbeat a dial: the main engine's full decision every 60 s (the parser's default), any beat from 5 to 120 s, every beat shown beside native with what it buys and what it costs; the monitors scaled to the beat (the exporter's measured beat, the alert at three beats, the panel's beat button); the cruise wait never under two readings, a fault the dial's ladder found at 90 and 120 s before it landed	`7c937d09e4031cc7`, 55 files	the model gate (`results/model_gate/V17_VS_V16.md`): every model that sets its own period identical to omni-v16, as shipped every gauge STEADY, no LOSS; every beat in our models (`results/model_gate/V17_LADDER.md`); every benchmark runs again on it

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Engine	What changed	Fingerprint	Results read on it
v16 (`docs/OMNI_V16.md`), the engine of 22 October (the founder's word)	v15 with every clock of the Kubernetes governor in seconds (the brain's minute and its C++ twin, the compass's damping and smoothing, the edge, the additions past the floor, a command's minute to land, the host's busy share, the heat law, one batch job a minute, the conveyance every 15 s, the loop's own schedule), so the decision period is a clean setting; the heartbeat, the full decision, every 15 s by default, the fast checks every 5 s	`5c5502f12b1c0ce4`, 55 files	the model gate (`results/model_gate/V16_VS_V15.md`): identical at 60 s (4,578 audit records, none different), STEADY on every gauge at 15 s; as shipped, omni-v16 at 15 s against omni-v15 at 60 s, every gauge STEADY; the pace study (`results/model_gate/V16_PACE.md`): against 60 s no LOSS at any pace, at 15 s alert minutes −14.7% (kind setting) and energy per core −2.6% (cloud setting); against native the pods waiting for a place LOSS at every pace and, in the cloud setting at 15 s, the 95th percentile (+0.36% to +0.47%) and the alert minutes (+2.31%), STEADY at 60 s only by the seeds' spread, the engine's rows, on the next version's list; three runs read the heartbeat (`docs/PACE_PREREGISTRATION.md`); every benchmark runs again

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
python3 tools/omni_version.py                    # this checkout: omni-
    ↪ v18, omni-v17, omni-v16, omni-v15, omni-v14, omni-v13, omni-v12,
    ↪  omni-v11, omni-v10, omni-v9, omni-v8, omni-v7, omni-v6, omni-v5
    ↪ , omni-v3, omni-v2, omni-v1, or every file that differs
python3 tools/omni_version.py --commit <sha>    # the engine at the co
    ↪ mmit any result ran on
```

A commit that holds every engine file but one not yet written reads as that version with the file named ("omni-v1, 37 of 38 files, all v1 bytes; not yet in this commit: tools/run_pandapower.py"): a v1 result for every test but the one that runner serves, which could not have run there. Every table made by rule checks each run's commit against the fingerprint and heads itself with a warning if any run is not the engine it claims or if the three are not separate runs.

The three-run rule (the founder's order of 24 October 2026: any change of 0.5% or more is a change; `tools/readings.py`, `three_run_reading`). Every benchmark runs as **A, B and C**, three separate GitHub runs on the same frozen bytes. With c_i the change of run i in percent of native, the change read is the mean $\bar{c} = (c[A] + c[B] + c[C])/3$, and all three runs are shown beside the reading:

- **GAIN**, **LOSS** or **power given in all three of our runs**: $|\bar{c}| \geq 0.5\%$, and every run moved the same way with its 95% interval clear of zero. The word comes from what the gauge is for, never from the sign; a resource spent that rose reads power given, never a loss.
- **GAIN**, **LOSS** or **power given by the mean of our three runs**: $|\bar{c}| \geq 0.5\%$, but not every run cleared zero the same way; the count of runs beyond the noise stands beside the word ("beyond the noise in 1 of 3").
- **STEADY, under the floor**: $|\bar{c}| < 0.5\%$. **STEADY, the same**: nothing moved.
- **The runs disagree**: runs with $|c_i| \geq 0.5\%$ pointed in opposite directions. The test itself is then unstable on that measure, and it is looked into before anything is claimed.
- A count that must never rise (failed requests, lost messages) has no floor: any rise in any run reads LOSS. Each run's own number wears that run's own word by the same floor; a deterministic simulator's three runs, which reproduce one another, read in all three of our runs from 0.5% up.

The tables are made by rule, never by hand (`tools/confirm_abc.py` for the paired Kubernetes and Azure tests, `pgbench_abc.py`, `kafka_abc.py`, `redis_abc.py`, `swarm_abc.py`, `mujoco_abc.py`, `pandapower_abc.py`, `citylearn_abc.py`; a deterministic simulator's three runs must reproduce each other to the digit, or to the declared tolerance where the simulator is not bit-reproducible across machines, and a score reads "the runs differ" when they do not), and `tests/test_confirm_abc.py`, run by `verify.py`, proves the rule on fixed cases.

Why three separate runs, and why the readings are what they are. One run of ten paired repetitions gives one interval, and an interval from one run can be narrow by luck: the runner was quiet that hour, the order of arms happened to favour one side. A second run on another day and another machine is a replication; a third makes a coincidence expensive. The three runs are separate GitHub runs, which means separate machines,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

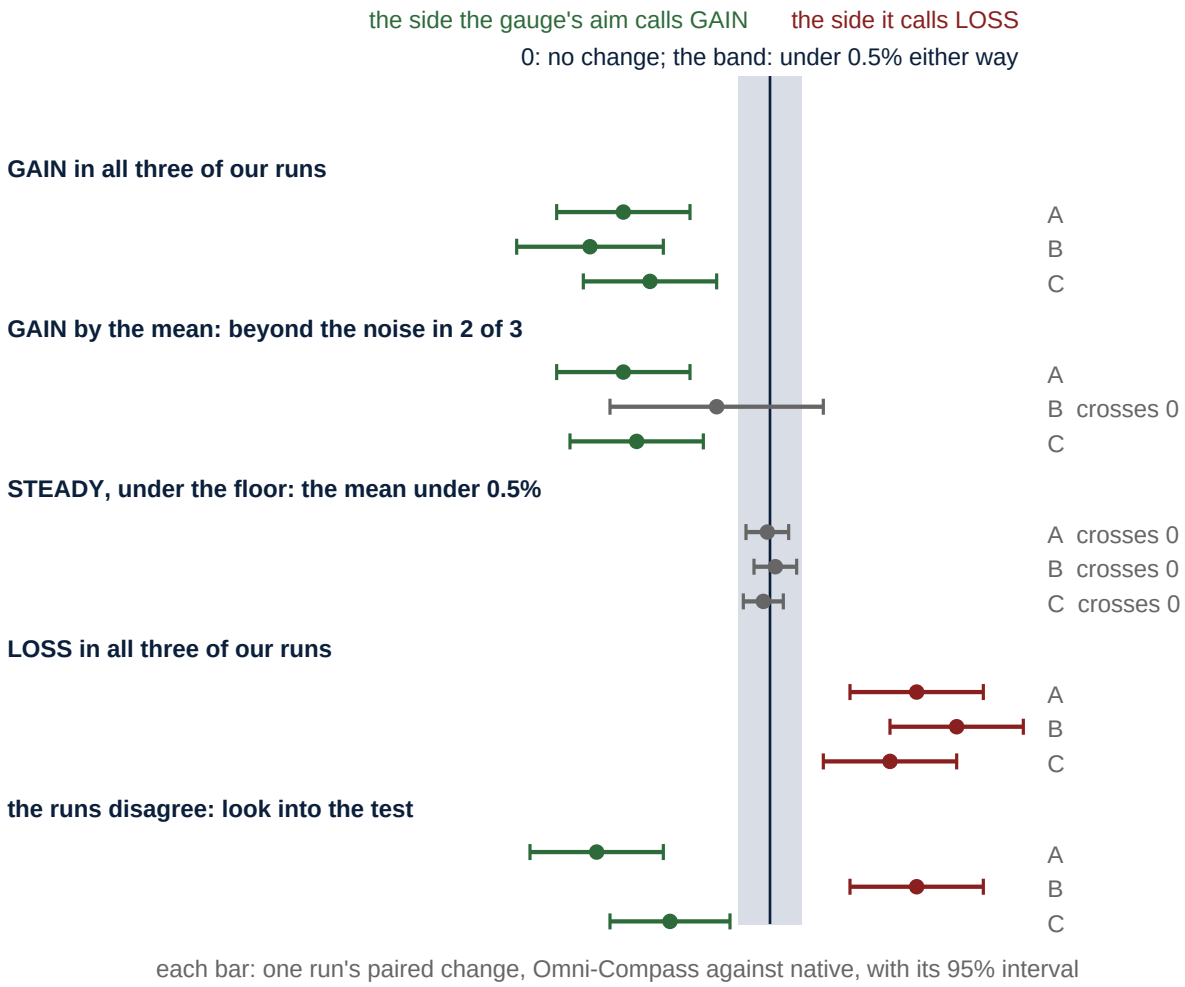


Figure 56. The three-run rule, read from the intervals, as drawn before the rule of 24 October (since then a mean change under 0.5% is STEADY and a change of 0.5% or more whose intervals do not all clear zero reads its word by the mean of our three runs). Each row is one gauge; each bar is one run's paired change of Omni-Compass against native with its 95% interval.

GAIN or LOSS in all three of our runs: the same side in all three, every interval clear of zero. STEADY, as drawn: at least one interval includes zero. The runs disagree: clear intervals on both sides, an unstable test to be looked into. Which side reads GAIN is set by what the gauge is for: for time waited, misses and failures a fall; for work done a rise; for a resource spent a fall is a cut and a rise is power given, never a loss.

separate hours and separate installations of every dependency, so what agrees across them is not an artefact of one machine. The readings are designed so that there is no reading a reader could mistake for a weaker form of "yes": a row reads "in all three of our runs" only when every run moved the same way with its interval clear of zero; a change of 0.5% or more that some run could not tell from the noise keeps its word, "by the mean of our three runs", with the count of runs beyond the noise beside it; a mean under 0.5% is STEADY; and runs of 0.5% or more that point in opposite directions are reported as a disagreement, which is an instability in the test

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

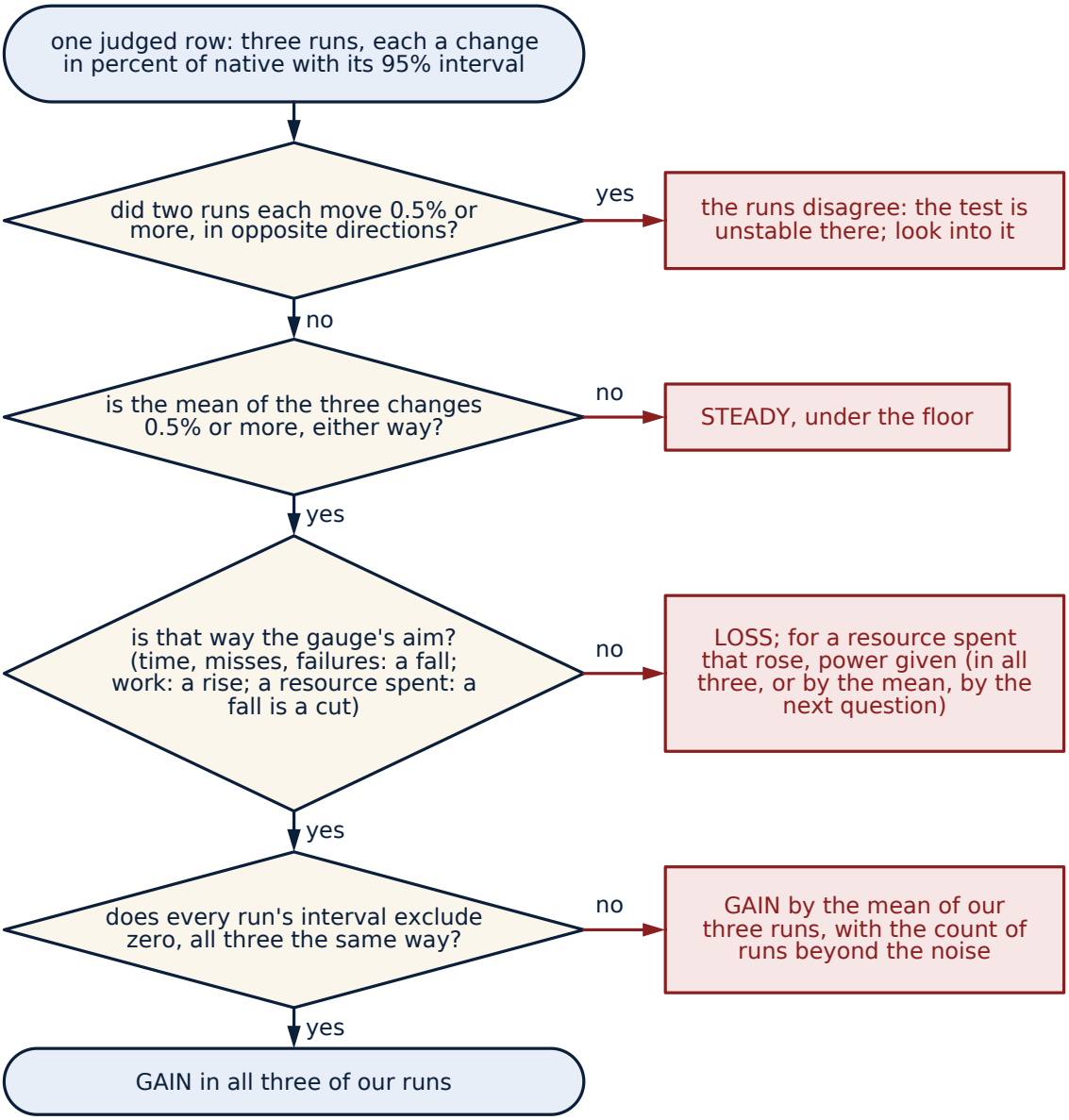


Figure 57. Flowchart: the three-run reading rule, as a procedure, as drawn before the rule of 24 October. For one judged row: if any of the three intervals includes zero the row reads STEADY, with the count of runs it holds in; if the three clear intervals do not point the same way the runs disagree and the test is unstable there; if they point the gauge's way the row reads GAIN in all three of our runs, and the other way LOSS, except for a resource spent that rose, which is power given.

Since 24 October the 0.5% floor on the mean of the three runs comes first, and an interval across zero no longer makes a row STEADY: it reads its word by the mean of our three runs, with the count beyond the noise (the text above).

to be found and fixed, not averaged away. The tables are made by tools, not by hand, and the tools check each run's engine and refuse to mix engines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The Omni index (`tools/omni_index.py`, `results/OMNI_INDEX.md`) is the one combined number. Every measure of every test is a ratio oriented so that above 1 is a GAIN for Omni-Compass on top of native: work (more), speed (a lower response time), machines (fewer), energy (less). A test's index is the geometric mean of its ratios, a category's the geometric mean of its tests, the headline the geometric mean of the real categories, each weighted the same. A measure enters only as its three-run reading allows: GAIN, LOSS or power given in all three of our runs counts as the geometric mean of the runs' ratios; anything else counts as exactly 1 (STEADY, the runs disagree, and a word read by the mean of our three runs), so nothing that did not hold in every run is claimed either way. Modelled muscles are shown beside the index, never inside it.

Why geometric means, and three rules a referee should check. A ratio of 2 and a ratio of 0.5 are the same size of effect in opposite directions; their arithmetic mean is 1.25, which would read as a gain, while their geometric mean is 1, which is the truth. Geometric means also make the index indifferent to which side a gauge is written from (work per machine or machines per unit of work), and they let a category of three tests and a category of six weigh the same. Three rules follow from the design and can be checked in the tool: first, the tuning workload or tuning cell of every benchmark is excluded, because the rules were fitted on it; second, no row inside the noise contributes anything, in either direction, so the index can never be moved by a run that happened to be quiet; third, the categories are the real ones only (Kubernetes, the database, messaging, the cache as it lands, Azure and the card as their tables land), each weighed the same whatever the number of tests inside it. The index therefore answers one question only: across every real category the same in all three of our runs, what did Omni-Compass on top of native deliver in more work, faster answers, fewer machines and less energy, counting the resources it spent against it. The index page says, where a ratio is large, why it is: in messaging, native sat at nine tenths of its measured capacity by design, so its queue grew at the high steps and Omni's did not, and a queue kept short against a queue that grows is a large ratio in response time.

CHAPTER 60

Results to Date

Every result below, read together in one place with what each shows and what it does not, by evidence class and engine, newest first: *The Receipts, in Their Totality* (`docs/THE_RECEIPTS.md`). The decisive receipt not yet produced is the physical one: the chip's own energy meter on a real GPU, on the current governor, on the same clock as the Kubernetes runs.

Every result below is Omni-Compass **on top of** a native system against the same native system alone, with the same work in both arms, read by the three-run rule of Chapter 59, on the engine named. Losses are in the tables beside the gains. The whole list, benchmark by benchmark with its native engine, its knob, its gauges and its file, is `docs/REGISTER.md`; the program that takes every benchmark to full size is `docs/PROOF_PROGRAM.md`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

How to read the chapter. Each section names the stack, the native controller Omni-Compass sat on, the one knob it moved, the result file in `results/live/` or `results/`, the three GitHub run numbers (or the rented machine's start and collect runs) and the engine version those runs were on; a reader can open the file, find the runs under the repository's Actions tab, and check the commit with `tools/omni_version.py --commit`. The figures quoted in prose are the table's figures and no others; where a range is given ("−13% to −37%") it is the span of the three runs' point estimates, not an interval. Every row of every table has one of the readings of the three-run rule (Chapter 59): GAIN, LOSS or power given in all three of our runs or by the mean of our three runs (with the count beyond the noise), STEADY under the floor or the same, or the runs disagree; rows that are shown and not judged say so and why. Every table was read again from its own data by the rule of 24 October (`tools/reread.py`); where a figure quoted below was written before that, its table's own word now stands. A loss is written in the same sentence as the gain it came with, never in a footnote, and the index (section 60.11) counts every loss in all three of our runs against Omni at full weight.

Two cautions a referee should hold while reading. The real-software results (evidence class L) were taken on GitHub's shared runners or on single rented machines, with the host's CPU-seconds as the only measure of energy and no meter on the wall; Chapters 57 and 58 say what that does and does not allow. And the modelled results (evidence class S) are reported as what they are, our own plant models run whole, and are never summed with the live ones; the index is built from the real categories alone, and the modelled realms sit beside it as a separate finding about the law's coherence at scale.

60.1 Real Kubernetes: seven tests, ten pairs each, three runs each

The six Kubernetes tests were designed to ask six different questions of the same cluster, and they are read together; a seventh, added on 8 October, asks the same questions under a day of demand that somebody else measured. The cluster is kind: a real Kubernetes control plane and six worker nodes as containers on one 4-core GitHub runner, with a real metrics-server, a real HPA at the operator's target of 50% CPU, a real PHP service behind a NodePort, and a real load generator whose replica count steps through a schedule identical in both arms. A real probe times HTTP requests every five seconds from outside the service. Native is that cluster with its HPA alone; omni is the same cluster with the compass law on the HPA target inside [30%, 50%] and on the machines through the release gate and the verdict. Both arms start from the same fresh cluster, run for 900 s after a 120 s warm-up, and end with the reset check.

Every table below ran on omni-v10, the engine before the one that stands (omni-v11 changed only the Kubernetes governor's hold on a reading with no machine in service, and every test runs again on it), ten pairs a run, three separate GitHub runs (A, B and C) released by the conductor on 18 October; each names its runs and its commits, and `tools/omni_version.py --commit` reads omni-v10 on every one. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

- **Steady work in steps** (`V10_STEADY.md`; runs 39107180703, 39114207589, 39121472594) asks the simplest question: the same work at a fixed rate, stepping through a schedule. The response time fell at the mean 48% to 49%, at the 95th percentile 64% to 65% and at the 99th 69% to 73%; the time over the line fell 98% to 100% and the alert minutes 100%, all GAIN in all three of our runs. Machines,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

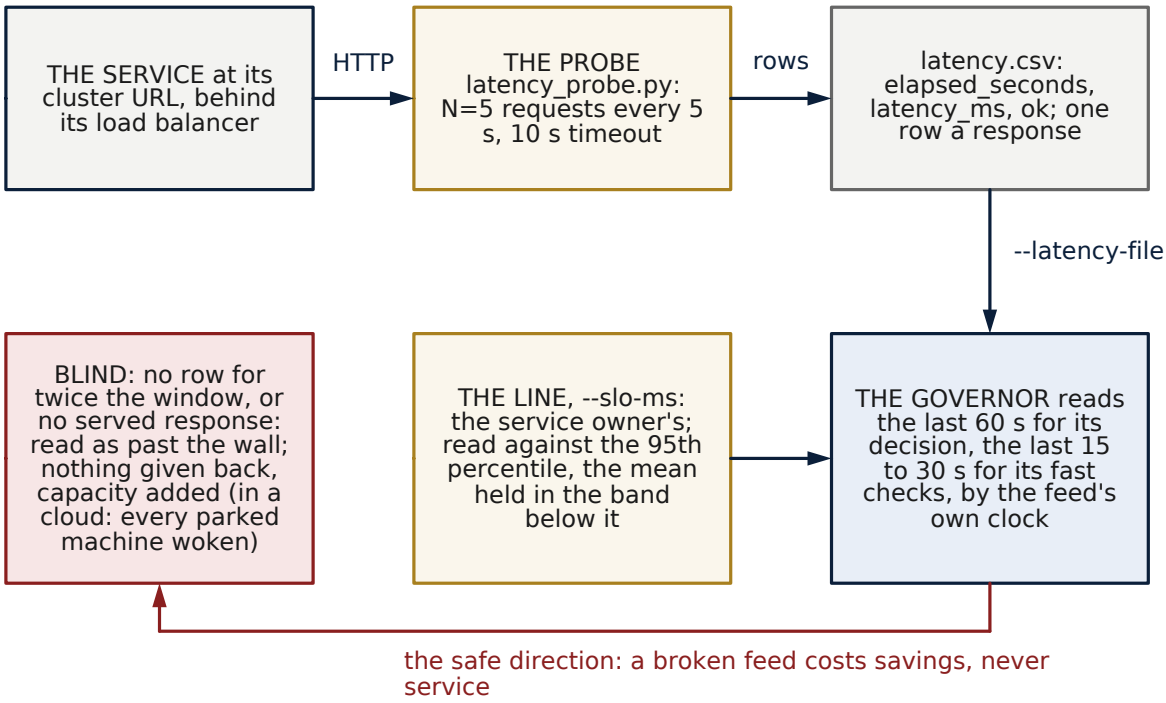


Figure 58. Wiring schematic: the response-time feed and the line. The probe times real requests to the service and writes one row a response to latency.csv (elapsed_seconds, latency_ms, ok); the governor reads the file by its own clock, the last 60 s for its decision and the last 15 to 30 s for its fast checks, against the line the service owner wrote (--slo-ms).

A feed that has not advanced for twice the window, or holds no served response, reads as blind, and a blind sense is read as past the wall: nothing is given back and capacity is added (in the cloud mode every parked machine is woken), the safe direction. A second feed, the service's own timings, may be written in the same three columns, measured at the same place as the line and never pre-smoothed.

node-hours, energy, HPA replicas, pods started and the pod start wait read STEADY; the service drew 5% to 6% less CPU (shown).

- **Demand that wanders** (`V10_WANDERING.md`; runs 39107183748, 39114211206, 39122521784): the load up and down one step at a time. The response time fell at the mean 45% to 50%, at the 95th percentile 57% to 66% and at the 99th 34% to 46%; the time over the line fell 38% to 45%, the alert minutes 21% to 32% and the energy 0.3% to 0.4%, all GAIN in all three of our runs. Failed requests fell 12% to 21% with one interval including zero (STEADY); machines, node-hours, replicas and pods started read STEADY.
- **All four in one run** (`V10_ALL_FOUR.md`; runs 39107187219, 39114214725, 39122525181) asks the product question: more work, faster, on fewer machines, with less energy, in one run. Work inside the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

response line rose **47% to 79%**, the 95th percentile fell 56% to 61%, the mean 38% to 46%, time over the line 35% to 46%, failed requests 9% to 14% and the alert minutes 16% to 34%, all GAIN in all three of our runs; the 99th percentile read STEADY (its intervals include zero), machines in service the same, node-hours and both energy models inside the noise.

- **Faults** (`V10_FAULTS.md`; runs 39107193417, 39115232348, 39122532509) asks: when a machine dies, traffic triples, a pod runs away and the probe goes blind, at the same moments in both arms, does the governor slow recovery? The 95th percentile fell 54% to 63%, the 99th 43% to 51%, the mean 33% to 41% and time over the line 26% to 32%, all GAIN in all three of our runs. Failed requests fell 8% to 15% and the alert minutes 13% to 20%, each with an interval including zero (STEADY). HPA replicas rose 14% to 20%, **power given in all three of our runs**: the pods the faster recovery took. Machines and energy inside the noise.
- **A queue of batch jobs** (`V10_BATCH.md`; runs 39107196814, 39115235566, 39122536157) asks: when the work is a pile, do cruise and idle finish it no later and hand the machines back? Machines in service and node-hours fell 6% to 14% over the window, the machines after the queue finished 10% to 22%, the standby-model energy 4% to 10% and the mean response 10% to 12%, all GAIN in all three of our runs; the queue finished no differently beyond the noise in all three runs.
- **Fairness** (`V10_FAIRNESS.md`; runs 39107190192, 39114217887, 39122528801) asks: does the governor help or hurt a noisy neighbour on the same workers? The governed service's time over the line fell 11%, GAIN in all three of our runs; every other judged row reads STEADY, the neighbour's own 95th and 99th percentiles, failures and time over the line included.
- **A public day of demand** (`V10_TRACE_GOOGLE2011.md`, `docs/TRACES_PREREGISTRATION.md`; runs 39108177259, 39115239009, 39122539586) asks: when the shape of the load is not ours, does the governor still read the same way? The first day of the Google cluster trace of 2011 (jobs submitted an hour, 18 public parts named with their SHA-256) was turned into a schedule by a rule written before the runs (min-max onto 1 to 8 generators, one step a bin, 24 steps of 108 s). The 95th percentile fell 56% to 62%, the 99th 38% to 51%, the mean 48% to 50%, time over the line 74% to 77%, failed requests 15% to 18%, the alert minutes 61% to 66% and the energy 0.3% to 0.4%, all GAIN in all three of our runs. Pods started rose 120% to 193%, **power given in all three of our runs**; machines in service the same and node-hours inside the noise; no pod was ever without a machine.
- **On top of Kubernetes with its add-ons** (KEDA on the CPU target and the requests in flight; KEDA's HTTP add-on), and on top of Goldilocks and of CruiseKube: section 60.2.

60.2 On the add-ons: four receipts and the capacity test (omni-v11; CruiseKube omni-v10)

omni-v11, the engine that stood until omni-v12 landed on 19 October (Chapter 59), has four of these three-run tables of its own, the newest until omni-v12's land: Goldilocks, KEDA's HTTP add-on, KEDA on the CPU target and the requests in flight, and the capacity test. Their runs were released by the conductor on 18 and 19

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

October, and `tools/omni_version.py --commit` reads omni-v11 on every run's commit. Each has ten pairs a run (five a cap in the capacity test), on the same kind cluster as section 60.1 with the add-on named as native, the served image pulled onto every worker before each arm. The CruiseKube receipt is still omni-v10's (18 October); its omni-v11 rerun, on the recorder corrected on 19 October, is running. They are read here gauge by gauge, native against Omni-Compass on top; the charts under each paragraph picture the table named and add nothing to it. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

On top of Goldilocks and the VPA's recommender (`V11_GOLDILOCKS.md`; runs 39171064099, 39175675418, 39177950720). Native is the service rightsized by Goldilocks from the VPA recommender's figures, with the HPA under it; omni is the same with Omni-Compass on top. The response time fell at the mean 47% to 48%, at the 95th percentile 61% to 64% and at the 99th 68% to 70%; the time over the line fell 98% to 100% and the alert minutes 100%; pods started fell 53% to 61% and the total pod start wait 48% to 63%: all GAIN in all three of our runs. Failed requests were zero in both arms. Machines in service, node-hours, HPA replicas and the mean pod start wait read STEADY. The energy counted with every parked worker at idle power rose 1.7% to 2.3%, power given in all three of our runs: the governor's own settings commands (16 to 18 a run, shown and not judged) conveyed processor time to the pods, the service drew 43% to 51% more CPU and answered faster for it, and the energy per core-hour fell 28% to 31% (shown).

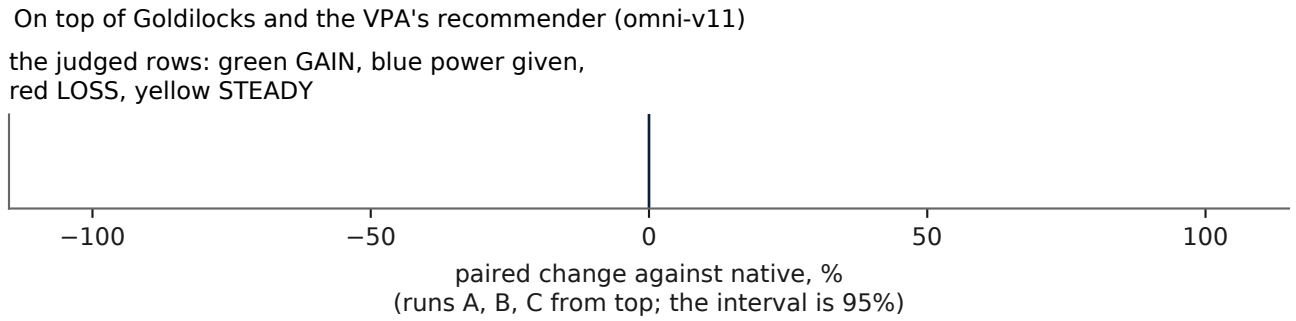


Figure 59. Receipt chart: Omni-Compass on top of Goldilocks and the VPA's recommender (omni-v11), the judged rows. Each row is one gauge; each line is one run's paired change against native with its 95% interval (runs A, B, C from top); green GAIN, amber power given, red LOSS, grey STEADY, by the reading key. The chart pictures results/live/V11_GOLDILOCKS.md and adds nothing to it.

On top of KEDA's HTTP add-on (`V11_KEDA_REQUESTS.md`; runs 39173411511, 39176842679, 39180252225). Native is KEDA 2.21 with its HTTP add-on 0.16 scaling the service on the requests in flight; omni reaches KEDA's ScaledObject through the plug, as section 60.1 describes. The response time fell at the mean 57%, at the 95th percentile 72% to 74% and at the 99th 58% to 62%; the time over the line fell 98% to 99% and the alert minutes 100%; HPA replicas fell 11% to 20%: all GAIN in all three of our runs. Machines, node-hours, failed requests, pods started and the pod start wait read STEADY. The pod start wait was the one LOSS on omni-v10; with the served image pulled onto every worker before each arm it reads STEADY (`docs/POD_START_WAIT.md`). The energy at idle power rose 1.6% to 2.2%, power given in all three of

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

On top of Goldilocks and the VPA's recommender (omni-v11)

	shown, not judged, runs A, B, C: the governor's own inputs and the mechanism's traces
settings commands the governor wrote on its own (no person touched either arm)	+15.9 +17 +17.5
pending pods in a 15 s snapshot, pod-minutes (a pod being started counts too: shown, not judged)	-42.3% -50.6% -70.6%
utilisation (used / allocatable)	+50.5% +43.3% +42.7%
CPU used (cores), mean	+50.5% +43.3% +42.7%
Omni's own CPU (cores), mean	+0.00824 +0.00789 +0.00736
CPU used with Omni's own (cores), mean	+51.5% +44.3% +43.7%
energy per core-hour (Wh, the 25 W standby model; a ratio over CPU used: shown, not judged)	-31.4% -27.9% -27.9%
host CPU busy, the real machine under kind (%)	+45.2% +40.2% +38.8%
host cores (the real machine under kind)	+0% +0% +0%

Figure 60. Receipt chart: on top of Goldilocks, the rows shown and not judged. The governor's own inputs and the mechanism's traces (the settings commands it wrote, the pods pending, the utilisation, the CPU used, its own CPU, the energy per core-hour, the host's busy share and cores), as the change in runs A, B and C. These rows explain how the judged rows came to be; they are not judged themselves.

our runs, the service drawing 47% to 56% more CPU for its faster answers while the energy per core-hour fell 30% to 34% (shown).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

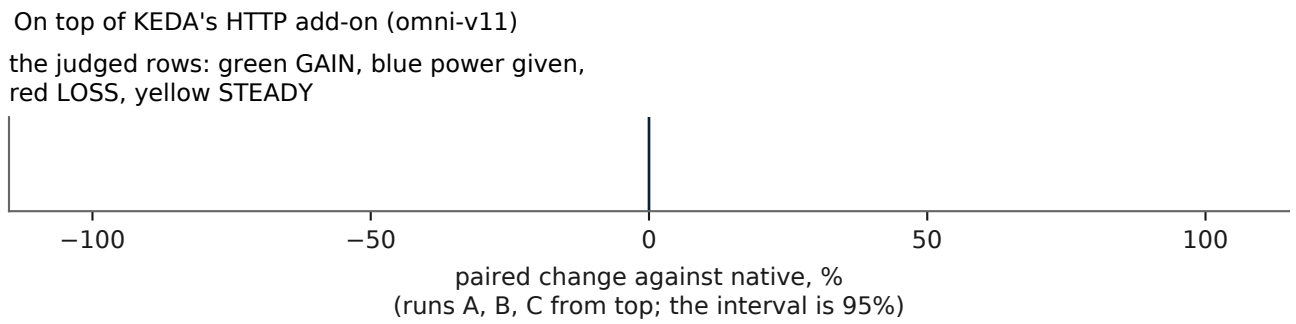


Figure 61. Receipt chart: Omni-Compass on top of KEDA's HTTP add-on (omni-v11), the judged rows, read as in the Goldilocks chart; it pictures results/live/V11_KEDA_REQUESTS.md.

On top of KEDA's HTTP add-on (omni-v11)

	shown, not judged, runs A, B, C: the governor's own inputs and the mechanism's traces
settings commands the governor wrote on its own (no person touched either arm)	+4.7 +4.1 +4.1
pending pods in a 15 s snapshot, pod-minutes (a pod being started counts too: shown, not judged)	+50% +113% +0%
utilisation (used / allocatable)	+53.9% +47.4% +55.7%
CPU used (cores), mean	+53.9% +47.4% +55.7%
Omni's own CPU (cores), mean	+0.00975 +0.00873 +0.00939
CPU used with Omni's own (cores), mean	+55.2% +48.7% +57%
energy per core-hour (Wh, the 25 W standby model; a ratio over CPU used: shown, not judged)	-33% -29.9% -33.6%
host CPU busy, the real machine under kind (%)	+44% +42.8% +50.3%
host cores (the real machine under kind)	+0% +0% +0%

Figure 62. Receipt chart: on top of KEDA's HTTP add-on, the rows shown and not judged: the governor's inputs and the mechanism's traces, as the change in runs A, B and C.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

On top of KEDA on the CPU target and the requests in flight (`V11_KEDA_CPU_REQUESTS.md`; the same three runs, the second setup in each). The response time fell at the mean 36% to 38%, at the 95th percentile 52% to 53% and at the 99th 57% to 61%, and the time over the line 97% to 100%, GAIN in all three of our runs. Failed requests, the alert minutes (native had none in run A), pods started and the pod start wait read STEADY; machines and node-hours STEADY. HPA replicas rose 2% to 3% and the energy at idle power 1.1% to 1.8%, power given in all three of our runs, for 28% to 30% more CPU drawn by the service and 19% to 21% less energy per core-hour (shown).

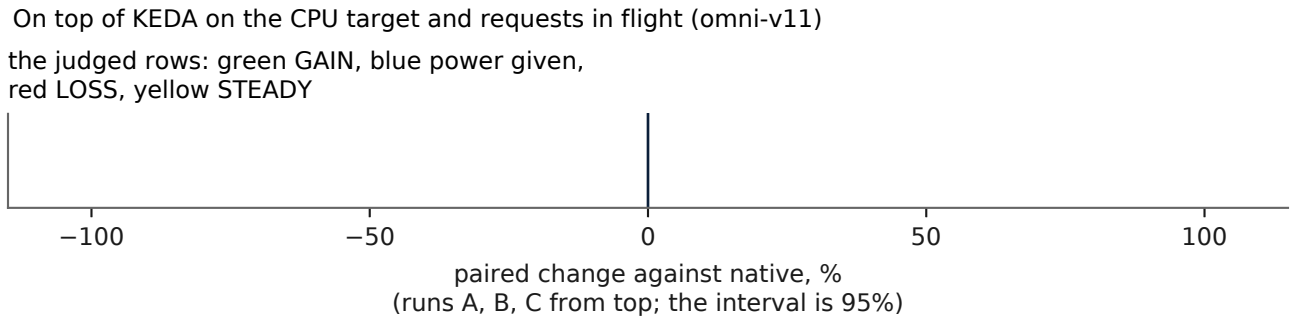


Figure 63. Receipt chart: Omni-Compass on top of KEDA on the CPU target and requests in flight (omni-v11), the judged rows, read as in the Goldilocks chart; it pictures results/live/V11_KEDA_CPU_REQUESTS.md.

On top of CruiseKube applying its own recommendations (`V10_CRUISEKUBE.md`; runs 39102512170, 39113183694, 39120444653). Native is CruiseKube 0.3.4 rightsizing the service's requests from its own recommendations; omni is the same with Omni-Compass on top. Every judged row reads STEADY: the response time at its mean, 95th and 99th percentile within 3.1% of native with an interval including zero in at least one run; the time over the line, failed requests and alert minutes zero in both arms; machines, node-hours and energy inside the noise. The governor wrote no settings command of its own in any run (shown): on the service CruiseKube had rightsized, nothing passed its rules for a move, and it took nothing away. The rows are the do-no-harm reading, native's work, speed and failures with Omni-Compass on top, and the receipt is kept as such.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

On top of KEDA on the CPU target and requests in flight (omni-v11)

	shown, not judged, runs A, B, C: the governor's own inputs and the mechanism's traces
settings commands the governor wrote on its own (no person touched either arm)	+18 +18 +17.7
pending pods in a 15 s snapshot, pod-minutes (a pod being started counts too: shown, not judged)	+36.2% -63.2% +197%
utilisation (used / allocatable)	+27.8% +28.8% +29.9%
CPU used (cores), mean	+27.8% +28.8% +29.9%
Omni's own CPU (cores), mean	+0.0113 +0.0117 +0.012
CPU used with Omni's own (cores), mean	+29.2% +30.2% +31.3%
energy per core-hour (Wh, the 25 W standby model; a ratio over CPU used: shown, not judged)	-19.4% -20.1% -21%
host CPU busy, the real machine under kind (%)	+26.6% +27.8% +29.9%
host cores (the real machine under kind)	+0% +0% +0%

Figure 64. Receipt chart: on top of KEDA on the CPU target and requests in flight, the rows shown and not judged: the governor's inputs and the mechanism's traces, as the change in runs A, B and C.

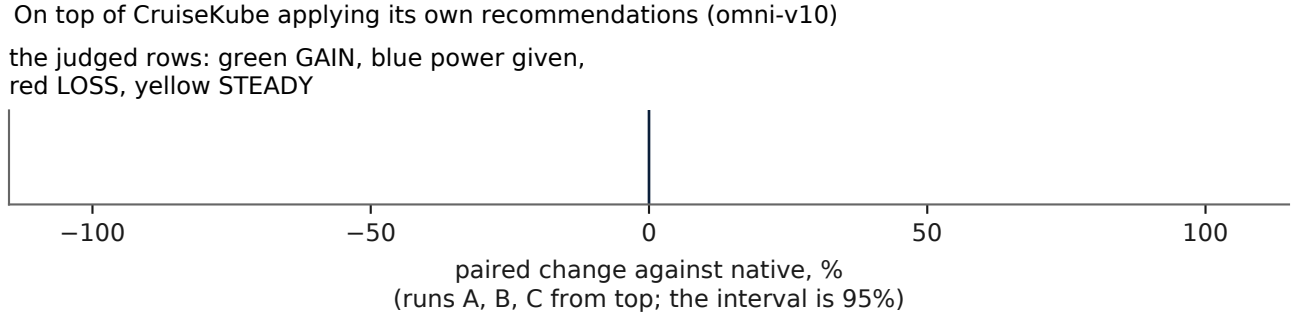


Figure 65. Receipt chart: Omni-Compass on top of CruiseKube applying its own recommendations (omni-v10), the judged rows, read as in the Goldilocks chart; it pictures results/live/V10_CRUISEKUBE.md.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

On top of CruiseKube applying its own recommendations (omni-v10)	shown, not judged, runs A, B, C: the governor's own inputs and the mechanism's traces
settings commands the governor wrote on its own (no person touched either arm)	+0 +0 +0
pending pods in a 15 s snapshot, pod-minutes (a pod being started counts too: shown, not judged)	+1.2% +712% -41.6%
utilisation (used / allocatable)	+1.1% -0.2% -0.2%
CPU used (cores), mean	+1.1% -0.2% -0.2%
Omni's own CPU (cores), mean	+0.00905 +0.00796 +0.00958
CPU used with Omni's own (cores), mean	+1.9% +0.5% +0.6%
energy per core-hour (Wh, the 25 W standby model; a ratio over CPU used: shown, not judged)	-1.2% -0.2% +0.2%
host CPU busy, the real machine under kind (%)	+2.1% +1.9% +1.2%
host cores (the real machine under kind)	+0% +0% +0%

Figure 66. Receipt chart: on top of CruiseKube, the rows shown and not judged: the governor's inputs and the mechanism's traces, as the change in runs A, B and C.

The capacity test at four caps (`V11_CAPACITY.md`; runs 39171068872, 39175679247, 39179020367). Native and Omni-Compass on top run under the same CPU quota on the served app, 1, 1.5, 2 and 3 cores, enforced by the cluster, five pairs a cap, with work waiting in line. The completed work and the work inside the line read STEADY at every cap, all eight rows in all three runs: under a cap that holds both arms, the governor on top did no less than native, and at these caps neither arm answered more inside the line than the other beyond the noise. Where a cap holds native back and Omni-Compass answers more within it, the row reads GAIN; on this cluster at these caps it did not, and the table says so.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

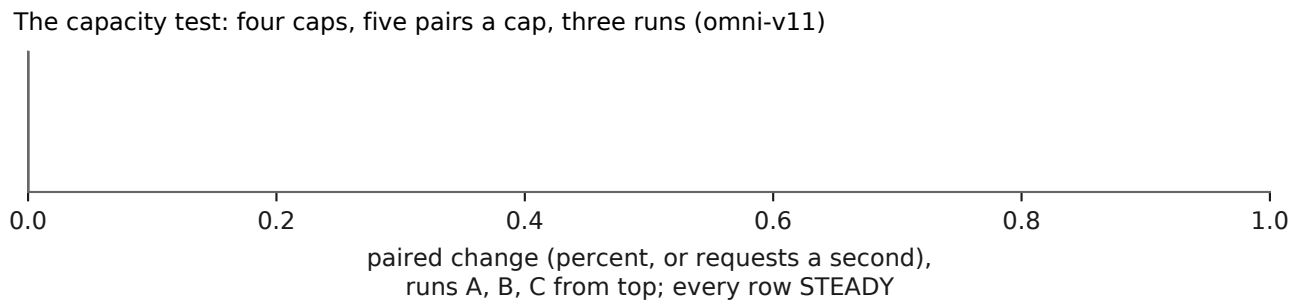


Figure 67. Receipt chart: the capacity test at four caps (omni-v11), native and Omni-Compass at the same resource cap of 1, 1.5, 2 and 3 cores, five pairs a cap, three runs. The completed work and the work inside the line, as the paired change with its 95% interval in runs A, B and C: every row STEADY, no difference beyond the noise, as results/live/V11_CAPACITY.md reads.

How a receipt comes to be. Every one of these tables carries the governor's inputs beside the service's outputs, and a referee reads them together. The inputs are the settings commands the governor wrote on its own, the host's busy share and the utilisation and CPU it raised, and its own CPU (seven to twelve thousandths of a core, shown); the outputs are the response time at its mean, 95th and 99th percentile, the time over the line and the alert minutes, and the energy of the declared idle-power model. Where the outputs fell and the energy rose, the rise is the power the faster answers took, read as power given; where the governor wrote nothing, the outputs read STEADY. The Goldilocks table drawn that way is the mechanism in one picture.

60.3 Omni-Compass direct: the three arms, three runs (omni-v18)

The founder's order of 23 October: native, Omni-Compass on native and Omni-Compass direct, always shown together. The test is Omni-Compass's own orchestrator (`omni_orchestrator/`, `docs/OMNI_ORCHESTRATOR.md`): native is Kubernetes' published autoscaling rule on the muscles, written by us as the yardstick (brain `yardstick`); Omni-Compass on native is Omni-Compass on top of that rule (brain `omni_native`); Omni-Compass direct is Omni-Compass alone on the muscles, no Kubernetes (brain `omni`). Three runs on GitHub's own machines, A, B and C from one commit (`0ca2bbdac97f`, engine omni-v18, runs 40023524338, 40023529155 and 40023534013), the full form: 18 steps of 25 s, the clients stepping 1 2 3 2 3 4 …, the busy room rule; each pair read by the one key, paired by step (`results/orchestrator/V18_THREE_ARMS_ABC.md`, class L):

Pair	Gauge	Three runs	A	B	C
Omni on native against native	Work (requests a step)	**GAIN in all three of our runs**	GAIN, up 5.2%	GAIN, up 5.3%	GAIN, up 7.4%
Omni on native against native	The 95th percentile (ms)	**GAIN in all three of our runs**	GAIN, down 9.0%	GAIN, down 12.1%	GAIN, down 7.5%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

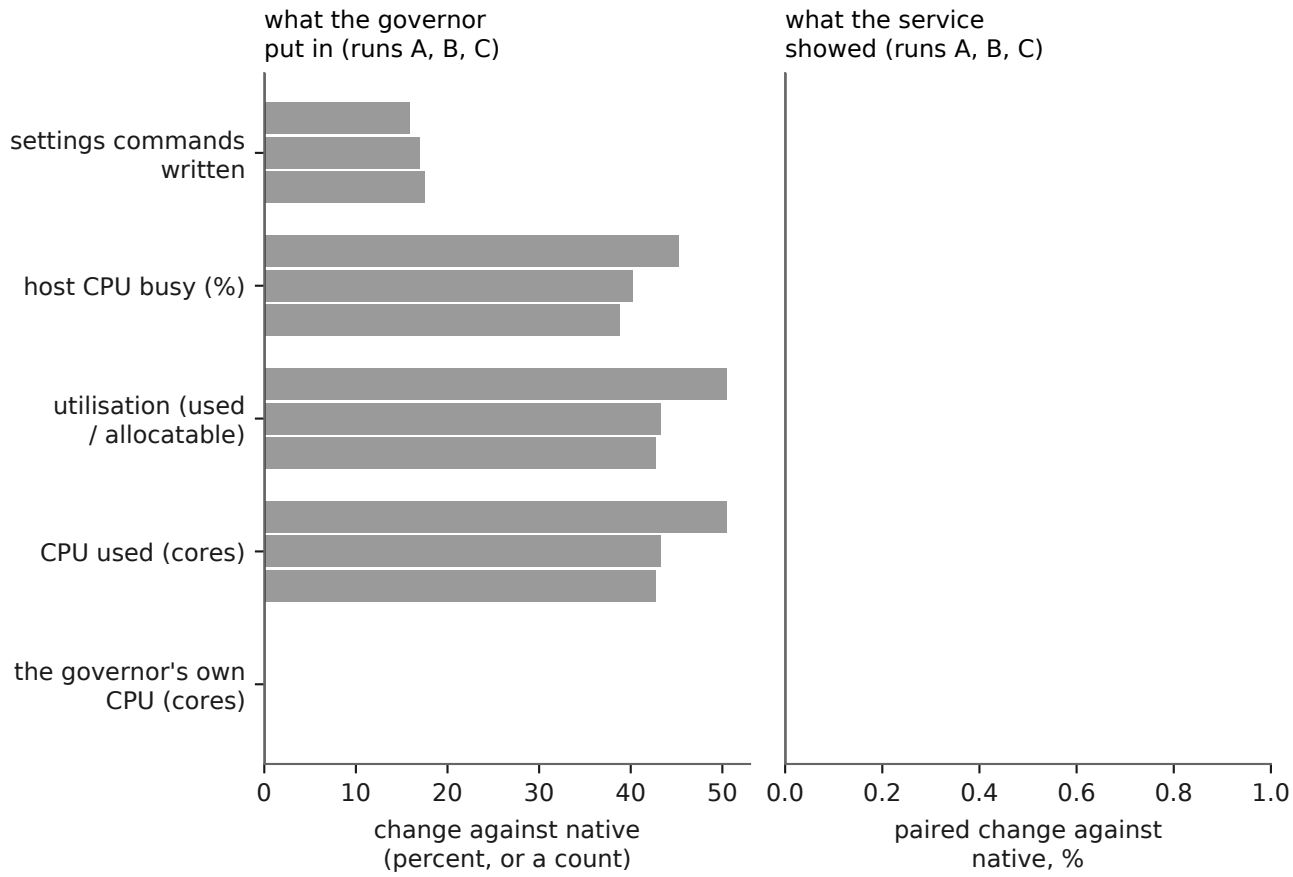


Figure 68. How a receipt came to be: the governor's inputs beside the service's outputs, from the Goldilocks table (runs A, B, C). Left, what the governor put in: the settings commands it wrote on its own, the host's busy share, the utilisation and the CPU used it raised, and its own CPU. Right, what the service showed: the response time at its mean, 95th and 99th percentile, the time over the line and the alert minutes, all down, and the energy of the idle-power model, up and read as power given.

Pair	Gauge	Three runs	A	B	C
Omni on native against native	Machines	**GAIN by the mean of our three runs, beyond the noise in 1 of 3**	GAIN, down 2.2%	GAIN, down 4.6%	GAIN, down 2.4%
Omni on native against native	Units	**power given in all three of our runs**	power given, up 11.0%	power given, up 8.9%	power given, up 10.8%
Omni direct against native	Work (requests a step)	**GAIN in all three of our runs**	GAIN, up 7.3%	GAIN, up 5.7%	GAIN, up 9.4%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Pair	Gauge	Three runs	A	B	C
Omni direct against native	The 95th percentile (ms)	**GAIN in all three of our runs**	GAIN, down 9.8%	GAIN, down 15.7%	GAIN, down 10.8%
Omni direct against native	Machines	**GAIN by the mean of our three runs, beyond the noise in 0 of 3**	GAIN, down 2.2%	GAIN, down 3.1%	GAIN, down 2.2%
Omni direct against native	Units	**power given by the mean of our three runs, beyond the noise in 2 of 3**	power given, up 6.9%	STEADY, down 0.3%	power given, up 9.0%
Omni direct against Omni on native	Work (requests a step)	**GAIN by the mean of our three runs, beyond the noise in 1 of 3**	GAIN, up 2.0%	STEADY, up 0.4%	GAIN, up 1.9%
Omni direct against Omni on native	The 95th percentile (ms)	**GAIN by the mean of our three runs, beyond the noise in 0 of 3**	GAIN, down 0.9%	GAIN, down 4.0%	GAIN, down 3.6%
Omni direct against Omni on native	Machines	**power given by the mean of our three runs, beyond the noise in 0 of 3**	STEADY, unchanged	power given, up 1.6%	STEADY, up 0.2%
Omni direct against Omni on native	Units	**GAIN by the mean of our three runs, beyond the noise in 1 of 3**	GAIN, down 3.7%	GAIN, down 8.5%	GAIN, down 1.7%

No request failed and none was answered wrong in any arm of any run (9,128 to 11,839 requests answered an arm). The means of the three runs, which the bottom line carries: Omni-Compass on native against native, work +6.0%, the 95th percentile +9.5% faster, machines +3.1% fewer, serving units +10.2% more; Omni-Compass direct against native, work +7.5%, the 95th percentile +12.1% faster, machines +2.5% fewer, serving units +5.2% more; Omni-Compass direct against Omni-Compass on native, work +1.4%, the 95th percentile +2.9% faster, machines +0.6% more, serving units +4.6% fewer. No pair reads LOSS on any gauge. The test runs again on omni-v23, the engine that stands. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

60.4 The organisms with the real cluster inside, and the big organisms

The six modelled organisms were run with the kind cluster inside as one more muscle at 1, 10 and 100 copies on GitHub (`V1_SIX_KUBE.md`, `V3_SIX_KUBE.md`) and at 1,000 copies on a rented Azure machine (`V1_BIG_ORGANISM.md`). At 10 and 100 copies on v3, every one of the twelve cells read GAIN on four to six gauges (response time, time over the line, the organism's energy and its time over its own line) and LOSS

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

on none beyond the noise, except a rounding-level work loss of six to eleven parts in a million in four cells, which the table shows. The four stacked at 100 copies ran off the clock in four of five repetitions (up to 277 s past a 1,440 s window): the machine could not step 170,000 muscles inside the window, both arms slipped alike, the pairing stands and the mark stays on the cell. At 1,000 copies on v1 the tower answered its slowest 5% in 0.2 s against native's 4.1 s and was over its line 0.2% of the time against 64%; the four stacked read p95 −74% and p99 −83% with the compass arm 300 to 615 s past its 2,880 s window in all three repetitions, marked off the clock, which is why the v3 stack was run with a 10,800 s window. **On v3 the four stacked at 1,000 copies** (`V3_BIG_ORGANISM.md`: 1.7 million muscles on one clock with the real cluster inside, three paired repetitions on one rented machine over 29 hours, 240 steps of 45 s an arm) **kept the clock in both arms**, 22 s behind a 10,800 s window (0.2%), and read: the cluster's slowest 5% answered in 150 ms against native's 2,882 ms (−95%), its slowest 1% in 165 ms against 5,405 ms (−97%), time over the response line 51% of samples to none, failed requests 0.14% to none, all clear of zero with three pairs; replicas, pods started and machines (six in both arms) the same; the standby-model energy −0.6% inside the noise; the host's CPU −1%, shown. The organism's own model (class S, shown beside the cluster and never inside the index) read energy −0.4% and work per energy +0.4% higher, time over its own line −0.7% lower, and work −0.0006%, a rounding-level model difference that the rule reads as LOSS and the table shows as such. The v1 stack's loss, the compass arm falling off the clock, is gone with the longer window; the gain, the cluster's tail, is the same shape at 1,000 copies as at 10 and 100. **The whole tower at 1,000 copies** (945,000 modelled muscles; started by run 37756284680, collected by run 37945866426 on 9 October after 25 hours on a second machine of the same type, commit `6f0f93e5`, Omni v3) read the same way and harder: the cluster's p95 6,690 → 160 ms (−98%), p99 9,068 → 187 ms (−98%), time over the response line 76% of samples to none, failed requests 4.8% to none, all clear of zero with three pairs; replicas (9.97 in both arms), pods started (4 against 3.7) and machines (6) the same or inside the noise; the standby-model energy −0.3% inside the noise; CPU with the governor's own −2.5% and the host −1.2%, shown; both arms 14 s behind the 10,800 s window (0.13%), on the clock in every repetition; the organism's own model energy −0.4% lower and work −0.0004%, the rounding-level loss the rule reads as LOSS. Native's own tail was longer under the tower than under the stack (6.7 s against 2.9 s at the 95th percentile) because that engine's tower (945 distinct muscles on omni-v3; 1,107 on the catalog that stands) drives the load generator harder at its peaks, and the governor's answer was the same 150 to 160 ms under both.

60.5 The bill on a real cloud

Azure Kubernetes Service with Azure's own cluster autoscaler as native (`V1_AKS_STEADY.md`, `V1_AKS_BURST.md`): five paired repetitions each on a 4-worker fleet, every machine billed every 15 s at list price, a fresh cluster per arm. No gauge cleared the noise in that one run; the bill read −4.7% on steady load and +5.0% on the burst, both with intervals across zero; the burst's p99 read −34% clear of the noise in that one run. This is the honest reading of a fleet in which one machine is a quarter of the fleet, so that only a saving of about 30% could clear the noise against an expected saving of a few percent (section 46.1). The fleet of 40 workers in nine machine families that can show one machine is preregistered (`docs/K8S_COMPASS_PREREGISTRATION.md`, amendments 1 to 4) and has been refused by Azure's

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

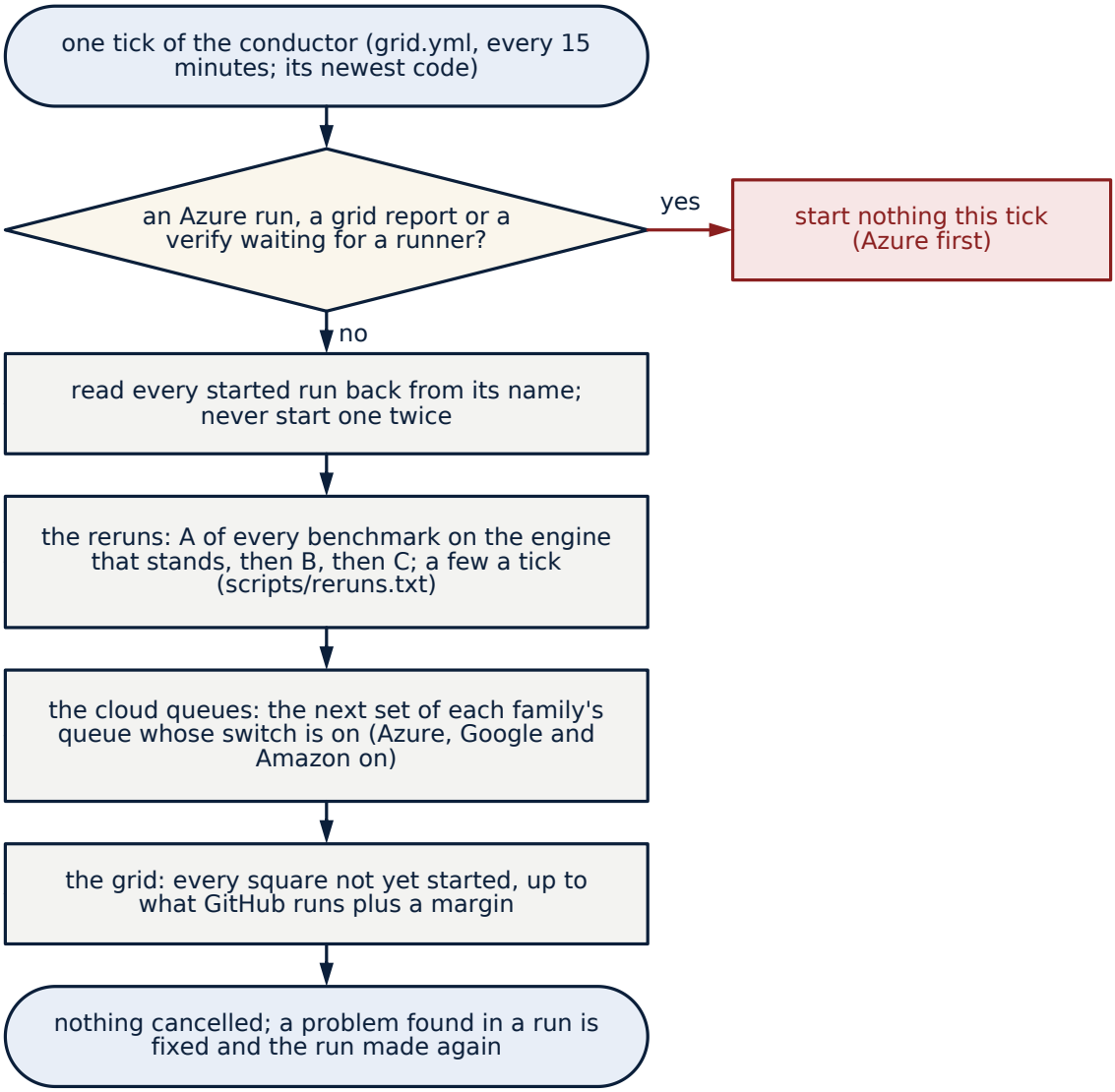


Figure 69. Flowchart: one tick of the grid conductor (grid.yml, every 15 minutes, on its newest code). It starts nothing while an Azure run, a grid report or a verify waits for a runner; it reads every started run back from its name and never starts one twice; it releases the reruns of every benchmark on the engine that stands (A of every benchmark, then B, then C), the next set of each cloud queue whose switch is on, and every grid square not yet started, up to what GitHub runs plus a margin. Nothing is cancelled.

own cluster capacity in its region on every dispatch so far, each refusal recorded; it runs the moment the region admits a cluster.

The refusals are themselves part of the record, and the preregistration's amendments 5 to 7 keep the count: six dispatches and 29 refused repetitions in eastus, each with Azure's own reason (its managed-Kubernetes capacity in the region, not the subscription's quota), a survey of every other region showing none with the vCPU quota the fleet needs, one dispatch cancelled by hand and recorded as such, and the seventh dispatch left to try. That seventh dispatch was refused on all five of its repetitions over the morning of 8 October, 34

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

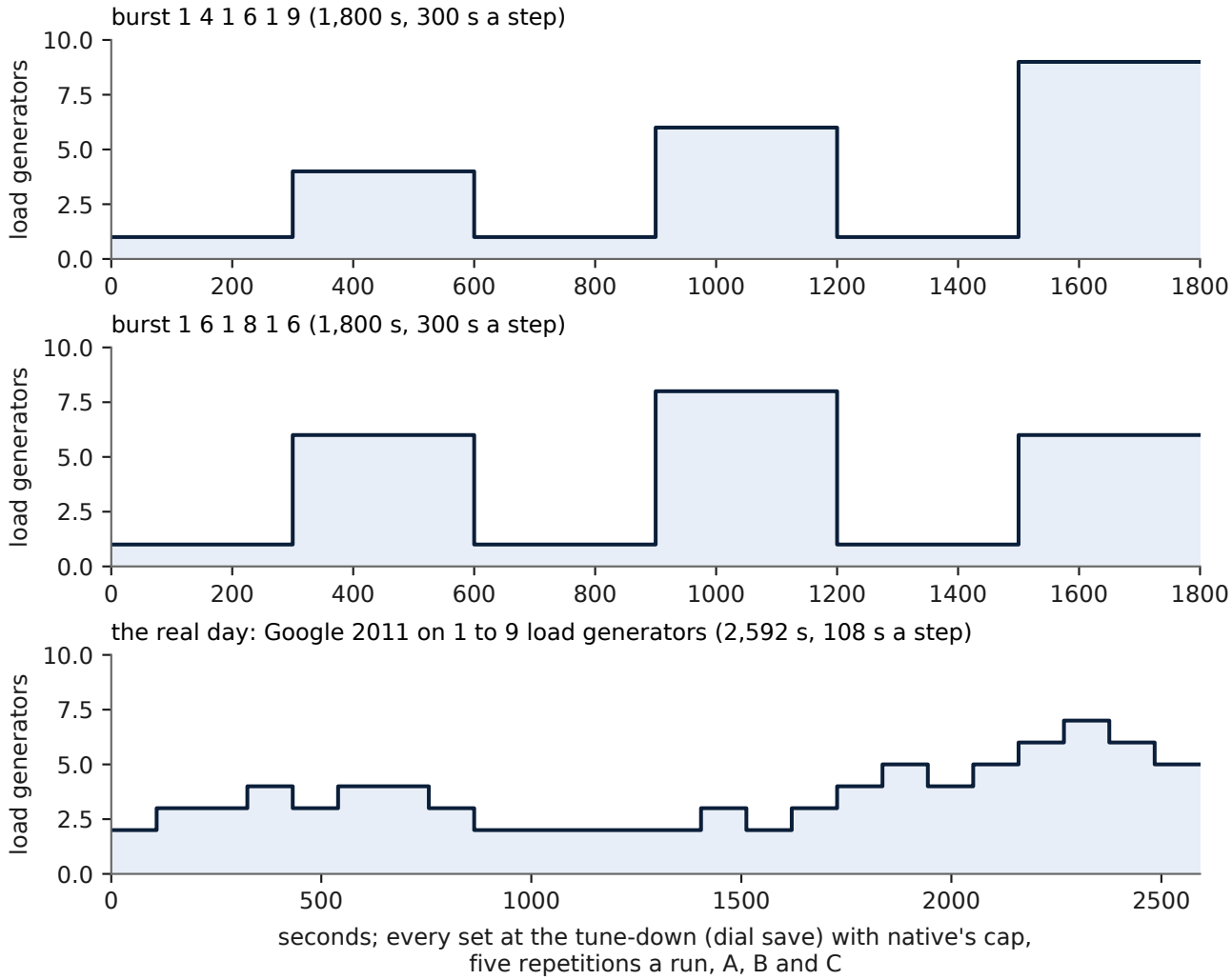


Figure 70. The load schedules of the cloud sets: the two bursts (1 4 1 6 1 9 and 1 6 1 8 1 6, 300 s a step) and the real day (the Google 2011 trace on one to nine load generators, 108 s a step). Every set runs at the tune-down with native's cap, five repetitions a run, as runs A, B and C; the load generator alone reads the schedule, never the governor.

refusals in all (amendment 8); whatever the region admits on a later dispatch is read by the same table as the four-worker runs, with the refused repetitions counted in the amendment, not in the table. A reader should take from this what it says and no more: the small fleet's bill reading is inside the noise by the arithmetic of its size, the large fleet's reading does not exist yet, and nothing about the bill on a real cloud is claimed in the index until it does.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

60.6 The cloud is a wire: three places, one organism, two routes (a composition, class S of L and P)

What the record shows in each place so far (the words for each place, and what each benchmark must show, are written once and carried word for word in this manual, the README and `DISCLOSURES.md`, section 10): at the software end the benefit, the Omni index over work, speed and machines (`results/OMNI_INDEX.md`; composed layer by layer in `results/WHOLE_SYSTEM.md`), with no bill in it; on bare metal, the A10's own bill and meter on omni-v23 (`results/TWO_BILLS.md`, `results/gpu/V23_A10_CAP_FULL_ABC.md`), fully loaded: the bill STEADY, the work done inside it the runs disagree (up in two runs, down in the third, that run's lost work on the next version's list), the chip's energy for each request served GAIN in all three of our runs (the A100's three runs on omni-v21 read work and energy STEADY, `results/gpu/V21_A100_CAP_ABC.md`); through the cloud, Azure's steady set on omni-v18, 98 paired repetitions (`results/live/AKS_STEADY_100_OMNI_V18.md`) reads the bill up 1.2%, inside the noise, and the 99th percentile GAIN: it holds. What the benefit cost the chip, the energy and the wire is listed row by row in section 3b of `results/WHOLE_SYSTEM.md`. In the cloud the yardstick is the work per dollar: the work answered within the line divided by the bill, against native; a bill that rises is power given beside the work it bought, and the work must rise more.

A cloud is just a wire (the founder's words of 24 October: "a big bold statement", carried in every manual). It is the wire between the software end (the owner's programs, Kubernetes and the open-source orchestrators, managers and autoscalers that carry the work) and the hardware end (the bare metal: the GPU chips, the Linux racks, the data center, on site or off site). Omni-Compass can live in all three places. The wire makes no benefit of its own. Until 31 October every cloud run in this repository was set on the wire alone (the tune-down on the machines only, the pods exactly native's), so it could only park machines the cloud's own autoscaler already made, after its own trial, and show at most a small cut in the bill of the wire. From 31 October (the founder's order of that day) every cloud set runs the whole engine at the max on the cloud's cluster, the software end, and what its receipt must show is that the engine holds there: no judged gauge LOSS against native, the work per dollar read beside the bill (the founder's words of 31 October, night: "We just need to show we hold"). The benefit is shown where it lives, at the software end, and explained through the wire. The energy and the throughput gained at the bare metal are the provider's and not under it; they are measured where Omni-Compass sits on the chip and composed with the wire into one organism and one bill by the product law $R(g) = \prod_\ell R_\ell(g)$, $\log R = \Sigma_\ell \log R_\ell$, the variances added (`tools/whole_system.py`, `results/WHOLE_SYSTEM.md`; regrouped into places and routes by `docs/engine/export_whole.py`). A provider that ran Omni-Compass on its own back end would see that benefit on its own chips and racks; no tenant can meter it, so it is never claimed here as a measurement. No run has stacked the layers: each factor is one layer's own run with the layers under it native, and the product assumes a layer's ratio does not change when the layers around it also carry Omni-Compass (tested on the governors' four ways: 18 of 30 pairs agree within the noise). The stacked run that replaces the composition is preregistered (`docs/WHOLE_SYSTEM_PREREGISTRATION.md`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Place	Measured so far (every interval carried, without the card's old run)	The company's expectation (the founder's figures of 24 October: an expectation, never a result)
The software end: the cluster (class L, 22 tests) and the services on it (class L, 17 tests)	the index **GAIN 18.5%**: work **GAIN 24.9%**, speed **GAIN 41.3%**, machines **GAIN 8.5%**, energy **GAIN 2.8%**	about 3 to 4%
The bare metal: the card and the machine	not measured in the headline scope: the machine's own layer (the kernel, Kepler, the host's meter) is not yet run; the card's one run (class P, 2 October, on a controller since replaced) read work per energy **GAIN 3.6%**, the energy drawn cut 3.5% (GAIN) and the 95th percentile 58.5% slower (LOSS)	2 to 3%
The cloud's wire alone: the machines it bills (Azure, class L, three one-run tests of 4 and 5 paired repetitions)	speed **STEADY** 4.4% (−21.5% to +38.8%), machines **STEADY** 2.4% (−10.0% to +5.8%), the bill **STEADY** 0.8% (−18.8% to +21.2%)	1 to 2%
The two ends together	see the routes below	about 5%

Where a measurement and the expectation differ, the measurement stands.

Route	Light	The whole workload against native (its index; the gauges correlated)	Gauge by gauge
On site: the software end × the bare metal	green, GAINING	**GAIN 18.5%** (+16.1% to +20.9%)	work **GAIN 24.9%**; speed **GAIN 41.3%**; machines **GAIN 8.5%**; energy **GAIN 2.8%**
Through the cloud: the software end × the cloud's wire × the bare metal at the back end	green, GAINING	**GAIN 14.8%** (+1.6% to +29.6%)	work **GAIN 24.9%**; speed **GAIN 47.5%**; machines **STEADY** 5.9% (−2.3% to +14.9%); energy **GAIN 2.8%**;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Route	Light	The whole workload against native (its index; the gauges correlated)	Gauge by gauge
			the bill **STEADY** 0.8% (−18.8% to +21.2%)

By the index's rule (only what held the same way in all three of our runs enters) the same routes read **+18.5%** and **+14.5%** GAIN (gauges correlated +13.3% to +15.8%), the second the repository's headline: speed +45.3%, machines +5.7%, energy +3.1%, work +24.6%, the bill +0.00%. With the card's one old run carried, its speed LOSS pulls the through-the-cloud index to +5.4%. The bare metal enters both routes as no change until it is measured, so both routes today are the software end's and, through the cloud, the wire's. Neither is a bill in money (section 76.8): only the cloud's compute is metered in money, and its one-run reading is the STEADY above. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

60.7 A real database, a real message broker, a real cache

Five stacks whose knob is a resource an operator sizes once and leaves, run as one shape of test on the engine that stands, omni-v11: three paired repetitions a run, three separate runs (A, B and C, released by the conductor on 18 and 19 October), the operator's fixed setting as native, the compass on that setting through the stack's own console as omni, a tuning workload shown and not counted, and the untouched workloads judged. MySQL, MongoDB and Redis run a second set beside the first with native exactly as the vendor ships it, every other rule unchanged (each preregistration's amendment of 17 October): where the factory setting is already at the knob's top, the knob is at its limit on native and Omni-Compass watches. On the engine that stands every move on these knobs passes the brain's verdict first (section 45.4): a give-back or a spend is written only after its paired trial on the stack itself reads no LOSS, so a knob that cannot prove it pays stays at the operator's setting, and a table that reads STEADY on every row is the do-no-harm reading, not a missing one. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

- **PostgreSQL 16 behind PgBouncer** (`V11_PGBENCH.md`; runs 39149254727, 39157791268, 39166378754): the pooler's pool size on `select`, `simple_update` and `tpcb_hot`. 35 judged rows read STEADY and one power given: on `simple_update` the most server connections open at once rose 8% to 10% in all three runs while the mean held open, the work inside the line, the latency and the host's CPU stayed inside the noise. Nothing measurable was bought with it, and the table shows it. Every pool was handed back. **At the max** (`V11_PGBENCH_SERVICE.md`; runs 39182915425, 39184166001, 39185403353): the server connections held open fell 18% to 22% on `select` and 10% to 16% on `tpcb_hot`, GAIN in all three of our runs; the other 34 judged rows STEADY.
- **MySQL 8.0 under sysbench** (`V11_SYSBENCH.md`; runs 39149259396, 39157795408, 39166382450): the InnoDB buffer pool on four untouched workloads. 46 judged rows STEADY and two where the runs disagree: on `update_index` the throughput (transactions and queries a second, one measure counted

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

twice) moved −0.02%, +0.26% and −0.35%, two runs clear of zero in opposite directions, a machine-level effect under an offered rate that pins throughput near 98% of what is offered (`docs/LIMITATIONS.md`, section 2). Work, p95, p99 and CPU inside the noise on every other row; every pool handed back. **Native as shipped** (`V11_SYSBENCH_SHIPPED.md`; runs 39150486066, 39159004803, 39167578494): from the factory's 128 MB pool Omni-Compass raised the pool 5% to 9% on `read_only` and `read_write` and the pages in use 4% to 16% on three workloads, power given in all three of our runs, with work, latency and failures STEADY: at this load the larger pool bought nothing measurable, and the table says so. **At the max** (`V11_SYSBENCH_SERVICE.md`; runs 39182906799, 39184157413, 39184175254): all 48 judged rows STEADY.

- **MongoDB under YCSB** (`V11_YCSB.md`; runs 39149263618, 39158992412, 39167564949): the storage engine's cache on `b`, `c`, `f` and `burst`. 43 judged rows STEADY and one GAIN: on `f` the cache held fell 0.9% to 1.0% in all three runs, with work, latency and reads into the cache inside the noise. **At the max** (`V11_YCSB_SERVICE.md`; runs 39182911063, 39184161631, 39184179710): the cache held fell 0.4% to 0.8% on `b` and 0.6% to 0.8% on `f`, GAIN in all three of our runs; the other 42 judged rows STEADY. **Native as shipped** (`V11_YCSB_SHIPPED.md`; runs 39150495654, 39160201436, 39169863638): the server's own cache (about 7.5 GB) is above the cover's top, the knob is at its limit on native, and all 44 judged rows read STEADY. Every cache handed back.
- **Apache Kafka 3.9.1** (`V11_KAFKA.md`, objective resource; `V11_KAFKA_SERVICE.md`, objective service; runs 39149249969, 39157787056, 39166374877 and 39150481307, 39159000835, 39167574190): the consumer group's size. Under both objectives the consumers running fell 0.8% on `burst`, GAIN in all three of our runs, every other judged row STEADY (the tuning workload, shown and not counted, read −0.3% under the service objective). Work inside the line, throughput, latency, lag and CPU read STEADY under both; no message lost; every group handed back.
- **Redis 7.0** (`V11_REDIS.md`, objective resource; `V11_REDIS_SERVICE.md`, objective service; `V11_REDIS_SHIPPED.md`, native as shipped; runs 39149244903, 39157782849, 39166371110, then 39149267883, 39158996767, 39167569649, then 39150490855, 39159008310, 39167582853): the cache's memory ceiling. Under the resource objective, on `large` the work inside the 2 ms line and the hit rate rose 1.4% to 1.6%, and on `burst` the ceiling fell 2.3% to 2.6% and the memory used 0.9% to 1.3%, all GAIN in all three of our runs; `small` read STEADY. Under the service objective, on `large` the work inside the line and the hit rate rose 1.4% to 1.6% and the mean latency fell 6.3% to 7.3%; on `small` the work and the hit rate rose 1.3% and the mean latency fell 1.7% to 2.0%: all GAIN in all three of our runs, for a ceiling 2.2% to 2.6% higher and 1.7% to 1.9% more memory used on `small`, **power given in all three of our runs**. With native exactly as shipped (no ceiling, `maxmemory 0`) the knob is at its limit on native: Omni-Compass wrote nothing, and every row reads STEADY but one, a memory rise of 0.01% on `small`, power given. The 99th percentile on `small` reads about 6 µs higher (0.1%) in single runs of the two operator's sets and STEADY by the rule: the harness runs the brain inside the load generator's own process, and a controlled test shows that presence alone lengthens the timed requests (`docs/TAIL_LATENCY.md`, section 4); the next version's harness runs it apart, as a deployment does. No request failed; every ceiling handed back.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

60.8 Robustness: the governor killed outright, the long run, its own cost

The robustness benchmark (`docs/ROBUSTNESS_PREREGISTRATION.md`) is a test of Omni-Compass itself, not of a gain. Both of its tests ran three times on omni-v11, on the kind rig that pulls the served image onto every worker before each arm; the runs of each test made before that rig are kept in the record and not counted, and they read no LOSS either.

The kill test on omni-v11 (`V11_ROBUST_KILL.md`; runs 39173416105, 39177935888, 39182898753; ten pairs of 900 s an arm in each run). The governor is killed outright at 40% of the window; the watchdog hands every setting back to the operator's; a second governor starts at 50% and governs to the end. In all 30 repetitions every setting was back at the operator's 8 to 11 s after the kill (the allowance is 60 s), and the second governor governed to the end in every one. In the 120 s after the kill, failed requests and the time over the line read STEADY against native in all three runs. Over the whole window the mean response time fell 29% to 44%, the 95th percentile 39% to 58%, the time over the line 30% to 31%, failed requests 7% to 12% and the alert minutes 16% to 23%, GAIN in all three of our runs; HPA replicas rose 3% to 6%, power given; machines the same. The 99th percentile reads STEADY: it rose 25% in run A and fell 25% to 27% in B and C. In nine of run A's ten repetitions both arms' 99th percentile is the probe's 10-second timeout; the rise is one repetition's, where the omni arm's slowest answers reached 9.2 s at the load's peak, 140 to 190 s after the kill, with every setting already back at native's and both arms at the same ten replicas, while the runner's processor was 100% busy for 70 s. The second governor had started 50 s earlier on the same four cores; that is the likely cause and is not proven (`docs/TAIL_LATENCY.md`, section 3).

The long run on omni-v11 (`V11_ROBUST_LONG.md`; runs 39173420146, 39177939062, 39182902844; three pairs of 7,200 s an arm in each run) asks whether the governor drifts, leaks or slows over hours. Its resident memory grew at most 2.3% (no leak: the limit is a quarter); it made 97.5% of the decisions its interval predicts in every repetition (valid: the limit is 95%); no decision failed; its decision time in the last hour was at most 1.13 times the first hour's (no growth: the limit is 1.5); and every setting was handed back at the end of every repetition and read back at the operator's. The mean response time fell 34% to 36%, the time over the line 32% to 35% and the alert minutes 14% to 19%, GAIN in all three of our runs; HPA replicas rose 49% to 52%, power given; the other service rows read STEADY (three pairs give wide intervals, as the preregistration says they would); machines the same. The earlier engines' robustness tables (`V10_ROBUST_LONG.md`, `V3_ROBUST_KILL.md`) stay their own record.

The governor's own CPU is shown in every Kubernetes table of sections 60.1 and 60.2: 0.007 to 0.012 of one core on average (all four in one run 0.0081 to 0.0100, faults 0.0094 to 0.0101, the Google day 0.0092 to 0.0098; on omni-v11 the long run 0.0104 to 0.0108 and the kill test 0.0126 to 0.0133, a second governor starting inside its window), shown and not judged.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

60.9 The modelled realms and the independent simulators

Every result here is evidence class S: a statement about a published model under its own native controller, never about hardware, and never counted in the Omni index. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

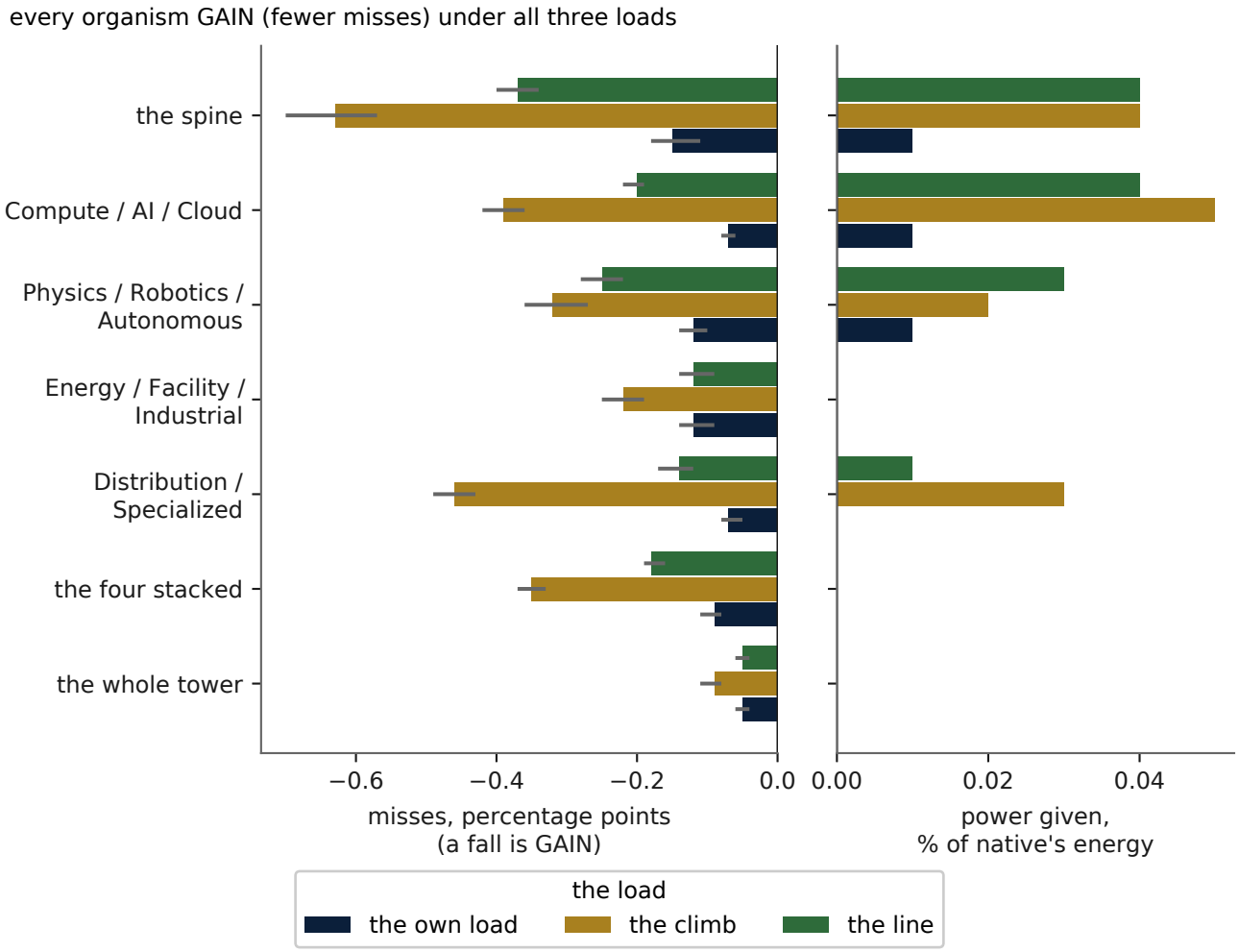


Figure 71. Receipt chart: the seven organisms under the three loads (the own load, the climb and the line), from results/realms_v5/REALMS_V5.md. Left, the change in misses in percentage points with its 95% interval, a fall read GAIN; right, the power given as a share of native's energy. Every organism reads GAIN (fewer misses) under all three loads; the organisms are class S, our own models.

- **The 1,107 muscles and the seven organisms** (`results/realms_v5/REALMS_V5.md`, run again on omni-v8 on 17 October, every reading the same to the digit): every muscle alone over its loads, 527 GAIN and 580 STEADY, none LOSS; every organism, the spine first, GAIN (fewer misses) under its own load, the climb and the line, the energy the max took shown as power given. The three GitHub runs of 16 October (38931342073, 38941579484, 38953616422) reproduced the committed result exactly.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The power grid** (`V11_PANDAPOWER.md`; runs 39151675293, 39160210556, 39169871088): eleven SimBench grids solved by pandapower, the tap changer's own controller as native, one whole tap per move as the knob. With ZIP loads the energy the loads drew and the net import fell in all eleven grids, the losses in lines and transformers fell in seven and rose in four (the rural and semi-urban grids with their own generation), **power given in all three of our runs**, and tap operations fell in ten and rose in one (power given, the declared wear); with constant-power loads the net import fell in seven and rose in the same four; the share of bus-steps outside the voltage band fell in two grids and was the same in nine. 67 gauge-rows GAIN, 14 power given, no LOSS.
- **Robot arms** (`V11_MUJOCO.md`; runs 39150499798, 39160206095, 39169867462): MuJoCo Menagerie arms with their shipped servos as native and the speed override of an axis with slack as the knob, after a paired physics trial. On the Kinova Gen3 the peak joint torque fell 28.6%, the tracking error 21.3%, the end-point error 11.0%, the copper heating 11.3%, the mechanical work 6.6% and the energy per takt 0.8%, GAIN in all three of our runs; the trial left the KUKA iiwa 14 and the UR5e native (a slower cycle no cheaper on the robot's own figures).
- **Drone swarms** (`V11_SWARM.md`; runs 39151682521, 39160282043, 39169877937): twenty Crazyflie 2.x quadrotors in gym-pybullet-drones, the shipped position controller as native, the cruise override inside the autopilot's limits as the knob. Energy a mission fell 7.4% (short), 17.8% (mixed) and 19.8% (long) and missions a charge rose 8.0%, 21.6% and 24.7%, GAIN in all three of our runs; no late mission, reserve breach, near miss or collision in either arm in the counted cells. Three gauges are shown and not judged (the closest approach, the tracking error inside its 0.25 m band, the cruise level), and they did not reproduce to one part in a thousand across GitHub's machines. The first of them carries a fault. The closest approach fell from native's 0.30 to 0.45 m to about 0.16 m in the short and mixed cells: the margin of the 0.30 m keep-out was spent to fly faster. On mission draws outside the counted cells the same wire let drones collide: on the mixed cell with draw 214, 3 collision ticks and 5 near-miss ticks (closest 0.083 m), where native flew the same draw with none (closest 0.207 m). The cause is in the wire, not the brain: its proximity reflex is written to return the planner's cruise at once, and the code lowered the speed by 0.01 a tick instead. The fix, the reflex at once and reaching out with the square of the speed as the stopping distance does, held every drone at or outside the full keep-out on every cell and draw tried, the untouched draws included, and kept most of the energy cut (`docs/LIMITATIONS.md`, section 3; `results/model_gate/V12_SWARM_REFLEX.md`). It is a guard, so it made the next engine version: it landed in omni-v12 (`docs/OMNI_V12.md`), whose swarm runs, under the preregistration's amendment 2, judge a collision with Omni-Compass on top as a LOSS and fly three mission draws no wire was ever run on.
- **Buildings with batteries** (`V11_CITYLEARN.md`; runs 39151678885, 39160214971, 39169874537): the eleven districts CityLearn 3.0.2 runs through this interface (eight more stop on CityLearn's own errors, listed in the table), its own rule-based control as native, each building's battery charge and discharge as the knob. In the four districts whose batteries moved, GAIN in all three of our runs:
  - the electricity bought fell 3.0% to 13.6%, the daily peak 3.2% to 16.9% and the ramping 3.4% to 10.1%;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- the daily unevenness fell 0.9% to 6.0% and the monthly unevenness 0.8% to 2.3%;
- the highest peak fell in two districts (2.7% and 4.4%);
- in the one district with a price, the 2022 robustness district, the bill fell 16.1% and the carbon 12.1%.

The distance from zero net energy fell in three districts and rose 0.02% in one, power given. The seven 2023 districts simulate power outages. Their batteries stay native, because the charge is also the building's reserve (omni-v8, after omni-v7 read LOSS there: energy not served +2.0% to +9.8%). Every score there reads STEADY, the same: the energy not served and the discomfort are native's to the last digit. Two rows read "the runs differ" on the time uncomfortable, though the paired change is exactly zero in every run: CityLearn's own value moved by at most 3.6×10^{-5} between runs, in both arms alike. In all, 27 score-rows GAIN, 1 power given and no LOSS. The wire's road since then: omni-v9 reads the coming hour, and omni-v10 glides at 0.0875 an hour (`docs/OMNI_V8.md` to `docs/OMNI_V10.md`); omni-v11 to omni-v23, the engine that stands, carry it unchanged.

60.10 The ten organisms two ways: the twenty scenarios (modelled, class S)

The ten organisms of section 34.7, each native and with Omni-Compass on native, on engine omni-v19, three runs (seeds 3000, 3001 and 3002, the same seed in both arms), under the climb for 240 decisions of 15 s, the brain going silent at 80% of every run and the lease walking every setting back to native (`results/scenarios/SCENARIOS_20.md`, every run's every gauge in `results/scenarios/SCENARIOS_20.json`; `python3 tools/scenarios_20.py`). Evidence class S: a model's energy is not a meter's, and a plant written by the people who wrote the law is not an independent test. Each organism is read gauge by gauge as the live benchmark engine's harness record reads it, the change per decision averaged over blocks of ten decisions with its 95% interval over the 24 blocks, by the key. The gauges: work (the work index, 1.0 native's own work; a rise is GAIN), muscles over the line (counted each decision; a fall is GAIN; the change is a share of native's own count), power (the organism's electrical draw; a fall is a cut, a rise power given). The record's table, every row (native's own gauges, the mean of runs A, B and C, then Omni-Compass on native against native, the word and the mean of the three runs' changes):

#	Organism	Muscles	Native (Omni-Compass off)	Omni-Compass on native, against native
1	The spine	267	work index 1.000; 21.38 muscles over the line a decision (8.01% of muscle-decisions); 3,418.4 kW (13,262.9 MJ)	work **STEADY** 0.00% · muscles over the line **GAIN** 8.36% · power **STEADY** 0.49%
2	Compute / AI / Cloud	540	work index 1.000; 29.80 muscles over the line a decision (5.52% of muscle-decisions); 3,890.7 kW (14,962.9 MJ)	work **STEADY** 0.00% · muscles over the line **GAIN** 7.41% · power **GAIN** 0.59%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Organism	Muscles	Native (Omni-Compass off)	Omni-Compass on native, against native
3	Physics / Robotics / Autonomous	404	work index 1.000; 38.22 muscles over the line a decision (9.46% of muscle-decisions); 4,811.8 kW (19,023.4 MJ)	work **STEADY** 0.00% · muscles over the line **GAIN** 3.89% · power **STEADY** 0.26%
4	Energy / Facility / Industrial	480	work index 1.000; 39.22 muscles over the line a decision (8.17% of muscle-decisions); 2,953,821.5 kW (10,659,898.0 MJ)	work **STEADY** 0.00% · muscles over the line **GAIN** 3.02% · power **STEADY** 0.00%
5	Distribution / Specialized	484	work index 1.000; 29.72 muscles over the line a decision (6.14% of muscle-decisions); 15,072.9 kW (55,218.8 MJ)	work **STEADY** 0.00% · muscles over the line **GAIN** 7.22% · power **STEADY** 0.17%
6	The four stacked	1,908	work index 1.000; 136.97 muscles over the line a decision (7.18% of muscle-decisions); 2,977,596.8 kW (10,749,103.1 MJ)	work **STEADY** 0.00% · muscles over the line **GAIN** 5.13% · power **STEADY** 0.00%
7	The whole tower	1,107	work index 1.000; 75.27 muscles over the line a decision (6.80% of muscle-decisions); 2,967,368.7 kW (10,709,412.1 MJ)	work **STEADY** 0.00% · muscles over the line **GAIN** 1.58% · power **STEADY** 0.00%
8	The orchestra	55	work index 1.000; 1.72 muscles over the line a decision (3.13% of muscle-decisions); 91.5 kW (329.4 MJ)	work **STEADY** 0.19% · muscles over the line **GAIN** 63.34% · power **power given** 52.38%
9	The live benchmark engine's organism	60	work index 1.000; 3.73 muscles over the line a decision (6.21% of muscle-decisions); 408,580.7 kW (1,471,946.9 MJ)	work **STEADY** 0.00% · muscles over the line **GAIN** 5.61% · power **STEADY** 0.00%
10	The whole organism, three sectors	layers	the reference of every factor	work **GAIN** 24.6% · speed **GAIN** 45.3% · machines **GAIN** 5.7% · energy **GAIN** 3.1% · bill **STEADY** 0.0%; the index **GAIN** 14.5% (+13.3% to +15.8%, the gauges perfectly correlated) (the composition, by the index's rule,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Organism	Muscles	Native (Omni-Compass off)	Omni-Compass on native, against native
				without the card)

Every GAIN and power given above holds in all three of our runs; every STEADY is under the floor or the same. **No row reads LOSS.** The record of 24 October also carried a third way, Omni-Compass by itself (each pool's own autoscaler switched off, `realms/harness.py` SOLO), which read LOSS in all three of our runs on work and on muscles over the line in eight organisms, 32 rows; the founder's order of 25 October took that way out, and its record, with every one of those rows and their cause, is kept whole in `docs/history/SCENARIOS_30_2026-10-24.md`. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

60.11 The one number, and the table

The one number. The Omni index over the real categories, judged across our three runs by the rule of 24 October, each test read from its own newest table with a reading (5 on omni-v16, 2 on omni-v11, 7 on omni-v10; no table is pooled across engines): **+3.4%, GAIN** (real Kubernetes **GAIN 28.6%** over its nine tests, the database's buffer pool **GAIN 1.6%**, real messaging and the real cache **power given 0.1%** each, the database's storage-engine cache **LOSS 1.4%**, the database behind its pooler **LOSS 4.9%**, each category weighed the same; Azure and the card join as their three-run tables land). The second reading, service alone (work and speed scored, machines and energy shown), stands at **+9.4%, GAIN** (Kubernetes +96.1% GAIN, the cache GAIN 0.4%, messaging STEADY, the buffer pool LOSS 0.00%, the storage-engine cache LOSS 1.2%, the database LOSS 12.0%). The benefit sheet reads the index's row GAIN AND LOSS. A category enters at whatever its tables show in all three of our runs, nothing included, which is the reader's guarantee that the number is not built from the categories that happened to work. The numbers of earlier engines are the road, in `docs/history`, and are never read beside these.

Result (the newest table of each test, three runs each)	Class	Reading	Source
Real Kubernetes, steady work in steps (omni-v10)	L	p95 −64% to −65%, p99 −69% to −73%, time over the line −98% to −100%, alert minutes −100%, GAIN; machines and energy STEADY	`results/live/V10_STEADY.md`
Real Kubernetes, demand that wanders (omni-v10)	L	p95 −57% to −66%, time over the line −38% to −45%, alert minutes −21% to −32%, energy −0.3% to −0.4%, GAIN	`results/live/V10_WANDERING.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

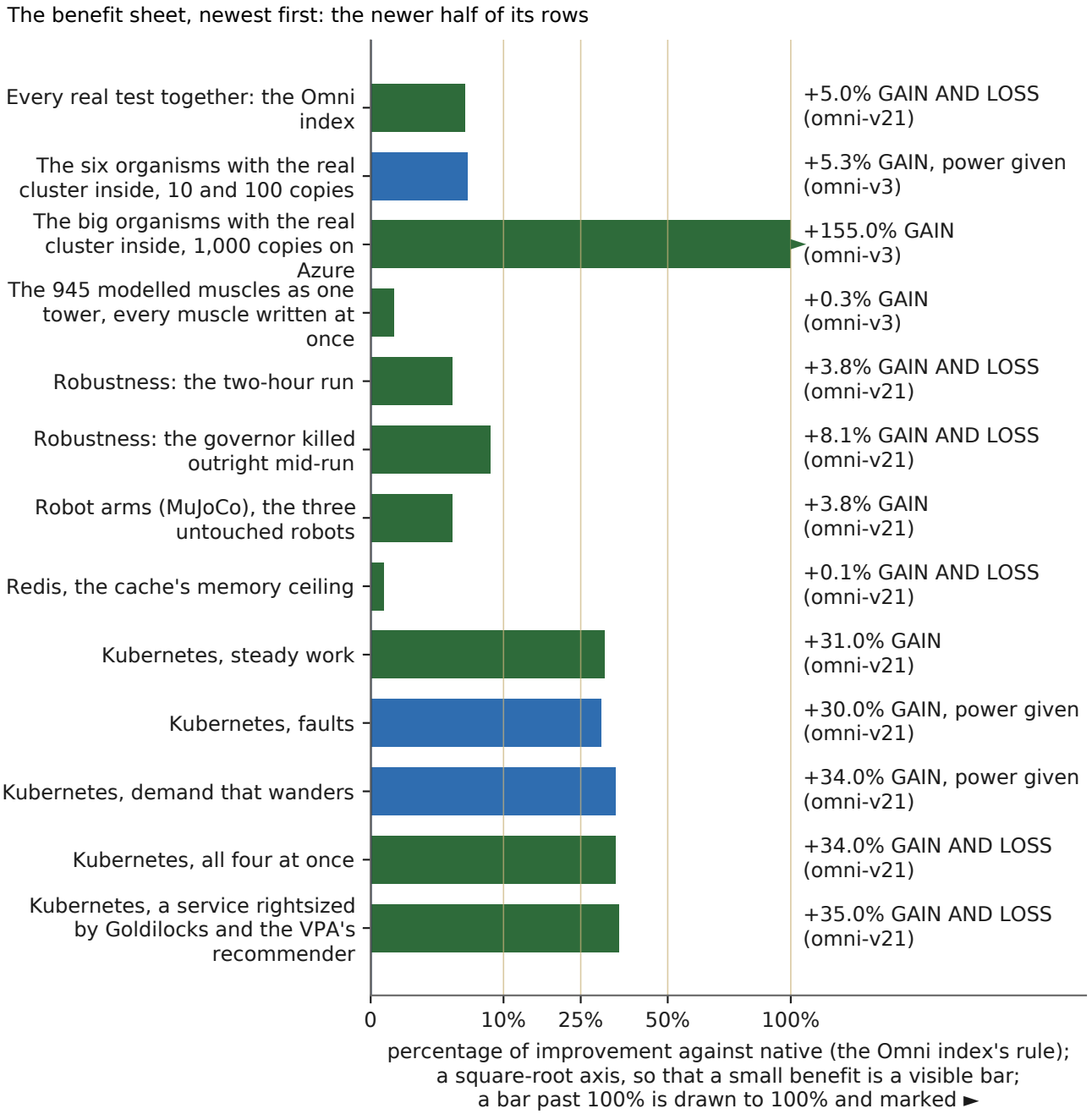


Figure 72. Chart: the benefit sheet, one number per benchmark, the newer half of its rows, from docs/BENEFIT_SHEET.md: the benefit by the Omni index's rule, its word, and the engine its table ran on; a result is never read across engines. Continued in the next figure.

Result (the newest table of each test, three runs each)	Class	Reading	Source
Real Kubernetes, all four in one run (omni-v10)	L	work inside the line +47% to +79%, p95 −56% to −61%,	`results/live/V10_ALL_FOUR.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

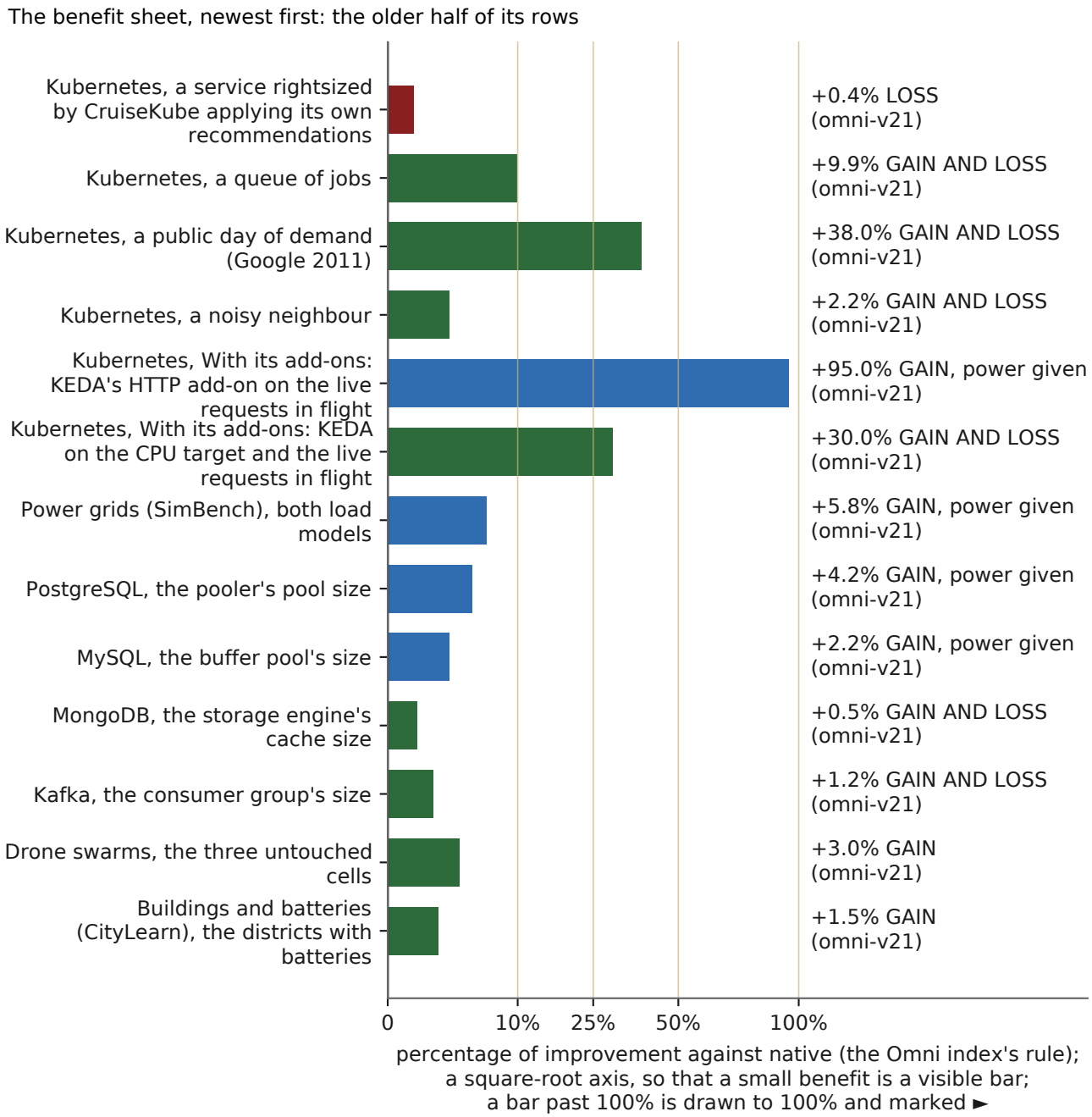


Figure 73. Chart: the benefit sheet, one number per benchmark, the older half of its rows, read as in the previous figure.

Result (the newest table of each test, three runs each)	Class	Reading	Source
		failed requests −9% to −14%, alert minutes −16% to −34%, all GAIN; machines the same	

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Result (the newest table of each test, three runs each)	Class	Reading	Source
Real Kubernetes, faults (omni-v10)	L	p95 −54% to −63%, p99 −43% to −51%, GAIN; replicas +14% to +20%, power given	`results/live/V10_FAULTS.md`
Real Kubernetes, a queue of batch jobs (omni-v10)	L	machines −6% to −14%, node-hours −6% to −14%, standby energy −4% to −10%, mean response −10% to −12%, GAIN; the queue finished STEADY	`results/live/V10_BATCH.md`
Real Kubernetes, a noisy neighbour (omni-v10)	L	time over the line −11%, GAIN; every other row STEADY, the neighbour's own included	`results/live/V10_FAIRNESS.md`
Real Kubernetes, a public day of demand, Google 2011 (omni-v10)	L	p95 −56% to −62%, time over the line −74% to −77%, failed requests −15% to −18%, alert minutes −61% to −66%, GAIN; pods started +120% to +193%, power given	`results/live/V10_TRACE_GOOGLE2011.md`
Real Kubernetes on Goldilocks, KEDA (two ways) and fixed capacity (omni-v11); on CruiseKube (omni-v10)	L	section 60.2	`results/live/V11_GOLDILOCKS.md`, `V11_KEDA_*.md`, `V11_CAPACITY.md`, `V10_CRUISEKUBE.md`
Robustness, the governor killed outright (omni-v11)	L	every setting back at the operator's 8 to 11 s after the kill in 30 of 30 repetitions; p95 −39% to −58%, failed requests −7% to −12%, alert minutes −16% to −23%, GAIN; replicas +3% to +6%, power given	`results/live/V11_ROBUST_KILL.md`
Robustness, the two-hour run (omni-v11)	L	no leak, decisions valid, no failed decision, every setting handed back; mean response −34% to −36%, GAIN; replicas +49% to +52%, power given	`results/live/V11_ROBUST_LONG.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Result (the newest table of each test, three runs each)	Class	Reading	Source
PostgreSQL, MySQL, MongoDB (omni-v11)	L	STEADY on every judged row but four: MongoDB's cache on `f` −0.9% to −1.0%, GAIN; PgBouncer's most connections open +8% to +10%, power given; MySQL's update throughput, the runs disagree; native as shipped: MySQL's pool +5% to +16%, power given, MongoDB STEADY; at the max: PgBouncer's connections held −10% to −22% and MongoDB's cache −0.4% to −0.8%, GAIN, MySQL STEADY	`results/live/V11_PGBENCH*.md`, `V11_SYSBENCH*.md`, `V11_YCSB*.md`
Kafka (omni-v11)	L	consumers −0.8% on `burst`, GAIN under both objectives; the rest STEADY	`results/live/V11_KAFKA.md`, `V11_KAFKA_SERVICE.md`
Redis (omni-v11)	L	work inside the line and hit rate +1.3% to +1.6%, mean latency −1.7% to −7.3%, GAIN; memory +1.7% to +1.9% on `small`, power given; native as shipped STEADY	`results/live/V11_REDIS.md`, `V11_REDIS_SERVICE.md`, `V11_REDIS_SHIPPED.md`
The 1,107 muscles and seven organisms	S	527 GAIN, 580 STEADY, none LOSS; every organism GAIN under three loads	`results/realms_v5/REALMS_V5.md`
Power grids (omni-v11)	S	67 gauge-rows GAIN, 14 power given, no LOSS	`results/live/V11_PANDAPOWER.md`
Robot arms (omni-v11)	S	the Gen3: torque −28.6%, tracking error −21.3%, GAIN; two arms left native	`results/live/V11_MUJOCO.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Result (the newest table of each test, three runs each)	Class	Reading	Source
Drone swarms (omni-v11)	S	energy a mission −7% to −20%, missions a charge +8% to +25%, GAIN; the closest approach spent to about 0.16 m (shown); collisions on a mission draw outside the counted cells, a fault in the wire fixed in omni-v12, its runs under way (section 60.9)	`results/live/V11_SWARM.md`
Buildings with batteries (omni-v11)	S	27 score-rows GAIN, 1 power given, no LOSS; the batteries of the outage districts native	`results/live/V11_CITYLEARN.md`

Running now and next. Every benchmark runs again on omni-v23, the engine that stands, released by the conductor (`scripts/dispatch_grid.py`, `scripts/reruns.txt`: A of every benchmark, then B, then C), the steady and wandering Kubernetes tests among them; Azure one run at a time in its region (the steady set, the bursts 1 4 1 6 1 9 and 1 6 1 8 1 6, the real day; A, B and C); the grid with the real cluster inside on GitHub's free runners. The runs of earlier engines already started run to their end and stand as their own engine's record. Google and Amazon are not yet run: their first sets, on omni-v10 on 18 October, stopped at the login at no cost, and since 24 October both queues are switched off and kept, wired as Azure's sets run (100 paired repetitions under the stopping rule, the 60 s beat, the $0 check first), each switched on by the founder's word (Google once its key is stored whole, Amazon once its account is on its Paid plan); the GPU chips wait for the founder's word; the CPU governor's measured run waits for a machine with a power meter.

60.12 Wire in, or watch: the verdict per knob

The founder's order of 9 October 2026 made a principle of what section 1.2 describes as a mechanism: Omni-Compass need not be wired into every muscle. Where it cannot beat the native controller on the muscle's own gauges, the muscle lives by itself and Omni only reads it, one wire out and no wire in. `tools/wiring_verdicts.py` applies that principle to every published result and prints the page `docs/WIRING_VERDICTS.md` (one row per knob and per muscle in `results/WIRING_VERDICTS.csv`; `verify.py` fails if the page differs from what the tables give). The rule is the same for every table: a knob reads **write** where at least one judged gauge is GAIN in all three of our runs and none is LOSS or power given in all three of our runs; **watch** where nothing is GAIN in all three of our runs, whether the rows are inside the noise, the runs disagree, or a gauge is LOSS or power given in all three of our runs with nothing against it (the LOSS is then named with its cause); and **operator's choice** where GAIN and LOSS (or power given) both hold in all three of our runs, a trade shown with both readings of the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

index. A modelled muscle takes its word from its realms label (`results/realms_v5`): a GAIN on work, speed or misses is write, the energy the max took for it power given and never a trade; SAME AS NATIVE and AT ITS LIMIT ON NATIVE are watch.

What the page says (written 16 October from the tables then standing; each row names its engine). On the real stacks, 10 of 26 knob-cases write, 2 are the operator's choice and 14 watch (the sets of 10 October on omni-v3 and the four v6 tables; each row names its engine and the v3 rows run again on v6). Of the independent simulators' 39 cases, 21 write, 15 are trades (the four SimBench grids that export, where the lower voltage costs line losses; the seven 2023 CityLearn districts, where the bill read power given) and 3 watch (two robots the engine itself left native, one grid where nothing was bought). Of the 1,107 modelled muscles on the v5 realms result, 527 write and 580 watch: 421 of the 580 are at their limit on native (every admission knob, 96 power knobs and 50 capacity and setpoint knobs that native already runs at the top of their range: nothing to push, read and left native) and 159 read the same as native (40 wrote and nothing moved beyond the noise, 119 wrote nothing). Every one of the seven organisms writes under every load, the power given shown beside its gain. Of the 14 organism cells with the real cluster inside, 12 write on the cluster's gauges, one is the operator's choice and one watches. Nothing on the page changes the engine, which stays the engine that stands (omni-v23); it changes what an operator connects.

The same verdict, live (built 9 October, running since 10 October). The page judges after the fact; the founder's order is that the brain judge in real time, on the knob, before it writes. On the five live stacks that is now in the harnesses (section 45.4; `tools/knob_verdict.py` around the engine's own verdict, the engine unchanged): every knob starts in watch and is written only inside the allowance a paired trial on the stack itself has earned under the declared objective, one notch a trial; a refused step is not taken, a trial holds the knob, a fail-up never spends beyond the allowance, the operator's setting is always free. The amendments are declared in the five preregistrations with the expectation written before the runs (Redis left native under the resource objective, the first notch refused; Kafka allowed step by step while each consumer pays; PostgreSQL's add above the operator's setting refused on `simple_update`; MySQL's `read_write` chunks refused; MongoDB's cache given back a notch a trial). The runs were dispatched on 10 October, three a stack and objective, in the day's rerun of every benchmark on one commit (`docs/RERUN_2026-10-10.md`), and the tables, the index and the page are then read again from them.

60.13 The second set of 10 October, and the four rules it taught the trial

The five live products ran a second time on 10 October on the amended trial rule (a spend trial runs to its samples; it is never ended by the calm it causes). The trials finished. The brain still refused the Kafka consumer almost every time, and in the one repetition where it allowed the third consumer early, the whole gain of the earlier law came back at a fraction of the spend: the 95th-percentile delay fell from 2,450 ms to 57 ms, the queue from 3,375 waiting messages to 253, the work inside the line rose 17%, with 2.52 consumers on average where the earlier law had spent 5.8 to 7.9. On Redis with the large working set the brain allowed 15 MB more memory and bought more hits, more work and less host CPU, a row the earlier law never showed

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

because it spent 200 MB; with the small working set nothing changed. PostgreSQL gave back about a tenth of its pool with the service inside the noise.

The Kafka reading is the finding. The gain is real and cheap; the brain's test of it is unreliable, and the fault is in how the harness measures, not in the law or the engine's judge. The cost sample it is shown is the age of the backlog, which does not fall until the backlog is gone. A consumer joining the group rebalances it and the settle time is eaten by the pause. The reference and the trial phases, fourteen seconds each, straddle a thirty-second load step and compare different loads. Four rules follow for the trial, written into the next engine (`docs/OMNI_V4_PLAN.md`): samples count only once the muscle reports itself settled; a spend on a queue or a cache is judged by the muscle's own reading, the lag and its drain rate, the misses, with the body's cost taken at the settled state; the reference and the trial are compared at the same load, else the trial waits for a steady step; and coarse to fine, one scout step to the far side of the cover when single notches read flat, so a hill of forty notches is not refused one notch at a time. The founder's words of the same day say it in one line: what the brain sends to a muscle has to coincide with what the muscle reports about itself and where it stands.

The next engine, Omni v4, is ordered on that day: one body, one brain, one tick a second; the body's cost judged; nothing anywhere pushed to a LOSS to move one thing to a GAIN; one trial at a time inside a body; nothing permanent; the wall belonging to the body; every wire forced through it. Part Two gains its chapter when v4 is written and run, and every result in this chapter is run again on it.

60.14 Threats to validity, stated by us

A referee will look for the ways these results could mislead. We list the ones we know, what each would do to the numbers, and what the record does about it.

1. **The machines are shared.** Every Kubernetes, database, broker and cache run is on one GitHub runner with four cores, and the governor, the load generator, the probe and the system under test share it. A governor that costs CPU can slow the service it governs, and a noisy neighbour on the host can move a run. The pairing on one runner with rotated order removes the between-machine noise; the host's own busy share is recorded beside every run; the governor's own CPU is a gauge in every table and a loss in all three of our runs where it is one (the database, Kafka's light workload); and three separate runs on separate machines must agree before anything counts.
2. **kind is not a cloud.** Idle machines stay powered and native has no node autoscaler, so a machine given back on kind is a machine-hour in service, not a watt and not a dollar. The energy rows on kind are a declared model and say so on every line. The bill is Azure's test, and its only result so far is inside the noise on a fleet too small to show one machine.
3. **Native is at its knee by design in the broker test.** The Kafka producer's peak offers nine tenths of the capacity measured for the operator's two consumers, so native's queue grows at the high steps and omni's does not. This is the situation the benchmark is of (a group sized for the quiet hours and overrun at the peak), it is preregistered, and it is why the latency ratios are large; the index page and the table say so. A

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

native group sized for the peak would show a smaller latency gain and a smaller consumer cost, and would be a different, also legitimate, benchmark.

4. **Declared penalties.** The cache's 5 ms store trip on a miss and the broker's 1, 2 or 5 ms of CPU per message are declared parameters of our own application, not measurements of anyone's store. The gauges that depend on them (work inside the line, the request latency) scale with them; the hit rate, the lag, the consumers and the memory held do not.
5. **Simulators are models.** Every S result is a statement about a published model under its own controller. A model may flatter or punish the governor in ways the real plant would not. S results are therefore shown beside the index and never inside it, and the muscles' models are the simplest physics that reproduces the meter's response to the knob, with the native controllers tuned as their authors ship them.
6. **The tuning case was seen.** Every benchmark's rules were set on one tuning case, and that case is shown and not counted. Where a rule changed after a counted result was seen (the swarm table's reproduction tolerance), the amendment is written in the preregistration with its time and reason, the table says so, and the runs were not rerun.
7. **The engine changed twice.** v1 to v2 added muscles without touching the law; v2 to v3 added one gate. Every result is labelled with its engine, the tools refuse to mix engines, and the v1 results stay as v1 results. A reader comparing a v1 number with a v3 number is comparing two engines, and the manual never does so except to say that the controllers were the same bytes and the readings the same.
8. **Three runs are three, not thirty.** The three-run rule is a replication standard, not a large sample. A row the same in all three of our runs of ten pairs is a row whose paired difference cleared zero in each of three independent samples of ten; the intervals are shown so a reader can judge their width. Where the intervals are wide, the manual says the test lacked power rather than calling the result a null.
9. **The founder's own harness.** The application in front of the cache, the producer in front of the broker, the load generator and the probe are ours. They are declared, versioned, archived with every run and reproducible from the repository; they are not independent of us. The independent pieces are the systems under test, their native controllers, the simulators and their shipped controllers, and the benchmark tools (pgbench, kind, Kubernetes, PyBullet, MuJoCo, pandapower, CityLearn) that others wrote. The load schedules of the Kubernetes tests were ours too, written one step at a time; the public-trace test (`docs/TRACES_PREREGISTRATION.md`) answers that by replaying a day of demand somebody else measured (the Google cluster trace of 2011, then the Azure Functions trace) under the wandering test's own rules, the derivation receipted and committed before the runs; its result (section 60.1) read the same way as the six, p95 65% to 71% faster and machines 6% to 10% fewer, the same in all three of our runs, with no row LOSS.

60.15 What is not yet shown, and the open program

Three things this manual does not show, and the program that will show them or show their absence:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **An energy or bill saving on real machines.** The fleet of 40 workers in nine Azure machine families that can show one machine is preregistered and dispatched on v3; it runs when Azure's region admits a cluster. If the saving is a few percent, a 40-worker fleet can see it; if the saving is not there, the table will say "STEADY" on a fleet that could have seen it, and that will be the result.
- **The real card.** Every earlier GPU result ran on a controller since replaced. The two-wire governor with the verdict runs on rented cards at one named commit, with the wire check first (`docs/GPU_RUN_GUIDE.md`, `docs/GPU_PREREGISTRATION.md`); until then the card has no result and the manual says so.
- **Robustness over a day.** The governor killed outright is a measured result (section 60.8: every setting back in 7 to 11 s, thirty times out of thirty), its own cost is tabulated, and the two-hour run is measured three times (no leak, no slowing, every decision expected but a few the cluster's API refused, every setting handed back); the 24-hour run, one pair on each of three rented eight-core machines of three families, is under way (`docs/ROBUSTNESS_PREREGISTRATION.md`, scenario 2b) and is read when its machines are collected; the same test on the other stacks remains open.
- **A demand shape that is not ours, on more than one trace.** The first public trace (a day of the Google cluster trace of 2011, section 60.1) read the same way as our own schedules, nine rows GAIN in all three of our runs and none LOSS; it is one day of one batch cluster's job arrivals. The Azure Functions invocation trace, a request-driven shape, follows by the same rule, and the "our own schedules" threat of section 60.14 is answered in full only when a second, differently shaped day reads the same way.

The queue beyond these, in order, is `docs/REGISTER.md` section 4, one or two at a time, each preregistered before it runs: PX4 and ArduPilot swarms, YCSB and HammerDB, Spark, OpenSearch, fio, Open-RMF, Basilisk, Orekit and GMAT, RocketPy and OpenRocket, Cantera. When the founder declares the engine final, the engine that stands then is published as Omni-Compass 1.0, and the older fingerprints go to `docs/history` as the road to it.

CHAPTER 61

Wire In, or Watch: The Verdict per Knob

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

How to read this receipt. Every judged row reads one of three words, decided by what its gauge is for and never by the plus or minus sign of its number. **GAIN**: the gauge moved toward its declared aim against native (more work done, a shorter wait, fewer misses or failures, or a resource cut). **STEADY**: native and Omni-Compass within 0.5% of each other (the mean of the three runs' changes under the 0.5% floor, or no

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

change at all), or the knob is at its limit on native. **LOSS**: the gauge moved away from its aim (less work done, a longer wait, more misses or failures). A resource spent or held (energy, the power drawn and its peaks and swings, machines, memory, processor time, a bill, wear) that fell is a cut, a GAIN; one that rose is **power given**, what Omni-Compass put in to buy the work and the speed shown beside it, always shown and never called a loss; work bought with each unit of a resource (work per energy) that fell is power given too. So a minus can read GAIN (a response time cut) and a plus can read LOSS (failed requests up). A change of 0.5% or more, the mean of the three runs' changes, reads GAIN, LOSS or power given: **in all three of our runs** where A, B and C moved the same way with every interval clear of zero, otherwise **by the mean of our three runs**, with the count of runs beyond the noise beside it; **the runs disagree** names runs that moved 0.5% or more in opposite directions, a test unstable there; **shown, not judged** is a gauge declared shown only. The output follows the input: every reading is the result of the tune, the wiring and the load it names, and where a reading elsewhere differs the first presumption is the wiring (`DISCLOSURES.md`, section 4). The full key, letter by letter: `docs/HOW_TO_READ_THE_RESULTS.md`. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

Omni-Compass is wired **out of** every muscle: it reads every reading. It is wired **into** a knob only where the paired measurement shows no gauge of the muscle read **LOSS** against native. Where the measurement shows nothing (**STEADY**), or shows a LOSS, the muscle stays native and Omni only watches it: one wire out, no wire in. This is not a new rule. It is what the engine does on the muscle itself before it moves anything (`omnicompass/verdict.py`: the paired trial, and the verdict **left native** where no step is allowed), and it is the watch arm of every realms run (`docs/REALMS_PREREGISTRATION.md`: the governor reads every period and writes nothing). This page applies the same principle to every published result, one verdict per knob, so that an operator can see which knobs earn a wire in and which stay native. The founder's order of 9 October 2026: where Omni cannot beat native, the muscle lives by itself; we only wire out of it.

Every verdict here is computed by `tools/wiring_verdicts.py` from the test's own result file; nothing is typed in, and `verify.py` fails if this page differs from what the tables give. One row per knob and case: `results/WIRING_VERDICTS.csv` (1214 rows, the 1,107 modelled muscles included one by one, and the seven organisms under three loads).

61.1 The rule

Every judged gauge of a table counts (rows marked "shown, not judged" never do). Every reading is **GAIN**, **STEADY** or **LOSS**, decided by what the gauge is for and never by the sign of its number (the key: `docs/HOW_TO_READ_THE_RESULTS.md`): response times and failed requests are time waited, missed or failed, so a fall is GAIN; work done is GAIN when it rises; machines, energy, memory, connections, consumers and processor time are resources spent, so a fall is a cut, a GAIN, and a rise is **power given**, the price named beside what it bought. Each gauge's aim is its preregistration's; an operator whose aim differs reads the same numbers the other way. A knob reads:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **write**: at least one gauge GAIN across our three runs, and no LOSS and no power given. Omni holds the knob.
- **watch**: no GAIN across our three runs. The knob stays native; Omni reads it and writes nothing. Where a gauge read LOSS or power given across our three runs, it is named and its cause given below. Where the runs disagree, nothing is settled and the knob stays native until it is.
- **operator's choice**: GAIN, and LOSS or power given, both judged across our three runs: a trade. Both readings of the Omni index are shown where the test is in it (the resource reading, the preregistered headline, and the service reading, work and speed alone), and the side that pays is named. The operator who wants the gain and can pay the price wires in; every other operator watches.

A modelled muscle (the 1,107 on their plants under its own load and work waiting in line, `results/realms_v5/MUSCLES_V5.csv`, engine omni-v5 at the max, the law omni-v6 carries unchanged) takes its verdict from its label by rule: a GAIN on work, speed or misses (MORE WORK, FASTER, FEWER MISSES, alone or together, on every load it ran) is write, and the energy the max took for it is power given, shown beside it and never a trade (the founder's order of 13 October); SAME AS NATIVE is watch, nothing moved beyond the noise; AT ITS LIMIT ON NATIVE is watch by design, the knob already at the top of its range on native, read and left native.

61.2 The count

Where	Knobs or cases	Write	Operator's choice	Watch
Real stacks, three runs each (Kubernetes under seven demands; the pool, the consumer group, the memory ceiling, the storage-engine cache and the buffer pool, each under its untouched workloads)	26	4	19	3
Independent simulators, three runs each (robot arms, grids, districts, swarms; evidence class S)	38	21	14	3
The 1,107 modelled muscles, alone on their plants under two loads (evidence class S)	1107	530	0	577
The organisms with the real cluster inside (GitHub at 10 and 100 copies; Azure at 1,000)	14	12	1	1

In words: on the real stacks Omni earns its wire in on 4 of 26 knob-cases, trades on 19 and watches on 3. Of the 1,107 modelled muscles, 530 earn a wire in and 577 stay native: 421 of the 577 are at their limit on native (the knob already at the top of its range: nothing to push, read and left native) and 156 read the same as native (nothing moved beyond the noise). A knob left native costs nothing and earns nothing; it needs no wire in, and the page below says so muscle by muscle.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

61.3 The real stacks (evidence class L: live software, three separate GitHub runs on the frozen engine)

The reading of every judged gauge is in the test's own table (the Source column). The index columns are the two readings of `results/OMNI_INDEX.md` for that test, where it has one.

61.3.1 Real Kubernetes: one knob under seven demands

The knob: the autoscaler's replicas, the floor and the machines, on top of the Horizontal Pod Autoscaler (kind clusters on GitHub, ten pairs a run).

Demand	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
steady work	mean response cut 48% to 49%, p95 cut 66%, p99 cut 72% to 74%, time over the line cut 100%, alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 100%, replicas cut 4.1% to 6.3%, pods started cut 20% to 49%, pod start wait, total (s) cut 20% to 54%, pod start wait, mean (s) cut 0% to 21%	none	**write**	+30.9%	+71.5%	9 gauges GAIN across our three runs (mean response cut 48% to 49%, p95 cut 66%, p99 cut 72% to 74%, time over the line cut 100% and 5 more), none LOSS or power given. Omni holds the knob	`results/live/V21_STEADY.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Demand	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
demand that wanders	mean response cut 40% to 45%, p95 cut 56% to 59%, p99 cut 11% to 35%, time over the line cut 35% to 40%, failed requests cut 12% to 15%, alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 22% to 29%, pod start wait, mean (s) cut 0.5% to 9.9%	power given: replicas up 0.1% to 2.9%	**operator's choice**	+33.6%	+138.5%	GAIN across our three runs: mean response cut 40% to 45%, p95 cut 56% to 59%, p99 cut 11% to 35%, time over the line cut 35% to 40% and 3 more; power given across our three runs: replicas up 0.1% to 2.9%	`results/live/V21_WANDERING.md`
all four at once	work inside the line up 30% to 52%, mean response cut 36% to 43%, p95 cut 51% to 59%, time over the line cut 32% to 37%, failed requests cut 5.1% to 5.4%, alert minutes (whole minutes whose 95th percentile, failed requests	LOSS: pod start wait, total (s) up 16% to 42%; power given: pods started up 8% to 37%	**operator's choice**	+33.5%	+78.3%	GAIN across our three runs: work inside the line up 30% to 52%, mean response cut 36% to 43%, p95 cut 51% to 59%, time over the line cut 32% to 37% and 2 more; LOSS across our three runs: pod start wait, total (s) up 16% to 42%;	`results/live/V21_ALL_FOUR.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Demand	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
	counted over, was over the line) cut 16% to 24%					power given across our three runs: pods started up 8% to 37%	
a noisy neighbour	mean response cut 12% to 17%, p95 cut 3% to 12%, time over the line cut 9% to 11%, alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 1.2% to 6.2%, pods started cut 9% to 28%, second app: response time (ms), 99th percentile cut 1% to 16%	LOSS: pod start wait, mean (s) up 5% to 51%, second app: failed requests (%) up 1.1% to 3.7%; power given: replicas up 0.1% to 2.9%	**operator's choice**	+2.2%	+6.8%	GAIN across our three runs: mean response cut 12% to 17%, p95 cut 3% to 12%, time over the line cut 9% to 11%, alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 1.2% to 6.2% and 2 more; LOSS across our three runs: pod start wait, mean (s) up 5% to 51%, second app: failed requests (%) up 1.1% to 3.7%; power given across our three runs: replicas up 0.1% to 2.9%	`results/live/V21_FAIRNESS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Demand	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
faults	parked-worker energy cut 0.3% to 1.0%, standby-model energy cut 0.3% to 1.0%, mean response cut 38% to 48%, p95 cut 51% to 58%, p99 cut 21% to 60%, time over the line cut 28% to 34%, failed requests cut 11% to 18%, alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 14% to 22%	power given: replicas up 10% to 20%, pods started up 12% to 22%	**operator's choice**	+30.0%	+119.7%	GAIN across our three runs: parked-worker energy cut 0.3% to 1.0%, standby-model energy cut 0.3% to 1.0%, mean response cut 38% to 48%, p95 cut 51% to 58% and 4 more; power given across our three runs: replicas up 10% to 20%, pods started up 12% to 22%	`results/live/V21_FAULTS.md`
a queue of jobs	machines cut 13% to 15%, machine-hours cut 13% to 15%, standby-model energy cut 9% to 10%, mean response cut 12% to 13%, p95 cut 0.4% to 5.7%, time over	LOSS: failed requests up from zero, pod start wait, total (s) up from zero, pod start wait, mean (s) up from zero; power given: replicas up 4% to 16%, pods started up from	**operator's choice**	+9.9%	+14.1%	GAIN across our three runs: machines cut 13% to 15%, machine-hours cut 13% to 15%, standby-model energy cut 9% to 10%, mean response cut 12% to 13% and	`results/live/V21_BATCH.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Demand	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
	the line cut 0.2% to 4.8%, machines after the queue cut 19% to 22%	zero				3 more; LOSS across our three runs: failed requests up from zero, pod start wait, total (s) up from zero, pod start wait, mean (s) up from zero; power given across our three runs: replicas up 4% to 16%, pods started up from zero	
a public day of demand (Google 2011)	mean response cut 50%, p95 cut 59% to 64%, p99 cut 48% to 56%, time over the line cut 75% to 80%, failed requests cut 10% to 18%, alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 61% to 72%	LOSS: pod start wait, total (s) up 88% to 279%; power given: replicas up 2% to 19%, pods started up 79% to 235%	**operator's choice**	+37.6%	+16 0.7 %	GAIN across our three runs: mean response cut 50%, p95 cut 59% to 64%, p99 cut 48% to 56%, time over the line cut 75% to 80% and 2 more; LOSS across our three runs: pod start wait, total (s) up 88% to 279%; power given across our three runs: replicas up 2% to 19%, pods started up 79% to	`results/live/V21_TRACE_GOOGLE2011.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Demand	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						235%	
With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs	mean response cut 37% to 38%, p95 cut 51% to 52%, p99 cut 54% to 58%, time over the line cut 91% to 100%, alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 100%	LOSS: pod start wait, total (s) up 27% to 84%; power given: parked-worker energy up 1.4% to 1.7%, standby-model energy up 1.4% to 1.7%, replicas up 11% to 16%, pods started up 31% to 49%	**operator's choice**	+27.3%	+106.1%	GAIN across our three runs: mean response cut 37% to 38%, p95 cut 51% to 52%, p99 cut 54% to 58%, time over the line cut 91% to 100% and 1 more; LOSS across our three runs: pod start wait, total (s) up 27% to 84%; power given across our three runs: parked-worker energy up 1.4% to 1.7%, standby-model energy up 1.4% to 1.7%, replicas up 11% to 16%, pods started up 31% to 49%	`results/live/V21_KEDA_CPU_REQUESTS.md`
With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass	mean response cut 57% to 58%, p95 cut 73% to 74%, p99 cut 60% to 66%, time over the line	power given: parked-worker energy up 1.9% to 2.4%, standby-model energy up 1.9%	**operator's choice**	+54.7%	+270.1%	GAIN across our three runs: mean response cut 57% to 58%, p95 cut 73% to 74%, p99 cut 60% to	`results/live/V21_KEDA_REQUESTS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Demand	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
on top, ten pairs, three runs	cut 95% to 97%, alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 94% to 100%, replicas cut 15% to 21%, pods started cut 21% to 25%, pod start wait, total (s) cut 22% to 26%, pod start wait, mean (s) cut 1.3% to 4.8%	to 2.4%				66%, time over the line cut 95% to 97% and 5 more; power given across our three runs: parked-worker energy up 1.9% to 2.4%, standby-model energy up 1.9% to 2.4%	

61.3.2 Real database (PostgreSQL behind PgBouncer, GitHub)

The knob: the pooler's pool size (server connections).

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
select	connections held open cut 6.0% to 7.2%	power given: host CPU busy up 0.4% to 1.3%, host CPU-seconds up 0.4% to 1.4%, CPU per 1,000 inside the line up 0.5% to 1.3%	**operator's choice**	+1.8%	+0.0%	the runs disagree on p95 (-13% to +5%), p99 (-12% to +5%), mean (-4.8% to +3.0%). Under the index's own scoring the GAIN outweighs what was given by	`results/live/V21_PGBENCH.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						+1.8% (resource reading); the service reading is +0.0%: a small trade, the operator's to take or leave.	
simple_update	connections held open cut 1.2% to 3.2%	power given: connections most at once up 6.7% to 8.3%	**operator's choice**	+0.5%	+0.0%	the runs disagree on p95 (-5.0% to +3.9%), p99 (-1% to +12%), mean (-2.2% to +3.7%). Why: connections most at once up 6.7% to 8.3% (power given), because amendment 1's add rule ("slow, clients waiting for a server: add") fired 4 to 15 times an arm on this slow write workload and bought servers above the operator's 20, up to 36 at the peak; work, latency and CPU inside the noise, so nothing was bought for them (`docs/POSTGRES_PREREGISTRATION.md`, the second set). The rule stands as written and the verdict for this workload is to watch. Under the index's	`results/live/V21_PGBENCH.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						own scoring the GAIN outweighs what was given by +0.5% (resource reading); the service reading is +0.0%: a small trade, the operator's to take or leave.	
tpcb_hot	p95 cut 0% to 64%, mean cut 0% to 43%, connections held open cut 1.7% to 7.5%, CPU per 1,000 inside the line cut 0.3% to 1.6%	power given: connections most at once up 0.0% to 6.7%	**operator's choice**	+10.5%	+19.2%	the runs disagree on p99 (-65% to +1%). The trade pays under both readings of the index (resource +10.5%, service +19.2%): the operator who runs PostgreSQL for its service wires in and pays in the resource named; an operator who counts that resource above the service watches.	`results/live/V21_PGBENCH.md`

61.3.3 Real messaging (Apache Kafka, GitHub)

The knob: the consumer group's size.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
tuning (tuning workload: shown, never counted)	none	LOSS: p95 up 1.1% to 5.7%, mean cut 0% to 11%, messages waiting, most up 0.1% to 5.5%, messages waiting, mean cut 0% to 11%; power given: consumers most at once up 0% to 17%	**watch** (not counted)			the runs disagree on p99 (-2.2% to +9.1%), host CPU busy (-1.0% to +1.4%), host CPU-seconds (-1.0% to +1.4%). Why: consumers most at once up 0% to 17% (power given), because the same price at its peak: the group grown to the partition count while the burst is drained.	`results/live/V21_KAFKA.md`
burst	p99 cut 0.1% to 3.0%, median cut 0.0% to 4.3%, mean cut 0.1% to 5.1%, messages waiting, mean cut 0.4% to 6.0%, host CPU busy cut 1.2% to 2.3%, host CPU-seconds cut 1.3% to 2.3%, CPU per 1,000 inside the line cut 2.0% to 2.5%	none	**write**	+0.0%	+0.0%	the runs disagree on p95 (-3.6% to +2.0%), messages waiting, most (-2.1% to +1.4%). No LOSS and no power given: Omni holds the knob.	`results/live/V21_KAFKA.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
heavy	none	LOSS: mean cut 0.4% to 3.0%, messages waiting, mean cut 0.1% to 3.9%; power given: consumers running up 0% to 12%, consumers most at once up 0% to 50%	**watch**	-1.2%	+0.0%	the runs disagree on p95 (-1.6% to +1.9%), p99 (-1.6% to +2.3%), messages waiting, most (-0.6% to +1.3%). Why: consumers running up 0% to 12% (power given), because the knob itself: Omni adds consumers while messages wait and the queue is kept short; the consumers running are the price, declared in advance as the gauge expected to rise: the power given (`docs/KAFKA_PREREGISTRATION.md`, the gauges); consumers most at once up 0% to 50% (power given), because the same price at its peak: the group grown to the	`results/live/V21_KAFKA.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						partition count while the burst is drained.	
light	none	LOSS: p99 up 0.6% to 9.1%, mean up 0% to 12%, messages waiting, mean up 0% to 11%; power given: consumers running up 0% to 17%, consumers most at once up 0% to 83%	**watch**	-2.4%	+0.0%	the runs disagree on p95 (-1% to +13%), median (-3.7% to +1.2%), messages waiting, most (-1.0% to +3.7%). Why: consumers running up 0% to 17% (power given), because the knob itself: Omni adds consumers while messages wait and the queue is kept short; the consumers running are the price, declared in advance as the gauge expected to rise: the power given (`docs/KAFKA_PREREGISTRATION.md`, the gauges); consumers most at once up 0% to 83% (power given), because	`results/live/V21_KAFKA.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						the same price at its peak: the group grown to the partition count while the burst is drained.	

61.3.4 Real cache (Redis, GitHub)

The knob: the cache's memory ceiling.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
tuning (tuning workload: shown, never counted)	work inside the line up 0.7% to 1.2%, hit rate up 0.7% to 1.2%, mean cut 2.2% to 3.8%	power given: memory ceiling held up 1.4% to 3.2%, memory used up 1.2% to 2.4%	**operator's choice** (not counted)			the runs disagree on host CPU busy (-8.4% to +4.9%), host CPU-seconds (-9.2% to +5.0%), CPU per 1,000 inside the line (-9.9% to +3.7%). Why: memory ceiling held up 1.4% to 3.2% (power given), because the knob itself: the ceiling grows while the cache is full and misses, so a wide	`results/live/V21_REDIS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						working set is held instead of evicted; the memory is the price of the hit rate, declared in advance as the gauge expected to rise: the power given (`docs/REDIS_PREREGISTRATION.md`, the gauges); memory used up 1.2% to 2.4% (power given), because the memory actually used follows the ceiling: the working set the operator's 64 MB could not hold is held.	
burst	memory ceiling held cut 1.3% to 2.4%, memory used cut 0.1% to 1.0%, host CPU busy cut 2% to 11%, host CPU-seconds cut 2% to 12%, CPU per 1,000 inside	none	**write**	+0.5%	+0.0%	No LOSS and no power given: Omni holds the knob.	`results/live/V21_REDIS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
	the line cut 2% to 12%						
large	work inside the line up 0.7% to 1.7%, hit rate up 0.7% to 1.7%, mean cut 2.7% to 7.8%	power given: memory ceiling held up 0.8% to 1.7%, memory used up 0.5% to 1.0%	**operator's choice**	-0.0%	+0.6%	the runs disagree on host CPU busy (-13% to +17%), host CPU-seconds (-15% to +19%), CPU per 1,000 inside the line (-15% to +17%). Why: memory ceiling held up 0.8% to 1.7% (power given), because the knob itself: the ceiling grows while the cache is full and misses, so a wide working set is held instead of evicted; the memory is the price of the hit rate, declared in advance as the gauge expected to rise: the power given (`docs/REDIS_PREREGISTRATION.md`, the gauges); memory used up	`results/live/V21_REDIS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						0.5% to 1.0% (power given), because the memory actually used follows the ceiling: the working set the operator's 64 MB could not hold is held. The trade pays under the service reading (+0.6%) and not under the resource reading (-0.0%, the preregistered headline): the operator who runs Redis for its speed and has the memory wires in; the operator who counts the memory as the cost watches.	
small	work inside the line up 0.6% to 1.3%, hit rate up 0.7% to 1.3%, mean cut 0.6% to 1.7%	LOSS: p99 up 0.2% to 1.7%; power given: memory ceiling held up 0.8% to 2.2%, memory used up 1.1% to 1.9%, host CPU	**operator's choice**	-0.2%	+0.5%	Why: memory ceiling held up 0.8% to 2.2% (power given), because the knob itself: the ceiling grows while the cache is full and	`results/live/V21_REDIS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
		busy up 1% to 11%, host CPU-seconds up 1% to 12%, CPU per 1,000 inside the line up 0% to 11%, process_cpu_seconds up 0.3% to 2.4%				misses, so a wide working set is held instead of evicted; the memory is the price of the hit rate, declared in advance as the gauge expected to rise: the power given (`docs/REDIS_PREREGISTRATION.md`, the gauges); memory used up 1.1% to 1.9% (power given), because the memory actually used follows the ceiling: the working set the operator's 64 MB could not hold is held. The trade pays under the service reading (+0.5%) and not under the resource reading (-0.2%, the preregistered headline): the operator who runs	

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						Redis for its speed and has the memory wires in; the operator who counts the memory as the cost watches.	

61.3.5 Real database cache (MongoDB under YCSB, GitHub)

The knob: the storage engine's cache size.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
tuning (tuning workload: shown, never counted)	none	LOSS: p95 up 0.0% to 2.1%, p99 cut 0.3% to 3.8%; power given: cache size held up 5% to 12%, cache in use up 4% to 11%, host CPU busy up 0.4% to 6.6%, host CPU-seconds up 0.5% to 8.7%, CPU per 1,000 inside the line up 0.6% to 8.7%, process_cpu_seconds cut 0.1% to 2.2%	**watch** (not counted)			the runs disagree on mean (-0.5% to +2.1%).	`results/live/V21_YCSB.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
b	host CPU busy cut 1.6% to 2.5%, host CPU-seconds cut 1.7% to 3.2%, CPU per 1,000 inside the line cut 1.8% to 3.2%	power given: process_cpu_seconds cut 0.3% to 2.0%	**operator's choice**	+0.0%	+0.0%	the runs disagree on p95 (-0.9% to +2.3%), p99 (-0.8% to +2.4%), mean (-0.9% to +1.4%). Under the index's own scoring the GAIN outweighs what was given by +0.0% (resource reading); the service reading is +0.0%: a small trade, the operator's to take or leave.	`results/live/V21_YCSB.md`
burst	mean cut 0.4% to 4.0%	LOSS: p95 up 0.2% to 5.4%, p99 up 0.4% to 2.5%; power given: cache size held cut 0.4% to 2.9%	**operator's choice**	-0.8%	-1.0%	the runs disagree on cache in use (-1.1% to +4.2%), host CPU busy (-15% to +25%), host CPU-seconds (-18% to +28%). Under the index's own scoring the GAIN outweighs what was given by -0.8% (resource reading); the service reading is -1.0%: a small trade, the operator's to take	`results/live/V21_YCSB.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						or leave.	
c	host CPU busy cut 0.2% to 3.8%, host CPU-seconds cut 0.2% to 4.7%, CPU per 1,000 inside the line cut 0.2% to 4.7%	power given: cache in use cut 0.3% to 3.4%, process_cpu_seconds up 0.2% to 1.1%	**operator's choice**	+0.0%	+0.0%	the runs disagree on p99 (-0.9% to +1.4%), mean (-0.6% to +0.6%), cache size held (-0.6% to +3.3%). Under the index's own scoring the GAIN outweighs what was given by +0.0% (resource reading); the service reading is +0.0%: a small trade, the operator's to take or leave.	`results/live/V21_YCSB.md`
f	none	power given: cache size held up 2.6% to 7.5%, cache in use up 2.8% to 7.2%	**watch**	-1.3%	+0.0%	the runs disagree on p95 (-0.7% to +4.5%), mean (-0.9% to +2.6%), host CPU busy (-1% to +17%).	`results/live/V21_YCSB.md`

61.3.6 Real database buffer pool (MySQL under sysbench, GitHub)

The knob: the buffer pool's size.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
tuning (tuning workload: shown, never counted)	p95 cut 2% to 20%, p99 cut 1.2% to 9.8%, mean cut 3.1% to 6.1%, host CPU busy cut 0.5% to 5.5%, host CPU-seconds cut 0.4% to 5.4%, CPU per 1,000 inside the line cut 0.4% to 5.4%	none	**write** (not counted)			the runs disagree on pool held (-12% to +11%), pages holding data (-11% to +11%). No LOSS and no power given: Omni holds the knob.	`results/live/V21_SYSBENCH.md`
burst	p95 cut 1.8% to 3.5%, p99 cut 2.4% to 4.1%, pool held cut 4% to 17%, pages holding data cut 2% to 14%	power given: host CPU busy cut 0.2% to 2.6%, host CPU-seconds cut 0.0% to 3.0%, CPU per 1,000 inside the line up 0.3% to 3.7%	**operator's choice**	+3.9%	+1.3%	The trade pays under both readings of the index (resource +3.9%, service +1.3%): the operator who runs MySQL for its service wires in and pays in the resource named; an operator who counts that resource above the service watches.	`results/live/V21_SYSBENCH.md`
read_only	p99 cut 0.6% to 2.4%, pool held cut 10% to 22%, pages holding data cut 8% to 21%	power given: host CPU busy up 0.5% to 2.0%, host CPU-seconds up 0.5% to 2.4%, CPU per 1,000	**operator's choice**	+5.0%	+0.0%	the runs disagree on p95 (-1.8% to +0.6%). Under the index's own scoring the GAIN outweighs what	`results/live/V21_SYSBENCH.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
		inside the line up 0.8% to 2.4%				was given by +5.0% (resource reading); the service reading is +0.0%: a small trade, the operator's to take or leave.	
read_write	p99 cut 0.6% to 1.4%	power given: pages holding data up 0.4% to 6.0%	**operator's choice**	+0.0%	+0.0%	the runs disagree on p95 (-2.4% to +1.8%), mean (-0.9% to +0.7%), pool held (-1.1% to +6.7%). Why: pages holding data up 0.4% to 6.0% (power given), because memory bought on real misses: a written working set keeps the pool missing above one percent, so the growth gate lets the pool grow to 1.1 to 1.6 GB in the wide notches; the host's CPU was cut 4% to 6% for it, a GAIN across our three runs, and latency did not move (`docs/MYSQL_PRE`	`results/live/V21_SYSBENCH.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Workload	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Index: resource	Index: service	Why	Source
						`REGISTRATION.md`, the third set). Under the index's own scoring the GAIN outweighs what was given by +0.0% (resource reading); the service reading is +0.0%: a small trade, the operator's to take or leave.	
update_index	pages holding data cut 0.6% to 0.8%, host CPU busy cut 2% to 38%, host CPU-seconds cut 2% to 38%	none	**write**	+0.0%	+0.0%	the runs disagree on work inside the line (-26% to +197%), p95 (-42% to +116%), p99 (-45% to +107%). No LOSS and no power given: Omni holds the knob.	`results/live/V21_SYSBENCH.md`

61.4 Every LOSS and every power given across our three runs on a real stack, and why

The founder's question was where the negatives are and why each is there. Every gauge that read LOSS or power given in all three of our runs on a real stack, with its cause read from the preregistration that carries the result:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Kubernetes, replicas: demand that wanders up 0.1% to 2.9%, a noisy neighbour up 0.1% to 2.9%, faults up 10% to 20%, a queue of jobs up 4% to 16%, a public day of demand (Google 2011) up 2% to 19%, With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs up 11% to 16%, power given** (15.09 / 2.11 / 8.376 / 8.865 / 9.512 / 9.699 → 10.08 to 9.651 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **Kubernetes, pod start wait, total (s): all four at once up 16% to 42%, a queue of jobs up from zero from zero, a public day of demand (Google 2011) up 88% to 279%, With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs up 27% to 84%, LOSS** (0 / 10 / 6.3 / 8.9 → 0.9 to 20.9 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **Kubernetes, pods started: all four at once up 8% to 37%, faults up 12% to 22%, a queue of jobs up from zero from zero, a public day of demand (Google 2011) up 79% to 235%, With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs up 31% to 49%, power given** (0 / 4.9 / 5.5 / 5.7 / 6.1 → 0.6 to 7.3 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **Kubernetes, pod start wait, mean (s): a noisy neighbour up 5% to 51%, a queue of jobs up from zero from zero, LOSS** (0 / 1.6 → 0.9 to 1.678 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **Kubernetes, second app: failed requests (%): a noisy neighbour up 1.1% to 3.7%, LOSS** (13.65 → 13.8 in run A). See the table. Verdict: **operator's choice**.
- **Kubernetes, failed requests: a queue of jobs up from zero from zero, LOSS** (0 → 0.01036 in run A). See the table. Verdict: **operator's choice**.
- **Kubernetes, parked-worker energy: With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs up 1.4% to 1.7%, With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass on top, ten pairs, three runs up 1.9% to 2.4%, power given** (157.9 / 159.1 → 161.6 to 161.7 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **Kubernetes, standby-model energy: With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs up 1.4% to 1.7%, With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass on top, ten pairs, three runs up 1.9% to 2.4%, power given** (157.9 / 159.1 → 161.6 to 161.7 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **PostgreSQL, host CPU busy: select up 0.4% to 1.3%, power given** (0.3247 → 0.3264 in run A). See the table. Verdict: **operator's choice**.
- **PostgreSQL, host CPU-seconds: select up 0.4% to 1.4%, power given** (486.9 → 489.6 in run A). See the table. Verdict: **operator's choice**.
- **PostgreSQL, CPU per 1,000 inside the line: select up 0.5% to 1.3%, power given** (0.1491 → 0.1499 in run A). See the table. Verdict: **operator's choice**.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **PostgreSQL, connections most at once: simple_update up 6.7% to 8.3%, tpcb_hot up 0.0% to 6.7%, power given** (20 → 21.33 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **Kafka, mean: heavy cut 0.4% to 3.0%, light up 0% to 12%, LOSS** (318.8 / 341 → 323.5 to 339.6 in run A). See the table. Verdict: **watch** on every workload named.
- **Kafka, messages waiting, mean: heavy cut 0.1% to 3.9%, light up 0% to 11%, LOSS** (277.4 / 62.6 → 281.1 to 62.55 in run A). See the table. Verdict: **watch** on every workload named.
- **Kafka, consumers running: heavy up 0% to 12%, light up 0% to 17%, power given** (2 → 2 in run A). See the table. Verdict: **watch** on every workload named.
- **Kafka, consumers most at once: heavy up 0% to 50%, light up 0% to 83%, power given** (2 → 2 in run A). See the table. Verdict: **watch** on every workload named.
- **Kafka, p99: light up 0.6% to 9.1%, LOSS** (3736 → 3759 in run A). See the table. Verdict: **watch**.
- **Redis, memory ceiling held: large up 0.8% to 1.7%, small up 0.8% to 2.2%, power given** (64 → 64.51 to 65.17 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **Redis, memory used: large up 0.5% to 1.0%, small up 1.1% to 1.9%, power given** (60.7 / 61.29 → 61.52 to 61.6 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **Redis, p99: small up 0.2% to 1.7%, LOSS** (5.74 → 5.75 in run A). See the table. Verdict: **operator's choice**.
- **Redis, host CPU busy: small up 1% to 11%, power given** (0.1463 → 0.1478 in run A). See the table. Verdict: **operator's choice**.
- **Redis, host CPU-seconds: small up 1% to 12%, power given** (246.5 → 249.5 in run A). See the table. Verdict: **operator's choice**.
- **Redis, CPU per 1,000 inside the line: small up 0% to 11%, power given** (0.5917 → 0.5932 in run A). See the table. Verdict: **operator's choice**.
- **Redis, process_cpu_seconds: small up 0.3% to 2.4%, power given** (308.9 → 309.7 in run A). See the table. Verdict: **operator's choice**.
- **MongoDB, process_cpu_seconds: b cut 0.3% to 2.0%, c up 0.2% to 1.1%, power given** (183.5 / 196.7 → 185.6 to 199.4 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **MongoDB, p95: burst up 0.2% to 5.4%, LOSS** (0.4083 → 0.409 in run A). See the table. Verdict: **operator's choice**.
- **MongoDB, p99: burst up 0.4% to 2.5%, LOSS** (0.577 → 0.5793 in run A). See the table. Verdict: **operator's choice**.
- **MongoDB, cache size held: burst cut 0.4% to 2.9%, f up 2.6% to 7.5%, power given** (512 → 514.9 to 540 in run A). See the table. Verdict: **operator's choice / watch** on every workload named.
- **MongoDB, cache in use: c cut 0.3% to 3.4%, f up 2.8% to 7.2%, power given** (416.7 / 418.5 → 419.6 to 439 in run A). See the table. Verdict: **operator's choice / watch** on every workload named.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **MySQL, host CPU busy: burst cut 0.2% to 2.6%, read_only up 0.5% to 2.0%, power given** (0.189 / 0.1893 → 0.189 to 0.1918 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **MySQL, host CPU-seconds: burst cut 0.0% to 3.0%, read_only up 0.5% to 2.4%, power given** (104.3 / 312.6 → 104.3 to 317.8 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **MySQL, CPU per 1,000 inside the line: burst up 0.3% to 3.7%, read_only up 0.8% to 2.4%, power given** (2.32 / 2.322 → 2.327 to 2.351 in run A). See the table. Verdict: **operator's choice** on every workload named.
- **MySQL, pages holding data: read_write up 0.4% to 6.0%, power given** (462 → 489.1 in run A). See the table. Verdict: **operator's choice**.

10 of these are a LOSS (failed requests, mean, messages waiting, mean, p95, p99, pod start wait, mean (s), pod start wait, total (s), second app: failed requests (%)): each is named above with its cause, and each is a fault to find and fix, our own wiring first. The rest are power given, a resource spent to buy the service the knob exists for. Where the resource was given and nothing was bought (PostgreSQL's `simple_update`), the verdict is watch; where it bought service, the verdict is the operator's, and both index readings say what the trade is worth.

61.5 The independent simulators (evidence class S: deterministic models with their own native controllers)

61.5.1 Robot arms (MuJoCo Menagerie, Google DeepMind)

The knob: the servo's speed override inside the takt.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
kinova_gen3	energy per takt cut 0.8%, copper heating per cycle cut 11%, mechanical work per cycle cut 6.6%, peak joint torque cut 29%, tracking error, RMS cut 21%, end-point error at the waypoints cut 11%	none	**write**	No LOSS: the same job inside the same takt, hit more accurately with less force; the cycle is slower inside the takt (shown, not judged).	`results/live/V7_MUJOCO.md`
kuka_iiwa_14	none	none	**watch**	The engine's own verdict: one cycle at full speed and one at override 0.8, in native mode before the	`results/live/V7_MUJOCO.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
				counted cycles, showed a slower cycle no cheaper on the robot's own figures (the gravity-holding torque is paid for longer), so the override was left at 1.0 and both arms are the same (`omnicompass/verdict.py`, left native). The trial's figures: 3157.3 J at full speed against 3757.3 J at the trial speed, reproduced over A, B, C: yes	
universal_robots_ur5e	none	none	**watch**	The engine's own verdict: one cycle at full speed and one at override 0.8, in native mode before the counted cycles, showed a slower cycle no cheaper on the robot's own figures (the gravity-holding torque is paid for longer), so the override was left at 1.0 and both arms are the same (`omnicompass/verdict.py`, left native). The trial's figures: 189.9 J at full speed against 210.5 J at the trial speed, reproduced over A, B, C: yes	`results/live/V7_MUJOCO.md`

61.5.2 The power grid (pandapower, SimBench grids)

The knob: the substation's voltage setpoint, one tap at a time.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
1-MV-comm--0-sw (ZIP loads)	energy the loads drew cut 1.5%, energy dissipated in lines and transformers cut 1.0%, net import from the upstream grid cut 2.9%, tap operations cut 34%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-comm--1-sw (ZIP loads)	energy the loads drew cut 1.4%, energy dissipated in lines and transformers cut 0.8%, net import from the upstream grid cut 15%, tap operations cut 36%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-comm--2-sw (ZIP loads)	energy the loads drew cut 1.4%, net import from the upstream grid cut 6.4%, tap operations cut 36%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-rural--1-sw (ZIP loads)	energy the loads drew cut 1.4%, net import from the upstream grid cut 1.4%	power given: energy dissipated in lines and transformers up 0.8%, tap operations up 100%	**operator's choice**	Why: line heating: an exporting grid pays the lower voltage in energy dissipated in its lines (the analysis under the table); taps: the declared cost on a grid that barely taps on its own (4 a year, doubled). The operator who wants the loads' energy and the upstream import cut wires in and gives	`results/live/V7_PANDAPOWER.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
				the line heating; the operator who runs the grid to its line heating watches.	
1-MV-rural--2-sw (ZIP loads)	energy the loads drew cut 1.3%, net import from the upstream grid cut 1.1%, bus-steps outside 0.95-1.05 cut 1.4e-04, tap operations cut 27%	power given: energy dissipated in lines and transformers up 1.5%	**operator's choice**	Why: line heating: an exporting grid pays the lower voltage in energy dissipated in its lines (the analysis under the table). The operator who wants the loads' energy and the upstream import cut wires in and gives the line heating; the operator who runs the grid to its line heating watches.	`results/live/V7_PANDAPOWER.md`
1-MV-semiurb--0-sw (ZIP loads)	energy the loads drew cut 1.4%, energy dissipated in lines and transformers cut 1.0%, net import from the upstream grid cut 7.0%, tap operations cut 38%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-semiurb--1-sw (ZIP loads)	energy the loads drew cut 1.4%, net import from the upstream grid cut 1.1%, tap operations cut 34%	power given: energy dissipated in lines and transformers up 0.7%	**operator's choice**	Why: line heating: an exporting grid pays the lower voltage in energy dissipated in its lines (the analysis under the table). The operator who wants the loads' energy and the upstream import	`results/live/V7_PANDAPOWER.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
				cut wires in and gives the line heating; the operator who runs the grid to its line heating watches.	
1-MV-semiurb--2-sw (ZIP loads)	energy the loads drew cut 1.4%, net import from the upstream grid cut 0.9%, bus-steps outside 0.95-1.05 cut 1.4e-05, tap operations cut 24%	power given: energy dissipated in lines and transformers up 1.1%	**operator's choice**	Why: line heating: an exporting grid pays the lower voltage in energy dissipated in its lines (the analysis under the table). The operator who wants the loads' energy and the upstream import cut wires in and gives the line heating; the operator who runs the grid to its line heating watches.	`results/live/V7_PANDAPOWER.md`
1-MV-urban--0-sw (ZIP loads)	energy the loads drew cut 1.5%, energy dissipated in lines and transformers cut 2.4%, net import from the upstream grid cut 2.0%, tap operations cut 31%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-urban--1-sw (ZIP loads)	energy the loads drew cut 1.5%, energy dissipated in lines and transformers cut 2.4%, net import from the upstream grid cut 2.2%, tap operations cut 30%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
1-MV-urban--2-sw (ZIP loads)	energy the loads drew cut 1.5%, energy dissipated in lines and transformers cut 2.2%, net import from the upstream grid cut 3.1%, tap operations cut 23%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-comm--0-sw (constant-power loads)	energy dissipated in lines and transformers cut 0.5%, tap operations cut 31%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-comm--1-sw (constant-power loads)	tap operations cut 31%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-comm--2-sw (constant-power loads)	tap operations cut 31%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-rural--1-sw (constant-power loads)	none	power given: energy dissipated in lines and transformers up 0.6%, tap operations up 100%	**watch**	Why: line heating: an exporting grid pays the lower voltage in energy dissipated in its lines (the analysis under the table); taps: the declared cost on a grid that barely taps on its own (4 a year, doubled).	`results/live/V7_PANDAPOWER.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
1-MV-rural--2-sw (constant-power loads)	bus-steps outside 0.95-1.05 cut 1.4e-04, tap operations cut 16%	power given: energy dissipated in lines and transformers up 1.2%	**operator's choice**	Why: line heating: an exporting grid pays the lower voltage in energy dissipated in its lines (the analysis under the table). The operator who wants the loads' energy and the upstream import cut wires in and gives the line heating; the operator who runs the grid to its line heating watches.	`results/live/V7_PANDAPOWER.md`
1-MV-semiurb--0-sw (constant-power loads)	energy dissipated in lines and transformers cut 0.9%, tap operations cut 29%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-semiurb--1-sw (constant-power loads)	tap operations cut 31%	power given: energy dissipated in lines and transformers up 0.6%	**operator's choice**	Why: line heating: an exporting grid pays the lower voltage in energy dissipated in its lines (the analysis under the table). The operator who wants the loads' energy and the upstream import cut wires in and gives the line heating; the operator who runs the grid to its line heating watches.	`results/live/V7_PANDAPOWER.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
1-MV-semiurb--2-sw (constant-power loads)	bus-steps outside 0.95-1.05 cut 1.7e-05, tap operations cut 21%	power given: energy dissipated in lines and transformers up 0.9%	**operator's choice**	Why: line heating: an exporting grid pays the lower voltage in energy dissipated in its lines (the analysis under the table). The operator who wants the loads' energy and the upstream import cut wires in and gives the line heating; the operator who runs the grid to its line heating watches.	`results/live/V7_PANDAPOWER.md`
1-MV-urban--0-sw (constant-power loads)	energy dissipated in lines and transformers cut 1.9%, tap operations cut 30%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-urban--1-sw (constant-power loads)	energy dissipated in lines and transformers cut 1.9%, tap operations cut 27%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`
1-MV-urban--2-sw (constant-power loads)	energy dissipated in lines and transformers cut 1.8%, tap operations cut 22%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_PANDAPOWER.md`

Why the line heating rose on four grids (power given). The lower setpoint carries the grid's power at a lower voltage and therefore a higher current, and the energy dissipated in the lines rises with the square of the current. The four grids where it rose are four of the five that export more than they import (net import negative: rural 1 and 2, semiurb 1 and 2); on the importing grids the loads' own draw falls with the voltage under ZIP loads and the line heating falls with it. Under constant-power loads the loads draw the same at any

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

voltage, so the setpoint saves nothing on the loads and the exporting grids give the line heating with nothing bought. Tap operations, declared in advance as Omni's expected cost (`docs/PANDAPOWER_PREREGISTRATION.md`): this grid barely taps on its own (4 a year), so Omni's moves double a very small count. The verdict on those four grids is the operator's: the loads' energy and the upstream import cut (GAIN) against the lines' heating up (power given); under constant-power loads on 1-MV-rural--1-sw, where nothing was bought, it is watch.

61.5.3 Buildings and batteries (CityLearn, UT Austin)

The knob: the electric batteries' charge and discharge commands.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
ca_alameda_county_neighborhood	electricity bought cut 5.9%, daily peak draw cut 3.6%, highest peak cut 3.0%, ramping cut 3.9%, daily load unevenness cut 1.0%, monthly load unevenness cut 0.7%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_CITYLEARN.md`
citylearn_challenge_2022_phase_all_robustness	electricity bill cut 8.6%, electricity bought cut 15%, carbon cut 12%, daily peak draw cut 14%, highest peak cut 2.7%, ramping cut 7.2%, daily load unevenness cut 2.9%, distance from zero net energy cut 6.1%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_CITYLEARN.md`
citylearn_challenge_2023_phase_2_local_evaluation	daily peak draw cut 1.6%, daily load unevenness cut 2.9%	LOSS: energy not served up 6.0%; power given: ramping up 6.2%	**operator's choice**	Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity	`results/live/V7_CITYLEARN.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
				bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch.	
citylearn_challenge_2023_phase_2_online_evaluation_1	daily peak draw cut 1.3%, highest peak cut 3.5%, daily load unevenness cut 2.0%, monthly load unevenness cut 1.9%	LOSS: energy not served up 5.5%; power given: ramping up 5.6%	**operator's choice**	Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch.	`results/live/V7_CITYLEARN.md`
citylearn_challenge_2023_phase_2_online_evaluation_2	daily peak draw cut 1.3%, highest peak cut 3.5%, daily load unevenness cut 1.7%, monthly load unevenness cut 1.9%	LOSS: energy not served up 6.2%; power given: ramping up 5.6%	**operator's choice**	Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch.	`results/live/V7_CITYLEARN.md`
citylearn_challenge_2023_phase_2_online_evaluation_3	daily peak draw cut 1.5%, highest peak cut 3.5%, daily load unevenness cut	LOSS: energy not served up 9.8%; power given: ramping up 5.5%	**operator's choice**	Why: the 2023 pattern: the bill, the ramping and the energy not served	`results/live/V7_CITYLEAR`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
	2.3%, monthly load unevenness cut 1.9%			up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch.	N.md
citylearn_challenge_2023_phase_3_1	daily peak draw cut 2.2%, daily load unevenness cut 4.0%, monthly load unevenness cut 1.2%	LOSS: energy not served up 2.0%; power given: highest peak up 2.5%, ramping up 4.5%	**operator's choice**	Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch.	results/live/V7_CITYLEARN.md
citylearn_challenge_2023_phase_3_2	daily peak draw cut 2.2%, daily load unevenness cut 4.1%, monthly load unevenness cut 0.5%	LOSS: energy not served up 4.1%; power given: highest peak up 2.5%, ramping up 4.9%	**operator's choice**	Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch.	results/live/V7_CITYLEARN.md

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
citylearn_challenge_2023_phase_3_3	daily peak draw cut 2.6%, daily load unevenness cut 4.7%, monthly load unevenness cut 1.0%	LOSS: energy not served up 2.2%; power given: highest peak up 2.5%, ramping up 4.8%	**operator's choice**	Why: the 2023 pattern: the bill, the ramping and the energy not served up, the electricity bought, the carbon and the daily peak down (the analysis under the table). By rule a trade; by what the battery is for (the bill), a watch.	`results/live/V7_CITYLEARN.md`
tx_travis_county_neighborhood	electricity bought cut 3.0%, daily peak draw cut 1.9%, ramping cut 2.9%, daily load unevenness cut 1.1%, monthly load unevenness cut 0.7%, distance from zero net energy cut 0.6%	none	**write**	No LOSS and no power given: Omni holds the knob.	`results/live/V7_CITYLEARN.md`
baeda_3dem			**no knob**	no electric battery in the district (4 buildings, 7 water tanks left native): there is nothing for Omni to hold, so there is nothing to decide; any difference between the arms is CityLearn's own run-to-run variation	`results/live/V7_CITYLEARN.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
				(scores that differ in A: none; reproduced: yes)	
quebec_neighborhood_with_demand_response_set_points			**no knob**	no electric battery in the district (20 buildings, 0 water tanks left native): there is nothing for Omni to hold, so there is nothing to decide; any difference between the arms is CityLearn's own run-to-run variation (scores that differ in A: time uncomfortable +0.25%; reproduced: no)	`results/live/V7_CITYLEARN.md`
quebec_neighborhood_without_demand_response_set_points			**no knob**	no electric battery in the district (20 buildings, 0 water tanks left native): there is nothing for Omni to hold, so there is nothing to decide; any difference between the arms is CityLearn's own run-to-run variation (scores that differ in A: time uncomfortable	`results/live/V7_CITYLEARN.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
				-0.28%; reproduced: no)	

Why the bill is up in the 2023 districts (power given), with the ramping and the energy not served (LOSS). The seven 2023 challenge districts are small (3 to 6 buildings) and short (720 to 2,208 hours), and in them the energy not served and the comfort rows move, where in the county neighbourhoods they read the same: the compass steers the batteries from the district's electricity reading alone (one wire, one muscle, `docs/CITYLEARN_PREREGISTRATION.md`), flattening the draw it sees, and in these districts that leaves less in the batteries for the hours that count against the bill, the hour-to-hour ramping and the energy not served. The bill, what the battery is bought to lower, is up 0.2% to 0.4% in all three of our runs here (power given), so for the operator who pays the bill this is a watch. The cause is a reading of the tables, not yet tested on its own.

61.5.4 Drone swarms (gym-pybullet-drones, University of Toronto)

The knob: the autopilot's cruise override.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
tuning (the tuning swarm: shown, never counted)	energy per mission cut 16%, fleet energy over the window cut 16%, missions per charge at the autopilot's reserve up 19%	none	**write** (not counted)	the runs differ on closest two drones ever came (-51% to -45%), tracking error, RMS (+88% to +91%), cruise override, mean (+49%). No LOSS: the tracking error is spent inside the compass's band (shown, the runs differ on its exact figure) and no safety gauge moved.	`results/live/V7_SWARM.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Case	GAIN across our three runs	LOSS or power given across our three runs	Verdict	Why	Source
long	energy per mission cut 20%, fleet energy over the window cut 20%, missions per charge at the autopilot's reserve up 25%	none	**write**	the runs differ on closest two drones ever came (-5.7% to -1.5%), tracking error, RMS (+92% to +93%), cruise override, mean (+52%). No LOSS: the tracking error is spent inside the compass's band (shown, the runs differ on its exact figure) and no safety gauge moved.	`results/live/V7_SWARM.md`
mixed	energy per mission cut 18%, fleet energy over the window cut 18%, missions per charge at the autopilot's reserve up 22%	none	**write**	the runs differ on closest two drones ever came (-46% to -42%), tracking error, RMS (+83% to +86%), cruise override, mean (+48% to +49%). No LOSS: the tracking error is spent inside the compass's band (shown, the runs differ on its exact figure) and no safety gauge moved.	`results/live/V7_SWARM.md`
short	energy per mission cut 7.4%, fleet energy over the window cut 7.4%, missions per charge at the autopilot's reserve up 8.0%	none	**write**	No LOSS: the tracking error is spent inside the compass's band (shown, the runs differ on its exact figure) and no safety gauge moved.	`results/live/V7_SWARM.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

61.6 The 1,107 modelled muscles, alone on their plants (evidence class S)

Every muscle's own row, with its verdict and the reason, is in `results/WIRING_VERDICTS.csv`; the labels and figures are the v5 realms table's (`results/realms_v5/REALMS_V5.md`, `results/realms_v5/MUSCLES_V5.csv`; engine omni-v5 at the max, every muscle its own card; seeds 3000 to 3009, ten paired seeds a muscle, under its own load and under work waiting in line). The label is the muscle's over every load it ran; the Why column gives each load's own.

Reading (the label over every load, its word first)	Muscles	Verdict
STEADY (at its limit on native)	421	**watch**
GAIN (faster + fewer misses)	396	**write**
STEADY (the same as native)	156	**watch**
GAIN (faster)	85	**write**
GAIN (fewer misses)	37	**write**
GAIN (more work + faster + fewer misses)	12	**write**

61.6.1 By knob

Knob	Muscles	Write	Watch: at its limit on native	Watch: the same as native	Of the watch, wrote nothing in any run
capacity	539	472	50	17	58
setpoint	174	44	0	130	111
power	119	14	96	9	96
admission	275	0	275	0	275

61.6.2 By family

Realm: family	Muscles	Write	Watch: at its limit on native	Watch: the same as native
Compute / AI / Cloud: AI Inference Serving	24	15	9	0
Compute / AI / Cloud: AI Training	18	16	2	0
Compute / AI / Cloud: Cloud VM & Capacity	21	20	1	0
Compute / AI / Cloud: Container Resources	25	22	3	0

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Realm: family	Muscles	Write	Watch: at its limit on native	Watch: the same as native
Compute / AI / Cloud: Cross-Cluster, Multi-Region & Edge	18	10	7	1
Compute / AI / Cloud: DPU SmartNIC & Programmable IO	12	10	2	0
Compute / AI / Cloud: Distributed Cluster Managers	15	8	7	0
Compute / AI / Cloud: GPU Fabric & RDMA	18	14	4	0
Compute / AI / Cloud: HPC & Distributed Compute	19	11	8	0
Compute / AI / Cloud: Host CPU & Memory	26	10	16	0
Compute / AI / Cloud: Kubernetes Dynamic Device Allocation	8	7	1	0
Compute / AI / Cloud: Kubernetes Placement & Scheduling	21	16	5	0
Compute / AI / Cloud: Kubernetes Workload Scaling	32	24	8	0
Compute / AI / Cloud: GPU Hardware	22	11	11	0
Compute / AI / Cloud: Node Fleet & Karpenter-Class Control	24	19	5	0
Compute / AI / Cloud: OpenShift & Machine API	9	5	4	0
Compute / AI / Cloud: Quantum Computing Control Simulation	16	11	5	0
Compute / AI / Cloud: Work Admission & Demand Shaping	19	0	19	0
Distribution / Specialized: Cache & Memory Services	16	12	4	0
Distribution / Specialized: Commerce & Payment Systems	15	7	8	0
Distribution / Specialized: Data Analytics & ETL	15	11	4	0

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Realm: family	Muscles	Write	Watch: at its limit on native	Watch: the same as native
Distribution / Specialized: Database & Transactions	19	14	5	0
Distribution / Specialized: Messaging & Streaming	22	13	7	2
Distribution / Specialized: Network Routing & Switching	19	12	7	0
Distribution / Specialized: Observability & Telemetry	9	6	3	0
Distribution / Specialized: Reliability, Security & Recovery	18	9	8	1
Distribution / Specialized: Runtime & Application	21	14	5	2
Distribution / Specialized: Search, Indexing & Vector DB	16	10	6	0
Distribution / Specialized: Service Mesh & API Reliability	16	9	7	0
Distribution / Specialized: Storage Block/File/Object	20	17	3	0
Distribution / Specialized: Telecom RAN & Edge Radio	12	8	4	0
Distribution / Specialized: Workflow, Logistics & Fulfillment	15	10	5	0
Energy / Facility / Industrial: Building & Critical Environment HVAC	15	0	4	11
Energy / Facility / Industrial: Cooling, Chillers & Thermodynamics	18	0	4	14
Energy / Facility / Industrial: Energy Storage & Microgrid	19	17	2	0
Energy / Facility / Industrial: Facility & Grid Optimization	15	9	5	1
Energy / Facility / Industrial: Grid Transmission & Distribution	12	0	8	4

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Realm: family	Muscles	Write	Watch: at its limit on native	Watch: the same as native
Energy / Facility / Industrial: Industrial PLC & Process Automation	18	0	12	6
Energy / Facility / Industrial: PDU, UPS & Electrical Distribution	18	7	3	8
Energy / Facility / Industrial: Semiconductor Fab & Precision Manufacturing	13	0	11	2
Energy / Facility / Industrial: Water Wastewater & Pumping	12	0	8	4
Physics / Robotics / Autonomous: Automotive EV & Mobile Powertrain	14	1	13	0
Physics / Robotics / Autonomous: Aviation & Autonomous Flight	28	14	14	0
Physics / Robotics / Autonomous: Robotics Fleet & Warehouse Automation	17	14	3	0
Physics / Robotics / Autonomous: Robotics Motion Control	26	19	5	2
Physics / Robotics / Autonomous: Spacecraft & Flight Software	14	0	14	0
Energy / Facility / Industrial: Healthcare Critical Environments	13	0	3	10
Distribution / Specialized: Medical Imaging & Clinical Systems	12	3	9	0
Energy / Facility / Industrial: Agriculture & Irrigation	14	0	4	10
Energy / Facility / Industrial: Oil & Gas Pipelines	14	0	6	8
Physics / Robotics / Autonomous: Rail Traction & Train Control	14	0	14	0
Physics / Robotics / Autonomous: Marine Propulsion & Vessel Automation	12	5	7	0
Distribution / Specialized: Ports & Maritime Logistics	12	6	6	0

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Realm: family	Muscles	Write	Watch: at its limit on native	Watch: the same as native
Energy / Facility / Industrial: Mining & Mineral Processing	14	0	6	8
Energy / Facility / Industrial: District Heating & Cooling	13	0	6	7
Energy / Facility / Industrial: Power Generation & Turbine Control	14	0	4	10
Energy / Facility / Industrial: Renewable Generation & Inverter Control	13	0	7	6
Physics / Robotics / Autonomous: Elevators & Vertical Transport	12	0	12	0
Energy / Facility / Industrial: Pharmaceutical & Food Manufacturing	14	0	4	10
Compute / AI / Cloud: Linux Kernel & OS Control	20	9	3	8
Compute / AI / Cloud: Serverless & Functions (FaaS)	9	6	2	1
Compute / AI / Cloud: Virtualization & Hypervisor	9	6	2	1
Distribution / Specialized: Content Delivery & Edge Cache	6	2	2	2
Compute / AI / Cloud: Gut: Data-plane Intake & Waste	8	0	5	3
Compute / AI / Cloud: Skin: Perimeter & Admission	5	1	4	0
Distribution / Specialized: Financial Markets & Trading Systems	14	7	3	4
Compute / AI / Cloud: Media Streaming & CDN	14	5	4	5
Compute / AI / Cloud: Game Servers & Real-Time Interactive	12	7	3	2
Compute / AI / Cloud: Blockchain & Ledger Nodes	10	5	3	2

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Realm: family	Muscles	Write	Watch: at its limit on native	Watch: the same as native
Compute / AI / Cloud: CI/CD & Build Farms	10	6	3	1

61.6.3 Why the watch muscles are watch

- **421 muscles are at their limit on native.** Native already runs the knob at the top of its range: every admission knob lets everything in while change is permitted (275 admission knobs), a power knob runs the full clock or the full battery power (96 power knobs), and 50 capacity and setpoint knobs sit where native's own controller already has what the plant needs. There is nothing for the max to push; the knob is read and left native, STEADY by design, and no wire in is needed.
- **156 muscles read the same as native**: 37 of them wrote and nothing moved beyond the noise on work, speed or misses, 119 wrote nothing because native held the reading and the law had nothing to push. Watch, by rule; these are the room the next engine looks for a gain in, in the model gate first.

61.6.4 Power given, muscle by muscle

The max buys work and speed; the energy it takes is **power given**, shown beside what it bought and never a trade or a loss (the founder's order of 13 October). The write muscles whose power given on their own load is the largest:

Muscle	Family	Knob	Reading	Misses (pp)	Work	Power given
`db_replicas`	Database & Transactions	capacity	GAIN (fewer misses)	-5.54	+0.27%	+104.31%
`io_capacity`	Database & Transactions	capacity	GAIN (fewer misses)	-6.89	+0.35%	+96.73%
`read_route`	Database & Transactions	capacity	GAIN (fewer misses)	-6.31	+0.35%	+90.41%
`synchronous_replication`	Database & Transactions	capacity	GAIN (fewer misses)	-3.90	+0.39%	+89.14%
`payment_failover`	Commerce & Payment Systems	capacity	GAIN (fewer misses)	-5.71	+0.17%	+88.88%
`buffer_pool`	Database & Transactions	capacity	GAIN (fewer misses)	-3.82	+0.23%	+88.65%
`placement_group`	Cloud VM & Capacity	capacity	GAIN (fewer misses)	-2.49	+0.10%	+88.00%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Muscle	Family	Knob	Reading	Misses (pp)	Work	Power given
parallel_workers	Database & Transactions	capacity	GAIN (fewer misses)	-3.92	+0.24%	+87.92%
pulse_duration	Quantum Computing Control Simulation	setpoint	GAIN (fewer misses)	-0.85	+0.00%	+86.36%
interruption_response	Cloud VM & Capacity	capacity	GAIN (fewer misses)	-2.32	+0.14%	+83.97%
multi_cluster_dispatch	Cross-Cluster, Multi-Region & Edge	capacity	GAIN (fewer misses)	-2.39	+0.14%	+83.95%
nodepool_weight	Node Fleet & Karpenter-Class Control	setpoint	GAIN (fewer misses)	-2.99	+0.19%	+83.62%
db_failover	Database & Transactions	capacity	GAIN (fewer misses)	-6.15	+0.44%	+83.17%
kubelet_reserved_resources	Node Fleet & Karpenter-Class Control	capacity	GAIN (fewer misses)	-2.22	+0.26%	+82.84%
asg_min	Cloud VM & Capacity	capacity	GAIN (fewer misses)	-1.53	+0.01%	+82.21%
runner_scale_set_max_runners	CI/CD & Build Farms	capacity	GAIN (fewer misses)	-2.57	+0.31%	+82.16%
checkpoint_rate	Database & Transactions	capacity	GAIN (fewer misses)	-5.18	+0.46%	+81.95%
background_writer	Database & Transactions	capacity	GAIN (fewer misses)	-3.94	+0.26%	+81.69%
pulse_phase	Quantum Computing Control Simulation	capacity	GAIN (fewer misses)	-2.01	+0.00%	+81.01%
db_memory	Database & Transactions	capacity	GAIN (fewer misses)	-5.32	+0.09%	+80.86%

Of the 530 write muscles, 526 took power for their gain on their own load and 4 took none (the gain came from timing, not from more energy). Every muscle's figure, on both loads, is in the CSV.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

61.7 The organisms

61.7.1 The seven modelled organisms under three loads (every muscle written at once, one brain)

Organism and load	Muscles	Reading	Verdict	Figures
The spine, the muscles every stack runs on, the normal load	267 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.145 pp (-0.176 to -0.115), work +0.000%, power given +0.011%; valid: yes
Compute / AI / Cloud, the normal load	540 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.070 pp (-0.084 to -0.057), work +0.000%, power given +0.009%; valid: yes
Physics / Robotics / Autonomous, the normal load	404 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.121 pp (-0.145 to -0.097), work +0.000%, power given +0.010%; valid: yes
Energy / Facility / Industrial, the normal load	480 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.118 pp (-0.144 to -0.093), work +0.000%, power given +0.000%; valid: yes
Distribution / Specialized, the normal load	484 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.067 pp (-0.082 to -0.052), work +0.000%, power given +0.002%; valid: yes
The four stacked, every duplicate kept, the normal load	1908 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.092 pp (-0.106 to -0.078), work +0.000%, power given +0.000%; valid: yes
The whole tower, every muscle once, the normal load	1107 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.051 pp (-0.062 to -0.040), work +0.000%, power given +0.000%; valid: yes
The spine, the muscles every stack runs on, the climb load	267 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.632 pp (-0.697 to -0.567), work +0.003%, power given +0.039%; valid: yes
Compute / AI / Cloud, the climb load	540 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.390 pp (-0.421 to -0.358), work +0.002%, power given +0.046%; valid: yes
Physics / Robotics / Autonomous, the climb load	404 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.317 pp (-0.361 to -0.274), work +0.002%, power given +0.021%; valid: yes

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Organism and load	Muscles	Reading	Verdict	Figures
Energy / Facility / Industrial, the climb load	480 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.217 pp (-0.247 to -0.187), work +0.000%, power given +0.000%; valid: yes
Distribution / Specialized, the climb load	484 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.460 pp (-0.489 to -0.431), work +0.003%, power given +0.026%; valid: yes
The four stacked, every duplicate kept, the climb load	1908 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.349 pp (-0.370 to -0.327), work +0.002%, power given +0.000%; valid: yes
The whole tower, every muscle once, the climb load	1107 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.094 pp (-0.107 to -0.082), work +0.000%, power given +0.000%; valid: yes
The spine, the muscles every stack runs on, the line load	267 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.371 pp (-0.402 to -0.340), work +0.000%, power given +0.038%; valid: yes
Compute / AI / Cloud, the line load	540 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.205 pp (-0.217 to -0.193), work +0.000%, power given +0.045%; valid: yes
Physics / Robotics / Autonomous, the line load	404 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.250 pp (-0.275 to -0.225), work +0.000%, power given +0.030%; valid: yes
Energy / Facility / Industrial, the line load	480 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.117 pp (-0.143 to -0.092), work +0.000%, power given +0.000%; valid: yes
Distribution / Specialized, the line load	484 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.142 pp (-0.167 to -0.117), work +0.000%, power given +0.006%; valid: yes
The four stacked, every duplicate kept, the line load	1908 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.176 pp (-0.190 to -0.163), work +0.000%, power given +0.000%; valid: yes
The whole tower, every muscle once, the line load	1107 muscles	GAIN (fewer misses)	**write**	GAIN (fewer misses): misses -0.051 pp (-0.062 to -0.040), work +0.000%, power given +0.000%; valid: yes

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

An organism's label is the whole's: every muscle written at once. The wiring an operator takes from this page is finer: the write muscles wired in, the watch muscles left native with one wire out. The organism result says the whole, written at once, read GAIN (fewer misses, and under the climb more work) in every organism under every load, the power given shown beside it; this page says which parts carried it.

61.7.2 The organisms with the real cluster inside

The knob is the cluster's (the autoscaler's replicas, the floor and the machines), under the organism's own demand, with the modelled muscles around it. The cluster's gauges decide the verdict; the organism's modelled work and energy are shown beside them and are judged muscle by muscle above.

Cell	Verdict	The table's reading, and the figures	Source
compute_ai_cloud at 10 copies (GitHub)	**write**	each gauge's word: GAIN on 5 (response time (ms), 95th percentile; time over the response line (% of samples); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 0, power given on 0, STEADY (inside the noise) on 6. The cluster's gauges: 2 GAIN (p95 -50.3%, time over the line -31.6%), 0 LOSS, 0 power given. The organism's modelled energy -0.1%.	`results/live/V3_SIX_KUBE.md`
physics_robotics_autonomous at 10 copies (GitHub)	**watch**	each gauge's word: GAIN on 3 (organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 0, power given on 0, STEADY (inside the noise) on 8. The cluster's gauges: 0 GAIN, 0 LOSS, 0 power given. The organism's modelled energy -0.1%.	`results/live/V3_SIX_KUBE.md`
energy_facility_industrial at 10 copies (GitHub)	**operator's choice**	each gauge's word: GAIN on 4 (time over the response line (% of samples); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 0, power given on 1 (HPA replicas, mean), STEADY (inside the	`results/live/V3_SIX_KUBE.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Cell	Verdict	The table's reading, and the figures	Source
		noise) on 6. The cluster's gauges: 1 GAIN (time over the line -32.6%), 0 LOSS, 1 power given (replicas +0.6%). The organism's modelled energy -0.3%.	
distribution_specialized at 10 copies (GitHub)	**write**	each gauge's word: GAIN on 5 (response time (ms), 95th percentile; time over the response line (% of samples); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 0, power given on 0, STEADY (inside the noise) on 6. The cluster's gauges: 2 GAIN (p95 -51.7%, time over the line -35.9%), 0 LOSS, 0 power given. The organism's modelled energy -0.2%.	`results/live/V3_SIX_KUBE.md`
tower at 10 copies (GitHub)	**write**	each gauge's word: GAIN on 6 (response time (ms), 95th percentile; time over the response line (% of samples); failed requests (%); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 0, power given on 0, STEADY (inside the noise) on 5. The cluster's gauges: 3 GAIN (p95 -41.6%, time over the line -20.1%, failed requests -8.4%), 0 LOSS, 0 power given. The organism's modelled energy -0.3%.	`results/live/V3_SIX_KUBE.md`
stack at 10 copies (GitHub)	**write**	each gauge's word: GAIN on 5 (time over the response line (% of samples); failed requests (%); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 1	`results/live/V3_SIX_KUBE.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Cell	Verdict	The table's reading, and the figures	Source
		(organism work), power given on 0, STEADY (inside the noise) on 6. The cluster's gauges: 2 GAIN (time over the line -12.1%, failed requests -8.5%), 0 LOSS, 0 power given. The organism's modelled work -0.0006% (6 parts in a million, rounding level) (LOSS or power given) is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.4%.	
compute_ai_cloud at 100 copies (GitHub)	**write**	each gauge's word: GAIN on 4 (time over the response line (% of samples); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 1 (organism work), power given on 0, STEADY (inside the noise) on 7. The cluster's gauges: 1 GAIN (time over the line -9.8%), 0 LOSS, 0 power given. The organism's modelled work -0.0008% (8 parts in a million, rounding level) (LOSS or power given) is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.1%.	`results/live/V3_SIX_KUBE.md`
physics_robotics_autonomous at 100 copies (GitHub)	**write**	each gauge's word: GAIN on 4 (time over the response line (% of samples); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 1 (organism work), power given on 0, STEADY (inside the	`results/live/V3_SIX_KUBE.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Cell	Verdict	The table's reading, and the figures	Source
		noise) on 7. The cluster's gauges: 1 GAIN (time over the line -8.4%), 0 LOSS, 0 power given. The organism's modelled work -0.0011% (11 parts in a million, rounding level) (LOSS or power given) is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.1%.	
energy_facility_industrial at 100 copies (GitHub)	**write**	each gauge's word: GAIN on 7 (time over the response line (% of samples); energy, parked workers still on at idle power (Wh); energy (Wh); organism work; organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 0, power given on 0, STEADY (inside the noise) on 5. The cluster's gauges: 3 GAIN (time over the line -14.9%, parked-worker energy -0.3%, standby-model energy -0.3%), 0 LOSS, 0 power given. The organism's modelled energy -0.4%.	`results/live/V3_SIX_KUBE.md`
distribution_specialized at 100 copies (GitHub)	**write**	each gauge's word: GAIN on 6 (response time (ms), 95th percentile; time over the response line (% of samples); organism work; organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 0, power given on 0, STEADY (inside the noise) on 6. The cluster's gauges: 2 GAIN (p95 -31.9%, time over the line -21.9%), 0 LOSS, 0 power given. The organism's modelled energy -0.2%.	`results/live/V3_SIX_KUBE.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Cell	Verdict	The table's reading, and the figures	Source
tower at 100 copies (GitHub)	**write**	each gauge's word: GAIN on 5 (time over the response line (% of samples); organism work; organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 0, power given on 0, STEADY (inside the noise) on 6. The cluster's gauges: 1 GAIN (time over the line -8.7%), 0 LOSS, 0 power given. The organism's modelled energy -0.4%.	`results/live/V3_SIX_KUBE.md`
stack at 100 copies (GitHub)	**write**	each gauge's word: GAIN on 4 (time over the response line (% of samples); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 1 (organism work), power given on 0, STEADY (inside the noise) on 6; OFF THE CLOCK in 3 of 5 repetitions (the organism ended up to 277 s after its window: the machine could not step this many muscles in time, and the cell's last steps saw a cluster at rest). The cluster's gauges: 1 GAIN (time over the line -14.6%), 0 LOSS, 0 power given. The organism's modelled work -0.0008% (8 parts in a million, rounding level) (LOSS or power given) is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.4%. Off the clock in 3 of 5 repetitions.	`results/live/V3_SIX_KUBE.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Cell	Verdict	The table's reading, and the figures	Source
tower at 1000 copies (Azure)	**write**	each gauge's word: GAIN on 7 (response time (ms), 95th percentile; response time (ms), 99th percentile; time over the response line (% of samples); failed requests (%); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 1 (organism work), power given on 0, STEADY (inside the noise) on 4. The cluster's gauges: 4 GAIN (p95 -97.6%, p99 -97.9%, time over the line -100.0%, failed requests -100.0%), 0 LOSS, 0 power given. The organism's modelled work -0.0004% (4 parts in a million, rounding level) (LOSS or power given) is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.4%.	`results/live/V3_BIG_ORGANISM.md`
stack at 1000 copies (Azure)	**write**	each gauge's word: GAIN on 7 (response time (ms), 95th percentile; response time (ms), 99th percentile; time over the response line (% of samples); failed requests (%); organism energy (J); organism time over the line (% of steps); organism work per energy), LOSS on 1 (organism work), power given on 0, STEADY (inside the noise) on 3. The cluster's gauges: 4 GAIN (p95 -94.8%, p99 -96.9%, time over the line -100.0%, failed requests -100.0%), 0 LOSS, 0 power given. The organism's modelled work	`results/live/V3_BIG_ORGANISM.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Cell	Verdict	The table's reading, and the figures	Source
		-0.0006% (6 parts in a million, rounding level) (LOSS or power given) is a model gauge judged muscle by muscle in the realms table and shown here beside the cluster's. The organism's modelled energy -0.4%.	

61.8 Not decided here

- **Azure's managed Kubernetes (the bill):** the two tables are on Omni v1 (`results/live/V1_AKS_STEADY.md`, `V1_AKS_BURST.md`) on a 4-worker fleet, where every gauge read STEADY, no difference beyond the noise, but the burst's p99 in one run. A fleet too small to show one machine decides nothing; the fleet that can (`docs/K8S_COMPASS_PREREGISTRATION.md`, the fleet that can show one machine) decides this knob on v3 when it runs. Until then the verdict on Azure is watch, for want of a result rather than because of one.
- **The card (NVIDIA, its own meter):** every earlier card result is obsolete; the current governor has not run on a real card. Nothing to decide yet.
- **The 24-hour robustness machines:** running; they test the governor's own staying power, not a knob's value.

61.9 What this means for the wiring

The two-way plug stays one wire in and one wire out (`docs/WIRING_GUIDE.md`). This page decides, per knob, whether the wire in is connected. A knob marked **write** is wired both ways. A knob marked **watch** is wired out only: Omni reads it, learns from it, and never writes it; the native controller lives by itself. A knob marked **operator's choice** is wired out, and in only by the operator who wants that trade. The engine's verdict (`omnicompass/verdict.py`) does the same on the muscle itself at run time, and a knob it has left native is a watch knob whatever this page says. Nothing on this page changes the engine, which stays Omni v3; it changes what an operator connects.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

61.10 The same verdict, live: the brain decides on the knob before it writes

This page judges after the fact, from published tables. The founder's order of 9 October is that the brain judge in real time, on the knob, before it writes, and that a knob which cannot prove it pays stay native. On the five live stacks (the pool, the consumer group, the memory ceiling, the storage-engine cache, the buffer pool) that is now built into the harnesses: `tools/knob_verdict.py` wraps the frozen engine's own verdict (`omnicompass/verdict.py`, unchanged) around every live knob. The knob starts in watch; a paired trial on the stack itself (the knob held at the deepest step already allowed, then one notch further, under the same traffic) allows one notch at a time under the declared objective, the resource reading of the index by default or the service reading at the operator's choice; a refused step is not taken and not retried for a minute; a trial holds the knob; a fail-up never spends beyond the allowance; the operator's setting is always free. The audit carries the cost sample and the verdict's state every second, the arm record sums it, and the three-run tables print the verdict per workload. It is declared as an amendment in each of the five preregistrations with the expectation written before the runs (Redis: left native under the resource objective; Kafka: allowed step by step while each consumer pays; PostgreSQL: the add above the operator's setting refused on `simple_update`; MySQL: the `read_write` chunks refused; MongoDB: the cache given back a notch a trial). The runs on it were dispatched on 10 October 2026 (`docs/RERUN_2026-10-10.md`); this page is then read again from their tables. Inside the modelled realms the organism still takes one directive for every muscle; the verdict per muscle there is the next engine, Omni v4, designed in `docs/OMNI_V4_PLAN.md` and not built.

Where the brain's verdict stands today, knob by knob. Who decides, in real time, whether a knob pays before it is written, and in which directions (spend: more resource for service; give back: less resource where the service holds):

Where	The knob	Who decides it pays, and when	Directions tried	Engine
The five live stacks: Redis's memory ceiling, Kafka's consumers, PostgreSQL's pool, MySQL's buffer pool, MongoDB's cache	one knob each	`tools/knob_verdict.py` around the engine's verdict: the knob starts in watch; a paired trial on the stack itself before every notch, one notch a trial, under the declared objective	spend and give back, both tried; the operator's setting always free	outside the engine (Omni v3 unchanged); running since 10 October
Kubernetes: the machines	nodes in service	the engine's verdict (`omnicompass/verdict.py`): a paired trial before a machine	give back (a machine taken only when the trial shows the service holds); gas and brake	Omni v3

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Where	The knob	Who decides it pays, and when	Directions tried	Engine
		is taken	on replicas and the floor by the compass law's band, no trial	
The card: the clock ceiling	one 15 MHz step at a time	the engine's verdict: a paired trial per step while the service is calm; no step passing, the card runs as it does alone; the power lid stays at the operator's	give back (slower clock for the same service); the speed floor never under the card's own busy clock	Omni v3 (`omni_controller/gpu_compass.py`)
Robot arms: the speed override; drones: the cruise	one knob each	the engine's verdict on the robot and the swarm; two of four robots left native by it	give back	Omni v3
Power grids: the tap; buildings: the batteries	one knob each	the compass law alone, no trial; the trades on four exporting grids and seven 2023 districts are read from the tables above as the operator's choice	the law's own push and pull	Omni v3 (a trial there is the next engine)
The 1,107 modelled muscles	each muscle's knob	the max on every muscle, every muscle its own card; the knobs native already runs at the top of their range read and left native	the law's own push and pull	Omni v5 at the max (the law omni-v6 carries), `docs/OMNI_V5.md`
The machine itself: the Linux kernel's frequency ceiling (`docs/CPU_POWER_PREREGISTRATION.md`)	one knob	`tools/knob_verdict.py` around the engine's verdict: every notch down tried on the machine first, under the index's	give back only (the ceiling cannot go above the top; the top always free)	outside the engine; built 10 October, waiting for a machine on the metal, no run yet

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Where	The knob	Who decides it pays, and when	Directions tried	Engine
		reading for one machine (energy x latency / work) or energy per request inside the line		

CHAPTER 62

The Pilot Protocol and Kit

62.1 Pilot Protocol

> **Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 42.5 of the manual (*Wired right or wired wrong*).

A pilot is the one proof our own runs cannot give: Omni-Compass on top of the licensee's own native controllers, on the licensee's own systems and work, read against native by the licensee's own engineers. Every result The Omni-Compass LLC publishes was run and scored by its own code, and every table says that no outside party has reviewed or approved it. A pilot run by this protocol is how that changes. The protocol was rewritten on 31 October 2026 for the engine that stands: native stays on in every arm, Omni-Compass moves native's knobs and never replaces native, and the readings are the receipts and the service gauges, read by the one key.

62.1.1 Before day one: the agreement and the license

- The evaluation license covers evaluation and simulation: test and staging systems, the models and `verify.py`.
- Governing live, customer-facing traffic is production use, and production use needs the signed, paid Omni-Compass Enterprise License first.
- A pilot on a production system, the read-only shadow included, runs only under a pilot agreement signed with The Omni-Compass LLC before day one. The agreement names the systems, the dates, the rules on this page and who receives the readings.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

62.1.2 What runs, and what does not

- **The governor** (`omni_controller/controller.py`; on a graphics card, `omni_controller/gpu_governor.py`) sits on top of the native controllers already there: Kubernetes' pod autoscaler and cluster autoscaler, the card's own power management. Native stays on in both arms. Omni-Compass sets native's knobs (the autoscaler's target and range, the machines in service, the CPU limits, the card's power limit) and never takes native's place.
- **The engine at its factory tune**, as shipped (`docs/FACTORY_PROFILE.md`), at the factory beat. Any knob may be turned. A changed tune's readings are its own, and turning a knob can change every metric in either direction (`DISCLOSURES.md`, section 3).
- **Not the pilot:** the C++ engine and `verify.py`. They check that the engine's two copies compute the same and that the code does what its rules say (phase 0). They govern nothing.
- **Omni-Compass by itself** (the third arm: native's autoscalers off, Omni-Compass the only controller on those knobs) runs only where the licensee chooses it, after phase 2, preregistered as `docs/SOLO_PREREGISTRATION.md` is.

62.1.3 Phase 0: the code checked (one day, the evaluator's own machine)

Run `python3 verify.py`. Pass: `VERIFICATION: PASS`. This includes the parity tests of the C++ engine against the Python (`tests/test_cpp_*_parity.py`), the shield's invariants and the reading rules. `python3 tools/reproduce.py` rebuilds every published table from its archived raw runs and exits 1 on any difference.

62.1.4 Phase 1: shadow (one week, no write rights)

- The shadow kit (`docs/PILOT_KIT.md`, `scripts/pilot_shadow.sh`): the governor in observe mode at the factory beat, under an identity that can read and cannot write. The kit proves this with `kubectl auth can-i` and stops if any write right exists.
- Logged at every decision: what Omni-Compass would set, what native did and what followed (`shadow_out/audit.jsonl`), with the count of writes, which must be 0.
- Pass, agreed before the start: no write, and no logged decision that breaks the shield's invariants I1 to I5 (claim C7, `docs/CLAIMS_REGISTER.md`).

62.1.5 The handover (one day)

One muscle at a time, in this order: the power cap, the machines in service, the pod autoscaler's target and range, then the CPU limits. `--native MUSCLE[,...]` leaves every muscle not yet handed over where native runs it. At each handover the reset is tested: everything goes back to native, and the native settings are read back equal to the snapshot taken at the start.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

62.1.6 Phase 2: the paired pilot (30 days: three runs of 10 days, A, B and C)

- **The pairing:** two matched pools at the same time, the same work split between them: native alone on one, Omni-Compass on top of native on the other. Where one pool is all there is, alternate whole days between the arms, with the same days of the week in each.
- **The three runs:** days 1 to 10 are run A, days 11 to 20 run B, days 21 to 30 run C. Each run is read on its own, and the three are read together by the three-run rule (`tools/confirm_abc.py`).
- **The stops:** the kill switch is tested on the first day and the last. It is instant, for security only, and puts every setting back to native (`omnicompass/master.py`, `tools/omni_switch.py`). The lease is tested once: stop the governor's process and watch the plug walk every setting back to native gradually.
- **Pass:** the pilot holds. Nothing the service is judged on reads LOSS against native in run A, B or C by the one key, and every resource that rose is read beside the work it bought, as power given.

62.1.7 What to read: the receipts and the service gauges

Where	The receipt, native against Omni-Compass	Read from
Through the cloud	the work per dollar: the work completed over the provider's bill for each arm's machines	the provider's own cost export and the licensee's request or job counts
The chip	the work per joule: the work completed over the card's own energy counter, with its clocks, temperature and throttle reasons beside it	the card's published interface (`nvidia-smi`, NVML)
On site	the work per machine-hour: the pool is the owner's, so its hours are no bill	the capture below
The service, every place	the 95th percentile response time (the 90th and 99th beside it), the minutes over the response line, the pods waiting for a place, the failed requests, the work completed	the licensee's own monitoring
Omni-Compass itself	every write it made, read back, and every reason	its audit (`--audit`)

Capture both arms read-only with `fleet/capture/kube_capture.sh`, then score each run:

```
python3 pilot/score.py --baseline native.csv --omni omni.csv [--idle-w W --dyn-w W]
```

It reports node-hours and energy per used CPU core-hour, utilisation, pending-pod minutes and the autoscaler's shortfall minutes, each with a bootstrap 95% interval over hourly blocks. Energy is measured only

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

where the capture holds a meter's `power_w`; where no meter reads it, energy is not scored. Each run needs at least 24 hourly blocks per arm (a 10-day run holds about 240).

62.1.8 Reading it

Every gauge is read by the one key (`tools/readings.py`, `docs/HOW_TO_READ_THE_RESULTS.md`): GAIN, STEADY or LOSS by what the gauge is for, never by the sign of its number. Time waited, missed or failed that fell is a GAIN; work that rose is a GAIN; a resource spent or held that fell is a cut, a GAIN, and one that rose is power given, never a LOSS. A mean change under 0.5% reads STEADY. Across the three runs a reading is GAIN or LOSS in all three when every run moved the same way clear of zero; the runs disagree when clear runs moved in opposite directions, and that is a finding to look into, not a result.

62.1.9 Reporting

Every row is reported, LOSS included, in the key's words, with the run it came from. The readings, the captures and their SHA-256 sums go to the licensee and to The Omni-Compass LLC as the pilot agreement says. Where the pilot's direction differs from the published benchmarks, the first presumption is the wiring (section 42.5 of the manual), and the pilot goes on once the wiring is checked.

62.2 Omni-Compass shadow pilot kit

What a customer runs first. Omni-Compass watches the cluster read-only beside its own autoscalers. At the engine's factory beat (every 60 s; `OMNI_INTERVAL` sets another) it decides what it would do, using the same closure law that was benchmarked. It never writes.

1. `kubectl config use-context <the cluster>`
2. `DURATION=86400 IDLE_W=<watts per idle node> DYN_W=<watts at full load> bash scripts/pilot_shadow.sh`
   - It proves with `kubectl auth can-i` that the identity can read and cannot write, and stops if any write permission exists.
   - It captures telemetry and logs every recommendation to `shadow_out/audit.jsonl`.
   - It writes `shadow_out/SHADOW_REPORT.md`: node-hours used against node-hours Omni-Compass would have used, and the write count, which must be 0.
3. Scoring against a matched baseline: `python pilot/score.py --baseline baseline.csv --omni omni.csv` (bootstrap intervals over hourly blocks).

Guarded control follows `docs/PILOT_PROTOCOL.md`: one loop at a time, the reset tested at each handover.

The kit is exercised end to end on kind by the `live-shadow` workflow.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 63

The GPU Bench

> **Before you wire anything:** read `DISCLOSURES.md`. Omni-Compass acts only through the wires it is given; it cannot be slapped on. If your paired receipts differ from the published benchmarks in direction, the first presumption is wiring: check the installation with section 42.5 of the manual (*Wired right or wired wrong*).

This is the test that answers "does Omni-Compass save real energy?" with the GPU's own power meter. There is no model in it. Nobody has run it yet on real hardware for this repository; the first run is the first real-meter result.

63.1 What you need

- A Linux machine with one NVIDIA GPU (a rented cloud GPU works: A100, H100, L4, A10, RTX), the NVIDIA driver and `nvidia-smi`.
- Python 3 with PyTorch built for CUDA (`pip install torch numpy`).
- Root, because setting a GPU power limit (`nvidia-smi -pl`) needs it. Persistence mode on is recommended (`sudo nvidia-smi -pm 1`); the receipt records it either way.

63.2 Run it

```
git clone <this repository> && cd <it>
sudo bash scripts/gpu_paired.sh                          # 5 repetitions,
    ↪  10 minutes per arm: about 3 hours
sudo REPS=5 DURATION=300 bash scripts/gpu_paired.sh   # 5-minute arms:
    ↪  about 1.5 hours
```

Options (environment variables): `REPS`, `DURATION` (s per arm), `DRAIN` (s after arrivals stop), `COOLDOWN` (s idle before each arm), `GPU` (index), `SAMPLE_MS`, `INTERVAL` (s between Omni decisions), `SLO_MS` (response-time target; default ten bare service times), `WORKLOAD_ARGS` (e.g. `--n 8192 --target-ms 80`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

63.3 Two phases

- `PHASE=smoke` (the default): look for faults and for an effect worth a counted run. Any number of repetitions.
- `PHASE=confirm`: the preregistered test (`docs/GPU_PREREGISTRATION.md`), 10 repetitions. Omni's code must be committed; its files are hashed before the first arm and again after the last, and any change invalidates the run.

```
sudo PHASE=confirm bash scripts/gpu_paired.sh
```

The **primary outcome** is work per energy: requests served per kilojoule the GPU drew.

63.4 What it does

1. **Receipt.** GPU name, UUID, VBIOS, driver, kernel, host, persistence mode, power management, the enforced limit, default, minimum, maximum and current power limit, the workload's hash, the mechanism id, git commit (`receipt.json`). The current power limit is the **snapshot**; every arm must start and end at it. The bench refuses to start unless power management is Enabled (otherwise a written limit would not bind).
2. **Calibrate once.** The workload times its request at the snapshot limit and picks the request size so one request takes about 50 ms (`calib.json`). Every arm uses the same calibration.
3. **Three arms per repetition, order rotated,** each after `COOLDOWN` s idle:
   - **native**: no Omni process.
   - **watch**: Omni runs, reads the GPU and decides, and is forbidden to write. This is the control: it shows what the machine does with Omni present and silent. If it writes even once, the run is invalid.
   - **omni**: Omni writes the GPU power limit.
4. **The same work in every arm.** `tools/gpu_workload.py` sends one seeded stream of requests (fp16 matrix products) at 30%, 60%, 80%, 30%, 60% and 30% of the GPU's full-power capacity. Every arm gets the same requests at the same moments.
5. **Measured by the device.** `nvidia-smi` samples power draw, temperature, utilisation, the power limit, the **enforced** power limit and the clock-limit reasons every 200 ms for the whole arm (the fields the driver reports, listed in `smi_fields.txt`); RAPL CPU package counters are read at both ends where the machine has them. This is receipt C, the outcome: Omni never supplies it.
6. **Reset.** After the omni arm Omni restores the snapshot limit and reads it back. The script checks the limit after every arm.
7. **The table** (`GPU_REPS.md`): each gauge for native, watch and omni, and three paired contrasts with 95% intervals — observation (watch − native), authority (omni − watch), total (omni − native). If an interval includes zero, it reads **STEADY, no difference beyond the noise**; wholly on one side, **GAIN** (or **LOSS**, or **power given**) **clear of the noise** (the words of amendment 14). The result label is chosen by rule

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(`docs/GPU_PREREGISTRATION.md`, amendment 1): SUPERIOR WITHIN GUARDRAILS, ENERGY IMPROVEMENT WITH SERVICE TRADEOFF, NONINFERIOR / INCONCLUSIVE, NOT ESTABLISHED, AWAY FROM ITS AIM, or INVALID. Also: actuator fidelity (receipt B), control effort and representation fidelity (receipt A). A meter that was not fitted prints UNAVAILABLE. The run folder holds every raw file and `SHA256SUMS.txt`.

63.5 What Omni does on the GPU (omni_controller/gpu_governor.py)

Every 2 s it reads the GPU and runs the Omni-Compass engine (the throughput law in `omnicompass/adapter.py`): load is GPU utilisation, power stress is draw over the snapshot limit, heat is temperature over 83 C, queue is response-time pressure. The engine's power cap becomes a power limit, inside hard rules:

- never below the GPU's current draw x 1.3, never below 0.70 of the snapshot (`--min-share`), never below the device minimum, never above the snapshot;
- a busy card (utilisation smoothed over decisions at or over 0.5, `--util-gate`) gets the snapshot limit back at once, and the cap returns only under 0.4;
- optional speed lock (`--baseline-file`, from `tools/gpu_baseline.py` on runs without Omni): the limit follows response time against that baseline, every gauge kept at least 1% faster (`docs/INTEGRATION_MANUAL.md`, level 5);
- no new write until the last one reads back from the device; every write is read back at once and recorded (requested, return code, read back, enforced limit, delay); the engine senses the enforced limit, the one the card obeys; a write the device refuses ends the arm (exit 4) and makes the run invalid; no clock locks are ever written;
- if it cannot read the GPU or the response times, the snapshot limit at once;
- if response time breaks its target, the snapshot limit at once, and for three decisions after;
- kill file or SIGTERM: the snapshot limit, read back.
- **one writer:** if the limit ever reads a value Omni did not write, Omni stops writing, leaves that limit alone, and exits 5 (the run is invalid);
- **heat fails up:** while the card reports a thermal or hardware slowdown, no lower limit is written;
- every decision names every rule that held the engine back (`blocked_by`) and the one that decided (`decided_by`).

Any workload plugs in through `WORKLOAD_CMD` (for example a vLLM or MLPerf inference harness), if it writes `latency.csv`, `requests.csv` and `summary.json` in `tools/gpu_workload.py`'s format and `SLO_MS` is given.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

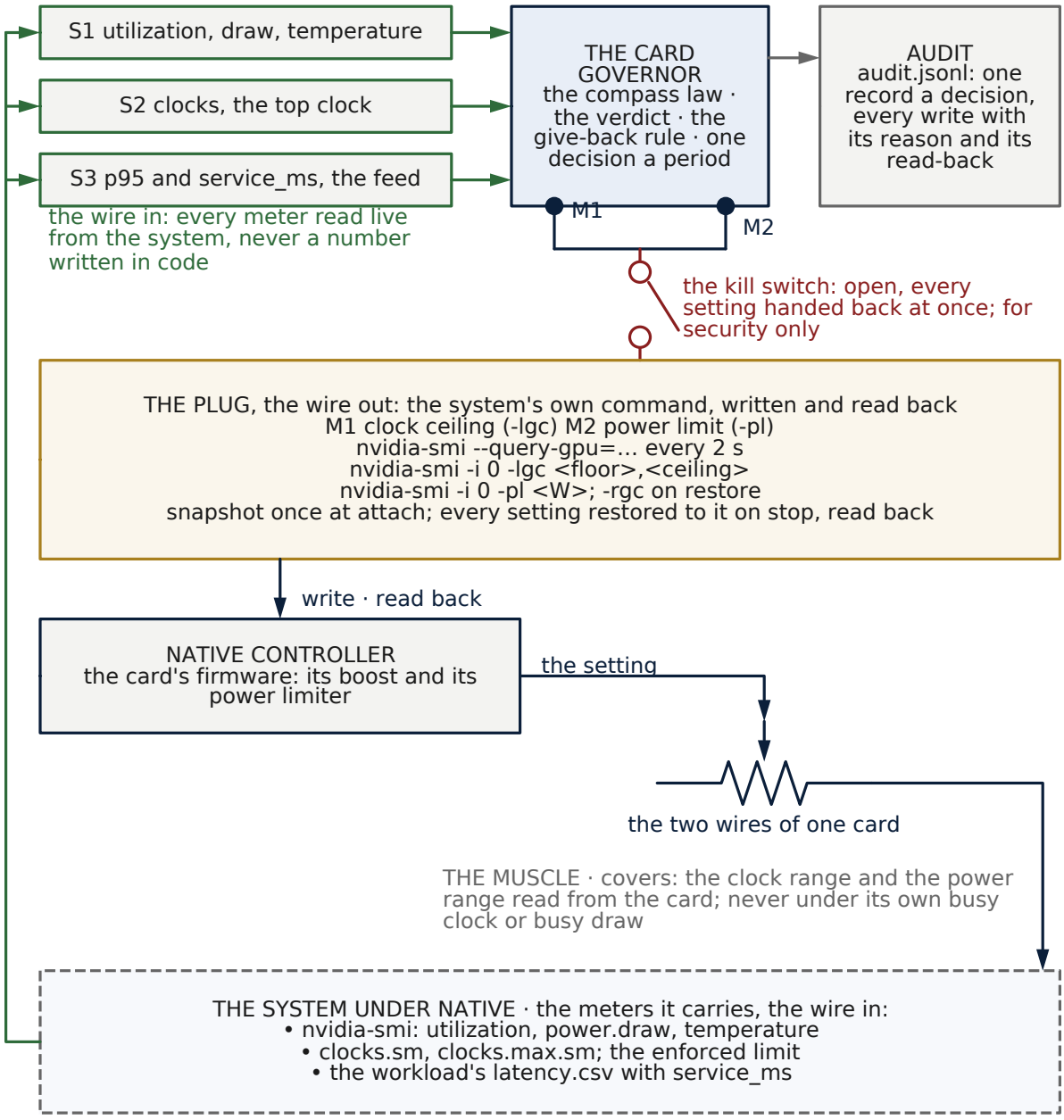


Figure 74. Wiring schematic: a graphics card, two wires. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the card's utilization, draw and temperature, its clocks and top clock, and the workload's feed; the two motors are the clock ceiling (-lgc) and the power limit (-pl), both through nvidia-smi and both read back. Learns the card on its own first (15 busy readings); races while

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

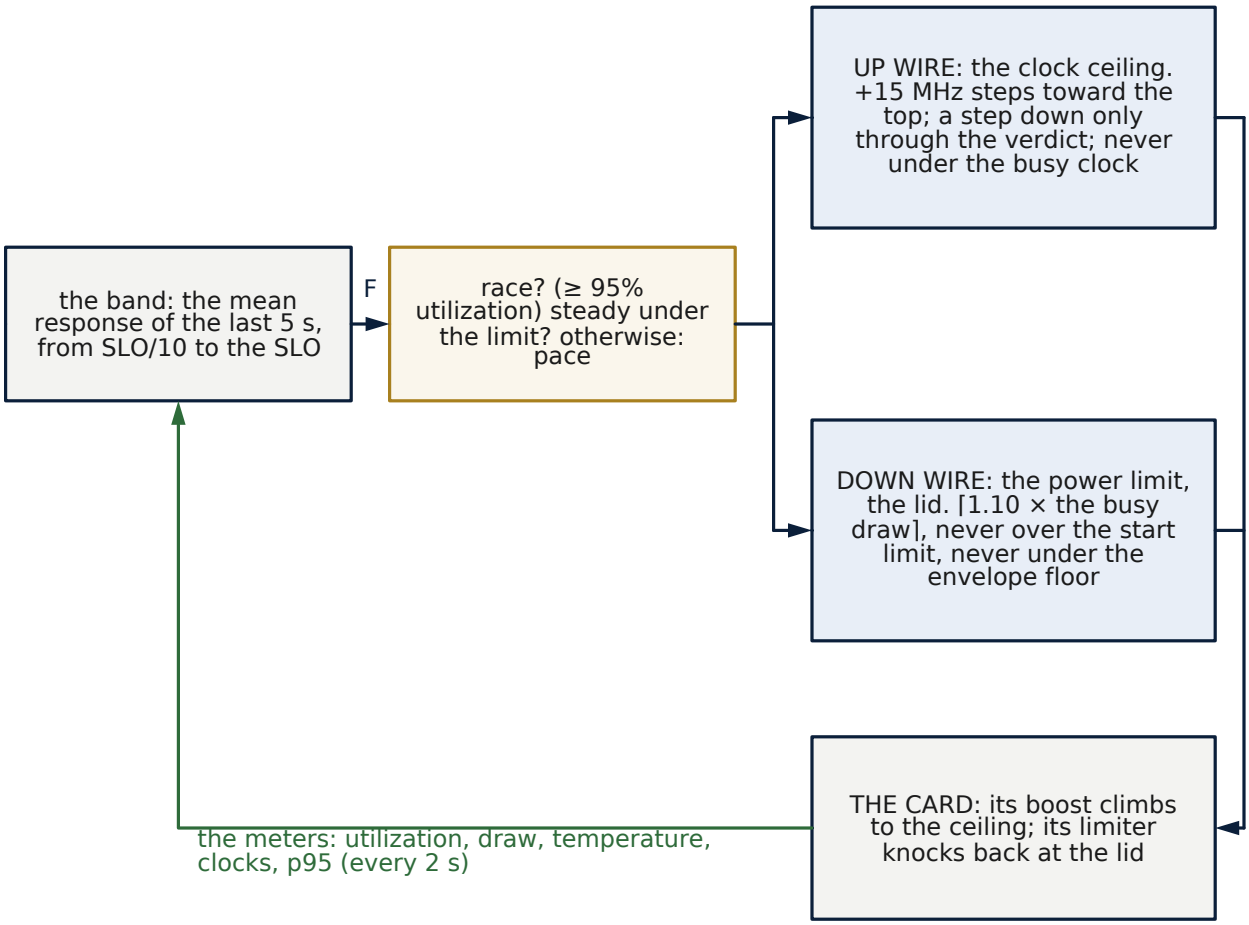


Figure 75. Block diagram: the two wires of a card as an antagonist pair. The band (the mean response of the last 5 s, from a tenth of the line to the line) yields the force F; the mode decides between racing (both wires to full while work waits, utilization ≥ 95%), steady under the limit (the card's own busy clock as the ceiling) and pacing.

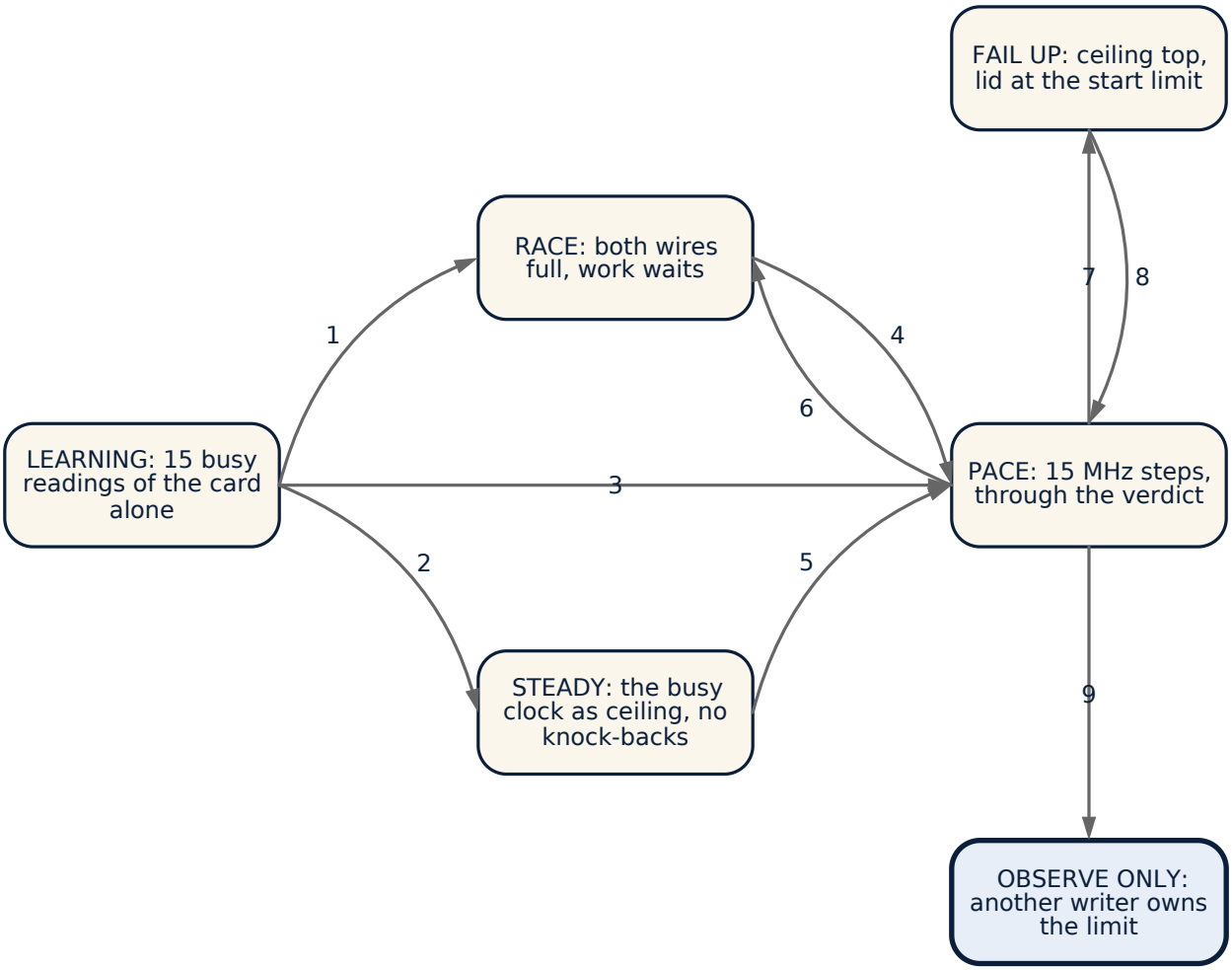
The up wire is the clock ceiling, raised in 15 MHz steps toward the top and lowered only through the verdict; the down wire is the power limit, the lid at ⌈1.10 × the busy draw⌉, never over the start limit and never under the envelope floor. The card's own boost climbs to the ceiling and its own limiter knocks back at the lid; the meters close the loop every 2 s.

work waits (≥ 95% utilization); at the max never below the top while the card serves; at the tune-down paces the slack in 15 MHz steps, each step through the paired trial.

The table also credits each write: joules and requests against native at the same moments of the same stream, grouped by the rule that decided it. CPU energy comes from RAPL by domain: package and DRAM apart,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

psys never added, wrap undone. The card's own energy counter (NVML) and its ECC and retired-page counters are read at both ends of each arm.



1 learned; utilization ≥ 95% · 2 learned; saturated at an operator's cap (draw ≥ 97% of it) · 3 learned; slack · 4 slack (the tune-down) · 5 off the cap: slack · 6 work waits again · 7 from any state: p95 at the line, a failed request, a blind feed, a heat slowdown · 8 clear for --slo-clear · 9 from any state: a limit not written by the governor

Figure 76. State machine: the two-wire card governor. It learns the card on its own first (15 busy readings), then races while work waits (utilization at 95% or more), holds steady at the card's own busy clock while saturated under an operator's cap, or paces the slack in 15 MHz steps through the verdict; from any state it fails up on a breach, a failed request, a blind feed or a heat slowdown, and it observes only, for the rest of the run, once a limit is found that it did not write.

63.6 Gauges

Gauge	From	Its aim
work per energy (served requests per kJ), primary	requests served / GPU energy	higher

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Gauge	From	Its aim
energy, GPU (J)	power.draw integrated over the arm's window	lower
energy per served request (J)	the same, over requests served	lower
power, GPU mean (W)	energy / window	lower
requests served, not served	the workload's own record	more served, fewer not
response time mean, 95th, 99th percentile (ms)	arrival to finish of each request	lower
temperature, peak and mean (C)	temperature.gpu	lower
energy, CPU package (J)	RAPL counters, when present	lower

63.7 Testing the bench without a GPU

`python tests/test_gpu_bench.py` runs the whole script against a stand-in `nvidia-smi` (`tests/standin_gpu/`), with the workload's `--sim` mode. The stand-in has no real power physics, so its numbers mean nothing; it proves the script, the controls and the validity checks work.

CHAPTER 64

The GPU Preregistration

Written before any hardware trial. The counted run (`PHASE=confirm`) hashes this file with the code in `FREEZE.json`; a change to either after the run starts invalidates it.

64.1 Question

On one NVIDIA GPU serving a fixed, seeded request stream, does Omni-Compass holding the GPU power limit change the successful work done per joule measured by the device, compared with the device left at its own limit?

Fixed here, before any smoke trial:

- the number of counted repetitions (10);
- the primary outcome;
- the guardrails;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- the analysis.

Nothing seen in smoke may change them.

64.2 Two phases

1. **Smoke** (`PHASE=smoke`, any number of repetitions). Its purpose is to find faults in the harness, and to see whether the effect is large enough to be worth a counted run. Smoke results are not published and are never reported as the result. After smoke, Omni's code may change.
2. **The counted run** (`PHASE=confirm`). Omni's code is committed and frozen before the first trial. The script refuses to run if any frozen file has uncommitted changes, and the table marks the run invalid if a frozen file changes during it. No inspection, tuning or rerun between counted trials. If the counted run fails, it is reported as failed; a new counted run needs a new commit and a new run, and both runs are reported.

64.3 Design

- **Arms:** native (no Omni), watch (Omni runs, may not write), omni (Omni writes the GPU power limit).
- **Repetitions:** 10 in the counted run, each with all three arms back to back, order rotated. Repetitions may run on separate machines of one GPU type (`REP_ONLY`, the GitHub workflow); the three arms of a repetition always share one machine, so every comparison is paired within a machine.
- **Per arm:** 60 s idle, then the pinned workload for 600 s plus 30 s drain.
- **Workload:** `tools/gpu_workload.py`, calibrated once at the start power limit before any arm. The seed is 20260928, and the load phases are 30, 60, 80, 30, 60 and 30% of full-power capacity.

64.4 Outcomes

- **Primary:** work per energy = requests served / GPU energy (kJ). GPU energy is `nvidia-smi` power.draw integrated over the arm's window.
- **Secondary:**
  - GPU energy;
  - energy per served request;
  - requests not served;
  - response time: mean, 95th and 99th percentile;
  - peak temperature;
  - CPU package energy (RAPL), where present;
  - where a smart plug is fitted (`WALL_METER`), whole-machine energy at the wall and work per wall kJ. The plug is read by the bench only, never by Omni. An arm whose plug readings have a gap over 5 s has no wall number.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

64.5 Analysis

- **Primary comparison:** omni against native, paired by repetition. Report the mean difference with a two-sided t 95% interval.
- **Result:**
  - The result is *a proven GAIN* if the interval lies entirely above zero.
  - It is *a a proven LOSS* if the interval lies entirely below zero.
  - Otherwise it is *not proven*.
- **Guardrails, fixed now:** a primary GAIN counts only if Omni did not buy it with the work. Both guardrails must hold:
  - **Requests served:** the 95% interval of (omni − native) must not reach below −1% of native.
  - **95th-percentile response time:** the interval must not reach above +10% of native.

If a guardrail fails, the verdict is *more requests per kJ, fails the service guardrail*. That is a different product and is reported as such.

- **Watch against native:** reported as the cost of Omni being present. If watch differs from native on the primary outcome as much as omni does, the effect is not attributed to Omni's authority.

64.6 Amendment 1 (2026-09-28, before any hardware trial; no smoke or counted data exist)

Added before any data, to make the chain from engine to plant identifiable. The question, the arms, the primary outcome, the repetitions, and the two guardrails above are unchanged.

- **Three contrasts, all reported, each paired by repetition:** observation = watch − native; authority = omni − watch; total = omni − native (the primary comparison). A total effect is not attributed to Omni's authority where the observation contrast differs materially from zero.
- **Third guardrail, errors:** the 95% interval of (omni − native) requests *not* served must not reach above +1% of native requests served.
- **Result label, by rule, never by hand** (`tools/gpu_reps.py`, `label()`):
  - primary a proven GAIN, all three guardrails held: **SUPERIOR WITHIN GUARDRAILS**;
  - primary a proven GAIN, a guardrail failed: **ENERGY IMPROVEMENT WITH SERVICE TRADEOFF**;
  - primary not proven, all guardrails held: **NONINFERIOR / INCONCLUSIVE**;
  - primary not proven, a guardrail failed: **NOT ESTABLISHED**;
  - primary a proven LOSS: **AWAY FROM ITS AIM**;
  - any invalidity below: **INVALID**.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The card obeys enforced.power.limit.** The snapshot records power.limit, enforced.power.limit, default, min and max limits, persistence mode and power management. The bench refuses to start unless power management is Enabled. The bench samples enforced.power.limit where the driver reports it. The governor senses the enforced limit and reads back every write at once; it never writes clock locks.
- **Also invalid:** a native or watch arm whose enforced limit differs from the snapshot; a governor that exits nonzero (a refused start, or a write the device refused — that write ends the arm); smoke and counted repetitions mixed in one table.
- **Three receipts, kept apart.** A (governor, `audit.jsonl` decisions): telemetry consumed; the six-state reading (history-dependent, `state_observed`); the memoryless state the telemetry alone points to (`state_measured`); the U-channel command u evaluated on the evolved state (`u_push`); admissibility; requested and granted authority; shield bound; holds. B (actuator, `actuator` records): requested limit, return code, power.limit read back, enforced limit, delay to realization, override (enforced under requested), restoration. C (outcome, the bench alone): nvidia-smi power, joules, temperature, utilization, clock-limit reasons; the workload's requests, latency and failures. Omni never supplies its own outcome.
- **Secondary, descriptive (no verdict):**
  - actuator fidelity: r[act] = read back − requested (mean and max absolute), delay (median, max), enforced-under-requested count, total variation of the realized limit, reversals, refused writes;
  - control effort: J_u = sum of abs(u[push]) × decision interval; mean and max abs(u[push]); saturations (abs(u[push]) = 25); shield interventions (a bound other than the engine decided); holds;
  - representation fidelity: R[int] = sqrt(mean over consecutive decision pairs of sum_k w_k (h(z_(k+1)) − F_h(h(z_k), u_k))_k^2) over E, U, I[U], S, B, weights 1, with h the memoryless map and F_h the engine's own projection (`state_projected_next`); directional accuracy = share of pairs where sign(projected − current reading) equals sign(next measured − current measured), both movements at least 0.01, per state and by predicted size (0.01–0.03, 0.03–0.1, ≥ 0.1);
  - energy of the rest of the machine = wall − GPU − CPU package, only when all three meters measured the arm. A missing meter prints UNAVAILABLE, never a modelled substitute.
- **Smoke never enters the counted run.** A smoke run that changes any code or parameter is followed by: repair, the tests, a new freeze, and a counted run collected from zero.

64.7 The run is invalid, and reported as invalid, if

- the watch arm executes any power-limit write;
- a native or watch arm sees a power limit other than the snapshot;
- any arm ends at a limit other than the snapshot (the reset failed);
- Omni's frozen files change during the run;
- the counted runs on uncommitted code.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

64.8 Recorded for every Omni decision

Every decision is logged in `audit.jsonl`:

- the raw device telemetry: utilisation, draw, temperature, limit, SM clock, and clock-limit reasons where the driver reports them;
- the engine's six-state reading, the state it had projected for this moment, and the error against it;
- the projection for the next decision. This is the engine's own evolved state from the same step that sets the cap; no separate predictor was added for the experiment;
- whether change was admissible, the requested and granted authority, and which shield bound decided the limit;
- the limit written, and the requests served in the window.

64.9 Amendment 2 (2026-09-28, before any hardware trial; no smoke or counted data exist)

Unchanged:

- the question;
- the arms;
- the primary outcome;
- the repetitions;
- the guardrails of amendment 1.
- **Watch must match native.** If the observation contrast (watch − native) on the primary outcome is proven in either direction, the label is **NOT ATTRIBUTABLE: WATCH DIFFERS FROM NATIVE** and no omni result is published.
- **One writer.** If the power limit ever reads a value that is neither Omni's last write nor the limit before it, another writer is present:
  - the governor stops writing for the rest of the run;
  - it leaves that writer's limit alone;
  - it exits 5;
  - the run is invalid.
- **Heat fails up.** While the device reports a thermal or hardware slowdown (clock-limit reason bits 0x8, 0x20, 0x40, 0x80), no lower limit is written.
- **Credit per write (descriptive).** Each write owns the interval to the next. For that interval, the table records GPU joules and requests finished, Omni minus native, at the same moments of the same seeded stream. It records who decided the write: the engine, a floor, the busy gate, a reflex, heat, or the speed lock. The table says which rule produced the joules; a speed-lock result is not credited to the engine.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **CPU side (secondary, never on the control path).** RAPL counters are read by domain name at both ends of each arm:
  - `package-N` is summed as CPU package; `dram` is summed separately;
  - `psys` is recorded and never added to either;
  - wrapping is undone with `max_energy_range_uj`;
  - a counter that went backwards without a known range, or is missing, prints UNAVAILABLE.
  - The governor never reads these counters.
- **Device energy counter (cross-check, secondary).** Where NVML reports it (Volta and newer), the card's total-energy counter is read at both ends of each arm, beside the integrated power.draw.
- **Card health (descriptive).** Uncorrected and corrected ECC error counts, and pages pending retirement, are read at both ends of each arm.
- **Workload plug.** Any workload may be served through `WORKLOAD_CMD` if it writes the pinned workload's files. It needs its own response-time target (`SLO_MS`). The command is recorded in the receipt. A counted run names its workload before the first trial.

64.10 Amendment 3 (2026-10-01, before any hardware trial; no smoke or counted data exist)

No GPU job has ever been given a machine (every gpu-bench run so far waited in the queue and was cancelled), so no trial data exist. Unchanged:

- the question;
- the arms;
- the primary outcome;
- the repetitions;
- the guardrails, labels and invalidity rules of amendments 1 and 2.
- **Declared envelope, before any trial.** The buyer's service envelope is written to a file before the first trial and recorded with the run (`envelope.json` in the run folder and in every repetition):
  - `power_min_w`, the lowest watts Omni may set. Default (`tools/declare_envelope.py`): max(device minimum, 70% of the power limit read at declaration), rounded up to whole watts;
  - `power_max_w`, the power limit read at declaration;
  - optionally `slo_ms`, the response-time target; without it the target comes from calibration (10 bare service times), as before.
  - The bench refuses an envelope whose floor lies outside [device minimum, starting limit].
  - **The counted run refuses to start without a declared envelope** (`scripts/gpu_paired.sh`, `ENVELOPE`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Envelope floor.** The governor never sets the limit under `power_min_w` (`--floor-w`). When the floor is what lifted a write, the decision record names it (`decided_by`: envelope_floor), so no saving below the floor can be credited to the engine.
- **Outer controller holds.** When the card's own controller (board, BMC or system policy) already holds enforced.power.limit under the current limit, a lower write that would still sit above that enforced limit changes nothing on the card. It is not written; the decision record marks it (`outer_controller_holds`). Only a write that would actually bind, or a return upward, goes out. This is not another writer (amendment 2): the set limit is unchanged and the governor keeps running.
- **Narrow cards are reported as they are.** The device's own limit range is in the snapshot. On a card whose range is narrow (for example a 70 W card that accepts 60–70 W), the envelope is that narrow range; the result is reported for that card and range and not extrapolated to wider cards.

64.11 Amendment 4 (2026-10-02, after an invalid smoke; no counted data exist)

The first smoke on a rented A10 (results/gpu/smoke-20261002T032459Z, never counted) was invalid by rule: two copies of `scripts/gpu_rented_run.sh` had been started on the same machine, so both benches wrote the same card's power limit. The native and watch arms saw limits they never wrote (116, 137 and 150 W), each governor refused to start beside the other one (exit 5), and one reset restore was undone by the other copy. The card also began at 116 W, a limit an earlier start had left behind, not its 150 W default, so native itself ran capped (83% of samples). None of these numbers measures Omni. The engine, the governor, the outcomes and the analysis are unchanged. The run script now:

- **runs once per machine:** it takes a lock and refuses to start while another copy runs;
- **runs alone on the card:** it refuses to start while any other process is using the GPU;
- **starts from the card's default limit:** it sets power.default_limit before the envelope is declared, so the envelope, the snapshot and every arm start from the card's own default, not from a limit an earlier run left behind.

64.12 Amendment 5 (2026-10-02, before any valid hardware trial; the only smoke so far was invalid, amendment 4)

The Omni arm changes engine. The outcomes, the arms' order, the guardrails, the analysis and the validity rules are unchanged.

- **The Omni arm holds two wires** (`omni_controller/gpu_compass.py`, the compass law of `omnicompass/compass_law.py`): the clock ceiling (`nvidia-smi -lgc`, reset with `-rgc`; cover 35% of the top clock to the top), which sets how high the card's own boost may climb, and the power limit (`-pl`), the lid at what a fully busy card draws at that ceiling plus 10%, never under the declared envelope floor and never over the start limit. The service is read as one position between calm

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

and the response-time line (the larger of p95 and utilization above half) and pulled to the middle; past 95% both wires go to full at once (fail up). The card's firmware keeps its own control; Omni sets only those two values. The earlier power-limit-only governor stays available (`OMNI_ENGINE=one_wire`) and is not the counted run's arm.

- **Why, before the run:** on a modelled card (`results/sim/gpu_two_wire/`, evidence class S, seeds never used while tuning) the one-wire governor gave +0.1% work per energy and the two-wire engine +9.0%. That is a model; this run is the card's own meter.
- **The watch arm** runs the same two-wire engine in watch mode: it computes and records both wires and writes neither.
- **The clock range is reset before and after every arm** (`-rgc`), as the power limit already was; the run script resets it once at the start.
- **The wire check runs first** (`tools/gpu_wire_check.py`): the card's clock must follow a lowered ceiling down and come back up when reset, the power limit must read back what was set, the governor must hand both wires back when stopped and must leave a limit set by another writer alone (exit 5). If any step fails, nothing else runs and the check's report names the wire, the step and what the card said.
- **Wire check, corrected before any trial (2026-10-02):** on the A10 the first wire check failed at "3 up wire (follows up)" although the wire works: under the heavy check load the card's own 150 W limit already held it near 990 MHz, so a ceiling at 60% of the top clock (1017 MHz) left no room for the clock to come back up above it. The check now reads the card's busy clock on its own first and locks at 60% of that. The governor likewise starts its ceiling at the clock the busy card actually runs (a ceiling above it holds nothing), and its clock cover is 35% of the top clock to the top. No trial had run.

64.13 Amendment 6 (2026-10-02, after the first counted run and before any further trial)

What the first counted run showed (A10, commit `c908054`, `results/gpu/run-20261002T082232Z/`): work per energy +3.6% (+2.7% to +4.5%, proven), the same requests served, no request lost, but the 95th-percentile response time +58.5% (510 to 809 ms), so the response-time guardrail failed and the label by rule was ENERGY IMPROVEMENT WITH SERVICE TRADEOFF. It stands as the result of that run.

Why, from the card's own samples: while busy the card ran at 736 to 768 MHz under Omni against 861 to 889 MHz on its own, and spent about 30% more time busy for the same work; requests queued behind each slower one. Three faults in the governor, not in the engine: (1) the position counted utilization above half as service trouble, so every burst read as past the wall (fail up in 46% of decisions) and every quiet gap pulled the ceiling down, so each burst began on a lowered clock; (2) the lid followed a curve from the top clock and sat at the 105 W envelope floor in 49 of 165 compass decisions, under the 135 W the card itself draws while busy; (3) the ceiling's cover reached 35% of the top clock, far under the clock the card's own power limit holds it at while busy.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The Omni arm from now on (`omni_controller/gpu_compass.py`; the outcomes, arms, guardrails, analysis and validity rules are unchanged):

- the position is response time only (p95 over 5 s, not 30 s); being busy is not a breach;
- **race while work waits:** at 95% utilization or more the ceiling goes to the top and the lid to the start limit; the compass paces only the slack between bursts;
- **the card's own level, learned from its own meter** while the ceiling is at the top and the card is busy: its busy clock (median) and busy draw (90th percentile); until 15 such readings are in, neither wire moves;
- **speed floor:** the ceiling never goes under the card's own busy clock; **lid floor:** the lid never goes under the card's own busy draw plus 10%;
- fail up (past 95% of the line, or blind) is unchanged.

On the modelled card, before any trial (`results/sim/gpu_two_wire/`, seeds 5000 to 5009): p95 122.1 ms native, 123.7 ms with the corrected compass; work per energy +8.2% (+6.3% to +10.1%); energy -7.5%; the median response 10.1 to 12.1 ms, slower in the quiet stretches the compass paces. That is a model; the next trial is the card's own meter.

64.14 Amendment 7 (2026-10-02, before any further trial)

The outcomes, arms, guardrails, analysis and validity rules are unchanged. The Omni arm of the counted runs the **service** profile.

- **Two profiles, one switch** (`--profile`, `omni_controller/gpu_compass.py`; the same in `realms/gpu_card.py`):
  - **service**, the default and the counted run's arm: down gain 0.0125, the compass's center at 0.4, the speed floor 3% above the card's own busy clock;
  - **batch**: down gain 0.015, center 0.5, the floor at the card's own busy clock, for work nobody waits on answer by answer. It may be run as a separate, declared counted run (`OMNI_ARGS="--profile batch"`) and is reported as its own result, never pooled with service.
- **How service was chosen, on the model, before the trial** (20 paired seeds, 5000-5009 and 5100-5109; ratios summarised as the geometric mean of the per-seed ratios): down gains 0.01, 0.0125, 0.015, 0.0175 and 0.02 alone, and 0.0125 to 0.0175 crossed with the compass's center (0.4, 0.5), the speed floor (1.00, 1.03 of the card's own busy clock) and the race threshold (0.90, 0.95). The rule: the most work per energy at which no seed's p95 is more than 10% slower than native. Service (0.0125, 0.4, 1.03) gave work per energy +5.3% (+4.2 to +6.5), energy -5.0%, p95 -4.1% (-9.7 to +1.7), worst seed +9%, 0 of 20 seeds more than 10% slower, p99 -1.8%; its neighbours gave the same within a point. 0.015 alone gave +6.2% but 3 of 20 seeds 10% to 48% slower at p95; 0.01 alone gave +3.8% with p95 -1.9%.
- **The ceiling moves in whole clock steps** (--min-change-mhz, 15 MHz), as the card's own clock does and as the model moves it: the force times the gain, as a share of the top clock, is rounded to whole steps, and a pull under half a step moves nothing and is not stored up.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The position reads the mean response time of the window** (as the model does), between the bare service time and the line; the 95th percentile at or past the line, or any failed request, is past the wall (fail up).
- **The model's report summarises ratios on the log scale** (`tools/run_gpu_card.py`): the arithmetic mean of per-seed percentages let one seed (+240%) stand for twenty.

64.15 Amendment 8 (2026-10-03, before any further trial)

The outcomes, arms, guardrails, analysis and validity rules are unchanged, except as stated here.

- **Omni-Compass moves the card only where it measures that the card is never behind for it** (`omnicompass/verdict.py`, in `omni_controller/gpu_compass.py` and `realms/gpu_card.py`). While the service is calm, the governor runs a paired trial:
  - first the ceiling at the top until 30 requests are measured;
  - then one 15 MHz step past the deepest step already allowed, until 30 more are measured.

Each request's cost is the card's own time on it: the workload's new `service_ms` column, start to done, with the wait in the queue left out. The step is allowed if its median cost is at most **2%** above the median at the top. Otherwise it is refused and not tried again for 900 decisions. The compass may move the ceiling only between the top and the deepest allowed step. Where no step passes, the ceiling stays at the top and the card runs as it does alone. Every trial and every judgement is in the audit (`verdict`, `verdict_state`, `verdict_deepest_step`).

- **One law, no profiles.** The service and batch profiles are removed. The law is: down gain 0.0125, the compass's center 0.4, the speed floor at the card's own busy clock, and the verdict's allowance of 2%. The batch profile paced past the 2% allowance, so it is gone.
- **Why 2%** (the card model, 20 paired seeds):
  - an allowance of 0 leaves the card native on every workload, because every clock step down adds some time to a request, and saves nothing;
  - 2% is the smallest allowance that saves energy, and it is below what a person or a service contract can notice.
- **A second workload, AI token generation** (`tools/gpu_workload.py --kind decode`): fp16 matrix-times-vector over 1 GiB of weights per pass, batch one, so each pass streams every weight from memory, as token generation does. It runs as a second counted run of the same design (10 repetitions × 3 arms × 600 s) after the compute-bound one (`scripts/gpu_rented_run.sh`; `SKIP_DECODE=1` skips it). It is labelled by the same rule and reported as its own result, never pooled.
- **Every comparison is a base alone against the same base with Omni on top.** The model's report (`tools/run_gpu_card.py`) runs two bases:
  - the card's own firmware, alone and with Omni on top;
  - an operator's fixed 105 W power cap, alone and with Omni on top.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The old one-wire governor is removed as an arm (it was Omni's own earlier version).

- **On the modelled card, before any trial** (`results/sim/gpu_two_wire/`, firmware alone against firmware with Omni on top; tuning seeds 5000-5009, fresh seeds 5100-5109):
  - compute-bound work: work per energy +0.70% (tuning) and +0.48% (fresh); median +1.56% and +1.47%; p95, p99 and time over the line unchanged within their intervals;
  - AI token generation: work per energy +3.36% and +3.86%; median +0.55% and +0.70%; p95 +0.29% and +0.26%; time over the line 0.

That is a model; the next trial is the card's own meter.

64.16 Amendment 9 (2026-10-03, before any trial on this code)

Everything in amendment 8 stands. Four additions, each run by `scripts/gpu_rented_run.sh` after the two counted runs and each reported as its own result:

1. **Steady under the limit** (`omni_controller/gpu_compass.py`, the same in `realms/gpu_card.py`). While the card is saturated against its own power limit (work waiting, the draw at 97% of the limit or more), the firmware boosts a step, hits the limit and is knocked back. The ceiling is then held at the card's own busy clock under that limit, so the same watts serve the work without the knock-backs. It is never held under that clock, the lid stays at the start limit, and a blind feed still fails up. In the card model, fully loaded under a fixed cap, this gives about +0.6% requests served from the same watts, with p95 about 2% faster.
2. **An operator's power cap underneath.** The card's limit is set to the envelope's lowest watts (70% of its default) before the run, and the paired bench runs native (the cap alone), watch and Omni-Compass on top of the cap (the lid never above the cap). It runs twice, 5 repetitions × 3 arms × 300 s each time:
   - at the usual load;
   - fully loaded (arrivals at 130% of the card's capacity under the cap), so the result is requests served from the same watts.

The card is returned to its default limit afterwards.

3. **The GPU fault drill** (`scripts/gpu_fault_drill.sh`). With the request stream running:
   - the governor is killed outright, and the watchdog must hand the card back;
   - the master switch is pulled, and the governor must hand back, exit and refuse to restart while OFF;
   - the response feed is paused, and the governor must fail up.

Every check must pass, and the card must end at its start limit.

4. **Real AI serving** (`scripts/gpu_vllm.sh`, `tools/llm_workload.py`). An open language model (Qwen/Qwen2.5-0.5B-Instruct) is served by vLLM, installed in its own environment. It is asked the same seeded stream of prompts in every arm, each for exactly 128 new tokens; 5 repetitions × 3 arms

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

× 300 s. Tokens per second and tokens per kilojoule follow from requests served. If vLLM cannot be installed or started on the machine, the stage says so and nothing measured before it changes.

64.17 Amendment 10 (2026-10-03, before any trial on several cards)

Everything in amendments 8 and 9 stands. **Several cards in one server** (`scripts/gpu_8card.sh`): every card runs `scripts/gpu_rented_run.sh` at the same time, each on its own card with its own wire check, envelope, smoke, the two counted runs and the power cap underneath, on the same committed code. Card i starts its arm rotation i steps later (`ROT_OFFSET`), so no arm always meets the same neighbours. Each repetition stays paired within its own card (its three arms on one card); the pooled table per workload holds every card's repetitions (card c, repetition r is pooled as repetition 100(c+1)+r) and is labelled by the same rule. Each card's own table is reported beside it; a card whose table is invalid is reported, never dropped. The fault drill runs once, on the first card, after every card has finished. Last, **one model across every card** (`scripts/gpu_vllm.sh` with GPU = every card): vLLM tensor parallel, Qwen/Qwen2.5-7B-Instruct, one governor per card on its own card's two wires, all reading the same response times; the energy is every card's summed; 5 repetitions × 3 arms × 300 s, labelled by the same rule. Each card's energy is read from that card alone (the bench's sampling filtered to the card in each repetition's own receipt).

The short design (`FAST=1`, written before any trial on several cards): each card runs one smoke round and 3 repetitions of the compute counted run (24 paired repetitions on 8 cards, pooled and labelled by the same rule, each card's own table reported beside it), then one model across every card with 3 repetitions. AI token generation and the power cap underneath are measured on the one-card machine (amendment 9) and are not repeated here.

The whole stacks with a real card inside, on several cards (written before any trial): `tools/run_hil.py` as on the one-card machine (amendment 9, the six organisms at 1x, 10x, 100x and 1,000x, repetitions 3, 3, 2, 1 by size, the adaptive step), each organism on its own card at the same time (organism i on card i; with fewer cards than organisms, the next organism waits for a card). Each organism's receipt is its own; nothing is pooled across organisms.

The pooled design (`POOLED=1`, written before any trial on several cards): every stage of the whole design, with the repetitions pooled across the cards. Each card runs one smoke round; 3 repetitions of the compute counted run and 3 of the AI token generation counted run (24 paired repetitions per test on 8 cards, pooled and labelled by the same rule, each card's own table beside it); 2 repetitions of each power cap load (16 per load); the six organisms with the card inside, one repetition per size, each organism on its own card; then one model across every card, 3 repetitions; the fault drill once. Arm durations are those of the whole design.

The time budget (`MAX_HOURS`, written before any trial on several cards): a run on several cards ends by its budget whatever happens. At the budget the master switch is pulled (every governor hands its card back), every stage still measuring stops and is reported as incomplete (an arm cut short is never counted: its repetition has no complete pair), the stages not started are skipped and named, and the finished ones are packed.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

64.18 Amendment 11 (2026-10-04, before any trial on this code)

The six organisms with the card inside (`tools/run_hil.py`), first run 2026-10-02 at commit `c908054`, labelled the card's part SUPERIOR in five organisms while its 95th percentile rose from about 500 ms to 600-935 ms in every organism. The harness counted the card's work and energy and never its response time (its time over the line was fixed at zero). That run is history: its governor was replaced in amendments 6 and 7, and its labels do not meet the founder's rule.

From this amendment the card harness measures what it never counted:

- the card's time over the line (requests slower than the line, ten bare service times, percent of requests) and its 95th percentile, each paired against native;
- each row keeps its preregistered label and names, beside it, every measure that came out behind native by any amount, so what Omni costs is read beside what it gains; every change is read in words (GAIN, EVEN or LOSS, and what it means); no new line is drawn. The 2% bound stays where the founder set it: inside the engine, as the trigger of the verdict (`omnicompass/verdict.py`: a knob moves only where a paired trial shows the muscle at most a 2% LOSS);
- the stacks' own label (`realms/harness.py label`) puts the band first: its guardrail is the mean time over the line at or under native's, where it was the upper end of the interval under +1 point;
- at one copy, each organism runs 5 paired repetitions (was 3), so an energy change of 1-3% on the card's own meter can be told from noise; 3, 2 and 1 at 10, 100 and 1,000 copies as before.

The single card and the eight cards run this code. A card run started before this amendment is reported in full from its own raw files (its response times included) before any figure is quoted.

Why the stacks were late more often in the first run (+0.25 points) and are not now (-0.03): two knobs moved in commit `2a9285d`. Omni could loosen the operator's HPA target to 0.95 (fewer pods, bursts wait); it is now held at the operator's. A machine went back with no headroom (the next burst waits for a boot); it now goes back only when the rest covers the recent peak at 0.6 of the release level. Each alone leaves the stacks late more often; together they are late less often. The headroom is the least that keeps every seed un-late (`results/realms/RELEASE_MARGIN_SWEEP.md`).

Every report reads each change in words (GAIN, EVEN or LOSS, and what it means), so a sign is never read alone.

64.19 Amendment 12 (2026-10-05, before any trial on this code)

The card model rerun on the current code (`tools/run_gpu_card.py`, 2026-10-05) showed amendment 9's steady hold making the card slower on its own firmware: compute work, p95 122 ms native against 149 ms with Omni on top, time over the line +0.39 points. A bisection over every commit since the last clean model run puts the change in commit `bd455f9` (amendment 9, item 1). Holding the ceiling at the card's median busy clock takes away the boost the firmware uses to serve a burst at its own factory limit. Under an operator's cap the same hold is what helps: the firmware sawtooths against the low cap, and the hold smooths it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

From this amendment the hold applies only under an operator's cap: the start limit under the card's factory limit, read from the card (`power.default_limit`). If the card will not say, it is treated as no cap, so no hold, and the firmware is left to serve the burst. On the card's own limit, saturated, Omni races, as it did before amendment 9. Test: `tests/test_gpu_compass.py` (no hold at the factory limit; the hold under a 150 W cap on a 214 W card).

The card model with this amendment, 20 seeds (tuning and fresh). Under the cap: p95 6.4% and 7.1% faster, p99 3.2% and 5.8% faster, time over the line 1.3 and 1.0 points lower. On the firmware: energy 0.5-3.7% lower, p95 even. The median is 0.6-1.5% slower, inside the verdict's allowance (`results/sim/gpu_two_wire/`). The real card is the test.

64.20 Amendment 13 (2026-10-13, before any trial on this code)

The founder's order of 12 October: 1, 10 and 100 only, to cut the time and the bill; no 1,000 copies and no 1,000 runs anywhere. The whole stacks with the card inside (`tools/run_hil.py`, step 8 of the one-card and eight-card runs) now run each organism at 1, 10 and 100 copies on one clock, with 5, 3 and 2 paired repetitions by size (amendment 11's numbers for those sizes); the 1,000-copy size is gone. On eight cards with the repetitions pooled (amendment 10), each card's organism runs once per size. Nothing else changes: the arms, the measures, the labels and the hand-back are as before. On one card the stage takes at least 16 hours (60 paired repetitions, each arm 240 steps of 2 s; a size whose step must be longer is measured on the machine and stated in the receipt), where it took about 25 with the 1,000 copies.

64.21 Amendment 14 (2026-10-17, wording only, before any trial on this code)

The rule of the analysis and of amendment 1 is unchanged; only its words change, so a reading never claims more than a 95% interval can carry and every word matches the house key (GAIN, EVEN, LOSS; `docs/HOW_TO_READ_THE_RESULTS.md`):

- "a proven GAIN" and "a proven LOSS" (the interval wholly on one side of zero) are written **GAIN, clear of the noise** and **LOSS, clear of the noise** (a card's energy that rose: **power given, clear of the noise**);
- "not proven" (the interval includes zero) is written **EVEN, no difference beyond the noise**;
- a contrast with fewer than two repetitions has no interval and says so.

The labels of amendment 1 read the same way: primary GAIN clear of the noise, all three guardrails held: SUPERIOR WITHIN GUARDRAILS; a guardrail failed: ENERGY IMPROVEMENT WITH SERVICE TRADEOFF; primary EVEN, guardrails held: NONINFERIOR / INCONCLUSIVE; a guardrail failed: NOT ESTABLISHED; primary LOSS or power given clear of the noise: AWAY FROM ITS AIM; the observation contrast clear of the noise: NOT ATTRIBUTABLE. The intervals use the two-sided 95% point of Student's t computed for the run's own degrees of freedom (`omnicompass/student_t.py`); the table it replaces held the same values to the last digit up to 20 degrees of freedom and fell to 1.96 beyond them. The first counted run's table (2 October) keeps the words it was printed with.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

64.22 Amendment 15 (2026-10-30, the rig, declared before any counted run on it): the window opens once the governor stands

Until this amendment `scripts/gpu_paired.sh` wrote `window_start.txt`, read the energy meters and started the workload the instant the governor process was launched, so the governor's own start-up (the interpreter, the first query of the card, the snapshot of its limits) was inside the omni and watch arms' windows and outside native's. On a rented card it is a second or two of a 600 s arm; on a loaded machine running the harness's own 5 s test arm it was most of the arm, and the governor had written nothing by the window's end (found 30 October 2026, `tests/test_gpu_bench.py` on a loaded runner). From this amendment every governed arm waits for each governor's first audit record (its snapshot, written before its first decision), records that moment in `governor_ready.txt`, and only then reads the meters and opens the window; a governor not standing within `GOV_READY_S` (60 s) refuses the arm. Native's window is unchanged. No gauge, rule, gain or guard of the governor changes, and `tools/gpu_reps.py` reads the window as before. The card tables already run (omni-v18's A100 run of 24 October; omni-v21's A100 tables of 27 October) stand as their own rig's; the next counted runs carry this rig.

64.23 Amendment 16 (2026-11-01, declared before any run of it): the stacks with the card fully loaded

The founder's design of 1 November reads every place by its bill and the work done inside it (`results/TWO_BILLS.md`): the chip native (A) and the chip with Omni-Compass on native (B), the cloud native (C) and the cloud with Omni-Compass on native (D). The whole stacks with the card inside (amendment 9; omni-v21's three runs, `results/gpu/V21_STACKS_A10_ABC.md`) offer the card the usual load (shares 0.3 to 0.8 of its full-power capacity), so the card serves every request in both arms and its work cannot differ: there the work is the same by construction and only the energy can move. This amendment adds a second form, `lambda-gpu.yml` mode `stacks-full`, and changes nothing in the first:

- the card's workload inside the stacks offers 1.3 times its full-power capacity in every phase (`--phases=1.3,1.3,1.3`, the same full load as the power cap's `cap-full` form), through `HIL_WORKLOAD_ARGS`, which reaches the stacks alone: the wire check and the smoke keep their own load;
- everything else is the stacks' as declared: the seven organisms (the spine, the four realms, the four stacked, the whole tower) at 1, 10 and 100 copies with the card inside, repetitions 5, 3 and 2 by size (`hil_profile standard`), native against Omni-Compass on native at the factory tune (the dial at the max), paired within each repetition, the card's firmware limit as it ships in both arms and no operator's cap;
- three runs, A, B and C, each on its own rented A10, on one commit that carries the engine that stands (omni-v23);

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- the reading: per organism and size, the card's requests served (work), its energy, its energy per request served, its 95th percentile and its time over the line, and the stacks' own gauges, each by the one key (`tools/readings.py`); the bill of each arm is the rental of its own window at the price the launch printed, so the bill reads the same in both arms unless a window differs, and the work per dollar moves with the work;
- said before the run: the A10 ships with its power limit at the top of its range, so on native the card already runs at its limit and the factory tune, never below native, may have nothing to raise (omni-v18's A100 run of 24 October wrote nothing for that reason). A STEADY row where the governor wrote nothing is that, said so: not a benefit and not a fault.

CHAPTER 65

The Realms Preregistration

Round 3 is the current one: its section at the end changes only how the realms are made up. Round 2's section replaced round 1's Omni layer. Round 1 below is kept as it was frozen. Each round was written and committed before its counted seeds were run. Evidence class **S**: every number the run produces comes from a declared model. Nothing here is a meter, and nothing here is evidence about a real machine.

65.1 Question

For each of the 656 muscles of the canonical tower, and for each realm and the whole tower run as one organism: does the frozen governor (`omnicompass.adapter.Governor`, the engine with u = 0, the stack law, the reset), holding that muscle's one knob on top of the plant's native controller, change work per energy against the native controller alone, without buying it with service?

65.2 What runs

- **The catalog** (`realms/catalog.csv`, built by `tools/realms_catalog.py`): the 656 rows of the XPASS package's canonical tower, unchanged, plus four columns given by fixed rules: realm, plant, parameter set, knob.
  - Compute / AI / Cloud: 250 muscles. Physics / Robotics / Autonomous: 88. Energy / Facility / Industrial: 121. Distribution / Specialized: 197.
- **Five plants** (`realms/plants.py`), each with its own native controller, all parameters in `realms/presets.py`:
  - compute_pool: request stream, servers with start-up delay, idle and dynamic power; native: the HPA rule (10% tolerance, scale-down stabilisation window), fixed admission limit, full clock;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

  - thermal_zone: zone heat balance, staged cooling units, COP from supply and outdoor temperature; native: PI on a fixed setpoint, units staged to the load;
  - energy_storage: site load, solar, battery, grid connection with a contract limit; native: self-consumption above a fixed reserve;
  - motion_axis: point-to-point moves from a task queue, motor copper losses and heating, derating; native: PID with feedforward at full speed;
  - process_loop: first-order process with dead time, pump or heater power; native: PI on a fixed setpoint.
- **A muscle is one knob of one plant.** Omni holds only that knob: capacity (the adapter's capacity law for one plant), setpoint (inside a declared band: calm end at rho0, stress end at rho_min), power (the directive's power cap), or admission (native while change is permitted; while not, only what clears inside the service target, or the flexible share deferred or shed).
- **Each muscle's plant is sized** by a factor 0.6 to 1.4 drawn from its muscle id.

65.3 Arms

On the same seed, so with the same demand, weather and disturbances:

- **native**: the plant and its native controller;
- **watch**: the governor reads every period and writes nothing;
- **omni**: the governor holds the knob; at 90% of the run it is killed and the knob returns to the native controller;
- **fixed_calm** (setpoint muscles only, descriptive): the native controller with the setpoint fixed at the band's calm end. It shows how much of a setpoint result the band alone gives.

Organisms: every plant of a realm (or all 656) on one 15 s clock for one hour. Plants are coupled:

- the electrical power of compute, motion and process plants is heat in the realm's thermal zones;
- the organism's load swing is load on its storage sites;
- the zones' temperature is the ambient every other plant reports.

One governor reads the organism's aggregate, and its one directive sets every muscle's knob.

65.4 Seeds and repetitions

Counted seeds 1000 to 1009: ten paired seeds per muscle and per organism. Development used seeds 0 and 1 only. Nothing from development seeds is reported.

65.5 Outcomes

- **Primary**, per seed: (work_omni / work_native) / (energy_omni / energy_native) − 1. Work is in the plant's own units: requests, IT heat held in specification, site load served, moves completed, or product

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

delivered in specification. For an organism, work is the mean over its plants of work_omni / work_native, and energy is total joules.

- **Guardrails**, both must hold:
  - work: the 95% interval of work_omni / work_native − 1 must not reach below −1%;
  - violations: the 95% interval of the change in the share of periods in violation must not reach above +1 percentage point.

Violation, per plant:

- compute: response time over the service target, or a dropped request;
- thermal: zone over its limit;
- storage: grid import over the contract limit;
- motion: tracking error over its bound, winding over temperature, or the oldest task waiting past its deadline;
- process: the process variable outside its specification.

- **Label, by rule** (`realms/harness.py`, `label()`, the GPU bench's rule):
  - SUPERIOR WITHIN GUARDRAILS;
  - ENERGY IMPROVEMENT WITH SERVICE TRADEOFF;
  - NONINFERIOR / INCONCLUSIVE;
  - NOT ESTABLISHED;
  - LESS WORK PER ENERGY;
  - INVALID.

65.6 Invalid

A muscle or organism is labelled INVALID if, on any seed:

- the watch arm differs from native in any meter or writes anything;
- the omni arm writes after the kill;
- a knob is not back at its native value after the kill;
- any contrast is not finite.

65.7 Recorded

Per muscle and organism:

- every per-seed contrast;
- writes per run;
- the native violation share;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- the fixed-setpoint comparison where it applies.

Per run: the commit, the seeds, and the fingerprints of the catalog, the engine, the governor and the whole frozen tree (`RUN.json`, `SHA256SUMS.txt`). The counted run refuses to run on uncommitted code.

65.8 Changes made on the development seeds, before this freeze

The development runs found bugs and sizing faults. Each was fixed before the counted seeds were run:

- **The admission knob was inverted**: it admitted only what cleared the service target while change was permitted. Fixed to the declared rule.
- **The compute admission limit compared the queue limit with the period's arrivals** instead of the backlog left after the period's service. Fixed for native and Omni alike.
- **The capacity law did not clip the queue and load readings to [0, 2]**, as the adapter's `observe_vector` does. Fixed.
- **Thermal and process utilisation was measured against the uncapped capacity** while the power cap was held, so the power law's trim ratcheted to its floor. Fixed: utilisation of the capacity actually available.
- **Compute plants reported a heat proxy of my own.** Replaced by the live controller's own thermal model (`omni_controller/muscles.py`).
- **Thermal zones kept fixed cooling units while their IT load was scaled.** The large halls were under-provisioned, so the native controller overheated. Cooling now scales with the hall.
- **The flight axis's motor thermal resistance was ten times too high**, so the native controller was always derated. Corrected. The UPS feed was sized under its own load; sized at 1.2 times.
- **The slow plants** (vehicle, spacecraft, flight) completed too few tasks in six minutes to measure. They now decide every 5, 10 and 2 seconds over the same 360 decisions.
- **Added the fixed_calm arm** for setpoint muscles, after development showed that the band's calm end alone accounts for much of the setpoint results.

None of these changes was chosen by its effect on Omni's result. The development runs showed losses as well as gains for Omni before and after them.

65.9 Not claimed

- **No row is evidence about a real machine.** A row says what the governor's law does to that model through that knob.
- **The plants, native controllers and bands were written by the same project as the governor.** That is a real conflict, so every one of them is in two files, to be read and contested.
- **A muscle's name chooses its knob and its plant's size; it does not get its own physics.** The 16 muscles of a family share the family's plant model at different sizes, through different knobs.
- **The plant code is Python only.** The governor's C++ twin is unchanged; a C++ twin of the plants is open.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

65.10 Round 2 (2026-10-01, after round 1's results; before any round-2 counted seed)

Round 1 (seeds 1000-1009) labelled all five organisms LESS WORK PER ENERGY. Reading the result against the shipped controller (`omni_controller/controller.py`, `omni_controller/muscles.py`, `omnicompass/nervous_system.py`) showed that round 1's Omni layer was not the Omni that runs on Kubernetes. Round 1 is kept unchanged in `results/realms/round1/`, with a note saying why it is superseded. Round 2 changes only how Omni commands a knob, the native machine-pool scaler and the declared budgets. These changes were made after seeing round 1, so they are listed with the source line each one follows:

Round 1	Round 2, as the shipped controller does it
Capacity muscles replaced the HPA with the stack simulator's capacity law (release one unit after convergence, dwell and a 20% band)	Pods: the HPA target written as min(rho*, the operator's target), never tighter, held one autoscaler window; the HPA scales (`controller.py`, HPA target patch). Machine pools: the node release gate, one machine per decision (`nervous_system.node_release_gate`)
HPA-type setpoints moved inside a band whose calm end was tighter than the operator's target	The same min(rho*, operator) rule: more headroom is always allowed, less never (`controller.py`)
No contraction authority and no SLO reflex	Every contraction needs the organ's authority (calm >= threshold, senses live) and three clean decisions; while service is breached the knob returns to native (`nervous_system.authority`, `muscles.py` SLO reflex)
Continuous knobs jumped to the governed value	Down by at most the calm share of the surplus per decision (`nervous_system`: step = calm)
Request-served pools had their power capped	Never: throttling request work saves no energy and adds wait (`muscles.py`, `_power_cap`); GPU and CPU-frequency pools use their envelope, never under draw x 1.3
Thermal setpoint by headroom	The live cooling law: warm while cool, cold as heat rises, under the envelope 18 + 9 x calm mapped onto the band (`muscles.py`, `_cooling`)
Admission paused every admission muscle, request traffic included, and resumed only when fully calm	Batch pacing of pausable work only: one muscle suspended per decision at power stress >= 0.95 or heat >= 0.96 (or a nervous pause), one resumed per decision at power stress <= 0.8 and heat < 0.90

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Round 1	Round 2, as the shipped controller does it
	(`muscles.py`, `_batch_pace` defaults); request traffic never paused
queue_ratio was the backlog in service-target units	pending starts per serving unit, or latency pressure (p95 / target − 1, or the failed share), capped at 2 (`controller.py`)
The governor's current cap followed the applied cap	1.0, as the live controller sets it
Site power budget 1.25 x a formula nominal that sat under the organism's real native draw (compute ran at 1.2 x it), so the governor read the site as always at its limit	1.25 x each plant's mean native draw on the calibration seed 999 (never a result seed); compute pools alone likewise
Organism heat = the hottest of up to 656 plants	The mean, as every other organism channel; local heat stays with each plant's own reflex
Native machine pools used the HPA rule	The Cluster Autoscaler's defaults: add while work waits, remove one machine after the rest has been under 50% for 10 minutes
Building cooling sized under its own peak (native overheated half the time)	Units of 35 kW: capacity 1.25 x the declared peak

Unchanged: the plants' physics, the catalog, every other parameter, the arms, the outcomes, the guardrails, the label rule and the invalidity rules.

- **Round 2 seeds:** 2000 to 2009. Development of round 2 used seeds 0 and 1 only, never reported.
- **Results:** `results/realms/` (round 1 in `results/realms/round1/`). Both rounds are cited together.

65.11 Round 3 (2026-10-02, after round 2's results; before any round-3 counted seed)

Only the make-up of the realm organisms changes. The plants, the Omni layer, the outcomes, the guardrails, the label rule and the invalidity rules are round 2's, unchanged.

- **Round 2 cut the 656 into four realms with no overlap.** No realm organism carried the infrastructure every real stack runs on unless that infrastructure was the realm's own. The data-centre realm had no cooling or power, and the robotics and plant realms had no Kubernetes, machines or GPUs.
- **Round 3 gives every realm the shared spine** (`tools/realms_catalog.py`, SPINE): Kubernetes Workload Scaling, Placement & Scheduling, Container Resources, Node Fleet, Cloud VM & Capacity,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

NVIDIA GPU Hardware, Host CPU & Memory, Network Routing, Storage, Observability, Reliability & Security, Cooling & Chillers, PDU / UPS & Electrical Distribution.

  - Each realm's organism is its own families plus the spine: Compute 345 muscles, Physics 262, Energy 282, Distribution 337.
  - The whole-tower organism still holds each of the 656 once.
- **Quantum Computing Control moves to the compute realm** (it behaves as a compute job queue).
- **Round 3 seeds:** 3000 to 3009. Round 2 is kept in `results/realms/round2/` with a note on why it is superseded.

65.12 Round 4: the stacked organism (2026-10-02, before any round-4 seed)

The four realm organisms of round 3, stacked on one 15 s clock (`realms/harness.py`, `run_stack`; `tools/run_stack.py`). Every muscle appears as often as it appears in the realms, duplicates included: 345 + 262 + 282 + 337 = 1,226. A duplicate is still a muscle that has to converge. Each realm keeps its own internal coupling. The plants, the Omni layer, the outcomes, the guardrails and the label rule are round 3's.

- **Arms:** native (no governor); separate (one governor per realm); one (one governor over the whole stack, reading the mean of all 1,226 muscles and the stack's total power against its total budget).
- **Check, required for validity:** the stacked native run equals each realm's own native run, plant by plant, on every seed. Stacking must change nothing natively.
- **Comparisons, each labelled by the rule:**
  - one governor against native;
  - separate governors against native;
  - one governor against separate governors (does one Omni over everything beat four).
- **Seeds:** 4000 to 4009. Development used seed 0 only, never reported.

65.13 Round 5: the whole stacks with the real card inside (written 2026-10-02, before any run)

One harness (`tools/run_hil.py`, started by `scripts/gpu_rented_run.sh` after a valid card smoke): each of the six organisms (the four realms, the four stacked with every duplicate kept (1,226), the whole tower of 656) runs on one clock as in round 3, with the machine's real GPU wired in as one more muscle of its NVIDIA GPU family (a spine family, so the card is in every organism). The card serves the pinned request stream; its own power.draw is heat in the organism's thermal zones and load on its storage sites.

- **Arms:** native (the stacks' own controllers, the card's own firmware) and omni (one engine on everything: the compass law on every simulated muscle, `realms/compass_arm.py`, and on the card's two wires,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`omni_controller/gpu_compass.py`). At 90% of each arm every knob and both wires are handed back; a knob not handed back, a card limit not back at its start, or a card governor exiting non-zero makes the run invalid (exit 2).

- **Seeds and repetitions:** 3 paired repetitions, seeds 6000-6002; arm order alternates by repetition and organism.
- **Clock:** 240 steps of 2 s of wall clock per arm (the card in real time).
- **Outcomes:** work per energy, work, energy and violations, Omni against native, for three parts kept apart: the simulated stacks (evidence S), the card (its own meter, evidence P), and both added (the card as one more plant, its joules added to the stacks'). Labels by the round 3 rule.

65.14 Round 6: the compute pools inside the band (2026-10-03, before any round-6 seed)

Round 3's rule (band first) was not held on the organisms: the compass law spent 0.20 to 0.29 points more time over the service line than native in every completed cell of the grid (`results/scale/GRID.md`). The cause was measured on the whole tower (seeds 7000-7003), muscle by muscle:

- **compute pools giving a machine back** (42 muscles): +1.77 points of their own time over the line;
- **the HPA target set looser than the operator's** (249 muscles): +0.22 points;
- the process and thermal setpoints, which save the most energy, added none.

Two changes, both in `realms/compass_arm.py`, everything else as round 3:

1. **The HPA target is held at the operator's own.** Loosening it past native costs the service time. This is the same rule as the card's speed floor and the live controller's cover.
2. **A machine goes back only when the machines left cover the recent peak at 0.6 of the plant's own release level** (`RELEASE_MARGIN`). A machine boots in minutes, so a burst that arrives after a release is served late until the machine is back.

How 0.6 was chosen (10 paired seeds per organism; the rule is the most work per energy whose whole interval of time over the line is at or under native): 1.0, 0.8, 0.75, 0.7, 0.65 and 0.6 on the tower; 0.65 and 0.6 on the other five.

- 0.65 failed on Physics, the upper end of its interval +0.001;
- 0.6 held on all six.

Checked on 20 paired seeds (7000-7019), before this round's official seeds:

Organism	Work per energy	Time over the line	Work done
Compute	+0.095%	−0.017 pp	unchanged within its interval (−0.001% to +0.000%)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Organism	Work per energy	Time over the line	Work done
Physics	+0.086%	−0.011 pp	unchanged within its interval (−0.001% to +0.000%)
Energy	+0.201%	−0.031 pp	unchanged within its interval (−0.001% to +0.000%)
Distribution	+0.090%	−0.015 pp	unchanged within its interval (−0.001% to +0.000%)
The four stacked	+0.153%	−0.022 pp	unchanged within its interval (−0.001% to +0.000%)
The whole tower	+0.194%	−0.010 pp	unchanged within its interval (−0.001% to +0.000%)

Every knob was handed back. The grid runs at 100× and 1,000× now on GitHub were started on round 3's law. They are recorded as round 3's result, and the grid is rerun on round 6's law.

65.15 Round 5, amended (2026-10-03, before any round-5 run on the corrected law)

The whole stacks with the real card inside run at four sizes, as the six-organism grid does: each organism as 1, 10, 100 and 1,000 copies governed together on one clock, with the one real card inside as one more muscle of its NVIDIA GPU family, native against Omni on top. Repetitions by size: 3, 3, 2 and 1 (seeds from 6000).

The step is 2 s of wall clock wherever the simulation keeps up. A size whose step takes longer gets a longer step: 1.5 times the measured time per muscle on that machine, times its muscles. The card's request stream runs for the same 240 steps, so the card and the stacks stay on one clock. The step of every size is in the receipt. At 1,000 copies the card is one muscle among hundreds of thousands, so its watts are a small share of the organism's; that is the arithmetic of one card in a large stack, and the card's own meter is reported apart from the stacks. Everything else is as round 5. The two GPU counted runs now run before this stage, so the most important results are in hand first.

65.16 Round 6 receipts and the one rule (2026-10-03)

At 1x and 10x (1,000 paired runs each) every organism is labelled superior within guardrails, with band first held and every knob handed back (`results/scale/receipts/`, `results/scale/GRID.md`). At 1,000 runs the intervals are narrow enough to show one cost: work done is lower by 0.0001% to 0.001%, wholly below zero in several organisms. Its cause was measured on the Physics organism, muscle by muscle. It is the thermal zones: a room held warmer keeps a little less margin, so a sudden heat spike puts it over its limit for a few extra seconds inside a step that already counts as over the line.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Limiting how far a room may drift (half, a quarter, none of the way toward its warm end; 40 paired seeds) does not remove it until the setpoint is held native, and then the Physics organism uses more energy than native. The cost is therefore disclosed and judged by the one rule now written for every muscle (`DISCLOSURES.md`, section 3): at most 2% in any measure, only where energy is saved. The largest work cost in the grid is 0.007% in a single run and 0.001% over 1,000 runs, far inside it.

65.16.1 The 1,000-copy size: memory and sharding (2026-10-03, before its re-run)

The first 1,000-copy run (six run 37090435347) stopped twice at the same point: every organism larger than physics and robotics ran out of memory about 3 minutes in (the four stacked at 1,000 copies is 1.2 million plants; at 33 KB a plant, and with the calibration organism built while the result organism was still held, it needed over 40 GB on a 16 GB runner). Three changes, none of which changes a number:

- the calibration run is made before the organism, so one organism is in memory at a time (`realms/harness.py`, `Body`);
- each plant is packed as it is made: its exogenous series as machine arrays, its random source dropped (every plant draws its series when it is made, never while it runs), plants with the same parameters sharing one table (`realms/plants.py`, `pack`, `_shared`);
- the six workflow sizes its shards by organism (`per_shard`, `workers` by organism), because one run of the four stacked at 1,000 copies takes 2 to 3 hours and a runner's job ends at 6.

Same outputs: four organisms, two seeds each, at 1x to 3x, byte-identical before and after. A plant now takes 6.7 KB; the four stacked at 1,000 copies needs about 9 GB. The re-run: 10 runs per organism at 1,000 copies (the 1-run and 10-run cells); 100 runs at 1,000 copies follows when the runners allow it (about 650 runner-hours).

The 100-run cells at 1,000 copies run on one rented machine (the one-machine grid script, removed on 13 October): every organism in turn, as many processes as the machine's cores and memory allow, the same seeds (7000 on), each finished organism saved at once, one receipt at the end (`SIX-1000x.md`), saved as `results/scale/v1/receipts/round6-1000x.md` in place of the 1-run and 10-run receipt it contains. From 12 October no 1,000-copy and no 1,000-run cell is started on any engine (the founder's order: 1, 10 and 100 only, the whole grid on GitHub's own machines, nothing rented for it); the rented-machine script is removed, and the cells already run at 1,000 copies stand as they were run.

Evaluation and Simulation use only. Copyright (c) 2026 The Omni-Compass LLC. Commercial use, commercialization or monetization of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. All rights reserved. See `LICENSE` *and* `NOTICE` *at the root of this repository. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

65.17 Amendment (2026-10-05): the compass law is the arm, written before its run

The tables of round 3 compared native against the allocation law ("omni"), the arm that came before the compass law. The engine is now one law on everything, the compass law on top of native, so the realms runner compares native against compass (`realms/harness.py` ARMS: native, watch, compass). The watch arm stays: Omni watching must write nothing and leave every muscle exactly as native. Seeds, plants, knobs, guardrails and the labelling rule are unchanged. The tables published until now describe the allocation law and stay as first measured in `docs/history/`.

65.18 Amendment (2026-10-05): four levers bounded by their physics, and the mirror label (written after the compass table was seen)

Disclosed: written after the first compass-law table (nine muscles LESS WORK PER ENERGY) was seen. Each change follows from the muscle's own model (`docs/MECHANISM_OF_ACTION.md` 9.6), applies to every muscle of its kind, and is tested:

1. Cooling power is native. A cap moves heat removal later, never away, and stages more units.
2. A battery's reserve is spent only at the connection's wall, and only when the battery can cover the excess. Below the operator's reserve every kWh costs (1 - eta)/sqrt(eta) to buy back.
3. A backup reserve (the UPS preset, `backup=True`) is never a lever.
4. A compute pool's power cap moves only where (9.8) says a lower cap saves energy. A quantum computer, a network switch and storage are left native (race to idle).

The labelling rule gains the mirror of ENERGY IMPROVEMENT WITH SERVICE TRADEOFF: less work per energy, no work lost (at most 1%), and the time over the line proven lower is SERVICE IMPROVEMENT WITH ENERGY TRADEOFF. Everything else is as before, and every other label keeps its definition.

Also in this amendment: an effort cap on a motion axis moves only where the copper loss it saves at full acceleration, (J a[max] / kt)^2 R, exceeds the standing draw a longer move pays, p[idle] + b v[max]^2 (a reaction wheel; not traction, a flight axis or a robot joint). The machine-release margin on node pools is 0.3, the largest at which no pool is later than native (`results/realms/RELEASE_MARGIN_SWEEP.md`, the per-muscle sweep).

65.19 Omni v2: the catalog grows to 945 muscles in 59 families (2026-10-06, before any v2 seed)

Written before any counted v2 run. **The law, the controllers, the plants, the Omni layer, the outcomes, the guardrails, the label rule and the invalidity rules are v1's, byte for byte.** Only the catalog and the presets it needs change, so by the change rule of `docs/OMNI_V1.md` this is a new version, Omni v2

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(`OMNI_V2.json`, `docs/OMNI_V2.md`), and every modelled result is run again on it. No v1 result is read as a v2 result.

What is added, each row with its real system, its setting and its source, built by `tools/realms_catalog_v2.py` from three committed files by the v1 rules (realm, membership, plant, preset and knob: `tools/realms_catalog.py`, unchanged):

- **The 656 of v1, byte for byte** (`realms/catalog_v1.csv`): every v1 muscle keeps its id, realm, plant, preset and knob.
- **118 real controls the realm study had already found and the tower lacked** (`docs/realm_study/TRUE_MUSCLES.csv`, the rows marked "added (real, not in the 656)"; six of them were already in the tower under another name and are listed as skipped in `realms/catalog_provenance.json`). They join the family the study filed them under; 70 of them are spine muscles (Kubernetes, nodes, hosts, GPUs, storage, network, observability, power, cooling) and so sit in all four realms.
- **171 muscles in thirteen new families** (`realms/wave4_families.csv`), the domains the tower did not cover, one new preset each (`realms/presets.py`, round engineering figures for the class of machine, as every other preset):

Family	Realm	Plant / preset	Muscles	Examples of the real system
Healthcare Critical Environments	Energy / Facility / Industrial	thermal_zone / hospital	13	ASHRAE 170 rooms on a hospital BMS (Metasys, Desigo); isolation-room pressure; pharmacy cold rooms
Medical Imaging & Clinical Systems	Distribution / Specialized	compute_pool / clinical	12	DICOM archives and routers (Orthanc, dcm4chee), EHR tiers, HL7/FHIR engines, monitoring gateways
Agriculture & Irrigation	Energy / Facility / Industrial	process_loop / irrigation	14	pivot and pump controllers (Valley, Lindsay, Netafim), greenhouse climate computers (Priva), grain dryers
Oil & Gas Pipelines	Energy / Facility / Industrial	process_loop / pipeline	14	compressor and pump station SCADA (Emerson, AVEVA, Honeywell), API 1130 leak detection, DRA injection

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Family	Realm	Plant / preset	Muscles	Examples of the real system
Rail Traction & Train Control	Physics / Robotics / Autonomous	motion_axis / rail_traction	14	traction converters and ATO (Siemens Mobility, Alstom, Hitachi Rail), CBTC headway, ERTMS
Marine Propulsion & Vessel Automation	Physics / Robotics / Autonomous	motion_axis / marine_propulsion	12	propulsion and power management (Kongsberg, Wärtsilä, MAN), IMO shaft power limitation
Ports & Maritime Logistics	Distribution / Specialized	compute_pool / port	12	terminal operating systems (Navis N4), automated cranes and AGVs (Kalmar, Konecranes), shore power
Mining & Mineral Processing	Energy / Facility / Industrial	process_loop / mill	14	grinding and flotation control (Metso, FLSmidth, ABB), ventilation on demand (Howden), dewatering
District Heating & Cooling	Energy / Facility / Industrial	process_loop / district_heat	13	network and substation control (Danfoss Leanheat, Kamstrup), CHP dispatch, thermal storage
Power Generation & Turbine Control	Energy / Facility / Industrial	process_loop / turbine	14	governors and plant DCS (GE Mark VIe, Emerson Ovation, Woodward), AVRs, hydro gates, reactor rods
Renewable Generation & Inverter Control	Energy / Facility / Industrial	process_loop / inverter	13	IEEE 1547 inverter curves (SMA, Enphase), wake steering (NREL FLORIS), pitch and curtailment (Vestas)
Elevators & Vertical Transport	Physics / Robotics / Autonomous	motion_axis / elevator_hoist	12	drives and destination dispatch (Otis, KONE, Schindler), ISO 25745 standby modes
Pharmaceutical & Food Manufacturing	Energy / Facility / Industrial	process_loop / batch_reactor	14	ISA-88 batch control (DeltaV, SIMATIC Batch), bioreactors (Sartorius), pasteurizers, CIP, cleanrooms

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Four wave-4 names collided with the tower and carry a domain prefix (`terminal_tank_level_target`, `train_auxiliary_power_budget`, `plant_reactive_power_target`, `inverter_frequency_droop_setpoint`). Eleven names the knob rules would have placed wrongly are set by hand in `tools/realms_catalog.py` OVERRIDE and listed there.

The organisms of v2: Compute / AI / Cloud 430, Physics / Robotics / Autonomous 376, Energy / Facility / Industrial 470, Distribution / Specialized 440, the four stacked 1,716, the whole tower 945; the shared spine is 257 muscles. In code and workflows the two whole-tower organisms are now named `tower` and `stack` (nothing is named by a count); the v1 names `organism_656` and `stack_1226` are read as aliases, so every v1 run folder and record still reads.

Checked before this freeze, on seed 7, every one of the 289 added muscles: deterministic, the watch arm equal to native, every knob handed back after the kill, no number undefined, Omni's time over the line never above native's. Two presets were adjusted on that check and before any counted run: the inverter plant's disturbance amplitude (0.8 to 0.5, with its noise 0.05 to 0.02) because native itself was outside its band a quarter of the time, and the elevator hoist's position gains (kp 2,000 to 3,000, kd 400 to 500) with a smaller load disturbance (400 to 100 N m), because the first figures put native's tracking error at its limit under passenger load. Where native is still often outside its band in some heavy-load muscles (the compute pools at their busiest sizes), that is the plant at the size the muscle id draws, the same for both arms, and it is reported as it comes.

v2 runs: the realms table (every muscle alone and the five organisms, seeds 3000-3009, three GitHub runs A, B, C), the six-organism grid at 1, 10, 100 and 1,000 copies, the six organisms with the real cluster inside, and the big organisms on Azure, each on the v2 fingerprint; the live benchmarks that import none of the realm files (Kubernetes alone, Azure alone, CityLearn, the grid, the robots, the database) are run again on v2 as GitHub's queue allows, their v1 tables kept as v1 tables.

65.20 Omni v3: the slack gate on speed knobs (2026-10-06, after the v2 table, before any v3 seed)

Declared on the v2 result and before any v3 run. The v2 table (`results/realms/REALMS.md`, three runs reproduced) read Physics and the whole tower as "energy improvement with a service tradeoff" where v1 had every organism superior. The rows that carried it were the new speed muscles: three rail traction muscles where Omni's slowing added 1.6 to 2.7 points of lateness on a train already near its timetable's capacity, and two marine and two elevator muscles where the slower cycle finished fewer moves in the hour. By the honesty rule the first suspect was our own wiring, and it was: the realm harness had no do-no-harm gate on the speed knob of a motion axis, so the compass slowed an axis that had no slack to spend. The robot benchmark has such a gate (its paired physics trial, `docs/ROBOTICS_PREREGISTRATION.md`); the realms get one now.

The rule (`realms/compass_arm.py`, `speed_slack`). A motion axis is offered its speed knob only where its duty at full speed, task rate × (move time at full speed + dwell), is at most SLACK = 0.5, computed from the plant's own figures before any decision. The compass may ease an axis down to 0.4 of its speed,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

which stretches a move by up to 2.5 times; an axis busy more than half the time at full speed cannot absorb that when tasks arrive in bunches, and its knob stays native. Where the knob is offered, nothing else changes: the compass, its gains, the cover [0.4, 1.0] and the fail-up are v2's.

The marine preset. Four 600 rad speed changes an hour made a one-hour run too coarse to count (one unfinished move is a quarter of the hour's work); the v3 preset is twelve changes of 200 rad an hour, a move of 126 s plus 30 s of dwell, deadline 1,200 s. No other preset changes.

What the gate does on the shipped presets (the middle size; a muscle's own size, 0.6 to 1.4 on the task rate, moves its duty either side):

Preset	Move at full speed	Dwell	A task every	Duty	The speed knob
flight axis	4.8 s	2.0 s	20 s	0.34	offered
reaction wheel	30.0 s	5.0 s	100 s	0.35	offered
marine propulsion	126.5 s	30 s	300 s	0.52	left native
elevator hoist	12.8 s	8.0 s	40 s	0.52	left native
robot joint	0.7 s	0.2 s	1.7 s	0.54	left native at the middle size, offered on the lightly loaded joints
EV traction	35.6 s	10 s	60 s	0.76	left native
rail traction	220 s	20 s	300 s	0.80	left native

The threshold is a declared number set on the v2 finding, not a fitted one: it is written here before the v3 runs and applies to every axis alike. Where it leaves a knob native the row will read as native does, and that is the result. Everything else, the plants, the outcomes, the guardrails, the label rule and the invalidity rules, is v2's. v3 runs the realms table three times (A, B, C), then the grid, then the organisms with the real cluster inside; the v2 table stays a v2 result.

65.21 Amendment (2026-10-16): the spine, the seventh organism (written before its run)

The founder's order of 16 October: the **spine** is an organism in its own right. It is the set of catalog muscles whose realm list names all four realms, 267 of the 1,107 (`tools/run_realms_v5.py`, `spine_rows`), in thirteen families: Kubernetes workload scaling, container resources, placement and scheduling, node fleets, cloud capacity, host CPU and memory, GPU hardware, network, storage, observability, reliability and security, electrical distribution, cooling. Run alone it is a data center with nothing of any one trade in it, the base every stack runs on (`docs/THE_SPINE.md`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- The spine runs as the seventh organism in every organism harness, under the same three loads (its own, the climb, work waiting in line), the same seeds (3000-3009 preregistered), the same two arms (native; Omni-Compass on top of native) and the same label rule as the other six; every table lists it first. In `tools/run_scale.py` and the grid conductor it is organism 7, so every grid run already started reads back under its own number; with the real cluster inside it is `tools/run_kil.py --organism spine`.
- The organisms are groupings of catalog rows made in the runners. No engine file changes: the fingerprint (`python3 tools/omni_version.py`) is the same before and after, and the six organisms' results on the engine that stands are unchanged by the seventh (the run is deterministic on its seeds; the rerun that adds the spine's rows reproduces theirs to the digit, checked by `realms-v5.yml`).
- Nothing is read across engines: the spine's rows carry the engine they ran on, like every other row.

CHAPTER 66

The Compass Law on Real Kubernetes: Preregistration

Written and committed before the run. The run's commit is the one that carries this file; nothing in the law, the harness or this rule changes after it starts.

66.1 What is run

Workflow `benchmark-reps` on `main`, 10 repetitions. Each repetition runs three arms back to back on the same runner, each on a fresh six-worker kind cluster, in an order rotated by repetition (`scripts/kind_paired.sh`):

Arm	What governs
native	Kubernetes alone (HPA at target 50, scheduler); Omni-Compass not started
omni	Omni-Compass on top with the engine's allocation law (`--law governor`, the law of sets 22 to 25)
compass	Omni-Compass on top with the compass law (`--law compass`, `omni_controller/controller.py`)

Load: fixed rate (`loadgen=open`), the same work in every arm. 900 measured seconds per arm. SLO 500 ms at the 95th percentile. Every Omni arm runs the six-state engine on every decision, the nervous system's authority and release gate, the shield, the compass, and ends with the reset, which must return the HPA target,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

its replica range, the pods' CPU limits and every worker to native, with no record left (`scripts/kind_bench.sh`).

66.2 The compass law in the live controller

The service position is the 95th-percentile response time over the SLO (0 calm, 1 the line); a blind probe or a pod waiting for a place reads as past the wall. The force is `A tanh((K_P (p - 0.5) + K_D v) / A)` with K[D] for critical damping times the realm push factor 3, the same law and gains as on every realm muscle (`realms/compass_arm.py`): up gain 0.10, down gain 0.02, release threshold -0.2. Two levers:

1. **HPA target**, cover from 60% of the operator's target to the operator's own: the up force lowers it (more pods), the down force returns it toward the operator's. It is never tighter than native.
2. **Node pool**: past the 0.95 wall one machine more at once; one machine back only while the force is below -0.2, the position is below the center, and the nervous system's release gate is open.

66.3 Outcomes and the rule

Primary: **worker nodes in service** (mean) and **95th-percentile response time**, each arm against native, paired over the 10 repetitions with a t-based 95% interval (`tools/live_reps.py`).

Band first: the compass arm is a win only if its p95 is not behind native's (the upper end of the 95% interval of the paired difference at or under 0) **and** failed requests are not higher. If that holds and machines in service fall with an interval wholly below 0, the label is **fewer machines within the band**. If machines fall but the band condition fails, the label is **tradeoff**. Otherwise **not established**.

Secondary, reported, not used for the label: p99, mean response time, HPA replicas, pods started, pod start wait, CPU including Omni-Compass's own, the declared energy models. The omni arm is reported against native and against the compass arm by the same rule. A run that fails its own checks (reset, controller stopped early, missing permission) is marked invalid and left out, never silently counted.

Evidence class **L**: real Kubernetes software on kind. Energy on kind is a declared model, not a meter.

66.4 Set 26 result

Compass arm against native: machines in service -17.2% (-26.4% to -7.9% of native), p95 -64.8%, failed requests 0 on both: **fewer machines within the band** (`results/live/LIVE_REPS_26.md`). The allocation law in the same set: machines -35.8%, p95 -55.4%.

66.5 Set 27 (written before the run)

The compass in the live controller now reads the service as the GPU compass does (`omni_controller/gpu_compass.py`, GPU amendments 6 and 7): the mean response time of the latency window between the bare service time (a tenth of the SLO) and the SLO, held at the compass's center

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

0.4 (the GPU service profile); p95 at or past the SLO, a blind probe or a pod waiting for a place is past the wall. Everything else, the arms (native, omni, compass), the load, the duration, the outcomes and the labelling rule above, is unchanged. The run's commit is the one that carries this section.

66.6 Set 27 result

Compass arm, aligned with the GPU governor, against native: machines in service -15.9% (-1.462 to -0.450 machines), p95 -65.5% (-311.8 to -156.1 ms), p99 -72.6%, failed requests 0 on both: **fewer machines within the band** (`results/live/LIVE_REPS_27.md`, run 37071353971, commit `d46c959`). The allocation law in the same set: machines -36.6%, p95 -53.1%.

66.7 The verdict in the live controller (2026-10-03, before any further set)

The compass law gives a machine back only where it measures that the service is never behind for it (`omnicompass/verdict.py`, stepwise, in `omni_controller/controller.py`). While the service is calm (inside the compass, no pod waiting, no breach), one more machine is given back on trial. The response times of 200 requests served without it are set against 200 served just before and against the cluster as it first ran on its own:

- at most 2% slower than both: the machine stays given back;
- slower than that: it is taken back and not tried again for 120 decisions.

Where no machine passes, the pool stays as the cluster runs it alone. The HPA target's cover is unchanged: from 60% of the operator's target up to the operator's own, never looser than native. Every trial is in the audit. The next set runs with this verdict; sets 26 and 27 ran before it and stay as they ran.

66.8 Set 28 and set 29 (2026-10-03, set 29 written before its run)

Set 28 (run 37087620193, commit `24666d7`) ran the compass law with the verdict. With Omni-Compass on top against native:

- machines in service −4.7%;
- p95 −65.3%, p99 −68.6%;
- pods waiting 0;
- failed requests 0.

Total CPU including Omni-Compass's own came out **+2.2%** (+0.003 to +0.042 cores), more than the 2% the one rule allows (`DISCLOSURES.md`, section 3). The cause is the controller's own cost: 0.063 cores, mostly a new kubectl process for every read, about 20 a minute. The compass law with the verdict freed only 0.040 cores of work. The allocation law in the same set: machines −29.6%, p95 −58.0%, total CPU including its own −1.2% (not significant). Set 28's receipt is `results/live/LIVE_REPS_28.md`, published with this section.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Set 29 runs the same arms, load, duration, outcomes and rule as set 28. One thing changes: the controller reads through one `kubectl proxy` started once, under the same least-privilege identity, so a read is a local HTTP request instead of a new kubectl process (`omni_controller/controller.py`, `Kube`; writes are unchanged; `tests/test_api_proxy.py`). The label also requires total CPU including Omni-Compass's own to be no more than 2% above native.

Set 29 result (run 37094338955, commit `a3721cc`): the compass law with the verdict, total CPU including its own **−4.7%** (−0.071 to −0.017 cores), passes; Omni-Compass's own CPU 0.011 cores (set 28: 0.063). Machines −5.1%, p95 −64.4%, p99 −71.1%, HPA replicas −7.6%, failed requests 0. No measure significantly behind native in either arm. Receipt: `results/live/LIVE_REPS_29.md`.

66.9 The cost to match (written before its run)

The question a buyer asks: what would native Kubernetes have to spend to answer as fast as it does with Omni-Compass on top? Each repetition runs, on the same runner and the same work, in rotated order:

- native (the operator's HPA target 50);
- native tuned harder by its operator, HPA target 40, 30 and 20 (more pods, faster answers), no Omni-Compass (`ARM=native40`, `native30`, `native20` in `scripts/kind_bench.sh`);
- native with Omni-Compass on top, the allocation law (`omni`);
- native with Omni-Compass on top, the compass law with the verdict (`compass`).

The report (`tools/live_reps.py`, "The cost to match") lists every arm's p95, p99, HPA replicas, CPU including Omni-Compass's own, and machines in service. For each Omni-Compass arm it names the cheapest native setting (by CPU) whose p95 is at or under Omni-Compass's, and that setting's extra replicas, CPU and machines over Omni-Compass. If no native setting tried reaches it, the report says so and gives the lowest native p95. 10 repetitions, 900 measured seconds per arm, fixed-rate load.

66.10 The fault test (written before its run)

Health, security and the babysitting a cluster needs, measured. Every arm (native; native with Omni-Compass on top, the allocation law; native with Omni-Compass on top, the compass law with the verdict) meets the same four faults at the same moments (`scripts/kind_faults.sh`, `FAULTS=1`):

1. at 15% of the run, a worker machine dies (its kind container is stopped) and comes back two minutes later;
2. at 35%, traffic triples for two minutes;
3. at 55%, a pod with no CPU limit burns CPU for two minutes;
4. at 75%, the response-time probe goes blind for one minute.

For each fault the report (`tools/live_reps.py`, "The fault test") gives the time to recover (from the fault's start until responses stay under the line for 30 s straight, at most 300 s) and the share of samples over the line or failed in the 300 s after it, paired against native over 10 repetitions. All the usual gauges are

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

reported as well, now including the share of response samples over the line (`pilot/bench_report.py`). The aim is lower in each.

The fault test, first run (run 37094604580, commit `a149d4e`; `results/live/FAULTS.md`). Omni-Compass on top recovered faster than native from every fault (allocation law: machine down −29%, runaway pod −35%, spike −7%; compass law: −14%, −17%, −6%), p95 −53% and −55%, p99 −27% (not significant) and −57%. One measure read LOSS: with the compass law, **HPA replicas +8.2%** (+0.41 to +0.99, significant), with CPU and machines unchanged and no energy saved, so outside the one rule (`DISCLOSURES.md`, section 3). The cause: past the wall the compass lowers the HPA target at once (more pods, the faster recovery), then handed the operator's target back step by step and held each step for the autoscaler's window, so the extra pods outlived the fault.

The change, written before the re-run (`omni_controller/controller.py`, the compass's push and pull on the HPA target; `tests/test_compass_controller.py`, "fault over"): once the responses are back inside the band (the compass's position under its center), the response line is clean and no pod is waiting, the operator's own target returns at once and is not held by the window. More pods only while the fault lasts. The re-run is the same fault test, arms, load, duration and rule (set 30 F); set 30 runs the same code without faults, to show nothing else moved.

Set 30 and set 30 F (runs 37105047258 and 37105046042, commit `acc1c4e`; `results/live/LIVE_REPS_30.md`, `results/live/FAULTS_30.md`). Set 30, no faults: no LOSS clear of the noise in either arm; the compass law's machines −9.9%, p95 −64.9%, time over the line −98.7%, HPA replicas −32.2%, total CPU −6.5%. Set 30 F: recovery faster than native from every fault in both arms; the compass law's HPA replicas **+5.6%** (first run +8.2%), still significant, with CPU and machines unchanged. The allocation law, which recovers as fast or faster, held no extra pods (+2.0%, not significant).

The second change, written before set 31 F (`omni_controller/controller.py`; `tests/test_compass_controller.py`, "blind"). Past the wall, the compass lowers the HPA target (more pods) only when the cause is load: the response line breached with every sense live and no pod waiting. Past the wall from a blind sense, or from pods waiting for a machine that is gone, more pods answer neither, so the target is the operator's own: fail up is native's own setting, as on the card, where fail up is the card's own clock and limit. The machine reflex (one machine more past the wall) is unchanged. Set 31 F is the same fault test, arms, load, duration and rule; set 31 the same without faults.

Set 31 and set 31 F (runs 37110007121 and 37110005322, commit `0a38e76`; `results/live/LIVE_REPS_31.md`, `results/live/FAULTS_31.md`). Set 31 F: the compass law's HPA replicas under faults **−1.1%** (not significant): the extra pods are gone. No LOSS clear of the noise in either arm; recovery faster than native from every fault (compass law, machine down −49%). Set 31: no LOSS clear of the noise; the compass law's machines −10.4%, p95 −66.0%, p99 −73.0%, time over the line −99.4%, HPA replicas −44.5%, pods started 0 against native's 4.3, total CPU −6.1%.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.11 The bill on a real cloud (written before its run)

The question a buyer pays for: the same work, a smaller bill? On kind every machine stays powered, so a machine given back saves only a declared model's energy. On a real cloud the machine is deleted and stops being billed. The run (`.github/workflows/aks-metered.yml`, `scripts/aks_paired.sh`, `scripts/kind_bench.sh` with `PLATFORM=aks`):

- **The cluster.** Azure Kubernetes Service, a fresh cluster for every arm, built the same way:
  - a system pool of one machine, tainted so no workload lands on it (AKS's add-ons and the load generator: kind's control plane);
  - a work pool starting at 4 machines (Standard_D2s_v5) under **Azure's own cluster autoscaler** (min 1, max 4; scale down after 2 minutes unneeded), which deletes a machine once it is empty.
- **The arms**, rotated in each repetition:
  - native: Kubernetes with Azure's autoscaler alone;
  - native with Omni-Compass on top, the compass law with the verdict (`compass`);
  - native with Omni-Compass on top, the allocation law (`omni`).

With Omni-Compass on top, the machines it gives back are idled (new pods go elsewhere, their pods leave first), and Azure's autoscaler then deletes them. Omni-Compass never deletes a machine itself.

- **The same work** in every arm: the fixed-rate load of sets 22 onward, 900 measured seconds after 120 s of warm-up.
- **The bill.** Every 15 s, the number of work machines that exist (in service or idle, every one is billed), integrated over the measured window: billed machine-hours, priced at Azure's list price for the machine (USD 0.096 an hour for Standard_D2s_v5, Linux, pay as you go, set in the workflow's input). Omni-Compass never reads this count.
- **Outcomes.** Billed machine-hours and the bill (the aim is lower), and every gauge of sets 28 and 29 (response time, failures, pods waiting, CPU including Omni-Compass's own), paired against native with 95% intervals over 5 repetitions. Labelled by the one rule (`DISCLOSURES.md`, section 3): no LOSS of more than 2%, and only where the bill or energy is saved.
- **Housekeeping.** One repetition at a time; each cluster deleted when its arm ends, before the next is made; the resource group deleted at the end of every repetition whatever happens. It needs the repository secret `AZURE_CREDENTIALS`.

66.12 The capacity test (written before its run)

The question behind "more work for the same cost": on the same machines, how much more work does Kubernetes serve with Omni-Compass on top before its answers break the line? Each repetition runs native, native with Omni-Compass on top (the allocation law) and with the compass law and the verdict, in rotated order, on the same six workers, the open-loop load rising in eight equal steps of one load generator each (6

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

requests a second per generator, 1 to 8 generators, 200 s a step, 1,600 measured seconds; `load_steps` in `benchmark-reps`). A run's capacity is the highest step at which no more than 5% of the response samples (every 5 s, through the Service) are over the line (500 ms) or failed, every lower step holding too, the first 30 s of each step left to settle (`tools/live_reps.py`, `capacity`). Reported: each arm's mean capacity in requests a second, its change against native and the 95% interval of the paired difference over 10 repetitions; every usual gauge beside it. Labelled by the one rule: no LOSS of more than 2%.

66.13 The fairness test (written before its run)

The question a buyer with many tenants asks: when one application surges, does Omni-Compass on top protect its neighbour, or starve it? Each repetition runs native, native with Omni-Compass on top (the allocation law) and with the compass law and the verdict, in rotated order, on the same six workers, with two applications on them (`TWO_APP=1`, `deploy/kind/noisy.yaml`): php-apache under its usual fixed-rate load, and a noisy neighbour, the same image and the same HPA rule, whose own fixed-rate load surges in steps (0, 0, 6, 0, 6, 0 load generators of 6 requests a second, the same moments in every arm). Each application's response time is probed through its own Service. Omni-Compass governs both HPAs (`deploy/kind/rbac-omni-noisy.yaml`: the second HPA and nothing else more). Reported: every usual gauge for php-apache, and the neighbour's p95, p99, time over the line and failed requests, each paired against native over 10 repetitions with its 95% interval. The label is the one rule, applied to both applications: no LOSS of more than 2% for either.

66.14 The capacity and fairness results, and the amendment they call for (2026-10-03 evening, before either is run again)

Capacity (`results/live/CAPACITY.md`, run 37150909816): the compass law served 33.0 requests a second within the line against native's 24.6, **+34.1% (95% interval of the paired difference +6.2 to +10.6 requests a second)**, with response times about half of native's and fewer failures. One measure significantly LOSS: pods started 7.3 against 5.6 (+30.4%) while the mean HPA replicas were 26.9% lower: churn, not more pods. **Fairness** (`results/live/FAIRNESS.md`, run 37154210570): the neighbour unharmed under both laws; with the compass law php-apache's failed requests 3.89% to 4.90% (+0.12 to +1.90 points), pending pods and pod start wait higher, no machine saved. By the one rule neither result is labelled GAIN. Both causes are read from the controller's code; both corrections are stated here, as law, before the tests run again.

1. Only a sensed muscle moves (the fairness cause). Let the probe measure the response time of the services in a set S (`--sensed ns/deployment,...`; empty means every HPA, the single-service case). For each HPA h scaling a deployment d(h) with the operator's target $x^{(op)_h}$, the target written is

$x_h(t)$ = the compass's (or the allocation law's) target, if $d(h) \in S$; $x^{(op)_h}$, if $d(h) \notin S$

The probe's position p is a statement about S alone. A push on a muscle outside S answers no sensed error and adds pods that compete with S for the same machines (the run's pending pods and failures). It is the GPU law's own rule, applied to Kubernetes: where Omni-Compass cannot sense, it stands at native's own setting.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2. Pods owed to a growing demand stay (the churn cause). Let u(t) be the CPU the workers use and W the HPA's scale-down window (W = 300 s unless the operator set one). The demand is *still growing* when

G(t) = [u(t) > (1 + ε) min[t − W ≤ s ≤ t] u(s)], ε = 0.05 .

After a breach the compass lowers the target to the bottom of its cover, x = x[lo] = 0.6 x^(op) (more pods, at once). Before this amendment, the first decision with the position back under the centre, p < c = 0.4, no breach and no pod waiting, returned x = x^(op) at once. Under a rising load the autoscaler then removed the extra pods one window later and started them again at the next step: the run's churn. The return is now

x ← x^(op) only if p < c, no breach, no pod waiting, ¬ G(t),

and while G(t) holds the target stays where the breach put it. A fault or a spike that has passed has a flat or falling u, so G is false and the operator's target returns at once, as the fault test requires (set 31 F: HPA replicas under faults −1.1%); only a demand still climbing keeps its pods. ε = 0.05 is set above the decision-to-decision noise of the node CPU reading. A rise too small to clear it leaves the rule as it was before this amendment (the target returns at once), so the correction can remove churn but cannot add any.

Nothing else changes: the compass's band, gains and centre, the verdict, the fail-up rules, the release gate and the node-pool law are as registered. Tests: `tests/test_compass_controller.py`, cases *demand* and *sensed*, beside the existing *fault over*, *blind* and reset cases.

The re-runs, on the commit that carries this amendment, unchanged in design: the capacity test (`load_steps` 1 to 8, 1,600 s), the fairness test (`two_app` 1, 900 s) and the fault test (`faults` 1, 900 s), 10 paired repetitions each, arms native / omni / compass. The second application is now probed and reported as before, and its HPA is held at the operator's target. Labelled by the one rule. Every number, whatever it says, is published beside the runs above.

66.15 The re-runs under the amendment: results (2026-10-04)

All three on commit `199f350`, 10 paired repetitions each, the design unchanged.

- **Fault test, set 32 F** (`results/live/FAULTS_32.md`, run 37162459956): **no LOSS clear of the noise under either law.** The compass law recovers from a lost machine 58% faster and starts 34% fewer pods; the amendment left the fault behaviour of set 31 F intact.
- **Fairness** (`results/live/FAIRNESS_2.md`, run 37162458834): **correction 1 holds.** With the compass law php-apache's failed requests are no longer lower (−14.0%, not significant; before +26.0%), pending pods −10.5% (before +65.2%); the neighbour unharmed under both laws. The compass law's response-time gains in this test are no longer significant. Still higher under both laws: the mean pod start wait (+1.1 s compass, +1.7 s allocation law).
- **Capacity** (`results/live/CAPACITY_2.md`, run 37162457542): the compass law served **24.6 against native's 16.2 requests a second, +51.9% (+6.2 to +10.6)**, the same paired difference as the first run on slower runners; response times −28% to −56%, failures −9.6%, HPA replicas −10.7%. **Correction 2 did not remove the extra pod starts** (5.1 against 3.8, +34.2%; before +30.4%): its premise, that the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

extra starts were pods removed and started again between steps, is not borne out. With the mean replicas lower, the reading the data support is pods started earlier on a rising load, the mechanism of the added capacity. Correction 2 stays (it removes no gain and added no measure power given); the pod-start rows stand as measured, and by the one rule neither the capacity nor the fairness arm is labelled GAIN while they do.

66.16 The pod record, and the second amendment (2026-10-04, before the next runs)

What the pod record shows (`tools/pod_report.py`, workflow `pod-report`, read from the stored artifacts of runs 37162457542 and 37162458834; nothing re-run). In the capacity test native started its pods once, early (33 of 38 in the first fifth of the window), and removed none. With Omni-Compass on top, two minutes into the window the controller raised the HPA target above the operator's (the conveyance: each pod given a larger CPU limit, the target raised by the same factor so each pod stays as busy, x = g x^(op); 114% with the compass law, 190% with the allocation law). The autoscaler then removed pods (14 with the compass law, 35 with the allocation law, over ten repetitions), and the next load steps started them again. **The extra starts are exactly those removals.** Correction 2 above read the wrong signal: the CPU used by the whole node, where one load step is lost in the node's own load.

3. A raise of the target waits for a steady demand. Let the demand on an HPA's service be read from the autoscaler's own status, $D(t) = \bar{u}(t)\, r(t)$: the pods' mean CPU utilisation of their request times the pods running, the CPU used in units of one pod's request. With W the HPA's scale-down window and $\varepsilon = 0.05$,

$$S(t) = [t - t_0 \geq 0.9\,W \;\wedge\; \max[[t-W,\, t]]\, D \leq (1+\varepsilon)\min[[t-W,\, t]]\, D]$$

(t_0 the oldest sample in the window). A new target above the one standing (fewer, larger pods) is written only when S(t) holds; the autoscaler itself removes pods only after its scale-down window, and Omni-Compass now asks the same of its own consolidation. A lower target (more pods) is never held, and the return of the operator's own target after a fault (rule 2) is unchanged. Correction 2's growth test G(t) now reads the same D(t) of the HPA concerned, not the node's CPU. Tests: `tests/test_compass_controller.py`, case *steady*.

The capacity, fairness and fault tests are run again on the commit that carries this amendment, unchanged in design, and published beside the runs above whatever they show. The bill run on Azure (`aks-metered`, run 37171672509) started on commit `199f350`, before this amendment, and is reported as of that commit.

66.17 The third amendment, and a change considered and declined (2026-10-04, before the next runs)

4. A lower target is never held. Until now every new HPA target, in either direction, was held for one autoscaler window W while the line was clean, so on a step up the compass's push (a lower target, more pods) could wait up to W and the pods arrive after the line is missed. The hold exists because a raise inside the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

window removes pods the autoscaler then starts again; a lower target asks for pods and removes none. The hold now applies to raises only:

write $x_h(t) \Leftrightarrow x_h(t) < x_h\hat{}(\text{now})$ `v` no write in [t−W, t) `v` the operator's own target returns,

with rule 3 still requiring a steady demand before any raise. Tests: `tests/test_convey.py` (a lower target written at once inside the window).

Declined: raising the replica cap above the operator's. `deploy/kind/demo.yaml` sets `maxReplicas: 10`, and at the top of the capacity test both arms meet it. Lifting it would let Omni-Compass run more pods than the operator authorised, which is more resources, not the same resources doing more, and it would break the shield's standing rule that Omni-Compass never moves past an operator's bound. Native would need the same cap for the comparison to stay fair. It stays at the operator's value in every arm; a higher cap is the operator's decision, tested as its own setting if ever asked for. Already in place and unchanged: a pod waiting for a place reads as past the wall and asks for one machine more at once.

A narrower form was proposed the same day (`docs/history/AMENDMENT_RISING_STEP_CAP.md`): while the demand rises and a pod of the sensed service is pending, raise `maxReplicas` by the pending count and lower the target to match. It is declined for a mechanical reason as well as the one above. The replica cap does not make pods pending: at the cap the autoscaler simply asks for no more pods, so the pending count there is zero and the write would never fire where it is aimed. A pod is pending only when the scheduler finds no machine with room for it, and more places under the cap give such a pod nowhere more to go; the answer to that is a machine, which the compass already asks for. The part of the proposal that holds, asking for pods at once on a step up instead of after a window, is rule 4 above.

On the CPU fill (used over allocatable about 0.10 in every arm): on kind every worker reports all of the host's cores as its own, so the allocatable counts the same cores once per worker and the fill reads far lower than the machine doing the work. Every receipt from the next runs on carries the host's own busy share and core count (`host_cpu.csv`, `tools/live_reps.py`), so the room left on the real machine is measured, not inferred.

The capacity, fairness and fault tests are run again on the commit that carries this amendment, beside the runs of commit `5d2e238` (rule 3 alone), so each rule's effect stays separable.

66.18 The second amendment's runs: results (2026-10-04)

`results/live/AMENDMENT_2_RUNS.md`, commit `5d2e238` (rule 3 alone), 10 paired repetitions each.

- **Capacity: no LOSS clear of the noise under either law.** Compass law 24.6 against native's 17.4 requests a second, +41.4% (+5.4 to +9.0); allocation law 25.2, +44.8% (+4.9 to +10.7). The compass law's pods started 5.7 against 4.1, interval −0.64 to +3.84: no longer significant. Rule 3 removed the significant pod-start excess and lifted the allocation law's capacity from +0.0% to +44.8%.
- **Faults: no LOSS clear of the noise under either law.**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Fairness: the compass law, no LOSS clear of the noise for either application.** The allocation law: php-apache much lower, and the neighbour's failed requests **+0.74 points (+0.19 to +1.28), significant**. The allocation law conveys idle CPU to the service it senses; on machines shared with a surging neighbour that CPU is the neighbour's headroom. The allocation law is not labelled GAIN in this test.

Standing after rule 3: the compass law is clean in all three tests (no LOSS clear of the noise, capacity +41.4%). Rule 4 (a lower target never held) runs next on commit `583c97f`, beside these, to see whether it adds capacity without costing a row.

66.19 The fourth amendment: the replica cap as a lever the operator grants (2026-10-04, before its run)

The founder's instruction: no number in the harness is hard-wired; every one moves where the system lets it. The cap of 10 in `deploy/kind/demo.yaml` is the value of Kubernetes' own php-apache example, not a choice of any operator, and an earlier session read it as an operator's bound. It becomes a lever, under the operator's grant.

5. Replica room. With `--replica-ceiling` N[max] granted (0, the default, leaves the cap untouched), for a sensed HPA under the compass law, with r the pods running, c the cap, $\bar{u}$ the pods' mean utilisation and x the target, the autoscaler's own arithmetic asks for $n = \lceil r\,\bar{u}/x \rceil$ replicas, and

c ← min(N[max], n) if r ≥ c, n > c, p ≥ centre or the line is breached; c ← c^(op) if n ≤ c^(op), p < centre, no breach, S(t).

The cap is raised only while it binds and the line is threatened, to what the autoscaler asks and never past the grant, and returns to the operator's once the demand has held still for a window. The operator's range is recorded before the first change; the reset restores it. Tests: `tests/test_compass_controller.py`, case *room*.

Its run (a setting of its own, labelled as such): the capacity test unchanged, `replica_ceiling` 30, beside the runs without it. Native keeps its cap of 10, as an operator who has not raised it would; the extra pods the compass uses are counted in the HPA replicas row, so the receipt shows what the capacity cost in pods, not the gain alone.

66.20 The third and fourth amendments, and the bill on a real cloud: results (2026-10-04)

Rule 4 (`results/live/AMENDMENT_3_RUNS.md`, commit `583c97f`): **the compass law has no LOSS of more than 2% on any measure in any of the three tests.** Capacity +48.1% (+5.7 to +9.9), no LOSS; fairness, no LOSS for either application (energy per core-hour on the declared standby model +1.5%, inside the allowance); faults, no LOSS, lost-machine recovery −61%. The allocation law: capacity +55.6%, no LOSS; in fairness the neighbour no longer LOSS. The first runs with the real machine metered: a 4-core GitHub runner 48% to 70% busy, where kind's "used / allocatable" reads 0.07 to 0.10.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Rule 5, the replica lever (`results/live/REPLICA_ROOM.md`, commit `2101c3d`, ceiling 30): the compass law served 24.6 requests a second, the same as without the lever, with 78% more pods and 26 pod starts against native's 4.6, both significant. **The replica cap was not what limited the service; the machine doing the work was.** The lever stays, off by default, as the operator's to grant where machines have room; it is not part of Omni-Compass's default setting and this setting is not labelled GAIN.

The bill on a real cloud (`results/live/AKS_BILL.md`, run 37187059424, commit `5b2832f`, four paired repetitions): **no difference in the bill either way** (allocation law −0.6%, compass law +0.8%, intervals across zero) and no measure significantly power given. Azure's own autoscaler already ran the workload on about 1.86 of 4 workers, so this light workload leaves no machine to give back. The machine savings measured on kind, where native has no node autoscaler, do not carry to a cloud with one at this load; that is the reading of record. The preregistered size `Standard_D2s_v5` is not allowed in the subscription's region; `Standard_D2s_v4` (same 2 vCPU, 8 GiB and list price) was used. Three earlier attempts stopped at the load generator's placement before any measurement and are not results.

66.21 The burst bill test (written before its run, 2026-10-04)

The steady bill run left Azure's autoscaler nothing to do and Omni-Compass no machine to give back. The question a buyer asks is the bill under load that moves: on a real cloud, with Azure's own cluster autoscaler underneath, does native with Omni-Compass on top bill fewer machine-hours than native alone when demand rises and falls? Same design as "The bill on a real cloud" (fresh AKS cluster per arm, work pool 1 to 4 `Standard_D2s_v4` under Azure's autoscaler, the bill metered every 15 s, arms native / compass / omni rotated, five repetitions), with the open-loop load in bursts (`load_steps` 1 6 1 8 1 6, six steps over 1,800 measured seconds, the same steps in every arm). Reported: machine-hours and the bill at list price, paired against native with their 95% intervals, beside every service gauge. Labelled by the one rule: a lower bill counts only if nothing is a LOSS of more than 2%.

66.22 The six organisms with the real cluster inside (written before its run, 2026-10-04)

Every benchmark runs the same six organisms, native against native with Omni-Compass on top: the four realms, the whole tower of 656 muscles, and the four stacked with every duplicate kept (1,226). Six organisms, two arms, twelve columns. The model grid (`six`) and the card in the loop (`tools/run_hil.py`, single card and eight cards) already do. The Kubernetes runs now do too (`tools/run_kil.py`, workflow `six-kube` on kind, input `organism` on `aks-metered`).

Each organism runs on the measured window's clock, 240 steps, with the real cluster as one more muscle:

- efferent: the organism's offered compute work at step k, D(k) = sum over its compute pools of lam_i(k) / sum of lam0_i, the seeded trace, identical in both arms. The load generator runs r(k) = 1 + round((LOAD_MAX - 1) (D(k) - min D) / (max D - min D)) replicas, LOAD_MAX = 6 declared here, open-loop load.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- afferent: the cluster's watts (capture.csv, the declared power model, every 15 s, identical accounting in both arms) are added to the organism's source, zone and site power, as the card's watts are in the card harness.
- the clock (stated 2026-10-07, after the 1,000-copy runs): the organism must keep the window's clock. `tools/run_kil.py` records how long after the window its last step ended (`behind_s`); the report (`tools/six_kube_report.py`) shows it as "organism behind its window (s)", shown and not judged, and marks a repetition OFF THE CLOCK when either arm ended more than 5% of the window late, because its last steps then saw a cluster whose load schedule had already ended. The pairing stands (both arms slip alike), the mark stays on the cell. The remedy is a longer window for that size on that machine (240 steps, each at least the machine's time to step the organism once), never a change to the organism or the law: the four stacked at 1,000 copies (1.7 million muscles) step in about 31 s on an 8-vCPU machine, so their window is 10,800 s (45 s steps), against 2,880 s for the tower at 1,000 copies.

Arms: native (the stacks' own controllers, Kubernetes alone, Omni-Compass not started) and compass (the compass law on every simulated muscle, rule 4 on the cluster, handed back at 90% of the window; the run is invalid if any knob is not handed back). Five paired repetitions per organism, order rotated, each arm on a fresh six-worker kind cluster, 960 measured seconds. Seed 6000 for every organism and arm.

Reported in `SIX_KUBE.md` (`tools/six_kube_report.py`): twelve columns of means, then per organism every gauge paired against native with its 95% interval. Cluster rows are measured; organism rows are models (evidence S). Each organism is labelled by the one rule: no LOSS of more than 2% on any measure (CPU and host load shown, not judged), and a gain counts only where energy or the bill is lower with an interval wholly below zero.

66.22.1 Result: the six organisms with the real cluster inside (run 37217362568, commit `d81d5ee`)

Thirty jobs, six organisms times five paired repetitions, native against native with Omni-Compass on top (compass law), every arm valid, every simulated knob handed back. Full report `results/live/SIX_KUBE.md`; raw files `results/live/raw/run-37217362568/`.

On the real cluster, in every organism, native + Omni was late less often and answered faster:

Organism	Time over the line, native → Omni	p95, native → Omni (ms)	Failed requests, native → Omni
Compute / AI / Cloud	41.0% → 26.7% (lower, -35%)	3,967 → 2,653 (lower, -33%, inside the noise)	14.9% → 13.3%
Physics / Robotics / Autonomous	28.4% → 15.5% (lower, -45%)	2,440 → 1,767 (lower, -28%)	5.7% → 5.0%
Energy / Facility / Industrial	21.6% → 10.4% (lower, -52%)	1,951 → 1,482 (lower, -24%)	3.8% → 3.2%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Organism	Time over the line, native → Omni	p95, native → Omni (ms)	Failed requests, native → Omni
Distribution / Specialized	40.7% → 21.1% (lower, -48%)	2,830 → 1,943 (lower, -31%)	11.1% → 9.3%
The whole tower (656)	25.8% → 14.7% (lower, -43%, inside the noise)	2,085 → 1,247 (lower, -40%, inside the noise)	12.8% → 9.1%
The four stacked (1,226)	55.6% → 43.0% (lower, -23%)	3,703 → 2,803 (lower, -24%)	31.7% → 27.6%

No measure read LOSS beyond the noise in any organism. Energy (declared model) and CPU moved by under 3%, lower with Omni in every organism. The modelled organisms around the cluster: work the same, energy 0.1-0.2% lower, time over the line 0.5-2% lower, in every organism (evidence S).

Read with it: at a peak of six open-loop load generators the 4-core runner is past what the six workers can serve in both arms (native late 22-56% of the time, 4-32% of requests failed; host 65-91% busy). The comparison is fair (same load, same seed, same runner per pair); the absolute levels are an overloaded cluster, and a lower `load_max` is the setting for a cluster inside its capacity.

66.23 More work, faster, on fewer machines, with less energy: all four in one run (written before its run, 2026-10-04)

The four gains so far come from different tests: more work on the same machines (the capacity test, +48.1%), the same work faster on fewer machines (sets 22-27), energy equal or lower in each. This test measures all four in one run.

Design: `benchmark-reps`, ten paired repetitions, arms native and compass (rule 4), order rotated, fresh six-worker kind cluster per arm, open-loop load rising and then falling, `load_steps` 1 2 3 4 5 6 7 8 7 6 5 4 3 2 1, 180 s a step (2,700 measured seconds). The rise is the capacity test (`tools/live_reps.py capacity`, read on the way up to the peak: the highest load step at which no more than 5% of response samples are over the 500 ms line or failed, every lower step too). The fall is where machines are no longer needed and can be handed back. Over the whole window: p95 response time, worker machines in service, energy (declared model; on kind it is not a meter), CPU with Omni's own.

Reported for each, native + Omni against native, paired, with the 95% interval and read in words (GAIN, EVEN or LOSS): work (capacity, requests a second inside the line), speed (p95), machines (worker machines in service, mean), energy.

66.24 The Omni index: one number for more for the same, or the same for less (written before its first use, 2026-10-04)

Each measure is turned into a ratio oriented so that above 1 is a GAIN for Omni-Compass:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- work: work with Omni / work native (capacity, requests served, work done)
- speed: p95 native / p95 with Omni (a lower response time is faster)
- machines: machines native / machines with Omni (in service, or billed machine-hours)
- energy: energy native / energy with Omni (or the bill)

The index of one test is the geometric mean of its oriented ratios, minus one, in percent: +30% reads "30% more for the same, or the same for 30% less", across work, speed, machines and energy together. A measure a test did not take is left out of that test, never filled in. A measure inside the noise is counted at its mean and flagged. A category (real Kubernetes on GitHub, Azure, the card, the eight cards, the modelled muscles) is the geometric mean of its tests; the headline is the geometric mean of the real categories, each weighted the same; the modelled muscles are shown beside it, never inside it. Computed by `tools/omni_index.py` from each test's own paired results; nothing is typed in.

The service reading, a second reading declared 2026-10-09 (the first unchanged). The founder asked what a cache is for. Redis is bought for speed and spends memory to deliver it; the index above counts that memory as a cost at the same weight as the speed it buys, so the Redis category reads −24.9% while its work inside the line and its hit rate read GAIN in all three of our runs in every run. The question is fair, and the answer is a second reading beside the first, never a change to the first: the same tests, the same three-run rule, scored on the service the stack is bought for, **work and speed only**, with the resources it spent or saved (machines, energy, memory, connections, consumers) shown beside it and not scored. It is computed for every test alike by the same tool (`service_pct`, `service_headline_pct`), never for one stack, and printed in the same tables as a second column; the resource reading above stays the headline this program preregistered. The two readings say different true things: where the governor buys service with a resource (Redis's memory ceiling, Kafka's consumers) the service reading is higher and the resource reading says what it cost; where it gives a resource back for the same service (the database pools and caches) the service reading is nothing and the resource reading carries the gain. On the day it was declared the service reading stood at +71.8% against the resource reading's +20.5% (Kubernetes +83.2%, Kafka +1,192.8%, Redis +8.8%, the three database stacks exactly nothing; on the sets of 10 October, the brain's verdict on every stack knob, the two readings stand at +10.7% and +4.8%); Kafka's hundredfold speed ratio is why the service headline is large, and the category table is where a reader sees it. A loss in all three of our runs in work or speed counts against Omni in both readings; neither hides one.

The wiring verdicts, declared 2026-10-09 (a reading of the tables, not a change to any rule). The founder's order of the same day: Omni need not be wired into every muscle; where it cannot beat native, the muscle stays native and Omni only reads it, one wire out and no wire in. `tools/wiring_verdicts.py` reads every three-run table, the simulator tables, the realms table and the organism tables by one rule into three words per knob and case: **write** (a judged gauge GAIN in all three of our runs, none LOSS or power given in all three of our runs), **watch** (nothing GAIN in all three of our runs, whether inside the noise, the runs disagreeing, or a loss with nothing against it) and **operator's choice** (gains and losses both the same in all three of our runs, shown with both readings of the index). Rows shown and not judged never count. Nothing is typed in, and `verify.py` fails if the page (`docs/WIRING_VERDICTS.md`) differs from what the tables

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

give. The verdicts change what an operator connects, never the engine (Omni v3) or any rule written here, and every test on this page is read as before.

66.25 Demand that wanders: up, spike, partway down, back up, down to idle (written before its run, 2026-10-04)

Real demand does not rise once and fall once. It climbs, spikes, eases part way, climbs again and finally settles to idle. This test drives that shape and asks whether machines follow it in order: the emptiest machine idles first on the way down (powered and Ready at its floor, never off), the warm machines wake first on the way up (no boot), one machine always in service for the first burst (`scripts/kind_nodepool.sh`).

Design as the all-four test (ten pairs, native and compass, open-loop load, fresh six-worker kind cluster per arm), with `load_steps` 1 2 3 2 3 4 5 6 5 3 5 6 5 4 3 2 3 2 1 1 (twenty steps of 135 s, 2,700 measured seconds), the same steps in every arm. Reported, native + Omni against native, paired with the 95% interval and read in words: time over the 500 ms line, failed requests, p95, worker machines in service (and its trace step by step), energy (declared model), CPU with Omni's own, pods started. No capacity is read (the load does not only rise).

66.25.1 Amendment to the wandering test, written before its run (2026-10-04)

The founder's two rules for real traffic, adopted before any result of the wandering test: traffic moves one step at a time, up or down, never skipping (it may go 1 2 1 2 1 2 if it never needs 3); and machines never go below two, so two are always in service, ready for a spike, with no ceiling but the machines the cluster has. The run started with the earlier steps (which jumped 5 to 3 and 3 to 5) was cancelled before it finished; none of it is reported.

From this amendment: the machine floor is two in every arm Omni-Compass governs (`MIN_NODES`, default 2: the controller's `--min-nodes`, the actuator `scripts/kind_nodepool.sh`, and every benchmark script). Native's own floor is the cluster's: kind keeps all six workers. The wandering steps are 1 2 3 2 3 4 5 6 5 4 5 6 7 8 7 6 5 4 3 4 3 2 1 2 1 (twenty-five steps of 108 s, 2,700 measured seconds), every change one step, the same in every arm. Everything else is as written above.

66.25.2 Result: all four in one run (run 37226863122)

Ten paired repetitions, every arm valid. **Work +29.3%** (capacity 24.6 to 31.8 requests a second inside the line, interval +5.4 to +9.0), **p95 -62.3%** (685.8 to 258.3 ms), time over the line -40.9%, failed requests -12.1%, **machines in service -3.6%**, **energy -0.3%** (declared model), CPU with Omni's own -1.9%: each proven. No measure power given beyond the noise (pending pod-minutes +19.6% and pods started +4.0%, both inside the noise). This run's machine floor was one; the floor of two was adopted after it started. Report `results/live/ALL_FOUR.md`; raw files `results/live/raw/run-37226863122/`; the Omni index updated (`results/OMNI_INDEX.md`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.25.3 The boundary is the band, not a machine held out (written 2026-10-04, for every run started after it)

Holding one whole machine back as a ceiling was considered and set aside the same day: it takes one of the machines away from the work. Every machine is usable (Omni-Compass's most is every machine that exists); the floor of two stays for quiet times. The protection against running into the wall is the band inside every machine: capacity is added at 95% of the response line, before the line is reached, and a machine goes back only if the ones left still run at or under the engine's utilisation target. An operator who wants whole machines held back can set `NODE_CUSHION` (off by default). The wandering run under way (37235231963) uses every machine, as every run after it does.

66.26 The six organisms at every size, the real cluster inside (written before its run, 2026-10-04)

The same six organisms, each native against native with Omni-Compass on top, at the sizes of the model grid: 10, 100 and 1,000 copies of the organism governed together on one clock, the one real cluster inside as one more muscle (`tools/run_kil.py --scale`; size 1 is the run already recorded, `results/live/SIX_KUBE.md`). Workflow `six-kube`, one job per organism, size and repetition, both arms on one runner, order rotated, fresh six-worker kind cluster per arm.

- Repetitions: 5 at 10 and 100 copies, 3 at 1,000. Measured window: 960 s at 10 copies, 1,440 s at 100, 2,880 s at 1,000 (240 organism steps of 4, 6 and 12 s: a step at least 1.5 times what the organism takes to compute on this runner, measured: 0.18 s at 10 copies of the four stacked, 2.4 s at 100).
- The organism is built before the window opens (`organism.ready`); the window opens the moment it is built (`organism.go`), so the cluster and the organism start on one clock however long the build takes. If the organism falls behind its own clock, `organism.json` records by how much (`behind_s`).
- Machine floor two, every machine usable (the founder's rules above), open-loop load, peak six generators.
- Declared before the run: 1,000 copies of the whole tower (656,000 modelled muscles) and of the four stacked (1.2 million) need about 6 and 11 GB of memory beside the cluster, past what one GitHub runner holds. They are run; if a runner cannot hold them, the job's failure is reported as that, never as a result.

Reported in `SIX_KUBE.md` by organism and size, every gauge paired against native with its 95% interval and read in words.

The run dimension of the model grid (1, 10, 100, 1,000 runs) is a count of seeds; on the real cluster each repetition is a real paired run of an hour or more, so the real cluster carries the repetitions above, and the model grid carries 1 to 1,000 runs.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.26.1 The two organisms too big for a GitHub runner: a larger rented machine (written before their run, 2026-10-05)

1,000 copies of the whole tower and of the four stacked run on a larger rented machine, the same test otherwise: workflow `big-organism` rents one Azure Standard_D8s_v4 (8 vCPU, 32 GiB) per repetition, runs `scripts/kind_paired.sh` on it (native against native with Omni-Compass on top, fresh six-worker kind cluster per arm, 2,880 measured seconds, the organism at 1,000 copies with the real cluster inside), copies every file back and deletes the machine whatever happens. Three paired repetitions per organism, two machines at a time. Its results join `SIX_KUBE.md` at 1,000 copies; the same cells from GitHub runners count only if the runner held them.

Result on Omni v3 (`results/live/V3_BIG_ORGANISM.md`*, both organisms, written 9 October after the second collect).* The machines were rented one at a time through `big-organism-detached` with a 10,800 s window an arm (the 2,880 s window of the v1 runs put the compass arm off the clock; the clock rule and the longer window are declared in the robustness and v3 notes). The four stacked: three repetitions over 29 hours (collected by run 37754612102), p95 2,882 → 150 ms, time over the line 51% → 0, failed requests 0.14% → 0, machines 6 in both arms, both arms 22 s behind the window. The whole tower: three repetitions over 25 hours (started by run 37756284680, collected by run 37945866426), p95 6,690 → 160 ms, p99 9,068 → 187 ms, time over the line 76% → 0, failed requests 4.8% → 0, machines 6 in both arms, both arms 14 s behind the window. Every cluster row that moved is clear of zero with three pairs; the organisms' own model rows are shown beside them and never enter the index; the rounding-level model work loss reads LOSS by rule in both. Six repetitions, six on the clock.

66.26.2 The burst bill test on Azure: two native arms lost to the harness, and the fix (2026-10-05)

Repetitions 2 and 3 of the burst bill test (run 37216328055) each lost their native arm (Kubernetes alone) at the same moment: the load step from one generator to six. Native's two work machines were full, answers timed out, and the API server stopped answering for minutes. The capture script ran with stop-on-first-error, so one unanswered reading ended the recording, and the load schedule stopped at its next unanswered scale; the arm failed its checks and was marked invalid. The Omni-Compass arms of the same repetitions ran every step. Those two arms are not counted.

That is a fault of the harness, and it threw away exactly the moments a cluster struggles. Fixed: a reading the API server does not answer is logged (`capture.csv.errors`) and skipped, and the recording goes on; a scale the API server does not answer is tried again, then logged, and the schedule goes on (`fleet/capture/kube_capture.sh`, `scripts/kind_bench.sh`, `tools/run_kil.py`). Repetitions 4 and 5 started on the earlier code. When the run ends, three more repetitions of the same test run on the fixed code, and the report counts every valid pair and names every invalid arm with its cause.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.27 Amendment 6: a pinned gauge is not a steady demand (written before its run, 2026-10-05)

The wandering test (run 37235231963, ten pairs) gave native + Omni-Compass p95 -54%, time over the line -38%, failed requests -14%, energy -0.3% and CPU -2%, each proven, and 1.5 more pod starts a run (+36%, proven). The founder holds that result back until the cause is fixed and the test is run again.

The cause, read from the run's own audit (repetition 4, 12 pod starts against native's 7): at 972 s the load eased from five generators to four, rule 3 found the demand steady, and the target was raised (50 to 190: fewer pods); the load then climbed back to eight and the pods were started again. The demand was not steady. The autoscaler stood at its replica cap, every pod as busy as it could be, so the reading (utilisation times pods) could not rise however much the load did: a pinned gauge reads flat.

Rule 5 (amends rule 3): a decision at which the autoscaler stands at its replica cap marks the window pinned; a window that touched the cap is not steady, so no raise of the target, and no return of a raised cap, until one whole autoscaler window after the cap was last touched (`omni_controller/controller.py` `_steady`, `self.pinned`; tested in `tests/test_compass_controller.py`). Nothing else changes.

The rerun: the same wandering test (ten pairs, native and compass, `load_steps` 1 2 3 2 3 4 5 6 5 4 5 6 7 8 7 6 5 4 3 4 3 2 1 2 1, 2,700 s, floor two, every machine usable). Reported in full, every gauge read in words; the first run is kept in its raw files and named beside the rerun.

66.28 Amendment 7: coasting, off the gas but still in gear (written before its run, 2026-10-05)

The founder's picture: a car in drive. At idle it creeps, ready (the floor of two machines). On the gas it speeds up at once (more pods, machines woken, no boot). Off the gas it coasts down gradually toward idle; it does not drop into neutral. The brake is for stopping (the reset hands everything back at once).

Rule 6: a raise of the HPA target (toward fewer pods) moves by at most `--coast-step` points of utilisation in one autoscaler window (default 25, `COAST_STEP`; 0 turns it off). The first wandering run raised the target 50 to 190 in one move; under rule 6 the same raise takes several calm windows (50, 75, 100, ...). A demand that comes back part way finds the pods still running and is met at once by a lower target, with no pod started again. A lower target (more pods) is never limited: the gas is always immediate. Rules 5 and 6 together: never ease off while the gauge is pinned, and then ease off a step at a time (`omni_controller/controller.py`; tested in `tests/test_convey.py`).

The rerun with rule 5 alone (run 37255770249) was stopped before it finished, to run the test once with both rules; none of it is reported. The rerun: the wandering test as written in amendment 6.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.29 Amendment 8: cruise and the emergency brake, and the batch test (written before its run, 2026-10-05)

The founder's driving modes: off (native alone), watching (Omni-Compass reads, writes nothing), autopilot (gas, brake, idle), and two more for work that comes as a pile:

- **Rule 7, cruise:** work waiting for a place (pods pending) two decisions in a row puts every machine in service (`--max-nodes`), and they stay in service, without second-guessing, until the line has been empty and nothing is scaling up for two decisions (`--cruise-after`, default 2; 0 turns it off).
- **Rule 8, the emergency brake:** nothing waiting, no response-time breach, nothing scaling up, and the sensed services' demand at or under 0.05 of one pod's request (`--brake-demand`): the machines go straight to the floor (two) in one move. Every check of the release gate still holds (every sense live, the last command landed, the machines left at or under the utilisation target) except one machine per decision. Never below the floor.
- Tested through the live controller: `tests/test_cruise_idle.py` (in `verify.py`).

The batch test: a queue of jobs on the six-worker kind cluster (`WORKLOAD=batch`, `deploy/kind/batch-jobs.yaml`): one Kubernetes Job of 240 pods, 60 at a time (more than the workers hold, so work waits for a place), each hashing 3,000 MB (real CPU work, identical in every arm), opened with the measured window (1,500 s); the service's load generator stands at zero. Ten pairs, native and compass, order rotated. Reported, paired with the 95% interval and read in words: how long the queue took to finish, the worker machines in service after it finished, machines in service and energy over the whole window, CPU with Omni-Compass's own, pods started. Cruise must not slow the queue; the brake must show in the machines held after it.

66.30 The frozen engine (rules 1-8, commit 353903683009): the fault and fairness results (2026-10-05)

Both tests ran again with nothing changed but the engine: ten pairs each, native and omni, order rotated.

- The fault test (GitHub run 37262797634, `results/live/FAULTS.md`): a proven GAIN on p95 (−60.1%), mean response (−34.5%), time over the line (−22.6%) and pending pods (−76.5%). No LOSS beyond the noise. Recovery after the fault, paired means: machine down 42 s against 70 s, a blind probe 54 s against 66 s, a runaway pod 98 s against 105 s, a spike 266 s against 267 s.
- The fairness test (GitHub run 37262799317, `results/live/FAIRNESS.md`): a proven GAIN on mean response (−15.0%). No LOSS beyond the noise, the neighbour's app included.

These replace the earlier engine's fault and fairness sections of `results/live/AMENDMENT_3_RUNS.md` in the Omni index. That file stays as first measured.

The Azure burst bill test and the big organisms, running them (2026-10-05): Azure no longer offers this subscription Standard_D2s_v5 in eastus, so AKS workers are Standard_D2s_v4 (2 vCPU, 8 GiB, the same list

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

price, USD 0.096 an hour). Both arms use the same size; the steady-load result of record (`results/live/AKS_BILL.md`) stays as measured on D2s_v5. The big-organism machine failed while seven kind nodes joined (kubelet-start). Fresh Ubuntu ships too few inotify watchers for that; the rented machine now raises them as kind's own documentation advises. Neither change touches the controller.

66.31 Amendment 9: idle read from the autoscaler's floor, and the batch test again (written before its run, 2026-10-05)

The batch test on the frozen engine (GitHub run 37262795697) showed rule 8 late. The queue emptied 664 s into the window, but the emergency brake fired at 1,574 s. The reason: a served service is never at zero CPU. The response probe alone keeps one pod at about 10% of its request, which reads as demand 0.10, above the brake's 0.05. Rule 8 now also counts a service as idle when its autoscaler stands at its least pods (`minReplicas`), wants no more, and runs at no more than half its target utilisation. Every other condition of rule 8 holds as before: nothing waiting, no breach, nothing scaling up, the release gate, every sense live, the last command landed, and the machines left carrying what runs now under the utilisation target. Test: `tests/test_cruise_idle.py` (it lifts to the idle floor, and it does not lift there above the floor or while busy).

Effect on what was already measured: none. The steady, fault, fairness, wandering and all-four runs on the frozen engine had the service at one pod with nothing waiting in 0 of 5,102 readings, so the amended rule could not have fired in any of them.

Three of the ten batch pairs stopped, all in the omni arm. The API server answered 500 under the batch load (60 jobs asking for 30 cores on a 4-core runner), and the recording script ended on an unanswered `kubectl top nodes`. That is a harness fault: a reading the API server does not answer is now logged and skipped, and the end-of-window reads are tried again. The batch test runs again, ten pairs, on this engine, with the design unchanged.

66.32 Amendment 10: two rows read for what they measure (written after the frozen-engine runs were seen, 2026-10-05)

Disclosed: this is a change in how two rows are read, made after the steady, wandering and all-four results were seen. Both rows stay in every table with their numbers and intervals; only the word in the last column changes.

- **Pending pods.** The row counts pods pending in each 15 s snapshot. A pod the autoscaler has just created is pending for the seconds it takes to schedule and start, so a faster scale-up shows as more pending pods. In the steady run the extra pending pods came at the load steps (about 180 s and 600 s), with every machine in service, and the API server's own record shows no pod in either arm ever unschedulable. The row is now shown, not judged. The judged gauge is the scheduler's own verdict, from the pod record:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

pods that no machine would take (PodScheduled False, reason Unschedulable), counted and timed in pod-minutes. In the five frozen-engine runs it is 0 in every arm.

- **Energy per core-hour.** It is energy divided by CPU used. CPU used is shown, not judged (more or less is not toward an aim by itself), so a ratio over it is not judged either. Energy itself stays judged.

Nothing else changes. With these readings, no row of the steady, wandering, all-four, fault or fairness results is behind native beyond the noise.

66.33 Amendment 11: the burst bill test sized to what the service can serve, and rounding read as the same (2026-10-05)

Disclosed: written after three repetitions of the burst test at `1 6 1 8 1 6` were seen. In them about 41% of requests failed in both arms. The service's own autoscaler sat pinned at its cap of 10 pods, and 6 and 8 load generators (36 and 48 requests a second) ask more than 10 pods can serve, whatever the machines. A test where neither arm can do the work cannot show a difference. The burst test runs at `1 3 1 4 1 3`: the peak is one step above the steady test's peak of 3, and inside what the capped service can serve. Everything else is as preregistered: five repetitions, 1,800 measured seconds, Azure's autoscaler underneath, the bill metered every 15 s, native against compass. The three repetitions at the old steps stay in the archive (`results/live/raw/run-37294579764/`) and are reported as the reason for this amendment, not as a result.

In every table a paired change under one part in a million of the value reads "same", the number still shown (`SAME_REL`, `docs/MECHANISM_OF_ACTION.md` 9.5).

66.34 Amendment 12: cruise steps back, and the organism's start file (written before the batch rerun, 2026-10-05)

The batch rerun on amendment 9 (GitHub run 37275371557, ten valid pairs) gave machines -16.9%, -27.2% once the queue was done, and energy -11.6% on the standby model. The queue finished 10 s later (584 s against 594 s, +1.8%). Omni held every machine in service throughout, so the time went elsewhere: every five seconds the floor step read every pod (240 batch pods), every node, node metrics and every HPA, on the one 4-core runner the queue was using. In cruise every machine is already in service, so those reads can change no action. From this amendment, while cruising the floor step makes no read and no write, and the service's target stays at the operator's own (`docs/MECHANISM_OF_ACTION.md` 9.3; `tests/test_compass_controller.py`). Cruise never engaged in the steady, wandering, all-four, fault or fairness runs, so this changes nothing there. The batch test runs again, ten pairs, design unchanged.

The organism with the real cluster inside waits for a start file. One repetition of six-kube (the four stacked, 1 copy, repetition 2) read that file in the instant it existed but was still empty, and stopped. The file is now written whole or not at all, and read only once it holds a number. That repetition is reported as stopped, never counted.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Also found and fixed on 2026-10-05, from the compass rename: each modelled muscle's compass was made fresh at every decision (`docs/MECHANISM_OF_ACTION.md` 9.1). That touched the modelled organism rows of the runs started after the rename (six-kube 37280090832, big organism 37287311224); their cluster rows are unaffected. The organism runs go again on the fixed code.

The batch test on amendment 12 (GitHub run 37344765837, ten pairs, `results/live/BATCH.md`): machines in service -20.7%, -32.4% once the queue was done, energy -14.3% on the standby model, mean response -11.7%, all proven. The queue finished +1.1% later, inside the noise (it was +1.8% and proven before cruise stepped back). No pod was left without a machine. No LOSS beyond the noise.

66.35 The fleet that can show one machine: Azure at 11 and 15 workers (written before its run, 2026-10-07)

The Azure runs so far used a work pool of 1 to 4 machines. One machine is a quarter to a half of that fleet, and the app's replica ceiling (10 pods of 200m, about one worker's worth) kept Azure's autoscaler at about 1.9 workers in both arms, so nothing under a quarter of the fleet could show. The steady run on v1 (5 pairs) read even on every gauge. That is a statement about the lever, not about the law. This run makes the lever big enough to see one machine.

- **Fleet.** The subscription allows 200 vCPUs in eastus but 10 vCPUs in each machine family (every family's request to raise it was refused through the API on 2026-10-07: the subscription's terms), so one family gives at most 5 machines. The fleet is therefore several work pools, one 2-vCPU machine family each, each capped at what its family allows: `worker_pools` = `Standard_D2s_v4:4,Standard_D2as_v4:5,Standard_D2_v4:5,Standard_D2a_v4:5,Standard_D2ds_v4:5,Standard_D2s_v3:5,Standard_E2s_v4:5,Standard_E2as_v4:5`, 39 workers (the DSv4 family also carries the system machine, hence 4). Every pool carries the label `omni-role=work`, the worker selector in both arms; the first pool keeps one machine, the others may scale to zero; every pool starts full and Azure's own autoscaler trims, as before. One machine is then 2.6% of the fleet. Mixed machine families are how real cloud fleets run; the bill is each pool's machine-hours at its own list price (`pool_prices`), Azure's own count every 15 s. The same autoscaler profile, a fresh cluster per arm. (Written first for 11 then 15 workers of one family; the family allowance made that impossible and this replaces it before any such run.)
- **Ceiling.** `hpa_max` = 9 × workers = 351 at 39 workers: nine 200m pods fit one 2-vCPU worker, so the fleet can fill. The same ceiling in every arm; the restore checks expect it back untouched.
- **Load.** The load generator's replica steps scale with the fleet, the same in every arm (one load replica drove about five pods at 50% on the 4-worker runs). Steady: `18 35 53 18 35 18` over 900 s at 39 workers (the peak asks for about 265 pods, thirty workers' worth). Burst: `11 35 11 53 11 35` over 1,800 s (the 4-worker burst was `1 3 1 4 1 3`). A fleet of another size scales the steps by workers/39, rounded.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Everything else as preregistered**: native (Azure's autoscaler alone) against omni (Omni-Compass on top of it, rule 4 on the node pool, the HPA target inside its range, handed back at 90% of the window), 5 paired repetitions, order rotated, the bill Azure's own machine count every 15 s at list price, the reading by `tools/live_reps.py`'s paired interval, every gauge reported. Engine: Omni v3 first (the newest), v1 after if the credits allow.
- **What it can say.** Lower, clear of the noise, in bill or response time: Omni has value on the managed service when the fleet is big enough to see a machine. Even again: Omni's value there is nil at this size too, and that is the reading. Higher: our own wiring is suspected first, found, fixed, and the run is made again.
- **Cost.** About $8 for the steady set and $15 for the burst set at 15 workers, at list price.
- **Amendment (2026-10-07 07:18 UTC, before the counted run).** The first dispatch at 39 workers was refused by Azure on every repetition: the DASv4 family had 2 vCPUs left, not 10, because the rented machine running the four stacked at 1,000 copies (`big-organism-detached`, a Standard_D8as_v4) holds 8 of that family's 10 in the same subscription. The run was cancelled (no arm ran) and dispatched again at **35 workers**: `Standard_D2as_v4:1`, the other seven pools as above; `hpa_max` 315; the steps scaled by 35/39 as this section says, steady `16 31 48 16 31 16`, burst `10 31 10 48 10 31`. One machine is 2.9% of this fleet. Nothing else changes. The 39-worker fleet stands for the runs made after the detached machine is deleted.
- **Amendment 2 (2026-10-07 09:40 UTC, before the counted run).** The 35-worker dispatch was refused too, on three repetitions: the DAv4 family (`Standard_D2a_v4`) has **no allowance at all** in this subscription ("remaining 0" is a limit of 0, not a holder), and the survey (`big-organism-detached`, mode survey, now listing every family's allowance and Azure's list prices) shows the same for the ESv4 and EAv4 families (`Standard_E2s_v4`, `Standard_E2as_v4`). Three of the eight pools could never be built. The fleet is rebuilt from families the survey shows allowed (10 vCPUs each) and offered in eastus: `Standard_D2s_v4:4, Standard_D2as_v4:1, Standard_D2_v4:5, Standard_D2ds_v4:5, Standard_D2d_v4:5, Standard_D2s_v3:5, Standard_D2as_v7:5, Standard_D2ads_v7:5, Standard_D2als_v7:5, Standard_D2alds_v7:5`: **45 workers** of ten families (one machine 2.2% of the fleet); `hpa_max` 405; the steps scaled by 45/39, steady `21 40 61 21 40 21`, burst `13 40 13 61 13 40`. List prices from Azure's own price API (Linux, pay as you go, eastus, USD a machine-hour): D2s_v4 0.096, D2as_v4 0.096, D2_v4 0.096, D2ds_v4 0.113, D2d_v4 0.113, D2s_v3 0.096, D2as_v7 0.0908, D2ads_v7 0.114, D2als_v7 0.0804, D2alds_v7 0.0952 (the 0.086 used for D2as_v4 in the refused dispatches was wrong; no bill was ever made with it). `scripts/aks_paired.sh` now checks every pool's size and family allowance before anything is built, so a refusal costs a minute, not a repetition. Nothing else changes.
- **Amendment 3 (2026-10-07 16:21 UTC, before the counted run).** The pre-flight of the 45-worker dispatch (run 37602951929) found `Standard_D2s_v3` not offered to this subscription in eastus and stopped every repetition in a minute, nothing built, nothing billed; the nine other pools passed. That pool is dropped: **40 workers** of nine families (one machine 2.5% of the fleet), `hpa_max` 360, the steps scaled

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

by 40/39, steady `18 36 54 18 36 18`, burst `11 36 11 54 11 36`, the pools' prices as in amendment 2. Nothing else changes.

- **Amendment 4 (2026-10-07 20:25 UTC, before the counted run).** Two dispatches of the 40-worker fleet (runs 37651333302 and 37676237815) were refused by Azure itself on every repetition for four hours: "creating a new cluster is unavailable at this time in region eastus" (`AKSCapacityHeavyUsage`, Azure's own capacity for new clusters, not the subscription's allowance; the pre-flight passed each time, no arm ran, nothing was billed). Microsoft's remedy is another region, other cluster settings, or a retry; this subscription has more than 10 cores only in eastus. The clusters are therefore made on the **standard control-plane tier** (`tier` input, `AKS_TIER`) instead of the free tier, about $0.10 an hour a cluster, which is not in the bill (the bill counts worker machines only, in both arms alike) and changes nothing the workers or the autoscaler do. If the standard tier is refused too, the run waits for Azure. Nothing else changes.
- **Amendment 5 (2026-10-08 03:55 UTC, before the counted run).** The standard-tier dispatch (run 37703460186) was refused by Azure's own cluster capacity in eastus on all five repetitions too, over three hours (23:39 to 02:48 UTC), the same `AKSCapacityHeavyUsage` on each: 22 refusals in eleven hours across four dispatches, the pre-flight passing each time, no arm run, nothing billed but the resource groups' minutes. The fleet is dispatched again unchanged; it runs when the region admits a cluster. The alternative, a second region, needs the subscription's regional vCPU allowance raised there, which only the account holder can request in the portal. Nothing else changes.
- **Amendment 6 (2026-10-08 05:00 UTC, before the counted run).** The fifth dispatch (run 37724667075) was refused on its first repetition after a forty-minute wait, the same `AKSCapacityHeavyUsage` in eastus, the twenty-third refusal; the pre-flight passed, no arm ran. We cancelled the run ourselves at that point to weigh a second region, and the answer is in the 6 October survey already on record (`big-organism-detached`, mode survey): eastus allows this subscription 32 vCPUs and the v4 machine sizes, and every other region allows 10 vCPUs in all, which is five two-core workers, not a fleet of forty. No other region can take this fleet until the account holder raises its allowance there. The fleet is therefore dispatched a sixth time in eastus, unchanged in every input, and runs when the region admits a cluster. The cancellation and the reason are recorded here so the count of dispatches reads right. Nothing else changes.
- **Amendment 7 (2026-10-08 08:30 UTC, before the counted run).** The sixth dispatch (run 37730249970) was refused on all five repetitions too, the same `AKSCapacityHeavyUsage` in eastus, over three and a half hours (05:01 to 08:26 UTC): 29 refusals in sixteen hours across six dispatches, the pre-flight passing each time, no arm run, nothing billed. The fleet is dispatched a seventh time, unchanged; nothing else changes. Each refused repetition costs GitHub runner minutes only.
- **Amendment 8 (2026-10-08 11:50 UTC, before the counted run).** The seventh dispatch (run 37750047768, 08:28 to 11:33 UTC) ran no arm either, but not for one reason. Repetitions 1 to 3 were refused as before (`AKSCapacityHeavyUsage` in eastus, the pre-flight passing each time): 32 refusals across seven dispatches. Repetitions 4 and 5 never reached Azure's cluster service: the pre-flight itself found only 2 of the 10 vCPUs free in two of the nine families (`standardDSv4Family`,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`StandardDasv7Family`), and the fleet is dispatched only when every family has room, so it stopped with nothing built and nothing billed. An inventory of the subscription (`.github/workflows/azure-inventory.yml`, run 37773586107, 11:58 UTC) answered it: the subscription holds exactly four machines, all ours and all running, and no cluster, scale set or group left behind by any earlier repetition. The holders are the three machines of the 24-hour robustness run (`docs/ROBUSTNESS_PREREGISTRATION.md`, scenario 2b): the detached start job, which tries the size it is given and falls back through the eight-core sizes it may rent when the allowance refuses it (the four-core size asked for was refused because the tower's machine held its family's allowance), rented `Standard_D8as_v7`, `Standard_D8s_v7` and `Standard_D8s_v4`, eight vCPUs in each of the Dasv7, Dsv7 and DSv4 families, where the allowance is 10, and the fleet's pre-flight (which asks every one of its nine families for 10 free vCPUs) found 2. The tower's `Standard_D8as_v4` holds 8 of the DASv4 family's 10 the same way. Nothing is deleted. The eighth dispatch, unchanged in every input, follows the collect that deletes the 24-hour machines (each machine runs two arms of 86,400 s one after the other, so about 2026-10-10 12:00 UTC, and by their 60-hour limit at the latest); until then the fleet cannot pass its own pre-flight, and that is the pre-flight doing its job.

66.36 The grid with the real cluster inside: protocol 2 (amendment, 2026-10-12, before the counted runs)

The grid's runs with the real cluster inside (workflow `six-kube`, every organism at 1, 10, 100 and 1,000 copies, every size to 1,000 repetitions; `scripts/dispatch_grid.py`) began on the max at 06:56 UTC. By 15:00 UTC three causes had marked repetitions invalid or ended jobs, all in the test rig and none in the engine (omni-v5, unchanged):

1. **The governor below the rig.** The four stacked at 100 copies, repetition 3 (job 114726933444): the governor made 47 of its 60 decisions by the end of its 3,600 s window, one every 78 s against its cadence of one a minute, and the run was invalid by the rule above (80%). The governor runs at the lowest CPU priority (nice 19) since live set 11, so that its own work never takes the service's place; the modelled organism, the heaviest process of the test rig on the runner (186,800 muscles), ran at normal priority (nice 0) in the same cgroup, where niceness ranks processes against each other: the rig outranked the governor and took its share. A second fault hid it: the bench's `wait` for the governor could not see it (the governor is started in a subshell: "wait: pid ... is not a child of this shell"), so the decisions were counted while a late governor was still deciding, and the job's own cleanup ended it.
2. **A refused download of kind.** `helm/kind-action` got no checksum file from GitHub ("no properly formatted checksum lines found"): the job ended before either arm ran.
3. **A refused download of the metrics-server manifest** (HTTP 504): the arm was marked invalid before its cluster was used; the other arm ran.

The amendment, protocol 2, carried in each run's name (`... window <s> protocol 2`):

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- With the organism inside, the governor runs at the organism's own priority (nice 0): no part of the test rig outranks it. Without the organism it stays at nice 19, as before. The service is not touched either way: it runs inside the kind nodes' containers, each its own cgroup, where a process's niceness on the runner does not reach.
- The decisions that count for validity are those inside the measured window (`window_start.txt` to `window_end.txt`), with the mean time between them shown; before, every decision record up to the reset counted, the kill switch's own included. The 80% rule stands. This rule, the governor's stop below and the downloads tried again hold for every run of `scripts/kind_bench.sh` from the commit carrying this amendment (the live sets, the add-on and robustness tests too); the governor's priority changes only where the organism runs beside it.
- A governor still running when the window has ended is stopped through the master switch (it hands every setting back and exits) and watched by its pid until it is gone, before any check reads the cluster.
- kind v0.31.0 (whose own node image is Kubernetes v1.35.0) and kubectl v1.35.0 are pinned at the versions every run so far used; a refused download is tried again with pauses and checked against its published SHA-256 (`scripts/kind_install.sh`); the metrics-server manifest likewise, once per machine, its pinned SHA-256 checked as before; a kind cluster that fails to come up is tried again twice. The rented machines install the same pinned versions (before: whatever was newest).

Counting. Every square with the cluster inside is filled on protocol 2 alone. The runs of protocol 1 (before this amendment: 100 repetitions of every organism at 1 and at 10 copies, 10 at 100 and at 1,000, and the first ten repetitions of the tower and of the four stacked at 1,000 copies on rented machines) are kept as they stand, archived and shown beside as a check, never pooled with protocol 2. A repetition whose job fails is started again, up to three times, and is paired within one run (native and Omni from the same run's files, `tools/six_kube_report.py`); one that fails three times is listed, never dropped. The modelled grid (`six`) is not touched by this amendment. Cost: GitHub's machines are free for this public repository (about 1,340 jobs run again, some seven hours of its queue); the first ten repetitions of the two rented squares run again cost about $110 at list price, within the founder's budget.

The founder's order, 12 October evening: 1, 10 and 100 only. The grid with the real cluster inside is every organism at 1, 10 and 100 copies, each to 100 repetitions; no 1,000 copies and no 1,000 repetitions, to cut the time and the bill. Every square runs on GitHub's own machines; the rented machines of the 1,000-copy squares are stopped and their repetitions (protocol 1) are kept as recorded, counted in no square. Protocol 2 and the counting rules above stand.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.37 The grid with the real cluster inside: protocol 3, and the live controller's ranges read live (amendment, 2026-10-13, before the counted runs)

What the card version's grid showed. On the card version (omni-v5, digest `5648f290`, protocol 2) native's own numbers show the test machine running out as the copies grew: GitHub's four-core machine busy 69 to 75% at 1 copy, 82 to 85% at 10 and 87 to 91% at 100; native's own failed requests 6 to 10%, 22 to 31% and 32 to 43%; the cluster's CPU in Omni-Compass's arm 0.7 to 1.7%, 1.2 to 2.5% and 4.0 to 7.0% under native's; and at 100 copies the four stacked off the window's clock in 49 of 100 repetitions, the tower in 1 of 100. The cause is the test rig, not the engine: the model of the organism (one process, at most one full core) and, in Omni-Compass's arm, Omni-Compass's math for every modelled muscle ran on the same four cores as the seven kind nodes, so the heavier the model, the less CPU the real cluster got, and more of it in Omni-Compass's arm, whose model also carries the math. The 100-copy squares measured the machine, not the governor; they stay as recorded and count in no square of the next grid.

Protocol 3, carried in each run's name (`... window <s> protocol 3`):

- **The rig's own cores.** With the organism inside on kind, the model runs on the runner's last core (`taskset`) and the kind nodes on the others (`docker update --cpuset-cpus`), the same in both arms; the placement is written into each arm's preflight (`scripts/kind_bench.sh`). The governor keeps the organism's own priority (protocol 2) and is not placed. The cluster then has the same cores at every size and in both arms, and the model's weight shows only on its own clock.
- **The window from a measured step.** Before either arm, the heavier arm (Omni-Compass's: the model with the math for every modelled muscle) is timed on the core it will run on, six steps (`tools/run_kil.py --calibrate`), and the window is lengthened, never shortened, to 240 steps of its slowest measured step with a quarter to spare, rounded up to the minute (`scripts/kind_paired.sh`). Both arms use the one window; the measurement is written into each arm's preflight. The conductor's windows (`scripts/dispatch_grid.py`) stay the least a square is given.

The live controller's ranges, read live (Omni v5 amendment 5, digest `6093b2b88aa911b9`; `docs/OMNI_V5.md`): the machine lever's top is the operator's `--max-nodes` where given, else every ready worker plus what the cluster's own autoscaler may still add, else (where Omni-Compass resizes the pool with the operator's command) one under the first count the pool refuses; cruise (rule 7) climbs toward that top one step a decision; the replica room (rule 5) is on by default, its ceiling what the machines can hold of the service's pods, one pod under it, read live (`--replica-ceiling N` the operator's own, 0 keeps the cap unmoved); at the max the power cap is never set under native (the capped pods get their machine's idle CPU, the frequency ceiling and the GPU limit stay at their top). The fourth amendment's text above stands as the record of what ran before.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Counting. Every square with the cluster inside is filled again on protocol 3 and the new engine alone: six organisms at 1, 10 and 100 copies, each to 100 repetitions, on GitHub's own machines (free for this public repository); the runs of protocols 1 and 2 are kept as recorded, archived and shown in history, never pooled with protocol 3. The modelled grid (`six`) is run again on the new engine too (its numbers are expected to be identical: the modelled code is unchanged but for comments, headers and one label's name), because no result is read across engine versions.

Evaluation and Simulation use only. Copyright (c) 2026 The Omni-Compass LLC. Commercial use, commercialization or monetization of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. All rights reserved. See `LICENSE` *and* `NOTICE` *at the root of this repository. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

66.38 The burst back at `1 6 1 8 1 6` (the founder's order, 2026-10-14, before the counted runs)

The founder's order of 14 October: the Azure burst runs at the steps first preregistered, `1 6 1 8 1 6`. Amendment 11 (5 October) had set `1 3 1 4 1 3` after three repetitions at `1 6 1 8 1 6` were seen; that amendment was ours, written after the results, and it is withdrawn for every run from this date. The run on the card version's engine at `1 3 1 4 1 3` (run 38455393389) was stopped after its second repetition and part of its third; its repetitions stay in the record as runs of the withdrawn setting and are counted in no burst result. The burst starts again at `1 6 1 8 1 6` (run 38500160429): five repetitions, 1,800 measured seconds, four workers of Standard_D2_v4 in eastus under Azure's autoscaler, the bill metered every 15 s, native against omni, then B and C. Disclosed beside it: at these steps in October's first runs about 41% of requests failed in both arms, because the service's own autoscaler was pinned at its cap of 10 pods; the table shows what each arm served at the steps the founder set.

66.39 Two more burst shapes on Azure (the founder's words, 2026-10-14, written before any run of either)

The founder's words of 14 October named two more shapes for the Azure burst: the steps `1 4 1 6 1 9`, and bursts that nobody picks, from 1 to 9. Both run beside the burst at `1 6 1 8 1 6`, each reported as its own setting, never in its place. Neither arm reads a schedule: the load generator alone steps through it (`scripts/kind_bench.sh`, the load generator's replica count set at each step), identically in both arms, and the governor sees only what the cluster shows (its processor use, the requests waiting, the response times), as in every burst run.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Setting	Load-generator replicas per step	Steps	Measured window	Machines (queue)
Burst `1 4 1 6 1 9`	`1 4 1 6 1 9`	6 of 300 s	1,800 s	four workers of Standard_D2_v4, eastus (after the `1 6 1 8 1 6` set, in its queue)
The real day on 1 to 9	`2 3 3 4 3 4 4 3 2 2 2 2 2 3 2 3 4 5 4 5 6 7 6 5`	24 of 108 s	2,592 s	four workers of Standard_D2s_v4, eastus (after the steady set, in its queue)

- **The real day on 1 to 9.** The 24 hourly counts of submitted jobs on the first day of Google's public 2011 cluster trace, the counts already derived for the trace test (`results/traces/google2011/schedule.json`, every part named with its SHA-256), placed on 1 to 9 by the same min-max rule with half-up rounding (`python3 tools/trace_schedule.py --out results/traces/google2011_1to9 --load-max 9 --counts-from results/traces/google2011`; receipt `results/traces/google2011_1to9/schedule.json`), then stepped by the founder's rule for real traffic, one step a bin toward the trace's level, never skipping. The demand is the trace's, not ours. Disclosed: the day touches 9 for one hour only (hour 21), and under the one-step rule the schedule climbs to 7 at most before the trace falls again; the schedule is run as derived, not stretched to reach 9.
- **Everything else as the burst bill test**: Azure Kubernetes Service with its own cluster autoscaler, a work pool of 1 to 4 machines, a fresh cluster for each arm, five paired repetitions with the arms' order rotated, the control plane on the standard tier, the machines Azure bills counted every 15 s at the list price ($0.096 an hour for either size), every gauge reported; native is Azure's autoscaler and the HPA alone, omni is Omni-Compass on top on the engine that stands, its replica room read live (Omni v5 amendment 5).
- **Reading.** Three separate runs of each setting, A, B and C (`tools/confirm_abc.py`); each row GAIN or LOSS in all three of our runs, EVEN (no difference beyond the noise) or the runs disagree. Replicas, machines and the bill that rise are power given, shown beside the response times, the time over the line and the failed requests they bought.
- **Order and dispatch.** The grid conductor starts every Azure set from `scripts/azure_queue.txt`: A of every set, then B, then C, one at a time in each machine family's queue, read back by its run's title, never started twice.
- **Amendment of 16 October (the three clouds).** Google Cloud and Amazon carry the same four sets, at the same settings and on the same harness, from `scripts/google_queue.txt` and `scripts/amazon_queue.txt` (the workflows `gke-metered.yml` and `eks-metered.yml`, one set at a time per machine-family queue, read back by title as on Azure), each behind its own `switch=` line, off until that cloud's secrets are in the repository and the founder turns it on; a run without its secrets stops before anything is built or billed. The Azure run is the measure of the other two: nothing

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

in a set differs between clouds but the machine type, the zone or region, and the list price. (18 October: both switched on by the founder, the secrets in the repository; nothing in a set changed.)

- **Disclosed.** At `1 6 1 8 1 6` in October's first runs about 41% of requests failed in both arms with the autoscaler pinned at its cap of 10 pods; a peak of 9 asks more of the four workers than 8 did. What each arm served at each shape is in its table. Cost: about $4 to $6 a set of five at list price; three sets of each shape, about $30 for both. Time: about nine hours a set.

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

66.40 In the cloud, the bill at native's work (the founder's order, 2026-10-14, night, before any run of it)

The founder's order of 14 October: in the cloud Omni-Compass is to show one thing, a smaller bill for the same work as native; the benefit of the max belongs where Omni-Compass sits in the engine before the cloud or in the chip at its other end, not in the cloud's bill. The first Azure runs on the engine that stands were at the max and showed the opposite of what the cloud needs: on the steady load, three repetitions of run 38455383137 raised the bill 49% at list price (machines in service 2.9 against 1.8, replicas 19 against 9.5) to buy faster replies and fewer failed requests. From this order every Azure set runs as follows, and nothing else changes:

- **Omni-Compass on top at the tune-down** (`dial save`, `OMNI_DIAL=save`): native's work on the least resource. Machines are given back only where the paired trial on the machines shows the response time no more than 2% slower (the verdict), through the release gate, one at a time; a breach puts one back at once.
- **Native's own replica cap** (`replica_ceiling 0`): Omni-Compass never raises the app's replica ceiling above the operator's. Inside that cap its HPA target still moves between 60% of the operator's target and the operator's own, so at a moment it can hold more pods than native's HPA would, and Azure's autoscaler may then add a machine: in a burst the bill can rise, and where it does the row reads power given and the setting is reported as failing its aim, never hidden. A setting in which the governor in a cloud only gives back (its HPA target never below the operator's) is a change to the engine and is proposed for the next version, not made here.
- **The aim, and the readings.** The bill (machine-hours at list price) and the machines in service aim lower: a cut is GAIN. Work served, response times, time over the line and failed requests must read EVEN or GAIN: a LOSS there is a LOSS of the test, its cause found and fixed. A rise in the bill is power given and is reported as the setting failing its aim.
- **Disclosed before the run.** Where native is short (at its cap and failing requests, as on the steady load), there is nothing to give back without costing work, and the bill is expected to read EVEN; the cuts are expected where native holds machines it does not need, in the quiet stretches between bursts and in the hours of a day's trough.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The runs of the max.** Steady A (run 38455383137: three repetitions finished at the max) and the burst at `1 6 1 8 1 6` (run 38500160429, its first repetition) are stopped and kept as runs of the max, counted in no bill result. Every Azure set starts again at the tune-down from `scripts/azure_queue.txt` (its run titles carry `dial save`), A of every set, then B, then C: the steady load, `1 4 1 6 1 9` (the founder's), `1 6 1 8 1 6`, and the real day on 1 to 9.

66.41 The babysitting tax: three rows in the receipt (the founder's order, 2026-10-14, night, before any run at the tune-down)

The founder's order of 14 October: show that the governor is an autopilot, self-running and self-steering like a self-driving car, acting ahead, so that nobody turns a knob and nobody has to react. Three rows join every Kubernetes and cloud receipt (`tools/live_reps.py`, `babysitting`), written here before any run at the tune-down; nothing else in the receipt or the readings changes.

- **Alert minutes**: the whole minutes of the measured window whose 95th percentile response time (nearest rank, a failed request counted over the line) was over the line, the minutes an on-call page would be open. Measured from the probe's own records, in both arms; judged, the aim lower (time missed: a fall is GAIN).
- **Settings commands the governor wrote on its own**: the writes in the governor's audit inside the window (a dry run is none). Native and the watching arm write nothing and read 0. Shown, not judged.
- **The governor's additions decided while the service was still inside its line**: of its decisions inside the window that add (a machine more than it saw, or a lower HPA target than its last), the share made while the 95th percentile it read was at or under the line. Shown, not judged.

No person touches a setting in either arm of any run; that is stated in every receipt beside the rows.

66.42 The decision period in the cloud, and two arms only (the founder's orders, 2026-10-14, night, before any run at the tune-down)

The founder's orders of 14 October: the governor is to move faster than every native controller under it (15 seconds, then 10, then 5 seconds a decision were asked the same night), and a change is made only where it is known to hurt nothing anywhere. Read against the governor before any run, a faster decision period is not a free setting today: four of its rules count in decisions, not in seconds, and every one of them would change with the period:

Rule (counted in decisions)	At 60 s (as built and tested)	At 15 s (native's own check)	At 5 s
a trial of giving one machine back (`max_trial`, 20 decisions a phase, written	20 minutes a phase, time to measure the 200 response times it needs	5 minutes, barely the time those 200 take at the probe's rate	100 seconds, while the probe gives at most one response time a second: every trial would be

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Rule (counted in decisions)	At 60 s (as built and tested)	At 15 s (native's own check)	At 5 s
in the engine)			abandoned for too little work, no machine would ever be given back, and the bill could not fall
the cool-down after a breach (3 decisions, written in the engine)	nothing given back for 3 minutes	45 seconds	15 seconds
cruise (`--cruise-after`, 2 decisions)	work waiting for 2 minutes	30 seconds	10 seconds
the spacing of trials and the wait after a refusal (`--verdict-every` 10, `--verdict-recheck` 120)	10 minutes; 2 hours	150 seconds; 30 minutes	50 seconds; 10 minutes

Two of the four are written in the engine and have no setting, so no run can keep them at their tested seconds at any period but 60: at native's own 15 seconds a machine would be given back after a trial a quarter as long, and given back again 45 seconds after a breach instead of 3 minutes, which is the risk of a person woken that the cloud exists to rule out. Native's own checks are never slowed by any of this: the HPA keeps its 15-second check and Azure's autoscaler its 10 seconds underneath, whatever Omni-Compass's period.

So every Azure set runs at the period the engine was built and tested at: the full decision every 60 seconds, the fast checks (the scheduling floor, the pod reflex, conveyance on every third, run between decisions) every 5 seconds, three times as fast as the HPA's own 15-second check. The rig accepts other periods (`OMNI_INTERVAL`, `OMNI_FLOOR_INTERVAL`, passed as `--interval` and `--floor-interval`), and a window in which the governor made no write needs at least one decision a minute to count, whatever the period. The faster decision belongs to the next engine version, with every rule above counted in seconds and the fast checks on their own clock, and is used only once a test shows it reads no LOSS on any gauge anywhere; until then the order holds as the founder put it: native's speed at least, faster only where it hurts nothing. The cloud's arms are native and Omni-Compass on top, nothing else (`ARMS` defaults to `native compass` in the workflow and in every cloud script); the steady run of the max had run those two.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.43 In the cloud, the machines alone: a bill that can only fall or stay (the founder's order, 2026-10-14, night, before any run at the tune-down)

The founder's order of 14 October: in the cloud Omni-Compass must never make the bill bigger. Read against the governor, the tune-down with native's replica cap still allowed it in a burst: the pod target moved down to 60% of the operator's, more pods than native's asked for machines, and Azure made them. From this order every Azure set runs the governor on the machines alone (`OMNI_MACHINES_ONLY=1` in `aks-metered.yml`; `scripts/kind_bench.sh`):

- **No pod autoscaler is moved.** The governor is told it senses no service (`--sensed none/none`), so every HPA keeps the operator's own target, and the replica room stays shut; the pods are exactly native's.
- **No processor is conveyed and no power is capped** (no `--cap-deployments`, no `--power-cmd`): in a cloud neither buys a smaller bill, and a pod given more processor would read busier to the HPA and ask for more pods.
- **The machines.** The governor's machine lever only parks a machine that Azure already made and wakes it again (`scripts/kind_nodepool.sh` never asks for more machines than the cluster has, and never tells Azure to make one). A machine is parked only after its own trial shows the response time holds within 2% (the verdict, through the release gate, one at a time), and a breach wakes it at once; a parked machine with no work is deleted by Azure's own autoscaler. Azure's autoscaler alone makes machines, for native's own pods.

So the omni arm can hold no more pods and no more machines than native's autoscalers give it: the bill can only fall or stay. The bill and the machines aim lower (a cut is GAIN); work, response time, failures and alert minutes must read EVEN or GAIN. The receipt's commands row shows how often a machine was parked or woken.

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

66.44 Amendment (2026-10-16): the spine, the seventh organism with the cluster inside

The founder's order of 16 October: the spine (the 267 catalog muscles in all four realms, the base every stack runs on; `docs/THE_SPINE.md`) is an organism in its own right. With the real cluster inside it runs as `tools/run_kil.py --organism spine`, on the same protocol (3), the same window rule, the same two arms and the same gauges as the other six; the grid conductor starts its squares, reads them back and never starts one twice, as it does for every organism, and the workflows `six-kube`, `aks-metered`, `gke-metered`, `eks-metered` and `cloud-organism` accept `spine`. The results already written on six organisms stand as they were run; nothing is read across engines or across organism sets.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.45 Amendment (2026-10-18): a window with no machine in service is refused in either arm; the add-on's set-up waits for its scaler

Two repetitions of the omni-v10 reruns failed for causes outside Omni-Compass, and each is now caught in words that say so. Neither change touches the engine, a setting, a gauge or a scored window.

- **No machine in service.** In the CruiseKube test (run 39120444653, repetition 8) every worker of the kind cluster left service for two minutes (eight of the capture's readings show no machine in service), while the service answered every probe. The omni arm's governor wrote nothing, and two of its fifteen decisions failed on that empty reading. The rig refused the window, correctly, but its words said the governor was not running. A reading with no machine in service leaves the window's machine count wrong in either arm, so such a window is now refused in both arms (`scripts/kind_bench.sh`). The capture records each node's taints and conditions at that moment (`fleet/capture/kube_capture.sh`), and a decision that failed is named by its error. The repetition runs again in place.
- **The add-on's scaler not yet answering.** In the KEDA test (run 39108180989, profile both, repetition 2), the native arm's set-up stopped before its window. KEDA's first call to the HTTP add-on's scaler met a pod still stopping after its rollout, and KEDA built the autoscaler without its HTTP metric within five minutes. The ScaledObject is now applied only once no KEDA pod is stopping and the scaler has an address. While the autoscaler lacks its metrics, the ScaledObject is touched once a minute with an annotation of the harness (never a setting), so KEDA builds the autoscaler again from the same spec (`scripts/kind_addons.sh`). The repetition runs again in place.

66.46 Amendment (2026-10-18, omni-v11): the governor holds on a reading with no machine in service

omni-v11 (`docs/OMNI_V11.md`) changes one thing in the governor these tests run. When the cluster's reading shows no machine in service, the decision is recorded as `held: the cluster's reading shows no machine in service; nothing decided, nothing written`. Nothing is decided from that reading, nothing is written, and the floor check returns at once. omni-v10 read the same reading as the highest overload and then divided by the empty capacity, so its decision failed and wrote nothing. The rig counts a held decision as the governor running (it is a decision record), never among the decisions of a scored window (it is no decision of the law). A window in which the capture saw no machine in service is refused in either arm (the amendment above), so no scored window carries a held decision that the capture also saw. Every setting, gauge, load, window and pair is unchanged; every test runs again on omni-v11.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.47 Amendment (2026-10-18): on kind, the served image pulled onto every worker before each arm

The KEDA add-on test's mean pod start wait read LOSS on omni-v10 under the http setup. The cause is the test machine (`docs/POD_START_WAIT.md`, the receipt `results/live/POD_START_PHASES.md`, 741 starts in five runs). A serving pod placed on a node that never served downloaded and unpacked its image (`registry.k8s.io/hpa-example`, untagged, pull policy Always) inside the window, on the one runner every kind node shares, and the unpacking took longer the busier that runner was. On a node already serving, a start took 1.3 to 1.7 s in both arms. On an empty node it took 7 to 12 s. Split exactly, the omni arm's rise in the mean (+1.42 s on omni-v10, +0.65 s on omni-v11) is the shift of its starts into a busier host (+1.57 s, +1.64 s), the host its conveyed processor keeps busy. At the same host load on the same kind of node its starts were as fast or faster (−0.14 s, −0.98 s).

From this amendment on, every kind arm runs an image pre-puller (`deploy/kind/prepull.yaml`) before the governor starts or the window opens. It is one pod on every worker, waited for on every worker and then deleted. A pre-puller that does not finish fails the arm's checks. Each arm's `preflight.txt` records `images=pre-pulled on every worker before the arm` and the digest each worker pulled. The rest stays as written: the operator's manifests, every setting, gauge, load, window and pair, the engine (omni-v11's digest), and the cloud sets, where every node is its own machine. The kind workflows' three runs on omni-v11 count only when their own commit carries the pre-puller (`RIG_FILE`, `scripts/dispatch_grid.py`). Their runs on the rig before end as their own record, and `tools/confirm_abc.py` marks any table whose runs differ in rig. In the grid with the real cluster inside this rig is protocol 4 (`KUBE_PROTOCOL`, carried in each run's name), and only its runs fill a square; no omni-v11 run with the cluster inside had started on protocol 3. Expected before any run: a pod on an empty node starts in about the time one beside a serving pod takes, in both arms. Whatever the runs read is reported.

66.48 Amendment (2026-10-18): the extended fault test, the two faults that blind a governor (written before its run)

The fault test above meets four faults. None of them takes away what a governor reads or the path it writes through, and a governor's worst hour is the one in which it cannot see. The extended test (`FAULTS=2`, `scripts/kind_faults.sh`) keeps the four at their moments and adds two after them, the same in every arm:

5. at 82% of the run, the API server is frozen for 30 s: its process in the control-plane node is stopped, then continued. The service's own path (kube-proxy on every node) keeps carrying traffic; no controller, native's or Omni-Compass's, can read the cluster or write to it.
6. at 88%, the metrics server is scaled to none for two minutes, then back to one: every autoscaler reads no CPU metric.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Each outage is written to `faults.log` only once it took hold; one that did not take hold is written as failed, and the report has no row for it. The report (`tools/live_reps.py`, "The fault test") reads the two like the four: the time to recover from the fault's start until responses stay under the line for 30 s straight, at most 300 s, and the share of samples over the line or failed in the 300 s after it, paired against native. The run's window is 2,500 s, so the second outage's 300 s end inside it (12% of 2,500 s is 300 s); the four keep their shares of the window, so they fall later in seconds than in the 900-second test, and its tables are not read beside this one's. Load (open, fixed rate), arms (native; native with Omni-Compass on top), the engine (omni-v11's digest), the rig (the pre-puller above), the gauges and the ten pairs are as written; three runs, A, B and C (`scripts/reruns.txt`, "kubernetes faults extended").

Two readings are named before the run. While the API server is frozen, the capture's readings are not answered and are skipped (`fleet/capture/kube_capture.sh`), in both arms. While the metrics server is gone, the capture reads no CPU used: the column records 0 for those two minutes in both arms, so the CPU row's change is diluted by that share of the window and never reversed by it. Expected before any run: Omni-Compass on top holds every setting while it cannot read (omni-v11 holds on a blind reading; a decision whose read fails writes nothing, and its lease runs 300 s), so it recovers from both as fast as native or faster. Whatever the runs read is reported.

66.49 Amendment (2026-10-19): a paid cloud set runs to its cap and stops when its bill is read (written before any run of it)

The founder's orders of 19 October: every cloud set gets a three-run reading, A, B and C on one engine, measured as finely as the cap allows ("the cap we can run it to ... that's all we can run"), at the least cost that still tells what Omni-Compass can do. Measured from the twelve Azure runs measured to the end (5 paired repetitions each), a run's bill swings about ±9% of native's (the median half-width of its 95% interval, 1.5% to 36% run to run), so a cut under about 9% read EVEN.

- **Repetitions.** Every Azure set (the steady set, the bursts `1 4 1 6 1 9` and `1 6 1 8 1 6`, the real day on 1 to 9) runs up to 100 paired repetitions a run, the founder's cap for paid runs (12 October). At 100 the bill's interval is about ±2%: a cut of about 3% reads GAIN eight times in ten.
- **The stopping rule** (`tools/stop_rule.py`, `aks-metered.yml` input `stop_looks`, set to `10,20,40,70`). At each look, on repetitions 1 to that look only, the bill's paired change is read with its 99% interval (four looks at 1% each keep the rule's error under 5%). The run stops at the first look where the interval clears zero (a cut: GAIN; a rise: power given), or holds zero within ±2% of native's bill (EVEN, measured to that precision). It never stops while its 95th percentile or its failed requests read a LOSS at that look: a LOSS is measured to the cap. A stopped run's remaining repetitions start, read the rule, and spend nothing on Azure. The table is read as every table is (95% intervals, `tools/live_reps.py`); the run's summary carries every look (`STOP_RULE.txt`).
- **Unchanged.** The arms (native, then Omni-Compass on top at the tune-down, the machines alone, native's replica cap), the sets' loads and windows, the machine family and region, the control plane's tier

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(standard: amendment 4's way past the free tier's capacity refusals; about $0.10 an hour a cluster, not in the bill), and the three-run rule.

- **Both arms at once, on twin clusters,** would remove most of the swing (today the arms run one after the other on fresh clusters, each meeting Azure at a different moment). It waits for Azure's processor allowance: one cluster (a system machine and four workers of `Standard_D2s_v4`) is the whole 10-vCPU allowance of that machine family in `eastus`, so twins need it raised to 20 (asked of Azure, never shrunk to fit). It is written here before any run of it and switched on only by a further amendment once the allowance is raised.
- **Counted apart.** A run under the rule carries "to the cap, stopping rule" in its title (`aks-metered.yml` run-name, `scripts/dispatch_grid.py` `cloud_title`), so the conductor counts its A, B and C from zero. The omni-v11 steady run at 5 repetitions (run 39162654399, finished 2026-10-19 01:46 UTC) and any set started before this landed stay as they ran, their own record; a set's three-run table pools only runs of one design and one engine.

66.50 Amendment (2026-10-19): a worker held by an add-on's own mark while it serves is in service

Three native repetitions of the omni-v11 CruiseKube test were refused for "no machine in service" while the service answered every probe: run 39171059299, repetitions 5 and 10, and run 39175670991, repetition 2. Each refused window's capture (`capture.csv.errors`) records the same cluster:

- every node Ready, the control plane included;
- every node marked `cruisekube.truefoundry.com/overloaded:NoSchedule`. CruiseKube 0.3.4 puts this mark on a node it judges overloaded, so the scheduler sends that node no new pod; the pods already on it keep running.

The recorder (`fleet/capture/kube_capture.sh`, `ACTIVE_ONLY=1`) took any NoSchedule mark as a machine out of service. It read 0 machines, and the window was refused under the amendment of 18 October. The add-on's own mark is a scheduling hint, not a machine leaving service. The recorder now counts a node in service when any one of these holds:

- it is open to new work;
- it is closed (cordoned, or marked idle by the governor) and still carries work that is not a DaemonSet's;
- it is Ready and carries such work, and its only marks are NoSchedule marks that are not Kubernetes' own.

Kubernetes' own marks still take a node out of service: `node.kubernetes.io/*` (unreachable, not ready, under pressure) and the control plane's role mark `node-role.kubernetes.io/*`. Any NoExecute mark does too, and so does a node that carries no such work. The power capture (`scripts/kind_power.sh`) already counted such a worker as active, so both captures now agree. `tests/test_capture_in_service.py` runs the recorder on the cluster as the refused repetitions saw

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

it: before this amendment it read (0, 0), now (2, 2). The test also checks what still takes a machine out of service, and that the governor's own marks read as before.

- **Unchanged.** The same rule holds in both arms. A window with truly no machine in service is still refused in either arm. Every setting, gauge, load, window and pair is unchanged, and no engine file changes.
- **Earlier runs.** Windows refused before this amendment stay refused. The CruiseKube tables of earlier runs stand as their own record. Where the add-on marked some workers, those runs' machine counts may read low in either arm: a stated threat to their validity. Repetition 8 of the omni-v10 run 39120444653 (the cause behind omni-v11) ran before the capture recorded marks, so its own files cannot say whether the same mark caused it; the same add-on was running.
- **The governor.** The governor (`omni_controller/controller.py`, an engine file of omni-v11) counts its machines by the recorder's old rule. Under the add-on's mark on every worker it reads no machine in service and holds; omni-v11 makes that hold safe, but the governor is blind there. Counting such a worker in service in the governor changes the engine. That is the next version's work, through the model gate, after the founder is told; this amendment does not touch it.
- **Run again.** The CruiseKube test runs again, A, B and C, on the corrected recorder (`scripts/reruns.txt`, block of 19 October). The three-run table pools only runs of one recorder.

66.51 Amendment (2026-10-22, omni-v16, written before any run of it): the full decision every 15 s

"The decision period in the cloud" above held every run at 60 s because four rules counted decisions. omni-v16 counts every clock of the governor in seconds (`docs/OMNI_V16.md`: the give-back trial, the cool-down, cruise and the trial spacing already counted seconds since omni-v7; now the brain's own minute, the compass's damping and smoothing, the edge's window and rates, the additions past the floor, a command's minute to land and the rest do too), so a period changes only how often the governor looks: at 15 s a give-back trial still lasts its 20 minutes, the wait after a breach its 3 minutes, and one machine is still given back at most once a minute. On the founder's word of 22 October ("15 sec all over heart beat"), the default decision period is 15 s, native's own pod autoscaler's pace, in every run from omni-v16's first on: the kind tests and the cloud sets alike (`scripts/kind_bench.sh`, `OMNI_INTERVAL`), the fast checks every 5 s as before. A window in which the governor made no write still needs its decisions to count, now one every 15 s. Runs before omni-v16 stand at the 60 s they ran at; nothing is read across engines.

66.52 Amendment (2026-10-22, omni-v17, written before any run of it): the main engine every 60 s, the beat a dial

On the founder's word of 22 October, afternoon, the heartbeat is a dial (`docs/OMNI_V17.md`): the main engine takes its full decision every 60 s, and any beat may be set (5, 15, 30, 60, 75, 90, 120 s and every value between), each shown beside native with what it buys and what it costs. From omni-v17's first run on, every run of this preregistration, the kind tests and the cloud sets alike, decides every 60 s by default

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(`scripts/kind_bench.sh`, `OMNI_INTERVAL`), the fast checks every 5 s as before; every clock counts seconds since omni-v16, so a give-back trial still lasts its 20 minutes, the wait after a breach its 3 minutes, and one machine is given back at most once a minute. The beats other than 60 s are read in their own runs (`docs/PACE_PREREGISTRATION.md`, amendment 3). Runs on omni-v16 stand at the 15 s they ran at; nothing is read across engines.

66.53 Amendment (2026-10-24, written before any run of it): Google's and Amazon's sets to the cap under the same stopping rule

On the founder's words of 24 October ("code it to wire it the same way that the Azure is running"), the Google and Amazon sets (`scripts/google_queue.txt`, `scripts/amazon_queue.txt`: the steady set, the bursts `1 4 1 6 1 9` and `1 6 1 8 1 6`, the real day on 1 to 9) run as Azure's do under the amendment of 19 October: up to 100 paired repetitions a run, the stopping rule's looks at 10, 20, 40 and 70 (`gke-metered.yml` and `eks-metered.yml` input `stop_looks`, `tools/stop_rule.py`, the same 99% intervals, never stopping on a LOSS), and "to the cap, stopping rule" in each run's title so their A, B and C are counted from zero. Everything else is as written for them: the arms, the loads, the windows, the machine families (`e2-standard-2` in `us-central1-a`; `m5.large` in `us-east-1`), the tune-down on the machines alone and the 60 s beat of the amendment above. Both queues are switched off and neither cloud has run a set on any engine since omni-v10's logins of 18 October; this is written before the first.

66.54 Amendment (25 October 2026): the ladder is 1, 10, 100 and 500

The founder's order of 25 October: the grid's top rung is 500 copies and 500 runs, in line with the Python's 500-run Monte Carlo (`tools/engine_runs.py`), replacing the 1,000 tier of 15 October. Every grid run started from this amendment on runs the ladder 1, 10, 100 and 500 (`scripts/dispatch_grid.py`); runs started before it stay as their own record. Paid cloud and GPU runs stay at 1, 10 and 100 copies with at most 100 repetitions.

66.55 Amendment (31 October 2026, written before any set at the max starts): every cloud set runs the whole engine at the max; the machines alone of 14 October struck

The founder's words of 31 October ("Why is there any hard rule written ... Whatever the math does and falls out is whatever happens. All we need to do is tune up what the math does to the best way we can and let whatever falls out fall out. And when it falls out wrong, the tuning is wrong"; "What I care is to make the cloud the best it can be with Omni-Compass in it, as well as on site with Omni-Compass on it, and we have to sit and tune and tweak both scenarios"). The section "In the cloud, the machines alone" above is struck from this amendment on: no set started after it runs `OMNI_MACHINES_ONLY=1`, `dial save` or the governor's `--give-back-only` (the switch is gone from the rig and the workflows; the flag leaves the governor with the next engine version, and nothing passes it). Every cloud set runs Omni-Compass on top at the max, every

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

muscle: the service sensed and its pod target moved inside the autoscaler's cover, the processor conveyed, the declared power model read, the replica room read live as on kind (`replica_ceiling` empty), and the machine lever parking and waking machines the cloud's own autoscaler made; the autoscaler, native's machine controller, stays in every arm and alone adds machines (`scripts/kind_nodepool.sh` asks the cloud for none). The bill (machine-hours at list price) is read by the one key beside work, response time, failures and alert minutes, and the cloud's judged number is the work per dollar: the work answered within the line for each dollar of the bill. The bill may rise where the work rises more, a rise read as power given beside the work it bought; a tune whose work per dollar falls against native is not admissible (`docs/THE_TUNE.md`), a fault of the tune, root-caused in the model first (`tools/model_gate.py`, whose cloud setting is the max from this amendment); the founder's order of 18 October, never raise its bill, is read the same way, by judging, never by tying the engine's hands. Every set at the max is its own A, B and C under the stopping rule of 19 October and the cap of 12 October (100 paired repetitions; 1, 10 and 100 copies); the sets started before this amendment (omni-v18's steady set; omni-v21's steady set, 25 of 100 repetitions when this was written) finish as the tune-down's record and are read in no table with a set at the max. Google's and Amazon's queues carry the same change behind their switches.

The same evening, before any set at the max started (the founder's words: "We never make the bill smaller ... they're looking to do more work with the money they're spending"; and then, "We're pushing 1% more bill, but we're getting ~12% more work ... anybody will pay 1% more for 10% more product"): the cloud is judged by its work per dollar, and these sets cannot read it as built. Each sends a fixed rate of requests (`LOADGEN=open`), so its work is the same in both arms by design and a set at the max could show only the bill beside a faster answer; the factory beat's ladder on omni-v23 reads the cloud setting at the max with the machine-hours up 9% to 35% across the beats, 14% at 60 s, on a plant whose work is fixed the same way (`results/model_gate/V23_LADDER.md`, our own models, class S). Azure's queue is held (`scripts/azure_queue.txt`, `switch=off`) until its sets carry the load whose work grows with the speed (the work meter, `LOADGEN=meter`, `tools/capacity_report.py`, the completed work counted in each arm), written into this preregistration before the switch goes on; then its sets start as written here, on the engine that stands then.

66.56 Amendment (31 October 2026, night, written before any set with the work meter runs): the cloud's sets read the work per dollar

The founder's words of 31 October, evening: "We're pushing 1% more bill, but we're getting ~12% more work ... anybody will pay 1% more for 10% more product". The cloud is judged by its work per dollar: the work answered within the line for each dollar of the bill, both from the same run.

- **The load.** Every cloud set runs the work meter (`LOADGEN=meter`, `deploy/kind/loadgen-meter.yaml`): one open-loop generator on the system pool's machine, off the measured workers, stepping through rising rates and counting every answer (inside the line, late, failed, not sent), the same rates in both arms; its warm-up requests are not counted. The meter was kind's (`docs/CAPACITY_PREREGISTRATION.md`); `scripts/kind_bench.sh` now runs it in a cloud

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

as well, and every cloud workflow takes the load (`loadgen`) and the meter's rates (`rate_steps`) as inputs and names a meter set by its rates.

- **The readings.** Beside the bill (machine-hours at list price), the receipt reads the work inside the line a second and the work per dollar of the bill (`tools/live_reps.py`), each run's paired difference and its interval, and over A, B and C the three readings of `tools/confirm_abc.py`: the work aims higher (a rise GAIN, a fall LOSS); the work per dollar is work bought with a resource (a rise GAIN, a fall power given); the bill a resource spent (a rise power given, never a LOSS by itself). Every other gauge is read as in every set, losses included.
- **The rates, frozen on one tuning case** (as the capacity test's, its amendment 1): before the counted set, one tuning case at the max on the steady set's machines, one paired repetition, the rates 10 20 40 60 80 100 120 140 a second over 1,200 s, never counted, shows where native and Omni-Compass saturate on Azure's machines (`scripts/azure_queue.txt`, its first line, run once: `runs=A`). The counted rates are then written here as an amendment, from below what native serves inside the line to above what Omni-Compass serves, before the counted set's run A.
- **The counted set.** A, B and C, each up to 100 paired repetitions under the stopping rule of 19 October, at the max, on the engine that stands when each run starts; its looks read the work per dollar.
- **Order.** Azure's queue is switched on again with this amendment, the tuning case first; it starts once the queue's machine family is free (the omni-v21 steady set already running finishes first). The four sets of the open load wait behind the meter's, each on the founder's word. Google's and Amazon's queues carry the same change behind their switches.

66.57 Amendment (31 October 2026, night, written before any counted set with the work meter runs): a cloud set must show that Omni-Compass holds

On the founder's words of 31 October, night ("We don't have to show benefit. We know that. We just need to show we hold, and then we know where the benefit lives, and we can explain"), the claim every cloud set of this preregistration is read for is that Omni-Compass holds through the cloud (`DISCLOSURES.md`, section 10; `tools/places.py`): every judged gauge (the work done, the response time, the minutes over the line, the pods waiting, the failures) reads STEADY or GAIN against native by the one key, in each of runs A, B and C as each run's own word reads it. The bill, the machine-hours and the energy are resources, read beside it (a rise power given, never a LOSS by itself), with the work per dollar beside the bill (the amendment above). A set holds when its runs read no LOSS; a LOSS is root-caused and fixed in the model first, as everywhere. The benefit is not this set's claim: it is shown where it lives, at the software end, and explained through the wire. Nothing else about the sets changes.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

66.58 Amendment (31 October 2026, night): the Omni index scores the software end; energy that no meter read is shown, never scored

where Omni-Compass holds). The index's headline scores from this amendment work, speed and machines, the software end's own results; energy is scored only where a meter read it (class P: the card's own meter). On kind and on the data stores no meter is read (a declared power model's watts; the host's processor seconds), so their energy is a model's estimate about the chip: shown in every row and scored as native's, the same juice, never dropped, because a geometric mean of fewer measures would weigh the rest more and read higher with nothing gained (`tools/omni_index.py`). No run changes and no table is read again: the same three-run tables give both readings, and the reading of 4 October, every measure scored, is printed beside the headline. Read from the tables that stand (omni-v21's): the headline GAIN, up 5.0% (the reading of 4 October: GAIN, up 5.2%); the service reading (work and speed alone, declared 9 October) unchanged, GAIN, up 12.9%.

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

CHAPTER 67

The Pod Start Wait on One Shared Host

One row of the Kubernetes add-on test read LOSS on the engines that stood on 18 October: the mean wait for a serving pod to start under KEDA's HTTP add-on. This page is that row's root cause, measured from the runs' own files. It sets out the mechanism and its exact decomposition, says what it predicts on a fleet of separate machines, and records the rig amendment that takes the test machine's own limit out of both arms. The engine is not changed. The row is never cut: every table that read it stands as recorded, and every page names it.

67.1 The row

The test (`.github/workflows/kind-addons.yml`, `docs/K8S_COMPASS_PREREGISTRATION.md`) runs native as KEDA with its HTTP add-on. In the http setup the add-on scales php-apache on the requests in flight. The omni arm is Omni-Compass on top of the same. On omni-v10, the three-run table of the http setup read the mean pod start wait **LOSS in all three of our runs** (+13.0%, +14.1%, +21.1% a pod). In the same three runs the total start wait read STEADY, pods started fell 24%, the response time fell 57% (mean) and 72% (95th percentile), and CPU used rose 50 to 54% (power given). On omni-v11, two runs of the same setup have landed. The third runs to its end on the rig it started on. Under the other setup, both (the CPU target and the requests in flight), the mean pod start wait, pooled over the repetitions, was lower in the omni arm on both engines (−7.8% on omni-v10, −6.3% on omni-v11; section 67.3).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

67.2 What a pod start is made of

From Kubernetes' own timestamps in each repetition's pod watch (`pod_watch.json`), every serving pod created and made Ready inside the measured window splits into three phases:

- **scheduling**, created → PodScheduled: the scheduler's choice of a node.
- **node side**, PodScheduled → container running: the kubelet, the pod's sandbox and network, and the image.
- **app and probe**, container running → Ready: php-apache's start and its readiness probe (TCP on port 80, every 2 s).

`tools/pod_phases.py` reads them and places each start by two things: the host's busy share over that start (`host_cpu.csv`, the real machine under kind) and whether its node already carried a Ready serving pod when the pod was created. The receipt is `results/live/POD_START_PHASES.md`: 741 starts in five runs, three on omni-v10 (39108180989, 39115242861, 39123583570) and two on omni-v11 (39148024948, 39157778756). Kubernetes stamps these times in whole seconds, so one start is read to ±1 s and the ranges below are over every stratum holding more than one start:

Phase	Node already serving	Empty node
scheduling	0.00 to 0.02 s	0.00 to 0.20 s
node side	0.72 to 0.94 s	6.94 to 11.86 s
app and probe	0.49 to 0.86 s	0.40 to 0.91 s

The whole difference is the node side on an empty node. php-apache runs `registry.k8s.io/hpa-example`, named without a tag, so its pull policy is Always. On a node that already serves, the kubelet asks the registry for the image's digest, finds every layer on the node, and starts the container in under a second. On a node that never served, it downloads the image and unpacks it, and that takes 7 to 12 s.

67.3 Why the omni arm's starts took longer: the exact decomposition

Let s run over strata, each a band b of the host's busy share (0 to 0.5, 0.5 to 0.7, 0.7 to 1) paired with the node's state (empty or already serving). For arm a, let p[a,s] be the share of its starts in stratum s and W[a,s] their mean wait. Then the change in the mean wait splits exactly in two:

$\Delta\bar{W} = \bar{W}[\text{omni}] - \bar{W}[\text{native}] = \Sigma_s\,(p[\text{omni},s] - p[\text{native},s]) \cdot W[\text{native},s] + \Sigma_s\, p[\text{omni},s] \cdot (W[\text{omni},s] - W[\text{native},s])$

The first term is the shift of starts into other strata, a busier host or an empty node. The second is the wait at the same host load on the same kind of node. Where native has no start in a stratum, the first term takes the omni arm's own wait there ($W[\text{native},s] := W[\text{omni},s]$), and the two terms still add to $\Delta\bar{W}$ exactly. `tests/test_pod_phases.py` checks the identity on a constructed run.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Engine, setup	Native W̄	Omni W̄	ΔW̄	Shift into other strata	Same stratum
omni-v10, http	8.52 s	9.94 s	+1.42 s (+16.7%)	+1.57 s	−0.14 s
omni-v11, http	8.62 s	9.27 s	+0.65 s (+7.6%)	+1.64 s	−0.98 s
omni-v10, both	2.64 s	2.43 s	−0.21 s (−7.8%)	−0.35 s	+0.14 s
omni-v11, both	2.99 s	2.80 s	−0.19 s (−6.3%)	−0.26 s	+0.07 s

Under the http setup, every rise in the mean is the shift: 21 of the omni arm's 68 starts on omni-v10, and 11 of its 44 on omni-v11, began while the host was at least 70% busy. Native's starts never did. At the same host load on the same kind of node, the omni arm's starts were as fast as native's or faster (−0.14 s and −0.98 s). These numbers are pooled over the runs' repetitions, so they differ a little from the table's paired means. They are shown, never judged. The judged row stays the three-run table's.

67.4 The mechanism on one machine

A kind cluster is one machine. All seven nodes (a control plane and six workers) are containers on one 4-core GitHub runner, and they share its processor and its disk. Omni-Compass conveys idle processor to the serving pods: it raises their CPU limits in place. That is the work behind the response-time GAIN, and it shows as CPU used, power given. On kind, that processor is the same one on which every node unpacks an image. A pod placed on an empty node therefore unpacks its image on a processor that the conveyed pods on other nodes are keeping busy. Unpacking is processor and disk work, so it took longer: the node side on an empty node rose from 6.9 to 7.9 s with the host under half busy to 10.4 to 11.9 s with it at least 70% busy. omni-v7's shared-host guard (`omni_controller/capacity.py`) already holds every new raise while the runner is 75% busy or unread. It cannot move the unpacking off the processor the raises already granted are using.

67.5 What it predicts on a fleet of separate machines

This section is a prediction, not a measurement. On a fleet, each node is its own machine. A new pod's image is unpacked on its own node's processor, and the processor conveyed to pods on other nodes does not touch it. A pod placed beside pods already serving finds the image on its node and starts in about the time this rig measures for such a pod (1.3 to 1.7 s in total). So the coupling in section 67.4 is a property of the single machine. It is a test machine's limit, never Omni's result, and the row that showed it is reported as measured. The cloud sets (Azure now; Google and Amazon behind their $0 check) run every node on its own machine, and they measure it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

67.6 The amendment (2026-10-18): the served image pulled onto every worker before each arm

On kind, before each arm and before the governor starts or the window opens, `scripts/kind_bench.sh` applies an image pre-puller (`deploy/kind/prepull.yaml`). This is a DaemonSet with one pod on every worker, which pulls `registry.k8s.io/hpa-example` as an operator's image pre-puller does on a production fleet. The bench waits until the pre-puller's pod is ready on every worker, then deletes it. A pre-puller that does not finish on every worker fails the arm's checks, so the arm is marked invalid and never runs on the other rig. Each arm's `preflight.txt` records `images=pre-pulled on every worker before the arm` and the digest each worker pulled.

- **The same in both arms.** Every worker carries the image before either arm is told apart.
- **The operator's manifests are left as written.** `deploy/kind/demo.yaml` still names the image without a tag, so the kubelet still asks the registry for its digest at every start, in both arms. It then finds every layer on the node.
- **Not the engine.** No fingerprinted file changes: the pre-puller is the rig's, and it moved no engine's digest (`python3 tools/omni_version.py`).
- **Not the clouds.** On AKS, GKE and EKS every node is its own machine and unpacks on its own processor. Their sets keep `demo.yaml` and their rig as their A, B and C started.
- **Never two rigs in one table.** A kind workflow's run on omni-v11 counts toward its three only when the run's own commit carries the pre-puller (`RIG_FILE` in `scripts/dispatch_grid.py`, read from GitHub at that commit). That covers the core benchmark, the seven Kubernetes repetition sets, the large organisms, fixed capacity, KEDA's add-ons, CruiseKube, Goldilocks and the robustness pair. Their omni-v11 runs on the rig before run to their end, since nothing is cancelled, and stay as their own record. `tools/confirm_abc.py` reads the rig from every arm's preflight record and heads any table whose A, B and C ran on different rigs with that fact. The grid's squares with the real cluster inside count runs by protocol, carried in each run's name. This rig is protocol 4 (`KUBE_PROTOCOL`), and no omni-v11 run with the cluster inside had started on protocol 3, so all of theirs run on it.

67.7 What the next runs are expected to show (written before any of them)

On the amended rig, a serving pod on an empty node should start in about the time one beside a serving pod takes, about 1.3 to 1.7 s in both arms. The host's busy share should then stop deciding the wait. Whatever the three runs read is the reading. A LOSS is root-caused again and never cut.

67.8 Threats to validity, and what stays

- **Every worker carries the image**, including workers no serving pod would reach in a given repetition. That costs disk before the window and nothing inside it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The registry is still asked for the digest** at every start, in both arms (pull policy Always, as the operator wrote it). The round trip is inside the 0.72 to 0.94 s node side measured on nodes already serving.
- **The host still couples the arms' other work.** Conveyed processor still shares the runner with the kubelets, the probes and the load generator. omni-v7's shared-host guard and the window's refusal rules stand as before.
- **The batch jobs' small image** (`busybox:1.36`, about 4 MB) is still pulled on first use inside the window, as before. It unpacks in well under a second.
- **The model gate carries the old rig.** The conveyance model's pod start time (`start_time` in `tools/model_gate.py`: 8 s up to half busy, rising to 12.4 s near the top) was fitted to the omni-v6 KEDA run's 151 starts. That curve is the empty-node unpacking of section 67.4 in model form, and it is why the gate reads the pod start wait under conveyance LOSS (+9.0% on omni-v10 and omni-v11, `docs/OMNI_V11.md`). The curve is refit from the first amended-rig KEDA runs' own files, never guessed. Until then the model's start-wait row reads this coupling, not the governor.

67.9 Reproduce

```
scripts/raw_archive.sh open /addons-run-39108180989/ /addons-run-39115
    ↪ 242861/ /addons-run-39123583570/ \
                            /addons-run-39148024948/ /addons-run-39157
    ↪ 778756/
python3 tools/pod_phases.py addons-run-39108180989 addons-run-39115242
    ↪ 861 addons-run-39123583570 \

    addons-run-39148024948 addons-run-39157778756 \
    --engine addons-run-39108180989=omni-v10 --engine addons-run-39115
    ↪ 242861=omni-v10 \
    --engine addons-run-39123583570=omni-v10 --engine addons-run-39148
    ↪ 024948=omni-v11 \
    --engine addons-run-39157778756=omni-v11 --out results/live/POD_ST

    ↪ ART_PHASES.md
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 68

The 95th and 99th Percentiles: Where a Tail Reads Higher, and Why

The founder asked on 19 October why the 95th and 99th percentiles show a loss on the live page. This page answers from the receipts themselves. Every number is read from omni-v11's three-run tables (the engine that stood when this page was written) and from the raw files of their runs. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

68.1 No tail reads LOSS

By the preregistered rule (a gauge reads LOSS only when it moves away from its aim in all three of our runs, every 95% interval clear of zero), **no 95th- or 99th-percentile row reads LOSS in any omni-v11 table** (23 tables), and none read LOSS in the omni-v10 tables before them. The one omni-v10 LOSS, the pod start wait under KEDA's HTTP add-on, was the test machine (an empty node unpacking the image inside the window); since the kind rig pulls the served image onto every worker before each arm, it reads STEADY on omni-v11 (`docs/POD_START_WAIT.md`, `results/live/V11_KEDA_REQUESTS.md`).

68.2 Real Kubernetes: Omni-Compass's tail was below native's in every repetition

The 90 paired repetitions of the three omni-v11 Kubernetes tests (Goldilocks, KEDA's HTTP add-on, KEDA on CPU and requests; ten pairs in each of runs A, B and C), each scored from its own raw files by `tools/live_reps.py arm_gauges`:

Test	Repetitions	Omni-Compass's 95th percentile above native's	99th percentile above native's	The 95th percentile, three-run reading
Goldilocks and the VPA	30	0	0	cut 61% to 64%, GAIN
KEDA's HTTP add-on	30	0	0	cut 72% to 74%, GAIN
KEDA on CPU and requests	30	0	0	cut 52% to 53%, GAIN

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

68.3 Where a single run's tail rose: the data stores and the kill test

Six tail rows on omni-v11 have one or two runs whose interval lies wholly above zero. None holds in all three runs, so each reads STEADY; a page that colours each run's cell on its own shows them red.

Table, workload	Gauge	A	B	C	What it is
Kafka at the max, light	end-to-end latency, 99th percentile	+4.2% (+2.0 to +6.5)	−1.6% (−11.8 to +8.6)	−2.8% (−12.6 to +7.0)	one run up, two down: run-to-run variation
MySQL at the tune-down, read and write	latency, 95th percentile	+11.4% (+6.9 to +15.9)	−3.6% (−57.6 to +50.4)	+0.2% (−76.5 to +76.9)	one run up, the other two within wide noise
Redis at the tune-down, small values	latency, 99th percentile	+0.1% (+0.1 to +0.2)	+0.1% (−0.2 to +0.4)	+0.2% (−0.1 to +0.5)	about 6 µs on 5,600 µs, consistent in sign
Redis at the max, small values	latency, 99th percentile	+0.1% (−0.5 to +0.6)	+0.2% (+0.1 to +0.4)	+0.2% (+0.1 to +0.3)	the same
The governor killed outright mid-run (robustness)	response time, 99th percentile	+25.2% (+7.7 to +42.7)	−27.3% (−60.1 to +5.5)	−24.5% (−55.7 to +6.8)	one repetition of run A (below)
Redis at the max, the tuning workload	latency, 99th percentile	+0.3% (−0.0 to +0.5)	+0.2% (+0.1 to +0.2)	+0.0% (−0.2 to +0.3)	the same (the tuning workload, not counted)

The Kafka and MySQL rows are single-run excursions. In each, one run rose and the others fell or sat inside a wide interval; the three-run rule exists to keep such a run from being read as a result. On MySQL's read-and-write workload the three runs' intervals are ±4.5, ±54 and ±77 points wide: with three pairs a run, the test cannot resolve a tail change of under about 10% of native (`docs/POWER.md`).

The kill test's row is one repetition. In nine of run A's ten repetitions both arms' 99th percentile is the probe's 10-second timeout. In the fifth, the omni arm's slowest answers reached 9.2 s 140 to 190 s after the kill, at the load's peak, with every setting already handed back to native's and both arms at the same ten replicas; the runner's processor was 100% busy for 70 s, and the second governor had started 50 s before on the same four cores. That is the likely cause, not proven (`docs/ROBUSTNESS_PREREGISTRATION.md`, "On omni-v11").

The Redis rows are a real, tiny, consistent rise: about 6 µs at the 99th percentile. Pooled over the 27 counted pairs of each set, Omni-Compass's 99th percentile is +5.8 µs (standard deviation 8.9 µs) at the tune-down and +7.1 µs (8.9 µs) at the max, while its mean latency falls about 32 µs (the larger ceiling turns

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

misses into hits). The order of the arms does not explain it: with Omni-Compass running second the rise is +4.9 µs, running first +8.5 µs. The cause is examined in section 68.4.

68.4 The Redis tail: the harness's own layout, not Omni-Compass's effect on Redis

In the Redis harness (`tools/run_redis.py`) the brain runs as a thread inside the load generator's own Python process. Once a second it reads the last second's requests (twice rebuilding a list of about 1,500 of them under the application's lock), decides and writes its audit line. While it computes it holds Python's interpreter lock, so a request thread that wakes from its store-trip sleep at that moment waits before its clock is read. Omni-Compass in a deployment is its own process and never a thread inside the application it governs; only this harness puts the two together. The harnesses of the other data stores do not: Kafka's producer and consumers, pgbench, sysbench and YCSB all run as processes of their own.

The test (`tools/redis_brain_probe.py`, the record in `results/REDIS_BRAIN_PROBE.json`, class S). It runs the harness's own application at the GitHub rate (1,500 requests a second, 32 request threads), with and without the harness's own brain loop in the same process, paired, the order alternating. Two outside parts are stand-ins, the same in both arms. The store answers every request in a fixed 0.1 ms (a sleep, as a socket wait is), a hit with probability 0.6 and otherwise the declared 5 ms store trip. The plug writes nowhere, so the brain reads, decides and runs its trials while the cache never changes. The only difference between the arms is the brain thread computing in the same process (1.4 ms of processor a second).

With the brain in the process, against without (12 pairs of 20 s)	Change	95% interval	Pairs with a rise
response time, 99th percentile	+101 µs	+50 to +151 µs	10 of 12
response time, 95th percentile	+6.8 µs	+1.2 to +12.4 µs	9 of 12
response time, median	+2.1 µs	+0.2 to +4.0 µs	9 of 12
response time, mean	+5.4 µs	+2.9 to +7.9 µs	11 of 12

What it shows. The brain's presence in the load generator's process, with nothing else changed, lengthens the timed requests, most in the tail. In this stand-in the effect is larger than on GitHub's machines (+101 µs at the 99th percentile against about +6 µs), because the stand-in's store answers in a fixed time and its tail is narrow. Three findings agree with it:

- in the Redis set run with native as shipped, the brain is idle (the knob is at its limit on native, and it computes nothing), and the 99th percentile shows no rise of consistent sign (`V11_REDIS_SHIPPED.md`);
- the rise appears in both of the operator's sets, where the brain computes every second;
- the mean still falls in those sets, because the larger ceiling turns misses into hits.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The tail rise is the harness's layout, and it weighs against Omni-Compass, because only the omni arm carries the brain inside its process. A direct test on a real Redis on this machine (three pairs) could not resolve it: the arms differed by hundreds of microseconds from pair to pair, and it was replaced by the test above.

The fix. The harness runs the brain in a process of its own, as a deployment does. It changes how the omni arm is measured, so it lands with the next engine version and its reruns, never inside the omni-v11 tables (`docs/CHANGE_CONTROL.md`).

68.5 Why the live page showed red

The live page (`docs/engine/OMNI_COMPASS_LIVE_ENGINE.html`, rebuilt by its own workflow) reads each benchmark's newest table. Until the omni-v11 tables reached main it read omni-v10's, where the KEDA pod start wait read LOSS. It also shows each run's own interval beside the three-run reading, so a single run's rise, like those in section 68.3, can show as that run's own excursion even where the three-run reading is STEADY. With the omni-v11 tables on main (19 October), the page's only LOSS is gone, and its next rebuild reads them.

CHAPTER 69

How Small an Effect Each Test Can See

Generated by `python3 tools/power.py` from the newest three-run table of each benchmark that stores its rows (`results/live/V*_*.json`); `tools/power.py --check`, run by `verify.py`, fails a push where this page is not what the tables give.

A row reads **STEADY** when a run's 95% interval includes zero: the change is smaller than the test resolves, not zero. The columns say how small:

- **MDE, one run**: the smallest true change one run detects with 80% power (two-sided 5%).
- **MDE, all three runs**: the smallest true change that reads GAIN or LOSS in all three of our runs with 80% power (each run at 92.8%).
- **Repetitions for 5%**: the paired repetitions a run would need for the three-run MDE to reach 5% of native.

Each figure is in percent of native's mean in the least favourable of the three runs. Every number is computed from the table's own intervals with the Student t quantiles the tables use (`omnicompass/student_t.py`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.1 All Four (omni-v21, `V21_ALL_FOUR.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
work inside the response line (requests a second; the capacity test's own gauge)	GAIN in all three of our runs	20.3%	25.0%	250
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.02%)	0.3%	0.3%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.11%)	0.4%	0.4%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.11%)	0.4%	0.4%	1
response time (ms), mean	GAIN in all three of our runs	19.6%	24.1%	232
response time (ms), 95th percentile	GAIN in all three of our runs	24.8%	30.5%	372
response time (ms), 99th percentile	the runs disagree	76.0%	93.4%	3493
time over the response line (% of samples)	GAIN in all three of our runs	20.6%	25.3%	257
failed requests (%)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	9.8%	12.0%	58
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	11.8%	14.6%	85
pending pods, pod-minutes	shown, not judged	196.6 %	241.5%	23336
utilisation (used / allocatable)	shown, not judged	2.9%	3.5%	5
CPU used (cores), mean	shown, not judged	2.9%	3.5%	5
CPU used with Omni's own (cores), mean	shown, not judged	2.9%	3.6%	6
energy per core-hour (Wh)	shown, not judged	5.4%	6.6%	18
HPA replicas, mean	the runs disagree	19.2%	23.6%	222

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pods started	power given by the mean of our three runs, beyond the noise in 0 of 3	62.0%	76.2%	2323
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	81.2%	99.8%	3981
pod start wait, mean (s)	the runs disagree	30.9%	38.0%	578
host CPU busy, the real machine under kind (%)	shown, not judged	2.9%	3.5%	5
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.2 Armada 4W Bare (omni-v12, `V12_ARMADA_4W_BARE.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	19.6%	24.1%	209
node-hours	GAIN by the mean of our three runs, beyond the noise in 1 of 3	19.4%	23.9%	205
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.06%)	0.6%	0.8%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	14.0%	17.2%	107
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	7.3%	9.0%	30
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	14.1%	17.4%	109
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	16.8%	20.6%	153

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
time over the response line (% of samples)	the runs disagree	12.5%	15.4%	86
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	28.5%	35.0%	442
pending pods, pod-minutes	shown, not judged	138.0 %	169.7%	10370
utilisation (used / allocatable)	shown, not judged	25.7%	31.6%	360
CPU used (cores), mean	shown, not judged	7.3%	9.0%	30
CPU used with Omni's own (cores), mean	shown, not judged	7.3%	9.0%	30
energy per core-hour (Wh)	shown, not judged	19.5%	24.0%	208
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	12.9%	15.8%	91
pods started	power given by the mean of our three runs, beyond the noise in 2 of 3	1090.6 %	1341.0%	647434
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	713.6 %	877.5%	277189
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	762.9 %	938.1%	316787
host CPU busy, the real machine under kind (%)	shown, not judged	1.4%	1.8%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	2.3%	2.8%	3
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	30.2%	37.1%	497

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	104.3%	128.3%	5927
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	85.3%	104.9%	3960
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	11.4%	14.0%	71

69.3 Armada 4W Both (omni-v12, `V12_ARMADA_4W_BOTH.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	6.8%	8.4%	26
node-hours	GAIN in all three of our runs	6.7%	8.2%	25
energy, parked workers still on at idle power (Wh)	GAIN in all three of our runs	1.4%	1.7%	2
energy (Wh)	GAIN in all three of our runs	4.4%	5.4%	11
response time (ms), mean	GAIN in all three of our runs	100.2%	123.2%	5464
response time (ms), 95th percentile	GAIN in all three of our runs	122.3%	150.4%	8146
response time (ms), 99th percentile	GAIN in all three of our runs	117.7%	144.8%	7543
time over the response line (% of samples)	GAIN in all three of our runs	127.8%	157.1%	8884
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	112.2%	138.0%	6855

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pending pods, pod-minutes	shown, not judged	108.3 %	133.2%	6387
utilisation (used / allocatable)	shown, not judged	19.1%	23.4%	198
CPU used (cores), mean	shown, not judged	19.4%	23.8%	204
CPU used with Omni's own (cores), mean	shown, not judged	19.3%	23.7%	203
energy per core-hour (Wh)	shown, not judged	67.9%	83.5%	2509
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 2 of 3	8.5%	10.5%	40
pods started	power given by the mean of our three runs, beyond the noise in 2 of 3	940.2 %	1156.0%	481121
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	794.8 %	977.3%	343846
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	733.4 %	901.8%	292753
host CPU busy, the real machine under kind (%)	shown, not judged	15.5%	19.1%	132
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	LOSS in all three of our runs	15.5%	19.0%	131
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	6940.6 %	8534.2%	26219684
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	1056.6 %	1299.2%	607663
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	110.5 %	135.9%	6648

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.4 Armada 4W Orch (omni-v12, `V12_ARMADA_4W_ORCH.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.08%)	0.5%	0.7%	1
energy, parked workers still on at idle power (Wh)	GAIN in all three of our runs	1.4%	1.7%	2
energy (Wh)	GAIN in all three of our runs	1.4%	1.7%	2
response time (ms), mean	GAIN in all three of our runs	96.5%	118.7%	5072
response time (ms), 95th percentile	GAIN in all three of our runs	121.0 %	148.8%	7972
response time (ms), 99th percentile	GAIN in all three of our runs	117.4 %	144.3%	7500
time over the response line (% of samples)	GAIN in all three of our runs	127.8 %	157.1%	8890
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	112.2 %	138.0%	6855
pending pods, pod-minutes	shown, not judged	108.5 %	133.3%	6402
utilisation (used / allocatable)	shown, not judged	19.4%	23.8%	205
CPU used (cores), mean	shown, not judged	19.4%	23.8%	205
CPU used with Omni's own (cores), mean	shown, not judged	19.4%	23.8%	205
energy per core-hour (Wh)	shown, not judged	79.8%	98.1%	3466
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	9.1%	11.2%	46

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pods started	the runs disagree	479.2 %	589.3%	125005
pod start wait, total (s)	the runs disagree	534.9 %	657.8%	155754
pod start wait, mean (s)	the runs disagree	750.9 %	923.3%	306888
host CPU busy, the real machine under kind (%)	shown, not judged	16.0%	19.7%	140
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	LOSS in all three of our runs	15.7%	19.3%	135
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	5666.8 %	6967.9%	17478855
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	793.8 %	976.1%	342992
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	111.8 %	137.5%	6809

69.5 Armada 4W Top (omni-v12, `V12_ARMADA_4W_TOP.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	6.8%	8.4%	26
node-hours	GAIN in all three of our runs	6.9%	8.5%	27
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.04%)	0.3%	0.4%	1
energy (Wh)	GAIN in all three of our runs	5.2%	6.4%	15
response time (ms), mean	GAIN in all three of our runs	16.3%	20.0%	145

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 95th percentile	GAIN in all three of our runs	18.0%	22.1%	177
response time (ms), 99th percentile	GAIN in all three of our runs	13.4%	16.5%	99
utilisation (used / allocatable)	shown, not judged	9.4%	11.6%	49
CPU used (cores), mean	shown, not judged	3.3%	4.0%	6
CPU used with Omni's own (cores), mean	shown, not judged	3.3%	4.1%	7
energy per core-hour (Wh)	shown, not judged	4.5%	5.5%	11
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 2 of 3	8.9%	10.9%	43
host CPU busy, the real machine under kind (%)	shown, not judged	3.7%	4.5%	8
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.01%)	0.4%	0.5%	1
queue: job wait, submitted to its first pod running, mean (s)	STEADY, under the floor (the three runs' mean 0.01%)	0.5%	0.7%	1
queue: job wait, submitted to its first pod running, 95th percentile (s)	STEADY, under the floor (the three runs' mean 0.00%)	0.4%	0.5%	1
queue: job turnaround, submitted to done, mean (s)	STEADY, under the floor (the three runs' mean 0.01%)	0.5%	0.7%	1
queue: jobs unfinished at the window's end	STEADY, the same	0.0%	0.0%	9

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.6 Armada 4W Vs (omni-v12, `V12_ARMADA_4W_VS.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	19.6%	24.1%	209
node-hours	GAIN by the mean of our three runs, beyond the noise in 1 of 3	19.6%	24.2%	211
energy, parked workers still on at idle power (Wh)	power given in all three of our runs	1.2%	1.4%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	15.6%	19.2%	133
response time (ms), mean	LOSS in all three of our runs	374.0 %	459.9%	76129
response time (ms), 95th percentile	LOSS in all three of our runs	1263.0 %	1553.0%	868286
response time (ms), 99th percentile	LOSS in all three of our runs	1506.6 %	1852.5%	1235478
utilisation (used / allocatable)	shown, not judged	70.3%	86.5%	2692
CPU used (cores), mean	shown, not judged	110.7 %	136.1%	6667
CPU used with Omni's own (cores), mean	shown, not judged	110.9 %	136.3%	6692
energy per core-hour (Wh)	shown, not judged	16.8%	20.7%	154
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	9.3%	11.4%	47
host CPU busy, the real machine under kind (%)	shown, not judged	59.8%	73.6%	1949
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	GAIN in all three of our runs	9.5%	11.7%	50

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, mean (s)	GAIN in all three of our runs	0.5%	0.6%	1
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN in all three of our runs	0.4%	0.4%	1
queue: job turnaround, submitted to done, mean (s)	GAIN in all three of our runs	12.2%	15.0%	82
queue: jobs unfinished at the window's end	GAIN in all three of our runs	0.0%	0.0%	9

69.7 Batch (omni-v21, `V21_BATCH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	15.1%	18.6%	139
node-hours	GAIN in all three of our runs	15.4%	18.9%	143
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.11%)	0.8%	0.9%	1
energy (Wh)	GAIN in all three of our runs	10.7%	13.1%	69
response time (ms), mean	GAIN in all three of our runs	6.3%	7.7%	24
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 1 of 3	7.0%	8.6%	30
response time (ms), 99th percentile	STEADY, under the floor (the three runs' mean 0.01%)	9.5%	11.7%	55
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	5.4%	6.7%	18
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	3.5%	4.2%	8
pending pods, pod-minutes	shown, not judged	5.0%	6.1%	15

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
utilisation (used / allocatable)	shown, not judged	20.6%	25.4%	258
CPU used (cores), mean	shown, not judged	4.7%	5.8%	14
CPU used with Omni's own (cores), mean	shown, not judged	4.7%	5.8%	14
energy per core-hour (Wh)	shown, not judged	13.2%	16.2%	106
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	36.3%	44.6%	796
host CPU busy, the real machine under kind (%)	shown, not judged	2.2%	2.7%	3
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.13%)	2.1%	2.6%	3
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	22.0%	27.0%	292

69.8 Binpack (omni-v12, `V12_BINPACK.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	17.1%	21.0%	177
node-hours	GAIN in all three of our runs	17.0%	20.9%	176
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.05%)	0.6%	0.7%	1
energy (Wh)	GAIN in all three of our runs	12.0%	14.8%	88
response time (ms), mean	GAIN in all three of our runs	10.2%	12.6%	64
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 0 of 3	14.5%	17.9%	128

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	8.1%	10.0%	40
time over the response line (% of samples)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	126.0 %	154.8%	9588
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	234.2 %	287.7%	33117
pending pods, pod-minutes	shown, not judged	50.6%	62.2%	1550
utilisation (used / allocatable)	shown, not judged	23.2%	28.6%	327
CPU used (cores), mean	shown, not judged	5.9%	7.2%	21
CPU used with Omni's own (cores), mean	shown, not judged	5.9%	7.2%	22
energy per core-hour (Wh)	shown, not judged	12.8%	15.7%	99
HPA replicas, mean	the runs disagree	13.7%	16.9%	115
pods started	power given by the mean of our three runs, beyond the noise in 0 of 3	818.9 %	1006.3%	405024
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	1074.4 %	1320.2%	697173
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	960.9 %	1180.8%	557738
host CPU busy, the real machine under kind (%)	shown, not judged	0.8%	1.1%	1
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.01%)	0.2%	0.3%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	28.0%	34.4%	475

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	197.8 %	243.1%	23638
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	0.9%	1.1%	1

69.9 Binpack 4W Bare (omni-v12, `V12_BINPACK_4W_BARE.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	17.7%	21.7%	189
node-hours	GAIN in all three of our runs	17.8%	21.9%	193
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.04%)	0.5%	0.6%	1
energy (Wh)	GAIN in all three of our runs	12.4%	15.2%	93
response time (ms), mean	GAIN in all three of our runs	8.2%	10.1%	41
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	11.6%	14.3%	82
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	16.1%	19.8%	158
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	14.7%	18.0%	131
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	19.0%	23.3%	218
pending pods, pod-minutes	shown, not judged	27.6%	33.9%	459
utilisation (used / allocatable)	shown, not judged	23.5%	28.9%	335
CPU used (cores), mean	shown, not judged	2.6%	3.2%	5

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
CPU used with Omni's own (cores), mean	shown, not judged	2.7%	3.3%	5
energy per core-hour (Wh)	shown, not judged	18.6%	22.9%	210
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	12.5%	15.4%	95
host CPU busy, the real machine under kind (%)	shown, not judged	1.6%	2.0%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.49%)	1.0%	1.3%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	28.7%	35.2%	497
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	165.4%	203.3%	16533
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	367.0%	450.9%	81337
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	9.4%	11.6%	54

69.10 Binpack 4W Both (omni-v12, `V12_BINPACK_4W_BOTH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	18.8%	23.1%	215
node-hours	GAIN in all three of our runs	18.9%	23.2%	216
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.08%)	0.5%	0.6%	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy (Wh)	GAIN in all three of our runs	13.0%	15.9%	102
response time (ms), mean	GAIN in all three of our runs	11.6%	14.3%	82
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	15.7%	19.3%	149
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	23.6%	29.0%	337
time over the response line (% of samples)	GAIN in all three of our runs	37.5%	46.1%	852
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	23.0%	28.3%	320
pending pods, pod-minutes	shown, not judged	35.2%	43.2%	748
utilisation (used / allocatable)	shown, not judged	28.3%	34.8%	485
CPU used (cores), mean	shown, not judged	5.0%	6.1%	15
CPU used with Omni's own (cores), mean	shown, not judged	4.9%	6.0%	15
energy per core-hour (Wh)	shown, not judged	20.6%	25.2%	256
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 2 of 3	15.3%	18.8%	142
host CPU busy, the real machine under kind (%)	shown, not judged	1.6%	2.0%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.18%)	1.0%	1.2%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	28.9%	35.5%	503
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	142.2 %	174.8%	12218

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	314.5 %	386.5%	59758
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	8.4%	10.3%	43

69.11 Binpack 4W Orch (omni-v12, `V12_BINPACK_4W_ORCH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.02%)	0.3%	0.4%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.02%)	0.6%	0.7%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.02%)	0.6%	0.7%	1
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	9.7%	11.9%	57
response time (ms), 95th percentile	the runs disagree	18.4%	22.6%	206
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	25.0%	30.7%	378
time over the response line (% of samples)	GAIN in all three of our runs	31.7%	39.0%	609
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	22.0%	27.0%	292
pending pods, pod-minutes	shown, not judged	27.6%	33.9%	461
utilisation (used / allocatable)	shown, not judged	5.5%	6.7%	19

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
CPU used (cores), mean	shown, not judged	5.5%	6.7%	19
CPU used with Omni's own (cores), mean	shown, not judged	5.5%	6.7%	19
energy per core-hour (Wh)	shown, not judged	9.9%	12.1%	59
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	9.4%	11.6%	54
host CPU busy, the real machine under kind (%)	shown, not judged	1.1%	1.3%	1
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.11%)	1.2%	1.5%	1
batch: worker machines in service after the queue finished, mean	STEADY, the same	0.0%	0.0%	10
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	136.3 %	167.5%	11222
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	256.8 %	315.6%	39839
queue: job turnaround, submitted to done, mean (s)	the runs disagree	9.3%	11.4%	53

69.12 Binpack 4W Top (omni-v12, `V12_BINPACK_4W_TOP.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	18.8%	23.1%	215
node-hours	GAIN in all three of our runs	18.9%	23.2%	216

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.06%)	0.6%	0.7%	1
energy (Wh)	GAIN in all three of our runs	13.2%	16.2%	105
response time (ms), mean	GAIN in all three of our runs	6.8%	8.4%	28
response time (ms), 95th percentile	the runs disagree	13.6%	16.7%	112
response time (ms), 99th percentile	the runs disagree	11.9%	14.6%	86
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	16.2%	19.9%	159
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	7.2%	8.8%	31
pending pods, pod-minutes	shown, not judged	27.1%	33.3%	444
utilisation (used / allocatable)	shown, not judged	27.3%	33.6%	451
CPU used (cores), mean	shown, not judged	2.3%	2.8%	4
CPU used with Omni's own (cores), mean	shown, not judged	2.3%	2.9%	4
energy per core-hour (Wh)	shown, not judged	14.4%	17.7%	126
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 2 of 3	14.1%	17.3%	120
host CPU busy, the real machine under kind (%)	shown, not judged	1.0%	1.3%	1
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.29%)	0.9%	1.1%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	28.9%	35.5%	503
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	103.3 %	126.9%	6444

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	221.0 %	271.6%	29510
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	6.3%	7.7%	24

69.13 Binpack 4W Vs (omni-v12, `V12_BINPACK_4W_VS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	17.7%	21.7%	189
node-hours	GAIN in all three of our runs	17.7%	21.8%	190
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.02%)	0.6%	0.8%	1
energy (Wh)	GAIN in all three of our runs	12.5%	15.4%	95
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	13.7%	16.9%	114
response time (ms), 95th percentile	the runs disagree	18.5%	22.7%	207
response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 0 of 3	22.6%	27.8%	310
time over the response line (% of samples)	LOSS in all three of our runs	40.1%	49.3%	971
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	43.5%	53.5%	1146
pending pods, pod-minutes	shown, not judged	31.9%	39.2%	616
utilisation (used / allocatable)	shown, not judged	21.9%	26.9%	289
CPU used (cores), mean	shown, not judged	3.7%	4.5%	9

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
CPU used with Omni's own (cores), mean	shown, not judged	3.8%	4.6%	9
energy per core-hour (Wh)	shown, not judged	13.5%	16.6%	110
HPA replicas, mean	the runs disagree	11.6%	14.2%	81
host CPU busy, the real machine under kind (%)	shown, not judged	1.6%	1.9%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	1.4%	1.8%	2
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	28.7%	35.2%	497
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	247.4 %	304.0%	36971
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	244.6 %	300.6%	36150
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	13.9%	17.1%	117

69.14 Crane 4W Bare (omni-v12, `V12_CRANE_4W_BARE.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.01%)	0.8%	0.9%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.03%)	0.8%	0.9%	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy (Wh)	STEADY, under the floor (the three runs' mean 0.03%)	0.8%	0.9%	1
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	3.9%	4.8%	10
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 2 of 3	6.9%	8.5%	29
response time (ms), 99th percentile	the runs disagree	7.7%	9.4%	36
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	65.4%	80.4%	2585
pending pods, pod-minutes	shown, not judged	209.2%	257.1%	26435
utilisation (used / allocatable)	shown, not judged	2.5%	3.0%	4
CPU used (cores), mean	shown, not judged	2.5%	3.0%	4
CPU used with Omni's own (cores), mean	shown, not judged	2.5%	3.1%	4
energy per core-hour (Wh)	shown, not judged	2.6%	3.2%	5
HPA replicas, mean	power given in all three of our runs	10.9%	13.4%	72
pods started	power given by the mean of our three runs, beyond the noise in 2 of 3	62.8%	77.2%	2382
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	71.3%	87.6%	3072
pod start wait, mean (s)	the runs disagree	34.9%	42.9%	737
host CPU busy, the real machine under kind (%)	shown, not judged	2.8%	3.4%	5
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.15 Crane 4W Both (omni-v12, `V12_CRANE_4W_BOTH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.11%)	0.7%	0.8%	1
energy, parked workers still on at idle power (Wh)	power given in all three of our runs	0.7%	0.9%	1
energy (Wh)	power given in all three of our runs	0.7%	0.9%	1
response time (ms), mean	LOSS by the mean of our three runs, beyond the noise in 0 of 3	3.8%	4.6%	9
response time (ms), 95th percentile	the runs disagree	6.0%	7.4%	22
response time (ms), 99th percentile	the runs disagree	9.1%	11.2%	51
time over the response line (% of samples)	the runs disagree	51.6%	63.4%	1609
pending pods, pod-minutes	shown, not judged	364.6%	448.0%	80289
utilisation (used / allocatable)	shown, not judged	4.0%	4.9%	10
CPU used (cores), mean	shown, not judged	4.0%	4.9%	10
CPU used with Omni's own (cores), mean	shown, not judged	4.2%	5.1%	11
energy per core-hour (Wh)	shown, not judged	2.5%	3.1%	4
HPA replicas, mean	power given in all three of our runs	14.2%	17.4%	122
pods started	power given in all three of our runs	59.8%	73.5%	2160
pod start wait, total (s)	LOSS in all three of our runs	77.8%	95.6%	3658
pod start wait, mean (s)	the runs disagree	31.3%	38.4%	591
host CPU busy, the real machine under kind (%)	shown, not judged	5.2%	6.4%	17

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.16 Crane 4W Orch (omni-v12, `V12_CRANE_4W_ORCH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.32%)	0.8%	1.0%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.31%)	0.8%	1.0%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.31%)	0.8%	1.0%	1
response time (ms), mean	LOSS in all three of our runs	3.7%	4.6%	9
response time (ms), 95th percentile	the runs disagree	4.6%	5.7%	13
response time (ms), 99th percentile	the runs disagree	8.9%	11.0%	49
time over the response line (% of samples)	the runs disagree	56.1%	69.0%	1903
pending pods, pod-minutes	shown, not judged	176.3%	216.6%	18770
utilisation (used / allocatable)	shown, not judged	2.8%	3.5%	5
CPU used (cores), mean	shown, not judged	2.8%	3.5%	5
CPU used with Omni's own (cores), mean	shown, not judged	2.8%	3.5%	5
energy per core-hour (Wh)	shown, not judged	1.8%	2.3%	3
HPA replicas, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	2.0%	2.4%	3

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pods started	power given by the mean of our three runs, beyond the noise in 1 of 3	28.6%	35.1%	494
pod start wait, total (s)	the runs disagree	46.2%	56.8%	1291
pod start wait, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	39.5%	48.5%	942
host CPU busy, the real machine under kind (%)	shown, not judged	2.7%	3.4%	5
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.17 Crane 4W Top (omni-v12, `V12_CRANE_4W_TOP.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.21%)	1.0%	1.2%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.32%)	0.9%	1.2%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.32%)	0.9%	1.2%	1
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	2.5%	3.1%	4
response time (ms), 95th percentile	the runs disagree	5.2%	6.4%	17
response time (ms), 99th percentile	the runs disagree	6.3%	7.8%	24
time over the response line (% of samples)	the runs disagree	64.2%	78.9%	2489

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pending pods, pod-minutes	shown, not judged	575.3%	706.9%	199906
utilisation (used / allocatable)	shown, not judged	2.6%	3.2%	5
CPU used (cores), mean	shown, not judged	2.6%	3.2%	5
CPU used with Omni's own (cores), mean	shown, not judged	2.8%	3.4%	5
energy per core-hour (Wh)	shown, not judged	2.3%	2.8%	4
HPA replicas, mean	power given in all three of our runs	14.4%	17.7%	126
pods started	power given in all three of our runs	54.2%	66.6%	1773
pod start wait, total (s)	LOSS in all three of our runs	80.2%	98.6%	3886
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	27.7%	34.0%	463
host CPU busy, the real machine under kind (%)	shown, not judged	2.7%	3.3%	5
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.18 Crane 4W Vs (omni-v12, `V12_CRANE_4W_VS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.31%)	0.9%	1.1%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.28%)	0.9%	1.1%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.28%)	0.9%	1.1%	1
response time (ms), mean	GAIN in all three of our runs	3.6%	4.4%	8

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 2 of 3	4.9%	6.1%	15
response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 0 of 3	8.8%	10.8%	48
time over the response line (% of samples)	the runs disagree	65.9%	81.0%	2622
pending pods, pod-minutes	shown, not judged	281.3%	345.7%	47807
utilisation (used / allocatable)	shown, not judged	2.7%	3.3%	5
CPU used (cores), mean	shown, not judged	2.7%	3.3%	5
CPU used with Omni's own (cores), mean	shown, not judged	2.7%	3.3%	5
energy per core-hour (Wh)	shown, not judged	2.9%	3.5%	6
HPA replicas, mean	power given in all three of our runs	11.3%	13.8%	77
pods started	power given by the mean of our three runs, beyond the noise in 2 of 3	55.0%	67.6%	1828
pod start wait, total (s)	LOSS in all three of our runs	77.2%	94.8%	3599
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	38.9%	47.8%	915
host CPU busy, the real machine under kind (%)	shown, not judged	2.2%	2.6%	3
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.19 Cruisekube (omni-v21, `V21_CRUISEKUBE.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
node-hours	STEADY, under the floor (the three runs' mean 0.11%)	0.7%	0.9%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.09%)	0.7%	0.8%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.09%)	0.7%	0.8%	1
response time (ms), mean	LOSS by the mean of our three runs, beyond the noise in 2 of 3	2.5%	3.0%	4
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	2.5%	3.1%	4
response time (ms), 99th percentile	the runs disagree	7.2%	8.9%	32
pending pods, pod-minutes	shown, not judged	336.8%	413.9%	68531
utilisation (used / allocatable)	shown, not judged	3.0%	3.7%	6
CPU used (cores), mean	shown, not judged	3.0%	3.7%	6
CPU used with Omni's own (cores), mean	shown, not judged	3.1%	3.8%	6
energy per core-hour (Wh)	shown, not judged	2.9%	3.5%	6
HPA replicas, mean	the runs disagree	12.8%	15.8%	100
pods started	the runs disagree	61.8%	76.0%	2309
pod start wait, total (s)	the runs disagree	86.2%	105.9%	4488
pod start wait, mean (s)	the runs disagree	50.1%	61.6%	1516
host CPU busy, the real machine under kind (%)	shown, not judged	2.2%	2.7%	3
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.20 Fairness (omni-v21, `V21_FAIRNESS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.08%)	0.6%	0.7%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.01%)	0.5%	0.7%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.01%)	0.5%	0.7%	1
response time (ms), mean	GAIN in all three of our runs	13.9%	17.1%	118
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 1 of 3	21.1%	26.0%	270
response time (ms), 99th percentile	the runs disagree	19.2%	23.6%	223
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	15.5%	19.1%	146
failed requests (%)	the runs disagree	37.5%	46.1%	851
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	7.7%	9.5%	37
pending pods, pod-minutes	shown, not judged	148.2 %	182.2%	13272
utilisation (used / allocatable)	shown, not judged	2.3%	2.8%	4
CPU used (cores), mean	shown, not judged	2.3%	2.8%	4
CPU used with Omni's own (cores), mean	shown, not judged	2.2%	2.8%	4
energy per core-hour (Wh)	shown, not judged	2.6%	3.1%	4
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	4.4%	5.4%	12

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pods started	GAIN by the mean of our three runs, beyond the noise in 1 of 3	35.0%	43.0%	741
pod start wait, total (s)	the runs disagree	70.4%	86.5%	2994
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	81.9%	100.7%	4053
host CPU busy, the real machine under kind (%)	shown, not judged	1.5%	1.8%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
second app: response time (ms), 95th percentile	the runs disagree	73.7%	90.5%	3278
second app: response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 0 of 3	84.9%	104.3%	4356
second app: time over the response line (% of samples)	the runs disagree	8.1%	9.9%	40
second app: failed requests (%)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	8.3%	10.2%	42

69.21 Faults (omni-v21, `V21_FAULTS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, under the floor (the three runs' mean 0.03%)	0.2%	0.2%	1
node-hours	STEADY, under the floor (the three runs' mean 0.46%)	0.8%	1.0%	1
energy, parked workers still on at idle power (Wh)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	0.9%	1.1%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	0.9%	1.1%	1
response time (ms), mean	GAIN in all three of our runs	34.9%	42.8%	734

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 95th percentile	GAIN in all three of our runs	25.8%	31.7%	402
response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 2 of 3	81.8%	100.5%	4042
time over the response line (% of samples)	GAIN in all three of our runs	22.2%	27.3%	299
failed requests (%)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	21.1%	25.9%	270
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	22.8%	28.1%	315
pending pods, pod-minutes	shown, not judged	256.5 %	315.1%	39728
utilisation (used / allocatable)	shown, not judged	5.3%	6.5%	18
CPU used (cores), mean	shown, not judged	5.3%	6.5%	18
CPU used with Omni's own (cores), mean	shown, not judged	5.3%	6.5%	17
energy per core-hour (Wh)	shown, not judged	4.9%	6.0%	15
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	24.1%	29.6%	350
pods started	power given by the mean of our three runs, beyond the noise in 1 of 3	37.0%	45.4%	826
pod start wait, total (s)	the runs disagree	253.0 %	310.9%	38658
pod start wait, mean (s)	the runs disagree	238.8 %	293.5%	34459
host CPU busy, the real machine under kind (%)	shown, not judged	4.1%	5.0%	11

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.22 Goldilocks (omni-v21, `V21_GOLDILOCKS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.39%)	0.8%	1.1%	1
energy, parked workers still on at idle power (Wh)	power given in all three of our runs	1.0%	1.2%	1
energy (Wh)	power given in all three of our runs	1.0%	1.2%	1
response time (ms), mean	GAIN in all three of our runs	20.7%	25.4%	259
response time (ms), 95th percentile	GAIN in all three of our runs	17.8%	21.8%	191
response time (ms), 99th percentile	GAIN in all three of our runs	16.3%	20.0%	160
time over the response line (% of samples)	GAIN in all three of our runs	124.9 %	153.5%	9430
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	314.5 %	386.5%	59758
pending pods, pod-minutes	shown, not judged	284.4 %	349.4%	48840
utilisation (used / allocatable)	shown, not judged	16.1%	19.8%	158
CPU used (cores), mean	shown, not judged	16.1%	19.8%	158
CPU used with Omni's own (cores), mean	shown, not judged	16.5%	20.3%	165
energy per core-hour (Wh)	shown, not judged	7.7%	9.5%	36

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
HPA replicas, mean	power given in all three of our runs	17.4%	21.4%	184
pods started	power given in all three of our runs	73.3%	90.1%	3249
pod start wait, total (s)	LOSS in all three of our runs	146.0 %	179.4%	12869
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	53.7%	66.0%	1745
host CPU busy, the real machine under kind (%)	shown, not judged	15.8%	19.4%	151
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.23 Kai (omni-v12, `V12_KAI.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	13.4%	16.5%	109
node-hours	GAIN in all three of our runs	13.7%	16.8%	113
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.01%)	0.6%	0.8%	1
energy (Wh)	GAIN in all three of our runs	9.3%	11.5%	53
response time (ms), mean	GAIN in all three of our runs	8.9%	10.9%	48
response time (ms), 95th percentile	the runs disagree	23.4%	28.8%	331
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	24.4%	30.0%	360
time over the response line (% of samples)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	74.5%	91.6%	3357
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	142.9 %	175.6%	12331

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pending pods, pod-minutes	shown, not judged	26.7%	32.8%	431
utilisation (used / allocatable)	shown, not judged	20.9%	25.7%	265
CPU used (cores), mean	shown, not judged	5.8%	7.1%	21
CPU used with Omni's own (cores), mean	shown, not judged	5.4%	6.6%	18
energy per core-hour (Wh)	shown, not judged	13.6%	16.7%	112
HPA replicas, mean	the runs disagree	16.2%	19.9%	159
host CPU busy, the real machine under kind (%)	shown, not judged	2.4%	2.9%	4
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.34%)	0.5%	0.6%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	24.9%	30.6%	374
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	12.2%	15.1%	91
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	41.9%	51.5%	1063
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	10.8%	13.2%	70

69.24 Kai 4W Bare (omni-v12, `V12_KAI_4W_BARE.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	16.4%	20.1%	162
node-hours	GAIN in all three of our runs	16.2%	19.9%	159

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.02%)	0.4%	0.5%	1
energy (Wh)	GAIN in all three of our runs	11.4%	14.0%	79
response time (ms), mean	GAIN in all three of our runs	10.2%	12.6%	64
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	14.4%	17.6%	125
response time (ms), 99th percentile	the runs disagree	21.7%	26.6%	284
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	9.4%	11.6%	54
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	7.3%	9.0%	33
pending pods, pod-minutes	shown, not judged	28.0%	34.4%	475
utilisation (used / allocatable)	shown, not judged	17.2%	21.2%	180
CPU used (cores), mean	shown, not judged	4.4%	5.4%	12
CPU used with Omni's own (cores), mean	shown, not judged	4.4%	5.4%	12
energy per core-hour (Wh)	shown, not judged	10.0%	12.3%	61
HPA replicas, mean	the runs disagree	19.0%	23.3%	218
host CPU busy, the real machine under kind (%)	shown, not judged	1.6%	2.0%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	1.3%	1.6%	2
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	26.8%	32.9%	433
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	129.1 %	158.6%	10063

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	165.8 %	203.7%	16600
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	15.5%	19.1%	146

69.25 Kai 4W Both (omni-v12, `V12_KAI_4W_BOTH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	13.8%	16.9%	115
node-hours	GAIN by the mean of our three runs, beyond the noise in 2 of 3	13.5%	16.6%	111
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.09%)	0.6%	0.8%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	9.5%	11.7%	55
response time (ms), mean	LOSS by the mean of our three runs, beyond the noise in 0 of 3	14.0%	17.2%	118
response time (ms), 95th percentile	LOSS in all three of our runs	42.0%	51.6%	1066
response time (ms), 99th percentile	LOSS in all three of our runs	46.0%	56.5%	1278
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	27.2%	33.4%	448
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	22.4%	27.5%	302
pending pods, pod-minutes	shown, not judged	122.9 %	151.0%	9118

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
utilisation (used / allocatable)	shown, not judged	13.4%	16.5%	109
CPU used (cores), mean	shown, not judged	5.2%	6.4%	17
CPU used with Omni's own (cores), mean	shown, not judged	5.3%	6.5%	17
energy per core-hour (Wh)	shown, not judged	8.6%	10.6%	45
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	20.4%	25.1%	253
host CPU busy, the real machine under kind (%)	shown, not judged	2.8%	3.5%	5
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS in all three of our runs	4.8%	5.8%	14
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	23.5%	28.9%	335
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	996.5 %	1224.5%	599760
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	724.5 %	890.3%	317048
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	42.0%	51.5%	1063

69.26 Kai 4W Orch (omni-v12, `V12_KAI_4W_ORCH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
node-hours	STEADY, under the floor (the three runs' mean 0.03%)	0.5%	0.7%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.06%)	0.5%	0.6%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.06%)	0.5%	0.6%	1
response time (ms), mean	LOSS in all three of our runs	16.1%	19.8%	157
response time (ms), 95th percentile	LOSS in all three of our runs	43.0%	52.9%	1118
response time (ms), 99th percentile	LOSS in all three of our runs	44.7%	54.9%	1206
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	22.6%	27.8%	308
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	17.6%	21.7%	188
pending pods, pod-minutes	shown, not judged	72.4%	89.0%	3166
utilisation (used / allocatable)	shown, not judged	2.8%	3.5%	5
CPU used (cores), mean	shown, not judged	2.8%	3.5%	5
CPU used with Omni's own (cores), mean	shown, not judged	2.8%	3.5%	5
energy per core-hour (Wh)	shown, not judged	3.1%	3.8%	6
HPA replicas, mean	the runs disagree	16.3%	20.0%	161
host CPU busy, the real machine under kind (%)	shown, not judged	2.2%	2.8%	4
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS in all three of our runs	4.0%	4.9%	10
batch: worker machines in service after the queue finished, mean	STEADY, the same	0.0%	0.0%	10

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	633.6 %	778.5%	242454
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	451.0 %	554.2%	122836
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	33.3%	40.9%	671

69.27 Kai 4W Top (omni-v12, `V12_KAI_4W_TOP.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	13.8%	16.9%	115
node-hours	GAIN by the mean of our three runs, beyond the noise in 2 of 3	13.7%	16.9%	114
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.02%)	0.6%	0.7%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	9.6%	11.8%	56
response time (ms), mean	GAIN in all three of our runs	12.5%	15.4%	96
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	16.1%	19.7%	156
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	13.5%	16.6%	110
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	17.6%	21.6%	188

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	7.6%	9.3%	35
pending pods, pod-minutes	shown, not judged	29.8%	36.6%	538
utilisation (used / allocatable)	shown, not judged	15.3%	18.8%	142
CPU used (cores), mean	shown, not judged	3.6%	4.5%	8
CPU used with Omni's own (cores), mean	shown, not judged	3.6%	4.4%	8
energy per core-hour (Wh)	shown, not judged	10.6%	13.0%	68
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	14.8%	18.2%	133
pods started	power given by the mean of our three runs, beyond the noise in 0 of 3	471.8 %	579.8%	134455
pod start wait, total (s)	the runs disagree	500.4 %	614.9%	151239
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	766.5 %	941.9%	354857
host CPU busy, the real machine under kind (%)	shown, not judged	1.5%	1.9%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	1.1%	1.4%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	23.5%	28.9%	335
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	45.3%	55.7%	1241

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	117.2 %	144.1%	8300
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	8.4%	10.3%	43

69.28 Kai 4W Vs (omni-v12, `V12_KAI_4W_VS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	16.4%	20.1%	162
node-hours	GAIN in all three of our runs	16.4%	20.1%	163
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.09%)	0.5%	0.6%	1
energy (Wh)	GAIN in all three of our runs	11.4%	14.1%	79
response time (ms), mean	GAIN in all three of our runs	19.2%	23.6%	222
response time (ms), 95th percentile	GAIN in all three of our runs	31.4%	38.5%	595
response time (ms), 99th percentile	GAIN in all three of our runs	32.2%	39.6%	627
time over the response line (% of samples)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	24.0%	29.4%	347
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	10.8%	13.3%	72
pending pods, pod-minutes	shown, not judged	39.8%	48.9%	956
utilisation (used / allocatable)	shown, not judged	19.5%	24.0%	231
CPU used (cores), mean	shown, not judged	3.2%	3.9%	7
CPU used with Omni's own (cores), mean	shown, not judged	3.1%	3.9%	6

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy per core-hour (Wh)	shown, not judged	12.8%	15.7%	99
HPA replicas, mean	the runs disagree	11.1%	13.7%	75
pods started	GAIN by the mean of our three runs, beyond the noise in 0 of 3	406.1 %	499.0%	99597
pod start wait, total (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	364.9 %	448.4%	80415
pod start wait, mean (s)	the runs disagree	454.3 %	558.3%	124680
host CPU busy, the real machine under kind (%)	shown, not judged	2.1%	2.6%	3
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	GAIN in all three of our runs	3.1%	3.9%	6
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	26.8%	32.9%	433
queue: job wait, submitted to its first pod running, mean (s)	GAIN in all three of our runs	39.7%	48.8%	952
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN in all three of our runs	80.2%	98.6%	3890
queue: job turnaround, submitted to done, mean (s)	GAIN in all three of our runs	21.8%	26.8%	289

69.29 Karmada 4W Bare (omni-v12, `V12_KARMADA_4W_BARE.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	21.1%	25.9%	243

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
node-hours	GAIN by the mean of our three runs, beyond the noise in 2 of 3	21.3%	26.2%	247
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.10%)	0.4%	0.5%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	15.3%	18.8%	127
response time (ms), mean	the runs disagree	5.8%	7.1%	19
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	11.4%	14.0%	71
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	15.9%	19.6%	138
time over the response line (% of samples)	the runs disagree	12.0%	14.8%	79
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	24.0%	29.5%	313
pending pods, pod-minutes	shown, not judged	99.6%	122.5%	5402
utilisation (used / allocatable)	shown, not judged	26.1%	32.1%	372
CPU used (cores), mean	shown, not judged	7.3%	9.0%	29
CPU used with Omni's own (cores), mean	shown, not judged	7.3%	9.0%	30
energy per core-hour (Wh)	shown, not judged	17.9%	22.1%	176
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 2 of 3	8.7%	10.7%	42
host CPU busy, the real machine under kind (%)	shown, not judged	2.0%	2.5%	3
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	2.4%	3.0%	4
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	32.8%	40.3%	585
queue: job wait, submitted to its first pod running, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	94.7%	116.5%	4884
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	115.2 %	141.6%	7223
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	9.2%	11.3%	47

69.30 Karmada 4W Both (omni-v12, `V12_KARMADA_4W_BOTH.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	19.7%	24.2%	211
node-hours	GAIN by the mean of our three runs, beyond the noise in 1 of 3	19.8%	24.4%	215
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.28%)	0.7%	0.9%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	14.2%	17.4%	109
response time (ms), mean	LOSS in all three of our runs	41.4%	50.9%	931
response time (ms), 95th percentile	LOSS in all three of our runs	66.3%	81.5%	2392
response time (ms), 99th percentile	LOSS in all three of our runs	54.1%	66.6%	1596
time over the response line (% of samples)	LOSS in all three of our runs	21.3%	26.1%	247

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS in all three of our runs	33.0%	40.6%	595
pending pods, pod-minutes	shown, not judged	325.8 %	400.5%	57756
utilisation (used / allocatable)	shown, not judged	25.9%	31.8%	365
CPU used (cores), mean	shown, not judged	10.9%	13.4%	65
CPU used with Omni's own (cores), mean	shown, not judged	10.9%	13.4%	65
energy per core-hour (Wh)	shown, not judged	18.8%	23.1%	193
HPA replicas, mean	power given in all three of our runs	13.3%	16.4%	97
host CPU busy, the real machine under kind (%)	shown, not judged	7.7%	9.5%	33
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	LOSS in all three of our runs	12.6%	15.5%	87
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	32.4%	39.8%	572
queue: job wait, submitted to its first pod running, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	20939. 2%	25746.9%	238644457
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	31127. 1%	38274.0%	527363696
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	59.7%	73.4%	1940

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.31 Karmada 4W Orch (omni-v12, `V12_KARMADA_4W_ORCH.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.00%)	0.5%	0.7%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.02%)	0.6%	0.8%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.02%)	0.6%	0.8%	1
response time (ms), mean	LOSS in all three of our runs	20.0%	24.5%	217
response time (ms), 95th percentile	LOSS in all three of our runs	40.4%	49.6%	887
response time (ms), 99th percentile	LOSS in all three of our runs	43.3%	53.2%	1021
time over the response line (% of samples)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	18.1%	22.2%	178
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS in all three of our runs	32.7%	40.2%	583
pending pods, pod-minutes	shown, not judged	246.3 %	302.9%	33032
utilisation (used / allocatable)	shown, not judged	10.7%	13.2%	63
CPU used (cores), mean	shown, not judged	10.7%	13.2%	63
CPU used with Omni's own (cores), mean	shown, not judged	10.7%	13.2%	63
energy per core-hour (Wh)	shown, not judged	8.3%	10.2%	38
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	9.8%	12.0%	53
host CPU busy, the real machine under kind (%)	shown, not judged	5.5%	6.7%	17

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	LOSS in all three of our runs	9.1%	11.2%	46
batch: worker machines in service after the queue finished, mean	STEADY, the same	0.0%	0.0%	9
queue: job wait, submitted to its first pod running, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	15916.3%	19570.7%	137884244
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	26101.8%	32094.8%	370828125
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	40.6%	49.9%	897

69.32 Karmada 4W Top (omni-v12, `V12_KARMADA_4W_TOP.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	19.7%	24.2%	211
node-hours	GAIN by the mean of our three runs, beyond the noise in 1 of 3	20.0%	24.6%	219
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.26%)	0.7%	0.8%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	14.5%	17.8%	115
response time (ms), mean	LOSS by the mean of our three runs, beyond the noise in 0 of 3	19.8%	24.4%	214
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 2 of 3	22.0%	27.0%	263

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 2 of 3	14.0%	17.2%	108
time over the response line (% of samples)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	22.1%	27.2%	267
failed requests (%)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	356.8 %	438.7%	69281
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	36.5%	44.9%	725
pending pods, pod-minutes	shown, not judged	99.0%	121.7%	5334
utilisation (used / allocatable)	shown, not judged	23.4%	28.7%	297
CPU used (cores), mean	shown, not judged	9.1%	11.2%	46
CPU used with Omni's own (cores), mean	shown, not judged	9.2%	11.3%	46
energy per core-hour (Wh)	shown, not judged	16.7%	20.6%	153
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 2 of 3	16.5%	20.2%	148
pods started	power given by the mean of our three runs, beyond the noise in 0 of 3	634.0 %	779.5%	218759
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	640.8 %	787.9%	223481
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	249.3 %	306.5%	33823
host CPU busy, the real machine under kind (%)	shown, not judged	6.3%	7.7%	22
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	4.6%	5.7%	12
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	32.4%	39.8%	572
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	230.4 %	283.3%	28900
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	309.3 %	380.3%	52060
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	15.7%	19.2%	134

69.33 Karmada 4W Vs (omni-v12, `V12_KARMADA_4W_VS.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	21.1%	25.9%	243
node-hours	GAIN by the mean of our three runs, beyond the noise in 1 of 3	21.5%	26.4%	252
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.07%)	0.8%	0.9%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	15.6%	19.2%	133
response time (ms), mean	GAIN in all three of our runs	17.9%	22.0%	175
response time (ms), 95th percentile	GAIN in all three of our runs	24.6%	30.2%	329
response time (ms), 99th percentile	GAIN in all three of our runs	21.5%	26.4%	252
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	12.3%	15.1%	83

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
failed requests (%)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	232.5 %	285.9%	29425
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	25.0%	30.8%	341
pending pods, pod-minutes	shown, not judged	101.9 %	125.3%	5651
utilisation (used / allocatable)	shown, not judged	25.1%	30.9%	343
CPU used (cores), mean	shown, not judged	11.7%	14.4%	75
CPU used with Omni's own (cores), mean	shown, not judged	11.7%	14.4%	75
energy per core-hour (Wh)	shown, not judged	16.5%	20.3%	149
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	12.4%	15.2%	84
pods started	power given by the mean of our three runs, beyond the noise in 0 of 3	281.1 %	345.6%	43004
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	246.9 %	303.6%	33185
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	236.2 %	290.4%	30367
host CPU busy, the real machine under kind (%)	shown, not judged	3.8%	4.6%	8
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	GAIN in all three of our runs	6.2%	7.7%	22
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	32.8%	40.3%	585

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	282.4 %	347.2%	43409
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	310.5 %	381.8%	52466
queue: job turnaround, submitted to done, mean (s)	GAIN in all three of our runs	21.7%	26.7%	257

69.34 Keda Cpu Requests (omni-v21, `V21_KEDA_CPU_REQUESTS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.09%)	0.9%	1.1%	1
energy, parked workers still on at idle power (Wh)	power given in all three of our runs	1.3%	1.6%	2
energy (Wh)	power given in all three of our runs	1.3%	1.6%	2
response time (ms), mean	GAIN in all three of our runs	15.2%	18.7%	140
response time (ms), 95th percentile	GAIN in all three of our runs	13.2%	16.2%	106
response time (ms), 99th percentile	GAIN in all three of our runs	13.8%	17.0%	116
time over the response line (% of samples)	GAIN in all three of our runs	90.0%	110.6%	4890
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	314.5 %	386.5%	59758
pending pods, pod-minutes	shown, not judged	328.2 %	403.3%	65073

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
utilisation (used / allocatable)	shown, not judged	13.3%	16.3%	107
CPU used (cores), mean	shown, not judged	13.3%	16.3%	107
CPU used with Omni's own (cores), mean	shown, not judged	13.9%	17.1%	117
energy per core-hour (Wh)	shown, not judged	7.0%	8.6%	30
HPA replicas, mean	power given in all three of our runs	13.2%	16.2%	106
pods started	power given by the mean of our three runs, beyond the noise in 2 of 3	51.6%	63.4%	1606
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	60.5%	74.3%	2210
pod start wait, mean (s)	the runs disagree	33.9%	41.7%	695
host CPU busy, the real machine under kind (%)	shown, not judged	12.1%	14.9%	89
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.35 Keda Requests (omni-v21, `V21_KEDA_REQUESTS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.29%)	0.6%	0.7%	1
energy, parked workers still on at idle power (Wh)	power given in all three of our runs	0.9%	1.1%	1
energy (Wh)	power given in all three of our runs	0.9%	1.1%	1
response time (ms), mean	GAIN in all three of our runs	18.4%	22.6%	205
response time (ms), 95th percentile	GAIN in all three of our runs	18.0%	22.1%	197

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 99th percentile	GAIN in all three of our runs	23.9%	29.3%	345
time over the response line (% of samples)	GAIN in all three of our runs	88.8%	109.1%	4758
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	89.8%	110.3%	4870
pending pods, pod-minutes	shown, not judged	454.4 %	558.4%	124729
utilisation (used / allocatable)	shown, not judged	14.7%	18.0%	130
CPU used (cores), mean	shown, not judged	14.7%	18.0%	130
CPU used with Omni's own (cores), mean	shown, not judged	15.0%	18.4%	136
energy per core-hour (Wh)	shown, not judged	2.9%	3.6%	6
HPA replicas, mean	GAIN in all three of our runs	12.3%	15.2%	92
pods started	GAIN in all three of our runs	22.4%	27.5%	303
pod start wait, total (s)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	44.5%	54.7%	1196
pod start wait, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	36.7%	45.1%	814
host CPU busy, the real machine under kind (%)	shown, not judged	14.2%	17.5%	122
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.36 Koordinator (omni-v12, `V12_KOORDINATOR.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, under the floor (the three runs' mean 0.44%)	2.0%	2.4%	3
node-hours	GAIN by the mean of our three runs, beyond the noise in 0 of 3	2.2%	2.7%	3
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.14%)	0.8%	1.0%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.45%)	1.6%	1.9%	2
response time (ms), mean	GAIN in all three of our runs	12.1%	14.9%	89
response time (ms), 95th percentile	GAIN in all three of our runs	7.7%	9.4%	36
response time (ms), 99th percentile	GAIN in all three of our runs	11.3%	13.9%	78
time over the response line (% of samples)	the runs disagree	277.2 %	340.6%	46417
pending pods, pod-minutes	shown, not judged	14.5%	17.8%	127
utilisation (used / allocatable)	shown, not judged	4.2%	5.2%	11
CPU used (cores), mean	shown, not judged	3.6%	4.4%	8
CPU used with Omni's own (cores), mean	shown, not judged	3.5%	4.3%	8
energy per core-hour (Wh)	shown, not judged	3.9%	4.8%	10
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	15.9%	19.6%	154
pods started	power given by the mean of our three runs, beyond the noise in 0 of 3	838.8 %	1030.7%	424944
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	909.5 %	1117.6%	499641

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	625.2 %	768.3%	236126
host CPU busy, the real machine under kind (%)	shown, not judged	3.0%	3.6%	6
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	6.1%	7.5%	23
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	73.2%	90.0%	3240
queue: job wait, submitted to its first pod running, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	39.7%	48.8%	953
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	42.0%	51.6%	1068
queue: job turnaround, submitted to done, mean (s)	the runs disagree	11.7%	14.4%	83
queue: jobs unfinished at the window's end	the runs disagree	59.2%	72.7%	2115

69.37 Koordinator 4W Bare (omni-v12, `V12_KOORDINATOR_4W_BARE.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	19.3%	23.7%	226
node-hours	GAIN in all three of our runs	19.4%	23.8%	227
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.20%)	0.5%	0.6%	1
energy (Wh)	GAIN in all three of our runs	13.5%	16.6%	111

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), mean	GAIN in all three of our runs	8.9%	11.0%	49
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	11.4%	14.0%	79
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	15.7%	19.3%	149
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	11.6%	14.2%	82
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	15.4%	18.9%	143
pending pods, pod-minutes	shown, not judged	40.5%	49.8%	994
utilisation (used / allocatable)	shown, not judged	24.0%	29.5%	348
CPU used (cores), mean	shown, not judged	4.3%	5.3%	12
CPU used with Omni's own (cores), mean	shown, not judged	4.3%	5.2%	11
energy per core-hour (Wh)	shown, not judged	16.9%	20.8%	173
HPA replicas, mean	the runs disagree	13.3%	16.4%	108
host CPU busy, the real machine under kind (%)	shown, not judged	1.7%	2.1%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.40%)	0.8%	0.9%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	30.3%	37.2%	554
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	264.4 %	324.9%	42237
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	480.5 %	590.4%	139435

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	13.9%	17.1%	117

69.38 Koordinator 4W Both (omni-v12, `V12_KOORDINATOR_4W_BOTH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	5.1%	6.3%	16
node-hours	GAIN by the mean of our three runs, beyond the noise in 0 of 3	5.3%	6.5%	17
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.27%)	1.1%	1.3%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	3.5%	4.4%	8
response time (ms), mean	GAIN in all three of our runs	55.0%	67.6%	1827
response time (ms), 95th percentile	GAIN in all three of our runs	70.9%	87.1%	3035
response time (ms), 99th percentile	GAIN in all three of our runs	68.3%	83.9%	2819
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	139.5 %	171.4%	11755
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	114.3 %	140.4%	7886
pending pods, pod-minutes	shown, not judged	5480.4 %	6734.4%	18140835
utilisation (used / allocatable)	shown, not judged	15.8%	19.4%	151
CPU used (cores), mean	shown, not judged	14.7%	18.1%	131

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
CPU used with Omni's own (cores), mean	shown, not judged	14.6%	17.9%	129
energy per core-hour (Wh)	shown, not judged	16.7%	20.5%	168
HPA replicas, mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	14.3%	17.5%	123
host CPU busy, the real machine under kind (%)	shown, not judged	11.2%	13.8%	77
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS in all three of our runs	30.9%	38.0%	577
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	65.7%	80.7%	2604
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	276459.2%	339720.2%	46163914789
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	325048.7%	399428.1%	63817130129
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	732.2%	899.8%	323831

69.39 Koordinator 4W Orch (omni-v12, `V12_KOORDINATOR_4W_ORCH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.15%)	0.6%	0.8%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.33%)	1.0%	1.2%	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy (Wh)	STEADY, under the floor (the three runs' mean 0.33%)	1.0%	1.2%	1
response time (ms), mean	GAIN in all three of our runs	52.8%	64.8%	1682
response time (ms), 95th percentile	GAIN in all three of our runs	70.9%	87.2%	3040
response time (ms), 99th percentile	GAIN in all three of our runs	68.8%	84.5%	2856
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	138.9 %	170.6%	11646
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	114.3 %	140.4%	7886
pending pods, pod-minutes	shown, not judged	4845.4 %	5954.2%	14181041
utilisation (used / allocatable)	shown, not judged	15.1%	18.5%	137
CPU used (cores), mean	shown, not judged	15.1%	18.5%	137
CPU used with Omni's own (cores), mean	shown, not judged	15.1%	18.5%	137
energy per core-hour (Wh)	shown, not judged	14.8%	18.2%	133
HPA replicas, mean	the runs disagree	8.9%	11.0%	49
host CPU busy, the real machine under kind (%)	shown, not judged	11.6%	14.2%	81
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS in all three of our runs	31.1%	38.3%	587
batch: worker machines in service after the queue finished, mean	STEADY, the same	0.0%	0.0%	10
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	343570 .5%	422188.2%	712971593 56
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	351017 .8%	431339.6%	744215384 95

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	697.6 %	857.3%	293968

69.40 Koordinator 4W Top (omni-v12, `V12_KOORDINATOR_4W_TOP.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	5.1%	6.3%	16
node-hours	GAIN by the mean of our three runs, beyond the noise in 0 of 3	5.4%	6.6%	18
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.06%)	0.6%	0.8%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	3.9%	4.7%	9
response time (ms), mean	GAIN in all three of our runs	10.2%	12.6%	64
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 2 of 3	10.3%	12.6%	64
response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 0 of 3	11.5%	14.2%	81
time over the response line (% of samples)	the runs disagree	97.0%	119.2%	5681
pending pods, pod-minutes	shown, not judged	14.2%	17.4%	122
utilisation (used / allocatable)	shown, not judged	5.7%	7.0%	20
CPU used (cores), mean	shown, not judged	3.8%	4.7%	9
CPU used with Omni's own (cores), mean	shown, not judged	3.7%	4.6%	9
energy per core-hour (Wh)	shown, not judged	4.8%	5.9%	14
HPA replicas, mean	the runs disagree	11.7%	14.4%	84

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
host CPU busy, the real machine under kind (%)	shown, not judged	3.2%	4.0%	7
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	7.6%	9.4%	36
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	65.7%	80.7%	2604
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	48.9%	60.1%	1446
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	45.6%	56.0%	1257
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	11.3%	13.9%	78
queue: jobs unfinished at the window's end	the runs disagree	49.9%	61.3%	1505

69.41 Koordinator 4W Vs (omni-v12, `V12_KOORDINATOR_4W_VS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	19.3%	23.7%	226
node-hours	GAIN in all three of our runs	19.1%	23.5%	222
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.13%)	1.1%	1.4%	1
energy (Wh)	GAIN in all three of our runs	14.3%	17.6%	125
response time (ms), mean	LOSS by the mean of our three runs, beyond the noise in 2 of 3	128.8 %	158.3%	10021

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 95th percentile	LOSS in all three of our runs	379.0 %	465.7%	86753
response time (ms), 99th percentile	LOSS in all three of our runs	374.8 %	460.6%	84858
time over the response line (% of samples)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	6359.1 %	7814.2%	24424617
pending pods, pod-minutes	shown, not judged	50.1%	61.6%	1518
utilisation (used / allocatable)	shown, not judged	15.6%	19.1%	147
CPU used (cores), mean	shown, not judged	17.4%	21.4%	183
CPU used with Omni's own (cores), mean	shown, not judged	17.5%	21.5%	185
energy per core-hour (Wh)	shown, not judged	7.8%	9.6%	37
HPA replicas, mean	the runs disagree	11.1%	13.6%	74
host CPU busy, the real machine under kind (%)	shown, not judged	10.9%	13.4%	72
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	GAIN in all three of our runs	17.6%	21.6%	187
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	22.4%	27.6%	304
queue: job wait, submitted to its first pod running, mean (s)	GAIN in all three of our runs	64.4%	79.1%	2502
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN in all three of our runs	65.0%	79.9%	2555
queue: job turnaround, submitted to done, mean (s)	GAIN in all three of our runs	42.9%	52.7%	1110
queue: jobs unfinished at the window's end	GAIN by the mean of our three runs, beyond the noise in 2 of 3	161.7 %	198.7%	15788

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.42 Kuberay 4W Bare (omni-v12, `V12_KUBERAY_4W_BARE.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	18.7%	23.0%	191
node-hours	GAIN in all three of our runs	18.6%	22.9%	189
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.15%)	0.7%	0.9%	1
energy (Wh)	GAIN in all three of our runs	13.0%	15.9%	92
response time (ms), mean	GAIN in all three of our runs	7.0%	8.7%	28
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	9.8%	12.1%	53
response time (ms), 99th percentile	LOSS in all three of our runs	11.4%	14.0%	72
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	7.8%	9.6%	34
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	13.3%	16.4%	97
pending pods, pod-minutes	shown, not judged	31.7%	39.0%	547
utilisation (used / allocatable)	shown, not judged	26.4%	32.4%	379
CPU used (cores), mean	shown, not judged	5.6%	6.9%	18
CPU used with Omni's own (cores), mean	shown, not judged	5.6%	6.9%	18
energy per core-hour (Wh)	shown, not judged	21.6%	26.5%	254
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	14.3%	17.6%	113
host CPU busy, the real machine under kind (%)	shown, not judged	1.9%	2.4%	3
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
batch: queue finished (s)	LOSS in all three of our runs	0.9%	1.2%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	30.0%	36.9%	492
queue: job wait, submitted to its first pod running, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	250.7 %	308.3%	34213
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	422.6 %	519.7%	97227
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	10.2%	12.6%	57

69.43 Kuberay 4W Both (omni-v12, `V12_KUBERAY_4W_BOTH.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	2.8%	3.4%	5
node-hours	GAIN by the mean of our three runs, beyond the noise in 0 of 3	2.7%	3.4%	5
energy, parked workers still on at idle power (Wh)	power given in all three of our runs	2.1%	2.5%	3
energy (Wh)	power given in all three of our runs	2.9%	3.6%	5
response time (ms), mean	LOSS in all three of our runs	24.8%	30.5%	336
response time (ms), 95th percentile	the runs disagree	57.6%	70.8%	1806
response time (ms), 99th percentile	the runs disagree	63.9%	78.6%	2222
time over the response line (% of samples)	LOSS in all three of our runs	69.8%	85.8%	2649

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS in all three of our runs	78.7%	96.8%	3371
pending pods, pod-minutes	shown, not judged	46.1%	56.7%	1158
utilisation (used / allocatable)	shown, not judged	25.3%	31.1%	348
CPU used (cores), mean	shown, not judged	25.2%	31.0%	347
CPU used with Omni's own (cores), mean	shown, not judged	25.2%	31.0%	346
energy per core-hour (Wh)	shown, not judged	25.1%	30.9%	343
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 2 of 3	10.7%	13.2%	63
host CPU busy, the real machine under kind (%)	shown, not judged	15.8%	19.4%	136
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	LOSS in all three of our runs	14.4%	17.7%	113
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	321.3%	395.1%	56203
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	1184832.0%	1456873.6%	764093080716
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	524088.4%	644421.0%	149500222022
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	304.5%	374.4%	50477

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.44 Kuberay 4W Orch (omni-v12, `V12_KUBERAY_4W_ORCH.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.04%)	0.4%	0.5%	1
energy, parked workers still on at idle power (Wh)	power given in all three of our runs	2.0%	2.4%	3
energy (Wh)	power given in all three of our runs	2.0%	2.4%	3
response time (ms), mean	LOSS in all three of our runs	37.2%	45.8%	755
response time (ms), 95th percentile	the runs disagree	64.7%	79.5%	2278
response time (ms), 99th percentile	the runs disagree	63.6%	78.2%	2201
time over the response line (% of samples)	LOSS in all three of our runs	75.6%	93.0%	3111
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS in all three of our runs	80.7%	99.2%	3545
pending pods, pod-minutes	shown, not judged	47.7%	58.7%	1241
utilisation (used / allocatable)	shown, not judged	25.3%	31.1%	348
CPU used (cores), mean	shown, not judged	25.3%	31.1%	348
CPU used with Omni's own (cores), mean	shown, not judged	25.3%	31.1%	348
energy per core-hour (Wh)	shown, not judged	25.0%	30.7%	340
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	14.1%	17.3%	108
host CPU busy, the real machine under kind (%)	shown, not judged	17.5%	21.5%	167
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
batch: queue finished (s)	LOSS in all three of our runs	13.9%	17.1%	105
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	1138780.6%	1400248.6%	705850630730
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	458440.2%	563699.7%	114392632456
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	276.6%	340.2%	41655

69.45 Kuberay 4W Top (omni-v12, `V12_KUBERAY_4W_TOP.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	2.8%	3.4%	5
node-hours	GAIN by the mean of our three runs, beyond the noise in 0 of 3	2.6%	3.2%	4
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.01%)	0.6%	0.7%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	1.8%	2.1%	2
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	12.5%	15.4%	86
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	15.0%	18.4%	122
response time (ms), 99th percentile	STEADY, under the floor (the three runs' mean 0.17%)	16.4%	20.2%	147
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	9.5%	11.7%	50

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	2.7%	3.3%	4
pending pods, pod-minutes	shown, not judged	30.1%	37.0%	494
utilisation (used / allocatable)	shown, not judged	2.5%	3.1%	4
CPU used (cores), mean	shown, not judged	1.3%	1.6%	1
CPU used with Omni's own (cores), mean	shown, not judged	1.3%	1.6%	1
energy per core-hour (Wh)	shown, not judged	1.9%	2.4%	3
HPA replicas, mean	the runs disagree	15.6%	19.2%	133
pods started	the runs disagree	1083.4%	1332.2%	638914
pod start wait, total (s)	the runs disagree	1000.8%	1230.6%	545161
pod start wait, mean (s)	the runs disagree	705.9%	868.0%	271230
host CPU busy, the real machine under kind (%)	shown, not judged	1.4%	1.7%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.37%)	1.7%	2.0%	2
queue: job wait, submitted to its first pod running, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	12.2%	15.1%	82
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	10.1%	12.4%	56
queue: job turnaround, submitted to done, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	5.2%	6.4%	15

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: jobs unfinished at the window's end	the runs disagree	15.5%	19.1%	132

69.46 Kuberay 4W Vs (omni-v12, `V12_KUBERAY_4W_VS.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	18.7%	23.0%	191
node-hours	GAIN in all three of our runs	18.6%	22.9%	189
energy, parked workers still on at idle power (Wh)	GAIN in all three of our runs	1.5%	1.9%	2
energy (Wh)	GAIN in all three of our runs	13.5%	16.6%	99
response time (ms), mean	GAIN in all three of our runs	18.3%	22.5%	183
response time (ms), 95th percentile	the runs disagree	60.4%	74.2%	1983
response time (ms), 99th percentile	the runs disagree	69.6%	85.5%	2635
time over the response line (% of samples)	GAIN in all three of our runs	23.7%	29.1%	306
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	24.9%	30.6%	337
pending pods, pod-minutes	shown, not judged	247.2 %	303.9%	33253
utilisation (used / allocatable)	shown, not judged	14.6%	17.9%	116
CPU used (cores), mean	shown, not judged	13.6%	16.8%	101
CPU used with Omni's own (cores), mean	shown, not judged	13.7%	16.8%	102
energy per core-hour (Wh)	shown, not judged	13.4%	16.4%	98
HPA replicas, mean	the runs disagree	13.6%	16.7%	101

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pods started	power given by the mean of our three runs, beyond the noise in 0 of 3	576.0 %	708.2%	180563
pod start wait, total (s)	the runs disagree	677.7 %	833.4%	250010
pod start wait, mean (s)	the runs disagree	677.7 %	833.4%	250010
host CPU busy, the real machine under kind (%)	shown, not judged	8.6%	10.5%	40
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	GAIN in all three of our runs	7.3%	9.0%	30
queue: job wait, submitted to its first pod running, mean (s)	GAIN in all three of our runs	83.2%	102.2%	3764
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN in all three of our runs	69.3%	85.2%	2613
queue: job turnaround, submitted to done, mean (s)	GAIN in all three of our runs	26.5%	32.6%	384
queue: jobs unfinished at the window's end	GAIN in all three of our runs	91.7%	112.7%	4575

69.47 Kueue 4W Bare (omni-v12, `V12_KUEUE_4W_BARE.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	14.8%	18.2%	119
node-hours	GAIN in all three of our runs	14.6%	17.9%	116
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.03%)	0.4%	0.5%	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy (Wh)	GAIN in all three of our runs	10.2%	12.5%	57
response time (ms), mean	GAIN in all three of our runs	8.1%	10.0%	36
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	13.1%	16.1%	94
response time (ms), 99th percentile	the runs disagree	16.4%	20.2%	147
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	9.3%	11.5%	48
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	7.2%	8.9%	29
pending pods, pod-minutes	shown, not judged	32.5%	40.0%	577
utilisation (used / allocatable)	shown, not judged	16.4%	20.1%	147
CPU used (cores), mean	shown, not judged	4.2%	5.2%	10
CPU used with Omni's own (cores), mean	shown, not judged	4.3%	5.2%	10
energy per core-hour (Wh)	shown, not judged	9.8%	12.0%	52
HPA replicas, mean	the runs disagree	11.5%	14.1%	72
host CPU busy, the real machine under kind (%)	shown, not judged	3.0%	3.7%	5
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.47%)	1.4%	1.7%	2
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	26.5%	32.6%	383
queue: job wait, submitted to its first pod running, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	101.2 %	124.5%	5577
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	319.5 %	392.8%	55558

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	11.0%	13.6%	67

69.48 Kueue 4W Both (omni-v12, `V12_KUEUE_4W_BOTH.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	14.3%	17.6%	112
node-hours	GAIN in all three of our runs	14.2%	17.5%	111
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.08%)	0.4%	0.5%	1
energy (Wh)	GAIN in all three of our runs	9.9%	12.2%	54
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	5.9%	7.3%	20
response time (ms), 95th percentile	LOSS in all three of our runs	13.9%	17.1%	105
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 2 of 3	15.7%	19.3%	135
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	5.7%	7.0%	18
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	11.7%	14.4%	75
pending pods, pod-minutes	shown, not judged	33.9%	41.7%	627
utilisation (used / allocatable)	shown, not judged	15.7%	19.3%	135
CPU used (cores), mean	shown, not judged	2.8%	3.4%	5
CPU used with Omni's own (cores), mean	shown, not judged	2.8%	3.4%	5
energy per core-hour (Wh)	shown, not judged	11.0%	13.5%	66

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	14.6%	17.9%	116
host CPU busy, the real machine under kind (%)	shown, not judged	2.4%	3.0%	4
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	LOSS in all three of our runs	1.2%	1.4%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	25.8%	31.8%	364
queue: job wait, submitted to its first pod running, mean (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	323.9 %	398.2%	57095
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	319.5 %	392.8%	55558
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	12.9%	15.8%	91

69.49 Kueue 4W Orch (omni-v12, `V12_KUEUE_4W_ORCH.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.01%)	0.5%	0.6%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.03%)	0.7%	0.8%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.03%)	0.7%	0.8%	1
response time (ms), mean	LOSS in all three of our runs	10.0%	12.3%	55

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 95th percentile	LOSS in all three of our runs	17.2%	21.2%	162
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 2 of 3	20.1%	24.7%	220
time over the response line (% of samples)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	7.4%	9.1%	31
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	9.7%	11.9%	51
pending pods, pod-minutes	shown, not judged	26.1%	32.1%	371
utilisation (used / allocatable)	shown, not judged	2.8%	3.4%	5
CPU used (cores), mean	shown, not judged	2.8%	3.4%	5
CPU used with Omni's own (cores), mean	shown, not judged	2.8%	3.4%	5
energy per core-hour (Wh)	shown, not judged	3.3%	4.1%	6
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	9.0%	11.1%	44
host CPU busy, the real machine under kind (%)	shown, not judged	3.1%	3.8%	6
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	1.4%	1.7%	2
batch: worker machines in service after the queue finished, mean	STEADY, the same	0.0%	0.0%	9
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	310.0 %	381.2%	52303
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	422.6 %	519.7%	97227

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	15.6%	19.1%	132

69.50 Kueue 4W Top (omni-v12, `V12_KUEUE_4W_TOP.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	14.3%	17.6%	112
node-hours	GAIN in all three of our runs	14.4%	17.7%	114
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.05%)	0.6%	0.8%	1
energy (Wh)	GAIN in all three of our runs	10.0%	12.3%	55
response time (ms), mean	GAIN in all three of our runs	7.8%	9.6%	34
response time (ms), 95th percentile	the runs disagree	14.7%	18.1%	119
response time (ms), 99th percentile	the runs disagree	13.0%	16.0%	93
time over the response line (% of samples)	GAIN in all three of our runs	8.6%	10.5%	41
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	15.6%	19.2%	133
pending pods, pod-minutes	shown, not judged	25.0%	30.8%	341
utilisation (used / allocatable)	shown, not judged	17.8%	21.9%	173
CPU used (cores), mean	shown, not judged	3.8%	4.7%	8
CPU used with Omni's own (cores), mean	shown, not judged	3.9%	4.8%	9
energy per core-hour (Wh)	shown, not judged	13.2%	16.2%	95
HPA replicas, mean	the runs disagree	17.0%	20.9%	157

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pods started	power given by the mean of our three runs, beyond the noise in 0 of 3	971.7 %	1194.8%	513909
pod start wait, total (s)	the runs disagree	805.8 %	990.8%	353409
pod start wait, mean (s)	the runs disagree	540.4 %	664.5%	158957
host CPU busy, the real machine under kind (%)	shown, not judged	2.3%	2.8%	3
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.37%)	0.9%	1.1%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	25.8%	31.8%	364
queue: job wait, submitted to its first pod running, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	107.4 %	132.1%	6284
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	69.3%	85.2%	2614
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	8.9%	10.9%	43

69.51 Kueue 4W Vs (omni-v12, `V12_KUEUE_4W_VS.md`, 9 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	14.8%	18.2%	119
node-hours	GAIN in all three of our runs	15.0%	18.5%	124

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.06%)	0.5%	0.7%	1
energy (Wh)	GAIN in all three of our runs	10.6%	13.0%	61
response time (ms), mean	GAIN in all three of our runs	12.1%	14.9%	80
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 1 of 3	17.1%	21.0%	160
response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 1 of 3	13.9%	17.2%	106
time over the response line (% of samples)	GAIN in all three of our runs	12.8%	15.8%	90
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	7.9%	9.7%	34
pending pods, pod-minutes	shown, not judged	26.6%	32.7%	386
utilisation (used / allocatable)	shown, not judged	17.0%	20.9%	158
CPU used (cores), mean	shown, not judged	3.2%	3.9%	6
CPU used with Omni's own (cores), mean	shown, not judged	3.1%	3.9%	6
energy per core-hour (Wh)	shown, not judged	10.8%	13.3%	64
HPA replicas, mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	11.1%	13.7%	68
pods started	GAIN by the mean of our three runs, beyond the noise in 0 of 3	467.2 %	574.4%	118796
pod start wait, total (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	412.1 %	506.7%	92438
pod start wait, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	412.1 %	506.7%	92438
host CPU busy, the real machine under kind (%)	shown, not judged	1.4%	1.7%	2

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	9
batch: queue finished (s)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	1.2%	1.5%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	26.5%	32.6%	383
queue: job wait, submitted to its first pod running, mean (s)	GAIN in all three of our runs	66.9%	82.3%	2437
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN in all three of our runs	47.9%	58.9%	1248
queue: job turnaround, submitted to done, mean (s)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	12.3%	15.1%	83

69.52 Ocm 4W Bare (omni-v12, `V12_OCM_4W_BARE.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	19.7%	24.2%	235
node-hours	GAIN by the mean of our three runs, beyond the noise in 2 of 3	20.0%	24.6%	243
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.01%)	0.6%	0.8%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	14.5%	17.8%	127
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	6.7%	8.2%	28
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	13.6%	16.7%	113

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	15.5%	19.1%	146
time over the response line (% of samples)	the runs disagree	4.6%	5.7%	13
failed requests (%)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	314.5 %	386.5%	59758
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	16.4%	20.1%	163
pending pods, pod-minutes	shown, not judged	94.8%	116.6%	5435
utilisation (used / allocatable)	shown, not judged	23.7%	29.1%	340
CPU used (cores), mean	shown, not judged	6.8%	8.3%	28
CPU used with Omni's own (cores), mean	shown, not judged	6.8%	8.3%	28
energy per core-hour (Wh)	shown, not judged	16.2%	20.0%	160
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	16.5%	20.2%	164
pods started	power given by the mean of our three runs, beyond the noise in 1 of 3	1167.5 %	1434.7%	823328
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	1190.8 %	1463.3%	856526
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	657.4 %	807.8%	261025
host CPU busy, the real machine under kind (%)	shown, not judged	2.0%	2.4%	3
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	1.9%	2.3%	3
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	32.0%	39.3%	619
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	85.6%	105.2%	4427
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	83.9%	103.1%	4254
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	7.2%	8.9%	32

69.53 Ocm 4W Both (omni-v12, `V12_OCM_4W_BOTH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	15.5%	19.1%	146
node-hours	GAIN by the mean of our three runs, beyond the noise in 1 of 3	15.4%	19.0%	145
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.23%)	0.7%	0.8%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	11.3%	13.9%	77
response time (ms), mean	LOSS in all three of our runs	39.4%	48.4%	939
response time (ms), 95th percentile	LOSS in all three of our runs	45.6%	56.1%	1258
response time (ms), 99th percentile	LOSS in all three of our runs	45.2%	55.5%	1234
time over the response line (% of samples)	LOSS in all three of our runs	35.6%	43.8%	767

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
failed requests (%)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	2822.2 %	3468.0%	4810738
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS in all three of our runs	47.4%	58.2%	1358
pending pods, pod-minutes	shown, not judged	275.0 %	337.9%	45665
utilisation (used / allocatable)	shown, not judged	17.8%	21.9%	193
CPU used (cores), mean	shown, not judged	15.2%	18.6%	139
CPU used with Omni's own (cores), mean	shown, not judged	15.2%	18.6%	139
energy per core-hour (Wh)	shown, not judged	15.8%	19.4%	151
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	28.4%	34.9%	488
pods started	power given by the mean of our three runs, beyond the noise in 2 of 3	1153.3 %	1417.2%	803409
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	1407.6 %	1729.8%	1196813
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	814.5 %	1000.9%	400736
host CPU busy, the real machine under kind (%)	shown, not judged	6.8%	8.4%	29
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS in all three of our runs	11.1%	13.6%	74
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	22.8%	28.0%	315

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	23578.6%	28974.0%	335796134
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	24739.4%	30400.4%	369672847
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	41.4%	50.8%	1034

69.54 Ocm 4W Orch (omni-v12, `V12_OCM_4W_ORCH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.02%)	0.4%	0.5%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.07%)	1.7%	2.0%	2
energy (Wh)	STEADY, under the floor (the three runs' mean 0.07%)	1.7%	2.0%	2
response time (ms), mean	LOSS by the mean of our three runs, beyond the noise in 2 of 3	66.5%	81.8%	2674
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 2 of 3	73.2%	90.0%	3238
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 2 of 3	71.4%	87.7%	3077
time over the response line (% of samples)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	72.5%	89.1%	3177
failed requests (%)	the runs disagree	1843.5%	2265.4%	2052766

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	75.1%	92.3%	3411
pending pods, pod-minutes	shown, not judged	305.9 %	375.9%	56518
utilisation (used / allocatable)	shown, not judged	30.2%	37.2%	553
CPU used (cores), mean	shown, not judged	30.2%	37.2%	553
CPU used with Omni's own (cores), mean	shown, not judged	30.2%	37.2%	553
energy per core-hour (Wh)	shown, not judged	112.4 %	138.1%	7632
HPA replicas, mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	9.2%	11.3%	52
pods started	GAIN by the mean of our three runs, beyond the noise in 0 of 3	314.5 %	386.5%	59758
pod start wait, total (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	314.5 %	386.5%	59758
pod start wait, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	314.5 %	386.5%	59758
host CPU busy, the real machine under kind (%)	shown, not judged	30.5%	37.5%	564
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS in all three of our runs	15.8%	19.4%	152
batch: worker machines in service after the queue finished, mean	STEADY, the same	0.0%	0.0%	10
queue: job wait, submitted to its first pod running, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	11078. 1%	13613.1%	74126399

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	12752.7%	15670.9%	98230285
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	38.0%	46.6%	871

69.55 Ocm 4W Top (omni-v12, `V12_OCM_4W_TOP.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	15.5%	19.1%	146
node-hours	GAIN by the mean of our three runs, beyond the noise in 1 of 3	15.4%	19.0%	145
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.31%)	1.1%	1.4%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	11.2%	13.8%	77
response time (ms), mean	the runs disagree	51.2%	62.9%	1584
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	55.0%	67.6%	1829
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	52.7%	64.8%	1680
time over the response line (% of samples)	the runs disagree	68.3%	83.9%	2816
failed requests (%)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	651.0%	800.0%	255984

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS in all three of our runs	55.2%	67.9%	1844
pending pods, pod-minutes	shown, not judged	188.6 %	231.7%	21482
utilisation (used / allocatable)	shown, not judged	21.4%	26.3%	277
CPU used (cores), mean	shown, not judged	22.4%	27.5%	302
CPU used with Omni's own (cores), mean	shown, not judged	22.4%	27.5%	303
energy per core-hour (Wh)	shown, not judged	73.4%	90.2%	3256
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	32.6%	40.1%	644
host CPU busy, the real machine under kind (%)	shown, not judged	29.1%	35.8%	514
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	9.7%	11.9%	57
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 1 of 3	22.8%	28.0%	315
queue: job wait, submitted to its first pod running, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	526.2 %	646.6%	167244
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	2098.2 %	2578.3%	2658990
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	17.8%	21.9%	192

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.56 Ocm 4W Vs (omni-v12, `V12_OCM_4W_VS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	19.7%	24.2%	235
node-hours	GAIN by the mean of our three runs, beyond the noise in 2 of 3	20.1%	24.6%	244
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.06%)	1.8%	2.3%	3
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	15.2%	18.7%	141
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	57.6%	70.7%	2002
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 2 of 3	65.8%	80.8%	2614
response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 2 of 3	67.1%	82.5%	2723
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	61.7%	75.8%	2299
failed requests (%)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	314.5 %	386.5%	59758
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	59.2%	72.8%	2118
pending pods, pod-minutes	shown, not judged	113.8 %	139.9%	7829
utilisation (used / allocatable)	shown, not judged	37.1%	45.6%	834
CPU used (cores), mean	shown, not judged	38.0%	46.7%	872
CPU used with Omni's own (cores), mean	shown, not judged	38.0%	46.7%	872

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy per core-hour (Wh)	shown, not judged	80.2%	98.5%	3883
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 2 of 3	11.7%	14.4%	83
host CPU busy, the real machine under kind (%)	shown, not judged	29.6%	36.3%	529
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	GAIN in all three of our runs	12.8%	15.7%	99
batch: worker machines in service after the queue finished, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	34.9%	42.9%	738
queue: job wait, submitted to its first pod running, mean (s)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	235.0 %	288.8%	33366
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	229.4 %	281.9%	31784
queue: job turnaround, submitted to done, mean (s)	GAIN in all three of our runs	26.5%	32.6%	426

69.57 Robust Kill (omni-v21, `V21_ROBUST_KILL.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.09%)	0.6%	0.7%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.14%)	0.6%	0.8%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.14%)	0.6%	0.8%	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), mean	GAIN in all three of our runs	32.5%	39.9%	637
response time (ms), 95th percentile	GAIN in all three of our runs	53.8%	66.1%	1750
response time (ms), 99th percentile	the runs disagree	59.1%	72.6%	2111
time over the response line (% of samples)	GAIN in all three of our runs	27.8%	34.1%	466
failed requests (%)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	20.0%	24.6%	242
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	18.3%	22.4%	202
pending pods, pod-minutes	shown, not judged	420.6 %	516.8%	106851
utilisation (used / allocatable)	shown, not judged	1.1%	1.4%	1
CPU used (cores), mean	shown, not judged	1.1%	1.4%	1
CPU used with Omni's own (cores), mean	shown, not judged	1.1%	1.4%	1
energy per core-hour (Wh)	shown, not judged	1.7%	2.1%	2
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 2 of 3	6.8%	8.4%	29
pods started	the runs disagree	90.1%	110.7%	4904
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	121.7 %	149.6%	8947
pod start wait, mean (s)	the runs disagree	37.9%	46.6%	870
host CPU busy, the real machine under kind (%)	shown, not judged	0.9%	1.1%	1
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
robust: failed requests in the 300 s after the kill (%)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	14.2%	17.5%	123

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
robust: time over the line in the 300 s after the kill (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	5.3%	6.5%	17

69.58 Robust Long (omni-v21, `V21_ROBUST_LONG.md`, 3 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.01%)	0.3%	0.4%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.20%)	0.6%	0.7%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.20%)	0.6%	0.7%	1
response time (ms), mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	106.0 %	131.4%	2072
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 0 of 3	150.7 %	186.9%	4191
response time (ms), 99th percentile	the runs disagree	96.2%	119.3%	1709
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	111.1 %	137.8%	2279
failed requests (%)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	65.0%	80.6%	781
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	62.8%	77.9%	729
pending pods, pod-minutes	shown, not judged	229.3 %	284.4%	9706
utilisation (used / allocatable)	shown, not judged	4.3%	5.3%	4

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
CPU used (cores), mean	shown, not judged	4.3%	5.3%	4
CPU used with Omni's own (cores), mean	shown, not judged	4.3%	5.3%	4
energy per core-hour (Wh)	shown, not judged	8.6%	10.6%	14
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	130.0 %	161.2%	3120
pods started	power given by the mean of our three runs, beyond the noise in 1 of 3	572.8 %	710.3%	60551
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	709.0 %	879.2%	92751
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	158.3 %	196.3%	4627
host CPU busy, the real machine under kind (%)	shown, not judged	3.2%	3.9%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	3

69.59 Steady (omni-v21, `V21_STEADY.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.04%)	0.9%	1.2%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.18%)	1.0%	1.2%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.18%)	1.0%	1.2%	1
response time (ms), mean	GAIN in all three of our runs	24.9%	30.7%	376

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
response time (ms), 95th percentile	GAIN in all three of our runs	32.0%	39.3%	618
response time (ms), 99th percentile	GAIN in all three of our runs	41.0%	50.4%	1016
time over the response line (% of samples)	GAIN in all three of our runs	104.5 %	128.4%	6593
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	114.3 %	140.4%	7891
pending pods, pod-minutes	shown, not judged	406.1 %	499.0%	99597
utilisation (used / allocatable)	shown, not judged	4.2%	5.2%	11
CPU used (cores), mean	shown, not judged	4.2%	5.2%	11
CPU used with Omni's own (cores), mean	shown, not judged	4.1%	5.0%	11
energy per core-hour (Wh)	shown, not judged	4.4%	5.4%	12
HPA replicas, mean	GAIN by the mean of our three runs, beyond the noise in 0 of 3	12.1%	14.9%	89
pods started	GAIN in all three of our runs	26.8%	32.9%	433
pod start wait, total (s)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	34.5%	42.4%	720
pod start wait, mean (s)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	25.3%	31.1%	386
host CPU busy, the real machine under kind (%)	shown, not judged	4.2%	5.2%	11
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.60 Trace Google2011 (omni-v21, `V21_TRACE_GOOGLE2011.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.01%)	0.3%	0.3%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.36%)	0.3%	0.4%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.36%)	0.3%	0.4%	1
response time (ms), mean	GAIN in all three of our runs	32.4%	39.8%	634
response time (ms), 95th percentile	GAIN in all three of our runs	37.1%	45.5%	831
response time (ms), 99th percentile	GAIN in all three of our runs	40.2%	49.4%	977
time over the response line (% of samples)	GAIN in all three of our runs	83.6%	102.8%	4225
failed requests (%)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	20.5%	25.1%	253
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	68.2%	83.8%	2806
pending pods, pod-minutes	shown, not judged	219.7 %	269.9%	29147
utilisation (used / allocatable)	shown, not judged	1.6%	1.9%	2
CPU used (cores), mean	shown, not judged	1.6%	1.9%	2
CPU used with Omni's own (cores), mean	shown, not judged	1.5%	1.9%	2
energy per core-hour (Wh)	shown, not judged	1.6%	2.0%	2
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 1 of 3	25.4%	31.2%	390

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pods started	power given in all three of our runs	99.8%	122.6%	6013
pod start wait, total (s)	LOSS in all three of our runs	137.0 %	168.4%	11339
pod start wait, mean (s)	the runs disagree	39.5%	48.5%	943
host CPU busy, the real machine under kind (%)	shown, not judged	1.5%	1.9%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.61 Vanilla (omni-v12, `V12_VANILLA.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	17.7%	21.7%	189
node-hours	GAIN in all three of our runs	17.8%	21.9%	192
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.15%)	0.8%	1.0%	1
energy (Wh)	GAIN in all three of our runs	12.6%	15.5%	97
response time (ms), mean	GAIN in all three of our runs	6.0%	7.4%	22
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	9.1%	11.2%	51
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 2 of 3	6.5%	8.0%	26
time over the response line (% of samples)	the runs disagree	33.6%	41.2%	681
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	45.5%	55.9%	1250
pending pods, pod-minutes	shown, not judged	56.7%	69.7%	1942

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
utilisation (used / allocatable)	shown, not judged	22.0%	27.1%	293
CPU used (cores), mean	shown, not judged	9.8%	12.1%	59
CPU used with Omni's own (cores), mean	shown, not judged	9.9%	12.1%	59
energy per core-hour (Wh)	shown, not judged	13.0%	16.0%	103
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	14.6%	18.0%	130
pods started	the runs disagree	409.4 %	503.1%	101256
pod start wait, total (s)	the runs disagree	378.0 %	464.5%	86317
pod start wait, mean (s)	the runs disagree	303.1 %	372.5%	55509
host CPU busy, the real machine under kind (%)	shown, not judged	0.8%	0.9%	1
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.12%)	0.3%	0.4%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	28.3%	34.8%	485
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	188.2 %	231.3%	21395
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	1.1%	1.3%	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.62 Volcano 4W Bare (omni-v12, `V12_VOLCANO_4W_BARE.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	16.6%	20.4%	167
node-hours	GAIN by the mean of our three runs, beyond the noise in 2 of 3	16.6%	20.4%	167
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.07%)	0.5%	0.6%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	11.7%	14.4%	83
response time (ms), mean	GAIN in all three of our runs	7.8%	9.6%	37
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	10.5%	12.9%	68
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 1 of 3	15.7%	19.3%	149
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	12.6%	15.4%	96
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	20.7%	25.5%	260
pending pods, pod-minutes	shown, not judged	30.6%	37.6%	567
utilisation (used / allocatable)	shown, not judged	21.3%	26.2%	275
CPU used (cores), mean	shown, not judged	3.9%	4.8%	10
CPU used with Omni's own (cores), mean	shown, not judged	3.9%	4.8%	10
energy per core-hour (Wh)	shown, not judged	11.9%	14.6%	86
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	16.8%	20.6%	170

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pods started	GAIN by the mean of our three runs, beyond the noise in 0 of 3	468.9 %	576.2%	132795
pod start wait, total (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	468.9 %	576.2%	132795
pod start wait, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	468.9 %	576.2%	132795
host CPU busy, the real machine under kind (%)	shown, not judged	1.8%	2.2%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.41%)	0.8%	1.0%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	27.3%	33.6%	452
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	150.3 %	184.7%	13649
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	335.7 %	412.5%	68058
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 2 of 3	10.4%	12.8%	66

69.63 Volcano 4W Both (omni-v12, `V12_VOLCANO_4W_BOTH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	18.8%	23.1%	214
node-hours	GAIN in all three of our runs	18.9%	23.2%	216
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.06%)	0.5%	0.7%	1

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	13.3%	16.4%	108
response time (ms), mean	GAIN in all three of our runs	20.3%	24.9%	249
response time (ms), 95th percentile	GAIN by the mean of our three runs, beyond the noise in 2 of 3	30.2%	37.1%	550
response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 1 of 3	31.3%	38.5%	594
time over the response line (% of samples)	GAIN in all three of our runs	50.9%	62.6%	1567
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	26.5%	32.6%	425
pending pods, pod-minutes	shown, not judged	52.2%	64.2%	1649
utilisation (used / allocatable)	shown, not judged	22.1%	27.2%	296
CPU used (cores), mean	shown, not judged	3.6%	4.5%	9
CPU used with Omni's own (cores), mean	shown, not judged	3.7%	4.5%	9
energy per core-hour (Wh)	shown, not judged	12.4%	15.3%	94
HPA replicas, mean	the runs disagree	15.8%	19.4%	152
pods started	power given by the mean of our three runs, beyond the noise in 0 of 3	629.1 %	773.0%	239031
pod start wait, total (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	1174.3 %	1443.0%	832918
pod start wait, mean (s)	LOSS by the mean of our three runs, beyond the noise in 0 of 3	1174.3 %	1443.0%	832918
host CPU busy, the real machine under kind (%)	shown, not judged	1.7%	2.0%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
batch: queue finished (s)	LOSS in all three of our runs	1.9%	2.4%	3
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	30.8%	37.8%	572
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	597.7 %	734.5%	215792
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	571.9 %	702.7%	197533
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	26.0%	32.0%	409

69.64 Volcano 4W Orch (omni-v12, `V12_VOLCANO_4W_ORCH.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.10%)	0.5%	0.6%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.09%)	0.5%	0.7%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.09%)	0.5%	0.7%	1
response time (ms), mean	GAIN in all three of our runs	23.4%	28.8%	331
response time (ms), 95th percentile	GAIN in all three of our runs	33.0%	40.6%	659
response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 1 of 3	30.4%	37.4%	559
time over the response line (% of samples)	GAIN in all three of our runs	47.6%	58.5%	1369

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN by the mean of our three runs, beyond the noise in 1 of 3	33.6%	41.2%	681
pending pods, pod-minutes	shown, not judged	45.8%	56.3%	1267
utilisation (used / allocatable)	shown, not judged	2.2%	2.8%	4
CPU used (cores), mean	shown, not judged	2.2%	2.8%	4
CPU used with Omni's own (cores), mean	shown, not judged	2.2%	2.8%	4
energy per core-hour (Wh)	shown, not judged	2.5%	3.1%	4
HPA replicas, mean	the runs disagree	12.2%	14.9%	90
pods started	GAIN by the mean of our three runs, beyond the noise in 0 of 3	367.0 %	450.9%	81337
pod start wait, total (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	367.0 %	450.9%	81337
pod start wait, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	468.9 %	576.2%	132795
host CPU busy, the real machine under kind (%)	shown, not judged	1.1%	1.3%	1
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	LOSS in all three of our runs	1.6%	1.9%	2
batch: worker machines in service after the queue finished, mean	STEADY, the same	0.0%	0.0%	10
queue: job wait, submitted to its first pod running, mean (s)	LOSS in all three of our runs	565.5 %	694.9%	193152
queue: job wait, submitted to its first pod running, 95th percentile (s)	LOSS in all three of our runs	273.7 %	336.3%	45234
queue: job turnaround, submitted to done, mean (s)	LOSS in all three of our runs	22.6%	27.8%	310

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.65 Volcano 4W Top (omni-v12, `V12_VOLCANO_4W_TOP.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN in all three of our runs	18.8%	23.1%	214
node-hours	GAIN in all three of our runs	18.6%	22.9%	210
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.02%)	0.5%	0.7%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	13.0%	16.0%	103
response time (ms), mean	GAIN in all three of our runs	14.1%	17.3%	120
response time (ms), 95th percentile	LOSS by the mean of our three runs, beyond the noise in 0 of 3	30.6%	37.6%	567
response time (ms), 99th percentile	the runs disagree	38.0%	46.7%	874
time over the response line (% of samples)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	56.5%	69.4%	1927
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	the runs disagree	21.6%	26.5%	282
pending pods, pod-minutes	shown, not judged	26.5%	32.5%	423
utilisation (used / allocatable)	shown, not judged	22.0%	27.1%	293
CPU used (cores), mean	shown, not judged	3.2%	3.9%	7
CPU used with Omni's own (cores), mean	shown, not judged	3.2%	4.0%	7
energy per core-hour (Wh)	shown, not judged	12.5%	15.4%	95
HPA replicas, mean	the runs disagree	14.5%	17.8%	127
host CPU busy, the real machine under kind (%)	shown, not judged	1.4%	1.7%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
batch: queue finished (s)	STEADY, under the floor (the three runs' mean 0.41%)	0.9%	1.1%	1
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	30.8%	37.8%	572
queue: job wait, submitted to its first pod running, mean (s)	the runs disagree	43.0%	52.9%	1119
queue: job wait, submitted to its first pod running, 95th percentile (s)	the runs disagree	103.9 %	127.7%	6522
queue: job turnaround, submitted to done, mean (s)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	8.8%	10.8%	47

69.66 Volcano 4W Vs (omni-v12, `V12_VOLCANO_4W_VS.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	GAIN by the mean of our three runs, beyond the noise in 2 of 3	16.6%	20.4%	167
node-hours	GAIN by the mean of our three runs, beyond the noise in 2 of 3	16.4%	20.1%	163
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.15%)	0.5%	0.6%	1
energy (Wh)	GAIN by the mean of our three runs, beyond the noise in 2 of 3	11.4%	14.1%	80
response time (ms), mean	LOSS by the mean of our three runs, beyond the noise in 1 of 3	26.9%	33.0%	436
response time (ms), 95th percentile	LOSS in all three of our runs	63.9%	78.5%	2464
response time (ms), 99th percentile	LOSS by the mean of our three runs, beyond the noise in 2 of 3	58.1%	71.4%	2039

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
time over the response line (% of samples)	LOSS in all three of our runs	101.0 %	124.1%	6165
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	LOSS by the mean of our three runs, beyond the noise in 1 of 3	34.8%	42.7%	731
pending pods, pod-minutes	shown, not judged	25.4%	31.2%	390
utilisation (used / allocatable)	shown, not judged	22.0%	27.0%	292
CPU used (cores), mean	shown, not judged	3.2%	4.0%	7
CPU used with Omni's own (cores), mean	shown, not judged	3.3%	4.1%	7
energy per core-hour (Wh)	shown, not judged	12.8%	15.8%	100
HPA replicas, mean	the runs disagree	16.3%	20.0%	160
host CPU busy, the real machine under kind (%)	shown, not judged	1.2%	1.5%	1
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10
batch: queue finished (s)	GAIN in all three of our runs	1.4%	1.8%	2
batch: worker machines in service after the queue finished, mean	GAIN in all three of our runs	27.3%	33.6%	452
queue: job wait, submitted to its first pod running, mean (s)	GAIN in all three of our runs	29.1%	35.7%	510
queue: job wait, submitted to its first pod running, 95th percentile (s)	GAIN in all three of our runs	76.6%	94.1%	3542
queue: job turnaround, submitted to done, mean (s)	GAIN in all three of our runs	13.9%	17.1%	117

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

69.67 Wandering (omni-v21, `V21_WANDERING.md`, 10 paired repetitions a run)

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
worker nodes in service, mean	STEADY, the same	0.0%	0.0%	1
node-hours	STEADY, under the floor (the three runs' mean 0.03%)	0.3%	0.4%	1
energy, parked workers still on at idle power (Wh)	STEADY, under the floor (the three runs' mean 0.32%)	0.3%	0.4%	1
energy (Wh)	STEADY, under the floor (the three runs' mean 0.32%)	0.3%	0.4%	1
response time (ms), mean	GAIN in all three of our runs	20.0%	24.6%	242
response time (ms), 95th percentile	GAIN in all three of our runs	25.4%	31.2%	390
response time (ms), 99th percentile	GAIN by the mean of our three runs, beyond the noise in 1 of 3	63.6%	78.2%	2445
time over the response line (% of samples)	GAIN in all three of our runs	23.5%	28.9%	335
failed requests (%)	GAIN in all three of our runs	14.3%	17.6%	125
alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line)	GAIN in all three of our runs	9.8%	12.1%	59
pending pods, pod-minutes	shown, not judged	282.4 %	347.0%	48153
utilisation (used / allocatable)	shown, not judged	1.4%	1.7%	2
CPU used (cores), mean	shown, not judged	1.4%	1.7%	2
CPU used with Omni's own (cores), mean	shown, not judged	1.4%	1.7%	2
energy per core-hour (Wh)	shown, not judged	3.6%	4.5%	8
HPA replicas, mean	power given by the mean of our three runs, beyond the noise in 0 of 3	14.9%	18.3%	135

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Measure	Reading	MDE, one run	MDE, all three runs	Repetitions for 5%
pods started	the runs disagree	66.8%	82.1%	2696
pod start wait, total (s)	the runs disagree	83.1%	102.1%	4173
pod start wait, mean (s)	GAIN by the mean of our three runs, beyond the noise in 0 of 3	36.0%	44.2%	781
host CPU busy, the real machine under kind (%)	shown, not judged	1.4%	1.7%	2
host cores (the real machine under kind)	shown, not judged	0.0%	0.0%	10

69.68 What the STEADY rows mean

157 rows read STEADY. In 2 of them the test could not resolve a change under 10% of native in all three runs, so STEADY there means "no change larger than the MDE shown", and nothing finer is claimed.

CHAPTER 70

Every Change to Every Preregistration

Generated by `python3 tools/deviations.py` from the preregistrations in `docs/`; `tools/deviations.py --check`, run by `verify.py`, fails a push where this register is not what they give. Each amendment's reason, and what it changed, is written in the preregistration it amends; this register is the index to all of them, newest first. The timing is the amendment's own statement in its heading. Earlier windows, runs and tables keep the rule they ran under, and a three-run table never pools runs from either side of an amendment that changes what it reads.

98 amendments in 18 preregistrations: 78 before the runs it governs; 13 not stated in its heading; 7 after runs began.

Date	Preregistration	Amendment	Timing
2026-11-01	`docs/GPU_PREREGISTRATION.md`	Amendment 16 (2026-11-01, declared before any run of it): the stacks with the card fully loaded	before the runs it governs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Preregistration	Amendment	Timing
2026-10-31	docs/SOLO_PREREGISTRATION.md	Amendment 2 (31 October 2026, written before the runs it governs): Omni direct keeps every muscle Omni-Compass cannot be	before the runs it governs
2026-10-31	docs/SOLO_PREREGISTRATION.md	Amendment 1 (31 October 2026, written before the runs it governs): the arm's pod writes were refused by the rig's own identity	before the runs it governs
2026-10-31	docs/PACE_PREREGISTRATION.md	Amendment 6 (31 October, night): the model's stand-in corrected; the ladder read again on it, in the cloud and on site	not stated in its heading
2026-10-31	docs/PACE_PREREGISTRATION.md	Amendment 5 (31 October, written before the ladder's cloud half runs on it): the cloud setting at the max	before the runs it governs
2026-10-31	docs/PACE_PREREGISTRATION.md	Amendment 4 (31 October, written before any reading of the ladder was seen): the factory beat read again on the engine that stands	before the runs it governs
2026-10-31	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (31 October 2026, written before any set at the max starts): every cloud set runs the whole engine at the max; the machines alone of 14 October struck	before the runs it governs
2026-10-31	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (31 October 2026, night, written before any set with the work meter runs): the cloud's sets read the work per dollar	before the runs it governs
2026-10-31	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (31 October 2026, night, written before any counted set with the work meter runs): a cloud set must show that Omni-Compass holds	before the runs it governs
2026-10-31	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (31 October 2026, night): the Omni index scores the software end; energy that no meter read is shown, never scored	not stated in its heading
Date	**Preregistration**	**Amendment**	**Timing**
2026-10-30	docs/YCSB_PREREGISTRATION.md	Amendment 6 (2026-10-30, declared before any run on it): the max on the cache, through	before the runs it governs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Preregistration	Amendment	Timing
		the server's own account of a miss (the next engine's candidate)	
2026-10-30	`docs/REDIS_PREREGISTRATION.md`	Amendment 5 (2026-10-30, declared before any run on it): the max on the cache's ceiling, with the working set's floor (the next engine's candidate)	before the runs it governs
2026-10-30	`docs/POSTGRES_PREREGISTRATION.md`	Amendment 4 (2026-10-30, declared before any run on it): the max on the pooler's pool (the next engine's candidate)	before the runs it governs
2026-10-30	`docs/MYSQL_PREREGISTRATION.md`	Amendment 6 (2026-10-30, declared before any run on it): the max on the buffer pool, through the server's own account of a miss (the next engine's candidate)	before the runs it governs
2026-10-30	`docs/KAFKA_PREREGISTRATION.md`	Amendment 4 (2026-10-30, declared before any run on it): the group's twin and the immune reflex (the next engine's candidate)	before the runs it governs
2026-10-30	`docs/GPU_PREREGISTRATION.md`	Amendment 15 (2026-10-30, the rig, declared before any counted run on it): the window opens once the governor stands	before the runs it governs
2026-10-25	`docs/ORCHESTRATORS_PREREGISTRATION.md`	Amendment 2: the Cluster Autoscaler acts after the scheduler's pass (written before the third run, 25 October 2026)	before the runs it governs
2026-10-25	`docs/ORCHESTRATORS_PREREGISTRATION.md`	Amendment 1: the response time and the pods waiting, corrected in the model (written before the second run, 25 October 2026)	before the runs it governs
2026-10-25	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment (25 October 2026): the ladder is 1, 10, 100 and 500	not stated in its heading
2026-10-24	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment (2026-10-24, written before any run of it): Google's and Amazon's sets to the cap under the same stopping rule	before the runs it governs
2026-10-22	`docs/PACE_PREREGISTRATION.md`	Amendment 3 (22 October, afternoon, written before any omni-v17 run): the beat a dial, the main engine at 60 s	before the runs it governs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Preregistration	Amendment	Timing
2026-10-22	docs/PACE_PREREGISTRATION.md	Amendment 2 (22 October, written before any run): the founder sets the heartbeat at 15 s	before the runs it governs
2026-10-22	docs/PACE_PREREGISTRATION.md	Amendment 1 (22 October, written before any run): how rule 1 reads the model	before the runs it governs
2026-10-22	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (2026-10-22, omni-v17, written before any run of it): the main engine every 60 s, the beat a dial	before the runs it governs
2026-10-22	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (2026-10-22, omni-v16, written before any run of it): the full decision every 15 s	before the runs it governs
2026-10-21	docs/YCSB_PREREGISTRATION.md	Amendment 5 (2026-10-21, declared before any run on it): the machine's operating system, Linux 6.8	before the runs it governs
2026-10-21	docs/GROUP_PREREGISTRATION.md	Amendment 3: what the counted runs of OCM and Armada found (written after those runs, 21 October 2026)	after runs began
2026-10-20	docs/GROUP_PREREGISTRATION.md	Amendment 2: Armada, frozen (written before its first run, 20 October 2026)	before the runs it governs
2026-10-20	docs/GROUP_PREREGISTRATION.md	Amendment 1: Open Cluster Management, frozen (written before its first run, 20 October 2026)	before the runs it governs
2026-10-20	docs/CRANE_PREREGISTRATION.md	Amendment 1: what the trial found in the rig (written after the trial of 20 October 2026, before the counted runs)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 9: what the trials of 20 October found in the rigs (written after the trials, before the counted runs)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 8: the Gödel scheduler (written before its first run, 20 October 2026)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 7: KubeRay (written before its first run, 20 October 2026)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 6: the four ways (written before their first run, 20 October 2026)	before the runs it governs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Preregistration	Amendment	Timing
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 5: the governor's own core (written before any run on it, 20 October 2026)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 4: Koordinator's runs repeated on the kubelet's cgroup root its own tests use (written before they were repeated, 20 October 2026)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 3: the Kubernetes scheduler alone, as shipped and with its bin-packing profile (written before its first run, 20 October 2026)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 2: Koordinator (written before its first run, 20 October 2026)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 1: the KAI scheduler (written before its first run, 20 October 2026)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 11: KubeRay's two LOSS rows root-caused (written from the runs' own files, 20 October 2026, before any run on the engine that fixes them)	before the runs it governs
2026-10-20	docs/BATCH_QUEUE_PREREGISTRATION.md	Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again)	before the runs it governs
2026-10-19	docs/SWARM_PREREGISTRATION.md	Amendment 2 (2026-10-19, before omni-v12's runs; written after the omni-v11 runs and the reflex study were seen, and said so)	after runs began
2026-10-19	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (2026-10-19): a worker held by an add-on's own mark while it serves is in service	not stated in its heading
2026-10-19	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (2026-10-19): a paid cloud set runs to its cap and stops when its bill is read (written before any run of it)	before the runs it governs
2026-10-18	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (2026-10-18, omni-v11): the governor holds on a reading with no machine in service	not stated in its heading

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Preregistration	Amendment	Timing
2026-10-18	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (2026-10-18): the extended fault test, the two faults that blind a governor (written before its run)	before the runs it governs
2026-10-18	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (2026-10-18): on kind, the served image pulled onto every worker before each arm	not stated in its heading
2026-10-18	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment (2026-10-18): a window with no machine in service is refused in either arm; the add-on's set-up waits for its scaler	not stated in its heading
2026-10-18	docs/CRUISEKUBE_PREREGISTRATION.md	Amendment 2: a window with no machine in service is refused in either arm (18 October, omni-v10 reruns)	after runs began
2026-10-18	docs/CITYLEARN_PREREGISTRATION.md	Amendment of 18 October (omni-v10): the glide that cuts the bill most	not stated in its heading
2026-10-17	docs/YCSB_PREREGISTRATION.md	Amendment 4 (2026-10-17, declared before any run on it): native as shipped, beside the operator's cache	before the runs it governs
2026-10-17	docs/REDIS_PREREGISTRATION.md	Amendment 4 (2026-10-17, declared before any run on it): native as shipped, beside the operator's ceiling	before the runs it governs
2026-10-17	docs/MYSQL_PREREGISTRATION.md	Amendment 5 (2026-10-17, declared before any run on it): native as shipped, beside the operator's pool	before the runs it governs
2026-10-17	docs/GPU_PREREGISTRATION.md	Amendment 14 (2026-10-17, wording only, before any trial on this code)	before the runs it governs
2026-10-17	docs/CITYLEARN_PREREGISTRATION.md	Amendment of 17 October, night: one key for every score	not stated in its heading
2026-10-17	docs/CITYLEARN_PREREGISTRATION.md	Amendment of 17 October (omni-v9): the battery wire reads the coming hour	not stated in its heading
2026-10-17	docs/CITYLEARN_PREREGISTRATION.md	Amendment of 17 October (omni-v8): the battery that carries its building through an outage stays native	not stated in its heading
2026-10-16	docs/REALMS_PREREGISTRATION.md	Amendment (2026-10-16): the spine, the seventh organism (written before its run)	before the runs it governs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Preregistration	Amendment	Timing
2026-10-16	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment (2026-10-16): the spine, the seventh organism with the cluster inside	not stated in its heading
2026-10-13	`docs/GPU_PREREGISTRATION.md`	Amendment 13 (2026-10-13, before any trial on this code)	before the runs it governs
2026-10-13	`docs/CRUISEKUBE_PREREGISTRATION.md`	Amendment 1: the trial run read, the chart pinned, and a decided window with no write counted (13 October, before the counted runs)	before the runs it governs
2026-10-12	`docs/CAPACITY_PREREGISTRATION.md`	Amendment 1 (12 October 2026, after the tuning case, before run A)	before the runs it governs
2026-10-10	`docs/YCSB_PREREGISTRATION.md`	Amendment 3 (2026-10-10 13:05 UTC, after the first counted set on amendment 2, before the next): a spend trial runs to its samples	before the runs it governs
2026-10-10	`docs/REDIS_PREREGISTRATION.md`	Amendment 3 (2026-10-10 13:05 UTC, after the first counted set on amendment 2, before the next): a spend trial runs to its samples	before the runs it governs
2026-10-10	`docs/POSTGRES_PREREGISTRATION.md`	Amendment 4 (2026-10-10 13:05 UTC, after the first counted set on amendment 3, before the next): a spend trial runs to its samples	before the runs it governs
2026-10-10	`docs/MYSQL_PREREGISTRATION.md`	Amendment 4 (2026-10-10 13:05 UTC, after the first counted set on amendment 3, before the next): a spend trial runs to its samples	before the runs it governs
2026-10-10	`docs/KAFKA_PREREGISTRATION.md`	Amendment 3 (2026-10-10 13:05 UTC, after the first counted set on amendment 2, before the next): a spend trial runs to its samples	before the runs it governs
2026-10-09	`docs/YCSB_PREREGISTRATION.md`	Amendment 2 (2026-10-09, declared before any run on it): the brain's own verdict on the cache	before the runs it governs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Preregistration	Amendment	Timing
2026-10-09	docs/REDIS_PREREGISTRATION.md	Amendment 2 (2026-10-09, declared before any run on it): the brain's own verdict on the ceiling	before the runs it governs
2026-10-09	docs/POSTGRES_PREREGISTRATION.md	Amendment 3 (2026-10-09, declared before any run on it): the brain's own verdict on the pool	before the runs it governs
2026-10-09	docs/MYSQL_PREREGISTRATION.md	Amendment 3 (2026-10-09, declared before any run on it): the brain's own verdict on the pool	before the runs it governs
2026-10-09	docs/KAFKA_PREREGISTRATION.md	Amendment 2 (2026-10-09, declared before any run on it): the brain's own verdict on the group's size	before the runs it governs
2026-10-08	docs/YCSB_PREREGISTRATION.md	Amendment 1 (2026-10-08, declared before the second counted set): the give-back asks whether the cache holds its working set	before the runs it governs
2026-10-08	docs/POSTGRES_PREREGISTRATION.md	Amendment 2 (2026-10-08, declared before the second counted set): where the CPU went, and the take-back gate	before the runs it governs
2026-10-08	docs/MYSQL_PREREGISTRATION.md	Amendment 2 (2026-10-08, declared before the third counted set): the pool grows only while it is missing	before the runs it governs
2026-10-07	docs/SWARM_PREREGISTRATION.md	Amendment 1 (2026-10-07 20:40 UTC, after the first A, B and C were seen; said so)	after runs began
2026-10-05	docs/REALMS_PREREGISTRATION.md	Amendment (2026-10-05): the compass law is the arm, written before its run	before the runs it governs
2026-10-05	docs/REALMS_PREREGISTRATION.md	Amendment (2026-10-05): four levers bounded by their physics, and the mirror label (written after the compass table was seen)	after runs began
2026-10-05	docs/K8S_COMPASS_PREREGISTRATION.md	Amendment 9: idle read from the autoscaler's floor, and the batch test again (written before its run, 2026-10-05)	before the runs it governs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Preregistration	Amendment	Timing
2026-10-05	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment 8: cruise and the emergency brake, and the batch test (written before its run, 2026-10-05)	before the runs it governs
2026-10-05	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment 7: coasting, off the gas but still in gear (written before its run, 2026-10-05)	before the runs it governs
2026-10-05	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment 6: a pinned gauge is not a steady demand (written before its run, 2026-10-05)	before the runs it governs
2026-10-05	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment 12: cruise steps back, and the organism's start file (written before the batch rerun, 2026-10-05)	before the runs it governs
2026-10-05	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment 11: the burst bill test sized to what the service can serve, and rounding read as the same (2026-10-05)	not stated in its heading
2026-10-05	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment 10: two rows read for what they measure (written after the frozen-engine runs were seen, 2026-10-05)	after runs began
2026-10-05	`docs/GPU_PREREGISTRATION.md`	Amendment 12 (2026-10-05, before any trial on this code)	before the runs it governs
2026-10-04	`docs/K8S_COMPASS_PREREGISTRATION.md`	Amendment to the wandering test, written before its run (2026-10-04)	before the runs it governs
2026-10-04	`docs/GPU_PREREGISTRATION.md`	Amendment 11 (2026-10-04, before any trial on this code)	before the runs it governs
2026-10-03	`docs/GPU_PREREGISTRATION.md`	Amendment 9 (2026-10-03, before any trial on this code)	before the runs it governs
2026-10-03	`docs/GPU_PREREGISTRATION.md`	Amendment 8 (2026-10-03, before any further trial)	before the runs it governs
2026-10-03	`docs/GPU_PREREGISTRATION.md`	Amendment 10 (2026-10-03, before any trial on several cards)	before the runs it governs
2026-10-02	`docs/GPU_PREREGISTRATION.md`	Amendment 7 (2026-10-02, before any further trial)	before the runs it governs
2026-10-02	`docs/GPU_PREREGISTRATION.md`	Amendment 6 (2026-10-02, after the first counted run and before any further trial)	before the runs it governs

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Date	Preregistration	Amendment	Timing
2026-10-02	`docs/GPU_PREREGISTRATION.md`	Amendment 5 (2026-10-02, before any valid hardware trial; the only smoke so far was invalid, amendment 4)	before the runs it governs
2026-10-02	`docs/GPU_PREREGISTRATION.md`	Amendment 4 (2026-10-02, after an invalid smoke; no counted data exist)	after runs began
2026-10-01	`docs/GPU_PREREGISTRATION.md`	Amendment 3 (2026-10-01, before any hardware trial; no smoke or counted data exist)	before the runs it governs
2026-09-28	`docs/GPU_PREREGISTRATION.md`	Amendment 2 (2026-09-28, before any hardware trial; no smoke or counted data exist)	before the runs it governs
2026-09-28	`docs/GPU_PREREGISTRATION.md`	Amendment 1 (2026-09-28, before any hardware trial; no smoke or counted data exist)	before the runs it governs

CHAPTER 71

Limitations and Threats to Validity

What the newest three-run tables (omni-v11's; omni-v23, the engine that stands since 31 October, runs every benchmark again) can and cannot support, written the way a referee reads a study: who ran it, what could bias a comparison, what a gauge really measures, how often the reading rule could be fooled, and how far a result travels. Each limitation names what would remove it. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

71.1 Independence

- **Every run is ours.** The harnesses, the runs, the tables and this page are The Omni-Compass LLC's own; no outside party has run, reproduced or reviewed them. What reduces the dependence on us: every table is rebuilt from its raw receipts by a public command and compared byte for byte (`tools/reproduce.py`, `reproduce.yml`), every rule was written before its runs (`docs/DEVIATIONS.md` lists every later change with its timing), and every raw file carries its SHA-256.
- **What would remove it:** an outside party running the same workflows from a fork, or the paired protocol on its own system (`docs/PILOT_PROTOCOL.md`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

71.2 Internal validity: could something other than Omni-Compass explain a difference?

- **Pairing and order.** Each repetition runs native and Omni-Compass back to back on the same machine, alternating which goes first. With three pairs a run (the data stores) the order cannot balance (two to one). Where it was checked (the Redis tail, `docs/TAIL_LATENCY.md`) the difference held in both orders.
- **The machine moves between runs.** GitHub's free machines differ from run to run. On MySQL's update workload the throughput was +0.3% in every pair of run B and −0.4% in every pair of run C, so the row reads "the runs disagree": a machine-level effect larger than the change, under an offered rate that pins throughput near 98% of what is offered.
- **One machine under kind.** Every Kubernetes node in the kind tests is a container on one runner; the governor counts that machine's processor once (omni-v6) and the served image is pulled onto every worker before each arm (18 October). A multi-machine cluster (Azure) is the test of the cloud's own behaviour.
- **The measuring process.** In the Redis harness the brain runs as a thread inside the load generator's own Python process, and its once-a-second work shares Python's interpreter lock with the request threads. The 99th percentile there reads about 6 µs higher with Omni-Compass in two of three runs. A controlled test shows the cause: the brain's presence in the process, with nothing else changed, lengthens the timed requests (99th percentile +101 µs, 10 of 12 pairs; mean +5 µs, 11 of 12) (`docs/TAIL_LATENCY.md`, section 4). The bias weighs against Omni-Compass, because only its arm carries the brain, and no deployment runs the brain inside the application. The next version's harness runs it in a process of its own. The other data stores' load generators already run as processes of their own.
- **Simulators that do not reproduce exactly.** CityLearn's comfort score moved by up to 3.6×10^{-5} between runs in both arms alike; PyBullet (the drone swarm) reproduces to about one part in a thousand across GitHub's machines. Rows that do not reproduce read "the runs differ", and the tables say when the paired change was exactly zero.

71.3 Construct validity: does each gauge measure what its name says?

- **Energy in the Kubernetes and swarm tables is a declared model**, not a meter (evidence class S inside a live test). No physical meter has measured Omni-Compass yet (class P: none), so less energy is **undetermined**.
- **Node-hours on kind are not a bill.** The cloud bill is measured only in the cloud (Azure, running).
- **The modelled muscles and organisms are our own models** (class S): 1,107 muscles, 527 GAIN, 0 LOSS (`results/realms_v5/REALMS_V5.md`) say what our models do, never what a real plant does.
- **A margin no gauge judges, and a fault it hid.** In the drone swarm the judged safety gauge is the near miss (two drones under half the 0.3 m keep-out, 0.15 m). The closest approach is shown, not judged. With Omni-Compass on top it fell from about 0.30 m to as low as 0.156 m (run C, the short cell). There was no

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

near miss and no collision in any counted run, but the margin native keeps was spent to fly faster. On mission draws outside the preregistered seeds it read LOSS: on the mixed cell with draw 214, the omni-v11 wire let two drones come within 0.083 m, with 5 near-miss ticks and 3 collision ticks, where native flew the same draw with none (`results/model_gate/V12_SWARM_REFLEX.md`). A preregistered set of seeds could not show it, and only flying draws the wire was never tuned on did.

- **The cause is in the wire, not the brain.** The proximity reflex is written to return the planner's cruise at once. The code lowered the speed by 0.01 a tick instead, so a drone that saw a neighbour within 0.5 m flew fast for about a second more.
- **The fix**: the reflex at once, reaching out with the square of the speed, as the stopping distance does. In our own flights it held every drone at or outside the full keep-out (the least, 0.304 m), with no near miss and no collision, on all ten cells and draws tried, the four preregistered cells and six untouched draws. It kept 49% to 92% of the energy cut.
- **What came with it.** The fix landed in omni-v12 (`docs/OMNI_V12.md`), and the same version's preregistration amendment (`docs/SWARM_PREREGISTRATION.md`, amendment 2, written before its runs) makes a collision with Omni-Compass on top a judged LOSS row, voids a cell only when native itself collides, and adds three mission draws no wire was ever run on.

71.4 Statistical conclusion validity: how often could the reading rule be fooled?

- **The three-run rule is strict.** A row reads GAIN or LOSS only when three separate runs all move the same way with every 95% interval clear of zero. With no true change, and runs independent, one run's interval is clear of zero on a given side with probability 0.025, so a false GAIN or LOSS in a row has probability $2 \times 0.025^3 \approx 3.1 \times 10^{-5}$. Over the 1,039 judged rows of the omni-v11 tables the expected number of false readings is at most 0.03. Of those rows, 613 are in tests that carry noise; the other 241 are in deterministic simulators, which reproduce to the digit. Over the 613 the expectation is 0.02. Runs share the harness, so they are not fully independent of it; the number is a guide, not a guarantee.
- **STEADY is not zero.** A row reads STEADY when the test could not resolve a change; `docs/POWER.md` gives, for every judged row, the smallest change all three runs would detect. Many data-store rows (three pairs a run) cannot resolve a change under 10% of native; STEADY there means "no change larger than that", and nothing finer is claimed.
- **Rows where the runs disagree** (omni-v11): MySQL's update throughput (above), CityLearn's comfort score in two districts (CityLearn's own drift; the paired change exactly zero), and the drone swarm's closest approach, tracking error and cruise override (PyBullet's reproduction; the first is the margin of section 71.3, the other two are shown, not judged).
- **A fixed set of seeds is a sample, not a guarantee.** Every preregistered test runs on seeds written before its runs, so a reading holds for those draws. The swarm's collisions (section 71.3) appeared only on a draw

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

outside them. The swarm's omni-v12 runs add three draws never flown (amendment 2); the other tests still run on their fixed seeds alone.

71.5 External validity: how far does a result travel?

- **Loads.** Open-loop load generators on synthetic schedules, one public day of demand (Google 2011) on kind, and the data stores' published benchmark workloads (sysbench, YCSB, pgbench). No production traffic.
- **Size.** kind clusters of six workers on one machine; an Azure cluster of four workers. No large production fleet.
- **Undetermined:** the cloud bill (Azure running, Google and Amazon gated by their $0 checks), energy (no meter), the GPU (no run on the engine that stands), production readiness (no deployment). These are written as undetermined wherever they appear, never as results.

71.6 The engine changed often

Twelve versions in twelve days (omni-v5 to omni-v16, `docs/CHANGE_CONTROL.md`), each through omni-v15 a fault found in a real run and fixed, and omni-v16 the decision pace made a clean setting on the founder's word. No table mixes engines, and each benchmark runs again on every version; the cost is that each version's three-run tables take a day or more to land, so the newest tables of some benchmarks are an older engine's until they do.

CHAPTER 72

The Evidence Ledger

Every material statement in the technical package gets exactly one primary evidence class. A statement never borrows a stronger class from the one printed beside it.

Class	Meaning	Layer
T	formally proved theorem, under stated assumptions	L1 mathematical mechanism
V	finite computational verification: exact over a declared finite set, never a theorem beyond it	L1
S	simulation / model result	L2 modelled systems
L	live observation of external software (a real Kubernetes API, a real process)	L3 external software

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class	Meaning	Layer
P	independent physical measurement (a meter Omni does not read or control)	L4 physical plant
O	open: not established	—

No layer stands in for another: a theorem is not a physical validation, a Monte Carlo result is not a theorem, and a model's energy is not a meter's.

Mechanism identity for every row: `results/MECHANISM_IDENTITY.json`. Canonical engine `symmetric_verified`, mechanism id `29d9808dfb8f…`; the printed configuration `printed_eight_line`, id `cd333dc166fb…`, is a named alternative (Option A of the directive: one canonical, one alternative embodiment; no equivalence is claimed).

72.1 Layer 1: the mechanism

Class	Statement	Where
T	Unsaturated, the U-channel error obeys de/dt = −KP e; V = e²/2 has dV/dt = −KP e² < 0 for e ≠ 0 (inside Omega_unsat only).	`docs/TRACKING_THEOREM.md` Thm 1
T	With the U drift bounded by F[bar] < u[max], V decreases wherever e ≠ 0, saturated or not; e never changes sign; saturated stretches end in bounded time (continuous time).	Thm 2
T	On the declared parameter box, F[bar] = 16.86 < 25, so Thm 2 holds over the whole box in continuous time.	Thm 2
T	The sampled controller (u held through each RK4 step) satisfies e_(k+1) = 0.88 e_k + d_k, hence abs(e_k) <= 0.88^k abs(e_0) + epsilon_h (1 − 0.88^k)/0.12.	Thm 3
V	epsilon_h = 0.00378 over every micro step of the 500 frozen fixtures (nearest target); ultimate bound 0.0315 < BASIN_TOL 0.10. Wrong target: 0.0243, bound 0.20 (wider than the basin).	`results/TRACKING_BOUNDS.json`
V	The discrete execution dithers inside that band: abs(e) grew on 27 061 of 100 000 micro steps; e changed sign 747 times. The continuous monotone decay is not inherited.	same
V	Saturated micro steps: 0 of 100 000 (nearest target), 5 of 100 000 (wrong target).	same

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class	Statement	Where
T	The admissible box A (E interval, abs(U − sigma) <= abs(U0 − sigma), S between S0 and S[plus]) is forward invariant in continuous time.	Thm 4
V	Discrete invariance of A: 0 failures over 100 000 micro steps of the frozen fixtures, both targets.	`results/TRACKING_BOUNDS.json`
O	Discrete invariance and epsilon_h proved over the whole box (interval arithmetic).	—
O	Global stability of the forced six-state system.	`docs/FORMAL_STATUS.md`
V	500 / 500 frozen fixtures satisfy CONVEY-5 and CERT-10 (dwell predicates, not external certification).	`results/core_evidence.json`
V	Frozen 500-fixture comparison: symmetric_verified final error 7.48e-5, integrated abs(u) 0.878, peak 13.93; printed_eight_line 5.13e-6, 11.79, 18.87. Both executable; not interchangeable.	`results/MECHANISM_IDENTITY.json`
V	Python and C++ produce identical results (engine, governor, shield, HPA law, conveyance, GPU rules); twins sealed.	`verify.py`, `results/SEAL.json`

72.2 Layer 2: models and simulations

Class	Statement	Where
S	Synthetic stack: energy for Omni over the Kubernetes reference 168.9 → 121.7 kWh per run (documented-behaviour reference, not upstream controllers).	`docs/CLAIMS_REGISTER.md` C8
S	Fleet harness, recorded PlanetLab shapes: −40.6% energy vs HPA+CA, −21.7% vs Karpenter-lite (omni_fleet).	C17
S	Single GPU physics model: the engine alone would save 10.6–13.6% work per kJ on card A but breaks the p95 guardrail by 15–28%; with the frozen guards, about +1 to +5% inside it.	`results/gpu/sim/FINDINGS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class	Statement	Where
S	Node exchange (CPU and GPU on one budget): +1.4 to +5.7% work against the separate budgets, never over budget.	`results/hardware/NODE_EXCHANGE_*.json`
S	Six organisms (345, 262, 282, 337, the four stacked 1,226, the whole tower 656), the compass law on every muscle against each organism's own controllers, 1,000 paired runs at 1× and 10× size: work per energy +0.20% to +0.30%, energy −0.21% to −0.32%; every knob handed back.	`results/scale/GRID.md`
S	GPU card model, the card's firmware alone against the firmware with Omni on top (amendment 8: the verdict, 2% allowance), geometric means over 10 seeds and 10 fresh seeds: AI token generation, energy −3.25% / −3.72%, median +0.55% / +0.70%, p95 +0.29% / +0.26%; compute-bound, energy −0.70% / −0.48%, median +1.56% / +1.47%. A fixed 105 W power cap alone against the cap with Omni on top: energy −2.09% / −2.37% (AI token generation).	`results/sim/gpu_two_wire/RESULT.md`
S	Realm harness round 3 (every realm carries the shared spine), preregistered, seeds 3000-3009: the whole 656-muscle tower native against one governor on top, work per energy +0.1% (+0.1 to +0.1), violations +0.5 pp, SUPERIOR WITHIN GUARDRAILS. Realms: Energy +0.2% with +1.9 pp violations (tradeoff); Compute 0.0% with +2.1 pp (not established); Distribution −0.1% (lower); Physics −0.7% (lower). Rounds 1 and 2 kept, superseded.	`results/realms/REALMS.md`
S	Stacked organism (round 4, seeds 4000-4009): the four realm organisms on one clock, 1,226 muscles with every duplicate; stacked native equals the four realms alone on every seed. One governor over the stack: energy −0.14%, work per energy +0.02%, violations +1.4 pp, ENERGY IMPROVEMENT WITH SERVICE TRADEOFF; four separate governors about the same (+0.04%); one governor against four separate: −0.02% (lower, by a hair).	`results/realms/stack/STACK.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

72.3 Layer 3: live external software

Class	Statement	Where
P	omni-v18 on a real card, NVIDIA A100 SXM4 40 GB (Lambda run 40079645681, 24 October), one run of 10 paired repetitions, the card's own meter: every service gauge STEADY (work per energy up 0.2%, the 95th percentile up 0.2%, requests served the same, none lost), the temperature up 1.4% to 1.5% (power given); wire check 7 of 7, fault drill 5 of 5; the governor wrote nothing in 3,050 decisions, both knobs at their top on native (at its limit on native).	`results/gpu/run-20261024T091235Z/GPU_REPS.md`
P	First real-GPU counted run, NVIDIA A10 (Lambda), 10 paired repetitions, the card's own meter: work per energy +3.6% (+2.7 to +4.5, proven), GPU energy −3.5%, same requests, none lost; wire check 7 of 7, every write read back, every arm restored.	`results/gpu/run-20261002T082232Z/GPU_REPS.md`
L	Set 25 (commit `6fa7a97`), real Kubernetes, 10 paired repetitions: machines in service −32.3%, p95 −57.3%, 0 failed requests, total CPU including Omni's own −0.6% (not proven).	`results/live/LIVE_REPS_25.md`
L	Set 26 (commit `e7f920d`), real Kubernetes, 10 paired repetitions, three arms: the allocation law machines −35.8%, p95 −55.4%; **the compass law in the live controller machines −17.2%, p95 −64.8%**, 0 failed requests, label by the preregistered rule *fewer machines within the band*.	`results/live/LIVE_REPS_26.md`
L	Set 27 (commit `d46c959`), real Kubernetes, 10 paired repetitions, three arms: the allocation law machines −36.6%, p95 −53.1%; **the compass law aligned with the GPU governor machines −15.9%, p95 −65.5%, p99 −72.6%**, 0 failed requests, label by the preregistered rule *fewer machines within the band*.	`results/live/LIVE_REPS_27.md`
L	Set 24 (2026-10-02, commit `c908054`), real Kubernetes (kind), 10 paired repetitions: machines in service −31.6% (proven), p95 response time −60.1% (proven), p99 −64.1% (proven), HPA replicas −38.6%	`results/live/LIVE_REPS_24.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class	Statement	Where
	(proven), 0 failed requests on both; total CPU including Omni's own −1.8% (not proven); energy with every machine powered −0.3% (declared model).	
L	Set 23 (set 22 repeated on the current code, 2026-10-02), real Kubernetes (kind), 10 paired repetitions, equal work: p95 response time −62% (proven), replicas −37% (proven), pods started −64% (proven), 0 failed requests; total CPU with Omni's own −1.0% and modelled energy −0.2% (no difference).	`results/live/LIVE_REPS_23.md`
L	Set 22, real Kubernetes (kind), 10 paired repetitions, equal work (fixed-rate load): p95 response time −61% (proven), replicas −23% (proven), pending pod-minutes −91% (proven), 0 failed requests.	`results/live/LIVE_REPS_22.md`
L	Set 21, real Kubernetes (kind), 10 paired repetitions: p95 response time −37% (proven), replicas −12% (proven), 0 failed requests.	`results/live/LIVE_REPS_21.md`
L	Omni patched a real Kubernetes API in place (no restart); the reset restored every setting in every run; watch mode wrote nothing.	same, `results/live/`
S	Set 21 energy is a declared model, not a meter: +1.8% higher with parked machines at idle power.	same (energy table)

72.4 Layer 4: independent physical measurement

Class	Statement	Where
O	Omni changes successful work per measured joule on a GPU. The bench is built with three receipts (governor, actuator, outcome), the three contrasts (observation, authority, total), and result labels by rule; it has **not** been run on a card.	`scripts/gpu_paired.sh`, `docs/GPU_PREREGISTRATION.md`
O	Production data-centre energy effect.	—

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class	Statement	Where
O	Whether the power limit is the right actuator for LLM serving. Published measurements (arXiv 2605.11999, H200) find memory-bound decode draws 137–300 W of 700 W, so no power cap binds; clock scaling is what saves energy there (arXiv 2501.08219, GreenLLM 2508.16449). The pinned bench workload is compute-bound, where the cap does bind: a result on it does not transfer to LLM decode.	external literature
O	Wall-plug (whole-machine) energy effect; CPU package and DRAM (RAPL) effect.	—
—	What the GPU bench will attribute: per write, joules and requests against native, and which rule decided it (engine, floor, gate, reflex, heat, speed lock).	`tools/gpu_reps.py`

72.5 Negative evidence, kept

Nothing here is deleted when a later result looks different.

Class	Statement	Where
P	Same run: p95 response time **+58.5% higher** (510 to 809 ms), mean +48%; label by rule ENERGY IMPROVEMENT WITH SERVICE TRADEOFF. Cause: governor wiring (busy bursts served below the card's own clock); corrected in amendments 6-7, not yet re-run on a card.	`results/gpu/run-20261002T082232Z/GPU_REPS.md`, `docs/GPU_PREREGISTRATION.md`
S	Two-wire GPU card, the profiles of amendment 7 (removed in amendment 8): service +5.3% work per energy with the median +14.8% slower; batch p95 +7.0% (−4.8 to +20.3). The governor before amendment 6 gave p95 +32.9% and +37.1%.	`docs/GPU_PREREGISTRATION.md`, history in git
S	Six organisms, same runs: time over the service line **+0.19 to +0.27 pp higher in every cell**; band first is not held anywhere.	`results/scale/GRID.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class	Statement	Where
S	Round 6 law (HPA target held at the operator's own, machine release margin 0.6), 20 paired seeds per organism, before the grid rerun: work per energy +0.086% to +0.201%, time over the line −0.010 to −0.031 pp (ahead of native in all six), work unchanged within its interval, every knob handed back.	`docs/REALMS_PREREGISTRATION.md` (round 6)
L	Set 21: modelled energy 1.8% **power given** with every machine powered (the only honest energy row on kind).	`results/live/LIVE_REPS_21.md`
S	Realm harness round 1 (superseded, kept): all five organisms **LOSS** (whole tower −0.1%). Its Omni layer did not follow the shipped controller (no contraction authority or SLO reflex, the stack law in place of the HPA, request traffic paused, a site budget under native draw).	`results/realms/round1/`
S	Realm harness round 2: the Physics / Robotics / Autonomous organism **LOSS** (−1.9%, violations +2.9 pp); the Compute organism's +2.5% costs +2.9 pp of service violations; 68 single muscles LOSS or power given, mostly batch pacing and cooling setpoints under the live cooling law.	`results/realms/REALMS.md`
L	Set 22, equal work: no CPU saving once Omni's own CPU is counted (service −7.6%, controller +0.070 cores, together −0.9%, not proven); modelled energy unchanged (−0.1%).	`results/live/LIVE_REPS_22.md`
L	Set 20: the "same work" reading was wrong; requests were +35%.	`results/live/LIVE_REPS_20.md` addendum
L	Sets 1–2: machine savings **withdrawn** — a broken probe had blinded the latency sense.	`results/live/LIVE_REPS_PROBE_DEFECT.md`
L	Set 3 with a working probe: Omni kept all 6 machines; no significant difference except more waiting pods.	`docs/HISTORY.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class	Statement	Where
L	Set 21: pending pod-minutes +238% (not proven); pod starts +37% (not proven).	`results/live/LIVE_REPS_21.md`
S	GPU and batch: energy 1–2% **power given**, power and heat margins 1–4% short of tight packers (AKS/Karpenter, CAST AI, Spot).	`docs/HISTORY.md`
S	Machine round trips 1.1 → 1.7 per run (**LOSS**) in 6 of 8 seed-target combinations.	C12
S	Power-protect mode: backlog violations 2.5% → 6.7% (**LOSS**).	C12p
S	Park strategy on PlanetLab shapes: energy +1.3% vs HPA+CA, +33.6% vs Karpenter-lite (**power given**).	C17
S	Typical response time about 20% slower than every platform in the web and four-cluster simulations.	`docs/HISTORY.md`
S	GPU model: the engine without its guards breaks the p95 guardrail.	`results/gpu/sim/FINDINGS.md`
S	The compass-stroke GPU variant was tried and not adopted.	same
S	Right-sizing against VPA: p95 +15%, memory (OOM) kills +531%.	`docs/history/BENCHMARK_REPORT.md`
—	Reported in the external master-build report (not reproducible from this repository): on fresh scenarios Karpenter+VPA sometimes used less modelled energy than Omni, while Omni had lower churn and fewer request-induced evictions. Kept here so it is not lost; to be re-run here before it is cited.	external

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 73

The Claims Register

Every claim, its evidence status and the command that reproduces it. Simulation results use the synthetic stack model in omnicompass/stack_sim.py; they are not production measurements.

ID	Claim	Status	Reproduce			
C1	The six-state mechanism is implemented identically in the reference engine, Python and C++.	Proven (exact parity tests)	`python verify.py`			
C2	The C++ governor reproduces the Python governor on recorded stack telemetry.	Proven (0 mismatches)	`python tests/test_cpp_governor_parity.py <oc_governor>`			
C2s	The C++ shield reproduces the Python shield (enforced actions, intervention counts and violations) on action sets recorded from every arm	Proven (0 mismatches; negative control)	`python verify.py`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	type; a C++ shield with invariant I1 weakened is rejected by the parity test.					
C2h	An independent C++ implementation of the HPA replica law (ratio rule, 10% tolerance, scale-up limit, 300 s scale-down window) reproduces the fleet harness HPA on every recorded step (hundreds of thousands of steps across Kubernetes and governor arms); a C++ HPA with 5% tolerance is rejected.	Proven (0 mismatches; negative control)	`python verify.py`			
C3	The S channel cannot cross S- (forward invariance).	Proven (Proposition 1)	`Manual Chapter 3`			
ID	**Claim**	**Status**	**Reproduce**			
C4	The governor completed	Measured (soak test);	`python verify.py`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	100,000,000 consecutive decisions (951 years at 5-minute intervals) with 0 failures and constant memory; failure-rate upper bound 3.0e-08 at 95% confidence.	reproduced by the full verifier				
C4b	Throughput-mode soak: 100,000,000 decisions, 0 failures; resident memory at 25/50/75/100% of each run is constant ([3852, 3852, 3852, 3852] kB power-protect, [3916, 3916, 3916, 3916] kB throughput).	Measured (soak test)	`python verify.py`			
C8p	Primary endpoint (energy, throughput vs reference target 0.7):	Measured in simulation	`python benchmarks/multiplicity.py`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	seed 346410161: -47.20 kWh, p = 1.0e-04; seed 360555127: -47.69 kWh, p = 1.0e-04. Secondary metrics Holm-adjusted over 24 tests: 21 toward their aims, 2 LOSS or power given, 1 not significant.					
C5	One governor decision costs 2.6 microseconds on one CPU core of the test machine (machine-dependent).	Measured	`cpp: oc_soak`			
C6	Observe mode leaves the stack bit-identical: over Kubernetes, identical to Kubernetes alone (500/500 and 500/500); over the native model,	Measured in simulation	`results/heldout_seed_*`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	identical to native (500/500 and 500/500).					
C7	Invariant violations per run (all arms scored identically): Kubernetes reference 9.56 / 9.44; power-protect 0.00 / 0.00; throughput 2.72 / 2.69, of which all are I4 (projected power), which throughput mode does not enforce; excluding I4: reference 6.91 / 6.80, throughput 0.00 / 0.00.	Measured in simulation	`results/heldout_seed_*`			
C8	Energy for Omni-Compass over Kubernetes (throughput mode, same power rules as the reference) versus the Kubernetes	Measured in simulation (synthetic stack, documented-behaviour reference model)	`results/heldout_seed_*`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	reference model (HPA target 0.7): 168.9 to 121.7 (lower) / 169.2 to 121.5 (lower) kWh per run; across HPA targets 0.5 to 0.8: GAIN in 8, LOSS in 0, not significantly different in 0 of 8 seed-target combinations.					
C9	Time healthy for Omni-Compass over Kubernetes (throughput mode) versus the reference model (target 0.7): 79.3% to 91.2% (higher) / 78.5% to 91.4% (higher); across targets: GAIN in 8, LOSS in 0, not significantly different in 0 of 8 seed-target combinations.	Measured in simulation	`results/heldout_seed_*`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
C10	Recovery time for Omni-Compass over Kubernetes (throughput mode) versus the reference model (target 0.7): 57.2 to 17.8 (lower) / 58.4 to 15.2 (lower) minutes; across targets: GAIN in 8, LOSS in 0, not significantly different in 0 of 8 seed-target combinations.	Measured in simulation	`results/heldout_seed_*`			
C11	Backlog violations for Omni-Compass over Kubernetes (throughput mode) versus the reference model (target 0.7): 2.5% to 1.6% (lower) / 2.6% to 1.5% (lower); across targets: GAIN in 8, LOSS in 0, not significantly	Measured in simulation	`results/heldout_seed_*`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	different in 0 of 8 seed-target combinations.					
C12	Machine round trips (started and later stopped), throughput mode versus the reference (target 0.7): 1.1 to 1.7 (higher) / 1.1 to 1.8 (higher) per run; across targets: GAIN in 2, LOSS in 6, not significantly different in 0 of 8 seed-target combinations. The reference removes a machine only after 10 minutes below 50% utilization and keeps post-event machines running; the governor releases them, which is the main source of	Measured in simulation	`results/heldout_seed_*`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	its energy saving.					
C12w	Scale direction reversals (back-and-forth wear), throughput mode versus the reference (target 0.7): 2.19 to 1.93 (lower) / 2.22 to 1.92 (lower) per run; across targets: GAIN in 4, LOSS in 4, not significantly different in 0 of 8 seed-target combinations. Power-protect: 2.19 to 3.06 (higher) / 2.22 to 3.08 (higher).	Measured in simulation	`results/heldout_seed_*`			
C12e	Engine mechanism in the flagship (throughput): removing the equation (2) release gate changes reversals from 1.93 / 1.92 to	Measured in simulation	`results/heldout_seed_*`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	3.19 / 3.18 and energy from 121.7 / 121.5 to 120.2 / 119.9 kWh; removing engine evolution changes energy to 125.4 / 125.1 kWh and reversals to 0.00 / 0.00.					
C12p	Power-protect mode (enforces the site power limit, which the reference does not): energy 168.9 to 132.2 (lower) / 169.2 to 132.3 (lower) kWh; power-limit violations 12.8% to 3.5% (lower) / 13.8% to 3.7% (lower); backlog violations 2.5% to 6.7% (higher) / 2.6% to 7.0% (higher).	Measured in simulation	`results/heldout_seed_*`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
C12a	Human pages: the governor issues no page actions. Zero pages is a design property, not a measured performance result.	Design	`omnicompass/adapter.py`			
C12b	The reference model is not the upstream Kubernetes controllers: Cluster Autoscaler scheduling simulation, Karpenter, VPA, scheduling constraints and disruption budgets are not represented.	Stated limitation	`benchmarks/stack_benchmark.py (K8sReference)`			
C15	15-second control plane, Omni-Compass as node-pool and power authority in place of the Cluster Autoscaler (HPA retained),	balanced: web -33.2% / -15.3%, reversals 4.0; multi -33.0% / -15.0%, reversals 15.5; batch -12.4% / -4.5%, reversals 5.7; gpu -5.0% /	wear-first: web -29.7% / -10.9%, reversals 3.6; multi -29.6% / -10.6%, reversals 14.5; batch -12.3% / -4.3%, reversals 4.9; gpu -4.5% /	park: web -15.9% / +6.7%, reversals 0.0; multi -15.6% / +7.0%, reversals 0.0; batch -15.3% / -7.7%, reversals 0.0; gpu -4.9% /	Measured in simulation (fleet harness, synthetic workloads, documented-behaviour execution layer)	`python -m fleet.benchmark --seeds 30 --seed-base 700000 --out out/`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	energy vs HPA+CA / vs HPA+Karpenter-lite: energy-first: web -34.5% / -16.9%, reversals 4.3; multi -34.3% / -16.6%, reversals 18.1; batch -17.2% / -9.7%, reversals 13.6; gpu -4.9% / -2.7%, reversals 5.1; gpu_always_on -4.7% / -3.2%, reversals 0.0	-2.8%, reversals 4.2; gpu_always_on -4.7% / -3.2%, reversals 0.0	-2.3%, reversals 3.0; gpu_always_on -4.2% / -2.8%, reversals 0.0	-2.7%, reversals 0.0; gpu_always_on -4.7% / -3.2%, reversals 0.0. Baseline reversals (CA / Karpenter): web 1.1 / 6.8; multi 5.5 / 25.4; batch 1.4 / 24.9; gpu 2.8 / 11.2; gpu_always_on 0.0 / 0.0. In gpu_always_on no arm powers a node off.		
C17	Recorded PlanetLab utilization shapes with a declared scale mapping (9 traces, 30 scenarios), governor as node-pool authority in place of the Cluster Autoscaler: omni_fleet: energy -40.6% vs HPA+CA (lower),	Measured in simulation driven by recorded traces (frozen laws, no retuning)	`python -m fleet.planetlab --dir fleet/traces/planetlab --scenarios 30 --seed-base 800000 --out out/`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	-21.7% vs Karpenter-lite (lower); omni_fleet_balanced: energy -37.5% vs HPA+CA (lower), -17.5% vs Karpenter-lite (lower); omni_fleet_wear: energy -31.0% vs HPA+CA (lower), -8.9% vs Karpenter-lite (lower); omni_fleet_park: energy +1.3% vs HPA+CA (higher), +33.6% vs Karpenter-lite (higher). Time healthy and work completed are significantly lower by 0.08 points and 0.01%; observe mode identical in 30/30.					
C16	Live-cluster capture, capture replay and the	Tested path; no real-data result	`python tests/test_fleet_realdata_paths.p`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	PlanetLab vessel are tested on generated inputs in the real formats. No real capture or recorded trace has been run in the package.		`y`			
C19	The live controller (omni_controller/) implements observe, target and nodepool modes with dry-run, audit log and a reset that restores HPA targets from annotations; tested against a stand-in kubectl only, never against a real cluster.	Tested path; no live result	`python tests/test_omni_controller.py`			
C18	Savings projection: the energy-first reduction relative to HPA + Karpenter-lite, applied to	Projection	`python benchmarks/savings.py`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	declared fleet profiles (results/SAVINGS.csv), computed identically in Python and C++. A projection from simulation, not measured savings.					
C21	End-to-end self-pilot (shipped controller, simulated cluster, real capture and scoring), default headroom 50%: energy per core-hour -7.7%, node-hours per core-hour -13.8%, pending-pod time not significantly different from HPA + Cluster Autoscaler; HPA shortfall minutes higher. Lower headroom	Measured in simulation	`python pilot/selfpilot.py`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	saves more energy with more pending-pod time (the manuscript's Section 8.12a).					
C20	pilot/score.py scores a user's own captures (node-hours and energy per used core-hour, utilisation, pending-pod and HPA-shortfall minutes, bootstrap intervals); tested to detect a real gain, report no difference for identical clusters and detect a service regression.	Tested tool; no pilot result	`python tests/test_pilot_score.py`			
C13	Decision components (autoscalers, power agents, paging, Terraform as controller) consume about	Modeled from published figures and stated assumptions	`python benchmarks/fleet_overhead.py`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

ID	Claim	Status	Reproduce			
	0.02% of fleet CPU; idle capacity is 92% of fleet CPU at 8% utilization.					
C14	Behaviour on production systems.	Not established; requires the pilot protocol	`docs/PILOT_PROTOCOL.md`			

CHAPTER 74

Comparison with Existing Controllers

People who run these systems often see "autoscaling" everywhere and assume it all does the same thing. It does not. This page sets out, from public descriptions, what each system controls, and where Omni-Compass sits among them. The five systems named most often, nine open-source pieces read from their own repositories (license, kind, where each can run, its register row) and the formal distinction between one governor and a collection of tools: `docs/THE_SEAT.md`.

How to read it. Borg and Twine are private to Google and Meta; Turbonomic is closed commercial software. None of them can be run here, so they are compared by what their makers have published (papers, documentation, product pages), never by a run. A mark of "—" means the capability is not in the system's public description, not that it is impossible for it. The only fair measured comparison is on the customer's own system: their stack as it runs today against the same stack with Omni-Compass on top, in paired runs (`docs/INTEGRATION_MANUAL.md`, section 6).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

74.1 What each system controls

System (public description)	Pods / replicas	Machines	GPU power	CPU clock / power	Moves watts between CPU and GPU	Site power budget	One decision across all of these
Kubernetes HPA	✓	—	—	—	—	—	—
Kubernetes VPA	✓ (requests)	—	—	—	—	—	—
Cluster Autoscaler, Karpenter	—	✓	—	—	—	—	—
KEDA	✓ (to zero)	—	—	—	—	—	—
Red Hat OpenShift / OKD	✓ (the Kubernetes autoscalers)	✓ (machine autoscaling)	—	—	—	—	—
IBM Turbonomic	✓ (sizing, scaling)	✓ (placement, sizing)	—	—	—	—	partial (resource actions across layers)
Google Borg	✓ (job scheduling)	✓ (cluster placement)	—	—	—	Google has published separate power-capping work	—
Meta Twine	✓	✓	—	—	—	Meta has published separate power management (Dynamo)	—
NVIDIA DCGM	—	—	✓ (limits, monitoring)	—	—	—	—

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

System (public description)	Pods / replicas	Machines	GPU power	CPU clock / power	Moves watts between CPU and GPU	Site power budget	One decision across all of these
Linux schedutil, RAPL	—	—	—	✓	—	—	—
AMD SmartShift, NVIDIA Dynamic Boost (laptops)	—	—	✓	✓	✓ (inside one laptop)	—	inside one device
Intel GEOPM (HPC)	—	—	partial	✓	partial (per job)	✓ (per job / cluster)	partial
Omni-Compass	✓ (on top of HPA; or deciding)	✓ (park and wake)	✓	✓	✓ (conveyance law; simulated)	✓ (simulated)	✓

74.2 How each one decides

	Typical rule	Omni-Compass
Pods	add replicas when average CPU passes a target	the same autoscaler keeps working; Omni-Compass sets its target from the whole system's state and raises floors ahead of bursts
Machines	add when pods cannot be placed, remove when underused	park and wake machines by one law, and only when every sense is live, pods are not rising and nothing waits
GPU	vendor default: full power, or a fixed cap set by hand	a cap from the engine, bounded by a floor and released at once when the card is busy or response time slips
CPU	the kernel's governor picks a clock from recent load	a ceiling from the same engine, inside the nervous system's envelope
Power	a fixed plan leaving room for every device at peak; a breaker or capper reacts when the site goes over	one conserved budget: watts move from organs holding surplus to organs in need, never created, never over the budget

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

	Typical rule	Omni-Compass
Safety	per tool	one OFF switch returns every organ to its recorded setting; watch mode first; service first

74.3 What is different, in plain words

1. **One brain instead of many separate reflexes.** Today each tool watches its own gauge: the pod autoscaler watches CPU, the node autoscaler watches pending pods, the GPU runs at its default, the power plan is fixed on paper. Nobody decides for the whole. Omni-Compass reads the whole system into one state and tells each tool the envelope it may act in. The tools keep doing their own jobs.
2. **Power as a budget that moves.** Today a building keeps room for every device to hit its peak at once, so much of its power sits reserved and unused. Omni-Compass treats the building's watts as one conserved total and moves them to where the work is: from CPUs and quiet GPUs to busy GPUs, never over the limit (`docs/CONVEYANCE_LAW.md`).
3. **A proof, not a tuned dial.** The budget law is proved to conserve the budget and to converge, and those properties are checked on every build (`tests/test_conveyance.py`).
4. **Service guarded on every power move.** Power is only taken back while response time is inside the target, and given back at once when it is not.
5. **Reversible in one move.** Every setting is recorded before it is changed and restored by one switch.

74.4 What the numbers show today, and what kind they are

Against	Result	Kind
Kubernetes as it runs by default (HPA + fixed nodes)	p95 response time −37% to −55%, replicas −12% to −25%, machines in service −17% to −21%	measured on real Kubernetes (kind), 10 paired runs each, `results/live/`
A GPU at its vendor default	+1.3% to +5.1% work per energy, p95 within +10%	modelled card (MLPerf-calibrated), `results/gpu/sim/after`
Today's power practice (every GPU at one fixed cap that leaves room for every CPU at peak)	+1.4% to +5.7% work served, backlog −16% to −28%, never over the site budget	modelled, `results/hardware/NODE_EXCHANGE_*.json`
A reactive site capper alone	60-196 minutes over the site budget without Omni-Compass; 0 with it	modelled

Turbonomic, Borg and Twine cannot be measured here. Against a customer who runs one of them, the comparison is made on the customer's own system, with the same paired method.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

74.5 Sources

- Kubernetes HPA, VPA, Cluster Autoscaler, Karpenter, KEDA: the projects' own documentation.
- Red Hat OpenShift / OKD: Red Hat's documentation of its autoscaling and machine management.
- IBM Turbonomic: IBM's product documentation.
- Borg: Verma et al., "Large-scale cluster management at Google with Borg", EuroSys 2015; Google's public Borg traces.
- Twine: Tang et al., "Twine: A Unified Cluster Management System for Shared Infrastructure", OSDI 2020.
- Meta Dynamo: Wu et al., "Dynamo: Facebook's Data Center-Wide Power Management System", ISCA 2016.
- NVIDIA DCGM and Dynamic Boost; AMD SmartShift; Intel GEOPM: the vendors' documentation.

Where a row above is wrong or out of date, correct it from the maker's own publication.

CHAPTER 75

The State of Play

How to read this receipt. Every judged row reads one of three words, decided by what its gauge is for and never by the plus or minus sign of its number. **GAIN**: the gauge moved toward its declared aim against native (more work done, a shorter wait, fewer misses or failures, or a resource cut). **STEADY**: native and Omni-Compass within 0.5% of each other (the mean of the three runs' changes under the 0.5% floor, or no change at all), or the knob is at its limit on native. **LOSS**: the gauge moved away from its aim (less work done, a longer wait, more misses or failures). A resource spent or held (energy, the power drawn and its peaks and swings, machines, memory, processor time, a bill, wear) that fell is a cut, a GAIN; one that rose is **power given**, what Omni-Compass put in to buy the work and the speed shown beside it, always shown and never called a loss; work bought with each unit of a resource (work per energy) that fell is power given too. So a minus can read GAIN (a response time cut) and a plus can read LOSS (failed requests up). A change of 0.5% or more, the mean of the three runs' changes, reads GAIN, LOSS or power given: **in all three of our runs** where A, B and C moved the same way with every interval clear of zero, otherwise **by the mean of our three runs**, with the count of runs beyond the noise beside it; **the runs disagree** names runs that moved 0.5% or more in opposite directions, a test unstable there; **shown, not judged** is a gauge declared shown only. The output follows the input: every reading is the result of the tune, the wiring and the load it names, and where a reading elsewhere differs the first presumption is the wiring (`DISCLOSURES.md`, section 4). The full key, letter by letter: `docs/HOW_TO_READ_THE_RESULTS.md`. Run and scored by The Omni-Compass LLC's

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

Current facts only. Earlier states, failures and chronology are kept whole in `docs/HISTORY.md`. The release this page describes is identified by `RELEASE_MANIFEST.json` (commit, fingerprints of the engine, the C++ twins, the GPU protocol, the live evidence and the verification receipt), which `verify.py` checks against the files.

Rerun everything: `pip install -r requirements.txt && python verify.py` ends with `VERIFICATION: PASS`.

Where things stand, read from the result files. Written by `python3 tools/summary.py --fix` from the files it names; `verify.py` fails a push in which this block no longer matches them. Nothing in it is typed by hand. The live results run on the engines named in their own tables, and every one of them runs again on the newest engine.

- **The engine on this checkout:** omni-v23, fingerprint c96ea4105d8747db (65 files).
- The muscles and the seven organisms on engine omni-v8 at the max (evidence class S, our own models; run 2026-10-17 15:39 UTC, commit 9819a903cba3, 1107 muscles; source: results/realms_v5/REALMS_V5.md):
  - every muscle alone, over its loads: STEADY 577, GAIN 530
  - the organisms under the normal load: the spine: GAIN (fewer misses), misses down 0.145 pp, power given +0.011%; Compute / AI / Cloud: GAIN (fewer misses), misses down 0.070 pp, power given +0.009%; Physics / Robotics / Autonomous: GAIN (fewer misses), misses down 0.121 pp, power given +0.010%; Energy / Facility / Industrial: GAIN (fewer misses), misses down 0.118 pp, energy the same; Distribution / Specialized: GAIN (fewer misses), misses down 0.067 pp, power given +0.002%; the four stacked: GAIN (fewer misses), misses down 0.092 pp, energy the same; the whole tower: GAIN (fewer misses), misses down 0.051 pp, energy the same
  - the organisms under the climb load: the spine: GAIN (fewer misses), misses down 0.632 pp, power given +0.039%; Compute / AI / Cloud: GAIN (fewer misses), misses down 0.390 pp, power given +0.046%; Physics / Robotics / Autonomous: GAIN (fewer misses), misses down 0.317 pp, power given +0.021%; Energy / Facility / Industrial: GAIN (fewer misses), misses down 0.217 pp, energy the same; Distribution / Specialized: GAIN (fewer misses), misses down 0.460 pp, power given +0.026%; the four stacked: GAIN (fewer misses), misses down 0.349 pp, energy the same; the whole tower: GAIN (fewer misses), misses down 0.094 pp, energy the same
  - the organisms under the line load: the spine: GAIN (fewer misses), misses down 0.371 pp, power given +0.038%; Compute / AI / Cloud: GAIN (fewer misses), misses down 0.205 pp, power given +0.045%; Physics / Robotics / Autonomous: GAIN (fewer misses), misses down 0.250 pp, power given +0.030%; Energy / Facility / Industrial: GAIN (fewer misses), misses down 0.117 pp, energy the same; Distribution / Specialized: GAIN (fewer misses), misses down 0.142 pp, power given +0.006%; the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

four stacked: GAIN (fewer misses), misses down 0.176 pp, energy the same; the whole tower: GAIN (fewer misses), misses down 0.051 pp, energy the same

- The grid (source: results/grid_v5/GRID.md): Across the grid: modelled, 36 of 36 squares of 10 and 100 runs GAIN (fewer misses); the 18 single runs: 18 with fewer misses; work +0.000%; power given +0.000% to +0.065%; with the cluster inside, 54 squares: 18 single repetitions (no reading); of the 36 of 10 and 100 repetitions, by gauge: "organism time over the line (% of steps)" GAIN in 36, LOSS in 0, "time over the response line (% of samples)" GAIN in 36, LOSS in 0, "response time (ms), 95th percentile" GAIN in 26, LOSS in 2, "energy (Wh)" GAIN in 26, power given in 0, "energy, parked workers still on at idle power (Wh)" GAIN in 26, power given in 0, "organism energy (J)" GAIN in 0, power given in 22, "organism work per energy" GAIN in 0, power given in 22, "failed requests (%)" GAIN in 18, LOSS in 0, "response time (ms), 99th percentile" GAIN in 12, LOSS in 2, "HPA replicas, mean" GAIN in 4, power given in 5, "pods started" GAIN in 3, power given in 0; off the clock in 3; invalid in 0.
- The Omni index (evidence class L, real software on real computers: the software end, not bare metal and not the chip; every measure judged across our three runs; the three-run tables of omni-v21, each named in its table): 5.0% GAIN on the software end's reading (work, speed and machines; energy shown, never scored where no meter read it), 12.9% GAIN on the service reading (work and speed alone); the reading declared on 4 October, the modelled energy scored too: 5.2% GAIN. Source: results/OMNI_INDEX.md.
  - Real Kubernetes (GitHub): 28.0% GAIN
  - Real database (PostgreSQL behind PgBouncer, GitHub): 4.2% GAIN
  - Real database buffer pool (MySQL under sysbench, GitHub): 2.2% GAIN
  - Real cache (Redis, GitHub): 0.1% GAIN
  - Real database cache (MongoDB under YCSB, GitHub): 0.5% LOSS
  - Real messaging (Apache Kafka, GitHub): 1.2% power given

On the engine that stands (omni-v23): 0 of 26 benchmarks so far. Every other row is that engine's own result, named in its Engine column, and runs again on the engine that stands (`scripts/reruns.txt`); results are never read across engines.

Benchmark	Engine	Its one number	Its word	Class
Every real test together: the Omni index	omni-v21	5.0%	GAIN AND LOSS	L
The six organisms with the real cluster inside, 10 and 100 copies	omni-v3	5.3%	GAIN, power given	L, the cluster; S, the organism around it
The big organisms with the real cluster inside, 1,000 copies on Azure	omni-v3	155.0%	GAIN	L, the cluster; S, the organism around it

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Benchmark	Engine	Its one number	Its word	Class
The 945 modelled muscles as one tower, every muscle written at once	omni-v3	0.3%	GAIN	S
Robustness: the two-hour run	omni-v21	3.8%	GAIN AND LOSS	L
Robustness: the governor killed outright mid-run	omni-v21	8.1%	GAIN AND LOSS	L
Robot arms (MuJoCo), the three untouched robots	omni-v21	3.8%	GAIN	S
Redis, the cache's memory ceiling	omni-v21	0.1%	GAIN AND LOSS	L
Real Kubernetes, steady work	omni-v21	30.9%	GAIN	L
Real Kubernetes, faults	omni-v21	30.0%	GAIN, power given	L
Real Kubernetes, demand that wanders	omni-v21	33.6%	GAIN, power given	L
Real Kubernetes, all four at once	omni-v21	33.5%	GAIN AND LOSS	L
Real Kubernetes, a service rightsized by Goldilocks and the VPA's recommender, Omni-Compass on top, ten pairs, three runs	omni-v21	34.7%	GAIN AND LOSS	L
Real Kubernetes, a service rightsized by CruiseKube applying its own recommendations, Omni-Compass on top, ten pairs, three runs	omni-v21	0.4%	LOSS	L
Real Kubernetes, a queue of jobs	omni-v21	9.9%	GAIN AND LOSS	L
Real Kubernetes, a public day of demand (Google 2011)	omni-v21	37.6%	GAIN AND LOSS	L
Real Kubernetes, a noisy neighbour	omni-v21	2.2%	GAIN AND LOSS	L

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Benchmark	Engine	Its one number	Its word	Class
Real Kubernetes, With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass on top, ten pairs, three runs	omni-v21	94.7%	GAIN, power given	L
Real Kubernetes, With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs	omni-v21	29.8%	GAIN AND LOSS	L
Power grids (SimBench), both load models	omni-v21	5.8%	GAIN, power given	S
PostgreSQL, the pooler's pool size	omni-v21	4.2%	GAIN, power given	L
MySQL, the buffer pool's size	omni-v21	2.2%	GAIN, power given	L
MongoDB, the storage engine's cache size	omni-v21	0.5%	GAIN AND LOSS	L
Kafka, the consumer group's size	omni-v21	1.2%	GAIN AND LOSS	L
Drone swarms, the three untouched cells	omni-v21	3.0%	GAIN	S
Buildings and batteries (CityLearn), the districts with batteries	omni-v21	1.5%	GAIN	S

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

75.1 The engine that stands

omni-v23 (`OMNI_V23.json`, digest `c96ea4105d8747db`, 65 files; `docs/OMNI_V23.md`), on main from 31 October: omni-v22 with one guard in the Kubernetes governor's pod muscle of its own, on the founder's order of that day that Omni direct keeps every muscle Omni-Compass cannot be. A write of a workload's own scale that the cluster refuses is recorded and the decision goes on; at three refusals in a row the knob goes back to native's own pod autoscaler, made from the operator's target and range recorded on the workload, and the muscle leaves the workload to it, so the organism never runs with a knob nobody moves. 64

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

of its 65 files are omni-v22's bytes; the model gate against omni-v22 (`results/model_gate/V23_VS_V22.md`, class S) read identical on every gauge of every model, no LOSS. Every benchmark runs again on it from 20:00 UTC on 31 October (`scripts/reruns.txt`); the omni-v22 runs started before run to their end as omni-v22's record.

omni-v22 (`OMNI_V22.json`, digest `0e23a0bc561b900a`, 65 files; `docs/OMNI_V22.md`), on main from 30 to 31 October: the five data stores' wires brought to the max. What a wire's own rule asks goes through unclamped, up to the operator's cover and never below native; memory is spent only where the store's own account of a miss (MySQL's performance schema timer on the data files' reads, WiredTiger's page-read time for the application's threads, the Redis application's own split) says the misses are at least the one key's floor of the service's time, and only while the host keeps its 5% cushion; a hand-back never goes below the working set's floor; Kafka's group has a twin of its arrivals and a reflex; PostgreSQL's pool is never taken below the pooler's own setting (`tools/store_brain.py`, the five `tools/run_*.py` wires, `tools/knob_verdict.py`, all in the fingerprint). The model gate against omni-v21 (class S) read no LOSS at the counted setting on every store wire; the Kubernetes governor's models read omni-v21's own numbers on every gauge (the one governor change, the hand-back tried five times against a transient failure of the cluster's API, moves no decision). Every benchmark runs again on it from the moment it reached main (`scripts/reruns.txt`, released by the conductor), the five stores first. omni-v21 (`docs/OMNI_V21.md`, the immune system knows itself, knob by knob) stood from the evening of 25 October to 30 October; its tables stand as its own record until omni-v22's three runs of each benchmark land. The block above is written from the result files on every push: each row names the engine its table ran on, and every three-run table of omni-v21 is rebuilt byte for byte from its archived runs by `tools/reproduce.py` (all 31 on 27 October). Each earlier engine is on its own page (`docs/OMNI_V<n>.md`); the chronology this page carried until 27 October is whole in `docs/history/STATE_OF_PLAY_2026-10-27.md`.

75.2 The whole system, every layer composed

`results/WHOLE_SYSTEM.md` (built by `tools/whole_system.py`) composes the layers from the tables that stand: each layer's ratio toward its gauge's aim, $R\ell(g) = o\ell/n\ell$ or $n\ell/o\ell$, measured with the layer under it as native runs it, composed by the product law $R(g) = \prod_\ell R\ell(g)$, every run's paired interval carried by the delta method, the three runs pooled by inverse variance. It is class S over class P and L tables and across engines, and says so; the stacked run, Omni-Compass at every layer at once on one machine (the GPU's stacks mode, `lambda-gpu.yml`), is the measurement that replaces it. The numbers stand on that page, never copied here.

75.3 Open

Each with its receipt, run by run, in `docs/RUN_LEDGER.md`.

1. **The card.** omni-v21 on an A100, three runs each (`results/gpu/V21_A100_SAVE_ABC.md`, `V21_A100_CAP_ABC.md`, `V21_A100_CAP_FULL_ABC.md`). At the factory dial (the max)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

STEADY with no write: native already runs both knobs at their top. The tune-down (dial save): work per energy GAIN by the mean of our three runs (up 0.75%, beyond the noise in 1 of 3), every service gauge STEADY. The cap at 70%: the mean response time LOSS by the mean (beyond the noise in 0 of 3, run A alone). The tune-down as the card's factory dial goes through the model gate as the next version on the founder's word; the cap's rows are on the next version's list. The stacked box (three A10s, `results/gpu/V21_STACKS_A10_ABC.md`): at 1 and 10 copies, complete in all three runs, the card STEADY in every organism; 100 copies one repetition a run, shown, not judged. Where the chip proof stands, what the card's own files say about the mechanism, and what it still needs, step by step with its cost: `docs/THE_CHIPS.md`.

2. **The three-arm showing on omni-v21** landed (`results/orchestrator/V21_THREE_ARMS_ABC.md`, runs 40474220398, 40474223636, 40474227699): Omni-Compass on top of native against native, work GAIN in all three of our runs (up 8.5%), the 95th percentile GAIN in all three (down 13.0%); the answer at the top of the README reads it.
3. **Azure**: the omni-v18 set of 100 paired repetitions finished 29 October (97 passed, 3 failed) as omni-v18's record; the omni-v21 steady set runs (16 of 100 by 30 October, 16:27 UTC: 14 passed, 2 failed, about 68 minutes a repetition); the omni-v22 sets queue behind it, one at a time, each behind Azure's own $0 check (`scripts/azure_queue.txt`; `docs/THE_CHIPS.md` section 6).
4. **Google and Amazon**: wired as Azure's sets are and switched off; GitHub's own check of 27 October (`cloud-check`) reads their four keys absent from this repository's secrets (`GCP_CREDENTIALS`, `GCP_PROJECT`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`; `docs/GOOGLE_SETUP.md` section 4, `docs/AMAZON_SETUP.md` sections 1a to 3).
5. **The CPU package** (`cpu-power`): needs a bare-metal machine; GitHub's are virtual, with no frequency governor and no energy meter, and the workflow fails rather than hide it.
6. **The three rows still on omni-v3** in the block above, until their omni-v21 runs land.
7. **Every LOSS row on omni-v21**, each with its cause in `docs/LOSS_LEDGER.md`: the next version's work, through the model gate first.
8. **Omni-Compass 1.0**: when the founder declares the engine final, the engine that stands then is published as 1.0 and the older fingerprints go to `docs/history` as the road to it.

75.4 Where things are

Path	What it is
`docs/OMNI_COMPASS_MANUAL.md` (PDF: `docs/OMNI_COMPASS_MANUAL.pdf`)	the Operator's Manual: how to embed, wire, tune, run and check Omni-Compass; every muscle's wiring and card (Appendix E)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Path	What it is
`docs/OMNI_COMPASS_OWNERS_MANUAL.md` (PDF: `docs/OMNI_COMPASS_OWNERS_MANUAL.pdf`)	the Owner's Manual: the theory, the mechanism, the engine's source, every result, every muscle (Appendix G)
`docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.md` (PDF: `docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.pdf`)	the Live Engine Manual: how to read the live engine page, its monitors, gauges and knobs, native beside Omni-Compass; what to do when a tile reads LOSS
`docs/BENEFIT_SHEET.md`	one number per benchmark, its word, its engine, every change in words
`docs/LEVELS.md`	every level on one page (muscle, organism, cluster, add-ons, service, plant, cloud, group of clusters), its counts of GAIN, STEADY, power given and LOSS, and any level whose gauges go the other way
`docs/REGISTER.md`	every muscle family wired, every benchmark run, every benchmark still to run
`docs/WIRING_VERDICTS.md`	write, watch or the operator's choice, per knob, from every result
`docs/OMNI_V22.md`	the engine that stands, what changed and how it was read; each earlier engine's page beside it (`docs/OMNI_V<n>.md`)
`docs/RUN_LEDGER.md`	every chip, cloud and machine run, what it cost, whether it finished, and where its receipt is
`docs/THE_CHIPS.md`	the chips: every GPU run, what the card shows by the one key, what its own files say about the mechanism, and what the chip proof still needs with each step's cost
`docs/AUDIT_2026-10-27.md`	the repository audited top to bottom on 27 October: every finding, its fix at its source, and what holds it
`docs/AUDIT_2026-10-30.md`	the repository graded top to bottom on 30 October: every area's grade on its record, what broke and was fixed, the founder's week, and what is open
`docs/REPORT_CARD.md`	the report card: every section and piece of the repository graded on its record on the engine that stands, with the count of everything and what moves each grade up

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Path	What it is
`docs/history/STATE_OF_PLAY_2026-10-10.md`	this page as it stood on 10 October, on Omni v3, moved here whole
`docs/history/STATE_OF_PLAY_2026-10-27.md`	this page's chronology as it stood on 27 October, moved here whole

PART EIGHT

Value, License and History

What a receipt is worth, how the license is priced against it, how the code is sealed, and how Omni-Compass came to be.

CHAPTER 76

Where the Value Comes From

76.1 The problem we are attacking

Computing has run into a wall that is not made of silicon. Data centers cannot get the power they need, cannot get it fast enough, and are running out of room to put more machines. AI made it harder: every new model needs more chips, and every chip needs more electricity and more cooling. The answers on the table are expensive and slow: new chips that do more work per watt (a new generation, new hardware to buy), new power plants and power contracts (billions, and years), and point fixes at a single layer (an AI serving trick, a scheduler, a power cap on one chip).

We attack it from the other side. Every system already running wastes room it paid for:

- GPUs boost to the top of their clock range and are knocked back by their own power limiter many times a second;
- Kubernetes keeps replicas and machines sized for the worst minute of the day;
- cooling runs colder than the heat requires; batteries hold more reserve than the hour needs.

That room is paid for in energy, in machines and in floor space, and it produces nothing.

Why the room is there is worth a paragraph, because it decides what can be done about it. Each layer of a stack is sized by someone who cannot see the others. The autoscaler's author does not know what the power cap will

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

do; the cooling engineer sizes for the hottest hour of the year; the database operator sets a buffer pool for the largest table and leaves it there; each of them, sensibly, leaves a margin against the layer next door, and the margins add up. None of these people is wrong. The waste is not a mistake anyone made; it is the price of layers that cannot talk to each other, and it is why a fix at one layer, however good, moves the margin rather than removing it. The benchmarks in this manual are each a measurement of one such margin on real software: the connections a pooler holds open that no client is using, the memory a cache keeps for data it will not be asked for, the machines a cluster keeps in service for a minute that has passed. Each is small on its own and each is paid for every hour of the year.

The attack, then, is not another autoscaler or another cache. It is a governor that reads every layer's own gauge, keeps each layer in the middle of its own band, and gives the margin back where the layer's own controller shows it is not needed, with the layer's own controller still underneath and the layer's own setting restored when the governor is gone. That is a different kind of thing from a point fix, and it is why it stacks on top of every point fix a data center has already made rather than competing with any of them.

76.2 What Omni-Compass is, in one paragraph

Omni-Compass is software that sits **on top of** the controllers a system already has (the GPU's firmware, the Kubernetes autoscaler, the building's thermostats) and governs every layer under one law: a bounded push and pull that keeps each service in the middle of its band instead of far below its limit. It never replaces the native controller, never asks for more than the native settings allow, hands every knob back the moment it is switched off or loses a sense, and refuses any step that measures a LOSS of more than 2% on anything. No new hardware. It stacks on top of every other fix a data center makes.

76.3 The benefit: more work from what is already paid for

The room that was spent on nothing becomes one of two things, and a company chooses which:

- **More work for the same cost:** the same machines and the same watts carry more work; or
- **The same work for less:** fewer machines in service, fewer watts, a smaller bill.

They are the same gain read from two sides (3.4). The health side comes with it: faster answers, faster recovery from failures, nothing waiting, every knob returned, no human babysitting the stack.

The results to date (Chapter 60) show both readings on real software, and show that which one a stack gets is decided by the stack, not by Omni-Compass. On the real Kubernetes cluster the gain was taken as **more work, faster**: the cluster carried more requests inside its response line on the same machines. On PostgreSQL behind its pooler, on MySQL's buffer pool and on MongoDB's storage-engine cache the gain was taken as **the same work for less**: the same transactions answered inside the same line with fewer connections open, a smaller pool or a smaller cache, each notch proved on the stack first and the resource handed back at the end of every run. On Apache Kafka the earlier set had taken the gain as more work, faster, at the cost of three to four times the consumers, and on Redis the compass had bought the hit rate with three to four times the memory; under the brain's own verdict on the knob (10 October) neither spend was judged on the first set, because the trial

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

rule of the day ended a spend trial on the calm the spend itself produced, so both read exactly nothing until the amended set lands (section 60.7). The combined index (section 60.11) is +4.8% over the six real categories on the sets of 10 October, with no loss in all three of our runs in any category. An operator should expect the same honesty from their own paired run: the receipts will say which side of the gain their stack took, and whether the compass's own cost (its CPU, its reads) ate into it.

76.4 How to read the benefit: the arithmetic

Every comparison is paired: the same system, the same work, the same moments, native alone against native with Omni-Compass on top. Let

- *W* be the work served (requests answered, tokens generated), held equal in both arms;
- *C* be a resource it used: machine-hours, CPU core-hours, or joules.

Productivity is work per resource, $P = W / C$. With the work held equal, the gain from Omni-Compass on top is

$G = P[omni] / P[native] − 1 = C[native] / C[omni] − 1.$

A gain *G* means the same resources can carry *G* more work. Read from the other side, the same work needs only

$C[omni] / C[native] = 1 / (1 + G)$

of the resources, a saving of $S = G / (1 + G)$. So **a third more work (G = 33%) is a quarter off the bill (S = 25%)**; 20% more work is 16.7% off; 50% more work is a third off. Both columns are the same measurement.

Three rules for reading any receipt:

1. **The work must be equal.** Every receipt states that the load was fixed-rate (open loop): the same requests at the same moments in every arm. Without that, a lower resource count could just mean less work was done.
2. **Nothing may read LOSS.** Beside every gain the receipt prints response times, failures, waiting and the controller's own CPU, each with its 95% interval. Our one rule (`DISCLOSURES.md`, section 3): no LOSS of more than 2% on any measure, and only where energy or the bill is saved. A GAIN paid for with a LOSS of service is not a GAIN.
3. **Know the class and the comparator.** A machine-hour gain on kind (where idle machines stay powered and native has no node autoscaler) is not yet a bill; a modelled energy line is not a meter. Chapter 58 lists the classes; Chapter 60 says, for every result, what it is measured against.

Three worked examples, from the tables. In the all-four Kubernetes test on v3, the same ten machines carried 35% to 49% more requests inside the response line: *G* is 0.35 to 0.49, so the same work would need 1/1.35 to 1/1.49 of the machines, a saving *S* of 26% to 33%, had the fleet been free to shrink; on kind it was not, which is why the table reports the work column and not a bill. In the database test, the pooler held 61% to 72% fewer connections open for the same work: the resource there is connections, $C[omni] / C[native]$ is 0.28

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

to 0.39, and the gain read as "more work for the same connections" would be G = 1.6 to 2.6, which the table reports as the machines-equivalent column, "fewer connections by". In the Kafka test the work inside the line rose 16% to 21% while the consumers running rose by a factor of 2.6 to 4: there the gain in work was bought with resources, and the table reports both, the one as GAIN in all three of our runs and the other as power given in all three of our runs, because that is what happened.

76.5 Where the evidence stands today, read honestly

Every number below is from a three-run table, each row naming the engine its table ran on; the evidence class of each is in the table it points to. The engine that stands is **omni-v23** (`OMNI_V23.json`, digest `c96ea4105d8747db`, 65 files; `docs/OMNI_V22.md`, on main since 30 October): the five data stores' wires at the max, each spend gated on the store's own account of what a miss costs and on the host's room, never below native (`tools/store_brain.py`); every table below is omni-v21's or an earlier engine's own record until omni-v22's three runs of each benchmark land. omni-v21 (`docs/OMNI_V21.md`, 25 to 30 October): the immune system knows itself, knob by knob (a knob redlining past its line that Omni-Compass's own pushing cannot bring back goes to native where native's twin reads it lower, and stays with Omni-Compass where native is not stronger). What each earlier version changed is in section 76.6 and on its own page (`docs/OMNI_V<n>.md`). Every benchmark runs again on the engine that stands; a row still on an older engine names it and stands as that engine's record until its three runs on the engine that stands land. Results are never read across engines.

The table is written from the benefit sheet (`docs/BENEFIT_SHEET.md`, `tools/benefit_sheet.py`) on every push, so this book quotes no number its tables have not given. Every table it names is rebuilt byte for byte from its archived runs by `tools/reproduce.py`, and every LOSS row is named with its cause in `docs/LOSS_LEDGER.md`. The questions a referee is entitled to ask of a row (whose load, which native, which window, what was spent to buy what) are answered in the table it points to.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
Every real test together: the Omni index (omni-v21)	**GAIN AND LOSS**	5.0 % GAIN	the six real categories together; each stack's own words are in its row	`results/OMNI_INDEX.md`
The six organisms with the real cluster inside, 10 and 100 copies (omni-v3)	**GAIN, power given**	5.3 % GAIN	p95 cut 32% to 52% (GAIN); time over the line cut 8% to 36% (GAIN); replicas up 0.6% (power given); failed requests cut 8.4% to 8.5% (GAIN); parked-worker energy cut 0.3% (GAIN); standby-model energy	`results/live/V3_SIX_KUBE.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
			cut 0.3% (GAIN)	
The big organisms with the real cluster inside, 1,000 copies on Azure (omni-v3)	**GAIN**	155% GAIN	p95 cut 95% to 98% (GAIN); p99 cut 97% to 98% (GAIN)	`results/live/V3_BIG_ORGANISM.md`
The 945 modelled muscles as one tower, every muscle written at once (omni-v3)	**GAIN**	0.3% GAIN	work per energy up 0.3% (GAIN)	`results/realms/REALMS.md`
Robustness: the two-hour run (omni-v21)	**GAIN AND LOSS**	3.8% LOSS	mean response cut 29% to 43% (GAIN); p95 cut 25% to 57% (GAIN); time over the line cut 30% to 41% (GAIN); failed requests cut 9% to 15% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 21% to 28% (GAIN); replicas up 29% to 50% (power given); pods started up 68% to 226% (power given); pod start wait, total (s) up 90% to 324% (LOSS); pod start wait, mean (s) up 24% to 27% (LOSS)	`results/live/V21_ROBUST_LONG.md`
Robustness: the governor killed outright mid-run (omni-v21)	**GAIN AND LOSS**	8.1% GAIN	mean response cut 30% to 40% (GAIN); p95 cut 28% to 48% (GAIN); time over the line cut 33% to 35% (GAIN); failed requests cut 9% to 13% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 18% to 25% (GAIN); replicas up 2.3% to 7.3% (power given); pod start wait, total (s) up 10% to 79%	`results/live/V21_ROBUST_KILL.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
			(LOSS); robust: failed requests in the 300 s after the kill (%) cut 3.7% to 4.3% (GAIN); robust: time over the line in the 300 s after the kill (% of samples) cut 0.5% to 2.6% (GAIN)	
Robot arms (MuJoCo), the three untouched robots (omni-v21)	**GAIN**	3.8 % GAIN	energy per takt cut 0.8% (GAIN); copper heating per cycle cut 11% (GAIN); mechanical work per cycle cut 6.6% (GAIN); peak joint torque cut 29% (GAIN); tracking error, RMS cut 21% (GAIN); end-point error at the waypoints cut 11% (GAIN)	`results/live/V21_MUJOCO.md`
Redis, the cache's memory ceiling (omni-v21)	**GAIN AND LOSS**	0.1 % GAIN	memory ceiling held cut 1.3% to 2.4% (GAIN); memory used cut 0.1% to 1.0% (GAIN); host CPU busy cut 2% to 11% (GAIN); host CPU-seconds cut 2% to 12% (GAIN); CPU per 1,000 inside the line cut 2% to 12% (GAIN); work inside the line up 0.6% to 1.7% (GAIN); hit rate up 0.7% to 1.7% (GAIN); mean cut 0.6% to 7.8% (GAIN); memory ceiling held up 0.8% to 2.2% (power given); memory used up 0.5% to 1.9% (power given); p99 up 0.2% to 1.7% (LOSS); host CPU busy up 1% to 11% (power given); host CPU-seconds up 1% to 12% (power given); CPU per 1,000 inside the line up 0% to	`results/live/V21_REDIS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
			11% (power given); process_cpu_seconds up 0.3% to 2.4% (power given)	
Real Kubernetes, steady work (omni-v21)	**GAIN**	31% GAIN	mean response cut 48% to 49% (GAIN); p95 cut 66% (GAIN); p99 cut 72% to 74% (GAIN); time over the line cut 100% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 100% (GAIN); replicas cut 4.1% to 6.3% (GAIN); pods started cut 20% to 49% (GAIN); pod start wait, total (s) cut 20% to 54% (GAIN); pod start wait, mean (s) cut 0% to 21% (GAIN)	`results/live/V21_STEADY.md`
Real Kubernetes, faults (omni-v21)	**GAIN, power given**	30% GAIN	parked-worker energy cut 0.3% to 1.0% (GAIN); standby-model energy cut 0.3% to 1.0% (GAIN); mean response cut 38% to 48% (GAIN); p95 cut 51% to 58% (GAIN); p99 cut 21% to 60% (GAIN); time over the line cut 28% to 34% (GAIN); failed requests cut 11% to 18% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 14% to 22% (GAIN); replicas up 10% to 20% (power given); pods started up 12% to 22% (power given)	`results/live/V21_FAULTS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
Real Kubernetes, demand that wanders (omni-v21)	**GAIN, power given**	34% GAIN	mean response cut 40% to 45% (GAIN); p95 cut 56% to 59% (GAIN); p99 cut 11% to 35% (GAIN); time over the line cut 35% to 40% (GAIN); failed requests cut 12% to 15% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 22% to 29% (GAIN); pod start wait, mean (s) cut 0.5% to 9.9% (GAIN); replicas up 0.1% to 2.9% (power given)	`results/live/V21_WANDERING.md`
Real Kubernetes, all four at once (omni-v21)	**GAIN AND LOSS**	34% GAIN	work inside the line up 30% to 52% (GAIN); mean response cut 36% to 43% (GAIN); p95 cut 51% to 59% (GAIN); time over the line cut 32% to 37% (GAIN); failed requests cut 5.1% to 5.4% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 16% to 24% (GAIN); pod start wait, total (s) up 16% to 42% (LOSS); pods started up 8% to 37% (power given)	`results/live/V21_ALL_FOUR.md`
Real Kubernetes, a service rightsized by Goldilocks and the VPA's recommender, Omni-Compass on top, ten pairs, three runs (omni-v21)	**GAIN AND LOSS**	35% GAIN	parked-worker energy up 1.5% to 1.8% (power given); standby-model energy up 1.5% to 1.8% (power given); mean response cut 45% to 47% (GAIN); p95 cut 59% to 61% (GAIN); p99 cut 62% to 65%	`results/live/V21_GOLDILOCKS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
			(GAIN); time over the line cut 96% to 100% (GAIN); replicas up 20% to 29% (power given); pods started up 44% to 103% (power given); pod start wait, total (s) up 75% to 137% (LOSS); pod start wait, mean (s) up 11% to 32% (LOSS)	
Real Kubernetes, a service rightsized by CruiseKube applying its own recommendations, Omni-Compass on top, ten pairs, three runs (omni-v21)	**LOSS**	0.4% LOSS	mean response up 0.9% to 2.4% (LOSS); p95 up 0.0% to 1.8% (LOSS); time over the line up 4.3% (LOSS)	`results/live/V21_CRUISEKUBE.md`
Real Kubernetes, a queue of jobs (omni-v21)	**GAIN AND LOSS**	9.9% GAIN	machines cut 13% to 15% (GAIN); machine-hours cut 13% to 15% (GAIN); standby-model energy cut 9% to 10% (GAIN); mean response cut 12% to 13% (GAIN); p95 cut 0.4% to 5.7% (GAIN); time over the line cut 0.2% to 4.8% (GAIN); machines after the queue cut 19% to 22% (GAIN); replicas up 4% to 16% (power given)	`results/live/V21_BATCH.md`
Real Kubernetes, a public day of demand (Google 2011) (omni-v21)	**GAIN AND LOSS**	38% GAIN	mean response cut 50% (GAIN); p95 cut 59% to 64% (GAIN); p99 cut 48% to 56% (GAIN); time over the line cut 75% to 80% (GAIN); failed requests cut 10% to 18% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over,	`results/live/V21_TRACE_GOOGLE2011.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
			was over the line) cut 61% to 72% (GAIN); pod start wait, total (s) up 88% to 279% (LOSS); replicas up 2% to 19% (power given); pods started up 79% to 235% (power given)	
Real Kubernetes, a noisy neighbour (omni-v21)	**GAIN AND LOSS**	2.2 % GAIN	mean response cut 12% to 17% (GAIN); p95 cut 3% to 12% (GAIN); time over the line cut 9% to 11% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 1.2% to 6.2% (GAIN); pods started cut 9% to 28% (GAIN); second app: response time (ms), 99th percentile cut 1% to 16% (GAIN); pod start wait, mean (s) up 5% to 51% (LOSS); second app: failed requests (%) up 1.1% to 3.7% (LOSS); replicas up 0.1% to 2.9% (power given)	`results/live/V21_FAIRNESS.md`
Real Kubernetes, With its add-ons: KEDA's HTTP add-on on the live requests in flight, Omni-Compass on top, ten pairs, three runs (omni-v21)	**GAIN, power given**	95% GAIN	parked-worker energy up 1.9% to 2.4% (power given); standby-model energy up 1.9% to 2.4% (power given); mean response cut 57% to 58% (GAIN); p95 cut 73% to 74% (GAIN); p99 cut 60% to 66% (GAIN); time over the line cut 95% to 97% (GAIN); alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line) cut 94% to 100% (GAIN);	`results/live/V21_KEDA_REQUESTS.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
			replicas cut 15% to 21% (GAIN); pods started cut 21% to 25% (GAIN); pod start wait, total (s) cut 22% to 26% (GAIN); pod start wait, mean (s) cut 1.3% to 4.8% (GAIN)	
Real Kubernetes, With its add-ons: KEDA on the CPU target and the live requests in flight, Omni-Compass on top, ten pairs, three runs (omni-v21)	**GAIN AND LOSS**	30% GAIN	parked-worker energy up 1.4% to 1.7% (power given); standby-model energy up 1.4% to 1.7% (power given); mean response cut 37% to 38% (GAIN); p95 cut 51% to 52% (GAIN); p99 cut 54% to 58% (GAIN); time over the line cut 91% to 100% (GAIN); replicas up 11% to 16% (power given); pods started up 31% to 49% (power given); pod start wait, total (s) up 27% to 84% (LOSS)	`results/live/V21_KEDA_CPU_REQUESTS.md`
Power grids (SimBench), both load models (omni-v21)	**GAIN, power given**	5.8 % GAIN	energy the loads drew cut 1.3% to 1.5% (GAIN); energy dissipated in lines and transformers cut 0.5% to 2.4% (GAIN); net import from the upstream grid cut 1% to 15% (GAIN); tap operations cut 16% to 38% (GAIN); energy dissipated in lines and transformers up 0.6% to 1.5% (power given); tap operations up 100% (power given)	`results/live/V21_PANDAPOWER.md`
PostgreSQL, the pooler's pool size (omni-v21)	**GAIN, power given**	4.2 % GAIN	connections held open cut 1.2% to 7.5% (GAIN); host CPU busy up 0.4% to 1.3% (power given); host CPU-seconds up 0.4% to	`results/live/V21_PGBENCH.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
			1.4% (power given); CPU per 1,000 inside the line up 0.5% to 1.3% (power given); connections most at once up 0.0% to 8.3% (power given); p95 cut 0% to 64% (GAIN); mean cut 0% to 43% (GAIN); CPU per 1,000 inside the line cut 0.3% to 1.6% (GAIN)	
MySQL, the buffer pool's size (omni-v21)	**GAIN, power given**	2.2% GAIN	p95 cut 1.8% to 3.5% (GAIN); p99 cut 0.6% to 4.1% (GAIN); pool held cut 4% to 22% (GAIN); pages holding data cut 1% to 21% (GAIN); host CPU busy up 0.2% to 2.6% (power given); host CPU-seconds up 0.0% to 3.0% (power given); CPU per 1,000 inside the line up 0.3% to 3.7% (power given); pages holding data up 0.4% to 6.0% (power given); host CPU busy cut 2% to 38% (GAIN); host CPU-seconds cut 2% to 38% (GAIN)	`results/live/V21_SYSBENCH.md`
MongoDB, the storage engine's cache size (omni-v21)	**GAIN AND LOSS**	0.5% LOSS	host CPU busy cut 0.2% to 3.8% (GAIN); host CPU-seconds cut 0.2% to 4.7% (GAIN); CPU per 1,000 inside the line cut 0.2% to 4.7% (GAIN); process_cpu_seconds up 0.2% to 2.0% (power given); mean cut 0.4% to 4.0% (GAIN); p95 up 0.2% to 5.4% (LOSS); p99 up 0.4% to 2.5% (LOSS); cache size held up 0.4% to 7.5%	`results/live/V21_YCSB.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit?	How much	Every change judged across our three runs, in words	Table
			(power given); cache in use up 0.3% to 7.2% (power given)	
Kafka, the consumer group's size (omni-v21)	**GAIN AND LOSS**	1.2% LOSS	p99 cut 0.1% to 3.0% (GAIN); median cut 0.0% to 4.3% (GAIN); mean cut 0.1% to 5.1% (GAIN); messages waiting, mean cut 0.4% to 6.0% (GAIN); host CPU busy cut 1.2% to 2.3% (GAIN); host CPU-seconds cut 1.3% to 2.3% (GAIN); CPU per 1,000 inside the line cut 2.0% to 2.5% (GAIN); mean up 0% to 12% (LOSS); messages waiting, mean up 0% to 11% (LOSS); consumers running up 0% to 17% (power given); consumers most at once up 0% to 83% (power given); p99 up 0.6% to 9.1% (LOSS)	`results/live/V21_KAFKA.md`
Drone swarms, the three untouched cells (omni-v21)	**GAIN**	3.0% GAIN	energy per mission cut 13% to 17% (GAIN); fleet energy over the window cut 13% to 17% (GAIN); missions per charge at the autopilot's reserve up 14% to 21% (GAIN)	`results/live/V21_SWARM.md`
Buildings and batteries (CityLearn), the districts with batteries (omni-v21)	**GAIN**	1.5% GAIN	electricity bought cut 3% to 14% (GAIN); daily peak draw cut 3% to 17% (GAIN); highest peak cut 2.7% to 4.4% (GAIN); ramping cut 3% to 10% (GAIN); daily load unevenness cut 0.9% to 6.0% (GAIN); monthly load unevenness cut 0.8% to 2.3% (GAIN);	`results/live/V21_CITYLEARN.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What (engine, three runs each)	Benefit ?	How much	Every change judged across our three runs, in words	Table
			electricity bill cut 16% (GAIN); carbon cut 12% (GAIN); distance from zero net energy cut 1.0% (GAIN)	

What is not in the table yet, and why, is in `docs/RUN_LEDGER.md`: the card's rentals and what came back from each, the clouds (Azure's bill at the tune-down; Google and Amazon wired and not yet run) and the CPU package, which needs a bare-metal machine. The bill becomes a claim, not a direction, when two tests land: the bill on a real cloud with a fleet big enough to show one machine, and the real card's own meter. We publish each of them as it comes, whatever it says.

76.6 One law, versioned; and what came before it

The engine in every result of this manual is **the compass law with the verdict** (chapter 30 and section 1.2), versioned and fingerprinted at every change: frozen as Omni v1 on 5 October 2026; grown to the 945-muscle catalog as v2; given the slack gate on speed knobs as v3; redesigned as **v5** (12 and 13 October) with the max as its default brain on the 1,107-muscle catalog, every muscle its own card, the living body and the lease's walk back (v4 was the written plan of that redesign, `docs/OMNI_V4_PLAN.md`, never fingerprinted); and given the shared-host capacity repair of the Kubernetes controller as **v6** (15 October); and given conveyance bounded by each namespace's quota, the verdict in the reports' words, every rule in seconds and the cloud's one flag as **v7** (16 October), tried in the model gate and in live trials first; given, as **v8** (17 October), the one wire omni-v7's CityLearn table showed was ours to fix: a battery that is also its building's reserve through an outage stays native; given, as **v9** (17 October), the battery wire reading the coming hour; given, as **v10** (18 October), the battery wire's glide at the speed that cuts the bill most with no LOSS, each proven on CityLearn and in the model gate before it was pushed; and given, as **v11** (18 October), by the founder's order that every fault found be fixed, the Kubernetes governor's hold on a reading with no machine in service, proven in a unit test that reproduces the fault and in the model gate; then, each the next version through the model gate, a raise the cluster refuses held and the swarm's reflex at once (**v12**), the limits handed back on a shared host at its top (**v13**), a knob that leaves any muscle native (**v14**), the power cap's request through the line's own clear time and every wire in the fingerprint (**v15**), every clock of the governor in seconds (**v16**), the heartbeat a dial with the main engine at 60 s (**v17**), Omni direct taken into the engine (**v18**), a pod muscle of its own (**v19**), the five data stores' wires fixed at their causes (**v20**), and, as **v21**, the engine that stands since 25 October, the immune system knowing itself, knob by knob (`docs/OMNI_V1.md`, `OMNI_V2.md`, `OMNI_V3.md`, `OMNI_V5.md` to `OMNI_V21.md`). From v11 on, a version's bookkeeping lands in one command (`tools/land_engine.py`), and the check it adds to `verify.py` refuses a push in which any place still names an older engine as the one that stands. Earlier engines, among them an allocation law that gave back about a third of the machines at the cost of response time and a compass law without the verdict, produced the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

numbered sets kept in `docs/history/`; their results are history, labelled as the engine that produced them, and are never read beside a later engine's. A buyer does not choose between modes: there is one law, and every table names the version of it that produced the table. When the founder declares the engine final, the engine that stands then is published as Omni-Compass 1.0, with the older fingerprints kept as the road to it, never as a second product.

"Versioned" has a precise meaning here. The engine is a fixed set of files (63 on v21), and `OMNI_V21.json` holds one SHA-256 for each of them and one digest over all of them; `OMNI_V20.json` back to `OMNI_V5.json`, `OMNI_V3.json`, `OMNI_V2.json` and `OMNI_V1.json` do the same for the earlier sets. `python3 tools/omni_version.py` hashes the files in a checkout and prints which fingerprint they match (for this edition, `omni-v23 (digest c96ea4105d8747db, 65 files)`), or, when they match none, which files differ from the nearest; with `--commit <sha>` it asks the same of any pushed commit, which is how every three-run table checks that its runs A, B and C were on the same engine before it combines them, and refuses if they were not. The versions differ in exactly the ways their records name, and a reader can check that from the fingerprints rather than from this sentence.

The rule that any change makes the next version is also the rule that keeps the results honest. A gain or a guard changed after a result is in would make that result unrepeatable on the engine that now stands, so the change moves the version, every result is run again on the new version, and the old tables keep their version in their name and move to history. The founder's standing order is that the engine is never changed without being told first; since 15 October every change is also tried in our own models against the engine that stands before any real run (`tools/model_gate.py`). This manual records no change made otherwise.

76.7 The output follows the input

Every number Omni-Compass produces is the result of three inputs, and a reader who changes any one of them has changed the result: **the tune** (the factory tune as shipped, or the dial turned down, or a card changed), **the wiring** (which muscles are wired in and which are watched, what each wire reads and writes, how fresh each feed is) and **the load** (the work offered, its shape and its line). The published receipts are the record of one setting of all three, and every receipt names it: the engine's fingerprint, the harness, the load and the window. The same engine wired onto another stack, or tuned to another purpose, gives another result, and that result is its own.

A team that wants a different outcome sets the inputs for it and reads its own receipts against native: the most work at any spend (the max, the default, in a cloud as on site since 31 October); the same work as native for a smaller bill (the tune-down, a tune anyone may set); a muscle held at a limit of the team's own choosing (its card, or its cover); a knob left native (watched, never written). Each of those is a legitimate setting of the inputs, and each has its own result. What no reader may do is read one tune's receipt as another's, or read a receipt without its inputs. The factory tune is the one setting whose receipts this book publishes (the Declaration of the factory tune, `DISCLOSURES.md`, section 3); the dial and the cards are the inputs a licensee may change, and a changed tune's results are its own (section 76.6). Where a result read elsewhere

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

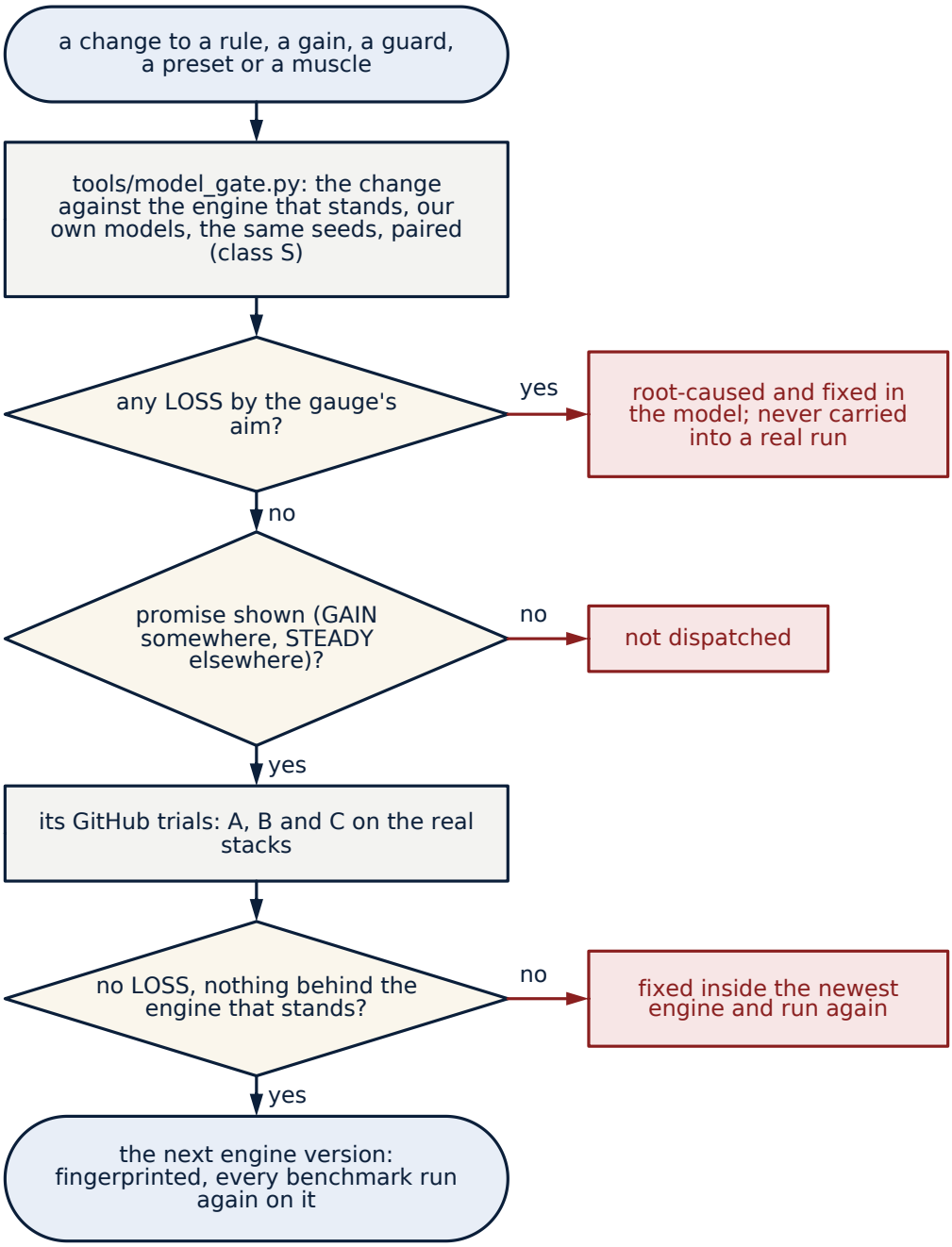


Figure 77. Flowchart: the model gate before any engine change runs for real. A change to a rule, a gain, a guard, a preset or a muscle runs first in our own models against the engine that stands, on the same seeds, paired (class S); a LOSS is root-caused and fixed in the model and never carried into a real run; a change that shows promise goes to its GitHub trials, and only one that reads no LOSS and nothing behind the engine that stands becomes the next fingerprinted version, on which every benchmark runs again.

differs from the published one, the first presumption is an input: the wiring, the settings or the reading (`DISCLOSURES.md`, section 4).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

76.8 Healthy production, how a configuration is chosen, the whole-workload benefit, and the scope of the validation

The founder's master specification of 24 October 2026, late night, is written once in `docs/FACTORY_PROFILE.md`; this section carries what the owner must hold of it, in its words.

What Omni-Compass consists of. A **governing engine** (the executable mathematical rules that turn observations and internal state into governing decisions), its **muscle adapters**, a **complete configuration profile** and a **validation harness** (the machinery that supplies workloads, executes comparisons, checks requirements and records results). The **factory profile** is the complete, versioned set of Omni-Compass default settings shipped together: the governing rules, gains, margins, the decision interval and every other default. A **candidate profile** is a proposed combination of settings under evaluation; the **active profile** is the settings actually governing the current run; a **custom profile** is an identifiable configuration that differs from the released factory profile. The released factory profile is omni-v23's (`OMNI_V23.json`): the governor's full decision every 60 s, the fast checks every 5 s, every muscle on the card found for it (`realms/muscle_cards_v5.csv`) and on the engine's own setting otherwise; it is the profile every published result ran on. The physiological words of this book (muscle, organism, nervous system, defensive system, health) are engineering analogies, each pointing to concrete code, signals, actuators and checks (`docs/FACTORY_PROFILE.md`, section 2).

A fixed tune governing a changing system. Let x be the governed system's state, z the governor's internal state, y its observations, θ the complete configuration profile and Δ the full governing decision interval. The public interval choices on the bench are $\Delta \in \{10, 15, 20, 30, 60, 90, 120\}$ s (the controller itself takes any value from 5 to 120 s, `--interval`). At decision k, with $t_{k+1} = t_k + \Delta$, the general contract is:

Step	Equation	What it is
observe	$y(k) = h(x(t_k)) + \varepsilon(k)$	the observation, with its sensing delay and noise ε
decide	$(u[req](k), z(k+1)) = G[\theta](z(k), y(k), A(k))$	the requested settings, inside the governing authority A(k)
constrain	$u[cmd](k) = \Pi[C](u[req](k))$	the constraints C(k): every muscle's own limits, the cover, never below native at the max
land	$u[ack](k) = R(u[cmd](k))$	the setting that actually landed, read back by the adapter
move	$x(t_{k+1}) = \Phi(x(t_k), u[ack](k), w(k))$	the system moves with the command that landed, under the workload and disturbances w

G[θ] is the governing engine at profile θ. The system evolves with the command that actually landed, so the requested, the commanded and the acknowledged settings are kept distinct and each is logged (the audit; a refused write is held, omni-v12). After release θ(k + 1) = θ(k) = θ[factory] for every k, while x, z and u go on changing: one fixed factory tune governs a changing workload. "One setting" means one complete profile, θ =

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(the rules, the gains, the margins and cushions, Δ, the dial, every muscle's card, …), more than a choice of heartbeat, and its settings are evaluated jointly, because changing one mechanism can move others.

Healthy production: the two objectives. Health is the explicit operating requirements: correctness, service limits, resource bounds and every applicable hardware limit. With $P(\theta)$ the useful output (work done inside the line), $C(\theta)$ the whole-workload cost inside the declared boundary, $H_j(\theta)$ each health reading and L_j its limit, the two objectives are kept explicit, and each reported result names its objective and its exact profile:

maximise $E[P(\theta)]$ subject to $H_j(\theta) \le L_j$ for every health requirement j

(the most healthy production the system can deliver: the max, the default brain), or

minimise $E[C(\theta) / P(\theta)]$ subject to the same requirements

(what that completed production costs: the tune-down, `dial save`, a tune anyone may set; the clouds ran it until 31 October, when the founder struck that rule and every cloud runs the max, the bill judged beside the work).

How a configuration is chosen. The factory baseline is established in eight steps: (1) verify the mechanisms: observations, units, authority, actuation, read-back, limits and recovery; (2) exercise the six governing intervals and the relevant knob combinations; (3) tune on development workloads, examining the interactions between muscles; (4) validate the selected profile on fresh workloads, bursts, sustained demand and the declared failure conditions included; (5) record every adverse result and resolve whether the candidate meets the release requirements; (6) freeze the complete factory profile, its engine identity and its validation receipts; (7) verify every corresponding implementation and adapter, the C++ harness included; (8) release that identified package as the factory baseline. The result is called **the best validated factory profile for the declared operating conditions**, which gives "optimal" an assessable meaning. Today the repository is at steps 1 and 2 for the next profile (the heartbeat sweep and the beat ladder, `results/model_gate/HEARTBEAT_SWEEP.md`, `results/model_gate/V17_LADDER.md`, class S); steps 3 to 8 come after the founder confirms the mechanisms, and no profile is locked as the baseline before them. Every engine version already passes the model gate and the three-run rule before its results stand. The tune rule of section 12.4 is how a released tune is read against the tunes tried so far; it is not the eight steps.

The whole-workload benefit: the bill. For each deployment route r (on-site hardware; through the cloud), the costs inside the declared boundary are added, each counted once, for each arm a (native, Omni-Compass on native, Omni-Compass direct):

$C(r, a) = \Sigma_k\, c(r, a, k)$, $S(r, a) = (C(r, \text{native}) - C(r, a)) / C(r, \text{native})$,

for a positive native cost and a valid, matched workload comparison. The two headline views are the total native cost, the total Omni-Compass cost and the percentage saved, on site and through the cloud. A component's percentage is a supporting measurement; the bill's percentage comes from adding the actual costs. A missing price or an incomplete workload stays visibly unresolved: today only the cloud's compute is metered in money (Azure, machine-hours at the list price, `results/live/AKS_BILL.md`); energy,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

machines, storage, network and operation carry no recorded price, so neither route's total is resolved yet, and the live benchmark engine shows each component as unresolved beside its measured change. The whole-workload index of section 60.6 is the composition's one number over its gauges, not a price.

The scope of the validation. Every result identifies the engine version, the configuration, the hardware or model, the workload and the measurement boundary that produced it, and says nothing beyond them. The Python, the C++ and the browser port share one behavioural contract: the same profile values, meanings and units; the same governing intervals and time-based rules; the same authority and constraint decisions; the same handling of stale observations, refused commands and recovery; exact agreement for discrete decisions, with tolerances fixed in advance where floating-point arithmetic needs them (`tools/seal.py`, `results/SEAL.json`, `docs/engine/check_live_engine.py`). A release record identifies the engine version and its fingerprint, the profile, the implementations' parity receipts and the validation receipts: a matching fingerprint establishes identity; the validation receipts establish what that identified implementation demonstrated, on the systems, loads and boundaries they name. The live benchmark engine's adjustments are variations of the mechanism on its stated backend (class S on the page's own model); an adjustment becomes a factory configuration only after validation and a new release declaration.

CHAPTER 77

The Economics of a Receipt

Run the stack native and print the receipt. Run the same stack with Omni-Compass and print the receipt. The difference is the gain, read from either side: more work for the same energy, or the same work for less energy and fewer machines.

The license fee is set against the measured gain on the customer's own receipt. On a stack that costs one hundred, a measured gain of ten is worth ten; the fee is a share of that ten, and the customer keeps the rest. Percentages from different parts of a stack are not added into one bill: the invoice is one gain, measured once.

The value a customer sees comes in three forms, each on its own line of the receipt:

1. **Work per energy** - more finished work per kilowatt-hour.
2. **Capacity** - the same work on fewer machines in service; it becomes a cost reduction when the released machines are returned to the cloud or switched off by the platform around Omni-Compass.
3. **Service** - faster responses and fewer breaches at the same load.

The babysitting tax - the people and tools kept on the clock to set caps, answer pages and turn knobs back after a run or a crash - is the cost Omni-Compass removes by holding the knobs and returning them itself.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The receipt's arithmetic, and the one number. With the work held equal, a gain G in productivity is the same measurement as a saving S = G / (1 + G) in resources: a third more work is a quarter off the bill (manual, section 76.4). Across every real stack the one number is the Omni index (`results/OMNI_INDEX.md`): each test's work, speed, machines and energy as ratios, combined by geometric means so that a gain and an equal loss cancel exactly, each real category weighed the same, and only rows the same in all three of our runs counted. The index is honest in both directions by construction. As this edition is written it reads +30.2%: real Kubernetes +25.5%, the database +14.2%, messaging +166.9%, and the cache −24.9%, the last negative because the memory the governor holds for a wide working set is the resource it trades and reads power given by rule, even as the cache's work and hit rate read GAIN. A buyer reads the category that matches their stack, and reads its losses in the same table as its gains.

What a receipt is not. A receipt is not a forecast. It is the measured difference between two arms on one system at one time, with its interval. The only number that applies to a customer's system is the one their own paired runs produce, and the method of those runs is the manual's Chapter 57, the same method every table in this book was made with.

CHAPTER 78

Due Diligence

Answers reference the Claims Register (C-numbers) and the Technical Manual.

78.1 Engineering

Does the mathematics hold? Equations (1) to (8), Propositions 1 to 3 and their proofs are in Manual Chapters 2 and 3. The 500/500 CONVEY/CERT result is a property of the controller (Proposition 3), shown by counterfactuals: the controller aimed at the wrong basin also scores 500/500. **Is Python the same as C++?** Yes (C1, C2). The verifier also builds a deliberately mutated C++ governor and checks that the parity test rejects it. **Can every number be reproduced?** `python verify.py` rebuilds the C++, reruns parity, reruns the 100,000,000-decision soak and compares it with the recorded result, replays held-out scenarios and checks the pre-registered SHA-256 hashes (program fingerprints are additionally checked when the Python version matches the recorded one).

78.2 Operations

Against what we run today? Compared against a documented-behaviour reference model of Kubernetes autoscaling (HPA tolerance 0.1, 300 s scale-down stabilization, HPA targets 0.5 to 0.8; simplified Cluster Autoscaler with 10-minute unneeded time, 0.5 utilization threshold, 10-minute delay after scale-up): C8 to C12, including where Omni-Compass read LOSS. The reference model is not the upstream controllers (C12b);

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

running the upstream controllers against the same scenarios is the next baseline step. **On real traffic?** Not yet. Results use a synthetic stack model. Replay of published production traces and the pilot protocol are the next evidence steps (C14). **What happens when it is wrong?** Observe mode changes nothing (C6). The reset returns control to the native managers at the next interval (omni_kill arm). The shield blocks actions that violate I1 to I5 (C7). **Will it wear hardware?** Machine start/stop cycles, power-cap travel and thermal travel are measured for every arm (C12, Manual Chapter 8).

78.3 Security

What authority does it hold? Capacity, replicas, power caps, rollback authorization and routing, only in AUTOPILOT, only through the shield. **Can it expand capacity during a security block?** No: invariant I1 is enforced before execution. **Does it replace encryption, identity or policy engines?** No. Those components are retained (fleet model, security role).

78.4 Finance

What does it save? Energy per run versus current autoscaling (C8). Fleet-scale figures are modeled (C13); the governance saving comes from reduced idle and padded capacity, not from removing the decision components' own consumption. **What does it cost to run?** C5.

78.5 Adoption

How is it introduced without risk? A week of shadow with no write rights, then one muscle at a time with the reset tested at each handover, then 30 days paired against native in three runs, native on in every arm (docs/PILOT_PROTOCOL.md); on a production system only under a pilot agreement signed with The Omni-Compass LLC, and live customer traffic only under the Enterprise License.

78.6 Referees

Was it tuned on the test data? No. Law, shield and baselines were frozen and fingerprinted before the held-out seeds 346410161 and 360555127 (results/PREREGISTRATION.json). **Where does it fail?** Backlog violations against current autoscaling (C11); the engine-dynamics ablation (Manual Chapter 8); open obligations (Manual Chapter 10).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 79

License and Commercial Terms

The software and this manual are licensed under the Omni-Compass Evaluation License (`LICENSE`): evaluation and simulation use only. Everything else - commercial use, production use, operating any system beyond evaluation, redistribution, a hosted or managed service, incorporation into a product or service, or using the software or its results to build a competing product - requires a written Omni-Compass Enterprise License signed by The Omni-Compass LLC and paid for. All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC. No patent or trademark license is granted for any other use. Contributions are accepted only on the terms in `.github/CONTRIBUTING.md`, which assign their rights to The Omni-Compass LLC.

The copyright in Omni-Compass belongs to The Omni-Compass LLC. The source is visible under the Evaluation License, which is not an open-source license; seeing the source grants no right beyond it. The owner of a copyright may grant it under more than one license at once (the Evaluation License to everyone, a paid Enterprise License to a customer), may set the terms of each new version, and may assign the copyright; copies already received keep the terms they were received under. The license texts (`LICENSE`, `NOTICE`, `DISCLOSURES.md`) govern.

Every paper is bound into this book in full, as it stands in the repository, in the chapters that follow this one: the licensing questions and answers, the third-party notices, the standard terms of the Enterprise License (the grant, the term, the fees' basis, the warranties and their limits, the indemnities, the audit right, the source escrow and the governing law), the patents and trademarks statements and the security policy; the Evaluation License itself is the License appendix, and the disclosures, declarations and disclaimers stand in the front matter. Where a paper and this chapter differ, the paper governs. The engine's whole source, every Python file the fingerprint `OMNI_V7.json` names, is printed in the Source appendix: nothing in the mechanism is withheld from the owner.

CHAPTER 80

Licensing: Questions and Answers

Plain answers to the questions buyers, engineers and their lawyers ask. The binding terms are in `LICENSE`; where this page and `LICENSE` differ, `LICENSE` governs. Every declaration is in `DISCLOSURES.md`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Is Omni-Compass open source? No. The source is published so that it can be evaluated, reproduced and audited. It is licensed under the Omni-Compass Evaluation License 1.0 (`LicenseRef-OmniCompass-Evaluation-1.0`), which permits evaluation and simulation only.

What may I do without a commercial license? Evaluation and simulation, free of charge (`LICENSE`, sections 1.4, 1.5 and 2): read the code; run `verify.py`, the simulations and the benchmarks; reproduce the published results and publish your own, every row included (`LICENSE`, section 4); run Omni-Compass in watch, dry-run or shadow mode, or in a test environment, on systems you own or control, where it writes to nothing that serves production traffic or real users and to no physical process outside a test bench. A limited patent license comes with it, for evaluation only (`LICENSE`, section 2.2).

What needs the Omni-Compass Enterprise License? Every commercialization or monetization of any kind; production use; operating any system beyond evaluation; offering it, or its function, as a hosted, managed or software-as-a-service offering; redistributing it; incorporating it, or any part or derivative of it, into a product, service, appliance or firmware; using it or its results to build or train a competing product (`LICENSE`, sections 3 and 5). The Enterprise License is a written agreement signed by The Omni-Compass LLC and paid for; its standard terms are summarized in `ENTERPRISE_LICENSE.md` (production, hosted service, OEM and embedded, managed service provider, research).

May I run it as a service for others? Not without an Enterprise License with a hosted-service rider (`LICENSE`, section 3(c); `ENTERPRISE_LICENSE.md`, section 1).

May I connect it to a grid, a plant, a vehicle or a medical device? Not under the evaluation license: such systems may be studied only in simulation. A live connection to any physical system needs an Enterprise License and a written safety case accepted by the system's owner (`LICENSE`, section 10).

How is the Enterprise License priced? Against the measured gain on the customer's own paired receipts (native against Omni-Compass on the same system, the same load and the same clock). Terms are set in each signed agreement; no price stated anywhere in this repository is an offer.

Are patents involved? All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (`PATENTS.md`). The evaluation license grants no patent license beyond evaluation.

May I use the name or the compass rose? Only to refer to Omni-Compass accurately (`TRADEMARKS.md`).

May I contribute? Contributions require the contributor license agreement (`.github/CLA.md`): every contribution is assigned to The Omni-Compass LLC, with a patent license from the contributor.

Does Omni-Compass send data anywhere? No. The software contains no telemetry and makes no network connection of its own except to the systems an operator points it at (a Kubernetes API, `nvidia-smi`, a meter command).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What do scanners report? The license is declared in machine-readable form for Black Duck, FOSSA, Snyk and REUSE (`pyproject.toml`, `REUSE.toml`, `.fossa.yml`, `.snyk`, `sbom/`); third-party components are listed in `THIRD_PARTY_NOTICES.md`.

Can the terms change? Yes. The Omni-Compass LLC reserves the right to change the software, its documentation, its results and these terms at any time, without notice; a changed version applies to copies obtained and uses made after it is published (`LICENSE`, section 9). A signed Enterprise License governs its own term.

Which law governs? The laws of the State of Delaware and the federal laws of the United States, with exclusive venue in the Delaware state courts and the United States District Court for the District of Delaware (`LICENSE`, section 19). The Omni-Compass LLC is based in Florida.

Who do I contact? The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com.

CHAPTER 81

Third-Party Notices

Omni-Compass uses the following third-party components. Each remains under its own license; nothing in the Omni-Compass Evaluation License changes those terms. Installed versions are those resolved from `requirements.txt`.

Component	Use	License
NumPy	numerical arrays	BSD-3-Clause
Matplotlib	charts in the reports	Matplotlib License (PSF-based, BSD-compatible)
ReportLab (open-source edition)	the printable book and reports	BSD-3-Clause
Numba	compiled inner loops of the motion-axis plant	BSD-2-Clause
PyTorch (on GPU machines only, not installed by `requirements.txt`)	the GPU workload of the benchmark	BSD-3-Clause
`tests/third_party/hpa_independent.py`	an independently written HPA reference used unmodified in tests	its own terms, stated in the file

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Component	Use	License
Liberation Serif and Liberation Sans fonts (embedded in the PDF book)	typesetting	SIL Open Font License 1.1
DejaVu Sans and DejaVu Sans Mono fonts (embedded in the PDF book)	typesetting	Bitstream Vera / DejaVu license (free)
PlanetLab / CoMon VM CPU traces, March 2011 (`fleet/traces/`, `k8s_controlplane/traces/`; source in their `PROVENANCE.json`)	held-out demand for the modelled fleet	their own terms (public research data); no Omni-Compass copyright is claimed in them
Google cluster-usage traces 2011 (`clusterdata-2011-2`), from which `results/traces/google2011/` is derived (`docs/TRACES_PREREGISTRATION.md`)	the public demand trace on the real cluster	Google's published terms for the traces; no Omni-Compass copyright is claimed in the trace itself

Kubernetes, kind, `kubectl`, the NVIDIA driver and `nvidia-smi` are not distributed with Omni-Compass; it calls them where an operator has installed them. Their names are the property of their owners (`DISCLOSURES.md`, section 1).

CHAPTER 82

The Enterprise License: Standard Terms

This page summarizes the standard terms of the Omni-Compass Enterprise License, the paid license that every production use, commercial use, hosted service, distribution, OEM or embedded use, and every other commercialization or monetization of Omni-Compass requires (`LICENSE`, section 5). It is a summary for evaluation and planning. It is not an offer, and it binds no one: only an Enterprise License signed by The Omni-Compass LLC and the licensee, with its order form, grants any right, and that signed agreement controls over this page in every respect.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

82.1 What can be licensed

License	What it allows
Production	The licensee runs Omni-Compass on top of its own systems (clusters, fleets, cards, sites, plants), on premises or in its own cloud accounts, within the scope in the order form
Hosted service	The licensee offers Omni-Compass's function to its own customers as a hosted, managed or software-as-a-service offering, under a hosted-service rider (per customer, per managed unit or revenue share)
OEM and embedded	The licensee ships Omni-Compass inside its own product, appliance, firmware or platform, under an OEM rider (per unit or per installed base)
Managed service provider	The licensee operates Omni-Compass for named third parties under a managed-service rider
Research	A university or public laboratory uses Omni-Compass beyond evaluation for non-commercial research and publication, under a research rider

Scope is set in each order form: the licensed systems and their size (machines, cores, cards, sites, muscles or managed units), the fields of use, the territory, the named sites or accounts, and the engine versions covered.

82.2 Fees

Fees are set in each order form: a subscription, a usage-based fee, a share of the measured value captured, or a combination, with any minimum commitment. Where fees track value, the value is measured on the licensee's own paired receipts (native against Omni-Compass on the same system, the same load and the same clock), by the method the order form names. Payment terms, taxes and late charges are as the order form states. No price, rate or benchmark in this repository is an offer.

82.3 What the licensee receives

1. The licensed engine at a named, sealed version (its fingerprint, `tools/omni_version.py`), and each later version the subscription covers.
2. The wiring kits and the plug for the licensed systems, the manual and the book, and the receipts tooling.
3. Support and maintenance at the tier in the order form: response targets, named contacts, and corrections of defects that reproduce on the licensed version.
4. Installation through the manual's levels: read-only first, the wire check, a watch arm equal to native, then writing levels one at a time, each with its receipt.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

82.4 Warranties

1. **Conformance.** For the warranty period in the order form, the licensed version performs materially as its documentation describes. The remedy is correction or, failing that, a refund of the fees paid for the nonconforming version for the affected period.
2. **The safeguards.** The kill switch hands every setting back to the value read before Omni-Compass's first write; the shield keeps every write inside the cover the operator granted; Omni-Compass never fights another writer.
3. **No other warranty.** No outcome, saving or result on the licensee's systems is warranted. Published results are evidence about the systems they ran on (`DISCLOSURES.md`, section 2).

82.5 Intellectual property

1. The Omni-Compass LLC keeps all right, title and interest in Omni-Compass, the Mechanism and the marks. The licensee receives the rights its order form lists, for its term, and no others.
2. **Patent license.** The Enterprise License includes a license under The Omni-Compass LLC's patent rights in the filed applications, and in any patent issuing from them, for the licensed scope.
3. **Infringement indemnity.** The Omni-Compass LLC defends the licensee against a third-party claim that the licensed version, used as licensed, infringes that party's intellectual property, and pays the damages finally awarded or the settlement it agrees, subject to prompt notice, control of the defense and the usual exclusions (modifications, combinations, unlicensed use, versions superseded by an offered fix). Remedies may include a license, a modification or a refund.
4. **Confidential filings.** The patent application number and other confidential information are disclosed to a licensee under the agreement's confidentiality terms.

82.6 Limits of liability

Each party's liability is limited to the fees paid or payable in the twelve months before the claim, and neither party is liable for indirect, incidental, special, consequential or punitive damages, except for the exclusions the signed agreement states (for example, breach of confidentiality, the infringement indemnity, and the licensee's breach of the license scope).

82.7 Confidentiality, data and security

1. **Confidentiality.** Mutual, for the term and a period after it, covering non-public information either party gives the other.
2. **Data.** Omni-Compass contains no telemetry. The licensee's data, readings, settings and receipts stay on the licensee's systems and remain the licensee's. Where the licensee chooses to share data for support, a data-processing agreement applies.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

3. **Security.** A software bill of materials ships with every version (`sbom/`); vulnerabilities are handled under `SECURITY.md`; the licensee may run the verifier (`python3 verify.py`) and the seal checks on every delivered version.

82.8 Safety and operation

1. The licensee remains the operator of its systems. The machines' own controls and protections stay in place; Omni-Compass sets only values they already accept.
2. Any connection to a physical system (grid, plant, vehicle, robot, medical device, facility) requires a written safety case accepted by the system's owner and every review its regulator requires; safety-critical systems are licensed only on those terms.
3. The kill switch stays operable at all times.

82.9 Compliance, audit and insurance

1. Each party complies with export-control, sanctions and anti-corruption law.
2. The licensee reports usage against the licensed scope as the order form provides, and The Omni-Compass LLC may verify it on reasonable notice, no more than once a year.
3. Insurance, where the order form requires it, is carried by each party at the limits stated there.

82.10 Term, renewal and transition

1. Term and renewal are as the order form states.
2. On termination or expiry, the licensee stops use, every setting Omni-Compass changed is handed back to native (the hand-back is built into the engine), and every copy is returned or destroyed; transition assistance is available at the rates in the order form.
3. Source-code escrow for the licensed version is available under a three-party escrow agreement.

82.11 Governing law and precedence

The signed agreement states its governing law and venue; the standard is the law of the State of Delaware, with venue in the Delaware state courts and the United States District Court for the District of Delaware. Its order of precedence is: the order form, then any rider, then the Enterprise License terms, then the documentation. This page and the evaluation license (`LICENSE`) are superseded by the signed agreement for the licensee and its scope.

82.12 How to obtain one

The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com. A license starts

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

with an evaluation on the licensee's own systems (`LICENSE`, section 1.4): watch mode, the wire check, and paired receipts on its own load.

CHAPTER 83

Patents and Trademarks

83.1 Patents

All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC.

The filings cover, among other things, the Omni-Compass engine (the six-state closed law and its bounded control command held through the integration step), the compass law (the service position pulled to the middle of its band by a bounded push and pull, with fail-up at the wall), the antagonist up/down wiring of each muscle, the plug contract (one restore point, read-back, the one-writer rule), the two-wire governance of a processor (clock ceiling and power limit), the two-way nervous system that grants and withdraws authority per organ, and one governor over many machines and realms on one clock.

83.1.1 What has been filed

Application	a nonprovisional utility patent application under 35 U.S.C. 111(a), filed with the United States Patent and Trademark Office
Title of the invention	**The Omni-Compass**
Inventor	Alan John Dubra, Kissimmee, Florida, United States
Drawing sheets	15
Application data sheet signed	20 September 2026 (form PTO/AIA/14, 37 CFR 1.76)
Publication	normal eighteen-month publication requested
Application number	withheld here on purpose; the filing is verifiable with the USPTO and is shown to a licensee under a signed Omni-Compass Enterprise License

The application data sheet names no assignee; the application stands in the inventor's name. Any assignment to The Omni-Compass LLC is recorded with the USPTO separately (37 CFR part 3).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

No patent license is granted by publishing this repository, by the evaluation license, or by any act of The Omni-Compass LLC other than a signed, paid Omni-Compass Enterprise License. Use outside the evaluation license is infringement. Everyone who downloads, forks, scans or runs this software is on notice of these filings.

83.2 Trademarks

All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC.

83.2.1 The marks

"Omni-Compass", "OmniCompass", the Omni-Compass compass rose with its ring of Greek letters, and the related names, logos and trade dress (the "Marks") are trademarks of The Omni-Compass LLC; trademark applications have been filed in the United States. Neither the evaluation license (`LICENSE`) nor publication of this repository grants any license to the Marks. A license to use them is granted only in a signed Omni-Compass Enterprise License, and only as it states.

83.2.2 What is allowed without a license

- Referring to Omni-Compass by name, accurately and in plain text, to say what it is, that you evaluated it, or that a result was produced with it, with the engine version and commit (`LICENSE`, section 4).
- Linking to the repository or to www.omni-compass.com.

83.2.3 What is not allowed

- Using any Mark, or anything confusingly similar, in the name of a product, service, company, domain, account, package, repository, fork or derivative.
- Using the logo or the compass rose, or altering any Mark.
- Suggesting endorsement, sponsorship, certification, partnership or affiliation by The Omni-Compass LLC.
- Using a Mark in advertising, comparative claims or sales material without a signed license.

83.2.4 Other companies' marks

Kubernetes is a trademark of The Linux Foundation; NVIDIA is a trademark of NVIDIA Corporation; KEDA and Karpenter are projects of their maintainers; other names belong to their owners. Their mention describes compatibility or a system Omni-Compass was tested with; it states no affiliation, endorsement or certification (`DISCLOSURES.md`, section 1).

83.2.5 Reporting misuse

The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 84

Security Policy

84.1 Reporting a vulnerability

Report a suspected vulnerability privately to The Omni-Compass LLC by telephone at its main office infrastructure (toll-free direct 1-888-339-9551; system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida) or through www.omni-compass.com, never in a public issue, pull request or discussion. Include the version and commit, what you saw, how to reproduce it, and its likely impact. We acknowledge reports, keep the reporter informed, and coordinate disclosure: a fix or a mitigation is published before the details are, and the reporter is credited if they wish. Please give us a reasonable time to fix before any disclosure, do not access data that is not yours, and do not disrupt any system; good-faith research that follows this policy will not be pursued by us.

84.2 What is in scope

The engine, the controllers, the plugs and wiring, the harnesses, the workflows and the verifier in this repository. Third-party software Omni-Compass runs on or connects to (Kubernetes, KEDA, Karpenter, NVIDIA drivers, databases) is reported to its own maintainers.

84.3 How Omni-Compass is built to fail safe

- The governor issues control actions only through the shield (`omnicompass/shield.py`, `cpp/src/shield.cpp`), inside the cover the operator granted, and never fights another writer.
- It snapshots every setting once, before its first write, and the kill switch (`python3 tools/omni_switch.py off`) hands every setting back; a dead-man lease hands them back if the governor stops.
- It runs under least privilege: on Kubernetes, a service account with `kubectl auth can-i` receipts for what it can and cannot do (`deploy/kind/rbac-omni.yaml`).
- The watchdog acts only on registry records its own user wrote that no other user can change, and never signals a process number the system gave to another process (`tools/omni_switch.py`). Put the switch and the registry in a directory only the governors' user can write (`OMNI_MASTER_OFF`, `OMNI_REGISTRY`); `status` warns when they are not.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- It contains no telemetry and makes no network connection of its own except to the systems an operator points it at.
- The threats of misuse, the guards against each and what is still open are set out in `docs/THREAT_MODEL.md`.
- Every release carries a software bill of materials (`sbom/`) and a seal of the engine and its twins (`results/SEAL.json`, `python3 verify.py`).
- The capture script (`fleet/capture/kube_capture.sh`) is read-only; no component writes to a live cluster unless the operator starts it in a writing mode.

CHAPTER 85

Repository Standards

The files the leading repositories carry, the open ones such as Kubernetes and the source-available, commercially licensed ones such as HashiCorp Terraform, Elastic, MongoDB, Redis, CockroachDB and Sentry, and where Omni-Compass carries each. Omni-Compass is published for evaluation and simulation; everything else is licensed (`LICENSE`, `LICENSING_FAQ.md`).

What a leading repository carries	Omni-Compass
License, machine-readable and in full	`LICENSE`, `LICENSES/LicenseRef-OmniCompass-Evaluation-1.0.txt`, SPDX headers, `REUSE.toml`
Licensing questions in plain words	`LICENSING_FAQ.md`
Notices and third-party components	`NOTICE`, `THIRD_PARTY_NOTICES.md`
Disclosures and disclaimers in one place	`DISCLOSURES.md`
Patents and trademarks	`PATENTS.md`, `TRADEMARKS.md`
Contributor license agreement	`.github/CLA.md`
Contributing, code of conduct, governance, ownership	`.github/CONTRIBUTING.md`, `.github/CODE_OF_CONDUCT.md`, `.github/GOVERNANCE.md`, `.github/OWNERS`, `.github/CODEOWNERS`
Security policy and contacts	`SECURITY.md`, `.github/SECURITY_CONTACTS`
Support	`.github/SUPPORT.md`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

What a leading repository carries	Omni-Compass
Changes and roadmap	`CHANGELOG.md`, `ROADMAP.md`
Citation and software metadata for archives and crawlers	`CITATION.cff`, `codemeta.json`, `pyproject.toml` (keywords, URLs)
Software bill of materials (SPDX and CycloneDX)	`sbom/omni-compass.spdx.json`, `sbom/omni-compass.cdx.json`
License scanners (Black Duck, FOSSA, Snyk, ScanCode, REUSE)	`pyproject.toml`, `.fossa.yml`, `.snyk`, `REUSE.toml`, SPDX headers
Issue and pull-request templates, including a licensing template	`.github/ISSUE_TEMPLATE/`, `.github/PULL_REQUEST_TEMPLATE.md`
Dependency updates	`.github/dependabot.yml`
Continuous verification	`.github/workflows/verify.yml`, `python3 verify.py`
Reproducibility: fingerprinted release, sealed twins, checksummed results	`RELEASE_MANIFEST.json`, `results/SEAL.json`, `SHA256SUMS.txt` in every result folder
Rules written before every benchmark run	`docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`
A manual and a printable book	`docs/OMNI_COMPASS_MANUAL.md`, `docs/OMNI_COMPASS_MANUAL.pdf`
Every result in one place, with charts	`docs/DOSSIER.md`
A one-file archive of the repository	GitHub's Download ZIP, at any commit (the copyright deposit is `release/copyright/`)

Files present at the commit this page describes; `python3 verify.py` checks the ones the results depend on.

CHAPTER 86

Python, C++ and the Seal

The laws are twinned: each has a Python version and a C++20 version that give the same answers, proven by a parity test on every build (`cmake -S cpp -B cpp/build && cmake --build cpp/build`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Law	Python	C++	Proven by
core engine	`omnicompass/core.py`	`cpp/src/core.cpp`	500 frozen fixtures
governor	`omnicompass/adapter.py`	`cpp/src/governor.cpp`	`tests/test_cpp_governor_parity.py`
safety shield	`omnicompass/shield.py`	`cpp/src/shield.cpp`	`tests/test_cpp_shield_parity.py`
HPA replica law	`fleet/harness.py`	`cpp/src/hpa.cpp`	`tests/test_cpp_hpa_parity.py`
closure law	`omnicompass/closure.py`	`cpp/src/closure.cpp`	`tests/test_cpp_closure_parity.py`
conveyance law	`omnicompass/conveyance.py`	`cpp/src/conveyance.cpp`	`tests/test_cpp_conveyance_parity.py`
nervous system	`omnicompass/nervous_system.py`	`cpp/src/nervous_system.cpp`	`tests/test_cpp_twins_parity.py`
compass and ledger	`omnicompass/compass.py`, `storage.py`	`cpp/src/compass.cpp`	`tests/test_cpp_twins_parity.py`
GPU governor rules	`omni_controller/gpu_governor.py`	`cpp/src/gpu_rules.cpp`	`tests/test_cpp_twins_parity.py`

The seal (`results/SEAL.json`) holds the SHA-256 fingerprint of every twinned file, written only after every parity test passes. `verify.py` fails, naming the file, if any sealed file changes afterwards. The compass law (`omnicompass/compass_law.py`) and the two-wire GPU governor are in Python today; their C++ twins are next.

Why two languages. A law that exists in one implementation can hide a bug that is also its behaviour: the tests pass because the tests were written against the same code. Two implementations written separately, in languages with different numerics and different defaults, that agree to fifteen decimal places on five hundred trajectories and on every fixture of the governor, the shield, the HPA law, the closure, the conveyance and the nervous system, are evidence that the text of the law is what both compute. The C++ twin is also the one a buyer would embed where Python does not run (a controller, a firmware, a PLC), and the soak (`results/SOAK.json`, one hundred million decisions, no failure) is run on it. The seal makes the twins' agreement a property of the repository rather than a claim: change a sealed byte and `verify.py` fails, naming the file.

The three fingerprints. The seal fingerprints the twinned laws; `OMNI_V1.json` to `OMNI_V7.json` fingerprint the engine as a whole (law, controllers, muscles, runners: 38 to 46 files); `RELEASE_MANIFEST.json` fingerprints the release (the commit, the engine, the twins, the GPU protocol,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the live evidence and the verification receipt) and is rewritten by `tools/release_manifest.py` before every push, under the Python GitHub runs. A reader holding any one of the three can tell whether the files in front of them are the files the results were made on.

CHAPTER 87

The Founder's Working Notes

Where a sentence is a design aim rather than what the code does today, or a figure that is not yet measured, the check column says so. Nothing in this chapter adds a result: every number that is a result names the file it comes from.

87.1 What Omni is

OmniCompass is the governor. It is the process running on the processor, and the chip executes it. The card, the replica count, the rack cap, the joint and the feeder are the levers: the muscles. Omni reads the meters, steps a bounded loop, and writes only the lever it is allowed to write. When it stops, it puts that lever back.

It is not the chip, not the card and not Kubernetes. It is the brain on the host, writing through a bolt.

Governor is the machine word. A governor on a steam engine did not build the engine and did not turn the shaft. It watched the speed and moved the throttle so the speed stayed in a band. The plant does the work: the chip, the card, the pods, the job. Omni reads a meter, writes one lever, and puts that lever back.

87.2 The loop

The chapter says	The code today	Check
Omni reads the plant's meters and maps them to the state E, U, S	`omnicompass/adapter.py`, `observe_vector` and `assimilate`	matches
It computes the drift with the command at zero, then raw = −drift + K[P]·(σ − U), clipped to the authority before the step	`omnicompass/core.py`, `control_command`: `u = clip(−f_U(x,t) + KP·(σ − U), ±U_AUTHORITY)`	matches
It holds that same u through every RK4 stage and does not rewrite U afterwards	`core.py`, `macro_step`: one `u` per micro step, passed unchanged into all four RK4 stages; U is never overwritten	matches (proved: `docs/TRACKING_THEOREM.md`)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The chapter says	The code today	Check
It writes the clipped command u to the lever	**Not today.** The frozen live governor runs the engine with u = 0 (`macro_step(..., target=None)`) and uses u only as its convergence signal (`push = u / U_AUTHORITY`). The lever is set by the allocation law and the nervous system's authority (`omnicompass/nervous_system.py`).	**design aim**, see "What to strengthen"
On exit it restores the lever	kill switch: `omni_controller/controller.py` restore; the GPU bench restores and reads back the start limit; the realm harness checks every knob is handed back	matches

So the step the chapter describes is the engine's own step, and it is proved. Writing u itself onto a lever, through an output map with a gain in watts (or replicas) per unit of u, is a further mechanism. It is not what produced the results below.

87.3 Wiring

A wrist wire moves the wrist. It does not move a finger that has its own tendon unless that tendon is also wired.

The card's clock wire goes to the NVIDIA driver. The replica wire goes to the Kubernetes API. The scheduler's wire goes to the kubelet. Those wires already exist and do not pass through Omni. Omni is an added process, and a tendon is connected to it only if the output map writes that lever and the input map reads it back.

A power-limit write can make the driver drop clocks. That is a side effect on one finger, not the replica nerve, which keeps running on its own path unless a second line is connected. The cluster starts its own wires: the API server accepts a replica write, the scheduler places the pod, the kubelet starts it, the driver moves the clocks, and HPA, if installed, is its own process. Omni joins those nerves. It is a client of the plant, not its origin.

87.4 Four realms, five organisms

1. Compute / AI / Cloud: GPU server, Kubernetes, the card, the job.
2. Physics / Robotics / Autonomous: joint, servo, PLC, car, line.
3. Energy / Facility / Industrial: rack, cooling, PDU, battery.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

4. Distribution / Specialized: network, feeder, logistics.
5. The whole organism: all 656 canonical muscles, each once, one clock.

A muscle may sit in more than one realm. Every realm stands on the same spine (Kubernetes, machines, GPUs and CPUs, network, storage, observability, security, cooling, electrical distribution), and then has its own domain muscles. The fifth run holds each of the 656 once.

Each organism is shown native, then with Omni, on the same membership, the same seed and the same clock: two receipts. The core does not change when the lever changes.

Check	
Built	`realms/` (round 3): spine of 190 muscles in all four realms; organisms of 345, 262, 282, 337 and 656 (`docs/REALM_MUSCLES.md`)
Result	Whole organism +0.1% work per energy, SUPERIOR WITHIN GUARDRAILS; the realms pay in service when they share the spine (`results/realms/REALMS.md`)
Limit	These are modelled plants (evidence S / rung E2). A catalog row is not a tendon. GitHub can host the compute organism for real (kind, and the GPU on a rented card); facility, machine and grid need their own hosts
Open	The 656 is a working catalog, not a census: `docs/realm_study/` checks it against the real systems' documentation and industry practice (waves 1 and 2 done; wave 3 maps every row)

87.5 Evidence rungs

Rung (chapter)	Class (repository, `docs/EVIDENCE_LEDGER.md`)	Meaning
E0	design	the written spec
E1	T / V	deterministic tests and proofs: Python against the C++ twin, the tracking theorem
E2	S	simulation on a made plant (the realms, the fleet and GPU models)
E3	L	real software: Kubernetes on kind, no card (sets 22 and 23)
E4	P	a physical meter: the card's own power reading (the GPU bench; first run on an NVIDIA A10, 2026-10-02, `results/gpu/run-20261002T082232Z/GPU_REPS.md`; the corrected governor not yet run on a card)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

A result does not climb a rung by itself. The four realms are plants; E1 to E4 are how hard the proof is on whichever plant is run.

87.6 What blocked the card

The hash did not stop the card. A Python–C++ mismatch can fail a test after a machine has started the job; it cannot stop GitHub from handing out a machine, because the runner is chosen before the code runs. The verify job started and failed (fixed since: it is green), and separately the GPU job never got a machine.

The organisation's and enterprise's settings show no GPU runner and no option to create one (checked 2026-10-02), so no budget could start it. The run therefore moves to a rented card (`scripts/gpu_rented_run.sh`, one command).

87.7 The bake-off that exists

Set 23 (`results/live/LIVE_REPS_23.md`, run 36940088922): real Kubernetes, ten pairs, fifteen minutes an arm, the same open-loop work, native against Omni, six kind workers, order rotated.

Gauge	Native	Omni	Change
Response, 95th percentile (ms)	327.3	123.8	−62.2%
Response, 99th percentile (ms)	503.4	169.3	−66.4%
Response, mean (ms)	146.6	80.1	−45.4%
Failed requests	0	0	0
HPA replicas, mean	8.51	5.40	−36.6%
Workers in service	6	4.28	−28.7%
Pods started	4.7	1.7	−63.8%
Pod start wait (s)	18.6	3.8	−79.6%
CPU, service plus Omni (cores)	0.903	0.894	−1.0% (no difference)
Energy, workers still on (Wh, declared model)	159.0	158.7	−0.2% (no difference)

The same requests were sent and none failed. They were answered faster, on fewer copies, at the same bill. Kind leaves every worker powered, so the watt line stays flat. Parking a worker and waking it again was tried; the churn ate the saving (`docs/CLAIMS_REGISTER.md`, C17), so that path is closed on this plant.

Arithmetic, not a result: if a run finishes 5% more work, nineteen runs do what twenty used to (20 / 1.05 = 19.05).

87.8 The chip

The card is the meter that can move the electric bill, and the meter that can move output.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Energy on the chip is the integral of power.draw. The same pile can be spent two ways:

- **Output:** more finished work on the same bill. The cap stays near where the native arm left it, the draw stays in the same band, and heat gets no new reason to rise.
- **The bill:** a lower enforced limit, the job still finishes, and joules fall.

A company short of cards spends it on output. A company paying for GPU hours with spare capacity spends it on the bill. A company against a building power cap splits it. The bench's primary outcome, work per joule, counts both, and its table prints requests and joules separately, so the receipt shows which way the pile went (`tools/gpu_reps.py`).

For the pile to be real:

- the cap has to bind;
- the write has to be enforced by the driver, and the enforced limit read back;
- the job has to finish;
- the limit has to be restored on exit, so the next native arm is not still capped.

All four are in the bench and its preregistration (`docs/GPU_PREREGISTRATION.md`, amendments 1–3).

87.9 The babysitting tax

The electric bill is the small pile. The babysitting tax is the people and tools kept on the clock to watch the levers, reset a cap, and stop the muscles fighting. A card's real cost is usually the card and the hours it sat idle or late, not the electrons.

What fills the seat today is a person: someone sets the cap, someone gets the page when it is left down, someone turns it back after the run or the crash. The product takes that knob and that page.

87.10 What we found nobody selling

We found no published product that does all of the following across compute, machines, facility and grid as one governor:

- reads all of their meters into one state;
- holds one clipped command through the step;
- puts each lever back when the run stops, and prints both receipts.

Borg, Twine and OpenShift govern compute. AWS is a cloud. Turbonomic covers the IT stack. Tesla runs the car, the battery and the factory as separate software. Each covers more than one thing as a company, not four realms as one governor.

Omni on such a stack does not replace it: Borg remains the muscle. This is a market observation from public material, not a claim tested here.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

87.11 The bill

Run the stack native and print the receipt. Run the same stack with Omni and print the receipt. The difference is the pile, and the fee is 20 percent of it.

On a stack that costs $100, a 10 percent win is $10. The fee is $2, and the customer keeps $8. In general, the fee is 0.2 × (measured win) × (spend). A gain and a cut are the same difference, read from opposite sides. Do not add CPU, GPU and babysitting percentages into one bill: the invoice is one pile.

Check: the 10 percent is an example, not a measured win. What is measured so far:

- on Kubernetes, response time −62% at the same energy (set 23);
- in simulation, +0.1% work per energy for the whole organism (realm round 3);
- on a card, nothing yet: the GPU bench is the first real-meter number.

Revenue figures should be computed from the measured win on each customer's own receipt. The per-company table in the working draft (2% of each company's published infrastructure spend) assumed a 10% win everywhere, and several of its spend figures do not match the companies' published capital expenditure. It is left out of the repository until both are sourced.

87.12 What to strengthen

1. **Wire the output map.** u becomes one lever, read back and restored, with the start value, the written value and the value after a kill printed on the receipt. The GPU bench already records the start limit, every write and its read-back, and the restored limit. A u-to-lever output map (a gain in watts or replicas per unit of u) is a new mechanism. It would be preregistered and tested as its own arm, first in the realms, then on a card, never mixed into the frozen confirmation run.
2. **Keep the step as it is:** clip before the stage, hold u through RK4, do not rewrite U (proved: `docs/TRACKING_THEOREM.md`).
3. **Run native against Omni on the same membership for each realm.** Compute is the one GitHub can host for real; the other three are modelled until their hosts exist.
4. **Aim the chip at about 5% more finished work on the same bill**, cap unchanged and temperature never behind, or at fewer joules for the same work. The buyer chooses, and the receipt prints which.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 88

The Road to the Engine That Stands: Omni v5, v6, v7, v8, v9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21 and 22

88.1 Omni v5 — the engine fingerprint

omni-v5 is a distinct engine, its own tree, no longer read as v3. It is fingerprinted file by file in `OMNI_V5.json` (`tools/omni_version.py`); `python3 tools/omni_version.py` prints `omni-v5` on this checkout. v1, v2 and v3 stay frozen, each kept for its own results; no result is ever read across versions.

88.1.1 The default brain: the max (12 October)

The founder's order of 12 October: "We're going to show a benchmark that shows it to the max. And we're going to show one native... anybody can tune it from the max down." v5's default (`V5CFG`) is therefore **the max**: every muscle pushed toward its own limit for the most work, faster, never past its limit and never below native. The energy that takes is reported as **LOSS**, beside the work and the speed it bought; the max never shaves (nothing is taken from one muscle to give to another). The dial turns it down: **the tune-down** (`V5SAVE`, the save setting described below) is native's work on the least energy, kept and tested on every muscle.

- **A pool** gets the machines its forecast needs before the rush lands: the recent peak arrivals widened by the largest rise the pool has shown within one machine start, at 95% utilisation, plus any **work waiting in line** cleared within one machine start, through a floor under its own autoscaler, never fewer than its **twin** runs (the brain's model of native: the same pool under its own autoscaler alone, on the same arrivals). The machines it added and no longer needs go back at once (a ceiling at the larger of its need and its twin's count; the autoscaler's own scale-down would hold them for its window). Its power cap and admission stay native: full clock, every request admitted.
- **A room** is cooled ahead of the heat as it nears its limit or its cooling nears flat out, never warmer than native.
- **A battery site** clips its line from its charge, native plus: the operator's own reserve while nothing would cross, and below it, down to its band's floor, exactly the draw that would cross; a UPS lends only the charge above its outage floor.
- **A motion axis alone**, while tasks are waiting, runs at **its own peak speed** (`s_peak`, `realms/presets.py`: the head the machine itself has above the everyday profile native runs: a joint's

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

drive and a multirotor's airframe 1.25, a road vehicle's drive 1.25, a ship's engine to its maximum continuous rating 1.15), never past it, back to the everyday setting at once when its motor passes 80% of its heat rise or its tracking error passes half its line. A reaction wheel's saturation, a train's line speed and an elevator's overspeed governor are their limits: those stay native. Inside a body an axis keeps its everyday speed (see the bone). Torque (effort) limits stay native: in these models a higher one changes nothing.

- **A process loop** native already runs flat out (the full actuator): wired, watched, native.
- **At its limit on native.** Every power limit at full, every admission gate open, a loop's actuator at its full range: native already runs these at the top of their range, the max writes nothing, and the tables say so in those words (`native_at_limit`).
- **The bone.** Inside an organism the body's lines (each site's grid line, each room's cooling and its degrees to its limit) are loaded by the whole body, so the max grants machines and cooling only out of the room the most loaded line has left, one decision ahead (its latest reading plus the larger of 3.09 of its own wobbles and the largest single climb it has shown, counted on the line without the max's own machines: its own steps are not the line's wobble). A machine is started only if the lines can still carry it when it is ready (the climb grown by the square root of its start time: one started and then cancelled only burns its start). A site with flexible load waiting gives its catch-up the room first. With no room, what the max added goes back, as much as the shortfall needs, the pools with the most extra first and the machines started last first, never below a pool's twin. An axis inside a body keeps its everyday speed: at the peak hour the organisms run, the bone almost never has room for its worst case, and in the development runs its peak gave the body nothing while its changed rhythm crossed neighbours at their own limits.
- **One pedal.** Push for more, lift off for less; lifting hands the setting back toward native. At idle the pool lifts all the way to its breathing floor (two machines) and wakes straight to the work. **The lease**: at 80% of every modelled run the brain is cut and the plug walks every setting back to native over 24 decisions (no jump the body must absorb at once); a floor that only held a pool at its twin's count adds nothing to walk back and is let go at once (a walking floor would hold machines native had already released). The security kill switch is instant.
- **Wear** is metered beside the work, never judged: a pool's machine-hours, a room's cooling unit-hours, a battery's kWh cycled, an axis's motor heat (degree-hours above ambient), a loop's actuator duty-hours.
- **Three loads**, fixed before the preregistered run (seeds 3000-3009): each organism's own load, **the climb** (the load stepping 1 2 3 2 3 4 ..., up to twice its own), and **the line** (`LINE` in `realms/harness.py`: work already waiting when the run starts, a quarter of the work the run itself brings; a pool's jobs, an axis's tasks, a site's held-over flexible load; a room's heat and a loop's demand cannot wait and have none). Every muscle alone runs its own load and the line; the organisms run all three. Speed is judged where there is a line: the arm that clears it sooner is faster.
- **The live controllers** carry the same dial (`--dial max`, the default): the Kubernetes controller holds the service well inside its line (the compass at 0.3) and never gives back a machine the cluster had on its own;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the GPU compass keeps the card's clock ceiling at the top while it serves (the card already runs flat out on its own power limit).

The organisms run the hour around each battery site's daily peak, where the UPS lines carry most of their rating: there the bone is tight and the max gives only what the lines can carry. Every muscle alone has no body around it, so the max gives it everything its own limits allow. `results/realms_v5/REALMS_V5.md` shows both, and the climb (the load stepping 1 2 3 2 3 4 ..., up to twice the organism's own) where native breaks first.

88.1.2 What v5 is

v5 is **a combination of everything**: it takes v3's proven engine (its live controllers, its realms and organisms, its runners) and adds the **v5 compass law** (`realms/compass_arm_v5.py`) and its arm in the harness (`realms/harness.py`). The goal is to be equal to or ahead of native and every prior version on every measure, and never behind on any — *do no harm*.

The v5 muscle catalog (`realms/catalog_v5.csv`) is **1,107 muscles in 70 families** — v3's 945, then a first wave of six new families of real control points: **Linux Kernel & OS Control** (cgroups, the CFS scheduler, DVFS/cpufreq, transparent hugepages, the I/O scheduler, NUMA balancing, dirty-ratio writeback, swappiness, TCP buffers, …), **Serverless & Functions (FaaS)**, **Virtualization & Hypervisor**, **Content Delivery & Edge Cache**, **Gut** (data-plane intake and waste), and **Skin** (perimeter and admission); then a second wave (12 October) of 95 muscles: five new families, **Financial Markets & Trading Systems** (matching engines, order gateways, pre-trade risk, market-data fan-out), **Media Streaming & CDN** (transcoders, packagers, origin shields, player buffers), **Game Servers & Real-Time Interactive** (Agones fleets, tick rates, matchmaking, relays), **Blockchain & Ledger Nodes** (a node's cache, mempool, peers and validation threads; never a consensus rule) and **CI/CD & Build Farms** (runner scale sets, build concurrency, caches, test shards), plus additions to seven families already there: security engines (an intrusion-detection system's capture threads and ring buffer, a web firewall's inspection workers, a SIEM's ingest), MQTT brokers, DNS and mail caches, ROS 2 executors, humanoids and manipulators, warehouse robot fleets, PX4 and ArduPilot drones and swarm spacing, and edge sites. The GPU family is named for the hardware, not one vendor (**GPU Hardware**), and the eight audited families carry numbers (64 to 71) like every other family. Each is a distinct real-world knob, modelled on the matching plant template, exactly as v3's 945 already share five templates. A third wave (12 October, night) adds the founder's nine open-source pieces as ten muscles, each in the spine family its knob belongs to (`docs/THE_SEAT.md`): Kepler's energy reading driving a pool's target (Kubernetes Workload Scaling), Climatik's inference power cap, Adaptive GPU Power Capping and NVIDIA's platform power budget (GPU Hardware), EAR's CPU frequency policy and global energy budget and the Kubernetes Power Manager's core profiles (Host CPU & Memory), CruiseKube's and Goldilocks's request rightsizing (Container Resources) and PEAKS's power-aware placement (Kubernetes Placement & Scheduling). The organisms on v5: Compute / AI / Cloud 540, Physics / Robotics / Autonomous 404, Energy / Facility / Industrial 480, Distribution / Specialized 484, the four stacked 1,908, the whole tower 1,107; the spine all four realms share is 267. (v3's own catalog `realms/catalog.csv` stays 945, frozen for v3's results.)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Two math-only guardrails, and otherwise free arms:

- **The edge** — a per-knob cushion that widens at once when service climbs and narrows slowly when it falls (`cushion = climb × reaction + z × wobble`), so nothing riding its own limit falls off.
- **The centre / the hole** — nothing is eased into a pit it cannot climb out of. A cooled zone is eased only by the degrees its cooling can still recover (the Lindley reach), and not at all while its cooling headroom is falling toward empty faster than the cooling can answer. A process loop's setpoint never sits closer to its band's edge than the loop's own largest swing toward that edge (the loop reach, read from the historian's low and high over each decision, carried ahead for a slow loop). A serving pool is never capped below the largest jump in demand it has shown (the drop floor) nor so far that a burst would push its responses over the line where native's would not (the latency floor). And nothing permanent: a machine handed back to a cloud is the cluster autoscaler's own decision, because the autoscaler shrinks from wherever it is left, so an early hand-back would leave the pool a machine short of native for good; and a motion axis is slowed only while the wake forecast over a whole deadline (the most that can arrive in one deadline at the recent peak rate) still finishes inside that deadline at the slowed speed, so a delay is always recovered before a later rush could make it permanent. When work comes, every muscle wakes straight to the level the work needs (the founder's rule: from idle straight to 99, never past 100).

Every other knob is free to move both ways inside its physical cover, generating units included (a unit's steam, speed or gate loop eased inside its band serves the same load with less effort), and **a knob stays native only where its own physics says moving it cannot help without harm**: an interactive pool's power cap (its response line is shorter than one decision, so a cap's one decision of slower service cannot be seen and undone inside the line; the metered GPU, class P, is where that cap is proven), a battery's peak-shave (round-trip energy bought for service), and a cloud pool's machine hand-back. A batch queue's, a training job's or a terminal's power cap is free under the drop and latency floors.

Amendment, 12 October. The first v5 table (commit `c0e80a852`, the whole tower +0.0%) ran an engine that held every generating unit's loop, every power cap and every inverter native by blanket gates. The gate on generating units alone threw away almost all of what v5 saves: on the tower, 1.09% with it off against 0.04% with it on, on development seeds. v5 was re-declared with the gates replaced by the physics floors above, developed on seeds 7100-7209 only, and the preregistered seeds 3000-3009 were run again unchanged.

Amendment 2, 12 October. The catalog's second wave (95 muscles, five new families and additions to seven) took it to 1,097 muscles in 70 families; the engine's law is unchanged, its fingerprint re-declared because the catalog is part of it. Developed on seeds 7100-7109 only (0 of 1,097 LOSS or power given, all six organisms superior); the preregistered seeds 3000-3009 were run again unchanged on commit `04d9eb3b3`, with the same savings on the whole tower, the four stacked and Energy / Facility (3.0 times v3's).

Amendment 3, 12 October (night): the living body. The founder's order of 12 October: "whatever one muscle is having a problem, the other muscles kick in ... the brain and nervous system to activate its immune system ... nothing can be locked." The engine now carries, as properties of the code (`docs/FORMAL_V5.md`, section 4): a native twin of every room (the brain's sense of the temperature native

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

would leave it at, exact by construction), the room's own knob curing it where it can and the body's heat debt handing back the max's machines where it cannot (V8, V11), every battery's charge counted as room only while it lasts and its discharge timed for its line (V6, V12), no machine taken from a queue that needs it (V7), a pull-down handed back gradually and granted only where it can be held (V9, V10), and every forecast starting at the plant's first reading (V13: a lock that kept every battery's timed discharge shut for the whole run, and a room twin that started wrong, both removed). The catalog's third wave took it to 1,107 muscles. Developed on seeds 7100-7199 only: under the climb, plants with less work or more misses inside the four realms fell from 19 to 0 in 400 organism runs while the misses fell about twice as far as before; on seeds 7100-7109 every muscle alone and the six organisms on all three loads showed 0 muscles and 0 plants with less work or more misses. Fingerprint re-declared (digest `9be48445438943c2`); the preregistered seeds 3000-3009 were run again unchanged.

Amendment 4, 13 October: every muscle its own card. The founder's question of 12 October, night: does every muscle have its own tuning, maxed to its own benefit, instead of one global setting for a thousand muscles? The brain stays one; its settings are now each muscle's own (`docs/FORMAL_V5.md`, section 5). A muscle's card (`realms/muscle_cards_v5.csv`) holds what the max uses for it: a machine pool's edge utilisation and window, a battery's line margin and, wired by its power, its hold window and hold margin. The cards were found by an automatic search over every muscle whose settings can change what the max does (`tools/tune_cards.py`), on development seeds only, today's global setting always one of the choices, and kept only where the card beat the global setting beyond the noise on seeds 7100-7109 and again on seeds 7200-7209 and never behind native on any of their seeds under any load (never over its line more often, never less work, never its line cleared later): 364 muscles (333 machine pools, 31 batteries); every other muscle runs the global setting. Inside a body, a battery wired by its power keeps the body's margins on its line (V14: its card was chosen alone; the line carries the whole body's swing). In development, 360 organism runs on the two sets: 0 plants with less work or more misses, the misses removed 1.7 times the global setting's. Fingerprint re-declared (digest `5648f2902f1fae68`, 44 files, the cards among them); the preregistered seeds were run again unchanged. Every card is printed in plain words in the muscle catalog (`docs/MUSCLE_CATALOG.md`).

Amendment 5, 13 October (evening): every muscle from 1 up to one click under its own capacity, read live. The founder's order of 13 October: "you can write it in code ... to show one, two, whatever a muscle's capacity is able to handle, guardrailed and skimmed on its border, one click down probably, so it never goes beyond. And, of course, can all scale right back down. It can never go be less than one." No top or floor in the live controllers is a number written down for a muscle any more; each muscle reads its own from the system it runs on:

- **Machines** (the Kubernetes controller): the top is the operator's own `--max-nodes` where given; else every ready worker plus what the cluster's own autoscaler may still add (its published status); where Omni-Compass resizes the pool with the operator's own command and no autoscaler reports, there is no top but the one the pool shows: one under the first count it refuses, read from the command's own refusal.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Before: a fixed 1,000. The floor stays the founder's two machines, and cruise climbs toward the top one step a decision.

- **Replicas** (the replica room): at the autoscaler's cap with the compass pressing, the cap rises to the replicas the autoscaler's own arithmetic asks for, never past what the machines can hold of the service's pods, one pod under it, read live (the allocatable CPU of every ready worker and of the machines the system can still add, less what every other pod asks for); where every node is a container on one machine (kind), each reports the whole machine's CPU as its own, so that machine's CPU counts once (on 4 October the room raised to 30 on kind served no more requests and ran 78% more pods, `results/live/REPLICA_ROOM.md`: the cap was never kind's limit, the machine was); `--replica-ceiling N` is the operator's own; 0 keeps the cap unmoved. Before: off unless the operator gave a number.
- **The power cap at the max is never set under native**: each machine's idle CPU goes to the capped service's pods (one click under the machine: its last 5% is never handed out, and no pod gets less than its operator gave it), and the frequency ceiling and the GPU limit stay at their top. Before: the governor's trim to demand could set the pods' CPU limit and the frequency ceiling as low as 69.5% even at the max (never on the sets with a response-time probe, where the muscle already conveyed). The tune-down trims as before, with no fixed floor: the pod's own request and its use plus headroom (before: a fixed 65%), and the card's own minimum power limit, read live.
- **GPU**: the clock ceiling's floor is the card's own lowest supported clock (before: 35% of the top); one degree under the card's own slowdown temperature reads as full heat where the card reports it (before: a fixed 83 C for every card).
- **Batch**: held work is admitted while load and power stress are under the compass's wall less one cushion (0.95 - 0.05 = 0.90, the nervous system's own admit gate; before: load 0.80).
- **The nervous system's site power envelope** is the living band of the device's own range, [0.05, 0.95] (before: [0.65, 0.95]); the C++ twin the same, re-proven equal value for value and re-sealed.
- **Words**: the engine's own files say which way a number went (the founder's order of 13 October); the older result label "WORSE" is "AWAY FROM ITS AIM", the GPU benchmark's own.

The modelled organisms are unchanged: their code differs only in comments, file headers and that one label's name, so the 1,107 muscles, the cards and the max give the same numbers on the same seeds. The test rig with the real cluster inside changes beside the engine (protocol 3, `docs/K8S_COMPASS_PREREGISTRATION.md`): the model of the organism on the runner's last core and the kind nodes on the others, and the window set from the heavier arm's measured step. Fingerprint re-declared (digest `6093b2b88aa911b9`, 44 files); the mechanism identity's actuator part re-recorded (the core, the law, the observation map, the governor, the time step and the shield unchanged); every result runs again on it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.1.3 The no-harm proof (modelled, class S)

On a fixed seed, for **every** muscle in the catalog, the v5 default does no less work than native, is over its line no more often, and hands every knob back to native on the kill (the dead-man lease); the energy it takes is power given, shown beside the work and the speed it bought. On the preregistered seeds 3000-3009 (commit `eea291501848`, digest `5648f2902f1fae68`, every muscle its own card): **0 of 1,107 muscles LOSS or power given or mixed** (527 toward their aims, 421 at their limit on native, 159 the same), every organism ahead of native under all three loads, and **not one plant with less work or more misses inside any organism under any load**; the misses fall 1.9% to 7.5% under each organism's own load, 1.4% to 7.2% under the climb and 1.1% to 8.3% with work waiting in line, against 1.6% to 6.4%, 0.9% to 3.5% and 1.0% to 7.2% with one global setting (digest `9be48445438943c2`), with less power given and less wear. This is checked in `tests/test_compass_arm_v5.py` and run in full by `tools/run_realms_v5.py` (`results/realms_v5/`). Models are class S: a model's energy is not a meter's, and a plant written by the same people who wrote the law is not an independent test.

88.1.4 In development

While v5 is in active development its fingerprint is **re-declared on each engine change** (`python3 tools/omni_version.py --write v5`), and **no v5 result is published** until the v5 freeze. At the freeze, every benchmark is rerun on v5, A/B/C, the founder hears each result before it goes up, and the fingerprint is final.

88.2 Omni v6: capacity repair and complete non-GPU rerun campaign

Omni v6 stood on main from 15 to 16 October; Omni v7 (`docs/OMNI_V7.md`) stood from 16 to 17 October, and since 17 October the engine that stands is Omni v8 (`docs/OMNI_V8.md`, the founder's order). This page and v6's four three-run tables (`results/live/V6_*.md`) stay as that engine's record, read beside no other engine's.

The engine combines the unchanged v5 muscle law and 1,107-muscle catalog with the v6 shared-host Kubernetes capacity repair. The controller samples Linux host CPU occupancy, counts kind's shared host once, refuses extra replica capacity when that host is busy or unmeasured, and rechecks a previously enlarged replica ceiling. This is a versioned evaluation engine; it does not establish that every workload gains. Every result retains its own engine fingerprint and evidence class.

The source of the capacity repair is branch `codex/v6-kind-headroom-repair`, commit `3bda41a00fab81052551be99e618a523b5c61347`. The damaged Markdown manuals on that branch are not imported. The existing intact manuals remain the starting point. The zero-valued machines-only flag repair is retained: `1` selects machines-only; `0`, empty and unset retain full wiring.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.2.1 Campaign

`scripts/benchmark_campaign.json` records this request. A successful `verify` run for the release commit marked `[all-nongpu]` starts the grid automatically; an unsuccessful release verification launches no campaign. `scripts/reruns.txt` lists every single-benchmark variant; the grid conductor releases A, B and C gradually alongside every organism at 1, 10, 100 and 1,000 copies and nested 1, 10, 100 and 1,000 runs on free GitHub runners. The v6 dispatch manifest lives under `results/grid_v6`; v5 receipts remain historical. Each job prints its engine fingerprint and source commit. A queued, failed or unavailable test is not a completed measurement.

The queue covers Kubernetes baseline/full/levers/observe, fairness, faults, batch and trace loads, KEDA, CruiseKube, Goldilocks, fixed capacity, scale control-plane checks, both muscle catalogs, all seven organisms (the spine the seventh since 16 October, `docs/THE_SPINE.md`) and their live-cluster grid, Redis, Kafka, PostgreSQL, MySQL, MongoDB, MuJoCo, pandapower, CityLearn, drone swarms, robustness, Linux CPU-governor checks, the Azure full-control burst and the larger Azure organism machine. Linux cpufreq/RAPL execution requires a host that exposes the interfaces; a hosted VM capability refusal remains visible as unavailable hardware.

The Azure burst is `1 4 1 6 1 9`, three paired repetitions, 600 measured seconds per arm, 100 seconds per load stage. Full wiring enables HPA control, pod CPU limits and the existing power model. It does not create host RAPL, clock control or physical power meters on managed Azure VMs.

Free GitHub Kubernetes, Linux, swarm and organism/model workloads may use 1,000 copies and 1,000 runs. Paid cloud and GPU organism scales remain 1, 10 and 100, with at most 100 repetitions. The conductor lists cells that exceed free runner memory/time as pending runner capacity; it never rents a cloud machine for them. The 1,000-copy planning estimates are extrapolations, not completed measurements.

88.2.2 Prepared, not launched

Google/GKE and Amazon/EKS retain their deployment scripts and use the new shared engine on a future manual launch. Their workflows, cloud-organism, gpu-bench and lambda-gpu are excluded from this campaign. No GPU machine is rented by it. `scripts/gpu_campaign_profiles.json` defines one physical card and eight physical cards, with all seven organisms at scales 1, 10 and 100 and 100 paired repetitions per scale. Reports show nested 1/10/100 run prefixes. These are model copies around real hardware; only these three organism scales are permitted.

The profiles are operational through the existing `HIL_SCALES` and `HIL_REPS_BY_SCALE` environment variables. Existing wall-clock budgets still apply: a stage that cannot finish within its budget must be reported incomplete.

© 2026 The Omni-Compass LLC. All rights reserved. **Evaluation and Simulation use only.** Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` and `DISCLOSURES.md`. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

88.3 Omni v7: tighter convergence on top of v6 (the engine of 16 to 17 October)

Omni v7 stood on main from 16 to 17 October; on 17 October it was followed by Omni v8 (`docs/OMNI_V8.md`, the founder's order), omni-v7 with one wire changed (the CityLearn runner: a battery whose building simulates outages stays native), and the same day by Omni v9 (`docs/OMNI_V9.md`), and on 18 October by Omni v10 (`docs/OMNI_V10.md`), the engine that stands. This page and v7's twenty-two three-run tables (`results/live/V7_*.md`) stay as that engine's record, read beside no other engine's.

Omni v7 was the engine that stood on main from 16 October (the founder's order: everything on the newest engine). Every benchmark ran again on it (`scripts/reruns.txt`; the grid conductor's reruns, grid and Azure queues restarted on it the same day), and every earlier table stands as its own engine's result until its three runs on v7 land; results from two versions are never pooled. v7 is v6 with five changes, each found from a v6 run's own files or from the governing rules themselves:

#	Change	Where	What it tightens
1	CPU conveyance obeys the namespace quota	`omni_controller/quota.py`, `muscles.py`	no write Kubernetes would refuse
2	On a shared host, no limit raised while the machine is at or over 75% busy or unread (the replica room's guard)	`omni_controller/muscles.py`	the host's top, nothing else
3	The verdict reads a trial in the reports' own words (GAIN, STEADY, LOSS)	`omnicompass/verdict.py`	the do-no-harm trigger
4	Every rule counts in seconds, not decisions	`omni_controller/controller.py`	the same behaviour at any decision period
5	The cloud's rule in one word, `--give-back-only`	`omni_controller/controller.py`, `deploy/install/omni-compass-cloud.yaml`, `scripts/kind_bench.sh`	the bill can only fall or stay

88.3.1 Quota-aware conveyance

The original capacity run 38722001872 tried to raise a pod CPU limit while its namespace CPU quota was already full. The candidate reads each namespace's ResourceQuota CPU-limit hard/used values, obeys the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

stricter pending spec/status ceiling, and reserves each accepted increase across all target deployments in one conveyance pass. It does not increase or delete an operator quota. PriorityClass scope selectors are matched from pod metadata, without deployment or quota names hardwired into the governor. Unknown quota state, read failures and unsupported scopes preserve native limits for expansion; reductions remain possible. Decreases do not create spendable headroom until observed by Kubernetes.

What the first live trial found, and the correction (16 October). The three trials of the candidate as first pushed (Goldilocks run 38825214203, the KEDA add-ons run 38825211370, the capacity test run 38825208939; one run of ten pairs each, so single runs, not three-run tables) read the candidate well short of omni-v6's three-run tables on the two add-on tests: Goldilocks response time at the 95th percentile -6% (STEADY) against v6's -58% GAIN in all three runs, HPA replicas +20% and pods started +200% (power given), the pod start wait +172% (LOSS); KEDA on the requests in flight p95 -2.7% (STEADY) against v6's -73%, replicas +17% (power given); KEDA on the CPU target and the requests p95 +3% (STEADY) against v6's -55%; the capacity test STEADY at every cap, as on v5. The governor wrote 5 commands a run against v6's 15 and the host was 4% busier against 40%: conveyance never raised a limit. The audit says why, 65 times in one repetition: `quota: CPU limit expansion blocked, reason: quota read unavailable: CalledProcessError`. The governor's role on every kind test (`deploy/kind/rbac-omni.yaml`, receipted in each run's `rbac_omni.txt`) cannot list ResourceQuotas, and the budget as first written read an unreadable quota as a zero allowance. The model gate had not seen it because its model of the cluster answered the quota read with an empty list. The correction: a namespace whose quotas cannot be read is **no budget known**, not a zero budget; the raise proceeds as it did on omni-v6 and Kubernetes' own admission, which refuses any write over a quota the governor cannot see, stays the authority (the plug reads every write back and a refused one is audited and never advertised); the audit says so once a pass. Where the role can read the quota, the budget bounds every raise as before. The model now runs the convey model a second time with the quota unreadable (section 88.3.6), the regression `test_an_unreadable_quota_holds_nothing` holds the v6 arithmetic under a forbidden read, and the three trials run again on the corrected candidate; the first trials stay in `results/live/raw/` on this branch as the record of the fault.

Kubernetes admission remains authoritative. A concurrent rejected write is audited and does not become an invented conveyance gain. The node's existing CPU cushion, operator CPU floor, security hold, workload schedule and benchmark validity checks remain intact. The separate runner shutdown in the historical run has no established cause and is not claimed fixed by this actuator change.

88.3.2 Conveyance on a shared host: one guard, two designs refused by the model

The v6 three-run KEDA table (`results/live/V6_KEDA_REQUESTS.md`) read the pod start wait, mean, +12.5%, +12.0% and +22.7% (STEADY on the three-run rule, LOSS in two single runs) beside response time -59% and CPU used +55%. The run's own files (`results/live/raw/addons-run-38725705826`): 149 raises, each pod's limit set to 1,850m (two serving pods on its node) or 3,700m (one), the HPA at 8.8 replicas on average, the host 0.27 busy under native and 0.34 under Omni-Compass, past 75% busy for 0.3% of its samples. On kind every node is a container on one 4-core machine and reports all four cores as its own,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

so the limits sum to about three machines: a ceiling, not a use; the kernel's fair scheduler arbitrates when the pods ask for more than the machine, as it does for native's pods.

Two repairs were written first and both were refused by the model gate (section 88.3.6) before any real run: **the host's CPU counted once** and shared by every serving pod (at the run's nine replicas 3,500m / 9 is under the operator's 500m, so nothing is ever conveyed: in the model the +64% work and -53% response time of conveyance are lost whole, requests served -16% against v6, LOSS); and **a hold while a pod starts on the host** (start wait 1.1% shorter for requests served -0.11% and response time +0.5%, both LOSS against v6 at the real regime). v7 keeps the one guard the model reads as no LOSS, the same as the replica room's (`omni_controller/capacity.py`): on a shared host no limit is raised while the machine's measured busy share is at or over 75% or cannot be read; limits already given stay; lowering is never held back; each hold is written to the audit (`"convey": "held"`). The slower pod start is the busier host serving more requests, the price of the work; it stays shown in every receipt and is judged as the table judges it.

88.3.3 The verdict in words

The verdict (`omnicompass/verdict.py`) is the engine's do-no-harm trigger: before a knob may move one step further from native, the step runs as a trial and its cost per piece of work is compared with the cost at the reference. Through v6 the comparison was the two medians alone: the step was allowed if m[trial] ≤ m[ref] · (1 + tolerance). On noisy work a median wanders, so a step that truly cost more than the allowance was often allowed by chance.

v7 (`judge="interval"`, the default) reads the trial exactly as a report reads a row (`tools/readings.py`). On the logarithm of each piece of work's cost, so that a heavy tail does not decide it, the change is

$d = \text{mean ln } c[\text{trial}] - \text{mean ln } c[\text{ref}]$, with the 95% interval $d \pm 1.96 \cdot \sqrt{s^2[\text{ref}] / n[\text{ref}] + s^2[\text{trial}] / n[\text{trial}]}$ (Welch's standard error),

and with $\tau = \ln(1 + \text{tolerance})$ the allowance in the same units:

- the interval wholly above zero is a **LOSS**: the step is refused and undone, whatever the allowance says;
- wholly below zero is a **GAIN**; across zero is **STEADY**;
- while the interval's half-width exceeds τ, the evidence is wider than the allowance, and the trial measures on, up to three times `min_samples`, before it decides;
- a GAIN or STEADY step is allowed only if both estimates are inside the allowance: the median (v6's guard, kept) and $d \le \tau$;
- a stepwise knob is read the same way against the costs first measured at native as well, so a run of small steps cannot add up past the allowance.

Every judgement writes its reading and its intervals to the audit (`"reading"`, `"interval_ln"`). `judge="median"` reproduces v6 exactly.

On noisy work (log-normal costs, σ = 0.25, a 2% allowance, 300 trials each, seeded):

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

True cost of the step	Allowed through v6 (medians)	Allowed in v7 (in words)
+0% (free)	63%	60%
+1%	56%	50%
+2% (at the allowance)	47%	39%
+3%	40%	28%
+5%	26%	14%

A step that costs more than the allowance is allowed about a third less often at +3% and about half as often at +5%; a free step is kept almost as often. A step whose cost is clearly higher reads LOSS and is refused. These are properties of the rule on modelled costs (class S), not benchmark results (`tests/test_verdict_interval.py`).

88.3.4 Every rule counts in seconds

Five of the governor's rules counted decisions: the clear time after a breach (3), the cruise wait (2), the strict window (5), the verdict's probe spacing (10), recheck (120) and trial length (20). At a decision every 60 seconds those are 3, 2, 5, 10, 120 and 20 minutes; at 15 seconds they would silently shrink fourfold, and a give-back trial's 20 decisions would last 5 minutes instead of 20. v7 states each rule in seconds (`--slo-clear-s 180`, `--cruise-after-s 120`, `--strict-window-s 300`, `--verdict-every-s 600`, `--verdict-recheck-s 7200`, `--verdict-trial-s 1200`) and converts it to decisions at the running period, n = max(1, round(seconds / period)). At 60 s every count is v6's; an explicit count (`--slo-clear`, `--cruise-after`, `--strict-window`, `--verdict-every`, `--verdict-recheck`) still wins.

Two moves are made rates in time as well: the compass moves the HPA target by the same amount a minute at any period (its step is scaled by period / 60, and coasting likewise), and a node pool gives back at most one machine a minute (`RELEASE_PERIOD_S = 60`), however often the governor decides.

88.3.5 The cloud's rule in one word

In a cloud Omni-Compass shows one thing: the same work as native for a smaller bill, never a bigger one. Through v6 that rule was assembled from several settings in the workflow (`OMNI_MACHINES_ONLY=1`, the dial at the tune-down, the sensed and capped deployments left empty). v7 makes it one flag, enforced in the governor itself: `--give-back-only` turns off every pod-, processor-, power-, clock-, cooling- and batch-actuator, sets the replica ceiling to 0 and the dial to the tune-down, and leaves the node pool as the only knob. Whatever else is passed, the governor can then only park a machine the cloud already made, after its own trial, and wake it on a breach: no more pods and no more machines than native. The Azure, Google and Amazon install file (`deploy/install/omni-compass-cloud.yaml`) and the machines-only branch of `scripts/kind_bench.sh` pass it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.3.6 The model gate: every change run in our own models first

The founder's order of 15 October, night: every fix, knob and rule is tried in a model before it is pushed, and goes to a real run only where it shows promise there. `tools/model_gate.py` runs two checkouts, the engine that stands and the candidate, each in its own process, on the same seeds, and reads the candidate against the standing engine paired by seed (the mean change, its 95% interval over the seeds, the word by the gauge's aim). Three models, all evidence class S, our own models, never a benchmark result:

- **cluster**: the shipped Kubernetes controller in the cloud setting (node-pool mode, the compass law, the tune-down, `--give-back-only`) on the model cluster of `pilot/selfpilot.py` as the cloud meets it (the cloud's own autoscaler makes machines in both arms; the governor only parks one that exists, which the cloud deletes after ten minutes empty, and wakes it while it exists); a decision every 60 s and the fast check every 15 s; **cluster15** the same with a decision every 15 s.
- **convey**: one 4-core host carrying three kind nodes, php-apache under its HPA (200m request, 500m limit, 50%, 1 to 10), load generators stepping 1 2 3 1 2 1 as `deploy/kind/loadgen.yaml` runs them, set so the native arm sits where the v6 KEDA run's native arm did (host busy 0.28 and p95 0.40, response p95 229 ms, 8.3 pods against the run's 0.27, 0.42, 248 ms and 8.8); the candidate's own `Muscles.convey` called every 15 s. Since 16 October the model runs twice: once with the namespace's quota readable and empty (`convey`), once with the quota unreadable, as the governor's role finds it on every kind test (`convey_unreadable_quota`): the first trial's fault (section 88.3.1) is now the model's to catch.
- **verdict**: the do-no-harm trigger alone on noisy work.

What it read on v7 against v6, 10 seeds (`results/model_gate/`):

Model	Reading
cluster, 60 s	v7 and v6 STEADY on every gauge; both against native: bill -1% (STEADY), energy per core-hour -2.4% GAIN, pods waiting for a place about three times native's (v6 LOSS, v7 STEADY). The cloud setting itself barely cuts the bill in this model and makes pods wait; checked against the Azure receipts' own pending-pod gauge before anything is concluded.
cluster, 15 s	v6: bill +43% (power given), pods waiting 10x (LOSS). v7: bill STEADY against native, pods waiting +111% LOSS, p95 +0.3% LOSS. Every rule now keeps its time, and faster is still not free: the default stays 60 s.
convey	v7 as first written (host once, start hold): requests served -16%, p95 +155% against v6, LOSS, nothing conveyed. The busy guard alone: identical to v6 at the real regime and at double load (the hold never binds under 75%).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Model	Reading
convey, quota unreadable (added 16 October)	the candidate as first pushed (digest 1a0d1d1b91bc49ad): requests served -39% and p95 +113% against v6, LOSS, the host and the limits given identical to native: nothing conveyed, as the live trials found. The corrected candidate (digest d6d467b824c585f1): identical to v6 on every gauge (+64% requests and -53% p95 against native), with the quota readable and empty and with it unreadable alike.
verdict	a step costing 3% more allowed 28% of the time instead of 40%; 5% more, 14% instead of 26%; a free step 60% instead of 63%.

88.3.7 The first three-run tables on v7 (16 October)

Twelve runs on real Kubernetes, ten pairs each, every one omni-v7 (`results/live/V7_*.md`), native against Omni-Compass on top. No row reads LOSS on the three-run rule.

Benchmark	What moved, GAIN in all three of our runs	STEADY	Power given
Goldilocks and the VPA's recommender	response time −45% to −47% mean, −60% to −62% p95, −67% to −69% p99; time over the line −100%	machines, failed requests, replicas, pods started, pod start wait, alert minutes	parked-worker energy +1.6% to +2.3%
KEDA's HTTP add-on	response time −57% to −58% mean, −71% to −75% p95; alert minutes −93% to −96%; replicas −15% to −19%; pods started −17% to −23%	machines, failed requests, pod start wait	parked-worker energy +2.2% to +2.3%
KEDA on CPU and requests	response time −36% to −38% mean, −52% to −54% p95, −57% to −59% p99; time over the line −98% to −100%	machines, failed requests, replicas, alert minutes	parked-worker energy +1.1% to +1.7%
CruiseKube applying its own recommendations	nothing: no setting written	every gauge	none
Fixed capacity, four CPU caps	nothing: the cap blocks the one lever	completed work and work inside the line at every cap	none

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Single runs clear of the noise that the other two did not repeat, each STEADY on the rule: CruiseKube's 99th percentile +3.1% (run A) and node-hours +1.0% (run B); Goldilocks replicas +3.4% (run C); capacity's completed work +3.8% at 1 core (run A).

88.3.8 The room search (16 October): the muscles that read the same as native

The founder's standing question is whether every muscle runs at its own edge. On the realms table of the engine that stands (`results/realms_v5/MUSCLES_V5.csv`, the preregistered seeds 3000-3009), 159 of the 1,107 muscles read SAME AS NATIVE on every load. Measured muscle by muscle on development seeds (7100 and 7101) in our own models (class S, never a benchmark result):

Muscles	Plant and knob	What the max does	Native's own misses	Room under the max's gauges
111	a process loop's setpoint (the process, irrigation, turbine, batch-reactor, pipeline, mill, district-heat and inverter presets)	writes nothing: the loop's actuator is native's full range, and the loop's work is the disturbance it handles, fixed by the load	median 0.00% of decisions, at most 0.56%; 6 of the 111 miss at all	none on work or speed; on misses, the 0.56% of six loops, which a setpoint eased toward the band's middle could touch: a card candidate for a later version, worth under one part in a hundred
36	a cooled zone's setpoint (the data-hall, building and hospital presets)	cools ahead of the heat (28 of the 36 wrote), and under native the room never nears its line either	0.00% on all 36	none: nothing above native to give
12	a battery's power knob (the UPS, microgrid and facility presets)	holds the charge for the line and gives native's full power once the line could need it	2.1% to 24.3% of decisions	every miss falls while the charge sits at the operator's reserve (0.6 for a UPS, 0.2 for a microgrid), above the outage floor; the power knob may not spend the reserve, and the max keeps that boundary. The charge between the reserve and the floor belongs to the setpoint knob, where the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Muscles	Plant and knob	What the max does	Native's own misses	Room under the max's gauges
				engine's cards already read GAIN (8 of 8)

The reading: the max already takes every muscle to the edge its own knob has. Where a muscle reads the same as native, the measurement says there is nothing above native for that knob to give, and the max leaves it native by design (do no harm). The one measurable room, six process loops' 0.56% of decisions, is noted for a later version's card search; it is not an engine change, and no engine file moved for this measurement.

88.3.9 Validation

Six shared-host regressions pass (`tests/test_convey_shared_host.py`: each node keeps its own arithmetic under the top, a busy or unread host holds every raise, a busy host still lowers, a start holds nothing, separate machines are never held, an unreadable quota holds nothing). Nine quota regressions pass (`tests/test_quota_convey.py`, among them: an unreadable quota is no budget known); the original source fails both the full-quota regression and the admission-rejection regression. Existing conveyance integration checks pass unchanged. The verdict's rule tests (`tests/test_verdict_interval.py`, `tests/test_verdict.py`: v6's medians kept under `judge="median"`, a noiseless 1.5% rise refused against a 1% allowance under v7), the time-correct tests (`tests/test_time_correct.py`, `tests/test_cloud_cadence.py`) and the cloud tests (tests/test_cloud_machines_only.py (removed on 31 October, when the founder struck the cloud's rule)) pass, as do every verdict user's tests (Redis, Kafka, pgbench, sysbench, YCSB, CPU power, GPU). Live GitHub trials on the v7 commit (KEDA's add-ons, Goldilocks, CruiseKube, capacity) repeats the original cap 1500m / repetition 4 / 1200-second-per-arm case at the original request rates. Its completion and measured result must be read from its artifact, with any LOSS retained. The trial cannot launch Azure, Google, Amazon or GPUs.

The organisms the candidate is run on are the seven of 16 October (the spine, the 267 muscles every stack runs on, the seventh; `docs/THE_SPINE.md`): they are groupings of catalog rows made in the runners, so the candidate's fingerprint did not move for them, and every organism table on this engine lists the spine first.

Reference: Kubernetes Resource Quotas and in-place container resource resizing. The CPU budget helper is included in both the engine and actuator fingerprints. README.md, LICENSE, NOTICE, DISCLOSURES.md, PATENTS.md and TRADEMARKS.md accompany the trial artifact.

© 2026 The Omni-Compass LLC. All rights reserved. **Evaluation and Simulation use only.** Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` and `DISCLOSURES.md`. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.4 Omni v8: the battery that carries its building through an outage stays native (the engine of 17 October)

Omni v8 stood on main on 17 October; later that day it was followed by Omni v9 (`docs/OMNI_V9.md`), omni-v8 with the CityLearn battery wire reading the coming hour, which cuts the one row this page names (section 88.3.3; printed LOSS, power given by the key, `docs/OMNI_V9.md`, section 1), and on 18 October by Omni v10 (`docs/OMNI_V10.md`), the engine that stands. This page stays as omni-v8's record, read beside no other engine's.

Omni v8 was the engine that stood on main from 17 October (the founder's order: "there's a new version 8 ... push in all and update all"). It is omni-v7 (`docs/OMNI_V7.md`) with one change to one wire, made because a three-run table of omni-v7 read LOSS and the cause was found to be ours. Its fingerprint is `OMNI_V8.json` (digest `f93d233136a1f902`, 46 files); of those 46 files, 45 are omni-v7's bytes, and the one that differs is the CityLearn runner. Every benchmark runs again on it (`scripts/reruns.txt`; the conductor's reruns, grid and Azure queues restarted on it the same day), and every omni-v7 table stands as omni-v7's result until its three runs on v8 land; results from two versions are never pooled. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

#	Change	Where	What it tightens
1	A battery whose building simulates power outages stays native: Omni-Compass moves only the batteries with no second duty	`tools/run_citylearn.py`	do no harm where the battery's charge is also its building's reserve

88.4.1 What omni-v7 read, and why

The omni-v7 CityLearn table (`results/live/V7_CITYLEARN.md`; runs 38937540021, 38949618073, 38959444615) read GAIN in all three of our runs on the electricity bought and the daily peak in all ten battery districts, and **LOSS in all three of our runs in the seven 2023 districts** on the energy not served (+2.0% to +9.8%) and the time uncomfortable (+0.01% to +0.15%), beside the ramping +4.5% to +6.2%, the highest peak +2.5% (three districts) and the monthly unevenness +0.07% (one), power held and so power given by the key (the table printed those three LOSS too, until the CityLearn tool was corrected on 17 October).

The cause was found on CityLearn itself (version 3.0.2, the version the runs installed), run locally on the same district files and reproducing the omni-v7 numbers of `citylearn_challenge_2023_phase_2_local_evaluation` to the last digit:

- The seven 2023 districts are the only districts CityLearn ships whose buildings simulate power outages (every building in each: 3, 3, 3, 3, 6, 6 and 6; every other district none, read from each district's own `schema.json`). CityLearn counts the energy not served only in outage hours (`citylearn/cost_function.py`, `normalized_unserved_energy`), and the time uncomfortable rises when an islanded building cannot run its cooling.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- The battery wire read the district's draw alone. It did not see the outage hours, and it did not see that in these buildings the battery has a second duty: carrying the building through the outage. A command that moves the battery for the district's draw during an outage asks for energy the islanded building cannot have.
- Three narrower repairs were tried on the tuning district first and refused there: charging only (the energy not served unchanged at +5.96%, LOSS, the ramping +3.5%); handing the battery back to native only during outage hours (the energy not served −2.7% and the discomfort −0.13%, both GAIN, with the ramping +7.0% and the monthly unevenness +0.06% beside them); and moving no more energy than the draw's excess over its band (the daily peak +0.5% and the ramping +2.6%). With peaks, swings and unevenness then read as misses, each was refused. By the key (17 October) those rises are power given, so the second, native only in outage hours, reads GAIN on both misses; it is studied again for the next engine version.

88.4.2 The change

The wire now leaves native every battery whose building simulates power outages (CityLearn's own `simulate_power_outage`, read from the building), as a water tank was left native in round 2: a battery that is also its building's reserve is a muscle with a second duty, which needs its own wire and its own reading (the outage risk) before Omni-Compass moves it. Every other battery is moved exactly as before: the compass law, its gain, its response, its glide, its band and its hand-back are unchanged. An action list the wire cannot place on its buildings moves nothing. The hour count of the file counts only hours where a command moved, and each district's file says how many batteries were left native for outages (`batteries_left_native_for_outages`).

88.4.3 What it does, read on CityLearn before any GitHub run

On the seven 2023 districts the omni arm is native's to the last digit (every battery left native; checked on `citylearn_challenge_2023_phase_2_local_evaluation` and `citylearn_challenge_2023_phase_3_1`). On every district without outage buildings the wire's commands are the same bytes as omni-v7's, so their results are omni-v7's (checked to the digit on `citylearn_challenge_2022_phase_all_robustness`, every gauge of both arms identical to omni-v7's GitHub run 38937540021, no battery left native). So v8 removes every LOSS of the 2023 districts and, with them, their small GAINs (the electricity bought −0.1% to −0.3%, the daily peak −1.3% to −2.6%), which were bought with the LOSS beside them.

One row the table printed LOSS remains, and it is said here: in `citylearn_challenge_2022_phase_all_robustness` (no outage buildings) the monthly unevenness rose 0.11% in all three of our runs (power given by the key: a ratio of the power held), beside the bill −8.6%, the electricity bought −14.7%, carbon −11.9%, the daily peak −14.0% and the highest peak −2.7%, all GAIN. The score is a ratio (one minus each month's mean over its peak), and it rose because the energy fell faster than each month's top hours, which sit at the batteries' limit. Its repair is looked into on its own, in the model gate first; v8 does not change it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.4.4 The model gate and the trial

The change touches no Kubernetes, card or cloud controller, so the model gate's models (`tools/model_gate.py`) read the same on both checkouts by construction; the gate was run all the same, omni-v8 against omni-v7 on the same seeds, and is recorded in `results/model_gate/V8_VS_V7.md`: seeds 1 to 8, a day each, every gauge of every model (the cluster at 60 s and at 15 s, conveyance with the quota readable and unreadable, the verdict on noisy work) identical between omni-v8 and omni-v7, to the last digit and in every count (steps allowed and refused, trials abandoned, machines parked, errors). The models' readings against native are omni-v7's and stay disclosed in `docs/OMNI_V7.md`, section 6 (at a decision every 15 s the cloud setting makes pods wait longer than native, which is why the decision period stays 60 s). These are our own models (class S), never a benchmark result.

The trial is the CityLearn benchmark itself, A, B and C on the v8 commit, read by `tools/citylearn_abc.py` into results/live/V8_CITYLEARN.md when they land: it must read no LOSS in the 2023 districts and the same table as omni-v7 everywhere else. The disclosure every amendment carries: every CityLearn district has now been seen, so no untouched district remains for this amendment; it was fixed on one district (`citylearn_challenge_2023_phase_2_local_evaluation`) and is reported on all (`docs/CITYLEARN_PREREGISTRATION.md`, amendment of 17 October).

88.4.5 What runs again on it

Everything, A, B and C, from the moment it landed (`scripts/reruns.txt`, released by the conductor a few a tick: A of every benchmark, then B, then C); the grid on its own manifest (`results/grid_v8/`); every Azure set from A again (`scripts/azure_queue.txt`). The omni-v7 runs already started run to their end (nothing is cancelled) and stay as omni-v7's record. Google Cloud and Amazon carry the same sets behind their switches, off until the founder turns them on; the GPU runs on Lambda wait for his word and one exact commit. The organisms are the seven of 16 October, the spine first (`docs/THE_SPINE.md`).

© 2026 The Omni-Compass LLC. All rights reserved. **Evaluation and Simulation use only.** Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` and `DISCLOSURES.md`. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

88.5 Omni v9: the battery wire reads the coming hour (the engine that stood on 17 and 18 October)

Omni v9 stood on main from 17 to 18 October; on 18 October it was followed by Omni v10 (`docs/OMNI_V10.md`), the engine that stands: omni-v9 with the battery glide at 0.0875 an hour, the speed that cuts the bill most with no LOSS (section 88.3.2 below gives the ladder it was chosen from). This page stays as omni-v9's record, read beside no other engine's.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Omni v9 was the engine on main from 17 October (the founder's orders: "we're going into another version of Engine ... Apply the fix, run it, and see in the model sim that it works"; "Omni-Compass should be tuned to exploit all that can be and keep all healthy and never raise its bill ... the tuning stays where it doesn't ever hurt anyone"). It is omni-v8 (`docs/OMNI_V8.md`) with one wire repaired and one statistical table made exact. It was written for one omni-v8 row the table printed as LOSS; on 17 October the founder found that word misapplied (section 88.3.1): the row was power given. The repair it led to stands on its own, because it cuts the bill further in every district. Its fingerprint is `OMNI_V9.json` (digest `7379f3a79ff2c475`, 47 files); 44 are omni-v8's bytes, two differ (the CityLearn runner and the realms harness) and one is new (`omnicompass/student_t.py`). It was proven in our own runs of CityLearn and in the model gate before it was pushed. Every benchmark runs again on it, and every earlier table stands as its own engine's result until its three runs on v9 land; results from two versions are never pooled. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

#	Change	Where	What it tightens
1	The battery wire reads the coming hour's draw (the mean of the same hour over the past seven days, only hours already seen) and its adjustment glides at up to 0.0625 of the command's range an hour (0.05 before), the fastest glide that read no LOSS on any score in any district	`tools/run_citylearn.py`	the battery answers the hour it moves, not the hour gone
2	Every 95% interval uses the exact Student t quantile for its number of pairs	`omnicompass/student_t.py`, `realms/harness.py` and the table tools	no interval narrower than its sample allows, at any count

88.5.1 What omni-v8 read, and why

One row of the omni-v8 table was printed LOSS: in `citylearn_challenge_2022_phase_all_robustness` (17 buildings, a battery in each, no outage buildings) the monthly load unevenness (`monthly_one_minus_load_factor_average`: one minus each month's mean draw over its peak) rose 0.11% in all three of our runs, beside the bill −8.6%, the electricity bought −14.7%, the daily peak −14.0% and the highest peak −2.7%, all GAIN (the omni-v7 table, whose commands omni-v8 repeats to the byte outside the 2023 districts). **By the key that row is power given, not a LOSS**: the unevenness is the shape of the power the district holds from the grid, a resource held, as a peak is; it is neither time waited nor work done nor a miss. The CityLearn table tool had filed the peaks, the swings and the unevenness as misses; it was corrected on 17 October (`tools/citylearn_abc.py`, `tools/readings.py`: one key for every gauge, as a cluster's peak power always was). Run locally, the same wire also raised, on districts the GitHub benchmark does not run (CityLearn ships them only in its

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

source repository), the tuning district's monthly unevenness +0.26%; in the 2022 phase 2 the highest peak +0.17%, the daily unevenness +1.03% and the monthly unevenness +0.68%; in the 2022 phase 3 the highest peak +0.45% and the ramping +3.37%: power given, every one, and shown here with the rest.

The cause was found in an hour-by-hour trace of the omni-v8 wire on the robustness district (CityLearn 3.0.2, run locally):

- The compass read the district's draw of the hour just gone and moved the batteries for the hour coming, and its adjustment could change by only 0.05 of the command's range an hour. Averaged by hour of day over the year, the adjustment discharged hardest at 00:00 and charged hardest at 15:00, while the district's draw peaked at 19:00 and bottomed at 13:00: it ran about five hours behind the day, still adding charge at 17:00 and 18:00 as the evening peak climbed.
- So the batteries were empty when each month's top hour came: at the month's highest net draw the mean state of charge was 0.000 to 0.037 in ten of the eleven months the trace covers.
- The score is a ratio. Lowering a month's mean (less energy bought) while its peak hour stays at the batteries' limit raises one minus mean over peak: the energy fell faster than the top hours.

Narrower repairs were tried on the robustness district first; each left the unevenness up, so none is in the engine:

Candidate	What it changed	Monthly unevenness against native
no discharge below the band's middle	Omni-Compass adds no discharge in a quiet hour	+0.11% (power given)
no new peak (the month's top so far, and the month's or the week's)	the charge it adds never lifts the district above its top so far	+0.11% (power given)
peak push	full discharge where the draw nears the month's top so far	+0.12% (power given)
a reserve of 30%, 50% or 70% of charge	its added discharge stops at the reserve	+0.42%, +0.25%, +0.25% (power given), and the highest peak's GAIN lost
the hour gone, glide 0.25	faster only	−0.61% (GAIN), the ramping +13.2% (power given)

88.5.2 The change, and how far it is pushed

The reading is the draw of the hour the batteries are about to move: the mean of the same hour of the day over the past seven days, h[t−24], h[t−48], …, h[t−168] (only hours already seen; before a day has been seen, the last hour's draw). Nothing else in the wire changed: the compass law, its gain (0.5), its response (1 h), its band

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(the 10th to the 90th percentile of the past week's draw), the hand-back at 90% of the year, the batteries left native in outage buildings (omni-v8) and the water tanks left native.

The glide was set where no score of any district rose: with the new reading, every glide from 0.05 to 0.10 an hour was run on the districts where the edge lies, and the fastest at which nothing rose in either is 0.0625. Read by the key, no glide on the ladder reads a LOSS anywhere: every rise below is power given, and the bill keeps falling up to 0.0875 to 0.10:

District	Score	0.05	0.0625	0.075	0.0875	0.10
phase 2	daily unevenness	+0.25% power given	-0.09% GAIN	-0.67% GAIN	-0.94% GAIN	-0.91% GAIN
phase 2	net-zero shortfall	-7.53% GAIN	-4.77% GAIN	-3.38% GAIN	-2.54% GAIN	-1.99% GAIN
phase 2	the bill	-14.12% GAIN	-17.22% GAIN	-18.75% GAIN	-19.17% GAIN	-19.03% GAIN
phase 3	the ramping	-5.05% GAIN	-1.91% GAIN	+2.32% power given	+6.18% power given	+9.59% power given
phase 3	net-zero shortfall	-2.21% GAIN	-0.76% GAIN	-0.04% STEADY	+0.25% power given	+0.41% power given
phase 3	the bill	-6.60% GAIN	-7.32% GAIN	-8.10% GAIN	-8.43% GAIN	-8.32% GAIN

At 0.05 the 2022 phase 2's daily unevenness rises; from 0.075 the 2022 phase 3's ramping does, and from 0.0875 its net-zero shortfall: power given, each. At 0.0625 every score of every district reads GAIN (section 88.3.3). The founder's rule for the tune (use everything that can be used, keep everything healthy, never raise the bill) takes the glide past 0.0625, to the speed that cuts the bill most with no LOSS: that is the next version's change, read on CityLearn first.

The second change is statistical and moves no command. The interval tables written in each tool stopped at a few sample sizes and fell back to 1.96 (the normal quantile) for a number of pairs they did not list, too narrow an interval for a small sample. Every tool now takes the exact quantile from one file, `omnicompass/student_t.py` (the regularized incomplete beta function, checked against the published table to three decimals). Every value the old tables held is unchanged; no published table used a count the old tables lacked, so no published reading moves.

88.5.3 What it does, read on CityLearn before any GitHub run

CityLearn 3.0.2, run locally on its own district files, native (CityLearn's BasicRBC) against Omni-Compass on top of it; every score is CityLearn's own (each over no battery at all), and every row is read by the one key. The omni-v8 column is the wire as it stood.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2022 robustness, 17 buildings (where the omni-v8 row stood)

CityLearn score	native	omni-v8 against native	omni-v9 against native
the bill	1.1008	-8.61% GAIN	-14.22% GAIN
electricity bought	1.1938	-14.66% GAIN	-14.92% GAIN
carbon	1.1494	-11.89% GAIN	-13.11% GAIN
the daily peak	1.1392	-13.95% GAIN	-20.82% GAIN
the highest peak	1.0272	-2.73% GAIN	-2.73% GAIN
the ramping	1.2231	-7.18% GAIN	-12.60% GAIN
daily unevenness	1.0084	-2.90% GAIN	-7.32% GAIN
monthly unevenness	0.9892	+0.11% power given	-2.29% GAIN
net-zero shortfall	1.1312	-6.10% GAIN	-2.76% GAIN

2022 phase 1, 5 buildings (the tuning district)

CityLearn score	native	omni-v8 against native	omni-v9 against native
the bill	1.1158	-10.54% GAIN	-17.63% GAIN
electricity bought	1.2275	-17.77% GAIN	-19.37% GAIN
carbon	1.1804	-14.67% GAIN	-17.15% GAIN
the daily peak	1.1417	-11.88% GAIN	-19.63% GAIN
the highest peak	1.0142	-1.40% GAIN	-5.04% GAIN
the ramping	1.1562	-4.41% GAIN	-10.45% GAIN
daily unevenness	0.9892	-0.29% GAIN	-3.09% GAIN
monthly unevenness	0.9900	+0.26% power given	-1.68% GAIN
net-zero shortfall	1.1369	-6.55% GAIN	-2.85% GAIN

2022 phase 2, 5 buildings

CityLearn score	native	omni-v8 against native	omni-v9 against native
the bill	1.1080	-10.83% GAIN	-17.22% GAIN
electricity bought	1.2124	-16.87% GAIN	-18.44% GAIN
carbon	1.1665	-14.09% GAIN	-16.69% GAIN
the daily peak	1.1124	-12.23% GAIN	-17.74% GAIN
the highest peak	0.9983	+0.17% power given	-11.26% GAIN

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CityLearn score	native	omni-v8 against native	omni-v9 against native
the ramping	1.1231	-3.97% GAIN	-10.99% GAIN
daily unevenness	0.9787	+1.03% power given	-0.09% GAIN
monthly unevenness	0.9828	+0.68% power given	-0.66% GAIN
net-zero shortfall	1.1970	-8.77% GAIN	-4.77% GAIN

2022 phase 3, 7 buildings

CityLearn score	native	omni-v8 against native	omni-v9 against native
the bill	1.0848	-4.17% GAIN	-7.32% GAIN
electricity bought	1.1573	-8.89% GAIN	-6.23% GAIN
carbon	1.1155	-6.65% GAIN	-5.50% GAIN
the daily peak	1.0973	-10.96% GAIN	-16.09% GAIN
the highest peak	0.9942	+0.45% power given	-3.61% GAIN
the ramping	1.1814	+3.37% power given	-1.91% GAIN
daily unevenness	1.0176	-4.83% GAIN	-9.88% GAIN
monthly unevenness	0.9838	-0.55% GAIN	-5.07% GAIN
net-zero shortfall	1.0809	-3.28% GAIN	-0.76% GAIN

2022 phase all, 17 buildings

CityLearn score	native	omni-v8 against native	omni-v9 against native
the bill	1.1007	not run	-14.20% GAIN
electricity bought	1.1941	not run	-14.92% GAIN
carbon	1.1496	not run	-13.11% GAIN
the daily peak	1.1392	not run	-20.77% GAIN
the highest peak	1.0272	not run	-2.73% GAIN
the ramping	1.2223	not run	-12.57% GAIN
daily unevenness	1.0084	not run	-7.29% GAIN
monthly unevenness	0.9892	not run	-2.28% GAIN
net-zero shortfall	1.1315	not run	-2.78% GAIN

Alameda County, 100 buildings

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CityLearn score	native	omni-v8 against native	omni-v9 against native
the bill	1.0000	not run	+0.00% STEADY
electricity bought	2.0205	not run	-6.95% GAIN
carbon	1.0000	not run	+0.00% STEADY
the daily peak	2.4165	not run	-8.92% GAIN
the highest peak	1.2582	not run	-4.40% GAIN
the ramping	1.3828	not run	-10.40% GAIN
daily unevenness	0.7972	not run	-2.75% GAIN
monthly unevenness	0.8053	not run	-1.79% GAIN
net-zero shortfall	9.8346	not run	-0.08% GAIN
energy not served	0.0000	not run	+0.00% STEADY
time uncomfortable	0.7607	not run	+0.00% STEADY

Travis County, 100 buildings

CityLearn score	native	omni-v8 against native	omni-v9 against native
the bill	1.0000	not run	+0.00% STEADY
electricity bought	3.8416	not run	-3.96% GAIN
carbon	1.0000	not run	+0.00% STEADY
the daily peak	3.1689	not run	-5.92% GAIN
the highest peak	1.3727	not run	+0.00% STEADY
the ramping	1.1329	not run	-7.58% GAIN
daily unevenness	0.5655	not run	-2.71% GAIN
monthly unevenness	0.6057	not run	-1.96% GAIN
net-zero shortfall	-4.6732	not run	-0.18% GAIN
energy not served	0.0000	not run	+0.00% STEADY
time uncomfortable	0.8454	not run	+0.00% STEADY

Across the districts above: omni-v8 29 GAIN, 0 STEADY, 7 power given, 0 LOSS (where it was run); omni-v9 58 GAIN, 9 STEADY, 0 power given, 0 LOSS.

The districts: the robustness district, where the omni-v8 row stood and where the cause was found; the tuning district (`citylearn_challenge_2022_phase_1`); the 2022 phases 2, 3 and all (shipped in CityLearn's source repository, not in its package, so the GitHub benchmark does not run them); and the two 100-building

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

neighbourhoods the benchmark counts (Alameda County, Travis County). The seven 2023 districts are unchanged by construction: every battery in them is left native (omni-v8), so their omni arm is native's to the last digit. The shipped runner (`tools/run_citylearn.py`) and the study's copy of it give the same numbers to the last digit on both arms (checked on the robustness district).

Shown, power given. Month by month, the peak hour with Omni-Compass on top rises above native's in some months: in 2 of 12 months in the robustness district, 3 in the tuning district, 1 in each of the 2022 phases 2 and 3, and 2 in phase all (omni-v8: 3, 3, 2 and 4); in Alameda and Travis in none. A peak is power held, so each is power given, beside the bill and the energy it bought; CityLearn's own scores over the year (the highest peak, the daily peak and the monthly unevenness) read GAIN. A guard that keeps Omni-Compass's added charge under the month's top so far cut one of the two months on the robustness district and left every score GAIN; it is a candidate for the next version, in the model gate first.

88.5.4 The model gate

The changes touch no Kubernetes, card or cloud controller, and the model gate's models (`tools/model_gate.py`) do not run the CityLearn wire, so they read the same on both checkouts by construction; the gate was run all the same, the candidate against omni-v8 on seeds 1 to 8, and is recorded in `results/model_gate/V9_VS_V8.md`: every gauge of every model (the cluster at 60 s and at 15 s, conveyance with the quota readable and unreadable, the verdict on noisy work) identical to omni-v8 to the last digit, in every count (it ran with the glide at 0.10, a setting no model reads). The same models show, for omni-v8 and the candidate alike, a pod start wait +9% against native beside 64% more requests served: the same row the real KEDA HTTP run showed (`docs/OMNI_V7.md`, section 6; `docs/STATE_OF_PLAY.md`, Open), the next repair. These are our own models (class S), never a benchmark result.

88.5.5 What runs again on it

Everything, A, B and C, from the moment it lands (`scripts/reruns.txt`, the omni-v9 campaign, released by the conductor a few a tick), with three entries added beside the operator's settings: MySQL, Redis and MongoDB with native exactly as shipped (MySQL's own 128 MB buffer pool; Redis with no memory ceiling and its own `noeviction` rule; MongoDB's own cache, half of the machine's memory less 1 GB). The grid on its own manifest (`results/grid_v9/`); every Azure set from A again. Google Cloud and Amazon stay behind their switches; the GPU runs on Lambda wait for the founder's word and one exact commit. The CityLearn table's three runs on GitHub make the table V9_CITYLEARN.md in `results/live/`; it must read no LOSS in any district, as it did locally.

88.6 Omni v10: the battery glide at the speed that cuts the bill most (the engine that stood on 18 October)

Omni v10 stood on main on 18 October; the same day it was followed by Omni v11 (`docs/OMNI_V11.md`), the engine that stands: omni-v10 with the Kubernetes governor's hold on a reading

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

with no machine in service. This page stays as omni-v10's record, read beside no other engine's. The founder's orders behind it: "Omni-Compass should be tuned to exploit all that can be and keep all healthy and never raise its bill ... whatever one can't be achieved doesn't get achieved. And the tuning stays where it doesn't ever hurt anyone"; and, on 17 October, "a minus doesn't mean a loss and neither does a plus mean a gain if you don't understand what it's against". It is omni-v9 (`docs/OMNI_V9.md`) with one setting changed: the most the CityLearn battery wire's adjustment may change in an hour (the glide) goes from 0.0625 to 0.0875 of the command's range. The CityLearn runner's own receipt now reads every score by the one key, as every table already did.

Its fingerprint is `OMNI_V10.json` (digest `4bf39b0d707784de`, 47 files). Forty-six files are omni-v9's bytes and one differs (the CityLearn runner). The change was proven in our own runs of CityLearn and in the model gate before it was pushed. Every benchmark runs again on it. Every earlier table stands as its own engine's result until its three runs on v10 land, and results from two versions are never pooled. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

#	Change	Where	What it does
1	The battery wire's glide: 0.0875 of the command's range an hour (omni-v9: 0.0625)	`tools/run_citylearn.py`	the batteries follow the coming hour faster, and the bill falls further in every district that has a price
2	The runner's receipt reads every score by the one key: a fall is a GAIN; a rise in energy not served or time uncomfortable is a LOSS; a rise in any other score is power given	`tools/run_citylearn.py`	one key for every output, as the three-run table (`tools/citylearn_abc.py`) already read

88.6.1 Why the glide moved

omni-v9 set the glide at the fastest speed at which no score in any district rose. That rule counted a rise in the ramping or in the distance from net zero against the engine. The one key of 17 October (`tools/readings.py`) does not. The ramping, the peaks, the unevenness and the net-zero distance describe the power the district holds from the grid: they are resources held. A rise in one of them is power given, shown beside what it bought, never a LOSS. Only what the occupants miss is a LOSS: energy not served and time uncomfortable.

Read by that key, the ladder omni-v9 ran shows that no glide reads a LOSS anywhere, and that the bill keeps falling up to 0.0875. At 0.10 it falls less in both districts that set the edge:

District	Score	0.05	0.0625 (omni-v9)	0.075	0.0875 (omni-v10)	0.10
phase 2	the bill	-14.12% GAIN	-17.22% GAIN	-18.75% GAIN	-19.17% GAIN	-19.03% GAIN

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

District	Score	0.05	0.0625 (omni-v9)	0.075	0.0875 (omni-v10)	0.10
phase 2	daily unevenness	+0.25% power given	-0.09% GAIN	-0.67% GAIN	-0.94% GAIN	-0.91% GAIN
phase 2	net-zero shortfall	-7.53% GAIN	-4.77% GAIN	-3.38% GAIN	-2.54% GAIN	-1.99% GAIN
phase 3	the bill	-6.60% GAIN	-7.32% GAIN	-8.10% GAIN	-8.43% GAIN	-8.32% GAIN
phase 3	the ramping	-5.05% GAIN	-1.91% GAIN	+2.32% power given	+6.18% power given	+9.59% power given
phase 3	net-zero shortfall	-2.21% GAIN	-0.76% GAIN	-0.04% STEADY	+0.25% power given	+0.41% power given

So 0.0875 is the glide that cuts the bill most with no LOSS: the founder's rule for the tune. Nothing else in the wire changed. The compass law, its gain (0.5), its response (1 h), its reading (the coming hour, from the same hour of the past seven days), its band (the 10th to the 90th percentile of the past week's draw), the hand-back at 90% of the year, the batteries left native in outage buildings and the water tanks left native are all omni-v9's.

88.6.2 What it does, read on CityLearn before any GitHub run

CityLearn 3.0.2, run locally on its own district files: native (CityLearn's BasicRBC) against Omni-Compass on top of it. Every score is CityLearn's own (each over no battery at all), and every row is read by the one key. The omni-v9 column is the wire as it stood.

2022 robustness, 17 buildings (where the omni-v8 row stood)

CityLearn score	native	omni-v9 against native	omni-v10 against native
the bill	1.1008	-14.22% GAIN	-16.09% GAIN
electricity bought	1.1938	-14.92% GAIN	-13.58% GAIN
carbon	1.1494	-13.11% GAIN	-12.07% GAIN
the daily peak	1.1392	-20.82% GAIN	-16.92% GAIN
the highest peak	1.0272	-2.73% GAIN	-2.73% GAIN
the ramping	1.2231	-12.60% GAIN	-9.41% GAIN
daily unevenness	1.0084	-7.32% GAIN	-6.02% GAIN
monthly unevenness	0.9892	-2.29% GAIN	-1.59% GAIN
net-zero shortfall	1.1312	-2.76% GAIN	-0.96% GAIN

2022 phase 1, 5 buildings (the tuning district)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CityLearn score	native	omni-v9 against native	omni-v10 against native
the bill	1.1158	-17.63% GAIN	-20.05% GAIN
electricity bought	1.2275	-19.37% GAIN	-18.23% GAIN
carbon	1.1804	-17.15% GAIN	-16.52% GAIN
the daily peak	1.1417	-19.63% GAIN	-17.06% GAIN
the highest peak	1.0142	-5.04% GAIN	-5.04% GAIN
the ramping	1.1562	-10.45% GAIN	-8.76% GAIN
daily unevenness	0.9892	-3.09% GAIN	-2.57% GAIN
monthly unevenness	0.9900	-1.68% GAIN	-1.09% GAIN
net-zero shortfall	1.1369	-2.85% GAIN	-1.22% GAIN

2022 phase 2, 5 buildings

CityLearn score	native	omni-v9 against native	omni-v10 against native
the bill	1.1080	-17.22% GAIN	-19.17% GAIN
electricity bought	1.2124	-18.44% GAIN	-17.80% GAIN
carbon	1.1665	-16.69% GAIN	-16.31% GAIN
the daily peak	1.1124	-17.74% GAIN	-15.67% GAIN
the highest peak	0.9983	-11.26% GAIN	-8.30% GAIN
the ramping	1.1231	-10.99% GAIN	-10.67% GAIN
daily unevenness	0.9787	-0.09% GAIN	-0.94% GAIN
monthly unevenness	0.9828	-0.66% GAIN	-0.52% GAIN
net-zero shortfall	1.1970	-4.77% GAIN	-2.54% GAIN

2022 phase 3, 7 buildings

CityLearn score	native	omni-v9 against native	omni-v10 against native
the bill	1.0848	-7.32% GAIN	-8.43% GAIN
electricity bought	1.1573	-6.23% GAIN	-4.70% GAIN
carbon	1.1155	-5.50% GAIN	-4.05% GAIN
the daily peak	1.0973	-16.09% GAIN	-12.57% GAIN
the highest peak	0.9942	-3.61% GAIN	-3.61% GAIN
the ramping	1.1814	-1.91% GAIN	+6.18% power given

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CityLearn score	native	omni-v9 against native	omni-v10 against native
daily unevenness	1.0176	-9.88% GAIN	-7.76% GAIN
monthly unevenness	0.9838	-5.07% GAIN	-4.74% GAIN
net-zero shortfall	1.0809	-0.76% GAIN	+0.25% power given

2022 phase all, 17 buildings

CityLearn score	native	omni-v9 against native	omni-v10 against native
the bill	1.1007	-14.20% GAIN	-16.08% GAIN
electricity bought	1.1941	-14.92% GAIN	-13.58% GAIN
carbon	1.1496	-13.11% GAIN	-12.07% GAIN
the daily peak	1.1392	-20.77% GAIN	-16.92% GAIN
the highest peak	1.0272	-2.73% GAIN	-2.73% GAIN
the ramping	1.2223	-12.57% GAIN	-9.33% GAIN
daily unevenness	1.0084	-7.29% GAIN	-6.02% GAIN
monthly unevenness	0.9892	-2.28% GAIN	-1.58% GAIN
net-zero shortfall	1.1315	-2.78% GAIN	-0.98% GAIN

Across the districts above: omni-v9 45 GAIN, 0 STEADY, 0 power given, 0 LOSS; omni-v10 43 GAIN, 0 STEADY, 2 power given, 0 LOSS.

Alameda County and Travis County (100 buildings each) were still running locally when this version was pushed; this machine runs each neighbourhood for hours, and their omni-v9 runs read no LOSS (`docs/OMNI_V9.md`, section 3). No score in them that can read a LOSS moves with the glide: they carry no price (the bill is 1.0 in both arms, STEADY by construction), no outage (energy not served is 0 in both arms) and no comfort the electric batteries touch (time uncomfortable +0.00% on omni-v9). Every other score there is a resource spent or held, where a rise is power given. Their three runs on GitHub, with every other district, make the omni-v10 CityLearn table; the local runs are added here when they finish.

What the faster glide trades. In every district with a price, the bill falls 1.1 to 2.4 points further than at omni-v9. Most other scores fall a little less than at omni-v9 and still read GAIN: the electricity bought, the carbon, the daily peak, the ramping, the unevenness and the net-zero distance. The highest peak holds where omni-v9 left it, except in the 2022 phase 2 (−8.30% against −11.26%), and that district's daily unevenness falls further (−0.94% against −0.09%). Two rows of the 2022 phase 3 rose and are power given: the ramping +6.18% and the net-zero shortfall +0.25%. A little more electricity is bought than at omni-v9 while the bill falls further, so each unit bought costs less on average. None of these five districts carries the occupants' own scores (energy not served, time uncomfortable); in the 2023 districts they are native's to the last digit, and in Alameda and Travis they read STEADY on omni-v9.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Shown, power given. In some months the peak hour with Omni-Compass on top rises above native's. With omni-v10 that happens in 3 months in the robustness district (omni-v9: 2), 3 in the tuning district (3), 1 in the 2022 phase 2 (1), 1 in the 2022 phase 3 (1) and 3 in phase all (2). By day it happens on 23, 25, 21, 56 and 23 days of the year (omni-v9: 16, 28, 15, 43 and 16). The two 100-building neighbourhoods were still running locally when this version was pushed (on omni-v9 neither showed a month whose peak rose above native's). A peak is power held, so each of these is power given. CityLearn's own scores over the year (the highest peak, the daily peak and the monthly unevenness) read GAIN in every one of the five districts.

The districts: the robustness district (`citylearn_challenge_2022_phase_all_robustness`); the tuning district (`citylearn_challenge_2022_phase_1`); the 2022 phases 2, 3 and all, which CityLearn ships in its source repository but not in its package, so the GitHub benchmark does not run them; and the two 100-building neighbourhoods the benchmark counts (Alameda County and Travis County). The seven 2023 districts are unchanged by construction. Every battery in them is left native (omni-v8), so their omni arm is native's to the last digit; this was checked with the v10 runner on `citylearn_challenge_2023_phase_2_local_evaluation`, where all eleven scores are identical.

88.6.3 Refused: the outage buildings' batteries outside the outage hours

One more change was tried before this version was set, at the same glide. The batteries in buildings that carry an outage would be steered outside the outage hours and handed back to native through each outage. Six of the seven 2023 districts were run:

2023 district	the bill	energy not served	time uncomfortable
phase 2, local evaluation	+0.19% power given	-2.68% GAIN	-0.12% GAIN
phase 2, online evaluation 1	-0.09% GAIN	**+0.69% LOSS**	-0.04% GAIN
phase 2, online evaluation 2	-0.11% GAIN	-0.15% GAIN	+0.00% STEADY
phase 2, online evaluation 3	-0.09% GAIN	-1.44% GAIN	-0.06% GAIN
phase 3, district 1	-0.25% GAIN	-2.47% GAIN	-0.04% GAIN
phase 3, district 2	-0.37% GAIN	-0.38% GAIN	-0.03% GAIN

In the first online evaluation, more energy went unserved. That is a miss, a LOSS. In the local evaluation, the bill rose. One LOSS is enough to refuse the change, and the founder's rule never raises the bill, so the seventh district was not needed. The batteries in outage buildings stay native all year, as in omni-v8 and omni-v9.

88.6.4 The model gate

The change touches no Kubernetes, card or cloud controller, and the model gate's models (`tools/model_gate.py`) do not run the CityLearn wire, so they read the same on both checkouts by construction. The gate was run all the same, the candidate against omni-v9 on seeds 1 to 8, and is recorded in `results/model_gate/V10_VS_V9.md`. Every gauge of every model reads identical to omni-v9 to the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

last digit (26 of 26: the cluster at 60 s and at 15 s, and conveyance with the quota readable and unreadable), and so do every count and the verdict on noisy work. These are our own models (class S), never a benchmark result.

88.6.5 What runs again on it

Everything runs again, A, B and C, from the moment v10 lands (`scripts/reruns.txt`, the omni-v10 campaign, released by the conductor a few at a time each tick). The grid runs on its own manifest (`results/grid_v10/`), and every Azure set starts again from A. Google Cloud and Amazon, switched on by the founder on 18 October with their secrets in, run the same four sets from A, one at a time in each machine-family queue. The GPU runs on Lambda wait for the founder's word and one exact commit. The three CityLearn runs on GitHub make the omni-v10 CityLearn table in `results/live/`. It must read no LOSS in any district, as it did locally in every district read before the push.

88.7 Omni v11: a reading with no machine in service is blind, never an overload (the engine that stood 18 to 19 October)

Omni v11 stood on main from 18 to 19 October; on the night of 19 October Omni v12 (`docs/OMNI_V12.md`) took its place, which is omni-v11 with a refused raise held and the swarm's reflex acting at once. This page stays as omni-v11's record, read beside no other engine's; its 23 three-run tables (`results/live/V11_*.md`) are its own. The founder's order behind it: every fault found is fixed, none set aside. It is omni-v10 (`docs/OMNI_V10.md`) with one guard added to the Kubernetes governor. When the cluster's reading shows no machine in service, the governor holds. It records why, decides nothing from that reading, writes nothing, and keeps the last reading that had machines. Every setting, gain, preset and muscle is omni-v10's, and so is every rule but this guard.

Its fingerprint is `OMNI_V11.json` (digest `745aa53ec8eeb67b`, 47 files). Forty-six files are omni-v10's bytes; one differs, the Kubernetes governor (`omni_controller/controller.py`). The change was proven in a unit test that reproduces the fault and in the model gate before it was pushed. Every benchmark runs again on it. Every earlier table stands as its own engine's result until its three runs on v11 land, and results from two versions are never pooled. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

#	Change	Where	What it does
1	A decision whose reading shows no machine in service holds: it is recorded as held, with its reason, and nothing is decided from it or written	`omni_controller/controller.py`, the decision	the engine is never fed a load measured against no capacity; the decision never divides by zero

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Change	Where	What it does
2	The fast floor check returns at once on the same reading	`omni_controller/controller.py`, the floor check	no machine is added from a blind reading
3	The last reading with machines stays the one the conveyance is bounded by	`omni_controller/controller.py`	a blind moment never lifts the bound on the CPU each pod may be given

88.7.1 The fault, found in a real run

In the omni-v10 CruiseKube test (run 39120444653, repetition 8, the arm with Omni-Compass on top), every worker of the kind cluster left service for two minutes. All six workers sit as containers on one GitHub runner. Eight of the capture's readings show no machine in service, while the service answered every one of its probes (no failed request in any minute of the window). In that reading the machines' capacity was zero. omni-v10's governor fed the engine a load measured against that zero capacity, read as the highest overload its scale allows (a load ratio of 2). It then divided by the capacity when it computed the machine floor. Two of its fifteen decisions and one floor check ended in a division by zero. Each wrote nothing, as the audit shows, and the cluster kept the settings it had. But the engine's own state had been given a false overload. The words of the rig named neither: they said the governor was not running.

omni-v11 reads such a reading as blind. The engine is not stepped, the floor is not computed and no machine is added. The audit says "held: the cluster's reading shows no machine in service; nothing decided, nothing written". When the machines return, the next decision reads them and the governor takes up where it was. `tests/test_blind_reading.py` builds the same cluster: two workers serving, then both out of service in the reading. On omni-v10 both of its tests end in the division by zero; on omni-v11 both pass, with no write and no error.

The rig was repaired beside it (no engine file; `docs/K8S_COMPASS_PREREGISTRATION.md`, amendment of 18 October). A window whose capture shows no machine in service is refused in either arm, because its machine count is wrong. A failed decision is named by its error. The capture records each node's taints and conditions at the moment it sees no machine in service.

88.7.2 The model gate

The candidate was run against omni-v10 on seeds 1 to 8 in our own models (`tools/model_gate.py`; `results/model_gate/V11_VS_V10.md`), on every model: the cluster at a decision every 60 s and every 15 s, the new blink model, conveyance with the quota readable and unreadable, and the verdict on noisy work. This is the first gate in which each arm is proven to import its own checkout's engine. From omni-v9's gate on, the gate had put its own tree first, so both arms ran the candidate; omni-v9 and omni-v10 changed no file a model runs, so their readings hold by construction, and each page carries the correction.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Model	Gauges identical to omni-v10	Decisions that failed (omni-v10 / omni-v11)	Decisions held (omni-v10 / omni-v11)
the cluster, a decision every 60 s	6 of 6	0 / 0	0 / 0
the cluster, a decision every 15 s	6 of 6	0 / 0	0 / 0
the blink: every machine out of service in the reading for 2 minutes every 6 hours	6 of 6	64 / 0	0 / 64
conveyance, the quota readable	8 of 8	(no decision of the governor)	
conveyance, the quota unreadable	8 of 8	(no decision of the governor)	
the verdict on noisy work	5 of 5	(no decision of the governor)	

Every gauge and every count is identical. A decision that failed and one that held both write nothing, so nothing the models measure moves; what moves is the record (a reason where there was an error) and the engine's own state, which a blind reading no longer reaches. Against native, the two engines read the same, and four rows read LOSS in both: the pod start wait under conveyance (+9.0% mean, +10.0% at the 95th percentile), the pending-pod minutes and the response time at a decision every 15 s (+111.7%, +0.33%; no run decides that often), and the alert minutes in the blink model (+2.35%). They are not this change's; they are the next version's work, in the gate first. The first is the test machine, not the governor: the model's pod start time was fitted to the empty-node image unpacking of the single kind host (`docs/POD_START_WAIT.md`, section 8), and the kind rig now pre-pulls the image. These are our own models (class S), never a benchmark result.

88.7.3 What runs again on it

Everything runs again, A, B and C, from the moment v11 lands (`scripts/reruns.txt`, the omni-v11 campaign, released by the conductor a few at a time each tick). The grid runs on its own manifest (`results/grid_v11/`), and every Azure set starts again from A. The omni-v10 runs already started run to their end as omni-v10's record; nothing is cancelled, and no table pools the two. Google Cloud and Amazon run the same four sets from A once their queues are switched back on (Google's key stored whole, Amazon's account on its Paid plan). The GPU runs on Lambda wait for the founder's word and one exact commit.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.8 Omni v12: a refused raise is held, and the swarm's reflex acts at once (the engine that stood 19 to 20 October)

Omni v12 stood on main from 19 to 20 October; on the evening of 20 October Omni v13 (`docs/OMNI_V13.md`) took its place, which is omni-v12 with the conveyance's hand-back. This page stays as omni-v12's record, read beside no other engine's. The founder's order behind it, of the night of 19 October: "Of course I want you to land version 12. I want you to keep fixing and building and straightening and enhancing and benchmarking and defensibility, and every time you do that ... it becomes a new version." It is omni-v11 (`docs/OMNI_V11.md`) with two faults fixed, both found on the omni-v11 runs on 19 October: one in the Kubernetes governor's conveyance, one in the swarm's wire. Every setting, gain, preset and muscle is omni-v11's, and so is every rule but these two.

Its fingerprint is `OMNI_V12.json` (digest `d38b58fbe28939b8`, 47 files). Forty-six files are omni-v11's bytes; one differs, the governor's muscles (`omni_controller/muscles.py`). The swarm's wire (`tools/run_swarm.py`) sits outside the fingerprint's scope, as the data-store wires do (it is versioned by git and tested); its change is declared here as part of omni-v12, and a swarm table counts only on a commit that carries it. Both changes were proven in their tests and in the model gate before they landed (`results/model_gate/V12_VS_V11.md`, `results/model_gate/V12_SWARM_REFLEX.md`). Every benchmark runs again on omni-v12, A, B and C, from the conductor's campaign of 20 October, 00:30 UTC. Every earlier table stands as its own engine's result until its three runs on v12 land, and results from two versions are never pooled. The readings below are our own models (evidence class S), never a benchmark result. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

#	Change	Where	The fault it fixes
1	A raise the cluster's admission refuses is not written again until the pod's limit changes, the raise shrinks, or a wait passes (5 minutes, doubled at each refusal up to an hour)	`omni_controller/muscles.py`, the conveyance	omni-v11 capacity test: admission refused every CPU-limit raise (`exceeded quota: omni-cap`) and the governor wrote it again at every fast check, about 9 refused writes a minute, 970 to 1,000 an hour (`results/STEERING.md`)
2	The swarm's proximity reflex hands back the planner's cruise in the tick it is triggered; its reach grows with the square of the speed (0.5 m at the planner's cruise, as before)	`tools/run_swarm.py`, `Drone.step`	the omni-v11 reflex let the override fall 0.01 a tick, so a drone kept flying fast about a second after it saw a neighbour; on one mission draw outside the counted cells two drones came within 0.083 m (`results/model_gate/V12_SWAR`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Change	Where	The fault it fixes
			`M_REFLEX.md`)

88.8.1 The refused raise

On the kind rig the governor's role cannot read the namespace's quota. As written since omni-v7, an unreadable quota is "no budget known", and the cluster's admission is the authority. In the omni-v11 capacity test admission refused each raise; the refusal was read back and audited, so nothing wrong was ever set, and the table read STEADY. But the governor asked again at every fast check: a controller that repeats a refused command every few seconds loads the API server and fills its own audit. In omni-v12 the refusal is the authority's answer. The same raise is held until something changes: the pod's limit (another writer moved it), a smaller raise (the budget moved), or the wait (5 minutes, then 10, 20, 40, 60, never longer). An accepted raise clears the hold. The audit records each hold and its wait (`held_s`), and `omni_errors_total` and `omni_writes_total` (`tools/omni_exporter.py`) show it to the operator.

The model gate's `convey_refused` model (the quota unreadable and every raise past the operator's limit refused, seeds 1 to 8): omni-v12 wrote **49.75 commands a day against omni-v11's 1,985.5 (−97.5%, GAIN)**, and every other gauge was identical to omni-v11's to the last digit: requests served, pod start wait, p95, alert minutes, host busy, CPU given and pod-seconds. On the convey models with a readable quota and with an unreadable quota that admission accepts, omni-v12 and omni-v11 are identical on every gauge. Tests: `tests/test_refused_raise.py`.

88.8.2 The swarm's reflex

`results/model_gate/V12_SWARM_REFLEX.md` holds the whole reading on gym-pybullet-drones, an outside simulator, on the counted cells and on mission draws the wire was never run on. With the fix, every drone stays at or outside the keep-out on every cell and draw tried, and 49% to 92% of omni-v11's energy cut is kept. That is the price of the reflex acting at once: on the long cell the energy cut falls from −19.8% to −14.5% against native, power given back to the margin between the drones.

88.8.3 The model gate against omni-v11

`results/model_gate/V12_VS_V11.md` (seeds 1 to 8; the cluster models at 60 s and 15 s and with the blink, the three convey models and the verdict model at 20 and 200 pieces of work a decision): omni-v12 reads **identical to omni-v11 on every gauge of every model but one**, the convey model whose every raise is refused, where it writes 97.5% fewer commands (GAIN). No reading of omni-v12 against omni-v11 is a LOSS. The standing engine's own readings against native in these models are omni-v11's and unchanged.

88.8.4 Studied and left out: the verdict's upper end (branch `v12-verdict-study`)

The candidate as first built also carried a stricter verdict: a step kept only when the upper end of every 95% interval of its change in cost is inside the 2% allowance, each phase measuring as many pieces of work as a free step needs to show that, and a trial that could not be read in its time never started. It answers a real fault

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(omni-v11 keeps a step whose interval reaches past the allowance on its estimate alone: on noisy work a step that truly costs 3% is kept about a quarter of the time). On work fast enough to read a step it kept a free step 98% of the time against omni-v11's 66%, and a 3% step 0% against 26%. But in the model cluster set from a real run's regime the governor reads about 80 requests a decision and the rule needs about 4,000 a phase: 1,102 of its give-back trials never started, no step was allowed, and the governor read identical to native, omni-v11's machine-hour and energy cuts gone (shown under each cluster table of `V12_VS_V11.md`). On the data stores (one cost sample a second, 40 to 120 a phase; 432 to 23,430 needed, read from the omni-v11 audits) every knob would stay native. Refused as written; the fault stays on the next version's list (`docs/CHANGE_CONTROL.md`) until a trial design that reads a 2% step in a trial's time is shown in the model gate.

88.8.5 What stays omni-v11's

Every other engine file, rule, gain, preset and muscle; the verdict (`omnicompass/verdict.py`) byte for byte; the control panel, the gauges feed, the alert rules and the steering report, which sit outside the engine and read or drive it only through the interfaces it already has (`docs/OBSERVABILITY.md`). The next version's list (`docs/CHANGE_CONTROL.md`) carries what is still to fix: the power cap's band, the after-kill window widened to 300 s, the Redis harness's brain in its own process, the commands without a shell, every governor wire in the fingerprint, the Kubernetes governor counting a worker an add-on marks while it serves, and the verdict's reading of a step. The kind rig's governor role still cannot read the namespace's quotas: the Operator's Manual, section 5.5, makes that grant optional, and the omni-v12 capacity rerun reads the hold in the real cluster.

88.9 Omni v13: no other muscle's room is conveyed (the engine that stood 20 to 21 October)

Omni v13 stood on main from 20 to 21 October; on 21 October Omni v14 (`docs/OMNI_V14.md`) took its place, which is omni-v13 with a knob for every muscle and the operator's commands run without a shell. This page stays as omni-v13's record, read beside no other engine's. The founder's order behind it, of 19 October, night: every fault found is fixed, and every fix is the next version, told, not asked. It is omni-v12 (`docs/OMNI_V12.md`) with one fault fixed, found on the omni-v12 runs of KubeRay's rig on 20 October: the governor's conveyance, which hands a machine's idle processor to the serving pods on it, kept handing it on a shared host after the host had none left, and the batch muscle beside the web paid. Every setting, gain, preset and muscle is omni-v12's, and so is every rule but this one.

Its fingerprint is `OMNI_V13.json` (digest `0088837333c6f0fc`, 47 files). Forty-six files are omni-v12's bytes; one differs, the governor's muscles (`omni_controller/muscles.py`). The change was proven in its tests (`tests/test_convey_handback.py`) and in the model gate before it landed (`results/model_gate/V13_VS_V12.md`). Every benchmark runs again on omni-v13, A, B and C. Every earlier table stands as its own engine's result until its three runs on v13 land, and results from two versions are never pooled. The readings from the model gate below are our own models (evidence class S),

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

never a benchmark result. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

#	Change	Where	The fault it fixes
1	On a shared host (kind) at its top (busy at or over 75%), while another workload there has room of its own (a pod, not a serving one, whose limit is above its request), every limit given goes back to the operator's; it is conveyed again only under 70%, the band's 5% cushion below the top	`omni_controller/muscles.py`, the conveyance	KubeRay's rig, the organism alone (`results/live/V12_KUBERAY_4W_BARE.md`): the web pods' limits were raised to 3,700m in the queue's first half minute and kept through the saturated queue; the batch queue finished 0.6% to 0.8% later and the web's 99th percentile rose 4% to 13%, LOSS in all three runs
2	Every other pod on a machine is counted by what it may use: its own limit where it has one, its request where it has none	the same	omni-v12 counted requests alone, so a batch pod's room between its request (500m) and its limit (1 core) read as idle and was conveyed

88.9.1 What the KubeRay runs show

The table's two LOSS rows were first put down to the give-back, the governor parking machines during the queue (the batch preregistration, amendment 10). The repetitions' own files say otherwise (`docs/BATCH_QUEUE_PREREGISTRATION.md`, amendment 11). Twenty-nine paired repetitions of the organism alone, A, B and C:

- **The cost sits where the host was saturated.** In 16 repetitions the native queue ran past 760 s: the runner's processor was slower and the batch filled the kind cores (the host's busy share 0.78 through the queue, 95% of readings at or over 0.75). There, each batch pod hashed its work in 63.1 s with Omni-Compass on top against 59.2 s natively, **+7.6%**, and the queue finished 1.1% later. In the other 13, where the host kept room, the same pods read +0.3% and the queue +0.1%.
- **No machine was parked during a saturated queue.** Six workers were in service through the queue in both arms in all 16; the machines Omni-Compass gave back were given back after the queue. The give-back is not the cause.
- **The web's limits were raised at the queue's start and kept.** In 15 of the 16 the conveyance raised the web pods' limits to 3,700m at 33 s (one at 65 s), on a reading of the host's busy share averaged over the 15 s since the governor's previous reading, which caught the dip as the stream's first pods started. From then on the host was at its top, so omni-v7's guard held every further raise, but limits already given stayed, for 92% to 96% of the queue.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The web gained nothing while the queue ran.** Pooled over the 29 repetitions, the web's median while the queue ran was 266 ms with Omni-Compass on top against 253 ms natively, its 99th percentile 3,429 ms against 3,226 ms, and the requests served the same (14,454 against 14,480). After the queue, with the host free, the conveyance did what it is for: the median fell from 114 ms to 69 ms and the 99th percentile from 194 ms to 113 ms.

So the lifted quota bought the web no processor on a host with none to spare (the kernel shares a saturated host by the weights of the requests, and the web's share there sat under its operator's limit), and the batch pods lost work per processor second while it was lifted. One saturated repetition had no raise during its queue and still read +4.0% on hashing: one repetition is not a reading, and the omni-v13 runs read what remains of the governor's other costs on the shared cores.

88.9.2 The rule

A limit above the operator's is idle processor conveyed. On a host at its top there is none, and what the serving pods take there beyond their operator's limit can only come out of what another muscle may use. omni-v13 hands it back. The hold is on a shared host only, where the nodes are containers on one machine and the machine's busy share is read directly; on separate machines each node's own arithmetic bounds the conveyance, now with every other pod counted by its limit, so every muscle on the machine can reach its own limit at the same moment as the serving pods reach theirs. A host where no other workload has room of its own (a web-only rig: system pods with no CPU limit, or a limit equal to the request) keeps omni-v7's rule: raises held at the top, limits given stay, because what the serving pods use there is the host's own idle and no other muscle's room. A host whose busy share cannot be read is blind, never at its top (omni-v11): raises held, nothing handed back. The audit names each hand-back (`"convey": "handed back"`) and each write gives the reason and the host's busy share.

88.9.3 The model gate against omni-v12

`results/model_gate/V13_VS_V12.md` (seeds 1 to 8). A new model, `convey_batch`, sets the batch muscle of KubeRay's rig beside the serving one on the batch rig's host: the kind nodes on 3 of 4 cores and the governor on the fourth, the stream of 24 jobs (one every 30 s, ten pods each, five at a time, 500m requested and a 1-core limit), each pod's work set so the native queue finishes where the saturated runners' did (834 s), the processor shared between cgroups by the weights of their requests, the pods' measured start lag (2.66 s) and the governor's own windowed reading of the host. The batch pods' loss of work while a serving pod's quota is lifted on a host at its top is the saturated repetitions' own (+7.6% hashing time, 0.071 of the work).

Against omni-v12, on the same seeds, **no reading is a LOSS in any model**:

- The three cluster models (a decision every 60 s, every 15 s, and with the machines blinking out of the reading) and the verdict model at 20 and 200 pieces of work a decision: identical to omni-v12 on every gauge, to the last digit. The cloud setting conveys nothing, and the verdict is omni-v11's byte for byte.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- The three web-only convey models (the quota readable and empty, unreadable, and refusing every raise): identical on every gauge. No other workload there has room of its own, so omni-v7's rule holds unchanged.
- `convey_batch`: against omni-v12, the batch queue finishes **1.48% sooner** and the job turnaround is **9.59% shorter** (GAIN), the web's 95th percentile **17.4% lower**, the requests served 0.43% more, the pod start wait 1.46% shorter, the commands written 15% fewer, the CPU given 1.2% less and the host's busy share 0.5% less (GAIN); the pod-seconds +0.84%, power given; the alert minutes identical.

Against native in `convey_batch`, omni-v12 read LOSS on six gauges: the queue +1.50%, the job turnaround +10.7%, the 95th percentile +14.8%, the requests served −2.5%, the pod start wait +1.9% and the alert minutes +9.8%. omni-v13 reads the queue identical to native's, the 95th percentile (−5.3%) and the start wait STEADY, and three LOSS rows. The job turnaround +0.05% (0.04 to 0.17 s a job): the queue's first reading spans the seconds before the batch pods ran, so the web's limits are raised once and handed back 15 s later; the rule that closes it is on the next version's list. The requests served −2.1% and the alert minutes +9.8%: both engines read the same way with no batch at all on this 3-core host, while the real KubeRay runs served 4% more after the queue, so the model's web model is out of its regime there (section 88.3.4). These are our own models, class S.

88.9.4 What stays omni-v12's

Every other engine file, rule, gain, preset and muscle; the verdict (`omnicompass/verdict.py`) byte for byte; the refused-raise hold; the swarm's reflex. The next version's list (`docs/CHANGE_CONTROL.md`) carries what is still to fix, with one item added: the model's web model on the batch rig's 3-core host, where both engines read fewer requests served than native (section 88.3.3) while the real runs read more after the queue, is to be brought to that host's regime from a real run's files.

88.10 Omni v14: a knob for every muscle, the operator's commands without a shell (the engine that stands since 21 October)

Omni v14 was the engine on main on 21 October, from 04:47 UTC until omni-v15 (`docs/OMNI_V15.md`) landed that evening. The founder's orders behind it: knobs on every monitor, "to turn off and on such a mechanism, a muscle, or tweak and turn any muscle" (20 October, evening), and, the next night, "tune the engine". It is omni-v13 (`docs/OMNI_V13.md`) with two changes, each written before it was built and read in the model gate against omni-v13 on the same seeds before it landed: a knob that leaves any one muscle native, and the operator's own commands run without a shell. Every setting, gain, preset and muscle is omni-v13's, and so is every other rule. A third change, to the conveyance's reading on a shared host, was built, tried in the gate and refused there; it is not in omni-v14 (section 88.3.3).

Its fingerprint is `OMNI_V14.json` (digest `7afba44615fcff8f`, 47 files). Forty-five files are omni-v13's bytes; two differ, the Kubernetes governor (`omni_controller/controller.py`) and its muscles (`omni_controller/muscles.py`). Every benchmark runs again on omni-v14, A, B and C. Every earlier table stands as its own engine's result until its three runs on v14 land, and results from two versions are

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

never pooled. The readings from the model gate are our own models (evidence class S), never a benchmark result. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

#	Change	Where	Why
1	`--native MUSCLE[,...]` leaves each named muscle where native runs it: never written, the rest running as before; a name that is not a muscle is refused at the start	the governor and its muscles; the control panel's muscle button	the founder's order of 20 October: a knob for every muscle; and his rule that a muscle Omni-Compass would hurt stays native
2	The operator's own commands (the site power reading, a muscle's read command, a wall meter's `cmd:`) run without a shell unless they hold a pipe, a redirect or a variable	`run_operator_cmd` (`omni_controller/muscles.py`), `tools/wall_meter.py`	the static scan's three high findings (`docs/SECURITY_SCAN.md`); the governor's site power reading no longer reaches a shell at all

88.10.1 The knob

`--native MUSCLE[,MUSCLE...]` (or `OMNI_NATIVE`) names the muscles the governor leaves native. A muscle named there is never written: it stays where native runs it, and every other muscle runs as before. The governor still reads the whole body and decides with all of it in view; only the write of a muscle left native is withheld, and the audit says so (`"held"`, and `"native_muscles"` at the start). A name that is not a muscle is refused at the start, never ignored, so a misspelling can never leave a muscle driven that the operator meant to leave native. With no name given, nothing is left native and the governor runs exactly as omni-v13 does.

Muscle	What is left native
`machines`	the node pool's size (the operator's node-scale command)
`replicas`	the pod autoscaler's target, the pod reflex's replica floor and the replica cap
`cpu`	the serving pods' CPU limits (the conveyance and the pod power cap)
`clock`	the processors' frequency ceiling
`gpu`	the GPU's power limit
`rollout`	the rollout guard (pause and undo)
`batch`	the batch admission and pacing

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Muscle	What is left native
`rightsize`, `coldstart`, `contain`, `cooling`	the pods' rightsizing, the cold-start reserve, the namespaces' containment, the cooling setpoint

The control panel (`tools/omni_control.py`) carries it as one more button, `POST /v1/muscle {"muscle": NAME, "on": true|false}`, role operator, every press audited with its holder and reason, and as a row of On and Native buttons on the console. Like a dial press it passes through native: the governor's own kill file is set, its exit awaited (every setting it wrote restored and read back), and it starts again with the new list. Nothing is changed under a running governor. The monitor wall on the live engine page (`docs/engine/OMNI_COMPASS_LIVE_ENGINE.html`, `#s-wall`) points every row at these buttons; the wall itself never presses one. Tests: `tests/test_native_muscles.py`.

88.10.2 The operator's commands without a shell

A command the operator configured (`--power-cmd`, a muscle's read command in the wiring file, a wall meter's `cmd:`) is split into its words as a shell would split them and run directly when it holds nothing only a shell reads: no pipe, list, redirect, subshell, variable, substitution, glob, home directory, comment or `NAME=value` before the command. One that does runs through the shell as before, so no operator's wiring changes. Tests: `tests/test_operator_cmd.py`. The static scan reads two high findings where it read three (`docs/SECURITY_SCAN.md`): the governor's own call to a shell is gone, and the two that remain are the fallback for a command that needs one.

88.10.3 Refused by the gate: the conveyance's reading on a shared host

The fault it was meant to fix: on the model set from KubeRay's rig (`results/model_gate/V13_VS_V12.md`), omni-v13 raises the web pods' CPU limits once early in the batch queue and hands them back 15 s later, and the batch's job turnaround reads +0.05% against native, a LOSS by the key. The governor reads the host's busy share as the change in its processor counters since its previous reading (every 15 s); the trace of that raise shows it lands on the reading the second job's pods are starting at, before their load is in the reading.

Three rules were written and tried, each against omni-v13 on the same eight seeds:

- **No raise on any reading older than another workload's newest running pod.** Refused: the web's 95th percentile read 6.4% higher than omni-v13's, a LOSS.
- **No raise on the reading that opens another workload's run** (none of its pods with room running at the previous reading). It never fires in the model: identical to omni-v13 on every gauge, because the batch is already running when the second job arrives.
- **That rule, and the starting pods counted:** a pod of another workload still starting ran for none of the interval, so what it may use is added to the reading, and nothing is raised when the sum reaches the 70% cushion. It fires where the fault is. The batch turnaround returns to native's exactly, and the web's queue period is native's second by second. But the web's 95th percentile reads 6.4% higher than omni-v13's, a

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

LOSS. Traced seed by seed: omni-v13's one raise costs the batch pods 15 s of work, which moves when every later batch pod finishes, and with it the seconds the web runs on its last pod; in six of eight seeds that lowered the web's 95th percentile against native. Without the raise the web's queue is native's, and the gain that rode on the batch's delay is gone.

By the gate's rule a LOSS row is not carried into a real run, so none of the three is in omni-v14. They are kept on the branch `v14-convey-study` with their tests and the model's readings. The next version's list (`docs/CHANGE_CONTROL.md`) carries the work that comes first: the model's web model set from a real run's own files (its record already says it is out of its regime on the 3-core batch host), then the third rule read again. The gate found one fault in the model itself and it is fixed for both engines: a batch pod was listed as Running from the moment it was made; a cluster lists it as Pending until its container runs (the rig's own pod start, about 3 s).

88.10.4 The model gate against omni-v13

`results/model_gate/V14_VS_V13.md`: the shipped controller on the model cluster in the cloud setting at 60 and 15 s and with every worker out of service for two minutes, the conveyance on one kind host (readable quota, unreadable quota, a quota that refuses every raise, and the batch muscle of KubeRay's rig beside the serving one), and the verdict on noisy work, eight seeds each, the same seeds for both engines. Every one of the 57 gauges reads identical between omni-v14 and omni-v13, and the verdict allows the same share of give-back steps at every cost: no LOSS. That is what both changes should read there: with no muscle named, the knob leaves nothing native, and no model calls an operator's command. The knob's own behaviour is proven in its tests (a muscle named is never written, an unknown name is refused, every other muscle runs as before), and the commands' in theirs.

88.10.5 What stays omni-v13's

Every other engine file; every rule, gain and guard, the conveyance and its hand-back of omni-v13 among them; the verdict byte for byte. The next version's list (`docs/CHANGE_CONTROL.md`) carries what is still to fix. The GPU governors (`omni_controller/gpu_governor.py`, `gpu_compass.py`) keep their own two levers, the power limit and the clock, and take the per-muscle knob with the version that next changes them.

88.11 Omni v15: the power cap's request through the line's own clear time, every wire in the fingerprint (the engine that stood 21 to 22 October)

Omni v15 was the engine on main from 21 October, evening, until omni-v16 (`docs/OMNI_V16.md`) landed on 22 October, under the founder's standing order that every fault found is fixed and every fix lands as the next version, told, not asked (19 October), and his word of 21 October ("push, push, push on GitHub, update, enhanced engine"). It is omni-v14 (`docs/OMNI_V14.md`) with one change to the brain and one to the fingerprint, each written before it was built and read in the model gate against omni-v14 on the same seeds

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

before it landed: the brain's request for the power cap trims only while the response line has been clean for the governor's own clear time, and every wire that holds a governor's rules is inside the fingerprint. Every gain, guard, preset and muscle is omni-v14's, and so is every other rule. A third item declared with it is not an engine change: the kill test's reading after the kill is 300 s wide (section 88.3.3). A second form of the first change, a dwell with a glide, was built, read in the gate and refused there; it is not in omni-v15 (section 88.3.1).

Its fingerprint is `OMNI_V15.json` (digest `0a53cdd7a558f803`, 55 files). Forty-five of its files are omni-v14's bytes; two differ, the brain (`omnicompass/adapter.py`) and the card governor (`omni_controller/gpu_governor.py`); eight are the wires taken into the fingerprint with this version, their bytes as they stood (`tools/run_redis.py`, `run_kafka.py`, `run_pgbench.py`, `run_sysbench.py`, `run_ycsb.py`, `run_swarm.py`, `run_mujoco.py`, `run_cpu_power.py`). The C++ twin of the brain (`cpp/src/governor.cpp`) carries the same change and is proven equal to the Python again (`tests/test_cpp_governor_parity.py`, on an observation stream that now carries the line's clear flag; `results/SEAL.json`). Every benchmark was to run again on omni-v15, A, B and C; none started before omni-v16 replaced it, because the grid's conductor read omni-v15's commits as no engine (its own copy of the version tool's scope predated the eight wires; fixed on 22 October, `CHANGELOG.md`). Every earlier table stands as its own engine's result until its three runs on v15 land, and results from two versions are never pooled. The readings from the model gate are our own models (evidence class S), never a benchmark result. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

#	Change	Where	Why
1	The brain trims the power cap to demand only while the response line has been clean for the governor's own clear time; a breach, or a line not yet clean, releases the request to the start limit in the same decision	`omnicompass/adapter.py` (the `power` rule, `observe_vector`'s `clean`), its C++ twin `cpp/src/governor.cpp`; the card governor hands the brain its clear flag (`omni_controller/gpu_governor.py`)	on the omni-v11 long run the request switched between the start limit and the trim at 126 of 350 decisions and turned back at 123 of them, at the one threshold on the queue and the state (`docs/CHANGE_CONTROL.md`, "the next version's list")
2	Every wire that holds a governor's rules is in the fingerprint	`tools/omni_version.py` (`SCOPE`; a version declared before them is read without those eight wires and only those)	a rule in a wire outside the fingerprint could change without the version changing (the swarm's reflex did, in omni-v12)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Change	Where	Why
3	Declared, not an engine change: the kill test reads the 300 s after the kill, where it read 120 s	`tools/live_reps.py`, `docs/ROBUSTNESS_PREREGISTRATION.md` ("On omni-v15"), `tests/test_robust.py`	the one tail rise of the omni-v11 kill test came 140 to 190 s after the kill, past the window declared

88.11.1 The request through the clear time

The fault. The brain (`omnicompass/adapter.Governor`) asks for a power cap at every decision: when the queue is under its guard (q < 0.05) and the state's U is at or over the gate (0.85 in the fleet law), the request is the trim to demand, clamp(cap_now × load × (1 + margin), cap_min, 1); otherwise it is the start limit, 1. On the omni-v11 long run (run 39173420146, the robustness long scenario: three paired repetitions of two hours on real Kubernetes, the wandering load) the governor's own audit shows the request switching between 1.0 and 0.695 (the fleet law's floor, which the trim reached at every calm decision) at 126 of its 350 decisions, and turning back the other way at 123 of them (41 of 116, 36 of 118 and 49 of 116 in the three repetitions; `results/STEERING.md` reads it as moves and reversals an hour). Each switch was the line breached (the controller folds the response-time pressure into the queue reading) and the next decision calm again. On Kubernetes the power wire is off, so nothing was written; but the card governor reads the same request for the GPU's power limit, and a request that hammers at one threshold is not the compass law's way (a band with cushions, smooth, not hammering).

The rule. The Kubernetes governor already declares the reflex: while the response line is breached, and for its clear time after (`--slo-clear-s`, 180 s), Omni-Compass may not pack replicas tighter than the HPA's target and may not cap power. It handed that flag to its muscles and not to the brain, whose request kept trimming the moment one decision read calm. The card governor applied the same gate outside the brain (`cap = requested if slo_clean else 1.0`, its `--slo-clear` 3 decisions). omni-v15 puts it inside: the observation vector carries the line's clear flag (`slo_clean`; a caller that senses no line is clean), and the trim is asked for only while it is set. A breach, or a line not yet clean for the clear time, releases the request to the start limit in the same decision; the spread under backlog at the power envelope and the backlog bound are untouched. No constant is added to the law: the clear time is each governor's own, as it already declared it.

Read on the run's own decisions. The three audits' decisions (their queue reading, state U, clear flag and the request recorded) are kept as a fixture (`tests/fixtures/robust_long_39173420146_decisions.json`, the source files' SHA-256 inside). The recorded request is omni-v14's rule on the recorded queue and state, decision for decision. On the same decisions, read with the recorded clear flag, omni-v15's rule leaves 35 moves of 126, 32 reversals of 123, and 28% of the travel (`tests/test_cap_band.py`). A fault's fix is read first where the fault was found.

Refused by the gate. A second form was written first: a dwell of three calm decisions and a glide of 0.05 a decision toward the trim, the release at once. On the run's own decisions it reads 67 moves and 33 reversals;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

but in the model gate, where the model cluster's calm stretches are long, the glide turned each trim into seven small moves (the request's moves in a day 149 against omni-v14's 38 on one seed, the travel the same), and on the model card it wrote the limit 25 times against 16 for the same travel and 0.17% more energy. More writes of smaller size is not the fix the fault asked for; by the gate's rule it is not carried. The clear-time gate alone is in omni-v15.

The card. With the flag inside the brain the card governor's own gate becomes a second reading of the same rule: the model card reads omni-v15 identical to omni-v14 on every gauge (section 88.3.4), as it should, and the GPU's run on Lambda starts from `gpu-omni-v15`, one exact commit.

88.11.2 Every wire in the fingerprint

`tools/omni_version.py` hashes every file whose bytes decide what Omni-Compass does. Up to omni-v14 the scope held the brain, the live governors, the realms and three outside-simulator wires (`tools/run_kil.py`, `run_citylearn.py`, `run_pandapower.py`); the data stores' wires (`run_redis.py`, `run_kafka.py`, `run_pgbench.py`, `run_sysbench.py`, `run_ycsb.py`) and `run_swarm.py`, `run_mujoco.py` and `run_cpu_power.py` were versioned by git and tested but did not move the digest, and one of them carried a fault fixed in omni-v12 without the digest moving (`docs/CHANGE_CONTROL.md`, "Gaps this page names"). From omni-v15 every one of them is in the scope: a change to any rule changes the version. A version declared before them is read without those eight wires and only those: a commit carrying omni-v14's 47 files byte for byte reads omni-v14 whatever those eight wires hold, because its fingerprint never covered them, while any other file omni-v14's list does not name (a new rule, a new module) still makes the tree a different engine (`tests/test_omni_version.py`); and the tool prints a version's own digest and file count as its fingerprint records them.

88.11.3 The kill test's reading: 300 s after the kill

Declared before omni-v15's runs, as `docs/ROBUSTNESS_PREREGISTRATION.md` ("On omni-v11") proposed: the kill scenario's paired gauges, the time over the line and the failed requests after the kill, are read over the 300 s after the kill mark, where they were read over 120 s (`tools/live_reps.py`, `ROBUST_AFTER_KILL_S`). The one tail rise of the omni-v11 kill test came 140 to 190 s after the kill, past the old window. The omni-v11 kill table stands as read then; omni-v15's kill runs are read over 300 s from their first. The rig is unchanged (the probe runs through the whole 900 s window).

88.11.4 The model gate against omni-v14

`results/model_gate/V15_VS_V14.md`: the shipped controller on the model cluster in the cloud setting at 60 and 15 s and with every worker out of service for two minutes, the conveyance on one kind host (readable quota, unreadable quota, a quota that refuses every raise, and the batch muscle of KubeRay's rig beside the serving one), the verdict on noisy work, and, new with this version, the GPU governor on the modelled card (the card whose voltage falls with its clock and the card near its voltage floor), eight seeds each, the same seeds for both engines. Every service gauge of every model reads identical between omni-v15 and

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

omni-v14: machine-hours, energy, pending-pod minutes, the HPA's shortfall, the 95th percentile and the alert minutes on the cluster models; requests served, start waits, the 95th percentile, host busy, limits given, pod seconds and writes on the conveyance models; the batch queue and turnaround; requests served, the 95th and 99th percentiles, energy, peak temperature and the limit's writes, moves, reversals and travel on both card models (the card governor applied the same gate outside the brain, so the brain's gate inside changes nothing there: 17 writes, 16 moves and 15 reversals a run on the card whose voltage falls with its clock, 35, 34 and 33 on the card near its floor, the same in both engines); and the verdict allows the same share of give-back steps at every cost. The request's own wear is the one place the two engines differ, and it reads STEADY: on the model cluster, where the response line is breached in about 27 of a day's 1,440 minutes and the calm stretches between are long, the request moved 67.3 times a day against 64.9 (+3.7%, STEADY), turned back 66.1 times against 63.8 (+3.7%, STEADY) and travelled 20.4 against 19.6 (+3.9%, STEADY) at a decision every 60 s; at a decision every 15 s 98.8 moves against 100.1 (−1.4%, STEADY), 96.1 reversals against 97.5 (−1.4%, STEADY), 29.3 travel against 29.7 (−1.4%, STEADY); with the blink, +3.0%, +2.9% and +3.0%, STEADY. The small rise at 60 s is the gate doing what the governor declares: omni-v14 kept trimming while the line was breached by less than the queue guard's 5% (a pressure between 0 and 0.05), and omni-v15 releases there too, so in a regime of rare, light breaches it releases a little more often. The fault was found in the opposite regime, the long run's clustered breaches (126 switches in 350 decisions), where the same gate leaves 35 (section 88.3.1); the cluster model's regime is the cloud setting's, set from a real run's own files, not the long run's, and the next long run on omni-v15 reads the request's moves and reversals an hour in `results/STEERING.md`. No LOSS anywhere: the change lands.

88.11.5 What stays omni-v14's

Every other engine file, byte for byte; every gain, guard and preset; the conveyance and its hand-back; the verdict. The next version's list (`docs/CHANGE_CONTROL.md`) carries what is still to fix, first among them the Redis harness's brain in its own process (its model, `tools/redis_brain_probe.py`, is read before the harness changes, with the version that changes it) and the model's web model set from KubeRay's after-queue files. The GPU governors keep their own two levers; the brain they call now reads the line's clear flag from them.

88.12 Omni v16: every clock of the governor in seconds; the heartbeat at 15 s

Omni v16 (`OMNI_V16.json`, digest `5c5502f12b1c0ce4`, 55 files) stood on main on 22 October, from the morning until omni-v17 landed that evening (`docs/OMNI_V17.md`: the same engine with the heartbeat a dial, the main engine at 60 s, and the cruise wait never under two readings at a slow beat). Two words of the founder's made it. That night ("Go"), on the plan put to him: put every clock of the Kubernetes governor in seconds, read the decision pace at 60, 15, 10 and 5 s in our own models, and run each pace three times on GitHub's own machines (`docs/PACE_PREREGISTRATION.md`). Then, once the models had read: "15 sec all over heart beat fix update code". So the governor's heartbeat, its full decision, comes every 15 s by default, native's own pod autoscaler's pace, where every engine before it decided every 60 s; the fast checks stay every

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

5 s. It is omni-v15 (`docs/OMNI_V15.md`) with the clocks changed and the heartbeat set, and nothing else: every gain, guard, preset and muscle is omni-v15's, and 50 of its 55 files are omni-v15's bytes (the five changed: `omnicompass/adapter.py`, `omnicompass/edge.py`, `omni_controller/controller.py`, `omni_controller/muscles.py`, `omni_controller/capacity.py`; the brain's C++ twin `cpp/src/governor.cpp` with it, sealed). Read in the model gate against omni-v15 on the same seeds before it was pushed (`results/model_gate/V16_VS_V15.md`), and the pace read alone on a plant that moves every 5 s (`results/model_gate/V16_PACE.md`). The readings from the model gate are our own models (evidence class S), never a benchmark result. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

88.12.1 Why

Every engine through omni-v15 took its full decision every 60 s. The number was written into the first harness on 26 September, with no reason recorded, and never measured against another; the engine was then tuned at 60 s and several of its rules were written in decisions, which locked 60 s in. Native's pod autoscaler checks every 15 s, the metrics it reads refresh every 15 s, the Cluster Autoscaler scans every 10 s, KEDA every 30 s. The founder asked for 15, 10 and 5 s on 14 October; six rules moved to seconds on 16 October (omni-v7) and the rest did not, so turning the period also changed what the remaining rules meant. omni-v16 finishes the job: turning the period changes only how often the governor looks. The cushion each muscle keeps is set by its own climb, wobble and reaction time in seconds, and is the same at any pace; the pace sets only how soon trouble is seen (within 15 s at 15 s, within a minute at 60 s), and so the default moves to native's 15 s.

The founder asked, before the default was set, whether a longer period would make a larger cushion and a stronger engine. It would not: a longer period leaves the governor blind for longer between looks (up to two minutes at 120 s, while native checks every 15 s), and to stay as safe while blind longer it would have to hold more back, which is less work done. The cushion has its own setting (each muscle's z of 3.09, 99.9% of steps inside it, and the dial's tune-down), and the period is not it.

88.12.2 What counted in decisions, and what it counts now

Every value below is the one omni-v15 used when a decision comes every 60 s (the reference step; `REF_PERIOD_S`).

Clock	omni-v15 (per decision)	omni-v16 (per second)	Where
the brain's blending of each reading	0.339 of the state a decision	1 − (1 − 0.339)^k a decision, k = period / 60 s: 0.339 a minute	`omnicompass/adapter.py` (`Governor.step`, `period_s`)
the brain's inner time	0.1 of its macro interval a decision	0.1 k a decision	the same

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Clock	omni-v15 (per decision)	omni-v16 (per second)	Where
the brain's dwell and wait before giving a machine back	1 and 5 decisions (the fleet law)	the same seconds: 60 and 300 s	the same
the brain's machines added or removed	up to 8 and 1 a decision	up to 8 and 1 over the last minute's decisions	the same; the C++ twin `cpp/src/governor.cpp` identically
the compass's damping (kd)	critical damping for the decision period	critical damping at the minute, so the law's push and pull per second are the same at any period	`omni_controller/controller.py`
the compass's velocity smoothing	0.3 a decision	0.3 a minute	the same
the edge's window, narrowing, the floor's forgetting, memory, wobble step	20 decisions; 0.05, 0.9995 a decision; one decision's change	20 minutes; 0.05, 0.9995 a minute; a minute's change	`omnicompass/edge.py` (`pace`)
what a breach teaches the edge	0.1 a decision of breach	0.1 a minute of breach	`omni_controller/controller.py`
machines added past the scheduling floor	one step a decision	one step a minute (the release has been one a minute since v7); the floor's machines at once	the same (`ADD_PERIOD_S`)
a command's time to land (proprioception)	the next decision	a minute, every decision reading it until it lands	the same (`LAND_S`)
the host's busy share (kind's shared runner)	since the last decision	over the last minute	`omni_controller/capacity.py` (`span`)
the heat law	0.86 of the heat kept a decision	0.86 a minute	`omni_controller/muscles.py`
held batch jobs admitted; jobs paused or resumed	one a decision	one a minute	the same
a quiet deployment scaled to zero	3 idle decisions	180 idle seconds (`--coldstart-idle-s`)	the same

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Clock	omni-v15 (per decision)	omni-v16 (per second)	Where
the coast toward fewer pods	coast step × period / 60 a raise	the coast step a raise (a raise comes at most once a window)	`omni_controller/controller.py`
the conveyance	every third fast check, never at a decision (so never at a decision every 5 s)	every 15 s, whichever check falls due (`--convey-every-s`)	the same
the loop	each wait began when the step ended (a 60 s decision came every 62 to 67 s on kind)	keeps its own schedule; a missed slot is skipped, never run in a burst	the same (`main`)

Six rules already counted seconds since omni-v7 and stay as they were: the line's clear time, cruise, the strict window, the trial spacing, the wait after a refusal, and a trial's length; so does the release of one machine a minute.

88.12.3 The heartbeat: a full decision every 15 s

The default `--interval` is 15 s and `--floor-interval` 5 s (`omni_controller/controller.py`; before, 60 s and 15 s, every rig and install file setting the fast checks to 5 s). Every place that starts the governor carries it: the rigs (`scripts/kind_bench.sh`, `kind_paired.sh`, `kind_robust.sh`, `kind_full.sh`, `kind_pilot.sh`), the install files (`deploy/install/omni-compass.yaml`, `omni-compass-cloud.yaml`, `deploy/controller.yaml`), the cloud runs (which start it through `kind_bench.sh`), the alert rules' staleness bound (`deploy/prometheus/omni-compass-rules.yaml`: three intervals, 45 s) and the manuals. Each run writes its period beside its files (`omni_interval.txt`), and the receipts read each run at its own (`tools/live_reps.py`; the robustness runs before it at 60 s). The page's controller trace and the modelled traces in the manuals keep the 60 s step they were drawn at, and say so. The levers rig (`scripts/kind_levers.sh`) was left at three, two and three decisions of 5 s when omni-v16 landed: under omni-v14 each of them was a minute of the brain's, under omni-v16 they are 15 s, and the two levers that give back (rightsize, coldstart), which wait for the brain's calm (its contraction authority, about a minute after it starts on an idle reading, `tests/test_pace_clocks.py`), failed three of the rig's 19 checks in the first omni-v16 run (22 October, 13:50 UTC). The rig now runs at the default 15 s for the 3, 2 and 3 minutes its checks were written for, with the idle time before cold start in seconds (120 s), and the levers check runs again, A, B and C (`scripts/reruns.txt`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.12.4 At 60 s it is omni-v15

On the model cluster at a decision every 60 s, omni-v16 makes exactly omni-v15's decisions: 4,578 audit records, none different (seed 1; the gate's eight seeds read every gauge identical, on every model). The brain's C++ twin equals the Python at 60, 15, 10 and 5 s (`tests/test_cpp_governor_parity.py`, 4,320 directives each, no difference), and every twin was sealed again after its parity test passed (`results/SEAL.json`). Two changes can be seen at 60 s on a real cluster and nowhere else: the conveyance comes every 15 s exactly (omni-v15: about 3.7 passes a minute), and the loop keeps its schedule (omni-v15's decisions drifted by the steps' own time). The model gate's other columns, each engine against native, read as they read for omni-v15 (the same rows, the same numbers at 60 s), because nothing but the clocks moved.

At 15 s, where omni-v15's rules counted in decisions ran four times as fast, every gauge reads STEADY against omni-v15, and the machines parked over the eight days fall from 4,212 to 1,782 (omni-v15 at 60 s: 1,464): the pace no longer changes what a rule means. As shipped (the model `cluster_shipped`: omni-v16 at its default of 15 s against omni-v15 at its 60 s, each started without a period, as an install file starts it), every gauge reads STEADY: machine hours +2.4%, energy per core-hour −0.5%, the pods waiting for a place −28.5%, the 95th percentile −0.3%, the alert minutes −0.9%. No LOSS against the engine it replaces.

The brain's lock (`results/LOCK_AMENDMENTS.json`) takes an amendment for `omnicompass/adapter.py`: the stack simulator steps at 60 s, so the held-out seeds replay unchanged (`verify.py`).

88.12.5 The pace in our models (`results/model_gate/V16_PACE.md`)

omni-v16 alone, eight seeds of a day each on a plant that moves every 5 s, at 60, 15, 10 and 5 s, each faster pace paired by seed against 60 s and against native. In the cloud's setting (the tune-down, machines alone), against 60 s, every service reading is STEADY at every pace (at 15 s: pending pod minutes −12.1%, the 95th percentile −0.0%, alert minutes +0.5%, machine hours −0.4%) and energy per core-hour falls 2.6% at 15 s (GAIN); the power cap's request moves more (power given). In the kind benchmark's setting (the compass law at the max, every autoscaler sensed), against 60 s, the alert minutes fall 14.7% at 15 s and 18.4% at 10 and 5 s (GAIN), the autoscaler's shortfall 6% at 10 and 5 s (GAIN), and every other reading is STEADY. Omni's own processor: 1.6 to 1.7 s an hour at 60 s, 2.7 at 15 s, 3.4 at 10 s, 5.5 at 5 s (under a thousandth of one core at 5 s); its reads: 1,650 an hour at 60 s, 2,190 at 15 s, 3,600 at 5 s. No LOSS against 60 s at any pace.

Against native, the rows read as the engine reads them at every pace, the old 60 s default included, and some read LOSS. The pods waiting for a place: in the kind setting about four times native's at every pace (pending pod minutes an hour: native 8.2; 32.0 at 60 s, 33.5 at 15 s, 28.8 and 28.6 at 10 and 5 s), LOSS at every pace; in the cloud's setting about twice (20.9 at 60 s, 18.4 at 15 s), LOSS at 15 and 5 s and STEADY at 60 and 10 s only by the seeds' spread; on the model gate's 15 s plant, +121.6% at 15 s (LOSS) and +209.8% at 60 s (STEADY by the spread). In the cloud's setting at 15 s two small rows read LOSS as well: the 95th percentile, +0.47% on the 5 s plant and +0.36% on the gate's (at 60 s +0.49% and +0.63%, STEADY by the spread), and

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the alert minutes, +2.31% on the 5 s plant (at 60 s +1.85%, STEADY). Against 60 s every one of them reads STEADY, the 15 s point estimates the smaller, so they are the engine's and not the pace's: the faster pace narrows the seeds' spread until the rows clear zero. They are on the next version's list to be root-caused (`docs/CHANGE_CONTROL.md`), beside the blink model's alert minutes (+2.35% against native at 60 s, LOSS, the same in every gate since omni-v13).

88.12.6 The real runs

`docs/PACE_PREREGISTRATION.md`: workflow `benchmark-reps`, five arms in every repetition (native; Omni-Compass on top at its default heartbeat of 15 s; the same deciding every 60, 10 and 5 s), the reruns' steady scenario, three runs of 10 repetitions on GitHub's own machines at no cost, released by the conductor as the campaign's entry `kubernetes decision pace`. They are the real-run reading of the heartbeat the founder set: a LOSS at 15 s there is root-caused and fixed in the next version, as any other.

88.13 Omni v17: the heartbeat a dial; the main engine at 60 s

Omni v17 (`OMNI_V17.json`, digest `7c937d09e4031cc7`, 55 files) stood on main on 23 October, from the afternoon until omni-v18 landed that night (`docs/OMNI_V18.md`). The founder's order of the afternoon before (22 October), once he had read the reads of the cluster a day in the pace study (about 86,000 at 5 s against 39,700 at 60 s): the governor's heartbeat, its full decision, is a dial, "5, 15, 30, 60, 75, 90, 120 ... Heartbeat scaled in its monitors. And you're going to set OmniCompass is main engine", because "who knows if maybe the purpose is somebody wants to set it at 5 seconds"; and before it, "we are going to write the engine to have the 60 ... we have to actually have to be able to show the same 15 second beats". So the main engine takes its full decision every 60 s; any beat can be set (`--interval`, 5 to 120 s and every value between), the fast checks every 5 s at every beat; and every beat is shown beside native with what it buys and what it costs, because an operator may want a fast beat for its reads and accept what it costs, or a slow one, and at each beat the other knobs may want their own combination. It is omni-v16 (`docs/OMNI_V16.md`) with two changes, both in `omni_controller/controller.py` (54 of its 55 files are omni-v16's bytes): the parser's default period moved from 15 to 60 s; and the cruise rule's wait never fewer than two readings, a fault the dial's own ladder found at 90 and 120 s before the version landed (below, "The fault the ladder found"). Every clock of the governor counts seconds since omni-v16, so the dial changes only how often it looks. Read in the model gate against omni-v16 on the same seeds (`results/model_gate/V17_VS_V16.md`), and every beat read beside native and beside 60 s on a plant that moves every 5 s (`results/model_gate/V17_LADDER.md`), before it was pushed. The readings from the model gate are our own models (evidence class S), never a benchmark result. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

88.13.1 Why a dial, and why 60 s for the main engine

Through omni-v15 the governor decided every 60 s, a number written into the first harness on 26 September and tuned around; omni-v16 put every clock in seconds so that the period became a clean setting, and set it at 15 s, native's own pod autoscaler's pace, on the founder's word that morning. The models then showed what

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

each pace buys and costs: a faster beat reads the cluster more often (about 39,700 reads a day at 60 s, 52,500 at 15 s, 86,400 at 5 s), spends more of the governor's own processor (1.6 to 1.7 s an hour at 60 s, 2.7 at 15 s, 5.5 at 5 s, under a thousandth of one core at any beat), and in the kind benchmark's setting sees trouble sooner (the minutes over the line 14.7% fewer at 15 s than at 60 s); against native the same rows read at every pace. No one beat is right for every operator. The founder's order makes the beat the operator's setting, as the tune-down is, and sets the main engine, the one the company ships and benchmarks first, at 60 s, the period every engine through omni-v15 decided at; 15 s, native's own pace, is one setting of the dial, run and shown beside it.

88.13.2 The dial in our own models (`results/model_gate/V17_LADDER.md`)

The engine alone on a plant that moves every 5 s, the fast checks every 5 s at every beat, eight seeds of a day each, the same seeds and plant at every beat: 5, 10, 15 and 60 s from the pace study of omni-v16 (`results/model_gate/V16_PACE.json`, read before omni-v16 landed, each run setting its period on the command line, so it reads the same on omni-v17), and 30, 75, 90 and 120 s run the same way for omni-v17; the native and 60 s runs the two share read the same seed by seed on every modelled gauge to the twelfth digit (the two were run under Python 3.11 and 3.12, whose float sums differ in the last bits of one gauge, the power cap's travel; `tools/beat_ladder.py` refuses a ladder where any shared reading differs by more). The governor's own processor is timed, not modelled, so each beat's is paired only with the 60 s of its own run. Each beat is paired by seed against native and against the main engine's 60 s.

In the cloud's setting (the compass law at the tune-down, the machines alone), each beat's mean over the eight seeds and its word against native:

Beat	Reads a day	Omni's own processor (s an hour)	Pods waiting (min an hour)	95th percentile (ms)	Minutes over the line (a day)	Autoscaler short (min an hour)	Machine-hours (a day)	Energy per core-hour (kWh)
native			8.18	79.94	27.00	1.20	133.08	0.0357
every 5 s	86,412	5.51	16.52 LOSS	80.18 STEADY	27.50 STEADY	1.20 STEADY	129.67 STEADY	0.0344 GAIN
every 10 s	60,924	3.38	15.92 STEADY	80.11 STEADY	27.12 STEADY	1.20 STEADY	133.40 STEADY	0.0346 GAIN
every 15 s	52,545	2.68	18.36 LOSS	80.31 LOSS	27.62 LOSS	1.20 STEADY	131.48 STEADY	0.0341 GAIN
every 30 s	44,009	2.07	18.59 LOSS	80.22 LOSS	27.38 STEADY	1.20 STEADY	132.19 STEADY	0.0347 GAIN

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Beat	Reads a day	Omni's own processor (s an hour)	Pods waiting (min an hour)	95th percentile (ms)	Minutes over the line (a day)	Autoscaler short (min an hour)	Machine-hours (a day)	Energy per core-hour (kWh)
every 60 s (the main engine)	39,691	1.69	20.88 STEADY	80.33 STEADY	27.50 STEADY	1.20 STEADY	131.98 STEADY	0.0350 STEADY
every 75 s	38,928	1.66	19.30 STEADY	80.20 STEADY	27.38 STEADY	1.20 STEADY	131.21 GAIN	0.0350 GAIN
every 90 s	38,396	1.60	18.27 STEADY	80.35 STEADY	27.62 STEADY	1.20 STEADY	131.29 GAIN	0.0350 GAIN
every 120 s	37,415	1.58	14.01 STEADY	80.08 STEADY	27.12 STEADY	1.20 STEADY	130.93 STEADY	0.0352 STEADY

In the kind benchmark's setting (the compass law at the max, every autoscaler sensed):

Beat	Reads a day	Omni's own processor (s an hour)	Pods waiting (min an hour)	95th percentile (ms)	Minutes over the line (a day)	Autoscaler short (min an hour)	Machine-hours (a day)	Energy per core-hour (kWh)
native			8.18	79.94	27.00	1.20	133.08	0.0357
every 5 s	86,412	5.55	28.55 LOSS	76.88 GAIN	19.38 GAIN	1.12 GAIN	149.67 power given	0.0385 power given
every 10 s	60,942	3.43	28.80 LOSS	76.85 GAIN	19.38 GAIN	1.12 GAIN	149.17 power given	0.0384 power given
every 15 s	52,490	2.66	33.51 LOSS	76.93 GAIN	20.25 GAIN	1.14 GAIN	151.78 power given	0.0389 power given
every 30 s	43,869	2.01	34.09 LOSS	76.99 GAIN	21.25 GAIN	1.16 GAIN	151.78 power given	0.0389 power given

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Beat	Reads a day	Omni's own processor (s an hour)	Pods waiting (min an hour)	95th percentile (ms)	Minutes over the line (a day)	Autoscaler short (min an hour)	Machine-hours (a day)	Energy per core-hour (kWh)
every 60 s (the main engine)	39,444	1.58	31.96 LOSS	77.18 GAIN	23.75 GAIN	1.19 STEADY	151.29 power given	0.0388 power given
every 75 s	38,836	1.66	29.15 LOSS	77.17 GAIN	21.75 GAIN	1.19 STEADY	149.50 power given	0.0384 power given
every 90 s	38,254	1.52	28.10 LOSS	77.01 GAIN	22.62 GAIN	1.20 STEADY	148.78 power given	0.0383 power given
every 120 s	37,536	1.47	22.51 LOSS	76.99 GAIN	22.88 GAIN	1.24 STEADY	148.18 power given	0.0382 power given

Each beat against the main engine's 60 s, and every gauge in full, is on the ladder's page.

88.13.3 The fault the ladder found: cruise on one reading at 90 and 120 s

The ladder's first reading of 90 and 120 s (kept on its page, "The first reading of every 90 and 120 s, before the fix") put the kind benchmark's setting in another regime from 75 s down: against 60 s the machine-hours +17.1% and +21.4%, the minutes over the line −71.6% at both, the pods waiting for a place −59.4% and −70.7%, the 95th percentile −15.4% and −21.3%, and the machines parked over the eight days 82 and 46 against 329. A jump that sharp between two neighbouring beats is a rule's count crossing a threshold, not a pace. The cause is the cruise rule (rule 7: work waiting for a place decision after decision puts every machine in service until the line has been empty as long). Its wait is written in seconds, `--cruise-after-s 120`, and v7 converts every such wait to decisions as n = max(1, round(seconds / period)): 2 at 60 and 75 s, but round(1.33) = 1 at 90 s and round(1.0) = 1 at 120 s. With one decision, a single reading with one pod waiting set every machine in service and a single empty reading let them go: the rule no longer asked for work that waits "decision after decision", the words it is written by. The fix (`omni_controller/controller.py`): the cruise wait is never fewer than two readings, `max(2, round(120 / period))`, when it is derived from seconds; a count an operator sets by hand (`--cruise-after`) stands as set. At every beat up to 80 s the count is what it was (24 at 5 s, 8 at 15 s, 4 at 30 s, 2 at 60 and 75 s), so nothing below 80 s moves; at 90 and 120 s it is 2 (`tests/test_pace_clocks.py`, which fails on the unfixed rule). Corrected, the kind benchmark's setting reads at 90 s, against 60 s, the machine-hours down 1.7%, STEADY, the minutes over the line down 4.7%, STEADY, the pods waiting down 12.1%, STEADY, the 95th percentile down 0.2%, STEADY; at 120 s

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the machine-hours down 2.1%, STEADY, the minutes over the line down 3.7%, STEADY, the pods waiting down 29.6%, GAIN; machines parked over the eight days 190 and 119 against 329 at 60 s. No other count of the governor's falls to a single reading between 5 and 120 s: the breach's clear time (180 s) is 2 decisions at 120 s, the strict window (300 s) 2, the trial's phase (1,200 s) 10; the brain's dwell is one step by design and its waits round to the nearest step. In the cloud's setting (the machines alone, at the tune-down) every service, machine-hour and energy reading at 90 and 120 s is the same before and after the fix, seed by seed; what moves is the governor's own reads of the cluster, 1.9% and 1.2% more (cruise pauses the five-second floor reads, and it now pauses less often), and the machines parked on three seed-beats (128 against 126, 148 against 147, 83 against 80).

88.13.4 What the slower main engine costs against 15 s

Set at 60 s, the main engine gives up what 15 s bought in the models, and it is disclosed here as the price of the founder's setting: in the kind benchmark's setting the minutes over the line are 23.75 a day at 60 s against 20.25 at 15 s (+17.3%: read the other way, 15 s against 60 s, −14.7%, GAIN, so 60 s against 15 s is a LOSS by the key), still 12.0% fewer than native's 27.0 (GAIN against native); in the cloud's setting the energy per core-hour is 2.6% more at 60 s than at 15 s (power given). Every other row reads STEADY between the two. The model gate's own models read the change as shipped (`cluster_shipped`, each checkout started without a period, as an install file starts it: omni-v17 at its 60 s against omni-v16 at its 15 s): every gauge reads STEADY: the machine-hours −2.3%, the energy per core-hour +0.5%, pods waiting for a place +39.8%, the autoscaler's shortfall +0.2%, the 95th percentile +0.3%, the minutes over the line +0.9%; no LOSS against omni-v16.

88.13.5 The rows against native that read LOSS (the engine's, at every beat)

Read by the one key, these rows read LOSS against native in our models (every seed's paired interval clear of zero):

- the cloud's setting, every 5 s: pods waiting for a place +101.8%.
- the cloud's setting, every 15 s: pods waiting for a place +124.4%; the 95th percentile +0.5%; the minutes over the line +2.3%.
- the cloud's setting, every 30 s: pods waiting for a place +127.1%; the 95th percentile +0.4%.
- the kind benchmark's setting, every 5 s: pods waiting for a place +248.9%.
- the kind benchmark's setting, every 10 s: pods waiting for a place +251.9%.
- the kind benchmark's setting, every 15 s: pods waiting for a place +309.5%.
- the kind benchmark's setting, every 30 s: pods waiting for a place +316.5%.
- the kind benchmark's setting, every 60 s: pods waiting for a place +290.6%.
- the kind benchmark's setting, every 75 s: pods waiting for a place +256.2%.
- the kind benchmark's setting, every 90 s: pods waiting for a place +243.4%.
- the kind benchmark's setting, every 120 s: pods waiting for a place +175.1%.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

They are the engine's at every beat, the main engine's included, and stay on the next version's list (`docs/CHANGE_CONTROL.md`) to be root-caused, beside the blink model's alert minutes (LOSS against native at 60 s in every gate since omni-v13).

88.13.6 The monitors scale with the beat

- The exporter (`tools/omni_exporter.py`) serves `omni_heartbeat_seconds`: the beat the governor runs at, measured as the median time between its last eleven full decisions, so a skipped slot moves it little (`tests/test_omni_exporter.py`, every beat from 5 to 120 s).
- The governor's Grafana screen (`deploy/grafana/omni-compass.json`, eighteen panels) shows the beat and counts the time since the last decision in beats of that setting (yellow past one and a half, red past three).
- The staleness alert (`OmniGovernorStale`, `deploy/prometheus/omni-compass-rules.yaml`) fires after three beats as measured: 180 s at the main engine's 60 s, 45 s at 15 s, 360 s at 120 s; the other rules count in minutes and hours, not decisions.
- The live engine page carries the dial (the page's section 2d, "The heartbeat dial"): every beat beside native, from the ladder, with the real runs' place; its manual reads it (`docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.md`, section 7.4).
- The control panel (`tools/omni_control.py`) carries the dial as a button: `POST /v1/beat {"seconds": 5 to 120}`, role operator, one console button for each stop (5, 15, 30, 60, 75, 90, 120 s); like a dial press it hands every setting back and starts the governor again at the new beat, and it serves the beat as `omni_control_beat_seconds` (`tests/test_omni_control.py`).
- Every run writes its beat beside its files (`omni_interval.txt`); the receipts read each arm at its own and record it (`tools/live_reps.py`, `"beats"`), and a three-run table names the beat its compass arm ran at (`tools/confirm_abc.py`).

88.13.7 Every place that starts the governor

The parser's default (`--interval 60`, `--floor-interval 5`); the rigs (`scripts/kind_bench.sh` `OMNI_INTERVAL`, `kind_paired.sh`, `kind_robust.sh`, `kind_levers.sh`, `kind_full.sh`, `kind_pilot.sh`); the install files (`deploy/install/omni-compass.yaml`, `omni-compass-cloud.yaml`, `deploy/controller.yaml`); the cloud runs, which start it through `kind_bench.sh`; and all three manuals. The levers rig gives each phase the 3, 2 and 3 minutes its checks were written for, three, two and three decisions at 60 s (`tests/test_pace_clocks.py`).

88.13.8 At 60 s it is omni-v16 at 60 s

omni-v17 changes no gain, preset or muscle, and one rule only at beats slower than 80 s: at any beat up to 80 s set on the command line it makes omni-v16's decisions at that beat (the cruise wait's count is unchanged there), and at 60 s those are omni-v15's (4,578 audit records on the model cluster, none different,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`results/model_gate/V16_VS_V15.md`). The model gate against omni-v16 (every model that sets its own period reads identical to omni-v16 on every gauge and every seed (9 models and the verdict); as shipped, omni-v17 at its 60 s against omni-v16 at its 15 s, every gauge reads STEADY; no LOSS against omni-v16.). The brain's C++ twin takes the period on every call and is unchanged (`results/SEAL.json`); the brain's lock takes no amendment.

88.13.9 The real runs

`docs/PACE_PREREGISTRATION.md`, amendment 3 (written before any omni-v17 run): workflow `benchmark-reps`, eight arms in every repetition on the same runner in an order rotated by repetition (native; Omni-Compass on top at the main engine's 60 s; the same deciding every 5, 15, 30, 75, 90 and 120 s), the reruns' steady scenario, three runs of 10 repetitions on GitHub's own machines at no cost, released by the conductor as the campaign's entry `kubernetes decision pace`. Each beat is read against native and against 60 s by the one key; a row that reads LOSS at any beat against native is named on the dial's page and root-caused like any other. Every benchmark runs again on omni-v17 at 60 s (`scripts/reruns.txt`), and the Kubernetes preregistration takes its amendment of the same day (`docs/K8S_COMPASS_PREREGISTRATION.md`).

88.13.10 Next: each beat's own combination

The founder's words: the engine "can always be tweaked, tuned, and turned with any knob, any button", and "when you do this inside, you might need to click and change and turn certain things to achieve a more optimal state with a different combination of buttons". The per-beat tuning is the next study, in the model first, on development seeds: at each beat, the combination of the other settings that reads GAIN with no LOSS against the main engine's tune at that beat; a combination that does lands as the next version, as that beat's card, never forced.

88.14 Omni v18: Omni direct in the engine; the three arms

Omni v18 (`OMNI_V18.json`, digest `1cb92edb64c4391d`, 62 files) stood on main from 23 October, night, to 25 October, when omni-v19 (`docs/OMNI_V19.md`) landed. The founder's orders of the same evening: "Native is Kubernetes on the muscles. Omni on native is Omni on top of Kubernetes on top of the muscles. And then Omni direct is simply Omni on top of the muscles, no Kubernetes ... they all need to be written in code and toned in ... and push it all"; and, once it was pushed, "It should be not 17 anymore, it should be 18 ... Give me this new live engine wired up now to where you're at".

omni-v18 is omni-v17 (`docs/OMNI_V17.md`) with Omni direct taken into the engine: the seven files of Omni-Compass's own orchestrator that hold its rules are fingerprinted like every other wire, so no rule of Omni direct can change without the version changing (the principle omni-v15 set for the eight wires it took in). 55 of its 62 files are omni-v17's bytes; the seven new ones are `omni_orchestrator/__init__.py`, `agent.py` (the machine's hand), `brain.py` (the brain), `cgroup.py` (the walls), `control.py` (the control plane), `front.py` (the front) and `scheduler.py` (the room rule). The showing's workload and

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

clients (`omni_orchestrator/workload.py`, `load.py`) are the harness and stay outside. A version declared before omni-v18 is read without the seven (`tools/omni_version.py`, `ORCHESTRATOR_WIRES`), so every earlier commit keeps the version it ran on.

88.14.1 The three arms

Every organism the orchestrator runs is shown three ways, all three in one showing and every pair read by the one key, paired by step (`tools/orchestrator_show.py`, `.github/workflows/orchestrator.yml`, `docs/OMNI_ORCHESTRATOR.md`):

arm	what moves the muscles	brain
native	Kubernetes' published autoscaling rule, written by us, every machine on, units spread	`yardstick`
Omni on native	that rule's units, never fewer; Omni's reflex and fail-up above it; Omni's conveyance, busy room rule, packing and machines on top; no fold, no verdict (the engine's strict form, as on the kind rigs)	`omni_native`
Omni direct	Omni alone on the muscles, no Kubernetes: everything Omni on native does, and the fold below the rule's count under the verdict	`omni`

88.14.2 What Omni direct carries

Every mechanism in Omni direct's brain is the engine's own, so the same gain goes to Omni on top of native:

- **conveyance** (omni-v7, omni-v13): each unit's processor wall set to the room its machine can spare under 75% busy, shared among the service's units there, never under the request, handed back between 70% and 75% and at the top beside another workload's room; journaled, written to the hand, read back from the kernel's `cpu.max`;
- **the reflex**: work waiting in line adds as many units as there are requests waiting, up to the step, in the same beat;
- **the twin**: Kubernetes' rule read on the same readings, each unit's use counted no higher than its request, as Kubernetes would see it; Omni never holds fewer units than the twin unless the verdict allowed the step;
- **the fold under the verdict** (`omnicompass/verdict.py`): a unit fewer than the twin tried on the service's own response times, under the same traffic (a trial whose arrivals move more than 15% is abandoned), kept only where it reads no LOSS within 2%; work waiting hands a step back at once (the immune reaction);
- **the hold on trials the traffic keeps abandoning** (new with omni-v18): a trial the traffic abandons holds the next off for 6 beats, doubled at each abandonment in a row up to the verdict's recheck (24 beats), cleared by a trial judged to its end; a phase counts its step from the higher of the twin it found and the twin now. Found on the first three-arm showing, where every trial was abandoned yet each held a unit below

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the twin while it ran (Omni direct against Omni on native, work LOSS, down 5.2%); fixed before the version landed (`docs/OMNI_ORCHESTRATOR.md`, section 2d);

- **the busy room rule** (omni-v7, `omni_controller/capacity.py`): no unit placed onto a machine at 75% of its measured busy share; the units packed onto the fewest machines, machines parked and woken;
- **the journal and the kill switch**: every change journaled before it is acted on and replayed when the control plane starts again; the kill switch holds the declared state and hands every conveyed wall back.

88.14.3 What it read before it landed

Our own showing on one host, class L, one run of each arm (`docs/OMNI_ORCHESTRATOR.md`, sections 2c and 2d):

- **The three arms, quick form** (six steps of 15 s, after the fix): Omni on native against native, work GAIN, up 8.3%; Omni direct against native, work GAIN, up 9.1%; Omni direct against Omni on native, the 95th percentile GAIN, down 13.2%, and units GAIN, a cut, down 9.5%. No LOSS in any pair; nothing failed or wrong in any arm, the batch job or the drills.
- **Omni direct against native, full form** (18 steps of 25 s): work GAIN, up 4.7%; the 95th percentile GAIN, down 9.9%; units GAIN, a cut, down 10.3%; machines STEADY, down 1.7%; 0 failed and 0 wrong in 20,452 requests.

The model gate against omni-v17 (`results/model_gate/V18_VS_V17.md`): every model reads omni-v17's numbers, because none of omni-v17's 55 files moved; the gate's models do not run the orchestrator, whose own gate is the showing above.

And after it landed, the three runs on GitHub's machines (the full form, A, B and C from one commit) read Omni on native against native work GAIN in all three (up 5.2% to 7.4%) and the 95th percentile GAIN in all three (down 7.5% to 12.1%), Omni direct against native work GAIN in all three (up 5.7% to 9.4%) and the 95th percentile GAIN in all three (down 9.8% to 15.7%), machines STEADY, the units power given or STEADY, and Omni direct against Omni on native STEADY on every gauge; no LOSS in any pair in any run (`results/orchestrator/V18_THREE_ARMS_ABC.md`).

88.14.4 What runs on it

Every benchmark runs again on omni-v18: the conductor's campaign counts from 02:51 UTC on 24 October, when the engine reached main, and only runs whose own commit carries `OMNI_V18.json` (`scripts/dispatch_grid.py`, `scripts/reruns.txt`), and the clouds' sets run on it once each cloud's own $0 check passes. In the clouds nothing changes: Omni on top of the cloud's Kubernetes at the tune-down, on the machines alone (the order of 14 October). Omni direct runs on GitHub's own machines, three runs from one commit (`orchestrator.yml`). Omni direct is ready on a rented machine in each cloud and runs there only on the founder's word: the test `direct` on the one-machine buttons (`big-organism-detached.yml` on Azure, `cloud-organism.yml` on Google and Amazon; `scripts/organism_detached.sh`) rents one 8-processor machine, ships it the exact commit, runs the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

same full form `reps` times one after another, brings every run's files back and gives the machine back; a dry run (the default) shows the price and rents nothing. The GPU runs on Lambda start from `gpu-omni-v18`, one exact commit.

88.14.5 What it does not claim

The three-run reading is the full form on GitHub's machines from one commit, on one host each; no cloud machine has run it yet. The comparison of Omni direct against Kubernetes itself on the kind rig is the next step. The trade between muscles judged on the organism stays the next version's work, through the model gate. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

88.15 Omni v19: Omni-Compass by itself, a pod muscle of its own

Omni v19 (`OMNI_V19.json`, digest `36ee67284c035235`, 62 files) lands on 24 October, 16:54 UTC, under the founder's order of 23 October that every organism is shown three ways: native; Omni-Compass and native; and Omni-Compass by itself, "running direct muscles" with native's own controller of that muscle switched off ("there's native, there's Omni and native, and then there's Omni asked. It is omni-v18 (`docs/OMNI_V18.md`: Omni direct in the fingerprint) with one muscle added to the Kubernetes governor, in `omni_controller/controller.py`, and the models' arm that runs it, in `realms/harness.py` and `realms/plants.py`; 59 of its 62 files are omni-v18's bytes. Nothing moves unless the operator names a workload to the muscle: with no workload named the engine is omni-v18, decision for decision (the model gate below). Preregistered before any run (`docs/SOLO_PREREGISTRATION.md`). Read in the model gate against omni-v18 on the same seeds (`results/model_gate/V19_VS_V18.md`), by itself against omni-v18 by itself arm by arm (`results/model_gate/SOLO_V19_VS_V18.md`) and by itself beside native and beside Omni-Compass and native (`results/model_gate/SOLO_V19.md`) before it was committed. The readings are our own models (evidence class S), never a benchmark result. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

88.15.1 Why it is omni-v19: one name, two engines

This change was first built on the live engine's branch and landed there on 24 October, 01:30 UTC, under the name omni-v18 (digest `22486290b19349eb`, 55 files, on top of omni-v17), while main's omni-v18 (Omni direct in the fingerprint, digest `1cb92edb64c4391d`, 62 files) had stood on main since 23 October, night. One version number names one engine, and no result is read across engines, so the two are kept apart: the branch's omni-v18 never stood on main, and nothing it recorded is read as main's omni-v18. Its change, the three files above and nothing else in the engine, is taken onto main's omni-v18 here, gated again against main's omni-v18 on the same seeds, and lands as omni-v19. Every rule is the branch's; the code's own comments, which named the branch's numbers (omni-v17, omni-v18), now name omni-v19, so the three files differ from the branch's bytes in those comments only. Main's omni-v18 files and records stay as they were.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.15.2 The pod muscle of its own

On every engine before it, the Kubernetes governor moved a workload's pods only through the pod autoscaler's own object (its range and its target): with the pod autoscaler switched off it had no pod muscle, so "by itself" held every workload at the operator's count. omni-v19 gives it one (`--own-replicas namespace/name,...`, or `OMNI_OWN_REPLICAS`; `_own_pods`). For each workload the operator names and no autoscaler governs, the governor sets the replica count itself, through the workload's own scale, by strict C's rule: replicas = ⌈current × measured busy share / target⌉, up at once, down only to the highest recommendation of the last 300 s, inside a tolerance of a tenth. The target is the one the compass moves for an autoscaler: the operator's own (the workload's `omnicompass.io/target-utilization` annotation, else `--own-target`), down to 60% of it, the most pods the cover allows past the wall. Never below the operator's own count at the max, never above the range's top (`omnicompass.io/replica-range`), never below one pod; the count it shipped is recorded once on the workload and handed back by the reset. A workload an autoscaler governs keeps its autoscaler: the muscle writes nothing there. `--native replicas` leaves it native and writes nothing; a security hold allows no expansion. The muscle keeps the pace of the controller it stands in for: a reconciliation every 15 s on the fast checks (`--own-every-s`, the pod autoscaler's sync), at the target the last decision set; the compass moves the target only in a decision, at any beat of the dial.

In the models (`realms/harness.py`, `SOLO`; `realms/plants.py`, `native_off`), the arm by itself is the max with every pool whose own count is the muscle's knob run with its autoscaler switched off, and its twin (the brain's sense of native) too. A power cap, an admission limit and every other plant's knob are a setting native already holds still, or run through native's own loop as its actuator: there by itself is the same arm as Omni-Compass and native.

88.15.3 The fault found before it landed

On the branch, rebuilt on omni-v17, whose main engine decides every 60 s (as main's omni-v18 does), the first candidate moved its pods only at a decision, once a minute. By itself in the model it then read the minutes over the response line LOSS against native (34.8 minutes a day against native's 26.4), the 95th percentile still a GAIN. The pod autoscaler it stands in for syncs every 15 s. The muscle now reconciles every 15 s between decisions; its window of fewer pods counts those reconciliations (the strict window's 300 s at any beat). Two unit tests hold it (`tests/test_own_replicas.py`, 11 in all).

88.15.4 The model gate against omni-v18 (`results/model_gate/V19_VS_V18.md`)

Every model, 8 seeds of a day: the cluster in the cloud's setting at 60 and 15 s, the blink, the shipped default, the four conveyance models, the card and the card near its voltage floor, and the verdict. Every gauge of every model reads **identical** to omni-v18, seed for seed (95 rows, and the verdict's every share; only the governor's own measured processor time, a clock reading, differs from run to run), because no model names a workload to the muscle. Every table is the one the branch's own gate against omni-v17 printed when it landed there, number for number and word for word, as it must be: main's omni-v18 added only Omni direct to omni-v17, and no model runs Omni direct. No LOSS against the engine that stands.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

In the models, the native arm and the max read byte-identical on both engines (the preregistration's step 3): 806 readings on the development seeds 7101 and 7102, 67 muscles each under its own load, the climb and the line, and one body of 60 muscles, every reading's hash the same on omni-v18 and omni-v19. The switch acts only in the arm by itself.

88.15.5 The rows against native that read LOSS (the engine's, as on omni-v18)

The same models read against native carry the rows omni-v18 and omni-v17 carry, unchanged, each on the next version's list with its cause: at a decision every 15 s in the cloud's setting the pods waiting for a place LOSS (up 121.6%) and the 95th percentile LOSS (up 0.36%); the alert minutes with the machines blinking out of the reading LOSS (up 2.35%); in the conveyance model the pod start wait LOSS (up 8.99%) and its 95th percentile LOSS (up 10.0%), beside the requests served GAIN (up 64.2%) and the 95th percentile GAIN (down 53.1%); in the batch model the requests served LOSS (down 2.11%), the alert minutes LOSS (up 9.82%) and the job turnaround LOSS (up 0.05%); the card's 95th percentile LOSS (up 2.38%; near its voltage floor up 5.78%), beside its energy GAIN (down 4.72%; down 1.30%). Every row is shown.

88.15.6 By itself, in our own model (`results/model_gate/SOLO_V19.md`, 8 seeds of a day, class S)

On the machines as the kind rig has them (every worker made at the start, no Cluster Autoscaler: 12 machines in every arm):

Gauge	native	Omni-Compass and native	Omni-Compass by itself
machine-hours	288.0	288.0	288.0
pods waiting for a place (pod-minutes an hour)	0.0	0.0	0.0
95th percentile response (ms)	80.0	76.8	64.4
minutes over the response line (a day)	26.4	22.0	9.2

By the one key, Omni-Compass by itself against native: the 95th percentile **GAIN** (down 19.4%: faster; its 95% interval down 12.2% to 26.6%), the minutes over the line **GAIN** (down 64.9%: fewer; down 32.4% to 97.5%), machine-hours, energy per core-hour and pods waiting STEADY (the same machines). Against Omni-Compass and native: the 95th percentile **GAIN** (down 16.0%), the minutes over the line **GAIN** (down 58.0%), the rest STEADY. Omni-Compass and native against native: the 95th percentile **GAIN** (down 4.0%), the minutes over the line **GAIN** (down 16.6%). Its own cost: the governor reads each workload it scales twice at each reconciliation (6,417 reads an hour in the model, twelve workloads, against 660 on top of native), the reads power given.

Against omni-v18 by itself, arm by arm (`results/model_gate/SOLO_V19_VS_V18.md`): native and Omni-Compass and native read identical on both engines, gauge for gauge; by itself, omni-v18 has no pod muscle and holds every workload at the operator's count (its 95th percentile 313,407 ms and 1,412 minutes

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

over the line in the day: the model's queue running away), where omni-v19 reads 64.4 ms and 9.2 minutes: the 95th percentile **GAIN** (down 100.0%), the minutes over the line **GAIN** (down 99.3%), the energy per used core-hour **GAIN** (down 4.2%), machine-hours and pods waiting identical. No LOSS in any arm.

88.15.7 Where it runs

On Kubernetes, the kind rig's `ARM=solo` (`scripts/kind_bench.sh`): the pod autoscaler removed after the deploy, the operator's target and range written on the workload, the workload at its range's floor, Omni-Compass the only controller of its pods; Kubernetes' own scheduler and kubelet stay; the kill switch's check reads the workload back at the count it started at and no record of Omni-Compass's left on it. The capture counts the workload's own replicas where no autoscaler stands (`fleet/capture/kube_capture.sh`). The three-run tables read it against native (`solo`) and against Omni-Compass and native (`solo|compass`) (`tools/confirm_abc.py`, `tools/live_reps.py`). Its runs: `kubernetes by itself` in `scripts/reruns.txt`, the steady load with the three arms in rotation (native, Omni-Compass and native, Omni-Compass by itself), A, B and C on GitHub's own machines, written before the omni-v19 campaign opened. In our models, the organisms runner's third arm (`tools/run_realms_v5.py`): every pool whose own autoscaler can be switched off, read against native and against Omni-Compass and native. Where native is a setting, or its own loop is the actuator, the two Omni-Compass arms are one. No cloud runs by itself without the founder's word (the order of 14 October: nothing in a cloud adds a machine).

88.15.8 Beside it: Omni direct

Main's omni-v18 took Omni direct into the engine: Omni-Compass's own orchestrator (`omni_orchestrator/`, `docs/OMNI_ORCHESTRATOR.md`) runs the organism with no Kubernetes at all, with its own walls, hand, front, room rule, brain and control plane, its seven rule files fingerprinted with every other wire. omni-v19 carries those seven files byte for byte. The two answer the same order from two sides: Omni direct replaces Kubernetes on the muscles (its arms native, Omni on native and Omni direct, `tools/orchestrator_show.py`); Omni-Compass by itself stays on Kubernetes, which still places and runs the pods, and switches off only the controller of the knob (on the kind rig the pod autoscaler), the governor its only controller.

88.15.9 The real runs

Every benchmark runs again on omni-v19 (`scripts/reruns.txt`; the conductor's campaign from 16:54:55 UTC on 24 October, the moment it stands), and `ARM=solo` joins the kind sets beside native and Omni-Compass and native, three runs, read by the one key.

88.15.10 What it does not claim

No real run of the arm by itself has been made yet: every reading on this page is our own model (class S). The trade between muscles judged on the organism stays the next version's work, through the model gate. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.16 Omni v20: the data stores' wires fixed at their causes

Omni v20 stood on main on 25 October until omni-v21 landed that evening (`docs/OMNI_V21.md`).

Omni v20 (`OMNI_V20.json`, digest `1df07f8e36a97985`, 62 files) lands on 25 October, 13:24 UTC, under the founder's words of 25 October ("Fix, fix, fix ... The SYS bench, Kafka, whatever ... has got a problem, you got to fucking fix it ... it can be fixed") and the standing order that every fault found is fixed and lands as the next version, told, not asked. It is omni-v19 (`docs/OMNI_V19.md`) with five of its wires changed: the data stores' runners `tools/run_sysbench.py`, `tools/run_kafka.py`, `tools/run_ycsb.py`, `tools/run_redis.py` and `tools/run_pgbench.py`. 57 of its 62 files are omni-v19's bytes. The faults were found by tracing why the Omni index read +4.32% before the 0.5% key of 24 October and +3.397% after it (`docs/STORE_LOSS_ROOT_CAUSES_2026-10-25.md`): seven data-store rows, each with 0 or 1 of its three runs clear of the noise, that the new key now counts. Three were faults in our own wiring, two were the test machine. The fixes were read in the store gate (`tools/store_gate.py`, `results/model_gate/STORE_GATE.md`) before they were committed. The gate's readings are our own model (evidence class S), never a benchmark result. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

88.16.1 What changed

- **sysbench: the brain reads its own workload.** `status()` counted only `statement/sql/{select,update,insert,delete}`. sysbench sends prepared statements, which those counters never see, so the brain read 4.1 statements a second (the harness's own monitoring queries) while sysbench sent 999 to 4,198. The compass force and the verdict's work sample were the brain's own queries, and it resized the buffer pool blind, 10 to 23 times an arm. Now sysbench connects as its own account (`omni_load`, created by `--setup` with rights on `sbtest` only), and the brain reads that account's statements from the per-user summary table, prepared ones included (`statement/com/Execute`, `statement/sql/execute`, `execute_sql`, and the four plain ones). Every sysbench row, the GAIN rows included, stood on the blind reading and runs again on omni-v20.
- **Kafka: an extra consumer is handed back.** Above the operator's count, the hand-back needed a consumer whose 200 ms poll came back empty, which never happens with 8 round-robin partitions: a third consumer allowed after a load peak stayed 170 of 451 seconds and bought nothing. Now, above the operator's count, calm gives one consumer back while the backlog is within one poll round (50 messages a consumer), 30 s after the last change, because every change pauses the whole group. "Idle" is a full second with nothing consumed, and a consumer starts busy; no trial starts on the seconds before the producer begins (the $2 \to 1 \to 2$ double rebalance at the start of every arm is gone).
- **YCSB: the cache grows back below native.** Below the operator's size, nothing grew the cache on misses; a give-back trial allowed before the working set widened left two arms 64 MB short with misses at 1% or more for 435 of 457 seconds. Now, below native, the cache takes 64 MB a second back while misses are 1% or more, up to native's size, and an open give-back trial is abandoned. "Full" is

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

WiredTiger's own 80%, or evicting in the last second (it was 90%, a level WiredTiger never holds). `serverStatus` is read in full once, then only for the sections the brain reads. mongod's CPU and the harness's are recorded apart, steal apart, beside the host's.

- **Redis: the CPU apart.** redis-server's, the App's, the brain's and the sampler's CPU are recorded apart, steal apart, beside the host's. The knob's rule is unchanged.
- **pgbench: the same state every arm.** Each arm starts from `pgbench -i` and a CHECKPOINT; the give-back waits 30 s after an add-back (5 s before); one aggregate line a second is kept from the per-transaction log. The workflow logs vmstat, iostat and PostgreSQL's checkpoints, lock waits and autovacuum into the run's files, so the runner's write stalls (6.2 s of schedule lag in a native arm far under capacity on the omni-v16 runs) can be told from the engine.
- **The readers.** `tools/omni_index.py`, `tools/ycsb_abc.py` and `tools/redis_abc.py` prefer the stack's own CPU where a run recorded it; older tables read as before (the index stays +3.397% on them).

88.16.2 The gate

The store gate replays every archived omni arm of the omni-v16 three-run tables, second by second, through omni-v19's wires and omni-v20's, each with its own verdict fed the recorded cost samples and its own knob, and reads the pair by the gauge's aim. It is open-loop: it shows what each wire decides on the same seconds, not how the store would have answered.

Wire	Arms	What omni-v20 does against omni-v19
Kafka	27	rebalances in the first 15 s 2.0 → 0, GAIN +100%; writes GAIN +92.9%; seconds below native while short GAIN +100%; seconds above native while calm (light) 37.6 → 6.3, GAIN +83.1% inside the arms' noise; consumers held +0.6%, inside the noise
YCSB	36	seconds below native while short 38.9 → 2.2, GAIN +94.4%; cache held power given +5.2%; writes power given +76.6%
pgbench	26	writes GAIN +21.0%; seconds below native while clients waited GAIN +12.7%; pool size =0.4%, under the floor
Redis	9,728 seconds	the two wires decide differently 0 times; the new CPU figures exist only in new runs
sysbench		the decisions cannot be replayed: the archived files hold only the old reading's counter (4.1 to 4.2 a second against sysbench's 999 to 4,198); a short real run reads the new one

No LOSS on any wire. The YCSB cache rise is a real change of behaviour: the new "full" opens the grow path on seconds that were slow, missing and evicting, which omni-v19 refused as "slow with room to spare". It is power given, and the replay cannot show what the extra memory buys; the real runs on omni-v20 read it. One

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

pgbench arm (select, run C rep 3) is left out of the gate because omni-v19's own replay matched only 32% of its record there; every other arm reproduces its record in 97% or more of the seconds.

88.16.3 What runs again

Every benchmark runs again on omni-v20 (`scripts/reruns.txt`, the conductor's campaign from 13:24 UTC on 25 October), the data stores first in meaning: sysbench, Kafka, YCSB, Redis and pgbench. A fourth pgbench repetition, balancing the arm order against the runner's drift, is written as a proposed amendment in `docs/POSTGRES_PREREGISTRATION.md`, not yet in force. Tests: `tests/test_store_wires.py` (each check fails on omni-v19's runners and passes on omni-v20's) and `tests/test_store_gate.py`.

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

88.17 Omni v21: the immune system knows itself, knob by knob

Omni v21 (`OMNI_V21.json`, digest `b658f05d43c1e602`, 63 files) stood on main from the evening of 25 October to 30 October, when omni-v22 (`docs/OMNI_V22.md`, the data stores' wires at the max) landed on it; its tables stand as its own record. It is omni-v20 with the immune system's self-knowledge, the founder's order of 25 October:

> "OmniCompass's mechanism must know when something is redlining out of its band when OmniCompass is on it. And if OmniCompass couldn't fix it to stop it redlining, then OmniCompass's own mechanism must know that if native is better, then it switches itself to native. If native is not better, well, then it'll stay wherever and redline there, because redlining less with OmniCompass than something else is still a benefit ... against native ... This code and mechanism need to be built within the C++ harness, within the immune system ... The immune system has to be activated in nervous system to propagate."

88.17.1 The rule, in plain words

Term	What it is
Redlining	A knob Omni-Compass holds whose gauge is over its own line (the reading divided by the line is over 1) for three decisions in a row.
Stuck	Redlining, and Omni-Compass's own pushing has not brought it back: the knob is at its own limit, or the reading has not fallen by 5% of where the redline began.
Native is stronger	Native's twin, the same moment under native's own setting, reads lower than Omni-Compass by more than 5%.
Hand back	Stuck and native stronger: the knob goes to native and stays there 10 decisions, the hold doubling for each hand-back in a row (up to 160) and halving after a quiet hold.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	What it is
Stay	Stuck and native not stronger: Omni-Compass keeps the knob and says so in its audit. It redlines less than native would, which is still a benefit against native.
Return	After the hold, Omni-Compass takes the knob back only when its own twin reads lower than native's by the same margin and nothing redlines with it.
The nervous system carries it	An organ every governed knob of which is handed to native has none of Omni-Compass's authority (`native_hold`) until it is returned.

88.17.2 Where it is written

Place	What
`omnicompass/immune.py`	the rule, one knob at a time
`cpp/src/immune.cpp`, `cpp/include/omnicompass/immune.hpp`	its C++ twin, line for line, proven alike to the last bit on 200 seeded scripts (`tests/test_cpp_immune_parity.py`, sealed by `tools/seal.py`)
`omnicompass/nervous_system.py`, `cpp/src/nervous_system.cpp`	`native_hold`: the hand-back carried to the organ's authority, in both twins
`omni_controller/controller.py`	each service's autoscaler target: the response time over its line is the redline; native's twin is the same demand under the operator's target, the replicas native's own rule would ask and the M/M/1 wait they would give

omni-v19's immune rules come with it onto omni-v20: the hand-back on a pod that waits, the immune memory, native's line, immune maturity and the step back (`tests/test_immune_system.py`).

88.17.3 What the model gate read

The model gate against omni-v20 (`results/model_gate/V21_VS_V20.md`, eight seeds, every model, our own models, evidence class S): no LOSS against omni-v20 in any model.

Model	Against omni-v20
the cluster (the cloud's setting), at 60 s, at 15 s, with blinks, as shipped	pods waiting for a place down 12.9% to 13.3%, fewer waiting, GAIN; machine-hours up 0.9% to 1.3% and energy per core-hour up 0.8% to 1.0%, power given; the power cap's moves, reversals and travel down 1.3% to 1.4%, GAIN

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Model	Against omni-v20
the conveyance, its quota cases, the batch beside the service	every gauge the same as omni-v20, STEADY
the card, the card near its floor	every gauge the same as omni-v20, STEADY
the verdict	the same as omni-v20, STEADY

Against native the gate reads the LOSS rows omni-v18, omni-v19 and omni-v20 carry, none caused by this version and each on the next version's list: the pod start wait in the conveyance models (up 9.0% and 10.0%), the batch beside the service (requests served down 2.1%, alert minutes up 9.8%), and the card's 95th and 99th percentiles (up 0.9% to 5.8%).

Every benchmark runs again on omni-v21 from the moment it reaches main; nothing is read across engines.

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

88.18 Omni v22: the data stores' wires at the max, through each store's own account of a miss

Omni v22 (`OMNI_V22.json`, digest `0e23a0bc561b900a`, 65 files) stood on main from 30 October to 31 October, evening, when omni-v23 (`docs/OMNI_V23.md`) landed on it; its runs stand as its own record. It is omni-v21 with the five data stores' wires brought to the max, the founder's order of 30 October:

> "Anything that has a problem is what your job is ... find out why they went one way or the other and tweak it and turn it and tune it to the best possible way for an Omni Compass engine to interact with this as a system governor, convergence control core plane that it is ... bring to a solid A mechanism."

The store wires were written on 9 and 10 October with the brain's verdict judging every move under the resource objective, before the founder's order of 12 October that the max is the default brain, and were never brought to it. On omni-v21 they carried 33 of the 50 LOSS rows (`docs/LOSS_LEDGER.md`); their causes, read from the runs' own files, are on `docs/STORE_ROOT_CAUSES_2026-10-30.md`: the wall abandoned every spend trial and the fail-up was clamped to the allowance those trials never earned, so in 4,426 of the 4,581 seconds Redis stood over its line the brain asked for memory and was refused, and Kafka's hand-back gave back a consumer the load still needed.

88.18.1 The rule, in plain words

Term	What it is
The max on a store's knob	The wire's own rule asks; what it asks goes through unclamped, up to the operator's cover, never below native. The resource it takes (memory, consumers,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	What it is
	servers) is power given, shown beside what it bought.
The store's own account of a miss	MySQL: the performance schema's timer on the data files' reads against the workload's statement timer. MongoDB: WiredTiger's time reading pages from disk for the application's threads against the server's own read latency. Redis: the application's own split of each request into hits and misses. Over the last 10 s, the misses' share of the service's own time.
The gate	Memory is spent only where that share is at least the one key's floor, a half of one percent (`tools/readings.py`): under it the misses are nothing the record could read, so memory could buy nothing it could read, and the knob holds and says why. Where the account is not read this second, nothing is known and nothing is taken.
The host's room	A notch is taken only while the host keeps 5% of its memory available after it (`/proc/meminfo`, the compass's cushion). Unread, nothing is taken.
The working set's floor	A hand-back never goes below the most memory the store held in the last 60 s (with the fullness margin, rounded up to a notch), nor below native: a knob handed back into its working set evicts what the service still asks for and buys it back at the store's price.
Kafka's twin and reflex	Each partition's arrivals (the topic's log-end offsets, once a second) placed on the members by the range assignor; a member's rate learned from the seconds the queue grew; the count goes at once to the smallest one that carries the arrivals and the backlog at nine tenths of a member's rate, never clamped; handed back in one move only to a count that carries the last minute's peak at seven tenths, 30 s after the last change; never below the operator's count; the immune hand-back to native when Omni-Compass's count redlines, stuck, and native's twin reads lower.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	What it is
PostgreSQL's pool	A server added while clients wait is never clamped; the pool is never taken below the pooler's own setting.
The dial	`OMNI_DIAL=save` keeps omni-v21's loop on every wire (the tune-down, where giving the resource back is the aim); every record carries its dial.
The fingerprint	`tools/store_brain.py` (what the three cache wires share) and `tools/knob_verdict.py` (the tune-down's verdict) join it: no rule of the store wires changes without the version changing.
The hand-back lands (the Kubernetes governor, the same version)	Every step of the governor's hand-back (the autoscalers' ranges and targets, the operator's own counts, every muscle's lever, the node pool) is tried 5 times against a transient failure of the cluster's API, 1 s apart doubling to 4 s, each on its own, and what did not land is raised and audited, never left silent. On 30 October (six-kube run 41044680505, repetition 346) one 500 from the API on the autoscaler list, at the reset, ended the whole hand-back on its first read: the conveyed CPU limits (1230m, 1850m) and the replica range (1 to 13) stayed behind and the arm was marked invalid. `omni_controller/controller.py` `_persist`, `omni_controller/muscles.py` `_guard_restore`, `RESTORE_TRIES`; `tests/test_restore_retry.py`.

What was tried and withdrawn: a slope of latency on miss share inside the arms as the price of a miss. It read 0.5 to 1.3 ms a unit of miss share on MySQL while the controlled comparison across pool sizes read none; the host's noise moves the misses and the latency together, and a regression reads that as a price. The root-cause page's first reading of the MySQL and MongoDB runs, that a miss cost the statement nothing, is withdrawn on the same page: the server's own meter, read on a machine of our own, puts the misses at 3% to 32% of what a statement waits for (a tuning-bench reading, class L on that host, no benchmark result).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

88.18.2 Where it is written

Place	What
`tools/store_brain.py`	the working set's floor, the store's account of its misses, the gate, the host's room, the max's record
`tools/run_sysbench.py`, `tools/run_ycsb.py`, `tools/run_redis.py`	each cache wire's `Brain`: its own compass law and rule around the shared max; the server's account read each second; the attach check that stops a MongoDB run on a server without the account
`tools/run_kafka.py`	the group's twin, the reflex, the hold, the immune hand-back; the tune-down under the dial
`tools/run_pgbench.py`	the pool's `Brain`: never clamped upward, never below the pooler's own setting
`tools/knob_verdict.py`	the tune-down's verdict, now in the fingerprint; its summary reads the max's records
`omni_controller/controller.py`, `omni_controller/muscles.py`	the hand-back that lands: every step tried 5 times against the API's weather, each muscle on its own, what did not land raised
`tools/store_gate.py`	the store gate: every candidate replayed on omni-v21's archived seconds (class S)
`tools/kafka_model.py`, `tools/redis_model.py`	the two closed-loop models (class S), every input set from omni-v21's runs' own files
`docs/MYSQL_PREREGISTRATION.md` (amendment 6), `docs/YCSB_PREREGISTRATION.md` (6), `docs/POSTGRES_PREREGISTRATION.md` (4), `docs/REDIS_PREREGISTRATION.md` (5), `docs/KAFKA_PREREGISTRATION.md` (4)	declared before any run on them

88.18.3 What the model gate read

Our own models, evidence class S, never a benchmark result.

Gate	Against omni-v21
the store gate, every store wire's candidate on omni-v21's archived seconds, open-loop (`results/model_gate/V22_STORE_GATE.md`)	no LOSS on any wire or workload; the seconds below native fall to none on every wire (MySQL 97.9 → 0 an arm, MongoDB 12.5 → 0, Redis 36.6 → 0, PostgreSQL 223.5 → 0); the three cache

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Gate	Against omni-v21
	candidates hold at native in the replay because omni-v21's audits carry neither the stores' accounts nor the host's memory; the pooler's servers +28.2% and Kafka's consumers +130.6%, power given
the Redis cache, closed loop (`results/model_gate/V22_REDIS_MODEL.md`)	work inside the line GAIN in every workload (up 7.7% to 22.9%), the mean latency GAIN (down 36% to 59%), the 95th and 99th percentiles GAIN or STEADY, the memory held power given (201 to 304 MB against 62 to 66 MB); no LOSS
the Kafka group, closed loop, 30 s a notch, the development seeds 101 to 108 (`results/model_gate/V22_KAFKA_MODEL.md`) and the untouched seeds 201 to 208 (`…_UNTOUCHED.md`)	no LOSS row; work inside the line GAIN, the 95th and 99th percentiles GAIN, the most messages waiting at once GAIN, consumers power given
the Kafka group at 20 s a notch (run C's setting, not the counted one; `…_20S.md`)	one LOSS row: the burst's most messages waiting at once, up 30.6% inside the seeds' spread (native's queue grows about 1,100 messages in a 20 s notch and one rebalance costs about the same); on the next version's list with its cause
the Kubernetes governor's models (`results/model_gate/V22_VS_V21.md`, `tools/model_gate.py`)	omni-v22's one change to the governor, the hand-back tried five times against the API's weather, moves no decision: every model reads omni-v21's own numbers on every gauge, STEADY, identical

MySQL's and MongoDB's accounts of a miss are read for real on the first runs of omni-v22 (omni-v21's audits do not carry them), and a closed-loop model of each store, set from those runs' own files, is the next model gate's work.

Every benchmark runs again on omni-v22 from the moment it reaches main; nothing is read across engines. The store benchmarks run first, every table at the counted 30 s a notch (`scripts/reruns.txt`).

Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 89

History

Earlier states of play, kept whole. Nothing here is current status; the current state is `docs/STATE_OF_PLAY.md`.

89.1 State of play as written on 27 September 2026 (with the update of 28 September)

Superseded for live results. The live evidence below predates sets 19 and 20. The current live table is `results/live/LIVE_PAIRED.md`; energy on kind is a declared model, not a meter. The first metered test is `scripts/gpu_paired.sh` (`docs/GPU_BENCH.md`).

Update, 28 September 2026. Added since this page was written, each with its evidence: - **Live, set 21** (conveyance only when response time needs it): p95 -37%, machines in service -21%, CPU used +26% (unchanged from set 20; its source is still open). `results/live/LIVE_REPS_21.md`. - **GPU governor bounded** (0.70 share floor, busy gate, 2 s decisions): on the MLPerf-calibrated modelled card, +5.1% and +1.3% work per kJ with p95 within +10% (the old governor failed that guardrail at +31% and +66%). Model only. `results/gpu/sim/`. - **Speed lock** (opt-in): speed won elsewhere spent on GPU watts, every gauge kept at least 1% faster than without Omni. Model only. `results/gpu/sim/pipeline/`. - **CPU and GPU on one conserved power budget** (conveyance law over CPU and GPU organs): +1.4% to +5.7% work served against today's fixed caps, never over the site budget. Model only. `docs/CONVEYANCE_LAW.md`, `results/hardware/NODE_EXCHANGE_*.json`. - **The manual in the box:** `docs/INTEGRATION_MANUAL.md`; every gauge: `docs/METRICS_CATALOG.md`; against what runs today: `docs/COMPARISON.md`.

This is the whole repository at the commit named in `STATE_OF_PLAY_COMMIT.txt`. Everything below can be rerun from it.

Run it live on real Kubernetes yourself: on any machine with Docker, kind, kubectl and Python, run `bash RUN_LIVE.sh 3`. It builds a fresh 7-node cluster per arm, runs native, Omni on top and Omni alone, and writes `live_runs/LIVE_REPS.md`. GitHub Actions runs the same comparison with a pushed commit whose message contains `[reps]`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Rerun the whole thing: `pip install -r requirements.txt && python verify.py`. It must end `VERIFICATION: PASS`, and it does at this commit.

89.2 Best measured results, and where each one comes from

Claim	Evidence	Where
B (Omni on top of each platform): 0 losing cells on held-out scenarios; strictly GAIN on 27 of 28 platform-workload pairs	simulation, pre-registered	`tuning/B_FIX.json`, `tuning/B_TONE_HELDOUT.json`
Four-cluster sites: B 0 losses on all 7 platforms; C 0 losses vs GKE, AKS/Karpenter, CAST AI, Spot	simulation, held-out	`tuning/SITE_LEAGUE.json`
C (Omni alone), one global setting, 400 never-used scenarios: 153 toward their aims / 174 equal / 37 LOSS or power given of 364 cells (Holm-corrected)	simulation, held-out	`tuning/CONFIRMATORY.json`
C with nervous-system coordination, 400 fresh scenarios: 151 / 173 / 40	simulation, held-out	`tuning/CONFIRMATORY_COORD.json`
Controllers fighting each other: lowest of all 8 systems on web (0.07 per day) and four-cluster sites (0.56 per day)	simulation, held-out	same file
Real demand, 1,052 recorded PlanetLab machines: 0 losing cells of 91	simulation on real traces	`tuning/PLANETLAB_LEAGUE.json`
Faults (machines dying, spikes, crash loops, noisy neighbours): fewest pages to a human, fastest recovery	simulation, 500 runs per workload	`results/protocol/`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Claim	Evidence	Where
GPU power law (engine cap over the MLPerf-measured performance law), against native at equal work: energy -27% to -33%, time over the heat limit -73% to -78%	simulation, held-out	`results/hardware/SUMMARY_HELDOUT.json`
Site power exchange (conveyance law, manuscript Ch. 29-31): 4 GPU groups under one site budget: 0 minutes over the site limit at 70/60/50% budgets (native 10.5/45.6/106); least energy at every budget; 31-115 fewer backlog minutes than the static split	simulation, held-out; law proved (conservation, Lyapunov, exponential convergence)	`docs/CONVEYANCE_LAW.md`, `results/hardware/SITE_EXCHANGE_HELDOUT_*.json`
Same-tower OFF / observe / ON receipt: observe identical to OFF; ON ahead of the graded-reflex comparison build on healthy time, recovery, energy, violations and peak power (peak below OFF)	simulation, held-out	`results/tower_off_on/`, `results/external_review/CHATGPT2_REVIEW.md`
Live levers under the nervous system: 19 of 19 checks on real Kubernetes; reset restores everything	live, kind	`results/live/LIVE_LEVERS_2_NERVOUS.txt`
Shadow pilot kit: a read-only identity made 0 writes across 40 decisions	live, kind	`results/live/LIVE_SHADOW_1.txt`
Safety shield: 2,000,000 adversarial cases, 0 violations (the test found 2 real bugs, both fixed); C++ twin matches	test	`tests/test_shield_properties.py`
C++ engine: 100,000,000 decisions, no failures, about 2.3 µs per decision	test	`results/SOAK.json`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

89.3 What still loses, measured

- **C, typical response time, web and four-cluster:** about 20% slower than every platform. Omni fills pods to 80%, where the others stop at 70%, and trades response time for energy.
- **C, machine starts and stops:** loses to plain Kubernetes and OpenShift, which hold machines steady and pay for it in machine-hours.
- **C, GPU and batch:** energy 1-2% lower and power and heat margins 1-4% lower than the tight packers (AKS/Karpenter, CAST AI, Spot).
- **Live, set 3, with a working probe:**
  - Omni kept all 6 machines. Latency near the 500 ms SLO blocks machine release.
  - There is no significant difference from native on anything else, except that C has more waiting pods.
  - The earlier live machine savings (sets 1-2) are withdrawn, because the broken probe had blinded the latency sense (`results/live/LIVE_REPS_PROBE_DEFECT.md`).
  - Set 4's decision trail showed why. Latency spikes from pod saturation at load steps kept the service from being clean, and the engine's calm stayed under the machine threshold (0.65 against 0.7). Both are addressed by the attribution gate and the two-way wiring above; the live test is pending (`results/live/LIVE_REPS_4_DIAGNOSIS.md`).
- **A bound, not a loss:** no controller, even one with perfect foresight, can match both the tightest packer's machine-hours and the calmest autoscaler's churn at once (`tuning/bound.py`).

89.4 Two-way nervous system (new, live test pending)

Manuscript Appendix J names delay and dropout handling and feedback interpretation as the nervous system's job. The live controller now wires both directions (`docs/TWO_WAY_NERVOUS_SYSTEM.md`).

- **Blind senses are detected against the wall clock.** A hung probe was the root cause of the live set 1-2 error.
  - A blind sense feeds the engine's `stale` channel.
  - It blocks every give-back.
- **Commands are read back.** Drift feeds the engine's `drift_ratio`. No new machine release goes out before the last one has landed.
- **The machine organ has its own engine view and a release gate.** Pods move first, and the headroom proof uses the engine's rho.

Tests: 75,000 blind states, 200,000 gate states, 300,000 nervous-system states.

Live sets 5 and 6 could not run. From 03:51 UTC GitHub refused every job, including `verify`: runner ID 0, no steps, failed within about 2 seconds. That is GitHub declining to start runners, which on a private repository is most often exhausted Actions minutes or a spending limit. It is not a code failure. `python verify.py` passes locally at this commit.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

89.5 Map

Path	What it is
`omnicompass/core.py`, `cpp/`	six-state engine; C++ twin
`omnicompass/closure.py`	closure law: forward projection, boundary correction, turning point, muscle tone
`omnicompass/nervous_system.py`	one engine state grants each organ its authority and envelope
`omnicompass/shield.py`	safety shield, downstream of everything
`omni_controller/`	live Kubernetes controller and levers
`fleet/`, `hardware/`	simulation plants
`tuning/`	benchmarks, pre-registrations, results
`results/live/`	every live run, including the failed and withdrawn ones
`docs/history/OMNICOMPASS_BUYER_EDITION.md`	the buyer-facing report
`docs/history/CONSENSUS_AUDIT.md`	the five-reviewer audit (written inside the project; internal, not independent)

89.6 The state of play as it stood on 2026-10-10 (Omni v3), moved whole on 2026-10-16

Kept as its own page, `docs/history/STATE_OF_PLAY_2026-10-10.md`: the 10 October narrative, every number Omni v3's, named as such.

89.7 The state of play as it stood on 2026-10-06 (moved here whole on 2026-10-07)

89.8 In one paragraph (2026-10-06, Omni v1)

Omni v1, the same in all three of our runs. On real Kubernetes, every test ran three times as separate GitHub runs on the frozen engine, ten pairs each (`docs/OMNI_V1.md`, the six `results/live/V1_*.md` tables). Work inside the response line **+42% to +48%** in all three runs of the all-four test (18.6 → 27.6 requests a second in run A), GAIN in all three of our runs; the 95th percentile **−57% to −69%** in every run of the steady, wandering, all-four and fault tests, GAIN in all three of our runs; failed requests −12% to −13% where load swings, GAIN in all three of our runs; machines −1.5% to −3.4% at steady load and −15% to −24% on the batch queue, GAIN in all three of our runs, no difference beyond the noise elsewhere; energy a declared model (kind never powers a machine down); beside a noisy neighbour, no

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

difference beyond the noise on every row. **The Omni index on v1: +26.2%** on real Kubernetes (`results/OMNI_INDEX.md`; only rows the same in all three of our runs count, a row inside the noise counts as zero; Azure and the card join when their v1 runs land). The paragraph below is the earlier engine's reading, kept as the record it was.

Omni-Compass sits on top of Kubernetes and hardware and gets more out of what is already there. On real Kubernetes, the engine before v1 (the live controller at rules 1-8, commit `353903683009`), ten pairs per test, native against compass with the same work sent to both: **41.7% more work handled inside the response line on the same machines** (capacity, all four in one run), responses **59-64% faster** at the 95th percentile (steady, wandering, all four, faults), 12% fewer failed requests where load swings, energy equal or lower, and not one pod left without a machine. Machines: up to 2% fewer. Omni gives a machine back only when a paired trial shows the service no slower without it, and on these clusters one machine fewer made requests 30-45% slower in most trials, so it kept them and spent them on speed and work (earlier engines without that check parked 29-36%: `docs/history/`). All of it together, real software on real computers (the software end, not bare metal and not the chip): **the Omni index +12.9%** (`results/OMNI_INDEX.md`).

Omni v1 (`docs/OMNI_V1.md`): the engine is frozen and fingerprinted (`OMNI_V1.json`), and every test above runs three times on it, as separate GitHub runs (A the result, B and C the replications); each table reads GAIN in all three of our runs, LOSS in all three of our runs, no difference beyond the noise, or the runs disagree, and replaces the earlier engine's table as it lands. Running on v1 now: the Kubernetes suite three times, the batch test, the six organisms at 1, 10, 100 and 1,000 copies with the real cluster inside, the big organisms on rented Azure machines, the Azure steady and burst bill tests, CityLearn, and the power grid.

What is shown and what is not. Shown: more work inside the response line on the same machines (34-52% across the three capacity runs on earlier engines) and a faster tail, on real Kubernetes. Not yet shown: an energy or cloud-bill saving on real machines. On kind the machines are containers on one runner, so a machine out of service saves modelled watts, not a metered bill, and Azure's bill did not move on the earlier engine (`results/live/AKS_BILL.md`); the v1 Azure runs are the test of that. The current card governor has not run on a real GPU.

The card: the first real run (NVIDIA A10, 2026-10-02) used a card controller since replaced; it is not a result to stand on. In simulation the current controller (amendment 12) is clearly faster under an operator's power cap (p95 6-7% faster, time over the line 1 point lower) and saves 0.5-3.7% energy on the card's own firmware with p95 even (`results/sim/gpu_two_wire/`). The real card runs next: one card, the card inside the six organisms, then eight cards.

The 945 muscles (Omni v2; 656 in v1) are models of real control systems. With the real cluster or the real card inside, they show the mechanism (work the same, energy 0.1-0.2% lower, time over the line lower than native in every organism); they are never counted in the headline.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Test (real)	Work	Speed	Machines	Energy	Source
All four in one run: load up and down one step at a time, 10 pairs	**+29%**	p95 -62%	**-3.6%**	-0.3%	`results/live/ALL_FOUR.md`
Capacity, load rising, 10 pairs	**+48%**	p95 -50%	-1%	-0.3%	`results/live/AMENDMENT_3_RUNS.md`
Sets 22-27, same work, 10 pairs each	same, none failed	p95 -55% to -65%	**-29% to -36%**	-0.1% to -0.5%	`results/live/LIVE_REPS_22.md` to `results/live/LIVE_REPS_27.md`
Six organisms, cluster inside, 5 pairs each	requests served +0.5% to +6%	p95 -24% to -40%	same to -2%	-0.1% to -0.6%	`results/live/SIX_KUBE.md`
Azure AKS, steady load, 5 pairs	same	p95 -20%	same	(billed) same	`results/live/AKS_BILL.md`

All four in one run is done: every one a proven GAIN in the same run (`results/live/ALL_FOUR.md`). Running now: demand that wanders (up, spike, down, back up, idle, 10 pairs), the Azure burst bill test, and the 1,000-copy grid of the six organisms on Lambda.

89.8.1 The engine now, and what runs on it (2026-10-05)

The live controller is frozen at rules 1-8 (`docs/K8S_COMPASS_PREREGISTRATION.md`, amendments 1-8): rules 5-8 were added today (a pinned gauge is not a steady demand; coasting; cruise; the emergency brake). Every result above was measured on an earlier version of the controller, and each names its commit. So that every number comes from one engine, the whole Kubernetes suite runs again on the frozen engine: all four in one run, demand that wanders, the steady same-work set, the fault and fairness tests, the batch test, and the six organisms at every size; the Azure burst bill test and the two big organisms on rented Azure machines too. The card harness changed only in how it judges (amendment 11 of the GPU preregistration), so the single card and the eight cards run once, on this engine, on Lambda.

Back on the frozen engine so far, ten pairs each:

- **Faults** (`results/live/FAULTS.md`): p95 −60%, mean response −35%, time over the line −23%, pending pods −77%. Recovery is faster after every fault: machine down 42 s against 70 s, a blind probe 54 s against 66 s, a runaway pod 98 s against 105 s, a spike the same.
- **Fairness** (`results/live/FAIRNESS.md`): a noisy neighbour on the same cluster. Mean response −15%, no LOSS, and the neighbour's own app never behind. The earlier engine scored −0.3% on the index here; this engine scores +5.6%.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Independent simulator: CityLearn round 1 split (the 2022 battery districts lower on bill -4% to -11%, electricity -9% to -17%, carbon -7% to -14%; the 2020-2021 water-tank districts higher on peaks and ramping), the fix (only the electric batteries are steered), and round 2 on every untouched district (`docs/CITYLEARN_PREREGISTRATION.md`).

89.8.2 Measured on real systems: the newest set, Omni-Compass against Kubernetes as it runs today

Set 24 (2026-10-02, commit `c908054`) repeats it again: machines in service −31.6%, p95 −60.1%, p99 −64.1%, HPA replicas −38.6%, 0 failed requests, total CPU including Omni-Compass's own −1.8% (not significant) (`results/live/LIVE_REPS_24.md`). Set 23 before it: p95 −62%, replicas −37%, pod starts −64%, 0 failed requests, no energy or total-CPU difference (`results/live/LIVE_REPS_23.md`). The set-22 table below stands as first measured.

Set 22 (`results/live/LIVE_REPS_22.md`): 10 paired repetitions on real Kubernetes (kind), each pair on one machine, Kubernetes with its autoscaler alone against the same Kubernetes with Omni-Compass on top. The load is sent at a fixed rate, so both arms were given **the same work**.

Result	Kubernetes alone	With Omni-Compass	Change (95% interval)
Energy, parked machines still on at idle power (declared model, no meter)	160.3 Wh	160.1 Wh	**−0.1%, no difference**
Response time, 95th percentile	407.9 ms	158.8 ms	**−61%** (proven)
Response time, 99th percentile	639.4 ms	245.1 ms	−62% (proven)
Response time, mean	179.9 ms	98.4 ms	−45% (proven)
Failed requests	0	0	equal
Pods waiting to start, pod-minutes	0.265	0.025	−91% (proven)
Replicas, mean	8.93	6.91	−23% (proven)
Machines in service, mean (all stayed powered)	6	4.14	−31% (proven)
CPU used by the service	1.036 cores	0.957 cores	−7.6% (proven)
Omni's own CPU (its controller and every command it ran)	0	0.070 cores	+0.070 (proven)
CPU used, service and Omni together	1.036 cores	1.026 cores	**−0.9%, no difference**

- **Same work, much faster answers**, with no failed requests and far less waiting.
- **No energy saving is shown on kind.** Every machine stays powered; energy is a declared model, and counted at the idle power a parked machine really draws it is unchanged.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **No CPU saving once Omni's own cost is counted.** The service used 7.6% less CPU; the controller spent almost all of it. Cutting the controller's cost is the next improvement.
- The reset restored every setting in every run.

89.8.3 Kubernetes sets 25 and 26 (2026-10-02, 10 paired repetitions each, equal work)

Run	Machines in service	p95	Failed	Total CPU incl. Omni's own	Receipt
Set 25, the engine's allocation law	**−32.3%**	**−57.3%**	0 / 0	−0.6% (not significant)	`results/live/LIVE_REPS_25.md`
Set 26, the engine's allocation law	**−35.8%**	**−55.4%**	0 / 0	−1.5% (not significant)	`results/live/LIVE_REPS_26.md`
Set 26, **the compass law in the live controller**	**−17.2%**	**−64.8%**	0 / 0	+1.0% (not significant)	same; label by the preregistered rule: **fewer machines within the band**
Set 27, the engine's allocation law	**−36.6%**	**−53.1%**	0 / 0	−0.0% (not significant)	`results/live/LIVE_REPS_27.md`
Set 27, **the compass law aligned with the GPU governor**	**−15.9%**	**−65.5%**	0 / 0	+0.2% (not significant)	same; label by the preregistered rule: **fewer machines within the band**

Set 27 (running): the compass in the live controller reads the service as the corrected GPU compass does (mean response time, center 0.4), against native and the allocation law (`docs/K8S_COMPASS_PREREGISTRATION.md`).

89.8.4 Measured on a real GPU: the card's own meter (evidence class P)

First counted run, NVIDIA A10 on Lambda, 2026-10-02 (`results/gpu/run-20261002T082232Z/GPU_REPS.md`, 10 paired repetitions × native / watch / Omni, 600 s each, frozen at commit `c908054`, checksums verified). Wire check 7 of 7; 2,144 writes, none refused, every one read back, every arm ended at the start limit; watch equal to native.

Gauge	Native	Omni	Change (95% interval)
Work per energy (requests per kJ)	50.79	52.62	**+3.6% (+2.7% to +4.5%), proven**
GPU energy	69,180 J	66,790 J	−3.5%, proven
Requests served / not served	3,514 / 0	3,514 / 0	equal
Response time, 95th percentile	510 ms	809 ms	**+58.5%, higher, proven**

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Result, by rule: ENERGY IMPROVEMENT WITH SERVICE TRADEOFF (the p95 guardrail of +10% failed). The six organisms with the same card inside (`results/hil/run-20261002T082232Z/HIL.md`, 3 repetitions each): the card's work per energy +1.6% to +2.8% in every organism, the same requests, its p95 500 to about 600-935 ms. The cause, from the card's own samples, was wiring in the governor (amendment 6 of `docs/GPU_PREREGISTRATION.md`): busy bursts served at 736-768 MHz against 861-889 MHz on its own. Corrected (amendments 6 and 7); the corrected governor has not yet run on a card.

89.8.5 Simulated (models: they show the mechanism, not a measurement)

Result	Where
GPU governor with share floor and busy gate (one wire, the power limit), MLPerf-calibrated card: +5.1% and +1.3% work per kJ, p95 within +10%	`results/gpu/sim/after`
Two-wire GPU card under the compass law, corrected governor (amendments 6-7), 10 seeds and 10 fresh seeds, geometric means: **service** profile work per energy **+6.9% / +3.8%**, energy −6.4% / −3.7%, p95 **−5.9% / −2.3%** (faster), time over the line −0.03 / −0.04 pp; **batch** profile +8.1% / +4.2%, p95 +7.0% / −2.3%; the one-wire governor +0.1%; both wires restored every seed	`results/sim/gpu_two_wire/RESULT.md`, `fresh/`
The six organisms (Compute 345, Physics 262, Energy 282, Distribution 337, the four stacked 1,226, the whole tower 656), native against the compass law on every muscle, 1,000 paired runs at 1× and at 10× size: work per energy +0.20% to +0.30%, energy −0.21% to −0.32%, work −0.01% to −0.02%, time over the service line **+0.19 to +0.27 pp in every cell (band first not held)**, every knob handed back. 100× and 1,000× are running	`results/scale/GRID.md`
Speed lock (speed won elsewhere spent on GPU watts)	`results/gpu/sim/pipeline/`
CPU and GPU on one conserved power budget: +1.4% to +5.7% work served against a fixed cap, never over the budget	`results/hardware/NODE_EXCHANGE_*.json`, `docs/CONVEYANCE_LAW.md`
GPU groups sharing a site budget: 0 minutes over the budget	`results/hardware/SITE_EXCHANGE_HELDOUT_*.json`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Result	Where
Platform leagues, faults, PlanetLab traces, stack benchmark	`tuning/`, `results/protocol/`, `results/` (see `docs/history/BENCHMARK_REPORT.md`)
The 656-muscle tower as organisms, round 3 (preregistered, 10 seeds; every realm carries the shared spine; Omni as the shipped controller commands): the whole tower native against one governor on top, work per energy **+0.1%, SUPERIOR WITHIN GUARDRAILS**; inside the realms the spine costs service: Energy +0.2% with +1.9 pp violations (tradeoff), Compute 0.0% (+2.1 pp, not established), Distribution −0.1% and Physics −0.7% (**AWAY FROM ITS AIM**). Rounds 1 and 2 kept, superseded	`results/realms/REALMS.md`, `docs/REALM_MUSCLES.md`

Back Matter

BACK MATTER

Glossary

Term	Meaning
Antagonist pair	the up side and the down side of a muscle's control, each with its own gain
Authority	how far the nervous system lets an organ move this decision
Band	a reading's or a knob's safe range
Compass	the band seen as a position from 0 (calm) to 1 (the line), with its bottom in the middle
Term	**Meaning**
STEADY	Equalized, Verified baseline, Equilibrium, Neutral: the mean change under 0.5% (under the floor), or

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	Meaning
	nothing moved (the same), or a knob at its limit on native; the key, front matter
GAIN	Growth, Advantage, Improvement, Net-positive: a gauge moved the way its declared aim asks, against native; a resource cut is a GAIN; the key, front matter
GAIN in all three of our runs	the mean change 0.5% or more, every run (A, B, C) moved the same way with its 95% interval clear of zero, and that way a GAIN for the gauge's aim
GAIN by the mean of our three runs	the mean change 0.5% or more toward the gauge's aim, not every run clear of zero; the count of runs beyond the noise stands beside it (Chapter 59)
LOSS	Leakage, Off-track, Setback, Subpar: a gauge moved against its declared aim (less work, a longer wait, more misses); it should never appear, and where it does it is shown, its cause found and fixed; the key, front matter
Power given	a resource spent or held that rose: what Omni put in to buy the work and speed beside it; shown, never called a loss
Cover	a knob's hard range; every write is clipped to it
Cushion	the 5% at each edge of the compass, 10% in all
DEX-YLL-1	the company's number, 1-888-DEX-YLL-1, read in order as the engine runs: DEX, Data Exchange (the input, the two-way plug); YLL, Yield Limit Line (the boundary: every muscle up to its own yield limit, never past it); 1, one closed loop (the output; in code 1 is true, in science 1 is unity)
Fail up	the return to full capacity the moment service crosses the wall or a sense goes blind
Governor	the process that reads, decides and writes; Omni-Compass
Muscle	any machine, service or controller Omni-Compass reads and sets

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	Meaning
Native	the system as it runs without Omni-Compass: every muscle under its own native controller, as its vendor ships it (the native baseline)
Omni-Compass on native	the second arm: the native system with Omni-Compass on top of it ("omni")
Omni-Compass direct	the third arm, Omni-Compass by itself: each knob's native controller switched off and Omni-Compass the only controller moving it (section 1.1)
Factory profile	the complete, versioned set of Omni-Compass default settings shipped together (section 76.8; `docs/FACTORY_PROFILE.md`)
Candidate, active and custom profile	a proposed combination under evaluation; the settings actually governing the current run; an identifiable configuration that differs from the released factory profile (section 76.8)
The best validated factory profile for the declared operating conditions	the profile the eight steps of section 76.8 establish; the meaning "optimal" has here, and the only one
Restore factory profile; Return to native	the two restoring controls: reapply the released configuration; hand governance back and restore the recorded native settings (the reset)
The ten organisms; the twenty scenarios	the seven of the catalog, the orchestra, the live benchmark engine's organism and the whole organism, each read native and on native (sections 34.7 and 60.10)
The cloud is a wire	a cloud is the wire between the software end and the bare metal; Omni-Compass can live in all three places (section 60.6)
Beyond the noise	a run whose 95% interval excludes zero; a three-run reading carries the count of its runs beyond the noise (Chapter 59)
Off the clock	a repetition in which either arm ended more than 5% of its window late; marked, the pairing stands
Organism	a set of muscles run together on one clock

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	Meaning
Omni index	the one combined number: the geometric mean of every real category's work, speed, machines and energy ratios, each read by the three-run rule
Omni v1, v2, v3	the frozen engines, fingerprinted file by file; a result belongs to the engine its commit carries
Plug	the two-way connection to one muscle: read, write, read back, restore
Pool (Azure)	one machine family under Azure's autoscaler with its own ceiling; several pools make the fleet
Profile (service or batch)	a named set of the compass's settings for one kind of work, service (the default) or batch: a setting inside a configuration profile
Race	full speed at once while work waits, so a burst is never served slowly
Yield limit line	a muscle's own limit, read live: Omni-Compass takes it right up to the line and never past it, as an engineer loads a material up to its yield limit, the most it carries and still springs back unharmed
Speed floor	the clock a card reaches on its own while busy, learned from its own meter; the governor never sets the ceiling under it
Receipt	the paired record of native against Omni-Compass for one run
Snapshot	a knob's value read once before the first write; the restore point
The runs disagree	runs clear of the noise point different ways; the test is unstable on that measure and is looked into
Wire check	the test that proves every wire follows, reads back and returns before anything runs
Work per energy	work done divided by energy used; the primary outcome
Work inside the line	the requests, transactions or messages a second answered within the response line; the product number on every real stack

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	Meaning
One pedal	push for more, lift off for less, idle (the pedal lifted all the way), reset (everything back to native): how the governor moves capacity under autopilot
Cruise	every machine in service while work waits for a place, held without second-guessing until the line has been empty for two decisions
Idle at once	the work done and demand at zero: the pedal lifted all the way in one move, every safety check still holding
Kill switch	security only: one switch in a human hand that turns every governor on the machine off at once; never the reset
Lease	the record every governor renews each decision; a lease older than its limit, or a dead process, makes the watchdog hand back for it
Watchdog	the service beside the governors that hands back for any governor that died or hung without doing it itself
Verdict	the paired trial on the muscle itself that allows a slow knob to move only where the cost is within 2%; "left native" where no step passes
Slack gate	v3's rule: a motion axis busy more than half the time at full speed keeps its speed native
Tuning case	the one workload, cell or swarm the rules were fitted on; shown in every table and never counted
Untouched case	a workload, cell or swarm the rules were applied to unchanged; the judged rows
Knee	nine tenths of a system's measured capacity, where its queue begins to grow; the broker's native was set there by design
Amendment	a change to a preregistration after it was written, with its time and reason; one made after a result was seen says so
Dwell	the seconds after a knob moves during which the law will not move it back

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	Meaning
Notch	the smallest step a knob moves in one decision (one consumer, 8 MB of cache, one tap)
Evidence class P, L, S	a physical meter; live software with no meter; a model of ours
Handed back	the knob read back at the operator's value at the end of an arm; every table says whether every arm was
Off the clock	see above; a repetition whose arm ended more than 5% of its window late
Own cost	the governor's CPU (its process and every command it ran) as a share of one core over the window, from its audit; a few thousandths of a core at every size
Smoke run	one repetition that exercises a harness end to end before any counted run; never counted, always recorded
L + S	an organism run with a real cluster inside: the cluster is L, the organism around it S
Working set	the records a workload actually touches over a window; a cache benchmark measures nothing unless the working set reaches the cache
Key space	the records a benchmark may ask for at one notch; in the MongoDB test the knob's load, stepped one notch at a time
Uniform draw	every record in the key space equally likely, so the working set is the key space; YCSB's shipped zipfian draw concentrates on a few thousand hot records whatever the key space
Ordered keys	records named in load order, so a run that asks for record n finds record n; a run with hashed keys against an ordered load asks for records that do not exist
Full-cache gate	the MongoDB and Redis rule: slow reads grow the cache only while the cache is full (nine tenths used), because a miss in a cache with room to spare is not the cache's to mend

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Term	Meaning
Buffer pool	InnoDB's cache of table and index pages, the one memory setting every MySQL operator sizes; resized online by the server in whole chunks
Chunk	the unit in which MySQL resizes its buffer pool (128 MB as shipped); the knob's notch on that stack
Page cache	the operating system's own cache of file contents; where the data fits in it, a storage-engine miss is a read from memory and a decompression, not a disk read, and the gain available to the cache knob is smaller

APPENDIX

Appendix A. Command Reference

Task	Command			
Verify everything	`python3 verify.py`			
Seven organisms, modelled	`python3 tools/run_scale.py --runs N --scale K --out <dir>`			
The full ladder	`bash scripts/scale_ladder.sh`			
Two-wire card, modelled	`python3 tools/run_gpu_card.py <dir> [fresh]`			
GPU wire check	`sudo python3 tools/gpu_wire_check.py --gpu 0`			
GPU, whole benchmark in one command	`sudo nohup bash scripts/gpu_rented_run.sh > run.log 2>&1 &`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Task	Command			
The seven organisms with the real card inside	`python3 tools/run_hil.py --out <dir>`			
GPU governor, two wires	`sudo python3 -m omni_controller.gpu_compass --mode watch	cap ...`			
Kubernetes, watch	`kubectl apply -f deploy/install/omni-compass.yaml`			
Kubernetes paired runs	`scripts/kind_paired.sh`, then `tools/live_reps.py`			
OFF (Kubernetes)	`touch /tmp/omni.kill`			
OFF (GPU)	`sudo touch /tmp/omni-gpu-kill`			
The whole harness OFF, ON, status, watchdog	`python3 tools/omni_switch.py off	on	status	watchdog`			
Seal check	`python3 tools/seal.py --check`			
Which engine a checkout or a commit carries	`python3 tools/omni_version.py [--commit <sha>]`			
The three-run table (Kubernetes, Azure)	`python3 tools/confirm_abc.py "<title>" raw/run-<A> raw/run-<B> raw/run-<C> --out results/live/V3_<TEST>.md`			
The three-run table (database, messaging, cache, swarms, robots, grid, buildings)	`tools/pgbench_abc.py`, `tools/kafka_abc.py`, `tools/redis_abc.py`, `tools/swarm_abc.py`, `tools/mujoco_abc.py`, `tools/pandapower_abc.py`, `tools/citylearn_abc.py`, the same arguments			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Task	Command			
The organisms with the cluster inside	`python3 tools/six_kube_report.py <raw dirs> --out results/live/V3_SIX_KUBE.md`			
The grid of copies and runs	`python3 tools/grid.py` (reads `results/scale/receipts/`)			
The Omni index	`python3 tools/omni_index.py`			
Which knobs earn a wire in (write, watch, operator's choice)	`python3 tools/wiring_verdicts.py` (`--check` compares the page with the tables; `verify.py` runs it)			
One number per benchmark, plus always toward the system's own aims	`python3 tools/benefit_sheet.py` (`--check` the same way; `verify.py` runs it)			
The database benchmark, one run	`python3 tools/run_pgbench.py --out <dir>` (workflow `pgbench`)			
The message broker, one run	`python3 tools/run_kafka.py --setup`, then `--workloads light,heavy,burst --out <dir>` (workflow `kafka`)			
The cache, one run	`python3 tools/run_redis.py --setup`, then `--workloads small,large,burst --out <dir>` (workflow `redis`)			
The database's storage-engine cache under YCSB, one run	`python3 tools/run_ycsb.py --setup`, then `--workloads b,c,f,burst --out <dir>` (workflow `ycsb`)			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Task	Command			
The database's buffer pool under sysbench, one run	`python3 tools/run_sysbench.py --setup`, then `--workloads read_only,read_write,update_index,burst --out <dir>` (workflow `sysbench`)			
The robustness test	workflow `robustness`: `scenario` kill or long, `reps`, `duration_s`; the own-cost table `python3 tools/own_cost.py <raw run dirs> --out results/live/V3_OWN_COST.md`			
The drone swarms, one cell	`python3 tools/run_swarm.py --cell short	mixed	long	all --out <dir> (workflow swarm`
The independent simulators	`python3 tools/run_citylearn.py`, `run_pandapower.py`, `run_mujoco.py` (workflows `citylearn`, `pandapower`, `mujoco`)			
Azure, the bill	workflow `aks-metered`: `reps`, `arms`, `duration_s`, `load_steps`, `worker_pools`, `pool_prices`, `hpa_max`			
The big organisms, detached	workflow `big-organism-detached`: `mode` start / collect / survey, `test` organism / scale / robust / direct, `organisms`, `size`, `duration_s`, `commit`, `max_hours`			
Task	**Command**			
Omni direct on a rented cloud machine	`test` = `direct` on `big-organism-detached`			

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Task	Command			
	(Azure) or `cloud-organism` (`cloud` gcp or aws): the three arms' full form, `reps` runs; `dry_run` yes shows the price and rents nothing			
Archive a finished run with the code	nothing to do: workflow `archive-run` takes it within the hour; to take it at once, add its id to `.github/archive_request.txt` and push			
Verify before a push, as GitHub runs it	`python3 tools/release_manifest.py` under Python 3.12: 0 FAIL			

APPENDIX

Appendix B. File Map

Path	Contents
`omnicompass/`	the engine, governor, nervous system, shield, compass, conveyance law, the compass
`omni_controller/`	the Kubernetes controller and muscles; the GPU governors (one and two wires)
`realms/`	the 1,107-muscle catalog of v5 (`catalog_v5.csv`; v3's 945 in `catalog.csv`, frozen), every muscle's own card (`muscle_cards_v5.csv`), the plant models, the realm harness, the compass on every muscle (the max, `compass_arm_v5.py`), the modelled card
`tools/`	benchmarks, receipts, the wire check, the scale ladder, the card search (`tune_cards.py`), manifests, seals
`scripts/`	one-command runs (GPU, Kubernetes, ladder)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Path	Contents
`deploy/`	container image, install and permission files
`cpp/`	the C++20 twins
`results/`	every published result, with its raw files and checksums
`results/live/V1_*.md`, `V3_*.md` (+ `.json`)	the three-run tables, one per test, named by the engine they ran on
`results/live/raw/run-<id>/`	every archived run's files, one SHA-256 manifest per repetition (the branch `raw`, opened by `scripts/raw_archive.sh`)
`results/grid_v5/GRID.md`, `results/realms_v5/REALMS_V5.md`, `results/OMNI_INDEX.md`	the grid of copies and runs, the modelled muscles and organisms, the one combined number (the older engines' tables stay under their own names)
`OMNI_V11.json` (the engine that stands), `OMNI_V10.json`, `OMNI_V9.json`, `OMNI_V8.json`, `OMNI_V7.json`, `OMNI_V6.json`, `OMNI_V5.json`, `OMNI_V3.json`, `OMNI_V2.json`, `OMNI_V1.json`	the engines' fingerprints
`docs/`	this manual, the preregistrations, the evidence ledger, the theorem, the realm study
`docs/OMNI_V7.md`, `OMNI_V6.md`, `OMNI_V5.md`, `OMNI_V3.md`, `OMNI_V2.md`, `OMNI_V1.md`	what each engine is and every result read on it
`docs/REGISTER.md`, `docs/PROOF_PROGRAM.md`	every muscle, every benchmark run and every benchmark still to run; the program to full size
`.github/workflows/`	every benchmark as GitHub runs it: `benchmark-reps`, `six`, `six-kube`, `big-organism`, `big-organism-detached`, `aks-metered`, `pgbench`, `kafka`, `redis`, `ycsb`, `sysbench`, `robustness`, `swarm`, `citylearn`, `pandapower`, `mujoco`, `archive-run`, `verify`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Path	Contents
`tests/`, `verify.py`	every unit and property test, the shield's adversarial cases, the Python-against-C++ agreement; `verify.py` runs them all and must end with its pass line
`tools/omni_version.py`, `tools/release_manifest.py`, `tools/layout_check.py`	which engine a checkout or commit carries; the release manifest and verification as GitHub runs it; the check that every path the documents name exists
`tools/confirm_abc.py` and its siblings (`pgbench_abc.py`, `kafka_abc.py`, `redis_abc.py`, `ycsb_abc.py`, `sysbench_abc.py`, `swarm_abc.py`, `mujoco_abc.py`, `pandapower_abc.py`, `citylearn_abc.py`)	the three-run tables, one tool per kind of raw record, each checking the engine of every run it reads
`tools/omni_index.py`, `tools/dossier.py`, `tools/own_cost.py`	the one number from the tables; the dossier from the tables; the governor's own cost from the archived audits
`tools/wiring_verdicts.py`, `docs/WIRING_VERDICTS.md`, `results/WIRING_VERDICTS.csv`	the verdict per knob from every table: write, watch or the operator's choice, every loss with its cause; one row per knob and per muscle
`tools/knob_verdict.py`	the brain's own verdict on a live knob, around the engine's `omnicompass/verdict.py`: watch first, one notch a trial, the allowance, the two objectives; used by the five live harnesses
`tools/benefit_sheet.py`, `docs/BENEFIT_SHEET.md`, `results/BENEFIT_SHEET.json`	one number per benchmark from the tables, plus always toward the system's own aims, with yes, no, none or trade beside it
`tools/legal.py`	the legal notice every generated report carries at its head and foot
`docs/book/`	the builder of this manual's PDF (`build_book.py`) and the theory chapters bound into it
`release/`	the copyright deposit and the release notes, printed at named commits
`CHANGELOG.md`, `docs/STATE_OF_PLAY.md`	what changed, by date; where everything stands today

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Path	Contents
`docs/*_PREREGISTRATION.md`	the rules of every benchmark, written before it ran, with every amendment and its time
`docs/DOSSIER.md`, `docs/dossier/`	every result in one place, with its charts, built by `tools/dossier.py` from the tables
`docs/HISTORY.md`, `docs/history/`	earlier states of play, earlier engines' sets and the pre-v1 index, kept whole
`LICENSE`, `NOTICE`, `LICENSES/`	the license and notices

APPENDIX

Appendix C. The Equations in Full

```
(1) dE/dt     = −α[E] E + β[int] + β[ext] + v[eff]
(2) dU/dt     = μ U (1 − U²) − (dE/dt)/E[max] − λ[U] U + u,
                |u| ≤ U_AUTHORITY = 25
(3) dI[U]/dt  = (1 − U) − σ₁ E − δ S − λ[I] I[U]
(4) v[eff]    = cos(ω[B] t / 2) · c · tanh(λ₀ + λ₁ (U − 0.5) + λ₂ S)
(5) Φ(S)      = αₛ S² / 2 + βₛ S³ / 4 − δ S

(6) dS/dt     = −dΦ/dS = δ − αₛ S − (3/4) βₛ S²
(7) dB/dt     = Ḃ ;  dḂ/dt = γ[c] δ S − (ω[B] / Q[B]) Ḃ − ω[B]² B
(8) R[B]      = finite-difference audit of (7) at every step;
                never fed back into the state

Control (one micro step, zero-order hold across all four RK4 stages):

    u = clip(−f[U](x, t) + K[P] (σ − U), −U_AUTHORITY, +U_AUTHORITY)
    K[P] = 12,  σ ∈ {−1, +1}

The compass (every muscle):
    p = position of the service reading in its band
        (0 calm, 1 the line)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    F = A tanh((K[P] (p − center) + K[D] v) / A)
    K[D] ≥ critical damping
    p ≥ 0.95  ⇒  full up force; down side held
    knob ← clip(knob + g[side] F span, cover)

The GPU governor (two wires, omni_controller/gpu_compass.py;

one law, amendments 8 to 10):
    p = (mean response of the last 5 s − S) / (SLO − S),  S = SLO / 10
    p95 ≥ SLO, a failure, blind  ⇒  fail up
    fail up / race (utilization ≥ 0.95)
        ⇒  ceiling = top clock, lid = start limit
           (the card's own settings)

    steady under the limit (saturated, drawing ≥ 0.97 of the limit)
        ⇒  ceiling = the card's own busy clock, lid = start limit
           (the same watts, no knock-backs)
    otherwise:
        step = round(g[side] F f[top] / 15 MHz) · 15 MHz
        ceiling ← clip(ceiling + step, c[floor], f[top])

        c[floor] = max(f[busy,own] × floor, f[top] − k* 15 MHz)
                   (never under the card's own busy clock, never past
                    the verdict's deepest step k*)
        lid = clip(ceil(1.10 P[busy,own]), envelope floor,
                   start limit)
    g[up] 0.10, g[down] 0.0125, center 0.4, floor 1.00

The verdict (omnicompass/verdict.py; every slow knob, the GPU ceiling
and the Kubernetes machine count):
    reference r = the costs of n requests at the deepest allowed step
                  (a card: at native), measured fresh
                  (Kubernetes n = 200; a card n = 30)

    trial t     = the costs of n more at one step past it, under the
                  same traffic
    d = mean ln c[t] − mean ln c[r] ± 1.96 √(s²[r]/n[r] + s²[t]/n[t])
                  (Welch); stepwise a second interval against the
                  costs first measured at native; τ = ln(1 + a), a = 0
    ↪ .02
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    LOSS   iff an interval lies wholly above 0  ⇒  refused
    wide   iff a half-width > τ and n[t] < 3n   ⇒  measure on
    allow  iff median t ≤ min(r, c[native]) (1 + a)  and  d ≤ τ
           (else refused, past the allowance)
    a trial begins only while calm, P after the last began
    (Kubernetes 600 s; a card 8 decisions); a phase past its limit

    (Kubernetes 1,200 s; a card 30) or out of calm is abandoned
    a refused step is not tried again for R (Kubernetes
    R = 7,200 s; a card 900 decisions = 30 min); the knob never goes p
    ↪ ast
    the deepest allowed step, so steps cannot add up beyond the
    allowance; on the max the machine count gives nothing back

Kubernetes, the compass on the HPA target
(omni_controller/controller.py; sets 30 and 31):
    target x ∈ [0.6 x[op], x[op]] (times the conveyed gain);
    the up force lowers x (more pods)
    p ≥ 0.95 from load (line breached, every sense live, nothing

    pending)  ⇒  x = 0.6 x[op] at once
    p ≥ 0.95 from a blind sense or pods waiting for a lost machine
        ⇒  x = x[op] (native's own)
    fault over (p < center, line clean, nothing pending)
        ⇒  x = x[op] at once, never held
    machines: one more at once past the wall; one back only when the

    verdict and the release gate agree
    g[up] 0.10, g[down] 0.02, release force < −0.2


The productivity arithmetic (every receipt; manual section 76.4):
    work W equal in both arms;  G = C[native] / C[omni] − 1;
    the same work needs 1/(1 + G) of the resources;

    saving S = G / (1 + G):  G = 1/3  ⇔  S = 1/4
```

The symbols, and their names in the code. The equations are written in the symbols above; the source (`omnicompass/core.py`, `omnicompass/adapter.py`, printed in Appendix H) spells each one out, and an engineer reading the two side by side needs the correspondence once. The values are those the live controllers run (`STACK_PARAMS` in `omnicompass/adapter.py`); the pressures and the drive are computed from the nine observations every decision and have no fixed value.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Symbol	In the code	What it is	Value in the live controllers
α[E]	`alpha_E`	the deviation's drain rate	3.487
β[int], β[ext]	`beta_int`, `beta_ext`	the internal and the external pressure on the deviation	computed each decision
v[eff], c	`v_eff`, `c`	the bounded drive and its bound	c = 1.0
λ_0, λ_1, λ_2	`lambda_0`, `lambda_1`, `lambda_2`	the drive's offset and its weights on the alignment and the stress	−0.40, 1.55, 1.15
μ	`mu`	the depth of the alignment's double well	3.782
E[max]	`E_max`	the deviation scale in the alignment's equation	2.0
λ[U]	`lambda_U`	the alignment's own damping	0.0
σ_1	`sigma_1`	the deviation's weight in the memory	0.38
δ	`delta`	the stress's drive (the engine's own default is 0.5)	0.705
λ[I]	`lambda_I`	the memory's leak	0.719
α_s, β_s	`alpha_s`, `beta_s`	the coefficients of the cubic potential Φ(S)	0.12, 0.10
γ[c]	`gamma_c`	the stress's drive on the bath	1.0
ω[B], Q[B]	`omega_B`, `Q_B`	the bath's natural frequency and quality	1.0, 2.5
Ḃ	`B_dot`	the bath's rate, the engine's only second-order element	state
R[B]	`R_B`	the finite-difference audit of the bath, never fed back	recorded
K[P]	`KP`	the control law's gain	12
u, U_AUTHORITY	`u`, `U_AUTHORITY`	the command and its authority	±25
σ	the target pole	+1 (healthy) or −1 (failed), the nearer basin by default	±1
f[U]	computed inside `control_command`	the drift of U with the command at zero	computed each micro step

Parameter ranges, defaults and the proof of convergence: `omnicompass/core.py`, `docs/TRACKING_THEOREM.md`, `docs/CANONICAL_ENGINE.md`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

APPENDIX

Appendix D. Metrics

Every gauge, where it comes from, and whether it is measured or modelled: `docs/METRICS_CATALOG.md`. The gauges that appear in the three-run tables of this manual, with their source and their direction, are these:

Gauge	Source	Measured or modelled	Direction
work inside the response line (requests, transactions or messages a second answered within the line)	the probe's own records, pgbench's log, the consumers' records, the application's records	measured	the aim is higher; the product number
response time, mean, median, 95th and 99th percentile	the same	measured	the aim is lower
time over the response line (share of samples)	the same, against the preregistered line	measured	the aim is lower
failed requests, lost messages	the same	measured	any increase is marked
alert minutes: whole minutes whose 95th percentile, failed requests counted over, was over the line	the probe's own records, minute by minute	measured	the aim is lower: the minutes an on-call page would be open
settings commands the governor wrote on its own	the governor's audit	measured	shown, not judged (no person touched either arm)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Gauge	Source	Measured or modelled	Direction
the governor's additions decided while the service was still inside its line (% of its additions)	the governor's audit	measured	shown, not judged
worker machines in service, mean; node-hours	the cluster's own node records every 15 s	measured (machine-hours on kind; Azure's own count on AKS)	the aim is lower
compute bill at list price	Azure's own machine count × list price per family	measured (a real bill)	the aim is lower
energy (Wh), the standby model	a declared formula over the machines in service (kind)	**modelled**, said so on every line	the aim is lower
host CPU busy; host CPU-seconds; CPU-seconds per 1,000 units inside the line	`/proc/stat` of the real machine under the test	measured	the aim is lower
Omni's own CPU (cores), mean	the governor's audit (its process and every command it ran)	measured	shown, not judged in the Kubernetes tables; a loss in all three of our runs where it is one
connections held open, consumers running, memory ceiling held	the pooler's, the group's, the cache's own reports	measured	the aim is lower (the resource held)
cache hit rate; consumer lag	Redis's INFO; the consumers' records	measured	the aim is higher; the aim is lower
HPA replicas, pods started, pod start wait	the API server's own records	measured	shown, not judged except pod start wait
organism work, energy, time over its line	the organism's plant models	**modelled**	by direction; S class
energy a mission, missions a charge (drones)	the simulator's motor constants through a declared model	**modelled**	lower; higher

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Gauge	Source	Measured or modelled	Direction
cache size held, mean; bytes in the cache, mean; pages read into the cache (MongoDB)	the server's own `serverStatus` (WiredTiger cache)	measured	the aim is lower (the resource held); shown, not judged (pages read)
seconds from the kill to every setting back at the operator's (robustness)	the harness polling the cluster once a second from the kill	measured	the aim is lower; past 60 s marked; never, INVALID
the governor's resident memory, last ten minutes over the first ten (robustness, long run)	the process table, sampled every 15 s by the harness	measured	a growth over a quarter is marked
decisions made of expected; failed decisions (robustness, long run)	the governor's audit	measured	under 95% INVALID; any failed decision shown with its reason
mission time, position error, collisions (drones)	the simulator's own state at every step	measured in the simulator (S class)	lower; under the safe error; any collision voids the cell
handed back (every arm)	the knob read back at the end of the arm	measured	yes in every arm, or the run is invalid
off the clock	the organism's own clock against its window	measured	shown; marks the cell

Three rules for reading any gauge in this appendix. A measured gauge is one the system under test, or the host's own kernel, reported about itself; a modelled gauge is one computed from a declared formula over measured inputs, and every table says which it is on the line itself, never only here. A resource gauge (machines, connections, consumers, memory, cache) is always judged as aiming lower, even where the governor's purpose was to add the resource to buy service, so that a gain bought with a resource is paid for in the same table and in the index. And a gauge marked "shown, not judged" never enters a reading or the index; it is there so that a reader can see what the governor did (how often it moved, how many pages it read, how many pods it started), not to be counted for or against it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

APPENDIX

Appendix E. Troubleshooting

Symptom	Cause	Fix
Wire check `FAIL 3 up wire (lock)`	the driver or VM refuses clock locking	use bare metal or full passthrough; or run the one-wire governor
Wire check `FAIL 3 up wire (follows down)`	the card ignores the ceiling	update the driver; check for another management agent holding clocks
Wire check `FAIL 4 down wire`	the power limit is refused or capped by the board	check `nvidia-smi -q -d POWER`; run as root
`another copy of this test is already running`	a second copy was started	wait for the first or reboot; start once
`something else is using the GPU`	another process holds the card	stop it; the benchmark must run alone
Governor exit 5	another writer changed a knob	find the other controller; Omni-Compass left its value alone
Governor exit 3	a restore did not read back	restore by hand (`nvidia-smi -rgc`, `-pl <start>`); investigate before rerunning
`decision failed (n in a row)`	the cluster API is unreachable	turn it OFF; native runs on
Energy saved but p95 slower than native	the card served bursts below its own clock, or the lid sat under its own draw	read section 42.5; in the audit compare `telemetry.clock_mhz` while busy with `native_busy_clock_mhz`, and `want_w` with `native_busy_draw_w`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Symptom	Cause	Fix
Fail-up in most decisions	`--slo-ms` set too low for the workload, or a stale feed read as blind	set `--slo-ms` from the workload's own target; check the feed's age
No saving at all, service unchanged	the card is saturated almost all the time (it races), or the service sits above the compass's center	expected on a card with no slack; the saving comes from the quiet stretches
Kubernetes: no machine ever given back	the release gate refuses (its reason is in the audit), or the service is above the compass's center	read `node_gate.reason` in the audit; check `--slo-ms` and the latency feed
Kubernetes: a repetition marked OFF THE CLOCK	the machine could not step the organism inside its window	a longer window for that size on that machine; never a faster reading of the same run
Broker: consumers climb to the cover and stay	messages wait at every step (the producer is past the group's capacity)	the benchmark's capacity step sets the peak at nine tenths of the operator's group; check the measured capacity in the run's log
Cache: the ceiling grows while little memory is used	cold misses read as pressure	the full-cache gate is the rule (used ≥ 90% of the ceiling); check `used_mb` beside `maxmemory_mb` in the run's log
Cache: the ceiling never grows	the working set fits the operator's ceiling	expected: a cache that fits has nothing for the ceiling to mend, and the row reads inside the noise
Storage-engine cache: the cache never fills, whatever the key space, and omni reads as a free memory saving	the benchmark's requests concentrate on a few hot records (YCSB's shipped zipfian draw), or the run asks for records the load did not write (hashed keys over an ordered load), so the reads are served from the index	a result that flatters the governor is the first thing to suspect: check "in cache" against the cache size in the per-notch lines; draw requests uniformly over the key space and name the same key order in load and run; a knob with nothing to do is not a saving
Storage-engine cache: the compass only ever reads calm	the line is set far above the stack's own latencies (a 2 ms line over 0.15 to 0.45 ms reads)	set the line on the tuning workload's own figures so that a hit is calm and a full-cache miss is slow, and write the figures in the preregistration before any counted

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Symptom	Cause	Fix
		run
Swarm: a cell reads "void"	two drones came within two collision radii in either arm	a task or simulator setting (keep-out, altitude stagger, depot spacing); fixed before any counted run and said in the preregistration
Three-run table heads itself with a warning	a run's commit is not the engine it claims, or the three are not separate runs	run `tools/omni_version.py --commit <sha>` on each; make the table only from runs on one frozen engine
Archive bot skips a run	the run was not finished when the bot passed, or the pass had taken its share of bytes	nothing to do: the next hourly pass copies it

APPENDIX

Appendix F. Evidence Map

`docs/EVIDENCE_LEDGER.md` (every claim and its class), `docs/CLAIMS_REGISTER.md` (what is claimed and what is not), `docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`, `docs/POSTGRES_PREREGISTRATION.md`, `docs/KAFKA_PREREGISTRATION.md`, `docs/REDIS_PREREGISTRATION.md`, `docs/SWARM_PREREGISTRATION.md`, `docs/CITYLEARN_PREREGISTRATION.md`, `docs/PANDAPOWER_PREREGISTRATION.md`, `docs/ROBOTICS_PREREGISTRATION.md`, `docs/ROBUSTNESS_PREREGISTRATION.md`, `docs/YCSB_PREREGISTRATION.md` and `docs/MYSQL_PREREGISTRATION.md` (the rules written before each run), `docs/OMNI_V1.md` to `OMNI_V3.md` (every result by engine), `docs/REGISTER.md` (every benchmark and its file), `docs/PROOF_PROGRAM.md` (the program to full size with its costs), `docs/DOSSIER.md` (every result in one place), `results/OMNI_INDEX.md` (the one number), `docs/WIRING_VERDICTS.md` (which knobs earn a wire in, and every loss with its cause), `docs/STATE_OF_PLAY.md` (where everything stands), `docs/HANDOFF.md` (every command in one page).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The map, claim by claim. Each claim this manual makes is listed here with the class of evidence behind it and the document that carries it, so that a reader can go from any sentence of the executive summary to the file that would have to be wrong for the sentence to be wrong.

Claim	Class	Where the evidence is	What would falsify it
The engine's six states stay inside their walls and settle	T / V (proof and test)	`docs/FORMAL_STATUS.md`, `docs/TRACKING_THEOREM.md`, `tests/`, `results/SOAK.json`	a trajectory leaving its wall, or the C++ twin disagreeing with the Python
No write ever leaves a knob's cover; a blind sense holds; fail-up is immediate	T / V	`tests/test_shield_properties.py` (two million cases), the plug tests per stack	one case of a write past the cover or a give-back while blind
On a real cluster the governor does more work inside the line, faster, on no more machines, at no more energy	L	the six Kubernetes tables in `results/live/`, A/B/C each, `docs/K8S_COMPASS_PREREGISTRATION.md`	a row LOSS in all three of our runs on work or speed, or power given on machines or energy with nothing bought in any of the six, which would stand in the table
Killed outright, the governor's settings are back at the operator's within seconds	L	`results/live/V11_ROBUST_KILL.md`, `docs/ROBUSTNESS_PREREGISTRATION.md`	a repetition handed back after 60 s, or not at all
Over two hours the governor neither leaks, slows nor drifts, and hands everything back at the end	L	`results/live/V11_ROBUST_LONG.md`, `docs/ROBUSTNESS_PREREGISTRATION.md`	memory growing past a quarter, the decision time growing past a half, fewer than 95% of expected decisions, or a setting not handed back
Governing costs a few thousandths of a core at every size	L	`results/live/V3_OWN_COST.md`	an own-cost figure growing with the organism
The pooler, the buffer pool and the storage-engine cache each give memory back on an untouched workload with no LOSS; the broker and the cache read nothing on the first set under the brain's verdict	L	`V3_PGBENCH.md`, `V3_KAFKA.md`, `V3_REDIS.md`, each preregistration	a product row LOSS in all three of our runs on an untouched workload; the memory row of the cache is such a loss and stands

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Claim	Class	Where the evidence is	What would falsify it
The one number is +4.8% across the real categories on the sets of 10 October, losses included	L, by rule	`results/OMNI_INDEX.md`, `tools/omni_index.py`	a category omitted, a loss not entered, a tuning row counted
A day of demand measured by someone else reads the same way as our own schedules	L	`results/live/V3_TRACE_GOOGLE2011.md`, `results/traces/google2011/schedule.json`, `docs/TRACES_PREREGISTRATION.md`	a schedule derived after the runs; a rule fitted to the governor; a trace part whose SHA-256 does not match
The modelled realms, grids, arms, buildings and swarms gain under their own native controllers	S	`results/realms/`, `V3_PANDAPOWER.md`, `V3_MUJOCO.md`, `V3_CITYLEARN.md`, `V3_SWARM.md`	a run of the same simulator at the same version and seed giving other digits
A physical meter shows less energy for the same work	P	**no current result**; `docs/GPU_PREREGISTRATION.md` names the run	the rerun on the current card controller reading no difference, or LOSS
Omni-Compass never changes the engine between a rule and its result	by construction	`OMNI_V3.json`, `tools/omni_version.py --commit <sha>` on every table's commits	a table whose runs' commits carry different fingerprints

What a referee asks of each document. Of a preregistration: is it dated before the first counted run, does it name the tuning case and the untouched cases, is every amendment dated and does any amendment made after a result was seen say so? Of a three-run table: are the three run ids separate runs, does the head name the engine at each commit, is every gauge of the preregistration present including the ones that went against the governor, and is the reading one of the three the rule allows? Of the index: is every real category in it, is every tuning row out of it, and does a loss in all three of our runs enter as a loss? Of the raw folder: does the SHA-256 manifest in it match the files, does the governor's audit show every decision with its reading and its write, and does the hand-back record read the operator's value? Every one of those questions has a yes in this program or a disclosure naming the exception; the disclosures are in section 60.14.

How to audit one result from this manual to its raw files. Take any row of Chapter 60. Its source column names a three-run table in `results/live/`; the table's head names the three GitHub run ids, the commit each ran on and the engine `tools/omni_version.py` reads at that commit. Each run id names a folder `results/live/raw/run-<id>/` (the branch `raw`, opened by `scripts/raw_archive.sh open /run-<id>/`) holding every repetition's files (the capture, the response times, the governor's audit and log,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the per-request records, one SHA-256 manifest per repetition). The table tool named in the table's text (`tools/confirm_abc.py` or its siblings) rebuilds the table from those folders; `tools/omni_index.py` rebuilds the index from the tables; `tools/dossier.py` rebuilds the dossier. The preregistration named in the table's text holds the rules, written before the first counted run, with every amendment dated. Nothing in that chain is by hand.

What one repetition leaves behind. Take the first repetition of the first kill-scenario run, archived at `results/live/raw/run-37716845219/paired-1/`. It holds two folders, one per arm, and the omni arm's folder holds twenty-nine files, every one of which a reader can open: `preflight.txt` (the UTC time, the arm, the git commit the arm ran at, the versions of Docker, kind, kubectl, the API server and Python, the metrics-server version with its SHA-256, the worker count and the probe's address); `rbac_omni.txt` (twenty lines of `kubectl auth can-i`, each verb the governor needs reading yes and each it must not have reading no); `window_start.txt` and `window_end.txt` (the window's clock); `load_schedule.log` (every load step and when it was applied); `latency.csv` (the probe's response times, one row a second); `capture.csv` (the cluster's state every fifteen seconds: replicas, workers, pods pending, the HPA's target); `host_cpu.csv` (the runner's own `/proc/stat`); `pod_watch.json` and `pods_end.json`, `nodes_end.txt`, `hpa_end.json` (the API server's own records at the end); `audit.jsonl` (one line per decision: the compass's position, heading and letter, the engine's state, every write with its value sent and its value read back); `controller.log` and `controller2.log` (the first governor's log, and the second one's after the kill); `audit_kill.jsonl` (the watchdog's restore writes, each with its command and its reason, such as "reset: remove record"); `watchdog.log` (one record: which governor, its process id, whether it was hung or dead, every command run and each exit code); `robust.log` (the kill at the 360th second with the settings as they stood, "target=50 range=1,10 limits=1850m 3700m workers=5/6", and the hand-back "every setting at the operator's after 8 s, target=50 limits=500m workers=6/6 left=none"); `rss.csv` (the governor's resident memory every fifteen seconds); `omni_writes.txt` (how many writes the governor made in the window: 42); `kill_switch.txt` (the reset check: target restored, workers in service, no record left); `omni2.pid` (the second governor's process id) and `kill` (the marker the harness wrote at the kill); `robust.out` (the same robustness log as the job printed it); `end_reads.err`, `pod_watch.err` and `capture.csv.errors` (empty, as they should be); `metrics-server-components.yaml` (the pinned manifest as applied); and `SHA256SUMS.txt`, one line per file. A reader who doubts a number in the kill table can find the second it was taken.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

APPENDIX

Appendix G. The Muscles: What Each Is For, and How It Is Wired

A **muscle** is one setting on one machine that already has its own control: a replica target, a node pool's size, a GPU's clock ceiling, a chiller's setpoint, a battery's reserve, a joint's effort. Omni-Compass does not replace that control. It reads the machine's meters and, at its default, the max, moves the setting the machine already accepts toward the machine's own limit for the most work, never past it and never below native, through a plug that reads the setting once before the first write, reads back every write, steps aside if another controller moves it, and walks it back to native at the end. A knob native already runs at the top of its range is at its limit on native and is left there.

This chapter lists all 1107 muscles of the v5 catalog (`realms/catalog_v5.csv`); each run records its engine separately. For each one it says what kind of machine it is, which of the four kinds of knob it is, what Omni-Compass reads and does with it, how such a muscle is reached in a real stack, and which organisms it belongs to. The plants behind the benchmark numbers are models of these machines (evidence class **S**); a muscle in this list is wired on a real system only through the levels and the checks of the manual (chapters 4 and 6). See `DISCLOSURES.md`.

G.1 How to read an entry

The four kinds of knob.

- **capacity**: how much of the machine is in service (replicas, instances, units, speed). At the max Omni-Compass pushes it to the machine's own limit for the most work, never past it and never below native.
- **admission**: what is let in (queues, rate limits, admission control). Native already lets everything in: the knob is at its limit on native, and Omni-Compass reads it and leaves it native.
- **power**: the power share or limit the machine may draw. Native already runs it at full: at its limit on native, read and left native.
- **setpoint**: the target the machine's own loop holds (a temperature, a reserve, a process value). At the max a room is cooled ahead of the heat and a battery's charge is held for its line, inside the safe band, never behind native on its line.

The five kinds of plant. Every muscle is modelled on one of five plants, each with its own service reading and lever:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Plant	What it is	What Omni-Compass reads	What Omni-Compass moves
`compute_pool`	a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs	how far its queue and its load sit toward the service line (queue or lateness, and load above half)	the capacity it may use: its replica or instance count (through its autoscaler's target or floor), its power share, or one unit given back through the release gate
`thermal_zone`	a data hall or building zone cooled by chillers, air handlers or fans	its temperature against its limit	its supply setpoint (colder is more cooling), its power share, or one cooling unit given back when the room has margin
`energy_storage`	a battery, UPS or microgrid store with a load and a grid connection	its draw on the grid connection and its reserve	its reserve setpoint (how much charge it holds back); its power limit is left native
`motion_axis`	a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor	its tracking error and its load	its speed and effort share, or its power share
`process_loop`	a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint	how far the process sits from its band	its setpoint within its safe band, its actuator range, or its power share

Every muscle its own card. The brain is one, its settings each muscle's own. A muscle's card holds the settings the max uses for that muscle (a machine pool: how full it runs on its forecast before it calls for more, and how many decisions of history it reads; a battery: how many wobbles of margin it keeps under its line, and, wired by its power, how long it reads its line before it holds and how much of it it keeps clear). The cards were found by an automatic search over every muscle on the development seeds, today's global setting always one of the choices, and kept only where the card beat the global setting beyond the noise on two separate sets of seeds and on no seed under any load fell behind native: never over its line more often, never less work, never its line cleared later (`tools/tune_cards.py`, `realms/muscle_cards_v5.csv`, `results/cards_v5/`). Inside a body, a battery wired by its power keeps the body's own margins on its line. 364 muscles carry a card of their own; every other muscle runs the global setting (machines started at 95% full, the last 20 decisions read, 3.09 wobbles of margin on a battery's line, 5% of it kept clear). Each muscle's card is in the last column of its family's table.

The organisms. The 1107 muscles build seven organisms: the spine (267 muscles, every muscle that sits in all four realms: the base every stack runs on, as an operating system is the base of every program), each of the four realms (every muscle whose realm list includes it, the spine among them), the four stacked with every

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

duplicate kept (1,908), and the whole tower with every muscle once (1107). Each is run native and with Omni-Compass on top of native (`docs/THE_SPINE.md`, `docs/REALMS.md`).

Organism	Muscles
The spine, the muscles every stack runs on	267
Compute / AI / Cloud	540
Physics / Robotics / Autonomous	404
Energy / Facility / Industrial	480
Distribution / Specialized	484
The four stacked, every duplicate kept	1,908
The whole tower, every muscle once	1107

How a muscle is reached in a real stack. The class of machine decides the wire:

Class of machine	What it is	The wire in a real stack
`batch`	batch and queued jobs	the job queue (Kubernetes Jobs, Slurm, Spark)
`batch_reactor`	a GMP batch reactor, fermenter or food process	the batch control system (ISA-88, OPC UA)
`building`	a building zone	the building management system
`chamber`	a controlled chamber (clean room, kiln, reactor)	the PLC or DCS
`clinical`	clinical systems: imaging archives, records, interface engines, monitoring gateways	the application's own scaling and admission settings (Kubernetes, the vendor console)
`commerce`	a customer-facing service with bursts	the Kubernetes API
`cpu_host`	a CPU host whose clock and power can be set	Linux cpufreq and RAPL
`data_hall`	a data hall cooling plant	the building management system (BACnet, Modbus) or its command line
`database`	a database cluster	the database operator (replicas, connection pools)
`district_heat`	a district heating or cooling network	the network control system and substation controllers (Modbus, M-Bus)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class of machine	What it is	The wire in a real stack
`elevator_hoist`	an elevator or escalator drive and its group control	the lift controller and destination dispatch (modelled only)
`ev_traction`	an electric vehicle traction motor	the vehicle's motor controller (CAN; modelled only)
`fabric`	a high-speed fabric (RDMA, DPU, SmartNIC)	the fabric manager's API
`facility`	a facility battery behind the meter	the energy management system
`feeder_voltage`	a distribution feeder's voltage	the distribution management system (IEC 61850, DNP3; modelled only)
`flight_axis`	a flight-control axis	the flight controller (modelled only)
`gpu`	GPU serving with a response-time target	nvidia-smi (clock ceiling and power limit) or the serving autoscaler
`gpu_batch`	GPU training and batch work	nvidia-smi, the job scheduler's pause and resume
`hospital`	a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites)	the hospital's building management system (BACnet)
`inverter`	a plant of grid-support inverters or wind turbines	the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547)
`irrigation`	pumps, pressure and climate on a farm or in a greenhouse	the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud)
`marine_propulsion`	a ship's propulsion shaft and power plant	the vessel automation and power management system (modelled only)
`microgrid`	a microgrid battery and loads	the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR)
`mill`	a grinding, flotation or materials-handling circuit	the plant DCS or PLC (OPC UA)
`network`	network routing and switching	the network controller (SDN, routing API)
`node`	a fleet of machines or VMs that boot in minutes	the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Class of machine	What it is	The wire in a real stack
`pipeline`	a pipeline segment with its compressors, pumps and valves	pipeline SCADA (DNP3, Modbus, OPC UA)
`port`	a container terminal's cranes and vehicles taking moves	the terminal operating system and fleet controllers
`process`	an industrial process loop	the PLC or DCS (OPC UA, EtherNet/IP, PROFINET)
`qpu`	a quantum or specialised accelerator queue	the accelerator's job queue
`rail_traction`	a train's traction and auxiliary systems	the train control and management system and ATO (modelled only)
`ran`	a radio access network cell or site	the RAN controller (O-RAN interfaces)
`reaction_wheel`	a spacecraft reaction wheel	the attitude controller (modelled only)
`robot_fleet`	a fleet of robots or vehicles taking tasks	the fleet manager's dispatch API
`robot_joint`	a robot joint servo	the robot controller (ROS 2, EtherCAT, the drive's fieldbus)
`server`	an application service on servers or pods	the Kubernetes API (HPA target and floor, pod resize)
`storage`	block, file or object storage	the storage system's QoS and tiering API
`turbine`	a generating unit's governor, boiler or excitation loop	the plant DCS and governor (OPC UA, IEC 61850; modelled only)
`ups`	an uninterruptible power supply	the UPS and PDU management interface (SNMP, Modbus)
`water`	a water or pumping process	the SCADA system (Modbus, DNP3)
`workflow`	a workflow or pipeline engine	the workflow engine's concurrency settings

Wires marked *modelled only* exist as plants in the benchmark and are not built for live use.

How to wire any muscle on your own stack (the plug, chapter 4). Every muscle below is wired the same way: Omni-Compass never replaces the native controller; it reads the controller's own meters (the wire out) and writes the one setting the controller already accepts (the wire in), through a plug that (1) reads the setting once before the first write and records it, (2) reads back every write and stops writing if another writer moved the setting, (3) renews a lease every decision so a dead governor's settings walk back to native by themselves, and (4) restores the recorded setting at the end. Each family's entry says which native controller owns its

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

muscles, where the reading is tapped, where the setting is written, how it is restored, and whether that wire has run live in this repository or only on a model (`docs/wiring_recipes.csv`). The muscle's name is the setting's own name on that controller. Where the entry says *modelled only*, the plant exists in the benchmark and the wire is written against the controller's documented interface, not yet run on the machine.

G.2 The spine: 267 muscles in all four realms, an organism of its own

The machines every realm stands on: servers, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution. The spine is the seventh organism (`spine`, the base every stack runs on, run native and with Omni-Compass on top like the other six; `docs/THE_SPINE.md`), and each spine muscle is also part of every realm's organism, counted once in the tower and four times in the stack.

G.2.1 Cloud VM & Capacity (21 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

- **Native controller:** the cloud's autoscaling group or scale set: AWS Auto Scaling, Azure VMSS, GCP MIG.
- **Wire out (read):** group size, instance states, the target-tracking metric (the cloud's API and monitoring).
- **Wire in (write):** desired capacity, min and max, the target-tracking policy's target, through the cloud CLI or SDK.
- **Restore:** the group's desired, min, max and policy recorded once; restored from it.
- **Run live?** live on AKS through the node pool (family 5); the cloud SDK path itself is written, not yet run live.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
97	vm count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
98	vm size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 40 decisions
99	vm start stop	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
100	vm migrate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
101	instance family	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
102	asg min	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
103	asg max	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
104	spot mix	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions
105	reservation mix	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
106	zone selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
107	region selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
108	architecture selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
109	accelerator selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
110	boot disk class	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
111	placement group	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions
112	interruption response	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
1038	target tracking policy	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1039	scaling cooldown warmup	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
1114	vm cpu cap by priority	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1120	predictive scaling	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
1121	instance refresh	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions

G.2.2 Container Resources (25 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the pod's own resource requests and limits (the Deployment template) and, where it runs, the VPA, Goldilocks or CruiseKube recommender.
- **Wire out (read):** each pod's CPU use (kubectl top), its request and limit, the node's allocatable CPU (Kubernetes API).
- **Wire in (write):** the running pod's CPU limit in place through pods/resize (no restart); the template is never changed, so no rollout.
- **Restore:** the operator's limit recorded in an annotation before the first write; every pod returned to it.
- **Run live?** live: omni_controller/muscles.py convey() (conveyance); KEDA and Goldilocks tables.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
33	cpu request	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
34	cpu limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
35	memory request	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
36	memory limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
37	ephemeral storage	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions
38	hugepages	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions
39	io weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
40	pids limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
41	memory high	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
42	memory reclaim	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
43	swap limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
44	cgroup io max	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
45	cpu weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
46	cpu quota	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
47	cpuset	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
48	runtime class	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1017	in place resize	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
1019	namespace quota	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1020	default limits	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
1022	memory hard limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
1023	memory protection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1024	io latency target	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
1025	freeze	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
2176	cruisekube request rightsizing	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
2177	goldilocks vpa request recommendation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 10 decisions

G.2.3 Host CPU & Memory (26 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the Linux kernel's own governors: cpufreq (schedutil or the processor driver), RAPL, cpuidle, the memory manager.
- **Wire out (read):** /sys/devices/system/cpu/cpufreq (frequencies and governor), /sys/class/powercap intel-rapl energy counters, /proc/stat busy share, /proc/pressure.
- **Wire in (write):** scaling_max_freq per policy and the RAPL package power limit through sysfs (root); on Kubernetes through the Kubernetes Power Manager's PowerProfile or EAR's policy.
- **Restore:** the governor's files snapshotted once and restored byte for byte (tools/run_cpu_power.py --restore).
- **Run live?** written and tested on the modelled machine (tools/run_cpu_power.py, cpu-power.yml); GitHub's machines are virtual and cannot write the governor: the live run needs a real Linux machine.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
113	cpufreq min	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
114	cpufreq max	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
115	rapl package power	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
116	uncore frequency	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
117	energy perf preference	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
118	cpu idle policy	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
119	memory bandwidth	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start,	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			never fewer than native runs; hands back what it no longer needs at once		
120	numa balance	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
121	irq affinity	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
122	llc allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
123	memory pressure gate	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
124	page reclaim rate	capacity	starts the machines the forecast needs before the rush lands and clears	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once		
125	transparent hugepages	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
126	core online offline	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
127	thermal throttle policy	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
128	host power profile	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
1040	frequency governor	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
1041	turbo boost	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			within one machine start, never fewer than native runs; hands back what it no longer needs at once		
1042	dram power limit	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
1043	swappiness reclaim	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
1112	core packing	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
1115	per pod cpu frequency range and c state access	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
1116	uncore frequency per node	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
2174	ear cpu frequency policy	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2175	ear global energy budget	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
2178	k8s power manager core profile	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global

G.2.4 Kubernetes Placement & Scheduling (21 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

- **Native controller:** kube-scheduler and its profiles, the descheduler, PriorityClasses, scheduling gates.
- **Wire out (read):** pods pending and their reasons, node fill, topology spread (Kubernetes API).
- **Wire in (write):** a pod's or deployment's placement fields (nodeSelector, affinity, topologySpreadConstraints, priorityClassName, schedulingGates) by kubectl patch, or the scheduler's profile.
- **Restore:** the fields as they stood before the first write, restored field by field.
- **Run live?** modelled; the API path is the same kubectl patch the live controller uses for the HPA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
49	node selector	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 85% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
50	node affinity	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
51	pod anti affinity	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions
52	topology spread	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions
53	numa placement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
54	gpu topology	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 20 decisions
55	storage locality	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
56	network locality	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
57	taint toleration	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
58	priority class	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
59	preemption policy	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
60	device claim	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
61	failure domain spread	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
62	scheduler backoff	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
63	gang admission	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
64	deschedule	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1026	scheduler scoring strategy	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
1027	scheduler plugin weights	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 40 decisions
1028	scheduling gates	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1029	cpu manager	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
2179	peaks power aware placement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

G.2.5 Kubernetes Workload Scaling (32 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

- **Native controller:** the HorizontalPodAutoscaler, KEDA ScaledObject or VPA that owns the workload's scale.
- **Wire out (read):** the HPA's current and desired replicas, the Deployment's ready replicas (Kubernetes API, metrics-server), the service's response time from the probe.
- **Wire in (write):** the HPA's target (spec.metrics averageUtilization) and floor (minReplicas) by kubectl patch; KEDA through its ScaledObject, so the add-on keeps its own loop.
- **Restore:** the operator's target and range recorded in an annotation before the first write, restored from it, annotation removed.
- **Run live?** live: scripts/kind_bench.sh and omni_controller (HPA target, KEDA, Goldilocks, CruiseKube), three-run tables in results/live.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
17	replicas	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
18	hpa cpu target	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
19	hpa memory target	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
20	hpa custom target	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
21	vpa apply	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
22	scale to zero	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 80% full; reads the last 40 decisions
23	keda threshold	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
24	rollout rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
25	max surge	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
26	max unavailable	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
27	deployment pause resume	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
28	rollout abort	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
29	pod eviction	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
30	pdb policy	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
31	scheduler queue priority	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
32	api priority fairness	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1001	hpa min replicas	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
1002	hpa max replicas	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions
1003	hpa scale down stabilisation window	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			machine start, never fewer than native runs; hands back what it no longer needs at once		
1004	hpa scale up stabilisation window	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
1005	hpa scaling rate policies	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
1006	hpa policy selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
1007	hpa tolerance	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			needs at once		
1008	hpa sync period	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
1009	vpa minmax allowed resources	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
1010	keda polling interval	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
1011	rollout progress deadline	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1012	rollout readiness delay	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			back what it no longer needs at once		
1014	unhealthy pod eviction policy	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1015	graceful termination	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
1016	probes	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
2172	kepler energy target autoscaling	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions

G.2.6 GPU Hardware (22 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

- **Native controller:** the card's own firmware limits through NVML and nvidia-smi (power limit, application clocks, MIG, compute mode).
- **Wire out (read):** nvidia-smi --query-gpu (clocks, power draw, temperature, utilisation, memory), the serving response time from the probe.
- **Wire in (write):** nvidia-smi -pl (power limit), -lgc/-rgc (clock ceiling and reset), -ac (application clocks), -mig and -c (modes), per card.
- **Restore:** every limit read once at start (the snapshot line in the GPU audit) and restored by the same commands at the end or on the lease.
- **Run live?** live: omni_controller/gpu_governor.py and scripts/gpu_paired.sh on a rented A10 (results/gpu); the new card runs wait on Lambda by the founder's order.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
129	gpu allocate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
130	gpu power limit	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
131	gpu sm clock	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
132	gpu memory clock	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
133	gpu persistence mode	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions
134	gpu compute mode	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
135	mig mode	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
136	mig geometry	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 85% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
137	gpu timeslice	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 40 decisions
138	gpu mps	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
139	gpu quarantine	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
140	gpu reset	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
141	gpu thermal limit	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
142	gpu ecc response	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			machine start, never fewer than native runs; hands back what it no longer needs at once		
143	gpu job power budget	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1044	application clocks	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
1045	gpu temperature target	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
1117	sync boost	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions
1118	target clocks	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
2173	climatik inference power cap	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2180	adaptive gpu power cap	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global
2181	nvidia platform power budget	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global

G.2.7 Node Fleet & Karpenter-Class Control (24 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

- **Native controller:** the cluster's own node autoscaler: Karpenter (NodePool), the Cluster Autoscaler, a MachineSet, a cloud node pool.
- **Wire out (read):** ready and schedulable workers, pending pods, allocatable and used CPU (Kubernetes API), the autoscaler's status ConfigMap.
- **Wire in (write):** the pool's size through the operator's own command (--node-scale-cmd: az aks nodepool scale, gcloud container clusters resize, eksctl scale nodegroup, kubectl scale machineset, a Karpenter NodePool limit); a machine parked by taint and drain, never deleted by Omni-Compass.
- **Restore:** the pool's starting count recorded once; every parked machine untainted and the pool restored to it at the end or on the lease.
- **Run live?** live: omni_controller nodepool mode on kind and on AKS (results/live/V1_AKS_*, the capacity and steady tables).

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
65	node desired	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			back what it no longer needs at once		
66	node pool min	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 10 decisions
67	node pool max	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 40 decisions
68	node provision	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 60 decisions
69	node cordon	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
70	node drain	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
71	node consolidate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
72	node replace	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
73	node shutdown	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 40 decisions
74	node power on	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
75	nodepool weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
76	disruption budget	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
77	consolidation policy	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 20 decisions
78	consolidate after	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 90% full; reads the last 20 decisions
79	expire after	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
80	capacity class	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
1030	scale down utilisation threshold	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1031	scale down unneeded time	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
1032	scale down delay after add	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 20 decisions
1033	expander	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 80% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1034	node pool limits	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
1035	kubelet eviction thresholds	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1036	kubelet reserved resources	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions
1037	max pods per node	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 40 decisions

G.2.8 Network Routing & Switching (19 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the network controller: the load balancer's weights, the SDN controller, the NIC's ethtool and tc settings.
- **Wire out (read):** link utilisation, drops, queue latency (SNMP, the controller's API, ethtool -S).
- **Wire in (write):** LB and route weights through the controller's API; rate limits and queue disciplines through tc; NIC queues through ethtool.
- **Restore:** every weight and setting recorded; restored with the same commands.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
225	lb weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
226	route weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
227	rate limit	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
228	bandwidth limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
229	qos class	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
230	connection limit	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
231	failover route	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
232	nic rate limit	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
233	nic queue count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
234	queue discipline	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			machine start, never fewer than native runs; hands back what it no longer needs at once		
235	congestion control	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
236	egress budget	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
237	ingress budget	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
238	ecmp weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
239	path selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
240	network isolation	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1046	nic interrupt coalescing	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
1047	nic ring size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 80% full; reads the last 10 decisions
1048	port link power	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global

G.2.9 Observability & Telemetry (9 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

- **Native controller:** the collector's own configuration: the OpenTelemetry Collector, Prometheus scrape and remote-write settings.
- **Wire out (read):** dropped samples, queue depth, memory, cardinality (the collector's self-metrics).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Wire in (write):** the collector's batch, queue and memory limits and scrape intervals through its config reload.
- **Restore:** the config recorded; reloaded from it.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
373	collector memory limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
374	export concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
376	cardinality budget	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
377	retention window	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 85% full; reads the last 40 decisions
378	remote write queue	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
379	telemetry shed	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1053	metric scrape interval	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
1054	batching	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 80% full; reads the last 10 decisions
1119	per pod energy attribution	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 60 decisions

G.2.10 Reliability, Security & Recovery (12 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the platform's own recovery and security controls: rollout and rollback, PodDisruptionBudgets, the security hold, a kill switch.
- **Wire out (read):** error rate, rollout status, the security hold (a ConfigMap key), the kill file.
- **Wire in (write):** rollout pause and undo by kubectl; the security hold read, never written; the kill switch (tools/omni_switch.py) is a human's.
- **Restore:** every write recorded and undone on the lease.
- **Run live?** live in part: the security hold and kill switch run in every Kubernetes benchmark (omnicompass/master.py).

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
385	restart	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
386	rollback	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
388	traffic divert recovery	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
389	degraded mode	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
390	actuator freeze	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
391	workload isolate	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
393	credential rotation gate	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
394	policy enforcement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
395	rate abuse gate	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
396	fault domain isolate	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
397	backup trigger	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 10 decisions
400	kill switch	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global

G.2.11 Storage Block/File/Object (20 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `storage` (block, file or object storage), reached through the storage system's QoS and tiering API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

Wire it on your stack.

- **Native controller:** the storage system's QoS and tiering: Ceph (ceph osd pool set, QoS), a cloud volume's IOPS class, S3 lifecycle.
- **Wire out (read):** IOPS, throughput, latency and queue depth from the storage system's metrics.
- **Wire in (write):** volume IOPS and throughput limits, replica counts and tiering through the storage API or CSI VolumeAttributesClass.
- **Restore:** the volume's class and limits recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
257	volume size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			than native runs; hands back what it no longer needs at once		
258	iops limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
259	throughput limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
260	replica count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
261	storage tier	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
262	volume placement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
263	snapshot trigger	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 10 decisions
264	rebalance	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 10 decisions
265	recovery rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
266	backfill rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
267	compaction pressure	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
268	cache allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 75% full; reads the last 10 decisions
269	object replication	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
270	erasure code profile	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
271	storage admission	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
272	degraded storage gate	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1049	scrub schedule	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	machines started at 70% full; reads the last 20 decisions
1050	io scheduler	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global
1051	disk power spin down	power	at its limit on native (the full clock): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1052	lifecycle tiering	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	the spine; all four realms; stack; tower	global

G.2.12 Cooling, Chillers & Thermodynamics (18 muscles)

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `data_hall` (a data hall cooling plant), reached through the building management system (BACnet, Modbus) or its command line. Omni-Compass reads its temperature against its limit. It belongs to every organism.

Wire it on your stack.

- **Native controller:** the building management system and the chiller plant's own supervisory controller (BACnet, Modbus).
- **Wire out (read):** supply and return temperatures, chiller load, fan and pump speeds, rack inlet temperatures (BACnet reads).
- **Wire in (write):** supply-air and chilled-water setpoints, fan and pump speeds within their safe bands by BACnet write; the plant's own staging keeps control.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled (the data-hall plant); the BACnet path is written.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
545	supply air temperature	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	the spine; all four realms; stack; tower	global
546	return air target	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
547	coolant supply temperature	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	the spine; all four realms; stack; tower	global
548	coolant flow	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	the spine; all four realms; stack; tower	global
549	pump speed	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	the spine; all four realms; stack; tower	global
550	fan speed	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	the spine; all four realms; stack; tower	global
551	chiller setpoint	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	the spine; all four realms; stack; tower	global
552	compressor authority	power	at its limit on native (the site envelope at full): read and left native	the spine; all four realms; stack; tower	global
553	cooling tower fan	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	the spine; all four realms; stack; tower	global
554	cooling capacity	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	the spine; all four realms; stack; tower	global
555	rack thermal budget	power	at its limit on native (the site envelope at full): read and left native	the spine; all four realms; stack; tower	global
556	gpu thermal envelope	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
557	cpu thermal envelope	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	the spine; all four realms; stack; tower	global
558	thermal workload migrate	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
559	thermal load shed	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1055	server fan speed	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	the spine; all four realms; stack; tower	global
1056	economiser	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	the spine; all four realms; stack; tower	global
1122	chiller plant supervisory control rl with safety layer	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	the spine; all four realms; stack; tower	global

G.2.13 PDU, UPS & Electrical Distribution (18 muscles)

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `ups` (an uninterruptible power supply), reached through the UPS and PDU management interface (SNMP, Modbus). Omni-Compass reads its draw on the grid connection and its reserve. It belongs to every organism.

Wire it on your stack.

- **Native controller:** the UPS and PDU management interfaces (SNMP, Modbus) and the servers' BMC power caps (IPMI DCMI, Redfish).
- **Wire out (read):** load per branch, UPS mode and charge, power per server (SNMP, Redfish).
- **Wire in (write):** server and rack power caps through Redfish or DCMI; UPS charge rate and mode through SNMP; breakers never written.
- **Restore:** every cap and mode read once and restored.
- **Run live?** modelled.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
529	server power cap	power	at its limit on native (full battery power): read and left native	the spine; all four realms; stack; tower	global
530	rack power cap	power	at its limit on native (full battery power): read and left native	the spine; all four realms; stack; tower	global
531	pdu branch power limit	power	at its limit on native (full battery power): read and left native	the spine; all four realms; stack; tower	global
532	pdu outlet control	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	the spine; all four realms; stack; tower	0 wobbles of margin on its line
534	ups operating mode	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	the spine; all four realms; stack; tower	0 wobbles of margin on its line
535	ups charge rate	power	at its limit on native (full battery power): read and left native	the spine; all four realms; stack; tower	global
536	ups discharge rate	power	at its limit on native (full battery power): read and left native	the spine; all four realms; stack; tower	global
537	phase balance	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	the spine; all four realms; stack; tower	0 wobbles of margin on its line
538	load transfer	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
539	reactive power target	setpoint	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	the spine; all four realms; stack; tower	0 wobbles of margin on its line
540	voltage target	setpoint	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	the spine; all four realms; stack; tower	0.25 wobbles of margin on its line
541	generator dispatch	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	the spine; all four realms; stack; tower	0 wobbles of margin on its line
542	electrical isolation	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
543	breaker trip gate	admission	at its limit on native (nothing paced or turned away): read and left native	the spine; all four realms; stack; tower	global
1057	ups shutdown battery test	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	the spine; all four realms; stack; tower	0 wobbles of margin on its line
1110	hierarchical power budget row pdu switchboard controllers	power	at its limit on native (full battery power): read and left native	the spine; all four realms; stack; tower	global
1111	priority aware capping throttle low priority first	power	at its limit on native (full battery power): read and left native	the spine; all four realms; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1113	power oversubscription with prediction	power	at its limit on native (full battery power): read and left native	the spine; all four realms; stack; tower	global

G.3 Realm: Compute / AI / Cloud (273 muscles of its own, 540 in its organism with the spine)

G.3.1 AI Inference Serving (24 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the serving engine's own autoscaler and batcher (vLLM, Triton, KServe, Ray Serve): replicas, batch size, concurrency, KV cache.
- **Wire out (read):** queue depth, tokens per second, p95 latency, KV-cache use from the server's /metrics.
- **Wire in (write):** the server's replica count or KServe/Knative autoscaling target, max_num_seqs and KV-cache budget through its API or its ConfigMap.
- **Restore:** the serving spec recorded once; restored to it.
- **Run live?** live in part: GPU serving under the card governor (results/gpu); the vLLM run is ready and waits on Lambda.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
161	model replicas	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
162	model route weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
163	model load	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
164	model unload	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
165	model instance count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
166	continuous batching	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
167	max batch size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
168	batch queue delay	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
169	inference concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
170	inference max tokens	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 85% full; reads the last 60 decisions
171	kv cache budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
172	prefix cache budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
173	speculative decode budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
174	model precision	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global
175	inference priority	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
176	inference slo gate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
1058	autoscaling target concurrency qps	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
1059	scale to zero panic window	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
1060	request priority levels	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
1061	kv cache swap space	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
1062	chunked prefill	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native	Compute; stack; tower	machines started at 80% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			runs; hands back what it no longer needs at once		
1063	tensor parallel degree	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 80% full; reads the last 40 decisions
1064	disaggregated prefill decode	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
1065	lora adapter load	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

G.3.2 AI Training (18 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu_batch` (GPU training and batch work), reached through nvidia-smi, the job scheduler's pause and resume. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the training job's launcher and scheduler (Kubeflow, Ray Train, Slurm, torchrun).
- **Wire out (read):** step time, GPU utilisation, checkpoint cadence, stragglers (the job's metrics).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Wire in (write):** the job's worker count (elastic training), batch sizes and checkpoint interval through the launcher's API or the job spec.
- **Restore:** the job spec recorded; the next run launched from it.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
177	training workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
178	global batch size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 85% full; reads the last 10 decisions
179	microbatch size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 90% full; reads the last 10 decisions
180	gradient accumulation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
181	data parallelism	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 60 decisions
182	tensor parallelism	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 10 decisions
183	pipeline parallelism	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 10 decisions
184	expert parallelism	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 85% full; reads the last 20 decisions
185	checkpoint interval	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
186	checkpoint trigger	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
187	training preempt	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
188	training gang size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 80% full; reads the last 10 decisions
189	elastic worker count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 90% full; reads the last 10 decisions
190	straggler mitigation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 20 decisions
191	training precision	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
192	compute comm overlap	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 80% full; reads the last 40 decisions
1066	activation checkpointing	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
1067	dataloader workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions

G.3.3 Cross-Cluster, Multi-Region & Edge (18 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the global load balancer and the fleet manager (Karmada, Fleet, the cloud's global LB).
- **Wire out (read):** per-region latency, capacity and cost, pending work (the global LB's and clusters' metrics).
- **Wire in (write):** region and cluster weights and quotas through the global controller's API.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
609	multi cluster dispatch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 60 decisions
610	region dispatch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 40 decisions
611	zone dispatch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
612	edge dispatch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
613	cloud capacity class	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start,	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			never fewer than native runs; hands back what it no longer needs at once		
614	workload migrate region	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
615	data residency gate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
616	latency region gate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
617	cost region gate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
618	carbon region gate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
619	global failover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 40 decisions
620	federation quota	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
621	cross cluster replication	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
622	edge offload	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
623	global admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5128	edge site node pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
V5-5129	edge inference gpu clocks	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global
V5-5130	edge sync interval	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

G.3.4 DPU SmartNIC & Programmable IO (12 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the DPU's own management (NVIDIA DOCA, the SmartNIC's SDK) and SR-IOV.
- **Wire out (read):** per-function rates, flow-steering hits, DPU CPU and memory (the DPU's counters).
- **Wire in (write):** rate limits, QoS groups and VF counts through the DPU management API.
- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0021	dpu pf tx rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
AUDIT-0022	dpu vf tx rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
AUDIT-0023	dpu sf tx rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0024	dpu bandwidth share	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
AUDIT-0025	dpu qos group	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
AUDIT-0026	sriov vf count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
AUDIT-0027	smartnic flow steering	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 80% full; reads the last 10 decisions
AUDIT-0028	smartnic offload enable	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0029	dpu cpu budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
AUDIT-0030	dpu memory budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
AUDIT-0031	dpu service placement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
AUDIT-0032	dpu failover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global

G.3.5 Distributed Cluster Managers (15 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the cluster manager's scheduler: YARN, Mesos, Nomad, Ray.
- **Wire out (read):** queue lengths, resource reservations, preemptions (the manager's API).
- **Wire in (write):** quotas, reservations and preemption settings through the manager's API.
- **Restore:** recorded once, restored.
- **Run live?** modelled.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
209	task admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
210	task priority	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
211	resource reservation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 80% full; reads the last 10 decisions
212	cluster quota	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
213	task preemption	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
214	binpack pressure	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 85% full; reads the last 10 decisions
215	spread pressure	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 80% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
216	gang schedule	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 90% full; reads the last 60 decisions
217	worker allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
218	maintenance evacuation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 90% full; reads the last 10 decisions
219	oversubscription	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 10 decisions
220	resource reclaim	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 85% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
221	queue fairness	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
222	deadline pressure	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
223	scheduler retry	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global

G.3.6 GPU Fabric & RDMA (18 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the fabric manager and the collective library (NVIDIA Fabric Manager, NCCL settings, RDMA CM).
- **Wire out (read):** link and NVSwitch counters (nvidia-smi nvlink, DCGM), collective time per step from the training job.
- **Wire in (write):** NCCL environment and the job's rank placement at launch; RDMA QoS through the fabric manager's API.
- **Restore:** the launch settings recorded; the next job launched with the originals.
- **Run live?** modelled; written against NCCL and Fabric Manager documentation.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
145	nvlink placement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			needs at once		
146	nvswitch route	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
147	gpu fabric quarantine	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
148	rdma bandwidth	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 10 decisions
149	rdma route	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
150	nic affinity	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
151	collective concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
152	collective algorithm	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
153	collective chunk size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
154	rank placement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
155	gpudirect policy	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
156	congestion response	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			within one machine start, never fewer than native runs; hands back what it no longer needs at once		
157	rail selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
158	fabric failover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
159	communication priority	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
160	fabric isolation	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
1068	collective protocol	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1069	nic hca selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global

G.3.7 HPC & Distributed Compute (19 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the batch scheduler: Slurm (scontrol, sacctmgr), PBS, LSF.
- **Wire out (read):** queue depth, node states, job wait times, power per node (sacct, sinfo, the power plugin).
- **Wire in (write):** partition limits, job priority and fair-share factors, node power-saving settings through scontrol and the scheduler config.
- **Restore:** the scheduler's settings recorded; restored with the same commands.
- **Run live?** modelled; EAR (family 8) is the live energy path on HPC.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
193	job slots	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
194	mpi ranks	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 20 decisions
195	rank mapping	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
196	node allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
197	job walltime	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
198	job priority	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
199	job preemption	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
200	checkpoint restart	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 80% full; reads the last 20 decisions
201	parallel io budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
202	collective budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
203	accelerator share	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 20 decisions
204	cpu gpu ratio	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
205	memory per rank	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
206	scratch budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
207	scheduler fair share	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
208	backfill policy	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
1070	node power saving	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global
1071	cpu frequency per job	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global
1072	partition limits	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

G.3.8 Kubernetes Dynamic Device Allocation (8 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** Kubernetes DRA: DeviceClass, ResourceClaim, ResourceSlice.
- **Wire out (read):** claims pending, devices allocated and tainted (the Kubernetes API).
- **Wire in (write):** DeviceClass selectors and claim capacity by kubectl apply; devices tainted, never removed.
- **Restore:** the objects recorded; restored.
- **Run live?** modelled; the API path is kubectl.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0001	dra device class selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 85% full; reads the last 40 decisions
AUDIT-0002	dra claim capacity	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
AUDIT-0003	dra claim sharing	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 20 decisions
AUDIT-0004	dra device taint	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0005	dra device eviction	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
AUDIT-0006	dra binding readiness	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 85% full; reads the last 20 decisions
AUDIT-0007	dra binding failure response	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
AUDIT-0008	dra device configuration	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

G.3.9 OpenShift & Machine API (9 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** OpenShift's Machine API operators: MachineHealthCheck, MachineConfigPool, the Cluster Version Operator.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Wire out (read):** machine phases, MCP status and pause, degraded counts (the OpenShift API).
- **Wire in (write):** MachineHealthCheck maxUnhealthy, MachineConfigPool paused and maxUnavailable by oc patch.
- **Restore:** the fields recorded before the first write, restored from the record.
- **Run live?** modelled; written against the documented OpenShift API, not yet run live.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
85	machine remediation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
88	machine health gate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
89	mcp pause	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
90	mcp max unavailable	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
91	node config rollout	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
92	operator reconcile budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
93	cluster version pacing	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
94	infra machine admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
95	machine failure domain	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

G.3.10 Quantum Computing Control Simulation (16 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `qpu` (a quantum or specialised accelerator queue), reached through the accelerator's job queue. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the quantum control stack's job queue and pulse schedules (Qiskit Runtime, the vendor's control software).
- **Wire out (read):** queue depth, calibration drift, error rates (the stack's metrics).
- **Wire in (write):** shot allocation, circuit admission and scheduling through the stack's API; pulse parameters never written live.
- **Restore:** recorded; restored.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
593	qubit mapping	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
594	circuit admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
595	shot allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
596	circuit scheduling	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
597	gate scheduling	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
598	pulse amplitude	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			needs at once		
599	pulse duration	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
600	pulse phase	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
601	pulse frequency	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global
602	coupling control	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
603	reset scheduling	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
604	measurement scheduling	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
605	dynamical decoupling	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
606	noise aware routing	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
607	error mitigation budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
608	quantum queue priority	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global

G.3.11 Work Admission & Demand Shaping (19 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the service's own admission layer: its API gateway, queue or admission controller (Envoy, NGINX, Kong, a job queue's concurrency setting).
- **Wire out (read):** queue depth, requests in flight and the response-time line from the gateway's or queue's metrics (Prometheus, the admin API).
- **Wire in (write):** the gateway's or queue's concurrency and rate settings (the limit named by the muscle) through its admin API or its config object.
- **Restore:** snapshot of each limit once before the first write; every limit restored from it.
- **Run live?** at its limit on native: read and left native in every run (nothing is paced or turned away); the wire is read-only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1	api concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
2	queue concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
3	queue backpressure	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
4	job admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
5	batch admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
6	inference admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
7	load shed	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
8	priority gate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
9	tenant admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
10	burst limit	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
11	deadline admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
12	work budget	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
13	request queue limit	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
14	retry admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
15	background work gate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
16	maintenance work gate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
1074	request concurrency limit	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
1123	adaptive concurrency limit	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
1124	priority request queue under limit	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global

G.3.12 Linux Kernel & OS Control (20 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the Linux kernel's own tunables: cgroup v2 files, /proc/sys/vm and net, the cpufreq and cpuidle governors, the I/O scheduler.
- **Wire out (read):** /sys/fs/cgroup, /proc/pressure, /proc/vmstat, /sys/block queue stats, the frequency files.
- **Wire in (write):** cgroup cpu.weight, cpu.max and memory.high; sysctl vm and net keys; scaling_max_freq, through sysfs and sysctl (root).
- **Restore:** every file snapshotted once and restored byte for byte.
- **Run live?** the CPU frequency path is written (tools/run_cpu_power.py); the other knobs are registered and queued, they need a real Linux machine.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5001	cgroup cpu weight	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5002	cgroup cpu quota	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5003	cgroup memory high	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 10 decisions
V5-5004	cfs scheduler latency	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5005	cpufreq dvfs governor	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global
V5-5006	transparent hugepages	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5007	io scheduler depth	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start,	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			never fewer than native runs; hands back what it no longer needs at once		
V5-5008	numa auto balancing	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5009	dirty ratio writeback	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5010	vm swappiness	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5011	tcp buffer autotune	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 60 decisions
V5-5012	block nr requests	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
V5-5013	ksm memory dedup	capacity	at its limit on native (the full actuator): read and left native	Compute; stack; tower	global
V5-5014	net rx coalesce	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs;	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			hands back what it no longer needs at once		
V5-5015	cpu cstate governor	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global
V5-5016	zswap pool fraction	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5017	page cache reclaim	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5018	irqbalance affinity	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 60 decisions
V5-5019	net qdisc codel target	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5020	hugepage pool reserve	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 20 decisions

G.3.13 Serverless & Functions (FaaS) (9 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Wire it on your stack.

- **Native controller:** the function platform's concurrency settings (Knative autoscaling annotations, AWS Lambda concurrency, OpenFaaS).
- **Wire out (read):** concurrency, cold starts, queue waits (the platform's metrics).
- **Wire in (write):** provisioned and reserved concurrency, scale-to-zero windows through the platform's API or annotations.
- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5021	fn concurrency limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5022	fn provisioned concurrency	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5023	fn scale to zero idle	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5024	fn memory size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5025	fn prewarm pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5026	fn reserved concurrency	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5027	fn burst admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5028	fn snapstart restore	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
V5-5029	fn timeout budget	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global

G.3.14 Virtualization & Hypervisor (9 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the hypervisor's management API (libvirt, vSphere, Hyper-V).
- **Wire out (read):** vCPU steal, balloon state, migration load (the hypervisor's metrics).
- **Wire in (write):** vCPU allocation, ballooning and migration pacing through the management API.
- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5030	vcpu allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5031	memory ballooning	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5032	vm overcommit density	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions
V5-5033	live migration pacing	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5034	numa vcpu pinning	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5035	sriov vf allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
V5-5036	vgpu partition	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
V5-5037	vm memory dedup ksm	capacity	at its limit on native (the full actuator): read and left native	Compute; stack; tower	global
V5-5038	hypervisor cpu pinning	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

G.3.15 Gut: Data-plane Intake & Waste (8 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the data plane's own intake pacing and compaction settings (the ingress, the queue, the store's compactor).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Wire out (read):** intake rate, backpressure, compaction debt, dead-letter depth (the components' metrics).
- **Wire in (write):** pacing, drain rates and compaction budgets through each component's settings API.
- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5045	ingress intake pacing	capacity	at its limit on native (the full actuator): read and left native	Compute; stack; tower	global
V5-5046	dead letter retry cap	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5047	gc compaction pacing	capacity	at its limit on native (the full actuator): read and left native	Compute; stack; tower	global
V5-5048	ttl eviction waste	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5049	backpressure threshold	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5050	queue drain rate	capacity	at its limit on native (the full actuator): read and left native	Compute; stack; tower	global
V5-5051	retry budget window	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5052	compaction debt drain	capacity	at its limit on native (the full actuator): read and left native	Compute; stack; tower	global

G.3.16 Skin: Perimeter & Admission (5 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the perimeter's own controls: the WAF, the DDoS shield, the TLS terminator.
- **Wire out (read):** request admission, inspection sampling, session cache hit rate (the perimeter's metrics).
- **Wire in (write):** token-bucket rates, sampling and session cache size through the perimeter's API; blocking rules never loosened.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5053	rate limit token bucket	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5054	connection admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5055	waf inspection sampling	capacity	at its limit on native (the full actuator): read and left native	Compute; stack; tower	global
V5-5056	tls session cache	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
V5-5057	ddos shield admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global

G.3.17 Media Streaming & CDN (14 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the streaming stack's transcoder pools, packager and CDN settings.
- **Wire out (read):** transcode queue, GPU clocks, buffer health, origin load (the stack's metrics).
- **Wire in (write):** worker pools, bitrate-ladder cap and cache TTLs through the stack's API; GPU clocks through nvidia-smi (family 9).
- **Restore:** recorded; restored.
- **Run live?** modelled; the GPU path is live (family 9).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5072	abr bitrate ladder cap	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5073	player buffer target	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5074	transcoder worker pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
V5-5075	live transcode gpu clocks	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global
V5-5076	vod transcode batch power	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global
V5-5077	packager workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 85% full; reads the last 60 decisions
V5-5078	origin shield cache size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5079	edge cache ttl	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5080	segment duration target	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5081	cdn request coalescing	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5082	drm license server pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 85% full; reads the last 40 decisions
V5-5083	thumbnail sprite batch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 80% full; reads the last 10 decisions
V5-5084	media ingest admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5085	encoder lookahead frames	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global

G.3.18 Game Servers & Real-Time Interactive (12 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** Agones Fleets and the matchmaker, the game node pool.
- **Wire out (read):** allocated and ready servers, matchmaking waits, tick rate (Agones and the matchmaker's metrics).
- **Wire in (write):** Fleet replicas and buffer sizes by kubectl patch; matchmaking admission through its API.
- **Restore:** the Fleet spec recorded; restored.
- **Run live?** modelled; the API path is kubectl patch.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5086	agones fleet buffer size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5087	agones fleet min replicas	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 60 decisions
V5-5088	agones fleet max replicas	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5089	game server tick rate	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5090	players per server cap	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5091	matchmaking admission rate	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5092	matchmaking wait window	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5093	relay server pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
V5-5094	voice chat mixer pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5095	game node pool autoscaler	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5096	replay render batch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
V5-5097	cloud gaming gpu clocks	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global

G.3.19 Blockchain & Ledger Nodes (10 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `storage` (block, file or object storage), reached through the storage system's QoS and tiering API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the node's configuration (cache sizes, peer limits, mempool) and its host's power profile.
- **Wire out (read):** block relay latency, mempool size, peer counts, validation backlog (the node's RPC).
- **Wire in (write):** cache and pool sizes through the node's config and RPC; the host's power profile through family 8.
- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5098	node db cache size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			needs at once		
V5-5099	mempool max size	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5100	max peer connections	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
V5-5101	script validation threads	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5102	block relay admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5103	rpc work queue depth	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5104	utxo cache flush interval	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5105	indexer batch workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5106	archive node storage tier	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
V5-5107	validator host power profile	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global

G.3.20 CI/CD & Build Farms (10 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

Wire it on your stack.

- **Native controller:** the runner scale set (actions-runner-controller, GitLab runners) and the build cache.
- **Wire out (read):** jobs queued, runners busy, cache hit rate (the controller's metrics).
- **Wire in (write):** min and max runners, build concurrency and cache size by kubectl patch of the scale set.
- **Restore:** the scale set spec recorded; restored.
- **Run live?** modelled; the API path is kubectl patch.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5108	runner scale set min runners	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5109	runner scale set max runners	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 60 decisions
V5-5110	build concurrency limit	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5111	build cache size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	global
V5-5112	test shard parallelism	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
V5-5113	job queue admission	admission	at its limit on native (nothing paced or turned away): read and left native	Compute; stack; tower	global
V5-5114	artifact retention ttl	setpoint	left native: native already holds it inside its band (nothing to win)	Compute; stack; tower	global
V5-5115	build node cpu frequency	power	at its limit on native (the full clock): read and left native	Compute; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5116	remote execution workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 70% full; reads the last 20 decisions
V5-5117	warm runner pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Compute; stack; tower	machines started at 75% full; reads the last 40 decisions

G.4 Realm: Physics / Robotics / Autonomous (137 muscles of its own, 404 in its organism with the spine)

G.4.1 Automotive EV & Mobile Powertrain (14 muscles)

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `ev_traction` (an electric vehicle traction motor), reached through the vehicle's motor controller (CAN; modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

Wire it on your stack.

- **Native controller:** the vehicle's motor and battery controllers (CAN, the OEM's diagnostic interface).
- **Wire out (read):** torque, speed, state of charge, temperatures (CAN signals).
- **Wire in (write):** torque, regeneration and charge limits within the OEM's envelope.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1083	state of charge limits	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
1084	smart charging schedule	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
AUDIT-0045	traction torque limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
AUDIT-0046	regen braking level	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
AUDIT-0047	battery charge limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
AUDIT-0048	battery discharge limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
AUDIT-0049	battery thermal target	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
AUDIT-0050	motor thermal limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
AUDIT-0051	vehicle speed envelope	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
AUDIT-0052	energy recovery target	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
AUDIT-0053	auxiliary power budget	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0054	fast charge current	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
AUDIT-0055	fast charge voltage	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
AUDIT-0056	vehicle safe state	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global

G.4.2 Aviation & Autonomous Flight (28 muscles)

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `flight_axis` (a flight-control axis), reached through the flight controller (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

Wire it on your stack.

- **Native controller:** the flight controller's own parameters: PX4 (MPC_XY_VEL_MAX, MPC_Z_VEL_MAX_UP, MPC_ACC_HOR, MC_ROLLRATE_MAX) and ArduPilot (WPNAV_SPEED, ATC_ACCEL_R_MAX), the battery failsafe.
- **Wire out (read):** mission progress, battery state, tracking error and near-miss distance over MAVLink telemetry.
- **Wire in (write):** the speed, acceleration and rate parameters over MAVLink PARAM_SET within the autopilot's own limits; failsafes and geofence never loosened.
- **Restore:** every parameter read (PARAM_REQUEST_READ) once and written back at the end.
- **Run live?** modelled: the swarm cells (results/live/V3_SWARM.md); the MAVLink path is written, not yet flown.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
481	throttle envelope	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
482	attitude target	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
483	attitude rate target	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
484	velocity target	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
485	altitude target	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
486	waypoint authority	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
487	flight hold	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
488	return to home	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
489	land action	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
490	mission admission	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
491	geofence response	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
492	failsafe selection	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
493	battery reserve threshold	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
494	actuator saturation envelope	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
495	flight mode transition	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
496	flight termination safe state	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
1079	attitude rate gains	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
1080	horizontal speed limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
1081	vertical speed limits	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1082	acceleration jerk limits	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
V5-5143	px4 mpc xy vel max	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
V5-5144	px4 mpc z vel max up	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
V5-5145	px4 mpc acc hor	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
V5-5146	px4 mc rollrate max	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
V5-5147	ardupilot wpnav speed	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
V5-5148	ardupilot atc accel r max	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
V5-5149	drone battery failsafe reserve	setpoint	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Physics; stack; tower	1.5 wobbles of margin on its line

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5150	swarm formation spacing	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global

G.4.3 Robotics Fleet & Warehouse Automation (17 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `robot_fleet` (a fleet of robots or vehicles taking tasks), reached through the fleet manager's dispatch API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Physics / Robotics / Autonomous.

Wire it on your stack.

- **Native controller:** the fleet manager's dispatch and charging rules (Open-RMF, the vendor fleet manager).
- **Wire out (read):** tasks waiting, robots active and charging, battery states (the fleet API).
- **Wire in (write):** active robot count, charge thresholds and zone admission through the fleet API.
- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
449	robot dispatch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global
450	task assignment	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
451	traffic reservation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global
452	robot route	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global
453	charging dispatch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global
454	battery reserve	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global
455	elevator request	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global
456	door request	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			than native runs; hands back what it no longer needs at once		
457	conveyor speed	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global
458	agv speed	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global
459	warehouse zone admission	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
460	robot quarantine	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
461	fleet failover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global
462	human safe stop	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
463	fleet concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
V5-5151	amr fleet charge threshold	setpoint	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Physics; stack; tower	1.5 wobbles of margin on its line
V5-5152	amr fleet active robots	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Physics; stack; tower	global

G.4.4 Robotics Motion Control (26 muscles)

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `robot_joint` (a robot joint servo), reached through the robot controller (ROS 2, EtherCAT, the drive's fieldbus). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

Wire it on your stack.

- **Native controller:** the robot's own controller: ros2_control and MoveIt parameters, the drive's limits over EtherCAT or the vendor fieldbus.
- **Wire out (read):** joint tracking error, effort, temperature, cycle time (joint_states, the drive's registers).
- **Wire in (write):** velocity and acceleration scaling, trajectory tolerances and controller rates through ROS 2 parameters (ros2 param set) or the drive's limit registers; never past the manufacturer's limits.
- **Restore:** every parameter read once (ros2 param get) and restored (ros2 param set).
- **Run live?** modelled: MuJoCo (results/live/V3_MUJOCO.md); the ROS 2 path is written, not yet run on a physical arm.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
433	joint position	capacity	its own peak speed while tasks wait, back to its everyday	Physics; stack;	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	tower	
434	joint velocity	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
435	joint acceleration	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
436	joint effort	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
437	cartesian velocity	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
438	trajectory speed	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
439	trajectory acceleration	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
440	jerk limit	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
441	collision margin	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
442	force limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
443	gripper force	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
444	locomotion speed	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
445	steering angle	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			(alone; inside a body, its everyday speed)		
446	braking force	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
447	balance correction	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
1076	trajectory tolerances	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
1077	velocity acceleration scaling	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
1078	controller update rate	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
V5-5135	ros2 executor threads	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native	Physics; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			runs; hands back what it no longer needs at once		
V5-5136	ros2 qos history depth	setpoint	left native: native already holds it inside its band (nothing to win)	Physics; stack; tower	global
V5-5137	ros2 control loop rate	setpoint	left native: native already holds it inside its band (nothing to win)	Physics; stack; tower	global
V5-5138	humanoid balance margin	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
V5-5139	humanoid gait speed	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
V5-5140	humanoid joint torque limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
V5-5141	humanoid step height	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
V5-5142	manipulator payload speed scaling	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			everyday speed)		

G.4.5 Spacecraft & Flight Software (14 muscles)

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `reaction_wheel` (a spacecraft reaction wheel), reached through the attitude controller (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

Wire it on your stack.

- **Native controller:** the flight software's command and mode tables (cFS, F Prime).
- **Wire out (read):** attitude error, wheel speeds, power and thermal state from telemetry.
- **Wire in (write):** payload duty cycle, communication allocation and wheel allocation through commands; safe modes never commanded.
- **Restore:** the command table recorded; restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
497	space command admission	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
499	flight task schedule	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
500	space mode transition	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
501	payload duty cycle	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
502	communication allocation	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
503	space power budget	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
504	space thermal command	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
505	attitude command envelope	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
506	reaction wheel allocation	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
507	rcs authority	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
508	safe mode transition	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
509	watchdog recovery	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
510	instrument activation	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
511	fault isolation	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global

G.4.6 Rail Traction & Train Control (14 muscles)

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `rail_traction` (a train's traction and auxiliary systems), reached through the train control and management system and ATO (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

Wire it on your stack.

- **Native controller:** the train control and management system and ATO.
- **Wire out (read):** speed, traction effort, timetable margin, auxiliary power (TCMS telemetry).
- **Wire in (write):** traction and coasting targets within the ATO's envelope.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2054	traction effort limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2055	regenerative braking share	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2056	coasting speed target	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
2057	dwell time schedule	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2058	hvac duty cycle	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2059	train auxiliary power budget	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2060	platform approach speed	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
2061	acceleration rate setpoint	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
2062	wheel slip protection mode	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2063	traction motor thermal derate	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2064	catenary voltage limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2065	timetable recovery margin	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2066	headway target	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
2067	door release hold	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

G.4.7 Marine Propulsion & Vessel Automation (12 muscles)

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `marine_propulsion` (a ship's propulsion shaft and power plant), reached through the vessel automation and power management system (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

Wire it on your stack.

- **Native controller:** the vessel's automation and power management system.
- **Wire out (read):** shaft speed and power, fuel rate, engine load (the automation system).
- **Wire in (write):** shaft speed and power limits, engine staging within the manufacturer's envelope.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2068	shaft speed target	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
2069	propeller pitch limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2070	shaft power limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2071	bow thruster duty	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2072	ballast pump mode	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2073	engine load sharing setpoint	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
2074	slow steaming speed target	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2075	auxiliary engine staging	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
2076	shore power mode	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2077	hotel load budget	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2078	trim target	capacity	its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed)	Physics; stack; tower	global
2079	rudder rate limit	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global

G.4.8 Elevators & Vertical Transport (12 muscles)

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `elevator_hoist` (an elevator or escalator drive and its group control), reached through the lift controller and destination dispatch (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

Wire it on your stack.

- **Native controller:** the lift controller and destination dispatch.
- **Wire out (read):** calls waiting, car positions, drive temperature (the controller).
- **Wire in (write):** speed and dispatch modes within the manufacturer's limits; safety modes never written.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2146	hoist speed target	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2147	acceleration limit	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
2148	regen drive mode	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2149	standby power mode	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2150	destination dispatch schedule	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2151	car parking mode	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2152	door dwell hold	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2153	escalator speed target	capacity	at its limit on native (its everyday speed is the machine's limit): read and left native	Physics; stack; tower	global
2154	group capacity mode	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2155	motor thermal derate	power	at its limit on native (full torque): read and left native	Physics; stack; tower	global
2156	brake test schedule	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global
2157	fire recall mode	admission	at its limit on native (nothing paced or turned away): read and left native	Physics; stack; tower	global

G.5 Realm: Energy / Facility / Industrial (213 muscles of its own, 480 in its organism with the spine)

G.5.1 Building & Critical Environment HVAC (15 muscles)

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `building` (a building zone), reached through the building management system. Omni-Compass reads its temperature against its limit. Its home realm is Energy / Facility / Industrial.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Wire it on your stack.

- **Native controller:** the building management system (BACnet) and the AHU and zone controllers.
- **Wire out (read):** zone temperatures, airflows, duct static pressure, occupancy (BACnet reads).
- **Wire in (write):** zone setpoints, fan speeds and damper positions within their bands by BACnet write; the BMS's own sequences keep control.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled: CityLearn districts (results/live/V3_CITYLEARN.md); the BACnet path is written.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1094	setpoint reset trim and respond	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
1095	duct static pressure setpoint	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
1096	optimal start stop	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	Energy; stack; tower	global
AUDIT-0081	zone temperature target	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
AUDIT-0082	zone airflow	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	Energy; stack; tower	global
AUDIT-0083	ahu fan speed	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	Energy; stack; tower	global
AUDIT-0084	damper position	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	Energy; stack; tower	global
AUDIT-0085	economizer position	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0086	boiler setpoint	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
AUDIT-0087	heat pump mode	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0088	humidity target	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
AUDIT-0089	occupancy ventilation	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0090	building demand limit	power	at its limit on native (the site envelope at full): read and left native	Energy; stack; tower	global
AUDIT-0091	thermal storage dispatch	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	Energy; stack; tower	global
AUDIT-0092	hvac emergency mode	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

G.5.2 Energy Storage & Microgrid (19 muscles)

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `microgrid` (a microgrid battery and loads), reached through the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR). Omni-Compass reads its draw on the grid connection and its reserve. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the battery's or site's energy management system and inverter (SunSpec, IEEE 2030.5, OpenADR, Modbus).
- **Wire out (read):** state of charge, grid import and export, PV output, load (the EMS or inverter registers).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Wire in (write):** the reserve setpoint, charge and discharge limits, import limit and curtailment through SunSpec or Modbus registers.
- **Restore:** registers read once and restored.
- **Run live?** modelled: CityLearn (results/live/V3_CITYLEARN.md) for the districts; the register path is written.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
513	battery charge power	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 10 decisions before it holds; keeps 0% of its line clear
514	battery discharge power	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear
515	battery soc reserve	setpoint	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	0.75 wobbles of margin on its line
516	grid import limit	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	global
517	grid export limit	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 5% of its line clear
518	pv curtailment	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear
519	ev charge power	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
520	heat pump power	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear
521	electrolyzer power	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 5 decisions before it holds; keeps 0% of its line clear
522	microgrid demand limit	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	global
523	peak shaving	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	0 wobbles of margin on its line
524	time of use schedule	setpoint	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	1 wobbles of margin on its line
525	energy load shed	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
526	flex load admission	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
527	storage dispatch	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	0 wobbles of margin on its line

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
528	microgrid emergency reserve	setpoint	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	1 wobbles of margin on its line
1085	volt var	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	0 wobbles of margin on its line
1086	volt watt	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	0 wobbles of margin on its line
1087	constant power factor reactive power	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	global

G.5.3 Facility & Grid Optimization (15 muscles)

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `facility` (a facility battery behind the meter), reached through the energy management system. Omni-Compass reads its draw on the grid connection and its reserve. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the facility's energy management system and demand-response gateway (OpenADR).
- **Wire out (read):** site power, utility signals, PUE, carbon intensity (the EMS and the utility's feed).
- **Wire in (write):** the site power budget, demand limit and battery dispatch through the EMS.
- **Restore:** recorded; restored.
- **Run live?** modelled.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
561	facility power budget	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear
562	utility demand limit	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear
563	demand response	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
564	electricity price gate	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
565	carbon intensity gate	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
566	renewable dispatch	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	0 wobbles of margin on its line
567	generator start stop	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	0 wobbles of margin on its line
568	site battery dispatch	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	0 wobbles of margin on its line

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
569	pue target	setpoint	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	1 wobbles of margin on its line
570	cooling power budget	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear
571	it power budget	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear
572	rack power allocation	power	at its limit on native (full battery power): read and left native	Energy; stack; tower	global
573	facility peak guard	capacity	holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor	Energy; stack; tower	0 wobbles of margin on its line
574	grid frequency response	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
575	facility islanding	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

G.5.4 Grid Transmission & Distribution (12 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `feeder_voltage` (a distribution feeder's voltage), reached through the distribution management system (IEC 61850, DNP3; modelled only). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Wire it on your stack.

- **Native controller:** the distribution management system and the devices' own controllers (IEC 61850, DNP3): regulators, capacitor banks, inverters.
- **Wire out (read):** feeder voltages, loads, tap positions, inverter output (the DMS).
- **Wire in (write):** regulator and transformer taps, capacitor switching and inverter reactive power within the planning envelope; protection never written.
- **Restore:** every setting read once and restored.
- **Run live?** modelled: SimBench grids (results/live/V3_PANDAPOWER.md).

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0033	capacitor bank switch	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0034	voltage regulator tap	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0035	transformer tap	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0036	inverter real power	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0037	inverter reactive power	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0038	feeder voltage target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0039	feeder load transfer	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0040	distribution storage dispatch	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0041	demand response dispatch	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0042	frequency droop setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0043	grid protection mode	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0044	grid restoration sequence	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

G.5.5 Industrial PLC & Process Automation (18 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the PLC or DCS loop (OPC UA, EtherNet/IP, PROFINET): the setpoint and the mode of each loop.
- **Wire out (read):** the process value, setpoint, output and alarm state of each loop (OPC UA reads).
- **Wire in (write):** the loop's setpoint within its safe band and its actuator range by OPC UA write; safety interlocks are never written.
- **Restore:** every setpoint read once and restored; a value outside the band is refused by the plug.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
465	plc cycle authority	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
466	machine cell admission	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
467	valve position	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
468	pump flow	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
469	compressor speed	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
470	heater power	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
471	furnace setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
472	pressure setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
473	temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
474	mass flow setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
475	tank level target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
476	conveyor rate	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
477	feed rate	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
478	purge vent action	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
1088	controller mode	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
1089	alarm limits	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1090	safety interlock trip	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
1091	opc ua writes	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global

G.5.6 Semiconductor Fab & Precision Manufacturing (13 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `chamber` (a controlled chamber (clean room, kiln, reactor)), reached through the PLC or DCS. Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the fab's MES and each tool's controller (SECS/GEM).
- **Wire out (read):** tool queues, chamber states, process values (MES and GEM).
- **Wire in (write):** lot dispatch and tool admission through the MES; chamber recipes and parameters never written outside the qualified window.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1092	run to run control	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
1093	idle sleep mode	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0057	tool job dispatch	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0058	wafer route	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0059	chamber recipe selection	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0060	chamber temperature	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0061	chamber pressure	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0062	gas flow	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0063	rf power	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0064	vacuum pump speed	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0065	robot transfer rate	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0066	lot priority	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0068	tool quarantine	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

G.5.7 Water Wastewater & Pumping (12 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `water` (a water or pumping process), reached through the SCADA system (Modbus, DNP3). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Wire it on your stack.

- **Native controller:** the utility's SCADA (Modbus, DNP3) and the pump station controllers.
- **Wire out (read):** levels, pressures, flows, aeration and dosing (SCADA reads).
- **Wire in (write):** pump speeds, level and pressure targets, aeration and dosing within their bands by SCADA write; emergency shutdowns never written.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0069	pump speed water	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0070	valve position water	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0071	reservoir level target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0072	line pressure target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0073	flow target water	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
AUDIT-0074	aeration rate	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0075	chemical dose rate	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
AUDIT-0076	filtration backwash	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0077	lift station dispatch	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0078	leak isolation	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0079	water demand shed	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
AUDIT-0080	water emergency shutdown	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

G.5.8 Healthcare Critical Environments (13 muscles)

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `hospital` (a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites)), reached through the hospital's building management system (BACnet). Omni-Compass reads its temperature against its limit. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the hospital's building management system (BACnet) with its clinical limits.
- **Wire out (read):** room pressures, air changes, temperatures and humidity (BACnet reads).
- **Wire in (write):** setpoints within the clinical envelope by BACnet write; pressure cascades and alarms never loosened.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2001	operating room air change setpoint	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2002	isolation room pressure target	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
2003	patient room temperature setpoint	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
2004	surgical suite humidity target	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
2005	ahu supply air temperature	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
2006	pharmacy cold room setpoint	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
2007	sterile storage humidity setpoint	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Energy; stack; tower	global
2008	imaging suite cooling capacity	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	Energy; stack; tower	global
2009	chiller plant staging	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	Energy; stack; tower	global
2010	exhaust fan capacity	capacity	left to its own staging (native already has the cooling it needs: nothing to win)	Energy; stack; tower	global
2011	ward night setback mode	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2012	medical gas plant demand limit	power	at its limit on native (the site envelope at full): read and left native	Energy; stack; tower	global
2013	emergency power load shed	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

G.5.9 Agriculture & Irrigation (14 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `irrigation` (pumps, pressure and climate on a farm or in a greenhouse), reached through the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the pump station controller, pivot panel or climate computer (Modbus, the vendor cloud).
- **Wire out (read):** pressure, soil moisture, temperature, flow (the controller's registers).
- **Wire in (write):** pump speed and climate setpoints within their bands by Modbus write.
- **Restore:** registers read once and restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2026	irrigation pump speed	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2027	mainline pressure setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2028	soil moisture target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2029	pivot speed setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2030	fertigation dose	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2031	greenhouse temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2032	greenhouse co2 target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2033	vent position setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2034	grain dryer temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2035	cold storage temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2036	barn ventilation rate	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2037	milking vacuum level setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2038	well drawdown level target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2039	drip zone dispatch	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

G.5.10 Oil & Gas Pipelines (14 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `pipeline` (a pipeline segment with its compressors, pumps and valves), reached through pipeline SCADA (DNP3, Modbus, OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** pipeline SCADA (DNP3, Modbus, OPC UA).
- **Wire out (read):** pressures, flows, line pack, compressor states (SCADA reads).
- **Wire in (write):** pressure and flow setpoints and unit staging within the safe envelope; protection never written.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2040	compressor discharge pressure setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2041	pump station suction pressure target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2042	line pack target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2043	pipeline flow setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2044	compressor unit staging	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2045	vfd pump speed	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2046	terminal tank level target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2047	leak detection shutdown	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2048	batch interface dispatch	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2049	heater outlet temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2050	drag reducing agent dose	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2051	valve position target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2052	cathodic protection voltage	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2053	pressure protection priority	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

G.5.11 Mining & Mineral Processing (14 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `mill` (a grinding, flotation or materials-handling circuit), reached through the plant DCS or PLC (OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the plant DCS or PLC (OPC UA).
- **Wire out (read):** mill load, speed, pressures, densities, power (OPC UA reads).
- **Wire in (write):** loop setpoints within their bands by OPC UA write; shutdowns never written.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2092	sag mill load setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2093	mill speed target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2094	crusher gap setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2095	flotation aeration rate	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2096	cyclone feed pressure target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2097	thickener underflow density target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2098	reagent dose	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2099	conveyor speed setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2100	dewatering pump level target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2101	ventilation on demand airflow	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2102	stockpile level target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2103	slurry pump speed	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2104	tailings discharge shutdown	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2105	ore blend dispatch	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

G.5.12 District Heating & Cooling (13 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `district_heat` (a district heating or cooling network), reached through the network control system and substation controllers (Modbus, M-Bus). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the network control system and substation controllers (Modbus, M-Bus).
- **Wire out (read):** supply and return temperatures, differential pressure, storage level (the controllers).
- **Wire in (write):** supply temperature and pressure setpoints, pump speed and staging within their bands.
- **Restore:** registers read once and restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2106	supply temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2107	return temperature target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2108	differential pressure setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2109	network pump speed	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2110	heat pump staging	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2111	chp dispatch	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2112	thermal storage level target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2113	peak boiler heater output	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2114	substation flow limit	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2115	peak demand shed	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2116	outdoor reset setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2117	cooling network supply temperature	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2118	cooling storage level target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global

G.5.13 Power Generation & Turbine Control (14 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `turbine` (a generating unit's governor, boiler or excitation loop), reached through the plant DCS and governor (OPC UA, IEC 61850; modelled only). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the plant DCS and the unit's governor (OPC UA, IEC 61850).
- **Wire out (read):** unit load, frequency, pressures, temperatures (the DCS).
- **Wire in (write):** load setpoints and ramp limits within the dispatch envelope; protection and rod positions never written.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2119	turbine speed droop setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2120	unit load setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2121	boiler steam pressure setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2122	feedwater level target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2123	agc participation limit	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2124	excitation voltage setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2125	hydro gate position target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2126	combustion air ratio setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2127	cooling water flow setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2128	inlet guide vane position	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2129	reserve dispatch priority	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2130	turbine ramp rate limit	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2131	emissions shutdown	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2132	nuclear rod position target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global

G.5.14 Renewable Generation & Inverter Control (13 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `inverter` (a plant of grid-support inverters or wind turbines), reached through the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547).
- **Wire out (read):** active and reactive power, voltage, curtailment state (SunSpec registers).
- **Wire in (write):** volt-var and volt-watt setpoints, curtailment and ramp limits by SunSpec write within the interconnection agreement.
- **Restore:** registers read once and restored.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Run live?** modelled: SimBench grids solved by pandapower (results/live/V3_PANDAPOWER.md); the register path is written.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2133	inverter volt var setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2134	inverter volt watt setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2135	active power curtailment	power	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2136	plant reactive power target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2137	wind turbine yaw offset	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2138	pitch angle limit	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2139	rotor speed setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2140	inverter frequency droop setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2141	ramp rate limit	capacity	at its limit on native (the full actuator): read and left native	Energy; stack; tower	global
2142	tracker stow mode	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2143	string mppt voltage setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2144	noise mode schedule	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2145	ice detection shutdown	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

G.5.15 Pharmaceutical & Food Manufacturing (14 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `batch_reactor` (a GMP batch reactor, fermenter or food process), reached through the batch control system (ISA-88, OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

Wire it on your stack.

- **Native controller:** the batch control system (ISA-88, OPC UA) with the validated recipe limits.
- **Wire out (read):** process values and phase states (OPC UA reads).
- **Wire in (write):** setpoints within the validated recipe band only; a value outside it is refused by the plug.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2158	reactor temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2159	agitator speed setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2160	fermenter dissolved oxygen target	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2161	ph setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2162	pasteurizer holding temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2163	freezer tunnel temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2164	cip cycle dispatch	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2165	cleanroom pressure cascade setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2166	lyophilizer shelf temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2167	chromatography flow setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2168	steam sterilizer cycle priority	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2169	oven zone temperature setpoint	setpoint	left native: native already holds it inside its band (nothing to win)	Energy; stack; tower	global
2170	refrigerant compressor authority	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global
2171	batch hold quarantine	admission	at its limit on native (nothing paced or turned away): read and left native	Energy; stack; tower	global

G.6 Realm: Distribution / Specialized (217 muscles of its own, 484 in its organism with the spine)

G.6.1 Cache & Memory Services (16 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the cache's own configuration: Redis CONFIG SET, Memcached runtime settings.
- **Wire out (read):** hit rate, evictions, used memory, p95 (INFO, the cache's stats).
- **Wire in (write):** maxmemory and eviction policy by CONFIG SET; replicas and shards through the operator.
- **Restore:** the setting read once (CONFIG GET) before the first write and restored (CONFIG SET).
- **Run live?** live: tools/run_redis.py, V3_REDIS.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
305	cache size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native	Distribution; stack; tower	machines started at 75% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			runs; hands back what it no longer needs at once		
306	cache ttl	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
307	cache eviction policy	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
308	cache replicas	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
309	cache sharding	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
310	cache prefetch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
311	cache writeback rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
312	cache admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
313	hot key isolation	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
314	cache connection limit	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
315	cache memory limit	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
316	cache compression	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
317	cache warmup	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			start, never fewer than native runs; hands back what it no longer needs at once		
318	cache failover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 60 decisions
319	cache flush rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
1106	memory size threads	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions

G.6.2 Commerce & Payment Systems (15 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `commerce` (a customer-facing service with bursts), reached through the Kubernetes API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the payment platform's gateway and its routing rules.
- **Wire out (read):** authorisation latency, decline and retry rates, queue depth (the platform's metrics).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Wire in (write):** concurrency, timeout and route weights through the platform's admin API.
- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
401	payment admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
402	payment concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
403	payment retry	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
404	payment timeout	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
405	fraud review gate	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
406	authorization route	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
407	processor route weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
408	transaction queue limit	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
409	idempotency window	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
410	order reservation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
411	inventory hold	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
412	checkout load shed	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
413	refund queue rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
414	settlement batch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			start, never fewer than native runs; hands back what it no longer needs at once		
415	payment failover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 60 decisions

G.6.3 Data Analytics & ETL (15 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** Spark's dynamic allocation and the SQL engine's slots (spark.dynamicAllocation, Flink's parallelism, warehouse slots).
- **Wire out (read):** executor count, task backlog, shuffle spill, stage times (the engine's metrics).
- **Wire in (write):** dynamic allocation min and max, parallelism and slots through the engine's config at submit or its REST API.
- **Restore:** the submit settings recorded; the next job launched from them.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
338	executor size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			longer needs at once		
339	dynamic allocation min	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
340	dynamic allocation max	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
341	shuffle partitions	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
342	shuffle bandwidth	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
343	etl concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
344	stage parallelism	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
345	query slots	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 85% full; reads the last 10 decisions
346	spill threshold	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
347	cache fraction	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 80% full; reads the last 20 decisions
348	batch interval	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 80% full; reads the last 20 decisions
349	stream backpressure	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
350	data locality wait	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 80% full; reads the last 20 decisions
351	speculation policy	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 75% full; reads the last 10 decisions
352	analytics admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

G.6.4 Database & Transactions (19 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `database` (a database cluster), reached through the database operator (replicas, connection pools). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the database's own pooler and server settings: PgBouncer's admin console, MySQL SET GLOBAL, MongoDB setParameter, the database operator.
- **Wire out (read):** pool waits and active connections (SHOW POOLS), buffer-pool size and hit rate, cache size and evictions, p95 per statement.
- **Wire in (write):** PgBouncer default_pool_size through its console; MySQL innodb_buffer_pool_size by SET GLOBAL; MongoDB wiredTigerEngineRuntimeConfig cache_size by setParameter; replicas through the operator.
- **Restore:** the setting read once before the first write and restored; a failed read-back stops the writer.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Run live?** live: tools/run_pgbench.py, run_sysbench.py, run_ycsb.py, three-run tables V3_PGBENCH, V3_SYSBENCH, V3_YCSB.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
273	db replicas	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 60 decisions
275	db memory	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
276	db cache	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 90% full; reads the last 40 decisions
277	query concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
278	read route	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
279	shard placement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 75% full; reads the last 40 decisions
280	db failover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
281	replication lag gate	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
282	db write throttle	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
283	db pool resize	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 75% full; reads the last 40 decisions
284	transaction concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
285	lock timeout	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
286	checkpoint rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
287	vacuum compaction rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
1099	parallel workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 60 decisions
1100	background writer	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
1101	synchronous replication	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
1102	buffer pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 75% full; reads the last 40 decisions
1103	io capacity	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions

G.6.5 Messaging & Streaming (22 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the broker's own controls: Kafka's admin API and consumer group, RabbitMQ policies, MQTT broker settings.
- **Wire out (read):** consumer lag, messages per second, p95 end to end, broker CPU (the broker's metrics, the consumer group's offsets).
- **Wire in (write):** the consumer group's size (consumers running), quotas and partition counts through the admin API.
- **Restore:** the group's size recorded; restored at the end.
- **Run live?** live: tools/run_kafka.py, V3_KAFKA.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
289	partition count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
290	partition placement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
291	producer quota	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
292	consumer quota	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
293	broker io quota	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
294	message retention	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
295	queue depth limit	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
296	consumer concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
297	producer batch size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
298	fetch batch size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
299	rebalance rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
300	replication factor	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 75% full; reads the last 10 decisions
301	retry backoff	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
302	dead letter divert	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
303	stream priority	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
304	broker failover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
1104	prefetch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
1105	memory disk alarm	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
V5-5124	mqtt broker max connections	capacity	starts the machines the forecast needs before the rush lands and clears work waiting	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			in line within one machine start, never fewer than native runs; hands back what it no longer needs at once		
V5-5125	mqtt max inflight messages	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global
V5-5126	mqtt session expiry	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global
V5-5127	broker cluster nodes	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions

G.6.6 Runtime & Application (21 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the application runtime's own settings: thread pools, JVM heap and GC, worker counts, connection pools.
- **Wire out (read):** queue waits, GC pauses, worker busy share, p95 from the application's metrics.
- **Wire in (write):** worker and pool sizes through the application's admin endpoint, environment or config reload; JVM flags at restart only.
- **Restore:** the settings recorded; restored (a restart-bound setting is left native).
- **Run live?** modelled.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
353	worker count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 75% full; reads the last 10 decisions
354	thread pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
355	jvm heap	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
356	gc budget	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
357	connection pool runtime	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
358	application cache size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			in line within one machine start, never fewer than native runs; hands back what it no longer needs at once		
359	runtime memory	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 60 decisions
360	async concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
361	event loop workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 80% full; reads the last 20 decisions
362	process count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
363	request timeout	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
364	background workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
365	runtime cpu budget	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
366	runtime io budget	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
367	runtime restart	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
1108	go runtime	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 60 decisions
1109	keepalive connection reuse	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 60 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5131	dns resolver cache size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
V5-5132	dns negative cache ttl	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global
V5-5133	mail queue concurrency	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5134	mail retry interval	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global

G.6.7 Search, Indexing & Vector DB (16 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the search engine's settings API: OpenSearch/Elasticsearch cluster and index settings, the vector database's config.
- **Wire out (read):** indexing and search latency, queue rejections, merge pressure (the engine's stats API).
- **Wire in (write):** index refresh interval, merge throttling, shard counts and search thread pools through the settings API.
- **Restore:** settings recorded; restored.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
321	index workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 75% full; reads the last 20 decisions
322	index refresh rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
323	segment merge rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
324	search concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
325	search timeout	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
326	shard count	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			longer needs at once		
327	shard replication	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
328	shard rebalance	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
329	vector search k	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
330	vector batch size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
331	embedding workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			runs; hands back what it no longer needs at once		
332	index memory budget	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
333	query route	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 60 decisions
334	hot shard isolation	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
335	search admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
1107	circuit breakers	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

G.6.8 Service Mesh & API Reliability (16 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the mesh's own policies: Istio DestinationRule and VirtualService, Linkerd ServiceProfile, Envoy filters.
- **Wire out (read):** request rate, error rate, p95 and outlier ejections from the mesh's telemetry.
- **Wire in (write):** connection pool, outlier detection, retry budget and traffic weights by kubectl patch of the policy objects.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Restore:** the policy objects recorded; restored.
- **Run live?** modelled; the API path is kubectl patch.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
241	circuit breaker	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
242	retry budget	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
243	service timeout	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
244	service concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
245	connection pool	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
246	traffic divert	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
247	traffic mirror	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			longer needs at once		
248	canary weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
249	outlier ejection	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
250	health threshold	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
251	dns traffic weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
252	session affinity	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions
253	request hedging	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			longer needs at once		
254	fault injection gate	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
255	service failover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
1097	load balancing policy	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions

G.6.9 Telecom RAN & Edge Radio (12 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `ran` (a radio access network cell or site), reached through the RAN controller (O-RAN interfaces). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the RAN's near-real-time RIC and O-RAN interfaces (E2, O1).
- **Wire out (read):** PRB use, connected UEs, cell load, handover rates (the RIC's KPIs).
- **Wire in (write):** scheduler weights, slice budgets and traffic steering through the RIC's xApps; transmit power within the licence.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0009	ran connection admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
AUDIT-0010	ran ue handover	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
AUDIT-0011	ran cell traffic steering	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
AUDIT-0012	ran slice resource budget	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
AUDIT-0013	ran prb allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
AUDIT-0014	ran scheduler weight	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
AUDIT-0015	ran tx power	power	at its limit on native (the full clock): read and left native	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
AUDIT-0016	ran antenna tilt	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
AUDIT-0017	ran carrier enable	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
AUDIT-0018	ran cell sleep	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
AUDIT-0019	ran du cu placement	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
AUDIT-0020	ran fronthaul budget	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

G.6.10 Workflow, Logistics & Fulfillment (15 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `workflow` (a workflow or pipeline engine), reached through the workflow engine's concurrency settings. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the workflow engine (Temporal, Airflow, Camunda) and the fulfilment system's dispatch rules.
- **Wire out (read):** task queue depth, worker utilisation, SLA misses (the engine's metrics).
- **Wire in (write):** worker rates, concurrency and retry policies through the engine's API.
- **Restore:** recorded; restored.
- **Run live?** modelled.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
417	workflow admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
418	workflow worker rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
419	task queue rate	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
420	workflow retry	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
421	workflow backoff	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
422	workflow timeout	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
423	inventory allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
424	fulfillment route	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
425	warehouse queue	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
426	carrier selection	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
427	dispatch priority	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
428	shipment batch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			in line within one machine start, never fewer than native runs; hands back what it no longer needs at once		
429	route replan	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
430	sla escalation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
431	compensation action	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 40 decisions

G.6.11 Medical Imaging & Clinical Systems (12 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `clinical` (clinical systems: imaging archives, records, interface engines, monitoring gateways), reached through the application's own scaling and admission settings (Kubernetes, the vendor console). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the clinical application's own scaling and admission settings (Kubernetes, the vendor console).
- **Wire out (read):** queue depths, interface backlog, reconstruction wait (the application's metrics).
- **Wire in (write):** replicas, concurrency and rate limits through the Kubernetes API or the vendor console.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2014	pacs archive tier target	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
2015	dicom router concurrency	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2016	ehr application replicas	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
2017	hl7 interface queue limit	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2018	fhir api rate limit	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2019	reconstruction gpu workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
2020	modality worklist timeout	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2021	patient monitoring gateway capacity	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2022	lab analyzer batch window	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2023	telehealth session admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2024	clinical backup window	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2025	imaging prefetch priority	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

G.6.12 Ports & Maritime Logistics (12 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `port` (a container terminal's cranes and vehicles taking moves), reached through the terminal operating system and fleet controllers. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the terminal operating system and the equipment fleet controllers.
- **Wire out (read):** moves waiting, cranes active, gate queue, equipment charge (the TOS).
- **Wire in (write):** crane and vehicle allocation, gate rate and charging windows through the TOS.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
2080	quay crane allocation	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
2081	yard crane fleet size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
2082	berth window admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2083	truck gate rate limit	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2084	horizontal transport fleet size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
2085	reefer plug power budget	power	at its limit on native (the full clock): read and left native	Distribution; stack; tower	global
2086	shore power connection capacity	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs;	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
			hands back what it no longer needs at once		
2087	rail mounted gantry dispatch	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
2088	container dwell priority	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2089	vessel arrival pacing	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
2090	stacking height target	setpoint	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
2091	equipment charging window	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

G.6.13 Content Delivery & Edge Cache (6 muscles)

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the CDN's configuration API and the edge cache's settings.
- **Wire out (read):** hit ratio, origin load, TTLs (the CDN's metrics).
- **Wire in (write):** cache TTLs, origin-shield concurrency and prefetch through the CDN API.
- **Restore:** recorded; restored.
- **Run live?** modelled.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5039	cdn cache ttl	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global
V5-5040	edge cache size	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
V5-5041	origin shield concurrency	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
V5-5042	prefetch aggressiveness	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global
V5-5043	tiered cache promotion	capacity	at its limit on native (the full actuator): read and left native	Distribution; stack; tower	global
V5-5044	edge pop admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

G.6.14 Financial Markets & Trading Systems (14 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the trading platform's gateway, matching engine and risk settings, the colocation BMS.
- **Wire out (read):** order latency, session counts, risk-check backlog, market-data fanout (the platform's metrics).
- **Wire in (write):** thread and session pools, conflation intervals and batch sizes through the platform's configuration; risk checks never relaxed.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Restore:** recorded; restored.
- **Run live?** modelled only.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5058	matching engine threads	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5059	order gateway sessions	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions
V5-5060	order admission rate limit	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
V5-5061	pre trade risk check workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global
V5-5062	market data fanout threads	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 10 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5063	market data conflation interval	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global
V5-5064	fix session heartbeat interval	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global
V5-5065	kernel bypass poll cores	power	at its limit on native (the full clock): read and left native	Distribution; stack; tower	global
V5-5066	tick store write batch	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global
V5-5067	post trade settlement workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 80% full; reads the last 20 decisions
V5-5068	end of day risk batch power	power	at its limit on native (the full clock): read and left native	Distribution; stack; tower	global
V5-5069	colocation rack cooling setpoint	setpoint	cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native	Distribution; stack; tower	global
V5-5070	backtest grid nodes	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5071	surveillance alert queue workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global

G.6.15 Reliability, Security & Recovery (6 muscles)

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

Wire it on your stack.

- **Native controller:** the platform's own recovery and security controls: rollout and rollback, PodDisruptionBudgets, the security hold, a kill switch.
- **Wire out (read):** error rate, rollout status, the security hold (a ConfigMap key), the kill file.
- **Wire in (write):** rollout pause and undo by kubectl; the security hold read, never written; the kill switch (tools/omni_switch.py) is a human's.
- **Restore:** every write recorded and undone on the lease.
- **Run live?** live in part: the security hold and kill switch run in every Kubernetes benchmark (omnicompass/master.py).

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5118	ids capture threads	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	global

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

#	Muscle (the setting's own name)	Knob	What Omni-Compass does with it	Organisms	Its own card
V5-5119	ids ring buffer size	setpoint	left native: native already holds it inside its band (nothing to win)	Distribution; stack; tower	global
V5-5120	ids rule group admission	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global
V5-5121	waf inspection workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 70% full; reads the last 20 decisions
V5-5122	siem ingest pipeline workers	capacity	starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once	Distribution; stack; tower	machines started at 80% full; reads the last 10 decisions
V5-5123	endpoint scan scheduling	admission	at its limit on native (nothing paced or turned away): read and left native	Distribution; stack; tower	global

APPENDIX

Appendix H. Source of the Engine

The engine that stands, omni-v23, is the 65 files named in `OMNI_V23.json` (digest c96ea4105d8747db): 60 Python files and 5 tables. The Python files are printed here in full from the repository, in the fingerprint's order, so that the book and the code can be read side by side and nothing in the mechanism is withheld from the owner: the Kubernetes governor, its muscles, the replica room, the quota budget and the two card governors (`omni_controller/`); the engine core, the compass law, the compass, the verdict, the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

conveyance law, the plug adapter, the nervous system, the master switch and the rest of the library (`omnicompass/`); the modelled plants, the harness, the presets, the max's arm and the card's model (`realms/`); and the three runners the fingerprint covers (`tools/`). The tables are the muscle catalogs: `realms/catalog_v5.csv` (the 1,107 muscles, printed muscle by muscle in the Muscles appendix with every card of `realms/muscle_cards_v5.csv`) and the earlier catalogs kept for their results (`realms/catalog.csv`, `realms/catalog_v1.csv`, `realms/wave4_families.csv`). `python3 tools/omni_version.py` hashes every one of the 65 and prints omni-v23 when none has changed; `verify.py` fails if a byte of them differs from the fingerprint.

H.1 `omni_controller/__init__.py`

5 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
```

H.2 `omni_controller/capacity.py`

126 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Capacity guards for shared-host Kubernetes clusters.

The kind runner is one physical host even when Kubernetes reports seve
    ↪ ral nodes.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
Sample the host's aggregate CPU counters and limit additional replicas
    ↪  to measured
headroom; an unavailable reading fails closed at the current replica c
    ↪ ount.
"""
from __future__ import annotations

import os
from pathlib import Path

KIND_SCALE_OUT_BUSY_LIMIT = 0.75

def one_host(nodes) -> bool:
    """Nodes that are containers on one machine (kind: providerID kind
    ↪ ://...): each reports the whole host's CPU as its
    own, so the cluster's real CPU is one host's, not the sum (the rep
    ↪ lica room on kind, results/live/REPLICA_ROOM.md:

    the cap raised to 30 served no more requests and ran 78% more pods
    ↪ )."""
    return bool(nodes) and all(str((n.get("spec") or {}).get("provider
    ↪ ID", "")).startswith("kind://") for n in nodes)

class HostCpuSampler:
    """Measure aggregate host CPU busy share from Linux /proc/stat, ov
    ↪ er the last `span` intervals between samples
    (omni-v16: the governor samples at each decision and reads the las
    ↪ t minute's share at any decision period; span 1, the
    share since the previous sample, is what every engine before omni-

    ↪ v16 read)."""

    def __init__(self, stat_path: str | os.PathLike[str] | None = None
    ↪ , span: int = 1):
        self.stat_path = Path(stat_path or os.environ.get("OMNI_HOST_P
    ↪ ROC_STAT", "/proc/stat"))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.span = max(1, int(span))
        self._readings: list[tuple[int, int]] = []      # the last spa
    ↪ n + 1 valid readings, oldest first
        self.busy_share: float | None = None

    def sample(self) -> float | None:

        """Update the busy share over the last span intervals (fewer a
    ↪ t the start), or None without two valid samples."""
        try:
            line = next(row for row in self.stat_path.read_text().spli
    ↪ tlines() if row.startswith("cpu "))
            fields = [int(value) for value in line.split()[1:]]

            if len(fields) < 4:
                self._readings = []
                self.busy_share = None
                return None
            total = sum(fields)
            idle = fields[3] + (fields[4] if len(fields) > 4 else 0)

            current = (total, idle)
        except (OSError, StopIteration, ValueError):
            self._readings = []
            self.busy_share = None
            return None

        self._readings = (self._readings + [current])[-(self.span + 1)
    ↪ :]
        previous = self._readings[0] if len(self._readings) > 1 else N
    ↪ one
        if previous is None:
            self.busy_share = None

            return None
        dt, di = current[0] - previous[0], current[1] - previous[1]
        if dt <= 0 or di < 0 or di > dt:
            self.busy_share = None
            return None
        self.busy_share = (dt - di) / dt
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return self.busy_share


def kind_replica_ceiling(
    allocatable_cpu_m: int,
    other_requested_cpu_m: int,

    pod_request_cpu_m: int,
    current_replicas: int,
    host_busy_share: float | None,
    reserve_share: float = 0.10,
    max_busy_share: float = KIND_SCALE_OUT_BUSY_LIMIT,
) -> int:

    """Return a total replica ceiling for kind's single shared host.


    Keep the existing Kubernetes-request ceiling as a hard upper bound
    ↪ . The
    host observation then limits *new* replicas to the CPU left below
    ↪ the

    configured reserve. If a host reading is unavailable, do not autho
    ↪ rize
    scale-out on an unmeasured shared machine.
    """
    alloc = max(0, int(allocatable_cpu_m))
    other = max(0, int(other_requested_cpu_m))

    request = int(pod_request_cpu_m)
    current = max(0, int(current_replicas))
    if request <= 0 or alloc <= 0:
        return current


    request_ceiling = max(1, (alloc - other) // request - 1)

    request_ceiling = max(current, request_ceiling)
    if host_busy_share is None:
        return current
    if not 0.0 <= host_busy_share <= 1.0:
        return current
    # A request-based headroom calculation can still authorize more po
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ds on a
    # host whose measured CPU is already under sustained pressure. At
    ↪ that
    # point, fail closed: do not expand the HPA cap until the host is
    ↪ below the
    # threshold. The threshold is a conservative starting guard for ki

    ↪ nd and
    # must be validated against the paired live workload before releas
    ↪ e.
    if host_busy_share >= min(1.0, max(0.0, float(max_busy_share))):
        return current

    reserve = min(0.95, max(0.0, float(reserve_share)))
    headroom_m = max(0, int(alloc * max(0.0, round(1.0 - reserve - hos
    ↪ t_busy_share, 12))))
    measured_ceiling = current + headroom_m // request
    return max(current, min(request_ceiling, measured_ceiling))

def bounded_replica_cap(
    operator_max: int,
    current_max: int,
    current_replicas: int,
    live_ceiling: int | None,

) -> int:
    """Recheck an Omni-raised HPA cap without setting it below live re
    ↪ plicas.

    If the live ceiling cannot be read, restore the operator's origina
    ↪ l cap;

    Kubernetes must still be allowed to keep its currently running rep
    ↪ licas.
    """
    original = max(1, int(operator_max))
    cap = max(original, int(current_max))
    running = max(0, int(current_replicas))

    measured = original if live_ceiling is None else max(0, int(live_c
    ↪ eiling))
    return max(original, running, min(cap, measured))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

H.3 omni_controller/controller.py

1748 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass live controller for a real Kubernetes cluster.

Modes
  observe   read the cluster every interval, run the fleet-mode govern

    ↪ or in OBSERVE, log every decision; no writes (default)
  target    additionally patch each HPA's CPU averageUtilization to rh
    ↪ o* (bounded, rate-limited); the Cluster Autoscaler and
            all node management remain unchanged
  nodepool  additionally size one node pool to the governor's recommen
    ↪ dation through --node-scale-cmd, a command template

            such as "aws autoscaling set-desired-capacity --auto-scali
    ↪ ng-group-name POOL --desired-capacity {n}"; the
            Cluster Autoscaler must not manage that pool
Laws (--law)
  governor  (default) the engine's allocation law sets the HPA target
    ↪ (rho*) and the closure law or the governor sizes the pool

  compass      the compass law (omnicompass/compass_law.py) holds the
    ↪ service in its band, read as the GPU compass reads it: the mean
            response time of the latency window between the bare servi
    ↪ ce time (a tenth of --slo-ms) and --slo-ms. At the
            max (--dial max, the default) the balance point stands at
    ↪ --compass-center (0.3: well inside the line, so every
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            rush finds its pods already there) and no machine the clus
    ↪ ter had on its own is ever given back; at the
            tune-down (--dial save) it is held at the v5 edge (omnicom
    ↪ pass/edge.py: the balance point moves with the
            service's own risk, 1 - (climb x reaction + 3.09 x wobble)
    ↪ , never below 0.4 unless a breach while machines were

            given back has taught it otherwise) and the verdict below
    ↪ gives machines back; p95 at or past the SLO, a blind
            sense or pods waiting for a place read as past the wall. I
    ↪ ts push and pull move two levers: each HPA's CPU target inside i
    ↪ ts cover (from 60% of the
            operator's target up to the operator's own, never tighter

    ↪ than native, so it only ever adds pods and gives them
            back) and the node pool (one machine back only while the f
    ↪ orce is clearly down, the position below the center
            and the nervous system's release gate open; past the 0.95
    ↪ wall, one machine up at once). The verdict
            (omnicompass/verdict.py, stepwise) decides how many machin

    ↪ es may be given back at all: while the service is
            calm, one more machine is given back on trial and the resp
    ↪ onse time of the requests served without it is set
            against the requests served just before and against the cl
    ↪ uster as it ran on its own at the start; at most
            --allow slower and that machine stays given back, slower t

    ↪ han that and it is taken back and not tried again
            for --verdict-recheck decisions. Where no machine passes,
    ↪ the pool stays as the cluster runs it alone. The
            six-state engine still runs every decision: it grants the
    ↪ authority, feeds the release gate and the compass,
            and is audited

Safety
  every node action passes through the shield (bounds, step limit); th
    ↪ e recommendation never falls below what the CPU
  requests of running and pending pods, or current usage, need, and ca
    ↪ pacity required by that floor is added in one step
  (the step limit applies only to the governor's own adjustments); --d
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ry-run logs intended writes without executing them; creating the
    ↪  kill file (or
  setting OMNI_KILL=1) restores every HPA target this controller chang
    ↪ ed, runs --node-restore-cmd if given, and returns to
  observe; every decision and action
  is appended to the audit log.

Requires kubectl on PATH with access to the cluster (metrics-server fo
    ↪ r kubectl top).
"""
from __future__ import annotations

import argparse, csv, json, math, os, re, shlex, subprocess, sys, time

from pathlib import Path
from types import SimpleNamespace

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from omnicompass import master

from omnicompass.adapter import Governor, mode_law, OBSERVE, REF_PERIO
    ↪ D_S
from omnicompass.shield import enforce, ShieldLimits
from omni_controller.muscles import Muscles, add_args as add_muscle_ar
    ↪ gs, run_operator_cmd, RESTORE_TRIES
from omni_controller.capacity import HostCpuSampler, bounded_replica_c

    ↪ ap, kind_replica_ceiling, one_host

ANNOTATION = "omnicompass.io/original-target-utilization"
RELEASE_PERIOD_S = 60.0   # the pool gives back at most one machine a
    ↪ minute (v7: a rate in time, not in decisions)
ADD_PERIOD_S = 60.0       # omni-v16: and adds at most --max-node-step

    ↪  machines a minute past the scheduling floor
LAND_S = 60.0             # omni-v16: a command is given a minute to l
    ↪ and before proprioception lets it go
COMPASS_UP, COMPASS_DOWN, COMPASS_RELEASE = 0.10, 0.02, -0.2   # the c
    ↪ ompass's gains, the same on every muscle (realms/compass_arm.py)
DEMAND_RISE = 0.05     # the demand moves: more than 5% between its le
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ast and its most over one HPA window
RANGE_ANN = "omnicompass.io/original-replica-range"
# the pod muscle of its own (Omni-Compass by itself, docs/SOLO_PREREGI
    ↪ STRATION.md): the operator names a workload no autoscaler
# governs (--own-replicas) and its target and range on the workload it
    ↪ self; the count it shipped is recorded at the first write

OWN_REPL_ANN = "omnicompass.io/original-own-replicas"
OWN_TARGET_ANN = "omnicompass.io/target-utilization"
OWN_RANGE_ANN = "omnicompass.io/replica-range"
# omni-v23 (the founder's order of 31 October: every muscle Omni-Compa
    ↪ ss cannot be stays; a knob left with no working controller
# fails the whole system): a write of the workload's own scale the clu

    ↪ ster refuses this many times in a row hands the knob back
# to native's own pod autoscaler, made from the operator's own target
    ↪ and range recorded on the workload
OWN_REFUSALS = 3

def to_milli(v: str) -> float:
    v = str(v).strip()
    if v.endswith("m"):
        return float(v[:-1])
    if v.endswith("n"):
        return float(v[:-1]) / 1e6

    return float(v) * 1000.0



# the reads the controller makes, as API paths (group/version, plural,
    ↪  namespaced): through one `kubectl proxy` started
# once (the same identity as every kubectl call), a read costs a local

    ↪  HTTP request instead of a new kubectl process
REST = {"nodes": ("api/v1", "nodes", False), "node": ("api/v1", "nodes
    ↪ ", False),
        "pods": ("api/v1", "pods", True), "pod": ("api/v1", "pods", Tr
    ↪ ue),
        "configmap": ("api/v1", "configmaps", True), "configmaps": ("a
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ pi/v1", "configmaps", True),
        "resourcequota": ("api/v1", "resourcequotas", True),
        "deployment": ("apis/apps/v1", "deployments", True), "deployme
    ↪ nts": ("apis/apps/v1", "deployments", True),
        "statefulset": ("apis/apps/v1", "statefulsets", True),
        "jobs": ("apis/batch/v1", "jobs", True), "job": ("apis/batch/v
    ↪ 1", "jobs", True),
        "hpa": ("apis/autoscaling/v2", "horizontalpodautoscalers", Tru
    ↪ e)}


def milli_cpu(q):
    """A metrics-API CPU quantity (n, u, m or cores) as kubectl top pr
    ↪ ints it: millicores."""
    q = str(q)
    if q.endswith("n"):
        return f"{int(int(q[:-1]) / 1e6)}m"
    if q.endswith("u"):
        return f"{int(int(q[:-1]) / 1e3)}m"
    if q.endswith("m"):
        return q
    return f"{int(float(q) * 1000)}m"

class Kube:
    def __init__(self, kubectl: str = "kubectl", dry_run: bool = False
    ↪ , audit=None, proxy: bool = False):
        self.kubectl, self.dry_run, self.audit = kubectl, dry_run, aud
    ↪ it
        self.base = None
        if proxy:
            self._start_proxy()

    def _start_proxy(self):
        import atexit
        p = subprocess.Popen([self.kubectl, "proxy", "--port=0"], stdo
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ut=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
        line = p.stdout.readline()                # "Starting to serv
    ↪ e on 127.0.0.1:PORT"
        if "serve on" not in line:
            p.kill()
            return                                # no proxy: every r

    ↪ ead stays a kubectl call, as before
        self.base = "http://" + line.strip().split()[-1]
        self.proxy_proc = p
        atexit.register(p.terminate)


    def _rest(self, args):

        """The read through the proxy, in kubectl's own output shape,
    ↪ or None when it is not one the proxy serves."""
        import urllib.parse, urllib.request, urllib.error
        a = list(args)
        if not a:
            return None

        verb, rest = a[0], a[1:]
        if verb not in ("get", "top") or not rest:
            return None
        kind, rest = rest[0], rest[1:]
        ns, name, sel, fsel, out, allns = None, None, None, None, None
    ↪ , False

        i = 0
        while i < len(rest):
            t = rest[i]
            if t == "-A":
                allns = True
            elif t == "-n":

                ns = rest[i + 1]; i += 1
            elif t == "-l":
                sel = rest[i + 1]; i += 1
            elif t.startswith("--field-selector="):
                fsel = t.split("=", 1)[1]
            elif t == "-o":
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            out = rest[i + 1]; i += 1
        elif t == "--no-headers":
            pass
        elif t.startswith("-"):
            return None                          # a flag the proxy
    ↪ path does not translate: use kubectl
        elif name is None:
            name = t
        else:
            return None
        i += 1
    if verb == "top":
        if kind not in ("nodes", "pods") or name:
            return None
        path = "apis/metrics.k8s.io/v1beta1/" + ("nodes" if kind =
    ↪ = "nodes" else (f"namespaces/{ns}/pods" if ns else "pods"))
    else:
        if kind not in REST or out not in ("json", "name"):
            return None
        group, plural, namespaced = REST[kind]
        path = f"{group}/" + (f"namespaces/{ns or 'default'}/" if
    ↪ namespaced and not allns else "") + plural
        if name:
            path += "/" + name
    q = {k: v for k, v in (("labelSelector", sel), ("fieldSelector
    ↪ ", fsel)) if v}
    url = f"{self.base}/{path}" + ("?" + urllib.parse.urlencode(q)
    ↪  if q else "")
    try:
        with urllib.request.urlopen(url, timeout=30) as r:
            body = json.loads(r.read())
    except urllib.error.HTTPError as e:
        raise subprocess.CalledProcessError(1, [self.kubectl, *arg
    ↪ s], stderr=f"API {e.code} for {path}")
    if verb == "top":
        if kind == "nodes":
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                return "".join(f"{it['metadata']['name']} {milli_cpu(i
    ↪ t['usage']['cpu'])} 0% 0Mi 0%\n" for it in body.get("items", [])
    ↪ )
            return "".join(f"{it['metadata']['name']} {milli_cpu(sum(i
    ↪ nt(milli_cpu(c['usage']['cpu'])[:-1]) for c in it['containers'])
    ↪  / 1000)} 0Mi\n"
                           for it in body.get("items", []))
        if out == "name":
            single = REST[kind][1][:-1]
            return "".join(f"{single}/{it['metadata']['name']}\n" for
    ↪ it in body.get("items", [body] if name else []))
        return body

    def get(self, *args):
        if self.base:
            r = self._rest(args)
            if r is not None:
                return r

        out = subprocess.run([self.kubectl, *args], capture_output=Tru
    ↪ e, text=True, check=True).stdout
        return json.loads(out) if "-o" in args and "json" in args else
    ↪  out

    def write(self, args, why):

        self.audit({"write": [self.kubectl, *args], "why": why, "dry_r
    ↪ un": self.dry_run})
        if not self.dry_run:
            subprocess.run([self.kubectl, *args], capture_output=True,
    ↪  text=True, check=True)

IDLE_TAINT = "omnicompass.io/idle"   # PreferNoSchedule: new pods go e
    ↪ lsewhere first, but land here at once if they must


def closed(n):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    """A machine I have closed to new work: marked idle (PreferNoSched
    ↪ ule) or cordoned."""
    spec = n.get("spec", {})
    return bool(spec.get("unschedulable")) or any(t.get("key") == IDLE
    ↪ _TAINT for t in spec.get("taints") or [])

def schedulable(n):
    """Open to new work: not closed by me, and not tainted NoSchedule
    ↪ (the control plane)."""
    spec = n.get("spec", {})
    return not closed(n) and not any(t.get("effect") == "NoSchedule" f
    ↪ or t in spec.get("taints") or [])


CP_ROLES = ("node-role.kubernetes.io/control-plane", "node-role.kubern
    ↪ etes.io/master")

def is_ready(n):
    return any(c["type"] == "Ready" and c["status"] == "True" for c in
    ↪  n["status"].get("conditions", []))

def worker(n):
    """A machine the services' pods may run on: not the control plane
    ↪ (its role label, or its role's NoSchedule taint)."""
    labels = (n.get("metadata", {}) or {}).get("labels", {}) or {}
    if any(r in labels for r in CP_ROLES):
        return False
    return not any(t.get("key") in CP_ROLES and t.get("effect") == "No
    ↪ Schedule" for t in (n.get("spec", {}) or {}).get("taints") or []
    ↪ )


UNBOUNDED = sys.maxsize   # no top but what the system itself refuses
    ↪ (the founder's rule of 13 October: from 1 up, unbounded)


def apply_give_back_only(a):
    """The cloud's rule in one word (v7, --give-back-only; the founder
    ↪ 's order of 14 October, night): the governor moves
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    the machines alone. No autoscaler moved (no service sensed), no pr
    ↪ ocessor conveyed, no power or clock capped, no pod
    resized, started or held, no replica cap raised, the tune-down dia
    ↪ l; machines are given back only after their own
    trial and woken on a breach, so the pods are exactly native's and
    ↪ the bill can only fall or stay. It is enforced here,

    in the governor, whatever else the command line says."""
    if not getattr(a, "give_back_only", False):
        return
    a.sensed = "none/none"
    for k in ("cap_deployments", "rightsize_deployments", "coldstart_d
    ↪ eployments", "power_cmd", "rapl_cmd", "cpufreq_cmd",

              "gpu_power_cmd", "cooling_cmd", "contain_namespaces"):
        if hasattr(a, k):
            setattr(a, k, "")
    for k in ("batch", "batch_pace", "strict_replicas", "pod_reflex_wr
    ↪ ites"):
        if hasattr(a, k):

            setattr(a, k, False)
    a.replica_ceiling = 0
    a.dial = "save"
    if getattr(a, "mode", "observe") == "target":
        a.mode = "nodepool"

# the muscles the operator can leave native, one by one (omni-v14; the
    ↪  founder's order of 20 October: "knobs on every monitor
# to turn off and on such a mechanism, a muscle"). A muscle named here
    ↪  is never written by the governor: it stays where native
# runs it, and every other muscle runs as before. The engine decides w

    ↪ ith the whole body in view either way.
MUSCLES = {"machines": "the node pool's size (the operator's node-scal
    ↪ e command)",
           "replicas": "the pod autoscaler's target, the pod reflex's
    ↪ replica floor and the replica cap",
           "cpu": "the serving pods' CPU limits (the conveyance and th
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ e pod power cap)",
           "clock": "the processors' frequency ceiling", "gpu": "the G
    ↪ PU's power limit",
           "rollout": "the rollout guard (pause and undo)", "batch": "
    ↪ the batch admission and pacing",
           "rightsize": "the pods' rightsizing", "coldstart": "the col
    ↪ d-start reserve", "contain": "the namespaces' containment",
           "cooling": "the cooling setpoint"}


def native_muscles(text) -> frozenset:
    """The muscles left native, from --native (or OMNI_NATIVE): a comm
    ↪ a list of MUSCLES' names; an unknown name is refused,
    never ignored, so a muscle the operator meant to leave native is n
    ↪ ever driven by a misspelling."""
    names = frozenset(x.strip() for x in str(text or "").split(",") if
    ↪  x.strip())
    bad = sorted(names - set(MUSCLES))
    if bad:
        raise SystemExit(f"--native: unknown muscle {', '.join(bad)} (
    ↪ the muscles: {', '.join(MUSCLES)})")
    return names

def autoscaler_room(k):
    """The machines the cluster's own autoscaler may still add, read l
    ↪ ive from its status (kube-system/
    cluster-autoscaler-status: per node group, its maxSize less the ma
    ↪ chines registered); None where no autoscaler reports
    (kind, a fixed pool, a pool Omni resizes with the operator's own c
    ↪ ommand). The founder's rule of 13 October: a
    muscle's top is what the system itself can hold, read live."""
    try:
        text = str(((k.get("get", "configmap", "cluster-autoscaler-sta
    ↪ tus", "-n", "kube-system", "-o", "json") or {})
                    .get("data", {}) or {}).get("status", ""))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    except Exception:
        return None
    if not text.strip():
        return None
    room = 0
    for line in text.splitlines():

    ↪ # the plain format: one node group's health per line
        m = re.search(r"registered=(\d+).*?maxSize=(\d+)", line)
        if m:
            room += max(0, int(m.group(2)) - int(m.group(1)))
    if room == 0 and "maxSize:" in text:
    ↪ # the YAML format of newer autoscalers

        for block in re.split(r"\n\s*-\s*name:", text)[1:]:
            mx = re.search(r"maxSize:\s*(\d+)", block)
            reg = re.search(r"registered:\s*\n\s*total:\s*(\d+)", bloc
    ↪ k)
            if mx and reg:
                room += max(0, int(mx.group(1)) - int(reg.group(1)))

    return room


def snapshot(k: Kube, active_only: bool = False):
    nodes = k.get("get", "nodes", "-o", "json")["items"]
    ready = [n for n in nodes if is_ready(n)]

    pool = sum(1 for n in ready if worker(n))
    ↪ # every ready worker, open or closed: the lever's pool
    pods = k.get("get", "pods", "-A", "-o", "json")["items"]
    if active_only:
        # in service: open to new work, or still carrying work (an idl
    ↪ ing worker keeps serving until its work is gone)

        carrying = {p["spec"].get("nodeName") for p in pods if p["stat
    ↪ us"].get("phase") in ("Running", "Pending")
                    and all(o.get("kind") != "DaemonSet" for o in p.ge
    ↪ t("metadata", {}).get("ownerReferences", []) or [])}
        ready = [n for n in ready if schedulable(n) or (closed(n) and
    ↪ n["metadata"]["name"] in carrying)]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    # open: the machines I keep open to new work. My machine orders ar
    ↪ e counted here; a machine I closed that still
    # carries work stays in service (in the energy count) until its wo
    ↪ rk leaves on its own
    open_n = sum(1 for n in ready if schedulable(n))
    names = {n["metadata"]["name"] for n in ready}

    alloc = sum(to_milli(n["status"]["allocatable"]["cpu"]) for n in r
    ↪ eady)
    req = sum(to_milli(c.get("resources", {}).get("requests", {}).get(
    ↪ "cpu", "0")) for p in pods
              if p["status"].get("phase") in ("Running", "Pending")
              and (not active_only or p["status"].get("phase") == "Pen

    ↪ ding" or p["spec"].get("nodeName") in names)
              for c in p["spec"]["containers"])
    pending = sum(1 for p in pods if p["status"].get("phase") == "Pend
    ↪ ing")
    top = k.get("top", "nodes", "--no-headers")
    used = sum(to_milli(line.split()[1]) for line in top.strip().split

    ↪ lines()
               if line.strip() and (not active_only or line.split()[0]
    ↪  in names))
    hpas = k.get("get", "hpa", "-A", "-o", "json")["items"]
    return {"nodes": len(ready), "open": open_n if active_only else le
    ↪ n(ready), "alloc_m": alloc, "req_m": req, "used_m": used, "pendi

    ↪ ng": pending, "hpas": hpas,
            "pool": pool}


def cpu_target(h):
    for i, m in enumerate(h["spec"].get("metrics", [])):

        if m.get("type") == "Resource" and m["resource"]["name"] == "c
    ↪ pu" and "averageUtilization" in m["resource"]["target"]:
            return i, int(m["resource"]["target"]["averageUtilization"
    ↪ ])
    return None, None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class Controller:
    def __init__(self, a, kube: Kube | None = None):
        self.a = a
        apply_give_back_only(a)
        a.native_muscles = native_muscles(getattr(a, "native", ""))

    ↪      # omni-v14: the muscles left native
        if a.native_muscles:
            a.pod_reflex_writes = False if "replicas" in a.native_musc
    ↪ les else getattr(a, "pod_reflex_writes", False)
            a.strict_replicas = False if "replicas" in a.native_muscle
    ↪ s else getattr(a, "strict_replicas", False)

            a.replica_ceiling = 0 if "replicas" in a.native_muscles el
    ↪ se getattr(a, "replica_ceiling", -1)
        # v7: every rule of the governor counts in seconds; its count
    ↪ of decisions follows from the decision period, so
        # the period can change without changing any rule's time (at -
    ↪ -interval 60 every count is the one v6 used)

        # --interval 0 is a model's or a test's loop that does not sle
    ↪ ep: each call is one decision of the shipped minute
        period = float(a.interval) if float(a.interval) > 0 else 60.0
        per = lambda sec: max(1, int(round(float(sec) / period)))
        pick = lambda n, sec: int(n) if n is not None else per(sec)
        self.n_slo_clear = pick(getattr(a, "slo_clear", None), getattr

    ↪ (a, "slo_clear_s", 180.0))
        # cruise waits for work that waits decision after decision: tw
    ↪ o readings at least, at any beat (omni-v17: at 90 and
        # 120 s its 120 s rounded to one decision, and a single readin
    ↪ g with a pod waiting put every machine in service)
        self.n_cruise_after = (int(a.cruise_after) if getattr(a, "crui

    ↪ se_after", None) is not None
                               else max(2, per(getattr(a, "cruise_afte
    ↪ r_s", 120.0)))) \
            if getattr(a, "cruise_after", None) != 0 else 0
        self.n_strict_window = pick(getattr(a, "strict_window", None),
    ↪  getattr(a, "strict_window_s", 300.0))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    self.n_verdict_every = pick(getattr(a, "verdict_every", None),
↪  getattr(a, "verdict_every_s", 600.0))
    self.n_verdict_recheck = pick(getattr(a, "verdict_recheck", No
↪ ne), getattr(a, "verdict_recheck_s", 7200.0))
    self.n_verdict_trial = per(getattr(a, "verdict_trial_s", 1200.
↪ 0))

    self.step_scale = period / 60.0               # the compass's
↪  gains are per minute: a decision moves its share of one
    self.n_release = per(RELEASE_PERIOD_S)        # decisions bet
↪ ween two machines given back (at most one a minute)
    # omni-v16: the rest of the governor's clocks in seconds as we
↪ ll, so the decision period changes only how often it

    # looks: the brain's minute, the compass's damping and smoothi
↪ ng, the edge, the additions, a command's time to
    # land, the host's busy share and the conveyance (at --interva
↪ l 60 every value is the one omni-v15 used)
    self.pace = period / REF_PERIOD_S             # one decision'
↪ s share of the brain's minute

    self.n_add = per(ADD_PERIOD_S)                # decisions bet
↪ ween two additions past the scheduling floor
    self.n_land = per(LAND_S)                     # decisions a c
↪ ommand is watched for landing
    self.n_convey = max(1, int(round(float(getattr(a, "convey_ever
↪ y_s", 15.0)) / float(getattr(a, "floor_interval", 15.0) or 15.0)

↪ )))
    # omni-v19: the pod muscle of its own keeps the pace of the co
↪ ntroller it stands in for (the pod autoscaler syncs every
    # 15 s), whatever the decision period: a reconciliation every
↪ --own-every-s on the fast checks, its window of fewer pods
    # counted in those reconciliations (the strict window's second

↪ s); the compass moves its target only in a decision
    self.n_own = max(1, int(round(float(getattr(a, "own_every_s",
↪ 15.0)) / float(getattr(a, "floor_interval", 15.0) or 15.0))))
    sw_s = float(a.strict_window) * period if getattr(a, "strict_w
↪ indow", None) is not None else float(getattr(a, "strict_window_s
↪ ", 300.0))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    self.n_own_window = max(1, int(round(sw_s / max(1e-9, float(ge
↪ tattr(a, "own_every_s", 15.0))))))
    self.own_checks = 0                             # fast checks t
↪ aken: the pod muscle of its own's clock between decisions
    self._own_last = None                           # (snapshot, ob
↪ servation, rho) of the last decision, for the checks between

    self.last_add = None                            # the decision
↪ at which the pool last added a machine
    self.cmd_at = {}                                # "nodes" or (n
↪ s, hpa) -> the decision its last command was written at
    self.checks = 0                                 # checks taken,
↪  a fast check or a decision: the conveyance's clock

    self.decisions = 0                              # decisions tak
↪ en, the governor's own clock for the rules above
    self.last_release = None                        # the decision
↪ at which the pool last gave a machine back
    self.log = open(a.audit, "a") if a.audit else None
    self.k = kube or Kube(a.kubectl, a.dry_run, self.audit, proxy=

↪ not getattr(a, "no_api_proxy", False))
    self.k.audit = self.audit
    # shared-host signal for kind (None means no scale-out proof),
↪  the busy share over the last minute (omni-v16)
    self.host_cpu = HostCpuSampler(span=per(60.0))
    self.g = Governor(law=mode_law("fleet"), period_s=period); sel

↪ f.g.set_mode(OBSERVE)
    # the machine organ's own engine view: fed with machine-attrib
↪ utable pressure only (pods waiting for a place)
    self.gn = Governor(law=mode_law("fleet"), period_s=period); se
↪ lf.gn.set_mode(OBSERVE)
    self.rec_n = None

    self.refused_at = None         # the first machine count the p
↪ ool refused (its top, read live), if any
    self.changed = {}
    self.target_at = {}            # (ns, hpa) -> when I last set
↪ its target
    self.nodes_restored = False
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.m = Muscles(self.k, a, self.audit)
        if a.native_muscles:
            self.audit({"native_muscles": sorted(a.native_muscles), "w
    ↪ hy": "left native by the operator (--native): never written"})
        self.cl = None
        if getattr(a, "closure", ""):

            from omnicompass.closure import ClosureLaw, ClosureNodes
            law = json.load(open(a.closure))
            law = law.get("setting", law).get("closure", law)
            law = {k: v for k, v in law.items() if k != "site"}
            # live decisions are 60 s apart, not simulator ticks: a ri
    ↪ se the release band absorbs over the release horizon

            # does not veto a release (derived from the band itself, n
    ↪ o new constant)
            from omnicompass.nervous_system import BAND
            law["rho_max"] = min(law.get("rho_max", 0.95), BAND[1])
    ↪   # the living band: no machine filled past 95%
            law.setdefault("turn_rise", law.get("delta_rel", -1.0) if

    ↪ law.get("delta_rel", -1.0) >= 0 else law.get("delta", 0.05))
            self.cl = ClosureNodes(ClosureLaw(**law))
        self.lp_hist = []
        self.cmd_nodes = None          # efferent record: the node cou
    ↪ nt last commanded (proprioception reads it back)
        self.cmd_hpa = {}              # efferent record: HPA targets

    ↪ last written
        self.pod_hold = {}             # (ns, hpa) -> (until, seconds)
    ↪ : the pod target held at native's after a pod waited (omni-v19)
        self.waited = False            # a fast check saw a pod waitin
    ↪ g for a place since the last decision (omni-v19)
        self.req_seen = []             # the requests on the machines

    ↪ at the last decisions, newest last (omni-v19)
        self.burst_m = 0.0             # the largest rise in requests
    ↪ seen inside two decisions: the immune memory (omni-v19)
        self.dem_seen = {}             # (ns, hpa) -> the service's de
    ↪ mand at the last decisions, newest last (omni-v19)
        self.immune = {}               # (ns, hpa) -> its knob's self-
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ knowledge (omnicompass/immune.py, omni-v21)
        self.dem_rise = {}            # (ns, hpa) -> the largest rise
    ↪  in its demand inside two decisions, as a ratio (omni-v19)
        self.hold_n = {}              # (ns, hpa) -> hand-backs in a
    ↪ row: each doubles the hold, up to two hours
        from omnicompass.compass import Compass

        gp = self.g.p                 # the compass reads the engine
    ↪ in the engine's own parameters
        self.compass = Compass(E_max=gp.E_max, alpha_s=gp.alpha_s, bet
    ↪ a_s=gp.beta_s, delta=gp.delta)
        self.s_floor = None           # bare service time (ms): the f
    ↪ astest a request is served with no queue ahead of it

        self.reflex = {}              # (ns, name) -> replica floor t
    ↪ he pod reflex holds while the queue says it is needed
        self.pod_cap = {}             # (ns, name) -> request / limit
    ↪  share: what the HPA's target means in queue terms
        self.compass_law = None
        self.edge = None

        self.dial = getattr(a, "dial", "max")
        if getattr(a, "law", "governor") == "compass":
            from omnicompass.compass_law import Band, CompassLaw
            # the max (the default dial): the service held well inside
    ↪  its line (the compass's balance point at
            # --compass-center, 0.3 at the max: more room than the lin

    ↪ e, so every rush finds pods already there), no
            # machine ever given back below what the cluster ran on it
    ↪ s own; the tune-down (--dial save): the v5 edge
            # moves the balance point toward the line and the verdict
    ↪ gives machines back where the response time allows
            center = getattr(a, "compass_center", None)

            if center is None:
                center = 0.3 if self.dial == "max" else 0.4
            # one decision every --interval s; the service follows a t
    ↪ arget in about one HPA sync plus a pod start (tau).
            # The damping and the velocity's smoothing are set for a d
    ↪ ecision a minute and kept in time at any period
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        # (omni-v16): kd is critical damping at the minute (compas
↪ s_law.py's rule with dt = 60 s; the compass's moves
        # are per minute, step_scale), and the smoothing 0.3 a min
↪ ute
        tau = getattr(a, "compass_tau", 60.0)
        self.compass_law = CompassLaw(Band(0.0, 1.0, center=center

↪ ), dt=period, tau=tau, kp=1.0,
                                      kd=max(0.0, 2.0 * math.sqrt(
↪ 1.0 * tau / REF_PERIOD_S) - 1.0) * REF_PERIOD_S,
                                      smooth=0.3 if self.pace == 1
↪ .0 else 0.3 ** self.pace)
        self.compass_law.kd *= 3.0                    # the same p

↪ ush as on every realm muscle (realms/compass_arm.py)
        # the v5 edge (the tune-down only): the knob's reaction is
↪  the time the service takes to follow it (one HPA
        # sync and a pod start, --compass-tau), in decisions; its
↪ window and rates per minute (omni-v16)
        from omnicompass.edge import Edge

        self.edge = Edge(react=tau / period, pace=self.pace) if se
↪ lf.dial == "save" else None
        self.edge_base = center
        from omnicompass.verdict import Verdict
        self.verdict = Verdict(tolerance=getattr(a, "allow", 0.02)
↪ , min_samples=getattr(a, "verdict_samples", 200),

                               probe_every=self.n_verdict_every, r
↪ echeck=self.n_verdict_recheck,
                               max_trial=self.n_verdict_trial, inc
↪ remental=True)
        # immune maturity (omni-v19): no give-back trial before th
↪ e governor has watched its organism for one recheck

        # period, so the immune memory has seen the organism's own
↪  rises before the first machine is given back
        self.verdict.last_probe = self.n_verdict_recheck - self.n_
↪ verdict_every
    self.n_native = None           # the machines the cluster ran
↪ on its own when Omni started (the verdict's step 0)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    self.cruise = False           # cruise (rule 7): work waiting
↪  for a place, so every machine in service until the line is empt
↪ y
    self.cruise_hits = 0; self.cruise_clear = 0
    self.lat_t = None             # elapsed_seconds of the last r
↪ equest the verdict has seen

    self.compass_law_x = {}               # (ns, name) -> the comp
↪ ass's continuous HPA target, before rounding
    self.own = [tuple(x.split("/", 1)) if "/" in x else ("default"
↪ , x) for x in filter(None, (getattr(a, "own_replicas", "") or ""
↪ ).split(","))]
    self.own_x = {}               # (ns, name) -> the compass's c

↪ ontinuous target for a workload of its own, before rounding
    self._own_F = 0.0             # the compass's force of the la
↪ st decision
    self.own_hist = {}            # (ns, name) -> the recommendat
↪ ions of the last window (fewer pods only to their highest)
    self.own_refused = {}         # (ns, name) -> writes of its s

↪ cale the cluster refused in a row (omni-v23)
    self.own_handed = set()       # (ns, name) handed back to nat
↪ ive's own autoscaler after OWN_REFUSALS refusals (omni-v23)
    self.demand = {}              # (ns, hpa) -> [(time, demand)]
↪ : the CPU its pods use, in units of one pod's request
    self.pinned = {}              # (ns, hpa) -> [time]: decision

↪ s at which the autoscaler stood at its replica cap


def _sensed(self, h):
    """Whether the probe senses the service this HPA scales (--sen
↪ sed ns/deployment,...; empty: every HPA, the
    single-service case). A muscle whose service I cannot sense st

↪ ays at the operator's own target: the probe's
    response time says nothing about it, and more pods for it comp
↪ ete for the machines of the service I do sense."""
    sel = [t.strip() for t in getattr(self.a, "sensed", "").split(
↪ ",") if t.strip()]
    ref = (h.get("spec", {}).get("scaleTargetRef", {}) or {}).get(
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ "name", "")
        return not sel or f'{h["metadata"]["namespace"]}/{ref}' in sel

    def _app_demand(self, h):
        """The demand on an HPA's service, read from the autoscaler's
    ↪ own status: its pods' mean CPU utilisation (of

        their request) times the pods running, i.e. the CPU used in un
    ↪ its of one pod's request. None if not reported."""
        st = h.get("status", {}) or {}
        for m in st.get("currentMetrics", []) or []:
            if m.get("type") == "Resource" and (m.get("resource") or {
    ↪ }).get("name") == "cpu":

                u = (m["resource"].get("current") or {}).get("averageU
    ↪ tilization")
                if u is not None:
                    return float(u) / 100.0 * int(st.get("currentRepli
    ↪ cas", 0) or 0)
        return None

    def _at_idle(self, h):
        """Whether an HPA's service stands at idle: at its least pods
    ↪ (spec.minReplicas), wanting no more, and running at
        no more than half its target utilisation. A served service is
    ↪ never at zero CPU (a health probe, a client's

        keep-alive), so idle is read from the autoscaler's own floor,
    ↪ not from a zero reading (amendment 9)."""
        lo = int((h.get("spec", {}) or {}).get("minReplicas", 1) or 1)
        st = h.get("status", {}) or {}
        cur, want = int(st.get("currentReplicas", 0) or 0), int(st.get
    ↪ ("desiredReplicas", 0) or 0)

        _, target = cpu_target(h)
        u = None
        for m in st.get("currentMetrics", []) or []:
            if m.get("type") == "Resource" and (m.get("resource") or {
    ↪ }).get("name") == "cpu":
                u = (m["resource"].get("current") or {}).get("averageU
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ tilization")
        return cur == lo and want <= lo and u is not None and target i
    ↪ s not None and float(u) <= 0.5 * float(target)

    def _rising(self, key, win):
        """Whether the demand is still growing: now above (1 + DEMAND_

    ↪ RISE) times its least over the last window.
        While it grows, the pods a breach called are still owed to it;
    ↪  handing the operator's target back would remove
        them and start them again on the next rise."""
        now = time.time(); h = self.demand.get(key, []); past = [d for
    ↪  t, d in h if now - t <= win]

        return bool(past) and h[-1][1] > (1.0 + DEMAND_RISE) * min(pas
    ↪ t)

    def _steady(self, key, win):
        """Whether the demand has held still for one whole window: sam
    ↪ ples spanning it, its most within (1 +

        DEMAND_RISE) of its least. The autoscaler itself removes pods
    ↪ only after its scale-down window; a raise of the
        target (fewer, larger pods) waits for the same, so pods are ne
    ↪ ver removed from a demand that is still moving
        and started again when it rises."""
        now = time.time(); h = [(t, d) for t, d in self.demand.get(key

    ↪ , []) if now - t <= win]
        # a pinned gauge is not a steady demand: while the autoscaler
    ↪ stands at its replica cap, every pod is as busy
        # as it can be and the reading cannot rise however much the lo
    ↪ ad does; a window that touched the cap is not
        # steady (the wandering test: a target raised at the cap was t

    ↪ aken back on the next climb, 1.5 pod starts a run)
        if any(now - t <= win for t in self.pinned.get(key, [])):
            return False
        return bool(h) and now - h[0][0] >= 0.9 * win and max(d for _,
    ↪  d in h) <= (1.0 + DEMAND_RISE) * min(d for _, d in h)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def _machine_room(self, pool):
        """The machines the system can still add to a pool of `pool` r
    ↪ eady workers, read live: up to the operator's own
        --max-nodes where given; else what the cluster's own autoscale
    ↪ r may add (its status); where Omni resizes the pool itself with
    ↪ the operator's command and no autoscaler

        reports, no top but the one the pool shows: one under the firs
    ↪ t count it refused (none refused: unbounded);
        otherwise none (a fixed pool)."""
        if (self.a.max_nodes or 0) > 0:
            return max(0, int(self.a.max_nodes) - pool)
    ↪    # the operator's own top

        room = autoscaler_room(self.k)
        if room is not None:
            return room
        if self.a.mode == "nodepool" and self.a.node_scale_cmd:
            return UNBOUNDED if self.refused_at is None else max(0, se
    ↪ lf.refused_at - 1 - pool)

        return 0


    def _max_nodes(self, s):
        """The machine lever's top, read live (the founder's rule of 1
    ↪ 3 October: from the floor up to what the system
        itself holds, never a number written in code): the operator's

    ↪ --max-nodes where given; else every ready worker
        the cluster has (the pool the lever opens and closes) plus the
    ↪  machines the system can still add."""
        if (self.a.max_nodes or 0) > 0:
            return int(self.a.max_nodes)
        if "_top" not in s:

    ↪     # read once a snapshot
            pool = int(s.get("pool", s["nodes"]))
            s["_top"] = max(int(self.a.min_nodes), min(UNBOUNDED, pool
    ↪  + self._machine_room(pool)))
        return s["_top"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def _pool_command(self, n, n_now, why, **extra):
        """One node-pool command (the operator's own, --node-scale-cmd
    ↪ ). A count the pool refuses is its top, read live:
        the lever stays one under it from then on, and the decision go
    ↪ es on (nothing else is held back by it)."""
        if "machines" in getattr(self.a, "native_muscles", ()):

            self.audit({"held": "machines", "why": f"{why}: the machin
    ↪ es are left native (--native); {n} not written"})
            return False
        cmd = self.a.node_scale_cmd.format(n=n)
        self.audit({"write": shlex.split(cmd), "why": why, "dry_run":
    ↪ self.a.dry_run, **extra})

        if self.a.dry_run:
            return True
        try:
            subprocess.run(shlex.split(cmd), check=True)
            return True
        except subprocess.CalledProcessError as e:

            if n > n_now:
                self.refused_at = n if self.refused_at is None else mi
    ↪ n(self.refused_at, n)
            self.audit({"refused": {"nodes": n, "now": n_now, "code":
    ↪ e.returncode}, "why": why,
                        "pool_top": None if self.refused_at is None el

    ↪ se self.refused_at - 1})
            return False


    def _pod_request_m(self, h):
        """One pod's CPU request (millicores) of the workload an HPA s
    ↪ cales, read from its Deployment; None where unread."""

        ref = (h.get("spec") or {}).get("scaleTargetRef") or {}
        if ref.get("kind") != "Deployment" or not ref.get("name"):
            return None
        try:
            dep = self.k.get("get", "deployment", ref["name"], "-n", h
    ↪ ["metadata"]["namespace"], "-o", "json")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        except Exception:
            return None
        tpl = dep["spec"]["template"]["spec"]
        req = sum(to_milli(c.get("resources", {}).get("requests", {}).
    ↪ get("cpu", "0")) for c in tpl.get("containers", []))
        return req if req > 0 else None

    def _placeable_target(self, h, s, lo_t, hi_t, rise=1.0):
        """The fail-up's target (omni-v19): the cover's floor lo_t whe
    ↪ re the machines in service can place the pods it asks
        for; else the lowest target whose pods they can place, ceil(10
    ↪ 0 dem / x) <= running + room, and the machines the rest

        need asked for now, so a pod is never asked for before there i
    ↪ s a machine to run it. Unread (no demand, no request,
        not the node-pool mode): the cover's floor, as before. rise is
    ↪  the immune memory, the largest rise in the service's
        demand seen inside two decisions: the pods asked for leave roo
    ↪ m for that rise times themselves, so the next rise of

        that size finds a place, as native's fewer pods do."""
        dem, req = self._app_demand(h), self._pod_request_m(h)
        if dem is None or not req or dem <= 0:
            return lo_t
        cur = int((h.get("status") or {}).get("currentReplicas", 0) or
    ↪  0)

        place = int((cur + max(0, int((s["alloc_m"] - s["req_m"]) // r
    ↪ eq))) / max(1.0, rise))
        if place <= 0:
            return hi_t
        fit = 100.0 * dem / place
        if fit <= lo_t:

            return lo_t
        short = int(math.ceil(100.0 * dem / lo_t - 1e-9)) - place
        if short > 0 and self.a.mode == "nodepool" and self.a.node_sca
    ↪ le_cmd and "machines" not in getattr(self.a, "native_muscles", (
    ↪ )):
            n = max(1, s["open"]); per_node = s["alloc_m"] / max(1, s[
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ "nodes"])
            need = min(self._max_nodes(s), int(math.ceil((s["req_m"] +
    ↪  short * req) * (1.0 + self.a.headroom) / per_node)))
            if need > n and need > (self.rec_n or 0) and self._pool_co
    ↪ mmand(need, n, "machines first: room for the pods the fail-up as
    ↪ ks (omni-v19)",

    ↪       pods_short=short):
                self.rec_n = need
        self.audit({"placeable": {"hpa": f"{h['metadata']['namespace']
    ↪ }/{h['metadata']['name']}", "target": round(min(hi_t, fit), 1),
                                  "asked": round(lo_t, 1), "pods_place

    ↪ able": place, "pods_short": max(0, short),
                                  "largest_rise": round(rise, 3)}})
        return min(hi_t, fit)

    def _live_ceiling(self, h):
        """The most replicas the machines can hold, one pod under it,

    ↪ read live: the allocatable CPU of every ready worker
        (closed ones included: the machine lever opens them) and of th
    ↪ e machines the system can still add, less the CPU
        every other pod asks for, in units of one of this service's po
    ↪ ds; where every node is a container on one machine
        (kind), that machine's CPU once. None where it cannot be read.

    ↪ """
        ref = (h.get("spec") or {}).get("scaleTargetRef") or {}
        ns = h["metadata"]["namespace"]
        if ref.get("kind") != "Deployment" or not ref.get("name"):
            return None
        try:

            dep = self.k.get("get", "deployment", ref["name"], "-n", n
    ↪ s, "-o", "json")
            nodes = [n for n in self.k.get("get", "nodes", "-o", "json
    ↪ ")["items"] if is_ready(n) and worker(n)]
            pods = self.k.get("get", "pods", "-A", "-o", "json")["item
    ↪ s"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        except Exception:
            return None
        tpl = dep["spec"]["template"]["spec"]
        req = sum(to_milli(c.get("resources", {}).get("requests", {}).
    ↪ get("cpu", "0")) for c in tpl.get("containers", []))
        if req <= 0 or not nodes:

            return None
        shared_host = one_host(nodes)
        if shared_host:
            # kind nodes are containers on this machine: count its CPU
    ↪  once, then use /proc/stat busy share
            # so host-side load generators and processes outside Kuber

    ↪ netes consume the same capacity budget.
            alloc = max(to_milli(n["status"]["allocatable"]["cpu"]) fo
    ↪ r n in nodes)
        else:
            alloc = sum(to_milli(n["status"]["allocatable"]["cpu"]) fo
    ↪ r n in nodes)

            alloc += self._machine_room(len(nodes)) * alloc / len(node
    ↪ s)
        sel = (dep["spec"].get("selector") or {}).get("matchLabels") o
    ↪ r {}
        names = {n["metadata"]["name"] for n in nodes}
        mine = lambda p: p["metadata"].get("namespace") == ns and sel

    ↪ and all((p["metadata"].get("labels") or {}).get(k_) == v for k_,
    ↪  v in sel.items())
        other = sum(to_milli(c.get("resources", {}).get("requests", {}
    ↪ ).get("cpu", "0")) for p in pods
                    if p["status"].get("phase") in ("Running", "Pendin
    ↪ g") and not mine(p)

                    and (p["spec"].get("nodeName") in names or p["stat
    ↪ us"].get("phase") == "Pending")
                    for c in p["spec"]["containers"])
        if shared_host:
            current = int((h.get("status") or {}).get("currentReplicas
    ↪ ", 0) or 0)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return kind_replica_ceiling(alloc, int(other), int(req), c
↪ urrent, self.host_cpu.busy_share)
    return max(1, int((alloc - other) // req) - 1)

def _replica_room(self, h, ns, name, win, obs, s):
    """The HPA's replica cap as a lever, up to what the machines c
↪ an hold, one pod under it, read live (the
    founder's rule of 13 October; --replica-ceiling N sets a ceili
↪ ng of the operator's own, 0 keeps the operator's cap
    unmoved). Only for a sensed service under the compass law. Whi
↪ le the autoscaler
    stands at its cap and the compass is at or past its centre (or
↪  the line is breached), the cap is raised to the
    replicas the autoscaler's own arithmetic asks for, ceil(r x u
↪ / x), never past the ceiling; once the demand has
    held still for one window, the compass is under its centre, no
↪ thing breaches and that arithmetic fits the
    operator's own cap again, the cap returns to it. The operator'
↪ s range is recorded before the first change, and
    the reset restores it."""
    given = int(getattr(self.a, "replica_ceiling", -1))
    if given == 0 or self.compass_law is None:
        return
    spec = h.get("spec", {}); st = h.get("status", {}) or {}
    ann = h["metadata"].get("annotations", {}) or {}
    lo0, hi0 = (int(x) for x in ann.get(RANGE_ANN, f"{spec.get('mi
↪ nReplicas', 1)},{spec.get('maxReplicas', 1)}").split(","))
    cap = int(spec.get("maxReplicas", hi0)); cur = int(st.get("cur
↪ rentReplicas", 0) or 0)
    _, tgt = cpu_target(h); dem = self._app_demand(h)
    if not tgt or dem is None:
        return
    need = int(math.ceil(100.0 * dem / float(tgt) - 1e-9))      #
↪ the replicas the autoscaler's arithmetic asks for
    pressed = self.compass_law.p >= self.compass_law.band.center o
↪ r not obs["slo_clean"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        ceil_ = given if given > 0 else None
        # Once Omni has raised the cap, refresh the shared-host ceilin
    ↪ g every tick:
        # utilization can climb after the first decision. Keep the liv
    ↪ e replica count
        # runnable if the refreshed ceiling is below it.

        if ceil_ is None and ((cur >= cap and pressed and need > cap)
    ↪ or cap > hi0):
            ceil_ = self._live_ceiling(h)
        if cap > hi0:
            safe_cap = bounded_replica_cap(hi0, cap, cur, ceil_)
            if safe_cap < cap:

                self.k.write(["patch", "hpa", name, "-n", ns, "--type=
    ↪ merge", "-p", json.dumps({"spec": {"maxReplicas": safe_cap}})],
                             f"replica room: cap {cap} -> {safe_cap} (
    ↪ live ceiling {ceil_}; current replicas {cur})")
                return
        if cur >= cap and pressed and need > cap and ceil_ is not None

    ↪  and cap < ceil_:
            new = min(ceil_, need)
            if RANGE_ANN not in ann:
                self.k.write(["annotate", "hpa", name, "-n", ns, "--ov
    ↪ erwrite", f"{RANGE_ANN}={lo0},{hi0}"], "replica room: record the
    ↪  operator's range")

            self.k.write(["patch", "hpa", name, "-n", ns, "--type=merg
    ↪ e", "-p", json.dumps({"spec": {"maxReplicas": new}})],
                         f"replica room: cap {cap} -> {new} (at the ca
    ↪ p, {need} asked, ceiling {ceil_})")
        elif cap > hi0 and need <= hi0 and self.compass_law.p < self.c
    ↪ ompass_law.band.center and obs["slo_clean"] and self._steady((ns

    ↪ , name), win):
            self.k.write(["patch", "hpa", name, "-n", ns, "--type=merg
    ↪ e", "-p", json.dumps({"spec": {"maxReplicas": hi0}})],
                         f"replica room: cap {cap} -> {hi0}, the opera
    ↪ tor's own (demand steady, {need} asked)")
            if int(spec.get("minReplicas", lo0)) == lo0:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            self.k.write(["annotate", "hpa", name, "-n", ns, f"{RA
↪ NGE_ANN}-"], "replica room: operator's range back, record remove
↪ d")

def _new_latencies(self):
    """Response times (ms) of the requests served since the last d
↪ ecision (the probe's latency file)."""
    lf = getattr(self.a, "latency_file", "")
    if not lf:
        return []
    try:
        rows = list(csv.DictReader(open(lf)))
    except OSError:
        return []
    out, last = [], self.lat_t
    for row in rows:
        t, ms = row.get("elapsed_seconds"), row.get("latency_ms")
        if row.get("ok") != "1" or not t or not ms:
            continue
        t = float(t)
        if last is None or t > last:
            out.append(float(ms))
            self.lat_t = t if self.lat_t is None else max(self.lat
↪ _t, t)
    return [] if last is None else out

def audit(self, rec):
    rec = {"time": time.time(), **rec}
    if self.log:
        self.log.write(json.dumps(rec) + "\n"); self.log.flush()
    return rec

def killed(self):
    # this controller's own switch, or the master switch for the w
↪ hole harness (omnicompass/master.py)
    return (os.environ.get("OMNI_KILL") == "1" or (self.a.kill_fil
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ e and Path(self.a.kill_file).exists())
                or master.is_off())

    def _persist(self, what, fn):
        """One step of the hand-back, tried RESTORE_TRIES times agains
    ↪ t a transient failure of the cluster's API; every

        failed try is in the audit. Returns (the step's value, whether
    ↪  it landed)."""
        for attempt in range(1, RESTORE_TRIES + 1):
            try:
                return fn(), True
            except (subprocess.CalledProcessError, OSError) as e:

                self.audit({"error": f"restore: {what}: {e!r}"[:500],
    ↪ "stderr": str(getattr(e, "stderr", "") or "")[-300:],
                            "attempt": attempt, "of": RESTORE_TRIES})
                if attempt < RESTORE_TRIES:
                    time.sleep(min(4.0, 2.0 ** (attempt - 1)))
        return None, False

    def restore(self):
        """Every setting handed back to the operator's, from the recor
    ↪ ds kept on the objects themselves. Each read and write
        is tried RESTORE_TRIES times, each muscle's hand-back on its o
    ↪ wn, and what did not land is raised at the end (the

        caller's audit records it; the master switch's watchdog runs t
    ↪ he hand-back again)."""
        missed = []
        hpas, ok = self._persist("list the autoscalers", lambda: self.
    ↪ k.get("get", "hpa", "-A", "-o", "json")["items"])
        if not ok:

            missed.append("the autoscalers' replica ranges and targets
    ↪  (the list could not be read)")
        for h in hpas or []:
            ns, name = h["metadata"]["namespace"], h["metadata"]["name
    ↪ "]
            rng = h["metadata"].get("annotations", {}).get(RANGE_ANN)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            if rng:
                lo0, hi0 = (int(x) for x in rng.split(","))

                def put_range(ns=ns, name=name, lo0=lo0, hi0=hi0):
                    self.k.write(["patch", "hpa", name, "-n", ns, "--t
    ↪ ype=merge", "-p", json.dumps({"spec": {"minReplicas": lo0, "maxR
    ↪ eplicas": hi0}})],
                                 "reset: restore the HPA's own replica
    ↪  range")
                self.k.write(["annotate", "hpa", name, "-n", ns, f
    ↪ "{RANGE_ANN}-"], "reset: remove range record")
                if not self._persist(f"{ns}/{name}: the HPA's own repl
    ↪ ica range", put_range)[1]:
                    missed.append(f"{ns}/{name}: the replica range")
            orig = h["metadata"].get("annotations", {}).get(ANNOTATION
    ↪ )
            if orig is not None:
                idx, _ = cpu_target(h)

                def put_target(ns=ns, name=name, idx=idx, orig=orig):
                    if idx is not None:
                        self.k.write(["patch", "hpa", name, "-n", ns,
    ↪ "--type=json", "-p",
                                      json.dumps([{"op": "replace", "p
    ↪ ath": f"/spec/metrics/{idx}/resource/target/averageUtilization",
    ↪  "value": int(orig)}])],
                                     "reset: restore original HPA targ
    ↪ et")
                self.k.write(["annotate", "hpa", name, "-n", ns, f
    ↪ "{ANNOTATION}-"], "reset: remove record")

                if not self._persist(f"{ns}/{name}: the operator's tar
    ↪ get", put_target)[1]:
                    missed.append(f"{ns}/{name}: the target")
        for ns, name in getattr(self, "own", []):
            dep, ok = self._persist(f"{ns}/{name}: read the workload",
    ↪  lambda ns=ns, name=name: self.k.get("get", "deployment", name,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ "-n", ns, "-o", "json"))
            if not ok:
                missed.append(f"{ns}/{name}: the operator's own count
    ↪ (the workload could not be read)")
                continue
            orig = (dep["metadata"].get("annotations", {}) or {}).get(
    ↪ OWN_REPL_ANN)
            if orig is None:
                continue

            def put_count(ns=ns, name=name, orig=orig):
                self.k.write(["scale", "deployment", name, "-n", ns, f
    ↪ "--replicas={int(orig)}"], "reset: restore the operator's own co
    ↪ unt")
                self.k.write(["annotate", "deployment", name, "-n", ns
    ↪ , f"{OWN_REPL_ANN}-"], "reset: remove the count's record")
            if not self._persist(f"{ns}/{name}: the operator's own cou
    ↪ nt", put_count)[1]:
                missed.append(f"{ns}/{name}: the operator's own count"
    ↪ )
        self.changed.clear()
        try:
            missed += [f"the muscle's hand-back: {n}" for n in (self.m
    ↪ .restore() or [])]   # each lever tried on its own (muscles.py)
        except Exception as e:  # noqa: BLE001
            self.audit({"error": f"restore: the muscles' hand-back: {e
    ↪ !r}"[:500]})
            missed.append("the muscles' hand-back")
        cmd = getattr(self.a, "node_restore_cmd", "")
        if cmd and not self.nodes_restored:
            self.audit({"write": shlex.split(cmd), "why": "reset: rest
    ↪ ore node pool", "dry_run": self.a.dry_run})
            if self.a.dry_run or self._persist("the node pool", lambda
    ↪ : subprocess.run(shlex.split(cmd), check=True))[1]:
                self.nodes_restored = True
            else:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                missed.append("the node pool")
        if missed:
            raise RuntimeError("hand-back incomplete after %d tries ea
    ↪ ch: %s" % (RESTORE_TRIES, "; ".join(missed)))

    def _gain(self, h):

        """The CPU each pod of the HPA's deployment is guaranteed, ove
    ↪ r the operator's limit (1 when nothing conveyed):
        g = min(mean conveyed limit, 0.95 A / ceil(maxReplicas / machi
    ↪ nes in service)) / L. The guaranteed share is the
        one each pod keeps with the autoscaler's largest count spread
    ↪ over the machines in service, so the target it

        sets moves only when a machine idles or wakes, never each time
    ↪  a pod starts or leaves."""
        from omnicompass.nervous_system import BAND
        ns = h["metadata"]["namespace"]; ref_ = h.get("spec", {}).get(
    ↪ "scaleTargetRef", {}) or {}
        key = (ns, ref_.get("name", ""))

        g = self.m.gain.get(key, 1.0); base = self.m.base.get(key, 0.0
    ↪ ); s = getattr(self, "_snap", None)
        if base > 0 and s and s.get("nodes"):
            ann = h["metadata"].get("annotations", {}) or {}
            hi0 = int(ann.get(RANGE_ANN, f"1,{h['spec'].get('maxReplic
    ↪ as', 1)}").split(",")[1])

            per_node = s["alloc_m"] / s["nodes"]
            g = min(g, BAND[1] * per_node / math.ceil(hi0 / s["nodes"]
    ↪ ) / base)
        return max(1.0, g)

    def strict_step(self, s, obs):

        """Strict C: Omni-Compass decides each deployment's replica fl
    ↪ oor itself: replicas = ceil(current x measured
        utilisation / target utilisation), up at once, down only to th
    ↪ e highest recommendation of the last
        --strict-window decisions; the HPA's minReplicas is set to tha
    ↪ t count, within its original range. Growth is never
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    blocked: maxReplicas stays the operator's, so the HPA remains
↪ the fast up-reflex between Omni's decisions (the HPA
    reacts every 15 s; a count frozen for a 60 s decision would an
↪ swer a load step up to 45 s late). The Unified
    Control Switch restores the original range."""
    hist = getattr(self, "_rec_hist", {}); self._rec_hist = hist

    for h in s["hpas"]:
        ns, name = h["metadata"]["namespace"], h["metadata"]["name
↪ "]
        ann = h["metadata"].get("annotations", {}) or {}
        lo0, hi0 = (int(x) for x in ann.get(RANGE_ANN, f"{h['spec'
↪ ].get('minReplicas', 1)},{h['spec']['maxReplicas']}").split(",")
↪ )
        idx, tgt = cpu_target(h)
        orig_t = int(ann.get(ANNOTATION, tgt or 50))
        cur = int(h.get("status", {}).get("currentReplicas", 0) or
↪  lo0)
        util = None
        for m in h.get("status", {}).get("currentMetrics", []) or
↪ []:
            if m.get("type") == "Resource" and m.get("resource", {
↪ }).get("name") == "cpu":
                util = m["resource"].get("current", {}).get("avera
↪ geUtilization")
        if util is None:
            continue
        rec = max(lo0, min(hi0, int(math.ceil(cur * float(util) /
↪ (orig_t * self._gain(h)) - 1e-9))))
        hh = (hist.get((ns, name), []) + [rec])[-self.n_strict_win
↪ dow:]; hist[(ns, name)] = hh
        want = rec if rec >= cur else max(hh)
        want = max(want, min(hi0, self.reflex.get((ns, name), 0)))
↪    # the fast pod reflex's floor stands while held
        if obs.get("security_block", 0.0) > 0.5:
            want = min(want, cur)                    # shield I1:
↪  no expansion during a security hold
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if RANGE_ANN not in ann:
            self.k.write(["annotate", "hpa", name, "-n", ns, "--ov
↪ erwrite", f"{RANGE_ANN}={lo0},{hi0}"], "strict: record original
↪ replica range")
        if h["spec"].get("minReplicas") != want or h["spec"]["maxR
↪ eplicas"] != hi0:

            self.k.write(["patch", "hpa", name, "-n", ns, "--type=
↪ merge", "-p", json.dumps({"spec": {"minReplicas": want, "maxRepl
↪ icas": hi0}})],
                         f"strict: replica floor {cur} -> {want} d
↪ ecided by Omni-Compass (utilisation {util}%, target {orig_t}%);
↪ growth stays free up to {hi0}")

def _own_pods(self, s, obs, rho, move=True):
    """The pod muscle of its own (omni-v19, the founder's order of
↪  23 October: Omni-Compass by itself, running the muscles
    directly; docs/SOLO_PREREGISTRATION.md). For a workload the op
↪ erator names (--own-replicas) and no autoscaler governs,

    Omni-Compass sets the replica count itself, through the worklo
↪ ad's own scale, by strict C's rule: replicas =
    ceil(current x measured busy share / target), up at once, down
↪  only to the highest recommendation of the last
    --strict-window, inside the autoscaler's tolerance of a tenth.
↪  The target is the one the compass moves for an

    autoscaler (the decision above): the operator's own (the workl
↪ oad's target-utilization annotation, else
    --own-target), down to 60% of it, the most pods the cover allo
↪ ws past the wall. Never below the operator's own count
    at the max (the tune-down may go to the range's floor), never
↪ above the range's top, never below one pod. The count

    it shipped is recorded once on the workload and handed back by
↪  the reset (restore). With move False (a fast check
    between decisions) the target stays where the last decision se
↪ t it and only the count is reconciled."""
    if "replicas" in self.a.native_muscles:
        return None                 # omni-v14: the replicas are
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ left native; nothing written
        if move:
            self._own_last = (s, obs, rho)
        governed = {(h["metadata"]["namespace"], (h["spec"].get("scale
    ↪ TargetRef") or {}).get("name")) for h in s["hpas"]}
        out = {}

        for ns, name in self.own:
            key = (ns, name)
            if key in self.own_handed:
                self.audit({"own_replicas": f"{ns}/{name}", "held": "h
    ↪ anded back to native's own autoscaler: nothing written"})
                continue

            if key in governed:
                self.audit({"own_replicas": f"{ns}/{name}", "held": "a
    ↪ n autoscaler governs it: its own muscle stands, nothing written"
    ↪ })
                continue
            dep = self.k.get("get", "deployment", name, "-n", ns, "-o"

    ↪ , "json")
            ann = dep["metadata"].get("annotations", {}) or {}
            cur = int(dep["spec"].get("replicas", 1) if dep["spec"].ge
    ↪ t("replicas") is not None else 1)
            orig = int(ann.get(OWN_REPL_ANN, cur))
            t_raw = ann.get(OWN_TARGET_ANN, getattr(self.a, "own_targe

    ↪ t", None))
            if t_raw is None:
                self.audit({"own_replicas": f"{ns}/{name}", "held": "n
    ↪ o target named (the workload's annotation or --own-target): noth
    ↪ ing written"})
                continue

            orig_t = float(t_raw)
            lo0, hi0 = (int(x) for x in str(ann.get(OWN_RANGE_ANN, f"1
    ↪ ,{10 ** 6}")).split(","))
            c0 = dep["spec"]["template"]["spec"]["containers"][0]
            req = to_milli(c0.get("resources", {}).get("requests", {})
    ↪ .get("cpu", "0") or "0")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        sel = ",".join(f"{k_}={v_}" for k_, v_ in ((dep["spec"].ge
↪ t("selector") or {}).get("matchLabels") or {}).items())
        if req <= 0 or not sel:
            self.audit({"own_replicas": f"{ns}/{name}", "held": "n
↪ o CPU request or no selector on the workload: nothing written"})
            continue

        top = self.k.get("top", "pods", "-n", ns, "-l", sel, "--no
↪ -headers")
        lines = [ln.split() for ln in str(top or "").strip().split
↪ lines() if ln.strip()]
        if not lines:
            continue                # no pod measured yet: nothin

↪ g decided from it
        used = sum(to_milli(x[1]) for x in lines)
        util = 100.0 * used / (len(lines) * req)
        dem = used / req
        self.demand[key] = [(t_, v_) for t_, v_ in self.demand.get
↪ (key, []) if time.time() - t_ <= 600.0] + [(time.time(), dem)]

        # the target: the compass's push and pull inside its cover
↪ , as on an autoscaler's target (the decision above)
        hi_t, lo_t = orig_t, max(10.0, 0.6 * orig_t)
        x = self.own_x.get(key, hi_t)
        if not move:
            pass                                                #

↪  between decisions: the target the last decision set
        elif self.compass_law is None:
            x = min(hi_t, max(lo_t, 100.0 * rho))
        else:
            load = s["pending"] == 0 and not obs["slo_clean"]
            pos_ = self.compass_law.p if self.compass_law.p is not

↪  None else 0.0   # no reading yet: the operator's own target
            if pos_ >= self.compass_law.band.wall_high and load:
                x = lo_t                                        #
↪  fail up: the most pods the cover allows, at once
            elif pos_ >= self.compass_law.band.wall_high:
                x = hi_t                                        #
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  past the wall, but not from load: the operator's own
                elif x < hi_t and pos_ < self.compass_law.band.center
    ↪ and obs["slo_clean"] and s["pending"] == 0 \
                        and not self._rising(key, 300.0):
                    x = hi_t
                else:

                    F_ = self._own_F
    ↪ # the compass's force of this decision (above)
                    x = x - (COMPASS_UP if F_ > 0 else COMPASS_DOWN) *
    ↪  F_ * (hi_t - lo_t) * self.step_scale
            x = max(lo_t, min(hi_t, x)); self.own_x[key] = x
            ratio = util / x

            rec = cur if abs(ratio - 1.0) <= 0.1 else int(math.ceil(cu
    ↪ r * ratio - 1e-9))
            floor_ = max(1, lo0, orig if self.dial == "max" else lo0)
            rec = max(floor_, min(hi0, rec))
            hh = (self.own_hist.get(key, []) + [rec])[-self.n_own_wind
    ↪ ow:]; self.own_hist[key] = hh

            want = rec if rec >= cur else max(hh)
            if obs.get("security_block", 0.0) > 0.5:
                want = min(want, cur)                                  #
    ↪  shield I1: no expansion during a security hold
            if want == cur:
                continue

            try:
                if OWN_REPL_ANN not in ann:
                    self.k.write(["annotate", "deployment", name, "-n"
    ↪ , ns, "--overwrite", f"{OWN_REPL_ANN}={cur}"], "own pods: record
    ↪  the operator's count")
                self.k.write(["scale", "deployment", name, "-n", ns, f

    ↪ "--replicas={want}"],
                             f"own pods: {cur} -> {want} (busy {util:.
    ↪ 0f}% of the request, target {x:.0f}%, the operator's {orig_t:.0f
    ↪ }%)")
            except subprocess.CalledProcessError as e:
                self._own_refused(key, orig_t, lo0, hi0, orig, e)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    # omni-v23: the knob never left with no controller
            continue
        self.own_refused.pop(key, None)
    # a write that landed ends the run of refusals
        out[key] = want
    if out:
        self.audit({"own_replicas": {f"{k[0]}/{k[1]}": v for k, v
in out.items()}})
    return out

def _own_refused(self, key, orig_t, lo0, hi0, orig, e):
    """omni-v23 (the founder's order of 31 October: Omni direct ke
eps every muscle Omni-Compass cannot be; a knob left with no
    working controller fails the whole system and is never a resul
t). A write of the workload's own scale that the cluster
    refuses (its permission, its admission, its API) is recorded,
and the decision goes on: the cluster keeps the last count
    that landed. At OWN_REFUSALS refusals in a row Omni-Compass ca
nnot be this knob's muscle, so the knob goes back to native's
    own pod autoscaler, made from the operator's own target and ra
nge recorded on the workload, and the muscle leaves the
    workload to it for the rest of the run (the snapshot's governe
d set holds it from the next decision on; the reset leaves
    native's autoscaler standing). Nothing is forced: the organism
 never runs with a knob nobody moves."""
    ns, name = key
    n = self.own_refused.get(key, 0) + 1
    self.own_refused[key] = n
    err = (getattr(e, "stderr", "") or str(e) or "")[-300:]
    self.audit({"own_replicas": f"{ns}/{name}", "refused": n, "err
or": err})
    if n < OWN_REFUSALS or key in self.own_handed:
        return
    self.own_handed.add(key)
    hi = hi0 if hi0 < 10 ** 6 else max(10, 10 * max(1, int(orig)))
    # no range recorded: ten times the operator's count
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        try:
            self.k.write(["autoscale", "deployment", name, "-n", ns, f
↪ "--cpu-percent={int(round(orig_t))}", f"--min={lo0}", f"--max={h
↪ i}"],
                         f"own pods: the cluster refused the muscle's
↪ write {n} times in a row: the knob handed back to native's "

                         f"own autoscaler at the operator's target {or
↪ ig_t:.0f}% and range {lo0},{hi}")
            self.audit({"own_replicas": f"{ns}/{name}", "handed_back":
↪  "native's own pod autoscaler", "after_refusals": n,
                        "target": orig_t, "range": [lo0, hi]})
        except subprocess.CalledProcessError as e2:

            self.audit({"own_replicas": f"{ns}/{name}", "hand_back_ref
↪ used": (getattr(e2, "stderr", "") or str(e2) or "")[-300:]})

    def pod_reflex(self):
        """The fast pod reflex (manuscript Section 5.3, preemptive coh
↪ erence): the HPA's own staffing rule,

        replicas = current x busy / target, read from the live queue i
↪ nstead of CPU averages a minute old.

        Busy comes from queueing physics: a replica serving requests i
↪ n processor sharing answers in R = S / (1 - u),
        so u = 1 - S / R, with S the bare service time (the fastest te

↪ nth of the window's requests, those with no queue
        ahead) and R the mean response of the same window. The operato
↪ r's target is a share of the pod's CPU request; the queue runs o
↪ n
        its limit, so the same promise in queue terms is target x requ
↪ est / limit. The reflex only raises the replica

        floor to what that rule needs now and hands it back the moment
↪  the queue no longer needs it: no padding, no new
        target, the operator's own promise met sooner. Zero cluster re
↪ ads while the queue is calm."""
        lf = getattr(self.a, "latency_file", "")
        if not lf or self.killed() or self.a.mode not in ("target", "n
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ odepool") or "replicas" in getattr(self.a, "native_muscles", ())
    ↪ :
            return None
        from omni_controller.muscles import latency_window
        w = latency_window(lf, getattr(self.a, "reflex_window_s", 30.0
    ↪ ))

        if w["blind"] or len(w["ms"]) < 5:
            return None
        p10 = sorted(w["ms"])[len(w["ms"]) // 10]
        # S and R from the same window: the fastest tenth of the reque
    ↪ sts answered now, at the CPU the pods have now. An
        # all-time fastest would read every later change of a pod's CP
    ↪ U limit (convey) as a queue that is not there
        self.s_floor = p10
        R = sum(w["ms"]) / len(w["ms"])
        u = max(0.0, min(0.99, 1.0 - self.s_floor / R)) if R > 0 else
    ↪ 0.0
        if u <= 0.0 and not self.reflex:
            return None                          # calm and nothing he
    ↪ ld: no read, no write
        out = {}
        for h in self.k.get("get", "hpa", "-A", "-o", "json")["items"]
    ↪ :
            idx, tgt = cpu_target(h)
            if tgt is None:
                continue
            ns, name = h["metadata"]["namespace"], h["metadata"]["name
    ↪ "]
            ann = h["metadata"].get("annotations", {}) or {}
            lo0, hi0 = (int(x) for x in ann.get(RANGE_ANN, f"{h['spec'
    ↪ ].get('minReplicas', 1)},{h['spec']['maxReplicas']}").split(",")
    ↪ )
            orig_t = int(ann.get(ANNOTATION, tgt)) / 100.0
            key = (ns, name)
            if key not in self.pod_cap:
                ref = h["spec"]["scaleTargetRef"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            c0 = self.k.get("get", ref["kind"].lower(), ref["name"
↪ ], "-n", ns, "-o", "json")["spec"]["template"]["spec"]["containe
↪ rs"][0]
            req = to_milli(c0.get("resources", {}).get("requests",
↪  {}).get("cpu", "0") or "0")
            lim = to_milli(c0.get("resources", {}).get("limits", {
↪ }).get("cpu", "0") or "0")
            self.pod_cap[key] = (req / lim) if req > 0 and lim > 0
↪  else 1.0
        u_star = max(0.05, orig_t * self.pod_cap[key])       # the
↪  operator's promise in queue terms
        cur = int(h.get("status", {}).get("currentReplicas", 0) or
↪  lo0)
        desired = int(h.get("status", {}).get("desiredReplicas", 0
↪ ) or cur)
        need = min(hi0, int(math.ceil(cur * u / u_star - 1e-9)))
        held = self.reflex.get(key)
        if not getattr(self.a, "pod_reflex_writes", False):
            # the autoscaler is the muscle that makes and removes
↪ pods; I gauge what the queue needs and write
            # nothing, so no pod is started or stopped that the mu
↪ scle itself would not start or stop
            if need > max(cur, desired):
                self.audit({"pod_reflex_reading": {f"{ns}/{name}":
↪  need}, "replicas": cur, "busy": round(u, 3),
                            "promise": round(u_star, 3), "R_ms": r
↪ ound(R, 1), "S_ms": round(self.s_floor, 1)})
            continue
        if need > max(cur, desired, h["spec"].get("minReplicas", 1
↪ ) or 1):
            if RANGE_ANN not in ann:
                self.k.write(["annotate", "hpa", name, "-n", ns, "
↪ --overwrite", f"{RANGE_ANN}={lo0},{hi0}"], "pod reflex: record o
↪ riginal replica range")
            self.k.write(["patch", "hpa", name, "-n", ns, "--type=
↪ merge", "-p", json.dumps({"spec": {"minReplicas": need}})],
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                            f"pod reflex: floor {need} (queue busy {u
↪ :.2f} vs promise {u_star:.2f}, R {R:.0f} ms, S {self.s_floor:.0f
↪ } ms)")
            self.reflex[key] = need; out[key] = need
        elif held is not None and need <= max(desired, lo0):
            # the queue no longer needs the floor (or the HPA has

↪ caught up): hand it back. On top, the operator's
            # own minimum returns; in strict C the floor is Omni's
↪  own replica decision again (strict_step)
            del self.reflex[key]; out[key] = None
            if not getattr(self.a, "strict_replicas", False) and h
↪ ["spec"].get("minReplicas") == held:

                self.k.write(["patch", "hpa", name, "-n", ns, "--t
↪ ype=merge", "-p", json.dumps({"spec": {"minReplicas": lo0}})],
                             f"pod reflex: release floor {held} ->
↪  {lo0} (queue busy {u:.2f})")
                out[key] = lo0
    if out:

        self.audit({"pod_reflex": {f"{k[0]}/{k[1]}": v for k, v in
↪  out.items()}, "busy": round(u, 3), "R_ms": round(R, 1), "S_ms":
↪  round(self.s_floor, 1)})
    return out

def _own_check(self):

    """Between decisions, the pod muscle of its own at the pod aut
↪ oscaler's pace (omni-v19): every n-th fast check
    (--own-every-s, 15 s), the count reconciled to the busy share
↪ at the target the last decision set."""
    if not self.own or self._own_last is None:
        return None

    self.own_checks += 1
    if self.own_checks % self.n_own:
        return None
    s, obs, rho = self._own_last
    try:
        return self._own_pods(s, obs, rho, move=False)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        except Exception as e:
            self.audit({"error": "own pods check: " + repr(e)[:500]})
            return None

    def floor_step(self):
        """Fast path between governor decisions: add the nodes that pe
    ↪ nding and running pod requests need (nodepool mode)."""
        if self.killed() or self.a.mode != "nodepool":
            return None
        self._own_check()
        if not self.a.node_scale_cmd:
            return None

        due = self._convey_due()
        if getattr(self, "cruise", False):
            # cruise (rule 7, amendment 12): every machine is already
    ↪ in service, so a floor check can add nothing; and a
            # queue of other work starves the service's responses with
    ↪ out any load on it, which the reflex would read as

            # a queue and answer with idle pods. Omni steps back: no r
    ↪ eads, no reflex, no conveyance until cruise ends
            return None
        self.pod_reflex()
        # energy to where the work is: a new serving pod gets its mach
    ↪ ine's idle CPU within --convey-every-s (15 s), not a

        # decision
        if getattr(self.a, "latency_file", "") and due:
            self._convey()
        # one light read every check; the full snapshot (nodes, all po
    ↪ ds, node metrics, HPAs) only when a pod waits. The
        # controller shares CPUs with the service it protects (on kind

    ↪ , one 4-core runner), so its reads cost latency
        if not str(self.k.get("get", "pods", "-A", "--field-selector=s
    ↪ tatus.phase=Pending", "-o", "name")).strip():
            return None
        self.waited = True                      # time waited between
    ↪  decisions is harm the next decision's verdict reads
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    s = snapshot(self.k, getattr(self.a, "active_nodes_only", Fals
↪ e))
    if s["alloc_m"] <= 0:
        return None                          # no machine in servi
↪ ce in the reading: blind, nothing to add from it
    if s["pending"] > 0:

        self._pods_wait_handback(s)
    n = max(1, s["open"]); per_node = s["alloc_m"] / max(1, s["nod
↪ es"])
    floor = max(int(math.ceil(s["req_m"] * (1.0 + self.a.headroom)
↪  / per_node)) if s["req_m"] > 0 else self.a.min_nodes, int(math.
↪ ceil(s["used_m"] / per_node)))

    floor = min(self._max_nodes(s), floor)
    if s["pending"] > 0 and floor > n and floor > (self.rec_n or 0
↪ ):
        if self._pool_command(floor, n, "scheduling floor (pending
↪  pods)"):
            self.rec_n = floor

            return floor
    return None


def _pods_wait_handback(self, s):
    """The immune hand-back (omni-v19, the founder's rule: where O
↪ mni-Compass cannot beat native, native stays). A pod waiting

    for a place is time waited, a LOSS: while one waits, every pod
↪  autoscaler whose target Omni-Compass holds below the
    operator's own (more pods than native would ask) gets the oper
↪ ator's target back at once, from this fast check, not the
    next decision, and keeps it for a hold of 10 minutes, doubled
↪ for each hand-back in a row up to two hours and halved back

    after a quiet hold. The machines the waiting pods need are sti
↪ ll asked for by the scheduling floor below."""
    now = time.time()
    for h in s["hpas"]:
        ns, name = h["metadata"]["namespace"], h["metadata"]["name
↪ "]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            key = (ns, name)
            orig, cmd = self.changed.get(key), self.cmd_hpa.get(key)
            idx, cur = cpu_target(h)
            if idx is None or orig is None or cmd is None or cur is No
    ↪ ne:
                continue

            # native's target in the pods' own terms: the operator's,
    ↪ times the conveyed gain while a pod's limit is raised
            # (the same promise in queue terms the decision keeps), ne
    ↪ ver over 100%
            back_to = min(100, int(round(float(orig) * self._gain(h)))
    ↪ )

            if int(cur) >= back_to:
                continue
            until, length = self.pod_hold.get(key, (0.0, 0.0))
            if until > now:
                continue                          # already handed back
    ↪  and held

            n = self.hold_n.get(key, 0)
            hold = min(7200.0, 600.0 * (2 ** n))
            self.pod_hold[key], self.hold_n[key] = (now + hold, hold),
    ↪  n + 1
            if "replicas" in self.a.native_muscles:
                continue

            self.k.write(["patch", "hpa", name, "-n", ns, "--type=json
    ↪ ", "-p",
                          json.dumps([{"op": "replace", "path": f"/spe
    ↪ c/metrics/{idx}/resource/target/averageUtilization", "value": ba
    ↪ ck_to}])],
                         f"immune hand-back: a pod waits, the HPA targ

    ↪ et back to native's {back_to}% for {int(hold)} s (omni-v19)")
            self.cmd_hpa[key] = back_to; self.target_at[key] = now
            self.compass_law_x[key] = float(back_to)
            self.audit({"immune": {"hpa": f"{ns}/{name}", "pending": s
    ↪ ["pending"], "target_was": int(cur), "target_back_to": back_to,
                                   "hold_s": int(hold), "in_a_row": n
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ + 1}})

    def _immune_native(self, h, ns, name, x, hi_t, lo_t, p95):
        """The immune system's self-knowledge for this service's autos
    ↪ caler target (omni-v21; omnicompass/immune.py; the
        founder's order of 25 October): while the service's response t
    ↪ ime is over its line with Omni-Compass's target on it,
        and Omni-Compass's own pushing does not bring it back, the tar
    ↪ get goes to native when native's twin reads the moment
        lower, and stays where it is when native is not stronger. Nati
    ↪ ve's twin is the same moment under the operator's target:
        the replicas native's own rule would ask for the measured dema
    ↪ nd, and the M/M/1 wait they would give, the response time
        scaled by (1 - rho_omni) / (1 - rho_native) with rho each pod'
    ↪ s busy share of its request. True while native holds it."""
        from omnicompass.immune import Immune
        slo = float(getattr(self.a, "slo_ms", 0) or 0)
        key = (ns, name)
        im = self.immune.setdefault(key, Immune())
        dem = self._app_demand(h)
        cur = int((h.get("status") or {}).get("currentReplicas", 0) or
    ↪  0)
        orig = int(h["metadata"].get("annotations", {}).get(ANNOTATION
    ↪ , cpu_target(h)[1] or 0) or 0)
        if not slo or p95 is None or dem is None or cur <= 0 or orig <
    ↪ = 0:
            return im.mode == "native"
        native_reps = max(1, int(math.ceil(100.0 * dem / orig - 1e-9))
    ↪ )
        rho_o, rho_n = min(0.98, dem / cur), min(0.98, dem / native_re
    ↪ ps)
        p_omni = float(p95) / slo
        p_native = p_omni * (1.0 - rho_o) / (1.0 - rho_n)
        moved = abs(x - self.compass_law_x.get(key, hi_t)) > 0.5
        at_limit = x <= lo_t + 1e-9 or cur >= int(h["spec"].get("maxRe
    ↪ plicas", 10 ** 9))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        mode, ev = im.tick(p_omni, p_native, moved, at_limit)
        if ev:
            self.audit({"immune": dict(ev, hpa=f"{ns}/{name}", native_
    ↪ replicas=native_reps, replicas=cur)})
        return mode == "native"

    def _held_native(self, key):
        """True while a pod autoscaler's target is held at the operato
    ↪ r's after a pod waited (_pods_wait_handback); a hold that
        has passed with no new hand-back for another hold's length hal
    ↪ ves the next one."""
        until, length = self.pod_hold.get(key, (0.0, 0.0))

        now = time.time()
        if until > now:
            return True
        if length and now > until + length and self.hold_n.get(key, 0)
    ↪  > 0:
            self.hold_n[key] -= 1

            self.pod_hold[key] = (now, length / 2.0)
        return False


    def _convey_due(self):
        """The conveyance's clock (omni-v16): every check counts, a fa
    ↪ st check or a decision, and the conveyance runs on

        every n-th, n = --convey-every-s / --floor-interval (15 s at t
    ↪ he 5 s checks), whatever the decision period. Before
        omni-v16 it ran on every third fast check and a decision was n
    ↪ ever one, so at a decision every 5 s it never ran."""
        self.checks += 1
        return self.checks % self.n_convey == 0

    def _convey(self):
        try:
            self.m.convey()
        except Exception as e:  # a failed conveyance writes nothing m
    ↪ ore; the next one comes on time
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            self.audit({"error": f"convey: {e}"})

    def step(self):
        self.decisions += 1
        convey_due = self._convey_due() if self.a.mode == "nodepool" a
    ↪ nd self.a.node_scale_cmd else False

        self.host_cpu.sample()              # two adjacent controller
    ↪ ticks provide a host CPU busy share
        if self.killed():
            self.restore()
            return self.audit({"decision": "killed", "mode": "observe"
    ↪ })

        s = snapshot(self.k, getattr(self.a, "active_nodes_only", Fals
    ↪ e))
        if s["alloc_m"] <= 0:
            # a reading with no machine in service is blind, never an
    ↪ overload (omni-v11; 18 Oct, the CruiseKube test, run
            # 39120444653: every worker of the kind cluster left servi

    ↪ ce for two minutes while the service answered every
            # probe, and omni-v10 read its load against no capacity, t
    ↪ hen divided by it). Nothing is decided from it and
            # nothing written; the last reading with machines stays th
    ↪ e one the conveyance is bounded by
            return self.audit({"decision": "held: the cluster's readin

    ↪ g shows no machine in service; nothing decided, nothing written"
    ↪ ,
                               "mode": "autopilot" if self.a.mode in (
    ↪ "target", "nodepool") else "observe"})
        self._snap = s
        # my orders are judged by the machines open to work; a machine

    ↪ 's capacity by the machines in service
        n = max(1, s["open"]); per_node = s["alloc_m"] / max(1, s["nod
    ↪ es"])
        if self.rec_n is None:
            self.rec_n = n
        repl = sum(int(h.get("status", {}).get("currentReplicas", 0) o
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ r 0) for h in s["hpas"]) or n
        power_stress = 0.0; blind = {}
        if self.a.power_cmd and self.a.site_limit_w:
            try:
                power_stress = float(run_operator_cmd(self.a.power_cmd
    ↪ ).stdout.strip()) / self.a.site_limit_w

                blind["power"] = False
            except ValueError:
                power_stress = 0.0; blind["power"] = True
        # proprioception (efferent -> afferent): did the last commands
    ↪  land? drift = |observed - commanded| / commanded
        drift = {}

        if self.cmd_nodes is not None:
            drift["nodes"] = abs(n - self.cmd_nodes) / max(1, self.cmd
    ↪ _nodes)
        for h in s["hpas"]:
            key = (h["metadata"]["namespace"], h["metadata"]["name"])
            if key in self.cmd_hpa:

                _, cur_t = cpu_target(h)
                if cur_t is not None:
                    drift["hpa " + "/".join(key)] = abs(cur_t - self.c
    ↪ md_hpa[key]) / max(1, self.cmd_hpa[key])
        nodes_landed = drift.get("nodes", 0.0) == 0.0
        # a command is given LAND_S to land, however often the governo

    ↪ r decides (omni-v16): every decision reads its drift
        # until it has landed or its minute has passed (at a decision
    ↪ a minute, the next decision only, as before)
        keep = lambda key, d_: d_ > 0.0 and self.decisions - self.cmd_
    ↪ at.get(key, self.decisions) < self.n_land
        if self.cmd_nodes is not None and not keep("nodes", drift.get(

    ↪ "nodes", 0.0)):
            self.cmd_nodes = None
        self.cmd_hpa = {k_: v_ for k_, v_ in self.cmd_hpa.items() if k
    ↪ eep(k_, drift.get("hpa " + "/".join(k_), 0.0))}
        obs = {"queue_ratio": min(2.0, s["pending"] / max(1, repl)), "
    ↪ load_ratio": min(2.0, s["used_m"] / max(1.0, self.rec_n * per_no
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ de)),
              "power_stress": power_stress, "thermal": 0.0, "network_
    ↪ stress": 0.0, "drift_ratio": max(drift.values(), default=0.0),
              "stale": 0.0, "security_block": 0.0}
        extra = self.m.sense(power_stress)
        lp = extra.pop("latency_pressure", 0.0); p95 = extra.pop("late
    ↪ ncy_p95_ms", None)
        if getattr(self.a, "latency_file", "") and getattr(self.a, "sl
    ↪ o_ms", 0):
            blind["latency"] = bool(extra.pop("latency_blind", 0.0))
        age = extra.pop("latency_age_s", None)
        obs.update(extra)

        # afferent integrity: the share of declared senses that are bl
    ↪ ind enters the engine as its stale channel
        obs["stale"] = (sum(blind.values()) / len(blind)) if blind els
    ↪ e 0.0
        obs["queue_ratio"] = min(2.0, max(obs["queue_ratio"], lp))
        # SLO reflex: while the response-time target is breached, and
    ↪ for --slo-clear decisions after, Omni-Compass may
        # not pack replicas tighter than the workload's own HPA target
    ↪  and may not cap power (no energy at service's cost)
        self.lp_hist.append(lp)
        guarded = bool(getattr(self.a, "latency_file", "")) and getatt
    ↪ r(self.a, "slo_ms", 0)

        n_clear = self.n_slo_clear
        obs["slo_clean"] = (not guarded) or (not blind.get("latency",
    ↪ False) and len(self.lp_hist) >= n_clear
                                             and all(x == 0.0 for x in
    ↪  self.lp_hist[-n_clear:]))
        self.g.nodes = self.rec_n; self.g.current_cap = 1.0

        d = self.g.step(obs, 0)
        floor = max(int(math.ceil(s["req_m"] * (1.0 + self.a.headroom)
    ↪  / per_node)) if s["req_m"] > 0 else self.a.min_nodes, int(math.
    ↪ ceil(s["used_m"] / per_node)))
        rec_n = max(self.a.min_nodes, min(self._max_nodes(s), max(self
    ↪ .rec_n + int(d["node_delta"]), floor)))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        rho = max(0.5, min(0.95, float(d["demand"])))
        from omnicompass.nervous_system import from_governor, node_rel
    ↪ ease_gate
        mode = "autopilot" if self.a.mode in ("target", "nodepool") el
    ↪ se "observe"
        # an organ every governed knob of which the immune system hand

    ↪ ed to native has no authority of Omni-Compass's (omni-v21)
        held = [k for k, im in self.immune.items() if im.mode == "nati
    ↪ ve"]
        auth = from_governor(self.g, obs, d, mode=mode, native_hold=("
    ↪ pods",) if held and len(held) == len(self.immune) else ())
        self.m.auth = auth

        breach_now = lp > 0.0 or blind.get("latency", False)
        # the machine organ's view: only pressure a machine release co
    ↪ uld cause (pods waiting for a place, a live
        # breach). Power and heat are relieved by a release, never wor
    ↪ sened by it, so they cannot veto one
        obs_n = dict(obs, queue_ratio=min(2.0, s["pending"] / max(1, r

    ↪ epl)), slo_clean=not breach_now,
                      power_stress=0.0, thermal=0.0)
        self.gn.nodes = self.rec_n; self.gn.current_cap = 1.0
        dn = self.gn.step(obs_n, 0)
        auth_n = from_governor(self.gn, obs_n, dn, mode=mode)
        if self.cl is not None:

            # the benchmarked law drives the machines: the closure law
    ↪  on requested cores (omnicompass/closure.py), the
            # scheduling floor stays underneath it
            self.cl.observe(s["req_m"] / 1000.0)
            # the machine organ's own pressure (as its release gate):
    ↪ pods waiting and live breaches, not modelled heat

            # or the whole body's latency push, which a machine releas
    ↪ e does not cause
            cl_n = self.cl.decide(n, per_node / 1000.0, self.gn.last_p
    ↪ ush, self.a.min_nodes, self._max_nodes(s))
            rec_n = max(self.a.min_nodes, min(self._max_nodes(s), max(
    ↪ cl_n, floor)))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        compass_rec = None
        if self.compass_law is not None:
            # the service position in its compass, read as the GPU com
↪ pass reads it (omni_controller/gpu_compass.py): the mean
            # response time of the window between the bare service tim
↪ e (a tenth of the SLO) and the SLO; p95 at or

            # past the SLO, a blind sense, or a pod waiting for a plac
↪ e is past the wall
            slo = float(getattr(self.a, "slo_ms", 0) or 0)
            pos = 0.0
            if slo > 0 and getattr(self.a, "latency_file", ""):
                from omni_controller.muscles import latency_window

                w = latency_window(self.a.latency_file, getattr(self.a
↪ , "latency_window_s", 60.0))["ms"]
                bare = slo / 10.0
                mean = sum(w) / len(w) if w else (p95 or 0.0)
                pos = max(0.0, (mean - bare) / (slo - bare))
            if blind.get("latency", False) or s["pending"] > 0 or (p95

↪  is not None and slo > 0 and p95 >= slo):
                pos = 1.0
            if self.edge is not None:
                # the v5 edge: a breach while machines are given back
↪ teaches the cushion a floor; then the balance point
                # and the wall move with the service's own climb and w

↪ obble (never behind the earlier law's own point
                # unless that lesson says so)
                if breach_now and self.n_native is not None and n < se
↪ lf.n_native:
                    self.edge.learn_fall(0.1 * self.pace)           #
↪ 0.1 a minute of breach (omni-v16)

                center = self.edge.update(pos)
                base = self.edge_base if self.edge.floor <= 0.0 else 0
↪ .0
                self.compass_law.band.center = min(0.995, max(0.0, max
↪ (base, center)))
                self.compass_law.band.cushion = min(0.05, max(0.005, s
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ elf.edge.react_part))
            F = self.compass_law.force(pos)
            self._own_F = F            # the same force moves a worklo
    ↪ ad's target of its own (_own_pods)
            # the verdict: the response times of the requests served s
    ↪ ince the last decision, at the machines the pool

            # stood at; then how many machines may be given back at al
    ↪ l (or the count a trial needs)
            if self.n_native is None:
                self.n_native = n
            self.verdict.observe(self._new_latencies())
            # a pod that waited for a place since the last decision, s

    ↪ een by a fast check, is harm as much as one waiting now
            waited, self.waited = self.waited or s["pending"] > 0, Fal
    ↪ se
            calm = self.compass_law.p < self.compass_law.band.wall_hig
    ↪ h and not waited and not breach_now
            if waited and self.verdict.allowed > 0:

                # the immune reaction (omni-v19; Omni direct's own, om
    ↪ ni_orchestrator/control.py): work waited under an
                # allowed depth, so one step goes back toward native a
    ↪ nd is not tried again until the recheck has passed
                self.verdict.allowed -= 1
                self.verdict.refused_until[self.verdict.allowed + 1] =

    ↪  self.verdict.t + self.verdict.recheck
                self.audit({"verdict": {"verdict": "depth handed back:
    ↪  a pod waited for a place", "step": self.verdict.allowed + 1},
                            "machines_given_back_allowed": self.verdic
    ↪ t.allowed, "machines_native": self.n_native})
            deepest, trial, ev = self.verdict.tick(calm)

            if self.dial == "max":
                deepest, trial = 0, False          # the max never giv
    ↪ es back a machine the cluster had on its own
            if ev:
                self.audit({"verdict": ev, "state": self.verdict.state
    ↪ , "machines_given_back_allowed": self.verdict.allowed,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                           "machines_native": self.n_native})
         if self.compass_law.p >= self.compass_law.band.wall_high:
             rec_n = max(floor, n + 1)
↪      # fail up: one machine more at once
         elif trial:
             rec_n = max(floor, self.n_native - deepest)

↪      # the trial's count (one machine per decision below)
         elif F < COMPASS_RELEASE and self.compass_law.p < self.com
↪ pass_law.band.center and self.n_native - (n - 1) <= deepest:
             rec_n = max(floor, n - 1)
↪      # one back, if the verdict and the release gate agree
         else:

             rec_n = max(floor, n)
         rec_n = max(self.a.min_nodes, min(self._max_nodes(s), rec_
↪ n))
         compass_rec = {"position": round(self.compass_law.p, 4), "
↪ velocity": round(self.compass_law.v, 4), "force": round(F, 4),
                        "center": round(self.compass_law.band.center,

↪ 4), "wall": round(self.compass_law.band.wall_high, 4),
                        "verdict_state": self.verdict.state, "machines
↪ _given_back_allowed": self.verdict.allowed}
     scaling_up = any(int(h.get("status", {}).get("desiredReplicas"
↪ , 0) or 0) > int(h.get("status", {}).get("currentReplicas", 0) o
↪ r 0)

                         for h in s["hpas"])
     gate = node_release_gate(n, per_node, s["used_m"], s["pending"
↪ ], scaling_up, breach_now, rho, auth_n,
                              senses_live=not any(blind.values()),
↪ last_command_landed=nodes_landed)
     # the immune memory (omni-v19): the largest rise in requests s

↪ een inside two decisions, about the time a new machine
     # takes to boot; a machine is given back only when the machine
↪ s left hold the requests and a reserve of the larger of
     # one machine and that rise, so the next rise of the same size
↪  lands on a machine already up, as it does on native's
     if s["req_m"] > 0:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.req_seen = (self.req_seen + [float(s["req_m"])])[-3:]
        self.burst_m = max(self.burst_m, float(s["req_m"]) - min(s
↪ elf.req_seen))
    reserve = max(per_node, self.burst_m)
    native_line = float(getattr(self.a, "native_release_util", 0.5
↪ ) or 0.5)

    if rec_n < n and (s["req_m"] + reserve > (n - 1) * per_node or
↪  s["req_m"] > native_line * (n - 1) * per_node):
        self.audit({"immune_memory": {"kept": n, "requests_m": int
↪ (s["req_m"]), "reserve_m": int(reserve),
                                      "largest_rise_m": int(self.b
↪ urst_m), "room_after_m": int((n - 1) * per_node),

                                      "native_line": native_line}}
↪ )
        rec_n = n            # the machines left would not hold th
↪ e requests and the reserve, or would stand thinner than
                             # native's own autoscaler lets them:
↪ the machine stays

    if getattr(self, "verdict", None) is not None and self.verdict
↪ .allowed > 0 and rec_n <= n and s["req_m"] > 0:
        # while machines stand given back, the same rule every dec
↪ ision: when the machines in service no longer hold the
        # requests and the reserve, or stand thinner than native's
↪  own line (native's own autoscaler may have removed one

        # of its own meanwhile), one machine comes back now, befor
↪ e a pod waits for it
        need = max(int(math.ceil((s["req_m"] + reserve) / per_node
↪  - 1e-9)),
                   int(math.ceil(s["req_m"] / (native_line * per_n
↪ ode) - 1e-9)))

        if need > n:
            self.audit({"immune_memory": {"machine_back": n + 1, "
↪ requests_m": int(s["req_m"]), "reserve_m": int(reserve),
                                          "native_line": native_li
↪ ne, "machines_needed": need}})
            rec_n = min(self._max_nodes(s), n + 1)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if rec_n < n and not gate["ok"]:
            rec_n = n            # nervous system: the machine organ m
    ↪ ay not give a machine back now (reason audited)
        elif rec_n < n and self.last_release is not None and self.deci
    ↪ sions - self.last_release < self.n_release:
            rec_n = n            # at most one machine a minute, howev
    ↪ er often the governor decides (v7)
        elif rec_n < n:
            rec_n = n - 1        # one machine at a time: release is t
    ↪ he slow, reversible direction
            self.last_release = self.decisions
        # cruise (rule 7): work waiting for a place, decision after de
    ↪ cision, so every machine goes in service and stays
        # there, without second-guessing, until the line has been empt
    ↪ y for as many decisions
        ca = self.n_cruise_after
        if ca > 0 and self.a.mode == "nodepool":
            self.cruise_hits = self.cruise_hits + 1 if s["pending"] >
    ↪ 0 else 0
            if not self.cruise and self.cruise_hits >= ca:
                self.cruise = True; self.cruise_clear = 0
                self.audit({"cruise": "on", "pending": s["pending"]})
            elif self.cruise:
                self.cruise_clear = self.cruise_clear + 1 if (s["pendi
    ↪ ng"] == 0 and not scaling_up) else 0
                if self.cruise_clear >= ca:
                    self.cruise = False
                    self.audit({"cruise": "off", "pending": s["pending
    ↪ "]})
            if self.cruise:
                rec_n = min(self._max_nodes(s), n + max(1, int(self.a.
    ↪ max_node_step)))   # toward the top, a step a decision
        # idle (rule 8: one pedal, lifted all the way): nothing waitin
    ↪ g, no breach, and the sensed services' demand at
        # zero: straight to the breathing floor in one move (the body
    ↪ idles, never at zero, and wakes straight to the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    # work when it comes). Every check of the release gate still h
↪ olds except one machine per decision
    eb = float(getattr(self.a, "idle_demand", 0.05) or 0)
    if eb > 0 and self.a.mode == "nodepool" and not self.cruise an
↪ d rec_n > self.a.min_nodes and s["pending"] == 0 \
            and not breach_now and not scaling_up:

        sensed = [h for h in s["hpas"] if self._sensed(h)]
        dem = [self._app_demand(h) for h in sensed]
        dem = [x for x in dem if x is not None]
        idle = bool(sensed) and all(self._at_idle(h) for h in sens
↪ ed)
        g_ok = gate["ok"] or gate["reason"] == "release permitted"

        if dem and (max(dem) <= eb or idle) and g_ok and not any(b
↪ lind.values()) and nodes_landed:
            low = max(self.a.min_nodes, floor)
            # the machines left must carry what runs now at or und
↪ er the engine's utilisation target
            while low < rec_n and s["used_m"] / max(low * per_node
↪ , 1e-9) > rho:
                low += 1
            if low < rec_n:
                self.audit({"idle_floor": {"from": rec_n, "to": lo
↪ w, "demand": round(max(dem), 3), "at_idle": idle}})
                rec_n = low

    out = self.audit({"authority": {"calm": round(auth["scalars"][
↪ "calm"], 3), "execute": auth["execute"],
                                    "contract": {o: v.get("contrac
↪ t") for o, v in auth["organs"].items()},
                                    "scalars": {k: round(v, 3) for
↪  k, v in auth["scalars"].items()},

                                    "node_view": {"calm": round(au
↪ th_n["scalars"]["calm"], 3),
                                                  "scalars": {k: r
↪ ound(v, 3) for k, v in auth_n["scalars"].items()}},
                                    "node_gate": {"ok": gate["ok"]
↪ , "reason": gate["reason"], "util_after": round(gate["util_after
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ "], 3)},
                                        "senses": {"blind": blind, "la
    ↪ tency_age_s": None if age is None else round(age, 1), "stale": o
    ↪ bs["stale"]},
                                        "proprioception": {k: round(v,
    ↪  3) for k, v in drift.items()}}})

        # the compass: where the engine stands on the wheel, whether e
    ↪ very level is inside Omega, whether the move at a
        # boundary points inward, and the ledger step
        fill = s["req_m"] / max(1.0, s["alloc_m"])
        levels = {"machine_fill": fill}
        moves = {"machine_fill": (1.0 if rec_n < n else -1.0 if rec_n
    ↪ > n else 0.0)}
        for o in ("cpufreq", "gpu", "power"):
            env = auth["organs"].get(o, {}).get("envelope")
            if env:
                levels[f"{o}_ceiling"] = env[1]
        forced = obs["queue_ratio"] > 0.0 or s["pending"] > 0

        self.audit({"compass": self.compass.read(d["state"]["E"], d["s
    ↪ tate"]["S"], levels, moves, forced, x=self.g.x, p=self.g.p)})
        out = self.audit({"decision": {"nodes_observed": n, "nodes_rec
    ↪ ommended": rec_n,
                                        "law": "compass" if self.compas
    ↪ s_law is not None else "closure" if self.cl is not None else "go
    ↪ vernor",
                                        "compass_law": compass_rec, "hp
    ↪ a_target_recommended": round(rho, 3),
                                        "E": d["state"]["E"], "U": d["s
    ↪ tate"]["U"], "pending": s["pending"],
                                        "power_cap": round(float(d["pow
    ↪ er_cap"]), 3), "change_permitted": bool(d["change_permitted"]),
                                        "rollback_authorized": bool(d["
    ↪ rollback_authorized"]),
                                        "thermal": round(obs["thermal"]
    ↪ , 3), "security_block": obs["security_block"],
                                        "power_stress": round(power_str
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ess, 3), "latency_p95_ms": p95,
                                        "queue_ratio": round(obs["queue
    ↪ _ratio"], 3), "slo_clean": obs["slo_clean"]}, "mode": self.a.mod
    ↪ e})
        if self.a.mode in ("target", "nodepool") and getattr(self.a, "
    ↪ strict_replicas", False):

            self.strict_step(s, obs)
            self.m.push(d, obs)
        elif self.a.mode in ("target", "nodepool"):
            for h in s["hpas"]:
                idx, cur = cpu_target(h)
                if cur is None:

                    continue
                ns, name = h["metadata"]["namespace"], h["metadata"]["
    ↪ name"]
                if not self._sensed(h):
                    continue     # a service the probe does not sense:
    ↪  its HPA stays at the operator's own target

                orig = int(h["metadata"].get("annotations", {}).get(AN
    ↪ NOTATION, cur))
                # the muscle's own clock: the autoscaler's scale-down
    ↪ window (300 s unless the operator set one)
                win = float(((h["spec"].get("behavior") or {}).get("sc
    ↪ aleDown") or {}).get("stabilizationWindowSeconds", 300))

                dem = self._app_demand(h)
                if dem is not None:
                    self.demand[(ns, name)] = [(t, v) for t, v in self
    ↪ .demand.get((ns, name), []) if time.time() - t <= 2 * win] \
                        + [(time.time(), dem)]
                if int((h.get("status") or {}).get("currentReplicas",

    ↪ 0) or 0) >= int(h["spec"].get("maxReplicas", 10 ** 9)):
                    self.pinned[(ns, name)] = [t for t in self.pinned.
    ↪ get((ns, name), []) if time.time() - t <= 2 * win] + [time.time(
    ↪ )]
                # more headroom is always allowed; less never: at a gi
    ↪ ven load fewer pods always means a longer M/M/c wait
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            # (no target above the operator's keeps the wait), so
↪ a raise only spends latency.
            # Omni on top earns its keep on machines, not by packi
↪ ng the operator's pods tighter
            # the same promise in queue terms: busy = target x req
↪ uest / limit. While convey() gives the pods a

            # limit g times the operator's, the target that keeps
↪ each pod exactly as busy is g times higher
            want = int(round(100 * min(rho, orig / 100.0) * self._
↪ gain(h)))
            back = False
            if self.compass_law is not None and getattr(self, "cru

↪ ise", False):
                # cruise (amendment 12): the service's own autosca
↪ ler runs it as it would alone (the operator's target);
                # Omni only holds every machine in service while t
↪ he queue drains
                self.compass_law_x[(ns, name)] = orig * self._gain

↪ (h)
                want = orig
            elif self.compass_law is not None:
                # the compass's push and pull on the target, insid
↪ e its cover [60% of the operator's, the operator's]
                # (in queue terms, times the conveyed gain): a low

↪ er target is more pods, so the up force lowers it
                g_ = self._gain(h); hi_t = orig * g_; lo_t = max(1
↪ 0.0, 0.6 * orig) * g_
                x = self.compass_law_x.get((ns, name), hi_t)
                load = not any(blind.values()) and s["pending"] ==
↪  0 and not obs["slo_clean"]

                if self.compass_law.p >= self.compass_law.band.wal
↪ l_high and load:
                    # fail up: the most pods the cover allows (pla
↪ ced below as far as the machines can hold them)
                    x = lo_t
                elif self.compass_law.p >= self.compass_law.band.w
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ all_high:
                        # past the wall, but not from load: a blind se
    ↪ nse, or pods waiting for a machine that is gone.
                        # More pods answer neither, so fail up is nati
    ↪ ve's own target, as on the card, where fail up is
                        # the card's own settings

                        x = hi_t; back = x != self.compass_law_x.get((
    ↪ ns, name), hi_t)
                    elif x < hi_t and self.compass_law.p < self.compas
    ↪ s_law.band.center and obs["slo_clean"] and s["pending"] == 0:
                        # responses back inside the band, nothing wait
    ↪ ing. If the demand that called the extra pods has

                        # stopped growing (a fault, a spike that passe
    ↪ d), they were for it only: the operator's own
                        # target returns at once. While the demand sti
    ↪ ll grows they stay, so the autoscaler does not
                        # remove them and start them again on the next
    ↪  rise

                        if not self._rising((ns, name), win):
                            x = hi_t; back = True
                    else:
                        F_ = compass_rec["force"]
                        x = x - (COMPASS_UP if F_ > 0 else COMPASS_DOW
    ↪ N) * F_ * (hi_t - lo_t) * self.step_scale

                    # the immune memory for pods (omni-v19): the large
    ↪ st rise in this service's demand seen inside two decisions
                    dem_now = self._app_demand(h)
                    if dem_now and dem_now > 0:
                        hist = (self.dem_seen.get((ns, name), []) + [d
    ↪ em_now])[-3:]

                        self.dem_seen[(ns, name)] = hist
                        self.dem_rise[(ns, name)] = max(self.dem_rise.
    ↪ get((ns, name), 1.0), dem_now / min(hist))
                    if self._held_native((ns, name)):
                        # a pod waited while the target stood below th
    ↪ e operator's: native's target is held (omni-v19)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                    x = hi_t
                if self._immune_native(h, ns, name, x, hi_t, lo_t,
↪  p95):
                    x = hi_t; back = x != self.compass_law_x.get((
↪ ns, name), hi_t)
                x = max(lo_t, min(hi_t, x))

                if x < hi_t and not back:
                    # every push for more pods, the fail-up's and
↪ the compass's alike, only as far as the machines in
                    # service can place them; the machines the res
↪ t need are asked for at once (omni-v19, Omni direct's
                    # rule: a unit never waits for a machine; a po

↪ d with no machine serves nothing, it only waits), and
                    # the pods follow as the room appears
                    x = max(x, self._placeable_target(h, s, x, hi_
↪ t, self.dem_rise.get((ns, name), 1.0)))
                self.compass_law_x[(ns, name)] = x
                self._replica_room(h, ns, name, win, obs, s)

                want = int(round(x))
            want_h = want if obs["slo_clean"] else min(want, orig)
            if abs(cur - want_h) < self.a.min_target_change and no
↪ t (not obs["slo_clean"] and cur > orig):
                continue
            if cur == want_h:

                continue
            if want_h > cur and not back and not self._steady((ns,
↪  name), win):
                continue     # a raise (fewer pods) waits for a de
↪ mand that has held still for one window
            if want_h > cur and self.a.coast_step > 0:

                # coasting (rule 6): off the gas, the target eases
↪  toward fewer pods by at most coast_step points a
                # window, never in one jump; a demand that comes b
↪ ack half way finds the pods still running and is
                # met at once by the lower target, with no pod sta
↪ rted again. More pods (a lower target) is never
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                # limited: the gas is always immediate
                # (a raise comes at most once a window, below, so
↪ the step is per raise at any period: omni-v16)
                want_h = min(want_h, cur + max(1, int(round(self.a
↪ .coast_step))))
            # the autoscaler takes its window to answer a target.
↪ A raise inside that window moves the muscle
            # mid-movement and removes pods it then starts again,
↪ so a raise is held for one window. A lower target
            # (more pods, the safe direction) is never held: on a
↪ step up the pods are asked for at once, before the
            # line is missed. A response-time breach returns the o
↪ perator's target at once
            last = self.target_at.get((ns, name))
            # handing the operator's own target back is never held
↪ : it is where native stands
            if obs["slo_clean"] and last is not None and time.time
↪ () - last < win and not back and want_h > cur:
                continue
            if "replicas" in self.a.native_muscles:
                continue    # omni-v14: the replicas are left nat
↪ ive; the decision is audited, nothing written
            self.target_at[(ns, name)] = time.time()
            self.changed.setdefault((ns, name), int(h["metadata"].
↪ get("annotations", {}).get(ANNOTATION, cur)))
            self.k.write(["annotate", "hpa", name, "-n", ns, "--ov
↪ erwrite", f"{ANNOTATION}={self.changed[(ns, name)]}"], "record o
↪ riginal target")
            self.k.write(["patch", "hpa", name, "-n", ns, "--type=
↪ json", "-p",
                          json.dumps([{"op": "replace", "path": f"
↪ /spec/metrics/{idx}/resource/target/averageUtilization", "value"
↪ : want_h}])],
                         f"HPA target to rho* = {want_h}%" + ("" i
↪ f obs["slo_clean"] else " (SLO reflex: not tighter than native)"
↪ ))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            self.cmd_hpa[(ns, name)] = want_h; self.cmd_at[(ns, na
↪ me)] = self.decisions
        self.m.push(d, obs)
    if self.own and self.a.mode in ("target", "nodepool"):
        self._own_pods(s, obs, rho)
    if self.a.mode in ("target", "nodepool"):

        self.pod_reflex()
    want_n = rec_n
    if rec_n > n and self.last_add is not None and self.decisions
↪ - self.last_add < self.n_add:
        # machines past the scheduling floor are added at most --m
↪ ax-node-step a minute, however often the governor

        # decides (omni-v16; the pool gives back at most one a min
↪ ute since v7); the floor's machines at once
        want_n = max(n, min(rec_n, floor))
    if self.a.mode == "nodepool" and self.a.node_scale_cmd and wan
↪ t_n != n:
        cfg = SimpleNamespace(minimum_nodes=self.a.min_nodes, maxi

↪ mum_nodes=self._max_nodes(s))
        if want_n > n:
            self.last_add = self.decisions
        acts, hits = enforce([{"action": "nodes", "target": want_n
↪ , "direction": 1 if want_n > n else -1}],
                             {"actual_nodes": n, "power_cap": 1.0}

↪ , {"power_stress": power_stress, "security_block": obs["security
↪ _block"]}, cfg,
                             ShieldLimits(power_limit=1e9, max_nod
↪ e_step=max(self.a.max_node_step, floor - n)))
        for act in acts:
            if self._pool_command(int(act["target"]), n, "node poo

↪ l size", shield_interventions=hits) and not self.a.dry_run:
                self.cmd_nodes = int(act["target"]); self.cmd_at["
↪ nodes"] = self.decisions
    if convey_due and getattr(self.a, "latency_file", "") and not
↪ self.cruise and not self.killed():
        self._convey()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.rec_n = rec_n
        return out


def parser():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--mode", choices=["observe", "target", "nodepool"
    ↪ ], default="observe")
    # the heartbeat, a dial (omni-v17, the founder's word of 22 Octobe
    ↪ r, afternoon): the main engine decides every 60 s, and any
    # beat can be set (5, 15, 30, 60, 75, 90, 120 s and every value be
    ↪ tween), each shown beside native; the fast checks run
    # every 5 s at every beat. Every rule counts seconds (omni-v16), s
    ↪ o either can be turned alone
    ap.add_argument("--interval", type=float, default=60.0, help="seco
    ↪ nds between governor decisions (the heartbeat, a dial; the main
    ↪ engine 60 s)")
    ap.add_argument("--floor-interval", type=float, default=5.0, help=
    ↪ "seconds between the fast checks: the scheduling floor, the pod
    ↪ reflex, the conveyance's clock (nodepool mode)")
    ap.add_argument("--convey-every-s", type=float, default=15.0,
                    help="seconds between conveyance passes, counted o
    ↪ n the checks (a fast check or a decision; omni-v16)")
    ap.add_argument("--iterations", type=int, default=0, help="0 = run
    ↪  until stopped")
    ap.add_argument("--dry-run", action="store_true")
    ap.add_argument("--kubectl", default="kubectl")
    ap.add_argument("--audit", default="omni_audit.jsonl")
    ap.add_argument("--kill-file", default="/tmp/omni.kill")
    ap.add_argument("--no-api-proxy", action="store_true", help="read
    ↪ through a new kubectl process every time instead of one kubectl
    ↪ proxy (the controller's own cost is higher)")
    ap.add_argument("--restore-only", action="store_true", help="put e
    ↪ very setting back from the records on the objects and exit (the
    ↪ watchdog's way back)")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ap.add_argument("--node-scale-cmd", default="")
    ap.add_argument("--cruise-after", type=int, default=int(os.environ
    ↪ ["CRUISE_AFTER"]) if os.environ.get("CRUISE_AFTER") else None,
                    help="decisions in a row with work waiting for a p
    ↪ lace before cruise puts every machine in service "
                         "(and decisions with the line empty before it
    ↪  ends); 0 turns cruise off; unset: --cruise-after-s")
    ap.add_argument("--cruise-after-s", type=float, default=120.0,
                    help="the same in seconds (v7: every rule of the g
    ↪ overnor counts in seconds, so its decision period can "
                         "change without changing any rule's time)")
    ap.add_argument("--idle-demand", "--brake-demand", dest="idle_dema
    ↪ nd", type=float,
                    default=float(os.environ.get("IDLE_DEMAND", os.env
    ↪ iron.get("BRAKE_DEMAND", 0.05))),
                    help="the sensed services' demand (in pods' worth
    ↪ of CPU) at or under which the pool idles: the "
                         "machines go straight to the breathing floor
    ↪ (one pedal lifted all the way); 0 turns it off")
    ap.add_argument("--coast-step", type=int, default=int(os.environ.g
    ↪ et("COAST_STEP", 25)),
                    help="the most the HPA target may ease toward fewe
    ↪ r pods in one autoscaler window (points of "
                         "utilisation; 0: no limit). Adding pods is ne
    ↪ ver limited")
    ap.add_argument("--min-nodes", type=int, default=int(os.environ.ge
    ↪ t("MIN_NODES", 2)),
                    help="machines always in service, ready for the ne
    ↪ xt burst (the founder's floor: two)")
    ap.add_argument("--max-nodes", type=int, default=0,
                    help="the machine lever's top; 0, the default: rea
    ↪ d live, every ready worker plus what the cluster's "
                         "autoscaler may still add")
    ap.add_argument("--max-node-step", type=int, default=2)
    ap.add_argument("--min-target-change", type=int, default=3)
    ap.add_argument("--law", choices=["governor", "compass"], default=
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ "governor",
                    help="governor (default): the engine's allocation
    ↪ law; compass: the compass law at the v5 edge on the HPA "
                         "target and the node pool")
    ap.add_argument("--allow", type=float, default=0.02, help="the mos
    ↪ t a machine given back may add to the response time")

    ap.add_argument("--verdict-samples", type=int, default=200, help="
    ↪ requests measured before and after a machine is given back on tr
    ↪ ial")
    ap.add_argument("--verdict-every", type=int, default=None, help="d
    ↪ ecisions between trials (unset: --verdict-every-s)")
    ap.add_argument("--verdict-every-s", type=float, default=600.0, he

    ↪ lp="seconds between give-back trials")
    ap.add_argument("--verdict-recheck", type=int, default=None, help=
    ↪ "decisions before a refused machine is tried again (unset: --ver
    ↪ dict-recheck-s)")
    ap.add_argument("--verdict-recheck-s", type=float, default=7200.0,
    ↪  help="seconds before a refused machine is tried again")

    ap.add_argument("--verdict-trial-s", type=float, default=1200.0, h
    ↪ elp="seconds a give-back trial's phase may take before it is aba
    ↪ ndoned")
    ap.add_argument("--give-back-only", action="store_true",
                    help="the cloud's rule in one word (v7): the machi
    ↪ nes alone, given back after their own trial and woken on a "

                         "breach; no autoscaler moved, no processor co
    ↪ nveyed, no power capped, the tune-down; so the bill can "
                         "only fall or stay")
    ap.add_argument("--compass-tau", type=float, default=60.0, help="s
    ↪ econds the service takes to follow a lever (the compass's dampin
    ↪ g)")

    ap.add_argument("--compass-center", type=float, default=None,
                    help="where the compass holds the service (0 the b
    ↪ are service time, 1 the SLO); default 0.3 at the max, "
                         "0.4 at the tune-down")
    ap.add_argument("--dial", choices=["max", "save"], default=os.envi
    ↪ ron.get("OMNI_DIAL", "max"),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                help="max (default): the most work, faster, never
↪ a machine given back below the cluster's own; save: "
                     "the tune-down (the v5 edge toward the line,
↪ machines given back where the response time allows)")
ap.add_argument("--replica-ceiling", type=int, default=-1,
                help="the most replicas Omni-Compass may raise a s

↪ ensed HPA's cap to while it binds (compass law): "
                     "-1, the default, reads it live (what the mac
↪ hines can hold, one pod under); N, a ceiling of the "
                     "operator's own; 0 keeps the operator's cap u
↪ nmoved")
ap.add_argument("--sensed", default="",

                help="ns/deployment,... whose response time the --
↪ latency-file measures; only their HPAs are moved "
                     "(empty: every HPA, the single-service case)"
↪ )
ap.add_argument("--closure", default="", help="JSON with the closu
↪ re-law setting (e.g. tuning/GLOBAL_LEAGUE_PREREGISTRATION.json):

↪  the benchmarked law decides the node count")
ap.add_argument("--strict-replicas", action="store_true", help="st
↪ rict C: Omni-Compass sets replica counts; the HPA is pinned")
ap.add_argument("--pod-reflex-writes", action="store_true",
                help="let the pod reflex raise the HPA's replica f
↪ loor itself (default: it gauges and writes nothing)")

ap.add_argument("--reflex-window-s", type=float, default=30.0, hel
↪ p="window of probe samples the fast pod reflex reads")
ap.add_argument("--strict-window", type=int, default=None, help="d
↪ ecisions a scale-down waits for (unset: --strict-window-s)")
ap.add_argument("--strict-window-s", type=float, default=300.0, he
↪ lp="seconds a scale-down waits for (highest recent recommendatio

↪ n)")
ap.add_argument("--headroom", type=float, default=0.5, help="spare
↪  capacity kept above pod requests (0.5 = 50%%, the default)")
ap.add_argument("--native-release-util", type=float, default=float
↪ (os.environ.get("NATIVE_RELEASE_UTIL", 0.5)),
                help="a machine is given back only while the machi
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ nes left stay at or under this share of requests to allocatable:
    ↪  native's own line (the Cluster Autoscaler's scale-down utilizat
    ↪ ion threshold, 0.5 by its documented default; set it to the clus
    ↪ ter's own), so Omni-Compass never leaves the machines thinner th
    ↪ an native's autoscaler would")
    ap.add_argument("--active-nodes-only", action="store_true",
                    help="count only nodes in service (schedulable, or
    ↪  cordoned but still carrying work) and the pods and usage on the
    ↪ m")
    ap.add_argument("--node-restore-cmd", default="", help="command ru
    ↪ n once when the reset fires, returning the node pool to native")
    add_muscle_args(ap)

    ap.add_argument("--power-cmd", default="")
    ap.add_argument("--site-limit-w", type=float, default=0.0)
    ap.add_argument("--own-every-s", type=float, default=15.0,
                    help="seconds between two reconciliations of the p
    ↪ od muscle of its own on the fast checks (the pod autoscaler's sy
    ↪ nc)")

    ap.add_argument("--own-replicas", default=os.environ.get("OMNI_OWN
    ↪ _REPLICAS", ""),
                    help="ns/deployment,... no autoscaler governs, who
    ↪ se replica count Omni-Compass sets itself through the "
                         "workload's own scale (omni-v19, Omni-Compass
    ↪  by itself): its target from the workload's "
                         "omnicompass.io/target-utilization annotation
    ↪  (else --own-target), its range from omnicompass.io/replica-rang
    ↪ e")
    ap.add_argument("--own-target", type=float, default=None,
                    help="the busy share (percent of the CPU request)
    ↪ a workload of its own is held at when it names none")

    ap.add_argument("--native", default=os.environ.get("OMNI_NATIVE",
    ↪ ""),
                    help="muscles left native, never written (omni-v14
    ↪ ): a comma list of " + ", ".join(MUSCLES))
    return ap
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def safe_step(c, fails):
    """One decision. A failed decision is recorded and skipped: it wri
    ↪ tes nothing, so the cluster keeps the last settings
    that landed, and the next decision comes at the normal cadence. Th
    ↪ ere is no automated fallback (manuscript Section

    5.8: the Unified Control Switch is mechanical, explicit, operator-
    ↪ controlled and 'does not rely on automated fallback
    inference'). Only a human flips the switch, for the whole harness
    ↪ at once: --kill-file present (or OMNI_KILL=1) turns
    Omni-Compass OFF and hands every setting back to native; removing
    ↪ it turns Omni-Compass back ON. Boundaries are never

    handled by switching anything off: the living band and the shield
    ↪ clamp every level inside its range."""
    try:
        c.step()
        return 0
    except Exception as e:

        fails += 1
        err = getattr(e, "stderr", "") or ""
        c.audit({"error": repr(e)[:500], "stderr": str(err)[-500:], "c
    ↪ onsecutive_failures": fails})
        print(f"decision failed ({fails} in a row): {e!r} {err}", file
    ↪ =sys.stderr, flush=True)

        return fails


def main(argv=None):
    a = parser().parse_args(argv)
    if a.restore_only:

        # the watchdog's way back for a controller that died: every se
    ↪ tting back to the operator's, from the records
        # kept on the objects themselves (annotations), then exit
        c = Controller(a); c.restore(); c.audit({"decision": "restore
    ↪ only: every setting handed back"})
        return 0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    master.refuse_if_off("kubernetes controller")
    c = Controller(a); i = 0; fails = 0
    args = list(sys.argv[1:] if argv is None else argv)
    master.register("kubernetes controller", restore=[[sys.executable,
    ↪  "-m", "omni_controller.controller", *args, "--restore-only"]],
                    stale_s=max(120.0, 5 * a.interval))

    import signal

    def master_off(*_):
        # the master switch pulled from the command line (tools/omni_s
    ↪ witch.py off): hand everything back now and exit
        try:

            c.restore()
        finally:
            c.audit({"decision": "master switch: OFF; every setting ha
    ↪ nded back; exiting"})
            sys.exit(0)
    signal.signal(signal.SIGTERM, master_off)

    import atexit, resource
    t0 = time.time()
    def overhead():   # the controller's own cost: CPU seconds of this
    ↪  process and every kubectl/script it ran
        me, kids = resource.getrusage(resource.RUSAGE_SELF), resource.
    ↪ getrusage(resource.RUSAGE_CHILDREN)

        cpu = me.ru_utime + me.ru_stime + kids.ru_utime + kids.ru_stim
    ↪ e
        c.audit({"overhead": {"cpu_s": round(cpu, 2), "wall_s": round(
    ↪ time.time() - t0, 1),
                              "cores_mean": round(cpu / max(1e-9, time
    ↪ .time() - t0), 4)}})

    atexit.register(overhead)
    # the loop keeps its own schedule (omni-v16): a decision every --i
    ↪ nterval s and a fast check every --floor-interval s
    # between, each slept to its time from the schedule, never --inter
    ↪ val after the last step ended, so the steps' own
    # time does not stretch every clock (before omni-v16 a decision of
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
↪  60 s came every 62 to 67 s on kind, and one of 5 s
# would have come every 7). A step that overruns its slot is follo
↪ wed at once by the next due one; missed slots are
# skipped, never run in a burst, and the lateness is in the audit
due = time.time()
while a.iterations == 0 or i < a.iterations:

    master.heartbeat()
    t_dec = time.time()
    fails = safe_step(c, fails); i += 1
    c.audit({"decision_ms": round(1000 * (time.time() - t_dec), 1)
↪ })
    if a.iterations == 0 or i < a.iterations:

        nxt = due + a.interval
        if a.mode == "nodepool":
            k = 1
            while due + k * a.floor_interval < nxt - 1e-6:
                slot = due + k * a.floor_interval
                if time.time() > slot + a.floor_interval:

↪         # this slot is long gone: skip it
                    k += 1
                    continue
                time.sleep(max(0.0, slot - time.time()))
                try:
                    c.floor_step()

                except Exception as e:
                    c.audit({"error": "floor check: " + repr(e)[:5
↪ 00], "stderr": str(getattr(e, "stderr", "") or "")[-500:]})
                k += 1
        late = time.time() - nxt
        if a.interval > 0 and late > a.interval:

            c.audit({"behind_s": round(late, 1), "why": "a step ou
↪ tran its slot; the slots missed are skipped, not run at once"})
            nxt = time.time()
        time.sleep(max(0.0, nxt - time.time()))
        due = nxt
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
if __name__ == "__main__":
    main()
```

H.4 omni_controller/gpu_compass.py

445 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on one GPU through two wires: the compass law (omnicom
    ↪ pass/compass_law.py) on a real card.

The card keeps its own control: NVIDIA's firmware still boosts and sti
    ↪ ll protects the chip. Omni holds two settings the
card already accepts, and nothing else:
  up wire    the clock ceiling: how high boost may climb (nvidia-smi -
    ↪ lgc MIN,MAX; -rgc resets it)
  down wire  the power limit: the lid above what that ceiling draws (n
    ↪ vidia-smi -pl W)

Every decision (--interval seconds):
  read      nvidia-smi: power.draw, temperature, utilization, power.li
    ↪ mit, clocks.sm; the workload's response times
  position  the service as one place in its compass, 0 calm to 1 the l
    ↪ ine: response time only (the mean over the last
            --latency-window-s, between a tenth of the line, the bare
    ↪ service time, and the line; the 95th percentile at
            or past the line, or a failed request, is past the wall).
    ↪ A card that is busy is doing its work; being busy
            is not a breach and is not read as one
  native    what the card does on its own, learned from its own meter
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
  ↪ before the compass may lower anything: while the
         ceiling is at the top and the card is busy, its clock and
  ↪ its draw (the clock its own power limit holds it
         at, and what that costs). Until --learn-samples busy readi
  ↪ ngs are in, the ceiling stays at the top
race     while the card is saturated (utilization at or over --race

  ↪ -util: work is waiting), the ceiling goes to the
         top and the lid to the start limit, so a burst is served a
  ↪ t full speed; the compass paces only the slack
force    the compass: pull to the center, push against what is risi
  ↪ ng, tanh-bounded; past the 0.95 wall: fail up
verdict  how far the ceiling may go (omnicompass/verdict.py): while

  ↪  the service is calm, a paired trial: the ceiling
         at the top until --verdict-samples requests are measured,
  ↪ then one 15 MHz step past the deepest step
         already allowed until as many again are measured; the card
  ↪ 's own time on each request (the workload's
         service_ms, the wait in the queue left out) is compared: a

  ↪ t most --allow slower and the step is allowed,
         slower than that and it is refused and not tried again for
  ↪  --verdict-recheck decisions. Where no step
         passes, the ceiling stays at the top: the card runs as it
  ↪ does alone
law      down gain 0.0125, the compass's center at the v5 edge (omn

  ↪ icompass/edge.py: 1 - (climb x reaction + 3.09 x
         wobble) of the card's own response-time position, never be
  ↪ low 0.4 unless a line crossed while the ceiling was
         held under the top has taught it otherwise; the wall moves
  ↪  out with the reaction part), the speed floor at the
         card's own busy clock (amendment 8)

steady   while the card is saturated against its own power limit (w
  ↪ ork waiting and the draw at the limit), the
         firmware boosts a step, hits the limit and is knocked back
  ↪ : a sawtooth. The ceiling is then held at the
         card's own busy clock under that limit (what the sawtooth
  ↪ averages to), so the same watts serve the work
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            without the knock-backs (amendment 9). Only under an opera
    ↪ tor's cap (the start limit under the card's
            factory limit, amendment 12): on the card's own limit the
    ↪ firmware's boost is left to serve the burst. It
            never holds under that clock, and blind always fails up
  write     up wire: the ceiling moves by the force (fast up, gently d
    ↪ own), inside its cover: from the card's own busy
            clock (never slower than native while there is work) to th
    ↪ e top; down wire: the lid at the card's own busy
            draw plus --lid-headroom, never under it and never over th
    ↪ e start limit, inside the declared envelope;
            fail up: ceiling to the top and the lid to the start limit
    ↪  at once
  guards    blind (meters unreadable, response times stale): fail up;
    ↪ heat (the card reports a thermal or hardware
            slowdown): never tighten; one writer: a power limit neithe
    ↪ r mine nor the start means another writer, so
            observe only from then on and exit 5; read-back: every wri
    ↪ te is read back
  restore   on exit, kill file or SIGTERM: clocks reset (-rgc) and the
    ↪  power limit back to the start, read back

The audit (--audit) carries the same records the bench reads from the
    ↪ one-wire governor (snapshot, decision with
telemetry and decided_by, write and actuator for each power-limit writ
    ↪ e, restored), plus clock_write records.
Exit: 0 clean, 3 restore failed, 4 a write refused, 5 another writer.
"""
from __future__ import annotations
import argparse
import json
import math
import os
import shlex
import signal
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
import subprocess
import sys
import time

from omnicompass import master
from omnicompass.compass_law import Band, CompassLaw, clamp

from omnicompass.verdict import Verdict
from omni_controller.gpu_governor import query, snapshot, throttle, sl
    ↪ owed, WriteFailed, SNAPSHOT
from omni_controller.muscles import latency_sense, latency_window

# the law (docs/GPU_PREREGISTRATION.md, amendment 8): the down gain (s
    ↪ hare of the top clock per unit of force), the compass's
# center (where it holds the response time, 0 the bare service time an
    ↪ d 1 the line) and the speed floor (a share of the
# card's own busy clock); the same in realms/gpu_card.py
LAW = {"down": 0.0125, "center": 0.4, "floor": 1.0}

def smi_run(smi, args):
    try:
        p = subprocess.run(shlex.split(smi) + args, capture_output=Tru
    ↪ e, text=True, timeout=20)

        return p.returncode, p.stderr.strip()[:300]
    except (subprocess.SubprocessError, OSError) as e:
        return -1, f"{type(e).__name__}: {e}"[:300]


def query_top_clock(smi, g):

    try:
        out = subprocess.run(shlex.split(smi) + ["-i", str(g), "--quer
    ↪ y-gpu=clocks.max.sm", "--format=csv,noheader,nounits"],
                             capture_output=True, text=True, timeout=1
    ↪ 0, check=True).stdout
        return float(out.strip().splitlines()[0])
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    except (subprocess.SubprocessError, OSError, ValueError, IndexErro
    ↪ r):
        return None


def query_default_limit(smi, g):
    """The card's factory power limit (watts), or None if the driver w
    ↪ ill not say. A start limit under it means an
    operator's cap is underneath (amendment 12)."""
    try:
        out = subprocess.run(shlex.split(smi) + ["-i", str(g), "--quer
    ↪ y-gpu=power.default_limit", "--format=csv,noheader,nounits"],
                             capture_output=True, text=True, timeout=1
    ↪ 0, check=True).stdout
        return float(out.strip().splitlines()[0])
    except (subprocess.SubprocessError, OSError, ValueError, IndexErro
    ↪ r):
        return None

def query_min_clock(smi, g):
    """The lowest graphics clock the card supports (the floor of a -lg
    ↪ c range); 300 MHz if the driver will not list them."""
    try:
        out = subprocess.run(shlex.split(smi) + ["-i", str(g), "--quer
    ↪ y-supported-clocks=graphics", "--format=csv,noheader,nounits"],
                             capture_output=True, text=True, timeout=1
    ↪ 0, check=True).stdout
        return int(min(float(x) for x in out.split() if x.strip()))
    except (subprocess.SubprocessError, OSError, ValueError):
        return 300


class GpuCompass:
    def __init__(self, a):
        self.a = a
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.g = int(str(a.gpus).split(",")[0])
        self.log = open(a.audit, "a")
        self.writes = 0
        self.foreign = False
        self.clock_set = False
        self.enforced_ok = query(a.smi, [self.g], enforced=True) is no
    ↪ t None
        s = query(a.smi, [self.g], self.enforced_ok)
        if s is None:
            raise SystemExit("nvidia-smi unreadable at start: no snaps
    ↪ hot, so I take no authority")
        snap = snapshot(a.smi, [self.g])

        self.start = s[self.g]["limit"]
        # an operator's cap underneath: the start limit under the card
    ↪ 's factory limit (unknown: treated as no cap)
        dflt = query_default_limit(a.smi, self.g)
        self.capped = dflt is not None and self.start < dflt - 1.0
        self.expect = self.start

        self.top = query_top_clock(a.smi, self.g) or s[self.g]["clock_
    ↪ mhz"]
        self.floor_w = max(float(a.floor_w or 0.0), s[self.g]["min"])
        self.c_floor = query_min_clock(a.smi, self.g)
        # the clock ceiling's cover: the card's own lowest supported c
    ↪ lock, read live (nothing fixed); an operator may set

        # a share of the top instead (--clock-min-share)
        self.c_lo = max(self.c_floor, a.clock_min_share * self.top)
        self.ceiling = self.top            # where the compass holds t
    ↪ he ceiling (continuous)
        self.written_ceiling = self.top    # what the card was last to
    ↪ ld

        self.busy_clk, self.busy_draw = [], []    # the card on its ow
    ↪ n: busy clock and busy draw, ceiling at the top
        self.prof = dict(LAW)
        if a.down_gain is not None:
            self.prof["down"] = a.down_gain
        self.verdict = Verdict(tolerance=a.allow, min_samples=a.verdic
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ t_samples, probe_every=a.verdict_every,
                               recheck=a.verdict_recheck)
        self.svc_t = None                  # elapsed_seconds of the la
    ↪ st request whose service time the verdict has seen
        self.brain = CompassLaw(Band(0.0, 1.0, center=self.prof["cente
    ↪ r"]), dt=a.interval, tau=2.0 * a.interval, kp=1.0, smooth=0.3)

        self.brain.kd *= 3.0
        # the v5 edge: the clock ceiling answers within the compass's
    ↪ own damping time (2 decisions)
        from omnicompass.edge import Edge
        self.edge = Edge(react=2.0)
        self.audit({"snapshot": {str(self.g): {"limit_w": self.start,

    ↪ "min_limit_w": s[self.g]["min"],
                                               "enforced_w": s[self.g]
    ↪ ["enforced"] if self.enforced_ok else None,
                                               "clock_top_mhz": self.t
    ↪ op,
                                               **{f: snap[f][str(self.

    ↪ g)] for f in SNAPSHOT}}},
                    "engine": "compass, two wires", "mode": a.mode, "l
    ↪ aw": self.prof,
                    "verdict": {"allow": a.allow, "samples": a.verdict
    ↪ _samples, "every": a.verdict_every,
                                "recheck": a.verdict_recheck},

                    "covers": {"clock_mhz": [round(self.c_lo), round(s
    ↪ elf.top)], "power_w": [self.floor_w, self.start]}})
        if a.mode == "cap" and snap["power.management"][str(self.g)].l
    ↪ ower() != "enabled":
            self.audit({"refused": "power management not Enabled: a wr
    ↪ itten limit would not bind"})

            raise SystemExit("power management not Enabled: cap mode r
    ↪ efused, no authority taken")

    def audit(self, rec):
        self.log.write(json.dumps({"time": time.time(), **rec}) + "\n"
    ↪ ); self.log.flush()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def position(self, util):
        a = self.a
        ls = latency_sense(a.latency_file, a.latency_window_s) if a.la
    ↪ tency_file and a.slo_ms else None
        if ls is not None and ls["blind"]:

            return None, ls
        resp = 0.0
        if ls is not None:
            # the position is the mean response time of the window bet
    ↪ ween the bare service time and the line (what
            # the model reads); the 95th percentile at or past the lin

    ↪ e, or any failed request, is past the wall
            bare = a.slo_ms / 10.0
            w = latency_window(a.latency_file, a.latency_window_s)["ms
    ↪ "]
            mean = sum(w) / len(w) if w else ls["p95"]
            resp = (mean - bare) / (a.slo_ms - bare)

            if ls["fail"] or ls["p95"] >= a.slo_ms:
                resp = max(resp, 1.0)
        return resp, ls


    def service_costs(self):
        """The card's own time (ms) on each request finished since the

    ↪  last decision (the workload's service_ms)."""
        import csv
        try:
            rows = list(csv.DictReader(open(self.a.latency_file)))
        except (OSError, TypeError):
            return []

        out, last = [], self.svc_t
        for row in rows:
            t, svc = row.get("elapsed_seconds"), row.get("service_ms")
            if row.get("ok") != "1" or not t or not svc:
                continue
            t = float(t)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            if last is None or t > last:
                out.append(float(svc))
                self.svc_t = t if self.svc_t is None else max(self.svc
↪ _t, t)
        if last is None:                    # the first read only finds
↪  where the file stands
            return []
        return out

    def learn(self, r):
        """The card on its own: while the ceiling is at the top and th
↪ e card is busy, its clock and draw are native's."""
        if r is not None and self.written_ceiling >= self.top and r["l
↪ imit"] >= self.start - 1.0 and r["util"] >= 0.9:
            self.busy_clk = (self.busy_clk + [r["clock_mhz"]])[-200:]
            self.busy_draw = (self.busy_draw + [r["draw"]])[-200:]

    def native(self):
        """(busy clock, busy draw) of the card on its own, or (None, N
↪ one) until enough busy readings are in."""
        if len(self.busy_clk) < self.a.learn_samples:
            return None, None
        c, d = sorted(self.busy_clk), sorted(self.busy_draw)
        return c[len(c) // 2], d[int(0.9 * (len(d) - 1))]

    def write_clock(self, mhz, why):
        mhz = int(round(mhz))
        cmd = ["-i", str(self.g), "-lgc", f"{self.c_floor},{mhz}"]
        if self.a.mode != "cap" or self.foreign:
            self.audit({"would_clock_write": cmd, "why": why}); return
        rc, err = smi_run(self.a.smi, cmd)
        self.audit({"clock_write": cmd, "why": why, "rc": rc, "stderr"
↪ : err})
        if rc != 0:
            raise WriteFailed(f"nvidia-smi -lgc {self.c_floor},{mhz} r
↪ eturned {rc}: {err}")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.clock_set = True
        self.writes += 1

    def write_limit(self, w, why):
        w = int(w)
        cmd = ["nvidia-smi", "-i", str(self.g), "-pl", str(w)]

        if self.a.mode != "cap" or self.foreign:
            self.audit({"would_write": cmd, "why": why}); return
        self.audit({"write": cmd, "why": why})
        t0 = time.time()
        rc, err = smi_run(self.a.smi, ["-i", str(self.g), "-pl", str(w
    ↪ )])

        act = {"gpu": self.g, "requested_w": w, "t_requested": t0, "rc
    ↪ ": rc, "stderr": err}
        if rc != 0:
            self.audit({"actuator": act, "write_failed": cmd})
            raise WriteFailed(f"nvidia-smi -pl {w} returned {rc}: {err
    ↪ }")

        self.writes += 1
        s = query(self.a.smi, [self.g], self.enforced_ok)
        if s is not None:
            back = s[self.g]
            act.update({"readback_w": back["limit"], "enforced_w": bac
    ↪ k["enforced"], "t_readback": time.time(),

                        "realized": abs(back["limit"] - w) < 1.0, "ove
    ↪ rride": back["enforced"] < back["limit"] - 1.0})
            if act["realized"]:
                act["delay_s"] = round(act["t_readback"] - t0, 3)
                self.expect = w
        self.audit({"actuator": act})

    def step(self):
        a, g = self.a, self.g
        s = query(a.smi, [g], self.enforced_ok)
        if s is None:
            p, ls, r = None, None, None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        else:
            r = s[g]
            if abs(r["limit"] - self.expect) >= 1.0 and not self.forei
    ↪ gn:
                self.foreign = True
                self.audit({"foreign_writer": {"gpu": g, "reads_w": r[
    ↪ "limit"], "expected_w": self.expect},
                            "action": "observe only from now on; both
    ↪ wires left to the other writer"})
            p, ls = self.position(r["util"])
        self.learn(r)
        n_clk, n_draw = self.native()
        thr = throttle(a.smi, [g]) if r is not None else None
        heat = slowed((thr or {}).get(g))
        saturated = r is not None and r["util"] >= a.race_util
        # the verdict: the card's own time on the requests since the l
    ↪ ast decision, at the step the ceiling stood at;
        # then the deepest step the compass may use now, or the step a
    ↪  trial needs
        self.verdict.observe(self.service_costs())
        if p is not None:
            # the v5 edge: past the line while the ceiling was held un
    ↪ der the top teaches the cushion a floor; then the
            # balance point and the wall move with the card's own clim
    ↪ b and wobble
            if p >= 1.0 and self.written_ceiling < self.top:
                self.edge.learn_fall(0.1)
            center = self.edge.update(p)
            base = self.prof["center"] if self.edge.floor <= 0.0 else
    ↪ 0.0
            self.brain.band.center = min(0.995, max(0.0, max(base, cen
    ↪ ter)))
            self.brain.band.cushion = min(0.05, max(0.005, self.edge.r
    ↪ eact_part))
        calm = p is not None and p < self.brain.band.wall_high and not
    ↪  saturated
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        deepest, trial, ev = self.verdict.tick(calm)
        if getattr(self.a, "dial", "max") == "max":
            # the max: the card's own clock ceiling stays at the top w
    ↪ hile it serves (its firmware boosts to its own power
            # limit: native already runs it flat out), so no step belo
    ↪ w the top is tried; the steady hold under an

            # operator's cap (no knock-backs) and fail up still apply
            deepest, trial = 0, False
        if ev:
            self.audit({"verdict": ev, "state": self.verdict.state, "d
    ↪ eepest_step": self.verdict.allowed})
        at_limit = r is not None and r["draw"] >= 0.97 * r["limit"]

        if p is not None and saturated and at_limit and n_clk is not N
    ↪ one and self.capped:
            # saturated against the card's own limit: hold the ceiling
    ↪  at the card's own busy clock under that limit,
            # no knock-backs; the lid stays at the start limit
            self.brain.force(p)

            ceiling = clamp(round(n_clk / a.min_change_mhz) * a.min_ch
    ↪ ange_mhz, self.c_lo, self.top)
            lid = self.start
            who = "steady_under_limit"
        elif p is None or p >= self.brain.band.wall_high or saturated:
            # fail up past the wall or blind; and race while work wait

    ↪ s (the card saturated: a queue is forming), so a
            # burst is always served at full speed and the compass pac
    ↪ es only the slack between bursts
            if p is not None:
                self.brain.force(p)
            ceiling, lid = self.top, self.start

            who = "blind_fail_up" if p is None else "fail_up" if p >=
    ↪ self.brain.band.wall_high else "race"
        elif trial:
            self.brain.force(p)
            ceiling = self.top - deepest * a.min_change_mhz
            lid = self.start
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            who = "verdict_trial"
        else:
            F = self.brain.force(p)
            # the speed floor: never under the clock the card reaches
    ↪ on its own while busy (until that is learned,
            # the top), so work waiting on the card is never served sl
    ↪ ower than native
            c_floor = self.top if n_clk is None else max(self.c_lo, mi
    ↪ n(self.top, n_clk * self.prof["floor"]))
            # and never past the deepest step the verdict has allowed
    ↪ (none allowed: the ceiling stays at the top)
            c_floor = max(c_floor, self.top - deepest * a.min_change_m
    ↪ hz)
            # the ceiling moves in whole clock steps, as the card's ow
    ↪ n clock does: the force times the gain (a share
            # of the top clock per unit of force) rounded to whole --m
    ↪ in-change-mhz steps; a pull under half a step
            # moves nothing and is not stored up, so a calm card is pa
    ↪ ced only when the pull is clearly down
            down = self.prof["down"]
            step = (a.up_gain if F > 0 else down) * F * self.top
            step = round(step / a.min_change_mhz) * a.min_change_mhz
            ceiling = clamp(self.ceiling + step, c_floor, self.top)
            if heat and ceiling < self.ceiling:
                ceiling, who = self.ceiling, "thermal_hold"
            else:
                who = "compass"
            # the lid: the start limit scaled to what a fully busy car
    ↪ d draws at this ceiling (a fifth of the draw does not
            # scale with the clock; the rest goes as clock^2.5, clock
    ↪ times voltage squared), plus headroom; at the top
            # clock the lid is the start limit, so the lid never adds
    ↪ a hammer of its own
            # the lid: never under what the card draws on its own whil
    ↪ e busy (its own meter, not a curve), plus
            # headroom; the start limit until that is learned
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        lid = self.start if n_draw is None else clamp(math.ceil(n_
↪ draw * (1.0 + a.lid_headroom)), self.floor_w, self.start)
    rec = {"decision": {str(g): {"telemetry": None if r is None el
↪ se {
                "util": r["util"], "draw_w": r["draw"], "temp_c": r
↪ ["temp"], "limit_w": r["limit"],

                "enforced_w": r["enforced"], "clock_mhz": r["clock_
↪ mhz"], "throttle": (thr or {}).get(g)},
            "position": None if p is None else round(p, 4), "veloci
↪ ty": round(self.brain.v, 4),
            "center": round(self.brain.band.center, 4), "wall": rou
↪ nd(self.brain.band.wall_high, 4),

            "latency_p95_ms": None if not ls else ls.get("p95"),
            "ceiling_mhz": round(ceiling), "want_w": int(lid), "dec
↪ ided_by": who,
            "native_busy_clock_mhz": n_clk, "native_busy_draw_w": n
↪ _draw,
            "verdict_state": self.verdict.state, "verdict_deepest_s

↪ tep": self.verdict.allowed}}}
    self.audit(rec)
    self.ceiling = ceiling
    if abs(ceiling - self.written_ceiling) >= a.min_change_mhz or
↪ (ceiling >= self.top and self.written_ceiling < self.top):
        if ceiling >= self.top and self.clock_set and a.mode == "c

↪ ap" and not self.foreign:
            rc, err = smi_run(a.smi, ["-i", str(g), "-rgc"])
            self.audit({"clock_write": ["-i", str(g), "-rgc"], "wh
↪ y": who, "rc": rc, "stderr": err})
            if rc != 0:
                raise WriteFailed(f"nvidia-smi -rgc returned {rc}:

↪  {err}")
            self.clock_set = False
        elif ceiling < self.top:
            self.write_clock(ceiling, who)
        self.written_ceiling = ceiling
    cur = r["limit"] if r is not None else self.expect
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if abs(lid - cur) >= a.min_change_w or (lid == self.start and
    ↪ cur != lid):
            self.write_limit(lid, who)

    def restore(self):
        failed = []

        if self.a.mode == "cap" and not self.foreign:
            rc, err = smi_run(self.a.smi, ["-i", str(self.g), "-rgc"])
            self.audit({"clock_write": ["-i", str(self.g), "-rgc"], "w
    ↪ hy": "restore", "rc": rc, "stderr": err})
            if rc != 0:
                failed.append(f"-rgc returned {rc}: {err}")

            s = query(self.a.smi, [self.g], self.enforced_ok)
            if s is None or abs(s[self.g]["limit"] - self.start) >= 1.
    ↪ 0:
                try:
                    self.write_limit(self.start, "reset: the limit rea
    ↪ d at start")

                except WriteFailed as e:
                    failed.append(str(e))
        s = query(self.a.smi, [self.g], self.enforced_ok) or {}
        back = s[self.g]["limit"] if self.g in s else None
        ok = self.foreign or (not failed and back is not None and abs(
    ↪ back - self.start) < 1.0)

        self.audit({"restored": {str(self.g): back}, "start": {str(sel
    ↪ f.g): self.start}, "ok": ok,
                    "writes": self.writes, "foreign_writer": self.fore
    ↪ ign, "restore_write_failed": failed})
        return ok

def parser():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--mode", choices=["watch", "cap"], default="watch
    ↪ ")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ap.add_argument("--dial", choices=["max", "save"], default=os.envi
    ↪ ron.get("OMNI_DIAL", "max"),
                    help="max (default): the clock ceiling never stepp
    ↪ ed below the top while the card serves; save: the "
                         "tune-down (the verdict's steps below the top
    ↪  where the card's time on a request allows)")

    ap.add_argument("--gpus", default="0")
    ap.add_argument("--smi", default=os.environ.get("NVIDIA_SMI", "nvi
    ↪ dia-smi"))
    ap.add_argument("--interval", type=float, default=2.0)
    ap.add_argument("--duration", type=float, default=0.0)
    ap.add_argument("--audit", default="gpu_compass_audit.jsonl")

    ap.add_argument("--kill-file", default="/tmp/omni-gpu-kill")
    ap.add_argument("--latency-file", default="")
    ap.add_argument("--slo-ms", type=float, default=0.0)
    ap.add_argument("--latency-window-s", type=float, default=5.0, hel
    ↪ p="seconds of response times the position reads")
    ap.add_argument("--race-util", type=float, default=0.95, help="uti

    ↪ lization at which work is waiting: race, never pace")
    ap.add_argument("--learn-samples", type=int, default=15, help="bus
    ↪ y readings of the card on its own before the compass may lower a
    ↪ nything")
    ap.add_argument("--floor-w", type=float, default=0.0, help="the de
    ↪ clared envelope's lowest watts")

    ap.add_argument("--clock-min-share", type=float, default=0.0,
                    help="the clock ceiling's cover, as a share of the
    ↪  top; 0 (the default): the card's own lowest supported clock")
    ap.add_argument("--allow", type=float, default=0.02, help="the mos
    ↪ t a ceiling step may add to the card's own time on a request")
    ap.add_argument("--verdict-samples", type=int, default=30, help="r

    ↪ equests measured at native and at the trial step")
    ap.add_argument("--verdict-every", type=int, default=8, help="deci
    ↪ sions between trials")
    ap.add_argument("--verdict-recheck", type=int, default=900, help="
    ↪ decisions before a refused step is tried again")
    ap.add_argument("--up-gain", type=float, default=0.10, help="share
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  of the top clock moved per unit of force, up")
    ap.add_argument("--down-gain", type=float, default=None, help="sha
    ↪ re of the top clock moved per unit of force, down (default: the
    ↪ law's)")
    ap.add_argument("--lid-headroom", type=float, default=0.10, help="
    ↪ the lid above the draw the ceiling takes")

    ap.add_argument("--min-change-mhz", type=float, default=15.0)
    ap.add_argument("--min-change-w", type=float, default=3.0)
    return ap


def main(argv=None):

    a = parser().parse_args(argv)
    master.refuse_if_off("GPU governor (two wires)")
    gov = GpuCompass(a)
    # what puts the card back if this process dies without doing it it
    ↪ self (the watchdog runs it)
    back = ([[a.smi, "-i", str(gov.g), "-rgc"], [a.smi, "-i", str(gov.

    ↪ g), "-pl", str(int(round(gov.start)))]]
            if a.mode == "cap" else [])
    master.register("GPU governor (two wires)", restore=back, stale_s=
    ↪ max(60.0, 5 * a.interval))
    stop = {"now": False}
    signal.signal(signal.SIGTERM, lambda *_: stop.__setitem__("now", T

    ↪ rue))
    t0 = time.time()
    failed = False
    try:
        while not stop["now"] and not os.path.exists(a.kill_file) and
    ↪ not master.is_off() and (a.duration <= 0 or time.time() - t0 < a

    ↪ .duration):
            try:
                master.heartbeat(); gov.step()
            except WriteFailed as e:
                gov.audit({"fatal": f"write failed: {e}"}); failed = T
    ↪ rue
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                break
            except Exception as e:  # noqa: BLE001  a failed decision
    ↪ is logged; the kill path still runs
                gov.audit({"error": f"{type(e).__name__}: {e}"})
            end = time.time() + a.interval
            while time.time() < end and not stop["now"] and not os.pat
    ↪ h.exists(a.kill_file) and not master.is_off():
                time.sleep(0.2)
    finally:
        ok = gov.restore()
    return 5 if gov.foreign else 3 if not ok else 4 if failed else 0

if __name__ == "__main__":
    sys.exit(main())
```

H.5 omni_controller/gpu_governor.py

529 lines.

```
"""Omni-Compass on one GPU box, no Kubernetes: the engine reads each G
    ↪ PU's own meter and may lower its power limit.

Every decision (--interval seconds), for each GPU in --gpus:
  sense     nvidia-smi: power.draw, temperature.gpu, utilization.gpu,
    ↪ power.limit (the device's own readings)
            the workload's response-time file (--latency-file, elapsed
    ↪ _seconds,latency_ms,ok) if given
  engine    the Omni-Compass governor, throughput law (omnicompass/ada
    ↪ pter.py): load = utilization, power stress =
            draw / the limit found at start, heat = temperature / --te
    ↪ mp-limit, queue = response-time pressure.
            Its directive power_cap is the share of the start limit th
    ↪ e GPU may draw.
  shield    limit = max(cap x start limit, draw x (1 + --headroom), --
    ↪ min-share x start limit, the device's minimum
            limit), never above the start limit; whole watts; a change
    ↪  under --min-change-w is not written.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
          --min-share bounds the slowdown: a card held at that share
  ↪  of its limit runs a burst only a little slower.
busy gate utilization (smoothed over decisions) at or above --util-g
  ↪ ate: the start limit. A busy card is the
          bottleneck; slowing it grows the queue faster than it save
  ↪ s energy. The cap returns once utilization is

          back under the gate less --util-band.
reflexes  a blind sense (nvidia-smi unreadable, or the response-time
  ↪  file stale or empty): the start limit, at once
          a response-time breach (p95 > --slo-ms), and for --slo-cle
  ↪ ar decisions after it: the start limit, at once
read-back no new write until the last one is read back from the devi

  ↪ ce (power.limit within 1 W of what I wrote)
enforced  the card obeys enforced.power.limit, which another authori
  ↪ ty (a lower board or system limit) can hold under
          the limit I request. The engine senses the enforced limit;
  ↪  each write is read back at once (requested,
          return code, power.limit, enforced limit, delay), and enfo

  ↪ rced under requested is recorded as an override.
          A driver without enforced.power.limit: power.limit is used
  ↪  and the snapshot says so.
speed lock (--baseline-file, from tools/gpu_baseline.py on a run wit
  ↪ hout Omni): the limit is set by response time
          alone. Over the last --lock-window-s of the response-time

  ↪ file, the mean, 95th and 99th percentile are each
          divided by the baseline's at the same arrival rate; the wo
  ↪ rst of the three is the speed ratio. The line is
          1 - --speed-gain (0.99: at least 1% faster than without Om
  ↪ ni). Above the line: the start limit, at once.
          Under the line less --lock-margin: lower by --lock-step x

  ↪ start, times the slack in margins (up to
          --lock-boost), at most once per --lock-hold-s (the window
  ↪ must see the step before the next). Between: held. Never under -
  ↪ -lock-floor x start.
          Speed won elsewhere (CPU conveyed to the serving pods, sho
  ↪ rter queues) shows as a ratio under the line,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            and the lock spends it on watts; with nothing won elsewher
    ↪ e it holds the start limit.
  one writer the limit must be what I last wrote (or the start limit).
    ↪  Any other value means another writer: I stop
            writing for the rest of the run (observe only), record "fo
    ↪ reign_writer", leave its limit alone on kill

            (restoring would fight it), and exit 5 so the bench marks
    ↪ the run invalid. Two controllers on one limit is
            how production is lost.
  heat      the device reporting a thermal or hardware slowdown (clock
    ↪ -limit reasons 0x8, 0x20, 0x40, 0x80): never
            tighten; the limit may only stay or rise (fail up).

  envelope  every decision records every rule that held the engine's r
    ↪ equest back ("blocked_by") and the one that
            decided ("decided_by": engine, a floor, the busy gate, a r
    ↪ eflex, heat, or the speed lock), so a limit that
            did not move names its cause, and a result is credited to
    ↪ the rule that produced it.

  envelope  --floor-w, the buyer's declared lowest watts (the confirma
    ↪ tion run requires it, scripts/gpu_paired.sh
  floor     ENVELOPE): the limit is never set under it. The highest wa
    ↪ tts are the limit read at start.
  outer     when the card's own controller (board, BMC or system polic
    ↪ y) already holds enforced.power.limit under the

  holds     limit I set, a lower write that stays above that enforced
    ↪ value would change nothing the card does: it is
            not written ("outer_controller_holds"). Only a write that
    ↪ would actually bind, or a return upward, goes out.
Modes
  watch     everything above is computed and audited; nothing is writt

    ↪ en (the control arm)
  cap       the limit is written with nvidia-smi -i <gpu> -pl <W>. Ref
    ↪ used at start unless every GPU reports
            power.management Enabled. A write the device refuses (nonz
    ↪ ero return) ends the run: recorded, the start
            limit restored, exit 4, and the bench marks the arm invali
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ d. No clock locks are ever written.
Kill switch
  the kill file (or SIGTERM) restores every GPU to the limit read at s
    ↪ tart, reads it back, and exits.
  The start limits are recorded first in the audit ("snapshot": power.
    ↪ limit, enforced.power.limit, default, min,

  max, persistence mode, power management, per GPU).
"""
from __future__ import annotations

import argparse, copy, json, math, os, shlex, signal, subprocess, sys,
    ↪  time

from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from omnicompass import master
from omnicompass.adapter import Governor, mode_law, AUTOPILOT, OBSERVE

    ↪ , observe_vector, assimilate, ASSIMILATION
from omnicompass.core import U_AUTHORITY, State
from omni_controller.muscles import latency_sense

FIELDS = "index,power.draw,temperature.gpu,utilization.gpu,power.limit
    ↪ ,power.min_limit,clocks.sm"

ENFORCED = "enforced.power.limit"
SNAPSHOT = ("power.limit", "enforced.power.limit", "power.default_limi
    ↪ t", "power.min_limit", "power.max_limit",
            "persistence_mode", "power.management")
REASONS = ("clocks_event_reasons.active", "clocks_throttle_reasons.act
    ↪ ive")   # newer, older drivers

HEAT_BITS = 0x8 | 0x20 | 0x40 | 0x80   # HW slowdown, SW thermal slowd
    ↪ own, HW thermal slowdown, HW power brake



def slowed(reason):
    """True when the device reports a thermal or hardware slowdown in
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ its clock-limit reasons (hex bit mask)."""
    try:
        return bool(int(str(reason), 16) & HEAT_BITS)
    except (TypeError, ValueError):
        return False

def blocked_by(requested, draw, start, min_w, headroom, min_share, slo
    ↪ _clean, gated, heat, lock, floor_w=0.0):
    """Every rule whose floor or override sits above the engine's own
    ↪ request (whole watts), in a fixed order."""
    engine_w = math.floor(requested * start)

    b = []
    if floor_w and floor_w > engine_w:
        b.append("envelope_floor")
    if not slo_clean:
        b.append("slo_or_blind_reflex")
    if gated:

        b.append("busy_gate")
    if math.ceil(draw * (1.0 + headroom)) > engine_w:
        b.append("draw_headroom_floor")
    if min_share * start > engine_w:
        b.append("share_floor")
    if min_w > engine_w:

        b.append("device_minimum")
    if heat:
        b.append("thermal_hold")
    if lock:
        b.append("speed_lock")
    return b

STATE = ("E", "U", "I_U", "S", "B", "B_dot")
MEASURED = ("E", "U", "I_U", "S", "B")   # the memoryless part of the
    ↪ observation map (B_dot exists only through history)


def measured_state(o, p):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    """h without memory: the state the telemetry alone points to (the
    ↪ targets assimilate() blends toward). Computed by
    assimilate() itself from a zero prior, so it is the engine's own m
    ↪ ap, not a second one."""
    z = assimilate(State(0.0, 0.0, 0.0, 0.0, 0.0, 0.0), o, copy.deepco
    ↪ py(p))

    return {k: getattr(z, k) / ASSIMILATION for k in MEASURED}


def window_stats(path, window_s, now=None):
    """Mean, 95th and 99th percentile (ms) and arrival rate (served pe
    ↪ r second) of the successful requests in the last

    window_s of the response-time file; None when blind (stale beyond
    ↪ two windows, or under 20 requests)."""
    try:
        age = (now if now is not None else time.time()) - os.path.getm
    ↪ time(path)
        rows = [l.split(",") for l in Path(path).read_text().splitline

    ↪ s()[1:] if l.strip()]
    except OSError:
        return None
    if not rows or age > 2.0 * window_s:
        return None
    t_end = float(rows[-1][0])

    ms = sorted(float(r[1]) for r in rows if float(r[0]) >= t_end - wi
    ↪ ndow_s and r[2].strip() == "1")
    if len(ms) < 20:
        return None
    span = min(window_s, max(1.0, t_end - float(rows[0][0])))
    tot = 0.0

    for v in ms:              # plain left-to-right addition, as the C
    ↪ ++ twin does (Python 3.12's sum() compensates)
        tot += v
    return {"mean": tot / len(ms), "p95": ms[min(len(ms) - 1, int(0.95
    ↪  * len(ms)))],
            "p99": ms[min(len(ms) - 1, int(0.99 * len(ms)))], "rate":
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ len(ms) / span, "n": len(ms)}


def baseline_at(base, rate):
    """The baseline's mean, p95 and p99 at an arrival rate, linear bet
    ↪ ween its rate bins, flat beyond the ends."""

    bins = base["bins"]
    if rate <= bins[0]["rate"]:
        return bins[0]
    if rate >= bins[-1]["rate"]:
        return bins[-1]
    for lo, hi in zip(bins, bins[1:]):

        if lo["rate"] <= rate <= hi["rate"]:
            f = (rate - lo["rate"]) / (hi["rate"] - lo["rate"])
            return {k: lo[k] + f * (hi[k] - lo[k]) for k in ("mean", "
    ↪ p95", "p99")}

def shield_limit(cap, draw, start, min_w, headroom, min_share, slo_cle
    ↪ an):
    """The shield: limit = max(cap x start, draw x (1 + headroom), min
    ↪ _share x start, device minimum), never above start,
    whole watts. Returns (want_w, the bound that decided it). Twinned
    ↪ in C++ (cpp/src/gpu_rules.cpp)."""

    share_floor = min_share * start
    floor = max(draw * (1.0 + headroom), min_w, share_floor)
    engine_w = math.floor(cap * start)
    want = int(min(start, max(engine_w, math.ceil(floor))))
    bound = ("slo_or_blind_reflex" if not slo_clean else "start_ceilin
    ↪ g" if want >= start and engine_w >= start

             else "share_floor" if math.ceil(floor) > engine_w and flo
    ↪ or == share_floor
             else "draw_headroom_floor" if math.ceil(floor) > engine_w
    ↪  and floor > min_w
             else "device_minimum" if math.ceil(floor) > engine_w else
    ↪  "engine")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return want, bound


def busy_gate(u_prev, util, gated_prev, gate, band):
    """Utilization smoothed over decisions (u_prev None: the first rea
    ↪ ding) and the gate with its band. Returns

    (u, gated). Twinned in C++."""
    u = 0.5 * (util if u_prev is None else u_prev) + 0.5 * util
    return u, (u >= gate or (gated_prev and u >= gate - band))


def lock_decide(ratio, cur, start, min_w, since_last, speed_gain, lock

    ↪ _margin, lock_step, lock_boost, lock_floor, lock_hold_s):
    """The speed lock's step from the worst response-time ratio agains
    ↪ t the baseline. Returns (want_w, bound, line,
    aim). Twinned in C++."""
    line = 1.0 - speed_gain
    aim = line * (1.0 - lock_margin)

    floor = max(min_w, lock_floor * start)
    if ratio > line:
        want, bound = start, "speed_lock_release"
    elif ratio < aim and since_last >= lock_hold_s:
        mult = min(lock_boost, max(1.0, (aim - ratio) / max(1e-6, line
    ↪  * lock_margin)))

        want, bound = max(floor, cur - mult * lock_step * start), "spe
    ↪ ed_lock_spend"
    else:
        want, bound = cur, "speed_lock_hold"
    return int(min(start, max(floor, want))), bound, line, aim

def throttle(smi, gpus):
    """The device's own record of why its clock is held down (power ca
    ↪ p, thermal, ...), as a bit mask; None if the
    driver does not report it."""
    for f in REASONS:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        try:
            out = subprocess.run(shlex.split(smi) + [f"--query-gpu=ind
    ↪ ex,{f}", "--format=csv,noheader,nounits"],
                                 capture_output=True, text=True, timeo
    ↪ ut=10, check=True).stdout
            return {int(x.split(",")[0]): x.split(",")[1].strip() for

    ↪ x in out.strip().splitlines()}
        except (subprocess.SubprocessError, OSError, ValueError, Index
    ↪ Error):
            continue
    return None

class WriteFailed(RuntimeError):
    """The device refused a power-limit write: the arm ends (the kill
    ↪ path still restores)."""

def snapshot(smi, gpus):
    """{field: {gpu: value}} for every SNAPSHOT field, each queried al
    ↪ one; "unsupported" where the driver refuses it."""
    out = {}
    for f in SNAPSHOT:
        try:

            r = subprocess.run(shlex.split(smi) + [f"--query-gpu=index
    ↪ ,{f}", "--format=csv,noheader,nounits"],
                               capture_output=True, text=True, timeout
    ↪ =10, check=True).stdout
            vals = {int(x.split(",")[0]): x.split(",", 1)[1].strip() f
    ↪ or x in r.strip().splitlines()}

        except (subprocess.SubprocessError, OSError, ValueError, Index
    ↪ Error):
            vals = {}
        out[f] = {str(g): vals.get(g, "unsupported") for g in gpus}
    return out

def query(smi, gpus, enforced=False):
    """{gpu: {"draw", "temp", "util", "limit", "min", "clock_mhz", "en
    ↪ forced"}} from the device, or None if nvidia-smi
    cannot be read. enforced: also read enforced.power.limit (else "en
    ↪ forced" is power.limit)."""
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    try:
        fields = FIELDS + ("," + ENFORCED if enforced else "")
        out = subprocess.run(shlex.split(smi) + [f"--query-gpu={fields
    ↪ }", "--format=csv,noheader,nounits"],
                             capture_output=True, text=True, timeout=1
    ↪ 0, check=True).stdout

        rows = {}
        for line in out.strip().splitlines():
            v = [x.strip() for x in line.split(",")]
            rows[int(v[0])] = {"draw": float(v[1]), "temp": float(v[2]
    ↪ ), "util": float(v[3]) / 100.0,
                               "limit": float(v[4]), "min": float(v[5]

    ↪ ), "clock_mhz": float(v[6]),
                               "enforced": float(v[7]) if enforced els
    ↪ e float(v[4])}
        return {g: rows[g] for g in gpus} if all(g in rows for g in gp
    ↪ us) else None
    except (subprocess.SubprocessError, OSError, ValueError, IndexErro

    ↪ r):
        return None


class GpuGovernor:
    def __init__(self, a):

        self.a = a
        self.gpus = [int(g) for g in str(a.gpus).split(",") if g.strip
    ↪ () != ""]
        self.log = open(a.audit, "a")
        self.eng = {g: Governor(law=mode_law("throughput")) for g in s
    ↪ elf.gpus}

        for e in self.eng.values():
            e.set_mode(AUTOPILOT if a.mode == "cap" else OBSERVE)
            e.nodes = 1
        self.cap = {g: 1.0 for g in self.gpus}
        self.written = {}          # gpu -> last limit written (W), un
    ↪ til read back
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.lp_hist = []
        self.writes = 0
        self.projected = {}
        self.reasons_ok = True     # False once the driver shows it do
    ↪ es not report clock-limit reasons
        self.util_avg = {}         # gpu -> utilization smoothed over

    ↪ decisions (busy gate)
        self.gated = {}            # gpu -> True while the busy gate h
    ↪ olds the start limit
        bf = getattr(a, "baseline_file", "")
        self.baseline = json.loads(Path(bf).read_text()) if bf else No
    ↪ ne   # speed lock: the run without Omni

        self.lock_at = {}          # gpu -> time of the lock's last ch
    ↪ ange
        self.clock = time.time     # replaceable, so a simulation in v
    ↪ irtual time can drive the lock's hold
        self.req_at = {}           # gpu -> (limit requested, time req
    ↪ uested) until it reads back

        self.foreign = False       # True once another writer changed
    ↪ a limit: observe only from then on
        self.enforced_ok = query(a.smi, self.gpus, enforced=True) is n
    ↪ ot None
        s = query(a.smi, self.gpus, self.enforced_ok)
        if s is None:

            raise SystemExit("nvidia-smi unreadable at start: no snaps
    ↪ hot, so I take no authority")
        snap = snapshot(a.smi, self.gpus)
        self.start = {g: s[g]["limit"] for g in self.gpus}
        self.expect = dict(self.start)   # gpu -> the limit that shoul
    ↪ d be there: the start, or my last write that landed

        self.min = {g: s[g]["min"] for g in self.gpus}
        self.audit({"snapshot": {str(g): {"limit_w": self.start[g], "m
    ↪ in_limit_w": self.min[g],
                                          "enforced_w": s[g]["enforced
    ↪ "] if self.enforced_ok else None,
                                          **{f: snap[f][str(g)] for f
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ in SNAPSHOT}} for g in self.gpus},
                   "senses": ENFORCED if self.enforced_ok else "power
    ↪ .limit (the driver does not report enforced.power.limit)",
                   "mode": a.mode})
        off = [g for g in self.gpus if snap["power.management"][str(g)
    ↪ ].lower() != "enabled"]

        if a.mode == "cap" and off:
            self.audit({"refused": f"power management not Enabled on G
    ↪ PU {off}: a written limit would not bind"})
            raise SystemExit(f"power management not Enabled on GPU {of
    ↪ f}: cap mode refused, no authority taken")

    def audit(self, rec):
        self.log.write(json.dumps({"time": time.time(), **rec}) + "\n"
    ↪ ); self.log.flush()


    def set_limit(self, g, w, why):
        cmd = ["nvidia-smi", "-i", str(g), "-pl", str(int(w))]

        if self.a.mode != "cap" or self.foreign:
            # watch: computed and recorded, never executed; after anot
    ↪ her writer appeared, likewise
            self.audit({"would_write": cmd, "why": why, **({"withheld"
    ↪ : "another writer owns the limit"} if self.foreign else {})})
            return

        self.audit({"write": cmd, "why": why})
        t0 = time.time()
        try:
            p = subprocess.run(shlex.split(self.a.smi) + ["-i", str(g)
    ↪ , "-pl", str(int(w))], capture_output=True, text=True, timeout=2
    ↪ 0)

            rc, err = p.returncode, p.stderr.strip()[:300]
        except (subprocess.SubprocessError, OSError) as e:
            rc, err = -1, f"{type(e).__name__}: {e}"[:300]
        act = {"gpu": g, "requested_w": int(w), "t_requested": t0, "rc
    ↪ ": rc, "stderr": err}
        if rc != 0:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            self.audit({"actuator": act, "write_failed": cmd})
            raise WriteFailed(f"GPU {g}: nvidia-smi -pl {int(w)} retur
    ↪ ned {rc}: {err}")
        self.writes += 1
        self.written[g] = int(w)
        s = query(self.a.smi, [g], self.enforced_ok)

        if s is not None:
            back = s[g]
            act.update({"readback_w": back["limit"], "enforced_w": bac
    ↪ k["enforced"], "t_readback": time.time(),
                        "realized": abs(back["limit"] - w) < 1.0, "ove
    ↪ rride": back["enforced"] < back["limit"] - 1.0})

            if act["realized"]:
                act["delay_s"] = round(act["t_readback"] - t0, 3); sel
    ↪ f.expect[g] = int(w)
        if not act.get("realized"):
            self.req_at[g] = (int(w), t0)
        self.audit({"actuator": act})

    def killed(self):
        return os.path.exists(self.a.kill_file)

    def restore(self):
        """Kill switch: every GPU back to the limit read at start, rea

    ↪ d back from the device."""
        s = query(self.a.smi, self.gpus, self.enforced_ok) or {}
        failed = []
        for g in self.gpus:
            if self.foreign:
                break                  # another writer owns the limit

    ↪ : restoring would fight it
            if self.a.mode == "cap" and (g not in s or abs(s[g]["limit
    ↪ "] - self.start[g]) >= 1.0):
                try:
                    self.set_limit(g, self.start[g], "kill switch: the
    ↪  limit read at start")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                except WriteFailed as e:
                    failed.append(str(e))
        s = query(self.a.smi, self.gpus, self.enforced_ok) or {}
        back = {str(g): (s[g]["limit"] if g in s else None) for g in s
    ↪ elf.gpus}
        ok = not failed and all(v is not None and abs(v - self.start[i

    ↪ nt(g)]) < 1.0 for g, v in back.items())
        self.audit({"restored": back, "enforced": {str(g): (s[g]["enfo
    ↪ rced"] if g in s else None) for g in self.gpus},
                    "start": {str(g): self.start[g] for g in self.gpus
    ↪ }, "ok": ok, "writes": self.writes,
                    "foreign_writer": self.foreign,

                    "restore_write_failed": failed})
        return ok


    def speed_lock(self, g, cur, slo_clean):
        """The limit from response time against the run without Omni (
    ↪ see the module docstring). Returns

        (want_w, bound, why, record)."""
        a, start = self.a, self.start[g]
        st = window_stats(a.latency_file, a.lock_window_s) if a.latenc
    ↪ y_file else None
        if st is None or not slo_clean:
            return int(start), "speed_lock_blind", "speed lock: no res

    ↪ ponse-time window (or SLO reflex), the start limit", None
        b = baseline_at(self.baseline, st["rate"])
        ratios = {k: st[k] / b[k] for k in ("mean", "p95", "p99")}
        ratio = max(ratios.values())
        want, bound, line, aim = lock_decide(ratio, cur, start, self.m
    ↪ in[g], self.clock() - self.lock_at.get(g, -1e18),

                                             a.speed_gain, a.lock_marg
    ↪ in, a.lock_step, a.lock_boost, a.lock_floor, a.lock_hold_s)
        if want != int(cur):
            self.lock_at[g] = self.clock()
        rec = {"rate": round(st["rate"], 2), "ratios": {k: round(v, 4)
    ↪  for k, v in ratios.items()},
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            "line": line, "aim": round(aim, 4), "n": st["n"]}
        why = f"speed lock: worst ratio {ratio:.3f} vs line {line:.3f}
↪  ({bound.split('_')[-1]}), limit {want} W"
        return want, bound, why, rec

    def step(self):

        a = self.a
        s = query(a.smi, self.gpus, self.enforced_ok)
        blind = s is None
        lp, p95, served = 0.0, None, None
        if a.latency_file and a.slo_ms:
            ls = latency_sense(a.latency_file, a.latency_window_s)

            served = ls["ok"]
            blind = blind or ls["blind"]
            if not ls["blind"]:
                p95 = ls["p95"]
                lp = max(0.0, ls["p95"] / a.slo_ms - 1.0)
                if ls["fail"]:

                    lp = max(lp, ls["fail"] / (ls["ok"] + ls["fail"]))
        self.lp_hist.append(lp)
        slo_clean = not blind and len(self.lp_hist) >= a.slo_clear and
↪  all(x == 0.0 for x in self.lp_hist[-a.slo_clear:])
        thr = throttle(a.smi, self.gpus) if s is not None and self.rea
↪ sons_ok else None

        self.reasons_ok = self.reasons_ok and (s is None or thr is not
↪  None)
        rec = {"decision": {}, "blind": blind, "served_in_window": ser
↪ ved, "latency_p95_ms": p95, "latency_pressure": round(lp, 3), "s
↪ lo_clean": slo_clean}
        for g in self.gpus:

            if s is None:
                # blind: no give-back of power I cannot see; the start
↪  limit, at once
                if self.cap[g] == 1.0 or self.foreign:
                    continue
                want, why, cur = self.start[g], "blind sense: the limi
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ t read at start", None
                self.cap[g] = 1.0
            else:
                r = s[g]; cur = r["limit"]
                if abs(cur - self.expect[g]) >= 1.0 and not (g in self
    ↪ .written and abs(cur - self.written[g]) < 1.0):

                    if g not in self.written or not self.foreign:
                        # neither what I last wrote nor what was there
    ↪  before: another writer
                        if not self.foreign:
                            self.audit({"foreign_writer": {"gpu": g, "
    ↪ reads_w": cur, "expected_w": self.expect[g],

                                                                "pending_w"
    ↪ : self.written.get(g)},
                                                "action": "observe only from n
    ↪ ow on; the limit left to the other writer"})
                        self.foreign = True; self.expect[g] = cur; sel
    ↪ f.written.pop(g, None)

                if g in self.written and abs(cur - self.written[g]) >=
    ↪  1.0:
                    # read-back: my last write has not landed; no new
    ↪ order on top of it
                    rec["decision"][str(g)] = {"hold": "last write not
    ↪  read back", "wrote_w": self.written[g], "reads_w": cur}

                    continue
                if g in self.written:
                    self.expect[g] = int(self.written.pop(g))
                if g in self.req_at:       # a write that had not read
    ↪  back at once has now landed
                    w0, t0 = self.req_at.pop(g)

                    self.audit({"actuator_realized": {"gpu": g, "reque
    ↪ sted_w": w0, "readback_w": cur,
                                                        "enforced_w": r[
    ↪ "enforced"], "delay_s": round(time.time() - t0, 3)}})
                # the limit the card obeys (enforced), not the one I m
    ↪ eant
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            e = self.eng[g]; e.current_cap = min(1.0, r["enforced"
↪ ] / self.start[g])
            obs = {"queue_ratio": min(2.0, lp), "load_ratio": r["u
↪ til"], "power_stress": r["draw"] / self.start[g],
                   "thermal": r["temp"] / a.temp_limit, "network_s
↪ tress": 0.0, "drift_ratio": 0.0,
                   "stale": 0.0, "security_block": 0.0, "slo_clean
↪ ": slo_clean}   # the brain sees the line's clear time too (omni
↪ -v15)
            # the chain, recorded whole: what the device said, the
↪  state it puts the engine in, what I had projected
            # for this moment, the projection for the next, what t
↪ he engine asked for, what the shield allowed
            seen = assimilate(e.x, observe_vector(obs, 0), copy.de
↪ epcopy(e.p))
            meas = measured_state(observe_vector(obs, 0), e.p)
            prev = self.projected.get(g)
            d = e.step(obs, 0)
            self.projected[g] = {k: getattr(e.x, k) for k in STATE
↪ }
            requested = float(d["power_cap"])
            cap = requested if slo_clean else 1.0
            want, bound = shield_limit(cap, r["draw"], self.start[
↪ g], self.min[g], a.headroom, getattr(a, "min_share", 0.0), slo_c
↪ lean)
            gate = getattr(a, "util_gate", 0.0)
            if gate:
                self.util_avg[g], self.gated[g] = busy_gate(self.u
↪ til_avg.get(g), r["util"], self.gated.get(g, False),
                                                            gate,
↪ getattr(a, "util_band", 0.1))
                if self.gated[g]:
                    want, bound = int(self.start[g]), "busy_gate"
                    cap = 1.0
            why = ("response-time reflex: the limit read at start"
↪  if not slo_clean
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                    else f"busy gate: utilization {self.util_avg[g]
↪ :.2f}, the limit read at start" if bound == "busy_gate"
                    else f"engine cap {cap:.3f} of {self.start[g]:.
↪ 0f} W; floor draw {r['draw']:.0f} W x {1 + a.headroom:.2f}, "
                         f"share floor {getattr(a, 'min_share', 0.0
↪ ):.2f}")

            lock = None
            if self.baseline is not None:
                want, bound, why, lock = self.speed_lock(g, cur, s
↪ lo_clean)
            heat = slowed((thr or {}).get(g))
            if heat and want < cur:

                # the device is slowing itself for heat or a hardw
↪ are limit: never tighten on top of it
                want, bound, why = int(cur), "thermal_hold", "ther
↪ mal or hardware slowdown reported: no tightening"
            floor_w = float(getattr(a, "floor_w", 0.0) or 0.0)
            if floor_w and want < floor_w:

                # the buyer's declared lowest watts: never under i
↪ t
                want, bound, why = int(math.ceil(min(floor_w, self
↪ .start[g]))), "envelope_floor", f"declared envelope floor {floor
↪ _w:.0f} W"
            blocks = blocked_by(requested, r["draw"], self.start[g

↪ ], self.min[g], a.headroom, getattr(a, "min_share", 0.0),
                                slo_clean, self.gated.get(g, False
↪ ) if gate else False, heat, self.baseline is not None, floor_w)
            # the card's own controller already holds it lower: a
↪ lower write above the enforced value changes nothing
            outer = r["enforced"] < cur - 1.0 and r["enforced"] -

↪ 1.0 <= want < cur
            self.cap[g] = want / self.start[g]
            rec["decision"][str(g)] = {
                "telemetry": {"util": r["util"], "draw_w": r["draw
↪ "], "temp_c": r["temp"], "limit_w": cur,
                              "enforced_w": r["enforced"],
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                                "clock_mhz": r["clock_mhz"], "throtte
    ↪ le": (thr or {}).get(g)},
                    "state_observed": {k: round(getattr(seen, k), 4) fo
    ↪ r k in STATE},
                    "state_measured": {k: round(v, 4) for k, v in measu
    ↪ .items()},

                    "state_projected_before": None if prev is None else
    ↪ e {k: round(v, 4) for k, v in prev.items()},
                    "prediction_error": None if prev is None else {k:
    ↪ round(getattr(seen, k) - prev[k], 4) for k in STATE},
                    "state_projected_next": {k: round(v, 4) for k, v in
    ↪ n self.projected[g].items()},

                    "u_push": round(e.last_push * U_AUTHORITY, 4),   #
    ↪  the U-channel command on the evolved state
                    "admissible": bool(d["change_permitted"]), "request
    ↪ ted_cap": round(requested, 4),
                    "granted_cap": round(cap, 4), "shield_bound": bound
    ↪ d, "want_w": want,

                    "engine_w": math.floor(requested * self.start[g]),
    ↪  "blocked_by": blocks,
                    "decided_by": "speed_lock" if bound.startswith("spe
    ↪ ed_lock") else bound,
                    "outer_controller_holds": outer,
                    "engine_cap": round(requested, 3), "E": round(d["st

    ↪ tate"]["E"], 3), "U": round(d["state"]["U"], 3),
                    "speed_lock": lock}
            if cur is not None and abs(want - cur) < a.min_change_w and
    ↪ not (want == self.start[g] and cur != want):
                continue
            if s is not None and rec["decision"].get(str(g), {}).get("

    ↪ outer_controller_holds"):
                continue
            if cur is not None and want == int(cur):
                continue
            self.set_limit(g, want, why)
        self.audit(rec)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def parser():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--mode", choices=["watch", "cap"], default="watch
    ↪ ")
    ap.add_argument("--gpus", default="0", help="GPU indexes, comma se
    ↪ parated")
    ap.add_argument("--smi", default=os.environ.get("NVIDIA_SMI", "nvi
    ↪ dia-smi"))
    ap.add_argument("--interval", type=float, default=2.0)
    ap.add_argument("--duration", type=float, default=0.0, help="secon
    ↪ ds to run (0: until killed)")
    ap.add_argument("--audit", default="gpu_audit.jsonl")
    ap.add_argument("--kill-file", default="/tmp/omni-gpu-kill")
    ap.add_argument("--headroom", type=float, default=0.3, help="limit
    ↪  never below draw x (1 + headroom)")
    ap.add_argument("--min-share", type=float, default=0.70, help="lim
    ↪ it never below this share of the start limit (0: off)")
    ap.add_argument("--floor-w", type=float, default=0.0, help="the de
    ↪ clared envelope's lowest watts: never under it (0: off)")
    ap.add_argument("--util-gate", type=float, default=0.5, help="smoo
    ↪ thed utilization at which the start limit returns (0: off)")
    ap.add_argument("--util-band", type=float, default=0.1, help="the
    ↪ cap resumes below --util-gate less this band")
    ap.add_argument("--baseline-file", default="", help="speed lock: b
    ↪ aseline from tools/gpu_baseline.py (off when empty)")
    ap.add_argument("--speed-gain", type=float, default=0.01, help="sp
    ↪ eed lock line: at least this much faster than baseline")
    ap.add_argument("--lock-boost", type=float, default=5.0, help="lar
    ↪ gest multiple of --lock-step in one step down")
    ap.add_argument("--lock-margin", type=float, default=0.08, help="s
    ↪ pend watts only while this far under the line")
    ap.add_argument("--lock-step", type=float, default=0.02, help="sha
    ↪ re of the start limit given up per decision")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ap.add_argument("--lock-hold-s", type=float, default=10.0, help="s
    ↪ econds between two steps down")
    ap.add_argument("--lock-window-s", type=float, default=60.0, help=
    ↪ "response-time window the lock reads")
    ap.add_argument("--lock-floor", type=float, default=0.5, help="the
    ↪  lock never holds the limit under this share of start")

    ap.add_argument("--min-change-w", type=float, default=5.0)
    ap.add_argument("--temp-limit", type=float, default=83.0, help="GP
    ↪ U temperature read as heat 1.0")
    ap.add_argument("--latency-file", default="")
    ap.add_argument("--slo-ms", type=float, default=0.0)
    ap.add_argument("--latency-window-s", type=float, default=30.0)

    ap.add_argument("--slo-clear", type=int, default=3)
    return ap


def main(argv=None):
    a = parser().parse_args(argv)

    master.refuse_if_off("GPU governor (one wire)")
    gov = GpuGovernor(a)
    back = [[a.smi, "-i", str(g), "-pl", str(int(round(gov.start[g])))
    ↪ ] for g in gov.gpus] if a.mode == "cap" else []
    master.register("GPU governor (one wire)", restore=back, stale_s=m
    ↪ ax(60.0, 5 * a.interval))

    stop = {"now": False}
    signal.signal(signal.SIGTERM, lambda *_: stop.__setitem__("now", T
    ↪ rue))
    t0 = time.time()
    failed = warned = False
    try:

        while not stop["now"] and not gov.killed() and not master.is_o
    ↪ ff() and (a.duration <= 0 or time.time() - t0 < a.duration):
            try:
                master.heartbeat(); gov.step()
            except WriteFailed as e:   # the actuator refused: the arm
    ↪  ends here, never carries on as if watching
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                gov.audit({"fatal": f"write failed: {e}"}); failed = T
    ↪ rue
                break
            except Exception as e:  # noqa: BLE001  a failed decision
    ↪ is logged; the kill path still runs
                gov.audit({"error": f"{type(e).__name__}: {e}"})

            if gov.foreign and not warned:
                print("omni-gpu: another writer changed the power limi
    ↪ t; observing only", file=sys.stderr, flush=True)
                warned = True
            end = time.time() + a.interval
            while time.time() < end and not stop["now"] and not gov.ki

    ↪ lled() and not master.is_off():
                time.sleep(0.2)
    finally:
        ok = gov.restore()
    return 5 if gov.foreign else 3 if not ok else 4 if failed else 0

if __name__ == "__main__":
    sys.exit(main())
```

H.6 omni_controller/muscles.py

886 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Live muscles for the Kubernetes controller beyond HPA and nodes.

Each muscle pulls (afferent) and pushes (efferent) through kubectl, re
    ↪ cords what it changed so the reset can hand
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
the muscle back, and tags every write with its muscle name in the audi
    ↪ t log.

  power_cap   push: at the max (the default dial) the cap is never set
    ↪  under native: each machine's idle CPU is conveyed
              to the capped deployments' running pods (convey: one cli

    ↪ ck under the machine, its last 5% never handed out,
              never under the operator's limit), and the frequency cei
    ↪ ling and the GPU limit stay at their top. The
              tune-down (--dial save): CPU limit of each running pod =
    ↪  base limit x the governor's power cap, never under
              the pod's own request nor its use plus headroom (nothing

    ↪  fixed), resized in place (no restart; Linux CFS
              quota throttles the containers for real); pull: power fr
    ↪ om --power-cmd
  heat        pull: thermal state from the harness heat law driven by
    ↪ live power stress (modelled: kind has no thermometer)
  security    pull: ConfigMap key "hold" == "true" means a security ho

    ↪ ld; the shield then blocks every expansion
  rollout     push: pause a deployment's rollout while the governor do
    ↪ es not permit change; resume it when it does;
              undo a rollout that has exceeded its progress deadline w
    ↪ hen the governor authorises rollback
  batch       push: admit (unsuspend) held batch Jobs, one a minute, o

    ↪ nly when change is permitted and load and power
              stress are under the compass's wall less one cushion (0.
    ↪ 95 - 0.05: one click under, the nervous system's
              own admit gate); running Jobs are never suspended
  latency     pull: 95th-percentile response time over the last --late
    ↪ ncy-window-s from --latency-file (CSV written by

              scripts/latency_probe.py); pressure max(0, p95 / --slo-m
    ↪ s - 1) enters the engine as queue pressure
  cpu_pstate  hardware connector. pull: --rapl-cmd prints package watt
    ↪ s (e.g. from /sys/class/powercap/intel-rapl);
              push: --cpufreq-cmd with {khz}, the frequency ceiling =
    ↪ max frequency x power cap (e.g. writing
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            scaling_max_freq with scripts/cpufreq_ceiling.sh {khz},
  ↪ see hardware/SCHEDUTIL.md, or `cpupower frequency-set -u {khz}kH
  ↪ z`); kill runs it with the maximum frequency
gpu         hardware connector. pull: --gpu-query-cmd prints "watts,
  ↪ celsius[,min_watts[,slowdown_celsius]]" (e.g.
            `nvidia-smi --query-gpu=power.draw,temperature.gpu,power

  ↪ .min_limit --format=csv,noheader,nounits` plus the
            card's slowdown temperature): the card's own figures, re
  ↪ ad live. One degree under its own slowdown
            temperature reads as heat 1.0 (--gpu-temp-limit only for
  ↪  a card that does not report it); push:
            --gpu-power-cmd with {w}, the power limit = max limit x

  ↪ power cap, never under the card's own minimum
            (e.g. `nvidia-smi -pl {w}`); kill restores the maximum l
  ↪ imit
rightsize   push: each running pod's CPU *request* follows its measu
  ↪ red use x (1 + headroom), resized in place through the
            pods/resize subresource (no restart); never above the li

  ↪ mit, never below --rightsize-min-m; no decrease while
            the SLO is breached, no increase during a security hold;
  ↪  kill restores every original request
coldstart   push: a deployment with no work waiting (ConfigMap key "
  ↪ queue" == 0, the queue a KEDA scaler would read) for
            --coldstart-idle-s seconds is scaled to zero; it is scal

  ↪ ed back to its recorded replica count the moment
            work is waiting; kill restores the recorded count
batch_pace  push: while power stress or heat is high, one running Jo
  ↪ b labelled omnicompass.io/pausable=true is suspended
            a minute (checkpointed training pauses instead of the si
  ↪ te going over its limit); it is resumed when

            stress and heat are back down; kill resumes every Job Om
  ↪ ni paused
contain     push: an agent namespace whose measured CPU use exceeds
  ↪ --contain-cpu-m, or any namespace during a security
            hold, gets a ResourceQuota (limits.cpu = budget) and its
  ↪  running pods' CPU limits are scaled in place so the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
              total fits the budget; lifted when use is back under bud
    ↪ get; kill deletes the quota and restores the limits
  cooling     facility connector. push: --cooling-cmd with {c}, the su
    ↪ pply-air setpoint moved between --cooling-min-c and
              --cooling-max-c by the heat state (warm setpoint while c
    ↪ ool, saving chiller energy; cold setpoint as heat

              rises); kill runs it with --cooling-restore-c
  Hardware connectors are off unless their commands are given (not ava
    ↪ ilable on CI runners); the mechanism is the same.
"""
from __future__ import annotations

import csv, json, math, shlex, subprocess, time
import re


# omni-v22 (30 October): a hand-back step is tried this many times aga
    ↪ inst a transient failure of the cluster's API (a 500 from
# the server, a dropped connection), 1 s apart doubling to 4 s, about

    ↪ 10 s in all, inside the rig's 60 s wait for the governor
# to exit and the master switch's stale time. Before, the first failed
    ↪  read ended the whole hand-back: six-kube run
# 41044680505, repetition 346, one 500 on the autoscaler list left eve
    ↪ ry conveyed limit and the replica range behind, and the
# arm was marked invalid. Each muscle's hand-back is tried on its own,

    ↪  and what did not land is raised at the end, never silent.
RESTORE_TRIES = 5


CPU_ANN = "omnicompass.io/original-cpu-limit"
PAUSE_ANN = "omnicompass.io/paused-by-omni"
BATCH_LABEL = "omnicompass.io/batch=true"

PAUSABLE_LABEL = "omnicompass.io/pausable=true"
REQ_ANN = "omnicompass.io/original-cpu-request"
REPL_ANN = "omnicompass.io/original-replicas"
PACE_ANN = "omnicompass.io/paced-by-omni"
QUOTA = "omni-containment"
LIM_ANN = "omnicompass.io/original-limits"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# a raise the cluster's admission refused is not written again until t
    ↪ he pod's limit changes or this wait passes, doubled at
# each refusal up to the last (v12): the refusal is the authority's an
    ↪ swer, and asking again every fast check is hammering
REFUSED_WAIT_S = (300.0, 3600.0)

def milli(v):
    v = str(v)
    return float(v[:-1]) if v.endswith("m") else float(v) * 1000.0

def claim_m(container):
    """The CPU (milli) a container may use: its own limit where it has
    ↪  one, else its request (omni-v13). A limit is the
    container's own room up to which the kernel lets it burst; that ro
    ↪ om is its muscle's, never idle for another."""
    res = container.get("resources", {}) or {}

    req = milli((res.get("requests", {}) or {}).get("cpu", "0") or "0"
    ↪ )
    lim = (res.get("limits", {}) or {}).get("cpu")
    return max(req, milli(lim)) if lim else req

def has_room(pod):
    """A pod with room of its own: a container whose limit is above it
    ↪ s request, the CPU between them the pod's to burst
    into (omni-v13). Conveyed to another muscle on a host with no idle
    ↪  CPU, that room is a trade the engine never makes."""
    for c in pod["spec"]["containers"]:

        res = c.get("resources", {}) or {}
        if claim_m(c) > milli((res.get("requests", {}) or {}).get("cpu
    ↪ ", "0") or "0"):
            return True
    return False

def ref(s):
    ns, name = s.split("/", 1)
    return ns, name
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# what only a shell reads in a command: a pipe, a list, a redirect, a
    ↪ subshell, a variable or a substitution, a glob, a
# home directory, a comment, a line break (quotes are read by shlex.sp
    ↪ lit as a shell reads them, and need no shell)
SHELL_ONLY = set("|&;<>()$`*?[]{}~#\n")

def run_operator_cmd(cmd, timeout=None):
    """The operator's own configured command (a site power reading, a
    ↪ muscle's read command, a wall meter's `cmd:`),
    with the governor's own rights, never text read from the cluster o
    ↪ r the network. omni-v14: a command that holds nothing

    only a shell reads is split into its words and run directly, witho
    ↪ ut a shell; one that does (a pipe, a redirect, a
    variable, VAR=value before the command) runs through the shell as
    ↪ before, so no operator's wiring changes. The
    static scan's three high findings were these calls (docs/SECURITY_
    ↪ SCAN.md)."""

    words = shlex.split(cmd) if not (set(cmd) & SHELL_ONLY) else None
    if words and "=" not in words[0]:
        return subprocess.run(words, capture_output=True, text=True, t
    ↪ imeout=timeout)
    return subprocess.run(cmd, shell=True, capture_output=True, text=T
    ↪ rue, timeout=timeout)

class Muscles:
    def __init__(self, k, a, audit):
        self.k, self.a, self.audit = k, a, audit
        self.gain = {}       # (ns, deployment) -> mean conveyed CPU l

    ↪ imit / operator's limit
        self.refused = {}    # "ns/pod" -> (limit when refused, the ra
    ↪ ise refused, retry at (s), wait (s)), v12
        self.base = {}       # (ns, deployment) -> operator's CPU limi
    ↪ t (millicores)
        self.thermal = 0.32  # harness initial thermal state
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    self.gpu_min_w = 0.0     # the card's own minimum power limit,
↪  read live (0 until the card reports it)
    self.gpu_heat_c = 0.0    # one degree under the card's own slo
↪ wdown temperature, read live (0: not reported)
    # omni-v16: the governor's decision period, so every muscle ru
↪ le written per decision keeps its time at any period

    # (--interval 0 is a model's loop: each call one decision of t
↪ he minute)
    self.period = float(getattr(a, "interval", 60.0) or 0.0) or 60
↪ .0
    self.pace = self.period / 60.0
    self.pushes = 0          # decisions this side has acted on: t

↪ he clock of the rules below

def _per(self, sec):
    """Decisions in sec seconds at this decision period (never few
↪ er than one)."""
    return max(1, int(round(float(sec) / self.period)))

def native(self, muscle):
    """omni-v14: a muscle the operator left native (--native) is n
↪ ever written; every other runs as before."""
    return muscle in getattr(self.a, "native_muscles", ())

def at_max(self):
    """The max (the default dial): no level is ever set under what
↪  native runs."""
    return getattr(self.a, "dial", "max") == "max"

# ---- afferent -----------------------------------------------

↪ ------------------------------------------------
def sense(self, power_stress):
    o = {}
    if getattr(self.a, "thermal_model", False):
        # the heat law's step is a minute (0.86 of the heat kept a
↪  minute, omni-v16 at any decision period)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        keep, take = (0.86, 0.14) if self.pace == 1.0 else (0.86 *
↪ * self.pace, 1.0 - 0.86 ** self.pace)
        self.thermal = min(1.35, max(0.0, keep * self.thermal + ta
↪ ke * (0.34 + 0.62 * min(1.35, power_stress))))
        o["thermal"] = self.thermal
    if getattr(self.a, "rapl_cmd", ""):

        o["cpu_watts"] = self._read(self.a.rapl_cmd)
    if getattr(self.a, "gpu_query_cmd", ""):
        out = self._run_out(self.a.gpu_query_cmd)
        try:
            vals = [float(x) for x in out.split(",")]
            w, t = vals[:2]

            o["gpu_watts"], o["gpu_celsius"] = w, t
            if len(vals) > 2 and vals[2] > 0:
                self.gpu_min_w = vals[2]                    # the
↪ card's own floor
            if len(vals) > 3 and vals[3] > 1:
                self.gpu_heat_c = vals[3] - 1.0             # one

↪ degree under the card's own slowdown
            limit = self.gpu_heat_c or self.a.gpu_temp_limit
            o["thermal"] = max(o.get("thermal", 0.0), min(1.35, t
↪ / max(1.0, limit)))
        except ValueError:
            pass

    lf = getattr(self.a, "latency_file", "")
    if lf and getattr(self.a, "slo_ms", 0):
        ls = latency_sense(lf, self.a.latency_window_s)
        o["latency_blind"] = 1.0 if ls["blind"] else 0.0
        o["latency_age_s"] = ls["age_s"]
        if not ls["blind"]:

            o["latency_p95_ms"] = ls["p95"]
            o["latency_pressure"] = max(0.0, ls["p95"] / self.a.sl
↪ o_ms - 1.0)
            if ls["fail"]:        # failed requests in a live wind
↪ ow are real service failures: full pressure share
                o["latency_pressure"] = max(o["latency_pressure"],
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  ls["fail"] / (ls["ok"] + ls["fail"]))
        cm = getattr(self.a, "security_configmap", "")
        if cm:
            ns, name = ref(cm)
            try:
                data = self.k.get("get", "configmap", name, "-n", ns,
    ↪ "-o", "json").get("data", {}) or {}
                o["security_block"] = 1.0 if str(data.get("hold", ""))
    ↪ .lower() == "true" else 0.0
            except Exception:
                o["security_block"] = 0.0
        return o

    def _run_out(self, cmd):
        return run_operator_cmd(cmd).stdout.strip()

    def _read(self, cmd):
        try:
            return float(self._run_out(cmd).split()[0])
        except (ValueError, IndexError):
            return float("nan")

    def _hw(self, template, value, why):
        cmd = template.format(khz=int(value), w=int(value), v=value)
        self.audit({"write": shlex.split(cmd), "why": why, "dry_run":
    ↪ self.a.dry_run})
        if not self.a.dry_run:
            subprocess.run(shlex.split(cmd), check=True)

    # ---- efferent ---------------------------------------------------
    ↪ ------------------------------------------------
    def push(self, d, obs, killed=False):
        if killed:
            return
        self.pushes += 1
        cap = 1.0 if self.at_max() else float(d["power_cap"])     # th
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ e max: never under native
        if not self.native("cpu"):
            self._power_cap(cap, obs)
        self._hardware(cap, obs)
        if not self.native("rollout"):
            self._rollout(bool(d["change_permitted"]), bool(d["rollbac
    ↪ k_authorized"]))
        if not self.native("batch"):
            self._batch(bool(d["change_permitted"]), obs)
        for name, muscle, fn in (("rightsize", "rightsize", lambda: se
    ↪ lf._rightsize(obs)), ("coldstart", "coldstart", self._coldstart)
    ↪ ,
                                 ("batch_pace", "batch", lambda: self.
    ↪ _batch_pace(obs)), ("contain", "contain", lambda: self._contain(
    ↪ obs)),
                                 ("cooling", "cooling", lambda: self._
    ↪ cooling(obs))):
            if not self.native(muscle):
                self._guard(name, fn)

    def _may(self, organ, what):
        """Nervous-system authority for an organ ("expand", "contract"
    ↪ , "admit", "pause"); without it, everything allowed."""
        a = getattr(self, "auth", None)
        if not a:
            return True
        return bool(a.get("organs", {}).get(organ, {}).get(what, False
    ↪ ))

    def _envelope(self, organ, default):
        a = getattr(self, "auth", None)
        return (a or {}).get("organs", {}).get(organ, {}).get("envelop
    ↪ e", default)

    def _guard(self, name, fn):
        """One lever failing is logged and never stops the others (nor
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  the reset)."""
        try:
            fn()
        except Exception as e:  # noqa: BLE001
            err = getattr(e, "stderr", "") or str(e)
            self.audit({"error": f"{name}: {type(e).__name__}: {str(er
    ↪ r).strip()[:400]}"})

    def _guard_restore(self, name, fn):
        """A lever's hand-back, tried RESTORE_TRIES times against a tr
    ↪ ansient failure of the cluster's API (omni-v22: the
        hand-back must land; a 500 from the server on one read had lef
    ↪ t the conveyed limits behind), every failed try in the
        audit; one lever failing never stops the others. A fault that
    ↪ is not the API's weather is reported at once, not
        retried. Returns whether it landed."""
        for attempt in range(1, RESTORE_TRIES + 1):
            try:
                fn()
                return True
            except Exception as e:  # noqa: BLE001
                err = getattr(e, "stderr", "") or str(e)
                self.audit({"error": f"{name}: {type(e).__name__}: {st
    ↪ r(err).strip()[:400]}", "attempt": attempt, "of": RESTORE_TRIES}
    ↪ )
                if not isinstance(e, (subprocess.CalledProcessError, O
    ↪ SError)):
                    return False
                if attempt < RESTORE_TRIES:
                    time.sleep(min(4.0, 2.0 ** (attempt - 1)))
        return False

    # ---- new levers ------------------------------------------------
    ↪ ------------------------------------------------
    def _named_usage(self, ns, sel=None):
        args = ["top", "pods", "-n", ns, "--no-headers"] + (["-l", sel
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ] if sel else [])
        try:
            out = self.k.get(*args)
        except Exception:
            return {}
        return {line.split()[0]: milli(line.split()[1]) for line in st
    ↪ r(out).strip().splitlines() if line.strip()}

    def _rightsize(self, obs):
        for target in filter(None, getattr(self.a, "rightsize_deployme
    ↪ nts", "").split(",")):
            ns, name = ref(target)
            dep = self.k.get("get", "deployment", name, "-n", ns, "-o"
    ↪ , "json")
            ann = dep["metadata"].get("annotations", {}) or {}
            c0 = dep["spec"]["template"]["spec"]["containers"][0].get(
    ↪ "resources", {})
            base = c0.get("requests", {}).get("cpu")
            if base is None:
                continue
            if REQ_ANN not in ann:
                self.k.write(["annotate", "deployment", name, "-n", ns
    ↪ , f"{REQ_ANN}={int(milli(base))}m"], "rightsize: record original
    ↪  CPU request")
            sel = ",".join(f"{k}={v}" for k, v in dep["spec"]["selecto
    ↪ r"]["matchLabels"].items())
            use = self._named_usage(ns, sel)
            for pod in self._pods(dep, ns):
                pn = pod["metadata"]["name"]
                if pn not in use:
                    continue
                res = pod["spec"]["containers"][0].get("resources", {}
    ↪ )
                cur = milli(res.get("requests", {}).get("cpu", base))
                lim = milli(res.get("limits", {}).get("cpu", "1000000m
    ↪ "))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            want = int(min(lim, max(self.a.rightsize_min_m, math.c
↪ eil(use[pn] * (1.0 + self.a.rightsize_headroom) / 10.0) * 10)))
            if want < cur and (not obs.get("slo_clean", True) or n
↪ ot self._may("pods", "contract")):
                continue  # SLO reflex / nervous system: no shrink
↪ ing without contraction authority

            if want > cur and obs.get("security_block", 0.0) > 0.5
↪ :
                continue  # shield I1: no expansion during a secur
↪ ity hold
            if abs(want - cur) < self.a.cap_min_change_m:
                continue

            self.k.write(["patch", "pod", pn, "-n", ns, "--subreso
↪ urce", "resize", "--type=json", "-p",
                          json.dumps([{"op": "replace", "path": "/
↪ spec/containers/0/resources/requests/cpu", "value": f"{want}m"}]
↪ )],
                         f"rightsize: CPU request {int(cur)}m -> {
↪ want}m in place (use {int(use[pn])}m)")

    def _coldstart(self):
        sig = getattr(self.a, "coldstart_signal", "")
        if not sig:
            return

        sns, sname = ref(sig)
        try:
            q = float((self.k.get("get", "configmap", sname, "-n", sns
↪ , "-o", "json").get("data") or {}).get("queue", "0"))
        except Exception:
            return

        self._idle = getattr(self, "_idle", 0) + 1 if q <= 0 else 0
        n_idle = int(self.a.coldstart_idle) if getattr(self.a, "coldst
↪ art_idle", None) is not None \
            else self._per(getattr(self.a, "coldstart_idle_s", 180.0))
↪           # the idle time in seconds (omni-v16)
        for target in filter(None, getattr(self.a, "coldstart_deployme
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ nts", "").split(",")):
            ns, name = ref(target)
            dep = self.k.get("get", "deployment", name, "-n", ns, "-o"
    ↪ , "json")
            ann = dep["metadata"].get("annotations", {}) or {}
            rep = int(dep["spec"].get("replicas", 1))

            if q > 0 and rep == 0 and REPL_ANN in ann:
                self.k.write(["scale", "deployment", name, "-n", ns, f
    ↪ "--replicas={ann[REPL_ANN]}"], f"coldstart: wake to {ann[REPL_AN
    ↪ N]} (queue {q:g})")
            elif self._idle >= n_idle and rep > 0 and self._may("pods"
    ↪ , "contract"):

                self.k.write(["annotate", "deployment", name, "-n", ns
    ↪ , "--overwrite", f"{REPL_ANN}={rep}"], "coldstart: record replic
    ↪ as")
                self.k.write(["scale", "deployment", name, "-n", ns, "
    ↪ --replicas=0"], f"coldstart: scale to zero (idle {self._idle} de
    ↪ cisions)")

    def _batch_pace(self, obs):
        if not getattr(self.a, "batch_pace", False):
            return
        hot = obs.get("power_stress", 0.0) >= self.a.pace_high or obs.
    ↪ get("thermal", 0.0) >= self.a.pace_heat \

            or (getattr(self, "auth", None) is not None and self._may(
    ↪ "batch", "pause"))
        calm = obs.get("power_stress", 1.0) <= self.a.pace_low and obs
    ↪ .get("thermal", 1.0) < self.a.pace_heat - 0.06
        if self.pushes - getattr(self, "_paced_at", -10 ** 9) < self._
    ↪ per(60.0):

            return               # one job paused or resumed a minute,
    ↪  however often the governor decides (omni-v16)
        jobs = self.k.get("get", "jobs", "-A", "-l", PAUSABLE_LABEL, "
    ↪ -o", "json")["items"]
        if hot:
            run = [j for j in jobs if not j["spec"].get("suspend")]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            if run:
                j = run[0]; ns, name = j["metadata"]["namespace"], j["
    ↪ metadata"]["name"]
                self.k.write(["annotate", "job", name, "-n", ns, "--ov
    ↪ erwrite", f"{PACE_ANN}=true"], "batch_pace: record pause")
                self.k.write(["patch", "job", name, "-n", ns, "--type=
    ↪ merge", "-p", json.dumps({"spec": {"suspend": True}})],
                             f"batch_pace: pause job (power stress {ob
    ↪ s.get('power_stress', 0.0):.2f}, heat {obs.get('thermal', 0.0):.
    ↪ 2f})")
                self._paced_at = self.pushes
        elif calm:
            paced = [j for j in jobs if j["spec"].get("suspend") and (
    ↪ j["metadata"].get("annotations") or {}).get(PACE_ANN) == "true"]
            if paced:
                self._resume_job(paced[0], "batch_pace: resume job (st
    ↪ ress and heat back down)")
                self._paced_at = self.pushes

    def _resume_job(self, j, why):
        ns, name = j["metadata"]["namespace"], j["metadata"]["name"]
        self.k.write(["patch", "job", name, "-n", ns, "--type=merge",
    ↪ "-p", json.dumps({"spec": {"suspend": False}})], why)
        self.k.write(["annotate", "job", name, "-n", ns, f"{PACE_ANN}-
    ↪ "], "batch_pace: clear pause record")

    def _quota(self, ns):
        try:
            return self.k.get("get", "resourcequota", QUOTA, "-n", ns,
    ↪  "-o", "json")
        except Exception:
            return None

    def _contain(self, obs):
        """Contain on over-budget use or a security hold; while contai
    ↪ ned each running pod gets an equal share of the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    budget as its CPU limit (in place); lift only when use is unde
↪ r 80% of the budget (release band) and no hold."""
    for ns in filter(None, getattr(self.a, "contain_namespaces", "
↪ ").split(",")):
        total = sum(self._named_usage(ns).values())
        budget = float(self.a.contain_cpu_m)

        hold = obs.get("security_block", 0.0) > 0.5
        q = self._quota(ns)
        if q is None and (total > budget or hold):
            self.k.write(["create", "quota", QUOTA, "-n", ns, f"--
↪ hard=limits.cpu={int(budget)}m"],
                         f"contain: quota limits.cpu={int(budget)}

↪ m (use {int(total)}m)")
            q = {"metadata": {"annotations": {}}}
        elif q is not None and total < 0.8 * budget and not hold:
            self._release(ns, q, "contain: use back under budget,
↪ lift containment")
            continue

        if q is None:
            continue
        held = json.loads((q["metadata"].get("annotations") or {})
↪ .get(LIM_ANN, "{}"))
        run = [p for p in self.k.get("get", "pods", "-n", ns, "-o"
↪ , "json")["items"] if p["status"].get("phase") == "Running"

               and p["spec"]["containers"][0].get("resources", {})
↪ .get("limits", {}).get("cpu") is not None]
        share = max(10, int(budget / max(1, len(run)) / 10) * 10)
        changed = False
        for p in run:
            pn = p["metadata"]["name"]; lim = p["spec"]["container

↪ s"][0]["resources"]["limits"]["cpu"]
            if pn not in held:
                held[pn] = lim; changed = True
            want = min(share, int(milli(held[pn])))
            if int(milli(lim)) != want:
                self.k.write(["patch", "pod", pn, "-n", ns, "--sub
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ resource", "resize", "--type=json", "-p",
                                  json.dumps([{"op": "replace", "path"
    ↪ : "/spec/containers/0/resources/limits/cpu", "value": f"{want}m"
    ↪ }])],
                                 f"contain: pod CPU limit {lim} -> {wa
    ↪ nt}m in place (budget share)")

            if changed:
                # the original limits live on the quota object, so a r
    ↪ eset in any process can restore them
                self.k.write(["annotate", "resourcequota", QUOTA, "-n"
    ↪ , ns, "--overwrite", f"{LIM_ANN}={json.dumps(held, sort_keys=Tru
    ↪ e)}"],

                             "contain: record original pod CPU limits"
    ↪ )

    def _release(self, ns, q, why):
        # the quota goes first: while it stands, Kubernetes refuses to
    ↪  raise a pod's limit back above the budget share

        held = json.loads((q["metadata"].get("annotations") or {}).get
    ↪ (LIM_ANN, "{}"))
        self.k.write(["delete", "resourcequota", QUOTA, "-n", ns], f"{
    ↪ why}: delete quota")
        for pn, lim in held.items():
            self._guard(f"{why} {pn}", lambda pn=pn, lim=lim: self.k.w

    ↪ rite(
                ["patch", "pod", pn, "-n", ns, "--subresource", "resiz
    ↪ e", "--type=json", "-p",
                 json.dumps([{"op": "replace", "path": "/spec/containe
    ↪ rs/0/resources/limits/cpu", "value": lim}])],
                f"{why}: restore pod CPU limit {lim}"))

    def _cooling(self, obs):
        cmd = getattr(self.a, "cooling_cmd", "")
        if not cmd:
            return
        th = obs.get("thermal", 0.5)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        x = min(1.0, max(0.0, (th - 0.5) / 0.5))
        c = round(self.a.cooling_max_c - (self.a.cooling_max_c - self.
    ↪ a.cooling_min_c) * x, 1)
        c = round(min(c, max(self.a.cooling_min_c, self._envelope("coo
    ↪ ling", [0, 99])[1])), 1)   # nervous-system envelope
        if abs(c - getattr(self, "_setpoint", -99.0)) >= 0.5:
            self._hw(cmd.replace("{c}", "{v}"), c, f"cooling: supply-a
    ↪ ir setpoint {c} C (heat {th:.2f})")
            self._setpoint = c

    def _pods(self, dep, ns):
        sel = ",".join(f"{k}={v}" for k, v in dep["spec"]["selector"][
    ↪ "matchLabels"].items())
        return [p for p in self.k.get("get", "pods", "-n", ns, "-l", s
    ↪ el, "-o", "json")["items"]
                if p["status"].get("phase") == "Running"]

    def _pod_usage(self, dep, ns):
        sel = ",".join(f"{k}={v}" for k, v in dep["spec"]["selector"][
    ↪ "matchLabels"].items())
        try:
            out = self.k.get("top", "pods", "-n", ns, "-l", sel, "--no
    ↪ -headers")
        except Exception:
            return []
        return [milli(line.split()[1]) for line in str(out).strip().sp
    ↪ litlines() if line.strip()]

    def _resize(self, pod, ns, value, why):
        self.k.write(["patch", "pod", pod["metadata"]["name"], "-n", n
    ↪ s, "--subresource", "resize", "--type=json", "-p",
                      json.dumps([{"op": "replace", "path": "/spec/con
    ↪ tainers/0/resources/limits/cpu", "value": value}])], why)

    def _power_cap(self, cap, obs):
        """In-place pod resize (no restart): each running pod's CPU li
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ mit = base limit x cap; the kernel's CFS quota
        enforces it. The deployment template is not changed, so no rol
    ↪ lout is triggered."""
        if not obs.get("slo_clean", True):
            cap = 1.0  # SLO reflex: no power capping while service is
    ↪  (or was just) over its response-time target

        if getattr(self.a, "latency_file", "") or self.at_max():
            # a request-served workload: its work is set by arrivals,
    ↪ not by the cap, so throttling it saves no energy
            # (the same CPU-seconds run later) and only adds wait. The
    ↪  energy goes where the work is: convey(). And at
            # the max every capped pod gets its machine's idle CPU, ne

    ↪ ver less than its operator gave it
            return self.convey(obs)
        for target in filter(None, getattr(self.a, "cap_deployments",
    ↪ "").split(",")):
            ns, name = ref(target)
            dep = self.k.get("get", "deployment", name, "-n", ns, "-o"

    ↪ , "json")
            ann = dep["metadata"].get("annotations", {}) or {}
            c0 = dep["spec"]["template"]["spec"]["containers"][0]
            tmpl = c0.get("resources", {}).get("limits", {}).get("cpu"
    ↪ )
            if tmpl is None:

                continue
            base = milli(ann.get(CPU_ANN, tmpl))
            req = milli(c0.get("resources", {}).get("requests", {}).ge
    ↪ t("cpu", "0"))
            want = int(max(req, round(base * max(self.a.cap_min, min(1
    ↪ .0, cap)) / 10.0) * 10))

            # reflex: never cap a pod below what it is using plus head
    ↪ room (as the node muscle never goes below requests)
            use = self._pod_usage(dep, ns)
            if use:
                want = int(min(base, max(want, math.ceil(max(use) * (1
    ↪ .0 + self.a.cap_headroom) / 10.0) * 10)))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            if CPU_ANN not in ann:
                self.k.write(["annotate", "deployment", name, "-n", ns
    ↪ , f"{CPU_ANN}={int(base)}m"], "power_cap: record original CPU li
    ↪ mit")
            for pod in self._pods(dep, ns):
                cur = milli(pod["spec"]["containers"][0].get("resource
    ↪ s", {}).get("limits", {}).get("cpu", tmpl))
                if obs.get("security_block", 0.0) > 0.5 and want > cur
    ↪ :
                    continue  # shield I1: no expansion during a secur
    ↪ ity hold
                if abs(want - cur) < self.a.cap_min_change_m:
                    continue
                self._resize(pod, ns, f"{want}m", f"power_cap: pod CPU
    ↪  limit to {want}m in place (cap {cap:.3f})")

    def convey(self, obs=None):
        """Energy to where the work is (omnicompass/conveyance.py, on
    ↪ one machine's CPU).

        A serving pod's CPU limit is a quota: a request that needs mor
    ↪ e CPU than one quota period allows waits for the
        next period, while the machine it runs on sits idle. That wait
    ↪  is energy withheld from the work, not saved: the
        request uses the same CPU-seconds either way. So each machine'
    ↪ s idle CPU is conveyed to the serving pods on it:

            c_i = min( max(L_i, (0.95 A_j - Q_j) / |P_j|), 0.95 A_j )

        A_j the machine's allocatable CPU, Q_j what every other pod on
    ↪  it may use (its own limit, or its request where it
        has no limit: omni-v13; the requests alone before), P_j the se
    ↪ rving pods on it, L_i the operator's own limit. The
        machine's last five percent is never handed out (the living ba
    ↪ nd), a pod never gets
        less than its operator gave it, and the requests (what the sch
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ eduler and the autoscaler read) are untouched.
        Resized in place through pods/resize: no pod restarts. No expa
    ↪ nsion during a security hold. The reset
        returns every pod to L_i.

        On kind every node is a container on one machine and reports t
    ↪ hat machine's whole CPU as its own, so the limits
        the nodes hand out sum to more than the machine: a ceiling, no
    ↪ t a use, and the kernel's fair scheduler arbitrates
        when the serving pods ask for more than the machine, as it doe
    ↪ s for native's pods. v7 adds one guard, the same
        as the replica room's (capacity.py): on such a shared host no
    ↪ limit is raised while the machine's measured busy
        share is at or over 75% or cannot be read; limits already give
    ↪ n stay, and lowering is never held back. Each hold
        is written to the audit ("convey": "held"). omni-v13 adds the
    ↪ hand-back: while another workload on the shared
        host has room of its own (a pod, not one of the serving pods,
    ↪ whose limit is above its request) and the host is
        at its top (busy at or over 75%), there is no idle CPU to conv
    ↪ ey, so every limit given goes back to the
        operator's; it is conveyed again only once the host's busy sha
    ↪ re is under 70% (the band's 5% cushion below the
        top), so the hand-back and the raise never chase each other. O
    ↪ n KubeRay's rig the limits given in the queue's
        first half minute stayed through the saturated queue: the batc
    ↪ h pods hashed 7.6% slower, the queue finished
        0.7% later and the web served no faster (20 October, docs/OMNI
    ↪ _V13.md). A host with no such workload keeps the
        v7 rule: what the serving pods use there at the top is the hos
    ↪ t's own idle, and no other muscle's room.
        Two other designs were tried in the model gate (tools/model_ga
    ↪ te.py,
        docs/OMNI_V7.md) and refused by it: the host's CPU counted onc
    ↪ e and shared by every serving pod (at the v6 KEDA
        runs' nine replicas, 3,500m / 9 is under the operator's 500m,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    so nothing is ever conveyed and the +64% work and
        -53% response time of conveyance are lost whole), and a hold w
    hile a pod starts on the host (a start wait 1%
        shorter for a LOSS on work and speed). The slower pod start of
    the v6 KEDA runs (+12.5% mean, EVEN on the
        three-run rule) is the busier host serving more requests: the

    price of the work, not a fault to hold against.
        The namespace's CPU quota, where the governor's role can read
    it, bounds every raise (omni_controller/quota.py);
        where it cannot be read, no budget is known and the raise proc
    eeds as on v6, Kubernetes' own admission the
        authority: the first v7 trial read an unreadable quota as zero

    and conveyed nothing (docs/OMNI_V7.md, section 1).


        Conveyance is on by default. It moves no work: the same reques
    ts run with less waiting. (The +26% CPU seen on
        kind is more requests served, because that load is closed-loop
    , not waste.) Optionally, --convey-on (a share

        of --slo-ms) engages it only once the 95th-percentile response
     time reaches convey-on x SLO and releases it
        below convey-off x SLO, returning every serving pod to L_i; a
    blind latency sense engages it (service first)."""
        from omnicompass.nervous_system import BAND
        obs = obs or {}

        targets = [ref(t) for t in filter(None, getattr(self.a, "cap_d
    eployments", "").split(","))]
        if not targets or self.native("cpu"):
            return None
        engaged = self._convey_engaged()
        from omni_controller.capacity import HostCpuSampler, KIND_SCAL

    E_OUT_BUSY_LIMIT, one_host
        node_items = self.k.get("get", "nodes", "-o", "json")["items"]
        alloc = {n["metadata"]["name"]: milli(n["status"]["allocatable
    "]["cpu"]) for n in node_items}
        pods = [p for p in self.k.get("get", "pods", "-A", "-o", "json
    ")["items"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            if p["status"].get("phase") in ("Running", "Pending")
↪ and p["spec"].get("nodeName")]
    shared = one_host(node_items)
    if not hasattr(self, "host_cpu"):
        self.host_cpu = HostCpuSampler()
    busy = self.host_cpu.sample() if shared else None

    # the shared host near its top, or unread: no limit is raised
↪ anywhere on it (v7); elsewhere nothing is held
    busy_hold = shared and (busy is None or busy >= KIND_SCALE_OUT
↪ _BUSY_LIMIT)
    from omni_controller.quota import CpuLimitBudget
    budget = CpuLimitBudget(self.k, self.audit)

    out = {}
    held = set()
    handed = False
    for ns, name in targets:
        self.gain[(ns, name)] = 1.0
        dep = self.k.get("get", "deployment", name, "-n", ns, "-o"

↪ , "json")
        ann = dep["metadata"].get("annotations", {}) or {}
        c0 = dep["spec"]["template"]["spec"]["containers"][0]
        tmpl = c0.get("resources", {}).get("limits", {}).get("cpu"
↪ )
        if tmpl is None:

            continue
        base = milli(ann.get(CPU_ANN, tmpl))
        sel = dep["spec"]["selector"]["matchLabels"]
        mine = lambda p: p["metadata"].get("namespace", "default")
↪  == ns and all(
            (p["metadata"].get("labels", {}) or {}).get(k) == v fo

↪ r k, v in sel.items())
        serving, other = {}, {}
        for p in pods:
            node = p["spec"]["nodeName"]
            if mine(p) and p["status"].get("phase") == "Running":
                serving.setdefault(node, []).append(p)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            elif not mine(p):
                # every other pod by what it may use: its own limi
↪ t where it has one, its request where it has none
                # (omni-v13). Counted by requests (omni-v12 and be
↪ fore), a batch pod's room between its request and
                # its limit read as idle and was conveyed to the s

↪ erving pod: the queue finished later and its tail
                # rose on KubeRay's rig (20 October, docs/OMNI_V13
↪ .md), a trade between muscles the engine never makes
                other[node] = other.get(node, 0.0) + sum(claim_m(c
↪ ) for c in p["spec"]["containers"])
        if CPU_ANN not in ann and serving:

            self.k.write(["annotate", "deployment", name, "-n", ns
↪ , f"{CPU_ANN}={int(base)}m"], "convey: record original CPU limit
↪ ")
        # omni-v13: on a shared host, another workload with room o
↪ f its own (a limit above its request) and the host
        # at its top: there is no idle CPU, every limit given goes

↪  back to the operator's; under the top but within
        # the band's cushion of it, nothing is raised again, so th
↪ e hand-back and the raise never chase each other
        contended = shared and busy is not None and any(not mine(p
↪ ) and has_room(p) for p in pods)
        give_back = contended and busy >= KIND_SCALE_OUT_BUSY_LIMI

↪ T
        cushion = contended and busy >= KIND_SCALE_OUT_BUSY_LIMIT
↪ - BAND[0]
        handed = handed or give_back
        got = []
        for node, ps in serving.items():

            a_j = alloc.get(node, 0.0)
            share = (BAND[1] * a_j - other.get(node, 0.0)) / len(p
↪ s)
            want = int(min(max(base, math.floor(share / 10.0) * 10
↪ ), BAND[1] * a_j)) if engaged and not give_back else int(base)
            for p in ps:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                cur = milli(p["spec"]["containers"][0].get("resour
↪ ces", {}).get("limits", {}).get("cpu", tmpl))
                if obs.get("security_block", 0.0) > 0.5 and want >
↪  cur:
                    got.append(cur); continue  # shield I1: no exp
↪ ansion during a security hold

                if (busy_hold or cushion) and want > cur:
                    held.add(node); got.append(cur); continue  # t
↪ he shared host near its top, or unread
                permitted, reservations = budget.limit(ns, p, cur,
↪  want)
                if abs(permitted - cur) < self.a.cap_min_change_m:

                    got.append(cur); continue
                key = f"{ns}/{p['metadata']['name']}"
                r = self.refused.get(key)
                if r is not None and (cur != r[0] or permitted < r
↪ [1]):
                    self.refused.pop(key); r = None            #

↪ the pod or the raise changed: the refusal is spent
                if r is not None and permitted > cur and time.time
↪ () < r[2]:
                    got.append(cur); continue                  #
↪ held: admission refused this raise and nothing changed
                try:

                    self._resize(p, ns, f"{permitted}m", (
                        f"convey: the shared host at its top (busy
↪  {busy:.2f}) beside another workload's room, operator's limit {p
↪ ermitted}m in place"
                        if give_back else f"convey: {node} idle CP
↪ U to its {len(ps)} serving pod(s), limit {permitted}m in place"

                        if engaged else f"convey: response time ca
↪ lm, operator's limit {permitted}m in place"))
                except Exception as error:
                    # Admission can race another writer. Never adv
↪ ertise a rejected increase to the HPA.
                    wait = REFUSED_WAIT_S[0] if r is None else min
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ (REFUSED_WAIT_S[1], 2.0 * r[3])
                        if permitted > cur:
                            self.refused[key] = (cur, permitted, time.
    ↪ time() + wait, wait)
                        self.audit({"error": f"convey resize rejected:
    ↪  {type(error).__name__}: {str(getattr(error, 'stderr', '') or er
    ↪ ror).strip()[:400]}",
                                    "pod": key, "accepted_limit_m": cu
    ↪ r,
                                    "held_s": wait if permitted > cur
    ↪ else 0.0})
                        got.append(cur)
                        # Do not reuse uncertain headroom for other po
    ↪ ds in this decision.
                        budget.reserve(reservations, cur, permitted)
                        continue
                    budget.reserve(reservations, cur, permitted)
                    self.refused.pop(key, None)
                    out[key] = permitted; got.append(permitted)
            # the gain g = mean conveyed limit / operator's limit: the
    ↪  replica organ reads it to keep the operator's
            # promise in queue terms (target x request / limit) while
    ↪ the limit is larger
            self.gain[(ns, name)] = (sum(got) / len(got) / base) if go
    ↪ t and base > 0 else 1.0
            self.base[(ns, name)] = base
        if held or handed:
            self.audit({"convey": "handed back" if handed else "held",
    ↪  "machines": sorted(held),
                        "host_busy": None if busy is None else round(b
    ↪ usy, 3), "shared_host": shared})
        return out

    def _convey_engaged(self):
        """Hysteresis on the response time: on at convey-on x SLO, off
    ↪  below convey-off x SLO; blind engages."""
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    on, off = getattr(self.a, "convey_on", 0.0), getattr(self.a, "
↪ convey_off", 0.0)
    lf, slo = getattr(self.a, "latency_file", ""), getattr(self.a,
↪  "slo_ms", 0.0)
    if not on or not lf or not slo:
        return True

    ls = latency_sense(lf, self.a.latency_window_s)
    was = getattr(self, "_conveying", False)
    now = True if ls["blind"] else (ls["p95"] >= on * slo or (was
↪ and ls["p95"] >= off * slo))
    if now != was:
        self.audit({"convey": "engaged" if now else "released", "p

↪ 95_ms": None if ls["blind"] else round(ls["p95"], 1),
                    "slo_ms": slo, "blind": ls["blind"]})
    self._conveying = now
    return now


def _hardware(self, cap, obs):

    cap = 1.0 if not obs.get("slo_clean", True) else max(self.a.ca
↪ p_min, min(1.0, cap))
    if obs.get("security_block", 0.0) > 0.5:
        cap = min(cap, getattr(self, "_last_cap", 1.0))  # shield
↪ I1: no expansion during a security hold
    if abs(cap - getattr(self, "_last_cap", 1.0)) < 0.02:

        return
    if getattr(self.a, "cpufreq_policy_root", "") and not self.nat
↪ ive("clock"):
        self._cpufreq_sysfs(cap, obs)
    if getattr(self.a, "cpufreq_cmd", "") and self.a.cpu_max_khz a
↪ nd not self.native("clock"):

        lo, hi = self._envelope("cpufreq", [0.0, 1.0]); cap = min(
↪ max(cap, lo), hi)
        self._hw(self.a.cpufreq_cmd, self.a.cpu_max_khz * cap, f"c
↪ pu_pstate: frequency ceiling {int(self.a.cpu_max_khz * cap)} kHz
↪  (cap {cap:.3f})")
    if getattr(self.a, "gpu_power_cmd", "") and self.a.gpu_max_w a
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ nd not self.native("gpu"):
            w = max(self.gpu_min_w, self.a.gpu_max_w * cap)
    ↪     # never under the card's own minimum
            self._hw(self.a.gpu_power_cmd, w, f"gpu: power limit {int(
    ↪ w)} W (cap {cap:.3f})")
        self._last_cap = cap

    def _cpufreq_sysfs(self, cap, obs):
        """CPU frequency ceiling through the kernel's own policy files
    ↪  (hardware/cpufreq.py).
        ceiling = min(max(cap, envelope floor, schedutil request), env
    ↪ elope ceiling), where the schedutil request is

        min(1, 1.25 u) at the current CPU utilisation u: the ceiling n
    ↪ ever cuts below the frequency schedutil itself
        would ask for, so it only removes headroom the scheduler is no
    ↪ t using. The envelope ceiling (heat) wins last.
        The first write snapshots every policy; the reset restores tho
    ↪ se exact values."""

        from hardware.cpufreq import CpuFreqPolicies, schedutil_freque
    ↪ ncy_invariant
        if getattr(self, "_cf", None) is None:
            self._cf = CpuFreqPolicies(self.a.cpufreq_policy_root)
            snap = self._cf.capture_original()
            if getattr(self.a, "cpufreq_require_schedutil", False):

                self._cf.assert_schedutil()
            self.audit({"cpu_pstate_snapshot": [r.as_dict() for r in s
    ↪ nap]})
        lo, hi = self._envelope("cpufreq", [0.0, 1.0])
        u = float(obs.get("cpu_util", obs.get("load_ratio", 1.0)))
        c = min(max(cap, lo, schedutil_frequency_invariant(u)), hi)

        w = self._cf.apply_cap(c, dry_run=self.a.dry_run)
        self.audit({"cpu_pstate_write": w, "why": f"cpu_pstate: ceilin
    ↪ g {c:.3f} (cap {cap:.3f}, envelope [{lo:.3f}, {hi:.3f}], "
                                                f"schedutil request
    ↪ {schedutil_frequency_invariant(u):.3f} at u {u:.3f})"})
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def _rollout(self, permitted, rollback):
        for target in filter(None, getattr(self.a, "rollout_guard", ""
    ↪ ).split(",")):
            ns, name = ref(target)
            dep = self.k.get("get", "deployment", name, "-n", ns, "-o"
    ↪ , "json")

            ann = dep["metadata"].get("annotations", {}) or {}
            paused = bool(dep["spec"].get("paused"))
            conds = {c["type"]: c for c in dep.get("status", {}).get("
    ↪ conditions", [])}
            stuck = conds.get("Progressing", {}).get("reason") == "Pro
    ↪ gressDeadlineExceeded"

            if rollback and stuck:
                self.k.write(["rollout", "undo", f"deployment/{name}",
    ↪  "-n", ns], "rollout: undo stuck rollout (rollback authorised)")
                continue
            if not permitted and not paused:
                self.k.write(["annotate", "deployment", name, "-n", ns

    ↪ , "--overwrite", f"{PAUSE_ANN}=true"], "rollout: record pause")
                self.k.write(["rollout", "pause", f"deployment/{name}"
    ↪ , "-n", ns], "rollout: pause (change not permitted)")
            elif permitted and paused and ann.get(PAUSE_ANN) == "true"
    ↪ :
                self.k.write(["rollout", "resume", f"deployment/{name}

    ↪ ", "-n", ns], "rollout: resume (change permitted)")
                self.k.write(["annotate", "deployment", name, "-n", ns
    ↪ , f"{PAUSE_ANN}-"], "rollout: clear pause record")

    def _batch(self, permitted, obs):
        if not getattr(self.a, "batch", False):

            return
        if not permitted or obs.get("security_block", 0.0) > 0.5:
            return
        if obs.get("load_ratio", 1.0) >= self.a.batch_load_max or obs.
    ↪ get("power_stress", 1.0) >= self.a.batch_power_max:
            return
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if not self._may("batch", "admit"):
            return
        if self.pushes - getattr(self, "_admitted_at", -10 ** 9) < sel
    ↪ f._per(60.0):
            return              # one held job admitted a minute, how
    ↪ ever often the governor decides (omni-v16)

        jobs = self.k.get("get", "jobs", "-A", "-l", BATCH_LABEL, "-o"
    ↪ , "json")["items"]
        held = sorted((j for j in jobs if j["spec"].get("suspend")), k
    ↪ ey=lambda j: j["metadata"].get("creationTimestamp", ""))
        if held:
            j = held[0]; ns, name = j["metadata"]["namespace"], j["met

    ↪ adata"]["name"]
            self.k.write(["patch", "job", name, "-n", ns, "--type=merg
    ↪ e", "-p", json.dumps({"spec": {"suspend": False}})],
                         "batch: admit held job (headroom)")
            self._admitted_at = self.pushes

    # ---- kill ------------------------------------------------------
    ↪ ------------------------------------------------
    def restore(self):
        """Every lever handed back, each tried RESTORE_TRIES times on
    ↪ its own; returns the names of the levers that did not land."""
        missed = []

        for name, fn in (("kill cooling", self._restore_cooling), ("ki
    ↪ ll contain", self._restore_contain),
                         ("kill batch_pace", self._restore_pace), ("ki
    ↪ ll coldstart", self._restore_coldstart),
                         ("kill rightsize", self._restore_rightsize),
    ↪ ("kill hardware and power cap", self._restore_core)):

            if not self._guard_restore(name, fn):
                missed.append(name)
        return missed

    def _restore_cooling(self):
        if getattr(self.a, "cooling_cmd", ""):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            self._hw(self.a.cooling_cmd.replace("{c}", "{v}"), self.a.
    ↪ cooling_restore_c, "reset: restore cooling setpoint")
            self._setpoint = None

    def _restore_contain(self):
        for ns in filter(None, getattr(self.a, "contain_namespaces", "
    ↪ ").split(",")):
            q = self._quota(ns)
            if q is not None:
                self._release(ns, q, "reset")

    def _restore_pace(self):
        if getattr(self.a, "batch_pace", False):
            for j in self.k.get("get", "jobs", "-A", "-l", PAUSABLE_LA
    ↪ BEL, "-o", "json")["items"]:
                if j["spec"].get("suspend") and (j["metadata"].get("an
    ↪ notations") or {}).get(PACE_ANN) == "true":
                    self._resume_job(j, "reset: resume paced job")

    def _restore_coldstart(self):
        for target in filter(None, getattr(self.a, "coldstart_deployme
    ↪ nts", "").split(",")):
            ns, name = ref(target)
            dep = self.k.get("get", "deployment", name, "-n", ns, "-o"
    ↪ , "json")
            ann = dep["metadata"].get("annotations", {}) or {}
            if REPL_ANN in ann:
                if int(dep["spec"].get("replicas", 1)) == 0:
                    self.k.write(["scale", "deployment", name, "-n", n
    ↪ s, f"--replicas={ann[REPL_ANN]}"], "reset: restore replicas")
                self.k.write(["annotate", "deployment", name, "-n", ns
    ↪ , f"{REPL_ANN}-"], "reset: clear replica record")

    def _restore_rightsize(self):
        for target in filter(None, getattr(self.a, "rightsize_deployme
    ↪ nts", "").split(",")):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        ns, name = ref(target)
        dep = self.k.get("get", "deployment", name, "-n", ns, "-o"
↪ , "json")
        orig = (dep["metadata"].get("annotations", {}) or {}).get(
↪ REQ_ANN)
        if orig:

            for pod in self._pods(dep, ns):
                if pod["spec"]["containers"][0].get("resources", {
↪ }).get("requests", {}).get("cpu") != orig:
                    self.k.write(["patch", "pod", pod["metadata"][
↪ "name"], "-n", ns, "--subresource", "resize", "--type=json", "-p
↪ ",

                                  json.dumps([{"op": "replace", "p
↪ ath": "/spec/containers/0/resources/requests/cpu", "value": orig
↪ }])],

                                 "reset: restore original CPU requ
↪ est in place")
            self.k.write(["annotate", "deployment", name, "-n", ns

↪ , f"{REQ_ANN}-"], "reset: remove request record")


def _restore_core(self):
    if getattr(self, "_cf", None) is not None:
        self.audit({"cpu_pstate_restore": self._cf.restore(dry_run
↪ =self.a.dry_run), "why": "reset: exact pre-Omni CPU ceilings"})

    if getattr(self.a, "cpufreq_cmd", "") and self.a.cpu_max_khz a
↪ nd getattr(self, "_last_cap", 1.0) != 1.0:
        self._hw(self.a.cpufreq_cmd, self.a.cpu_max_khz, "reset: r
↪ estore maximum CPU frequency")
    if getattr(self.a, "gpu_power_cmd", "") and self.a.gpu_max_w a
↪ nd getattr(self, "_last_cap", 1.0) != 1.0:

        self._hw(self.a.gpu_power_cmd, self.a.gpu_max_w, "reset: r
↪ estore maximum GPU power limit")
    self._last_cap = 1.0
    for target in filter(None, getattr(self.a, "cap_deployments",
↪ "").split(",")):
        ns, name = ref(target)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            dep = self.k.get("get", "deployment", name, "-n", ns, "-o"
    ↪ , "json")
            orig = (dep["metadata"].get("annotations", {}) or {}).get(
    ↪ CPU_ANN)
            if orig:
                for pod in self._pods(dep, ns):

                    if pod["spec"]["containers"][0].get("resources", {
    ↪ }).get("limits", {}).get("cpu") != orig:
                        self._resize(pod, ns, orig, "reset: restore or
    ↪ iginal pod CPU limit in place")
                self.k.write(["annotate", "deployment", name, "-n", ns
    ↪ , f"{CPU_ANN}-"], "reset: remove CPU record")

        for target in filter(None, getattr(self.a, "rollout_guard", ""
    ↪ ).split(",")):
            ns, name = ref(target)
            dep = self.k.get("get", "deployment", name, "-n", ns, "-o"
    ↪ , "json")
            if (dep["metadata"].get("annotations", {}) or {}).get(PAUS

    ↪ E_ANN) == "true":
                self.k.write(["rollout", "resume", f"deployment/{name}
    ↪ ", "-n", ns], "reset: resume rollout")
                self.k.write(["annotate", "deployment", name, "-n", ns
    ↪ , f"{PAUSE_ANN}-"], "reset: clear pause record")

def latency_sense(path, window_s, now=None):
    """The latency afferent with its integrity (manuscript Appendix J:
    ↪  the nervous system owns delay and dropout
    handling). Returns p95 of successful requests in the last window,
    ↪ the ok and failed counts in it, and the age of

    the newest sample against the wall clock. The sense is blind when
    ↪ its newest sample is older than two windows, or
    when the window holds no successful request: a hung probe stops wr
    ↪ iting, and its last clean window must not be read
    as the present."""
    import os, time
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    try:
        age = (now if now is not None else time.time()) - os.path.getm
    ↪ time(path)
        rows = list(csv.DictReader(open(path)))
    except OSError:
        return {"p95": float("nan"), "ok": 0, "fail": 0, "age_s": floa
    ↪ t("inf"), "blind": True}
    if not rows:
        return {"p95": float("nan"), "ok": 0, "fail": 0, "age_s": age,
    ↪  "blind": True}
    t_end = float(rows[-1]["elapsed_seconds"])
    win = [r for r in rows if float(r["elapsed_seconds"]) >= t_end - w
    ↪ indow_s]
    ms = sorted(float(r["latency_ms"]) for r in win if r.get("ok") ==
    ↪ "1")
    fails = sum(1 for r in win if r.get("ok") != "1")
    p95 = ms[min(len(ms) - 1, int(0.95 * len(ms)))] if ms else float("
    ↪ nan")
    return {"p95": p95, "ok": len(ms), "fail": fails, "age_s": age, "b
    ↪ lind": age > 2.0 * window_s or not ms}


def latency_window(path, window_s, now=None):
    """The last window_s seconds of successful probe samples (ms), wit
    ↪ h the same integrity rule as latency_sense."""
    import os, time
    try:
        age = (now if now is not None else time.time()) - os.path.getm
    ↪ time(path)
        rows = list(csv.DictReader(open(path)))
    except OSError:
        return {"ms": [], "blind": True}
    if not rows:
        return {"ms": [], "blind": True}
    t_end = float(rows[-1]["elapsed_seconds"])
    ms = [float(r["latency_ms"]) for r in rows if r.get("ok") == "1" a
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ nd float(r["elapsed_seconds"]) >= t_end - window_s]
    return {"ms": ms, "blind": age > 2.0 * window_s or not ms}


def latency_p95(path, window_s):
    """95th-percentile response time (ms) of successful requests in th
    ↪ e last window_s seconds of the probe CSV."""
    try:
        rows = list(csv.DictReader(open(path)))
    except OSError:
        return float("nan")
    if not rows:
        return float("nan")
    t_end = float(rows[-1]["elapsed_seconds"])
    ms = sorted(float(r["latency_ms"]) for r in rows if r.get("ok") ==
    ↪  "1" and float(r["elapsed_seconds"]) >= t_end - window_s)
    fails = sum(1 for r in rows if r.get("ok") != "1" and float(r["ela
    ↪ psed_seconds"]) >= t_end - window_s)
    if fails and not ms:
        return 1e9
    return ms[min(len(ms) - 1, int(0.95 * len(ms)))] if ms else float(
    ↪ "nan")

def add_args(ap):
    ap.add_argument("--cap-deployments", default="", help="ns/name[,ns
    ↪ /name]: power_cap muscle scales their CPU limit")
    ap.add_argument("--cap-min", type=float, default=0.0,
                    help="lowest power cap the tune-down applies, a sh
    ↪ are of the operator's limit; 0 (the default): nothing "
                         "fixed, the pod's own request and its use plu
    ↪ s headroom are the floor")
    ap.add_argument("--cap-min-change-m", type=int, default=20, help="
    ↪ smallest CPU-limit change written, millicores")
    ap.add_argument("--cap-headroom", type=float, default=0.3, help="p
    ↪ ower cap never below pod CPU usage x (1 + headroom)")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ap.add_argument("--latency-file", default="", help="probe CSV (ela
    ↪ psed_seconds,latency_ms,ok) for the latency afferent")
    ap.add_argument("--slo-ms", type=float, default=0.0, help="95th-pe
    ↪ rcentile response-time target, ms")
    ap.add_argument("--latency-window-s", type=float, default=60.0)
    ap.add_argument("--convey-on", type=float, default=0.0,
                    help="0 (default): convey idle CPU always; > 0: on
    ↪ ly once p95 reaches this share of --slo-ms")
    ap.add_argument("--convey-off", type=float, default=0.25, help="re
    ↪ turn pods to the operator's limit once p95 is below this share")
    ap.add_argument("--slo-clear", type=int, default=None, help="decis
    ↪ ions the SLO must stay met before densifying or capping again (u
    ↪ nset: --slo-clear-s)")
    ap.add_argument("--slo-clear-s", type=float, default=180.0, help="
    ↪ seconds the SLO must stay met before densifying or capping again
    ↪ ")
    ap.add_argument("--thermal-model", action="store_true", help="heat
    ↪  muscle: thermal state from the harness heat law")
    ap.add_argument("--security-configmap", default="", help="ns/name
    ↪ of a ConfigMap whose key 'hold' signals a security hold")
    ap.add_argument("--rollout-guard", default="", help="ns/name[,ns/n
    ↪ ame]: rollout muscle pauses, resumes, undoes")
    ap.add_argument("--batch", action="store_true", help="batch muscle
    ↪ : admit held Jobs labelled " + BATCH_LABEL)
    ap.add_argument("--batch-load-max", type=float, default=0.9,
                    help="admit held work only under this load: the co
    ↪ mpass's wall less one cushion (0.95 - 0.05)")
    ap.add_argument("--batch-power-max", type=float, default=0.9,
                    help="admit held work only under this power stress
    ↪ : the wall less one cushion, as the nervous system")
    ap.add_argument("--rapl-cmd", default="", help="cpu_pstate pull: p
    ↪ rints CPU package watts")
    ap.add_argument("--cpufreq-cmd", default="", help="cpu_pstate push
    ↪ : command template with {khz}")
    ap.add_argument("--cpufreq-policy-root", default="", help="cpu_pst
    ↪ ate push through sysfs policies (e.g. /sys/devices/system/cpu/cp
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ufreq); exact restore on kill")
    ap.add_argument("--cpufreq-require-schedutil", action="store_true"
    ↪ , help="refuse to act unless every policy runs schedutil")
    ap.add_argument("--cpu-max-khz", type=float, default=0.0)
    ap.add_argument("--gpu-query-cmd", default="", help="gpu pull: pri
    ↪ nts 'watts,celsius'")

    ap.add_argument("--gpu-power-cmd", default="", help="gpu push: com
    ↪ mand template with {w}")
    ap.add_argument("--gpu-max-w", type=float, default=0.0)
    ap.add_argument("--gpu-temp-limit", type=float, default=83.0,
                    help="GPU temperature read as heat 1.0 when the ca
    ↪ rd does not report its own slowdown temperature "

                         "(--gpu-query-cmd's fourth field; then one de
    ↪ gree under it)")
    ap.add_argument("--rightsize-deployments", default="", help="ns/na
    ↪ me[,ns/name]: rightsize muscle sets pod CPU requests from use")
    ap.add_argument("--rightsize-headroom", type=float, default=0.3)
    ap.add_argument("--rightsize-min-m", type=int, default=50)

    ap.add_argument("--coldstart-deployments", default="", help="ns/na
    ↪ me[,ns/name]: scaled to zero when idle, woken on work")
    ap.add_argument("--coldstart-signal", default="", help="ns/name of
    ↪  a ConfigMap whose key 'queue' is the waiting work")
    ap.add_argument("--coldstart-idle", type=int, default=None, help="
    ↪ idle decisions before scaling to zero (unset: --coldstart-idle-s

    ↪ )")
    ap.add_argument("--coldstart-idle-s", type=float, default=180.0, h
    ↪ elp="idle seconds before scaling to zero (omni-v16)")
    ap.add_argument("--batch-pace", action="store_true", help="batch_p
    ↪ ace muscle: pause Jobs labelled " + PAUSABLE_LABEL + " under str
    ↪ ess")

    ap.add_argument("--pace-high", type=float, default=0.95, help="pow
    ↪ er stress that pauses a job")
    ap.add_argument("--pace-low", type=float, default=0.8, help="power
    ↪  stress under which a paused job resumes")
    ap.add_argument("--pace-heat", type=float, default=0.96, help="hea
    ↪ t state that pauses a job")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ap.add_argument("--contain-namespaces", default="", help="ns[,ns]:
    ↪  agent namespaces held to --contain-cpu-m")
    ap.add_argument("--contain-cpu-m", type=float, default=1000.0)
    ap.add_argument("--cooling-cmd", default="", help="cooling push: c
    ↪ ommand template with {c} (supply-air setpoint, C)")
    ap.add_argument("--cooling-min-c", type=float, default=18.0)

    ap.add_argument("--cooling-max-c", type=float, default=27.0)
    ap.add_argument("--cooling-restore-c", type=float, default=22.0)
```

H.7 omni_controller/quota.py

119 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Conservative CPU-limit admission budget; Kubernetes remains the fin
    ↪ al authority."""
from decimal import Decimal, InvalidOperation
import re

def cpu_milli(value):
    match = re.fullmatch(r'([+\-]?(?:\d+(?:\.\d*)?|\.\d+)(?:[eE][+\-]?
    ↪ \d+)?)([numkMGTPE]?)', str(value))
    if not match:

        raise ValueError('invalid CPU quantity')
    factor = {'n': '0.000001', 'u': '0.001', 'm': '1', '': '1000', 'k'
    ↪ : '1000000',
              'M': '1e9', 'G': '1e12', 'T': '1e15', 'P': '1e18', 'E':
    ↪ '1e21'}
    result = Decimal(match[1]) * Decimal(factor[match[2]])
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if not result.is_finite() or result < 0:
        raise ValueError('invalid CPU quantity')
    return result


def matches(quota, pod):
    """True/False for PriorityClass selectors; None means applicabilit
    ↪ y is unknown, so expansion stops."""
    spec = quota.get('spec', {})
    if spec.get('scopes'):
        return None
    expressions = spec.get('scopeSelector', {}).get('matchExpressions'
    ↪ , [])
    unknown = False
    for expr in expressions:
        if expr.get('scopeName') != 'PriorityClass':
            unknown = True
            continue
        value = pod.get('spec', {}).get('priorityClassName', '')
        operator, values = expr.get('operator'), expr.get('values', []
    ↪ )
        if operator == 'In':
            selected = value in values
        elif operator == 'NotIn':
            selected = value not in values
        elif operator == 'Exists':
            selected = bool(value)
        elif operator == 'DoesNotExist':
            selected = not value
        else:
            unknown = True
            continue
        if not selected:
            return False
    return None if unknown else True
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class CpuLimitBudget:
    """One convey pass: share each namespace's headroom across all tar
    ↪ get deployments.

    Never credit a decrease before the server reports it. Reserve succ
    ↪ essful increases locally
    so an eventually updated quota status cannot spend the same allowa
    ↪ nce twice in this pass.
    A quota whose status or scopes cannot be read as a number leaves t
    ↪ he native CPU limit in place.

    A namespace whose quotas cannot be read at all (the governor's rol
    ↪ e has no list permission, or the API refused) is
    **no budget known**, not a zero budget: the raise proceeds as it d
    ↪ id before the budget existed (omni-v6), and
    Kubernetes' own admission, which refuses any write over a quota th
    ↪ e governor cannot see, stays the authority; the

    plug reads every write back and a refused one is audited and never
    ↪  advertised. The first v7 trial (Goldilocks, run
    38825214203) read the unreadable quota as zero and so never raised
    ↪  a limit: 5 commands a run against v6's 15, the
    host 4% busier against 40%, the response-time GAIN gone and the po
    ↪ d start wait a LOSS (docs/OMNI_V7.md, section 1).

    """
    def __init__(self, kube, audit):
        self.kube, self.audit, self.namespaces, self.unreadable = kube
    ↪ , audit, {}, set()

    def _load(self, namespace):

        if namespace in self.namespaces:
            return self.namespaces[namespace]
        try:
            quotas = self.kube.get('get', 'resourcequota', '-n', names
    ↪ pace, '-o', 'json')['items']
            budgets = []
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            for quota in quotas:
                spec = quota.get('spec', {}).get('hard', {})
                status = quota.get('status', {})
                hard = status.get('hard', {})
                if 'limits.cpu' not in spec and 'limits.cpu' not in ha
    ↪ rd:
                    continue
                try:
                    ceiling = min(cpu_milli(x['limits.cpu']) for x in
    ↪ (spec, hard))
                    remaining = max(Decimal(0), ceiling - cpu_milli(st
    ↪ atus['used']['limits.cpu']))
                except (KeyError, ValueError, InvalidOperation):
                    remaining = Decimal(0)
                budgets.append([quota, remaining])
        except Exception as error:
            if namespace not in self.unreadable:                    #
    ↪ said once a pass, not once a pod
                self.audit({'quota': 'no budget known: the raise proce
    ↪ eds, Kubernetes admission is the authority',
                            'namespace': namespace, 'reason': f'quota
    ↪ read unavailable: {type(error).__name__}'})
            self.unreadable.add(namespace)
            budgets = []
        self.namespaces[namespace] = budgets
        return budgets

    def limit(self, namespace, pod, current, desired):
        current, desired = Decimal(str(current)), Decimal(str(desired)
    ↪ )
        if desired <= current:
            return int(desired), []
        budgets = self._load(namespace)
        applicable = []
        allowance = desired - current
        for entry in budgets:                           # an unreadable na
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ mespace has no entries: the allowance stands
            selected = matches(entry[0], pod)
            if selected is False:
                continue
            applicable.append(entry)
            allowance = min(allowance, entry[1] if selected is True el
    ↪ se Decimal(0))
        permitted = int(current + allowance)
        if permitted < desired:
            self.audit({'quota': 'CPU limit expansion bounded', 'names
    ↪ pace': namespace,
                        'pod': pod['metadata']['name'], 'desired_m': i
    ↪ nt(desired), 'permitted_m': permitted})
        return permitted, applicable

    @staticmethod
    def reserve(entries, current, accepted):
        delta = max(Decimal(0), Decimal(str(accepted)) - Decimal(str(c
    ↪ urrent)))
        for entry in entries:
            entry[1] = max(Decimal(0), entry[1] - delta)
```

H.8 `omni_orchestrator/__init__.py`

28 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass's own orchestrator: the organism run without Kubernete
    ↪ s (the founder's order of 23 October 2026).

Written from scratch on Python's standard library and Omni-Compass's o
    ↪ wn compass law (omnicompass/compass_law.py); no
code of Kubernetes, Docker Swarm, Nomad, Slurm or any other orchestrat
    ↪ or is in it. The layers under it stay native: the
Linux kernel shares each machine's processor among its units and enfor
    ↪ ces the walls set here.

  cgroup.py     a machine's and a unit's walls: CPU and memory limits
    ↪ through the kernel's control groups
  agent.py      a machine's hand, one process per machine: starts, wat
    ↪ ches, restarts and stops the units on its
                machine, inside its walls, and answers the control pla
    ↪ ne over HTTP

  workload.py   the showing's organism: a service whose every answer c
    ↪ an be checked, and batch tasks
  front.py      one address for a service: requests queued and handed
    ↪ to ready units one at a time each, retried on
                another unit when one dies; the response time it measu
    ↪ res is the brain's sense

  scheduler.py  where a unit goes: the room on each machine, a service
    ↪  never on one machine alone, packed onto the
                fewest machines (Omni) or spread over all (the yardsti
    ↪ ck)
  brain.py      how many units and how many machines: the compass law
    ↪ on the response time, never below its native

                twin (the yardstick's rule computed on the same readin
    ↪ gs); the yardstick's rule itself beside it
  control.py    the control plane: the ledger of what is wanted and wh
    ↪ at runs, journaled and replayed after a crash;
                the checks every second; the brain on its beat; the ki
    ↪ ll switch; the routes; the audit
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
  load.py       closed-loop clients stepping by one, every answer chec
    ↪ ked against its known digest
"""

# © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Sim
    ↪ ulation use only; any commercialization or

# monetization requires a signed, paid Omni-Compass Enterprise License
    ↪ . All patents, copyrights and trademarks filed in the USA. www.o
    ↪ mni-compass.com
```

H.9 omni_orchestrator/agent.py

273 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""A machine's hand: one process per machine, the only thing that star
    ↪ ts or stops a unit on it.

It holds the machine's walls (omni/<machine>: its CPU quota and memory

    ↪ ), starts each unit in walls of its own under them
and on the machine's cores, and watches every unit twice a second: a s
    ↪ ervice is probed on its health path (ready at its
first answer; killed and started again after three missed answers, or
    ↪ when it never answers in its first 30 s); a unit
whose process ends is started again after a wait that doubles from 1 s
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  to 30 s; a batch task that ends with 0 is done,
and one that fails three times is failed. The units keep running whate
    ↪ ver happens to the control plane: the hand needs
no one to keep them alive.

HTTP on 127.0.0.1:<port>:

  GET  /status          the machine, its walls, the processor it used,
    ↪  and every unit with its state
  POST /units           start a unit: {"id", "kind": "service"|"task",
    ↪  "cmd": [...], "port", "limit_m", "mem_mb",
                        "health", "env"}
  POST /units/<id>/limit  move the unit's CPU wall: {"limit_m"}; answe
    ↪ rs with the wall as the kernel now holds it
                        (the control plane's conveyance: a unit given
    ↪ the processor its machine can spare, or handed back)
  DELETE /units/<id>    stop a unit: SIGTERM to its process group, SIG
    ↪ KILL after 3 s, its walls removed
  POST /stop_all        stop every unit
"""
from __future__ import annotations

import argparse
import json
import os

import signal
import subprocess
import threading
import time
import urllib.request
from concurrent.futures import ThreadPoolExecutor

from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path

from omni_orchestrator.cgroup import Walls
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def probe(port: int, path: str, timeout: float = 1.0) -> bool:
    try:
        with urllib.request.urlopen(f"http://127.0.0.1:{port}{path}",
    ↪ timeout=timeout) as r:
            return r.status == 200
    except Exception:
        return False


class Unit:
    def __init__(self, spec: dict):
        self.spec = spec
        self.id = spec["id"]
        self.kind = spec.get("kind", "service")
        self.proc: subprocess.Popen | None = None
        self.state = "starting"
        self.ready = False
        self.restarts = 0
        self.fails = 0
        self.exit_code = None
        self.started = 0.0
        self.not_before = 0.0
        self.backoff = 1.0
        self.want = True

class Agent:
    def __init__(self, name: str, cores: set[int], cpu_m: float, mem_m
    ↪ b: float, state_dir: str):
        self.name, self.cores, self.cpu_m, self.mem_mb = name, cores,
    ↪ cpu_m, mem_mb
        self.dir = Path(state_dir)
        self.dir.mkdir(parents=True, exist_ok=True)
        self.walls = Walls(name)
        for p in self.walls.pids():                      # a restarted
    ↪  hand finds its machine empty: what a lost hand
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            try:                                            # left behind
↪  is gone with it
                os.kill(p, signal.SIGKILL)
            except OSError:
                pass
        self.walls.remove()

        self.walled = self.walls.make(cpu_m, mem_mb)
        self.units: dict[str, Unit] = {}
        self.lock = threading.Lock()
        self.pool = ThreadPoolExecutor(8)

    # ---- the units ------------------------------------------------

↪ ----------------------------------------
    def start_unit(self, spec: dict) -> None:
        with self.lock:
            if spec["id"] in self.units:
                return
            u = Unit(spec)

            self.units[u.id] = u
        Walls(self.name, u.id).make(spec.get("limit_m"), spec.get("mem
↪ _mb"))
        self._spawn(u)

    def _spawn(self, u: Unit) -> None:

        env = {**os.environ, "OMNI_UNIT": u.id, "OMNI_MACHINE": self.n
↪ ame,
               **{k: str(v) for k, v in (u.spec.get("env") or {}).item
↪ s()}}
        with open(self.dir / f"{u.id}.log", "ab") as log:
            u.proc = subprocess.Popen(u.spec["cmd"], stdout=log, stder

↪ r=subprocess.STDOUT, env=env,
                                      cwd=u.spec.get("cwd"), start_new
↪ _session=True)
        Walls(self.name, u.id).join(u.proc.pid)          # inside its
↪ walls before it has done any work
        if self.cores:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            try:
                os.sched_setaffinity(u.proc.pid, self.cores)
            except OSError:
                pass
        u.started, u.state, u.ready, u.fails = time.time(), "starting"
    ↪ , False, 0

    def _kill(self, u: Unit, grace: float) -> None:
        p = u.proc
        if p is None:
            return
        try:
            os.killpg(p.pid, signal.SIGTERM)
        except OSError:
            pass
        try:
            p.wait(grace)
        except subprocess.TimeoutExpired:
            try:
                os.killpg(p.pid, signal.SIGKILL)
            except OSError:
                pass
            p.wait(5)

    def stop_unit(self, uid: str) -> bool:
        with self.lock:
            u = self.units.pop(uid, None)
        if u is None:
            return False
        u.want = False
        self._kill(u, 3.0)
        Walls(self.name, uid).remove()
        return True

    def stop_all(self) -> int:
        with self.lock:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            ids = list(self.units)
        for uid in ids:
            self.stop_unit(uid)
        return len(ids)

    def set_limit(self, uid: str, limit_m: float) -> float | None:

        """Move a unit's CPU wall and read back what the kernel holds;
    ↪  None when the unit is not here."""
        with self.lock:
            u = self.units.get(uid)
            if u is None:
                return None

            u.spec["limit_m"] = limit_m                  # a unit star
    ↪ ted again keeps the wall it was given
        w = Walls(self.name, uid)
        w.set_cpu(limit_m)
        return w.cpu_limit_m()

    # ---- the watch ------------------------------------------------
    ↪ ------------------------------------------
    def supervise(self) -> None:
        while True:
            now = time.time()
            with self.lock:

                units = [u for u in self.units.values() if u.want]
            probes = {}
            for u in units:
                if u.proc is None:
                    if now >= u.not_before:
                        self._spawn(u)

                    continue
                code = u.proc.poll()
                if code is not None:
                    u.exit_code, u.ready, u.proc = code, False, None
                    if u.kind == "task" and code == 0:
                        u.state, u.want = "succeeded", False
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                    elif u.kind == "task" and u.restarts >= 2:
                        u.state, u.want = "failed", False
                    else:
                        u.restarts += 1
                        u.state = "crashed"
                        u.not_before = now + u.backoff

                        u.backoff = min(30.0, u.backoff * 2)
                    continue
                if u.kind == "task":
                    u.state = "running"
                elif u.spec.get("health"):
                    probes[u.id] = self.pool.submit(probe, u.spec["por
    ↪ t"], u.spec["health"])
            for u in units:
                f = probes.get(u.id)
                if f is None:
                    continue
                if f.result():

                    u.ready, u.fails, u.state = True, 0, "running"
                    if now - u.started > 60:
                        u.backoff = 1.0                 # a unit that
    ↪  has held for a minute starts its waits afresh
                else:
                    u.fails += 1

                    if (u.ready and u.fails >= 3) or (not u.ready and
    ↪ now - u.started > 30):
                        u.ready, u.state = False, "unhealthy"
                        self._kill(u, 0.5)              # the next wa
    ↪ tch sees it end and starts it again
            time.sleep(0.5)

    def status(self) -> dict:
        with self.lock:
            units = list(self.units.values())
        return {"machine": self.name, "cpu_m": self.cpu_m, "mem_mb": s
    ↪ elf.mem_mb, "walled": self.walled,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                "cores": sorted(self.cores), "cpu_s": self.walls.cpu_s
    ↪ (), "time": time.time(),
                "units": [{"id": u.id, "kind": u.kind, "state": u.stat
    ↪ e, "ready": u.ready, "restarts": u.restarts,
                           "pid": u.proc.pid if u.proc else None, "por
    ↪ t": u.spec.get("port"),

                           "exit_code": u.exit_code, "cpu_s": Walls(se
    ↪ lf.name, u.id).cpu_s(),
                           "limit_m": Walls(self.name, u.id).cpu_limit
    ↪ _m()} for u in units]}

def serve(agent: Agent, port: int) -> None:
    class Handler(BaseHTTPRequestHandler):
        def _send(self, code: int, obj) -> None:
            body = json.dumps(obj).encode()
            self.send_response(code)
            self.send_header("Content-Type", "application/json")

            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)

        def _body(self) -> dict:
            n = int(self.headers.get("Content-Length") or 0)

            return json.loads(self.rfile.read(n) or b"{}")

        def do_GET(self):
            if self.path == "/status":
                self._send(200, agent.status())
            else:

                self._send(404, {"error": "no such path"})

        def do_POST(self):
            if self.path == "/units":
                agent.start_unit(self._body())
                self._send(200, {"ok": True})
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            elif self.path.startswith("/units/") and self.path.endswit
    ↪ h("/limit"):
                got = agent.set_limit(self.path.split("/")[2], float(s
    ↪ elf._body()["limit_m"]))
                self._send(200 if got is not None else 404, {"limit_m"
    ↪ : got})

            elif self.path == "/stop_all":
                self._send(200, {"stopped": agent.stop_all()})
            else:
                self._send(404, {"error": "no such path"})

        def do_DELETE(self):

            if self.path.startswith("/units/"):
                self._send(200, {"stopped": agent.stop_unit(self.path.
    ↪ split("/", 2)[2])})
            else:
                self._send(404, {"error": "no such path"})

        def log_message(self, *a):
            pass

    srv = ThreadingHTTPServer(("127.0.0.1", port), Handler)
    srv.daemon_threads = True
    srv.serve_forever(poll_interval=0.2)

def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__.split("\n\n")[0])
    ap.add_argument("--name", required=True)
    ap.add_argument("--port", type=int, required=True)

    ap.add_argument("--cores", default="", help="the machine's cores,
    ↪ e.g. 0 or 0,1 (empty: any)")
    ap.add_argument("--cpu-m", type=float, required=True, help="the ma
    ↪ chine's processor, millicores")
    ap.add_argument("--mem-mb", type=float, default=2048)
    ap.add_argument("--state-dir", required=True)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    a = ap.parse_args(argv)
    cores = {int(c) for c in a.cores.split(",") if c.strip()}
    agent = Agent(a.name, cores, a.cpu_m, a.mem_mb, a.state_dir)
    threading.Thread(target=agent.supervise, daemon=True).start()
    signal.signal(signal.SIGTERM, lambda *_: (agent.stop_all(), agent.
    ↪ walls.remove(), os._exit(0)))

    serve(agent, a.port)


if __name__ == "__main__":
    main()

# © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Sim
    ↪ ulation use only; any commercialization or
# monetization requires a signed, paid Omni-Compass Enterprise License
    ↪ . All patents, copyrights and trademarks filed in the USA. www.o
    ↪ mni-compass.com
```

H.10 `omni_orchestrator/brain.py`

134 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""How many units, how many machines, and how much processor each unit
    ↪  may use.

Yardstick   the Kubernetes Horizontal Pod Autoscaler's published rule,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  written here from its documentation as the
         measure Omni is read against (it is not Kubernetes' code):
    ↪  every 15 s the units become
         ceil(units × busy ÷ target), busy being the processor the
    ↪ units used over their requests (a pod never uses
         more than its limit, here its request, so busy is read cap

    ↪ ped at the request), target 50%; no move inside a
         10% tolerance; a rise at most to max(2 × units, units + 4)
    ↪  at once; a fall only to the highest count it
         recommended over the last 300 s.

OmniBrain   Omni-Compass's own, the max riding the edge of the organis

    ↪ m:
         the reflex    work waiting in line at the front (every han
    ↪ d busy) adds units now, one for each request
                       waiting, up to the step; a request in line i
    ↪ s the organism's own signal, read in the beat;
         the compass   the compass law (omnicompass/compass_law.py)

    ↪  on the service's 95th percentile response time in
                       the band from 0 to the service's line: a for
    ↪ ce up adds units at once in proportion to it (past
                       the wall, the most it may add); a force down
    ↪  gives back one unit, only after it has held down
                       for three beats with no request waiting;

         the twin      the yardstick's rule computed on the same re
    ↪ adings, what Kubernetes would hold for this work:
                       Omni holds at least that, less only the dept
    ↪ h the verdict has allowed (omnicompass/verdict.py,
                       the engine's own trigger): one unit fewer th
    ↪ an the twin is tried on the service itself and kept

                       only where the response time measured under
    ↪ it reads no LOSS against the count before, within
                       the 2% allowance; a trial walks to its step
    ↪ one unit a beat, stands there while it measures,
                       and is abandoned the moment work waits or th
    ↪ e wall is reached. Work waiting in line hands one
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                            allowed step back (the immune reaction: harm
    ↪  seen, the knob steps toward native).

conveyance  the processor each unit may use (its CPU wall), the engine
    ↪ 's own rule (omni_controller/muscles.py, omni-v7
           and omni-v13): the room under 75% of its machine, less the
    ↪  other workloads' units on it counted by their
           limits, shared equally among the service's units there, ne
    ↪ ver under a unit's request and never over the
           machine; no new raise while the machine is 75% busy or its
    ↪  busy share is unread; at the top beside another
           workload's room every limit goes back to the request, conv
    ↪ eyed again under 70%. A unit whose wall stands
           above its request answers at its machine's own speed inste
    ↪ ad of its paper quota's: faster, with no unit and
           no machine added. Kubernetes' autoscaler never moves a lim
    ↪ it; it adds pods.

machines    how many machines the units need: their requests, plus roo
    ↪ m for the most units the brain may add at once,
           so a unit never waits for a machine; never fewer than the
    ↪ floor (two).
"""
from __future__ import annotations

import math

from omnicompass.compass_law import Band, CompassLaw

CONVEY_TOP, CONVEY_BACK = 0.75, 0.70     # the engine's own lines on a
    ↪  machine's busy share (omni-v7, omni-v13)


def clamp(v, lo, hi):
    return lo if v < lo else hi if v > hi else v
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class Yardstick:
    def __init__(self, mn: int, mx: int, target: float = 0.5, toleranc
    ↪ e: float = 0.1, window_s: float = 300.0):
        self.mn, self.mx, self.target, self.tolerance, self.window_s =
    ↪  mn, mx, target, tolerance, window_s

        self.hist: list[tuple[float, int]] = []

    def recommend(self, now: float, units: int, busy: float | None) ->
    ↪  int:
        if busy is None or units <= 0:
            rec = max(units, self.mn)

        else:
            ratio = busy / self.target
            rec = units if abs(ratio - 1.0) <= self.tolerance else mat
    ↪ h.ceil(units * ratio)
        rec = clamp(rec, self.mn, self.mx)
        self.hist = [(t, r) for t, r in self.hist if now - t <= self.w

    ↪ indow_s] + [(now, rec)]
        if rec < units:
            rec = max(r for _, r in self.hist)           # a fall only
    ↪  to the highest of the window
        if rec > units:
            rec = min(rec, max(2 * units, units + 4))

        return clamp(rec, self.mn, self.mx)


class OmniBrain:
    def __init__(self, line_ms: float, beat_s: float, mn: int, mx: int
    ↪ , max_step: int = 3, hold: int = 3):

        self.law = CompassLaw(Band(0.0, line_ms), dt=beat_s, tau=2.0 *
    ↪  beat_s)
        self.twin = Yardstick(mn, mx)
        self.mn, self.mx, self.max_step, self.hold = mn, mx, max_step,
    ↪  hold
        self.calm = 0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def decide(self, now: float, units: int, busy: float | None, p95_m
    ↪ s: float, waiting: int,
                depth: int = 0, trial: bool = False, twin: int | None =
    ↪  None, anchor: int | None = None) -> tuple[int, dict]:
        """One beat. depth: how many units below the twin the verdict

    ↪ allows (a trial's own step included); trial: a
        verdict phase is on, so the count walks to that depth one unit
    ↪  a beat and stands there while it measures.
        anchor: the twin the depth is counted from during a phase (the
    ↪  twin as it stood when the phase began, so the
        step does not follow the count down: inside its 10% tolerance

    ↪ the yardstick's rule holds whatever stands)."""
        if twin is None:
            twin = self.twin.recommend(now, units, busy)
        # during a phase the step is counted from the higher of the tw
    ↪ in as the phase found it and the twin now: the floor
        # never follows the count down (the twin's tolerance would acc

    ↪ ept the cut) and never stays below a twin the
        # traffic has raised since the phase began
        floor = max(self.mn, (twin if anchor is None else max(anchor,
    ↪ twin)) - max(0, int(depth)))
        f = self.law.force(p95_ms)
        reflex = min(self.max_step, int(waiting)) if waiting > 0 else

    ↪ 0
        if reflex or f > 0.05:
            self.calm = 0
            want = units + max(reflex, round(f * self.max_step) if f >
    ↪  0.05 else 0, 1)
        elif trial:

            self.calm = 0
            want = max(floor, units - 1)                # a trial wal
    ↪ ks to its step one unit a beat and stands there
        elif f < -0.05:
            self.calm += 1
            want = units - 1 if self.calm >= self.hold else units
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        else:
            self.calm = 0
            want = units
        want = clamp(max(want, floor), self.mn, self.mx)
        return want, {"force": round(f, 3), "position": round(self.law
    ↪ .p or 0.0, 3), "twin": twin, "floor": floor,

                      "depth": int(depth), "trial": bool(trial), "p95_
    ↪ ms": round(p95_ms, 1),
                      "busy": None if busy is None else round(busy, 3)
    ↪ , "waiting": waiting}

def conveyed_limits(cpu_m: float, busy_m: float | None, ours: list[dic
    ↪ t], others_m: float,
                    others_room: bool = False) -> dict[str, float] | N
    ↪ one:
    """The CPU wall (millicores) each of the service's units on one ma
    ↪ chine may have, by the engine's rule: the room

    under 75% of the machine, less the other workloads' units counted
    ↪ by their limits (others_m), shared equally among
    ours; never under a unit's request, never over the machine, in ste
    ↪ ps of 10 m. None: nothing changes (the machine's
    busy share unread, or between 70% and 75%, where what stands is he
    ↪ ld). At 75% or over: no new raise; and beside

    another workload's room (others_room) every wall goes back to the
    ↪ request."""
    if busy_m is None or not ours:
        return None
    if busy_m >= CONVEY_TOP * cpu_m:
        return {u["id"]: float(u["req_m"]) for u in ours} if others_ro

    ↪ om else None
    if busy_m >= CONVEY_BACK * cpu_m:
        return None
    share = (CONVEY_TOP * cpu_m - others_m) / len(ours)
    return {u["id"]: float(min(cpu_m, max(u["req_m"], math.floor(share
    ↪  / 10.0) * 10.0))) for u in ours}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def machines_needed(units: int, req_m: float, room_m: float, step_unit
    ↪ s: int, floor: int, extra_m: float = 0.0) -> int:
    return max(floor, math.ceil((units * req_m + step_units * req_m +
    ↪ extra_m) / room_m - 1e-9))

# © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Sim
    ↪ ulation use only; any commercialization or
# monetization requires a signed, paid Omni-Compass Enterprise License
    ↪ . All patents, copyrights and trademarks filed in the USA. www.o
    ↪ mni-compass.com
```

H.11 omni_orchestrator/cgroup.py

145 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""A machine's and a unit's walls: the kernel's own control groups, ve
    ↪ rsion 2 (the unified hierarchy) or version 1.

A group is omni/<machine> for a machine and omni/<machine>/<unit> for
    ↪ a unit. A machine's CPU quota caps everything in
it; a unit's quota caps the unit; the kernel's own counter gives the p
    ↪ rocessor time each one used. Version 2 (cgroup.controllers
at the root, the default on modern kernels and on GitHub's runners): c
    ↪ pu.max, memory.max, cpu.stat, and the cpu and memory
controllers handed down to each level through cgroup.subtree_control;
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ processes live only in the leaves (the units). Version 1
(the cpu, cpuacct and memory hierarchies mounted apart): cpu.cfs_quota
    ↪ _us, memory.limit_in_bytes, cpuacct.usage. Where the
groups cannot be written, nothing is walled and `Walls.make` says so (
    ↪ the caller records it)."""
from __future__ import annotations

import os
from pathlib import Path

ROOT = Path(os.environ.get("OMNI_CGROUP_ROOT", "/sys/fs/cgroup"))
HIER = ("cpu", "cpuacct", "memory")

PERIOD_US = 100_000


def v2() -> bool:
    return (ROOT / "cgroup.controllers").exists()

def _write(p: Path, text: str) -> bool:
    try:
        p.write_text(text)
        return True
    except OSError:

        return False


class Walls:
    def __init__(self, *parts: str):
        self.parts = ("omni", *parts)

    def _dir(self, h: str = "") -> Path:
        return ROOT.joinpath(h, *self.parts) if h else ROOT.joinpath(*
    ↪ self.parts)

    # ---- make ----------------------------------------------------
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ---------------------------------------------
    def make(self, cpu_m: float | None = None, mem_mb: float | None =
    ↪ None) -> bool:
        """Create the group; set its CPU quota (millicores) and memory
    ↪  limit. True when the CPU wall is in place."""
        return self._make_v2(cpu_m, mem_mb) if v2() else self._make_v1
    ↪ (cpu_m, mem_mb)

    def _make_v2(self, cpu_m, mem_mb) -> bool:
        try:
            d = ROOT
            for part in self.parts:                       # every level
    ↪  above the leaf hands the controllers down
                d = d / part
                d.mkdir(exist_ok=True)
                _write(d.parent / "cgroup.subtree_control", "+cpu +mem
    ↪ ory")
            if cpu_m:
                ok = _write(d / "cpu.max", f"{int(cpu_m * PERIOD_US /
    ↪ 1000)} {PERIOD_US}")
                if not ok:
                    return False
            if mem_mb:
                _write(d / "memory.max", str(int(mem_mb) * 1024 * 1024
    ↪ ))     # a limit the kernel refuses: the CPU wall stands
            return True
        except OSError:
            return False

    def _make_v1(self, cpu_m, mem_mb) -> bool:
        try:
            for h in HIER:
                if (ROOT / h).is_dir():
                    self._dir(h).mkdir(parents=True, exist_ok=True)
            if cpu_m:
                (self._dir("cpu") / "cpu.cfs_period_us").write_text(st
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ r(PERIOD_US))
                (self._dir("cpu") / "cpu.cfs_quota_us").write_text(str
    ↪ (int(cpu_m * PERIOD_US / 1000)))
            if mem_mb and (ROOT / "memory").is_dir():
                _write(self._dir("memory") / "memory.limit_in_bytes",
    ↪ str(int(mem_mb) * 1024 * 1024))

            return True
        except OSError:
            return False

    def set_cpu(self, cpu_m: float) -> bool:
        """Move the group's CPU wall to cpu_m millicores (the quota th

    ↪ e kernel enforces); True when the kernel took it."""
        q = str(int(cpu_m * PERIOD_US / 1000))
        if v2():
            return _write(self._dir() / "cpu.max", f"{q} {PERIOD_US}")
        return _write(self._dir("cpu") / "cpu.cfs_quota_us", q)

    def cpu_limit_m(self) -> float | None:
        """The group's CPU wall as it stands, in millicores (None: no
    ↪ wall, or none readable)."""
        try:
            if v2():
                q, p = (self._dir() / "cpu.max").read_text().split()

                return None if q == "max" else int(q) * 1000.0 / int(p
    ↪ )
            q = int((self._dir("cpu") / "cpu.cfs_quota_us").read_text(
    ↪ ))
            p = int((self._dir("cpu") / "cpu.cfs_period_us").read_text
    ↪ ())

            return None if q < 0 else q * 1000.0 / p
        except (OSError, ValueError):
            return None

    # ---- join, read, list, remove ------------------------------------
    ↪ ------------------------------------------------
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def join(self, pid: int) -> None:
        if v2():
            _write(self._dir() / "cgroup.procs", str(pid))
            return
        for h in HIER:
            f = self._dir(h) / "cgroup.procs"

            if f.exists():
                _write(f, str(pid))

    def cpu_s(self) -> float:
        """The processor time used by everything in the group so far,
    ↪ in seconds."""

        try:
            if v2():
                for line in (self._dir() / "cpu.stat").read_text().spl
    ↪ itlines():
                    if line.startswith("usage_usec"):
                        return int(line.split()[1]) / 1e6

                return 0.0
            return int((self._dir("cpuacct") / "cpuacct.usage").read_t
    ↪ ext()) / 1e9
        except (OSError, ValueError):
            return 0.0

    def pids(self) -> set[int]:
        """Every process in the group and the groups under it."""
        out: set[int] = set()
        base = self._dir() if v2() else self._dir("cpu")
        if not base.is_dir():
            return out

        for d, _, _ in os.walk(base):
            try:
                out |= {int(x) for x in (Path(d) / "cgroup.procs").rea
    ↪ d_text().split()}
            except (OSError, ValueError):
                pass
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return out

    def remove(self) -> None:
        """Remove the group and the groups under it, deepest first (a
    ↪ group with a process left in it stays)."""
        for base in ([self._dir()] if v2() else [self._dir(h) for h in
    ↪  HIER]):
            if not base.is_dir():
                continue
            for d, _, _ in sorted(os.walk(base), key=lambda w: -len(w[
    ↪ 0])):
                try:
                    os.rmdir(d)
                except OSError:
                    pass

# © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Sim
    ↪ ulation use only; any commercialization or
# monetization requires a signed, paid Omni-Compass Enterprise License
    ↪ . All patents, copyrights and trademarks filed in the USA. www.o
    ↪ mni-compass.com
```

H.12 omni_orchestrator/control.py

629 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The control plane: one process that decides; the hands on the machi
    ↪ nes and the front do the rest.

The ledger is what is wanted (units per service, the state of every ma
    ↪ chine, batch tasks done) and what was placed
(every unit: its machine, its port). Every change to it is a line in t
    ↪ he journal before it is acted on, so a control
plane that is killed and started again replays the journal and finds t
    ↪ he same ledger; the units it finds running on the
machines are taken back as they are, none started again. Every second
    ↪ it:

  1. asks every machine's hand for its status; a machine silent three
    ↪ times running is down, and its units are placed
     again elsewhere; a unit a machine runs that the ledger does not k
    ↪ now is stopped;
  2. stops the units retiring once their drain is over (out of the rou
    ↪ tes first, so no request is lost);
  3. on the brain's beat, decides the units of each service (brain.py:
    ↪  the reflex on work in line, the compass law on the
     95th percentile, the yardstick twin less the depth the verdict ha
    ↪ s allowed on the service's own response times) and,
     for Omni, the machines (wake before any unit would wait, park one
    ↪  at a time after the load has held low for three
     beats, its units moved first) and each unit's CPU wall (the engin
    ↪ e's conveyance: the room its machine can still
     give under 75%, journaled, written to the hand and read back);
  4. places the units the ledger lacks (the scheduler's room rule), re
    ↪ tires the extra ones, and lets batch tasks into
     the room the services leave (a task yields its room the moment a
    ↪ service needs it);
  5. writes the routes for the front (only ready units) and one line o
    ↪ f the timeline.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
The kill switch is a file: while it exists the brain is off and the de
    ↪ clared state is held (the declared units, every
machine on, every wall at its request). Every decision is a line in th
    ↪ e audit with its reason.
"""
from __future__ import annotations

import argparse
import json
import os
import signal
import sys

import time
import urllib.request
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

from omni_orchestrator.brain import OmniBrain, Yardstick, conveyed_lim

    ↪ its, machines_needed
from omni_orchestrator.scheduler import BUSY_TOP, place
from omnicompass.verdict import Verdict

ROOT = str(Path(__file__).resolve().parents[1])

def http(method: str, url: str, body: dict | None = None, timeout: flo
    ↪ at = 5.0):
    data = json.dumps(body).encode() if body is not None else None
    req = urllib.request.Request(url, data=data, method=method, header
    ↪ s={"Content-Type": "application/json"})

    with urllib.request.urlopen(req, timeout=timeout) as r:
        return json.loads(r.read() or b"{}")


class Control:
    def __init__(self, spec: dict, state_dir: str, brain: str, beat: f
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ loat, kill_file: str | None = None,
                room_rule: str = "requests"):
        self.spec, self.brain_kind, self.beat, self.room_rule = spec,
    ↪ brain, beat, room_rule
        self.dir = Path(state_dir)
        self.dir.mkdir(parents=True, exist_ok=True)

        self.journal_path, self.audit_path = self.dir / "journal.jsonl
    ↪ ", self.dir / "audit.jsonl"
        self.timeline_path = self.dir / "timeline.jsonl"
        self.routes_path = Path(spec["routes"])
        self.kill_file = Path(kill_file) if kill_file else self.dir /
    ↪ "KILL"

        self.floor, self.boot_s, self.drain_s = spec.get("floor_machin
    ↪ es", 2), spec.get("boot_s", 10), spec.get("drain_s", 2)
        self.step_units, self.hold = spec.get("step_units", 2), spec.g
    ↪ et("park_hold_beats", 3)
        self.how = "pack" if brain in ("omni", "omni_native") else "sp
    ↪ read"

        now = time.time()
        self.machines = {m["name"]: {"name": m["name"], "url": m["url"
    ↪ ], "cpu_m": m["cpu_m"], "state": "awake",
                                     "miss": 0, "until": 0.0, "since":
    ↪  now} for m in spec["machines"]}
        self.services = {s["name"]: s for s in spec["services"]}

        self.desired = {n: s["replicas"] for n, s in self.services.ite
    ↪ ms()}
        self.jobs = {j["name"]: j for j in spec.get("jobs", [])}
        self.done: dict[str, set] = {n: set() for n in self.jobs}
        self.units: dict[str, dict] = {}
        self.seq, self.port = 0, spec.get("port_base", 9100)

        self.replayed = self.replay()
        self.brains = {n: OmniBrain(s["line_ms"], beat, s["min"], s["m
    ↪ ax"], spec.get("max_step", 3), self.hold)
                       for n, s in self.services.items()}
        self.yard = {n: Yardstick(s["min"], s["max"]) for n, s in self
    ↪ .services.items()}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    # the verdict (omnicompass/verdict.py), one a service: a unit
↪ fewer than the twin is tried on the service's own
    # response times, stepwise, within the 2% allowance; a trial e
↪ very 2 beats at the earliest, a refused step tried
    # again after 24 beats, a phase given up after 12 beats
    self.verdicts = {n: Verdict(tolerance=0.02, min_samples=30, pr
↪ obe_every=2, recheck=24, max_steps=s["max"],
                                max_trial=12, incremental=True) fo
↪ r n, s in self.services.items()} if brain == "omni" else {}
    self.front_seq = 0                                  # the front'
↪ s answers already handed to the verdict
    self.last_n: dict[str, int] = {}                    # the units
↪ standing at the last beat, and the step the phase
    self.phase_floor: dict[str, int] = {}               # asked for
↪ then: the trial counts only work served at its step
    self.phase_anchor: dict[str, int] = {}              # the twin a
↪ s each phase found it: its steps are counted from there
    self.phase_rate: dict[str, float] = {}              # the arriva
↪ ls (last 5 s) as each phase found them: its traffic
    # a trial the traffic abandoned cost a unit below the twin and
↪  measured nothing; the next waits 6 beats, doubled at
    # each abandonment in a row, up to the verdict's recheck (24 b
↪ eats), and a trial judged to its end clears the hold
    self.abandoned: dict[str, int] = {}

    self.trial_hold: dict[str, int] = {}
    self.busy_raw: dict[str, float | None] = {}
    self.cpu_prev: dict[str, tuple[float, float]] = {}
    self.next_beat, self.next_yard = now + beat, now + 15.0
    self.short: dict[str, int] = {}
    self.stopping: set[str] = set()

    self.park_calm, self.killed, self.stop = 0, False, False
    self.pool = ThreadPoolExecutor(12)
    self.routes_last = None

# ---- the ledger and its journal ---------------------------------
↪ ------------------------------------------------
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def apply(self, ev: dict) -> None:
        k = ev["ev"]
        if k == "desired":
            self.desired[ev["service"]] = ev["n"]
        elif k == "unit":
            u = dict(ev["u"])

            u.setdefault("ready", False)
            self.units[u["id"]] = u
            self.seq = max(self.seq, u["seq"] + 1)
            self.port = max(self.port, u["port"] + 1)
        elif k == "retire":
            if ev["id"] in self.units:

                self.units[ev["id"]].update(state="retiring", retire_a
    ↪ t=ev["at"])
        elif k == "gone":
            self.units.pop(ev["id"], None)
        elif k == "machine":
            m = self.machines.get(ev["name"])

            if m is not None:
                m.update(state=ev["state"], until=ev.get("until", 0.0)
    ↪ , since=ev.get("t", time.time()))
        elif k == "task_done":
            self.done.setdefault(ev["job"], set()).add(ev["index"])
        elif k == "limit":

            if ev["id"] in self.units:
                self.units[ev["id"]]["conveyed_m"] = ev["limit_m"]

    def record(self, ev: dict) -> None:
        ev = {"t": time.time(), **ev}
        with open(self.journal_path, "a") as f:

            f.write(json.dumps(ev) + "\n")
            f.flush()
            os.fsync(f.fileno())
        self.apply(ev)

    def replay(self) -> int:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if not self.journal_path.exists():
            return 0
        n = 0
        for line in self.journal_path.read_text().splitlines():
            try:
                self.apply(json.loads(line))
                n += 1
            except (ValueError, KeyError):
                continue
        return n

    def audit(self, kind: str, **kw) -> None:
        with open(self.audit_path, "a") as f:
            f.write(json.dumps({"t": time.time(), "kind": kind, **kw})
    ↪  + "\n")

    # ---- the machines' hands ------------------------------------
    ↪ ------------------------------------------------

    def poll(self) -> dict[str, dict | None]:
        def get(m):
            try:
                return http("GET", m["url"] + "/status", timeout=1.0)
            except Exception:
                return None
        futs = {n: self.pool.submit(get, m) for n, m in self.machines.
    ↪ items()}
        return {n: f.result() for n, f in futs.items()}

    def _delete(self, url: str, uid: str) -> None:
        try:
            http("DELETE", f"{url}/units/{uid}", timeout=10.0)
        except Exception:
            pass
        self.stopping.discard(uid)

    def stop_unit(self, m: dict, uid: str) -> None:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if uid not in self.stopping:
            self.stopping.add(uid)
            self.pool.submit(self._delete, m["url"], uid)

    def sync(self, status: dict, now: float) -> None:
        for name, m in self.machines.items():
            st = status.get(name)
            if st is None:
                m["miss"] += 1
                if m["miss"] >= 3 and m["state"] != "down":
                    lost = [u for u in self.units.values() if u["machi
    ↪ ne"] == name]
                    self.record({"ev": "machine", "name": name, "state
    ↪ ": "down"})
                    for u in lost:
                        self.record({"ev": "gone", "id": u["id"]})
                    self.audit("machine down", machine=name, why="no a
    ↪ nswer three times running",
                               units_lost=[u["id"] for u in lost])
                continue
            m["miss"] = 0
            prev = m.get("_cpu")                          # the machine
    ↪ 's measured busy share, millicores, smoothed over its polls
            m["_cpu"] = (st.get("time", now), st.get("cpu_s", 0.0))
            if prev and m["_cpu"][0] > prev[0]:
                rate = max(0.0, (m["_cpu"][1] - prev[1]) / (m["_cpu"][
    ↪ 0] - prev[0]) * 1000.0)
                m["busy_m"] = rate if m.get("busy_m") is None else 0.5
    ↪  * m["busy_m"] + 0.5 * rate
            if m["state"] == "down":
                self.record({"ev": "machine", "name": name, "state": "
    ↪ awake"})
                self.audit("machine back", machine=name, why="its hand
    ↪  answers again; it joins empty")
            seen = set()
            for r in st["units"]:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                uid = r["id"]
                seen.add(uid)
                u = self.units.get(uid)
                if u is None or u["machine"] != name:
                    if uid not in self.stopping:
                        self.audit("stray unit stopped", machine=name,
    ↪  unit=uid, why="not in the ledger")
                    self.stop_unit(m, uid)
                    continue
                u.update(ready=bool(r["ready"]), agent_state=r["state"
    ↪ ], restarts=r["restarts"], cpu_s=r["cpu_s"],
                         limit_m=r.get("limit_m"), ever_ready=u.get("e
    ↪ ver_ready", False) or bool(r["ready"]))
                if u.get("kind") == "service" and not u["ever_ready"]
    ↪ and r["restarts"] >= 2 and u["state"] != "retiring":
                    # a unit that cannot come up where it stands (its
    ↪ port taken, its machine lying) is placed again
                    # elsewhere with a new port, instead of being star
    ↪ ted on the same spot forever
                    self.audit("unit replaced", unit=uid, machine=name
    ↪ , why=f"{r['restarts']} starts and never ready")
                    self.record({"ev": "gone", "id": uid})
                    self.stop_unit(m, uid)
                    continue
                if u.get("kind") == "task" and r["state"] in ("succeed
    ↪ ed", "failed") and u["state"] != "retiring":
                    if r["state"] == "succeeded":
                        self.record({"ev": "task_done", "job": u["job"
    ↪ ], "index": u["index"]})
                    else:
                        self.audit("task failed", unit=uid, why="three
    ↪  tries; it goes back in the queue")
                    self.record({"ev": "gone", "id": uid})
                    self.stop_unit(m, uid)
            for u in [u for u in self.units.values() if u["machine"] =
    ↪ = name and u["id"] not in seen]:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                if u["state"] != "retiring" and now - u["born"] > 5.0:
                    self.record({"ev": "gone", "id": u["id"]})
                    self.audit("unit lost", unit=u["id"], machine=name
↪ , why="its machine no longer runs it")

    def retire(self, u: dict, why: str, drain: float | None = None) ->
↪  None:
        self.record({"ev": "retire", "id": u["id"], "at": time.time()
↪ + (self.drain_s if drain is None else drain)})
        self.audit("unit retiring", unit=u["id"], machine=u["machine"]
↪ , why=why)

    def retire_due(self, now: float) -> None:
        for u in [u for u in self.units.values() if u["state"] == "ret
↪ iring" and now >= u["retire_at"]]:
            self.stop_unit(self.machines[u["machine"]], u["id"])
            self.record({"ev": "gone", "id": u["id"]})

    # ---- placing ------------------------------------------------
↪ ------------------------------------------
    def _start(self, u: dict, spec: dict) -> None:
        self.record({"ev": "unit", "u": u})
        url = self.machines[u["machine"]]["url"]

        def go():
            try:
                http("POST", url + "/units", spec)
            except Exception:
                pass                                        # not started
↪ : the next checks find it missing and place it again

        self.pool.submit(go)

    def place_service(self, name: str, svc: dict) -> bool:
        m = place(svc["req_m"], name, self.machines, list(self.units.v
↪ alues()), self.how, self.room_rule)
        if m is None:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        victim = next((u for u in self.units.values() if u.get("ki
↪ nd") == "task" and u["state"] != "retiring"
                      and self.machines[u["machine"]]["state"] ==
↪  "awake"), None)
        if victim is not None:
            self.retire(victim, f"a batch task yields its room to
↪ {name}", drain=0.0)
        return False
    uid, port = f"{name}-{self.seq}", self.port
    u = {"id": uid, "seq": self.seq, "kind": "service", "service":
↪  name, "machine": m, "port": port,
         "req_m": svc["req_m"], "state": "placed", "born": time.ti
↪ me()}
    spec = {"id": uid, "kind": "service", "port": port, "limit_m":
↪  svc["limit_m"], "mem_mb": svc.get("mem_mb", 256),
            "health": svc.get("health", "/health"), "cwd": ROOT,
            "cmd": [sys.executable, "-m", "omni_orchestrator.workl
↪ oad", "serve", "--port", str(port), "--k",
                    str(svc["k"]), "--unit", uid]}
    self._start(u, spec)
    self.audit("unit placed", unit=uid, machine=m, service=name)
    return True

def place_task(self, jname: str, j: dict, index: int) -> bool:
    m = place(j["req_m"], f"job:{jname}", self.machines, list(self
↪ .units.values()), "pack", self.room_rule)
    if m is None:
        return False
    uid = f"{jname}-{index}-{self.seq}"
    u = {"id": uid, "seq": self.seq, "kind": "task", "job": jname,
↪  "index": index, "machine": m, "port": 0,
         "req_m": j["req_m"], "state": "placed", "born": time.time
↪ ()}
    spec = {"id": uid, "kind": "task", "limit_m": j["limit_m"], "m
↪ em_mb": j.get("mem_mb", 256), "cwd": ROOT,
            "cmd": [sys.executable, "-m", "omni_orchestrator.workl
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ oad", "task", "--job", jname, "--index",
                       str(index), "--k", str(j["k"]), "--out", j["ou
    ↪ t"]]}
        self._start(u, spec)
        self.audit("task placed", unit=uid, machine=m, job=jname, inde
    ↪ x=index)

        return True

    def count_on(self, machine: str, service: str | None = None) -> in
    ↪ t:
        return sum(1 for u in self.units.values() if u["machine"] == m
    ↪ achine and u["state"] != "retiring"

                   and (service is None or u.get("service") == service
    ↪ ))

    def reconcile(self) -> None:
        self.short = {}
        for name, svc in self.services.items():

            mine = [u for u in self.units.values() if u.get("service")
    ↪  == name and u["state"] != "retiring"]
            movers = [u for u in mine if self.machines[u["machine"]]["
    ↪ state"] == "draining"]
            staying = [u for u in mine if u not in movers]
            want = self.desired[name]

            for k in range(want - len(staying)):
                if not self.place_service(name, svc):
                    self.short[name] = want - len(staying) - k
                    break
            staying = [u for u in self.units.values() if u.get("servic
    ↪ e") == name and u["state"] != "retiring"

                       and self.machines[u["machine"]]["state"] != "dr
    ↪ aining"]
            if movers and len(staying) >= want and sum(1 for u in stay
    ↪ ing if u["ready"]) >= want:
                for u in movers:
                    self.retire(u, "its machine is being parked; a rep
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ lacement elsewhere is ready")
            extra = len(staying) - want
            if extra > 0:
                if self.how == "pack":
                    key = lambda u: (u["ready"], self.count_on(u["mach
    ↪ ine"]), -u["born"])
                else:
                    key = lambda u: (u["ready"], -self.count_on(u["mac
    ↪ hine"], name), -u["born"])
                for u in sorted(staying, key=key)[:extra]:
                    self.retire(u, f"{want} units wanted")

    def batch(self) -> None:
        if any(self.short.values()):
            return                                      # the service
    ↪ s come first
        for jname, j in self.jobs.items():
            running = [u for u in self.units.values() if u.get("job")

    ↪ == jname and u["state"] != "retiring"]
            busy = {u["index"] for u in running}
            pending = [i for i in range(j["tasks"]) if i not in self.d
    ↪ one[jname] and i not in busy]
            for i in pending[:max(0, j["parallel"] - len(running))]:
                if not self.place_task(jname, j, i):
                    break

    # ---- the brain ------------------------------------------------
    ↪ ---------------------------------------------
    def front_stats(self) -> dict | None:
        try:
            return http("GET", self.spec["front_stats"], timeout=1.0)
        except Exception:
            return None

    def busy(self, name: str, now: float) -> float | None:
        """The service's busy share as the yardstick reads it: the pro
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ cessor its units used over their requests, each
        unit's use capped at its request (a Kubernetes pod never uses
    ↪ more than its limit, here its request; a unit
        Omni has conveyed more to would otherwise read as more pods th
    ↪ an Kubernetes would hold). The uncapped share is
        kept in busy_raw for the audit."""

        used = raw = req = 0.0
        for u in self.units.values():
            if u.get("service") != name or u["state"] == "retiring" or
    ↪  "cpu_s" not in u:
                continue
            prev = self.cpu_prev.get(u["id"])

            self.cpu_prev[u["id"]] = (now, u["cpu_s"])
            if prev is None or now - prev[0] <= 0:
                continue
            dt = now - prev[0]
            d = max(0.0, u["cpu_s"] - prev[1])
            raw += d

            used += min(d, u["req_m"] / 1000.0 * dt)
            req += u["req_m"] / 1000.0 * dt
        self.busy_raw[name] = raw / req if req > 0 else None
        return used / req if req > 0 else None

    def front_recent(self) -> dict | None:

        """The front's reading with every answer's response time since
    ↪  the last beat (for the verdict)."""
        try:
            out = http("GET", f"{self.spec['front_stats']}?since={self
    ↪ .front_seq}", timeout=1.0)
            self.front_seq = int(out.get("seq", self.front_seq))

            return out
        except Exception:
            return None

    def think(self, now: float) -> None:
        stats = self.front_recent() if self.brain_kind in ("omni", "om
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ni_native") else None
        for name, svc in self.services.items():
            n = len([u for u in self.units.values() if u.get("service"
    ↪ ) == name and u["state"] != "retiring"
                     and self.machines[u["machine"]]["state"] != "drai
    ↪ ning"])

            if self.brain_kind == "off":
                want, why = svc["replicas"], {"rule": "the declared un
    ↪ its, no brain"}
            elif self.brain_kind == "yardstick":
                if now < self.next_yard:
                    continue

                self.next_yard = now + 15.0
                busy = self.busy(name, now)
                want = self.yard[name].recommend(now, n, busy)
                why = {"rule": "the yardstick: Kubernetes' published a
    ↪ utoscaling rule", "busy": busy}
            elif self.brain_kind == "omni_native":

                # Omni on top of the native rule (the engine's strict
    ↪ form, as on the kind rigs): the units are the
                # yardstick's, Kubernetes' rule on the same readings,
    ↪ never fewer; Omni's reflex and fail-up add above it;
                # no fold below the twin and no verdict; Omni's convey
    ↪ ance, room rule, packing and machines on top

                busy = self.busy(name, now)
                p95 = (stats or {}).get("p95_5s", 0.0)
                waiting = int((stats or {}).get("line", 0))
                reading = svc["line_ms"] if stats and waiting > 2 * ma
    ↪ x(1, int(stats.get("hands", 1))) else p95
                brain = self.brains[name]

                twin = brain.twin.recommend(now, n, busy)
                want, why = brain.decide(now, n, busy, reading, waitin
    ↪ g, 0, False, twin=twin)
                why["rule"] = "Omni on native: the yardstick's units a
    ↪ t least; Omni's reflex, conveyance and machines on top"
                why["busy_raw"] = None if self.busy_raw.get(name) is N
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ one else round(self.busy_raw[name], 3)
            else:
                busy = self.busy(name, now)
                p95 = (stats or {}).get("p95_5s", 0.0)
                waiting = int((stats or {}).get("line", 0))
                reading = p95

                if stats and waiting > 2 * max(1, int(stats.get("hands
    ↪ ", 1))):
                    reading = svc["line_ms"]             # work waitin
    ↪ g in line past twice the hands: at the wall
                brain, v = self.brains[name], self.verdicts.get(name)
                twin = brain.twin.recommend(now, n, busy)

                depth, trial, anchor = 0, False, None
                if v is not None:
                    calm = waiting == 0 and brain.law.band.position(re
    ↪ ading) < brain.law.band.wall_high
                    rate = float((stats or {}).get("n_5s", 0.0))
    ↪ # the arrivals of the last 5 s

                    r0 = self.phase_rate.get(name)
                    if v.phase is not None and r0 is not None and abs(
    ↪ rate - r0) > 0.15 * max(r0, 5.0):
                        # a trial is paired under the same traffic (om
    ↪ nicompass/verdict.py): the arrivals moved by more
                        # than 15% since the phase began, so what it m

    ↪ easured is the traffic, not the step; abandoned
                        k = self.abandoned.get(name, 0) + 1
                        wait = min(v.recheck, 6 * 2 ** (k - 1))
                        self.abandoned[name], self.trial_hold[name] =
    ↪ k, v.t + 1 + wait
                        self.audit("verdict", service=name, verdict="t

    ↪ rial abandoned (the traffic changed)", step=v.trial,
                                   arrivals_5s_was=r0, arrivals_5s_now
    ↪ =rate, next_trial_after_beats=wait)
                        calm = False
                    elif v.phase is None and v.t + 1 < self.trial_hold
    ↪ .get(name, 0):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                    calm = False                    # the traffic
↪  has been moving faster than a trial: none starts yet
                # the work served over the last beat counts for th
↪ e trial only if the units stood at its step the
                # whole beat (the knob at the step the verdict ask
↪ ed for, and unchanged since)

                if stats and stats.get("recent") and n == self.las
↪ t_n.get(name) == self.phase_floor.get(name):
                    v.observe(stats["recent"])
                if waiting > 0 and v.allowed > 0:
                    # the immune reaction: work waited under an al
↪ lowed depth, so one step goes back to the twin and

                    # is not tried again until the recheck has pas
↪ sed
                    v.allowed -= 1
                    v.refused_until[v.allowed + 1] = v.t + v.reche
↪ ck
                    self.audit("verdict", service=name, verdict="d

↪ epth handed back: work waited in line",
                               step=v.allowed + 1, waiting=waiting
↪ )
                depth, trial, ev = v.tick(calm)
                if ev:
                    self.audit("verdict", service=name, **ev)

                    if ev.get("verdict") == "trial":
                        self.phase_anchor[name] = twin   # the ste
↪ p is counted from the twin as the phase found it
                        self.phase_rate[name] = rate     # and jud
↪ ged only under the traffic the phase found
                    elif str(ev.get("verdict", "")).startswith("st

↪ ep "):
                        self.abandoned[name] = 0         # a trial
↪  judged to its end: the traffic held still for one
                anchor = self.phase_anchor.get(name, twin) if tria
↪ l else None
            want, why = brain.decide(now, n, busy, reading, waitin
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ g, depth, trial, twin=twin, anchor=anchor)
                why["busy_raw"] = None if self.busy_raw.get(name) is N
    ↪ one else round(self.busy_raw[name], 3)
                self.last_n[name], self.phase_floor[name] = n, why["fl
    ↪ oor"]
            if want != self.desired[name]:

                self.audit("units", service=name, was=self.desired[nam
    ↪ e], now=want, why=why)
                self.record({"ev": "desired", "service": name, "n": wa
    ↪ nt})
        if self.brain_kind in ("omni", "omni_native"):
            self.machine_brain(now)

            self.convey(now)

    def _set_limit(self, url: str, uid: str, limit_m: float) -> None:
        try:
            http("POST", f"{url}/units/{uid}/limit", {"limit_m": limit
    ↪ _m}, timeout=5.0)

        except Exception:
            pass                                            # not taken:
    ↪ the read-back differs and the next beat writes again

    def convey(self, now: float) -> None:
        """The engine's conveyance (omni-v7, omni-v13) on Omni's own m

    ↪ achines: each service unit's CPU wall set to the
        room its machine can still give under 75%, shared among the se
    ↪ rvice's units there, never under the request;
        batch tasks on the machine are the other workload, counted by
    ↪ their limits. Written to the hand, journaled,
        read back; audited when the intention changes."""

        for m in self.machines.values():
            if m["state"] != "awake":
                continue
            on = [u for u in self.units.values() if u["machine"] == m[
    ↪ "name"] and u["state"] != "retiring"]
            ours = [u for u in on if u.get("kind") == "service"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        others_m = sum(float(u.get("conveyed_m") or u["req_m"]) fo
↪ r u in on if u.get("kind") != "service")
        want = conveyed_limits(m["cpu_m"], m.get("busy_m"), ours,
↪ others_m)
        if want is None:
            continue

        for u in ours:
            w = want[u["id"]]
            meant = float(u.get("conveyed_m") or u["req_m"])
            if abs(w - meant) >= 10.0:
                self.record({"ev": "limit", "id": u["id"], "limit_
↪ m": w})

                self.audit("limit conveyed" if w > meant else "lim
↪ it handed back", unit=u["id"], machine=m["name"],
                           was=meant, now=w, why=f"the machine {m[
↪ 'busy_m']:.0f} m busy of {m['cpu_m']} m; "
                           f"{len(ours)} unit(s) share the room un
↪ der 75%, other workloads {others_m:.0f} m")

            held = u.get("limit_m")
            if abs(w - meant) >= 10.0 or (held is not None and abs
↪ (held - w) >= 10.0):
                self.pool.submit(self._set_limit, m["url"], u["id"
↪ ], w)

def hand_back_limits(self) -> None:
    """Every conveyed wall back to the request (the declared state
↪ ), as the kill switch asks."""
    for u in self.units.values():
        if u.get("kind") == "service" and u.get("conveyed_m") and
↪ u["conveyed_m"] != u["req_m"]:

            self.record({"ev": "limit", "id": u["id"], "limit_m":
↪ float(u["req_m"])})
            self.pool.submit(self._set_limit, self.machines[u["mac
↪ hine"]]["url"], u["id"], float(u["req_m"]))

def machine_brain(self, now: float) -> None:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    room = max(m["cpu_m"] for m in self.machines.values()) * (BUSY
↪ _TOP if self.room_rule == "busy" else 1.0)
    req = max(s["req_m"] for s in self.services.values())
    tasks_m = sum(u["req_m"] for u in self.units.values() if u.get
↪ ("kind") == "task" and u["state"] != "retiring")
    units = sum(self.desired.values())

    need = machines_needed(units, req, room, self.step_units, self
↪ .floor, tasks_m)
    live = [m for m in self.machines.values() if m["state"] in ("a
↪ wake", "booting")]
    awake = [m for m in self.machines.values() if m["state"] == "a
↪ wake"]

    booting = [m for m in self.machines.values() if m["state"] ==
↪ "booting"]
    draining = [m for m in self.machines.values() if m["state"] ==
↪  "draining"]
    short = any(self.short.values())
    if need > len(live) or (short and not booting):

        # a unit with no room waits only for a machine already on
↪ its way: one is woken, not one a beat
        self.park_calm = 0
        k = max(1, need - len(live))
        for m in draining[:k]:
            self.record({"ev": "machine", "name": m["name"], "stat

↪ e": "awake"})
            self.audit("machine kept", machine=m["name"], why=f"th
↪ e units wanted need {need} machines; its drain is undone")
            k -= 1
        for m in [m for m in self.machines.values() if m["state"]
↪ == "parked"][:max(0, k)]:

            self.record({"ev": "machine", "name": m["name"], "stat
↪ e": "booting", "until": now + self.boot_s})
            self.audit("machine woken", machine=m["name"],
                       why=f"{units} units wanted need {need} mach
↪ ines with room for {self.step_units} more"
                       if need > len(live) else "a unit found no r
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ oom")
        elif short:
            self.park_calm = 0                              # a machine i
    ↪ s booting for the unit that waits
        elif need < len(awake) and not draining and not booting:
            self.park_calm += 1

            if self.park_calm >= self.hold:
                m = min(awake, key=lambda m: (self.count_on(m["name"])
    ↪ , m["name"]))
                self.record({"ev": "machine", "name": m["name"], "stat
    ↪ e": "draining"})
                self.audit("machine draining", machine=m["name"],

                           why=f"{units} units wanted need {need} mach
    ↪ ines; {len(awake)} awake")
                self.park_calm = 0
        else:
            self.park_calm = 0

    def machines_step(self, now: float) -> None:
        for m in self.machines.values():
            if m["state"] == "booting" and now >= m["until"]:
                self.record({"ev": "machine", "name": m["name"], "stat
    ↪ e": "awake"})
                self.audit("machine awake", machine=m["name"], why="bo

    ↪ oted")
            elif m["state"] == "draining":
                if not any(u["machine"] == m["name"] for u in self.uni
    ↪ ts.values()):
                    self.record({"ev": "machine", "name": m["name"], "
    ↪ state": "parked"})

                    self.audit("machine parked", machine=m["name"], wh
    ↪ y="empty")
                elif now - m["since"] > 60:
                    self.record({"ev": "machine", "name": m["name"], "
    ↪ state": "awake"})
                    self.audit("machine kept", machine=m["name"], why=
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ "its units found no room elsewhere in 60 s")

    def kill_switch(self, now: float) -> None:
        on = self.kill_file.exists()
        if on and not self.killed:
            self.killed = True

            for name, svc in self.services.items():
                if self.desired[name] != svc["replicas"]:
                    self.record({"ev": "desired", "service": name, "n"
    ↪ : svc["replicas"]})
            for m in self.machines.values():
                if m["state"] == "draining":

                    self.record({"ev": "machine", "name": m["name"], "
    ↪ state": "awake"})
                elif m["state"] == "parked":
                    self.record({"ev": "machine", "name": m["name"], "
    ↪ state": "booting", "until": now + self.boot_s})
            self.hand_back_limits()

            self.audit("kill switch on", why="the brain is off; the de
    ↪ clared state is held: the declared units, every "
                       "machine on, every wall at its request")
        elif not on and self.killed:
            self.killed = False
            self.audit("kill switch off", why="the brain resumes")

    # ---- the front and the record ---------------------------------
    ↪ ----------------------------------------------
    def write_routes(self) -> None:
        routes = {name: sorted(({"id": u["id"], "host": "127.0.0.1", "
    ↪ port": u["port"], "ready": True}

                               for u in self.units.values() if u.get(
    ↪ "service") == name and u["ready"]
                               and u["state"] != "retiring" and self.
    ↪ machines[u["machine"]]["state"] != "down"),
                              key=lambda r: r["id"]) for name in self
    ↪ .services}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if routes == self.routes_last:
            return
        tmp = self.routes_path.with_suffix(".tmp")
        tmp.write_text(json.dumps(routes))
        os.replace(tmp, self.routes_path)
        self.routes_last = routes

    def timeline(self, now: float) -> None:
        states = {}
        for m in self.machines.values():
            states[m["state"]] = states.get(m["state"], 0) + 1
        row = {"t": now, "brain": self.brain_kind, "killed": self.kill
    ↪ ed, "desired": dict(self.desired),
                "units": {n: sum(1 for u in self.units.values() if u.ge
    ↪ t("service") == n and u["state"] != "retiring")
                          for n in self.services},
                "ready": {n: sum(1 for u in self.units.values() if u.ge
    ↪ t("service") == n and u["ready"]
                                 and u["state"] != "retiring") for n in
    ↪  self.services},
                "machines": states, "short": dict(self.short),
                "tasks_running": sum(1 for u in self.units.values() if
    ↪ u.get("kind") == "task" and u["state"] != "retiring"),
                "tasks_done": {j: len(d) for j, d in self.done.items()}
    ↪ ,
                # the processor conveyed above the requests (millicores
    ↪ , every service unit standing) and the verdict's state
                "conveyed_m": sum(max(0.0, float(u.get("conveyed_m") or
    ↪  u["req_m"]) - u["req_m"]) for u in self.units.values()
                                  if u.get("kind") == "service" and u["
    ↪ state"] != "retiring"),
                "depth": {n: v.allowed for n, v in self.verdicts.items(
    ↪ )},
                "trial": {n: v.phase for n, v in self.verdicts.items()
    ↪ if v.phase}}
        with open(self.timeline_path, "a") as f:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            f.write(json.dumps(row) + "\n")

    def step(self, now: float) -> None:
        self.sync(self.poll(), now)
        self.retire_due(now)
        self.kill_switch(now)

        if not self.killed and now >= self.next_beat:
            self.next_beat = now + self.beat
            self.think(now)
        self.machines_step(now)
        self.reconcile()
        self.batch()

        self.write_routes()
        self.timeline(now)

    def run(self) -> None:
        self.audit("control plane started", brain=self.brain_kind, bea
    ↪ t=self.beat, journal_lines_replayed=self.replayed,

                   units_in_ledger=len(self.units))
        while not self.stop:
            t = time.time()
            try:
                self.step(t)
            except Exception as e:                       # a failed ch

    ↪ eck is recorded and the next comes on time
                self.audit("error", error=repr(e)[:300])
            time.sleep(max(0.05, 1.0 - (time.time() - t)))


def main(argv=None):

    ap = argparse.ArgumentParser(description=__doc__.split("\n\n")[0])
    ap.add_argument("--spec", required=True)
    ap.add_argument("--state-dir", required=True)
    ap.add_argument("--brain", choices=("omni", "omni_native", "yardst
    ↪ ick", "off"), default="omni",
                    help="omni: Omni direct, Omni alone on the machine
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ s; omni_native: Omni on top of the native rule "
                        "(Kubernetes' autoscaling rule holds the unit
    ↪ s, Omni conveys, places, parks and adds above it); "
                        "yardstick: the native rule alone; off: the d
    ↪ eclared units, no brain")
    ap.add_argument("--beat", type=float, default=60.0, help="the brai

    ↪ n's beat in seconds (the dial: 5 to 120 s)")
    ap.add_argument("--kill-file", default=None)
    ap.add_argument("--room-rule", choices=("requests", "busy"), defau
    ↪ lt="requests",
                    help="room by requests alone (Kubernetes' way), or
    ↪  also under 75%% of the machine's measured busy share")

    a = ap.parse_args(argv)
    c = Control(json.loads(Path(a.spec).read_text()), a.state_dir, a.b
    ↪ rain, a.beat, a.kill_file, a.room_rule)
    signal.signal(signal.SIGTERM, lambda *_: setattr(c, "stop", True))
    c.run()

if __name__ == "__main__":
    main()

# © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Sim
    ↪ ulation use only; any commercialization or

# monetization requires a signed, paid Omni-Compass Enterprise License
    ↪ . All patents, copyrights and trademarks filed in the USA. www.o
    ↪ mni-compass.com
```

H.13 `omni_orchestrator/front.py`

218 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""One address for a service: its own process, so the service answers
    ↪ while the control plane is down.

Every request joins one line; each ready unit has one hand (a thread w
    ↪ ith one kept-open connection) that takes the next
request from the line, hands it to its unit and waits for the answer,
    ↪ so a unit works on one request at a time and the
waiting happens here, where it is measured. The units are read from th
    ↪ e routes file the control plane writes (only
ready units are in it). A unit that fails to answer sends the request
    ↪ back to the head of the line for another unit
(three tries in all) and rests half a second before taking work again;
    ↪  a unit that leaves the routes finishes the
request in hand and takes no more.

HTTP on 127.0.0.1:<port>: any GET is the service's (GET /work?n=...);
    ↪ GET /_stats is this front's own reading: the
answers of the last 5 and 10 s (count, failures, the 50th, 95th and 99
    ↪ th percentiles of the response time), the line's
length and the hands working; GET /_stats?since=<seq> adds every answe
    ↪ r's response time after that sequence number
(the pieces of work the control plane's verdict measures a trial on).
    ↪ Every second the same reading is appended to the
stats file.
"""
from __future__ import annotations

import argparse
import collections
import http.client
import json
import threading
import time
import urllib.parse
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path


def pct(xs: list[float], q: float) -> float:
    if not xs:
        return 0.0
    s = sorted(xs)
    return s[min(len(s) - 1, int(q * len(s)))]


class Job:
    __slots__ = ("path", "done", "status", "body", "tries", "t0")

    def __init__(self, path: str):
        self.path, self.done, self.status, self.body, self.tries, self
    ↪ .t0 = path, threading.Event(), 0, b"", 0, time.time()

class Hand(threading.Thread):
    def __init__(self, front: "Front", uid: str, host: str, port: int)
    ↪ :
        super().__init__(daemon=True)
        self.front, self.uid, self.host, self.port, self.alive = front
    ↪ , uid, host, port, True

    def run(self):
        conn = None
        misses = 0
        while self.alive:
            j = self.front.take(self)
            if j is None:
                break
            try:
                if conn is None:
                    conn = http.client.HTTPConnection(self.host, self.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ port, timeout=15)
                conn.request("GET", j.path)
                r = conn.getresponse()
                j.body, j.status = r.read(), r.status
                j.done.set()
                misses = 0

            except Exception:
                if conn is not None:
                    conn.close()
                    conn = None
                j.tries += 1
                if j.tries < 3:

                    self.front.requeue(j)
                else:
                    j.status, j.body = 502, b"no unit answered"
                    j.done.set()
                misses += 1
                time.sleep(0.5 if misses < 3 else 2.0)

        if conn is not None:
            conn.close()


class Front:
    def __init__(self, routes: str, service: str, stats_path: str):

        self.routes, self.service, self.stats_path = Path(routes), ser
    ↪ vice, Path(stats_path)
        self.line: collections.deque[Job] = collections.deque()
        self.cv = threading.Condition()
        self.hands: dict[str, Hand] = {}
        self.done: collections.deque = collections.deque(maxlen=200_00

    ↪ 0)   # (time, ms, ok)
        self.recent: collections.deque = collections.deque(maxlen=20_0
    ↪ 00)  # (seq, ms) of the answers, for the verdict
        self.seq = 0
        self.mtime = 0.0
        self.busy = 0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def submit(self, path: str) -> Job:
        j = Job(path)
        with self.cv:
            self.line.append(j)
            self.cv.notify()

        if not j.done.wait(30):
            j.status, j.body = 504, b"no answer in 30 s"
        ms = 1000.0 * (time.time() - j.t0)
        self.done.append((time.time(), ms, j.status == 200))
        if j.status == 200:
            with self.cv:

                self.seq += 1
                self.recent.append((self.seq, ms))
        return j

    def since(self, seq: int) -> dict:
        """The response times of every answer after sequence number se

    ↪ q (the last 5,000 at most), and the newest seq."""
        with self.cv:
            xs = [ms for s, ms in self.recent if s > seq]
            return {"seq": self.seq, "recent": [round(x, 2) for x in x
    ↪ s[-5000:]]}

    def take(self, hand: Hand) -> Job | None:
        with self.cv:
            while hand.alive and not self.line:
                self.cv.wait(0.5)
            if not hand.alive:
                return None

            return self.line.popleft()

    def requeue(self, j: Job) -> None:
        with self.cv:
            self.line.appendleft(j)
            self.cv.notify()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def read_routes(self) -> None:
        try:
            m = self.routes.stat().st_mtime
        except OSError:
            return

        if m == self.mtime:
            return
        self.mtime = m
        try:
            units = {u["id"]: u for u in json.loads(self.routes.read_t
    ↪ ext()).get(self.service, []) if u.get("ready")}

        except (OSError, ValueError):
            return
        for uid, u in units.items():
            if uid not in self.hands:
                h = Hand(self, uid, u.get("host", "127.0.0.1"), int(u[
    ↪ "port"]))

                self.hands[uid] = h
                h.start()
        for uid in [x for x in self.hands if x not in units]:
            self.hands.pop(uid).alive = False
        with self.cv:
            self.cv.notify_all()

    def reading(self) -> dict:
        now = time.time()
        out = {"t": now, "line": len(self.line), "hands": len(self.han
    ↪ ds)}
        for w in (1, 5, 10):

            xs = [(ms, ok) for t, ms, ok in list(self.done) if t >= no
    ↪ w - w]
            ok_ms = [ms for ms, ok in xs if ok]
            out[f"n_{w}s"] = len(xs)
            out[f"fail_{w}s"] = sum(1 for _, ok in xs if not ok)
            out[f"p50_{w}s"] = round(pct(ok_ms, 0.50), 1)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            out[f"p95_{w}s"] = round(pct(ok_ms, 0.95), 1)
            out[f"p99_{w}s"] = round(pct(ok_ms, 0.99), 1)
        return out

    def loop(self) -> None:
        nxt = time.time()
        while True:
            self.read_routes()
            if time.time() >= nxt:
                with open(self.stats_path, "a") as f:
                    f.write(json.dumps(self.reading()) + "\n")
                nxt += 1.0
            time.sleep(0.2)


def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__.split("\n\n")[0])
    ap.add_argument("--port", type=int, required=True)
    ap.add_argument("--service", required=True)
    ap.add_argument("--routes", required=True)
    ap.add_argument("--stats", required=True)
    a = ap.parse_args(argv)
    front = Front(a.routes, a.service, a.stats)
    class Handler(BaseHTTPRequestHandler):
        protocol_version = "HTTP/1.1"

        def do_GET(self):
            if self.path == "/_stats" or self.path.startswith("/_stats
    ↪ ?"):
                q = urllib.parse.parse_qs(urllib.parse.urlsplit(self.p
    ↪ ath).query)
                out = front.reading()
                if "since" in q:
                    out.update(front.since(int(q["since"][0])))
                body, code = json.dumps(out).encode(), 200
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            else:
                j = front.submit(self.path)
                body, code = j.body, j.status
            self.send_response(code)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)

        def log_message(self, *a):
            pass
    threading.Thread(target=front.loop, daemon=True).start()
    srv = ThreadingHTTPServer(("127.0.0.1", a.port), Handler)
    srv.daemon_threads = True
    srv.request_queue_size = 256
    srv.serve_forever(poll_interval=0.2)
if __name__ == "__main__":
    main()

# © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Sim
    ↪ ulation use only; any commercialization or
# monetization requires a signed, paid Omni-Compass Enterprise License
    ↪ . All patents, copyrights and trademarks filed in the USA. www.o
    ↪ mni-compass.com
```

H.14 `omni_orchestrator/scheduler.py`

52 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Where a unit goes: the room rule.

A machine's room is its processor less the requests of the units on it
    ↪ ; a unit goes only where its request fits, so no

machine is ever asked for more than it has. Only machines awake take n
    ↪ ew units (one booting, draining or parked takes
none). Two ways to choose among the machines that fit:

  pack    Omni's: the machine left fullest by the unit (the best fit),
    ↪  so whole machines empty out and can be parked;

          but a service of two or more units is never left on one mach
    ↪ ine alone, so one machine lost never takes the
          whole service with it
  spread  the yardstick's, as Kubernetes' own scheduler scores by defa
    ↪ ult: the machine with the fewest units of the
          service, then the one with the most room left

`place` returns the machine's name, or None when no machine awake has
    ↪ the room (the brain then wakes one)."""
from __future__ import annotations

BUSY_TOP = 0.75     # the engine's own rule (omni_controller/capacity.
    ↪ py, omni-v7): no unit onto a machine this busy


def room(machine: dict, units: list[dict], rule: str = "requests") ->
    ↪ float:

    """The room on a machine: its processor less the requests on it; u
    ↪ nder the busy rule also less what its measured
    busy share leaves under 75% of its processor, so a machine at its
    ↪ top takes no unit whatever its requests say."""
    by_req = machine["cpu_m"] - sum(u["req_m"] for u in units if u["ma
    ↪ chine"] == machine["name"])
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if rule == "busy" and machine.get("busy_m") is not None:
        return min(by_req, BUSY_TOP * machine["cpu_m"] - machine["busy
    ↪ _m"])
    return by_req

def place(req_m: float, service: str, machines: dict[str, dict], units
    ↪ : list[dict], how: str = "pack",
          rule: str = "requests") -> str | None:
    awake = [m for m in machines.values() if m["state"] == "awake"]
    fits = [m for m in awake if room(m, units, rule) >= req_m]
    if not fits:

        return None
    mine = {m["name"]: sum(1 for u in units if u["machine"] == m["name
    ↪ "] and u.get("service") == service) for m in awake}
    if how == "spread":
        return min(fits, key=lambda m: (mine[m["name"]], -room(m, unit
    ↪ s, rule), m["name"]))["name"]

    on = {name for name, n in mine.items() if n > 0}
    if len(on) == 1:
        # the service stands on one machine: its next unit goes to ano
    ↪ ther machine where there is one with the room
        others = [m for m in fits if m["name"] not in on]
        if others:

            fits = others
    return min(fits, key=lambda m: (room(m, units, rule) - req_m, m["n
    ↪ ame"]))["name"]

# © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Sim
    ↪ ulation use only; any commercialization or

# monetization requires a signed, paid Omni-Compass Enterprise License
    ↪ . All patents, copyrights and trademarks filed in the USA. www.o
    ↪ mni-compass.com
```

H.15 `omnicompass/__init__.py`

5 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
```

H.16 omnicompass/adapter.py

344 lines.

```
"""Omni-Compass stack governor.

  sense      : stack telemetry -> normalized observation vector
  assimilate : observation blended into the six-state engine state
  evolve     : one macro step of equations (1)-(7) with u = 0
  allocate   : engine state -> directive (capacity, change gating, pow
    ↪ er, rollback, routing, alerting, paging)
  authority  : OBSERVE | AUTOPILOT; kill() revokes authority and retur
    ↪ ns control to the native managers
"""
from __future__ import annotations

import math
from collections import deque
from dataclasses import dataclass, asdict, field
from typing import Any, Dict, List, Optional

from .core import State, Params, macro_step, v_eff, control_command, U
    ↪ _AUTHORITY, MACRO_DT

OBSERVE = "observe"
AUTOPILOT = "autopilot"

# Engine parameters used for stack governance (fixed, not sampled).
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
STACK_PARAMS = Params(alpha=4.2, beta_int=0.0, beta_ext=0.0, k=1.7, si
    ↪ gma_1=0.38, delta=0.705, gamma_1=1.35,
                      gamma_c=1.0, lambda_0=-0.40, lambda_1=1.55, lamb
    ↪ da_2=1.15, c=1.0, E_max=2.0,
                      omega_B=1.0, Q_B=2.5, alpha_s=0.12, beta_s=0.10,
    ↪  mu=3.782, alpha_E=3.487, alpha_U=4.2,

                      lambda_I=0.719, lambda_U=0.0)
INITIAL_STATE = State(E=0.12, U=0.82, I_U=0.0, S=0.0, B=0.0, B_dot=0.0
    ↪ )
ASSIMILATION = 0.339
REF_PERIOD_S = 60.0     # omni-v16: the step every per-step constant o
    ↪ f the brain was set at (one decision a minute)

def clamp(v: float, lo: float, hi: float) -> float:
    return lo if v < lo else hi if v > hi else v

def observe_vector(obs: Dict[str, float], conflicts: int) -> Dict[str,
    ↪  float]:
    """Normalize telemetry. Inputs are the stack's delayed observation
    ↪ s."""
    return {
        "q": clamp(float(obs.get("queue_ratio", 0.0)), 0.0, 2.0),

        "load": clamp(float(obs.get("load_ratio", 0.0)), 0.0, 2.0),
        "power": clamp(float(obs.get("power_stress", 0.0)), 0.0, 1.5),
        "thermal": clamp(float(obs.get("thermal", 0.0)), 0.0, 1.5),
        "network": clamp(float(obs.get("network_stress", 0.0)), 0.0, 1
    ↪ .5),
        "drift": clamp(float(obs.get("drift_ratio", 0.0)), 0.0, 1.5),

        "stale": clamp(float(obs.get("stale", 0.0)), 0.0, 1.0),
        "security": clamp(float(obs.get("security_block", 0.0)), 0.0,
    ↪ 1.0),
        "conflict": clamp(conflicts / 4.0, 0.0, 1.0),
        # the response line clean for the governor's own clear time (o
    ↪ mni-v15): the Kubernetes governor's slo_clean (its

        # --slo-clear-s, 180 s) and the card governor's (its --slo-cle
    ↪ ar decisions); a caller that senses no line is clean
        "clean": 1.0 if obs.get("slo_clean", True) else 0.0,
    }
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def assimilate(x: State, o: Dict[str, float], p: Params, a: float = AS
    ↪ SIMILATION) -> State:
    """Blend the observation into the engine state and set the forcing
    ↪  terms (a: the observation's weight in one step)."""
    q, load, pw, th, nw, dr, st, sec, cf = (o[k] for k in
        ("q", "load", "power", "thermal", "network", "drift", "stale",
    ↪  "security", "conflict"))
    e_obs = clamp(0.25 * q + 0.18 * max(0.0, load - 0.85) + 0.16 * pw
    ↪ + 0.13 * th + 0.10 * nw + 0.08 * dr
                  + 0.06 * cf + 0.04 * st, 0.0, 1.5)
    u_obs = clamp(1.0 - (0.27 * q + 0.18 * pw + 0.16 * th + 0.12 * nw
    ↪ + 0.12 * dr + 0.10 * cf + 0.05 * st), 0.0, 1.0)
    s_obs = clamp(0.36 * th + 0.28 * pw + 0.18 * nw + 0.10 * sec + 0.0
    ↪ 8 * st - 0.20 * q, -1.0, 1.0)
    b_obs = clamp(0.52 * pw + 0.26 * th + 0.22 * dr, -1.0, 1.5)
    i_obs = clamp(q + dr + cf, 0.0, 2.0)
    p.beta_int = clamp(0.58 * q + 0.42 * max(0.0, load - 0.75), 0.0, 1
    ↪ .0)
    p.beta_ext = clamp(0.33 * pw + 0.27 * th + 0.20 * nw + 0.12 * st +
    ↪  0.08 * sec, 0.0, 1.0)
    return State(E=(1 - a) * x.E + a * e_obs, U=(1 - a) * x.U + a * u_
    ↪ obs, I_U=(1 - a) * x.I_U + a * i_obs,
                 S=(1 - a) * x.S + a * s_obs, B=(1 - a) * x.B + a * b_
    ↪ obs, B_dot=(1 - a) * x.B_dot + a * (b_obs - x.B))

@dataclass(frozen=True)
class AllocationLaw:
    """Engine state -> governing directive.
    release    capacity removed only while push <= push_release (the e
    ↪ ngine's controller reports convergence)
    add        without backlog, capacity added only while push >= push
    ↪ _add (the engine's controller reports need)
    push       equation (2) control law evaluated on the evolved state
    ↪  with target sigma = +1:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
           u = clip(-f_U(x) + KP (1 - U), -U_AUTHORITY, U_AUTHORIT
↪ Y);  push = u / U_AUTHORITY
           (f_U contains the double-well drift and -(dE/dt)/E_max
↪ from equation (1))
headroom   rho* = clamp(rho0 - kI*I_U - kE*E, rho_min, rho0)
sizing     load_s = load * cap_now if size_at_full_cap else load

capacity   n_req = ceil(n * load / rho*) + ceil(kq * q * n);  delt
↪ a = clamp(n_req - n, -down_max, +up_max)
guard      no capacity removal while q >= guard_queue
band       capacity removed only when n_req <= n - down_band
dwell      capacity removed only after surplus persists for down_d
↪ well intervals and no earlier than

           down_after_add intervals after the last addition
envelope   no capacity addition while observed power >= power_ceil
↪ ing (power_ceiling_backlog while q >= guard_queue)
security   no capacity addition while a security block is observed
change     Terraform plans only while U >= U_gate, only in the dir
↪ ection of the live capacity decision

power      q < guard_queue and U >= U_gate and the line clean: cap
↪  = clamp(cap_now * load * (1 + margin), cap_min, 1)
           (trim to demand). The line is clean when the caller's o
↪ wn clear time has passed since its last breach
           (obs slo_clean: 180 s on the Kubernetes governor, 3 dec
↪ isions on the card's; omni-v15), so the request

           cannot switch back and forth at the one threshold; a re
↪ lease is never delayed (the branches below, and
           cap = 1 while the line is not clean)
           q >= guard_queue at the power envelope: cap = max(cap_m
↪ in, cap_now - spread_step)      (spread)
           q >= guard_queue below the envelope: cap held (capacity

↪  added as nodes; 1 at max_nodes)
           otherwise cap = 1; always cap <= 1 - cap_gain * max(0,
↪ B - B_cap)
rollback   a proposed rollback is authorised if S >= S_rollback, U
↪  is falling, or security blocks change
replicas   direct actuation: desired = ceil(replicas * load / rho*
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ) (the HPA formula with the engine-set target),
              increases unless capacity is being reduced, decreases o
    ↪ nly while reducing;
              otherwise autoscaler increases approved unless reducing
    ↪ , decreases only while reducing
    consistency a step that expands capacity never tightens the power

    ↪ cap
    paging     none: the governor holds every action a paged operator
    ↪ would take
    """
    rho0: float = 0.914
    rho_min: float = 0.627

    kI: float = 0.252
    kE: float = 0.297
    kq: float = 0.795
    up_max: int = 2
    down_max: int = 1
    guard_queue: float = 0.05

    power_ceiling: float = 0.95
    power_ceiling_backlog: float = 1.00
    spread_step: float = 0.05
    backlog_release: bool = False
    size_at_full_cap: bool = False
    push_release: float = 0.005

    push_add: float = -9.0
    U_gate: float = 0.481
    margin: float = 0.006
    cap_min: float = 0.706
    B_cap: float = 0.40
    cap_gain: float = 0.60

    S_rollback: float = 0.55
    route_network: float = 0.20
    down_band: int = 4
    down_dwell: int = 4
    down_after_add: int = 4
    min_nodes: int = 8
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    max_nodes: int = 40


@dataclass
class Governor:
    law: AllocationLaw = field(default_factory=AllocationLaw)

    mode: str = AUTOPILOT
    killed: bool = False
    x: State = field(default_factory=lambda: State(**asdict(INITIAL_ST
    ↪ ATE)))
    p: Params = field(default_factory=lambda: Params(**asdict(STACK_PA
    ↪ RAMS)))

    t: float = 0.0
    current_cap: float = 1.0
    nodes: int = 20
    surplus_streak: int = 0
    since_add: int = 99
    last_push: float = 0.0

    log: deque = field(default_factory=lambda: deque(maxlen=4096))
    evolve: bool = True
    period_s: float = REF_PERIOD_S          # omni-v16: the caller's s
    ↪ econds between steps (step keeps the brain's own minute)
    recent: Optional[deque] = None          # omni-v16: (added, remove
    ↪ d) at each of the other steps of the last minute

    def kill(self) -> None:
        """Revoke authority; the engine continues to compute and log d
    ↪ irectives."""
        self.killed = True

    def set_mode(self, mode: str) -> None:
        if mode not in (OBSERVE, AUTOPILOT):
            raise ValueError(mode)
        self.mode = mode

    @property
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def has_authority(self) -> bool:
        return self.mode == AUTOPILOT and not self.killed

    def step(self, obs: Dict[str, float], conflicts: int) -> Dict[str,
    ↪  Any]:
        """Sense, assimilate, evolve, allocate.

        omni-v16: the brain keeps its own minute whatever the caller's
    ↪  step. Its constants were set for one step a minute;
        a caller stepping every period_s seconds moves the brain k = p
    ↪ eriod_s / 60 of its minute each step: the
        observation enters with weight 1 - (1 - 0.339)^k, the engine a
    ↪ dvances k of its macro interval, a dwell or a wait
        counted in steps lasts as many seconds as at one step a minute
    ↪ , and (for a step shorter than a minute) the
        machines added or removed over the last minute's steps stay wi
    ↪ thin up_max and down_max. At 60 s every value is
        the one the engines before omni-v16 used."""

        k = self.period_s / REF_PERIOD_S
        o = observe_vector(obs, conflicts)
        U_before = self.x.U
        xa = assimilate(self.x, o, self.p, ASSIMILATION if k == 1.0 el
    ↪ se 1.0 - (1.0 - ASSIMILATION) ** k)
        if self.evolve:

            xn, _ = macro_step(xa, self.p, self.t, target=None, macro_
    ↪ dt=MACRO_DT * k)
        else:
            xn = xa
        self.x = xn
        self.t += 0.1 * k

        L = self.law
        dwell, after = (L.down_dwell, L.down_after_add) if k == 1.0 el
    ↪ se \
            (max(1, int(round(L.down_dwell / k))), max(1, int(round(L.
    ↪ down_after_add / k))))
        n = int(self.nodes)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        u_push, _ = control_command(xn, self.p, self.t, +1)
        push = u_push / U_AUTHORITY
        self.last_push = push
        rho = clamp(L.rho0 - L.kI * xn.I_U - L.kE * xn.E, L.rho_min, L
    ↪ .rho0)
        converged = push <= L.push_release

        load_sized = o["load"] * self.current_cap if L.size_at_full_ca
    ↪ p else o["load"]
        n_req = math.ceil(n * load_sized / rho) + math.ceil(L.kq * o["
    ↪ q"] * n)
        delta = max(-L.down_max, min(L.up_max, n_req - n))
        if delta < 0 and not converged:

            delta = 0
        if delta > 0 and o["q"] < L.guard_queue and push < L.push_add:
            delta = 0
        if delta < 0 and (o["q"] >= L.guard_queue or n_req > n - L.dow
    ↪ n_band):
            delta = 0

        self.surplus_streak = self.surplus_streak + 1 if delta < 0 els
    ↪ e 0
        if delta < 0 and (self.surplus_streak < dwell or self.since_ad
    ↪ d < after):
            delta = 0
        ceiling = L.power_ceiling_backlog if o["q"] >= L.guard_queue e

    ↪ lse L.power_ceiling
        if delta > 0 and (o["power"] >= ceiling or n >= L.max_nodes):
            delta = 0
        if o["security"] > 0.5:
            delta = min(delta, 0)
        if k < 1.0:

            # up_max and down_max a minute, not a step (omni-v16): thi
    ↪ s step with the other steps of the last minute
            m = max(1, int(round(1.0 / k)))
            if self.recent is None:
                self.recent = deque()
            if delta > 0:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            delta = max(0, min(delta, L.up_max - sum(a_ for a_, _
↪ in self.recent)))
        elif delta < 0:
            delta = min(0, max(delta, -(L.down_max - sum(r_ for _,
↪  r_ in self.recent))))
        self.recent.append((max(0, delta), max(0, -delta)))

        while len(self.recent) > m - 1:
            self.recent.popleft()
    self.since_add = 0 if delta > 0 else self.since_add + 1
    if delta < 0:
        self.surplus_streak = 0
    at_envelope = o["power"] >= ceiling

    if o["q"] < L.guard_queue and xn.U >= L.U_gate and o["clean"]
↪ >= 0.5:
        # the trim to demand, only while the line has been clean f
↪ or the caller's own clear time (omni-v15): on the
        # omni-v11 long run the request switched between the start
↪  limit and the trim at 126 of 350 decisions and

        # turned back at 123 of them, at the one threshold on the
↪ queue and the state; on the same decisions this
        # gate leaves 35 moves (tests/test_cap_band.py). The card
↪ governor had applied the same gate outside the brain
        cap = clamp(self.current_cap * o["load"] * (1.0 + L.margin
↪ ), L.cap_min, 1.0)

    elif o["q"] >= L.guard_queue and L.spread_step > 0.0:
        if at_envelope:
            cap = clamp(self.current_cap - L.spread_step, L.cap_mi
↪ n, 1.0)
        else:
            cap = 1.0 if L.backlog_release else (self.current_cap

↪ if n < L.max_nodes else 1.0)
    elif o["q"] >= L.guard_queue and L.backlog_release:
        cap = 1.0
    else:
        cap = 1.0
    cap = min(cap, clamp(1.0 - L.cap_gain * max(0.0, xn.B - L.B_ca
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ p), L.cap_min, 1.0))
        if delta > 0:
            cap = max(cap, self.current_cap)
        command = clamp(0.5 * (v_eff(xn, self.p) / max(self.p.c, 1e-9)
    ↪  + 1.0), 0.0, 1.0)
        d = {

            "node_delta": int(delta),
            "change_permitted": xn.U >= L.U_gate and o["security"] <=
    ↪ 0.5,
            "power_cap": cap,
            "rollback_authorized": xn.S >= L.S_rollback or xn.U < U_be
    ↪ fore or o["security"] > 0.5,

            "route_shift": o["network"] > L.route_network,
            "state": {"E": xn.E, "U": xn.U, "I_U": xn.I_U, "S": xn.S,
    ↪ "B": xn.B},
            "demand": rho,
            "load": o["load"],
            "command": command,

        }
        self.log.append({"mode": self.mode, "killed": self.killed, **{
    ↪ k: v for k, v in d.items() if k != "state"},
                         **d["state"]})
        return d

def directive_to_actions(d: Dict[str, Any], stack_state: Dict[str, Any
    ↪ ], proposals: List[Dict[str, Any]],
                         cfg, direct_actuation: bool = False) -> List[
    ↪ Dict[str, Any]]:
    """Directive -> executable stack actions. direct_actuation: capaci

    ↪ ty is owned by the governor; no Terraform plans."""
    tag = "Omni-Compass"
    out: List[Dict[str, Any]] = []
    cur = int(stack_state["actual_nodes"])
    target = int(max(cfg.minimum_nodes, min(cfg.maximum_nodes, cur + i
    ↪ nt(d["node_delta"]))))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if target != cur:
        out.append({"manager": tag, "action": "nodes", "target": targe
    ↪ t, "direction": 1 if target > cur else -1})
    if not direct_actuation and d["change_permitted"] and any(p["actio
    ↪ n"] == "terraform_plan" for p in proposals):
        tf_dir = 1 if target > stack_state["terraform_desired_nodes"]

    ↪ else -1
        live_dir = 1 if target > cur else -1 if target < cur else 0
        if abs(target - stack_state["terraform_desired_nodes"]) >= 2 a
    ↪ nd live_dir in (0, tf_dir):
            out.append({"manager": tag, "action": "terraform_plan", "t
    ↪ arget": target,

                        "direction": 1 if target > stack_state["terraf
    ↪ orm_desired_nodes"] else -1})
    reps = [int(p["target"]) for p in proposals if p["action"] == "rep
    ↪ licas"]
    if direct_actuation:
        cur_r = int(stack_state["replicas"])

        rt = max(20, min(500, math.ceil(cur_r * float(d["load"]) / max
    ↪ (float(d["demand"]), 1e-9))))
        if (rt > cur_r and d["node_delta"] >= 0) or (rt < cur_r and d[
    ↪ "node_delta"] < 0):
            out.append({"manager": tag, "action": "replicas", "target"
    ↪ : rt, "direction": 1 if rt > cur_r else -1})

        reps = []
    if reps:
        cur_r = int(stack_state["replicas"])
        rt = None
        if d["node_delta"] >= 0 and max(reps) > cur_r:
            rt = max(reps)

        elif d["node_delta"] < 0 and min(reps) < cur_r:
            rt = min(reps)
        if rt is not None:
            out.append({"manager": tag, "action": "replicas", "target"
    ↪ : rt,
                        "direction": 1 if rt > cur_r else -1})
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    cap = float(d["power_cap"])
    if any(a["direction"] > 0 and a["action"] in ("nodes", "terraform_
    ↪ plan", "replicas") for a in out):
        cap = max(cap, float(stack_state["power_cap"]))
    if abs(cap - stack_state["power_cap"]) > 1e-9:
        out.append({"manager": tag, "action": "power_cap", "target": c
    ↪ ap,
                    "direction": -1 if cap < stack_state["power_cap"]
    ↪ else 1})
    if d["route_shift"]:
        out.append({"manager": tag, "action": "route_shift", "target":
    ↪  1.0, "direction": 1})
    if any(p["action"] == "rollback" for p in proposals) and d["rollba
    ↪ ck_authorized"]:
        out.append({"manager": tag, "action": "rollback", "target": 1.
    ↪ 0, "direction": -1})
    alerts = [p for p in proposals if p["action"] == "alert"]
    if alerts:
        out.append({"manager": tag, "action": "alert", "target": max(i
    ↪ nt(p["target"]) for p in alerts), "direction": 1})
    return out


THROUGHPUT_MODE = dict(backlog_release=True, power_ceiling=9.0, power_
    ↪ ceiling_backlog=9.0, spread_step=0.0, B_cap=9.0,
                       size_at_full_cap=True, kI=0.05, kE=0.40, cap_mi
    ↪ n=0.65, down_band=1, down_dwell=10,
                       down_after_add=3, kq=0.30, U_gate=0.45, push_re
    ↪ lease=0.01)
FLEET_MODE = dict(THROUGHPUT_MODE, rho0=0.829, rho_min=0.769, kI=0.072
    ↪ , kE=0.487, kq=0.635, down_band=3, down_dwell=1, down_after_add=
    ↪ 5, push_release=0.053, cap_min=0.695, margin=0.131, U_gate=0.85,
    ↪  up_max=8)
FLEET_BALANCED_MODE = dict(FLEET_MODE, push_release=0.0098, push_add=0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ .0098)
FLEET_WEAR_MODE = dict(FLEET_MODE, push_release=0.0035, push_add=0.003
    ↪ 5)
FLEET_DECISION_SECONDS = 60

def mode_law(mode: str, base: "AllocationLaw" = None) -> "AllocationLa
    ↪ w":
    """Operating modes.

    power_protect  the site power limit is enforced (envelope, spread-
    ↪ and-throttle, bath cap; shield I4 active)

    fleet          throughput mode with constants selected on the 15-s
    ↪ econd fleet harness (fleet/), decisions every 60 s;
                   energy-first: the equation (2) release gate is leni
    ↪ ent and the add gate is off
    fleet_balanced fleet with the equation (2) gate active in both dir
    ↪ ections at 0.0098 (release and add)

    fleet_wear     fleet with the equation (2) gate active in both dir
    ↪ ections at 0.0035 (release and add)
    throughput     the power envelope is not enforced (same power rule
    ↪ s as the native stack); caps released on backlog;
                   capacity sized at full power (load * cap_now), so p
    ↪ ower trimming does not trigger machine starts;

                   constants selected by constrained search on develop
    ↪ ment seeds (minimum energy subject to backlog,
                   health, recovery, reversal and invariant constraint
    ↪ s)
    """
    from dataclasses import replace

    base = base or AllocationLaw()
    if mode == "power_protect":
        return base
    if mode == "throughput":
        return replace(base, **THROUGHPUT_MODE)
    if mode == "fleet":
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return replace(base, **FLEET_MODE)
    if mode == "fleet_balanced":
        return replace(base, **FLEET_BALANCED_MODE)
    if mode == "fleet_wear":
        return replace(base, **FLEET_WEAR_MODE)
    raise ValueError(mode)
```

H.17 omnicompass/authority_contract.py

59 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Executable authority/evidence contracts for Omni-Compass.

This module does not claim that a plant obeyed Omni. It defines the mi
    ↪ nimum links that must

exist before software may promote a claim from COMPUTED -> REQUESTED -
    ↪ > ENFORCED -> ATTRIBUTED
-> BENEFICIAL -> RESTORED. Missing evidence can only lower a claim lev
    ↪ el, never raise it.
"""
from __future__ import annotations

from dataclasses import dataclass, asdict
from enum import IntEnum
from typing import Any, Iterable

class ClaimLevel(IntEnum):
    COMPUTED=0; REQUESTED=1; ENFORCED=2; ATTRIBUTED=3; BENEFICIAL=4; R
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ESTORED=5

@dataclass(frozen=True)
class AuthorityContract:
    muscle_id: str
    resource: str

    writer: str
    command: str
    bounds: str
    readback: str
    conflict_set: tuple[str,...]
    meter: tuple[str,...]

    guardrails: tuple[str,...]
    restore: str
    evidence_class: str
    def to_dict(self):
        d=asdict(self); d['conflict_set']=list(self.conflict_set); d['
    ↪ meter']=list(self.meter); d['guardrails']=list(self.guardrails);

    ↪  return d

GPU_POWER = AuthorityContract(
    'nvidia.gpu.power_limit','GPU board power envelope','omni_controll
    ↪ er/gpu_governor.py',
    'nvidia-smi -i <gpu> -pl <watts>','device min <= watts <= frozen s

    ↪ tart limit',
    'power.limit + enforced.power.limit',
    ('VBIOS','BMC/SMBPBI/chassis','application clocks','foreign host w
    ↪ riter'),
    ('DCGM/NVML device joules','integrated board power','optional wall
    ↪  meter','RAPL package meter (separate domain)'),

    ('successful work','SLO','no thermal/HwPowerBrake confound','Watch
    ↪  writes == 0'),
    'restore frozen starting board-power limit and read back','physica
    ↪ l')

RAPL_METER = AuthorityContract(
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    'intel.rapl.meter','CPU package energy meter','read-only',
    'read energy_uj','read-only','start/end energy_uj + max_energy_ran
    ↪ ge_uj',(),
    ('RAPL package joules','optional DRAM joules','optional psys joule
    ↪ s'),
    ('wrap-aware delta','domain labels remain separate'), 'none (read-
    ↪ only)','physical-meter')

K8S_REPLICA = AuthorityContract(
    'kubernetes.replicas','workload replica allocation','Omni Kubernet
    ↪ es adapter/native controller',
    'scale/patch replica target','declared min/max + safety envelope',
    ↪ 'observed desired/actual replicas',
    ('HPA','operator','GitOps/deployment writer'),('work','latency','p
    ↪ ending/backlog','optional node/wall energy'),
    ('SLO','health','backlog','unsafe == 0'),'restore/hand back origin
    ↪ al controller state','external-software')
CONTRACTS={x.muscle_id:x for x in (GPU_POWER,RAPL_METER,K8S_REPLICA)}

def catalogue(): return {k:v.to_dict() for k,v in CONTRACTS.items()}
```

H.18 omnicompass/ce_score.py

32 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Control-effort index. Experiment-defined. Sweep weights; don't trop
    ↪ hy one set."""
from __future__ import annotations
from typing import Dict
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
SWEEPS = {
    "balanced": dict(replica_change=1, pod_move=1, node_provision=8, n
    ↪ ode_suspend=8, resize=2, reversal=4, power_cap_travel=1),
    "node_heavy": dict(replica_change=1, pod_move=1, node_provision=16
    ↪ , node_suspend=16, resize=2, reversal=4, power_cap_travel=1),
    "replica_heavy": dict(replica_change=4, pod_move=2, node_provision
    ↪ =8, node_suspend=8, resize=2, reversal=8, power_cap_travel=1),
    "resize_heavy": dict(replica_change=1, pod_move=1, node_provision=
    ↪ 8, node_suspend=8, resize=8, reversal=4, power_cap_travel=1),
    "uniform": dict(replica_change=1, pod_move=1, node_provision=1, no
    ↪ de_suspend=1, resize=1, reversal=1, power_cap_travel=1),
}


def ce(row: Dict, weights: Dict[str, float]) -> float:
    return (
        weights.get("replica_change", 0) * float(row.get("replica_chan
    ↪ ge_units", 0))
        + weights.get("pod_move", 0) * float(row.get("evictions", 0))
        + weights.get("node_provision", 0) * float(row.get("node_start
    ↪ s", 0))
        + weights.get("node_suspend", 0) * float(row.get("node_stops",
    ↪  0))
        + weights.get("resize", 0) * float(row.get("request_changes",
    ↪ 0))
        + weights.get("reversal", 0) * float(row.get("replica_reversal
    ↪ s", 0))
        + weights.get("power_cap_travel", 0) * float(row.get("power_ca
    ↪ p_travel", 0))
    )


def all_sweeps(row: Dict) -> Dict[str, float]:
    return {name: ce(row, w) for name, w in SWEEPS.items()}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

H.19 omnicompass/closure.py

250 lines.

```
"""Closure governor: machines governed by the Omni-Compass closure law
    ↪ , preemptive coherence and the dual-bath tracker.

Sources (the owner's manuscript): Section 5 (5c preemptive coherence:
    ↪ forward projection tau_fwd = Phi_dt(x) and
correction -lambda grad psi), Chapter 20 (master closure law F = G0 +
    ↪ Gc, Gc = -rho beta Dh, zero in the interior,
inward near the boundary with margin delta), Chapters 29-30 (the wheel
    ↪ : expansion, turning point dE/dt = 0, compression;
release only after the turn, ignition before the threshold), Chapter 3
    ↪ 1 (dual-bath exchange dU/dt = -alpha E + beta S,
dE/dt = alpha U - gamma B).

Cluster realisation (one pool)
  state        r(t)  requested cores (what the pods ask for);  n machi
    ↪ nes;  c = cores x ALLOC per machine
  dual bath    level L and rate v of r, the exchange pair of Chapter 3
    ↪ 1 in discrete form:

                innovation e = r - (L + v dt)
                L <- L + v dt + a e          (coherence bath absorbs
    ↪ the observation)
                v <- (1 - g) v + b e / dt    (deviation bath exchange
    ↪ s with it; g = redistribution damping)
  projection   r_fwd(tau) = L + v tau  for tau in [0, H]          (5c:
    ↪  Phi over the look-ahead horizon)
  admissible   h(tau) = r_fwd(tau) / (n c) - rho_max  <= 0      (Chapt
    ↪ er 20: the admissible domain)
  closure Gc   if max_{tau <= H_add} h(tau) > -delta (projected bounda
    ↪ ry approach, within one boot of lead):
                add k = ceil( (max r_fwd / (rho_max - delta) - n c) /
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  c ) machines        (beta sized to restore delta)
  interior G0  energy descent: remove one machine only if it stays adm
    ↪ issible with margin for the whole release horizon,
                 max_{tau <= H_rel} r_fwd(tau) / ((n - 1) c) <= rho_ma
    ↪ x - delta,
               and only after the turn (v <= 0: past the peak, the met
    ↪ ric flip), and only while the six-state engine's
               equation (2) push <= push_release
  interior     nothing else happens: inside the domain the platform ph
    ↪ ysics runs unmodified (Gc = 0)
"""
from __future__ import annotations

import math
from dataclasses import dataclass


@dataclass(frozen=True)

class ClosureLaw:
    rho_max: float = 0.95      # admissibility boundary: requested cor
    ↪ es / allocatable cores
    delta: float = 0.05        # inwardness margin
    H_add: int = 8             # look-ahead for the closure correction
    ↪  (ticks); >= the boot delay

    H_rel: int = 40            # look-ahead a release must survive (ti
    ↪ cks)
    a: float = 0.5             # dual-bath coupling: level absorption
    b: float = 0.1             # dual-bath coupling: rate exchange
    g: float = 0.05            # redistribution damping of the rate ba
    ↪ th

    push_release: float = 0.2  # engine gate on release
    turn: bool = True          # release only after the turning point
    ↪ (v <= 0)
    delta_rel: float = -1.0    # release band: a release must fit at r
    ↪ ho_max - delta_rel (hysteresis; < 0 means = delta)
    z: float = 0.0             # deviation-bath band: a release must s
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ urvive z standard deviations of the tracker's
                               # innovation over the release horizon (
    ↪ Chapter 31: the deviation bath carries the noise;
                               # the band is wide where demand is nois
    ↪ y, tight where it is calm)
    tone: bool = False         # muscle tone: release = park (alive, l
    ↪ ow power, instant wake), not power-off; parked
                               # machines beyond the reserve needed wi
    ↪ thin tone_H are powered off
    site: bool = False         # whole body (multi-cluster sites): the
    ↪  law runs on the site total, one warm reserve for
                               # the site, and traffic shift lets a cl
    ↪ uster use another's already-powered machines
    tone_H: int = 960          # reserve horizon (ticks): the largest
    ↪ demand seen over this window sets the warm reserve
    coord: bool = False        # nervous-system coordination: pods dec
    ↪ ided first, machines on those pods, and the two
                               # organs never move in opposite directi
    ↪ ons in one decision
    wake_first: bool = False   # whole body: a site that needs a machi
    ↪ ne wakes a parked one in any cluster before it cold-
                               # boots one (traffic shift carries the
    ↪ load; a wake costs no start, a boot costs one)
    dwell: int = 0             # release only after this many consecut
    ↪ ive calm decisions (resource-aware envelope,
                               # Proposition 2: no release outside the
    ↪  calm set; 0 = no dwell requirement)
    turn_rise: float = 0.0     # the turning-point veto binds only for
    ↪  a rise larger than this share of one machine over
                               # the release horizon (v * H_rel > turn
    ↪ _rise * c); 0 = any rise vetoes (the benchmarked
                               # law). A rise smaller than the release
    ↪  band cannot carry demand through the band, so it
                               # is no reason to hold a machine (one s
    ↪ mall rise must not hold every machine for a run)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class ClosureNodes:
    def __init__(self, law: ClosureLaw = ClosureLaw()):
        self.L = None; self.v = 0.0; self.law = law; self.calm = 0; se
    ↪ lf.s2 = 0.0; self.hist = []

    def observe(self, r: float, dt: float = 1.0) -> None:
        a, b, g = self.law.a, self.law.b, self.law.g
        if self.law.tone:
            self.hist.append(r)
            if len(self.hist) > self.law.tone_H:
                self.hist.pop(0)

        if self.L is None:
            self.L = r; return
        pred = self.L + self.v * dt
        e = r - pred
        self.s2 = 0.95 * self.s2 + 0.05 * e * e
        self.L = pred + a * e

        self.v = (1.0 - g) * self.v + b * e / dt

    def reserve(self, c: float) -> int:
        """Machines the law expects to need within the tone horizon (l
    ↪ argest recent demand at the working boundary)."""
        if not self.hist:

            return 0
        return int(math.ceil(max(self.hist) / ((self.law.rho_max - sel
    ↪ f.law.delta) * c) - 1e-9))

    def fwd_max(self, H: int) -> float:
        return max(self.L + self.v * tau for tau in range(0, H + 1)) i

    ↪ f self.L is not None else 0.0

    def decide(self, n: int, c: float, push: float, n_min: int, n_max:
    ↪  int) -> int:
        """Target machine count for this tick."""
        L = self.law
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if self.L is None:
            return n
        peak_add = max(self.fwd_max(L.H_add), 0.0)
        if n <= 0 or peak_add / (n * c) - L.rho_max > -L.delta:
            self.calm = 0
            need = math.ceil(peak_add / ((L.rho_max - L.delta) * c) -
    ↪ 1e-9)
            return int(min(n_max, max(n_min, need, n)))
        peak_rel = max(self.fwd_max(L.H_rel), 0.0) + L.z * math.sqrt(s
    ↪ elf.s2 * max(1, L.H_rel)) * L.a
        band = L.delta if L.delta_rel < 0 else L.delta_rel
        calm = n - 1 >= n_min and peak_rel / ((n - 1) * c) <= L.rho_ma
    ↪ x - band \
            and (not L.turn or self.v * L.H_rel <= L.turn_rise * c) an
    ↪ d push <= L.push_release
        self.calm = self.calm + 1 if calm else 0
        if calm and self.calm > L.dwell:
            self.calm = 0
            return n - 1
        return n


# ------------------------------------------------------------------
    ↪ ------------------------------------------------

# Self-calibrating closure law: every margin is derived, none is tuned
    ↪  per workload.
#
#   tracker      dual bath (level L, rate v); the gains are chosen by
    ↪ the data: a bank of trackers from slow-trend to fast-
#                trend (a, b) runs side by side and the one with the s
    ↪ mallest running one-step prediction error drives the
#                forecast (model selection by prediction error; the fa
    ↪ stest member is the critically damped a = 1/2,
#                b = a^2 / (2 - a))
#   noise        sigma^2 = running variance of the tracker's one-step
    ↪ innovation (the deviation bath of Chapter 31)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
#   add horizon  H_add = boot delay + 1 tick: a machine ordered now is
    ↪  serving when the projection arrives
#   release hor. H_rel = boot delay / park fraction: the energy break-
    ↪ even between keeping a machine parked and booting
#                it cold again (a parked machine draws park_frac x idl
    ↪ e per tick, a cold start draws boot x idle)

#   boundary     requested cores <= allocatable cores (rho_max = 1, th
    ↪ e physical limit); no fixed margin
#   margin       z(t) sigma sqrt(H): the forecast error over the horiz
    ↪ on at confidence z;
#                z(t) = z95 x max(1, S / S*), S the six-state engine's
    ↪  stress (equation 6) and S* its equilibrium,

#                delta - alpha_s S* - (3/4) beta_s S*^2 = 0: the engin
    ↪ e widens the margin when it is stressed
#   Gc (add)     if projected peak over H_add + margin > n c: add ceil
    ↪ ((peak + margin) / c) - n machines
#   G0 (release) one machine, only if projected peak over H_rel + marg
    ↪ in over max(H_rel, T*) <= (n - 1) c (the machine is

#                not expected back within the learned break-even T* be
    ↪ low), after the turn (v <= 0), and while
#                the engine's push <= push_release; the released machi
    ↪ ne is parked
#   power-off    learned break-even (ski rental with measured return t
    ↪ imes): every release at level k opens a record

#                that closes when demand needs k machines again; X = t
    ↪ he time it took. A parked machine is powered off
#                when it has been parked T* ticks, T* = argmin_T sum_i
    ↪  cost_i(T), cost_i = park_frac X_i if X_i <= T,
#                else park_frac T + boot (energy in idle-machine ticks
    ↪ ; still-open records are right-censored at their

#                age). Before any record exists T* = boot / park_frac,
    ↪  the deterministic ski-rental rule (never more than
#                twice the energy of a controller that knows the futur
    ↪ e). Nothing is tuned: T* is computed from the
#                workload's own return times, per workload, while it r
    ↪ uns
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# ----------------------------------------------------------------------
    ↪ ------------------------------------------------
@dataclass(frozen=True)
class AutoClosureLaw:
    boot: int = 6                  # boot delay of a cold machine (tic
    ↪ ks): a property of the plant

    park_frac: float = 0.25        # parked power / idle power: a prop
    ↪ erty of the hardware
    z95: float = 1.6449            # one-sided 95% normal quantile
    a: float = 0.5
    push_release: float = 0.2
    site: bool = False

    # read by the plant (same interface as ClosureLaw)
    tone: bool = True
    rho_max: float = 1.0
    delta: float = 0.0

    @property

    def b(self) -> float:
        return self.a * self.a / (2.0 - self.a)

    @property
    def H_add(self) -> int:
        return self.boot + 1

    @property
    def H_rel(self) -> int:
        return int(math.ceil(self.boot / self.park_frac))

    @property

    def tone_H(self) -> int:
        return self.H_rel


def stress_equilibrium(delta: float, alpha_s: float, beta_s: float) ->
    ↪  float:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    """Positive root of delta - alpha_s S - (3/4) beta_s S^2 = 0 (the
    ↪ engine's stress equilibrium, equation 6)."""
    qa = 0.75 * beta_s
    return (-alpha_s + math.sqrt(alpha_s * alpha_s + 4.0 * qa * delta)
    ↪ ) / (2.0 * qa) if qa > 0 else delta / alpha_s

class AutoClosureNodes:
    def __init__(self, law: AutoClosureLaw = AutoClosureLaw(), s_eq: f
    ↪ loat = None):
        self.law = law; self.L = None; self.v = 0.0; self.s2 = 0.0; se
    ↪ lf.n_obs = 0

        self.S = 0.0; self.s_eq = s_eq if s_eq else stress_equilibrium
    ↪ (0.5, 0.12, 0.10)
        self.hist = []; self.ages = []; self.n_last = 0
        self.t = 0; self.open = []; self.closed = []   # release recor
    ↪ ds: (level, t0) open; X closed

    BANK = ((0.2, 0.002), (0.2, 0.02), (0.5, 0.005), (0.5, 0.05), (0.5
    ↪ , 1.0 / 6.0))

    def observe(self, r: float, dt: float = 1.0) -> None:
        if self.L is None:
            self.L = r; self.bank = [[r, 0.0, 0.0] for _ in self.BANK]

    ↪ ; return
        self.n_obs += 1; self.t += 1
        self.ages = [x + 1 for x in self.ages]
        w = max(1.0 / self.n_obs, 0.02)           # running mean firs
    ↪ t, then a 50-tick exponential window
        for (a, b), m in zip(self.BANK, self.bank):

            pred = m[0] + m[1] * dt
            e = r - pred
            m[2] = (1.0 - w) * m[2] + w * e * e
            m[0] = pred + a * e
            m[1] = m[1] + b * e / dt
        best = min(self.bank, key=lambda m: m[2])
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.L, self.v, self.s2 = best

    def z(self) -> float:
        return self.law.z95 * max(1.0, self.S / self.s_eq) if self.s_e
    ↪ q > 0 else self.law.z95

    def peak(self, H: int, H_noise: int = None) -> float:
        """Projected peak over H ticks (level + trend) plus the foreca
    ↪ st-error margin accumulated over H_noise ticks."""
        if self.L is None:
            return 0.0
        Hn = H if H_noise is None else H_noise

        return max(self.L + self.v * tau for tau in range(0, H + 1)) +
    ↪  self.z() * math.sqrt(self.s2 * Hn)

    GRID = (0, 6, 12, 24, 48, 96, 192, 384, 768, 1440)

    def threshold(self) -> int:

        L = self.law
        if not self.closed and not self.open:
            return L.H_rel
        cens = [self.t - t0 for _, t0 in self.open]
        def cost(T):
            c = sum(L.park_frac * x if x <= T else L.park_frac * T + L

    ↪ .boot for x in self.closed)
            return c + sum(L.park_frac * T + L.boot if a > T else L.pa
    ↪ rk_frac * a for a in cens)
        return min(self.GRID, key=lambda T: (cost(T), T))

    def reserve(self, c: float) -> int:

        """Machines to keep powered (running + parked younger than the
    ↪  learned break-even T*); older parked go off."""
        T = self.threshold()
        self.ages = [x for x in self.ages if x < T]
        return self.n_last + len(self.ages)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def decide(self, n: int, c: float, push: float, n_min: int, n_max:
↪  int) -> int:
        L = self.law
        if self.L is None:
            return n
        need = int(math.ceil(max(self.peak(L.H_add), 0.0) / c - 1e-9))

        still = []
        for lvl, t0 in self.open:            # a release record closes
↪  when demand needs its level again
            if need >= lvl:
                self.closed.append(self.t - t0)
            else:

                still.append((lvl, t0))
        self.open = still[-200:]; self.closed = self.closed[-400:]
        if n <= 0 or need > n:
            t = int(min(n_max, max(n_min, need)))
            self.ages.sort(); del self.ages[max(0, len(self.ages) - (t
↪  - n)):]     # wake the longest parked first

            self.n_last = t
            return t
        # release only if the machine is not expected back within the
↪ learned break-even T*: the trend is projected over
        # the release horizon, the forecast error is accumulated over
↪ max(H_rel, T*)

        if n - 1 >= n_min and self.peak(L.H_rel, max(L.H_rel, self.thr
↪ eshold())) <= (n - 1) * c and self.v <= 0.0 \
                and push <= L.push_release:
            self.ages.append(0); self.n_last = n - 1; self.open.append
↪ ((n, self.t))
            return n - 1

        self.n_last = n
        return n
```

H.20 `omnicompass/compass.py`

163 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
"""The Omni-Compass, computed.

I read my own state on the compass every decision and say where I am,
    ↪ in the compass's own terms.

The wheel (Chapter 29). My deviation E, scaled by its ceiling E_max, r
    ↪ uns along the horizontal; its rate of change runs
along the vertical. A reading is a point on that wheel. Its heading is
    ↪  measured clockwise from north, as on the face:

    +   A     0 deg   rising through rest
    ⇄   Δ    45 deg   expansion: above rest and rising, exchange under
    ↪  way
    >   B    90 deg   peak extension: the turn, where the metric flips
    ↪  (k -> -k)
    ⊤   Λ   135 deg   dispersion: above rest and easing, the ceiling h
    ↪ olds
    -   Ω   180 deg   falling through rest
    ≈   Π   225 deg   compression: below rest and settling into the ba
    ↪ sin
    <   Γ   270 deg   deepest compression: the turn at the floor
    ✦   Ψ   315 deg   re-alignment: below rest and rising, ignition ah
    ↪ ead

The four quadrants are the strokes of the closed circle (Chapters 29-3
    ↪ 0):
    I   Expansion     above rest, rising
    II  Dispersion    above rest, easing
    III Compression   below rest, falling
    IV  Re-alignment  below rest, rising

The wheel turns I -> II -> III -> IV -> I. Crossing from IV into I is
    ↪ ignition; the turn at peak extension is the
metric flip. The state is continuous through every crossing: nothing s
    ↪ witches off, nothing restarts.

The rim carries the 24 letters in their places on the face, 15 degrees
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  each, clockwise from A at north.

The axle is the structural basin S. Its rest point S* solves equation
    ↪ (6): delta - alpha_s S - (3/4) beta_s S^2 = 0.

Closing the circle (Chapter 1, the Unified Circle Principle):

    X' = G(X),  X(0) in Omega           Omega is the living band: ever
    ↪ y level inside [0.05, 0.95]
    G(X) . n(X) <= 0 on the boundary    at a boundary the next move po
    ↪ ints inward, never outward
    grad L(X) . G(X) <= 0               the ledger L, the engine's com
    ↪ posite storage over all six states

                                        (omnicompass.storage.composite
    ↪ _V), descends when nothing is forcing it
    => lim X(t) in M*                   the state settles into the bas
    ↪ in
I check all three every decision and report them. The ledger may rise
    ↪ while outside load forces me; I report that as

forcing, never hide it.
"""
from __future__ import annotations

import math
from dataclasses import dataclass, field

from typing import Dict, List, Optional

from omnicompass.nervous_system import BAND

RIM = ["A", "E", "Z", "Δ", "H", "Θ", "B", "I", "K", "Λ", "M", "N",
       "Ω", "Ξ", "O", "Π", "P", "Σ", "Γ", "T", "Y", "Ψ", "Φ", "X"]

POINTS = [("+", "rising through rest"), ("⇄", "expansion, exchange und
    ↪ er way"), (">", "peak extension, the turn"),
          ("⊤", "dispersion, the ceiling holds"), ("−", "falling throu
    ↪ gh rest"), ("≈", "compression, settling into the basin"),
          ("<", "deepest compression, the turn at the floor"), ("✦", "
    ↪ re-alignment, ignition ahead")]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
QUADRANTS = {1: "Expansion", 2: "Dispersion", 3: "Compression", 4: "Re
    ↪ -alignment"}


def phi(S: float, alpha_s: float, beta_s: float, delta: float) -> floa
    ↪ t:
    """Basin potential, equation (5)."""
    return alpha_s * S * S / 2.0 + beta_s * S ** 3 / 4.0 - delta * S


def axle(alpha_s: float, beta_s: float, delta: float) -> float:
    """S*, the rest point of the axle: the positive root of delta - al
    ↪ pha_s S - (3/4) beta_s S^2 = 0."""
    q = 0.75 * beta_s
    if q <= 0:
        return delta / alpha_s if alpha_s > 0 else 0.0
    return (-alpha_s + math.sqrt(alpha_s * alpha_s + 4.0 * q * delta))
    ↪  / (2.0 * q)

def heading(e: float, rate: float) -> float:
    """Degrees clockwise from north: north is rising (rate > 0), east
    ↪ is extension (e > 0)."""
    if abs(e) < 1e-12 and abs(rate) < 1e-12:
        return 0.0
    return math.degrees(math.atan2(e, rate)) % 360.0


def quadrant(e: float, rate: float) -> int:
    if e > 0:
        return 1 if rate > 0 else 2
    return 3 if rate < 0 else 4


def in_omega(levels: Dict[str, float]) -> Dict[str, bool]:
    lo, hi = BAND
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return {k: lo - 1e-9 <= v <= hi + 1e-9 for k, v in levels.items()}


def inward(level: float, move: float, eps: float = 0.01) -> bool:
    """G . n <= 0 on the boundary of Omega: at (or past) the floor the
    ↪  move is not downward, at the ceiling not upward."""

    lo, hi = BAND
    if level >= hi - eps and move > 0:
        return False
    if level <= lo + eps and move < 0:
        return False
    return True

@dataclass
class Compass:
    """Reads the engine each decision and keeps the previous reading t
    ↪ o form rates and the ledger step."""

    E_max: float = 1.0
    alpha_s: float = 0.12
    beta_s: float = 0.10
    delta: float = 0.5
    prev_e: Optional[float] = None
    prev_L: Optional[float] = None

    scale: float = 0.0           # running RMS of the rate, so the wh
    ↪ eel is round in its own units
    turns: int = 0               # full circles closed (IV -> I cross
    ↪ ings)
    last_q: Optional[int] = None
    trail: List[str] = field(default_factory=list)

    def read(self, E: float, S: float, levels: Dict[str, float] = None
    ↪ , moves: Dict[str, float] = None,
             forced: bool = False, x=None, p=None) -> Dict:
        """x, p: the engine's full state and parameters; with them the
    ↪  ledger is the composite storage of all six
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    states. Without them it is the storage of the two the reading
↪ names, E^2/2 + Phi(S) - Phi(S*)."""
    e = E / max(self.E_max, 1e-12)
    raw = 0.0 if self.prev_e is None else e - self.prev_e
    self.scale = raw * raw if self.scale == 0.0 else 0.9 * self.sc
↪ ale + 0.1 * raw * raw

    rate = raw / math.sqrt(self.scale) if self.scale > 1e-18 else
↪ 0.0
    h = heading(e, rate)
    q = quadrant(e, rate)
    if self.last_q == 4 and q == 1:
        self.turns += 1

    pt, meaning = POINTS[int(((h + 22.5) % 360) // 45)]
    letter = RIM[int(((h + 7.5) % 360) // 15)]
    s_star = axle(self.alpha_s, self.beta_s, self.delta)
    parts = {}
    if x is not None and p is not None:
        from omnicompass.storage import composite_V

        cv = composite_V(x, p, 1)
        L = cv["V"]; parts = {k: round(v, 5) for k, v in cv.items(
↪ ) if k.startswith("V_")}
        s_star = cv["S_star"]
    else:
        L = 0.5 * e * e + phi(S, self.alpha_s, self.beta_s, self.d

↪ elta) - phi(s_star, self.alpha_s, self.beta_s, self.delta)
    dL = None if self.prev_L is None else L - self.prev_L
    levels = levels or {}; moves = moves or {}
    omega = in_omega(levels)
    lo, hi = BAND
    outside = {k: ("below the floor" if levels[k] < lo else "above

↪  the ceiling") for k, ok in omega.items() if not ok}
    heading_in = {k: (moves.get(k, 0.0) > 0 if levels[k] < lo else
↪  moves.get(k, 0.0) < 0) for k in outside}
    inw = {k: inward(levels[k], moves.get(k, 0.0)) for k in levels
↪ }
    r = {"heading_deg": round(h, 1), "letter": letter, "point": pt
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ , "meaning": meaning,
              "quadrant": q, "stroke": QUADRANTS[q], "e": round(e, 4),
    ↪ "rate": round(rate, 3),
              "axle_S": round(S, 4), "axle_rest": round(s_star, 4), "le
    ↪ dger": round(L, 5),
              "ledger_step": None if dL is None else round(dL, 5), "led

    ↪ ger_parts": parts,
              "descent": None if dL is None else (dL <= 1e-9 or forced)
    ↪ , "forced": forced,
              "omega_held": all(omega.values()) if omega else True, "in
    ↪ ward": all(inw.values()) if inw else True,
              "outside": outside, "returning": heading_in,

              "circles_closed": self.turns}
          self.prev_e, self.prev_L, self.last_q = e, L, q
          return r


def omega_words(r: Dict) -> str:

    if r["omega_held"]:
        return "Ω held"
    parts = [f"{k} {w}, {'returning' if r['returning'].get(k) else 'he
    ↪ ld'}" for k, w in r["outside"].items()]
    return "Ω: " + "; ".join(parts)

def say(r: Dict) -> str:
    """One line in the compass's own terms."""
    step = "" if r["ledger_step"] is None else f" {r['ledger_step']:+.
    ↪ 4f}{' (forced)' if r['forced'] and r['ledger_step'] > 0 else ''}
    ↪ "

    return (f"{r['letter']} {r['point']} {r['heading_deg']:5.1f}° {r['
    ↪ stroke']}: {r['meaning']} | "
            f"{omega_words(r)} | G·n≤0 {'yes' if r['inward'] else 'NO'
    ↪ } | "
            f"L {r['ledger']:.4f}{step} | axle S {r['axle_S']} (rest {
    ↪ r['axle_rest']}) | circles {r['circles_closed']}")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

H.21 omnicompass/compass_law.py

176 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The compass: one smooth law for every muscle, and the plug every mu
    ↪ scle is wired through.

A muscle is a plug with one wire in and one wire out. The plug reads i

    ↪ ts meters and writes its lever; all the
thinking is here, in the brain, written once for every muscle.

The plug (Plug)
  read      the service reading (response time, temperature, queue) an
    ↪ d the lever as it stands

  write     one value; the plug clips it to the cover, sends it, and r
    ↪ eads back what the device took
  restore   the lever back to the snapshot, the value read once before
    ↪  the first write; it never moves after that
  one writer if the lever is found at a value the brain did not write,
    ↪  someone else owns it: the brain stops writing

            and leaves that value alone (restoring over it would fight
    ↪  the new owner)

Two bands
  cover     the outer band, the lever's hard range (the device's or th
    ↪ e buyer's lowest and highest setting); every
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            write is clipped to it, so nothing the math does can set t
    ↪ he lever outside
  compass      the service reading as a position in its band, 0 to 1 (
    ↪ 0: calm, 1: the service line); the brain pulls it
            to the bottom of the compass, the center (0.5 by default);
    ↪  the walls run to 0.05 and 0.95, and the last 5% on

            each side is cushion (10% in all) before the line

The force (CompassLaw.force), the physics of a ball in a compass with
    ↪ the right friction
  pull      K_P x (position - center): gentle near the bottom, harder
    ↪ up the walls

  push      K_D x velocity: whatever is shoving the position (a load r
    ↪ ising, heat building) is met by an equal and
            opposite push; it is also the friction that stops the ball
    ↪  sloshing past the bottom (with K_D chosen for
            critical damping it glides to the center and stops, no ove
    ↪ rshoot, no ringing)

  smooth    the force is A x tanh(raw / A): a spring near the center t
    ↪ hat bends over and flattens into its maximum A,
            never a corner, never a hammer; the same shape the engine
    ↪ already uses for v_eff (omnicompass/core.py)
  fail up   past the 0.95 wall the up side goes to its full force at o
    ↪ nce, and the down side may not act until the

            position is back inside the compass

Two forces (antagonist pairs)
  up        a positive force adds capacity, power, cooling or speed (s
    ↪ cale out, raise a clock, start a chiller)
  down      a negative force takes it back (scale in, lower a limit, w

    ↪ arm a setpoint)
  each side has its own gain (lever units per unit of force), because
    ↪ adding and taking back do not cost the same;
  a muscle with two wires (a GPU's clock range and power limit) gives
    ↪ each side its own lever; a one-wire muscle
  carries both directions on its one lever
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
The frozen engine (omnicompass/core.py) and its proofs are untouched;
    ↪ this is a separate law, run as its own arm.
"""
from __future__ import annotations

import math
from dataclasses import dataclass, field
from typing import Optional


def clamp(v, lo, hi):

    return lo if v < lo else hi if v > hi else v


@dataclass
class Band:
    """A reading's band: lo is calm (position 0), hi is the line (posi

    ↪ tion 1)."""
    lo: float
    hi: float
    center: float = 0.5
    cushion: float = 0.05

    def position(self, x: float) -> float:
        return (x - self.lo) / (self.hi - self.lo)


    @property
    def wall_low(self) -> float:
        return self.cushion

    @property
    def wall_high(self) -> float:
        return 1.0 - self.cushion

@dataclass
class CompassLaw:
    """The brain's law for one muscle. dt: seconds between decisions;
    ↪ tau: the muscle's response time in seconds
    (how long the service reading takes to follow the lever); kd defau
    ↪ lts to critical damping for that response."""
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    band: Band
    dt: float
    tau: float
    kp: float = 1.0
    kd: Optional[float] = None
    authority: float = 1.0

    smooth: float = 0.5          # the velocity estimate's smoothing (
    ↪ 0: raw difference, near 1: heavily smoothed)
    p: Optional[float] = field(default=None, init=False)
    v: float = field(default=0.0, init=False)

    def __post_init__(self):

        if self.kd is None:
            # a first-order muscle with response tau under a P-D pull:
    ↪  critical damping when kd = 2 sqrt(kp tau) - 1
            # scaled to the decision period; never negative
            self.kd = max(0.0, 2.0 * math.sqrt(self.kp * self.tau / se
    ↪ lf.dt) - 1.0) * self.dt

    def sense(self, reading: float) -> float:
        p = self.band.position(reading)
        if self.p is not None:
            self.v = self.smooth * self.v + (1.0 - self.smooth) * (p -
    ↪  self.p) / self.dt

        self.p = p
        return p

    def force(self, reading: float) -> float:
        """Positive: the up side (add capacity); negative: the down si
    ↪ de (take it back). Bounded by the authority."""

        p = self.sense(reading)
        if p >= self.band.wall_high:
            return self.authority                                    # f
    ↪ ail up: past the wall, full force up at once
        raw = self.kp * (p - self.band.center) + self.kd * self.v
        f = self.authority * math.tanh(raw / self.authority)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if f < 0.0 and self.v > 0.0 and p > self.band.center:
            f = 0.0                                                 # n
    ↪ ever take back while it climbs above the center
        return f

class ForeignWriter(Exception):
    pass


class Plug:
    """One wire in, one wire out. Subclasses implement _read_service,
    ↪ _read_lever and _send; everything else is here."""

    def __init__(self, lo: float, hi: float, tolerance: float = 1e-6):
        self.lo, self.hi, self.tol = lo, hi, tolerance
        self.snapshot: Optional[float] = None
        self.last_written: Optional[float] = None

        self.owned = True
        self.writes = 0
        self.clipped = 0

    # the wires ------------------------------------------------------
    ↪ ---------------------------------------------

    def _read_service(self) -> float:
        raise NotImplementedError

    def _read_lever(self) -> float:
        raise NotImplementedError

    def _send(self, value: float) -> None:
        raise NotImplementedError

    # the contract ---------------------------------------------------
    ↪ ---------------------------------------------
    def attach(self) -> float:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        """Read the lever once, before any write: the restore point. I
    ↪ t never moves after this."""
        self.snapshot = self._read_lever()
        return self.snapshot

    def read(self) -> float:
        return self._read_service()

    def lever(self) -> float:
        return self._read_lever()

    def write(self, value: float) -> Optional[float]:
        """Clip to the cover, check nobody else moved the lever, send,
    ↪  read back. None: not written (not owned)."""
        if not self.owned:
            return None
        now = self._read_lever()
        expected = self.last_written if self.last_written is not None
    ↪ else self.snapshot
        if expected is not None and abs(now - expected) > self.tol:
            self.owned = False                                      # s
    ↪ omeone else moved it: it is theirs now
            raise ForeignWriter(f"lever at {now}, last written {expect
    ↪ ed}")
        v = clamp(value, self.lo, self.hi)
        self.clipped += int(v != value)
        if abs(v - now) <= self.tol:
            return now
        self._send(v)
        self.last_written = self._read_lever()
        self.writes += 1
        return self.last_written

    def restore(self) -> bool:
        """The lever back to the snapshot. Left alone when someone els
    ↪ e owns it. True: at the snapshot (or not ours)."""
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if not self.owned or self.snapshot is None:
            return True
        if abs(self._read_lever() - self.snapshot) > self.tol:
            self._send(self.snapshot)
        self.last_written = None
        return abs(self._read_lever() - self.snapshot) <= self.tol
```

H.22 omnicompass/configurations.py

70 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Named engine configurations.

Default remains omnicompass.core (symmetric_verified). This module doe
    ↪ s not

replace it. printed_eight_line is the marketing-plate transcription.

  symmetric_verified  claim-2 / package core
    dE = -alpha_E E + beta_int + beta_ext + v_eff
    dU = mu U (1-U^2) - dE/E_max - lambda_U U + u
    dI = 1-U - sigma_1 E - delta S - lambda_I I_U

    v  = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U-0.5) + l
    ↪ ambda_2 S)

  printed_eight_line  plate species (not the held-out evidence core)
    dE = -alpha E + beta_int + beta_ext + v_eff
    dU = alpha (1-U) - dE/E_max - k U (1-U) + u
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    dI = 1-U - sigma_1 E - delta S
    v  = c * tanh(lambda_0 + lambda_1 (U-U_t) + lambda_2 S)
"""
from __future__ import annotations

import math

from typing import Callable

from .core import EPS, KP, U_AUTHORITY, Params, State, derivatives as
    ↪ core_derivatives, v_eff as core_v_eff

SYMMETRIC = "symmetric_verified"
PRINTED = "printed_eight_line"
DEFAULT = SYMMETRIC


def printed_v_eff(x: State, p: Params, t: float = 0.0, U_t: float = 0.
    ↪ 5) -> float:
    z = p.lambda_0 + p.lambda_1 * (x.U - U_t) + p.lambda_2 * x.S
    return p.c * math.tanh(z)


def printed_derivatives(x: State, p: Params, t: float, u: float = 0.0,
    ↪  U_t: float = 0.5) -> State:
    v = printed_v_eff(x, p, t, U_t)
    dE = -p.alpha * x.E + p.beta_int + p.beta_ext + v
    dU = p.alpha * (1.0 - x.U) - dE / max(p.E_max, 1e-6) - p.k * x.U *
    ↪  (1.0 - x.U) + u
    dI = (1.0 - x.U) - p.sigma_1 * x.E - p.delta * x.S
    dS = p.delta - p.alpha_s * x.S - 0.75 * p.beta_s * x.S * x.S
    dBd = p.gamma_c * p.delta * x.S - (p.omega_B / max(p.Q_B, EPS)) *
    ↪ x.B_dot - p.omega_B ** 2 * x.B
    return State(dE, dU, dI, dS, x.B_dot, dBd)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def derivatives_for(name: str) -> Callable:
    if name == SYMMETRIC:
        return core_derivatives
    if name == PRINTED:
        return printed_derivatives
    raise ValueError(name)

def control_command_for(name: str, x: State, p: Params, t: float, targ
    ↪ et: float):
    deriv = derivatives_for(name)
    drift = deriv(x, p, t, 0.0).U

    raw = -drift + KP * (float(target) - x.U)
    return max(-U_AUTHORITY, min(U_AUTHORITY, raw)), raw


def describe(name: str) -> str:
    return {

        SYMMETRIC: "package / claim-2 cubic U; owns held-out evidence"
    ↪ ,
        PRINTED: "plate logistic U; executable; not the default",
    }[name]
```

H.23 omnicompass/conveyance.py

101 lines.

```
"""Conveyance law: moving a conserved budget ("juice") between organs.

Manuscript sources (interpretation of the English, not only the symbol
    ↪ s):
  Ch. 29 §8   E_system = E + U + S + B, dE_system/dt = 0 over the cycl
    ↪ e: "Energy redistributes. Energy reorganizes.

              Energy does not diverge."
    ↪     -> the budget is conserved, only moved
  Ch. 31 §4   "When E grows locally, B increases to distribute gradien
    ↪ ts." -> flow runs down the gradient of need
  Ch. 31 §5   stability requires Re(lambda_i) <= 0
    ↪       -> the flow must provably converge
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
  Ch. 30 §5-7 ignition is a "redistribution surge ... controlled relea
    ↪ se", with state continuity (no discontinuity)

    ↪       -> the reserve is released continuously
  Ch. 29 §6   compression: "backpressure develops through S ... struct
    ↪ ured inward rotation"
    ↪       -> surplus is pulled back smoothly

The law. Organs i hold allocations a_i of a budget A (watts, cores, ..
    ↪ .), have need d_i (the budget that would serve
their work at the engine's utilisation target rho), and a deficit ("lo
    ↪ cal deviation")
        e_i = d_i / (rho a_i) - 1        (> 0 needs juice, < 0 holds s
    ↪ urplus)
The redistribution mediator moves budget by the replicator flow
        da_i/dt = kappa a_i (e_i - e_bar),       e_bar = sum_j a_j e_j
    ↪  / A

Properties (proved in docs/CONVEYANCE_LAW.md, checked in tests/test_co
    ↪ nveyance.py):
  P1 conservation      sum_i da_i/dt = 0 exactly: the budget is moved,
    ↪  never created
  P2 gradient flow     it is the Shahshahani gradient of F(a) = sum_i
    ↪ (d_i / rho) ln a_i on the simplex sum a_i = A;
                       F is strictly concave, so F rises monotonically
    ↪  (Lyapunov function)
  P3 convergence       unique equilibrium a_i* = A d_i / D (D = sum d_
    ↪ i): every organ at the same deficit e*,
                       reached exponentially: da_i/dt = (kappa/rho)(d_
    ↪ i - a_i D/A), rate kappa D / (rho A), and the
                       linearisation has eigenvalues -kappa D/(rho A)
    ↪ < 0 (Re(lambda) <= 0, Ch. 31 §5)
  P4 bounds            organ floors and ceilings (device minimum, TDP)
    ↪  are kept by projection; budget an organ cannot
                       take because it is at its ceiling returns to th
    ↪ e others (water-filling), and budget left over
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                        when every organ is at its need is held back as
    ↪  reserve, not spent
Reserve (Ch. 30 surge): R = the budget needed to cover the demand rise
    ↪  the rate tracker projects over one actuation
delay; released continuously to organs whose deficit stays positive af
    ↪ ter the flow."""

from __future__ import annotations

from typing import List, Sequence


def deficits(a: Sequence[float], d: Sequence[float], rho: float) -> Li

    ↪ st[float]:
    return [di / (rho * max(ai, 1e-12)) - 1.0 for ai, di in zip(a, d)]


def replicator_step(a: Sequence[float], d: Sequence[float], rho: float
    ↪ , kappa: float, dt: float = 1.0) -> List[float]:

    """One exact step of da_i/dt = (kappa/rho)(d_i - a_i D/A), the clo
    ↪ sed form of the replicator flow (P3):
    a_i(t+dt) = a_i* + (a_i - a_i*) exp(-kappa D dt / (rho A)); conser
    ↪ ves sum a_i to rounding (P1)."""
    import math
    A = sum(a); D = sum(d)

    if A <= 0 or D <= 0:
        return list(a)
    k = math.exp(-kappa * D * dt / (rho * A))
    return [A * di / D + (ai - A * di / D) * k for ai, di in zip(a, d)
    ↪ ]

def project(a: Sequence[float], lo: Sequence[float], hi: Sequence[floa
    ↪ t], budget: float) -> List[float]:
    """Water-filling projection onto {lo_i <= a_i <= hi_i, sum a_i <=
    ↪ budget}: organs pinned at a bound keep it, the
    rest share the remainder in proportion to their unconstrained allo
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ cation (P4). Exact in <= n rounds."""
    a = [min(max(x, l), h) for x, l, h in zip(a, lo, hi)]
    for _ in range(len(a) + 1):
        tot = sum(a)
        if tot <= budget + 1e-9:
            return a

        free = [i for i in range(len(a)) if a[i] > lo[i] + 1e-12]
        over = tot - budget
        share = sum(a[i] - lo[i] for i in free)
        if share <= 0:
            return a
        a = [max(lo[i], a[i] - over * (a[i] - lo[i]) / share) if i in

    ↪ free else a[i] for i in range(len(a))]
    return a


class Conveyance:
    """The budget law for a set of organs sharing one budget (e.g. GPU

    ↪  groups under one site power cap).
    step(need, lo, hi) returns allocations; the flow moves budget towa
    ↪ rd need, the projection keeps bounds, and the
    reserve holds back what the projected rise needs, releasing it con
    ↪ tinuously where deficits stay positive."""

    def __init__(self, n: int, budget: float, rho: float = 0.8, kappa:
    ↪  float = 1.0):
        self.budget, self.rho, self.kappa = budget, rho, kappa
        self.a = [budget / n] * n
        self.prev = None
        self.rate = [0.0] * n

    def step(self, need: Sequence[float], lo: Sequence[float], hi: Seq
    ↪ uence[float]) -> List[float]:
        # the living band: every organ keeps at least 5% of its range
    ↪ (idle, never off) and never takes more than 95%
        from omnicompass.nervous_system import BAND
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        lo = [max(l, BAND[0] * h) for l, h in zip(lo, hi)]
        hi = [max(l, BAND[1] * h) for l, h in zip(lo, hi)]
        need = [max(x, 1e-9) for x in need]
        if self.prev is not None:           # rate tracker for the res
    ↪ erve (the dual-bath rate channel)
            self.rate = [0.5 * r + 0.5 * (x - p) for r, x, p in zip(se
    ↪ lf.rate, need, self.prev)]
        self.prev = list(need)
        reserve = min(0.5 * self.budget, sum(max(0.0, r) for r in self
    ↪ .rate))
        want = [min(h, x / self.rho) for x, h in zip(need, hi)]
    ↪    # budget each organ can use at rho

        want = [max(w, l) for w, l in zip(want, lo)]
    ↪      # a device cannot go below its floor
        spend = max(sum(lo), min(self.budget - reserve, sum(want)))
    ↪     # never spend budget nobody needs
        a = replicator_step(self.a, [w * self.rho for w in want], self
    ↪ .rho, self.kappa)

        a = [x * spend / max(sum(a), 1e-12) for x in a]
        a = project(a, lo, [max(l, min(h, w)) if sum(want) <= spend el
    ↪ se h for l, h, w in zip(lo, hi, want)], spend)
        # surge (Ch. 30): release the reserve continuously to organs s
    ↪ till in deficit, up to their ceiling
        left = self.budget - sum(a)

        short = [max(0.0, min(h, x / self.rho) - ai) for x, h, ai in z
    ↪ ip(need, hi, a)]
        tot = sum(short)
        if tot > 0 and left > 0:
            a = [ai + min(s, left * s / tot) for ai, s in zip(a, short
    ↪ )]

        self.a = a
        return a
```

H.24 `omnicompass/core.py`

240 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
"""Omni-Compass core: six-state ODE, RK4 integrator, bounded feedforwa
    ↪ rd-plus-proportional controller.

State x = (E, U, I_U, S, B, B_dot).

  (1) dE/dt    = -alpha_E E + beta_int + beta_ext + v_eff

  (2) dU/dt    = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u,   |u
    ↪ | <= U_AUTHORITY
  (3) dI_U/dt  = (1 - U) - sigma_1 E - delta S - lambda_I I_U
  (4) v_eff    = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U
    ↪ - 0.5) + lambda_2 S)
  (5) Phi(S)   = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S

  (6) dS/dt    = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
  (7) dB/dt    = B_dot ;  dB_dot/dt = gamma_c delta S - (omega_B/Q_B)
    ↪ B_dot - omega_B^2 B
  (8) R_B[n]   = finite-difference audit of (7); never fed back into t
    ↪ he state.

Controller (one micro step, zero-order hold across all four RK4 stages
    ↪ ):
  u = clip(-f_U(x, t) + KP (sigma - U), -U_AUTHORITY, +U_AUTHORITY)
where f_U is the uncontrolled U drift and sigma in {-1, +1} is the loc
    ↪ ked target.
"""

from __future__ import annotations

import math
from dataclasses import dataclass, asdict, fields
from typing import Dict, List, Optional, Tuple

EPS = 1e-12
KP = 12.0
U_AUTHORITY = 25.0
MACRO_STEPS = 20
MICRO_STEPS = 10
MACRO_DT = 0.1
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
MICRO_DT = MACRO_DT / MICRO_STEPS
BASIN_TOL = 0.10
CONVEY_WINDOW = 5
CERT_WINDOW = 10

PARAMETER_RANGES: Dict[str, Tuple[float, float]] = {
    "alpha": (4.2, 4.2), "beta_int": (0.0, 1.0), "beta_ext": (0.0, 1.0
    ↪ ), "k": (1.7, 1.7),
    "sigma_1": (0.38, 0.38), "delta": (0.01, 1.0), "gamma_1": (1.35, 1
    ↪ .35), "gamma_c": (1.0, 1.0),
    "lambda_0": (-2.0, 2.0), "lambda_1": (0.1, 3.0), "lambda_2": (0.0,
    ↪  2.0), "c": (0.5, 5.0),
    "E_max": (1.0, 10.0), "omega_B": (0.1, 5.0), "Q_B": (0.5, 10.0), "
    ↪ alpha_s": (0.05, 0.25),
    "beta_s": (0.05, 0.25), "mu": (2.0, 6.0), "alpha_E": (4.2, 4.2), "
    ↪ alpha_U": (4.2, 4.2),
    "lambda_I": (0.5, 2.0), "lambda_U": (0.0, 0.5),
}
# alpha, k, gamma_1, alpha_U do not enter equations (1)-(8).


@dataclass
class State:
    E: float
    U: float
    I_U: float
    S: float
    B: float
    B_dot: float
    def as_tuple(self) -> Tuple[float, ...]:
        return (self.E, self.U, self.I_U, self.S, self.B, self.B_dot)


@dataclass
class Params:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    alpha: float = 4.2
    beta_int: float = 0.0
    beta_ext: float = 0.0
    k: float = 1.7
    sigma_1: float = 0.38
    delta: float = 0.5

    gamma_1: float = 1.35
    gamma_c: float = 1.0
    lambda_0: float = 0.0
    lambda_1: float = 1.0
    lambda_2: float = 0.5
    c: float = 1.0

    E_max: float = 2.0
    omega_B: float = 1.0
    Q_B: float = 2.5
    alpha_s: float = 0.12
    beta_s: float = 0.10
    mu: float = 4.0

    alpha_E: float = 4.2
    alpha_U: float = 4.2
    lambda_I: float = 1.0
    lambda_U: float = 0.1

# ------------------------------------------------------------------ eq
    ↪ uations
def v_eff(x: State, p: Params, t: float = 0.0) -> float:
    z = p.lambda_0 + p.lambda_1 * (x.U - 0.5) + p.lambda_2 * x.S
    return math.cos(0.5 * p.omega_B * t) * (p.c * math.tanh(z))

def phi(S: float, p: Params) -> float:
    return 0.5 * p.alpha_s * S * S + 0.25 * p.beta_s * S ** 3 - p.delt
    ↪ a * S

def s_flow(S: float, p: Params) -> float:
    return p.delta - p.alpha_s * S - 0.75 * p.beta_s * S * S


def derivatives(x: State, p: Params, t: float, u: float = 0.0) -> Stat
    ↪ e:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    v = v_eff(x, p, t)
    dE = -p.alpha_E * x.E + p.beta_int + p.beta_ext + v
    dU = p.mu * x.U * (1.0 - x.U * x.U) - dE / max(p.E_max, 1e-6) - p.
    ↪ lambda_U * x.U + u
    dI = (1.0 - x.U) - p.sigma_1 * x.E - p.delta * x.S - p.lambda_I *
    ↪ x.I_U

    dS = s_flow(x.S, p)
    dBd = p.gamma_c * p.delta * x.S - (p.omega_B / max(p.Q_B, EPS)) *
    ↪ x.B_dot - p.omega_B ** 2 * x.B
    return State(dE, dU, dI, dS, x.B_dot, dBd)

def _axpy(x: State, k: State, h: float) -> State:
    return State(x.E + h * k.E, x.U + h * k.U, x.I_U + h * k.I_U, x.S
    ↪ + h * k.S, x.B + h * k.B, x.B_dot + h * k.B_dot)


def rk4_step(x: State, p: Params, t: float, h: float, u: float = 0.0)

    ↪ -> State:
    k1 = derivatives(x, p, t, u)
    k2 = derivatives(_axpy(x, k1, 0.5 * h), p, t + 0.5 * h, u)
    k3 = derivatives(_axpy(x, k2, 0.5 * h), p, t + 0.5 * h, u)
    k4 = derivatives(_axpy(x, k3, h), p, t + h, u)
    c = h / 6.0

    return State(*(a + c * (b1 + 2 * b2 + 2 * b3 + b4) for a, b1, b2,
    ↪ b3, b4 in
                   zip(x.as_tuple(), k1.as_tuple(), k2.as_tuple(), k3.
    ↪ as_tuple(), k4.as_tuple())))

def bath_residual(prev: State, cur: State, nxt: State, p: Params, h: f
    ↪ loat) -> float:
    d2 = (nxt.B - 2.0 * cur.B + prev.B) / (h * h)
    dm = (cur.B - prev.B) / h
    return d2 + (p.omega_B / max(p.Q_B, EPS)) * dm + p.omega_B ** 2 *
    ↪ cur.B - p.gamma_c * p.delta * cur.S

# ------------------------------------------------------------------ co
    ↪ ntroller
def control_command(x: State, p: Params, t: float, target: int) -> Tup
    ↪ le[float, float]:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    """Return (u, raw) for one micro step. raw is the unclipped comman
    ↪ d."""
    drift = derivatives(x, p, t, 0.0).U
    raw = -drift + KP * (float(target) - x.U)
    return max(-U_AUTHORITY, min(U_AUTHORITY, raw)), raw

def basin_sign(U: float) -> int:
    if abs(U - 1.0) <= BASIN_TOL:
        return 1
    if abs(U + 1.0) <= BASIN_TOL:
        return -1

    return 0


def nearest_basin_sign(U: float) -> int:
    return 1 if abs(U - 1.0) <= abs(U + 1.0) else -1

def macro_step(x: State, p: Params, t0: float, target: Optional[int],
               micro_steps: int = MICRO_STEPS, macro_dt: float = MACRO
    ↪ _DT) -> Tuple[State, Dict[str, float]]:
    """Advance one macro interval. target=None runs the plant with u=0
    ↪  (observe)."""

    h = macro_dt / micro_steps
    t = t0
    u_abs = u_sq = u_peak = 0.0
    sat = 0
    for _ in range(micro_steps):
        if target is None:

            u = raw = 0.0
        else:
            u, raw = control_command(x, p, t, target)
        u_abs += abs(u) * h
        u_sq += u * u * h
        u_peak = max(u_peak, abs(u))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        sat += int(abs(raw) > U_AUTHORITY + EPS)
        x = rk4_step(x, p, t, h, u)
        t += h
    return x, {"u_abs": u_abs, "u_sq": u_sq, "u_peak": u_peak, "satura
    ↪ ted": sat}

# ------------------------------------------------------------------ tr
    ↪ ajectory
@dataclass
class Result:
    final: State

    U_hist: List[float]
    target_sign: int
    born: int
    convey: int
    convey_start: int
    convey_confirm: int

    cert: int
    cert_start: int
    cert_complete: int
    max_streak: int
    actuator_abs_integral: float
    actuator_sq_integral: float

    actuator_peak: float
    actuator_saturated_periods: int


def simulate(x0: State, p: Params, target: Optional[int] = None, contr
    ↪ olled: bool = True) -> Result:

    """Canonical 20-macro-step trajectory with CONVEY-5 / CERT-10 pred
    ↪ icates.

    target defaults to the nearest basin of U0. controlled=False runs
    ↪ u=0.
    """
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    tgt = nearest_basin_sign(x0.U) if target is None else int(target)
    b0 = basin_sign(x0.U)
    born = int(b0 != 0)
    streak = 1 if born else 0
    ssign = b0 if born else 0
    best = streak

    convey = cert = 0
    cs = cc = ts = tc = -1
    ua = usq = upk = 0.0
    nsat = 0
    x = x0
    hist = [x0.U]

    for m in range(1, MACRO_STEPS + 1):
        x, a = macro_step(x, p, (m - 1) * MACRO_DT, tgt if controlled
    ↪ else None)
        ua += a["u_abs"]; usq += a["u_sq"]; upk = max(upk, a["u_peak"]
    ↪ ); nsat += a["saturated"]
        hist.append(x.U)

        b = basin_sign(x.U)
        if b:
            if b == ssign:
                streak += 1
            else:
                ssign, streak = b, 1

        else:
            ssign, streak = 0, 0
        best = max(best, streak)
        if not convey and streak >= CONVEY_WINDOW:
            convey, cs, cc = 1, m - CONVEY_WINDOW + 1, m
        if not cert and streak >= CERT_WINDOW:

            cert, ts, tc = 1, m - CERT_WINDOW + 1, m
    return Result(x, hist, tgt, born, convey, cs, cc, cert, ts, tc, be
    ↪ st, ua, usq, upk, nsat)


def sample_state(rng) -> State:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return State(float(rng.uniform(0.0, 1.0)), float(rng.uniform(-1.0,
    ↪  1.0)), float(rng.uniform(0.0, 1.0)),
                 float(rng.uniform(-1.0, 1.0)), float(rng.uniform(-1.0
    ↪ , 1.0)), float(rng.uniform(-1.0, 1.0)))

def sample_params(rng) -> Params:
    return Params(**{k: float(rng.uniform(lo, hi)) for k, (lo, hi) in
    ↪ PARAMETER_RANGES.items()})


def s_roots(p: Params) -> Tuple[float, float]:
    """Roots of equation (6): S_minus < 0 < S_plus."""
    a = 0.75 * p.beta_s
    d = math.sqrt(p.alpha_s ** 2 + 4.0 * a * p.delta)
    return (-p.alpha_s - d) / (2.0 * a), (-p.alpha_s + d) / (2.0 * a)
```

H.25 omnicompass/disruption_budget.py

31 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Karpenter-shaped voluntary node-down budget.

Not Karpenter. Caps how many nodes Omni may remove in a window.
allowed ≈ ceil(n0 * pct/100), then optional absolute ceiling.

"""
from __future__ import annotations
import math


def allowed_downs(n0: int, pct: float, abs_cap: int = 0) -> int:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    n0 = max(0, int(n0))
    raw = int(math.ceil(n0 * max(0.0, float(pct)) / 100.0)) if pct els
    ↪ e n0
    if abs_cap and abs_cap > 0:
        raw = min(raw, int(abs_cap))
    return max(0, raw)

def apply_node_delta(delta: int, used: int, allow: int) -> tuple[int,
    ↪ int, bool]:
    """Return (delta, used, blocked). Only clips negative (voluntary s
    ↪ hrink)."""

    if delta >= 0:
        return int(delta), int(used), False
    room = max(0, int(allow) - int(used))
    if room <= 0:
        return 0, int(used), True
    take = max(delta, -room)  # delta is negative

    return int(take), int(used) - int(take), False
```

H.26 omnicompass/edge.py

107 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The v5 edge: the compass's balance point is not a fixed middle, it
    ↪ moves with each muscle's own risk.

One Edge per knob. Every decision it reads the service position (0 cal
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ m, 1 the service line) and keeps the cushion the
knob needs in front of the line:

    cushion = climb x reaction + z x wobble

  climb     the largest rise of the position seen within one reaction
    ↪ time, per decision (how fast the service has been
            climbing lately); times the reaction, it is the rise that
    ↪ can land before the knob's own action does
  reaction  the decisions the knob takes to act: its own, not the plan
    ↪ t's (a power cap answers in one decision; a machine
            pool only once its machines have started)
  wobble    the step-to-step spread of the position over the window (d
    ↪ ivided by root 2: the spread of one step)
  z         3.09: 99.9% of steps inside the cushion

It widens at once when the need rises and narrows slowly when it falls
    ↪  (a driver doubles the following distance the
moment rain starts, and closes it slowly after). A fall (the service o
    ↪ ver its line) while, or soon after, Omni held
something back teaches the knob a floor under its cushion, forgotten s
    ↪ lowly. With memory, the worst climb and wobble the
knob has shown are kept as long as its own physics carries them.

The balance point is 1 - cushion; the wall (where the law fails up at
    ↪ once) moves with the reaction part of it. The
realms use this through realms/compass_arm_v5.py; the live controllers
    ↪  through omni_controller/ (--law v5). Pure math,
no I/O; the frozen engine (omnicompass/core.py) is untouched.

  pace      (omni-v16) the length of one reading in the knob's referen
    ↪ ce steps: the window, the narrowing, the floor's
            forgetting and the memory are given per reference step and
    ↪  kept so in time, and the wobble is the spread of
            the change over one reference step, so a knob read more of
    ↪ ten keeps every rule's time and only reads more
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            often. The Kubernetes governor's reference step is its min
    ↪ ute (pace = its decision period / 60 s); at pace 1
            every value is the one given, as every engine before omni-
    ↪ v16 read it
"""
from __future__ import annotations

import math


class Edge:
    def __init__(self, react: float, z: float = 3.09, window: int = 20
    ↪ , narrow: float = 0.05, start: float = 0.5,
                 memory: float = 0.0, fall_decay: float = 0.9995, est:
    ↪  int = 2, pace: float = 1.0):
        self.react = max(1.0, float(react))
        self.z, self.window, self.narrow, self.est = z, int(window), n
    ↪ arrow, est

        self.mem = memory
        self.fall_decay = fall_decay
        self.lag = 1                            # readings in one refere
    ↪ nce step (the wobble's and the climb's step)
        if pace != 1.0:
            # every count and rate given per reference step, kept in t

    ↪ ime at this pace (omni-v16)
            self.window = max(int(window), int(round(window / pace)))
            self.narrow = 1.0 - (1.0 - narrow) ** pace
            self.fall_decay = fall_decay ** pace
            self.mem = memory ** pace if memory > 0 else 0.0
            self.lag = max(1, int(round(1.0 / pace)))

        self.hist = []
        self.cushion = 1.0 - start
        self.react_part = 0.05
        self.climb_mem = 0.0
        self.wob_mem = 0.0
        self.floor = 0.0                        # learned from a fall: t
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ he least cushion this knob needs

    def learn_fall(self, step: float) -> None:
        """A fall while, or soon after, Omni held something back: this
    ↪  knob needs more room than its statistics said."""
        self.floor = min(1.0, max(self.floor, self.cushion) + step)

    def update(self, p: float) -> float:
        """Read one position; return the balance point (1 - cushion)."
    ↪ ""
        self.hist.append(min(p, 1.0))
        if len(self.hist) > self.window + 1:

            self.hist.pop(0)
        h, lag = self.hist, self.lag
        if len(h) >= lag + 2 and self.est == 2:
            r = max(1, int(round(self.react)))
            d = [b - a for a, b in zip(h, h[lag:])]              # th
    ↪ e change over one reference step

            md = sum(d) / len(d)
            wob = math.sqrt(sum((x - md) ** 2 for x in d) / max(1, len
    ↪ (d) - 1)) / math.sqrt(2.0)
            if r < len(h):
                rise = max(h[i + r] - h[i] for i in range(len(h) - r))
            else:

                rise = max(0.0, max(d)) / lag * r                # lo
    ↪ nger than the window: the steepest climb, extrapolated
            climb = max(0.0, rise) / r                           # pe
    ↪ r reading, so climb x reaction is the rise
        elif len(h) >= lag + 2:
            climb = max(max(0.0, b - a) for a, b in zip(h, h[lag:])) /

    ↪  lag
            m = sum(h) / len(h)
            wob = math.sqrt(sum((x - m) ** 2 for x in h) / (len(h) - 1
    ↪ ))
        else:
            climb, wob = 0.05, 0.05
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if self.mem > 0:
            # held longer: the worst climb and spread this knob has sh
    ↪ own, forgotten slowly
            self.climb_mem = max(climb, self.climb_mem * self.mem)
            self.wob_mem = max(wob, self.wob_mem * self.mem)
            climb, wob = self.climb_mem, self.wob_mem

        react_part = min(1.0, climb * self.react)
        need = min(1.0, max(react_part + self.z * wob, self.floor))
        self.floor *= self.fall_decay
        if need >= self.cushion:
            self.cushion = need                                    # wi
    ↪ den at once

        else:
            self.cushion = max(need, self.cushion * (1.0 - self.narrow
    ↪ ))   # narrow slowly
        self.react_part = max(0.005, react_part)
        return 1.0 - self.cushion

def physics_memory(horizon: float, window: int) -> float:
    """The forgetting factor for a knob whose risks last `horizon` dec
    ↪ isions (its own slowest physics): none if that is
    inside the window, else 1 - 1 / horizon per decision."""
    return (1.0 - 1.0 / horizon) if horizon > window else 0.0
```

H.27 omnicompass/immune.py

94 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The immune system's self-knowledge, knob by knob (the founder's ord
    ↪ er of 25 October): Omni-Compass knows when a knob it holds
is redlining out of its band; if its own pushing cannot bring the knob
    ↪  back and native is stronger there, it hands the knob to

native itself; if native is not stronger, it stays, because redlining
    ↪ less than native would is still a benefit against native.

  "OmniCompass's mechanism must know when something is redlining out o
    ↪ f its band when OmniCompass is on it. And if OmniCompass
  couldn't fix it to stop it redlining, then OmniCompass's own mechani

    ↪ sm must know that if native is better, then it switches
  itself to native. If native is not better, well, then it'll stay."

Each decision a knob gives three readings:

  p_omni     the knob's gauge against its own line with Omni-Compass's

    ↪  setting, as a ratio: 1 is on the line, over 1 is redlining
  p_native   the native twin's reading of the same moment under native
    ↪ 's own setting (the caller's twin of the native rule)
  moved      Omni-Compass changed the knob this decision
  at_limit   Omni-Compass's push on the knob is at the knob's own limi
    ↪ t (it cannot push further)

The rule (all in decisions, nothing tuned to a result):

  redlining   p_omni > 1 for `streak` decisions in a row
  stuck       redlining, and Omni-Compass's pushing has not brought it
    ↪  back: at its limit, or p_omni has not fallen by `relief`

               (a fraction of its reading when the redline began) since
    ↪  then
  stronger    native's twin reads lower than Omni-Compass by more than
    ↪  `margin`: p_native < p_omni · (1 − margin)
  hand back   stuck and native stronger: the knob goes to native and s
    ↪ tays there `hold` decisions, the hold doubling for each
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
              hand-back in a row (up to `hold_max`) and halving after
    ↪ a quiet hold
  stay        stuck and native not stronger: Omni-Compass keeps the kn
    ↪ ob (it redlines less than native would)
  return      after the hold, while native holds the knob: Omni-Compas
    ↪ s takes it back only when its own twin reads lower than
              native's by the same margin and nothing redlines with it

The nervous system carries the answer to the organ (omnicompass/nervou
    ↪ s_system.py, `native_hold`): an organ handed to native has
no authority of Omni-Compass's until it is returned. The C++ twin is c
    ↪ pp/src/immune.cpp (tests/test_cpp_immune_parity.py)."""
from __future__ import annotations

from typing import Dict, Optional, Tuple


class Immune:
    """One knob's self-knowledge. tick() answers ("omni" or "native",
    ↪ the event for the audit or None)."""

    def __init__(self, streak: int = 3, relief: float = 0.05, margin:
    ↪ float = 0.05, hold: int = 10, hold_max: int = 160):
        self.streak, self.relief, self.margin = int(streak), float(rel
    ↪ ief), float(margin)
        self.hold0, self.hold_max = int(hold), int(hold_max)
        self.mode = "omni"
        self.red = 0                      # decisions in a row over th
    ↪ e line, with Omni-Compass on the knob
        self.red_from: Optional[float] = None   # p_omni when the redl
    ↪ ine began
        self.left = 0                     # decisions of the hold stil
    ↪ l to run
        self.hold_len = int(hold)         # the next hold's length
        self.quiet = 0                    # decisions since the last h
    ↪ and-back
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.handbacks = 0

    def tick(self, p_omni: float, p_native: float, moved: bool = False
    ↪ , at_limit: bool = False) -> Tuple[str, Optional[Dict]]:
        stronger = p_native < p_omni * (1.0 - self.margin)
        if self.mode == "native":

            self.left -= 1
            self.quiet += 1
            if self.left > 0:
                return "native", None
            if p_omni <= 1.0 and p_omni < p_native * (1.0 - self.margi
    ↪ n):

                self.mode, self.red, self.red_from = "omni", 0, None
                if self.quiet >= 2 * self.hold_len:          # a quiet
    ↪  hold: the next one is halved
                    self.hold_len = max(self.hold0, self.hold_len // 2
    ↪ )
                return "omni", {"immune": "returned to Omni-Compass",

    ↪ "p_omni": round(p_omni, 6), "p_native": round(p_native, 6)}
            self.left = 1                                     # native
    ↪  still the stronger: held, read again next decision
            return "native", None
        if p_omni > 1.0:
            self.red += 1

            if self.red_from is None:
                self.red_from = p_omni
        else:
            self.red, self.red_from = 0, None
            self.quiet += 1
            return "omni", None

        if self.red < self.streak:
            return "omni", None
        stuck = at_limit or p_omni > self.red_from * (1.0 - self.relie
    ↪ f)
        if not stuck:
            return "omni", None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if stronger:
            self.mode, self.left = "native", self.hold_len
            event = {"immune": "handed to native", "p_omni": round(p_o
    ↪ mni, 6), "p_native": round(p_native, 6),
                     "redline_decisions": self.red, "hold": self.hold_
    ↪ len}
            self.handbacks += 1
            self.hold_len = min(self.hold_max, self.hold_len * 2)
            self.quiet, self.red, self.red_from = 0, 0, None
            return "native", event
        return "omni", {"immune": "redlining, native not stronger: sta
    ↪ ys", "p_omni": round(p_omni, 6),
                        "p_native": round(p_native, 6), "redline_decis
    ↪ ions": self.red}


# © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Sim
    ↪ ulation use only; any commercialization or
# monetization requires a signed, paid Omni-Compass Enterprise License
    ↪ . All patents, copyrights and trademarks filed in the USA. www.o
    ↪ mni-compass.com
```

H.28 omnicompass/master.py

150 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The kill switch (the master switch): one OFF for the whole harness,
    ↪  not one muscle at a time.

Its purpose is security: if anything gets into Omni-Compass's brain an
    ↪ d tries to drive a system for its own ends, or
anything rogue happens, one switch in a human hand turns Omni-Compass'
    ↪ s governing off everywhere at once. It is not
the reset (the one pedal lifted all the way at the end of a run, which
    ↪  hands one governor's settings back).

Every Omni-Compass governor on a machine (the Kubernetes controller, t
    ↪ he GPU governors, any muscle they drive) obeys one
switch, in a human hand:

  OFF   the switch file exists (OMNI_MASTER_OFF, by default /tmp/omni-
    ↪ compass/OFF). Every running governor sees it at its
        next decision, puts every setting it ever wrote back to the va
    ↪ lue it read before its first write, reads each one
        back, writes its last audit line and exits. No governor starts
    ↪  while the switch is OFF.
  ON    the switch file is removed. Nothing restarts by itself: a gove
    ↪ rnor runs again only when someone starts it.

The switch is also pulled from the command line, which signals every r
    ↪ egistered governor at once so none waits for its
next decision:

  python3 tools/omni_switch.py off [--reason TEXT]    turn the whole h
    ↪ arness off, wait for every governor to restore
  python3 tools/omni_switch.py on                      allow governors
    ↪  to be started again
  python3 tools/omni_switch.py status                  OFF or ON, and
    ↪ every governor running
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
A governor that cannot read the switch (a filesystem it cannot reach)
    ↪ treats that as OFF: when in doubt, hand back.

If a governor dies without handing back (killed outright, the machine
    ↪ crashed, it hung), nothing it wrote may be left
in place. So every governor records, before its first write, the exact
    ↪  commands that put every setting back, and
renews a lease every decision. The watchdog finds any governor whose p
    ↪ rocess is gone or whose lease is older than its
limit and runs those commands for it:

  python3 tools/omni_switch.py watchdog [--stale 60] [--every 10]    k
    ↪ eep watching (run it as its own service)
  python3 tools/omni_switch.py watchdog --once                       o
    ↪ ne pass
"""
from __future__ import annotations
import atexit
import json
import os
import time

DEFAULT_DIR = "/tmp/omni-compass"
def switch_path() -> str:
    return os.environ.get("OMNI_MASTER_OFF", os.path.join(DEFAULT_DIR,
    ↪  "OFF"))
def registry_dir() -> str:
    return os.environ.get("OMNI_REGISTRY", os.path.join(os.path.dirnam
    ↪ e(switch_path()) or DEFAULT_DIR, "running"))
def is_off() -> bool:
    """True when the master switch is OFF (or cannot be read: when in
    ↪ doubt, hand back)."""
    p = switch_path()
    try:
        return os.path.exists(p)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    except OSError:
        return True


def reason() -> str:
    try:
        return json.loads(open(switch_path()).read()).get("reason", ""
    ↪ )
    except (OSError, ValueError):
        return ""

_ME = {}


def register(name: str, restore=None, stale_s: float = 60.0) -> str:
    """Record this governor as running, so the switch can signal it, w
    ↪ ith the commands (argument lists) that put every

    setting back if it dies without doing so itself; removed when the
    ↪ process exits cleanly."""
    d = registry_dir()
    os.makedirs(d, exist_ok=True)
    path = os.path.join(d, f"{os.getpid()}.json")
    rec = {"pid": os.getpid(), "name": name, "started": time.time(), "

    ↪ lease": time.time(), "stale_s": stale_s,
           "restore": restore or []}
    with open(path + ".tmp", "w") as f:
        json.dump(rec, f)
    os.replace(path + ".tmp", path)
    _ME.update(path=path, rec=rec)

    def _gone():
        try:
            os.remove(path)
        except OSError:
            pass
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    atexit.register(_gone)
    return path


def heartbeat():
    """Renew this governor's lease (called every decision)."""
    if not _ME:
        return
    _ME["rec"]["lease"] = time.time()
    try:
        with open(_ME["path"] + ".tmp", "w") as f:
            json.dump(_ME["rec"], f)
        os.replace(_ME["path"] + ".tmp", _ME["path"])
    except OSError:
        pass


def _alive(pid):
    try:
        os.kill(int(pid), 0)
        return True
    except OSError:
        return False
def records():
    out = []
    d = registry_dir()
    if not os.path.isdir(d):
        return out
    for fn in sorted(os.listdir(d)):
        if not fn.endswith(".json"):
            continue
        try:
            rec = json.loads(open(os.path.join(d, fn)).read())
            rec["_path"] = os.path.join(d, fn)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            out.append(rec)
        except (OSError, ValueError):
            pass
    return out

def orphans(now=None):
    """Governors that are gone without handing back (process dead) or
    ↪ hung (lease older than its limit)."""
    now = now or time.time()
    return [r for r in records() if not _alive(r.get("pid", -1)) or no
    ↪ w - r.get("lease", 0) > r.get("stale_s", 60.0)]

def running():
    """Every governor registered as running whose process is alive."""
    return [r for r in records() if _alive(r.get("pid", -1))]

def refuse_if_off(name: str):
    """Called by every governor at start: no authority is taken while
    ↪ the master switch is OFF."""
    if is_off():
        raise SystemExit(f"{name}: the Omni-Compass master switch is O
    ↪ FF ({switch_path()}"
                         f"{': ' + reason() if reason() else ''}); not
    ↪ hing started. Turn it on with: "
                         "python3 tools/omni_switch.py on")
```

H.29 omnicompass/mathdrive.py

86 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Math-driven actuation: the Omni-Compass equations move the system d
    ↪ irectly.

The frozen governor (adapter.Governor) evolves the equations open-loop
    ↪  (u = 0) and reads only a few derived numbers.
Here the equations are closed-loop and their own quantities ARE the ac
    ↪ tuator commands. No equation is changed; the
integrator, parameters and observation map are the frozen ones (omnico
    ↪ mpass/core.py, omnicompass/adapter.py).

Per decision
  1. sense      x <- assimilate(x, observe_vector(telemetry))
    ↪               (the frozen observation map)
  2. evolve     x <- macro_step(x, target = +1, macro_dt = clock)
    ↪               equations (1)-(7) with equation
                (2)'s controller ON: u = clip(-f_U(x) + 12 (1 - U), -2
    ↪ 5, 25) holds the healthy basin U = +1
  3. read the   effort  w = integral |u| dt / (25 clock)   in [0, 1]:
    ↪ how hard the math had to work to stay healthy
     math       need    I_U                                 equation (
    ↪ 3): unrelieved need, slow memory
                tide    B, B_dot                            equation (
    ↪ 7): the bath, second order, anticipates
                drive   v_eff                               equation (
    ↪ 4): the tanh-bounded drive
  4. actuate    capacity   n = ceil(fit (1 + g_w w + g_I max(0, I_U) +
    ↪  g_B max(0, B_dot))) machines
                           (released one at a time only while w <= w_r
    ↪ elease: the math reports calm)
                pods       HPA target rho = clamp(rho0 - k_w w - k_I m
    ↪ ax(0, I_U), rho_min, rho0)
                power      cap = clamp(1 - g_cap max(0, B) + 2 w, cap_
    ↪ min, 1)   (the bath lowers power only when calm)
  clock: engine time advanced per decision. It sets how fast the math
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ moves relative to the system: the bath's
  natural period is 2 pi / omega_B = 6.28 engine units, i.e. 6.28 / cl
    ↪ ock decisions.
"""
from __future__ import annotations

import math
from dataclasses import dataclass, replace

from omnicompass.adapter import STACK_PARAMS, INITIAL_STATE, observe_v
    ↪ ector, assimilate
from omnicompass.core import State, macro_step, v_eff, U_AUTHORITY

@dataclass(frozen=True)
class MathLaw:
    clock: float = 0.1
    g_w: float = 1.0

    g_I: float = 0.3
    g_B: float = 0.5
    w_release: float = 0.02
    dwell: int = 3
    rho0: float = 0.70
    rho_min: float = 0.50

    k_w: float = 0.5
    k_I: float = 0.1
    g_cap: float = 0.0
    cap_min: float = 0.8

class MathDrive:
    def __init__(self, law: MathLaw = MathLaw()):
        self.law = law
        self.p = replace(STACK_PARAMS)
        self.x = State(**vars(INITIAL_STATE))
        self.t = 0.0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.calm = 0
        self.w = 0.0

    def step(self, obs, n, requests, cores_per_node):
        """Returns (hpa_target, node_target, power_cap, readout)."""
        L = self.law

        o = observe_vector(obs, 0)
        xa = assimilate(self.x, o, self.p)
        self.x, stats = macro_step(xa, self.p, self.t, target=+1, macr
    ↪ o_dt=L.clock)
        self.t += L.clock
        w = min(1.0, stats["u_abs"] / (U_AUTHORITY * L.clock))

        self.w = w
        x = self.x
        need = max(0.0, x.I_U); tide = max(0.0, x.B_dot)
        fit = requests / cores_per_node
        want = math.ceil(fit * (1.0 + L.g_w * w + L.g_I * need + L.g_B
    ↪  * tide) - 1e-9)

        tgt = n
        if want > n:
            tgt = want; self.calm = 0
        elif want < n:
            self.calm = self.calm + 1 if w <= L.w_release else 0
            if self.calm >= L.dwell:

                tgt = n - 1; self.calm = 0
        else:
            self.calm = 0
        rho = min(L.rho0, max(L.rho_min, L.rho0 - L.k_w * w - L.k_I *
    ↪ need))
        cap = min(1.0, max(L.cap_min, 1.0 - L.g_cap * max(0.0, x.B) +

    ↪ 2.0 * w))
        return rho, int(tgt), cap, {"w": w, "I_U": x.I_U, "B": x.B, "B
    ↪ _dot": x.B_dot, "U": x.U, "E": x.E,
                                    "v_eff": v_eff(x, self.p, self.t)}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

H.30 omnicompass/muscle_sdk.py

316 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""OmniCompass Universal Muscle SDK.

This module does not change the six-state governor.  It standardises h
    ↪ ow a plant

surface earns authority beneath it: observe -> bound -> request -> exe
    ↪ cute ->
readback -> receipt -> restore.  Adapters may target Kubernetes, host
    ↪ controls,
GPU/accelerator controls, cloud/VM APIs, storage, network, runtimes, c
    ↪ ooling or

facility power without changing the governing state law.
"""
from __future__ import annotations

from dataclasses import asdict, dataclass, field
from enum import Enum

from time import monotonic, time
from typing import Any, Callable, Dict, Iterable, Mapping, Optional, P
    ↪ rotocol, Tuple


class Evidence(str, Enum):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    DESIGN = "D"
    SIMULATION = "S"
    LIVE_SOFTWARE = "L"
    PHYSICAL = "P"
    COMMERCIAL = "C"

class AutomationMode(str, Enum):
    DISABLED = "disabled"
    RECOMMEND = "recommend"
    MANUAL = "manual"
    EXTERNAL_APPROVAL = "external_approval"

    AUTOMATIC = "automatic"


class MuscleStatus(str, Enum):
    AVAILABLE = "available"
    DEGRADED = "degraded"

    BLIND = "blind"
    FAILED = "failed"
    RESTORING = "restoring"


@dataclass(frozen=True)

class TimingContract:
    observation_period_s: float
    decision_period_s: float
    actuation_delay_s: float = 0.0
    minimum_hold_s: float = 0.0
    cooldown_s: float = 0.0

    timeout_s: float = 5.0
    stale_after_s: float = 30.0


@dataclass(frozen=True)
class CapabilityContract:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    minimum: float
    maximum: float
    resolution: float = 0.0
    max_step: Optional[float] = None
    max_rate_per_s: Optional[float] = None
    units: str = "unit"

    reversible: bool = True
    readback: bool = True

    def clamp(self, value: float, previous: Optional[float] = None, dt
    ↪ : Optional[float] = None) -> float:
        v = min(self.maximum, max(self.minimum, float(value)))

        if self.max_step is not None and previous is not None:
            v = min(previous + self.max_step, max(previous - self.max_
    ↪ step, v))
        if self.max_rate_per_s is not None and previous is not None an
    ↪ d dt is not None:
            step = max(0.0, dt) * self.max_rate_per_s

            v = min(previous + step, max(previous - step, v))
        if self.resolution > 0:
            v = round(v / self.resolution) * self.resolution
            v = min(self.maximum, max(self.minimum, v))
        return v

@dataclass(frozen=True)
class SafetyContract:
    hard_min: float
    hard_max: float
    fail_closed: bool = True

    safe_default: Optional[float] = None
    forbidden: Tuple[str, ...] = ()

    def permits(self, value: float, context: Mapping[str, Any]) -> Tup
    ↪ le[bool, str]:
        if value < self.hard_min or value > self.hard_max:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            return False, "outside_hard_bounds"
        for key in self.forbidden:
            if bool(context.get(key, False)):
                return False, f"forbidden:{key}"
        return True, "permitted"

@dataclass(frozen=True)
class MuscleSpec:
    muscle_id: str
    plant_id: str
    resource_type: str

    adapter_version: str
    capability: CapabilityContract
    timing: TimingContract
    safety: SafetyContract
    automation: AutomationMode = AutomationMode.RECOMMEND
    evidence: Evidence = Evidence.DESIGN

    native_owner: str = "native"
    authority_priority: int = 100
    description: str = ""


@dataclass

class Observation:
    value: float
    timestamp: float = field(default_factory=time)
    quality: float = 1.0
    metadata: Dict[str, Any] = field(default_factory=dict)

    def age(self, now: Optional[float] = None) -> float:
        return max(0.0, (time() if now is None else now) - self.timest
    ↪ amp)


@dataclass
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class MuscleReceipt:
    muscle_id: str
    timestamp: float
    mode: str
    requested: Optional[float]
    bounded: Optional[float]

    realized: Optional[float]
    units: str
    permitted: bool
    reason: str
    observation_age_s: Optional[float] = None
    actuation_latency_s: Optional[float] = None

    actuator_residual: Optional[float] = None
    restored: Optional[bool] = None
    evidence: str = "D"
    metadata: Dict[str, Any] = field(default_factory=dict)

    def as_dict(self) -> Dict[str, Any]:

        return asdict(self)


class MuscleAdapter(Protocol):
    spec: MuscleSpec
    def observe(self) -> Observation: ...

    def snapshot_native(self) -> Any: ...
    def execute(self, value: float) -> None: ...
    def readback(self) -> float: ...
    def restore_native(self, snapshot: Any) -> None: ...

class FunctionalMuscle:
    """Small adapter for public APIs/commands without coupling them to
    ↪  the brain."""
    def __init__(self, spec: MuscleSpec, observe: Callable[[], Observa
    ↪ tion], execute: Callable[[float], None],
                 readback: Callable[[], float], snapshot: Callable[[],
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  Any], restore: Callable[[Any], None]):
        self.spec = spec
        self._observe, self._execute, self._readback = observe, execut
    ↪ e, readback
        self._snapshot, self._restore = snapshot, restore

    def observe(self): return self._observe()
    def execute(self, value): return self._execute(value)
    def readback(self): return float(self._readback())
    def snapshot_native(self): return self._snapshot()
    def restore_native(self, snapshot): return self._restore(snapshot)

class MuscleRuntime:
    """Uniform authority gate around one actuator.  The governor propo
    ↪ ses; this runtime earns and verifies authority."""
    def __init__(self, adapter: MuscleAdapter, audit: Optional[Callabl
    ↪ e[[Dict[str, Any]], None]] = None):

        self.adapter = adapter
        self.spec = adapter.spec
        self.audit = audit or (lambda _: None)
        self.native_snapshot: Any = None
        self.last_realized: Optional[float] = None
        self.last_write_mono: Optional[float] = None

        self.status = MuscleStatus.AVAILABLE


    def arm(self) -> None:
        if self.native_snapshot is None:
            self.native_snapshot = self.adapter.snapshot_native()

    def command(self, requested: float, context: Optional[Mapping[str,
    ↪  Any]] = None,
                approved: bool = False, now: Optional[float] = None) -
    ↪ > MuscleReceipt:
        context = dict(context or {})
        obs = self.adapter.observe()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        wall = time() if now is None else now
        age = obs.age(wall)
        mode = self.spec.automation
        reason = ""
        permitted = True
        bounded: Optional[float] = None

        realized: Optional[float] = None
        latency: Optional[float] = None

        if age > self.spec.timing.stale_after_s or obs.quality <= 0:
            permitted, reason, self.status = False, "stale_or_invalid_
    ↪ observation", MuscleStatus.BLIND

        elif mode == AutomationMode.DISABLED:
            permitted, reason = False, "disabled"
        elif mode in (AutomationMode.RECOMMEND, AutomationMode.MANUAL)
    ↪ :
            permitted, reason = False, mode.value
        elif mode == AutomationMode.EXTERNAL_APPROVAL and not approved

    ↪ :
            permitted, reason = False, "approval_required"

        if permitted:
            dt = None if self.last_write_mono is None else max(0.0, mo
    ↪ notonic() - self.last_write_mono)

            bounded = self.spec.capability.clamp(requested, self.last_
    ↪ realized, dt)
            permitted, reason = self.spec.safety.permits(bounded, cont
    ↪ ext)

        if permitted and self.last_write_mono is not None:

            elapsed = monotonic() - self.last_write_mono
            hold = max(self.spec.timing.minimum_hold_s, self.spec.timi
    ↪ ng.cooldown_s)
            if elapsed < hold:
                permitted, reason = False, "hold_or_cooldown"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if permitted:
            self.arm()
            t0 = monotonic()
            self.adapter.execute(float(bounded))
            self.last_write_mono = monotonic()
            if self.spec.capability.readback:

                realized = self.adapter.readback()
                latency = monotonic() - t0
                self.last_realized = realized
            else:
                realized = float(bounded)
                self.last_realized = realized

            self.status = MuscleStatus.AVAILABLE
        elif reason == "":
            reason = "blocked"

        receipt = MuscleReceipt(
            muscle_id=self.spec.muscle_id, timestamp=wall, mode=mode.v
    ↪ alue,
            requested=float(requested), bounded=bounded, realized=real
    ↪ ized,
            units=self.spec.capability.units, permitted=permitted, rea
    ↪ son=reason,
            observation_age_s=age, actuation_latency_s=latency,

            actuator_residual=None if realized is None or bounded is N
    ↪ one else realized - bounded,
            evidence=self.spec.evidence.value,
            metadata={"observation_quality": obs.quality, "status": se
    ↪ lf.status.value},
        )

        self.audit(receipt.as_dict())
        return receipt

    def restore(self) -> MuscleReceipt:
        self.status = MuscleStatus.RESTORING
        ok = False
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        reason = "no_snapshot"
        realized = None
        if self.native_snapshot is not None:
            try:
                self.adapter.restore_native(self.native_snapshot)
                realized = self.adapter.readback() if self.spec.capabi
    ↪ lity.readback else None
                ok, reason = True, "restored"
                self.status = MuscleStatus.AVAILABLE
            except Exception as exc:  # deliberately auditable fail-cl
    ↪ osed boundary
                reason = f"restore_failed:{type(exc).__name__}"
                self.status = MuscleStatus.FAILED
        receipt = MuscleReceipt(
            muscle_id=self.spec.muscle_id, timestamp=time(), mode=self
    ↪ .spec.automation.value,
            requested=None, bounded=None, realized=realized, units=sel
    ↪ f.spec.capability.units,
            permitted=False, reason=reason, restored=ok, evidence=self
    ↪ .spec.evidence.value,
            metadata={"status": self.status.value},
        )
        self.audit(receipt.as_dict())
        return receipt

class MuscleRegistry:
    """Catalog of qualified plant surfaces.  Registration expands musc
    ↪ les, never the six-state core."""
    def __init__(self):
        self._muscles: Dict[str, MuscleRuntime] = {}

    def register(self, runtime: MuscleRuntime) -> None:
        mid = runtime.spec.muscle_id
        if mid in self._muscles:
            raise ValueError(f"duplicate muscle_id: {mid}")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self._muscles[mid] = runtime

    def get(self, muscle_id: str) -> MuscleRuntime:
        return self._muscles[muscle_id]

    def command(self, muscle_id: str, requested: float, **kwargs) -> M
    ↪ uscleReceipt:
        return self.get(muscle_id).command(requested, **kwargs)

    def restore_all(self) -> Dict[str, MuscleReceipt]:
        return {mid: m.restore() for mid, m in reversed(list(self._mus
    ↪ cles.items()))}

    def catalog(self) -> list[Dict[str, Any]]:
        out = []
        for mid, m in sorted(self._muscles.items()):
            s = m.spec
            out.append({
                "muscle_id": mid, "plant_id": s.plant_id, "resource_ty
    ↪ pe": s.resource_type,
                "mode": s.automation.value, "evidence": s.evidence.val
    ↪ ue, "units": s.capability.units,
                "min": s.capability.minimum, "max": s.capability.maxim
    ↪ um,
                "readback": s.capability.readback, "reversible": s.cap
    ↪ ability.reversible,
                "stale_after_s": s.timing.stale_after_s, "status": m.s
    ↪ tatus.value,
            })
        return out

# Standard actuator classes.  These are categories/contracts, not clai
    ↪ ms that a connector is already physically verified.
STANDARD_MUSCLE_CLASSES = (
    "kubernetes.replicas", "kubernetes.cpu_request", "kubernetes.cpu_l
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ imit", "kubernetes.memory_request",
    "kubernetes.memory_limit", "kubernetes.hpa_target", "kubernetes.po
    ↪ d_placement", "kubernetes.node_pool",
    "host.cpufreq", "host.cpu_quota", "host.cpuset", "host.memory", "h
    ↪ ost.io",
    "gpu.power_limit", "gpu.clock_envelope", "gpu.mig", "gpu.admission
    ↪ ",
    "cloud.instance_count", "cloud.instance_size", "cloud.placement",
    ↪ "cloud.start_stop", "cloud.capacity_policy",
    "storage.volume_size", "storage.tier", "storage.iops", "storage.th
    ↪ roughput", "storage.placement",
    "network.traffic_weight", "network.rate_limit", "network.bandwidth
    ↪ ", "network.routing",
    "application.concurrency", "application.worker_count", "applicatio
    ↪ n.queue_admission", "application.jvm_heap",
    "application.thread_pool", "application.batch", "facility.cooling_
    ↪ setpoint", "facility.rack_power", "facility.site_power",
)
```

H.31 omnicompass/muscles.py

159 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Infrastructure component catalog and fleet overhead model.

Each component records its role, deployment pattern, resource footprin
    ↪ t, the source of that
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
footprint and its evidence status, and its classification under the ke
    ↪ ep-or-remove rule:

  decision       control logic whose function the governor performs
    ↪    -> candidate for removal
  observability  telemetry and storage for humans, audit and control
    ↪    -> retained; control path replaced by direct sensing
  execution      machinery that performs the work
    ↪    -> retained
  security       cryptography, identity, policy, runtime detection
    ↪    -> retained unless a stronger guarantee is proven

Evidence status:
  measured       benchmark or production measurement published by the
    ↪ source
  documented     formula or default published in official documentatio
    ↪ n
  example        request values from a published example manifest
  assumption     modelling assumption; parameter exposed for sensitivi
    ↪ ty analysis

Energy per resource follows the Cloud Carbon Footprint method:
  W = vCPU * (W_min + u * (W_max - W_min)) + GB * W_mem, scaled by PUE
    ↪ .
"""
from __future__ import annotations

from dataclasses import dataclass, asdict
from typing import Dict, List

W_MIN_PER_VCPU = 0.434
W_MAX_PER_VCPU = 1.948
W_PER_GB = 0.392
ENERGY_SOURCE = ("Cloud Carbon Footprint coefficients (SPECpower-deriv
    ↪ ed), AMD EPYC 3rd generation: "
                 "0.434 W min, 1.948 W max per vCPU; memory 0.392 W pe
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ r GB (arXiv 2510.26413)")


@dataclass(frozen=True)
class Component:
    name: str

    role: str
    scope: str
    vcpu: float
    gib: float
    status: str
    source: str

    verdict: str
    note: str = ""


def kube_reserved_cpu(cores: int) -> float:
    """AKS kube-reserved CPU table (Microsoft Learn), cores -> vCPU, l

    ↪ inear between table points."""
    pts = [(1, 0.060), (2, 0.100), (4, 0.140), (8, 0.180), (16, 0.260)
    ↪ , (32, 0.420), (64, 0.740)]
    if cores <= 1:
        return pts[0][1]
    for (c0, v0), (c1, v1) in zip(pts, pts[1:]):

        if cores <= c1:
            return v0 + (v1 - v0) * (cores - c0) / (c1 - c0)
    c0, v0 = pts[-2]; c1, v1 = pts[-1]
    return v1 + (v1 - v0) * (cores - c1) / (c1 - c0)

@dataclass
class Fleet:
    nodes: int = 100_000
    nodes_per_cluster: int = 5_000
    cores_per_node: int = 64
    gib_per_node: int = 256
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    pods_per_node: int = 30
    series_per_node: int = 2_000
    long_term_replication: int = 3
    mean_cpu_utilization: float = 0.30
    pue: float = 1.18
    mesh: str = "sidecar"

    @property
    def clusters(self) -> int:
        return -(-self.nodes // self.nodes_per_cluster)

def catalog(f: Fleet) -> List[Component]:
    series = f.nodes * f.series_per_node
    C = []
    C.append(Component("kubelet + container runtime (kube-reserved)",
    ↪ "execution", "per node",
                       kube_reserved_cpu(f.cores_per_node) * f.nodes,
    ↪ 0.0, "documented",
                       "Microsoft Learn, AKS node resource reservation
    ↪ s (CPU table)", "retain",
                       "memory reservation not counted here"))
    C.append(Component("node-exporter (metrics agent)", "observability
    ↪ ", "per node", 0.050 * f.nodes,
                       100 / 1024 * f.nodes, "example", "OneUptime Dae
    ↪ monSet resource example (requests 50m / 100Mi)",
                       "retain for humans; control path replaced by di
    ↪ rect sensing"))
    C.append(Component("log collector (Fluentd / Fluent Bit)", "observ
    ↪ ability", "per node", 0.100 * f.nodes,
                       200 / 1024 * f.nodes, "example", "OneUptime Dae
    ↪ monSet resource example (requests 100m / 200Mi)",
                       "retain for audit and humans"))
    C.append(Component("OpenTelemetry agent collector", "observability
    ↪ ", "per node", 0.100 * f.nodes,
                       128 / 1024 * f.nodes, "example", "OneUptime age
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ nt-gateway topology (requests 100m / 128Mi)",
                      "retain for humans; control path replaced by di
    ↪ rect sensing"))
    if f.mesh == "sidecar":
        C.append(Component("service-mesh sidecar proxies (Envoy, per p
    ↪ od)", "security", "per pod",
                           0.20 * f.pods_per_node * f.nodes, 60 / 1024
    ↪  * f.pods_per_node * f.nodes, "measured",
                           "Istio 1.24 benchmark at 1,000 req/s, 1 KB:
    ↪  ~0.20 vCPU, ~60 MB per sidecar",
                           "retain (mTLS); independent option: ambient
    ↪  mode",
                           "load-dependent; an idle sidecar measured ~
    ↪ 8 millicores / 40 Mi (Defense Unicorns)"))
    else:
        C.append(Component("service-mesh ztunnel (ambient, per node)",
    ↪  "security", "per node",
                           0.06 * f.nodes, 12 / 1024 * f.nodes, "measu
    ↪ red",
                           "Istio 1.24 benchmark: ztunnel ~0.06 vCPU,
    ↪ ~12 MB", "retain (mTLS)"))
    C.append(Component("Prometheus (in-cluster head series)", "observa
    ↪ bility", "per cluster",
                      0.0, 3.0e-6 * series, "measured", "Robust Perce
    ↪ ption: ~3 kB per head series (Prometheus 2.20+)",
                      "retain for humans; control scrape load removab
    ↪ le", "series_per_node is an assumption"))
    C.append(Component("long-term metrics (Mimir ingesters)", "observa
    ↪ bility", "fleet",
                      series * f.long_term_replication / 300_000, 2.5
    ↪  * series * f.long_term_replication / 300_000,
                      "documented", "Grafana Mimir capacity planning:
    ↪  1 core and 2.5 GB per 300,000 in-memory series",
                      "retain for audit"))
    C.append(Component("cluster autoscaler / Karpenter", "decision", "
    ↪ per cluster", 1.0 * f.clusters,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                      1.0 * f.clusters, "assumption", "typical contro
    ↪ ller sizing; not measured here",
                      "remove: governor owns capacity"))
    C.append(Component("HPA / VPA / KEDA controllers", "decision", "pe
    ↪ r cluster", 0.5 * f.clusters,
                      1.0 * f.clusters, "assumption", "typical contro
    ↪ ller sizing; not measured here",
                      "remove: governor owns replicas and sizing"))
    C.append(Component("power / thermal policy agents", "decision", "p
    ↪ er node", 0.010 * f.nodes,
                      32 / 1024 * f.nodes, "assumption", "typical nod
    ↪ e agent sizing; not measured here",
                      "remove: governor owns power caps"))
    C.append(Component("alert routing and paging (Alertmanager)", "dec
    ↪ ision", "per cluster", 0.2 * f.clusters,
                      0.5 * f.clusters, "assumption", "sizing scales
    ↪ with firing alerts (Grafana Mimir guidance)",
                      "remove paging path; keep audit notifications")
    ↪ )
    C.append(Component("Terraform as live capacity controller", "decis
    ↪ ion", "fleet", 0.0, 0.0, "assumption",
                      "run cost negligible; harm is stale overwrites
    ↪ (measured in the stack benchmark)",
                      "remove from the live control path; retain as a
    ↪ udit record"))
    return C


def watts(vcpu: float, gib: float, u: float, pue: float) -> float:
    return pue * (vcpu * (W_MIN_PER_VCPU + u * (W_MAX_PER_VCPU - W_MIN
    ↪ _PER_VCPU)) + gib * 1.073741824 * W_PER_GB)


def fleet_report(f: Fleet) -> Dict:
    comps = catalog(f)
    total_vcpu = f.nodes * f.cores_per_node
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    total_gib = f.nodes * f.gib_per_node
    fleet_w = watts(total_vcpu, total_gib, f.mean_cpu_utilization, f.p
    ↪ ue)
    rows = []
    for c in comps:
        lo, hi = watts(c.vcpu, c.gib, 0.0, f.pue), watts(c.vcpu, c.gib
    ↪ , 1.0, f.pue)
        rows.append({**asdict(c), "watts_low": lo, "watts_high": hi,
                     "mwh_per_year_low": lo * 8760 / 1e6, "mwh_per_yea
    ↪ r_high": hi * 8760 / 1e6,
                     "share_of_fleet_vcpu": c.vcpu / total_vcpu,
                     "share_of_fleet_energy_low": lo / fleet_w, "share
    ↪ _of_fleet_energy_high": hi / fleet_w})
    by_role = {}
    for r in rows:
        g = by_role.setdefault(r["role"], {"vcpu": 0.0, "share_of_flee
    ↪ t_vcpu": 0.0,
                                           "share_of_fleet_energy_low
    ↪ ": 0.0, "share_of_fleet_energy_high": 0.0,
                                           "mwh_per_year_low": 0.0, "
    ↪ mwh_per_year_high": 0.0})
        for k in g:
            g[k] += r[k]
    idle_vcpu = total_vcpu * (1.0 - f.mean_cpu_utilization)
    idle_w = f.pue * idle_vcpu * W_MIN_PER_VCPU
    return {"fleet": asdict(f), "clusters": f.clusters, "fleet_vcpu":
    ↪ total_vcpu, "fleet_mwh_per_year": fleet_w * 8760 / 1e6,
            "energy_model": ENERGY_SOURCE, "components": rows, "by_rol
    ↪ e": by_role,
            "idle_vcpu": idle_vcpu, "idle_share_of_fleet_vcpu": 1.0 -
    ↪ f.mean_cpu_utilization,
            "idle_vcpu_share_of_fleet_energy": idle_w / fleet_w}
```

H.32 `omnicompass/nervous.py`

232 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Nervous system: one engine, many muscles.

Each muscle is a connector with five parts (domain map):
  afferent  sense

  engine    shared six-state field (not duplicated)
  efferent  push
  reflex    shield / bounds before push
  kill      return that muscle to its native controller

Status:

  wired     sense and push are executed in this tree
  sensed    telemetry is accepted; push is not issued
  open      connector registered; no plant; no push

This module does not replace adapter.Governor. It sits around it:
afferents merge into the observation vector the governor already under

    ↪ stands;
efferents are tagged by muscle so a kill is per-nerve, not a mystery w
    ↪ rite.
"""
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Callable, Dict, List, Optional


# Fit from the domain map. Advisory muscles never push from this runti
    ↪ me.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
DIRECT = "direct"
SUPERVISORY = "supervisory"
ADVISORY = "advisory"
NOT_OURS = "not_omni"

WIRED = "wired"

SENSED = "sensed"
OPEN = "open"


@dataclass(frozen=True)
class Muscle:

    name: str
    family: str
    fit: str
    status: str
    reads: tuple
    pushes: tuple

    reflexes: tuple
    note: str = ""


# Registered nerves. Status is a claim about THIS tree, not a product
    ↪ slide.

CATALOG: List[Muscle] = [
    Muscle("nodes", "infra", DIRECT, WIRED,
           ("load_ratio", "pending", "requests"),
           ("node_delta", "actuation"),
           ("request_floor", "step_limit", "I2", "I5"),
           "fleet + k8s_controlplane"),

    Muscle("hpa", "infra", DIRECT, WIRED,
           ("utilization", "replicas"),
           ("hpa_target",),
           ("target_bounds_50_95",),
           "target write only when authority is on"),
    Muscle("power_cap", "infra", DIRECT, WIRED,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        ("power_stress", "site_limit"),
        ("power_cap",),
        ("I4", "cap_lo_hi"),
        "harness; live: CPU limits of capped deployments (omni_cont
    ↪ roller/muscles.py)"),
    Muscle("heat", "infra", DIRECT, SENSED,

        ("thermal",),
        (),
        ("thermal_limit",),
        "feeds E/S; live: harness heat law on live power, or GPU te
    ↪ mperature; no chiller write"),
    Muscle("network", "infra", DIRECT, SENSED,

        ("network_stress",),
        ("route_shift",),
        ("link_capacity",),
        "directive flag only"),
    Muscle("security", "infra", DIRECT, WIRED,
        ("security_block",),

        ("block_expansion",),
        ("I1",),
        "live: hold signal from a ConfigMap; blocks node, cap and b
    ↪ atch expansion"),
    Muscle("deployments", "infra", DIRECT, WIRED,
        ("rollout_progress",),

        ("pause", "resume", "undo"),
        ("change_permitted", "rollback_authorized"),
        "live: kubectl rollout pause / resume / undo"),
    Muscle("gpu", "infra", DIRECT, WIRED,
        ("gpu_power", "gpu_temp"),
        ("gpu_power_cap",),

        ("temp_limit", "cap_floor"),
        "hardware connector (nvidia-smi query and power limit); tes
    ↪ ted with stand-in hardware, not run on a GPU; MIG open"),
    Muscle("cpu_pstate", "infra", DIRECT, WIRED,
        ("rapl_watts",),
        ("frequency_ceiling",),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        ("cap_floor",),
        "hardware connector (RAPL read, cpufreq ceiling); tested wi
    ↪ th stand-in hardware, not run on bare metal"),
    Muscle("memory", "infra", DIRECT, OPEN,
        ("mem_used", "mem_request"),
        ("mem_request",),

        ("oom_margin",)),
    Muscle("storage", "infra", SUPERVISORY, OPEN,
        ("io_load", "io_latency", "capacity"),
        ("tier", "volume_scale"),
        ("durability",)),
    Muscle("batch_queue", "infra", DIRECT, WIRED,

        ("queue_depth", "load_ratio", "power_stress"),
        ("admit",),
        ("headroom", "I1"),
        "live: admit held Jobs (spec.suspend) with headroom; priori
    ↪ ty and deadlines open"),
    Muscle("cooling", "facility", DIRECT, OPEN,

        ("inlet_c", "chiller_load"),
        ("setpoint_c", "fan_curve"),
        ("inlet_limit",)),
    Muscle("grid", "facility", DIRECT, OPEN,
        ("price", "carbon", "demand_signal"),
        ("shed_kw", "shift"),

        ("contract_limit",)),
    Muscle("training", "ai", DIRECT, OPEN,
        ("gpu_power", "step", "progress"),
        ("power_cap", "pack"),
        ("no_kill_midstep",)),
    Muscle("inference", "ai", DIRECT, OPEN,

        ("qps", "latency", "gpu_mem"),
        ("replicas", "batch"),
        ("latency_slo",)),
    Muscle("agent_containment", "ai", DIRECT, OPEN,
        ("actions", "spend", "net"),
        ("cpu_cap", "mem_cap", "net_cap", "spend_cap"),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            ("hard_cap", "instant_kill"),
            "containment only; not value alignment"),
    Muscle("value_alignment", "ai", NOT_OURS, OPEN,
            (),
            (),
            (),
            "not a muscle Omni governs"),
]


def catalog_by_name() -> Dict[str, Muscle]:
    return {m.name: m for m in CATALOG}

def wired() -> List[Muscle]:
    return [m for m in CATALOG if m.status == WIRED]

def open_muscles() -> List[Muscle]:
    return [m for m in CATALOG if m.status == OPEN]


@dataclass
class NerveEvent:

    muscle: str
    kind: str  # sense | push | hold | kill | reflex
    payload: Dict[str, Any] = field(default_factory=dict)


@dataclass

class Bundle:
    """One tick of the whole body."""
    obs: Dict[str, float]
    events: List[NerveEvent] = field(default_factory=list)
    kills: List[str] = field(default_factory=list)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def sense(self, muscle: str, **vals: float) -> None:
        m = catalog_by_name()[muscle]
        for k, v in vals.items():
            self.obs[k] = float(v)
        self.events.append(NerveEvent(muscle, "sense", dict(vals)))
        if m.status == OPEN:
            self.events.append(NerveEvent(muscle, "hold", {"reason": "
    ↪ open"}))

    def kill(self, muscle: str) -> None:
        self.kills.append(muscle)
        self.events.append(NerveEvent(muscle, "kill", {}))

def merge_stack_obs(bundle: Bundle) -> Dict[str, float]:
    """Map extra nerves onto the governor's existing observation names
    ↪ ."""
    o = dict(bundle.obs)
    aliases = {
        "utilization": "load_ratio",
        "gpu_util": "load_ratio",
        "inlet_c": "thermal",
        "gpu_temp": "thermal",
        "rapl_watts": "power_stress",
        "gpu_power": "power_stress",
        "queue_depth": "queue_ratio",
        "spend": "queue_ratio",
    }
    for src, dst in aliases.items():
        if src in o and dst not in o:
            x = float(o[src])
            if src == "inlet_c":
                x = max(0.0, (x - 18.0) / 12.0)
            elif src == "gpu_temp":
                x = max(0.0, (x - 40.0) / 40.0)
            o[dst] = x
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return o


def allow_push(muscle: str, authority: bool, killed: List[str]) -> boo
    ↪ l:
    m = catalog_by_name()[muscle]

    if muscle in killed:
        return False
    if not authority:
        return False
    if m.fit in (ADVISORY, NOT_OURS):
        return False

    return m.status == WIRED


def tag_directive(d: Dict[str, Any], authority: bool, killed: List[str
    ↪ ]) -> Dict[str, Any]:
    """Attach which muscle may execute which field. Unwired fields sta

    ↪ y logged."""
    out = dict(d)
    nerves = {
        "node_delta": "nodes",
        "actuation": "nodes",
        "hpa_target": "hpa",

        "power_cap": "power_cap",
        "route_shift": "network",
    }
    permitted = {}
    for field_name, muscle in nerves.items():
        if field_name in out:

            permitted[field_name] = allow_push(muscle, authority, kill
    ↪ ed)
    out["nerve_permit"] = permitted
    out["nerve_killed"] = list(killed)
    return out
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

H.33 omnicompass/nervous_system.py

166 lines.

```
"""The supervisory nervous system: one evolving engine state grants au
    ↪ thority to every organ family.

Omni-Compass does not replace the specialist controllers. Each keeps i
    ↪ ts own mechanism:
- the HPA and VPA size pods;
- the Cluster Autoscaler and Karpenter size node pools;
- Linux schedutil picks CPU frequency;
- NVIDIA DCGM manages GPUs;
- rollout controllers and routers do their own work.

The nervous system decides, from one shared state, the envelope inside
    ↪  which each of them may act.

Inputs, all from the six-state engine (omnicompass/core.py, via the go
    ↪ vernor), and nothing tuned per organ:
  kappa  convergence   = 1 if push <= push_release, else push_release
    ↪ / push. This is the equation-(2) gate made
                         continuous: the further the engine is from it
    ↪ s basin, the less it may give back.
  h      basin health  = clamp((U - U_gate) / (1 - U_gate), 0, 1)
  sigma  stress ratio  = S / S*, where S* solves delta - alpha_s S - (
    ↪ 3/4) beta_s S^2 = 0 (equation 6)
  nu     unmet need    = max(0, I_U) (equation 3)
  calm                 = kappa * h * (1 - clamp(sigma - 1, 0, 1)) * (1
    ↪  - clamp(nu, 0, 1))
                         in [0, 1]; 1 means fully settled

Authority per organ:
  expand     may the organ add capacity or performance. Always yes, ex
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ cept during a security hold.
  contract   may it give capacity or performance back. Only when calm
    ↪ >= the organ's reversibility threshold
             (theta_pods 0.5, theta_nodes 0.7: a node costs a boot to
    ↪ reverse) and the SLO is clean.
  step       fraction of the organ's surplus it may shed this decision

    ↪ . Equals calm.
  envelope   for continuous organs, the allowed range:
               cpufreq ceiling (fraction of cpuinfo_max) in [1 - 0.35
    ↪ calm, 1];
               GPU power limit (fraction of max) in [1 - 0.30 calm, 1]
    ↪ ;

               site power cap in the living band of its own range (not
    ↪ hing fixed: the device's own floor and top);
               traffic-shift fraction in [0, 0.5 calm];
               cooling supply-air setpoint (C) in [18, 18 + 9 calm].
             Heat above 0.96 forces the frequency and GPU ceilings dow
    ↪ n to 1 - 0.35 x excess.

  batch      admit held work only if calm >= 0.5 and power stress < 0.
    ↪ 9; pause pausable work if sigma > 1, power
             stress >= 0.95, or heat >= 0.96.
  rollback   the engine's rollback authorisation (S high, U falling, o
    ↪ r a security hold).

Global rules, applied last:
  - observe: every authority is computed and logged; execute is False
    ↪ everywhere.
  - kill: no authority at all.
  - security hold: no capacity organ may expand. Cooling is the one pr
    ↪ otective organ: "expand" there means more

    cooling, which adds protection and no capacity, so it stays allowe
    ↪ d.
The safety shield (omnicompass/shield.py) stays downstream and can sti
    ↪ ll veto any action.
"""
from __future__ import annotations
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
from dataclasses import dataclass
from typing import Any, Dict, Tuple

from omnicompass.closure import stress_equilibrium

ORGANS = ("pods", "nodes", "cpufreq", "gpu", "power", "batch", "routin
    ↪ g", "rollback")
# The living band (the founder's rule): every level the nervous system
    ↪  hands out lives between 5% and 95% of its range.
# Nothing is driven to zero (a part with no work idles down to its flo
    ↪ or, alive and ready) and nothing is driven to its

# absolute top (the last 5% is never spent). The band is applied last,
    ↪  to every envelope, so no organ can leave it.
BAND = (0.05, 0.95)


def in_band(lo: float, hi: float) -> list:

    """Clip an envelope [lo, hi] (fractions of the organ's range) into
    ↪  the living band, keeping lo <= hi."""
    b0, b1 = BAND
    h = min(b1, max(b0, hi))
    return [min(h, max(b0, lo)), h]

THETA = {"pods": 0.5, "nodes": 0.7, "cpufreq": 0.3, "gpu": 0.3, "power
    ↪ ": 0.3, "routing": 0.5}


def clamp(x, lo, hi):

    return lo if x < lo else hi if x > hi else x


@dataclass(frozen=True)
class NervousInputs:
    E: float; U: float; I_U: float; S: float
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    push: float
    push_release: float = 0.2
    U_gate: float = 0.5
    s_eq: float = 1.0
    security_block: float = 0.0
    slo_clean: bool = True

    power_stress: float = 0.0
    thermal: float = 0.0
    rollback: bool = False
    stale: float = 0.0               # fraction of declared senses tha
    ↪ t are blind (dropout, frozen, unreadable)
    mode: str = "autopilot"          # "observe" or "autopilot"

    killed: bool = False
    native_hold: Tuple[str, ...] = ()   # organs the immune system han
    ↪ ded to native (omnicompass/immune.py): no authority of Omni's


def scalars(i: NervousInputs) -> Dict[str, float]:

    kappa = 1.0 if i.push <= i.push_release else i.push_release / max(
    ↪ i.push, 1e-12)
    h = clamp((i.U - i.U_gate) / max(1.0 - i.U_gate, 1e-12), 0.0, 1.0)
    sigma = i.S / i.s_eq if i.s_eq > 0 else 0.0
    nu = max(0.0, i.I_U)
    calm = kappa * h * (1.0 - clamp(sigma - 1.0, 0.0, 1.0)) * (1.0 - c

    ↪ lamp(nu, 0.0, 1.0))
    return {"kappa": kappa, "h": h, "sigma": sigma, "nu": nu, "calm":
    ↪ clamp(calm, 0.0, 1.0)}


def authority(i: NervousInputs) -> Dict[str, Any]:

    if i.killed:
        return {"execute": False, "killed": True, "organs": {}}
    sc = scalars(i); calm = sc["calm"]
    sec = i.security_block > 0.5
    hot = i.thermal >= 0.96
    excess = clamp(i.thermal - 0.96, 0.0, 1.0)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    org: Dict[str, Dict[str, Any]] = {}
    seeing = i.stale <= 0.0          # never give capacity back on a b
    ↪ lind sense; expanding stays allowed (the safe side)
    for o, th in THETA.items():
        ok = calm >= th and i.slo_clean and seeing
        org[o] = {"expand": not sec, "contract": ok, "step": calm if o

    ↪ k else 0.0}
    cf_lo = 1.0 - 0.35 * calm; gp_lo = 1.0 - 0.30 * calm
    cf_hi = 1.0 - 0.35 * excess if hot else 1.0; gp_hi = 1.0 - 0.35 *
    ↪ excess if hot else 1.0
    org["cpufreq"]["envelope"] = in_band(min(cf_lo, cf_hi), cf_hi)
    org["gpu"]["envelope"] = in_band(min(gp_lo, gp_hi), gp_hi)

    org["power"]["envelope"] = in_band(0.0, 1.0)      # the device's o
    ↪ wn range, its last 5% at each end never spent
    # routing is an amount moved, not a level: it may move nothing, ne
    ↪ ver more than the band's top
    org["routing"]["envelope"] = [0.0, min(BAND[1], 0.5 * calm) if not
    ↪  sec else 0.0]

    org["cooling"] = {"expand": True, "protective": True, "contract":
    ↪ calm >= 0.3 and seeing, "step": calm, "envelope": [18.0, 18.0 +
    ↪ 9.0 * calm]}
    org["batch"] = {"expand": (not sec) and seeing and calm >= 0.5 and
    ↪  i.power_stress < 0.9, "contract": True, "protective_contract":
    ↪ True,

                    "admit": (not sec) and seeing and calm >= 0.5 and
    ↪ i.power_stress < 0.9,
                    "pause": sc["sigma"] > 1.0 or i.power_stress >= 0.
    ↪ 95 or hot, "step": calm}
    org["rollback"] = {"expand": False, "contract": False, "authorized
    ↪ ": bool(i.rollback or sec), "step": 0.0}

    # the immune system's hand-back propagated (the founder's order of
    ↪  25 October): an organ whose knob redlined, that
    # Omni-Compass could not bring back and where native is the strong
    ↪ er, has none of Omni-Compass's authority until returned
    for o in i.native_hold:
        if o in org:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            org[o] = {**org[o], "expand": False, "contract": False, "s
    ↪ tep": 0.0, "native": True}
    return {"execute": i.mode == "autopilot", "killed": False, "securi
    ↪ ty_hold": sec, "scalars": sc, "organs": org,
            "native_hold": [o for o in i.native_hold if o in org]}

def from_governor(g, obs: Dict[str, Any], d: Dict[str, Any] = None, mo
    ↪ de: str = None, native_hold: Tuple[str, ...] = ()) -> Dict[str,
    ↪ Any]:
    """Authority from a live omnicompass.adapter.Governor after its st
    ↪ ep (d = the step's directive); native_hold: the organs

    the immune system handed to native (omnicompass/immune.py)."""
    x = g.x; p = g.p; L = g.law
    return authority(NervousInputs(
        E=x.E, U=x.U, I_U=x.I_U, S=x.S, push=g.last_push, push_release
    ↪ =getattr(L, "push_release", 0.2),
        U_gate=getattr(L, "U_gate", 0.5), s_eq=stress_equilibrium(p.de

    ↪ lta, p.alpha_s, p.beta_s),
        security_block=float(obs.get("security_block", 0.0)), slo_clea
    ↪ n=bool(obs.get("slo_clean", True)),
        power_stress=float(obs.get("power_stress", 0.0)), thermal=floa
    ↪ t(obs.get("thermal", 0.0)),
        rollback=bool((d or {}).get("rollback_authorized", False)), st

    ↪ ale=float(obs.get("stale", 0.0)),
        mode=mode or g.mode, killed=g.killed, native_hold=tuple(native
    ↪ _hold)))


def node_release_gate(n: int, per_node_m: float, used_m: float, pendin

    ↪ g: int, pods_scaling_up: bool,
                      latency_breach_now: bool, rho: float, node_auth:
    ↪  Dict[str, Any],
                      senses_live: bool = True, last_command_landed: b
    ↪ ool = True) -> Dict[str, Any]:
    """May the machine organ give one machine back now? Attribution: a
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ n organ is held back only by stress it can cause
    or cure. The node organ reads its own engine view (fed with machin
    ↪ e-attributable pressure: pods waiting for a
    place), and the release must also pass:
      coordination  pods are not scaling up and latency is not breache
    ↪ d now (pods move first; machines never move

                    against them, the rule of the benchmarked coordina
    ↪ tion)
      headroom      nothing is pending, and after the release the rema
    ↪ ining machines run at or below the engine's own
                    utilisation target rho: used / ((n - 1) x per_node
    ↪ ) <= rho

      authority     the node organ's own calm, security and stress gat
    ↪ es (authority() above) grant contraction
      senses        every declared sense is live (afferent integrity)
      proprioception the node organ's last command landed (efferent fe
    ↪ edback: no new order to a muscle that did not
                    carry out the previous one)

    Returns {"ok": bool, "reason": str, "util_after": float}."""
    util_after = used_m / max((n - 1) * per_node_m, 1e-9) if n > 1 els
    ↪ e float("inf")
    checks = [("one machine left", n > 1), ("pods waiting", pending ==
    ↪  0), ("pods scaling up", not pods_scaling_up),
              ("latency breached now", not latency_breach_now), (f"uti

    ↪ l after {util_after:.2f} > rho {rho:.2f}", util_after <= rho),
              ("node organ has no contraction authority", bool(node_au
    ↪ th.get("organs", {}).get("nodes", {}).get("contract", False))),
              ("a sense is blind", senses_live), ("last node command d
    ↪ id not land", last_command_landed)]
    failed = [name for name, ok in checks if not ok]

    return {"ok": not failed, "reason": "; ".join(failed) or "release
    ↪ permitted", "util_after": util_after}
```

H.34 omnicompass/pools.py

38 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Pool actuation policy. Does not change the frozen allocation law.

elastic     node off is allowed (HPA+CA cattle)
always_on   node off is illegal (GPU / pet). Negative node_delta is dr

    ↪ opped.
idle_power  negative node_delta becomes park (Ready, idle watts), not
    ↪ power-off.

Pack profiles are overlays on rho0 only. Frozen law stays at rho0=0.91
    ↪ 4.

"""
from __future__ import annotations

from typing import Any, Dict

POOLS = ("elastic", "always_on", "idle_power")

PACK_MID = {"rho0": 0.82}
PACK_HARD = {"rho0": 0.95}


def apply_pool(directive: Dict[str, Any], pool: str) -> Dict[str, Any]

    ↪ :
    if pool not in POOLS:
        raise ValueError(pool)
    d = dict(directive)
    delta = int(d.get("node_delta", 0))
    if pool == "always_on":
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if delta < 0:
            d["node_delta"] = 0
        d["actuation"] = "hold"
    elif pool == "idle_power" and delta < 0:
        d["actuation"] = "park"
    else:
        d["actuation"] = "nodes" if delta != 0 else "hold"
    d["pool"] = pool
    return d
```

H.35 omnicompass/shield.py

106 lines.

```
"""Safety shield: every governor action set is checked against declare
    ↪ d invariants before execution.

Invariants (each enforced by construction; every intervention is count
    ↪ ed):
  I1 security   no capacity expansion (nodes, Terraform, rollout) whil
    ↪ e a security block is observed
  I2 bounds     node targets lie in [minimum_nodes, maximum_nodes]; po
    ↪ wer cap in [0.65, 1.0]. When the current node
                count is itself outside the bounds and the rate limit
    ↪ I5 forbids reaching them in one step, the target
                must move toward the bounds (the only admissible direc
    ↪ tion): n > max -> max <= t < n, n < min -> n < t <= min
  I3 coherence  an action set never both expands capacity and tightens
    ↪  the power cap
  I4 power      no node addition whose projected power stress exceeds
    ↪ power_limit:
                projected = observed_power_stress * (n + k) / n
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
  I5 rate       node change per interval bounded by max_node_step

The shield is a runtime monitor: it guarantees the invariants for exec
    ↪ uted governor actions
under the observed state. It does not certify the plant model.
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Dict, List, Tuple

EXPAND = ("nodes", "terraform_plan", "rollout", "replicas")

@dataclass(frozen=True)
class ShieldLimits:
    power_limit: float = 1.00
    max_node_step: int = 4

    cap_lo: float = 0.65
    cap_hi: float = 1.00


def violations(actions: List[Dict[str, Any]], st: Dict[str, Any], obs:
    ↪  Dict[str, float], cfg,

               lim: ShieldLimits = ShieldLimits()) -> List[str]:
    """Invariants violated by an action set (used to score every arm i
    ↪ dentically)."""
    v = []
    n = max(1, int(st["actual_nodes"]))
    sec = float(obs.get("security_block", 0.0)) > 0.5

    for a in actions:
        k, t, d = a["action"], a.get("target"), a.get("direction", 0)
        if sec and d > 0 and k in ("nodes", "terraform_plan", "rollout
    ↪ "):
            v.append("I1")
        if k == "nodes":
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            ti = int(t)
            inside = cfg.minimum_nodes <= ti <= cfg.maximum_nodes
            toward = (n > cfg.maximum_nodes and cfg.maximum_nodes <= t
    ↪ i < n) or (n < cfg.minimum_nodes and n < ti <= cfg.minimum_nodes
    ↪ )
            if not inside and not toward:

                v.append("I2")
            add = int(t) - n
            if abs(add) > lim.max_node_step:
                v.append("I5")
            if add > 0 and float(obs.get("power_stress", 0.0)) * (n +
    ↪ add) / n > lim.power_limit:

                v.append("I4")
        if k == "power_cap" and not lim.cap_lo <= float(t) <= lim.cap_
    ↪ hi:
            v.append("I2")
    up = any(a.get("direction", 0) > 0 and a["action"] in EXPAND for a
    ↪  in actions)

    tighten = any(a["action"] == "power_cap" and float(a["target"]) <
    ↪ float(st["power_cap"]) - 1e-9 for a in actions)
    if up and tighten:
        v.append("I3")
    return v

def enforce(actions: List[Dict[str, Any]], st: Dict[str, Any], obs: Di
    ↪ ct[str, float], cfg,
            lim: ShieldLimits = ShieldLimits()) -> Tuple[List[Dict[str
    ↪ , Any]], int]:
    """Return the largest invariant-satisfying subset/modification of

    ↪ the action set and the intervention count."""
    n = max(1, int(st["actual_nodes"]))
    sec = float(obs.get("security_block", 0.0)) > 0.5
    ps = float(obs.get("power_stress", 0.0))
    out, hits = [], 0
    for a in actions:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        a = dict(a)
        k = a["action"]
        if sec and a.get("direction", 0) > 0 and k in ("nodes", "terra
    ↪ form_plan", "rollout"):
            hits += 1
            continue

        if k == "nodes":
            r = int(a["target"])
            toward = (n > cfg.maximum_nodes and cfg.maximum_nodes <= r
    ↪  < n) or (n < cfg.minimum_nodes and n < r <= cfg.minimum_nodes)
            t = r if toward else max(cfg.minimum_nodes, min(cfg.maximu
    ↪ m_nodes, r))   # minimal intervention

            t = max(n - lim.max_node_step, min(n + lim.max_node_step,
    ↪ t))
            if t > n and ps > 0.0:
                k_max = int((lim.power_limit / ps) * n - n + 1e-9)
                t = min(t, n + max(0, k_max))
            if sec and t > n:

                hits += 1          # I1 after clamping: a clamp may tu
    ↪ rn a request into an expansion; not during a hold
                continue
            if t != int(a["target"]):
                hits += 1
            if t == n:

                continue
            a["target"], a["direction"] = t, (1 if t > n else -1)
        if k == "power_cap":
            c = max(lim.cap_lo, min(lim.cap_hi, float(a["target"])))
            if c != float(a["target"]):
                hits += 1

            a["target"] = c
        out.append(a)
    up = any(a.get("direction", 0) > 0 and a["action"] in EXPAND for a
    ↪  in out)
    if up:
        kept = []
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        for a in out:
            if a["action"] == "power_cap" and float(a["target"]) < flo
    ↪ at(st["power_cap"]) - 1e-9:
                hits += 1
                continue
            kept.append(a)

        out = kept
    return out, hits
```

H.36 omnicompass/speed.py

96 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Speed-first allocation: the frozen six-state engine, a law that nev
    ↪ er trades response time for energy.

The frozen fleet law (adapter.AllocationLaw, FLEET_MODE) uses one numb

    ↪ er, rho*, both as the HPA target (how busy each
pod may run) and as the node-sizing target (how busy each machine may
    ↪ run). Saving machines therefore always meant
running pods hotter, which is where response time was lost. This law s
    ↪ eparates the two:

  pods      HPA target = rho* from the engine (equation: rho* = clamp(
    ↪ rho0 - kI I_U - kE E, rho_min, rho0)), with
            speed constants (rho0 near the latency knee) and the respo
    ↪ nse-time nerve feeding the queue observation
  machines  sized to what the pods ask for, not to how busy they are:
            n_up   = ceil(requests / (cores * ALLOC) * (1 + headroom_u
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ p)) + ceil(kq * q * n)
            n_keep = ceil(peak requests over the last `window` decisio
    ↪ ns / (cores * ALLOC) * (1 + headroom))
            added at once when n < n_up (a pending pod costs response
    ↪ time);
            removed one per decision only when n >= n_keep + band for
    ↪ dwell decisions, no sooner than after_add
            decisions after an addition, and only while the engine's e
    ↪ quation (2) push <= push_release
  ahead     while requests are rising, n_up += ceil(antic * trend * n)
    ↪ , trend = relative rise over antic_lag
            decisions (anticipation: machines boot before the pods tha
    ↪ t need them are created; antic = 0 disables)
  power     cap = 1 whenever work is waiting or pods are busy above ca
    ↪ p_busy; otherwise cap_idle
            (a cap slows service: S = S0 / cap, so capping a busy mach
    ↪ ine always costs response time)

The engine state (E, U, I_U, S, B, B_dot) is evolved by omnicompass.co
    ↪ re unchanged; only this mapping is new.
"""
from __future__ import annotations

import math

from dataclasses import dataclass, replace

from omnicompass.adapter import Governor, mode_law, AUTOPILOT


@dataclass(frozen=True)

class SpeedLaw:
    rho0: float = 0.60
    rho_min: float = 0.50
    kI: float = 0.10
    kE: float = 0.10
    kq: float = 0.50
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    headroom: float = 0.10
    headroom_up: float = 0.0
    window: int = 10
    band: int = 1
    dwell: int = 5
    after_add: int = 5

    push_release: float = 0.05
    cap_idle: float = 1.0
    cap_busy: float = 0.30
    antic: float = 0.0          # anticipation: extra machines = ceil(
    ↪ antic x rising request trend x n)
    antic_lag: int = 4          # decisions over which the trend is me

    ↪ asured


class SpeedGovernor:
    def __init__(self, law: SpeedLaw = SpeedLaw()):
        self.law = law

        base = replace(mode_law("throughput"), rho0=law.rho0, rho_min=
    ↪ law.rho_min, kI=law.kI, kE=law.kE)
        self.g = Governor(law=base)
        self.g.set_mode(AUTOPILOT)
        self.streak = 0
        self.since_add = 99

        self.hist = []
        self.rhist = []

    def step(self, obs, n, requests, cores_per_node):
        """Returns (hpa_target, node_target, power_cap)."""
        L = self.law

        self.g.nodes = n
        d = self.g.step(obs, 0)
        q = obs["queue_ratio"]
        self.hist = (self.hist + [requests])[-max(1, L.window):]
        n_up = math.ceil(requests / cores_per_node * (1.0 + L.headroom
    ↪ _up) - 1e-9) + math.ceil(L.kq * q * n)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.rhist = (self.rhist + [requests])[-64:]
        if L.antic > 0 and len(self.rhist) > L.antic_lag:
            past = self.rhist[-1 - L.antic_lag]
            trend = max(0.0, requests - past) / max(past, 1e-9)
            n_up += math.ceil(L.antic * trend * n - 1e-9)
        n_keep = math.ceil(max(self.hist) / cores_per_node * (1.0 + L.
    ↪ headroom) - 1e-9)
        tgt = n
        if n < n_up:
            tgt = n_up
            self.streak = 0
            self.since_add = 0

        else:
            self.since_add += 1
            if n >= max(n_keep, n_up) + L.band and q < 0.05:
                self.streak += 1
                if self.streak >= L.dwell and self.since_add >= L.afte
    ↪ r_add and self.g.last_push <= L.push_release:

                    tgt = n - 1
                    self.streak = 0
            else:
                self.streak = 0
        busy = obs["load_ratio"] > L.cap_busy or q > 0.0
        cap = 1.0 if busy or tgt > n else L.cap_idle

        return float(d["demand"]), int(tgt), cap
```

H.37 omnicompass/stack_sim.py

291 lines.

```
"""Synthetic fragmented-stack model: proposal behaviour of Kubernetes,
    ↪  Terraform, HPA/VPA,
Karpenter, Argo CD/Rollouts, Istio/Cilium, power/thermal, alerting and
    ↪  paging roles.

Each block is extracted verbatim from reference/omni_compass_reference
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ _engine.py at the line shown.
"""
from __future__ import annotations
import math
from dataclasses import dataclass, field
from typing import Any, Dict, List, Tuple

import numpy as np


# --- source line 21897: ManagerBenchmarkConfig
@dataclass
class ManagerBenchmarkConfig:
    profile: str = "core"

    scenarios: int = 500
    steps: int = 48
    interval_minutes: int = 5
    initial_nodes: int = 20
    minimum_nodes: int = 8
    maximum_nodes: int = 40

    capacity_units_per_node: float = 100.0
    site_power_limit_kw: float = 33.0
    pue: float = 1.18
    node_idle_kw: float = 0.38
    node_dynamic_kw: float = 1.12
    terraform_apply_delay_steps: int = 8

    grafana_refresh_steps: int = 2
    datadog_eval_steps: int = 3
    prometheus_eval_steps: int = 2
    certification_streak: int = 4


    @property

    def dt_hours(self) -> float:
        return float(self.interval_minutes / 60.0)


# --- source line 21922: ManagerScenario
@dataclass
class ManagerScenario:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    scenario_id: int
    seed: int
    family: str
    event_start: int
    event_duration: int
    base_load_fraction: float

    spike_multiplier: float
    failed_node_fraction: float
    telemetry_delay_steps: int
    telemetry_loss_probability: float
    network_efficiency_drop: float
    power_derate: float

    security_block_probability: float
    rollout_multiplier: float
    kafka_lag_steps: int
    alert_noise_probability: float
    batch_pressure_fraction: float
    operator_delay_steps: int

# --- source line 22021: _mom_clamp
def _mom_clamp(x: float, lo: float, hi: float) -> float:
    return float(min(hi, max(lo, x)))


# --- source line 22007: _MOM_FAMILY_WEIGHTS

_MOM_FAMILY_WEIGHTS = [
    ("normal_variation", 0.10),
    ("demand_spike", 0.14),
    ("node_rack_failure", 0.12),
    ("telemetry_fault", 0.12),
    ("kubernetes_terraform_conflict", 0.12),

    ("alert_incident", 0.10),
    ("power_thermal_network", 0.12),
    ("security_policy", 0.08),
    ("deployment_rollout", 0.06),
    ("compound_failure", 0.04),
]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# --- source line 22025: _mom_allocate_families
def _mom_allocate_families(n: int) -> List[str]:
    n = max(1, int(n))
    raw = [(name, n * weight) for name, weight in _MOM_FAMILY_WEIGHTS]
    counts = {name: int(math.floor(value)) for name, value in raw}

    remainder = n - sum(counts.values())
    order = sorted(raw, key=lambda item: (item[1] - math.floor(item[1]
    ↪ )), reverse=True)
    for i in range(remainder):
        counts[order[i % len(order)][0]] += 1
    families: List[str] = []

    for name, _ in _MOM_FAMILY_WEIGHTS:
        families.extend([name] * counts[name])
    return families[:n]

# --- source line 22039: _mom_generate_scenarios
def _mom_generate_scenarios(cfg: ManagerBenchmarkConfig, seed_base: in

    ↪ t) -> List[ManagerScenario]:
    families = _mom_allocate_families(cfg.scenarios)
    scenarios: List[ManagerScenario] = []
    for idx, family in enumerate(families):
        seed = int(seed_base) + idx * 1009 + 17
        rng = np.random.default_rng(seed)

        event_start = int(rng.integers(max(4, cfg.steps // 8), max(5,
    ↪ cfg.steps // 2)))
        event_duration = int(rng.integers(max(4, cfg.steps // 10), max
    ↪ (6, cfg.steps // 3)))
        values = {
            "spike_multiplier": 1.0,

            "failed_node_fraction": 0.0,
            "telemetry_delay_steps": int(rng.integers(0, 2)),
            "telemetry_loss_probability": float(rng.uniform(0.0, 0.015
    ↪ )),
            "network_efficiency_drop": 0.0,
            "power_derate": 1.0,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            "security_block_probability": 0.0,
            "rollout_multiplier": 1.0,
            "kafka_lag_steps": int(rng.integers(0, 2)),
            "alert_noise_probability": float(rng.uniform(0.0, 0.02)),
            "batch_pressure_fraction": float(rng.uniform(0.02, 0.10)),
        }

        if family == "demand_spike":
            values["spike_multiplier"] = float(rng.uniform(1.55, 2.35)
    ↪ )
            values["batch_pressure_fraction"] = float(rng.uniform(0.12
    ↪ , 0.32))
        elif family == "node_rack_failure":

            values["failed_node_fraction"] = float(rng.uniform(0.12, 0
    ↪ .38))
        elif family == "telemetry_fault":
            values["telemetry_delay_steps"] = int(rng.integers(3, 9))
            values["telemetry_loss_probability"] = float(rng.uniform(0
    ↪ .10, 0.35))

            values["kafka_lag_steps"] = int(rng.integers(2, 7))
        elif family == "kubernetes_terraform_conflict":
            values["spike_multiplier"] = float(rng.uniform(1.30, 1.85)
    ↪ )
            values["rollout_multiplier"] = float(rng.uniform(1.10, 1.3
    ↪ 5))

        elif family == "alert_incident":
            values["alert_noise_probability"] = float(rng.uniform(0.14
    ↪ , 0.38))
            values["telemetry_delay_steps"] = int(rng.integers(1, 5))
        elif family == "power_thermal_network":
            values["network_efficiency_drop"] = float(rng.uniform(0.18

    ↪ , 0.45))
            values["power_derate"] = float(rng.uniform(0.68, 0.86))
            values["spike_multiplier"] = float(rng.uniform(1.20, 1.65)
    ↪ )
        elif family == "security_policy":
            values["security_block_probability"] = float(rng.uniform(0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ .38, 0.82))
            values["rollout_multiplier"] = float(rng.uniform(1.10, 1.3
    ↪ 0))
        elif family == "deployment_rollout":
            values["rollout_multiplier"] = float(rng.uniform(1.30, 1.8
    ↪ 0))

            values["spike_multiplier"] = float(rng.uniform(1.10, 1.40)
    ↪ )
        elif family == "compound_failure":
            values["spike_multiplier"] = float(rng.uniform(1.55, 2.25)
    ↪ )
            values["failed_node_fraction"] = float(rng.uniform(0.12, 0

    ↪ .30))
            values["telemetry_delay_steps"] = int(rng.integers(3, 8))
            values["telemetry_loss_probability"] = float(rng.uniform(0
    ↪ .10, 0.28))
            values["network_efficiency_drop"] = float(rng.uniform(0.15
    ↪ , 0.40))

            values["power_derate"] = float(rng.uniform(0.70, 0.88))
            values["security_block_probability"] = float(rng.uniform(0
    ↪ .12, 0.42))
            values["rollout_multiplier"] = float(rng.uniform(1.15, 1.5
    ↪ 5))
            values["kafka_lag_steps"] = int(rng.integers(2, 7))

            values["alert_noise_probability"] = float(rng.uniform(0.08
    ↪ , 0.24))
            values["batch_pressure_fraction"] = float(rng.uniform(0.15
    ↪ , 0.35))
        scenarios.append(ManagerScenario(
            scenario_id=idx + 1,

            seed=seed,
            family=family,
            event_start=event_start,
            event_duration=event_duration,
            base_load_fraction=float(rng.uniform(0.52, 0.76)),
            operator_delay_steps=int(rng.integers(2, 9)),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            **values,
        ))
    return scenarios

# --- source line 22123: _mom_event_active
def _mom_event_active(scenario: ManagerScenario, step: int) -> bool:
    return scenario.event_start <= step < (scenario.event_start + scen
    ↪ ario.event_duration)

# --- source line 22127: _mom_source_signals
def _mom_source_signals(scenario: ManagerScenario, step: int, cfg: Man
    ↪ agerBenchmarkConfig, rng: np.random.Generator) -> Dict[str, floa
    ↪ t]:
    phase = 2.0 * math.pi * step / max(1, cfg.steps)
    demand = scenario.base_load_fraction * (0.88 + 0.14 * math.sin(pha
    ↪ se - 0.7))
    demand *= float(rng.lognormal(mean=-0.5 * 0.035**2, sigma=0.035))
    active = _mom_event_active(scenario, step)
    if active:
        demand *= scenario.spike_multiplier
    batch = scenario.batch_pressure_fraction * (1.0 + 0.35 * math.sin(
    ↪ phase * 1.7 + 0.5))
    if active and scenario.family in ("demand_spike", "compound_failur
    ↪ e"):
        batch *= 1.65
    failed_fraction = scenario.failed_node_fraction if active else 0.0
    network_efficiency = 1.0 - (scenario.network_efficiency_drop if ac
    ↪ tive else 0.0)
    power_derate = scenario.power_derate if active else 1.0
    security_block = bool(active and rng.random() < scenario.security_
    ↪ block_probability)
    rollout = scenario.rollout_multiplier if active and scenario.famil
    ↪ y in ("kubernetes_terraform_conflict", "deployment_rollout", "se
    ↪ curity_policy", "compound_failure") else 1.0
    return {
        "demand_fraction": float(max(0.0, demand)),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        "batch_fraction": float(max(0.0, batch)),
        "failed_fraction": float(failed_fraction),
        "network_efficiency": float(_mom_clamp(network_efficiency, 0.4
    ↪ 0, 1.0)),
        "power_derate": float(_mom_clamp(power_derate, 0.55, 1.0)),
        "security_block": float(1.0 if security_block else 0.0),

        "rollout_multiplier": float(rollout),
        "event_active": float(1.0 if active else 0.0),
        "telemetry_drop": float(1.0 if rng.random() < scenario.telemet
    ↪ ry_loss_probability else 0.0),
        "alert_noise_datadog": float(1.0 if rng.random() < scenario.al
    ↪ ert_noise_probability else 0.0),

        "alert_noise_prometheus": float(1.0 if rng.random() < scenario
    ↪ .alert_noise_probability * 0.85 else 0.0),
    }

# --- source line 22157: _mom_delayed_observation
def _mom_delayed_observation(history: List[Dict[str, float]], delay_st

    ↪ eps: int) -> Dict[str, float]:
    if not history:
        return {}
    idx = max(0, len(history) - 1 - max(0, int(delay_steps)))
    return dict(history[idx])

# --- source line 22164: _mom_proposals
def _mom_proposals(state: Dict[str, Any], obs: Dict[str, float], scena
    ↪ rio: ManagerScenario, step: int, cfg: ManagerBenchmarkConfig, no
    ↪ ise_flags: Dict[str, float]) -> List[Dict[str, Any]]:
    proposals: List[Dict[str, Any]] = []
    actual_nodes = int(state["actual_nodes"])

    queue_ratio = float(obs.get("queue_ratio", 0.0))
    load_ratio = float(obs.get("load_ratio", 0.0))
    thermal = float(obs.get("thermal", 0.0))
    power_stress = float(obs.get("power_stress", 0.0))
    network_stress = float(obs.get("network_stress", 0.0))
    stale = bool(obs.get("stale", 0.0) > 0.5)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    security_block = bool(obs.get("security_block", 0.0) > 0.5)
    rollout = float(obs.get("rollout_multiplier", 1.0))

    hpa_target = int(round(state["replicas"] * _mom_clamp(0.90 + 0.55
    ↪ * load_ratio + 0.45 * queue_ratio, 0.65, 1.65)))
    hpa_target = max(20, min(500, hpa_target))

    if hpa_target != state["replicas"]:
        proposals.append({"manager":"HPA/VPA", "action":"replicas", "t
    ↪ arget":hpa_target, "direction":1 if hpa_target > state["replicas
    ↪ "] else -1})

    effective_need = max(load_ratio, queue_ratio + 0.70)

    karp_target = int(math.ceil(actual_nodes * _mom_clamp(effective_ne
    ↪ ed, 0.55, 1.65)))
    karp_target = max(cfg.minimum_nodes, min(cfg.maximum_nodes, karp_t
    ↪ arget))
    if queue_ratio > 0.25:
        karp_target = min(cfg.maximum_nodes, max(karp_target, actual_n

    ↪ odes + 1))
    elif queue_ratio < 0.05 and load_ratio < 0.63:
        karp_target = max(cfg.minimum_nodes, min(karp_target, actual_n
    ↪ odes - 1))
    if karp_target != actual_nodes:
        proposals.append({"manager":"Karpenter", "action":"nodes", "ta

    ↪ rget":karp_target, "direction":1 if karp_target > actual_nodes e
    ↪ lse -1})

    sustained = float(state["smoothed_load"])
    tf_target = int(math.ceil(cfg.initial_nodes * _mom_clamp(0.72 + 0.
    ↪ 55 * sustained + 0.35 * queue_ratio, 0.55, 1.70)))

    tf_target = max(cfg.minimum_nodes, min(cfg.maximum_nodes, tf_targe
    ↪ t))
    if scenario.family == "kubernetes_terraform_conflict" and _mom_eve
    ↪ nt_active(scenario, step):
        tf_target = max(cfg.minimum_nodes, min(cfg.maximum_nodes, tf_t
    ↪ arget + (2 if step % 2 == 0 else -2)))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if tf_target != state["terraform_desired_nodes"]:
        proposals.append({"manager":"Terraform", "action":"terraform_p
    ↪ lan", "target":tf_target, "direction":1 if tf_target > state["te
    ↪ rraform_desired_nodes"] else -1})

    if rollout > 1.05:

        proposals.append({"manager":"Argo CD / Rollouts", "action":"ro
    ↪ llout", "target":rollout, "direction":1})
        if thermal > 0.95 or power_stress > 1.0 or security_block:
            proposals.append({"manager":"Argo CD / Rollouts", "action"
    ↪ :"rollback", "target":1.0, "direction":-1})

    if thermal > 0.86 or power_stress > 0.93:
        cap = _mom_clamp(1.0 - 0.22 * max(thermal - 0.80, power_stress
    ↪  - 0.86), 0.70, 0.98)
        proposals.append({"manager":"Power/Thermal", "action":"power_c
    ↪ ap", "target":cap, "direction":-1})

    if network_stress > 0.18:
        proposals.append({"manager":"Istio/Cilium", "action":"route_sh
    ↪ ift", "target":_mom_clamp(network_stress, 0.0, 1.0), "direction"
    ↪ :1})

    if security_block:

        proposals.append({"manager":"Vault/OPA", "action":"deny_change
    ↪ ", "target":1.0, "direction":-1})

    warning = queue_ratio > 0.20 or thermal > 0.82 or power_stress > 0
    ↪ .90 or network_stress > 0.25
    critical = queue_ratio > 0.48 or thermal > 0.98 or power_stress >

    ↪ 1.02
    if warning or noise_flags.get("alert_noise_datadog", 0.0) > 0.5:
        proposals.append({"manager":"Datadog", "action":"alert", "targ
    ↪ et":2 if critical else 1, "direction":1})
    if warning or noise_flags.get("alert_noise_prometheus", 0.0) > 0.5
    ↪ :
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        proposals.append({"manager":"Prometheus/Alertmanager", "action
    ↪ ":"alert", "target":2 if critical else 1, "direction":1})
    if critical and not stale:
        proposals.append({"manager":"PagerDuty", "action":"page", "tar
    ↪ get":1, "direction":1})

    if obs.get("batch_fraction", 0.0) > 0.15:
        proposals.append({"manager":"Slurm/Ray/Kueue", "action":"batch
    ↪ _admission", "target":obs.get("batch_fraction", 0.0), "direction
    ↪ ":1})
    return proposals

# --- source line 22228: _mom_conflict_count
def _mom_conflict_count(proposals: List[Dict[str, Any]]) -> Tuple[int,
    ↪  int]:
    conflicts = 0
    duplicate_alerts = 0
    node_dirs = [p["direction"] for p in proposals if p["action"] in (

    ↪ "nodes", "terraform_plan")]
    replica_dirs = [p["direction"] for p in proposals if p["action"] =
    ↪ = "replicas"]
    rollout_dirs = [p["direction"] for p in proposals if p["action"] i
    ↪ n ("rollout", "rollback")]
    if node_dirs and min(node_dirs) < 0 < max(node_dirs):

        conflicts += 1
    if replica_dirs and min(replica_dirs) < 0 < max(replica_dirs):
        conflicts += 1
    if rollout_dirs and min(rollout_dirs) < 0 < max(rollout_dirs):
        conflicts += 1
    has_scale_up = any(p["direction"] > 0 and p["action"] in ("nodes",

    ↪  "terraform_plan", "replicas", "rollout") for p in proposals)
    has_cap = any(p["action"] == "power_cap" for p in proposals)
    has_deny = any(p["action"] == "deny_change" for p in proposals)
    if has_scale_up and has_cap:
        conflicts += 1
    if has_scale_up and has_deny:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        conflicts += 1
    alerts = [p for p in proposals if p["action"] == "alert"]
    if len(alerts) > 1:
        duplicate_alerts = len(alerts) - 1
        conflicts += 1
    return int(conflicts), int(duplicate_alerts)
```

H.38 omnicompass/storage.py

57 lines.

```
"""The engine's composite storage: the ledger that closes the circle.

Ported exactly from the engine source (composite_practical_lyapunov_va
    ↪ lue and its bath matrix, weights and S rest point,
docs/handoff/mathematics/omni_compass_engine_source_c527df2d.py); test
    ↪ s/test_core_parity.py checks it against that

source on 5,000 random states. The frozen engine (omnicompass/core.py)
    ↪  is not touched.
"""
from __future__ import annotations

import math

from typing import Dict, Tuple

from .core import EPS, Params, State, phi


def stable_S(p: Params) -> float:

    """S*, the stable rest of the axle: the root of delta - alpha_s S
    ↪ - (3/4) beta_s S^2 = 0 where the flow bends back."""
    a, b, c = 0.75 * p.beta_s, p.alpha_s, -p.delta
    if abs(a) <= EPS:
        return p.delta / max(p.alpha_s, EPS)
    disc = max(0.0, b * b - 4.0 * a * c)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    for r in ((-b + math.sqrt(disc)) / (2.0 * a), (-b - math.sqrt(disc
    ↪ )) / (2.0 * a)):
        if -p.alpha_s - 1.5 * p.beta_s * r < 0.0:
            return r
    raise RuntimeError("no stable S rest point")

def bath_P(p: Params) -> Tuple[float, float, float]:
    """(p11, p12, p22) of the positive-definite P solving A_B^T P + P
    ↪ A_B = -I for the bath (7)."""
    w = max(abs(p.omega_B), EPS); damp = max(w / max(abs(p.Q_B), EPS),
    ↪  EPS)

    p12 = 1.0 / (2.0 * w * w); p22 = (p12 + 0.5) / damp; p11 = damp *
    ↪ p12 + w * w * p22
    return p11, p12, p22


def composite_weights(p: Params) -> Dict[str, float]:

    alpha_s, beta_s = max(p.alpha_s, EPS), max(p.beta_s, EPS)
    p11, p12, p22 = bath_P(p)
    g = p.gamma_c * p.delta
    pg = (p12 * g, p22 * g)
    m_floor = alpha_s / 2.0
    return {"a_U": 1.0, "b_W": 1.0 / (2.0 * max(p.mu, EPS)), "c_E": 1.

    ↪ 0 / max(p.alpha_E, EPS),
            "w_I": 1.0 / max(p.lambda_I, EPS), "w_S": (pg[0] ** 2 + pg
    ↪ [1] ** 2 + 0.25) / (m_floor * m_floor),
            "d_B": 1.0, "S_floor": -alpha_s / (3.0 * beta_s)}

def composite_V(x: State, p: Params, sigma: int = 1) -> Dict[str, floa
    ↪ t]:
    """The composite storage of the whole engine, the ledger that clos
    ↪ es the circle:
    V = a_U (U - sigma)^2 / 2 + b_W W(U) + c_E E^2 / 2 + w_S (Phi(S) -
    ↪  Phi(S*)) + w_I I_U^2 / 2 + d_B z^T P z / 2,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    z = (B - B*, B_dot), B* = gamma_c delta S* / omega_B^2, W(U) = mu
    ↪ (U^2 - 1)^2 / 4."""
    sig = 1.0 if sigma >= 0 else -1.0
    w = composite_weights(p); s_star = stable_S(p)
    b_star = p.gamma_c * p.delta / max(abs(p.omega_B), EPS) ** 2 * s_s
    ↪ tar

    p11, p12, p22 = bath_P(p)
    z0, z1 = x.B - b_star, x.B_dot
    parts = {"V_U": 0.5 * w["a_U"] * (x.U - sig) ** 2, "V_W": w["b_W"]
    ↪  * 0.25 * p.mu * (x.U * x.U - 1.0) ** 2,
            "V_E": 0.5 * w["c_E"] * x.E ** 2, "V_S": w["w_S"] * (phi(
    ↪ x.S, p) - phi(s_star, p)),

            "V_I": 0.5 * w["w_I"] * x.I_U ** 2, "V_B": 0.5 * w["d_B"]
    ↪  * (p11 * z0 * z0 + 2 * p12 * z0 * z1 + p22 * z1 * z1)}
    return {"V": sum(parts.values()), **parts, "S_star": s_star, "B_st
    ↪ ar": b_star}
```

H.39 omnicompass/student_t.py

91 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The two-sided 95% point of Student's t distribution, for any number
    ↪  of degrees of freedom, computed (no table).

t95(ν) is the t with P(|T| ≤ t) = 0.95 for T ~ Student's t with ν degr
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ees of freedom. It is found by bisection on the
distribution's own CDF, P(|T| ≤ t) = 1 − I_x(ν/2, 1/2) with x = ν/(ν +
    ↪  t²), where I is the regularized incomplete beta
function (continued fraction, Numerical Recipes 6.4), and rounded to t
    ↪ hree decimals, as the printed tables were: 12.706 at
ν = 1, 4.303, 3.182, …, 2.262 at 9, 2.228 at 10, 2.086 at 20, 1.984 at
    ↪  100, 1.960 as ν grows without bound. So every
interval printed before (all on 9 or fewer degrees of freedom) reads t
    ↪ he same to the last digit. The hand-copied tables it
replaces fell back to the normal 1.96 past their last row, which narro
    ↪ ws an interval of 11 to 30 repetitions by 2% to 14%.

  python3 -m omnicompass.student_t 9 10 20 100   prints each ν and its
    ↪  t95
"""
from __future__ import annotations

import math

import sys
from functools import lru_cache


def _betacf(a: float, b: float, x: float) -> float:
    """The continued fraction of the incomplete beta function (modifie
    ↪ d Lentz)."""
    tiny, qab, qap, qam = 1e-300, a + b, a + 1.0, a - 1.0
    c, d = 1.0, 1.0 - qab * x / qap
    d = 1.0 / (d if abs(d) > tiny else tiny)
    h = d
    for m in range(1, 10000):
        m2 = 2 * m
        aa = m * (b - m) * x / ((qam + m2) * (a + m2))
        d = 1.0 + aa * d; d = 1.0 / (d if abs(d) > tiny else tiny)
        c = 1.0 + aa / c; c = c if abs(c) > tiny else tiny
        h *= d * c
        aa = -(a + m) * (qab + m) * x / ((a + m2) * (qap + m2))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        d = 1.0 + aa * d; d = 1.0 / (d if abs(d) > tiny else tiny)
        c = 1.0 + aa / c; c = c if abs(c) > tiny else tiny
        delta = d * c
        h *= delta
        if abs(delta - 1.0) < 1e-15:
            break
    return h


def ibeta(a: float, b: float, x: float) -> float:
    """The regularized incomplete beta function I_x(a, b), 0 ≤ x ≤ 1."
    ↪ ""
    if x <= 0.0:
        return 0.0
    if x >= 1.0:
        return 1.0
    ln_front = math.lgamma(a + b) - math.lgamma(a) - math.lgamma(b) +
    ↪ a * math.log(x) + b * math.log1p(-x)
    if x < (a + 1.0) / (a + b + 2.0):
        return math.exp(ln_front) * _betacf(a, b, x) / a
    return 1.0 - math.exp(ln_front) * _betacf(b, a, 1.0 - x) / b


def coverage(t: float, df: float) -> float:
    """P(|T| ≤ t) for Student's t with df degrees of freedom."""
    return 1.0 - ibeta(df / 2.0, 0.5, df / (df + t * t))


@lru_cache(maxsize=None)
def t95(df: int) -> float:
    """The two-sided 95% point for df degrees of freedom (df ≥ 1), to
    ↪ three decimals; NaN below one."""
    if df < 1:
        return float("nan")
    lo, hi = 0.0, 64.0                                  # t95(1) =
    ↪ 12.706 is the largest
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    for _ in range(80):
        mid = (lo + hi) / 2.0
        lo, hi = (mid, hi) if coverage(mid, df) < 0.95 else (lo, mid)
    return round((lo + hi) / 2.0, 3)

class _T95:
    """t95 read as the tables were read, T95.get(ν, fallback) and T95[
    ↪ ν], so every caller stays as written; the fallback
    answers only below one degree of freedom, where no interval exists
    ↪ ."""

    def get(self, df: int, default: float = float("nan")) -> float:
        return t95(df) if df >= 1 else default

    def __getitem__(self, df: int) -> float:
        return t95(df)

T95 = _T95()


if __name__ == "__main__":
    for arg in sys.argv[1:] or ["1", "2", "9", "10", "20", "100", "100
    ↪ 0"]:
        print(arg, f"{t95(int(arg)):.3f}")
```

H.40 omnicompass/verdict.py

190 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The verdict: Omni-Compass moves a knob only where it measures that
    ↪ the muscle loses nothing by it.

Omni-Compass sits on top of a muscle that already works (a card's firm
    ↪ ware, an autoscaler, a chiller's loop). It does not
replace that muscle, and it has no reason to move the muscle's knob un
    ↪ less moving it costs the muscle nothing. The
verdict is how the brain finds that out, on the muscle itself, before
    ↪ it acts and again from time to time:

  steps     the knob's settings from native outward, in whole steps (s
    ↪ tep 0 is native: the muscle as it runs alone)
  cost      what each setting costs the muscle's own work, measured pe
    ↪ r piece of work (a card: the milliseconds it spends
            on one request, the wait in the queue left out, so more tr
    ↪ affic is never mistaken for a slower card)

  probe     while the service is calm, a paired trial: first the knob
    ↪ at native until min_samples pieces of work are
            measured (the reference, fresh, under the same traffic), t
    ↪ hen at one step past the deepest step already
            allowed until min_samples more are measured
  judge     the trial step's median cost against the reference's: no h

    ↪ igher (within the measurement's resolution,
            tolerance) and the step is allowed; higher and it is refus
    ↪ ed, the knob goes back, and that step is not tried
            again until recheck decisions have passed (the work may ha
    ↪ ve changed). A trial that sees the service leave
            calm is abandoned, and the knob goes back at once

  allowed   the compass may move the knob only between native and the
    ↪ deepest allowed step; where no step is allowed, the
            verdict is "left native" and the knob is never moved


  stepwise  for a knob that is slow to move back (a machine given back
    ↪  takes minutes to return), incremental=True runs
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            the reference at the deepest step already allowed instead
    ↪ of at native, and judges the trial step against
            both that reference and the cost first measured at native,
    ↪  so steps can never add up past the allowance

The verdict never makes the knob more aggressive than the law: it only
    ↪  narrows where the law may go. Every probe and
every judgement is returned to the caller for the audit.

  in words  (Omni v7, judge="interval": the reports' own words, GAIN,
    ↪ EVEN and LOSS, as the code that steers the knob,
            docs/omni_v5/NEXT_VERSION_PLAN.md section 1) the trial ste
    ↪ p is read exactly as a report reads a row: the 95%
            interval of its change in cost, on the logarithm of each p
    ↪ iece of work's cost (so a heavy tail does not
            decide it), d = mean ln c_trial − mean ln c_ref ± 1.96 se.
    ↪  The interval wholly above zero is a LOSS: the step
            is refused and undone, whatever the allowance says. Wholly
    ↪  below zero is a GAIN, and across zero is EVEN.
            While the interval is wider than the allowance (its half-w
    ↪ idth over τ = ln(1 + tolerance)) the trial measures
            on, up to three times min_samples. A GAIN or an EVEN step
    ↪ is allowed only if both estimates of its change
            are within the allowance: the median (the guard of the eng
    ↪ ines before v7, kept) and the mean of the log
            costs, d ≤ τ. A stepwise step is read, the same way, again
    ↪ st the costs first measured at native. On noisy
            work this refuses a step that truly costs 3% more against
    ↪ a 2% allowance about a quarter less often than the
            medians alone allowed it, and one that costs 5% more about
    ↪  half as often, while a free step is kept almost
            as often (tests/test_verdict_interval.py). judge="median"
    ↪ keeps the engines before v7 (the medians alone).
"""
from __future__ import annotations
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
import math

Z95 = 1.96                       # the two-sided 95% interval every re
    ↪ port reads (tools/readings.py)

def median(xs):
    s = sorted(xs)
    n = len(s)
    return None if n == 0 else (s[n // 2] if n % 2 else 0.5 * (s[n //
    ↪ 2 - 1] + s[n // 2]))

def log_stats(xs):
    """(mean, variance, n) of the logarithms of the positive costs; th
    ↪ e variance is infinite with fewer than two."""
    ls = [math.log(x) for x in xs if x is not None and x > 0]
    n = len(ls)

    if n == 0:
        return None
    m = sum(ls) / n
    var = sum((x - m) ** 2 for x in ls) / (n - 1) if n > 1 else math.i
    ↪ nf
    return m, var, n

def change_interval(ref, got, z=Z95):
    """The 95% interval of the change in log cost from ref to got (Wel
    ↪ ch's standard error), or None if either is empty."""
    a, b = log_stats(ref), log_stats(got)

    if a is None or b is None:
        return None
    d = b[0] - a[0]
    se = math.sqrt(a[1] / a[2] + b[1] / b[2]) if math.isfinite(a[1]) a
    ↪ nd math.isfinite(b[1]) else math.inf
    return d - z * se, d + z * se

class Verdict:
    def __init__(self, tolerance=0.005, min_samples=30, probe_every=12
    ↪ 0, recheck=1800, max_steps=60, max_trial=30,
                 incremental=False, judge="interval"):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.tol = tolerance
        self.judge = judge
        self.native_ref = None            # the costs measured at nati
    ↪ ve (incremental, judge="interval")
        self.min_n = min_samples
        self.probe_every = probe_every

        self.recheck = recheck
        self.max_steps = max_steps
        self.max_trial = max_trial        # decisions a trial phase ma
    ↪ y take before it is abandoned
        self.incremental = incremental
        self.native_cost = None           # the cost measured at nativ

    ↪ e (incremental: the bound every step is held to)
        self.allowed = 0                  # deepest step allowed (0: n
    ↪ ative only)
        self.refused_until = {}           # step -> decision index bef
    ↪ ore which it is not tried again
        self.phase = None                 # None, "ref" (knob at nativ

    ↪ e) or "trial" (knob at the trial step)
        self.trial = None
        self.ref, self.got = [], []
        self.phase_t = 0
        self.t = 0
        self.last_probe = -10 ** 9

    @property
    def state(self):
        return "left native" if self.allowed == 0 else "acting"


    def observe(self, costs):

        """Costs (one per piece of work) measured since the last decis
    ↪ ion, under the step tick() last returned."""
        if self.phase == "ref":
            self.ref.extend(costs)
        elif self.phase == "trial":
            self.got.extend(costs)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def _end(self):
        self.phase, self.trial, self.ref, self.got = None, None, [], [
    ↪ ]

    def tick(self, calm):

        """One decision. calm: the service is inside its compass and n
    ↪ othing waits (a trial is never run under stress).
        Returns (the step the knob must stand at during a trial, or th
    ↪ e deepest step the law may use; is_trial; event).
        The event, if any, is for the audit."""
        self.t += 1

        event = None
        if self.phase is not None and (not calm or self.t - self.phase
    ↪ _t > self.max_trial):
            event = {"verdict": "trial abandoned (service not calm)" i
    ↪ f not calm else "trial abandoned (too little work)",
                     "step": self.trial}

            self._end()
        elif self.phase == "ref" and len(self.ref) >= self.min_n:
            self.phase, self.phase_t = "trial", self.t
        elif self.phase == "trial" and len(self.got) >= self.min_n:
            m0, mk = median(self.ref), median(self.got)
            if self.incremental and self.allowed == 0:

                self.native_cost = m0
                self.native_ref = list(self.ref)
            bound = m0 if self.native_cost is None else min(m0, self.n
    ↪ ative_cost)
            within = mk <= bound * (1.0 + self.tol)
            if self.judge == "interval":

                reading, ivs, wide, d_max = self._reading()
                if reading != "LOSS" and wide and len(self.got) < 3 *
    ↪ self.min_n:
                    return self._stand()                        # the
    ↪  evidence is wider than the allowance: it measures on
                if reading != "LOSS" and within and d_max <= math.log(
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ 1.0 + self.tol):
                    self.allowed = self.trial
                    event = {"verdict": "step allowed", "step": self.t
    ↪ rial, "cost_native": m0, "cost_step": mk,
                             "reading": reading, "interval_ln": ivs}
                else:

                    self.refused_until[self.trial] = self.t + self.rec
    ↪ heck
                    event = {"verdict": "step refused: the muscle is s
    ↪ lower there", "step": self.trial,
                             "cost_native": m0, "cost_step": mk,
                             "reading": "LOSS" if reading == "LOSS" el

    ↪ se "past the allowance", "interval_ln": ivs}
            elif within:
                self.allowed = self.trial
                event = {"verdict": "step allowed", "step": self.trial
    ↪ , "cost_native": m0, "cost_step": mk}
            else:

                self.refused_until[self.trial] = self.t + self.recheck
                event = {"verdict": "step refused: the muscle is slowe
    ↪ r there", "step": self.trial,
                         "cost_native": m0, "cost_step": mk}
            self._end()
        elif (self.phase is None and calm and self.t - self.last_probe

    ↪  >= self.probe_every
              and self.allowed < self.max_steps and self.refused_until
    ↪ .get(self.allowed + 1, -1) <= self.t):
            self.phase, self.trial, self.phase_t, self.last_probe = "r
    ↪ ef", self.allowed + 1, self.t, self.t
            self.ref, self.got = [], []

            event = {"verdict": "trial", "step": self.trial}
        if self.phase == "ref":
            return (self.allowed if self.incremental else 0), True, ev
    ↪ ent
        if self.phase == "trial":
            return self.trial, True, event
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return self.allowed, False, event

    def _stand(self):
        """The knob stays at the trial step for another decision (the
    ↪ trial measures on)."""
        return self.trial, True, None

    def _reading(self):
        """The trial's reading in the reports' words (tools/readings.p
    ↪ y: a 95% interval wholly on one side of zero is a
        GAIN or a LOSS, across zero is EVEN), against the reference an
    ↪ d, stepwise, against native: LOSS if any interval

        is wholly above zero, GAIN if every interval is wholly below i
    ↪ t, else EVEN. Returns the reading, the intervals
        (for the audit), whether any interval is wider than the allowa
    ↪ nce (half-width over τ) and the largest estimate d."""
        tau = math.log(1.0 + self.tol)
        ivs = [change_interval(self.ref, self.got)]

        if self.incremental and self.native_ref:
            ivs.append(change_interval(self.native_ref, self.got))
        ivs = [iv for iv in ivs if iv is not None]
        if not ivs:
            return "EVEN", [], True, math.inf
        if any(lo > 0.0 for lo, _ in ivs):

            reading = "LOSS"
        elif all(hi < 0.0 for _, hi in ivs):
            reading = "GAIN"
        else:
            reading = "EVEN"
        wide = any((hi - lo) / 2.0 > tau for lo, hi in ivs)

        d_max = max((lo + hi) / 2.0 for lo, hi in ivs)
        return reading, [[round(lo, 5), round(hi, 5)] for lo, hi in iv
    ↪ s], wide, d_max
```

H.41 `realms/__init__.py`

6 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The realm harness: the muscle tower (945 in Omni v2) on modelled pl
    ↪ ants, native against Omni on top (docs/REALMS.md)."""
```

H.42 realms/compass_arm.py

186 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The compass law on every realm muscle (omnicompass/compass_law.py):
    ↪  each plant's knob held by its own compass.

Each plant's service is read as one position in its compass (0 calm, 1

    ↪  the line): the worst of its queue or lateness, its
load above half, and, for a storage site, its draw on the grid connect
    ↪ ion above half. The last period in violation
reads as past the wall (fail up). The compass's force moves the knob:
    ↪ a positive force adds capacity, power, cooling or
protection, a negative force gives it back, each knob inside its own c
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ over. At the kill every knob returns to native.

Knob by plant (the override key the plant already obeys, its native va
    ↪ lue, its cover, and which way is "more"):
  compute_pool   power: limit share 0.4-1; replicas-type: the HPA targ
    ↪ et held at native (loosening it past the

                 operator's own costs the service time, the same rule
    ↪ as the card's speed floor); machine pools: one
                 machine released while the force is clearly down and
    ↪ the machines left cover the recent peak with
                 margin (RELEASE_MARGIN); admission: native
  thermal_zone   setpoint between the band's stress (colder, more cool

    ↪ ing) and calm ends; units: one released while the
                 force is clearly down; power: native (a cooling cap m
    ↪ oves heat later, never away)
  energy_storage reserve between the band's ends (a lower reserve give
    ↪ s the battery to the site); power: native
  motion_axis    speed and effort: 0.4-1

  process_loop   setpoint from native toward the band's calm end; actu
    ↪ ator range 0.3-1; power 0.4-1
"""
from __future__ import annotations

from omnicompass.compass_law import Band, CompassLaw, clamp

UP, DOWN, RELEASE = 0.10, 0.02, -0.2
RELEASE_MARGIN = 0.3     # a machine goes back only when the rest cove
    ↪ rs the recent peak at 0.3 of the plant's own release level
# 0.3: the largest margin at which no node pool is later than native (
    ↪ results/realms/RELEASE_MARGIN_SWEEP.md, the

# per-muscle sweep of 2026-10-05: 0.6 saved 2.0% energy with 32 of 42
    ↪ pools later, 0.4 left 4 later, 0.3 none). A machine
# boots in minutes and a burst that comes in the meantime is served la
    ↪ te, so speed first holds the machines (the same
# rule as the verdict on real Kubernetes: a machine goes back only whe
    ↪ re it costs no speed)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def position(plant) -> float:
    if plant.vhist and plant.vhist[-1]:
        return 1.0
    if plant.template == "thermal_zone":                      # a zon
    ↪ e's service is its temperature against its limit
        P = plant.P
        return (plant.T - min(P["stress"], P["calm"])) / (P["t_limit"]
    ↪  - min(P["stress"], P["calm"]))
    o = plant.observe()
    p = max(o["queue_ratio"], (o["load_ratio"] - 0.5) / 0.5)
    if plant.template == "energy_storage":
        p = max(p, (o["power_stress"] - 0.5) / 0.5)
    return p


def release_safe(plant) -> bool:
    """A discrete unit goes back only when what is left covers the rec
    ↪ ent peak (the plant's own release window), with
    RELEASE_MARGIN of headroom on a machine pool: a machine boots in m
    ↪ inutes, so a burst that arrives after a release
    is served late until it is back."""
    P = plant.P
    if plant.template == "compute_pool":
        from .plants import RELEASE_WINDOW, RELEASE_FRAC
        n = plant.n
        return (n > P["n_min"] and not plant.pending and plant.Q <= 0.
    ↪ 0
                and len(plant.ahist) == RELEASE_WINDOW * P["startup_st
    ↪ eps"]
                and max(plant.ahist) / ((n - 1) * plant.mu(plant.cap))
    ↪  <= RELEASE_FRAC * 0.95 * RELEASE_MARGIN)
    u = plant.units_on
    return u > 1 and plant.T <= plant.setpoint() and plant.qc_cmd_avg
    ↪ / ((u - 1) * P["q_unit_w"]) <= 0.8
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def capping_saves(plant) -> bool:
    """Whether a lower power cap saves energy on this pool at all (rac
    ↪ e to idle, MECHANISM_OF_ACTION 9.6).

    With throughput mu(c) ~ c^eps at power cap c and the autoscaler ho
    ↪ lding utilisation near its target u*, the energy
    per unit of work is (p_idle / u* + p_dyn c) / mu(c). Its slope at
    ↪ c = 1 is positive (a lower cap saves) only if
    eps < p_dyn / (p_idle / u* + p_dyn). Where idle power dominates (a
    ↪  quantum computer's cryostat, a network switch),

    running slower keeps the machines on longer and costs more than it
    ↪  saves: the cap is left native."""
    import math
    P = plant.P
    eps = math.log(plant.mu(1.0) / plant.mu(0.99)) / math.log(1.0 / 0.
    ↪ 99)   # the machine's own speed curve at full power

    u = P.get("target", 0.7)
    return eps < P["p_dyn"] / (P["p_idle"] / u + P["p_dyn"])


def effort_cap_saves(plant) -> bool:
    """Whether a lower effort cap saves energy on this axis at all (ME

    ↪ CHANISM_OF_ACTION 9.6).


    A cap on effort lowers the copper loss of a full-acceleration move
    ↪ , (J a_max / kt)^2 R, and stretches the move, during
    which the axis keeps drawing its idle power and turning against vi
    ↪ scous friction, p_idle + b v_max^2. It pays only

    where the first exceeds the second (a reaction wheel); elsewhere (
    ↪ traction, a flight axis, a robot joint) a slower
    move costs more than the current it saves, and the cap is left nat
    ↪ ive."""
    P = plant.P
    copper = (P["J"] * P["a_max"] / P["kt"]) ** 2 * P["R"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    standing = P["p_idle"] + P["b"] * P["v_max"] ** 2
    return copper > standing


SLACK = 0.5              # a speed knob is offered only where the axis
    ↪  at full speed is busy at most half the time (Omni v3)

def speed_slack(plant) -> bool:
    """Whether the axis has slack to spend on a slower, gentler move a
    ↪ t all (Omni v3, docs/REALMS_PREREGISTRATION.md).

    The compass may ease a motion axis down to 0.4 of its speed, which
    ↪  stretches every move by up to 2.5 times. An axis
    whose duty at full speed, task rate x (move time + dwell), is alre
    ↪ ady above one half has no room for that: tasks
    arrive in bunches, and a stretched move pushes the next ones past
    ↪ their deadline (a train on its timetable, a lift at

    rush hour). There the speed knob is left native, as the robot benc
    ↪ hmark's paired trial leaves it
    (docs/ROBOTICS_PREREGISTRATION.md). Computed from the plant's own
    ↪ figures before any decision."""
    import math
    P = plant.P

    v, a, D = P["v_max"], P["a_max"], P["D"]
    move = 2.0 * math.sqrt(D / a) if D < v * v / a else D / v + v / a
    return P["task_rate"] * (move + P["dwell"]) <= SLACK


def lever(plant, knob):

    """(override key, native value, low, high, sign): sign +1 when a h
    ↪ igher value is more capacity."""
    P, t = plant.P, plant.template
    if knob == "admission":
        return None
    if t == "compute_pool":
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if knob == "power":
            return ("power", 1.0, 0.4, 1.0, 1) if capping_saves(plant)
    ↪  else None
        if P.get("ca"):
            return ("release", None, 0, 0, 0)
        return None                                              # the H
    ↪ PA target is held at the operator's own
    if t == "thermal_zone":
        if knob == "setpoint":
            return ("setpoint", P["t_set"], min(P["stress"], P["calm"]
    ↪ ), max(P["stress"], P["calm"]), -1)
        if knob == "capacity":

            return ("release", None, 0, 0, 0)
        # the cooling power cap gives nothing back: the heat the zone
    ↪ makes must leave it either way, so a lower cap only
        # lets the temperature drift up while the PI command grows and
    ↪  stages more units (and their fans). Native
        # (MECHANISM_OF_ACTION 9.6)

        return None
    if t == "energy_storage":
        if knob == "setpoint" and P.get("backup"):
            return None                                          # a bac
    ↪ kup reserve (a UPS) is kept for an outage: native
        if knob == "setpoint":

            # the reserve may go down to the stress end (the battery g
    ↪ iven to the site under strain), never above the
            # operator's own: a kWh held back in a calm battery is a k
    ↪ Wh bought from the grid (MECHANISM_OF_ACTION 9.6)
            return ("setpoint", P["reserve"], min(P["stress"], P["rese
    ↪ rve"]), P["reserve"], -1)

        return None                                              # a bat
    ↪ tery's power limit gives nothing back: native
    if t == "motion_axis":
        if knob == "power":
            return ("power", 1.0, 0.4, 1.0, 1) if effort_cap_saves(pla
    ↪ nt) else None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return ("capacity", 1.0, 0.4, 1.0, 1) if speed_slack(plant) el
    ↪ se None
    if t == "process_loop":
        if knob == "setpoint":
            lo, hi = min(P["sp"], P["calm"]), max(P["sp"], P["calm"])
            return ("setpoint", P["sp"], lo, hi, 1 if P["sp"] > P["cal

    ↪ m"] else -1)
        if knob == "power":
            return ("power", 1.0, 0.4, 1.0, 1)
        return ("capacity", 1.0, 0.3, 1.0, 1)
    return None

def compass_apply(plant, knob, dt=1.0, tau=3.0):
    """One decision: the plant's compass reads its position and moves
    ↪ its knob. Returns the override written."""
    lv = lever(plant, knob)
    if lv is None:

        plant.override = {}
        return {}
    if not hasattr(plant, "_compass_law"):
        plant._compass_law = CompassLaw(Band(0.0, 1.0), dt=dt, tau=tau
    ↪ , kp=1.0, smooth=0.3)
        plant._compass_law.kd *= 3.0

        plant._compass_law_x = lv[1]
    b = plant._compass_law
    F = b.force(position(plant))
    key, native, lo, hi, sign = lv
    if key == "release":
        ok = F < RELEASE and b.p < b.band.center and release_safe(plan

    ↪ t)
        plant.override = {"release": 1.0} if ok else {}
        return plant.override
    span = hi - lo
    if key == "setpoint" and plant.template == "energy_storage":
        # a battery's reserve is spent only where it buys service: at
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ the connection's wall the reserve drops to the
        # stress end at once; anywhere else the operator's own reserve
    ↪ , so no round trip is paid for nothing (9.6)
        P = plant.P
        excess = plant.imp - P["p_lim_w"]                       # what
    ↪  the connection is over its limit by, now

        coverable = 0.0 < excess <= P["p_batt_w"] * plant.override.get
    ↪ ("power", 1.0)
        x = lo if (b.p >= b.band.wall_high and (coverable or plant.imp
    ↪  >= 0.95 * P["p_lim_w"] and excess <= 0.0)) else native
        plant._compass_law_x = x
        plant.override = {} if abs(x - native) < 1e-9 else {key: x}

        return plant.override
    if b.p >= b.band.wall_high:
        x = hi if sign > 0 else lo                                  # f
    ↪ ail up: full capacity at once
    else:
        g = UP if F > 0 else DOWN

        x = clamp(plant._compass_law_x + sign * g * F * span, lo, hi)
    plant._compass_law_x = x
    plant.override = {} if abs(x - native) < 1e-9 else {key: x}
    return plant.override
```

H.43 realms/compass_arm_v5.py

1309 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# The Omni v5 compass law, in the engine (additive beside the frozen v
    ↪ 3 law in realms/compass_arm.py). It is the engine
# change that makes this tree omni-v5: two math-only guardrails (the e
    ↪ dge, so nothing riding its limit falls off; the

# centre/hole, so nothing drops into a pit it cannot climb out of), fr
    ↪ ee arms otherwise, every move proven to harm nothing
# native holds on the modelled organism. Run by the v5 arm in realms/h
    ↪ arness.py and tools/run_realms_v5.py. Evidence class S.
#
#   free arms     every knob may move both ways inside its physical co

    ↪ ver; the only exclusions are knobs whose own
#                 physics says moving them cannot lower energy (a cool
    ↪ ing cap, a battery's power cap, a backup reserve,
#                 admission, a power cap on a machine racing to idle):
    ↪  the brain's knowledge, not a guardrail
#   the edge      the compass's balance point is not a fixed middle: i

    ↪ t is 1 - cushion, where
#                   cushion = climb (how fast the service has climbed
    ↪ lately, per decision) x reaction (decisions the knob
#                             takes to act) + z x wobble (the service'
    ↪ s own spread lately)
#                 widened at once when the need rises, narrowed slowly

    ↪  when it falls; past 1 - climb x reaction the
#                 law fails up at once (the wall moves with the cushio
    ↪ n too)
#   the hole      nothing is eased into a pit it cannot climb out of:
    ↪ a cooled zone never warmer than its cooling can
#                 recover (the Lindley reach), a process loop's setpoi

    ↪ nt never closer to its band's edge than the loop's
#                 own largest swing toward that edge (the loop reach),
    ↪  a serving pool never capped below the largest jump
#                 in demand it has shown; and nothing permanent: a mac
    ↪ hine handed back to a cloud is the cluster
#                 autoscaler's own decision (it shrinks from wherever
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ it is left, so an early hand-back would leave the
#                 pool a machine short of native for good)
from __future__ import annotations

import copy
import csv

import math
from pathlib import Path

from omnicompass.compass_law import Band, CompassLaw
from omnicompass.edge import Edge as _CoreEdge, physics_memory
from .compass_arm import (UP, DOWN, RELEASE, clamp, position, release_

    ↪ safe, capping_saves, effort_cap_saves,
                        speed_slack)
from .plants import CATCH_UP


class Cfg:

    def __init__(self, z=3.09, window=20, react_mult=1.0, narrow=0.05,
    ↪  start=0.5, target_hi=0.95, spend_beyond=False,
                memory=0.0, target_free=True, est=1, snap=True, learn
    ↪ _fall=0.0, fall_decay=0.9995, hybrid=False,
                base_center=0.5, base_wall=0.95, headroom=False, phys
    ↪ ics_memory=False, motion_safe=False,

                motion_q=0.999, burst_window=20, hole_guard=False, dr
    ↪ op_guard=False, jump0=1.0, band=False,
                hole_floor=True, precool=False, storage_shave=False,
    ↪ power_cap=False, loop_reach=False,
                ca_release=False, dial="save", edge_util=0.95, line_z
    ↪ =3.09, hold_window=None, hold_margin=0.05,

                heat_head=0.8, strain=0.5):
        self.dial = dial                        # "max" (the default bra
    ↪ in: every muscle pushed to its own limit for the most
                                                # work, the energy that
    ↪ takes shown as power given) or "save" (the tune-down:
                                                # the least energy for n
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ative's work)
        self.edge_util = edge_util            # max: how full a machin
    ↪ e pool runs on the forecast before it calls for more
        self.line_z = line_z                  # max: the wobbles of ma
    ↪ rgin a battery keeps its site's draw under its line
        self.hold_window = hold_window        # max: decisions a batte
    ↪ ry wired by its power reads its line before it holds
                                              # anything (None: the wi
    ↪ ndow)
        self.hold_margin = hold_margin        # max: the share of its
    ↪ line a battery wired by its power keeps clear at least
        self.heat_head = heat_head            # max: an axis is pushed
    ↪  only while its motor is under this share of its heat rise
        self.strain = strain                  # max: and its tracking
    ↪ error under this share of its line
        self.loop_reach = loop_reach          # a process loop's setpo
    ↪ int never closer to its band's calm-side edge than the
                                              # loop's own largest swi
    ↪ ng toward that edge leaves room for (loop_clamp)
        self.ca_release = ca_release          # a machine pool under a
    ↪  cluster autoscaler: Omni may hand a machine back
                                              # (off in the default: t
    ↪ he autoscaler shrinks from wherever it is left, so an
                                              # early hand-back is per
    ↪ manent; the reflex rule: nothing permanent)
        self.storage_shave = storage_shave    # battery peak-shaving (
    ↪ discharge to hold an import limit) costs round-trip
                                              # energy, so it is a ser
    ↪ vice<->energy tradeoff, off in the no-harm default;
                                              # a service/Max dial tur
    ↪ ns it on. Off: the storage setpoint stays native.
        self.power_cap = power_cap            # a power cap on a servi
    ↪ ng pool whose response line is longer than one decision
                                              # (lever_v5); an interac
    ↪ tive pool's cap stays native either way
        self.hole_floor = hole_floor          # the hole guard never b
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ elow the earlier law's balance point (hybrid)
        self.precool = precool                # a cooled zone may also
    ↪  sit warm while its headroom can pull it back in time
        self.band = band                      # a process loop's margi
    ↪ n read against its band's edge, not its setpoint
        self.jump0 = jump0                    # the jump in demand the
    ↪  brain assumes a pool can make in one decision until it
                                              # has seen a larger one
    ↪ (1.0: demand can double)
        self.drop_guard = drop_guard          # a pool that can turn r
    ↪ equests away is never capped below the largest jump
                                              # in demand it has seen
    ↪ (a request turned away is work lost for good)
        self.hole_guard = hole_guard          # a cooled zone is never
    ↪  warmer than it can be pulled back from before its
                                              # cooling could run out
    ↪ (the hole a saturated cooling plant cannot climb out of)
        self.motion_safe = motion_safe        # a motion axis's speed
    ↪ and effort never below what its own physics needs
        self.motion_q = motion_q              # the share of next-deci
    ↪ sion arrival counts the speed must cover (z = 3.09)
        self.burst_window = burst_window      # decisions of arrival h
    ↪ istory the rate estimate keeps its peak over
        self.z, self.window, self.react_mult, self.narrow, self.start
    ↪ = z, window, react_mult, narrow, start
        self.memory = memory                  # 0: only the recent win
    ↪ dow; near 1: the worst climb ever seen, forgotten slowly
        self.target_free = target_free        # the autoscaler's targe
    ↪ t free both ways (False: held at the operator's, as v3)
        self.est = est                        # 1: level spread and th
    ↪ e steepest one-step climb x reaction; 2: the step-to-step
                                              # wobble and the largest
    ↪  rise seen within one reaction time
        self.snap = snap                      # at the wall: snap back
    ↪  to native (True) or climb back at the law's full force
        self.learn_fall = learn_fall          # > 0: a miss while Omni
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  had given something back raises that plant's cushion floor
        self.fall_decay = fall_decay          # how slowly that floor
    ↪ is forgotten, per decision
        self.hybrid = hybrid                  # True: everything the e
    ↪ arlier law did stays (its balance point, its wall, its
                                              # fail-up), and the edge

    ↪  only ever adds: the balance point is the higher of the
                                              # earlier one and the ed
    ↪ ge, the wall the higher of the earlier one and the edge's
        self.base_center, self.base_wall = base_center, base_wall
        self.headroom = headroom              # a cooling zone's headr
    ↪ oom as a second gauge

        self.physics_memory = physics_memory  # each plant remembers i
    ↪ ts risks for as long as its own physics carries them
        self.target_hi = target_hi            # a compute pool's utili
    ↪ zation may never be set past this (a queue at 1 explodes)
        self.spend_beyond = spend_beyond      # spend beyond native aw
    ↪ ay from the wall (the speed edge); False: the energy edge

def reaction_steps(plant, key=None) -> float:
    """Decisions the knob takes to act: the knob's own, not the plant'
    ↪ s (a power cap answers in one decision; a machine
    pool answers only once its machines have started)."""

    P, t = plant.P, plant.template
    dt = float(P.get("dt", P.get("dt_dec", 15.0)))
    if t == "compute_pool":
        if key == "power":
            return 1.0
        return max(1.0, float(P.get("startup_steps", 1)))

    if t == "process_loop":
        return max(1.0, (3.0 * P.get("tau", dt) + P.get("theta", 0.0))
    ↪  / dt)
    if t == "thermal_zone":
        # the cooling's own reaction: how long the spare cooling takes
    ↪  to pull the zone back across its band (the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        # passive time constant, hours, is how slowly it drifts withou
    ↪ t cooling, not how fast the loop answers)
        spare = max(P["units"] * P["q_unit_w"] - P["q_it_w"], 0.1 * P[
    ↪ "q_it_w"])
        return max(1.0, P["c_j_k"] * (P["t_limit"] - P["t_set"]) / spa
    ↪ re / dt)

    return 1.0


def lever_v5(plant, knob, cfg):
    """(override key, native, low, high, sign): sign +1 when a higher
    ↪ value is more capacity."""

    P, t = plant.P, plant.template
    if knob == "admission":
        return None
    if t == "compute_pool":
        if knob == "power":
            # a pool whose response line is shorter than one decision

    ↪ (an interactive GPU or CPU, a radio, a switch) cannot
            # have a cap's one decision of slower service seen and und
    ↪ one inside its line, so its cap stays native (the
            # metered GPU, class P, is where an interactive cap is pro
    ↪ ven); a pool whose line is longer than a decision
            # (a batch queue, a training job, a terminal's moves) abso

    ↪ rbs it, held by the drop and latency floors
            ok = cfg.power_cap and capping_saves(plant) and P["slo_s"]
    ↪  >= P["dt"]
            return ("power", 1.0, 0.4, 1.0, 1) if ok else None
        if P.get("ca"):
            return ("release", None, 0, 0, 0) if cfg.ca_release else N

    ↪ one
        # the autoscaler's target, free both ways: lower is more machi
    ↪ nes, higher is fewer, never past target_hi
        if not cfg.target_free:
            return None
        return ("target", P["target"], 0.6 * P["target"], max(P["targe
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ t"], cfg.target_hi), -1)
    if t == "thermal_zone":
        if knob == "setpoint":
            return ("setpoint", P["t_set"], min(P["stress"], P["calm"]
    ↪ ), max(P["stress"], P["calm"]), -1)
        if knob == "capacity":

            return ("release", None, 0, 0, 0)
        return None
    if t == "energy_storage":
        if knob == "setpoint" and not P.get("backup") and cfg.storage_
    ↪ shave:
            return ("setpoint", P["reserve"], min(P["stress"], P["rese

    ↪ rve"]), P["reserve"], -1)
        return None
    if t == "motion_axis":
        if knob == "power":
            return ("power", 1.0, 0.4, 1.0, 1) if effort_cap_saves(pla
    ↪ nt) else None

        return ("capacity", 1.0, 0.4, 1.0, 1) if speed_slack(plant) el
    ↪ se None
    if t == "process_loop":
        if knob == "setpoint":
            lo, hi = min(P["sp"], P["calm"]), max(P["sp"], P["calm"])
            return ("setpoint", P["sp"], lo, hi, 1 if P["sp"] > P["cal

    ↪ m"] else -1)
        if knob == "power":
            return ("power", 1.0, 0.4, 1.0, 1)
        return ("capacity", 1.0, 0.3, 1.0, 1)
    return None

def memory_horizon(plant) -> float:
    """Decisions a plant carries a risk: its own slowest physics (a ro
    ↪ om's heat capacity over its loss to outside, a
    process loop's three time constants, a machine pool's start)."""
    P, t = plant.P, plant.template
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    dt = float(P.get("dt", P.get("dt_dec", 15.0)))
    if t == "thermal_zone":
        return 3.0 * P["c_j_k"] / P["ua"] / dt
    if t == "process_loop":
        return (3.0 * P.get("tau", dt) + P.get("theta", 0.0)) / dt
    if t == "compute_pool":

        return float(P.get("startup_steps", 1))
    return 1.0


class Edge(_CoreEdge):
    """The v5 edge (omnicompass/edge.py) for one plant's knob: the kno

    ↪ b's own reaction, and, with physics memory, its
    risks kept as long as the plant's own slowest physics carries them
    ↪ ."""

    def __init__(self, plant, cfg, key=None):
        mem = cfg.memory

        if cfg.physics_memory:
            mem = physics_memory(memory_horizon(plant), cfg.window)
        super().__init__(reaction_steps(plant, key) * cfg.react_mult,
    ↪ z=cfg.z, window=cfg.window, narrow=cfg.narrow,
                         start=cfg.start, memory=mem, fall_decay=cfg.f
    ↪ all_decay, est=cfg.est)

        self.cfg = cfg


def deadline_share(plant) -> float:
    """A motion axis's lateness ahead: the oldest waiting task's wait
    ↪ plus the time to clear the whole backlog at the

    axis's present speed, as a share of the deadline (0 when nothing w
    ↪ aits). It reads a late task before it is late."""
    w = getattr(plant, "waits", None)
    if not w:
        return 0.0
    P = plant.P
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    clear = len(w) * (plant.move_time(plant.s) + P["dwell"])
    return (w[0] + clear) / P["deadline_s"]


def poisson_hi(mean: float, q: float) -> int:
    """The smallest count n with P(N <= n) >= q for a Poisson count of
    ↪  this mean."""
    if mean <= 0.0:
        return 0
    p = math.exp(-mean)
    c, n = p, 0
    while c < q and n < 100000:
        n += 1
        p *= mean / n
        c += p
    return n

def serve_time(plant, n: int, s: float) -> float:
    """Seconds to finish n tasks when the moves started within the nex
    ↪ t decision run at speed s and every later one at
    full speed (the brain's next decision can restore the speed only f
    ↪ or moves not yet started)."""
    if n <= 0:
        return 0.0
    P = plant.P
    cs = plant.move_time(s) + P["dwell"]
    c1 = plant.move_time(1.0) + P["dwell"]
    m = min(n, max(1, math.ceil(P["dt_dec"] / cs)))
    return m * cs + (n - m) * c1

def safe_speed(plant, cfg, lo: float) -> float:
    """The slowest speed a motion axis may be given at this decision,
    ↪ from its own physics:
      a move that would outlast one decision is never slowed (once sta
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ rted its speed cannot be taken back, so the knob's
        reaction time is the whole move, and every task arriving meanw
    ↪ hile would wait behind it);
      a queue is never slowed (every task behind the move would finish
    ↪  later);
      the tasks waiting now and the most that can arrive in the next d

    ↪ ecision (the motion_q share of a Poisson count at the
        recent peak rate) are all finished inside the deadline even if
    ↪  the whole burst arrives at once;
      and the wake forecast over a whole deadline (the most that can a
    ↪ rrive in one deadline at the recent peak rate) is
        finished inside that deadline at the slowed speed, so the axis

    ↪  always goes idle again and a delay is recovered
        before a later rush could make it permanent (an axis one task
    ↪ behind native in a rush stays behind it).
    1.0 is native (full speed): when work comes, the axis wakes straig
    ↪ ht to it."""
    P = plant.P

    dt = P["dt_dec"]
    hist = getattr(plant, "_v5_arr", None)
    if hist is None:
        hist = plant._v5_arr = []
    k = plant.k
    if k > 0:

        hist.append(plant.arrivals[k - 1])        # what the brain cou
    ↪ nted arriving in the last decision
        if len(hist) > cfg.burst_window:
            hist.pop(0)
    if plant.backlog > 1 or plant.move_time(1.0) + P["dwell"] > dt:
        return 1.0

    lam = max(plant.demand_rate, (max(hist) / dt) if hist else 0.0)
    burst = plant.backlog + poisson_hi(lam * dt, cfg.motion_q)
    w0 = plant.waits[0] if plant.waits else 0.0
    room = P["deadline_s"] - w0
    # the wake forecast over a whole deadline: the motion_q share of t
    ↪ he tasks that can arrive in one deadline at the recent
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    # peak rate. A slowed axis must still finish all of them inside th
    ↪ at deadline, so it always goes idle again and any
    # delay is recovered before a later rush could make it permanent (
    ↪ a task one behind native in a rush stays behind)
    H = P["deadline_s"]
    n_h = poisson_hi(lam * H, cfg.motion_q)

    s = lo
    while s < 1.0:
        cycle = plant.move_time(s) + P["dwell"]
        if cycle <= dt and serve_time(plant, burst, s) <= room and n_h
    ↪  * cycle <= H:
            return s

        s = min(1.0, s + 0.02)
    return 1.0


def effort_floor(plant) -> float:
    """The least effort a motion axis may be capped to: the torque its

    ↪  next full-speed move needs (inertia times the
    acceleration, the viscous drag at top speed, the standing load and
    ↪  the largest disturbance), as a share of its limit.
    Below it the axis cannot follow its own trajectory and falls behin
    ↪ d it."""
    P = plant.P

    need = P["J"] * P["a_max"] + P["b"] * P["v_max"] + abs(P["load_bia
    ↪ s"]) + P["dist"]
    return min(1.0, need / P["tau_max"])


class PoolTwin:

    """What the operator's own autoscaler would have had with no cap:
    ↪ the same arrivals the brain counts, the same rules
    (the HPA's tolerance, its scale-down window, the machines' start t
    ↪ ime). The brain knows the controller it sits on."""

    def __init__(self, P):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.n = int(P["n0"])
        self.pending = []
        self.Q = 0.0
        self.hist = []

    def step(self, P, k, lam):

        self.n += sum(1 for s in self.pending if s <= k)
        self.pending = [s for s in self.pending if s > k]
        dt, mu = P["dt"], P["mu"]
        A = lam * dt
        C = self.n * mu * dt
        adm = min(A, max(0.0, C + P["queue_limit_s"] * self.n * mu - s
    ↪ elf.Q))
        served = min(self.Q + adm, C)
        self.Q += adm - served
        util = served / C if C > 0 else 1.0
        cur = self.n + len(self.pending)
        ratio = util / P["target"]

        rec = cur if abs(ratio - 1.0) <= 0.1 else math.ceil(self.n * r
    ↪ atio)
        self.hist = (self.hist + [rec])[-P["stab_steps"]:]
        want = rec if rec >= cur else min(cur, max(self.hist))
        want = int(clamp(want, P["n_min"], P["n_max"]))
        if want > cur:

            self.pending += [k + P["startup_steps"]] * min(want - cur,
    ↪  max(4, cur))
        elif want < cur:
            cut = cur - want
            while cut and self.pending:
                self.pending.pop()

                cut -= 1
            self.n = max(P["n_min"], self.n - cut)

    def machines_at(self, k):
        return self.n + sum(1 for s in self.pending if s <= k)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def drop_floor(plant, cfg) -> float:
    """The lowest power cap a request-serving pool may be given now. A
    ↪  pool with a bounded queue turns requests away once
    the queue is full, and a request turned away is work lost for good
    ↪ ; a burst that starts while the pool is capped finds

    less room than native would have had. So the capped pool must stil
    ↪ l serve, at once, what is queued plus the arrivals
    after the largest jump in demand this pool has shown (never less t
    ↪ han jump0, nor than the jump its operator's own target
    leaves room for, 1/target - 1), and nothing is capped while reques
    ↪ ts are queued or were turned away."""

    P = plant.P
    hist = getattr(plant, "_v5_lam", None)
    if hist is None:
        hist = plant._v5_lam = []
        plant._v5_jump = max(cfg.jump0, 1.0 / P["target"] - 1.0)
        plant._v5_twin = PoolTwin(P) if not P.get("ca") else None

    k = plant.k
    twin = plant._v5_twin
    if k > 0:
        a = plant.lam[k - 1]
        if hist and hist[-1] > 0:
            plant._v5_jump = max(plant._v5_jump, a / hist[-1] - 1.0)

        hist.append(a)
        if len(hist) > cfg.window:
            hist.pop(0)
        if twin is not None:
            twin.step(P, k - 1, a)
    if plant.drop_share > 0.0 or not hist or plant.Q > (twin.Q if twin

    ↪  is not None else 0.0):
        return 1.0                                                    #
    ↪  turning requests away, or behind native: no cap
    # (1) no less serving capacity than native would have now: the cap
    ↪  spends only machines the autoscaler has added
    # beyond native's (it adds them because the cap raises the utilisa
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ tion it sees)
    surplus = 1.0
    if twin is not None:
        n_arm = plant.n + sum(1 for s in plant.pending if s <= k)
        n_nat = twin.machines_at(k)
        surplus = (n_nat / n_arm) ** 2.5 if n_arm >= n_nat else 1.0

    # a cap acts until the next decision, so a burst that starts now m
    ↪ eets one decision of capped service: the queue it
    # leaves behind is at most the capacity the cap took away. It turn
    ↪ s requests away only if, on top of what native's
    # own queue would reach before the autoscaler's new machines start
    ↪  (startup_steps), it overfills the queue's bound

    dt, n, S = P["dt"], plant.n, P["startup_steps"]
    C1 = n * P["mu"] * dt
    B1 = P["queue_limit_s"] * n * P["mu"]
    X1 = max(0.0, hist[-1] * (1.0 + plant._v5_jump) * dt - C1)
    room = B1 - plant.Q - S * X1                                        #
    ↪  what native's queue would leave free in the burst

    # (2) or a shortfall a burst cannot turn into requests turned away
    ↪ ; either one alone keeps the work
    if room <= 0.0:
        burst = 1.0
    elif room >= C1:
        burst = 0.0

    else:
        burst = (1.0 - room / C1) ** 2.5                                #
    ↪  mu(cap) = mu cap^0.4: C(cap) >= C1 - room
    return min(surplus, burst)

def latency_floor(plant, cfg) -> float:
    """The lowest power cap that keeps a serving pool's response time
    ↪ inside its line through a burst whenever native's
    would stay inside it (read after drop_floor, which keeps the pool'
    ↪ s arrival history and its largest jump). A cap acts
    until the next decision: if the burst (the latest arrivals after t
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ he largest jump this pool has shown) outruns the
    capped capacity, the work the cap held back waits behind everythin
    ↪ g native itself would queue until its new machines
    start (startup_steps), and every request is served a little slower
    ↪ . At most, one capped decision leaves behind all the
    capacity it took away, whatever the burst. The cap is allowed only

    ↪  while that extra wait fits in the room native's own
    worst response time in the burst leaves under the line; where nati
    ↪ ve itself would be late, nothing is capped."""
    P = plant.P
    hist = getattr(plant, "_v5_lam", None)
    if not hist or plant.drop_share > 0.0:

        return 1.0
    dt, n, S, slo = P["dt"], max(plant.n, 1), max(1, int(P["startup_st
    ↪ eps"])), P["slo_s"]
    mu1 = plant.mu(1.0)
    lam_b = hist[-1] * (1.0 + plant._v5_jump)
    grow = max(0.0, lam_b * dt - n * mu1 * dt)                        #

    ↪  native's own queue growth per decision in the burst
    room = slo - ((plant.Q + plant.H + grow * S) / (n * mu1) + 1.0 / m
    ↪ u1)
    if room <= 0.0:
        return 1.0
    c = 0.4

    while c < 1.0:
        mu_c = plant.mu(c)
        held = n * (mu1 - mu_c) * dt
    ↪ # the most one capped decision can leave behind:

    ↪  # all the capacity it took away (any burst)

        if held / (n * mu1) + (1.0 / mu_c - 1.0 / mu1) <= room:
            return c
        c = min(1.0, c + 0.02)
    return 1.0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# How far a wandering load runs over its line: the 99.9% quantile, ove
    ↪ r one hour of decisions, of the deepest excursion
# R = max(0, R - (k + x)) of a load whose spare capacity wanders as a
    ↪ standardized AR(phi) process x with mean k spreads
# (Lindley's recursion: the spare capacity pulls the excursion back, i
    ↪ ts shortfall pushes it on). In spreads x decisions.

# Rows phi = 0.8, 0.9, 0.95, 0.98; columns k = 0, 0.25, ..., 6. (Monte
    ↪  Carlo, 20,000 hours each: scratchpad/lindley_mc.py)
LINDLEY_PHI = (0.8, 0.9, 0.95, 0.98)
LINDLEY = (
    (152.708, 104.52, 70.88, 48.499, 34.735, 26.563, 20.498, 15.714, 1
    ↪ 1.983, 9.01, 6.777, 4.898, 3.411, 2.233, 1.426,

     0.814, 0.432, 0.152, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0),
    (213.775, 159.008, 116.768, 83.079, 60.201, 46.878, 37.396, 29.04,
    ↪  22.051, 16.486, 12.974, 9.636, 6.317, 4.438,
     2.726, 1.381, 0.499, 0.157, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0),
    (289.729, 238.03, 190.297, 148.51, 112.981, 84.525, 66.894, 48.288
    ↪ , 36.794, 26.801, 17.795, 11.811, 7.975, 4.807,

     2.508, 1.017, 0.251, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0),
    (424.6, 365.52, 307.666, 254.623, 207.105, 162.118, 126.978, 97.33
    ↪ 1, 69.401, 51.264, 38.243, 24.412, 16.657, 9.051,
     3.737, 1.082, 0.082, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0),
)

def lindley_reach(phi: float, k: float) -> float:
    """The table read between its rows and columns (beyond its edges:
    ↪ the nearest row, and nothing past k = 6)."""
    k = max(0.0, k)
    if k >= 6.0:

        return 0.0
    j = int(k / 0.25)
    fk = k / 0.25 - j

    def row(r):
        return r[j] + (r[j + 1] - r[j]) * fk
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    phi = clamp(phi, LINDLEY_PHI[0], LINDLEY_PHI[-1])
    for i in range(len(LINDLEY_PHI) - 1):
        if phi <= LINDLEY_PHI[i + 1]:
            f = (phi - LINDLEY_PHI[i]) / (LINDLEY_PHI[i + 1] - LINDLEY
    ↪ _PHI[i])
            return row(LINDLEY[i]) + (row(LINDLEY[i + 1]) - row(LINDLE
    ↪ Y[i])) * f
    return row(LINDLEY[-1])


def cooling_hole_ceiling(plant, cfg, native: float) -> float:
    """The warmest setpoint a cooled zone may be given now, from its o
    ↪ wn physics: the hole is a cooling plant that runs
    out (the heat in exceeds everything its units can remove); then no
    ↪  setpoint can pull the zone down, and only the
    degrees the zone has in hand below its limit carry it through. So
    ↪ the zone may be held warmer than the operator's
    setpoint only by the degrees its worst plausible saturation would
    ↪ not need.

    The brain reads the heat the zone sheds each decision from its own
    ↪  energy balance (the cooling delivered plus what
    the room's mass stored) as the cooling's headroom h, the share of
    ↪ its installed capacity left, and keeps h's mean,
    spread and memory (lag-one correlation) over everything it has see
    ↪ n; the spread is never taken below what the
    step-to-step swing implies over the wander's correlation time. How
    ↪  far such a wander runs the room over its setpoint
    in an hour at 99.9% is the Lindley reach, in spreads x decisions;
    ↪ times the spread, Q / C and the decision gives the
    degrees. The ceiling leaves those degrees, plus the loop's own tra
    ↪ cking overshoot seen so far, below the limit.
    Nothing is raised until the brain has a window of readings."""
    P = plant.P
    Q = P["units"] * P["q_unit_w"] * plant.override.get("power", 1.0)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    t_prev = getattr(plant, "_v5_Tprev", plant.T)
    plant._v5_Tprev = plant.T
    q_heat = plant.qc_avg + P["c_j_k"] * (plant.T - t_prev) / P["dt"]
    h = 1.0 - q_heat / max(Q, 1e-9)
    st = getattr(plant, "_v5_hs", None)
    if st is None:

        st = plant._v5_hs = {"n": 0, "m": h, "v": 0.0, "dv": 0.0, "las
    ↪ t": h, "c": 0.0, "track": 0.1}
    st["n"] += 1
    n = st["n"]
    a = 1.0 / n                                                    #
    ↪ every reading kept: the whole memory

    d = h - st["last"]
    dm = h - st["m"]
    st["m"] += a * dm
    st["v"] = (1.0 - a) * (st["v"] + a * dm * dm)
    st["c"] = (1.0 - a) * st["c"] + a * (h - st["m"]) * (st["last"] -
    ↪ st["m"])     # lag-one covariance

    st["dv"] = (1.0 - a) * st["dv"] + a * d * d
    st["last"] = h
    hh = st.setdefault("hh", [])                                   #
    ↪ the headroom's recent readings, for its trend
    hh.append(h)
    if len(hh) > cfg.window:

        hh.pop(0)
    if h > 0.0:
        st["track"] = max(st["track"], plant.T - plant.setpoint())  #
    ↪ the loop's own overshoot while it had headroom
    if len(hh) > 1:
        # the headroom read ahead: falling at its present rate (the st

    ↪ eeper of the fall over the window and over the
        # cooling's own answer time), would it run out before the cool
    ↪ ing could pull the zone back? Then the zone goes
        # into that hole at the operator's own temperature, never warm
    ↪ er: the setpoint is native until the fall stops
        react = reaction_steps(plant)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        r = max(1, min(len(hh) - 1, int(math.ceil(react))))
        fall = max(0.0, (hh[0] - hh[-1]) / (len(hh) - 1), (hh[-1 - r]
    ↪ - hh[-1]) / r)
        if h - fall * react <= 0.0:
            return native
    hi = max(P["stress"], P["calm"])

    # never behind what came before: the earlier law's own balance poi
    ↪ nt (the zone held at the middle of its band) stays
    # allowed from the first decision; only the degrees beyond it must
    ↪  be proven by the reach
    lo = min(P["stress"], P["calm"])
    floor = lo + cfg.base_center * (P["t_limit"] - lo) if (cfg.hybrid

    ↪ and cfg.hole_floor) else native
    if n < cfg.window:
        return clamp(floor, native, hi)
    phi = clamp(st["c"] / st["v"], 0.0, 0.999) if st["v"] > 1e-12 else
    ↪  0.0
    tc = max(float(cfg.window), 1.0 / (1.0 - phi))                    #

    ↪  the wander's correlation time, in decisions
    sd = max(math.sqrt(st["v"]), math.sqrt(st["dv"]) * math.sqrt(tc /
    ↪ 2.0), 1e-6)
    reach = lindley_reach(1.0 - 1.0 / tc, st["m"] / sd)               #
    ↪  spreads x decisions
    rise = sd * Q / P["c_j_k"] * P["dt"] * reach                      #

    ↪  degrees the worst hour's saturation adds
    ceiling = max(P["t_limit"] - rise - st["track"], floor)
    if cfg.precool and h > 0.0:
        # or, while the cooling has headroom now: the degrees it can p
    ↪ ull back before that headroom could run out (it
        # falls by at most z step-to-step swings times the square root

    ↪  of the steps the pull-back takes), held only as
        # long as each decision proves it again; the zone is cooled ah
    ↪ ead of a surge instead of held back all hour
        fall = cfg.z * max(math.sqrt(st["dv"]), 1e-6)
        room = (h / fall) ** 2 - 1.0
        if room > 0.0:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            ceiling = max(ceiling, native + room * P["dt"] * h * Q / P
    ↪ ["c_j_k"] - st["track"])
    return clamp(ceiling, native, hi)


def cooling_headroom_position(plant) -> float:
    """A cooling zone's second gauge: how close its cooling is to runn
    ↪ ing flat out ((load ratio - 0.5) / 0.5). A room whose
    cooling has no spare capacity cannot be pulled back by any setpoin
    ↪ t, so its margin must already be there."""
    o = plant.observe()
    return (o["load_ratio"] - 0.5) / 0.5

def v5_position(plant) -> float:
    """The service position the v5 law reads: the plant's own, and for
    ↪  a motion axis the deadline margin too, and for a
    cooling zone the cooling's own headroom too (every gauge the plant
    ↪  offers, the worst one decides)."""
    p = position(plant)
    if plant.template == "motion_axis":
        p = max(p, deadline_share(plant))
    elif plant.template == "thermal_zone" and getattr(plant, "_v5_head
    ↪ room", False):
        p = max(p, cooling_headroom_position(plant))
    return p


def band_position(plant) -> float:
    """A process loop's place in its own band: how far it strayed from
    ↪  its setpoint in the last decision plus how far
    that setpoint sits from the band's middle, in half-bands (1 is the
    ↪  band's edge). The plant's deviation reading is
    taken from wherever the setpoint is, so a setpoint eased toward th
    ↪ e band's calm end reads as calm while the margin
    to the edge shrinks; this is the margin itself."""
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    P = plant.P
    return plant.dev + abs(plant.setpoint() - P["mid"]) / P["half"]


def loop_reach(plant, cfg, calm_low: bool):
    """How far this loop's disturbance pushes it off its setpoint towa
    ↪ rd the band's calm-side edge, in half-bands, from
    everything the brain has seen (the historian's low, or high, over
    ↪ each decision: a swing toward the band's middle
    cannot take the loop out, so it is not counted). The larger of the
    ↪  deepest such swing seen and its mean plus z
    spreads, the spread never below what the step-to-step swing implie
    ↪ s over the swing's own correlation time (the reading
    the cooling hole guard uses). A loop slower than one decision is a
    ↪ lso read ahead: its swing now, carried on at its
    present climb (the steeper of the climb over the window and over t
    ↪ he loop's own answer time) for as long as the loop
    takes to answer once its setpoint goes back. None until the brain
    ↪ has a window of readings."""
    d = plant.dev_dn if calm_low else plant.dev_up
    st = getattr(plant, "_v5_ls", None)
    if st is None:
        st = plant._v5_ls = {"n": 0, "m": d, "v": 0.0, "dv": 0.0, "las
    ↪ t": d, "c": 0.0, "peak": 0.0, "h": []}
    st["n"] += 1
    n = st["n"]
    a = 1.0 / n                                                  #
    ↪ every reading kept: the whole memory
    dd = d - st["last"]
    dm = d - st["m"]
    st["m"] += a * dm
    st["v"] = (1.0 - a) * (st["v"] + a * dm * dm)
    st["c"] = (1.0 - a) * st["c"] + a * (d - st["m"]) * (st["last"] -
    ↪ st["m"])     # lag-one covariance
    st["dv"] = (1.0 - a) * st["dv"] + a * dd * dd
    st["last"] = d
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    st["peak"] = max(st["peak"], d)
    h = st["h"]
    h.append(d)
    if len(h) > cfg.window:
        h.pop(0)
    if n < cfg.window:

        return None
    phi = clamp(st["c"] / st["v"], 0.0, 0.999) if st["v"] > 1e-12 else
    ↪  0.0
    tc = max(float(cfg.window), 1.0 / (1.0 - phi))                    #
    ↪  the swing's correlation time, in decisions
    sd = max(math.sqrt(st["v"]), math.sqrt(st["dv"]) * math.sqrt(tc /

    ↪ 2.0), 1e-9)
    reach = max(st["peak"], st["m"] + cfg.z * sd)
    P = plant.P
    answer = (P.get("tau", 0.0) + P.get("theta", 0.0)) / P["dt"]
    ↪  # decisions the loop takes to answer its setpoint
    if answer > 1.0:

        r = max(1, min(len(h) - 1, int(math.ceil(answer))))
        climb = max(0.0, (h[-1] - h[0]) / (len(h) - 1), (h[-1] - h[-1
    ↪ - r]) / r)
        reach = max(reach, d + climb * answer)
    return reach

def loop_clamp(plant, cfg, x: float, native: float) -> float:
    """The setpoint never sits closer to its band's calm-side edge tha
    ↪ n the loop's own reach leaves room for: at most
    (1 - reach) half-bands from the band's middle, and native once tha
    ↪ t room is gone. Before the brain has a window of

    readings the band reading alone holds the loop (the earlier law's
    ↪ own behaviour, as the hole guard allows)."""
    P = plant.P
    reach = loop_reach(plant, cfg, P["calm"] < P["mid"])
    if reach is None:
        return x
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    room = (1.0 - reach) * P["half"]
    if room <= abs(native - P["mid"]):
        return native
    return clamp(x, P["mid"] - room, P["mid"] + room)

def raw_position(plant) -> float:
    """The service position without the last decision's violation flag
    ↪  (for the climb and wobble statistics)."""
    if plant.template == "motion_axis":
        o = plant.observe()
        return max(o["queue_ratio"], (o["load_ratio"] - 0.5) / 0.5, de

    ↪ adline_share(plant))
    if plant.template == "thermal_zone":
        P = plant.P
        t = (plant.T - min(P["stress"], P["calm"])) / (P["t_limit"] -
    ↪ min(P["stress"], P["calm"]))
        return max(t, cooling_headroom_position(plant)) if getattr(pla

    ↪ nt, "_v5_headroom", False) else t
    o = plant.observe()
    p = max(o["queue_ratio"], (o["load_ratio"] - 0.5) / 0.5)
    if plant.template == "energy_storage":
        p = max(p, (o["power_stress"] - 0.5) / 0.5)
    # (a process loop's statistics stay on its deviation from its own

    ↪ setpoint: the cushion is for what the disturbance
    # does, while the setpoint's own offset toward the band's edge is
    ↪ the brain's known choice, read by the law itself)
    return p

# ------------------------------------------------------------------
    ↪ ------------------------------------------------
# MAX, the work edge: the default brain (the founder's orders of 11 an
    ↪ d 12 October). Every muscle is pushed toward its
# own limit to do the most work, faster; never past its limit and neve
    ↪ r below native (native's own setting is always one
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# of the brain's choices, so a muscle native already runs flat out sta
    ↪ ys native, wired and watched). The energy that
# takes is reported as power given, beside the work and the speed it b
    ↪ ought. One pedal: push for more, lift off for
# less, and lifting hands the setting back toward native.
#

#   compute pool   the machines the forecast needs are started before
    ↪ the rush lands: the recent peak arrivals widened
#                  by the largest rise this pool has shown within one
    ↪ machine start, plus the standing queue, at the edge
#                  utilisation. The HPA target is lowered just enough
    ↪ that the operator's own autoscaler starts them (a

#                  machine pool: a floor under its cluster autoscaler)
    ↪ . Never fewer machines than native would keep; the
#                  power cap and admission stay native (full clock, ev
    ↪ ery request admitted)
#   cooled zone    cooled ahead of the heat: the setpoint goes from th
    ↪ e operator's toward the band's cold end as the

#                  room nears its limit or its cooling nears flat out,
    ↪  both read ahead by the cooling's own answer time;
#                  never warmer than native
#   battery site   the charge is held for the line instead of spent on
    ↪  the first minutes of load, and only the draw
#                  that would cross the site's line (the forecast draw

    ↪  plus line_z of its step-to-step wobbles) is taken
#                  from it; a UPS lends only the charge above its outa
    ↪ ge floor (its band's stress end), never below it
#   motion axis    while tasks are waiting, its speed goes to the axis
    ↪ 's own peak (s_peak, realms/presets.py: the head the
#                  machine itself has above the everyday profile nativ

    ↪ e runs), never past it, and back to the everyday
#                  setting at once when the motor nears its heat limit
    ↪  or the tracking strains (an axis alone; inside a
#                  body it keeps its everyday speed, max_body_step); a
    ↪ n axis whose everyday setting is its limit (a
#                  wheel's saturation, a line speed, an overspeed gove
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ rnor) and every effort (torque) limit stay native:
#                   native already runs them at their limit
#   process loop    native: the full actuator, native already runs it f
    ↪ lat out
def max_pool_want(plant, cfg) -> int:
    """The machines a pool needs when a machine started now is ready:

    ↪ the recent peak arrivals (what the brain counted,
    never the decision still to come) widened by the largest rise this
    ↪  pool has shown within one machine start (at
    first, the room its operator's own target leaves: 1/target - 1), p
    ↪ lus the standing queue, at the edge utilisation."""
    P = plant.P

    k = plant.k
    S = max(1, int(P["startup_steps"]))
    st = getattr(plant, "_mx", None)
    if st is None:
        st = plant._mx = {"lam": [], "rise": max(0.0, 1.0 / P["target"
    ↪ ] - 1.0)}

    h = st["lam"]
    if k > 0:
        a = plant.lam[k - 1]
        if len(h) > 0:
            lo = min(h[-S:])
            if lo > 0.0:

                st["rise"] = max(st["rise"], a / lo - 1.0)
        h.append(a)
        if len(h) > max(cfg.window, S + 1):
            h.pop(0)
    line = getattr(plant, "B", 0.0) / (S * P["dt"])   # work waiting i
    ↪ n line: cleared within one machine start

    if not h and line <= 0.0:
        return int(P["n_min"])
    fore = max(h) * (1.0 + st["rise"]) if h else 0.0   # (nothing coun
    ↪ ted yet: the line alone)
    need = (fore + plant.Q / P["dt"] + line) / (plant.mu(1.0) * cfg.ed
    ↪ ge_util)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return int(clamp(math.ceil(need - 1e-9), P["n_min"], P["n_max"]))


def max_cool(plant, cfg) -> float:
    """A cooled zone's setpoint for the most work: the operator's, mov
    ↪ ed toward the band's cold end as the room nears its

    limit or its cooling nears flat out. The cooling's load is read fr
    ↪ om the room's own energy balance (the cooling
    delivered plus what the room's mass stored) against its installed
    ↪ capacity, carried ahead at its present climb for the
    cooling's own answer time; heat that outruns the cooling is carrie
    ↪ d by the degrees the room already has in hand."""

    P = plant.P
    native = P["t_set"]
    cold = min(P["stress"], P["calm"])
    Q = P["units"] * P["q_unit_w"]
    t_prev = getattr(plant, "_mx_T", plant.T)
    plant._mx_T = plant.T

    load = (plant.qc_avg + P["c_j_k"] * (plant.T - t_prev) / P["dt"])
    ↪ / max(Q, 1e-9)
    hist = getattr(plant, "_mx_q", None)
    if hist is None:
        hist = plant._mx_q = []
    hist.append(load)

    if len(hist) > cfg.window:
        hist.pop(0)
    react = reaction_steps(plant)
    climb = 0.0
    if len(hist) > 1:
        r = max(1, min(len(hist) - 1, int(math.ceil(react))))

        climb = max(0.0, (hist[-1] - hist[0]) / (len(hist) - 1), (hist
    ↪ [-1] - hist[-1 - r]) / r)
    ahead = load + climb * react
    room = (plant.T - cold) / max(P["t_limit"] - cold, 1e-9)
    risk = max(clamp((ahead - 0.8) / 0.2, 0.0, 1.0), clamp((room - 0.6
    ↪ ) / 0.3, 0.0, 1.0))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return min(native, native - (native - cold) * risk)


def max_hold(plant, cfg, key):
    """A battery site's knob for the most work held inside its line: t
    ↪ he charge kept for the line, and only the draw

    that would cross the line taken from it. The draw is forecast from
    ↪  the brain's own readings: the last decision's net
    draw, plus its latest rise, plus line_z of its step-to-step wobble
    ↪ s. A UPS lends only the charge above its outage
    floor (its band's stress end); any other site, down to its band's
    ↪ stress end. A battery wired by its power holds its

    charge while the line has room and gives native's full power once
    ↪ the line could need it (nothing locked: the
    discharge timed for the line)."""
    P = plant.P
    st = getattr(plant, "_mx_s", None)
    if st is None or plant.k <= 1 and st["n"] == 0:

        # the forecast starts at the site's first reading: before its
    ↪ first step a plant shows its initial values (no
        # draw at all), and the jump from them to the first reading is
    ↪  no climb of the line
        st = plant._mx_s = {"last": plant.net, "n": 0, "dv": 0.0, "ris
    ↪ e": 0.0}

        if plant.k < 1:
            st["last"] = None
    d = plant.net - st["last"] if st["last"] is not None else 0.0
    if st["last"] is not None and plant.k > 1:
        st["n"] += 1
        a = 1.0 / st["n"]

        st["dv"] = (1.0 - a) * st["dv"] + a * d * d
        st["rise"] = max(st["rise"], d)
    st["last"] = plant.net
    fore = plant.net + max(0.0, d) + cfg.line_z * math.sqrt(st["dv"])
    over = max(0.0, fore - P["p_lim_w"])                          # the
    ↪  draw that would cross the line this decision
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    floor = P["stress"]
    if key == "power":
        # nothing locked: the battery's discharge timed for the line.
    ↪ While the line has room even after the largest
        # climb it has shown (and line_z of its wobbles, and a twentie
    ↪ th of the line), the charge is held: nothing is

        # spent, charging stays free; once the line could need it, the
    ↪  full power native gives. The charge native spends
        # early is still there when the line needs it
        win = cfg.window if cfg.hold_window is None else cfg.hold_wind
    ↪ ow
        if plant.net <= 0.0 or st["n"] < win:

            return {}                                                 #
    ↪  nothing to hold, or the line not yet read for a window
        room = max(cfg.line_z * math.sqrt(st["dv"]), st["rise"], cfg.h
    ↪ old_margin * P["p_lim_w"])
        return {} if fore + room >= P["p_lim_w"] else {"power": 0.0}
    if key == "setpoint":

        E, sq = P["e_wh"] * 3600.0, math.sqrt(P["eta_rt"])
        dt = P["dt"]
        # native plus: the operator's own reserve while nothing would
    ↪ cross the line (the site's own use of its
        # battery is native's), and below it, down to the floor, exact
    ↪ ly the draw that would cross

        r = clamp(plant.soc - over * dt / (E * sq), floor, P["reserve"
    ↪ ])
        return {} if abs(r - P["reserve"]) < 1e-12 else {"setpoint": r
    ↪ }
    # the ceiling the battery defends (the capacity knob): the line le
    ↪ ss the forecast's own margin

    cap = clamp(1.0 - (fore - plant.net) / P["p_lim_w"], 0.5, 1.0)
    return {"capacity": cap}


def max_units(plant, cfg):
    """A room's cooling units (its capacity knob, both ways): one more
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  unit held on than its staging asks for while the
    room is at risk (its load read ahead, or its degrees in hand, as m
    ↪ ax_cool reads them); the staging's own otherwise."""
    P = plant.P
    native = P["t_set"]
    x = max_cool(plant, cfg)

    if x >= native - 1e-9 or plant.units_on >= P["units"]:
        return {}
    return {"units_min": float(plant.units_on + 1)}


def machines(pool) -> int:

    """A pool's machines in service and starting."""
    return pool.n + len(pool.pending)


HEAT_HEAD, STRAIN = 0.8, 0.5     # an axis is pushed only while its mo
    ↪ tor is under 80% of its heat rise and its tracking

                                 # error under half its line (the glob
    ↪ al setting; a muscle's card may set its own)


def max_speed(plant, cfg):
    """An axis's speed for the most work: its own peak while tasks are

    ↪  waiting; None (the everyday setting, native) when
    it has no head above it, nothing waits, the motor is near its heat
    ↪  limit or the tracking strains."""
    P = plant.P
    peak = P.get("s_peak", 1.0)
    if peak <= 1.0 or plant.backlog <= 0:

        return None
    if plant.Tm > P["t_amb"] + cfg.heat_head * (P["t_lim"] - P["t_amb"
    ↪ ]) or plant.err_max > cfg.strain * P["e_max"]:
        return None
    return peak
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def native_at_limit(template: str, knob: str) -> bool:
    """Whether native already runs this kind of knob at the top of its
    ↪  range, so the max has nothing above native to
    give it: every power limit at full (clock, torque, battery power,
    ↪ the site envelope), every admission gate open

    (nothing paced or turned away), a loop's actuator at its full rang
    ↪ e (an axis's speed is at its limit only where its
    machine has no peak above the everyday setting: the max then write
    ↪ s nothing)."""
    return knob in ("power", "admission") or (knob == "capacity" and t
    ↪ emplate in ("motion_axis", "process_loop"))

def max_ask(plant, knob, cfg, twin=None):
    """What a muscle asks for the most work this decision. A pool: the
    ↪  machines its forecast needs, never fewer than its
    twin (the brain's model of native: the same plant under its own au

    ↪ toscaler alone, on the same arrivals) runs now. A
    cooled zone at risk: a colder setpoint and the most its pull-down
    ↪ can add. None: the muscle asks nothing (it holds
    native, a donor's spare or a battery's own line)."""
    P, t = plant.P, plant.template
    if t == "compute_pool" and knob not in ("power", "admission"):

        base = machines(twin) if twin is not None else int(P["n_min"])
        want = max(max_pool_want(plant, cfg), base)
        return {"want": want, "base": base, "cur": machines(plant), "h
    ↪ eld": int(plant.override.get("floor", 0)),
                "unit_w": P["p_idle"], "line": "src"}
    if t == "thermal_zone" and knob == "setpoint":

        x = max_cool(plant, cfg)
        if x >= P["t_set"] - 1e-9:
            return None
        cop = plant.cop(x, plant.tout[min(plant.k, plant.steps - 1)])
        cost = max(0.0, P["units"] * P["q_unit_w"] - plant.qc_avg) / c
    ↪ op   # the most the pull-down can add
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return {"x": x, "cost": cost, "line": "zone"}
    if t == "motion_axis" and knob == "capacity":
        s = max_speed(plant, cfg)
        if s is None:
            return None
        # the power a faster profile can add, counted at its worst: th
    ↪ e axis's rated draw, going with the cube of the
        # speed scale (the torque of a faster acceleration, times the
    ↪ faster moves done each decision while tasks wait)
        return {"s": s, "cost": max(plant.rated_w, plant.power_w) * (s
    ↪  ** 3 - 1.0), "line": "axis"}
    return None

# Every muscle its own card (the founder's question of 12 October, nig
    ↪ ht: does every muscle have its own tuning, maxed to
# its own benefit, instead of one global setting for a thousand muscle
    ↪ s?). A muscle's card is the brain's settings for

# that muscle alone (a pool's edge utilisation and window, a battery's
    ↪  line margin and window), found by an automatic
# search over every muscle on the development seeds and kept only wher
    ↪ e it beats the global setting beyond the noise on
# two separate sets of them and does no harm against native on any of
    ↪ their seeds under any load: never over its line more

# often, never less work, never its line cleared later (tools/tune_car
    ↪ ds.py; realms/muscle_cards_v5.csv). A muscle
# without a card runs the global setting; the tune-down never reads ca
    ↪ rds.
CARD_FILE = Path(__file__).with_name("muscle_cards_v5.csv")
CARD_KEYS = {"edge_util": float, "window": int, "line_z": float, "hold

    ↪ _window": int, "hold_margin": float,
             "heat_head": float, "strain": float}


def load_cards(path=CARD_FILE) -> dict:
    """{muscle_id: {setting: value}}: each muscle's own settings, an e
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ mpty cell meaning the global one."""
    if not Path(path).exists():
        return {}
    out = {}
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            c = {k: cast(row[k]) for k, cast in CARD_KEYS.items() if r
    ↪ ow.get(k, "").strip()}
            if c:
                out[row["muscle_id"]] = c
    return out

CARDS = load_cards()


def card(plant, cfg):
    """The settings the max uses for this muscle: its own card over th
    ↪ e brain's (the same brain, the muscle's own numbers
    where it has them); the brain's own where it has none, and always
    ↪ under the tune-down."""
    own = getattr(plant, "_card", None)
    if not own or cfg.dial != "max":
        return cfg
    memo = getattr(plant, "_card_cfg", None)
    if memo is None or memo[0] is not cfg:
        c = copy.copy(cfg)
        for k, v in own.items():
            setattr(c, k, v)
        plant._card_cfg = memo = (cfg, c)
    return memo[1]


def body_card(plant, cfg, knob):
    """A muscle's card inside a body: its own, except that a battery w
    ↪ ired by its power keeps the body's margins on its
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    line (its line_z and its hold margin never slimmer than the body's
    ↪ ): the line carries the whole body's swing, which
    the card, chosen for the battery alone, never saw. The body has th
    ↪ e last word on a line it shares."""
    c = card(plant, cfg)
    if c is cfg or knob != "power":

        return c
    memo = getattr(plant, "_body_card", None)
    if memo is None or memo[0] is not c:
        b = copy.copy(c)
        b.line_z = max(c.line_z, cfg.line_z)
        b.hold_margin = max(c.hold_margin, cfg.hold_margin)

        plant._body_card = memo = (c, b)
    return memo[1]


def max_apply(plant, knob, cfg, twin=None):
    """The max for a muscle alone (no body around it to share with): a

    ↪  pool's machines started before the rush and
    never fewer than its twin runs, a room cooled ahead of the heat, a
    ↪  battery's line clipped from its charge; every
    other knob native. The muscle's own card where it has one."""
    cfg = card(plant, cfg)
    P, t = plant.P, plant.template

    ov = {}
    a = max_ask(plant, knob, cfg, twin)
    if a is not None and t == "compute_pool":
        ov = {"floor": float(a["want"])} if a["want"] > P["n_min"] els
    ↪ e {}
        plant._mx_guard = a["want"] <= a["base"]        # the floor ad

    ↪ ds nothing beyond the twin's machines
        keep = max(a["want"], a["base"])
        if twin is not None and a["cur"] > keep:
            ov["cap"] = float(keep)       # the pedal lifted: machines
    ↪  the max added and no longer needs go back at once
                                          # (the autoscaler's own scal
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ e-down holds them for its window), never below its twin
    elif a is not None and t == "thermal_zone":
        ov = {"setpoint": a["x"]}
    elif a is not None and t == "motion_axis":
        ov = {"capacity": a["s"]}
    elif t == "energy_storage" and knob in ("setpoint", "capacity", "p
    ↪ ower"):
        ov = max_hold(plant, cfg, knob)
    elif t == "thermal_zone" and knob == "capacity":
        ov = max_units(plant, cfg)
    plant.override = ov
    return ov

class Bone:
    """The body's skeleton for the max: the lines every muscle stands
    ↪ on. A battery site's grid line and a room's
    cooling are loaded by the whole body (the body's power swing is lo
    ↪ ad on every site, the machines' power is heat in
    every room), so no muscle may take power the bone cannot carry: wh
    ↪ at the max adds is granted only out of the room the
    most loaded line has left, read one decision ahead (the line's lat
    ↪ est reading plus line_z of its own step-to-step
    wobbles; a battery that clips its own line counts the power it can
    ↪  still clip). With no room, the machines this
    brain added go back at once, never below what the pool's twin runs
    ↪  (the brain's model of native: the same pool under
    its own autoscaler alone, on the same arrivals), so a pool never h
    ↪ as fewer machines than native and the lines never
    carry more of this brain's power than they have room for. Nothing
    ↪ is taken from one muscle to give to another: the
    max never shaves (the tune-down does that, on the dial).

    A battery's charge counts as room on its line only while it lasts:
    ↪  a site whose battery covers its draw now, but
    whose charge above its floor runs out before power granted now cou
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ld be handed back, has as room only its line less
    its whole draw (the line a decision later carries all of it). And
    ↪ every room wired by its setpoint has a twin too:
    the same room under its operator's setpoint, warmed by native's he
    ↪ at (each pool's twin in place of the pool, every
    other source as it is), so the brain knows the temperature native

    ↪ would leave it at and never leaves it warmer."""

    def __init__(self, body, cfg):
        self.body, self.cfg = body, cfg
        self.st = {}
        self.twins = {}

        self.room_twins = {}
        from .harness import make_plant, scale_load_plant
        from .plants import pack
        for i, (p, r) in enumerate(zip(body.plants, body.rows)):
            pool = p.template == "compute_pool" and r["knob"] not in (
    ↪ "power", "admission")

            if pool or p.template == "thermal_zone":
    ↪ # every room: the brain's sense of native
                tw = pack(make_plant(r, body.seed, organism=True))
                if getattr(body, "load", None) is not None:
                    scale_load_plant(tw, body.load)
                (self.twins if pool else self.room_twins)[i] = tw

        zi = [i for i, p in enumerate(body.plants) if p.template == "t
    ↪ hermal_zone"]
        self._heat_of = {i: kh for i, kh in zip(zi, body.k_heat)}
    ↪   # each room's heat per W of the body's sources

    def advance_twin(self):

        """Step every twin one decision (each decides on the same arri
    ↪ vals native sees). First, the power the max's own
        machines added over their twins in the decision just metered:
    ↪ the lines' readings carry it, and it is this brain's
        step, not the line's own wobble, so the forecast is kept on th
    ↪ e line without it. Each room's twin is warmed by
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    native's sources over the same decision: each pool's twin in p
↪ lace of the pool, every other source as it is."""
    b = self.body
    self.extra_w = sum(max(0, machines(b.plants[i]) - machines(tw)
↪ ) * b.plants[i].P["p_idle"]
                       for i, tw in self.twins.items())

    # native's source power: the body's own reading with each pool
↪ 's power replaced by its twin's (before the first
    # step the body reads its sources' nominal power, and so does
↪ native)
    native_src = b.src_w + sum(tw.power_w - b.plants[i].power_w fo
↪ r i, tw in self.twins.items())

    for tw in self.twins.values():
        tw.step()
    n_z = max(1, len(b.zones))
    for i, tw in self.room_twins.items():
        tw.ext = {"heat_w": self._heat_of[i] * native_src / n_z}
        tw.T_read = tw.T                                        #
↪  its temperature when the real room is read
        tw.step()


def _ahead(self, key, x, prior):
    """How far a line's reading can climb in one decision, from ev
↪ erything the brain has seen of it: the larger of

    line_z of its step-to-step wobbles and the largest single clim
↪ b it has shown (never forgotten: a line that once
    jumped can jump again), and never less than the prior (a twent
↪ ieth of the line) until it has shown more."""
    st = self.st.get(key)
    if st is None:

        st = self.st[key] = {"last": x, "n": 0, "dv": 0.0, "rise":
↪  0.0}
        return prior
    d = x - st["last"]
    st["last"] = x
    st["n"] += 1
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        a = 1.0 / st["n"]
        st["dv"] = (1.0 - a) * st["dv"] + a * d * d
        st["rise"] = max(st["rise"], d)
        return max(self.cfg.line_z * math.sqrt(st["dv"]), st["rise"],
    ↪ prior)

    def room(self):
        """(room for machines, room for cooling): the W the body may s
    ↪ till add, one decision ahead. Each line is kept as
        (its gap to its limit, how far it can climb in one decision, i
    ↪ ts load per W of the body's power), so the room
        over a longer reach can be read too (room_for)."""

        b = self.body
        own = getattr(self, "extra_w", 0.0)                           #
    ↪  this brain's own machines in the last reading
        self._all, self._src = [], []
        n_st = max(1, len(b.stores))
        for s, kl in zip(b.stores, b.k_load):

            P = s.P
            g = kl / n_st                                             #
    ↪  site draw per W of the body's power
            if g <= 0:
                continue
            E, sq = P["e_wh"] * 3600.0, math.sqrt(P["eta_rt"])

            clip = 0.0
            floor = s.override.get("setpoint", P["reserve"])
            if getattr(s, "override_kind", None) == "clip":
                clip = min(P["p_batt_w"], max(0.0, s.soc - P["stress"]
    ↪ ) * E * sq / P["dt"])
                floor = min(floor, P["stress"])

            cover = max(0.0, s.net - s.imp)                           #
    ↪  what the battery covers of the site's draw now
            lasts = max(0.0, s.soc - floor) * E * sq / (cover * P["dt"
    ↪ ]) if cover > 0.0 else math.inf   # decisions
            # the site's whole draw climbs on its own while the batter
    ↪ y covers it (the import then reads nothing): its
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        # climb is read too, for the moment the battery stops cove
↪ ring
        up_net = self._ahead(("net", id(s)), s.net - g * own, 0.05
↪  * P["p_lim_w"])
        self._all.append((P["p_lim_w"] + clip - s.imp, self._ahead
↪ (id(s), s.imp - g * own, 0.05 * P["p_lim_w"]), g,

                          P["p_lim_w"] - s.net, lasts, up_net))
        if s.deferred_j > 0.0:
            # flexible load waiting at the site (held over, or the
↪  line): its catch-up has the connection's room first,
            # and the body may add only what the catch-up leaves (
↪ else every W added is a W of waiting work not done)

            base = s.net - getattr(s, "catch", 0.0)             #
↪  the site's draw before its catch-up
            self._all.append((CATCH_UP * P["p_lim_w"] - base - s.d
↪ eferred_j / P["dt"],
                              self._ahead(("catch", id(s)), base -
↪  g * own, 0.05 * P["p_lim_w"]), g, None, math.inf, 0.0))

    n_z = max(1, len(b.zones))
    for zn, kh in zip(b.zones, b.k_heat):
        P = zn.P
        h = kh / n_z                                            #
↪  room heat per W of the body's machine power
        if h <= 0:

            continue
        Q = P["units"] * P["q_unit_w"]
        self._src.append((Q - zn.qc_cmd_avg, self._ahead(id(zn), z
↪ n.qc_cmd_avg - h * own, 0.05 * Q), h))
        # and the room's own degrees: the heat that would carry it
↪  to its limit within one decision, its temperature

        # read ahead by its own step-to-step climb (a room at its
↪ limit has no room for heat, whatever its cooling has)
        c = P["c_j_k"] / P["dt"]
        self._src.append(((P["t_limit"] - zn.T) * c, self._ahead((
↪ "T", id(zn)), zn.T, 0.05) * c, h))
    return self.room_for(1)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def room_for(self, reach):
        """(room for machines, room for cooling) over reach decisions:
    ↪  each line's gap less its one-decision climb grown
        as the square root of the reach (its wobbles add up like a ran
    ↪ dom walk). A machine that takes reach decisions to

        start is started only if the lines can still carry it when it
    ↪ is ready: one started and cancelled burns its start.
        A battery whose charge runs out before power granted now could
    ↪  be off its line (reach, the decision its line
        takes to show it, and the body's cooling tail, TAIL) leaves it
    ↪ s line only the line less the site's whole draw, read

        at the draw's own climb."""
        f = math.sqrt(max(1, reach))
        r_all = math.inf
        for gap, up, g, after, lasts, up_net in self._all:
            if after is not None and lasts < reach + 1 + TAIL:
                # the battery stops covering before power granted now,

    ↪  and the cooling its heat calls for, is off the line:
                # the line then carries the site's whole draw, climbin
    ↪ g at the draw's own pace
                r_all = min(r_all, (min(gap, after) - f * max(up, up_n
    ↪ et)) / g)
            else:

                r_all = min(r_all, (gap - f * up) / g)
        r_src = min([(gap - f * up) / h for gap, up, h in self._src],
    ↪ default=math.inf)
        return min(r_src, r_all), r_all

TAIL = 2   # decisions the body's power stays on its lines after machi
    ↪ nes are handed back: their heat reaches the rooms a
           # decision later, and the rooms' cooling answers over its o
    ↪ wn averaging, one more
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def _units(room: float, unit: float, cap: int) -> int:
    """How many units of unit W fit in room W (cap when the room is un
    ↪ bounded)."""
    if not math.isfinite(room):
        return cap
    return max(0, min(cap, int(room // unit)))

def walk_room(p):
    """A room's pull-down handed back as the pedal is lifted: toward t
    ↪ he operator's setpoint over the cooling's own
    reaction time, never snapped (a setpoint snapped back kicks the ro

    ↪ om's own controller, which then overshoots warm
    of where native would have it)."""
    x = p.override.get("setpoint") if p.override else None
    native = p.P["t_set"]
    if x is None:
        return {}

    step = (native - min(p.P["stress"], p.P["calm"])) / max(1.0, react
    ↪ ion_steps(p))
    x = min(native, x + max(step, 1e-9))
    return {} if x >= native - 1e-9 else {"setpoint": x}

def queue_need(p):
    """The machines a pool's queue and its latest arrivals need now: n
    ↪ ever handed back while they are needed."""
    lam = p.lam[p.k - 1] if p.k > 0 else 0.0
    return int(math.ceil((lam + p.Q / p.P["dt"]) / max(p.mu(1.0), 1e-9
    ↪ ) - 1e-9))

def room_guard(body, bone):
    """Never warmer than native: a room warmer than its twin (the same
    ↪  room under its operator's settings and native's
    heat) is cured by its own knob where it has one: a room wired by i

    ↪ ts setpoint is cooled by the difference. A room
    whose own knob cannot cool it (its cooling units, a power or admis
    ↪ sion gate, or cooling already flat out) is cured by
    the body: its heat debt is handed back by the pools (heat_debt, ma
    ↪ x_body_step)."""
    for i, tw in bone.room_twins.items():
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        p = body.plants[i]
        excess = p.T - getattr(tw, "T_read", tw.T)
        if excess <= 1e-9:
            continue
        knob = body.knobs[i]
        if knob == "setpoint":

            native = p.P["t_set"]
            x = max(min(p.P["stress"], p.P["calm"]), min(p.override.ge
    ↪ t("setpoint", native), native - excess))
            p.override = {"setpoint": x}

def heat_debt(body, bone):
    """The body's power (W) that would carry, over one decision, the h
    ↪ eat by which the rooms that cannot cure themselves
    stand warmer than their twins: a room wired by its cooling units o
    ↪ r a power or admission gate, or any room whose
    cooling is already flat out (its own knob cannot cool it further).

    ↪  The pools hand it back (the other muscles kick
    in): the max's added machines go back until no such room is warmer
    ↪  than native's."""
    debt = 0.0
    n_z = max(1, len(body.zones))
    for i, tw in bone.room_twins.items():

        p = body.plants[i]
        excess = p.T - getattr(tw, "T_read", tw.T)
        if excess <= 1e-9:
            continue
        P = p.P
        flat_out = p.qc_cmd_avg >= p.units_on * P["q_unit_w"] * p.over

    ↪ ride.get("power", 1.0) - 1e-9
        if body.knobs[i] == "setpoint" and not flat_out:
            continue
        h = bone._heat_of[i] / n_z                                  #
    ↪  room heat per W of the body's sources
        if h > 0:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            debt = max(debt, excess * P["c_j_k"] / P["dt"] / h)
    return debt


def max_body_step(body, cfg, bone):
    """One decision of the max over one body: every battery that can c
    ↪ lips its own line or times its discharge for it,
    and the machines and cooling the muscles ask for are granted out o
    ↪ f the room the bone has left; with no room, or
    while a room that cannot cure itself stands warmer than native's (
    ↪ its heat debt), what this brain added goes back at
    once, never below what the twin runs nor what a pool's queue needs
    ↪ , and a room's pull-down is walked back, never
    snapped. A room is pulled down only if the bone can carry it for t
    ↪ he cooling's own reaction time, and no room wired by
    its setpoint is left warmer than its twin. Every other muscle is w
    ↪ ired, watched and native."""
    asks = []
    for i, (p, knob) in enumerate(zip(body.plants, body.knobs)):
        t = p.template
        p.override_kind = None
        if t == "energy_storage" and knob in ("setpoint", "capacity",
    ↪ "power"):
            p.override = max_hold(p, body_card(p, cfg, knob), knob)
            p.override_kind = "clip" if knob in ("setpoint", "capacity
    ↪ ") else knob
            continue
        a = max_ask(p, knob, card(p, cfg), bone.twins.get(i))
        if a is None:
            p.override = walk_room(p) if t == "thermal_zone" and knob
    ↪ == "setpoint" else {}
            continue
        asks.append((p, a))
    r_src, r_all = bone.room()
    debt = heat_debt(body, bone)
    if debt > 0.0:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    r_src = min(r_src, -debt)                                          #
↪  a room that cannot cure itself: the pools give back
if r_src < 0.0 or r_all < 0.0:
    # no room on a line: what this brain added goes back, as much
↪ as the shortfall needs (the pools holding the most
    # extra machines first, and in each the machines started last)

↪ , never below a pool's twin (nothing native has is
    # taken) nor below what its queue needs (a machine taken from
↪ a queue leaves its work waiting); the machines that
    # fit stay, held where they are. No room is pulled further bel
↪ ow the operator's setpoint until the bone has room
    # again: its pull-down is walked back

    short = -min(r_src, r_all)                                         #
↪  W of the body's power to give back
    for p, a in sorted(asks, key=lambda pa: -(pa[1].get("cur", 0)
↪ - pa[1].get("base", 0))):
        if a["line"] != "src":
            p.override = walk_room(p) if a["line"] == "zone" else

↪ {}   # rooms walked back; axes: the everyday setting
            continue
        base = max(a["base"], int(p.P["n_min"]), min(a["cur"], que
↪ ue_need(p)))
        extra = max(0, a["cur"] - base)
        give = min(extra, int(math.ceil(short / a["unit_w"] - 1e-9

↪ ))) if short > 0.0 else 0
        short -= give * a["unit_w"]
        keep = a["cur"] - give
        if keep > base:                                                #
↪  extra machines that fit: held, not added to
            p.override = {"floor": float(keep), "cap": float(keep)

↪ }
            p._mx_guard = False
        else:                                                          #
↪  pinned at native's count: adds nothing
            p.override = ({"floor": float(base), "cap": float(base
↪ )} if extra else
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                            ({"floor": float(base)} if base > p.P["n
↪ _min"] else {}))
            p._mx_guard = True
    room_guard(body, bone)
    return
# pools by their shortfall first, then the rooms' pull-downs (cool
↪ ing ahead of the heat protects every plant in the room)
order = {"src": 0, "zone": 1, "axis": 2}
far, used = {}, 0.0                                              #
↪  the rooms over each start time; W granted so far
for p, a in sorted(asks, key=lambda pa: (order[pa[1]["line"]],
                                         -(pa[1].get("want", 0) -
↪ max(pa[1].get("cur", 0), pa[1].get("base", 0))))):
    if a["line"] == "src":
        now = max(a["cur"], a["base"])
        add = max(0, a["want"] - now)
        can = 0
        if add:
            reach = int(p.P["startup_steps"])                    #
↪  the lines must carry it until it is ready
            if reach not in far:
                far[reach] = bone.room_for(reach)
            f_src, f_all = far[reach]
            can = min(_units(min(r_src, f_src - used), a["unit_w"]
↪ , add),
                      _units(min(r_all, f_all - used), a["unit_w"]
↪ , add))
        floor = max(now + can if add else a["want"], a["base"])
        r_src -= can * a["unit_w"]
        r_all -= can * a["unit_w"]
        used += can * a["unit_w"]
        p.override = {"floor": float(floor)} if floor > p.P["n_min
↪ "] else {}
        p._mx_guard = floor <= a["base"]
    elif a["line"] == "axis":
        # inside a body an axis keeps its everyday speed: the orga
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ nisms run the hour around each battery site's peak,
            # where the bone almost never has room for an axis's worst
    ↪  case, and a faster axis's changed rhythm lands on
            # rooms and UPS lines already at their own limits (in the
    ↪ development runs its peak gave the body nothing
            # and crossed neighbours at their limits). Its peak is for
    ↪  an axis alone, with its own power and heat
            p.override = {}
        else:
            ok = a["cost"] <= r_all
            if ok and not p.override.get("setpoint"):
                # a new pull-down only if the bone can carry it for th
    ↪ e cooling's own reaction time: one granted and taken
                # back a decision later kicks the room's own controlle
    ↪ r and leaves it warm of native
                reach = int(math.ceil(reaction_steps(p)))
                if reach not in far:
                    far[reach] = bone.room_for(reach)
                ok = a["cost"] <= far[reach][1] - used
            if ok:
                p.override = {"setpoint": a["x"]}
                r_all -= a["cost"]
            else:
                p.override = walk_room(p)                              #
    ↪  the pull-down waits for room, walked back
    room_guard(body, bone)


def compass_apply_v5(plant, knob, cfg, dt=1.0, tau=3.0):
    if cfg.dial == "max":
        return max_apply(plant, knob, cfg)
    lv = lever_v5(plant, knob, cfg)
    if lv is None:
        plant.override = {}
        return {}
    if not hasattr(plant, "_v5"):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        plant._v5_headroom = cfg.headroom
        plant._v5_band = cfg.band
        law = CompassLaw(Band(0.0, 1.0), dt=dt, tau=tau, kp=1.0, smoot
    ↪ h=0.3)
        law.kd *= 3.0
        plant._v5 = (law, Edge(plant, cfg, lv[0]))

        plant._v5_x = lv[1]
    law, edge = plant._v5
    p = v5_position(plant)
    edge.since = 0 if plant.override else getattr(edge, "since", 10 **
    ↪  9) + 1
    if plant.template == "motion_axis":

        blame = max(3.0, plant.P["deadline_s"] / plant.P["dt_dec"])
    ↪   # a late task was decided a deadline ago
    else:
        # a fall within the knob's own reaction/settling time of Omni
    ↪ holding back is Omni's doing: a power cap on a
        # serving pool shows up as latency a few decisions later (its

    ↪ startup lag), a process loop's overshoot within its
        # three time constants. So the cap teaches that plant's cushio
    ↪ n floor even when the fall is not the same decision.
        blame = memory_horizon(plant)
    if cfg.learn_fall > 0 and plant.vhist and plant.vhist[-1] and edge
    ↪ .since <= blame:

        # a fall while, or soon after, Omni held something back: this
    ↪ plant needs more room than its statistics said
        edge.floor = min(1.0, max(edge.floor, edge.cushion) + cfg.lear
    ↪ n_fall)
    center = edge.update(raw_position(plant))
    if cfg.hybrid:

        # never behind what came before: the edge only moves the balan
    ↪ ce point and the wall further out; but a plant
        # that has fallen while Omni held something back has taught th
    ↪ e brain otherwise, and its own lesson decides
        base = cfg.base_center if edge.floor <= 0.0 else 0.0
        law.band.center = clamp(max(base, center), 0.0, 0.995)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        law.band.cushion = min(1.0 - cfg.base_wall, max(0.005, edge.re
    ↪ act_part))
    else:
        law.band.center = clamp(center, 0.0, 0.995)
        law.band.cushion = min(0.5, max(0.005, edge.react_part))
    ↪ # the wall moves with the reaction part

    if cfg.band and plant.template == "process_loop":
        # the band's own edge (1 is the edge). A loop faster than one
    ↪ decision is pushed about within the decision by its
        # disturbance alone, so it is read ahead: the setpoint's offse
    ↪ t from the band's middle plus the most the
        # disturbance has pushed it off its setpoint (mean plus z time

    ↪ s the spread over the window). A slower loop's
        # deviation from a moving setpoint is mostly its own lag, so i
    ↪ t is read as it stands (offset plus deviation) and
        # acts only at the wall. Either way, once the reading reaches
    ↪ the wall the law fails up; away from the edge the
        # law reads the plant as before.

        P = plant.P
        off = abs(plant.setpoint() - P["mid"]) / P["half"]
        if (3.0 * P.get("tau", 0.0) + P.get("theta", 0.0)) / P["dt"] <
    ↪ = 1.0:
            hist = getattr(plant, "_v5_band_h", None)
            if hist is None:

                hist = plant._v5_band_h = []
            hist.append(plant.dev)
            if len(hist) > cfg.window:
                hist.pop(0)
            md = sum(hist) / len(hist)
            sd = math.sqrt(sum((x - md) ** 2 for x in hist) / max(1, l

    ↪ en(hist) - 1)) if len(hist) > 1 else md
            p = max(p, off + md + cfg.z * sd)
        elif off + plant.dev >= law.band.wall_high:
            p = max(p, off + plant.dev)
    F = law.force(p)
    key, native, lo, hi, sign = lv
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if key == "release":
        ok = F < RELEASE and law.p < law.band.center and release_safe(
    ↪ plant)
        plant.override = {"release": 1.0} if ok else {}
        return plant.override
    span = hi - lo

    if key == "setpoint" and plant.template == "energy_storage":
        P = plant.P
        excess = plant.imp - P["p_lim_w"]
        coverable = 0.0 < excess <= P["p_batt_w"] * plant.override.get
    ↪ ("power", 1.0)
        x = lo if (law.p >= law.band.wall_high and (coverable or plant

    ↪ .imp >= 0.95 * P["p_lim_w"] and excess <= 0.0)) else native
        plant._v5_x = x
        plant.override = {} if abs(x - native) < 1e-9 else {key: x}
        return plant.override
    if law.p >= law.band.wall_high:
        # at the wall: the energy edge snaps straight back to native (

    ↪ always free, never beyond it); the speed edge
        # and the hybrid fail up to full capacity at once, as the earl
    ↪ ier law did
        if cfg.spend_beyond or cfg.hybrid:
            x = hi if sign > 0 else lo
        elif cfg.snap:

            x = native
        else:
            x = clamp(plant._v5_x + sign * UP * span, lo, hi)
    ↪  # the law's full force up, smooth, never a snap
            x = min(x, native) if sign > 0 else max(x, native)
    else:

        g = UP if F > 0 else DOWN
        x = clamp(plant._v5_x + sign * g * F * span, lo, hi)
        if not (cfg.spend_beyond or cfg.hybrid):
            # the energy edge: away from the wall nothing is spent bey
    ↪ ond native (native is always free)
            x = min(x, native) if sign > 0 else max(x, native)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if cfg.drop_guard and plant.template == "compute_pool" and key ==
    ↪ "power":
        x = max(x, drop_floor(plant, cfg))
    ↪  # no request turned away (keeps the history)
        x = max(x, latency_floor(plant, cfg))
    ↪  # no response pushed over the line

    if cfg.hole_guard and plant.template == "thermal_zone" and key ==
    ↪ "setpoint":
        x = min(x, cooling_hole_ceiling(plant, cfg, native))
    if cfg.motion_safe and plant.template == "motion_axis":
        # the edge read from the axis's own physics, at once: never sl
    ↪ ower than its queue and the next decision's burst

        # allow, never weaker than its next move needs
        x = max(x, safe_speed(plant, cfg, lo) if key == "capacity" els
    ↪ e effort_floor(plant))
    if cfg.loop_reach and plant.template == "process_loop" and key ==
    ↪ "setpoint":
        x = loop_clamp(plant, cfg, x, native)

    plant._v5_x = x
    plant.override = {} if abs(x - native) < 1e-9 else {key: x}
    return plant.override


# The tune-down setting (the dial turned from the max down to save): n

    ↪ ative's work on the least energy, no muscle giving up
# anything native holds on the modelled plants. The autoscaler's targe
    ↪ t held at the operator's; the step-to-step
# estimator; a fall while Omni held back raises that plant's cushion f
    ↪ loor; the hybrid never behind the earlier law; each
# plant remembers its own physics; motion speed and effort never below

    ↪  what the axis needs; a process loop read against
# its band's edge, its setpoint never closer to that edge than its own
    ↪  largest swing (generators included); a request pool
# never capped below the largest demand jump; a cooled zone never warm
    ↪ er than its cooling can recover; a power cap only on
# a pool whose response line outlasts a decision, never below the larg
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ est demand jump or past the line; the battery shave
# and a cloud machine pool's hand-back held native.
V5SAVE = Cfg(dial="save", target_free=False, est=2, learn_fall=0.1, hy
    ↪ brid=True, physics_memory=True, motion_safe=True,
             band=True, drop_guard=True, hole_guard=True, loop_reach=T
    ↪ rue, power_cap=True)

# The engine's default: the max (the founder's order of 12 October: "W
    ↪ e're going to show a benchmark that shows it to the
# max. And we're going to show one native... anybody can tune it from
    ↪ the max down"). Every muscle pushed to its own limit
# for the most work, faster; the energy that takes is power given.

V5CFG = Cfg(dial="max", window=20, edge_util=0.95, line_z=3.09)


def apply_v5(plant, knob):
    """compass_apply_v5 with the engine's default v5 configuration (V5
    ↪ CFG, the max)."""

    return compass_apply_v5(plant, knob, V5CFG)


def apply_v5_save(plant, knob):
    """compass_apply_v5 turned down to save (V5SAVE)."""
    return compass_apply_v5(plant, knob, V5SAVE)
```

H.44 realms/gpu_card.py

262 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""A GPU card on its real physics, its own firmware, and the two-wire
    ↪ plug (evidence class S: a model, not a meter).

The card

  clock     f, a share of the top clock (1.0), moved by the firmware i
    ↪ n bins of 0.9% (about 15 MHz on a 1.7 GHz part)
  power     P = idle + leakage(T) + P_dyn x busy x f x V(f)^2, with V(
    ↪ f) = 0.6 + 0.4 f: dynamic power rises with the
            clock and with the square of the voltage the clock needs,
    ↪ so the top of the clock range costs the most

  heat      C dT/dt = P - (T - T_amb) / R_th; leakage rises with tempe
    ↪ rature
  work      requests arrive (a slow wave, bursts, noise, drawn from th
    ↪ e seed); the card serves f x mu per second;
            a request's response time is its wait plus its service tim
    ↪ e

The firmware (native, what ships on the card), 50 times a second
  boost     while there is work and room, raise the clock one bin, up
    ↪ to the clock ceiling
  hammer    while the draw is over the power limit, or the chip is ove
    ↪ r its slowdown temperature, drop three bins

  So the clock saws against the limit for as long as the work keeps co
    ↪ ming: up a bin, knocked down three, up again.

The work
  memb      the share of a request's time at the top clock spent waiti
    ↪ ng on memory, which does not speed up with the

            clock: 0 is compute-bound work (large matrix products, the
    ↪  bench's pinned workload), 0.85 is typical of AI
            token generation (each token streams the model's weights f
    ↪ rom memory). The card serves 1 / ((1 - memb) / f
            + memb) of its top speed
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
The arms: each base alone, and the same base with Omni-Compass on top
    ↪ (Omni never runs the card itself)
  native    the card as shipped: power limit 150 W, clock ceiling at t
    ↪ he top, firmware alone
  omni      native + Omni-Compass on top, through two wires (omnicompa
    ↪ ss/compass_law.py): the clock ceiling is the up wire (how

            high boost may push), the power limit is the down wire (th
    ↪ e lid, set just above what the ceiling draws,
            never under the card's own busy draw). The compass's readi
    ↪ ng is the response time as a position between the
            bare service time (0) and the service line (1); the brain
    ↪ pulls it to the center. The verdict

            (omnicompass/verdict.py) decides how far the ceiling may g
    ↪ o: a step down is allowed only after a paired
            trial shows it costs each request at most ALLOW of the car
    ↪ d's own time on it; where no step passes, the
            ceiling stays where the firmware has it. Both wires are re
    ↪ stored to their snapshot at 90% of the run

  cap       a fixed power cap set by the operator and left (105 W, 70%
    ↪ ), firmware alone under it
  cap_omni  the cap + Omni-Compass on top: the same law, the lid never
    ↪  above the operator's cap
"""
from __future__ import annotations

import math
import random
from typing import Dict

from omnicompass.compass_law import Band, CompassLaw, Plug, clamp

from omnicompass.verdict import Verdict

TICK = 0.02                # firmware period, s
DECIDE = 1.0               # brain period, s
BIN = 0.009                # one clock bin, share of the top clock
F_MIN = 0.12
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
P_IDLE, P_DYN = 22.0, 165.0
LEAK0, LEAK_T = 8.0, 0.02   # leakage at 40 C, and its rise per C
T_AMB, R_TH, C_TH = 30.0, 0.33, 90.0     # C, C/W, J/C  (150 W -> abou
    ↪ t 80 C, time constant about 30 s)
T_SLOW = 87.0
MU = 100.0                  # requests per second at the top clock

LIMIT_DEFAULT, LIMIT_MIN = 150.0, 105.0
SLO_S = 10.0 / MU           # the service line: ten bare service times
    ↪  (as the GPU bench sets it)
# the law of omni_controller/gpu_compass.py (docs/GPU_PREREGISTRATION.
    ↪ md, amendment 8): up and down gains (share of the top
# clock per unit of force), the compass's center, the speed floor (a s

    ↪ hare of the card's own busy clock), the verdict's
# allowance (the most a step may add to the card's own time on a reque
    ↪ st) and how often it runs a trial (decisions)
UP, DOWN, CENTER, FLOOR, ALLOW, PROBE_EVERY = 0.10, 0.0125, 0.4, 1.0,
    ↪ 0.02, 8
OMNI = ("omni", "cap_omni")

SATURATED = 0.95            # busy share over a decision at which work
    ↪  is waiting: race, never pace



def volt(f):
    return 0.6 + 0.4 * f

def power(f, busy, T):
    return P_IDLE + LEAK0 * (1.0 + LEAK_T * (T - 40.0)) + P_DYN * busy
    ↪  * f * volt(f) ** 2

class Card:
    def __init__(self, seed: int, duration: float, memb: float = 0.0):
        r = random.Random(seed)
        self.memb = memb
        n = int(duration / TICK)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        phase = r.uniform(0, 2 * math.pi)
        self.lam, left, amp, z = [], 0, 0.0, 0.0
        per = int(1.0 / TICK)
        for k in range(n):
            if k % per == 0:
                z = 0.85 * z + r.gauss(0, 0.05)

                if left <= 0 and r.random() < 0.02:
                    left, amp = r.randint(5, 25), r.uniform(0.1, 0.3)
                left -= 1
            wave = 0.45 + 0.15 * math.sin(2 * math.pi * k * TICK / 300
    ↪ .0 + phase)
            self.lam.append(max(0.0, MU * (wave + (amp if left > 0 els

    ↪ e 0.0)) * math.exp(z)))
        self.n = n
        self.k = 0
        self.f, self.T, self.Q = 0.5, 45.0, 0.0
        self.last_p, self.last_busy = P_IDLE, 0.0
        self.limit, self.ceiling = LIMIT_DEFAULT, 1.0

        self.m = {"energy_j": 0.0, "served": 0.0, "arrived": 0.0, "ham
    ↪ mer": 0, "reversals": 0, "viol_s": 0.0,
                  "t_peak": 0.0, "t_sum": 0.0, "f_sum": 0.0, "f_sq": 0
    ↪ .0}
        self.resp = []           # (response time, requests) per tick,
    ↪  for the percentiles

        self.last_dir = 0
        self.window = []         # response times this decision period
        self.bwin = []           # busy shares this decision period
        self.swin = []           # the card's own time per request thi
    ↪ s decision period (the wait in the queue left out)

    def tick(self):
        k = self.k
        a = self.lam[k] * TICK
        sp = 1.0 / ((1.0 - self.memb) / self.f + self.memb)
        cap = sp * MU * TICK
        served = min(self.Q + a, cap)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.Q += a - served
        busy = served / cap if cap > 0 else 1.0
        P = power(self.f, busy, self.T)
        self.T += (P - (self.T - T_AMB) / R_TH) * TICK / C_TH
        w = self.Q / (sp * MU) + 1.0 / (sp * MU)
        self.resp.append((w, a))

        self.window.append(w)
        self.bwin.append(busy)
        if a > 0:
            self.swin.extend([1.0 / (sp * MU)] * max(1, int(round(a)))
    ↪ )
        m = self.m

        self.last_p, self.last_busy = P, busy
        m["energy_j"] += P * TICK; m["served"] += served; m["arrived"]
    ↪  += a
        m["t_peak"] = max(m["t_peak"], self.T); m["t_sum"] += self.T
        m["f_sum"] += self.f; m["f_sq"] += self.f * self.f
        if w > SLO_S:

            m["viol_s"] += TICK
        # the firmware: boost up a bin, or the hammer down three
        if P > self.limit or self.T > T_SLOW:
            d = -3
            m["hammer"] += 1
        elif busy > 0.05 and self.f + BIN <= self.ceiling + 1e-9:

            d = 1
        elif self.f > self.ceiling + 1e-9:
            d = -1
        else:
            d = 0
        if d and self.last_dir and (d > 0) != (self.last_dir > 0):

            m["reversals"] += 1
        if d:
            self.last_dir = d
        self.f = clamp(self.f + d * BIN, F_MIN, 1.0)
        self.k += 1
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class CeilingPlug(Plug):
    """The up wire: the clock ceiling the firmware may boost to (nvidi
    ↪ a-smi -lgc on a real card; -rgc restores)."""
    def __init__(self, card):
        super().__init__(0.3, 1.0)
        self.card = card

    def _read_service(self):
        """The service as a position in its compass: response time onl
    ↪ y (bare service time 0, the line 1). A busy card is
        doing its work; being busy is not a breach and is not read as
    ↪ one."""
        w = self.card.window
        return (sum(w) / len(w) - 1.0 / MU) / (SLO_S - 1.0 / MU) if w
    ↪ else 0.0

    def _read_lever(self):
        return self.card.ceiling

    def _send(self, v):
        self.card.ceiling = round(v / BIN) * BIN if v < 1.0 else 1.0
class LimitPlug(Plug):
    """The down wire: the power limit, the lid (nvidia-smi -pl on a re
    ↪ al card), whole watts."""
    def __init__(self, card, top=LIMIT_DEFAULT):
        super().__init__(LIMIT_MIN, top, tolerance=0.5)
        self.card = card
    def _read_service(self):
        return self.card.T

    def _read_lever(self):
        return self.card.limit
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def _send(self, v):
        self.card.limit = float(round(v))


def run(seed: int, arm: str, duration: float = 600.0, memb: float = 0.
    ↪ 0) -> Dict:
    card = Card(seed, duration, memb)
    per = int(DECIDE / TICK)
    kill = int(0.9 * card.n)
    restored = True
    up = down = brain = vd = None
    base_limit = LIMIT_MIN if arm in ("cap", "cap_omni") else LIMIT_DE
    ↪ FAULT
    card.limit = base_limit
    events = {"trials": 0, "allowed": 0, "refused": 0}
    if arm in OMNI:
        up, down = CeilingPlug(card), LimitPlug(card, top=base_limit)
        up.attach(); down.attach()
        brain = CompassLaw(Band(lo=0.0, hi=1.0, center=CENTER), dt=DEC
    ↪ IDE, tau=2.0, kp=1.0, authority=1.0, smooth=0.3)
        brain.kd *= 3.0                                  # the push: t
    ↪ hree times the damping that only stops the slosh,
                                                         # so a rising
    ↪  load is met before it reaches the wall
        vd = Verdict(tolerance=ALLOW, probe_every=PROBE_EVERY)
    learn_f, learn_p = [], []          # the card under its base while
    ↪  busy, ceiling at the top: its clock and its draw
    for k in range(card.n):
        if brain is not None and card.ceiling >= 1.0 and card.limit >=
    ↪  base_limit and card.last_busy >= 0.9:
            learn_f.append(card.f); learn_p.append(card.last_p)
        if brain is not None and k % per == 0 and k:
            if k >= kill:
                if k - per < kill:
                    restored = up.restore() and down.restore()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            else:
                F = brain.force(up.read())
                # the speed floor: never under the clock the card reac
    ↪ hes under its base while busy (the top until 15
                # busy readings are in), so waiting work is never serv
    ↪ ed slower than the base

                f_nat = sorted(learn_f[-10000:])[len(learn_f[-10000:])
    ↪  // 2] if len(learn_f) >= 15 else 1.0
                p_nat = sorted(learn_p[-10000:])[int(0.9 * (len(learn_
    ↪ p[-10000:]) - 1))] if len(learn_p) >= 15 else base_limit
                b = card.bwin
                saturated = bool(b) and sum(b) / len(b) >= SATURATED

                # the verdict: what the last decision's requests cost
    ↪ the card at the step it stood at, then how far
                # the ceiling may go now (or the step a trial needs)
                vd.observe(card.swin)
                deepest, trial, ev = vd.tick(brain.p < brain.band.wall
    ↪ _high and not saturated)

                if ev:
                    key = {"trial": "trials", "step allowed": "allowed
    ↪ "}.get(ev["verdict"], "refused" if "refused" in ev["verdict"] el
    ↪ se None)
                    if key:
                        events[key] += 1

                at_limit = card.last_p >= 0.97 * card.limit
                if saturated and at_limit and len(learn_f) >= 15 and b
    ↪ ase_limit < LIMIT_DEFAULT:
                    # saturated against an operator's cap: hold at the
    ↪  card's own busy clock under it (amendments 9, 12);
                    # on the card's own factory limit the firmware's b

    ↪ oost serves the burst
                    c = up.write(max(0.3, min(1.0, f_nat)))
                elif brain.p >= brain.band.wall_high or saturated:
                    # fail up past the wall; and race while work waits
    ↪  (the card saturated: a queue is forming), so a
                    # burst is always served at full speed; the compas
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ s paces only the slack between bursts
                    c = up.write(1.0)
                elif trial:
                    c = up.write(1.0 - deepest * BIN)
                else:
                    g = UP if F > 0 else DOWN                    # up
    ↪ fast (service first), down gently
                    c_lo = max(min(1.0, f_nat * FLOOR), 1.0 - deepest
    ↪ * BIN)
                    c = up.write(max(c_lo, card.ceiling + g * F))
                # the lid just above what the ceiling draws fully busy
    ↪ , never under the card's own busy draw
                lid = max(power(c, 1.0, card.T), p_nat) * 1.06
                down.write(lid)
            card.window, card.bwin, card.swin = [], [], []
        elif k % per == 0:
            card.window, card.bwin, card.swin = [], [], []
        card.tick()

    m = dict(card.m)
    n = card.n
    rs = sorted(card.resp)
    tot = sum(a for _, a in rs) or 1.0

    def pct(q):
        acc = 0.0
        for w, a in rs:
            acc += a
            if acc >= q * tot:
                return w
        return rs[-1][0]

    mean_f = m["f_sum"] / n
    return {"energy_j": m["energy_j"], "served": m["served"], "work_pe
    ↪ r_kj": m["served"] / (m["energy_j"] / 1000.0),
            "p50_ms": 1000 * pct(0.5), "p95_ms": 1000 * pct(0.95), "p9
    ↪ 9_ms": 1000 * pct(0.99),
            "viol_share": m["viol_s"] / duration, "hammer_per_s": m["h
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ammer"] / duration,
           "reversals_per_s": m["reversals"] / duration, "clock_mean"
    ↪ : mean_f,
           "clock_jitter": math.sqrt(max(0.0, m["f_sq"] / n - mean_f
    ↪ ** 2)), "t_peak": m["t_peak"],
           "t_mean": m["t_sum"] / n, "backlog_end": card.Q, "restored
    ↪ ": restored,
           "writes": (up.writes + down.writes) if up else 0, "verdict
    ↪ _steps_allowed": vd.allowed if vd else 0,
           "verdict_events": events}
```

H.45 realms/harness.py

608 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Realm harness: every muscle native, watched and governed; every rea
    ↪ lm, and the whole tower, as one organism.

Arms, all on the same seed (the same demand, weather, disturbances):

  native  the plant and its native controller alone
  watch   the frozen governor reads the plant every period and writes
    ↪ nothing; must equal native exactly
  omni    the frozen governor holds the muscle's one knob; at 90% of t
    ↪ he run it is killed, the knob returns to the
          native controller, and the harness checks that it did
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
The governor is omnicompass.adapter.Governor, unchanged (the engine wi
    ↪ th u = 0, the stack law, the reset).

Single muscle: one plant, one governor reading that plant.
Organism: all the plants of a realm (or the whole tower) on one 15 s c
    ↪ lock, coupled: the electrical power of compute, motion
and process plants is heat in the realm's thermal zones; the organism'
    ↪ s load swing is load on its storage sites; the
zones' temperature is the ambient every other plant reports. One gover
    ↪ nor reads the organism's aggregate and its one
directive sets every muscle's knob.

Primary outcome per seed: work per energy, omni against native: (work_
    ↪ omni / work_native) / (energy_omni /
energy_native) - 1. For an organism, work is the mean over its plants
    ↪ of work_omni / work_native (their work units
differ) and energy is total joules. Guardrails: work not lower by more
    ↪  than 1%; the share of periods in violation not
higher by more than 1 percentage point. Labels by rule (label()).
"""
from __future__ import annotations

import csv

import math
from pathlib import Path
from typing import Dict, List

from omnicompass.adapter import Governor, OBSERVE
from omnicompass.nervous_system import from_governor

from .plants import TEMPLATES, ThermalZone, EnergyStorage, pack
from .compass_arm import compass_apply
from .compass_arm_v5 import apply_v5, apply_v5_save, max_apply, max_bo
    ↪ dy_step, Bone, V5CFG, CARDS   # the v5 law: the max
                                                    # (arm v5, the de
    ↪ fault) and the tune-down (arm v5save)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
from .presets import STEPS_SINGLE, ORGANISM_STEPS, CAL_SEED, params_fo
    ↪ r

ROOT = Path(__file__).resolve().parents[1]
LAWS = {"compass": compass_apply, "v5": lambda p, k: max_apply(p, k, V
    ↪ 5CFG), "v5save": apply_v5_save}

ARMS = ("native", "watch", "compass")   # native; Omni watching (write
    ↪ s nothing); the compass law on top (amendment, 2026-10-05)
SOLO = "solo"     # Omni-Compass by itself (omni-v19, docs/SOLO_PREREG
    ↪ ISTRATION.md): the max, with every pool whose own count is
                  # the muscle's knob run with its autoscaler switched
    ↪  off (and its twin, the brain's sense of native, too)

def native_off_pool(template: str, knob: str) -> bool:
    """Whether by itself switches off this muscle's own native control
    ↪ ler: a pool whose knob is its count (capacity, or the
    autoscaler's target), where native is the autoscaler. A power cap,

    ↪  an admission limit and every other plant's knob are
    a setting native already holds still, or run through native's own
    ↪ loop as their actuator: by itself is the same arm."""
    return template == "compute_pool" and knob not in ("power", "admis
    ↪ sion")
# for a setpoint muscle, a fourth arm: the native controller with the

    ↪ setpoint simply fixed at the band's calm end.
# It shows how much of any Omni result on that muscle the band alone w
    ↪ ould give, with no governor.
FIXED = "fixed_calm"
KILL_AT = 0.8
WALK = 24         # decisions over which the lease walks every setting

    ↪  back to native after the brain goes silent (the
                  # plug's own reflex: a room warmed or a loop eased i
    ↪ s brought back slowly enough that the body repays it
                  # without a jump its lines must absorb at once); the
    ↪  security kill switch stays instant. The run keeps
                  # going native after the walk, so every setting is p
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ roven back at the operator's before it ends
CLIMB = (1, 2, 3, 2, 3, 4, 5, 6, 5, 4, 5, 6, 7, 8, 7, 6, 5, 4, 3, 4, 3
    ↪ , 2, 1, 2, 1)
# the climb: the load steps by one, up and down (1 2 3 2 3 4 ...), lev
    ↪ el 4 is the organism's own normal load and level
# 8 twice it; every muscle's work demand follows it (requests, the roo

    ↪ ms' IT heat, the axes' tasks, the loops' demand),
# so the organism is pushed past what native can carry and the most ea
    ↪ ch arm can do shows. A site's own base load stays
# its own: the climb reaches it through the body's power swing on its
    ↪ line

LINE = 0.25       # the line: work already waiting when the run starts
    ↪ , a quarter of the work the run itself brings (the
                  # founder's order of 12 October: work waiting in lin
    ↪ e, so the most each arm can clear shows). A pool's
                  # line is jobs submitted before the start (its machi

    ↪ nes take them from what the requests leave them),
                  # an axis's is tasks queued, a site's is flexible lo
    ↪ ad held over; a room's heat and a loop's demand
                  # cannot wait, so they have none. Both arms of a see
    ↪ d start with the same line. Fixed before any
                  # preregistered run (docs/OMNI_V5.md, the max)

def climb_level(k: int, steps: int) -> float:
    """The climb's load multiplier at decision k of steps (level / 4).
    ↪ """
    return CLIMB[min(len(CLIMB) - 1, k * len(CLIMB) // steps)] / 4.0

def put_line(p):
    """Put the line on one plant (in place): a quarter of its own run'
    ↪ s work, waiting when the run starts."""
    P = p.P

    if p.template == "compute_pool":
        p.B = p.line0 = LINE * sum(p.lam) * P["dt"]
    elif p.template == "motion_axis":
        n = int(round(LINE * sum(p.arrivals)))
        p.backlog += n
        p.waits = [0.0] * n + p.waits
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        p.line_n = n
    elif p.template == "energy_storage":
        p.line_e = LINE * P["flex"] * sum(p.load) * P["dt"]
        p.deferred_j += p.line_e
    return p

def scale_load_plant(p, profile="climb"):
    """Put a load on one plant's work demand (in place): the climb, th
    ↪ e line, or a constant multiple."""
    if profile == "line":
        return put_line(p)

    steps = p.steps
    m = [climb_level(k, steps) for k in range(steps)] if profile == "c
    ↪ limb" else [float(profile)] * steps
    if p.template == "compute_pool":
        p.lam = [x * m[k] for k, x in enumerate(p.lam)]
    elif p.template == "thermal_zone":

        p.qit = [x * m[k] for k, x in enumerate(p.qit)]
    elif p.template == "motion_axis":
        p.arrivals = [int(round(x * m[k])) for k, x in enumerate(p.arr
    ↪ ivals)]
    elif p.template == "process_loop":
        p.d = [x * m[k] for k, x in enumerate(p.d)]

    return p


def scale_load(body, profile="climb"):
    """Put a load on a body's work demand (in place; every arm of a se
    ↪ ed gets the same load)."""

    body.load = profile
    for p in body.plants:
        scale_load_plant(p, profile)
    return body
from omnicompass.student_t import T95  # noqa: E402  (the two-sided 95
    ↪ % point of Student's t, any degrees of freedom)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def catalog(path=None) -> List[dict]:
    return list(csv.DictReader((Path(path) if path else ROOT / "realms
    ↪ " / "catalog.csv").open()))

# the six organisms: the four realms, the whole tower (every muscle on
    ↪ ce) and the stack (the four realms on one clock,
# every duplicate kept). Nothing is named by a count: the v1 names car
    ↪ ried one and are read as aliases.
TOWER, STACK = "tower", "stack"

ORGANISM_ALIAS = {"organism_656": TOWER, "tower_656": TOWER, "stack_12
    ↪ 26": STACK}



def organism_name(name: str) -> str:
    return ORGANISM_ALIAS.get(name, name)

def organism_sizes(rows=None) -> dict:
    """{organism: muscles} for the six, from the catalog."""
    rows = rows or catalog()
    realms = ("compute_ai_cloud", "physics_robotics_autonomous", "ener

    ↪ gy_facility_industrial", "distribution_specialized")
    sizes = {r: sum(1 for x in rows if r in x["realms"].split(";")) fo
    ↪ r r in realms}
    sizes[STACK] = sum(sizes.values())
    sizes[TOWER] = len(rows)
    return sizes

def seed_for(muscle_id: str, seed: int) -> int:
    h = 0
    for ch in f"{muscle_id}:{seed}":
        h = (h * 1000003 + ord(ch)) % 2_147_483_647

    return h



def make_plant(row, seed, organism=False):
    steps = ORGANISM_STEPS if organism else STEPS_SINGLE[row["template
    ↪ "]]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    p = TEMPLATES[row["template"]](params_for(row, organism), seed_for
    ↪ (row["muscle_id"], seed), steps)
    if row["template"] == "compute_pool" and not organism:
        p.budget_w = 1.25 * native_mean_power(row)
    p._card = CARDS.get(row["muscle_id"])        # the muscle's own se
    ↪ ttings for the max (none: the global setting)

    return p


_CAL = {}

def native_mean_power(row):
    """A compute pool's declared power budget basis: its mean native d
    ↪ raw on the calibration seed."""
    key = row["muscle_id"]
    if key not in _CAL:
        c = TEMPLATES[row["template"]](params_for(row), seed_for(key,

    ↪ CAL_SEED), STEPS_SINGLE[row["template"]])
        tot = 0.0
        for _ in range(c.steps):
            c.step()
            tot += c.power_w
        _CAL[key] = tot / c.steps

    return _CAL[key]


def calibrate_organism(rows):
    """Each plant's mean native power in the organism on the calibrati
    ↪ on seed: the organism's declared budget and

    coupling basis (the same for every arm and every result seed)."""
    key = tuple(r["muscle_id"] for r in rows)
    if key not in _CAL:
        nat = run_organism_arm(rows, CAL_SEED, "native", nominal=None)
        _CAL[key] = nat["mean_power"]
    return _CAL[key]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def authority(g, obs, d, clean):
    """The nervous system's authority after the governor's step, as th
    ↪ e live controller computes it."""
    return from_governor(g, dict(obs, slo_clean=clean), d, mode="autop
    ↪ ilot")


def _apply(plant, knob, g, d, auth):
    """Set the plant's override from the directive and authority; retu
    ↪ rn it ({} when the knob is native)."""
    if g is None or not g.has_authority:
        plant.override = {}
        return {}
    plant.override = plant.omni_override(knob, d, g, auth)
    return plant.override
def _restored(plant, knob):
    return plant.override == {}


def _native_of(plant, key):
    """The operator's own value of an override key (what the snapshot
    ↪ holds); None: the key has no setting of its own
    (a floor, a release, a paced job, a battery's clip ceiling) and is
    ↪  simply let go."""
    P, t = plant.P, plant.template
    if key == "target":
        return P["target"]
    if key == "setpoint":
        return P["t_set"] if t == "thermal_zone" else P["reserve"] if
    ↪ t == "energy_storage" else P["sp"]
    if key in ("power",) or (key == "capacity" and t in ("motion_axis"
    ↪ , "process_loop")):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return 1.0
    if key == "floor":
        return float(P["n_min"])          # a floor walks down to the
    ↪ pool's least machines (the autoscaler governs)
    return None

def walk_back(plant, left):
    """One decision of the lease's walk back to native: every setting
    ↪ moves 1/left of the way to the operator's own;
    a key with no setting of its own is let go. left = 1 is the last s
    ↪ tep: everything native."""

    out = {}
    for key, x in plant.override.items():
        nat = _native_of(plant, key)
        if nat is None or left <= 1:
            continue
        if key == "floor" and getattr(plant, "_mx_guard", False):

            continue        # a floor that only held the pool at nativ
    ↪ e's own count (it added nothing) is let go at once:
                            # the autoscaler has the pool, and a walki
    ↪ ng floor would hold machines native has released
        y = x + (nat - x) / left
        if abs(y - nat) > 1e-9:

            out[key] = y
    plant.override = out
    return out


PACE_HIGH, PACE_LOW, PACE_HEAT = 0.95, 0.8, 0.96             # the liv

    ↪ e batch_pace defaults (omni_controller/muscles.py)


def pace(paced_plants, obs, auth):
    """The live batch_pace rule over the admission muscles: while hot,
    ↪  suspend the next running one; while calm, resume
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    the first suspended one; one per decision either way. Band first:
    ↪ a job whose service is out of its line, or would
    leave it if held one more period, is never suspended, and if suspe
    ↪ nded it is resumed at once (fail up)."""
    if not paced_plants:
        return

    for p in paced_plants:
        if p.paced and not (p.slo_clean and p.can_hold()):
            p.paced = False
    hot = (obs["power_stress"] >= PACE_HIGH or obs["thermal"] >= PACE_
    ↪ HEAT
           or bool(auth and auth["organs"]["batch"].get("pause")))

    calm = obs["power_stress"] <= PACE_LOW and obs["thermal"] < PACE_H
    ↪ EAT - 0.06
    if hot:
        for p in paced_plants:
            if not p.paced and p.slo_clean and p.can_hold():
                p.paced = True

                return
    elif calm:
        for p in paced_plants:
            if p.paced:
                p.paced = False
                return

def line_left(plant):
    """The share of a plant's line still waiting at the end (0: cleare
    ↪ d; None: the plant had no line)."""
    if getattr(plant, "line_n", 0):

        return max(0, plant.line_n - plant.line_done) / plant.line_n
    if getattr(plant, "line_e", 0.0):
        return max(0.0, plant.deferred_j) / plant.line_e
    if plant.template == "compute_pool" and getattr(plant, "line0", 0.
    ↪ 0):
        return plant.B / plant.line0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return None


def run_muscle_arm(row, seed, arm, load=None):
    plant = make_plant(row, seed)
    if load is not None:
        scale_load_plant(plant, load)
    knob = row["knob"]
    if arm == FIXED:
        for k in range(plant.steps):
            plant.override = plant.fixed_calm()
            plant.step()
        plant.finalize()
        return dict(plant.m, writes=0, after_kill_writes=0, restore_ok
    ↪ =True)
    g = None if arm in ("native", FIXED, "compass", "v5", "v5save", SO
    ↪ LO) else Governor()
    if arm == "watch":
        g.set_mode(OBSERVE)
    kill_at = int(KILL_AT * plant.steps)
    writes = after_kill_writes = 0
    last = None
    restore_ok = True
    twin = make_plant(row, seed) if arm in ("v5", SOLO) else None    #
    ↪  the brain's model of native (the max's floor)
    if twin is not None and load is not None:
        scale_load_plant(twin, load)
    if arm == SOLO and native_off_pool(row["template"], knob):
        plant.native_off = twin.native_off = True                    #
    ↪  by itself: the pool's own autoscaler off, in both
    for k in range(plant.steps):
        if arm in ("compass", "v5", "v5save", SOLO):
            apply = LAWS["v5" if arm == SOLO else arm]
            if k < kill_at:
                if twin is not None:
                    twin.step()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                    v = max_apply(plant, knob, V5CFG, twin)
                else:
                    v = apply(plant, knob)
                if v and v != last:
                    writes += 1
            else:

                v = walk_back(plant, kill_at + WALK - k)
                if k >= kill_at + WALK - 1:
                    restore_ok = restore_ok and _restored(plant, knob)
            last = v
        elif g is not None:
            obs = plant.observe()

            if arm == "omni" and k == kill_at:
                g.kill()
            g.current_cap = 1.0                                    # a
    ↪ s the live controller sets it
            d = g.step(obs, 0)
            auth = authority(g, obs, d, plant.slo_clean) if g.has_auth

    ↪ ority else None
            if knob == "admission" and g.has_authority and (row["templ
    ↪ ate"] != "compute_pool" or plant.P.get("pausable")):
                pace([plant], obs, auth)
            v = _apply(plant, knob, g, d, auth)
            if v:

                if v != last:
                    writes += 1
                last = v
            if arm == "omni" and k >= kill_at:
                after_kill_writes += int(bool(v))
                restore_ok = restore_ok and _restored(plant, knob)

        plant.step()
    plant.finalize()
    m = dict(plant.m)
    m.update(writes=writes, after_kill_writes=after_kill_writes, resto
    ↪ re_ok=restore_ok and after_kill_writes == 0)
    left = line_left(plant)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if left is not None:                    # the line: the decision
    ↪ it was cleared at (the run's length if it was not)
        m.update(line_left=left, line_k=plant.line_k if plant.line_k i
    ↪ s not None else plant.steps)
    return m

def run_muscle(row, seed) -> Dict:
    out = {arm: run_muscle_arm(row, seed, arm) for arm in ARMS}
    if row["knob"] == "setpoint":
        out[FIXED] = run_muscle_arm(row, seed, FIXED)
    core = ("work", "energy_j", "viol", "steps")

    out["watch_equal"] = all(out["watch"][f] == out["native"][f] for f
    ↪  in core) and out["watch"]["writes"] == 0
    return out


# ---------------------------------------------------------------------

    ↪ ----------------------------------------
class Body:
    """One organism's plants and its internal coupling: the electrical
    ↪  power of compute, motion and process plants is
    heat in its thermal zones, its load swing is load on its storage s
    ↪ ites, and its zones' temperature is the ambient

    every other plant reports."""

    def __init__(self, rows, seed, nominal="calibrated"):
        self.rows, self.seed, self.nominal = rows, seed, nominal
        self.load = None                    # the load profile put on
    ↪  the body's work demand (None: its own)

        # the calibration run first, then this body: one organism in m
    ↪ emory at a time (1,000 copies of the four stacked
        # is 1.2 million plants). Each plant is packed as it is made (
    ↪ plants.pack): the same numbers, a quarter of the room
        cal = calibrate_organism(rows) if nominal == "calibrated" else
    ↪  None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.plants = [pack(make_plant(r, seed, organism=True)) for r
    ↪ in rows]
        if cal is not None:
            for p, w in zip(self.plants, cal):
                p.nominal_w = max(w, 1.0)
        self.knobs = [r["knob"] for r in rows]

        self.paced = [p for p, r in zip(self.plants, rows) if r["knob"
    ↪ ] == "admission"
                      and (r["template"] != "compute_pool" or p.P.get(
    ↪ "pausable"))]
        self.zones = [p for p in self.plants if isinstance(p, ThermalZ
    ↪ one)]

        self.stores = [p for p in self.plants if isinstance(p, EnergyS
    ↪ torage)]
        self.sources = [p for p in self.plants if not isinstance(p, (T
    ↪ hermalZone, EnergyStorage))]
        self.nom_src = sum(p.nominal_w for p in self.sources) or 1.0
        self.nom_sz = self.nom_src + sum(z.nominal_w for z in self.zon

    ↪ es)
        self.nom_all = self.nom_sz + sum(s.nominal_w for s in self.sto
    ↪ res)
        self.k_heat = [0.3 * z.P["q_it_w"] / (self.nom_src / len(self.
    ↪ zones)) for z in self.zones] if self.zones else []
        self.k_load = [0.3 * s.P["load_w"] / (self.nom_sz / len(self.s

    ↪ tores)) for s in self.stores] if self.stores else []
        self.budget = 1.25 * self.nom_all
        self.src_w, self.sz_w, self.all_w = self.nom_src, self.nom_sz,
    ↪  self.nom_all
        self.psum = [0.0] * len(self.plants)

    def couple(self):
        zones = self.zones
        th = sum(z.zone_thermal() for z in zones) / len(zones) if zone
    ↪ s else None
        for z, kh in zip(zones, self.k_heat):
            z.ext = {"heat_w": kh * self.src_w / len(zones)}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        for s, kl in zip(self.stores, self.k_load):
            s.ext = {"load_w": kl * (self.sz_w - self.nom_sz) / len(se
    ↪ lf.stores)}
        if th is not None:
            for p in self.plants:
                if not isinstance(p, ThermalZone):
                    p.ext["thermal"] = th

    def step(self):
        for i, p in enumerate(self.plants):
            p.step()
            self.psum[i] += p.power_w
        self.src_w = sum(p.power_w for p in self.sources)
        self.sz_w = self.src_w + sum(p.power_w for p in self.zones)
        self.all_w = self.sz_w + sum(p.power_w for p in self.stores)


def aggregate(bodies):
    """One governor's reading of the bodies it serves: the mean of eve
    ↪ ry plant's observation, and power stress as the
    bodies' total draw against their total budget."""
    obs = [p.observe() for b in bodies for p in b.plants]
    n = len(obs)
    agg = {key: sum(o[key] for o in obs) / n for key in ("queue_ratio"
    ↪ , "load_ratio", "network_stress", "drift_ratio",
                                                         "thermal")}
    agg.update(power_stress=sum(b.all_w for b in bodies) / sum(b.budge
    ↪ t for b in bodies), stale=0.0,
               security_block=0.0)
    return agg

def omni_bones(bodies):
    """The max's skeleton for each body (realms/compass_arm_v5.py, Bon
    ↪ e), with its pools' twins."""
    return [Bone(b, V5CFG) for b in bodies]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def omni_step(bodies, bones, k, kill_at):
    """One decision of the engine's default brain over bodies: the max
    ↪  with each body's bone before the kill, the lease's
    walk back to native after it. Returns the plants whose setting cha

    ↪ nged (for the write count)."""
    changed = 0
    for bi, b in enumerate(bodies):
        if k >= kill_at:
            for p in b.plants:
                walk_back(p, kill_at + WALK - k)

            continue
        before = [dict(p.override) for p in b.plants]
        bones[bi].advance_twin()
        max_body_step(b, V5CFG, bones[bi])
        changed += sum(1 for p, o in zip(b.plants, before) if p.overri
    ↪ de and p.override != o)

    return changed


def run_bodies(bodies, arm, groups):
    """Run bodies on one 15 s clock. groups: lists of body indices, on
    ↪ e governor per group (one group of all bodies is

    one governor over the whole stack; one group per body is a separat
    ↪ e governor per body). arm native: no governor."""
    govs = []
    if arm not in ("native", "compass", "v5", "v5save", SOLO):
        for grp in groups:
            g = Governor()

            if arm == "watch":
                g.set_mode(OBSERVE)
            govs.append((g, [bodies[i] for i in grp]))
    kill_at = int(KILL_AT * ORGANISM_STEPS)
    writes = after_kill_writes = 0
    restore_ok = True
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    last = {}
    bones, twins = [], []
    if arm in ("v5", SOLO):
        bones = [Bone(b, V5CFG) for b in bodies]
    ↪  # each body's skeleton, with its pools' twins
    if arm == SOLO:

    ↪  # by itself: every pool whose count is its knob runs with
        for b, bone in zip(bodies, bones):
    ↪  # its autoscaler off, and so does its twin
            for i, (p, r) in enumerate(zip(b.plants, b.rows)):
                if native_off_pool(p.template, r["knob"]):
                    p.native_off = True

                    if i in bone.twins:
                        bone.twins[i].native_off = True
    for k in range(ORGANISM_STEPS):
        for b in bodies:
            b.couple()
        if arm in ("compass", "v5", "v5save", SOLO):

            apply = LAWS["v5" if arm == SOLO else arm]
            for bi, b in enumerate(bodies):
                if k >= kill_at:
                    for p in b.plants:
                        walk_back(p, kill_at + WALK - k)
                        if k >= kill_at + WALK - 1:

                            restore_ok = restore_ok and _restored(p, N
    ↪ one)
                    continue
                if arm in ("v5", SOLO):
                    bones[bi].advance_twin()
                    max_body_step(b, V5CFG, bones[bi])

    ↪  # the max: one body, its bone, its spare shared
                    for p in b.plants:
                        v = p.override
                        if v and v != last.get(id(p)):
                            writes += 1
                        last[id(p)] = dict(v)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                    continue
                for p, knob in zip(b.plants, b.knobs):
                    v = apply(p, knob)
                    if v and v != last.get(id(p)):
                        writes += 1
                    last[id(p)] = v

        for g, bs in govs:
            agg = aggregate(bs)
            if arm == "omni" and k == kill_at:
                g.kill()
            g.current_cap = 1.0
            d = g.step(agg, 0)

            auth = {c: authority(g, agg, d, c) for c in (True, False)}
    ↪  if g.has_authority else {True: None, False: None}
            if g.has_authority:
                pace([p for b in bs for p in b.paced], agg, auth[True]
    ↪ )
            for b in bs:

                for p, knob in zip(b.plants, b.knobs):
                    v = _apply(p, knob, g, d, auth[p.slo_clean])
                    if v:
                        if v != last.get(id(p)):
                            writes += 1
                        last[id(p)] = v

                    if arm == "omni" and k >= kill_at:
                        after_kill_writes += int(bool(v))
                        restore_ok = restore_ok and _restored(p, knob)
        for b in bodies:
            b.step()
        for tw in twins:

            tw.couple()
            tw.step()
    for b in bodies:
        for p in b.plants:
            p.finalize()
    return {"plants": [dict(p.m) for b in bodies for p in b.plants], "
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ writes": writes,
            "after_kill_writes": after_kill_writes, "restore_ok": rest
    ↪ ore_ok and after_kill_writes == 0,
            "mean_power": [x / ORGANISM_STEPS for b in bodies for x in
    ↪  b.psum]}

def run_organism_arm(rows, seed, arm, nominal="calibrated", load=None)
    ↪ :
    b = Body(rows, seed, nominal)
    if load is not None:
        scale_load(b, load)

    return run_bodies([b], arm, [[0]])


def run_organism(rows, seed) -> Dict:
    out = {arm: run_organism_arm(rows, seed, arm) for arm in ARMS}
    out["watch_equal"] = out["watch"]["plants"] == out["native"]["plan

    ↪ ts"] and out["watch"]["writes"] == 0
    for arm in ARMS:
        out[arm].pop("mean_power", None)
    return out

# ----------------------------------------------------------------------
    ↪ ----------------------------------------
def paired(arm_m, nat_m):
    """Per-seed contrasts of one muscle: primary, work change, violati
    ↪ on-share change (percentage points)."""
    w = arm_m["work"] / nat_m["work"] if nat_m["work"] > 0 else (1.0 i

    ↪ f arm_m["work"] == 0 else math.inf)
    e = arm_m["energy_j"] / nat_m["energy_j"] if nat_m["energy_j"] > 0
    ↪  else math.nan
    v = (arm_m["viol"] / arm_m["steps"] - nat_m["viol"] / nat_m["steps
    ↪ "]) * 100.0
    return {"primary": w / e - 1.0, "work": w - 1.0, "energy": e - 1.0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ , "viol_pp": v}


def paired_organism(arm, nat):
    ws = [a["work"] / n["work"] for a, n in zip(arm["plants"], nat["pl
    ↪ ants"]) if n["work"] > 0]
    w = sum(ws) / len(ws)
    e = sum(a["energy_j"] for a in arm["plants"]) / sum(n["energy_j"]
    ↪ for n in nat["plants"])
    va = sum(a["viol"] for a in arm["plants"]) / sum(a["steps"] for a
    ↪ in arm["plants"])
    vn = sum(n["viol"] for n in nat["plants"]) / sum(n["steps"] for n
    ↪ in nat["plants"])
    return {"primary": w / e - 1.0, "work": w - 1.0, "energy": e - 1.0
    ↪ , "viol_pp": (va - vn) * 100.0}


def interval(xs):
    n = len(xs)
    m = sum(xs) / n
    if n < 2:
        return m, m, m
    sd = math.sqrt(sum((x - m) ** 2 for x in xs) / (n - 1))
    t = T95.get(n - 1, 1.96)
    h = t * sd / math.sqrt(n)
    return m, m - h, m + h


def label(per_seed, valid=True):
    """Result label by rule (the GPU bench's rule, docs/GPU_PREREGISTR
    ↪ ATION.md, amendment 1)."""
    if not valid or not all(all(math.isfinite(c[k]) for k in c) for c
    ↪ in per_seed):
        return "INVALID"
    p = interval([c["primary"] for c in per_seed])
    w = interval([c["work"] for c in per_seed])
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    v = interval([c["viol_pp"] for c in per_seed])
    guard = w[1] >= -0.01 and v[0] <= 0.0        # band first: time ov
    ↪ er the line no higher than native's (mean)
    if p[2] < 0:
        # the mirror of the energy tradeoff: less work per energy, but
    ↪  no work lost and the time over the line proven

        # lower (the energy bought service; a battery's round trip is
    ↪ the usual price). Without that proof: work per
        # energy away from its aim, and nothing bought with it
        if w[1] >= -0.01 and v[2] < 0.0:
            return "SERVICE IMPROVEMENT WITH ENERGY TRADEOFF"
        return "AWAY FROM ITS AIM"

    if p[1] > 0:
        return "SUPERIOR WITHIN GUARDRAILS" if guard else "ENERGY IMPR
    ↪ OVEMENT WITH SERVICE TRADEOFF"
    return "NONINFERIOR / INCONCLUSIVE" if guard else "NOT ESTABLISHED
    ↪ "

def summarize(per_seed):
    return {k: interval([c[k] for c in per_seed]) for k in ("primary",
    ↪  "work", "energy", "viol_pp")}

# ---------------------------------------------------------------------
    ↪ ----------------------------------------
def run_stack(realm_rows, seed) -> Dict:
    """The four realm organisms stacked on one clock, every muscle as
    ↪ often as it appears (duplicates included), each
    realm keeping its own internal coupling. Arms:

      native     no governor
      separate   one governor per realm (as if each realm had its own
    ↪ Omni)
      one        one governor over the whole stack
    native_equal: the stacked native run reproduces each realm's own n
    ↪ ative run exactly, plant by plant."""
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    out = {}
    for arm, groups in (("native", None), ("separate", [[i] for i in r
    ↪ ange(len(realm_rows))]),
                        ("one", [list(range(len(realm_rows)))])):
        bodies = [Body(rows, seed) for rows in realm_rows]
        out[arm] = run_bodies(bodies, "native" if arm == "native" else
    ↪  "omni", groups or [])
    alone = []
    for rows in realm_rows:
        alone += run_organism_arm(rows, seed, "native")["plants"]
    out["native_equal"] = alone == out["native"]["plants"]
    for arm in ("native", "separate", "one"):
        out[arm].pop("mean_power", None)
    return out
```

H.46 realms/plants.py

864 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Realm plants: the machines the muscles act on (evidence class S: de
    ↪ clared models, not meters).

Five plant models. Each one carries its own native controller and runs
    ↪  correctly with no Omni at all:

  compute_pool    servers, nodes, GPUs, robots, radio cells or workers
    ↪  serving a request stream; native: the Kubernetes
                HPA rule (desired = ceil(n * utilisation / target),
    ↪ 10% tolerance, scale-down stabilisation window,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                  start-up delay), a fixed admission limit, full clock
  thermal_zone    a data hall or building zone cooled by staged units;
    ↪  native: PI on a fixed setpoint, units staged to
                  the load, chiller COP from supply and outdoor temper
    ↪ ature
  energy_storage  a site with load, solar and a battery behind a grid

    ↪ connection; native: self-consumption above a
                  fixed reserve
  motion_axis     a joint, flight axis, spacecraft axis or vehicle doi
    ↪ ng point-to-point moves from a task queue;
                  native: PID with feedforward at full speed, motor he
    ↪ ating and derating

  process_loop    a first-order process with dead time (level, pressur
    ↪ e, chamber temperature, feeder voltage);
                  native: PI on a fixed setpoint, pump or heater power
    ↪  from the actuator


A muscle is one knob of one plant. Omni may hold that knob and only th

    ↪ at one; the native controller keeps the rest.
Omni commands a knob the way the shipped controller commands its organ
    ↪ s (omni_controller/controller.py,
omni_controller/muscles.py), through the nervous system's authority (o
    ↪ mnicompass/nervous_system.py, from_governor):

  expand     adding capacity, performance or protection is always allo
    ↪ wed (except during a security hold)
  contract   giving any of it back (fewer machines, a lower speed or p
    ↪ ower limit, a warmer setpoint, a lower reserve
             of protection) only while the organ has contraction autho
    ↪ rity (calm >= the organ's threshold, senses live)

             and the plant's service has been clean for the last three
    ↪  decisions (the SLO reflex); continuous organs
             shed at most the calm fraction of their surplus per decis
    ↪ ion, discrete organs one unit, through a release
             gate (nothing waiting, and after the release the rest run
    ↪ s at or under the engine's utilisation target)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
  fail up    while service is breached, the knob returns to the native
    ↪  controller at once
  pacing     admission muscles are batch pacing (the live batch_pace m
    ↪ uscle): one muscle's work suspended per decision
             while power stress >= 0.95 or heat >= 0.96 or the nervous
    ↪  system calls a pause; one resumed per decision

             once power stress <= 0.8 and heat < 0.90 (realms/harness.
    ↪ py, pace)

By plant and knob:
  compute_pool  replicas and HPA-type knobs: the HPA target written as
    ↪  min(rho*, the operator's target), never tighter,

               held for one autoscaler window (the live controller's
    ↪ rule); machine pools (Cluster Autoscaler native):
               the node release gate; power: the GPU or CPU-frequency
    ↪  envelope, never under the work's draw plus 30%,
               and never on a request-served pool (the live controlle
    ↪ r: throttling it saves no energy, only adds

               wait); admission: batch pacing of pausable work only (
    ↪ request traffic is never paused)
  thermal_zone  setpoint: the live cooling law (warm while cool, cold
    ↪ as heat rises, inside [stress, stress + band x
               calm]); units: the release gate; power: the site power
    ↪  envelope above the load plus 10%; admission:

               paced, the migratable 10% is shed
  energy_storage reserve by power stress (released under stress, held
    ↪ when calm only with authority); peak-shave
               ceiling at rho* of the connection (expand only); batte
    ↪ ry power: the site power envelope; admission:
               paced, the flexible share is deferred; resumed, it is

    ↪ caught up
  motion_axis   speed: up at once to demand, down by the calm share of
    ↪  the surplus; effort: the power envelope;
               admission: paced, no new move starts
  process_loop  setpoint: toward the efficient end of its band by the
    ↪ calm share; actuator range: as motion speed;
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                power: the site power envelope above use plus 10%; adm
    ↪ ission: paced, 10% is shed

Every disturbance trace is drawn from the seed when the plant is built
    ↪ , so all arms of a seed see the same ones.
Meters (work, energy_j, viol, steps) are kept by the plant from its ow

    ↪ n state; Omni never supplies them.
"""
from __future__ import annotations

import math
import random

from array import array
from typing import Dict, Optional

KNOBS = ("capacity", "setpoint", "power", "admission")
RELEASE_WINDOW, RELEASE_FRAC = 2, 0.7
CATCH_UP = 0.8      # a site catches its held-over flexible load up wh

    ↪ ile its draw is under this share of its connection


def clamp(v, lo, hi):
    return lo if v < lo else hi if v > hi else v

def rho_of(d):
    """The engine's utilisation target as the live controller reads it
    ↪ ."""
    return clamp(float(d["demand"]), 0.5, 0.95)

def organ(auth, name):
    return auth["organs"].get(name, {}) if auth and auth.get("execute"
    ↪ ) else {}

def may_contract(auth, name, clean):
    return bool(organ(auth, name).get("contract")) and clean


def calm_of(auth):
    return auth["scalars"]["calm"] if auth and "scalars" in auth else
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ 0.0


def envelope_cap(auth, name, d, floor):
    """A power-type limit inside the organ's envelope, never under the
    ↪  floor (what the work draws plus headroom)."""
    lo, hi = organ(auth, name)["envelope"]
    return clamp(max(clamp(float(d["power_cap"]), lo, hi), floor), lo,
    ↪  1.0)


def continuous_capacity(x, load, rho, auth, organ_name, clean, lo):
    """Up at once to x * load / rho*; down only with contraction autho
    ↪ rity, by the calm share of the surplus."""
    req = x * clamp(load, 0.0, 2.0) / rho
    if req > x:
        return min(1.0, req)
    if may_contract(auth, organ_name, clean):
        return max(lo, x - calm_of(auth) * (x - req))
    return x


def ar_trace(rng, n, rho, sd):
    z, out = 0.0, []
    for _ in range(n):
        z = rho * z + rng.gauss(0.0, sd)
        out.append(z)
    return out
def _jit(f):
    """Compile a pure-number kernel with numba when it is installed (s
    ↪ ame arithmetic, in the same order); else plain."""
    try:
        import numba
        return numba.njit(cache=True)(f)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    except Exception:  # noqa: BLE001  numba missing or unable to comp
    ↪ ile: the plain function runs, identically
        return f


@_jit
def _motion_substeps(n, dt, theta, omega, integ, Tm, backlog, dwell, p
    ↪ os, mv_on, mv_t, mv_v, mv_a, mv_T, mv_start,
                     mv_sgn, s, effort, admit, tau_dk, v_max, a_max, D
    ↪ , i_max, kp, kd, ki, J, load_bias, tau_max,
                     t_lim, b, R, kt, p_idle, regen, t_amb, r_th, c_th
    ↪ , tol, dwell_p):
    """MotionAxis.step's inner loop, every fine step of one decision,
    ↪ as plain numbers (see MotionAxis.step)."""
    e = 0.0
    err_max = 0.0
    tau_peak = 0.0
    hot = False
    done = 0
    for _ in range(n):
        if not mv_on:
            if dwell > 0.0:
                dwell -= dt
            elif backlog > 0 and admit:
                mv_sgn = 1.0 if pos == 0.0 else -1.0
                mv_v = v_max * s
                mv_a = a_max * s
                mv_T = 2.0 * math.sqrt(D / mv_a) if D < mv_v * mv_v /
    ↪ mv_a else D / mv_v + mv_v / mv_a
                mv_t = 0.0
                mv_start = pos
                mv_on = True
        if mv_on:
            ta = mv_v / mv_a if D >= mv_v * mv_v / mv_a else math.sqrt
    ↪ (D / mv_a)
            vp = mv_a * ta
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            t = mv_t
            if t < ta:
                off, vr, ar = 0.5 * mv_a * t * t, mv_a * t, mv_a
            elif t < mv_T - ta:
                off, vr, ar = 0.5 * mv_a * ta * ta + vp * (t - ta), vp
    ↪ , 0.0
            elif t < mv_T:
                tr = mv_T - t
                off, vr, ar = D - 0.5 * mv_a * tr * tr, mv_a * tr, -mv
    ↪ _a
            else:
                off, vr, ar = D, 0.0, 0.0
            th_r = mv_start + mv_sgn * off
            w_r = mv_sgn * vr
            a_r = mv_sgn * ar
            mv_t += dt
        else:
            th_r, w_r, a_r = pos, 0.0, 0.0
        err = th_r - theta
        integ = integ + err * dt
        integ = -i_max if integ < -i_max else i_max if integ > i_max e
    ↪ lse integ
        tau = kp * err + kd * (w_r - omega) + ki * integ + J * a_r + l
    ↪ oad_bias
        lim = tau_max * effort
        if Tm > t_lim:
            lim *= 0.5
            hot = True
        tau = -lim if tau < -lim else lim if tau > lim else tau
        if abs(tau) > tau_peak:
            tau_peak = abs(tau)
        acc = (tau - b * omega - tau_dk) / J
        omega += acc * dt
        theta += omega * dt
        q = tau / kt
        i2r = R * (q * q)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        mech = tau * omega
        pw = p_idle + i2r + (mech if mech > 0 else regen * mech)
        e += pw * dt
        Tm += (i2r - (Tm - t_amb) / r_th) * dt / c_th
        if mv_on:
            if abs(err) > err_max:

                err_max = abs(err)
            if mv_t >= mv_T and abs(err) < tol and abs(omega) < 10 * t
    ↪ ol:
                pos = mv_start + mv_sgn * D
                mv_on = False
                dwell = dwell_p

                backlog -= 1
                done += 1
    return (theta, omega, integ, Tm, backlog, dwell, pos, mv_on, mv_t,
    ↪  mv_v, mv_a, mv_T, mv_start, mv_sgn, e, err_max,
            tau_peak, hot, done)

_SHARED: Dict[tuple, dict] = {}


def _shared(P: dict) -> dict:
    """One parameter table for every plant with the same parameters (t

    ↪ he copies of a muscle in a stack of 1,000): no
    plant writes to its table, so they share it."""
    try:
        key = tuple(sorted(P.items()))
        hash(key)
    except TypeError:

        return dict(P)
    return _SHARED.setdefault(key, dict(P))


def pack(p: "Plant") -> "Plant":
    """A plant made, packed for a large organism: its exogenous series
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  (arrivals, load, sun, the motion axes' tasks and
    disturbances) as machine arrays instead of lists of Python numbers
    ↪ , the same values in a quarter of the room, and
    its random source dropped (every plant draws its series when it is
    ↪  made, never while it runs)."""
    for name in ("lam", "load", "pv", "tau_d", "arrivals"):

        v = getattr(p, name, None)
        if isinstance(v, list):
            setattr(p, name, array("q" if all(type(x) is int for x in
    ↪ v) else "d", v))
    p.rng = None
    return p

class Plant:
    template = ""

    def __init__(self, P: dict, seed: int, steps: int):

        self.P = _shared(P)
        self.rng = random.Random(seed)
        self.steps = steps
        self.k = 0
        self.override: Dict[str, float] = {}
        self.ext: Dict[str, float] = {}

        self.m = {"work": 0.0, "energy_j": 0.0, "viol": 0, "steps": 0,
    ↪  "wear": 0.0}
        # wear (Omni v5, the max: shown beside the work, never judged)
    ↪ : what running harder costs the machine itself, in
        # the plant's own unit: a pool's machine-hours, a room's cooli
    ↪ ng unit-hours, a battery's kWh cycled through it,

        # an axis's motor heat (degree-hours above ambient), a loop's
    ↪ actuator duty-hours
        self.power_w = 0.0
        self.vhist = []
        self.paced = False          # an admission muscle's work suspe
    ↪ nded by batch pacing (set by the harness)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def record(self, violated):
        self.m["viol"] += int(violated)
        self.vhist = (self.vhist + [bool(violated)])[-3:]

    @property
    def slo_clean(self):
        """No violation in the last three decisions (the live controll
    ↪ er's --slo-clear 3)."""
        return not any(self.vhist)

    def omni_override(self, knob, d, g, auth):
        """The plant's override for this period from the directive d a
    ↪ nd the nervous system's authority; {} is native."""
        raise NotImplementedError

    def can_hold(self):
        """Whether batch pacing may suspend this plant's work now with
    ↪ out taking its service out of the line."""
        return True

    def fixed_calm(self):
        """The descriptive comparator for setpoint muscles: the setpoi
    ↪ nt fixed at its band's calm end, no governor."""
        return {"target" if self.template == "compute_pool" else "setp
    ↪ oint": self.P["calm"]}

    def thermal_obs(self, own):
        return self.ext.get("thermal", own)
    def finalize(self):
        pass


# ------------------------------------------------------------------------
    ↪ ----------------------------------------
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class ComputePool(Plant):
    template = "compute_pool"
    # Omni-Compass by itself (omni-v19, docs/SOLO_PREREGISTRATION.md):
    ↪  the pool's own autoscaler switched off, so the count
    # holds where it stands unless the brain's floor or ceiling moves
    ↪ it; set on a plant (and on its twin) by the arm solo only
    native_off = False

    def __init__(self, P, seed, steps):
        super().__init__(P, seed, steps)
        P, r = self.P, self.rng
        noise = ar_trace(r, steps, 0.9, P["noise"])
        phase = r.uniform(0.0, 2.0 * math.pi)
        self.lam, left, amp = [], 0, 0.0
        for k in range(steps):
            if left == 0 and r.random() < P["burst_p"]:
                left, amp = r.randint(4, 20), r.uniform(0.3, P["burst_
    ↪ max"])
            b = amp if left > 0 else 0.0
            left = max(0, left - 1)
            day = 1.0 + P["diurnal"] * math.sin(2.0 * math.pi * k / P[
    ↪ "period_steps"] + phase)
            self.lam.append(max(0.0, P["lam0"] * day * (1.0 + b) * mat
    ↪ h.exp(noise[k])))
        self.n = int(P["n0"])
        self.pending = []
        self.Q = 0.0
        self.util, self.W, self.drift, self.cap = 0.5, 0.0, 0.0, 1.0
        self.hist = []
        self.th = 0.32
        self.H = 0.0
        self.drop_share = 0.0
        self.unneeded = 0
        self.omni_t, self.omni_t_k = None, 0
        self.ahist = []
        self.B = 0.0                # work waiting in line (the 'line'
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  load, realms/harness.py: jobs already submitted
        self.line_k = None          # when the run starts); the machin
    ↪ es take it from what the requests leave them, and
                                    # line_k is the decision the line
    ↪ was cleared at
        self.nominal_w = P["lam0"] / (P["mu"] * P["target"]) * (P["p_i
    ↪ dle"] + P["target"] * P["p_dyn"])
        self.budget_w = 1.5 * self.nominal_w

    def mu(self, cap):
        return self.P["mu"] * cap ** 0.4          # dynamic power ~ f^
    ↪ 2.5, so throughput ~ (power share)^0.4

    def omni_override(self, knob, d, g, auth):
        P, clean, rho = self.P, self.slo_clean, rho_of(d)
        if not clean:                                              # b
    ↪ and first: outside the band, every knob is native
            self.omni_t = None
            return {}
        if knob == "power":
            org = P.get("power_organ")
            if not org or not P.get("pausable") or not may_contract(au
    ↪ th, org, clean):
                return {}                                          # a
    ↪  request-served pool never gives watts back
            proposed = envelope_cap(auth, org, d, min(1.0, self.cap *
    ↪ self.util * 1.3))
            if self.W > 0 and self.mu(proposed) < self.mu(1.0) * (self
    ↪ .W / max(P["slo_s"], 1e-9)):
                return {}                                          # t
    ↪ he cut would push the wait past the line
            return {"power": proposed}
        if knob == "admission":
            if not P.get("pausable"):
                return {}                                          # r
    ↪ equest traffic is never paused (live: pausable jobs only)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            return {"admission": 0.0} if self.paced else {}
        if P.get("ca"):                                          # a
    ↪  machine pool: the node release gate
            n = self.n
            ok = (may_contract(auth, "nodes", clean) and n > P["n_min"
    ↪ ] and not self.pending and self.Q <= 0.0
                  and len(self.ahist) == RELEASE_WINDOW * P["startup_s
    ↪ teps"]
                  and max(self.ahist) / ((n - 1) * self.mu(self.cap))
    ↪ <= RELEASE_FRAC * rho)   # the rest covers the recent peak
            return {"release": 1.0} if ok else {}
        # pods: the HPA target, never tighter than the operator's, hel
    ↪ d for one autoscaler window
        want = min(rho, P["target"])
        if self.omni_t is None or self.k - self.omni_t_k >= P["stab_st
    ↪ eps"]:
            self.omni_t, self.omni_t_k = want, self.k
        return {"target": self.omni_t}

    def can_hold(self):
        """Pacing may suspend this job only if one more period of held
    ↪  work stays inside half the line even after the
        autoscaler has shrunk the idle pool to its floor (a suspended
    ↪ job's pods are scaled in; on resume the held work

        lands on what is left)."""
        mu = self.mu(self.cap)
        held = self.Q + self.H + self.lam[self.k] * self.P["dt"]
        return held / (self.P["n_min"] * mu) + 1.0 / mu <= 0.5 * self.
    ↪ P["slo_s"]

    def observe(self):
        P = self.P
        pw = self.power_w / self.budget_w
        latency_pressure = max(0.0, self.W / P["slo_s"] - 1.0, self.dr
    ↪ op_share)
        return {"queue_ratio": min(2.0, max(len(self.pending) / max(1,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  self.n), latency_pressure)),
                "load_ratio": self.util, "power_stress": pw, "thermal"
    ↪ : self.thermal_obs(self.th),
                "network_stress": self.util if P.get("network") else 0
    ↪ .0, "drift_ratio": self.drift,
                "stale": 0.0, "security_block": 0.0}

    def hpa(self, util):
        target = self.override.get("target", self.P["target"])
        cur = self.n + len(self.pending)
        ratio = util / target
        rec = cur if abs(ratio - 1.0) <= 0.1 else math.ceil(self.n * r
    ↪ atio)
        self.hist.append(rec)
        self.hist = self.hist[-self.P["stab_steps"]:]
        return rec if rec >= cur else min(cur, max(self.hist))

    def cluster_autoscaler(self, util, mu):
        """Native machine pool: add machines while work is waiting; re
    ↪ move one after it has been unneeded (the rest
        under 50% utilised) for the unneeded time (Cluster Autoscaler
    ↪ defaults: 0.5, 10 minutes)."""
        P = self.P
        total = self.n + len(self.pending)
        if (self.Q > 0.0 or self.B > 0.0) and util >= 0.99:
            self.unneeded = 0
            return max(total, math.ceil(self.lam[self.k] / (mu * 0.9))
    ↪ )
        if self.n > P["n_min"] and not self.pending and util * self.n
    ↪ / (self.n - 1) < 0.5:
            self.unneeded += 1
        else:
            self.unneeded = 0
        if self.unneeded >= P["ca_unneeded_steps"]:
            self.unneeded = 0
            return total - 1
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return total

    def step(self):
        P, k, dt = self.P, self.k, self.P["dt"]
        self.n += sum(1 for s in self.pending if s <= k)
        self.pending = [s for s in self.pending if s > k]

        self.cap = cap = self.override.get("power", 1.0)
        mu = self.mu(cap)
        A = self.lam[k] * dt
        if self.override.get("admission", None) == 0.0:            # p
    ↪ aced: the job is suspended, its work held
            self.H += A

            A = 0.0
        elif self.H > 0.0:                                         # r
    ↪ unning: held work comes back into the pool's spare
            back = min(self.H, max(0.0, 0.85 * P["target"] * self.n *
    ↪ mu * dt - A))   # room, under the autoscaler band
            A += back                                              # (

    ↪ a resumed job continues at its parallelism)
            self.H -= back
        limit = P["queue_limit_s"] * self.n * mu
        C = self.n * mu * dt
        adm = min(A, max(0.0, C + limit - self.Q))                 # w
    ↪ hat this period serves, plus the queue limit

        drop = A - adm
        served = min(self.Q + adm, C)
        self.Q += adm - served
        took = 0.0
        if self.B > 0.0:                                           # t
    ↪ he line: what the requests leave the machines

            took = min(self.B, C - served)
            self.B -= took
            if self.B <= 1e-9:
                self.B, self.line_k = 0.0, k
        util = (served + took) / C if C > 0 else 1.0
        W = (self.Q + self.H) / max(self.n * mu, 1e-9) + 1.0 / mu
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        p = self.n * (P["p_idle"] + P["p_dyn"] * cap * util) + len(sel
    ↪ f.pending) * P["p_idle"]
        self.m["work"] += served + took
        self.m["energy_j"] += p * dt
        self.m["steps"] += 1
        self.m["wear"] += (self.n + len(self.pending)) * dt / 3600.0

        self.record(W > P["slo_s"] or drop > 1e-9)
        self.drop_share = drop / max(A, 1e-9) if A > 0 else 0.0
        self.power_w, self.util, self.W = p, util, W
        self.ahist = (self.ahist + [self.lam[k]])[-RELEASE_WINDOW * P[
    ↪ "startup_steps"]:]
        self.th = min(1.35, max(0.0, 0.86 * self.th + 0.14 * (0.34 + 0

    ↪ .62 * min(1.35, p / self.budget_w))))
        self.drift = abs(self.lam[k] - self.lam[k - 1]) / P["lam0"] if
    ↪  k else 0.0
        total = self.n + len(self.pending)
        if self.native_off:
            want = total                                            # b

    ↪ y itself: no autoscaler; the count holds
        else:
            want = self.cluster_autoscaler(util, mu) if P.get("ca") el
    ↪ se self.hpa(util)
        if self.override.get("release"):
            want = min(want, total - 1)                             # o

    ↪ ne machine per decision
        if "floor" in self.override:                                # a
    ↪  floor under the autoscaler: machines started
            want = max(want, int(self.override["floor"]))           # b
    ↪ efore the rush (it only ever adds)
        if "cap" in self.override:                                  # a

    ↪  ceiling over it (its maxReplicas): the
            want = min(want, int(self.override["cap"]))             # m
    ↪ achines handed back at once, to the count set
        want = int(clamp(want, P["n_min"], P["n_max"]))
        if want > total:
            self.pending += [k + P["startup_steps"]] * min(want - tota
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ l, max(4, total))
        elif want < total:
            cut = total - want
            while cut and self.pending:
                self.pending.pop(); cut -= 1
            self.n = max(P["n_min"], self.n - cut)

        self.k += 1


# ---------------------------------------------------------------------
    ↪ ------------------------------------------
class ThermalZone(Plant):

    template = "thermal_zone"

    def __init__(self, P, seed, steps):
        super().__init__(P, seed, steps)
        P, r = self.P, self.rng
        noise = ar_trace(r, steps, 0.95, P["noise"])

        ph = r.uniform(-0.5, 0.5)
        per = 86400.0 / P["dt"]
        self.qit = [P["q_it_w"] * (1.0 + P["it_amp"] * math.sin(2 * ma
    ↪ th.pi * (k / per + ph)) + noise[k])
                    for k in range(steps)]
        self.tout = [P["t_out"] + P["t_out_amp"] * math.sin(2 * math.p

    ↪ i * (k / per - 0.25 + P["start_frac"]))
                     + r.gauss(0, 0.3) for k in range(steps)]
        self.T = P["t_set"]
        self.integ = 0.0
        self.units_on = P["units"]
        self.qc_avg, self.qc_cmd_avg, self.drift = P["q_it_w"], P["q_i

    ↪ t_w"], 0.0
        cop = self.cop(P["t_set"], P["t_out"])
        self.nominal_w = P["q_it_w"] / cop + 0.6 * P["units"] * P["p_u
    ↪ nit_w"]
        self.full_w = P["units"] * P["q_unit_w"] / cop + P["units"] *
    ↪ P["p_unit_w"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def cop(self, tset, tout):
        tsup = tset - self.P["approach"]
        return clamp(self.P["eta"] * (tsup + 273.15) / max(3.0, tout +
↪  5.0 - tsup), 2.0, 10.0)

    def setpoint(self):
        return self.override.get("setpoint", self.P["t_set"])


    def omni_override(self, knob, d, g, auth):
        P, clean, rho = self.P, self.slo_clean, rho_of(d)
        if knob == "setpoint":

            # the live cooling law: warm setpoint while cool, cold as
↪ heat rises, under the nervous-system ceiling
            th = self.zone_thermal()
            x = clamp((th - 0.5) / 0.5, 0.0, 1.0)
            c = P["calm"] - (P["calm"] - P["stress"]) * x
            c = min(c, P["stress"] + (P["calm"] - P["stress"]) * calm_

↪ of(auth))
            if c > P["t_set"] and not (organ(auth, "cooling").get("con
↪ tract") and clean):
                c = P["t_set"]                                  # w
↪ arming gives cooling back: needs authority
            return {"setpoint": c}

        if knob == "capacity":                                  # c
↪ ooling units: the release gate
            u = self.units_on
            ok = (organ(auth, "cooling").get("contract") and clean and
↪  u > 1 and self.T <= self.setpoint()
                  and self.qc_cmd_avg / ((u - 1) * P["q_unit_w"]) <= r

↪ ho)
            return {"release": 1.0} if ok else {}
        if knob == "power":
            if not may_contract(auth, "power", clean):
                return {}
            floor = min(1.0, 1.1 * self.qc_cmd_avg / max(self.units_on
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  * P["q_unit_w"], 1e-9))
            return {"power": envelope_cap(auth, "power", d, floor)}
        return {"admission": 0.0} if self.paced else {}

    def observe(self):
        P = self.P

        base = P["t_set"] - 6.0
        th = clamp((self.T - base) / (P["t_limit"] - base), 0.0, 1.5)
        return {"queue_ratio": max(0.0, self.T - self.setpoint()) / ma
    ↪ x(P["t_limit"] - self.setpoint(), 0.5),
                "load_ratio": self.qc_avg / max(self.units_on * P["q_u
    ↪ nit_w"] * self.override.get("power", 1.0), 1e-9),

                "power_stress": self.power_w / self.full_w, "thermal":
    ↪  th, "network_stress": 0.0,
                "drift_ratio": self.drift, "stale": 0.0, "security_blo
    ↪ ck": 0.0}

    def zone_thermal(self):

        return self.observe()["thermal"]


    def step(self):
        P, k = self.P, self.k
        sub = P["sub"]
        dts = P["dt"] / sub

        tset = self.setpoint()
        cap = self.override.get("power", 1.0)
        shed = 0.9 if self.override.get("admission", 1.0) == 0.0 else
    ↪ 1.0
        q_own = max(0.0, self.qit[k]) * shed
        q_load = q_own + max(0.0, self.ext.get("heat_w", 0.0))

        qmax = self.units_on * P["q_unit_w"] * cap
        cop = self.cop(tset, self.tout[k])
        e = qc_sum = cmd_sum = 0.0
        hot = False
        for _ in range(sub):
            err = self.T - tset
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            cmd = q_load + P["kp"] * err + P["ki"] * self.integ
            qc = clamp(cmd, 0.0, qmax)
            if 0.0 < cmd < qmax or (cmd >= qmax and err < 0) or (cmd <
    ↪ = 0 and err > 0):
                self.integ += err * dts
            self.T += (q_load + P["ua"] * (self.tout[k] - self.T) - qc
    ↪ ) * dts / P["c_j_k"]
            p = qc / cop + self.units_on * P["p_unit_w"]
            e += p * dts
            qc_sum += qc
            cmd_sum += max(cmd, 0.0)
            if self.T > P["t_limit"]:
                hot = True
            else:
                self.m["work"] += q_own * dts
        self.m["energy_j"] += e
        self.m["steps"] += 1
        self.m["wear"] += self.units_on * P["dt"] / 3600.0

        self.record(hot)
        self.power_w = e / P["dt"]
        self.qc_avg, self.qc_cmd_avg = qc_sum / sub, cmd_sum / sub
        self.drift = abs(self.qit[k] - self.qit[k - 1]) / P["q_it_w"]
    ↪ if k else 0.0
        staged = int(clamp(math.ceil(self.qc_cmd_avg / (0.8 * P["q_uni
    ↪ t_w"]) - 1e-9), 1, P["units"]))
        if self.override.get("release"):
            staged = min(staged, self.units_on - 1)                # o
    ↪ ne unit per decision
        if "units_min" in self.override:                           # ah
    ↪ ead of the heat: units held on before the
            staged = max(staged, int(self.override["units_min"]))  # s
    ↪ taging asks for them (the knob both ways)
        self.units_on = int(clamp(staged, 1, P["units"]))
        self.k += 1
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# ------------------------------------------------------------------------
    ↪ ----------------------------------------
class EnergyStorage(Plant):
    template = "energy_storage"

    def __init__(self, P, seed, steps):
        super().__init__(P, seed, steps)
        P, r = self.P, self.rng
        noise = ar_trace(r, steps, 0.9, P["noise"])
        cloud = ar_trace(r, steps, 0.97, 0.08)
        self.load, self.pv = [], []
        for k in range(steps):
            h = (P["start_h"] + k * P["dt"] / 3600.0) % 24.0
            shape = 1.0 + P["load_amp"] * math.cos(2 * math.pi * (h -
    ↪ P["peak_h"]) / 24.0)
            self.load.append(max(0.0, P["load_w"] * shape * (1.0 + noi
    ↪ se[k])))
            sun = max(0.0, math.sin(math.pi * (h - 6.0) / 12.0)) if 6.
    ↪ 0 <= h <= 18.0 else 0.0
            self.pv.append(P["pv_w"] * sun * clamp(0.85 + cloud[k], 0.
    ↪ 2, 1.0))
        self.soc = self.soc0 = P["soc0"]
        self.deferred_j = 0.0
        self.line_e, self.line_k = 0.0, None    # flexible energy wait
    ↪ ing in line when the run starts (the 'line' load)
                                                # and the decision it
    ↪ was all served at
        self.imp, self.net, self.drift = 0.0, 0.0, 0.0
        self.E = P["e_wh"] * 3600.0
        self.nominal_w = max(P["load_w"] - 0.32 * P["pv_w"], 0.2 * P["
    ↪ load_w"])

    def omni_override(self, knob, d, g, auth):
        P, clean = self.P, self.slo_clean
        if knob == "setpoint":
            # reserve by power stress: released as the connection near
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ s its limit (more protection: always allowed);
            # held above native while calm only with contraction autho
    ↪ rity (it withholds battery energy)
            x = clamp((self.imp / P["p_lim_w"] - 0.5) / 0.5, 0.0, 1.0)
            r = P["calm"] - (P["calm"] - P["stress"]) * x
            if r > P["reserve"] and not may_contract(auth, "power", cl

    ↪ ean):
                r = P["reserve"]
            return {"setpoint": r}
        if knob == "capacity":                                      # t
    ↪ he ceiling the battery defends: rho* (expand only)
            return {"capacity": rho_of(d)}

        if knob == "power":
            if not may_contract(auth, "power", clean):
                return {}
            return {"power": envelope_cap(auth, "power", d, 0.0)}
        return {"admission": 0.0} if self.paced else {}

    def observe(self):
        P = self.P
        return {"queue_ratio": self.deferred_j / (P["p_lim_w"] * P["dt
    ↪ "] * 10.0),
                "load_ratio": max(0.0, self.net) / P["p_lim_w"], "powe
    ↪ r_stress": self.imp / P["p_lim_w"],

                "thermal": self.thermal_obs(0.3), "network_stress": 0.
    ↪ 0, "drift_ratio": self.drift,
                "stale": 0.0, "security_block": 0.0}


    def step(self):
        P, k, dt = self.P, self.k, self.P["dt"]

        ext = self.ext.get("load_w", 0.0)
        L_own = self.load[k]
        L = max(0.0, L_own + ext)
        pv = self.pv[k]
        if self.override.get("admission", None) == 0.0:             # p
    ↪ aced: defer the flexible share
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            defer = P["flex"] * L_own
            self.deferred_j += defer * dt
            L -= defer
            served_own = L_own - defer
            self.catch = 0.0
        else:                                                   # c
    ↪ atch up while the connection has room
            room = max(0.0, CATCH_UP * P["p_lim_w"] - (L - pv))
            s = min(self.deferred_j / dt, room)
            self.deferred_j -= s * dt
            L += s
            served_own = L_own + s

            self.catch = s                                      # (
    ↪ what this decision caught up, W)
            if self.line_e and self.line_k is None and self.deferred_j
    ↪  <= 1e-6:
                self.line_k = k
        net = L - pv

        reserve = self.override.get("setpoint", P["reserve"])
        pmax = P["p_batt_w"] * self.override.get("power", 1.0)
        sq = math.sqrt(P["eta_rt"])
        if net < 0.0:
            ch = min(-net, pmax, (1.0 - self.soc) * self.E / (dt * sq)
    ↪ )

            self.soc += ch * dt * sq / self.E
            imp = 0.0
            self.m["wear"] += ch * dt / 3.6e6
        else:
            d = min(net, pmax, max(0.0, self.soc - reserve) * self.E *
    ↪  sq / dt)

            if "capacity" in self.override:
                thr = self.override["capacity"] * P["p_lim_w"]
                low = P["stress"] if P.get("backup") else 0.0      # a
    ↪  UPS never below its outage floor
                d += max(0.0, min(net - d - thr, pmax - d, max(0.0, se
    ↪ lf.soc - low) * self.E * sq / dt - d))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            self.soc -= d * dt / (self.E * sq)
            imp = net - d
            self.m["wear"] += d * dt / 3.6e6
        self.m["work"] += served_own * dt
        self.m["energy_j"] += (imp - ext) * dt                    # t
    ↪ he external load is metered where it is drawn

        self.m["steps"] += 1
        self.record(imp > P["p_lim_w"])
        self.drift = abs(pv - self.pv[k - 1]) / max(P["pv_w"], 1.0) if
    ↪  k else 0.0
        self.imp, self.net, self.power_w = imp, net, imp - ext
        self.k += 1

    def finalize(self):
        # battery energy left below (or above) where it started is bou
    ↪ ght back (or credited) at the round trip
        self.m["energy_j"] += (self.soc0 - self.soc) * self.E / math.s
    ↪ qrt(self.P["eta_rt"])

# ---------------------------------------------------------------------
    ↪ ----------------------------------------
class MotionAxis(Plant):
    template = "motion_axis"

    def __init__(self, P, seed, steps):
        super().__init__(P, seed, steps)
        P, r = self.P, self.rng
        self.arrivals, self.tau_d = [], []
        rate_noise = ar_trace(r, steps, 0.95, 0.15)

        td = 0.0
        for k in range(steps):
            lam = P["task_rate"] * math.exp(rate_noise[k]) * P["dt_dec
    ↪ "]
            # Poisson count by inversion
            n, p, u = 0, math.exp(-lam), r.random()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            c = p
            while u > c and n < 50:
                n += 1; p *= lam / n; c += p
            self.arrivals.append(n)
            if r.random() < 0.1:
                td = r.uniform(-P["dist"], P["dist"])

            self.tau_d.append(P["load_bias"] + td)
        self.theta = self.omega = self.integ = 0.0
        self.Tm = P["t_amb"]
        self.backlog, self.waits = 0, []
        self.line_n = self.line_done = 0    # tasks waiting in line wh
    ↪ en the run starts (the 'line' load), how many of

        self.line_k = None                  # them are done, and the d
    ↪ ecision the last one was finished at
        self.move = None
        self.dwell = 0.0
        self.pos = 0.0
        self.s = 1.0

        self.demand_rate = P["task_rate"]
        self.err_max = self.drift = 0.0
        self.tau_peak = 0.0
        self.rated_w = P["p_idle"] + P["R"] * (P["tau_max"] / P["kt"])
    ↪  ** 2 * 0.3
        self.nominal_w = self.rated_w

    def can_hold(self):
        """Pacing may hold new moves only while the oldest waiting tas
    ↪ k stays two decisions inside its deadline."""
        return not self.waits or self.waits[0] + 2.0 * self.P["dt_dec"
    ↪ ] <= self.P["deadline_s"]

    def move_time(self, s):
        v, a, D = self.P["v_max"] * s, self.P["a_max"] * s, self.P["D"
    ↪ ]
        return 2.0 * math.sqrt(D / a) if D < v * v / a else D / v + v
    ↪ / a
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def omni_override(self, knob, d, g, auth):
        P, clean = self.P, self.slo_clean
        if knob == "admission":
            return {"admission": 0.0} if self.paced else {}
        if not clean:
            return {}                                                # S
    ↪ LO reflex: full speed and effort at once
        if knob == "power":
            if not may_contract(auth, "gpu", clean):
                return {}
            return {"power": envelope_cap(auth, "gpu", d, min(1.0, 1.3
    ↪  * self.tau_peak / P["tau_max"]))}
        load = self.demand_rate * (self.move_time(self.s) + P["dwell"]
    ↪ )
        return {"capacity": continuous_capacity(self.s, load, rho_of(d
    ↪ ), auth, "pods", clean, 0.4)}

    def observe(self):
        P = self.P
        return {"queue_ratio": self.backlog / max(P["task_rate"] * 30.
    ↪ 0, 1.0),
                "load_ratio": self.demand_rate * (self.move_time(self.
    ↪ s) + P["dwell"]),
                "power_stress": self.power_w / self.rated_w,
                "thermal": self.thermal_obs(clamp((self.Tm - P["t_amb"
    ↪ ]) / (P["t_lim"] - P["t_amb"]), 0.0, 1.5)),
                "network_stress": 0.0, "drift_ratio": self.drift, "sta
    ↪ le": 0.0, "security_block": 0.0}

    def ref(self, t):
        """Trapezoid (or triangle) reference from the move's start: po
    ↪ sition offset, velocity, acceleration."""
        mv = self.move
        v, a, D, T = mv["v"], mv["a"], self.P["D"], mv["T"]
        ta = v / a if D >= v * v / a else math.sqrt(D / a)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        vp = a * ta
        if t < ta:
            return 0.5 * a * t * t, a * t, a
        if t < T - ta:
            return 0.5 * a * ta * ta + vp * (t - ta), vp, 0.0
        if t < T:

            tr = T - t
            return D - 0.5 * a * tr * tr, a * tr, -a
        return D, 0.0, 0.0

    def step(self):
        P, k = self.P, self.k

        dt = P["dt"]
        n = int(round(P["dt_dec"] / dt))
        self.backlog += self.arrivals[k]
        self.waits += [0.0] * self.arrivals[k]
        self.demand_rate = 0.9 * self.demand_rate + 0.1 * self.arrival
↪ s[k] / P["dt_dec"]

        self.s = self.override.get("capacity", 1.0)
        effort = self.override.get("power", 1.0)
        admit = self.override.get("admission", 1.0) >= 0.5
        mv = self.move
        out = _motion_substeps(
            n, dt, self.theta, self.omega, self.integ, self.Tm, self.b

↪ acklog, self.dwell, self.pos, mv is not None,
            mv["t"] if mv else 0.0, mv["v"] if mv else 0.0, mv["a"] if
↪  mv else 0.0, mv["T"] if mv else 0.0,
            mv["start"] if mv else 0.0, mv["sgn"] if mv else 0.0, self
↪ .s, effort, admit, self.tau_d[k],
            P["v_max"], P["a_max"], P["D"], P["i_max"], P["kp"], P["kd

↪ "], P["ki"], P["J"], P["load_bias"], P["tau_max"],
            P["t_lim"], P["b"], P["R"], P["kt"], P["p_idle"], P["regen
↪ "], P["t_amb"], P["r_th"], P["c_th"], P["tol"],
            P["dwell"])
        (self.theta, self.omega, self.integ, self.Tm, self.backlog, se
↪ lf.dwell, self.pos, mv_on, mv_t, mv_v, mv_a, mv_T,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
         mv_start, mv_sgn, e, err_max, tau_peak, hot, done) = out
        self.move = {"t": mv_t, "v": mv_v, "a": mv_a, "T": mv_T, "star
    ↪ t": mv_start, "sgn": mv_sgn} if mv_on else None
        del self.waits[:done]                                       # t
    ↪ he finished moves are the oldest waiting tasks
        if self.line_n and self.line_k is None:                     # (

    ↪ the line's tasks are the oldest of all)
            self.line_done += done
            if self.line_done >= self.line_n:
                self.line_k = k
        self.waits = [w + P["dt_dec"] for w in self.waits]
        late = bool(self.waits) and self.waits[0] > P["deadline_s"]

        self.m["work"] += done
        self.m["energy_j"] += e
        self.m["steps"] += 1
        self.m["wear"] += max(0.0, self.Tm - P["t_amb"]) * P["dt_dec"]
    ↪  / 3600.0
        self.record(err_max > P["e_max"] or hot or late)

        self.tau_peak, self.err_max = tau_peak, err_max
        self.power_w = e / P["dt_dec"]
        self.drift = abs(self.tau_d[k] - self.tau_d[k - 1]) / P["tau_m
    ↪ ax"] if k else 0.0
        self.k += 1

    def theta_ref_end(self):
        return self.pos


# ---------------------------------------------------------------------
    ↪ -----------------------------------------

class ProcessLoop(Plant):
    template = "process_loop"

    def __init__(self, P, seed, steps):
        super().__init__(P, seed, steps)
        P, r = self.P, self.rng
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        noise = ar_trace(r, steps, 0.9, P["noise"])
        per = 86400.0 / P["dt"]
        ph = r.uniform(0.0, 1.0)
        self.d = [max(0.0, P["d_mean"] * (1.0 + P["d_amp"] * math.sin(
    ↪ 2 * math.pi * (k / per + ph))) + noise[k])
                  for k in range(steps)]

        self.y = P["sp"]
        self.integ = (P["sp"] + P["d_mean"]) / P["K"] / P["ki"] if P["
    ↪ ki"] else 0.0
        nd = max(1, int(round(P["theta"] / (P["dt"] / P["sub"]))))
        self.delay = [(P["sp"] + P["d_mean"]) / P["K"]] * nd
        self.s = 1.0

        self.umax_eff = P["u_max"]
        self.u_avg, self.dev, self.drift = (P["sp"] + P["d_mean"]) / P
    ↪ ["K"], 0.0, 0.0
        self.dev_dn = self.dev_up = 0.0     # the historian's low and
    ↪ high over the decision: how far below and above its
                                            # setpoint the loop went,

    ↪ in half-bands (read by the v5 law; native never reads it)
        self.nominal_w = P["p_max_w"] * (((P["sp"] + P["d_mean"]) / P[
    ↪ "K"]) / P["u_max"]) ** P["aff"]

    def setpoint(self):
        return self.override.get("setpoint", self.P["sp"])

    def omni_override(self, knob, d, g, auth):
        P, clean = self.P, self.slo_clean
        if knob == "admission":
            return {"admission": 0.0} if self.paced else {}
        if not clean:

            return {}                                               # S
    ↪ LO reflex: native at once
        if knob == "setpoint":
            if not may_contract(auth, "power", clean):
                return {}
            sp = self.setpoint()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            return {"setpoint": sp + calm_of(auth) * (P["calm"] - sp)}
        if knob == "power":
            if not may_contract(auth, "power", clean):
                return {}
            return {"power": envelope_cap(auth, "power", d, min(1.0, 1
↪ .1 * (self.u_avg / P["u_max"]) ** P["aff"]))}

        return {"capacity": continuous_capacity(self.s, self.u_avg / s
↪ elf.umax_eff, rho_of(d), auth, "pods", clean,
                                                0.3)}


    def observe(self):
        P = self.P

        return {"queue_ratio": self.dev, "load_ratio": self.u_avg / se
↪ lf.umax_eff,
                "power_stress": self.power_w / P["p_max_w"], "thermal"
↪ : self.thermal_obs(0.3),
                "network_stress": 0.0, "drift_ratio": self.drift, "sta
↪ le": 0.0, "security_block": 0.0}

    def step(self):
        P, k = self.P, self.k
        sub = P["sub"]
        dts = P["dt"] / sub
        sp = self.setpoint()

        self.s = self.override.get("capacity", 1.0)
        umax = self.umax_eff = P["u_max"] * self.s * self.override.get
↪ ("power", 1.0) ** (1.0 / P["aff"])
        frac = 0.9 if self.override.get("admission", 1.0) < 0.5 else 1
↪ .0
        dk = self.d[k] * frac

        e = u_sum = dev = dn = up = 0.0
        out = False
        for _ in range(sub):
            err = sp - self.y
            u_raw = P["kp"] * err + P["ki"] * self.integ
            u = clamp(u_raw, 0.0, umax)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            if 0.0 < u_raw < umax or (u_raw >= umax and err < 0) or (u
    ↪ _raw <= 0 and err > 0):
                self.integ += err * dts
            self.delay.append(u)
            ud = self.delay.pop(0)
            self.y += (-self.y + P["K"] * ud - dk) * dts / P["tau"]

            e += P["p_max_w"] * (u / P["u_max"]) ** P["aff"] * dts
            u_sum += u
            dv = abs(self.y - P["mid"]) / P["half"]
            dev = max(dev, abs(self.y - sp) / P["half"])
            dn = max(dn, (sp - self.y) / P["half"])
            up = max(up, (self.y - sp) / P["half"])

            if dv > 1.0:
                out = True
            else:
                self.m["work"] += dk * dts
        self.m["energy_j"] += e
        self.m["steps"] += 1

        self.m["wear"] += (u_sum / sub) / P["u_max"] * P["dt"] / 3600.
    ↪ 0
        self.record(out)
        self.power_w = e / P["dt"]
        self.u_avg, self.dev = u_sum / sub, dev
        self.dev_dn, self.dev_up = dn, up

        self.drift = abs(self.d[k] - self.d[k - 1]) / max(P["d_mean"],
    ↪  1e-9) if k else 0.0
        self.k += 1


TEMPLATES = {c.template: c for c in (ComputePool, ThermalZone, EnergyS

    ↪ torage, MotionAxis, ProcessLoop)}
```

H.47 `realms/presets.py`

198 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Parameter sets of the realm plants, fixed before any confirmation r
    ↪ un (docs/REALMS_PREREGISTRATION.md).

One set per preset named in realms/catalog.csv. Each muscle runs its f
    ↪ amily's set at its own size: the work scale
(request rate, IT load, site load, process demand) is multiplied by 0.
    ↪ 6 to 1.4, drawn from the muscle id, so the 16
muscles of a family are 16 differently sized plants. Values are round
    ↪ engineering figures for the class of machine,
not measurements of any product.

Two clocks:
  single     each plant on its own natural decision period and horizon
    ↪  (STEPS_SINGLE)
  organism   every plant on one shared 15 s decision clock for 240 ste
    ↪ ps (one hour), so they can be coupled

"""
from __future__ import annotations

import hashlib

STEPS_SINGLE = {"compute_pool": 720, "thermal_zone": 720, "energy_stor
    ↪ age": 720, "motion_axis": 360,
                "process_loop": 720}
ORGANISM_DT = 15.0
ORGANISM_STEPS = 240
CAL_SEED = 999          # the calibration seed: sets declared power bu
    ↪ dgets from native draw; never a result seed
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
_CP = dict(dt=15.0, mu=50.0, slo_s=0.5, startup_steps=4, n0=6, n_min=2
    ↪ , n_max=80, p_idle=60.0, p_dyn=140.0,
           target=0.70, lam0=200.0, diurnal=0.35, period_steps=720, no
    ↪ ise=0.08, burst_p=0.01, burst_max=1.0,
           queue_limit_s=30.0, stab_steps=20, calm=0.914, stress=0.627

    ↪ )

PRESETS = {
    # compute_pool ----------------------------------------------------
    ↪ ---------------------------------------
    "server": dict(_CP),

    "node": dict(_CP, ca=1, ca_unneeded_steps=40, mu=80.0, slo_s=1.0,
    ↪ startup_steps=12, p_idle=120.0, p_dyn=230.0, lam0=320.0),
    "cpu_host": dict(_CP, power_organ="cpufreq", mu=60.0, p_idle=80.0,
    ↪  p_dyn=170.0, lam0=240.0),
    "gpu": dict(_CP, power_organ="gpu", mu=8.0, slo_s=2.0, startup_ste
    ↪ ps=8, p_idle=60.0, p_dyn=240.0, lam0=40.0, n0=6),

    "gpu_batch": dict(_CP, power_organ="gpu", pausable=1, mu=8.0, slo_
    ↪ s=600.0, startup_steps=8, p_idle=60.0, p_dyn=240.0, lam0=40.0, d
    ↪ iurnal=0.1,
                      burst_p=0.005, queue_limit_s=3600.0),
    "batch": dict(_CP, pausable=1, mu=20.0, slo_s=300.0, startup_steps
    ↪ =8, lam0=80.0, diurnal=0.2, queue_limit_s=3600.0),

    "fabric": dict(_CP, mu=100.0, slo_s=0.2, startup_steps=1, p_idle=2
    ↪ 0.0, p_dyn=30.0, lam0=400.0, network=1),
    "robot_fleet": dict(_CP, mu=0.05, slo_s=120.0, startup_steps=2, n0
    ↪ =8, n_max=60, p_idle=50.0, p_dyn=300.0,
                        lam0=1.0, queue_limit_s=1800.0),
    "qpu": dict(_CP, pausable=1, mu=2.0, slo_s=60.0, startup_steps=20,

    ↪  n0=3, n_min=1, n_max=12, p_idle=600.0, p_dyn=200.0,
                lam0=6.0, queue_limit_s=1800.0),
    "network": dict(_CP, mu=1000.0, slo_s=0.05, startup_steps=2, p_idl
    ↪ e=15.0, p_dyn=10.0, lam0=4000.0, network=1),
    "storage": dict(_CP, mu=200.0, slo_s=0.05, startup_steps=2, p_idle
    ↪ =8.0, p_dyn=4.0, lam0=800.0),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    "database": dict(_CP, mu=30.0, slo_s=0.2, startup_steps=20, n0=4,
    ↪ p_idle=100.0, p_dyn=150.0, lam0=120.0),
    "commerce": dict(_CP, slo_s=1.0, startup_steps=6, burst_p=0.02, bu
    ↪ rst_max=1.5),
    "workflow": dict(_CP, pausable=1, mu=0.5, slo_s=60.0, lam0=4.0, qu
    ↪ eue_limit_s=1800.0),

    "ran": dict(_CP, mu=100.0, slo_s=0.05, startup_steps=4, n0=6, n_mi
    ↪ n=2, n_max=12, p_idle=300.0, p_dyn=500.0,
                lam0=350.0, diurnal=0.6, network=1),
    # thermal_zone ------------------------------------------------
    ↪ ----------------------------------------
    "data_hall": dict(dt=60.0, sub=12, c_j_k=2.0e6, ua=500.0, q_it_w=1

    ↪ 50e3, it_amp=0.15, noise=0.03, units=6,
                     q_unit_w=40e3, p_unit_w=1.5e3, t_set=24.0, t_lim
    ↪ it=27.0, calm=26.0, stress=22.0, t_out=25.0,
                     t_out_amp=7.0, start_frac=0.25, eta=0.45, approa
    ↪ ch=8.0, kp=40e3, ki=200.0),
    "building": dict(dt=60.0, sub=12, c_j_k=8.0e6, ua=2000.0, q_it_w=6

    ↪ 0e3, it_amp=0.4, noise=0.05, units=4,
                    q_unit_w=35e3, p_unit_w=0.8e3, t_set=22.0, t_limi
    ↪ t=25.0, calm=24.0, stress=21.0, t_out=30.0,
                    t_out_amp=6.0, start_frac=0.25, eta=0.40, approac
    ↪ h=10.0, kp=15e3, ki=60.0),
    # energy_storage ----------------------------------------------

    ↪ ----------------------------------------
    "microgrid": dict(dt=120.0, e_wh=400e3, p_batt_w=100e3, eta_rt=0.9
    ↪ 0, load_w=120e3, load_amp=0.4, peak_h=19.0,
                     pv_w=150e3, p_lim_w=140e3, reserve=0.2, calm=0.5
    ↪ , stress=0.1, flex=0.15, soc0=0.5,
                     start_h=0.0, noise=0.05),

    "ups": dict(dt=120.0, e_wh=50e3, p_batt_w=200e3, eta_rt=0.92, load
    ↪ _w=150e3, load_amp=0.1, peak_h=15.0,
                pv_w=0.0, p_lim_w=180e3, reserve=0.6, calm=0.8, stress
    ↪ =0.5, flex=0.05, soc0=0.9, start_h=0.0,
                noise=0.03, backup=True),   # a UPS reserve is held fo
    ↪ r an outage, not for a peak: never Omni's lever
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    "facility": dict(dt=120.0, e_wh=1000e3, p_batt_w=300e3, eta_rt=0.9
    ↪ 0, load_w=400e3, load_amp=0.25, peak_h=17.0,
                     pv_w=200e3, p_lim_w=450e3, reserve=0.2, calm=0.5,
    ↪  stress=0.1, flex=0.10, soc0=0.5,
                     start_h=0.0, noise=0.04),
    # motion_axis ----------------------------------------------------

    ↪ ----------------------------------------
    # s_peak (Omni v5, the max): the axis's own peak speed against the
    ↪  everyday setting native runs (v_max, a_max are
    # the everyday profile), the head the machine itself has above it.
    ↪  A joint's programmed speed and a multirotor's
    # default speed sit below what the drive and the airframe can do (

    ↪ 1.25); a road vehicle's everyday acceleration below
    # its drive's peak (1.25); a ship's service speed below its engine
    ↪ 's maximum continuous rating (1.15). None where the
    # everyday setting is the limit itself: a reaction wheel's saturat
    ↪ ion, a train's line speed, an elevator's rated
    # speed under its overspeed governor (s_peak absent: 1.0, native a

    ↪ lready at the limit). Native never reads it.
    "robot_joint": dict(dt_dec=1.0, dt=0.004, J=0.05, b=0.02, kt=0.5,
    ↪ R=0.8, tau_max=12.0, p_idle=20.0, D=1.5,
                        v_max=3.0, a_max=15.0, dwell=0.2, tol=0.01, e_
    ↪ max=0.05, c_th=300.0, r_th=1.2, t_amb=30.0,
                        t_lim=90.0, kp=400.0, kd=8.0, ki=100.0, i_max=

    ↪ 0.05, task_rate=0.6, dist=2.0,
                        load_bias=0.0, regen=0.0, deadline_s=30.0, s_p
    ↪ eak=1.25),
    "flight_axis": dict(dt_dec=2.0, dt=0.01, J=2.0, b=0.5, kt=0.53, R=
    ↪ 0.15, tau_max=40.0, p_idle=30.0, D=10.0,
                        v_max=3.0, a_max=2.0, dwell=2.0, tol=0.1, e_ma

    ↪ x=0.5, c_th=400.0, r_th=0.2, t_amb=25.0,
                        t_lim=80.0, kp=30.0, kd=12.0, ki=10.0, i_max=0
    ↪ .5, task_rate=0.05, dist=4.0,
                        load_bias=19.6, regen=0.0, deadline_s=120.0, s
    ↪ _peak=1.25),
    "reaction_wheel": dict(dt_dec=10.0, dt=0.02, J=50.0, b=0.01, kt=0.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ 05, R=2.0, tau_max=0.2, p_idle=5.0, D=0.5,
                          v_max=0.02, a_max=0.004, dwell=5.0, tol=0.0
    ↪ 02, e_max=0.01, c_th=100.0, r_th=5.0,
                          t_amb=20.0, t_lim=70.0, kp=12.5, kd=40.0, k
    ↪ i=0.5, i_max=0.01, task_rate=0.01,
                          dist=0.002, load_bias=0.0, regen=0.0, deadl
    ↪ ine_s=600.0),
    "ev_traction": dict(dt_dec=5.0, dt=0.02, J=1600.0, b=25.0, kt=40.0
    ↪ , R=0.05, tau_max=6000.0, p_idle=300.0,
                        D=400.0, v_max=14.0, a_max=2.0, dwell=10.0, to
    ↪ l=1.0, e_max=3.0, c_th=5000.0, r_th=0.05,
                        t_amb=25.0, t_lim=120.0, kp=800.0, kd=1800.0,
    ↪ ki=20.0, i_max=5.0, task_rate=1.0 / 60.0,
                        dist=300.0, load_bias=0.0, regen=0.6, deadline
    ↪ _s=300.0, s_peak=1.25),
    # process_loop ----------------------------------------------------
    ↪ ----------------------------------------
    "process": dict(dt=10.0, sub=10, tau=120.0, K=1.0, theta=5.0, sp=0
    ↪ .6, mid=0.6, half=0.2, calm=0.5, stress=0.6,
                    u_max=2.0, aff=3.0, p_max_w=30e3, kp=2.0, ki=0.02,
    ↪  d_mean=0.3, d_amp=0.3, noise=0.02),
    "water": dict(dt=10.0, sub=10, tau=30.0, K=1.0, theta=3.0, sp=1.0,
    ↪  mid=1.0, half=0.15, calm=0.92, stress=1.0,
                  u_max=3.0, aff=3.0, p_max_w=75e3, kp=1.5, ki=0.05, d
    ↪ _mean=0.6, d_amp=0.4, noise=0.03),
    "chamber": dict(dt=10.0, sub=10, tau=300.0, K=1.0, theta=10.0, sp=
    ↪ 1.0, mid=1.0, half=0.02, calm=0.995,
                    stress=1.0, u_max=3.0, aff=1.0, p_max_w=20e3, kp=6
    ↪ .0, ki=0.02, d_mean=0.5, d_amp=0.05,
                    noise=0.002),
    "feeder_voltage": dict(dt=10.0, sub=10, tau=5.0, K=1.0, theta=1.0,
    ↪  sp=1.0, mid=1.0, half=0.05, calm=0.97,
                          stress=1.0, u_max=2.0, aff=1.0, p_max_w=500
    ↪ e3, kp=1.0, ki=0.5, d_mean=0.1, d_amp=0.5,
                          noise=0.005),
    # Omni v2, wave 4 of the realm study (docs/REALMS_PREREGISTRATION.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ md): thirteen new families, one preset each. Round
    # engineering figures for the class of machine, as above; none is
    ↪ a measurement of any product.
    # thermal_zone: a hospital's critical rooms (tighter band, smaller
    ↪  plant than a data hall)
    "hospital": dict(dt=60.0, sub=12, c_j_k=6.0e6, ua=1500.0, q_it_w=8

    ↪ 0e3, it_amp=0.25, noise=0.04, units=4,
                     q_unit_w=40e3, p_unit_w=1.0e3, t_set=21.0, t_limi
    ↪ t=24.0, calm=22.5, stress=20.5, t_out=28.0,
                     t_out_amp=6.0, start_frac=0.25, eta=0.40, approac
    ↪ h=10.0, kp=20e3, ki=80.0),
    # compute_pool: clinical systems (imaging archives, records, inter

    ↪ face engines: a strict line, a daytime swing)
    "clinical": dict(_CP, mu=40.0, slo_s=1.0, startup_steps=6, n0=4, p
    ↪ _idle=80.0, p_dyn=150.0, lam0=120.0, diurnal=0.5,
                     burst_p=0.015, burst_max=1.2),
    # compute_pool: a container terminal's cranes and vehicles taking
    ↪ moves from a queue (a move is a job; pausable)

    "port": dict(_CP, pausable=1, mu=0.02, slo_s=1800.0, startup_steps
    ↪ =2, n0=8, n_min=2, n_max=40, p_idle=15e3,
                 p_dyn=150e3, lam0=0.1, diurnal=0.3, burst_p=0.01, bur
    ↪ st_max=1.0, queue_limit_s=7200.0),
    # process_loop: pumps and pressure in a field or a greenhouse (a d
    ↪ aily cycle, a fast loop)

    "irrigation": dict(dt=10.0, sub=10, tau=60.0, K=1.0, theta=5.0, sp
    ↪ =1.0, mid=1.0, half=0.2, calm=0.9, stress=1.0,
                       u_max=3.0, aff=3.0, p_max_w=90e3, kp=1.2, ki=0.
    ↪ 03, d_mean=0.5, d_amp=0.5, noise=0.03),
    # process_loop: a pipeline segment (line pack is slow, compressor
    ↪ power goes as the cube)

    "pipeline": dict(dt=30.0, sub=10, tau=600.0, K=1.0, theta=30.0, sp
    ↪ =1.0, mid=1.0, half=0.1, calm=0.95, stress=1.0,
                     u_max=2.5, aff=3.0, p_max_w=5e6, kp=1.5, ki=0.005
    ↪ , d_mean=0.5, d_amp=0.3, noise=0.02),
    # process_loop: a grinding and flotation circuit (mill power goes
    ↪ roughly as the square of the drive setting)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    "mill": dict(dt=10.0, sub=10, tau=120.0, K=1.0, theta=10.0, sp=1.0
    ↪ , mid=1.0, half=0.15, calm=0.93, stress=1.0,
                 u_max=2.0, aff=2.0, p_max_w=10e6, kp=2.0, ki=0.02, d_
    ↪ mean=0.5, d_amp=0.25, noise=0.03),
    # process_loop: a district heating network (half an hour of therma
    ↪ l inertia, a strong daily swing)

    "district_heat": dict(dt=60.0, sub=10, tau=1800.0, K=1.0, theta=12
    ↪ 0.0, sp=1.0, mid=1.0, half=0.15, calm=0.9,
                          stress=1.0, u_max=2.0, aff=2.0, p_max_w=20e6
    ↪ , kp=1.0, ki=0.001, d_mean=0.5, d_amp=0.6,
                          noise=0.02),
    # process_loop: a generating unit's steam, speed or gate loop (fas

    ↪ t, tight band, near-linear fuel-to-power)
    "turbine": dict(dt=5.0, sub=10, tau=20.0, K=1.0, theta=2.0, sp=1.0
    ↪ , mid=1.0, half=0.05, calm=0.97, stress=1.0,
                    u_max=2.0, aff=1.2, p_max_w=300e6, kp=3.0, ki=0.2,
    ↪  d_mean=0.4, d_amp=0.3, noise=0.01),
    # process_loop: a plant of grid-support inverters or turbines (ver

    ↪ y fast, the resource itself the disturbance)
    "inverter": dict(dt=5.0, sub=10, tau=2.0, K=1.0, theta=0.2, sp=1.0
    ↪ , mid=1.0, half=0.05, calm=0.97, stress=1.0,
                     u_max=1.5, aff=1.0, p_max_w=5e6, kp=2.0, ki=1.0,
    ↪ d_mean=0.3, d_amp=0.5, noise=0.02),
    # process_loop: a GMP batch reactor, fermenter or food process (sl

    ↪ ow, very tight band, small disturbance)
    "batch_reactor": dict(dt=10.0, sub=10, tau=600.0, K=1.0, theta=20.
    ↪ 0, sp=1.0, mid=1.0, half=0.05, calm=0.98,
                          stress=1.0, u_max=3.0, aff=1.0, p_max_w=200e
    ↪ 3, kp=4.0, ki=0.01, d_mean=0.5, d_amp=0.1,
                          noise=0.005),

    # motion_axis: a train's traction axis (the EV axis scaled tenfold
    ↪  in inertia, torque and gains; the same dynamics)
    "rail_traction": dict(dt_dec=5.0, dt=0.02, J=16000.0, b=250.0, kt=
    ↪ 400.0, R=0.005, tau_max=60000.0, p_idle=3000.0,
                          D=2500.0, v_max=12.0, a_max=1.0, dwell=20.0,
    ↪  tol=2.0, e_max=5.0, c_th=50000.0, r_th=0.01,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                           t_amb=25.0, t_lim=150.0, kp=8000.0, kd=18000
    ↪ .0, ki=50.0, i_max=10.0, task_rate=1.0 / 300.0,
                           dist=3000.0, load_bias=0.0, regen=0.7, deadl
    ↪ ine_s=600.0),
    # motion_axis: a ship's propulsion shaft (large inertia, heavy dra
    ↪ g, a standing hull load)

    # (Omni v3: twelve speed changes an hour of 200 rad instead of fou
    ↪ r of 600, so a one-hour run holds enough moves for a
    # count; a move of 126 s plus 30 s of dwell at one every 300 s is
    ↪ a duty of 0.52)
    "marine_propulsion": dict(dt_dec=10.0, dt=0.05, J=50000.0, b=3000.
    ↪ 0, kt=2000.0, R=0.005, tau_max=500000.0,

                              p_idle=50000.0, D=200.0, v_max=10.0, a_m
    ↪ ax=0.05, dwell=30.0, tol=2.0, e_max=5.0,
                              c_th=50000.0, r_th=0.002, t_amb=30.0, t_
    ↪ lim=120.0, kp=25000.0, kd=56000.0, ki=100.0,
                              i_max=50.0, task_rate=1.0 / 300.0, dist=
    ↪ 50000.0, load_bias=10000.0, regen=0.0,

                              deadline_s=1200.0, s_peak=1.15),
    # motion_axis: an elevator hoist (a stiff position loop, a counter
    ↪ weighted load, regeneration on the way down)
    "elevator_hoist": dict(dt_dec=1.0, dt=0.005, J=20.0, b=2.0, kt=8.0
    ↪ , R=0.3, tau_max=2500.0, p_idle=300.0, D=60.0,
                           v_max=8.0, a_max=1.5, dwell=8.0, tol=0.02,

    ↪ e_max=0.2, c_th=2000.0, r_th=0.3, t_amb=30.0,
                           t_lim=110.0, kp=3000.0, kd=500.0, ki=50.0,
    ↪ i_max=1.0, task_rate=1.0 / 40.0, dist=100.0,
                           load_bias=0.0, regen=0.5, deadline_s=120.0)
    ↪ ,
}

# the parameter that sets each plant's size
SCALE_KEY = {"compute_pool": ("lam0",), "thermal_zone": ("q_it_w", "q_
    ↪ unit_w", "c_j_k", "ua", "kp", "ki", "p_unit_w"), "energy_storage
    ↪ ": ("load_w", "pv_w"),
             "motion_axis": ("task_rate",), "process_loop": ()}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def muscle_scale(muscle_id: str) -> float:
    h = int(hashlib.sha256(muscle_id.encode()).hexdigest()[:8], 16) /
    ↪ 0xFFFFFFFF
    return 0.6 + 0.8 * h

def params_for(row: dict, organism: bool = False) -> dict:
    """The parameter set one muscle's plant runs with."""
    P = dict(PRESETS[row["preset"]])
    s = muscle_scale(row["muscle_id"])

    for key in SCALE_KEY[row["template"]]:
        P[key] = P[key] * s
    if row["template"] == "compute_pool":
        P["n0"] = max(P["n_min"], int(round(P["n0"] * s)))
    if row["template"] == "energy_storage":
        P["p_lim_w"] *= s

        P["e_wh"] *= s
        P["p_batt_w"] *= s
    if row["template"] == "process_loop":
        P["p_max_w"] *= s
    if organism:
        if row["template"] in ("compute_pool", "thermal_zone", "energy

    ↪ _storage", "process_loop"):
            ratio = ORGANISM_DT / P["dt"]
            P["dt"] = ORGANISM_DT
            if "sub" in P:
                P["sub"] = max(1, int(round(P["sub"] * ratio)))
            if "startup_steps" in P:

                P["startup_steps"] = max(1, int(round(P["startup_steps
    ↪ "] / ratio)))
                P["stab_steps"] = max(1, int(round(P["stab_steps"] / r
    ↪ atio)))
                if "ca_unneeded_steps" in P:
                    P["ca_unneeded_steps"] = max(1, int(round(P["ca_un
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ needed_steps"] / ratio)))
                P["period_steps"] = ORGANISM_STEPS * 4
            if row["template"] == "energy_storage":
                P["start_h"] = P["peak_h"] - 0.5                  # t
    ↪ he hour around the site's peak
        else:

            P["dt_dec"] = ORGANISM_DT
    return P
```

H.48 tools/knob_verdict.py

211 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The brain's own verdict on one live knob, in real time (the founder
    ↪ 's order of 9 October 2026).

Every knob starts in watch: one wire out, nothing written. It is writt

    ↪ en only between native and the deepest step a paired
trial on the stack itself has allowed under the declared objective, an
    ↪ d a step that fails its trial is not taken. The
trial, the judgement, the allowance and the recheck are the frozen eng
    ↪ ine's (omnicompass/verdict.py, Omni v3, byte for
byte); this module gives that Verdict the knob's notch, the cost it ju

    ↪ dges and the condition under which a trial of this
knob is fair. Nothing in the engine changes.

  two directions   spend (more of the resource: a bigger ceiling, more
    ↪  consumers, more connections) and give back (less:
                   fewer connections, a smaller pool or cache), each i
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
  ↪ ts own Verdict with its own steps outward from native.
                Restoring native is never a step: the knob may alwa
  ↪ ys be moved back toward the operator's setting.
the condition    a give-back step is tried while the service is calm
  ↪  and the stack's own give-back condition holds (the
                engine's rule: a trial is never run under stress);

  ↪ a spend step is tried while the compass asks to spend
                and the stack's own spend condition holds (a cache
  ↪ full and missing, messages waiting, clients queued),
                because that is the only time a spend can show what
  ↪  it buys. A give-back trial once started runs on
                until its samples are in unless the service leaves

  ↪ calm, when it is abandoned (the engine's rule). A
                spend trial runs to its samples whatever the compas
  ↪ s's force: the spend's own effect calms the service
                within seconds, and a trial ended on that calm coul
  ↪ d never be judged (the first counted set of 10
                October 2026 abandoned every Kafka spend trial that

  ↪  way; amendment 3 in the five preregistrations). Only
                the wall (a fail-up) or the engine's own time limit
  ↪  ends a spend trial early.
the cost         one sample a second from the stack's own readings.
  ↪ Under the resource objective, the preregistered
                reading of the Omni index, cost = resource held x h

  ↪ ost CPU busy share x latency / work: a step passes only
                if the service gained outweighs the resource and CP
  ↪ U spent, by the index's own arithmetic (a geometric mean
                of the four ratios no lower than one is the same st
  ↪ atement). Under the service objective cost = latency /
                work: the resources are shown and not judged. The a

  ↪ im is the lower cost; "no higher within the tolerance" passes,
                which is the engine's judge, so a step that buys no
  ↪ thing and costs nothing is allowed and a step that costs
                more than it buys is refused.
the hold         during a trial the knob stands at the phase's value
  ↪  whatever the compass asks: the reference at the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                    deepest step already allowed (native at first), the
    ↪ n the trial step one notch further. Between trials the
                    compass moves the knob by its own law, clamped to t
    ↪ he allowance. A fail-up never spends beyond the
                    allowance; toward native it is always free.
  the record        every trial, allowance, refusal and abandonment is
    ↪ kept for the audit, with the counts and the state:
                    "left native" (no step allowed in either direction)
    ↪  or "acting" with the allowance.
"""
from __future__ import annotations

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from omnicompass.verdict import Verdict  # noqa: E402  (the frozen eng
    ↪ ine's verdict, unchanged)

RESOURCE, SERVICE, PER_WORK = "resource", "service", "per-work"
OBJECTIVES = (RESOURCE, SERVICE, PER_WORK)
SPEND, GIVE = "spend", "give back"

def sample_cost(objective, work, latency, resource, cpu_share):
    """One second's cost under the objective (its aim: lower); None wh
    ↪ en nothing worked (nothing was measured).

    resource   the index's reading: resource x host CPU busy share x l
    ↪ atency / work (declared 9 October)

    service    latency / work: the resources shown, not judged (declar
    ↪ ed 9 October)
    per-work   resource / work: the resource spent per unit of work in
    ↪ side the line, the speed left to the compass's own line
               as the guardrail (the card's reading, work per energy;
    ↪ added 10 October for the machine's own power, where the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                resource is the energy the processor's meter reads)"""
    if work is None or latency is None or work <= 0 or latency < 0:
        return None
    if objective == SERVICE:
        return latency / work
    if objective == PER_WORK:

        return max(float(resource), 1e-9) / work
    cpu = 1.0 if cpu_share is None else max(float(cpu_share), 1e-3)
    return max(float(resource), 1e-9) * cpu * latency / work


class KnobVerdict:

    def __init__(self, native, notch, cover, objective=RESOURCE, toler
    ↪ ance=0.02, min_samples=8, probe_every=20, recheck=60,
                 max_trial=40, settle_s=2.0):
        if objective not in OBJECTIVES:
            raise ValueError(f"objective must be one of {OBJECTIVES},
    ↪ not {objective!r}")

        lo, hi = cover
        self.native, self.notch, self.lo, self.hi = native, notch, lo,
    ↪  hi
        self.objective = objective
        self.settle_s = settle_s
        self.verdicts = {

            SPEND: Verdict(tolerance, min_samples, probe_every, rechec
    ↪ k, max_steps=max(0, int((hi - native) // notch)),
                           max_trial=max_trial, incremental=True),
            GIVE: Verdict(tolerance, min_samples, probe_every, recheck
    ↪ , max_steps=max(0, int((native - lo) // notch)),
                          max_trial=max_trial, incremental=True),

        }
        self.phase_dir = None                       # the direction whos
    ↪ e trial phase holds the knob, or None
        self.events = []
        self.counts = {"trials": 0, "allowed": 0, "refused": 0, "aband
    ↪ oned": 0, "clamped": 0, "held": 0, "samples": 0}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.last_value, self.changed_at, self.t = native, None, 0.0

    # ------------------------------------------------------------------
    ↪ ------------------------------ the steps
    def value_of(self, direction, step):
        v = self.native + step * self.notch if direction == SPEND else

    ↪  self.native - step * self.notch
        return max(self.lo, min(self.hi, v))

    @property
    def allowed_low(self):
        return self.value_of(GIVE, self.verdicts[GIVE].allowed)

    @property
    def allowed_high(self):
        return self.value_of(SPEND, self.verdicts[SPEND].allowed)

    @property

    def state(self):
        if self.verdicts[SPEND].allowed == 0 and self.verdicts[GIVE].a
    ↪ llowed == 0:
            return "left native"
        return f"acting: {self.allowed_low} to {self.allowed_high}"

    # ------------------------------------------------------------------
    ↪ ------------------------------ the samples
    def observe(self, cost, t, settled=True):
        """This second's cost sample, measured under the value the kno
    ↪ b stood at. Fed to the direction in a trial phase,
        once the knob has settled after its last change; nothing else

    ↪ is kept."""
        if cost is None or not settled or self.phase_dir is None:
            return False
        if self.changed_at is not None and t - self.changed_at < self.
    ↪ settle_s:
            return False
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.verdicts[self.phase_dir].observe([cost])
        self.counts["samples"] += 1
        return True

    # ------------------------------------------------------------------
    ↪ ----------------------------- the decision

    def decide(self, wanted, spend_ok, give_ok, t, why="", stress=None
    ↪ , calm=None, fail_up=False):
        """The compass's wanted value becomes the knob's target: the p
    ↪ hase's value while a trial holds the knob, the wanted
        value clamped to the allowance otherwise. spend_ok / give_ok:
    ↪ the stack's own conditions for a step in each direction

        this second (the compass asking to spend with the stack full a
    ↪ nd missing; calm with the stack holding its demand).
        stress / calm: the service past the cushion toward the line /
    ↪ inside the calm cushion (the compass's force): a
        give-back trial is abandoned when the service leaves calm (the
    ↪  engine's rule); a spend trial runs to its samples

        whatever the force (amendment 3, 10 October 2026). fail_up: th
    ↪ e service at the wall: every trial is abandoned and
        the knob goes where the fail-up says, native always free, neve
    ↪ r beyond the allowance.
        Returns (target, why, info)."""
        self.t = t

        order = [SPEND, GIVE] if self.phase_dir != GIVE else [GIVE, SP
    ↪ END]
        ok = {SPEND: bool(spend_ok), GIVE: bool(give_ok)}
        stress = bool(spend_ok) if stress is None else bool(stress)
        calm = bool(give_ok) if calm is None else bool(calm)
        holding = self.phase_dir

        results = {}
        for name in order:
            v = self.verdicts[name]
            if fail_up:
                go_on = False
            elif v.phase is not None:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            # a give-back trial is abandoned when the service leav
↪ es calm (the engine's rule: never a trial under stress);
            # a spend trial runs to its samples whatever the compa
↪ ss's force, because the spend's own effect calms the
            # service within seconds and ending the trial on that
↪ calm would mean no spend could ever be judged (seen on

            # every Kafka trial of the first counted set, 10 Octob
↪ er 2026: amendment 3); only the wall ends it early
            go_on = (not stress) if name == GIVE else True
        else:
            go_on = ok[name] and holding in (None, name)
↪    # a new trial only in a free second, in the direction asked f

↪ or
        step, is_trial, ev = v.tick(go_on)
        if ev:
            self.events.append({"t": round(t, 1), "direction": nam
↪ e, **ev})
            self._count(ev)

        results[name] = (step, is_trial)
        if v.phase is not None:
            holding = name
        elif holding == name:
            holding = None
    self.phase_dir = holding

    info = {"verdict_phase": None, "verdict_direction": self.phase
↪ _dir, "allowed_low": self.allowed_low, "allowed_high": self.allo
↪ wed_high}
    if self.phase_dir is not None:
        v = self.verdicts[self.phase_dir]
        step, _ = results[self.phase_dir]

        target = self.value_of(self.phase_dir, step)
        info["verdict_phase"] = v.phase
        if target != wanted:
            self.counts["held"] += 1
        why = f"{why}; held by the verdict: {self.phase_dir} {'ref
↪ erence' if v.phase == 'ref' else 'trial'} at {target}"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        else:
            target = min(max(wanted, self.allowed_low), self.allowed_h
    ↪ igh)
            if target != wanted:
                self.counts["clamped"] += 1
                why = f"{why}; the verdict allows {self.allowed_low} t
    ↪ o {self.allowed_high}: held at {target}"
        if target != self.last_value:
            self.changed_at, self.last_value = t, target
        return target, why, info

    def _count(self, ev):
        s = ev.get("verdict", "")
        if s == "trial":
            self.counts["trials"] += 1
        elif s == "step allowed":
            self.counts["allowed"] += 1
        elif s.startswith("step refused"):
            self.counts["refused"] += 1
        elif s.startswith("trial abandoned"):
            self.counts["abandoned"] += 1

    # ------------------------------------------------------------------
    ↪ ----------------------------- the record
    def record(self, keep_events=400):
        return {"objective": self.objective, "state": self.state, "nat
    ↪ ive": self.native, "notch": self.notch,
                "spend_allowed_steps": self.verdicts[SPEND].allowed, "
    ↪ give_back_allowed_steps": self.verdicts[GIVE].allowed,
                "allowed_low": self.allowed_low, "allowed_high": self.
    ↪ allowed_high, "counts": dict(self.counts),
                "events": self.events[:keep_events], "events_total": l
    ↪ en(self.events)}


def summarize(records):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    """One line for a table from the omni arms' records of one workloa
    ↪ d: under the tune-down the verdict's state and allowance in
    each arm; under the max (omni-v22, records with dial "max") the ma
    ↪ x's own counts (tools/store_brain.py)."""
    if not records:
        return "no verdict record"

    parts = []
    for r in records:
        if r.get("dial") == "max":
            from tools.store_brain import summarize_max
            parts.append(summarize_max([r]))
            continue

        c = r.get("counts", {})
        parts.append(f"{r.get('state', '?')} ({c.get('trials', 0)} tri
    ↪ als, {c.get('allowed', 0)} allowed, {c.get('refused', 0)} refuse
    ↪ d)")
    return "; ".join(parts)
```

H.49 `tools/run_citylearn.py`

280 lines.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES

    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of CityLearn's own controller: an independent,
    ↪ recognized simulator we did not write.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
CityLearn (Intelligent Environments Lab, University of Texas at Austin
    ↪ ; github.com/intelligent-environments-lab/citylearn,
MIT license) simulates districts of real buildings from measured data:
    ↪  each building's load, its solar panels, and its
battery, hour by hour for a year. It ships its own controllers. Its ru
    ↪ le-based controller (citylearn.agents.rbc.BasicRBC)

is the native arm: it charges and discharges every battery by the hour
    ↪  of the day.

Arms, the same district, the same year, the same weather and loads:
  native  CityLearn's BasicRBC alone
  omni    the same BasicRBC, with the compass law (omnicompass/compass

    ↪ _law.py) on top of its electric battery commands only (a
          cold or hot water tank is a thermal muscle with its own read
    ↪ ing and stays native). The reading is the
          district's draw without its batteries in the coming hour, re
    ↪ ad from the same hour of the past week's days
          (the demand the batteries answer, not their own effect; caus

    ↪ al: only hours already seen; omni-v9); its band is
          the 10th to the 90th percentile of the past week's draw. The
    ↪  compass's force moves every battery command the same way:
          a positive force (the district drawing hard) pushes toward d
    ↪ ischarge, a negative force (a quiet district)
          toward charge, by at most GAIN of the command's range, alway

    ↪ s inside the battery's own limits [-1, 1]. At
          KILL_AT of the year Omni-Compass hands every command back to
    ↪  the native controller, and the file checks it did

Scored by CityLearn's own evaluation (env.evaluate): every cost functi
    ↪ on as CityLearn defines it, each the controller's

value over the value with no battery at all (CityLearn's direction is
    ↪ lower for every one). Each is read by the one key
(tools/readings.py): energy not served and time uncomfortable are miss
    ↪ es, a rise is a LOSS; the bill, the electricity, the
carbon and the power held (its peaks, swings and unevenness) are resou
    ↪ rces, a fall is a cut and a rise is power given.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
Nothing here is scored by Omni-Compass's own code.

  python3 tools/run_citylearn.py --datasets citylearn_challenge_2022_p
    ↪ hase_all,... --out results/citylearn
"""
from __future__ import annotations

import argparse

try:                                                    # the legal
    ↪  notice every generated report carries
    from tools.legal import stamp as _legal_stamp

except ImportError:
    import sys as _s, pathlib as _p; _s.path.insert(0, str(_p.Path(__f
    ↪ ile__).resolve().parents[1])); from tools.legal import stamp as
    ↪ _legal_stamp
from tools.readings import EVEN as W_EVEN, GAIN as W_GAIN, GIVEN as W_
    ↪ GIVEN, LOSS as W_LOSS   # the one key's words, written

                                                        # once (too
    ↪ ls/readings.py); GAIN below is the compass's gain
import hashlib
import json
import math
import sys

import time
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from omnicompass.compass_law import Band, CompassLaw  # noqa: E402

import os
GAIN = float(os.environ.get("CL_GAIN", 0.5))   # the most the compass
    ↪ moves a battery command, as a share of its range
# the three settings below were chosen on the tuning district (citylea
    ↪ rn_challenge_2022_phase_1) only and frozen before
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# any other district ran (docs/CITYLEARN_PREREGISTRATION.md); the glid
    ↪ e and the coming hour's reading are omni-v9's
# amendment of 17 October, found on the district whose LOSS they repai
    ↪ r and read on every district (docs/OMNI_V9.md); the
# glide's speed is omni-v10's (docs/OMNI_V10.md)
TAU = float(os.environ.get("CL_TAU", 1.0))     # the battery's respons
    ↪ e the compass is damped for (hours)
SLEW = float(os.environ.get("CL_SLEW", 0.0875))  # the most Omni-Compa
    ↪ ss's adjustment may change in one hour (omni-v10: the
                                                  # glide that cuts the
    ↪  bill most with no LOSS in any district, read by the one key
                                                  # of 17 October; omni
    ↪ -v9 0.0625, where nothing rose; 0.05 before)
FORECAST_DAYS = 7   # omni-v9: the coming hour's draw is the mean of t
    ↪ he same hour over the past week's days
WINDOW_H = 168      # the band's window: the past week of hourly draw
KILL_AT = 0.9       # Omni-Compass hands every command back at 90% of
    ↪ the year

KPIS = ["cost_total", "electricity_consumption_total", "carbon_emissio
    ↪ ns_total", "daily_peak_average",
        "all_time_peak_average", "ramping_average", "daily_one_minus_l
    ↪ oad_factor_average",
        "monthly_one_minus_load_factor_average", "zero_net_energy", "a
    ↪ nnual_normalized_unserved_energy_total",
        "discomfort_proportion"]
WORDS = {"cost_total": "electricity bill", "electricity_consumption_to
    ↪ tal": "electricity bought",
         "carbon_emissions_total": "carbon", "daily_peak_average": "da
    ↪ ily peak draw", "all_time_peak_average": "highest peak",
         "ramping_average": "ramping (swings hour to hour)", "daily_on
    ↪ e_minus_load_factor_average": "daily load unevenness",
         "monthly_one_minus_load_factor_average": "monthly load uneven
    ↪ ness", "zero_net_energy": "distance from zero net energy",
         "annual_normalized_unserved_energy_total": "energy not served
    ↪ ", "discomfort_proportion": "time uncomfortable"}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def percentile(xs, q):
    s = sorted(xs)
    k = (len(s) - 1) * q
    f = int(math.floor(k)); c = min(f + 1, len(s) - 1)
    return s[f] + (s[c] - s[f]) * (k - f)

def coming_hour(hist, days=FORECAST_DAYS):
    """The district's draw in the coming hour, read from the same hour
    ↪  of the past days (hist[-24], hist[-48], ... up to
    `days` of them; causal: only hours already seen), or the last hour

    ↪ 's own draw until a day has been seen. omni-v9: the
    reading of the hour gone left the adjustment about five hours behi
    ↪ nd the day (its strongest discharge at midnight, the
    draw's peak at 19:00), still charging as the evening peak climbed,
    ↪  so the batteries were empty at each month's top
    hour (docs/OMNI_V9.md)."""

    same = hist[-24::-24][:days]
    return sum(same) / len(same) if same else hist[-1]


def glide(off, target, slew=None):
    """The adjustment moved toward its target by at most `slew` (SLEW)

    ↪  in one hour: it glides, never jumps."""
    step = SLEW if slew is None else slew
    return off + max(-step, min(step, target - off))


def schema_of(name, data_dir):

    if data_dir:
        p = Path(data_dir) / name / "schema.json"
        if p.exists():
            return str(p)
    from citylearn.data import DataSet
    return DataSet().get_schema(name)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def run_arm(schema, arm):
    from citylearn.citylearn import CityLearnEnv
    from citylearn.agents.rbc import BasicRBC
    try:                        # CityLearn's own rule controllers need
    ↪  its flat interface; newer districts ship another
        env = CityLearnEnv(schema, central_agent=True, interface="flat
    ↪ ")
    except TypeError:           # a CityLearn without the choice is fla
    ↪ t already
        env = CityLearnEnv(schema, central_agent=True)

    agent = BasicRBC(env)
    obs, _ = env.reset()
    steps = env.time_steps
    kill = int(KILL_AT * steps)
    compass_law = None
    hist, moved, handed_back, off = [], 0, True, 0.0

    # one wire, one muscle: the compass steers only the electric batte
    ↪ ries from the district's electricity reading; a cold
    # or hot water tank (cooling_storage, dhw_storage) is a thermal mu
    ↪ scle that needs its own reading and stays native
    names = [n for agent in env.action_names for n in agent]
    battery = [n == "electrical_storage" for n in names]

    # omni-v8 (17 October): a battery that also carries its building t
    ↪ hrough power outages has a second duty, the reserve
    # for the outage, which the district's draw cannot show; it stays
    ↪ native, as a water tank does, until it has its own wire
    # and its own reading. The building's own setting says which batte
    ↪ ries those are (CityLearn's simulate_power_outage).

    if len(env.action_names) == 1:                # the central agent
    ↪ : one list, each building's actions in its own order
        owner = [bi for bi, b in enumerate(env.buildings) for _ in b.a
    ↪ ctive_actions]
    else:
        owner = [bi for bi, agent_names in enumerate(env.action_names)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  for _ in agent_names]
    reserve = [bool(getattr(b, "simulate_power_outage", False)) for b
    ↪ in env.buildings]
    steer = ([bt and not reserve[owner[i]] for i, bt in enumerate(batt
    ↪ ery)] if len(owner) == len(battery)
             else [False] * len(battery))      # an action list it can

    ↪ not place on its buildings: nothing is steered
    t0, k = time.time(), 0
    while not env.terminated:
        a = agent.predict(obs)
        if arm == "omni":
            # demand the batteries do not cause: the district's draw w

    ↪ ithout storage, recorded hour by hour (the compass reads
            # the room, not its own heater: a reading that includes th
    ↪ e battery feeds every battery move straight back); the
            # compass reads the coming hour from this record (coming_h
    ↪ our)
            seen = len(env.net_electricity_consumption)

            ws = env.net_electricity_consumption_without_storage
            net = float(ws[seen - 1]) if seen and len(ws) >= seen else
    ↪  None
            if net is not None:
                hist.append(float(net))
            if k < kill and len(hist) >= 24:

                w = hist[-WINDOW_H:]
                lo, hi = percentile(w, 0.10), percentile(w, 0.90)
                if hi - lo > 1e-9:
                    band = Band(lo, hi)
                    if compass_law is None:
                        compass_law = CompassLaw(band, dt=1.0, tau=TAU

    ↪ )
                    compass_law.band = band
                    f = compass_law.force(coming_hour(hist))
                    off = glide(off, -GAIN * f)                    #
    ↪ the adjustment glides, never jumps more than SLEW
                    if abs(off) > 1e-12:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                        flat = [x for row in a for x in row]
                        if any(steer):
                            flat = [max(-1.0, min(1.0, x + off)) if b
    ↪ else x for x, b in zip(flat, steer)]
                            a = [flat]
                            moved += 1                                  #
    ↪ an hour counts as moved only where a command moved
            elif k >= kill:
                handed_back = handed_back and True          # from her
    ↪ e every command is the native controller's own
        obs, _, _, _, _ = env.step(a)
        k += 1
    ev = env.evaluate()
    d = ev[ev["level"] == "district"].set_index("cost_function")["valu
    ↪ e"].to_dict()
    return {"arm": arm, "buildings": len(env.buildings), "steps": step
    ↪ s,
            "batteries": sum(battery), "batteries_left_native_for_outa
    ↪ ges": sum(battery) - sum(steer), "thermal_stores_left_native": s
    ↪ um(1 for n in names if n in ("cooling_storage", "dhw_storage")),
    ↪  "seconds": round(time.time() - t0, 1),
            "moved_hours": moved, "handed_back": handed_back and k >=
    ↪ kill,
            "kpis": {x: (None if (d.get(x) is None or (isinstance(d.ge
    ↪ t(x), float) and math.isnan(d.get(x)))) else float(d[x]))
                     for x in KPIS}}


MISSES = ("annual_normalized_unserved_energy_total", "discomfort_propo
    ↪ rtion")   # what the occupants miss; every other score
    ↪            # is a resource spent or held


def reading(score, native, omni):
    """This run's word by the one key (tools/readings.py): a fall is a
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  GAIN (a cut, or fewer misses); a rise in a miss is a
    LOSS, a rise in a resource spent or held (the bill, the electricit
    ↪ y, the carbon, the peaks, swings and unevenness) is
    power given. A word that counts is the three-run table's (tools/ci
    ↪ tylearn_abc.py), never one run's."""
    if native is None or omni is None:

        return "not scored"
    if abs(omni - native) < 1e-9:
        return W_EVEN
    return W_GAIN if omni < native else (W_LOSS if score in MISSES els
    ↪ e W_GIVEN)

def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--datasets", default="")
    ap.add_argument("--report-only", default="", help="build the repor

    ↪ t from the per-district files in this folder")
    ap.add_argument("--out", required=True)
    ap.add_argument("--data-dir", default="", help="a local copy of Ci
    ↪ tyLearn's data/datasets (else CityLearn fetches it)")
    a = ap.parse_args(argv)
    out = Path(a.out); out.mkdir(parents=True, exist_ok=True)

    try:
        import citylearn
        version = citylearn.__version__
    except ImportError:
        version = "(see each district file)"
    res = {}

    if a.report_only:
        for f in sorted(Path(a.report_only).glob("*.json")):
            res[f.stem] = json.loads(f.read_text())
    for name in [x for x in a.datasets.split(",") if x]:
        schema = schema_of(name, a.data_dir)
        res[name] = {}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        for arm in ("native", "omni"):
            try:
                res[name][arm] = run_arm(schema, arm)
            except Exception as e:                          # a district
↪ an arm cannot run is reported, never left out
                import traceback

                res[name][arm] = {"arm": arm, "error": f"{type(e).__na
↪ me__}: {e}"[:600],
                                  "where": traceback.format_exc().stri
↪ p().splitlines()[-3:]}
        res[name]["citylearn_version"] = version
        res[name]["settings"] = {"gain": GAIN, "tau_h": TAU, "glide_pe

↪ r_h": SLEW, "window_h": WINDOW_H, "kill_at": KILL_AT,
                                 "forecast_days": FORECAST_DAYS}
        (out / f"{name}.json").write_text(json.dumps(res[name], indent
↪ =1) + "\n")
        for arm in ("native", "omni"):
            r = res[name][arm]

            print(f"== {name} {arm}: " + (f"{r['seconds']} s" if "seco
↪ nds" in r else f"ERROR {r['error']} at {r['where']}"),
                  flush=True)
    L = ["# Omni-Compass on top of CityLearn's own controller", "",
         f"CityLearn {version} (Intelligent Environments Lab, Universi
    ↪ ty of Texas at Austin; MIT license): an "

         "independent simulator of real buildings from measured data,
    ↪ with its own controllers and its own scoring. "
         "Native: CityLearn's rule-based battery controller (BasicRBC)
    ↪ . Omni: the same controller with the compass "
         f"law on top of its battery commands (gain {GAIN}, response {
    ↪ TAU} h, glide {SLEW} an hour, reading the coming hour from the s

    ↪ ame hour of the past {FORECAST_DAYS} days, band from the past we
    ↪ ek's district draw, handed back at "
         f"{int(KILL_AT * 100)}% of the year). Every number is CityLea
    ↪ rn's own score: the controller over no battery at "
         "all, read by the one key: a fall is a GAIN; a rise in energy
    ↪  not served or time uncomfortable is a LOSS, a rise in "
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
     "any other score (a resource spent or held) is power given. E
↪ vidence class: an independent recognized simulator "
     "(not our model).", ""]
for name, r in res.items():
    n, o = r["native"], r["omni"]
    if "error" in n or "error" in o:
        L += [f"## {name}: not run to the end", "",
              f"- native: {n.get('error', 'ran')}", f"- omni: {o.g
↪ et('error', 'ran')}", ""]
        continue
    if name.startswith("citylearn_challenge_2022_phase_1"):
        name += " (the tuning district: not counted in the confirm
↪ ation)"
    L += [f"## {name}: {n['buildings']} buildings, {n['steps']:,}
↪ hours", "",
          f"Omni-Compass moved the commands in {o['moved_hours']:,
↪ } hours; every command handed back at "
          f"{int(KILL_AT * 100)}%: {'yes' if o['handed_back'] else
↪  'NO'}.", "",
          "| CityLearn score (over no battery) | native | omni | C
↪ hange | Reading (this run) |",
          "|---|---:|---:|---:|---|"]
    for x in KPIS:
        nv, ov = n["kpis"][x], o["kpis"][x]
        if nv is None and ov is None:
            continue
        ch = f"{100 * (ov - nv) / abs(nv):+.2f}%" if nv not in (No
↪ ne, 0) and ov is not None else ""
        rd = reading(x, nv, ov)
        L.append(f"| {WORDS[x]} (`{x}`) | {'' if nv is None else f
↪ '{nv:.4f}'} | {'' if ov is None else f'{ov:.4f}'} | {ch} | "
                 f"{rd + (': less ' + WORDS[x] if rd == W_GAIN els
↪ e ': more ' + WORDS[x] if rd in (W_LOSS, W_GIVEN) else '')} |")
    L.append("")
L += ["## Across every district", "",
      "The tuning district is left out of this table.", "",
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
          "| CityLearn score | Districts where it fell | Districts whe
    ↪ re it rose | Mean change | A rise reads |",
          "|---|---:|---:|---:|---|"]
    for x in KPIS:
        pairs = [(r["native"]["kpis"][x], r["omni"]["kpis"][x]) for nm
    ↪ , r in res.items() if nm != "citylearn_challenge_2022_phase_1"
                 and "kpis" in r["native"] and "kpis" in r["omni"]
                 if r["native"]["kpis"][x] not in (None, 0) and r["omn
    ↪ i"]["kpis"][x] is not None]
        if not pairs:
            continue
        b = sum(1 for nv, ov in pairs if ov < nv - 1e-9); w = sum(1 fo
    ↪ r nv, ov in pairs if ov > nv + 1e-9)
        m = sum(100 * (ov - nv) / abs(nv) for nv, ov in pairs) / len(p
    ↪ airs)
        L.append(f"| {WORDS[x]} | {b} of {len(pairs)} | {w} of {len(pa
    ↪ irs)} | {m:+.2f}% | {W_LOSS if x in MISSES else W_GIVEN} |")
    (out / "CITYLEARN.md").write_text("\n".join(_legal_stamp(L)) + "\n
    ↪ ")
    sums = [f"{hashlib.sha256(p.read_bytes()).hexdigest()}  {p.name}"
    ↪ for p in sorted(out.glob("*.json"))]
    (out / "SHA256SUMS.txt").write_text("\n".join(sums) + "\n")
    print("\n".join(L))
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

H.50 `tools/run_cpu_power.py`

789 lines.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of the Linux kernel's own CPU frequency governo
    ↪ r, energy read by the processor's own meter
(docs/CPU_POWER_PREREGISTRATION.md).

The machine as it runs: the kernel's frequency governor as shipped (sc
    ↪ hedutil, or the processor driver's own) choosing the
clock every few milliseconds under a fixed ceiling, the processor's en
    ↪ ergy counter (RAPL, /sys/class/powercap) ticking in
microjoules, and a CPU-bound service answering requests offered at a f

    ↪ ixed rate that steps up and down. The governor is the
native controller here; it keeps running in both arms.


  native  the machine as found: the governor as shipped, the frequency
    ↪  ceiling where the operator left it (the top)
  omni    the same machine, the same governor, with the compass law (o

    ↪ mnicompass/compass_law.py) on one knob, the frequency
          ceiling (cpufreq scaling_max_freq, written through the kerne
    ↪ l's own interface on every CPU), inside the cover
          [half the top clock, the top]: the compass reads the service
    ↪ 's own request latency on a band from 0 to the response
          line and holds it at 40% of the line; a slow service raises

    ↪ the ceiling by notches, calm gives one notch back after
          a dwell, at 95% of the line the ceiling goes to the top at o
    ↪ nce (fail up); the brain's own verdict
          (tools/knob_verdict.py around the engine's omnicompass/verdi
    ↪ ct.py) tries every notch down on the machine itself
          before it is allowed, under the declared objective, and a no

    ↪ tch that does not pay is not taken; the ceiling is
          handed back to the operator's at the end and read back; a ce
    ↪ iling found at a value Omni did not write stops it
          writing (one writer); the master switch and the lease (omnic
    ↪ ompass/master.py) hand back if Omni dies
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
Gauges from the service's own records (every request's latency), the p
    ↪ rocessor's energy counter (joules over the window,
class P: the processor's own meter), a wall plug where one is fitted (
    ↪ WALL_METER, tools/wall_meter.py: the whole machine,
independent of Omni), the kernel's own frequency readings and /proc/st
    ↪ at. Energy is the resource; the machine count is one

in both arms. Every row is reported, losses included.

  sudo python3 tools/run_cpu_power.py --probe                        #
    ↪ can this machine run it? (the governor writable, the meter reada
    ↪ ble)
  sudo python3 tools/run_cpu_power.py --workloads tuning --reps 3 --ou

    ↪ t DIR
  python3 tools/run_cpu_power.py --report-only DIR                   #
    ↪ re-read finished <workload>.json files and write CPU_POWER.md
  python3 tools/run_cpu_power.py --restore DIR/snapshot.json         #
    ↪ put the ceiling back by hand (the watchdog runs this if Omni die
    ↪ s)

  python3 tools/run_cpu_power.py --plant sim --workloads tuning --reps
    ↪  1 --step-s 2 --out DIR   # the harness on a modelled machine (t
    ↪ ests only)
"""
from __future__ import annotations

import argparse
import glob
import hashlib
import json
import math
import multiprocessing as mp

import os
import queue
import random
import statistics
import subprocess
import sys
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
import threading
import time
from pathlib import Path

try:                                                    # the legal
    ↪  notice every generated report carries

    from tools.legal import stamp as _legal_stamp
except ImportError:
    import pathlib as _p; sys.path.insert(0, str(_p.Path(__file__).res
    ↪ olve().parents[1])); from tools.legal import stamp as _legal_sta
    ↪ mp
ROOT = Path(__file__).resolve().parents[1]

sys.path.insert(0, str(ROOT))
from omnicompass import master  # noqa: E402
from omnicompass.compass_law import Band, CompassLaw  # noqa: E402
from tools.knob_verdict import KnobVerdict, sample_cost, OBJECTIVES, R
    ↪ ESOURCE as DEFAULT_OBJECTIVE  # noqa: E402
from tools.readings import word as gel_word, aim as gel_aim, EVEN_SAME

    ↪ , EVEN_NOISE, SHOWN  # noqa: E402  (the words every table uses)

SYSFS_CPU = "/sys/devices/system/cpu"
SYSFS_RAPL = "/sys/class/powercap"
LINE_MS = 20.0           # the response line: a request answered with
    ↪ in 20 ms (the service itself takes about 5 ms at the top clock)

CENTER = 0.4             # the compass holds the service's latency at
    ↪  40% of the line
DT = 1.0                 # one decision a second
TAU = 2.0                # a ceiling written shows in the latency wit
    ↪ hin a couple of seconds
SMOOTH = 0.5

NOTCH_SHARE = 0.05       # one notch of the knob: 5% of the top clock
FLOOR_SHARE = 0.5        # the cover: never under half the top clock
    ↪ (and never under the processor's own minimum)
DWELL_S = 5.0            # after raising the ceiling, no giving back
    ↪ within five seconds (no hunting)
SERVICE_MS = 5.0         # the work unit: about 5 ms of one core at t
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ he top clock, calibrated once per run and then fixed for every a
    ↪ rm
UTIL_AT_TOP_STEP = 0.9    # the offered rate at step 8 is 90% of the m
    ↪ achine's capacity at the top clock
TOP_STEP = 8
STEPS = "1 2 3 2 3 4 5 6 5 4 3 2 1 2 1"

STEADY = "3 3 3 3 3 3"
BURST = "1 6 1 8 1 6"
from omnicompass.student_t import T95  # noqa: E402  (the two-sided 95
    ↪ % point of Student's t, any degrees of freedom)
SAME_REL = 1e-6

WORKLOADS = {
    # name: (steps, tuning?)
    "tuning": (STEADY, True),     # the tuning case: steady at step 3
    ↪ (the rules are frozen on it; shown, not counted)
    "wandering": (STEPS, False),  # demand that wanders one step at a
    ↪ time

    "burst": (BURST, False),      # bursts sized to the machine
}

GAUGES = [  # key, label, direction
    ("work_inside_line_rps", "work inside the response line (requests
    ↪ a second answered within the line)", "higher"),

    ("rps", "throughput (requests a second)", "higher"),
    ("p95_ms", "latency, 95th percentile (ms)", "lower"),
    ("p99_ms", "latency, 99th percentile (ms)", "lower"),
    ("mean_ms", "latency, mean (ms)", "lower"),
    ("failed", "failed requests (not answered by the end of the arm)",
    ↪  "never more"),

    ("energy_j", "energy, CPU package (J; the processor's own meter, t
    ↪ he resource)", "spent"),
    ("power_w_mean", "power, CPU package mean (W)", "spent"),
    ("j_per_1k_inside", "energy per 1,000 requests inside the line (J)
    ↪ ", "spent"),
    ("wall_j", "energy, whole machine at the wall (J; where a plug is
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ fitted)", "spent"),
    ("ceiling_mhz_mean", "frequency ceiling held, mean (MHz; the knob)
    ↪ ", "shown"),
    ("freq_mhz_mean", "frequency run at, mean (MHz; the governor's own
    ↪  choice under the ceiling)", "shown"),
    ("cpu_busy_share", "CPU busy (share of the run)", "shown"),

    ("writes", "ceiling changes written (the knob's moves)", "shown"),
]


# ---------------------------------------------------------------------
    ↪ ---------------------------------- the plant

class Plant:
    """The machine through the kernel's own files: the frequency ceili
    ↪ ng of every CPU (the knob), the governor (read only,
    the native controller), the processor's energy counter (the meter)
    ↪ . `root` is / on a real machine; a modelled tree in a
    directory for the tests (SimPlant)."""

    def __init__(self, root="/"):
        self.root = Path(root)
        self.cpu_dir = self.root / SYSFS_CPU.lstrip("/")
        self.rapl_dir = self.root / SYSFS_RAPL.lstrip("/")
        self.cpus = sorted((d for d in self.cpu_dir.glob("cpu[0-9]*")

    ↪ if (d / "cpufreq").is_dir()), key=lambda d: int(d.name[3:]))
        self.packages = sorted(d for d in self.rapl_dir.glob("*rapl*:[
    ↪ 0-9]*") if d.name.count(":") == 1 and (d / "energy_uj").exists()
    ↪ )
        self.snapshot, self.last_written, self.expected = None, None,
    ↪ None

        self._e_prev = None


    # the knob
    def _read(self, cpu, name):
        return (cpu / "cpufreq" / name).read_text().strip()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def _put(self, path: Path, text: str):
        """One kernel attribute written (the kernel takes the value wh
    ↪ ole); the modelled machine replaces the file whole."""
        path.write_text(text)

    def top_khz(self):
        """The top clock: the highest any CPU reaches (on a machine wi
    ↪ th two kinds of core the smaller cores clamp a higher
        ceiling to their own top, which the kernel does by itself)."""
        return max(int(self._read(c, "cpuinfo_max_freq")) for c in sel
    ↪ f.cpus)
    def min_khz(self):
        return max(int(self._read(c, "cpuinfo_min_freq")) for c in sel
    ↪ f.cpus)

    def governors(self):
        return sorted({self._read(c, "scaling_governor") for c in self
    ↪ .cpus})

    def driver(self):
        try:
            return self._read(self.cpus[0], "scaling_driver")
        except OSError:
            return "?"

    def cur_khz_mean(self):
        vals = []
        for c in self.cpus:
            for name in ("scaling_cur_freq", "cpuinfo_cur_freq"):
                try:
                    vals.append(int(self._read(c, name))); break
                except (OSError, ValueError):
                    continue
        return sum(vals) / len(vals) if vals else float("nan")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def attach(self):
        """The snapshot, taken once: every CPU's governor and ceiling,
    ↪  as found. The knob starts at the operator's ceiling."""
        if self.snapshot is None:
            self.snapshot = {c.name: {"governor": self._read(c, "scali
    ↪ ng_governor"), "max_khz": int(self._read(c, "scaling_max_freq"))
    ↪ ,
                                      "min_khz": int(self._read(c, "sc
    ↪ aling_min_freq"))} for c in self.cpus}
        self.expected = {c.name: int(self._read(c, "scaling_max_freq")
    ↪ ) for c in self.cpus}
        self.last_written = max(self.expected.values())
        return self.last_written

    def ceiling_actual(self):
        """The ceiling as the kernel holds it now (the highest over th
    ↪ e CPUs)."""
        return max(int(self._read(c, "scaling_max_freq")) for c in sel
    ↪ f.cpus)

    def lever(self, check=True):
        """The knob's value (the ceiling last written, the operator's
    ↪ at first); every CPU's ceiling must still read what
        our last write left there, or another writer has the knob and
    ↪ Omni stops writing."""
        cur = {c.name: int(self._read(c, "scaling_max_freq")) for c in
    ↪  self.cpus}
        if check and self.expected is not None and cur != self.expecte
    ↪ d:
            moved = {k: (self.expected[k], v) for k, v in cur.items()
    ↪ if self.expected.get(k) != v}
            raise RuntimeError(f"another writer moved the ceiling (exp
    ↪ ected -> found, kHz): {moved}: Omni stops writing")
        return self.last_written if self.last_written is not None else
    ↪  max(cur.values())
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def write(self, khz):
        """One value to every CPU; the kernel clamps a core's ceiling
    ↪ to its own range, and what it reads back is what
        Omni expects to find there from now on."""
        for c in self.cpus:
            self._put(c / "cpufreq" / "scaling_max_freq", f"{int(khz)}
    ↪ \n")
        self.expected = {c.name: int(self._read(c, "scaling_max_freq")
    ↪ ) for c in self.cpus}
        self.last_written = int(khz)
        return int(khz)

    def restore(self):
        try:
            if not self.snapshot:
                return False
            for c in self.cpus:
                self._put(c / "cpufreq" / "scaling_max_freq", f"{self.
    ↪ snapshot[c.name]['max_khz']}\n")
            return all(int(self._read(c, "scaling_max_freq")) == self.
    ↪ snapshot[c.name]["max_khz"] for c in self.cpus)
        except OSError:
            return False

    def writable(self):
        """Can the ceiling be written? Tried with the value already th
    ↪ ere, so nothing changes."""
        try:
            c = self.cpus[0]; cur = self._read(c, "scaling_max_freq")
            self._put(c / "cpufreq" / "scaling_max_freq", cur + "\n")
            return True
        except (OSError, IndexError):
            return False

    # the meter
    def energy_j(self):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        """The processor's energy counter summed over the packages, in
    ↪  joules, wrap-around taken care of."""
        tot = 0.0
        cur = []
        for p in self.packages:
            e = int((p / "energy_uj").read_text().strip())
            try:
                rng = int((p / "max_energy_range_uj").read_text().stri
    ↪ p())
            except (OSError, ValueError):
                rng = None
            cur.append((e, rng))
        if self._e_prev is None:
            self._e_prev = cur; self._acc = 0.0
            return 0.0
        for (e, rng), (e0, _) in zip(cur, self._e_prev):
            d = e - e0
            if d < 0 and rng:
                d += rng + 1
            tot += max(0, d)
        self._e_prev = cur
        self._acc += tot / 1e6
        return self._acc

    def meter_readable(self):
        try:
            return bool(self.packages) and all(int((p / "energy_uj").r
    ↪ ead_text().strip()) >= 0 for p in self.packages)
        except (OSError, ValueError):
            return False

    def tick(self, busy_share):  # the real machine needs nothing; the
    ↪  modelled one advances its counter
        pass
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class SimPlant(Plant):
    """A modelled machine in a directory for the tests: four CPUs with
    ↪  the kernel's files, a governor that reads
    'schedutil', a top clock of 2,800 MHz, an energy counter that grow
    ↪ s as idle power plus a dynamic part rising with busy share
    and the clock to the power 2.5; the service time of a request grow
    ↪ s as the top clock over the ceiling."""

    TOP_KHZ, MIN_KHZ = 2_800_000, 800_000
    P_IDLE_W, P_DYN_W = 5.0, 80.0

    def __init__(self, root, cpus=4):
        root = Path(root)
        for i in range(cpus):
            d = root / SYSFS_CPU.lstrip("/") / f"cpu{i}" / "cpufreq";
    ↪ d.mkdir(parents=True, exist_ok=True)
            for name, val in (("scaling_governor", "schedutil"), ("sca
    ↪ ling_driver", "modelled"), ("scaling_max_freq", self.TOP_KHZ),
                              ("scaling_min_freq", self.MIN_KHZ), ("cp
    ↪ uinfo_max_freq", self.TOP_KHZ), ("cpuinfo_min_freq", self.MIN_KH
    ↪ Z),
                              ("scaling_cur_freq", self.TOP_KHZ)):
                (d / name).write_text(f"{val}\n")
        r = root / SYSFS_RAPL.lstrip("/") / "intel-rapl:0"; r.mkdir(pa
    ↪ rents=True, exist_ok=True)
        (r / "energy_uj").write_text("0\n"); (r / "max_energy_range_uj
    ↪ ").write_text("262143328850\n"); (r / "name").write_text("packag
    ↪ e-0\n")
        super().__init__(root)
        self._uj = 0

    def _put(self, path: Path, text: str):
        tmp = path.with_name(path.name + ".tmp")
        tmp.write_text(text); os.replace(tmp, path)

    def service_factor(self):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        """How much slower a request is than at the top clock, under t
    ↪ he ceiling as the modelled governor runs it."""
        return self.TOP_KHZ / max(self.MIN_KHZ, self.ceiling_actual())

    def tick(self, busy_share, dt=DT):
        cur = self.ceiling_actual(); f = cur / self.TOP_KHZ

        for c in self.cpus:
    ↪  # the modelled governor runs at the ceiling while busy
            self._put(c / "cpufreq" / "scaling_cur_freq", f"{int(cur)}
    ↪ \n")
        p = self.P_IDLE_W + self.P_DYN_W * max(0.0, min(1.0, busy_shar
    ↪ e)) * f ** 2.5

        self._uj = (self._uj + int(p * dt * 1e6)) % (262143328850 + 1)
        self._put(self.packages[0] / "energy_uj", f"{self._uj}\n")


# --------------------------------------------------------------------
    ↪ ---------------------------------- the service

def work_unit(k, buf=b"\x5a" * 65536):
    """One request's work: k passes of SHA-256 over 64 KB, the same co
    ↪ unt in every arm (CPU-bound, no I/O)."""
    h = b""
    for _ in range(k):
        h = hashlib.sha256(buf + h).digest()

    return h


def calibrate(target_ms=SERVICE_MS, trials=25):
    """k such that one request takes about target_ms of one core at th
    ↪ e clock the machine runs at now; fixed for the run."""

    t = time.perf_counter(); work_unit(20); per_pass = (time.perf_coun
    ↪ ter() - t) / 20
    k = max(1, int(round(target_ms / 1000.0 / max(per_pass, 1e-6))))
    ts = []
    for _ in range(trials):
        t = time.perf_counter(); work_unit(k); ts.append(time.perf_cou
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ nter() - t)
    return k, statistics.median(ts) * 1000.0


VERDICT_KW = {}            # the brain's verdict as the engine ships i
    ↪ t; the tests shorten its trials here

def _worker(req_q, res_q, k, sim_root, service_ms, seed=0):
    plant = SimPlant.__new__(SimPlant) if sim_root else None
    rng = random.Random(seed)          # the modelled service's own sp
    ↪ read, seeded: the same draws in both arms and in every run

    if plant is not None:
        Plant.__init__(plant, sim_root)
    while True:
        item = req_q.get()
        if item is None:
            break

        rid, t_issue = item
        if plant is not None:
            try:
                factor = plant.service_factor()
            except (OSError, ValueError):
    ↪  # the modelled file mid-replacement: the last factor stands

                factor = getattr(plant, "_last_factor", 1.0)
            plant._last_factor = factor
            time.sleep(service_ms / 1000.0 * factor * rng.uniform(0.95
    ↪ , 1.05))
        else:
            work_unit(k)

        res_q.put((rid, t_issue, time.monotonic()))


class Service(threading.Thread):
    """The CPU-bound service and the open-loop load in front of it: re
    ↪ quests offered at the step's rate whether or not the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    last were answered (equal work in both arms), a pool of one worker
    ↪  per CPU, every request's latency recorded."""

    JITTER = (0.5, 1.5)        # the spacing of requests around the ra
    ↪ te (a Poisson-like spread); the tests narrow it

    def __init__(self, steps, step_s, unit_rps, k, workers, seed, sim_
    ↪ root=None, service_ms=SERVICE_MS):
        super().__init__(daemon=True)
        self.steps, self.step_s, self.unit, self.k, self.n_workers = [
    ↪ int(s) for s in steps.split()], step_s, unit_rps, k, workers
        self.last_done_t = None

        self.rng, self.seed, self.sim_root, self.service_ms = random.R
    ↪ andom(seed), seed, sim_root, service_ms
        self.ctx = mp.get_context("fork") if "fork" in mp.get_all_star
    ↪ t_methods() else mp.get_context()
        self.req_q, self.res_q = self.ctx.Queue(), self.ctx.Queue()
        self.records, self.lock = [], threading.Lock()

        self.issued = self.done = 0
        self.t0 = None
        self.stop_flag = threading.Event()

    def _collect(self):
        while not self.stop_flag.is_set() or self.done < self.issued:

            try:
                rid, t_issue, t_done = self.res_q.get(timeout=0.2)
            except queue.Empty:
                if self.stop_flag.is_set():
                    break
                continue

            with self.lock:
                self.records.append((t_issue - self.t0, t_done - t_iss
    ↪ ue)); self.done += 1; self.last_done_t = t_done

    def run(self):
        procs = [self.ctx.Process(target=_worker, args=(self.req_q, se
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ lf.res_q, self.k, self.sim_root, self.service_ms, (self.seed or
    ↪ 0) * 7919 + i), daemon=True)
                 for i in range(self.n_workers)]   # each worker's mod
    ↪ elled spread seeded from the arm's seed: no unseeded draw anywhe
    ↪ re in the harness
        for p in procs:

            p.start()
        self.t0 = time.monotonic()
        col = threading.Thread(target=self._collect, daemon=True); col
    ↪ .start()
        rid = 0
        for step in self.steps:

            rate = step * self.unit; period = 1.0 / rate
            n_step = int(round(rate * self.step_s))
    ↪          # the step's requests, the same count in both arms
            next_t = time.monotonic()
            for _ in range(n_step):
                while True:

                    now = time.monotonic()
                    if now >= next_t:
                        break
                    time.sleep(min(0.002, next_t - now))
                self.req_q.put((rid, now)); rid += 1; self.issued += 1
                next_t += period * self.rng.uniform(*self.JITTER)

    ↪          # a Poisson-like spread around the rate, the same seed
    ↪ in both arms
        deadline = time.monotonic() + 5.0
    ↪         # the queue drains for at most five seconds
        while self.done < self.issued and time.monotonic() < deadline:
            time.sleep(0.05)

        self.stop_flag.set()
        for _ in procs:
            self.req_q.put(None)
        col.join(timeout=2.0)
        for p in procs:
            p.join(timeout=2.0)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            if p.is_alive():
                p.terminate()

    def last_second(self, line_ms, since_s=1.0):
        """(answered, inside the line, mean latency s) over the last s
    ↪ econd's answered requests."""
        if self.t0 is None:
            return 0, 0, None
        cut = time.monotonic() - self.t0 - since_s
        with self.lock:
            lat = [l for t, l in self.records[-5000:] if t + l >= cut]
        if not lat:
            return 0, 0, None
        return len(lat), sum(1 for l in lat if l * 1000.0 <= line_ms),
    ↪  sum(lat) / len(lat)

    def reading(self, since_s=1.0):
        """What the compass reads: the mean latency of the last second
    ↪ 's answered requests, in seconds; the line when requests
        have waited longer than the line with nothing answered (a serv
    ↪ ice that answers nothing is at the line); nothing
        when nothing is asked."""
        n, _, mean_s = self.last_second(LINE_MS, since_s)
        if mean_s is not None:
            return mean_s
        if self.issued > self.done and self.t0 is not None:
            waited = time.monotonic() - (self.last_done_t or self.t0)
            if waited > LINE_MS / 1000.0:
                return LINE_MS / 1000.0
        return 0.0

    @property
    def failed(self):
        return max(0, self.issued - self.done)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# ------------------------------------------------------------------
    ↪ ---------------------------------- the brain
def decide(p, force, cur, top, floor, notch, last_change_age, wall=0.9
    ↪ 5):
    """The knob's move this second, in kHz: a slow service raises the
    ↪ ceiling by ceil(force / 0.1) notches toward the top;

    calm gives one notch back after the dwell, never under the floor;
    ↪ past the wall the top at once."""
    if p is not None and p >= wall:
        return top, "fail up: the line is at hand, the ceiling to the
    ↪ top at once"
    if force > 0.05:

        return min(top, cur + notch * max(1, math.ceil(force / 0.1))),
    ↪  "the service slow: the ceiling up"
    if force < -0.05 and cur > floor and last_change_age >= DWELL_S:
        return max(floor, cur - notch), "calm: a notch given back"
    return cur, "hold"

def cpu_times():
    with open("/proc/stat") as fh:
        f = fh.readline().split()
    vals = list(map(int, f[1:]))
    return sum(vals), vals[3] + vals[4]

def grid(top, min_khz):
    """The notch and the floor on this machine, from the top clock."""
    notch = max(1000, int(round(top * NOTCH_SHARE / 1000.0)) * 1000)
    floor = max(min_khz, int(top * FLOOR_SHARE))

    n = int((top - floor) // notch)
    return notch, top - n * notch


class Omni(threading.Thread):
    """The brain, one decision a second, writing the audit; the knob i
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ s handed back at the end and read back. The ceiling
    starts in watch and is written only inside the allowance a paired
    ↪ trial on the machine itself has earned under the
    declared objective (tools/knob_verdict.py around the engine's own
    ↪ Verdict); a trial holds the ceiling; a fail-up never
    spends beyond the allowance; toward the operator's ceiling is alwa

    ↪ ys free. The master switch and the lease are honoured."""

    def __init__(self, plant: Plant, svc: Service, audit_path: Path, l
    ↪ ine_ms=LINE_MS, objective=DEFAULT_OBJECTIVE, power=None):
        super().__init__(daemon=True)
        self.plant, self.svc, self.audit, self.line_ms, self.objective

    ↪  = plant, svc, audit_path, line_ms, objective
        self.power = power
    ↪  # a callable giving the last second's package power (W), or Non
    ↪ e
        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE
    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)

        self.stop_flag = threading.Event(); self.last_change = -1e9; s
    ↪ elf.writes = 0; self.failups = 0
        self.foreign = False; self.handed_back = False; self.verdict =
    ↪  None; self.switched_off = False

    def run(self):

        top = self.plant.attach(); notch, floor = grid(self.plant.top_
    ↪ khz(), self.plant.min_khz())
        top = min(top, self.plant.top_khz())
        t0 = time.monotonic()
        self.verdict = KnobVerdict(top, notch, (floor, top), objective
    ↪ =self.objective, **{"settle_s": TAU, **VERDICT_KW})

        with open(self.audit, "w") as fh:
            fh.write(json.dumps({"snapshot": self.plant.snapshot, "top
    ↪ _khz": top, "notch_khz": notch, "floor_khz": floor, "objective":
    ↪  self.objective,
                                 "governors": self.plant.governors(),
    ↪ "driver": self.plant.driver()}) + "\n")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            while not self.stop_flag.is_set():
                tick = time.monotonic()
                try:
                    master.heartbeat()
                    if master.is_off():
                        self.switched_off = True

                        fh.write(json.dumps({"t": round(tick - t0, 2),
↪  "master": "OFF: handing back"}) + "\n")
                        break
                    reading = self.svc.reading(1.0)
                    n_req, inside, mean_s = self.svc.last_second(self.
↪ line_ms)

                    f = self.law.force(reading)
                    cur = self.plant.lever()
                    p_w = self.power() if self.power else None
                    cost = sample_cost(self.objective, inside, mean_s,
↪  p_w if p_w is not None else 1.0, None)   # one machine: energy
↪ x latency / work

                    self.verdict.observe(cost, tick - t0)
                    target, why = decide(self.law.p, f, cur, top, floo
↪ r, notch, tick - self.last_change, wall=self.law.band.wall_high)
                    target, why, vinfo = self.verdict.decide(target, s
↪ pend_ok=False, give_ok=(f < -0.05), t=tick - t0, why=why,
                                                           stress=f

↪ > 0.05, calm=f < -0.05, fail_up=why.startswith("fail up"))
                    wrote = None
                    if target != cur:
                        wrote = self.plant.write(target); self.writes
↪ += 1
                        if target > cur:

                            self.last_change = tick
                        if why.startswith("fail up"):
                            self.failups += 1
                    fh.write(json.dumps({"t": round(tick - t0, 2), "re
↪ ading_ms": round(reading * 1000, 3), "p": round(self.law.p, 4),
↪ "force": round(f, 4),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                                        "ceiling_khz": cur, "target_k
    ↪ hz": target, "wrote_khz": wrote, "requests_last_s": n_req, "insi
    ↪ de_last_s": inside,
                                        "power_w": None if p_w is Non
    ↪ e else round(p_w, 2), "cost": None if cost is None else round(co
    ↪ st, 9),

                                        **vinfo, "why": why}) + "\n")
                except Exception as e:                          # a fore
    ↪ ign writer or a lost file: say so, stop writing
                    fh.write(json.dumps({"t": round(tick - t0, 2), "er
    ↪ ror": repr(e)}) + "\n")
                    self.foreign = True

                    break
                time.sleep(max(0.0, DT - (time.monotonic() - tick)))
        self.handed_back = self.plant.restore()


class Sampler(threading.Thread):

    """Once a second in both arms: the energy counter, the ceiling, th
    ↪ e clock run at, the host's busy share, the wall plug."""

    def __init__(self, plant: Plant, svc: Service, wall_spec=None):
        super().__init__(daemon=True)
        self.plant, self.svc, self.wall_spec = plant, svc, wall_spec

        self.stop_flag, self.rows, self.lock = threading.Event(), [],
    ↪ threading.Lock()
        self.last_power_w = None

    def run(self):
        e_base = e_prev = self.plant.energy_j(); cpu_prev = cpu_times(

    ↪ ); t_prev = time.monotonic()
        while not self.stop_flag.is_set():
            time.sleep(DT)
            now = time.monotonic()
            cpu_now = cpu_times(); d_total = cpu_now[0] - cpu_prev[0];
    ↪  d_idle = cpu_now[1] - cpu_prev[1]; cpu_prev = cpu_now
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            busy = (1.0 - d_idle / d_total) if d_total > 0 else 0.0
            if isinstance(self.plant, SimPlant):
    ↪  # the modelled machine: busy from the service's own work
                n, _, mean_s = self.svc.last_second(LINE_MS)
                busy = min(1.0, n * (self.svc.service_ms / 1000.0) * s
    ↪ elf.plant.service_factor() / max(1, self.svc.n_workers))

                self.plant.tick(busy, now - t_prev)
            e = self.plant.energy_j(); p_w = (e - e_prev) / max(1e-3,
    ↪ now - t_prev); e_prev = e; t_prev = now
            wall_w = None
            if self.wall_spec:
                try:

                    from tools.wall_meter import read as wall_read
                    wall_w = float(wall_read(self.wall_spec))
                except Exception:
                    wall_w = None
            with self.lock:
                self.last_power_w = p_w

                self.rows.append({"t": now, "energy_j": e - e_base, "p
    ↪ ower_w": p_w, "ceiling_khz": self.plant.ceiling_actual(),
                                  "cur_khz": self.plant.cur_khz_mean()
    ↪ , "busy": busy, "wall_w": wall_w})

    def power(self):

        with self.lock:
            return self.last_power_w


# ---------------------------------------------------------------------
    ↪ ---------------------------------- the arms

def probe(plant: Plant, wall_spec=None, out=print):
    """Can this machine run the benchmark? Prints what it has; returns
    ↪  0 when it can, 2 when it cannot (and why)."""
    why = []
    if not plant.cpus:
        why.append("no cpufreq under /sys/devices/system/cpu: the kern
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ el exposes no frequency governor here (a virtual machine, or a k
    ↪ ernel without cpufreq)")
    else:
        out(f"CPUs with a governor: {len(plant.cpus)}; governor(s): {'
    ↪ , '.join(plant.governors())}; driver: {plant.driver()}; "
            f"clock {plant.min_khz() / 1000:.0f} to {plant.top_khz() /
    ↪  1000:.0f} MHz")
        if not plant.writable():
            why.append("the frequency ceiling (scaling_max_freq) is no
    ↪ t writable: run as root on a machine whose driver allows it")
    if not plant.packages:
        why.append("no energy counter under /sys/class/powercap (RAPL)
    ↪ : the processor's meter is not exposed here")
    elif not plant.meter_readable():
        why.append("the energy counter (energy_uj) is not readable: ru
    ↪ n as root")
    else:
        out(f"energy meter: {len(plant.packages)} package(s) under {pl
    ↪ ant.rapl_dir}")
    out(f"wall plug: {wall_spec or 'none (set WALL_METER=<spec> for th
    ↪ e whole machine at the wall)'}")
    if why:
        for w in why:
            out("CANNOT RUN HERE: " + w)
        return 2
    out("this machine can run the benchmark")
    return 0


def run_arm(arm, wl_dir: Path, rep, steps, step_s, line_ms, seed, k, u
    ↪ nit_rps, workers, plant: Plant, objective=DEFAULT_OBJECTIVE, wal
    ↪ l_spec=None,
            sim_root=None):
    d = wl_dir / f"rep-{rep}" / arm; d.mkdir(parents=True, exist_ok=Tr
    ↪ ue)
    svc = Service(steps, step_s, unit_rps, k, workers, seed, sim_root=
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ sim_root)
    sampler = Sampler(plant, svc, wall_spec); sampler.start()
    omni = None
    if arm == "omni":
        master.refuse_if_off("cpu_power")
        snap_path = d / "snapshot.json"

        omni = Omni(plant, svc, d / "audit.jsonl", line_ms, objective,
    ↪  power=sampler.power)
        plant.attach(); snap_path.write_text(json.dumps({"root": str(p
    ↪ lant.root), "snapshot": plant.snapshot}, indent=1))
        master.register("cpu_power", restore=[[sys.executable, str(ROO
    ↪ T / "tools" / "run_cpu_power.py"), "--restore", str(snap_path)]]

    ↪ )
        omni.start()
    cpu0 = cpu_times(); t0 = time.time()
    svc.start(); svc.join()
    t1 = time.time(); cpu1 = cpu_times()
    if omni:

        omni.stop_flag.set(); omni.join(timeout=30)
    sampler.stop_flag.set(); sampler.join(timeout=5)
    seconds = t1 - t0
    with open(d / "requests.log", "w") as fh:
        for t, lat in svc.records:
            fh.write(f"{t:.3f} {lat * 1000:.3f}\n")

    rows = sampler.rows
    with open(d / "samples.jsonl", "w") as fh:
        for r in rows:
            fh.write(json.dumps({k2: (round(v, 3) if isinstance(v, flo
    ↪ at) else v) for k2, v in r.items()}) + "\n")
    lat_sorted = sorted(l * 1000.0 for _, l in svc.records); n = len(l

    ↪ at_sorted)
    def q(p):
        return lat_sorted[min(n - 1, int(p * n))] if n else float("nan
    ↪ ")
    inside = sum(1 for l in lat_sorted if l <= line_ms)
    energy = rows[-1]["energy_j"] if rows else float("nan")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    wall_rows = [r["wall_w"] for r in rows if r.get("wall_w") is not N
    ↪ one]
    wall_j = sum(wall_rows) * DT if wall_rows and len(wall_rows) >= 0.
    ↪ 9 * len(rows) else float("nan")
    total = cpu1[0] - cpu0[0]; idle = cpu1[1] - cpu0[1]
    g = {"arm": arm, "rep": rep, "seconds": round(seconds, 1), "reques
    ↪ ts": n, "issued": svc.issued,
         "work_inside_line_rps": inside / seconds, "rps": n / seconds,
         "p95_ms": q(0.95), "p99_ms": q(0.99), "mean_ms": (sum(lat_sor
    ↪ ted) / n) if n else float("nan"), "failed": svc.failed,
         "energy_j": energy, "power_w_mean": energy / seconds if secon
    ↪ ds else float("nan"), "j_per_1k_inside": 1000.0 * energy / max(1
    ↪ , inside),
         "wall_j": wall_j, "ceiling_mhz_mean": (sum(r["ceiling_khz"] f
    ↪ or r in rows) / len(rows) / 1000.0) if rows else float("nan"),
         "freq_mhz_mean": (sum(r["cur_khz"] for r in rows) / len(rows)
    ↪  / 1000.0) if rows else float("nan"),
         "cpu_busy_share": (total - idle) / max(1, total), "writes": (
    ↪ omni.writes if omni else 0),
         "handed_back": (omni.handed_back if omni else True), "foreign
    ↪ _writer": (omni.foreign if omni else False), "failups": (omni.fa
    ↪ ilups if omni else 0),
         "switched_off": (omni.switched_off if omni else False), "verd
    ↪ ict": (omni.verdict.record() if omni and omni.verdict else None)
    ↪ }
    (d / "arm.json").write_text(json.dumps(g, indent=1))
    print(f"   {arm} rep {rep}: {n} requests, inside the line {g['work
    ↪ _inside_line_rps']:.0f}/s, p95 {g['p95_ms']:.2f} ms, energy {ene
    ↪ rgy:.0f} J "
          f"({g['power_w_mean']:.1f} W), ceiling {g['ceiling_mhz_mean'
    ↪ ]:.0f} MHz, failed {g['failed']}, handed back {g['handed_back']}
    ↪ "
          + (f", verdict {g['verdict']['state']}" if g.get("verdict")
    ↪ else ""), flush=True)
    return g
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def paired(reps, key, direction):
    """omni against native on one gauge over the repetitions: means, t
    ↪ he mean paired difference, its 95% interval."""
    d = [r["omni"][key] - r["native"][key] for r in reps if _num(r["om
    ↪ ni"].get(key)) and _num(r["native"].get(key))]

    if not d:
        return None
    n = len(d)
    nat = sum(r["native"][key] for r in reps if _num(r["native"].get(k
    ↪ ey))) / n
    om = sum(r["omni"][key] for r in reps if _num(r["omni"].get(key)))

    ↪  / n
    mean = sum(d) / n
    if n > 1:
        sd = math.sqrt(sum((x - mean) ** 2 for x in d) / (n - 1))
        half = T95.get(n - 1, 1.96) * sd / math.sqrt(n)
    else:

        half = float("nan")
    lo, hi = mean - half, mean + half
    same = abs(mean) <= SAME_REL * max(abs(nat), 1e-12)
    clear = n > 1 and (lo > 0 or hi < 0) and not same
    if direction == "shown":
        reading = SHOWN

    elif direction == "never more" and any(x > 0 for x in d):
        reading = "**LOSS in a repetition**"
    elif same:
        reading = EVEN_SAME
    elif not clear:
        reading = EVEN_NOISE

    else:
        reading = gel_word(mean < 0, gel_aim(direction, key))      # t
    ↪ he word by what the gauge is for, never by its sign
    return {"native": nat, "omni": om, "diff": mean, "ci95": [lo, hi],
    ↪  "n": n, "significant": clear, "reading": reading}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def _num(x):
    return isinstance(x, (int, float)) and not (isinstance(x, float) a
    ↪ nd math.isnan(x))

def engine():
    try:
        out = subprocess.run([sys.executable, str(ROOT / "tools" / "om
    ↪ ni_version.py")], cwd=ROOT, capture_output=True, text=True, time
    ↪ out=120).stdout.strip()
        sha = subprocess.run(["git", "rev-parse", "HEAD"], cwd=ROOT, c

    ↪ apture_output=True, text=True).stdout.strip()
        return {"version": out.splitlines()[0] if out else "unknown",
    ↪ "commit": sha[:12]}
    except (OSError, subprocess.SubprocessError):
        return {"version": "unknown", "commit": "?"}

def machine(plant: Plant):
    model = "?"
    try:
        for line in open("/proc/cpuinfo"):
            if line.startswith("model name"):

                model = line.split(":", 1)[1].strip(); break
    except OSError:
        pass
    return {"model": model, "cpus": len(plant.cpus), "governors": plan
    ↪ t.governors(), "driver": plant.driver(),
            "top_mhz": plant.top_khz() / 1000.0, "min_mhz": plant.min_

    ↪ khz() / 1000.0, "kernel": os.uname().release, "modelled": isinst
    ↪ ance(plant, SimPlant)}


def run_workload(name, out: Path, reps, step_s, line_ms, plant: Plant,
    ↪  objective=DEFAULT_OBJECTIVE, wall_spec=None, sim_root=None, wor
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ kers=None):
    steps, tuning = WORKLOADS[name]
    wl_dir = out / f"cpu-power-{name}"; wl_dir.mkdir(parents=True, exi
    ↪ st_ok=True)
    workers = workers or len(plant.cpus) or os.cpu_count() or 2
    if sim_root:

        k, measured_ms = 0, SERVICE_MS
    else:
        plant.attach(); plant.write(plant.top_khz())
    ↪      # the work unit is calibrated at the top clock, once, then
    ↪ fixed
        k, measured_ms = calibrate(); plant.restore()

    capacity = workers * 1000.0 / measured_ms
    ↪     # requests a second the machine answers at the top clock
    unit = capacity * UTIL_AT_TOP_STEP / TOP_STEP
    notch, floor = grid(plant.top_khz(), plant.min_khz())
    print(f"== {name}: steps {steps} x {step_s} s, {unit:.1f} requests
    ↪ /s a step (step {TOP_STEP} = {UTIL_AT_TOP_STEP:.0%} of {capacity

    ↪ :.0f}/s), work unit "
          + (f"modelled, {measured_ms:.2f} ms" if sim_root else f"{k}
    ↪ passes = {measured_ms:.2f} ms") + f", {workers} workers; the cei
    ↪ ling's cover "
          f"{floor / 1000:.0f} to {plant.top_khz() / 1000:.0f} MHz by
    ↪ {notch / 1000:.0f} MHz; the verdict's objective {objective}", fl

    ↪ ush=True)
    rec = {"workload": name, "tuning": tuning, "steps": steps, "step_s
    ↪ ": step_s, "line_ms": line_ms, "unit_rps": unit, "work_unit_pass
    ↪ es": k,
           "service_ms_at_top": measured_ms, "workers": workers, "cove
    ↪ r_khz": [floor, plant.top_khz()], "notch_khz": notch, "objective

    ↪ ": objective,
           "wall_meter": bool(wall_spec), "machine": machine(plant), "
    ↪ engine": engine(), "reps": []}
    for rep in range(1, reps + 1):
        order = ("native", "omni") if rep % 2 else ("omni", "native")
        r = {}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        for arm in order:
            r[arm] = run_arm(arm, wl_dir, rep, steps, step_s, line_ms,
    ↪  1000 + rep, k, unit, workers, plant, objective, wall_spec, sim_
    ↪ root)
        rec["reps"].append(r)
        (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1)
    ↪ )
    rec["paired"] = {k2: paired(rec["reps"], k2, dr) for k2, _, dr in
    ↪ GAUGES}
    (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1))
    return rec

def fmt(v):
    if v is None or (isinstance(v, float) and math.isnan(v)):
        return "n/a"
    return f"{v:,.0f}" if abs(v) >= 100 else f"{v:,.3g}"

def report(recs, out: Path):
    L = ["# CPU power: Omni-Compass on top of the Linux kernel's own f
    ↪ requency governor", "",
         "The machine as it runs is native: the kernel's frequency gov
    ↪ ernor as shipped under the operator's ceiling (the top clock). "
         f"Omni is the compass law on the ceiling through the kernel's
    ↪  own interface inside [half the top, the top], holding the servi
    ↪ ce's "
         f"own request latency at {CENTER:.0%} of the {LINE_MS:.0f} ms
    ↪  line, a notch given back only after a paired trial on the machi
    ↪ ne itself "
         "allowed it under the declared objective. The same requests i
    ↪ n both arms. Energy is the processor's own meter (RAPL); the who
    ↪ le "
         "machine at the wall where a plug is fitted. Every row is rep
    ↪ orted, losses included.", ""]
    for rec in recs:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        m = rec.get("machine", {})
        L += [f"## {rec['workload']}: steps {rec['steps']} × {rec['ste
    ↪ p_s']} s" + (" (the tuning workload)" if rec.get("tuning") else
    ↪ ""), "",
              f"{m.get('model', '?')}, {m.get('cpus', '?')} CPUs, gove
    ↪ rnor {', '.join(m.get('governors', []))} ({m.get('driver', '?')}
    ↪ ), kernel {m.get('kernel', '?')}"
              + (" — **a modelled machine, tests only**" if m.get("mod
    ↪ elled") else "") + f"; {rec['unit_rps']:.1f} requests a second a
    ↪  step; the work unit "
              f"{rec['work_unit_passes']} passes = {rec['service_ms_at
    ↪ _top']:.2f} ms at the top clock; the cover {rec['cover_khz'][0]
    ↪ / 1000:.0f} to "
              f"{rec['cover_khz'][1] / 1000:.0f} MHz by {rec['notch_kh
    ↪ z'] / 1000:.0f} MHz; {len(rec['reps'])} paired repetitions; engi
    ↪ ne "
              f"{rec['engine']['version']} at `{rec['engine']['commit'
    ↪ ]}`.", "",
              "| Gauge | native | omni | change | 95% interval of the
    ↪ difference | reading |", "|---|---:|---:|---:|---:|---|"]
        for k, label, direction in GAUGES:
            p = rec.get("paired", {}).get(k)
            if not p:
                continue
            ch = "" if p["native"] == 0 else f"{100 * p['diff'] / abs(
    ↪ p['native']):+.1f}%"
            L.append(f"| {label} | {fmt(p['native'])} | {fmt(p['omni']
    ↪ )} | {ch} | {fmt(p['ci95'][0])} to {fmt(p['ci95'][1])} | {p['rea
    ↪ ding']} |")
        hb = all(r["omni"]["handed_back"] for r in rec["reps"]); fw =
    ↪ any(r["omni"]["foreign_writer"] for r in rec["reps"])
        L += ["", f"Every omni arm handed the ceiling back to the oper
    ↪ ator's and read it back: {'yes' if hb else 'NO'}; another writer
    ↪  seen: "
              f"{'yes' if fw else 'no'}; fail-ups: {sum(r['omni']['fai
    ↪ lups'] for r in rec['reps'])}; the brain's verdict: "
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                + "; ".join(f"rep {r['omni']['rep']}: {r['omni']['verdic
    ↪ t']['state']}" for r in rec["reps"] if r["omni"].get("verdict"))
    ↪  + ".", ""]
    (out / "CPU_POWER.md").write_text("\n".join(_legal_stamp(L)) + "\n
    ↪ ")
    return out / "CPU_POWER.md"

def restore_from(path):
    """Put every CPU's ceiling back from a snapshot file (the watchdog
    ↪ 's hand-back when Omni died)."""
    s = json.loads(Path(path).read_text())

    plant = Plant(s.get("root", "/")); plant.snapshot = s["snapshot"]
    ok = plant.restore()
    print(f"ceiling restored from {path}: {'yes' if ok else 'NO'}")
    return 0 if ok else 1

def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--probe", action="store_true", help="say whether
    ↪ this machine can run the benchmark, and exit")
    ap.add_argument("--plant", default="real", help="real (the machine

    ↪  through /sys) or sim (a modelled machine in a directory, tests
    ↪ only)")
    ap.add_argument("--sim-dir", default="", help="where the modelled
    ↪ machine's files live (default: a fresh temporary directory)")
    ap.add_argument("--workloads", default="tuning")
    ap.add_argument("--reps", type=int, default=3)

    ap.add_argument("--step-s", type=float, default=60.0)
    ap.add_argument("--line-ms", type=float, default=LINE_MS)
    ap.add_argument("--workers", type=int, default=0, help="service wo
    ↪ rkers (default: one per CPU)")
    ap.add_argument("--out", default="cpu-power-out")
    ap.add_argument("--objective", default=DEFAULT_OBJECTIVE, choices=
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ OBJECTIVES,
                   help="what the brain's verdict judges a notch by:
    ↪ resource (energy, speed and work together, the index's reading f
    ↪ or one machine), "
                        "per-work (energy per request inside the line
    ↪ , the line itself the guardrail) or service (speed and work alon
    ↪ e)")
    ap.add_argument("--report-only", default="")
    ap.add_argument("--restore", default="", help="restore the ceiling
    ↪  from this snapshot file and exit")
    a = ap.parse_args(argv)
    if a.restore:
        return restore_from(a.restore)
    if a.report_only:
        recs = [json.loads(f.read_text()) for f in sorted(Path(a.repor
    ↪ t_only).rglob("cpu-power-*/*.json"))]
        print(report(recs, Path(a.report_only))); return 0
    sim_root = None
    if a.plant == "sim":
        import tempfile
        sim_root = a.sim_dir or tempfile.mkdtemp(prefix="omni-cpu-sim-
    ↪ ")
        plant = SimPlant(sim_root)
    else:
        plant = Plant("/")
    wall_spec = os.environ.get("WALL_METER") or None
    rc = probe(plant, wall_spec)
    if a.probe or rc:
        return rc
    out = Path(a.out); out.mkdir(parents=True, exist_ok=True)
    names = list(WORKLOADS) if a.workloads == "all" else a.workloads.s
    ↪ plit(",")
    recs = [run_workload(n, out, a.reps, a.step_s, a.line_ms, plant, a
    ↪ .objective, wall_spec, sim_root, a.workers or None) for n in nam
    ↪ es]
    print(report(recs, out))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

H.51 tools/run_kafka.py

839 lines.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of a Kafka consumer group's operator-set size (
    ↪ docs/KAFKA_PREREGISTRATION.md).

Apache Kafka as shipped (one KRaft broker on the machine, its server.p
    ↪ roperties untouched but for the log directory) carries
a topic of 8 partitions. A producer offers messages at a stepped rate,
    ↪  the same in both arms; a consumer group reads them,
spends a declared service time of CPU on each (the work the messages a
    ↪ re for), and records each message's end-to-end

latency, produce to consume. An operator sets the consumer count once;
    ↪  that fixed number is the native controller here.

  native  the consumer group at the operator's count (2 consumers) for
    ↪  the whole run
  omni    the same group, with the compass law (omnicompass/compass_la
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ w.py) on one knob, the consumer count, inside the
          cover [1, 8] (never under one consumer, never more consumers
    ↪  than partitions): the compass reads the group's own
          end-to-end latency on a band from 0 to the response line and
    ↪  holds it at 40% of the line; the service slow and
          messages waiting add consumers; calm gives back one idle con

    ↪ sumer a second (above the operator's count, calm with
          nothing waiting gives one back with no idle test; idle is a
    ↪ full second with no message: store fix of 25 October);
          at 95% of the line every consumer the
          topic can use is started at once (fail up); the count is han
    ↪ ded back to the operator's at the end and read back

Gauges from the consumers' own records (every message's latency), the
    ↪ group's lag (produced minus consumed) and the
host's /proc/stat (CPU seconds, the consumers' and the compass's own c
    ↪ ost, counted against Omni). No energy is claimed
beyond the host's CPU seconds. Every row is reported, losses included.

  python3 tools/run_kafka.py --setup                      # start a br
    ↪ oker and make the topics (the workflow does this)
  python3 tools/run_kafka.py --workloads tuning --reps 3 --out DIR
  python3 tools/run_kafka.py --report-only DIR            # re-read fi
    ↪ nished <workload>.json files and write KAFKA.md

"""
from __future__ import annotations

import argparse
import json
import math

import multiprocessing as mp
import os
import statistics
import subprocess
import sys
import threading
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
import time
from collections import deque
from pathlib import Path

try:                                                    # the legal
    ↪  notice every generated report carries

    from tools.legal import stamp as _legal_stamp
except ImportError:
    import pathlib as _p; sys.path.insert(0, str(_p.Path(__file__).res
    ↪ olve().parents[1])); from tools.legal import stamp as _legal_sta
    ↪ mp
ROOT = Path(__file__).resolve().parents[1]

sys.path.insert(0, str(ROOT))
from omnicompass.compass_law import Band, CompassLaw  # noqa: E402
from omnicompass.immune import Immune  # noqa: E402  (omni-v22: the im
    ↪ mune system's self-knowledge on the group's knob)
from tools.knob_verdict import KnobVerdict, sample_cost, OBJECTIVES, R
    ↪ ESOURCE as DEFAULT_OBJECTIVE  # noqa: E402  (amendment 2: the br

    ↪ ain's own verdict)
from tools.readings import word as gel_word, aim as gel_aim, EVEN_SAME
    ↪ , EVEN_NOISE, SHOWN  # noqa: E402  (the words every table uses)

BOOTSTRAP = os.environ.get("KAFKA_BOOTSTRAP", "localhost:9092")
PARTITIONS = 8

LINE_MS = 500.0          # the response line: a message handled withi
    ↪ n 500 ms of being produced
CENTER = 0.4             # the compass holds the service at 40% of th
    ↪ e line
DT = 1.0                 # one decision a second
TAU = 3.0                # a consumer added or removed shows in the l

    ↪ atency within a few seconds (the group rebalances)
SMOOTH = 0.5
NATIVE_CONSUMERS = 2     # the operator's count
COVER = (1, PARTITIONS)  # never under one consumer, never more than
    ↪ partitions
DWELL_S = 5.0            # no taking back within five seconds of a ch
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ange
DIAL = os.environ.get("OMNI_DIAL", "max")   # omni-v22: max, the facto
    ↪ ry tune; save, the tune-down
POLL_RECORDS = 50         # the most messages a consumer takes in one
    ↪ poll (KafkaConsumer max_poll_records)
HEARTBEAT_MS = 3000       # the consumers' heartbeat (KafkaConsumer he

    ↪ artbeat_interval_ms, the client's own default): a member learns
    ↪ of a
                          # rebalance at its next heartbeat, so a chan
    ↪ ge reaches every member within one interval
SESSION_MS = 10000        # the consumers' session timeout (KafkaConsu
    ↪ mer session_timeout_ms, the client's own default)

HANDBACK_DWELL_S = 30.0   # store fix of 25 October: a consumer above
    ↪ the operator's count is handed back only 30 s after the last cha
    ↪ nge (each
                          # change rebalances the whole group, and a h
    ↪ and-back that is undone at once is two pauses for nothing)
IDLE_S = 1.0              # store fix of 25 October: a consumer is idl

    ↪ e only after a full second with no message (a sustained state, n
    ↪ ever one
                          # empty 200 ms poll), and a consumer starts
    ↪ busy: the omni-v16 runs read a momentary empty poll as idle, whi
    ↪ ch made a
                          # 2 -> 1 -> 2 give-back trial and its fail-u

    ↪ p (two rebalances) in the first 11 seconds of every omni arm, an
    ↪ d with 8
                          # partitions keyed round-robin no consumer a
    ↪ bove the operator's count was ever idle, so none was ever given
    ↪ back
STEPS = "1 2 3 2 3 4 5 6 5 4 3 2 1 2 1"

BURST = "1 6 1 8 1 6"
PEAK_SHARE = 0.9          # the peak notch offers nine tenths of nativ
    ↪ e's measured capacity
from omnicompass.student_t import T95  # noqa: E402  (the two-sided 95
    ↪ % point of Student's t, any degrees of freedom)
SAME_REL = 1e-6
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
WORKLOADS = {
    # name: (service ms of CPU per message, payload bytes, steps, tuni
    ↪ ng?)
    "tuning": (2.0, 256, STEPS, True),
    "light": (1.0, 128, STEPS, False),

    "heavy": (5.0, 1024, STEPS, False),
    "burst": (2.0, 256, BURST, False),
}

GAUGES = [  # key, label, direction
    ("work_inside_line_mps", "work inside the response line (messages

    ↪ a second handled within the line)", "higher"),
    ("mps", "throughput (messages a second consumed)", "higher"),
    ("p95_ms", "end-to-end latency, 95th percentile (ms)", "lower"),
    ("p99_ms", "end-to-end latency, 99th percentile (ms)", "lower"),
    ("p50_ms", "end-to-end latency, median (ms)", "lower"),
    ("mean_ms", "end-to-end latency, mean (ms)", "lower"),

    ("lag_max", "consumer lag, most messages waiting at once", "lower"
    ↪ ),
    ("lag_mean", "consumer lag, mean messages waiting", "lower"),
    ("lost", "messages produced and never consumed", "never more"),
    ("consumers_mean", "consumers running, mean (the resources held)",
    ↪  "spent"),

    ("consumers_max", "consumers running, most at once", "spent"),
    ("cpu_busy_share", "host CPU busy (share of the run)", "spent"),
    ("cpu_seconds", "host CPU-seconds", "spent"),
    ("cpu_s_per_1k_inside", "host CPU-seconds per 1,000 messages insid
    ↪ e the line", "spent"),
    ("rebalances", "consumer changes written (the knob's moves)", "sho

    ↪ wn"),
]


# ------------------------------------------------------------------ t
    ↪ he producer and the consumers
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def busy_ms(ms):
    """Spend `ms` of CPU on a message (the work the message is for)."""
    end = time.perf_counter() + ms / 1000.0
    x = 0.0
    while time.perf_counter() < end:
        x = math.sqrt(x + 1.0)
    return x


def producer_proc(topic, rate_steps, step_s, payload_bytes, produced, stop, log_path):
    """Offer messages at the stepped rate; every message carries its produce time (ns, the machine's monotonic clock)."""
    from kafka import KafkaProducer
    p = KafkaProducer(bootstrap_servers=BOOTSTRAP, linger_ms=5, acks=1)
    pad = b"x" * max(0, payload_bytes - 40)
    seq = 0
    with open(log_path, "w") as log:
        for rate in rate_steps:
            t_end = time.perf_counter() + step_s
            interval = 1.0 / max(rate, 1e-9)
            nxt = time.perf_counter()
            log.write(f"{time.time():.3f} rate {rate:.1f}\n"); log.flush()
            while time.perf_counter() < t_end and not stop.is_set():
                now = time.perf_counter()
                if now >= nxt:
                    p.send(topic, key=str(seq % PARTITIONS).encode(), value=f"{seq}:{time.perf_counter_ns()}:".encode() + pad)
                    seq += 1; nxt += interval
                    with produced.get_lock():
                        produced.value = seq
                else:
                    time.sleep(min(0.001, nxt - now))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            if stop.is_set():
                break
        p.flush(); p.close()


def consumer_proc(topic, group, service_ms, consumed, stop_evt, rec_pa
    ↪ th, idle_flag):
    """One consumer of the group: poll, spend the service time per mes
    ↪ sage, record (consume time, latency ms)."""
    from kafka import KafkaConsumer
    c = KafkaConsumer(topic, bootstrap_servers=BOOTSTRAP, group_id=gro
    ↪ up, auto_offset_reset="earliest",
                      enable_auto_commit=True, max_poll_records=POLL_R
    ↪ ECORDS, session_timeout_ms=SESSION_MS, heartbeat_interval_ms=HEA
    ↪ RTBEAT_MS)
    n = 0
    last_msg = time.monotonic()                                  # a co
    ↪ nsumer starts busy: idle only after IDLE_S with no message
    with open(rec_path, "a") as rec:
        while not stop_evt.is_set():
            batch = c.poll(timeout_ms=200)
            got = 0
            for msgs in batch.values():
                for m in msgs:
                    seq, t_ns, _ = m.value.split(b":", 2)
                    busy_ms(service_ms)
                    lat_ms = (time.perf_counter_ns() - int(t_ns)) / 1e
    ↪ 6
                    rec.write(f"{time.time():.3f} {seq.decode()} {lat_
    ↪ ms:.3f}\n")
                    got += 1
            if got:
                n += got; rec.flush()
                with consumed.get_lock():
                    consumed.value += got
            if got:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                last_msg = time.monotonic()
            idle_flag.value = 1 if is_idle(last_msg, time.monotonic())
    ↪  else 0
    c.close()

def is_idle(last_msg_t, now, idle_s=IDLE_S):
    """A consumer is idle when no message has reached it for idle_s se
    ↪ conds or more (a sustained state, store fix of 25 October)."""
    return now - last_msg_t >= idle_s

class Group:
    """The consumer group as a fleet of processes: start one, stop one
    ↪ , how many run, which are idle."""

    def __init__(self, topic, group, service_ms, out: Path):
        self.topic, self.group, self.service_ms, self.out = topic, gro
    ↪ up, service_ms, out
        self.consumed = mp.Value("l", 0); self.procs = []; self.k = 0;
    ↪  self.changes = 0

    def start_one(self):
        stop = mp.Event(); idle = mp.Value("i", 0)                # a
    ↪  consumer starts busy (store fix of 25 October)
        self.k += 1
        p = mp.Process(target=consumer_proc, args=(self.topic, self.gr
    ↪ oup, self.service_ms, self.consumed, stop, str(self.out / f"cons
    ↪ umer-{self.k}.log"), idle), daemon=True)
        p.start(); self.procs.append((p, stop, idle)); self.changes +=
    ↪  1

    def stop_one(self, prefer_idle=True):
        if not self.procs:
            return False
        order = sorted(range(len(self.procs)), key=lambda i: -self.pro
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ cs[i][2].value) if prefer_idle else range(len(self.procs))
        i = list(order)[0]
        p, stop, _ = self.procs.pop(i); stop.set(); p.join(timeout=15)
    ↪ ; self.changes += 1
        return True

    def count(self):
        return len(self.procs)


    def idle(self):
        return sum(1 for _, _, f in self.procs if f.value)

    def stop_all(self):
        for p, stop, _ in self.procs:
            stop.set()
        for p, _, _ in self.procs:
            p.join(timeout=15)
        self.procs = []

# ------------------------------------------------------------------ t
    ↪ he compass on the consumer count
def service_sample(rec_files, since_s, line_ms=LINE_MS):
    """The brain's sample from the consumers' own records: (the mean e

    ↪ nd-to-end latency in s of the messages consumed in the
    last second, or None when none was; how many were consumed; how ma
    ↪ ny of them inside the line)."""
    now = time.time(); lat = []
    for f in rec_files:
        try:

            with open(f) as fh:
                fh.seek(0, 2); size = fh.tell(); fh.seek(max(0, size -
    ↪  65536))
                for ln in fh.read().splitlines()[-400:]:
                    t, _, l = ln.split(" ")
                    if float(t) >= now - since_s:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                        lat.append(float(l) / 1000.0)
        except (OSError, ValueError):
            pass
    if not lat:
        return None, 0, 0
    return sum(lat) / len(lat), len(lat), sum(1 for l in lat if l * 10
    ↪ 00.0 <= line_ms)


def service_reading(rec_files, since_s):
    """The group's own reading: the mean end-to-end latency (s) of the
    ↪  messages consumed in the last second; nothing

    consumed reads as the lag's age (messages waiting with no one taki
    ↪ ng them), capped at the line."""
    return service_sample(rec_files, since_s)[0]


def decide(p, force, lag, idle, cur, last_change_age, wall=0.95, snaps
    ↪ hot=None, consumed=None):
    """The knob's move this second: (target, direction, why). The serv
    ↪ ice slow with messages waiting adds consumers,
    ceil(force / 0.1) at a time; calm gives back one idle consumer a s
    ↪ econd, after the dwell; past the wall, every
    consumer the topic can use at once.

    Store fix of 25 October: above the operator's own count (snapshot)
    ↪ , calm with no more waiting than the group takes in one
    poll round (POLL_RECORDS a consumer: the messages in flight betwee
    ↪ n the producer and the consumers, which keep the lag
    above zero in 388 of 389 calm seconds of an omni-v16 arm) gives on
    ↪ e consumer back, HANDBACK_DWELL_S after the last change,

    with no idle test (every consumer of a topic keyed round-robin has
    ↪  work, so the idle test held a consumer added above
    native to the end of the arm). Below the operator's count the idle
    ↪  test and an empty backlog stay. A second in which the
    group consumed nothing (consumed 0: before the producer starts, or
    ↪  a rebalance) says nothing about the demand: nothing
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    is given back in it."""
    lo, hi = COVER
    if p is not None and p >= wall:
        return hi, 0, "fail up: the line is at hand, every consumer at
    ↪  once"
    if force > 0.05 and lag > 0:

        want = min(hi, cur + max(1, math.ceil(force / 0.1)))
        return want, (1 if want != cur else 0), f"slow with {lag} wait
    ↪ ing: add {want - cur}"
    if consumed is not None and consumed <= 0:
        return cur, 0, "nothing consumed in the last second: nothing k
    ↪ nown, nothing taken"

    if (snapshot is not None and cur > snapshot and force < -0.05 and
    ↪ lag <= cur * POLL_RECORDS
            and last_change_age >= HANDBACK_DWELL_S):
        return cur - 1, -1, "calm, nothing waiting beyond one poll rou
    ↪ nd, above the operator's count: one consumer given back"
    if force < -0.05 and idle > 0 and cur > lo and last_change_age >=

    ↪ DWELL_S and lag == 0 and (snapshot is None or cur <= snapshot):
        return cur - 1, -1, "calm, an idle consumer given back"
    return cur, 0, "hold"


# omni-v22: the group's twin and the immune reflex (docs/KAFKA_PREREGI

    ↪ STRATION.md, amendment 4; docs/STORE_ROOT_CAUSES_2026-10-30.md).
# On omni-v21 the verdict's allowance clamped every move, the wall's f
    ↪ ail-up included, and the wall abandoned every spend trial, so in
# 1,413 of the 1,428 seconds the service stood over its line the brain
    ↪  asked for consumers and was refused; and the hand-back gave
# a consumer back that the load still needed, each give-back and re-ad

    ↪ d a rebalance of the whole group (a median 2.58 s and 1.24 s
# of every message waiting). The twin knows which count carries the lo
    ↪ ad; the reflex goes there and holds it.
RHO_SPEND = 0.9           # the twin spends to the smallest count whos
    ↪ e busiest member runs at or under nine tenths of its rate
RHO_HAND = 0.7            # it hands back only to a count whose busies
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ t member stays at or under seven tenths at the window's peak
HANDBACK_WINDOW_S = 60.0  # the window the hand-back reads the arrival
    ↪ s' peak over
DRAIN_S = 2 * TAU         # a backlog the twin finds, and the one a ch
    ↪ ange's own rebalance makes, is drained within this
MU_KEEP = 15              # the member's rate: the median of the twin'
    ↪ s last 15 estimates
STALL_KEEP = 9            # a change's own stall: the median of the la
    ↪ st 9 the group showed


class RateTwin:
    """The consumer group's twin: each partition's arrivals (the topic
    ↪ 's log-end offsets, once a second) placed on the members by
    Kafka's range assignor, and one member's rate learned from the gro
    ↪ up's own seconds. In a second in which the queue grew
    and no change was settling, the group consumed sum(min(load_i, mu)
    ↪ ) over its members (the members over their rate serve
    mu, the others their load), solved for mu. The stall a change cost
    ↪ s is learned the same way: the seconds after a change in
    which the group consumed under half its rate before it. Nothing in
    ↪  it is a number written for a workload."""

    def __init__(self, partitions=PARTITIONS):
        self.partitions = partitions
        self.mu_est = deque(maxlen=MU_KEEP)
        self.stalls = deque(maxlen=STALL_KEEP)
        self.watch = None                       # (time of the change,
    ↪  the rate consumed before it, seconds stalled so far)

    @property
    def mu(self):
        return statistics.median(self.mu_est) if self.mu_est else None

    @property
    def stall_s(self):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    """A change's own stall: the median the group showed, or, befo
↪ re it has shown one, the protocol's bound (a rebalance
    reaches every member within one heartbeat interval, the consum
↪ ers' own setting)."""
    return statistics.median(self.stalls) if self.stalls else HEAR
↪ TBEAT_MS / 1000.0

def ranges(self, c):
    c = max(1, min(c, self.partitions))
    n, extra = divmod(self.partitions, c)
    out, start = [], 0
    for i in range(c):

        size = n + (1 if i < extra else 0)
        out.append(range(start, start + size)); start += size
    return out


def loads(self, c, arrivals, backlog=None):
    """Each member's load (messages a second) at count c: its part

↪ itions' arrivals, and a backlog to drain within DRAIN_S."""
    total = sum(arrivals) or 1.0
    out = []
    for r in self.ranges(c):
        a = sum(arrivals[p] for p in r)
        if backlog:

            a += backlog * (a / total) / DRAIN_S       # the backl
↪ og sits where the messages arrive
        out.append(a)
    return out


def learn(self, c, arrivals, consumed):

    loads = self.loads(c, arrivals)
    total = sum(loads)
    if consumed <= 0 or consumed >= 0.98 * total:
        return None
    lo, hi = 0.0, max(loads)
    for _ in range(48):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            mid = (lo + hi) / 2
            if sum(min(x, mid) for x in loads) < consumed:
                lo = mid
            else:
                hi = mid
        mu = (lo + hi) / 2

        if mu > 0:
            self.mu_est.append(mu)
        return mu

    def changed(self, t, consumed_before):
        self.watch = [t, consumed_before, 0]

    def after_change(self, t, consumed):
        """Counts the seconds after a change the group consumed under
    ↪ half its rate before it: the change's own stall."""
        if self.watch is None:
            return

        t0, before, stalled = self.watch
        if before > 0 and consumed < 0.5 * before and t - t0 <= 10:
            self.watch[2] = stalled + 1
            return
        if t - t0 >= 1:
            self.stalls.append(float(stalled)); self.watch = None

    def busiest(self, c, arrivals, backlog=0.0):
        mu = self.mu
        return None if mu is None else max(self.loads(c, arrivals, bac
    ↪ klog)) / mu

    def needed(self, arrivals, lo, hi, rho, backlog=0.0):
        """The smallest count in [lo, hi] whose busiest member runs at
    ↪  or under rho; where none does, the smallest count with the
        least busy busiest member (a count that adds no relief is neve
    ↪ r chosen over a smaller one)."""
        best = None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        for c in range(lo, hi + 1):
            b = self.busiest(c, arrivals, backlog)
            if b is None:
                return None
            if b <= rho:
                return c

            if best is None or b < best[0] - 1e-9:
                best = (b, c)
        return best[1]


class Brain:

    """The knob's decision, one a second (omni-v22): the same code the
    ↪  real run (Omni) and the closed-loop model
    (tools/kafka_model.py) drive. obs: reading (s, or None), n_msg, in
    ↪ side, lag, idle, consumers, cpu_share and arrivals (each
    partition's messages a second, or None while unread).

      the reflex   with the twin's rate known: the count cannot carry
    ↪ the arrivals and the backlog (its own and the one a change's
                   stall would add) at nine tenths of its rate, so the
    ↪  knob goes at once to the smallest count that can, up to the
                   partitions; the immune system's reflex, never clamp
    ↪ ed by the verdict's allowance (the allowance judges the

                   compass's own moves). At the wall with no rate know
    ↪ n, the compass's fail-up goes one consumer up, and the
                   twin learns the rate from the second it sees the qu
    ↪ eue grow
      the hold     above the operator's count the knob stays where the
    ↪  reflex put it; it is handed back, in one move (one

                   rebalance), only to the smallest count at or above
    ↪ the operator's whose busiest member stays at or under seven
                   tenths of its rate at the arrivals' peak of the las
    ↪ t 60 s, 30 s after the last change, calm, nothing waiting
                   beyond one poll round a consumer
      the floor    never below the operator's count (the max, the fact
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ory tune)
      the immune   Omni-Compass's count redlining past the line, stuck
    ↪ , and native's twin (the operator's count on the same
                   arrivals) reading lower: the knob goes back to nati
    ↪ ve for the immune hold (omnicompass/immune.py)
      the rest     the compass law and the verdict as on omni-v21, ins
    ↪ ide the allowance, never under the reflex's count"""

    def __init__(self, snapshot, objective=DEFAULT_OBJECTIVE, line_ms=
    ↪ LINE_MS):
        self.snapshot = snapshot; self.objective = objective; self.lin
    ↪ e_s = line_ms / 1000.0
        self.law = CompassLaw(Band(0.0, self.line_s, center=CENTER), d
    ↪ t=DT, tau=TAU, smooth=SMOOTH)
        self.verdict = KnobVerdict(snapshot, 1, COVER, objective=objec
    ↪ tive, settle_s=2 * TAU)   # a consumer that joins rebalances the
    ↪  group first
        self.twin = RateTwin()
        self.immune = Immune()
        self.window = deque()                    # (t, arrivals) over
    ↪ HANDBACK_WINDOW_S
        self.last_change = -1e9; self.prev_lag = None; self.prev_consu
    ↪ med = 0; self.failups = 0

    def step(self, t, obs):
        lo, hi = max(COVER[0], self.snapshot), COVER[1]
        reading = obs["reading"]; lag, n_msg, cur, idle = obs["lag"],
    ↪ obs["n_msg"], obs["consumers"], obs["idle"]
        if reading is None:
            reading = min(self.line_s, 0.0 if lag == 0 else self.line_
    ↪ s)        # waiting with no one consuming: the line
        f = self.law.force(reading)
        cost = sample_cost(self.objective, obs["inside"], reading if n
    ↪ _msg else None, cur, obs["cpu_share"])
        self.verdict.observe(cost, t)
        arrivals = obs.get("arrivals")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        settling = t - self.last_change < 2 * TAU
        self.twin.after_change(t, n_msg)
        if arrivals is not None:
            self.window.append((t, list(arrivals)))
            while self.window and t - self.window[0][0] > HANDBACK_WIN
    ↪ DOW_S:

                self.window.popleft()
            grew = self.prev_lag is not None and lag > self.prev_lag
            if grew and not settling:
                self.twin.learn(cur, arrivals, n_msg)
        self.prev_lag = lag
        info = {"mu": None if self.twin.mu is None else round(self.twi

    ↪ n.mu, 1), "stall_s": round(self.twin.stall_s, 2)}
        target, why = None, None
        if arrivals is not None and self.twin.mu is not None:
            rate = sum(arrivals)
            backlog = lag + self.twin.stall_s * rate             # wha
    ↪ t waits now, and what one change's stall would add

            busy_now = self.twin.busiest(cur, arrivals, lag)
            info["busiest"] = round(busy_now, 3)
            if busy_now > RHO_SPEND and not settling:
                want = self.twin.needed(arrivals, max(lo, cur), hi, RH
    ↪ O_SPEND, backlog)
                if want is not None and want > cur:

                    target, why = want, f"the twin: the busiest member
    ↪  at {busy_now:.2f} of its rate; {want} consumers carry it"
            elif cur > lo and not settling and t - self.last_change >=
    ↪  HANDBACK_DWELL_S and f < -0.05 and lag <= cur * POLL_RECORDS:
                peak = [max(a[p] for _, a in self.window) for p in ran
    ↪ ge(PARTITIONS)]

                want = self.twin.needed(peak, lo, cur, RHO_HAND, 0.0)
                if want is not None and want < cur:
                    target, why = want, f"calm, and {want} consumers c
    ↪ arry the last {HANDBACK_WINDOW_S:.0f} s's peak at {RHO_HAND} of
    ↪ their rate: handed back in one move"
        if target is None:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        d_target, _, d_why = decide(self.law.p, f, lag, idle, cur,
↪  t - self.last_change, wall=self.law.band.wall_high,
                                    snapshot=self.snapshot, consum
↪ ed=n_msg)
        fail_up = d_why.startswith("fail up")
        if fail_up and self.twin.mu is None:
            target, why = min(hi, cur + 1), "fail up, the twin's r
↪ ate not yet known: one consumer up"
            self.failups += 1
        else:
            v_target, why, vinfo = self.verdict.decide(d_target, s
↪ pend_ok=(f > 0.05 and lag > 0), give_ok=False, t=t, why=d_why,
                                                        stress=f >
↪ 0.05, calm=f < -0.05, fail_up=fail_up)
            info.update(vinfo)
            target = max(lo, v_target)
            if cur > lo and target < cur:
                target, why = cur, f"{why}; held at the reflex's c
↪ ount until the hand-back's terms are met"
            if target > cur and (settling or (arrivals is not None
↪  and self.twin.mu is not None)):
                # a change's own stall is not demand, and where th
↪ e twin knows the group only the twin adds: the compass's
                # add, read on the seconds a hand-back's rebalance
↪  stalled the group, re-added the consumer just handed back
                # (the model, heavy, seeds 105 and 108: 2 -> 3 ->
↪ 2 every 30 s)
                target, why = cur, f"{why}; the twin carries the s
↪ pend: held"
    # the immune system's self-knowledge: Omni-Compass's count red
↪ lining and stuck, native's twin reading lower
    p_omni = reading / self.line_s
    p_native = p_omni
    if arrivals is not None and self.twin.mu is not None:
        b_nat = self.twin.busiest(self.snapshot, arrivals, lag)
        b_cur = self.twin.busiest(cur, arrivals, lag)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            p_native = p_omni * (b_nat / b_cur) if b_cur else p_omni
        mode, ev = self.immune.tick(p_omni, p_native, moved=target !=
    ↪ cur, at_limit=cur >= hi)
        if mode == "native":
            target, why = self.snapshot, "the immune system: Omni-Comp
    ↪ ass's count redlined, stuck, and native's twin reads lower: nati
    ↪ ve holds the knob"
        if ev:
            info["immune"] = ev
        if target != cur:
            self.twin.changed(t, n_msg)
            self.last_change = t
        info.update({"p": round(self.law.p, 4), "force": round(f, 4)})
        return target, why, info


class Offsets:
    """The topic's log-end offset of each partition, read from the bro
    ↪ ker once a second (one wire in): each partition's arrivals,
    as any deployment reads them (the consumer group tool's LOG-END-OF
    ↪ FSET)."""

    def __init__(self, topic):
        from kafka import KafkaConsumer, TopicPartition
        self.c = KafkaConsumer(bootstrap_servers=BOOTSTRAP, group_id=N
    ↪ one, enable_auto_commit=False)
        self.tps = [TopicPartition(topic, p) for p in range(PARTITIONS
    ↪ )]
        self.last, self.last_t = None, None
    def rates(self, now):
        try:
            end = self.c.end_offsets(self.tps)
        except Exception:
            return None
        cur = [end.get(tp, 0) for tp in self.tps]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        out = None
        if self.last is not None and now > self.last_t:
            out = [max(0.0, (a - b) / (now - self.last_t)) for a, b in
    ↪  zip(cur, self.last)]
        self.last, self.last_t = cur, now
        return out

    def close(self):
        try:
            self.c.close()
        except Exception:
            pass

class Omni(threading.Thread):
    """The brain, one decision a second, writing the audit; the knob i
    ↪ s handed back at the end and read back. Amendment 2 (the
    brain's own verdict, 2026-10-09): the compass's own moves are writ

    ↪ ten only inside the allowance a paired trial on the group
    itself has earned under the declared objective (tools/knob_verdict
    ↪ .py around the engine's own Verdict). From omni-v22 the
    decision is the Brain's (above): the twin's reflex and hold, the f
    ↪ loor at the operator's count, the immune system."""

    def __init__(self, grp: Group, produced, audit_path: Path, line_ms
    ↪ =LINE_MS, objective=DEFAULT_OBJECTIVE, topic=None, dial=DIAL):
        super().__init__(daemon=True)
        self.grp, self.produced, self.audit, self.line_ms, self.object
    ↪ ive, self.topic, self.dial = grp, produced, audit_path, line_ms,
    ↪  objective, topic, dial

        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE
    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)   # the tune-down's own law
        self.stop_flag = threading.Event(); self.failups = 0; self.han
    ↪ ded_back = False; self.verdict = None; self.brain = None; self.l
    ↪ ast_change = -1e9
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def govern_record(self):
        """The record of what governed the knob: the max's twin and ve
    ↪ rdict (omni-v22) or the verdict alone (the tune-down)."""
        return self.verdict.record() if self.verdict else None

    def run(self):

        snapshot = self.grp.count()                          # read once
    ↪ , before the first write: the operator's count
        t0 = time.monotonic(); cpu_prev = cpu_times()
        self.verdict = KnobVerdict(snapshot, 1, COVER, objective=self.
    ↪ objective, settle_s=2 * TAU)   # a consumer that joins rebalance
    ↪ s the group first

        if self.dial == "max":
            self.brain = Brain(snapshot, self.objective, self.line_ms)
    ↪ ; self.verdict = self.brain.verdict
        offsets = None
        if self.topic:
            try:

                offsets = Offsets(self.topic)
            except Exception:
                offsets = None                               # unread ar
    ↪ rivals: the twin waits, the compass decides as on omni-v21
        with open(self.audit, "w") as fh:
            while not self.stop_flag.is_set():

                tick = time.monotonic()
                recs = [self.grp.out / f"consumer-{k}.log" for k in ra
    ↪ nge(1, self.grp.k + 1)]
                lag = max(0, self.produced.value - self.grp.consumed.v
    ↪ alue)
                reading, n_msg, inside = service_sample(recs, 1.0, sel

    ↪ f.line_ms)
                cur = self.grp.count(); idle = self.grp.idle()
                cpu_now = cpu_times(); d_total = cpu_now[0] - cpu_prev
    ↪ [0]; d_idle = cpu_now[1] - cpu_prev[1]; cpu_prev = cpu_now
                cpu_share = (1.0 - d_idle / d_total) if d_total > 0 el
    ↪ se None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            arrivals = offsets.rates(tick) if offsets else None
            shown = reading if reading is not None else min(LINE_M
↪ S / 1000.0, 0.0 if lag == 0 else LINE_MS / 1000.0)
            if self.brain is not None:
↪                   # the max (omni-v22): the twin, the reflex, t
↪ he hold

                obs = {"reading": reading, "n_msg": n_msg, "inside
↪ ": inside, "lag": lag, "idle": idle, "consumers": cur,
                       "cpu_share": cpu_share, "arrivals": arrival
↪ s}
                target, why, info = self.brain.step(tick - t0, obs
↪ )

                fail_up = False
            else:
↪                   # the tune-down: omni-v21's loop
                f = self.law.force(shown)
                cost = sample_cost(self.objective, inside, shown i
↪ f n_msg else None, cur, cpu_share)

                self.verdict.observe(cost, tick - t0)
↪                    # measured under the count of the last secon
↪ d
                target, d, why = decide(self.law.p, f, lag, idle,
↪ cur, tick - self.last_change, wall=self.law.band.wall_high, snap
↪ shot=snapshot,

                                        consumed=n_msg)
                fail_up = why.startswith("fail up")
                # a give-back above the operator's count is the ha
↪ nd-back of a spend, inside the allowance: no give-back trial (wh
↪ ose
                # steps run below native) is started for it

                target, why, info = self.verdict.decide(target, sp
↪ end_ok=(f > 0.05 and lag > 0), give_ok=(f < -0.05 and idle > 0 a
↪ nd lag == 0 and cur <= snapshot and n_msg > 0),

                                                        t=tick - t
↪ 0, why=why, stress=f > 0.05, calm=f < -0.05, fail_up=fail_up)
                info.update({"p": round(self.law.p, 4), "force": r
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ound(f, 4), "cost": None if cost is None else round(cost, 9), "d
    ↪ ial": "save"})
                if target > cur:
                    for _ in range(target - cur):
                        self.grp.start_one()
                    self.last_change = tick

                    if fail_up:
                        self.failups += 1
                elif target < cur:
                    for _ in range(cur - target):
                        self.grp.stop_one()
                    self.last_change = tick

                fh.write(json.dumps({"t": round(tick - t0, 2), "readin
    ↪ g_ms": round(shown * 1000, 2), "lag": lag, "idle": idle,
                                     "consumers": cur, "target": targe
    ↪ t, "consumed_last_s": n_msg, "inside_last_s": inside,
                                     "cpu_share": None if cpu_share is
    ↪  None else round(cpu_share, 4),

                                     "arrivals": None if arrivals is N
    ↪ one else [round(x, 1) for x in arrivals],
                                     "allowed_low": self.verdict.allow
    ↪ ed_low, "allowed_high": self.verdict.allowed_high,
                                     **info, "why": why}) + "\n")
                time.sleep(max(0.0, DT - (time.monotonic() - tick)))

        if self.brain is not None:
            self.failups = self.brain.failups
        if offsets:
            offsets.close()
        # the hand-back: the operator's count, read back
        while self.grp.count() > snapshot:

            self.grp.stop_one(prefer_idle=False)
        while self.grp.count() < snapshot:
            self.grp.start_one()
        self.handed_back = self.grp.count() == snapshot
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class Sampler(threading.Thread):
    """Consumers running and lag, once a second, in both arms."""

    def __init__(self, grp: Group, produced):
        super().__init__(daemon=True)
        self.grp, self.produced, self.stop_flag, self.rows = grp, prod
    ↪ uced, threading.Event(), []

    def run(self):
        while not self.stop_flag.is_set():
            self.rows.append((self.grp.count(), max(0, self.produced.v
    ↪ alue - self.grp.consumed.value)))
            time.sleep(DT)


def cpu_times():
    with open("/proc/stat") as fh:
        f = fh.readline().split()
    vals = list(map(int, f[1:]))
    idle = vals[3] + vals[4]
    return sum(vals), idle


def kafka_bin(name):
    home = os.environ.get("KAFKA_HOME", "")
    return str(Path(home) / "bin" / name) if home else name


def make_topic(topic):
    subprocess.run([kafka_bin("kafka-topics.sh"), "--bootstrap-server"
    ↪ , BOOTSTRAP, "--create", "--if-not-exists", "--topic", topic,
                    "--partitions", str(PARTITIONS), "--replication-fa
    ↪ ctor", "1"], check=True, capture_output=True, text=True)


def delete_topic(topic):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    subprocess.run([kafka_bin("kafka-topics.sh"), "--bootstrap-server"
    ↪ , BOOTSTRAP, "--delete", "--topic", topic], capture_output=True,
    ↪  text=True)


# ------------------------------------------------------------------ o

    ↪ ne arm, one repetition
def run_arm(arm, wl_dir: Path, rep, service_ms, payload, steps, step_s
    ↪ , base, line_ms, objective=DEFAULT_OBJECTIVE):
    d = wl_dir / f"rep-{rep}" / arm; d.mkdir(parents=True, exist_ok=Tr
    ↪ ue)
    topic = f"omni-{wl_dir.name}-{rep}-{arm}-{int(time.time())}"; grou
    ↪ p = topic + "-g"
    make_topic(topic)
    rates = [base * int(s) for s in steps.split()]
    produced = mp.Value("l", 0); stop_prod = mp.Event()
    grp = Group(topic, group, service_ms, d)
    for _ in range(NATIVE_CONSUMERS):
        grp.start_one()
    time.sleep(8)                                                # the g
    ↪ roup forms and settles (both arms)
    sampler = Sampler(grp, produced); sampler.start()
    omni = None
    if arm == "omni":
        omni = Omni(grp, produced, d / "audit.jsonl", line_ms, objecti
    ↪ ve, topic=topic); omni.start()
    cpu0 = cpu_times(); t0 = time.time()
    prod = mp.Process(target=producer_proc, args=(topic, rates, step_s
    ↪ , payload, produced, stop_prod, str(d / "producer.log")))
    prod.start(); prod.join()

    # drain: up to 30 s for the group to catch up, then stop
    t_drain = time.time()
    while grp.consumed.value < produced.value and time.time() - t_drai
    ↪ n < 30:
        time.sleep(0.5)
    t1 = time.time(); cpu1 = cpu_times()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if omni:
        omni.stop_flag.set(); omni.join(timeout=60)
    sampler.stop_flag.set(); sampler.join(timeout=5)
    changes = grp.changes
    grp.stop_all()
    # the gauges, from every consumer's record

    lats = []
    for f in sorted(d.glob("consumer-*.log")):
        for ln in f.read_text().splitlines():
            try:
                t, seq, lat = ln.split(" "); lats.append((float(t), fl
    ↪ oat(lat)))

            except ValueError:
                pass
    seconds = t1 - t0
    lat_sorted = sorted(l for _, l in lats)
    n = len(lat_sorted)
    def q(p):

        return lat_sorted[min(n - 1, int(p * n))] if n else float("nan
    ↪ ")
    inside = sum(1 for l in lat_sorted if l <= line_ms)
    cons = [c for c, _ in sampler.rows] or [NATIVE_CONSUMERS]; lags =
    ↪ [l for _, l in sampler.rows] or [0]
    total = (cpu1[0] - cpu0[0]); idle = (cpu1[1] - cpu0[1]); tick_hz =

    ↪  os.sysconf("SC_CLK_TCK")
    cpu_s = (total - idle) / tick_hz
    g = {"arm": arm, "rep": rep, "seconds": round(seconds, 1), "produc
    ↪ ed": produced.value, "consumed": n,
         "work_inside_line_mps": inside / seconds, "mps": n / seconds,
    ↪  "p95_ms": q(0.95), "p99_ms": q(0.99), "p50_ms": q(0.5),

         "mean_ms": (sum(lat_sorted) / n) if n else float("nan"), "lag
    ↪ _max": max(lags), "lag_mean": sum(lags) / len(lags),
         "lost": max(0, produced.value - n), "consumers_mean": sum(con
    ↪ s) / len(cons), "consumers_max": max(cons),
         "cpu_busy_share": (total - idle) / max(1, total), "cpu_second
    ↪ s": cpu_s, "cpu_s_per_1k_inside": 1000.0 * cpu_s / max(1, inside
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ),
         "rebalances": changes, "handed_back": (omni.handed_back if om
    ↪ ni else True), "failups": (omni.failups if omni else 0),
         "verdict": (omni.verdict.record() if omni and omni.verdict el
    ↪ se None)}
    (d / "arm.json").write_text(json.dumps(g, indent=1))

    delete_topic(topic)
    print(f"   {arm} rep {rep}: {n} of {produced.value} consumed, insi
    ↪ de the line {inside / seconds:.1f}/s, p95 {g['p95_ms']:.0f} ms,
    ↪ "
          f"lag max {g['lag_max']}, consumers {g['consumers_mean']:.2f
    ↪ }, CPU {cpu_s:.1f} s, handed back {g['handed_back']}"

          + (f", verdict {g['verdict']['state']}" if g.get("verdict")
    ↪ else ""), flush=True)
    return g


def capacity(wl_dir: Path, service_ms, payload, calib_s):

    """Native's unlimited capacity: the operator's consumers fed faste
    ↪ r than they can take for calib_s seconds."""
    d = wl_dir / "capacity"; d.mkdir(parents=True, exist_ok=True)
    topic = f"omni-{wl_dir.name}-cap-{int(time.time())}"; make_topic(t
    ↪ opic)
    produced = mp.Value("l", 0); stop_prod = mp.Event()

    grp = Group(topic, topic + "-g", service_ms, d)
    for _ in range(NATIVE_CONSUMERS):
        grp.start_one()
    time.sleep(8)
    c0 = grp.consumed.value
    prod = mp.Process(target=producer_proc, args=(topic, [5000.0], cal

    ↪ ib_s, payload, produced, stop_prod, str(d / "producer.log")))
    prod.start(); t0 = time.time(); prod.join(); t1 = time.time()
    rate = (grp.consumed.value - c0) / (t1 - t0)
    grp.stop_all(); delete_topic(topic)
    return rate
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def paired(reps, key, direction):
    """omni against native on one gauge over the repetitions: means, t
    ↪ he mean paired difference, its 95% interval."""
    d = [r["omni"][key] - r["native"][key] for r in reps if _num(r["om
    ↪ ni"].get(key)) and _num(r["native"].get(key))]

    if not d:
        return None
    n = len(d)
    nat = sum(r["native"][key] for r in reps) / n
    om = sum(r["omni"][key] for r in reps) / n
    mean = sum(d) / n

    if n > 1:
        sd = math.sqrt(sum((x - mean) ** 2 for x in d) / (n - 1))
        half = T95.get(n - 1, 1.96) * sd / math.sqrt(n)
    else:
        half = float("nan")
    lo, hi = mean - half, mean + half

    same = abs(mean) <= SAME_REL * max(abs(nat), 1e-12)
    clear = n > 1 and (lo > 0 or hi < 0) and not same
    if direction == "shown":
        reading = SHOWN
    elif direction == "never more" and any(x > 0 for x in d):
        reading = "**LOSS in a repetition**"

    elif same:
        reading = EVEN_SAME
    elif not clear:
        reading = EVEN_NOISE
    else:
        reading = gel_word(mean < 0, gel_aim(direction, key))      # t

    ↪ he word by what the gauge is for, never by its sign
    return {"native": nat, "omni": om, "diff": mean, "ci95": [lo, hi],
    ↪  "n": n, "significant": clear, "reading": reading}


def _num(x):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return isinstance(x, (int, float)) and not (isinstance(x, float) a
    ↪ nd math.isnan(x))


def engine():
    try:
        out = subprocess.run([sys.executable, str(ROOT / "tools" / "om
    ↪ ni_version.py")], cwd=ROOT, capture_output=True, text=True, time
    ↪ out=120).stdout.strip()
        sha = subprocess.run(["git", "rev-parse", "HEAD"], cwd=ROOT, c
    ↪ apture_output=True, text=True).stdout.strip()
        return {"version": out.splitlines()[0] if out else "unknown",
    ↪ "commit": sha[:12]}
    except (OSError, subprocess.SubprocessError):
        return {"version": "unknown", "commit": "?"}


def run_workload(name, out: Path, reps, step_s, calib_s, line_ms, obje
    ↪ ctive=DEFAULT_OBJECTIVE):
    service_ms, payload, steps, tuning = WORKLOADS[name]
    wl_dir = out / f"kafka-{name}"; wl_dir.mkdir(parents=True, exist_o
    ↪ k=True)
    print(f"== {name}: {service_ms} ms of work a message, {payload} B,
    ↪  steps {steps}, {step_s} s a notch, the verdict's objective {obj
    ↪ ective}", flush=True)
    cap = capacity(wl_dir, service_ms, payload, calib_s)
    peak = max(int(s) for s in steps.split())
    base = PEAK_SHARE * cap / peak
    print(f"   native capacity with {NATIVE_CONSUMERS} consumers: {cap
    ↪ :.0f} messages a second; base rate {base:.1f}/s (peak notch {pea
    ↪ k} offers {PEAK_SHARE:.0%} of it)", flush=True)
    rec = {"workload": name, "tuning": tuning, "service_ms": service_m
    ↪ s, "payload_bytes": payload, "steps": steps, "step_s": step_s,
           "line_ms": line_ms, "native_consumers": NATIVE_CONSUMERS, "
    ↪ cover": list(COVER), "capacity_mps": cap, "base_rate_mps": base,
           "objective": objective, "dial": DIAL, "engine": engine(), "
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ reps": []}
    for rep in range(1, reps + 1):
        order = ("native", "omni") if rep % 2 else ("omni", "native")
        r = {}
        for arm in order:
            r[arm] = run_arm(arm, wl_dir, rep, service_ms, payload, st
    ↪ eps, step_s, base, line_ms, objective)
        rec["reps"].append(r)
        (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1)
    ↪ )
    rec["paired"] = {k: paired(rec["reps"], k, dr) for k, _, dr in GAU
    ↪ GES}
    (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1))
    return rec


def fmt(v):
    if v is None or (isinstance(v, float) and math.isnan(v)):
        return "n/a"
    return f"{v:,.0f}" if abs(v) >= 100 else f"{v:,.3g}"


def report(recs, out: Path):
    L = ["# Kafka: Omni-Compass on top of a consumer group's operator-
    ↪ set size", "",
         f"Apache Kafka as shipped (one broker, a topic of {PARTITIONS
    ↪ } partitions); native: the consumer group at the operator's coun
    ↪ t "
         f"({NATIVE_CONSUMERS}); omni: the compass law on the consumer
    ↪  count inside [{COVER[0]}, {COVER[1]}], holding the group's own
    ↪ "
         f"end-to-end latency at {CENTER:.0%} of the {LINE_MS:.0f} ms
    ↪ line. The same offered load in both arms. Energy is the host's C
    ↪ PU "
         "seconds, the compass's own cost included. Every row is repor
    ↪ ted, losses included.", ""]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    for rec in recs:
        L += [f"## {rec['workload']}: {rec['service_ms']} ms of work a
    ↪  message, {rec['payload_bytes']} B, steps {rec['steps']} × {rec[
    ↪ 'step_s']} s"
              + (" (the tuning workload)" if rec.get("tuning") else ""
    ↪ ), "",
              f"Native capacity {rec['capacity_mps']:.0f} messages a s
    ↪ econd; base rate {rec['base_rate_mps']:.1f}/s; {len(rec['reps'])
    ↪ } paired repetitions; "
              f"engine {rec['engine']['version']} at `{rec['engine']['
    ↪ commit']}`.", "",
              "| Gauge | native | omni | change | 95% interval of the
    ↪ difference | reading |", "|---|---:|---:|---:|---:|---|"]
        for k, label, direction in GAUGES:
            p = rec.get("paired", {}).get(k)
            if not p:
                continue
            ch = "" if p["native"] == 0 else f"{100 * p['diff'] / abs(
    ↪ p['native']):+.1f}%"
            L.append(f"| {label} | {fmt(p['native'])} | {fmt(p['omni']
    ↪ )} | {ch} | {fmt(p['ci95'][0])} to {fmt(p['ci95'][1])} | {p['rea
    ↪ ding']} |")
        hb = all(r["omni"]["handed_back"] for r in rec["reps"])
        L += ["", f"Every omni arm handed back to the operator's count
    ↪ : {'yes' if hb else 'NO'}; fail-ups: "
              f"{sum(r['omni']['failups'] for r in rec['reps'])}.", ""
    ↪ ]
    (out / "KAFKA.md").write_text("\n".join(_legal_stamp(L)) + "\n")
    return out / "KAFKA.md"

def setup():
    """The workflow's broker: Kafka as shipped from KAFKA_HOME, one KR
    ↪ aft node, the log directory the only change."""
    home = Path(os.environ["KAFKA_HOME"]); props = Path("/tmp/omni-kaf
    ↪ ka-server.properties"); logs = Path("/tmp/omni-kraft-logs")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    txt = (home / "config" / "kraft" / "server.properties").read_text(
    ↪ ).replace("log.dirs=/tmp/kraft-combined-logs", f"log.dirs={logs}
    ↪ ")
    props.write_text(txt)
    uuid = subprocess.run([str(home / "bin" / "kafka-storage.sh"), "ra
    ↪ ndom-uuid"], capture_output=True, text=True, check=True).stdout.

    ↪ strip().splitlines()[-1]
    subprocess.run([str(home / "bin" / "kafka-storage.sh"), "format",
    ↪ "-t", uuid, "-c", str(props)], check=True, capture_output=True,
    ↪ text=True)
    subprocess.Popen([str(home / "bin" / "kafka-server-start.sh"), str
    ↪ (props)], stdout=open("/tmp/omni-kafka-server.log", "w"), stderr

    ↪ =subprocess.STDOUT)
    for _ in range(60):
        if "Kafka Server started" in Path("/tmp/omni-kafka-server.log"
    ↪ ).read_text():
            print("broker up"); return 0
        time.sleep(2)

    print("broker did not start", file=sys.stderr); return 1


def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)

    ap.add_argument("--setup", action="store_true")
    ap.add_argument("--workloads", default="tuning")
    ap.add_argument("--reps", type=int, default=3)
    ap.add_argument("--step-s", type=float, default=20.0)
    ap.add_argument("--calib-s", type=float, default=20.0)
    ap.add_argument("--line-ms", type=float, default=LINE_MS)

    ap.add_argument("--out", default="kafka-out")
    ap.add_argument("--objective", default=DEFAULT_OBJECTIVE, choices=
    ↪ OBJECTIVES,
                    help="what the brain's verdict judges a step by: r
    ↪ esource (the index's reading: work, speed, consumers and CPU tog
    ↪ ether) or service (work and speed alone)")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ap.add_argument("--report-only", default="")
    a = ap.parse_args(argv)
    if a.setup:
        return setup()
    out = Path(a.out)
    if a.report_only:
        recs = [json.loads(f.read_text()) for f in sorted(Path(a.repor
    ↪ t_only).rglob("kafka-*/*.json"))]
        print(report(recs, Path(a.report_only))); return 0
    names = list(WORKLOADS) if a.workloads == "all" else a.workloads.s
    ↪ plit(",")
    recs = [run_workload(n, out, a.reps, a.step_s, a.calib_s, a.line_m
    ↪ s, a.objective) for n in names]
    print(report(recs, out))
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

H.52 `tools/run_kil.py`

199 lines.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The whole stacks with the real Kubernetes cluster inside: one organ
    ↪ ism, one clock, one engine.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
Started by scripts/kind_bench.sh when ORGANISM is set (kind on GitHub,
    ↪  or AKS on Azure). One of the six organisms of
tools/run_hil.py (the four realms, the four stacked with every duplica
    ↪ te kept, and the whole tower) runs
on the measured window's clock, ORGANISM_STEPS steps, and the real clu
    ↪ ster is wired into it as one more muscle:

  efferent  the organism's own compute demand drives the cluster. Dema
    ↪ nd at step k is the organism's offered work
            over its declared mean (sum over its compute pools of lam[
    ↪ k] / lam0, the seeded trace, the same in both
            arms). The load generator runs 1 + round((LOAD_MAX - 1) *

    ↪ (demand[k] - least) / (most - least))
            replicas: the organism's quietest step is one generator, i
    ↪ ts peak is LOAD_MAX, so the cluster serves the
            organism's own swings across the whole declared range.
  afferent  the cluster's watts (capture.csv, the declared power model
    ↪ , the same in both arms) are heat in the

            organism's thermal zones and load on its storage sites, as
    ↪  the card's watts are in tools/run_hil.py.

Arms, the same organism, the same seed, the same demand:
  native  the stacks' own controllers, and Kubernetes alone underneath
    ↪  (Omni-Compass not started)

  omni / compass  one engine on everything: the compass law (omnicompa
    ↪ ss/compass_law.py) on every simulated muscle
          (realms/compass_arm.py) and the live controller on the clust
    ↪ er (started by kind_bench.sh). At KILL_AT of the
          window every simulated knob is handed back, and this file ch
    ↪ ecks it was.

Writes organism.json (every plant's receipt, the steps, the replica tr
    ↪ ace, the cluster's watts as the organism saw
them) and appends each replica change to load_schedule.log in the form
    ↪ at scripts/kind_bench.sh writes.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
Sizes (--scale): 1, 10, 100 or 1,000 copies of the organism governed t
    ↪ ogether on one clock (as tools/run_hil.py), the
one real cluster inside. A large organism is built before the window o
    ↪ pens: this file writes organism.ready when it is
built and starts on the epoch written to the go file (--go-file), so t
    ↪ he cluster's window and the organism's clock open

together however long the build takes.

  python3 tools/run_kil.py --organism tower --arm compass --duration 9
    ↪ 60 --out DIR [--scale 10] [--go-file F]
"""
from __future__ import annotations

import argparse
import csv
import json
import os
import subprocess

import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from realms.harness import Body, KILL_AT  # noqa: E402
from realms.harness import omni_bones, omni_step, WALK  # noqa: E402
    ↪ (the engine's default brain, the max, omni-v5)
from realms.presets import ORGANISM_STEPS  # noqa: E402
from tools.run_hil import groups, bodies_for, NAMES, SEED0  # noqa: E4
    ↪ 02

OMNI_ARMS = ("omni", "compass", "strict")


def demand_trace(body):
    """The organism's offered compute work over its declared mean, per
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  step (the seeded trace: arm-independent)."""
    pools = [p for p in body.plants if hasattr(p, "lam")]
    if not pools:
        return [1.0] * ORGANISM_STEPS
    lam0 = sum(p.P["lam0"] for p in pools) or 1.0
    return [sum(p.lam[k] for p in pools) / lam0 for k in range(ORGANIS
    ↪ M_STEPS)]


def replicas_of(trace, load_max):
    """The organism's lowest demand is one load generator, its peak is
    ↪  load_max, linear between: its own swings,
    across the whole range the operator declared."""
    lo, hi = min(trace), max(trace)
    if hi - lo < 1e-12:
        return [max(1, load_max // 2)] * len(trace)
    return [max(1, min(load_max, 1 + round((load_max - 1) * (d - lo) /
    ↪  (hi - lo)))) for d in trace]

def cluster_w(capture, fallback):
    """The cluster's latest watts from capture.csv (the declared power
    ↪  model, every 15 s, in every arm)."""
    try:
        rows = list(csv.DictReader(open(capture)))
        for r in reversed(rows):
            if r.get("power_w"):
                return float(r["power_w"])
    except (OSError, ValueError):
        pass
    return fallback


def calibrate(organism, scale, seed, steps):
    """Seconds per step of Omni-Compass's arm, the heavier of the two
    ↪ (the model and Omni-Compass's math for every
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    modelled muscle), on this machine: the organism built as the windo
    ↪ w builds it, then `steps` steps timed one by one."""
    bodies = bodies_for(organism, scale, seed)
    bones = omni_bones(bodies)
    kill_at = int(KILL_AT * ORGANISM_STEPS)
    took = []

    for k in range(steps):
        t = time.perf_counter()
        for b in bodies:
            b.couple()
        omni_step(bodies, bones, k, kill_at)
        for b in bodies:

            b.step()
        took.append(time.perf_counter() - t)
    return {"organism": organism, "scale": scale, "steps": steps, "ste
    ↪ p_s_mean": sum(took) / len(took),
            "step_s_max": max(took), "steps_in_window": ORGANISM_STEPS
    ↪ }

def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--organism", required=True, choices=sorted(NAMES)

    ↪ )
    ap.add_argument("--arm", required=True)
    ap.add_argument("--duration", type=float, required=True, help="the
    ↪  measured window (s); one step is duration / steps")
    ap.add_argument("--out", required=True)
    ap.add_argument("--start-at", type=float, default=0.0, help="epoch

    ↪  second the window opens (built before, run after)")
    ap.add_argument("--go-file", default="", help="wait for this file
    ↪ (the window's start epoch inside) after organism.ready")
    ap.add_argument("--scale", type=int, default=int(os.environ.get("O
    ↪ RGANISM_SCALE", 1)),
                    help="copies of the organism on one clock (1, 10,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ 100, 1000)")
    ap.add_argument("--seed", type=int, default=int(os.environ.get("OR
    ↪ GANISM_SEED", SEED0)))
    ap.add_argument("--load-max", type=int, default=int(os.environ.get
    ↪ ("LOAD_MAX", 6)),
                    help="load-generator replicas at the organism's pe
    ↪ ak demand (the operator's load, declared before the run)")
    ap.add_argument("--kubectl", default=os.environ.get("KUBECTL_ADMIN
    ↪ ", "kubectl"))
    ap.add_argument("--dry", action="store_true", help="do not call ku
    ↪ bectl (tests)")
    ap.add_argument("--calibrate", type=int, default=0,
                    help="time this many steps of Omni-Compass's arm (
    ↪ no cluster, no clock) and print the seconds per step "
                         "as JSON: the window is set from it (scripts/
    ↪ kind_paired.sh)")
    a = ap.parse_args(argv)
    out = Path(a.out); out.mkdir(parents=True, exist_ok=True)

    if a.calibrate:
        print(json.dumps(calibrate(a.organism, a.scale, a.seed, a.cali
    ↪ brate)))
        return 0
    bodies = bodies_for(a.organism, a.scale, a.seed)
    body = bodies[0]                                                # t
    ↪ he body the cluster lives in
    trace = demand_trace(body)
    reps = replicas_of(trace, a.load_max)
    omni = a.arm in OMNI_ARMS
    step_s = a.duration / ORGANISM_STEPS
    kill_at = int(KILL_AT * ORGANISM_STEPS)

    (out / "organism.ready").write_text(f"{time.time():.0f}\n")
    if a.go_file:
        start = None
        while start is None:          # the start time, once the file
    ↪ holds it whole (never a half-written file)
            try:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                start = float(Path(a.go_file).read_text().split()[0])
            except (OSError, IndexError, ValueError):
                time.sleep(0.5)
        time.sleep(max(0.0, start - time.time()))
    elif a.start_at:
        time.sleep(max(0.0, a.start_at - time.time()))

    t0 = time.time()
    nominal_cluster = None
    writes, last, set_r, seen_w = 0, {}, None, []
    bones = omni_bones(bodies) if omni else []
    log = open(out / "load_schedule.log", "a")
    for k in range(ORGANISM_STEPS):

        if reps[k] != set_r:
            log.write(f"{time.strftime('%H:%M:%S', time.gmtime())} loa
↪ d-generator replicas -> {reps[k]}\n"); log.flush()
            if not a.dry:
                for _ in range(6):                                    #
↪  tried again if the API server does not answer

                    try:
                        if subprocess.run([a.kubectl, "scale", "deploy
↪ ment/load-generator", f"--replicas={reps[k]}",
                                           "--request-timeout=20s"], c
↪ apture_output=True, timeout=30).returncode == 0:
                            break

                    except subprocess.TimeoutExpired:
                        pass
                    time.sleep(5)
            set_r = reps[k]
        for b in bodies:
            b.couple()

        if omni:
            writes += omni_step(bodies, bones, k, kill_at)       # the
↪  max with its bone; after the kill the lease walks back
        for b in bodies:
            b.step()
        w = cluster_w(out / "capture.csv", nominal_cluster or 0.0)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if nominal_cluster is None and w:
            nominal_cluster = w
        seen_w.append(w)
        body.src_w += w; body.sz_w += w; body.all_w += w          # th
    ↪ e cluster's watts in the organism
        wait = t0 + (k + 1) * step_s - time.time()

        if wait > 0:
            time.sleep(wait)
    restore_ok = True
    plants = [p for b in bodies for p in b.plants]
    for p in plants:
        p.finalize()

        restore_ok = restore_ok and (not omni or p.override == {})
    rec = {"organism": a.organism, "name": NAMES[a.organism], "arm": a
    ↪ .arm, "omni": omni, "seed": a.seed,
           "muscles": len(plants), "scale": a.scale, "steps": ORGANISM
    ↪ _STEPS, "step_s": step_s, "load_max": a.load_max,
           "demand": trace, "replicas": reps, "cluster_w": seen_w, "si

    ↪ m_writes": writes, "sim_restore_ok": restore_ok,
           "plants": [dict(p.m) for p in plants], "wall_s": round(time
    ↪ .time() - t0, 1)}
    (out / "organism.json").write_text(json.dumps(rec) + "\n")
    behind = max(0.0, time.time() - t0 - a.duration)
    rec["behind_s"] = round(behind, 1)

    (out / "organism.json").write_text(json.dumps(rec) + "\n")
    print(f"{NAMES[a.organism]} x{a.scale}, arm {a.arm}: behind the wi
    ↪ ndow by {behind:.0f} s; {len(plants)} muscles, {ORGANISM_STEPS}
    ↪ steps of {step_s:.2f} s, "
          f"replicas {min(reps)}..{max(reps)}, simulated writes {write
    ↪ s}, handed back {restore_ok}")

    return 0 if restore_ok else 2


if __name__ == "__main__":
    sys.exit(main())
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

H.53 tools/run_mujoco.py

305 lines.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of a robot arm's own position servos, in MuJoCo
    ↪  (docs/ROBOTICS_PREREGISTRATION.md).

MuJoCo (Google DeepMind) integrates the arm; MuJoCo Menagerie supplies
    ↪  the robot as its maker describes it, with the
position servos the model ships with. The task is a pick-and-place cyc
    ↪ le between fixed joint-space waypoints scaled to
each robot's own ranges, played as a minimum-jerk trajectory with a pl
    ↪ anned duration, under a time line (the slowest
cycle the task allows). Arms:

  native  the shipped servo tracks the trajectory at its planned speed
    ↪  (speed override 1.0)
  omni    the same servo and trajectory, with the compass law (omnicom
    ↪ pass/compass_law.py) on one knob, the speed override,
          read from the cycle's time margin: with slack the override e
    ↪ ases down (gentler motion, less torque, less

          energy); as the margin shrinks it comes back up; past the wa
    ↪ ll it is full speed at once. Guards: override in
          [0.5, 1.0], at most 0.01 per control tick (100 Hz), handed b
    ↪ ack to 1.0 at the end of every cycle and read back.
Before the counted cycles, one paired trial in native mode (a cycle at
    ↪  override 1.0 and one at 0.8, neither counted) asks
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
the robot's own figures whether a slower cycle costs less energy at al
    ↪ l, standing draw included; if it does not, the
override is left native for that robot and the result reads "nothing f
    ↪ or Omni to move".
Energy is a declared model, the same for both arms, from MuJoCo's own
    ↪ joint torques and velocities: mechanical work

|torque x velocity|, copper heating torque^2 x C (I²R, C declared per
    ↪ robot), and a standing draw (declared per robot) drawn for
the whole takt (the time line), moving or waiting, as a robot on a lin
    ↪ e is: the unit is energy per takt. The planned speed
is set per robot, before any Omni run, as the fastest at which the rob
    ↪ ot's own servo tracks the task within 0.05 rad. Every

cycle's record is written; the report (MUJOCO.md) gives each gauge its
    ↪  word by the preregistration's aims (GAIN, EVEN,
LOSS, or power given for a resource spent that rose: tools/readings.py
    ↪ ).
  python3 tools/run_mujoco.py --menagerie DIR --robots franka_emika_pa
    ↪ nda --cycles 100 --out out

  python3 tools/run_mujoco.py --report-only DIR --out DIR
"""
from __future__ import annotations

import argparse
import csv

import json
import math
import sys
import time
from pathlib import Path

try:                                                    # the legal
    ↪  notice every generated report carries
    from tools.legal import stamp as _legal_stamp
except ImportError:
    import sys as _s, pathlib as _p; _s.path.insert(0, str(_p.Path(__f
    ↪ ile__).resolve().parents[1])); from tools.legal import stamp as
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ _legal_stamp
ROOT = Path(__file__).resolve().parents[1]; sys.path.insert(0, str(ROO
    ↪ T))
from omnicompass.compass_law import Band, CompassLaw, clamp
from tools.readings import word as gel_word, aim as gel_aim, EVEN_SAME
    ↪ , SHOWN  # noqa: E402  (the words every table uses)

CTRL_DT = 0.01            # the control tick (100 Hz)
SEG_S = 2.0               # planned seconds per trajectory segment, th
    ↪ e starting point of each robot's feasibility calibration
SPAN = 0.25               # waypoints 25% of each joint's own range ei
    ↪ ther side of home, capped at MAX_OFFSET: a brisk pick-and-place,
    ↪  the
MAX_OFFSET = 0.6          # same for every arm (rad; a model whose joi
    ↪ nts turn fully would otherwise be asked for 90-degree swings)
WAYPOINTS = 4             # segments per cycle (the last returns home)
LINE_FACTOR = 1.5         # the time line: the slowest cycle the task
    ↪ allows, as a multiple of the planned cycle

OVERRIDE_MIN, OVERRIDE_MAX, OVERRIDE_MID = 0.5, 1.0, 0.75
SLEW = 0.01               # the override moves at most this much per c
    ↪ ontrol tick
TRIAL_OVERRIDE = 0.8      # the paired physics trial's slower cycle
TRACK_OK = 0.05           # rad RMS: the task is feasible for the robo
    ↪ t's own servo when native tracks within this

# the declared energy model per robot: copper coefficient C in W per (
    ↪ N m)^2 at the joint (R / (k_t N)^2 for a geared
# motor; 0.01 is a 1-ohm, 0.1 N m/A motor behind a 100:1 gear), and th
    ↪ e standing draw in W, the same in both arms
MODEL = {"franka_emika_panda": {"copper_w_per_nm2": 0.01, "idle_w": 60
    ↪ .0, "note": "idle about 60 W per the maker's datasheet class"},
          "universal_robots_ur5e": {"copper_w_per_nm2": 0.01, "idle_w":
    ↪  90.0, "note": "idle about 90 W per the maker's datasheet class"
    ↪ },
          "kuka_iiwa_14": {"copper_w_per_nm2": 0.01, "idle_w": 100.0, "
    ↪ note": "declared"},
          "kinova_gen3": {"copper_w_per_nm2": 0.01, "idle_w": 36.0, "no
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ te": "idle about 36 W per the maker's datasheet class"}}
DEFAULT_MODEL = {"copper_w_per_nm2": 0.01, "idle_w": 100.0, "note": "d
    ↪ eclared default"}
SCENES = {"franka_emika_panda": "scene.xml", "universal_robots_ur5e":
    ↪ "scene.xml", "kuka_iiwa_14": "scene.xml", "kinova_gen3": "scene.
    ↪ xml"}

def minimum_jerk(s):
    """The minimum-jerk profile from 0 to 1 over phase s in [0, 1]."""
    return 10 * s ** 3 - 15 * s ** 4 + 6 * s ** 5

class Arm:
    """A Menagerie arm with its shipped position servos: which actuato
    ↪ rs drive joints, their ranges, the home pose."""

    def __init__(self, menagerie: Path, robot: str):

        import mujoco
        self.mujoco = mujoco
        d = menagerie / robot
        xml = d / SCENES.get(robot, "scene.xml")
        if not xml.exists():
            xml = next(p for p in sorted(d.glob("*.xml")) if "mjx" not

    ↪  in p.name and "scene" not in p.name)
        self.model = mujoco.MjModel.from_xml_path(str(xml))
        self.data = mujoco.MjData(self.model)
        m = self.model
        self.act = [i for i in range(m.nu) if m.actuator_trntype[i] ==
    ↪  mujoco.mjtTrn.mjTRN_JOINT]   # joint servos only

        if not self.act:
            raise RuntimeError("the model ships no joint servos (no jo
    ↪ int-transmission actuators)")
        self.joint = [int(m.actuator_trnid[i][0]) for i in self.act]
        self.qadr = [int(m.jnt_qposadr[j]) for j in self.joint]
        self.vadr = [int(m.jnt_dofadr[j]) for j in self.joint]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    lo = [float(m.actuator_ctrlrange[i][0]) if m.actuator_ctrllimi
↪ ted[i] else float(m.jnt_range[j][0]) for i, j in zip(self.act, s
↪ elf.joint)]
    hi = [float(m.actuator_ctrlrange[i][1]) if m.actuator_ctrllimi
↪ ted[i] else float(m.jnt_range[j][1]) for i, j in zip(self.act, s
↪ elf.joint)]

    self.lo, self.hi = lo, hi
    if m.nkey > 0:
        mujoco.mj_resetDataKeyframe(m, self.data, 0)
        home = [float(self.data.qpos[a]) for a in self.qadr]
    else:
        home = [(a + b) / 2 for a, b in zip(lo, hi)]

    self.home = [clamp(h, a, b) for h, a, b in zip(home, lo, hi)]
    self.substeps = max(1, int(round(CTRL_DT / m.opt.timestep)))
    # waypoints scaled to each joint's own range: a quarter of the
↪  span either side of home, alternating per joint
    span = [(b - a) for a, b in zip(lo, hi)]
    pattern = [(1, -1, 0.5, -0.5), (-1, 1, -0.5, 0.5), (0.5, -0.5,

↪  1, -1)]
    self.waypoints = []
    for k in range(WAYPOINTS - 1):
        w = [clamp(self.home[j] + pattern[j % 3][k] * min(SPAN * s
↪ pan[j], MAX_OFFSET), lo[j], hi[j]) for j in range(len(self.act))
↪ ]

        self.waypoints.append(w)
    self.waypoints.append(list(self.home))         # the last segm
↪ ent returns home

def target(self, phase):
    """The trajectory's joint targets at phase in [0, WAYPOINTS)."

↪ ""
    k = min(int(phase), WAYPOINTS - 1); s = minimum_jerk(clamp(pha
↪ se - k, 0.0, 1.0))
    a = self.waypoints[k - 1] if k > 0 else self.home; b = self.wa
↪ ypoints[k]
    return [x + (y - x) * s for x, y in zip(a, b)]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def reset(self):
        self.mujoco.mj_resetData(self.model, self.data)
        if self.model.nkey > 0:
            self.mujoco.mj_resetDataKeyframe(self.model, self.data, 0)
        for i, a, h in zip(self.act, self.qadr, self.home):

            self.data.qpos[a] = h; self.data.ctrl[i] = h
        self.mujoco.mj_forward(self.model, self.data)

    def tick(self, targets):
        """One control tick: write the servo targets, step the physics
    ↪ , return torque and velocity per joint."""

        for i, t in zip(self.act, targets):
            self.data.ctrl[i] = t
        for _ in range(self.substeps):
            self.mujoco.mj_step(self.model, self.data)
        tau = [float(self.data.actuator_force[i]) for i in self.act]
        vel = [float(self.data.qvel[v]) for v in self.vadr]

        pos = [float(self.data.qpos[a]) for a in self.qadr]
        return tau, vel, pos


def cycle(arm: Arm, params: dict, law: CompassLaw | None, seg: float,
    ↪ line: float, fixed_override: float | None = None):

    """One pick-and-place cycle at seg planned seconds per segment; re
    ↪ turns its gauges. law None: native (override
    fixed_override or 1.0). At the end the override is handed back to
    ↪ 1.0 and read back."""
    phase, t, override = 0.0, 0.0, (fixed_override if fixed_override i
    ↪ s not None else 1.0)

    mech = copper = 0.0; peak = 0.0; tau_abs = 0.0; n = 0; err2 = 0.0;
    ↪  wp_err = []; overrides = []
    next_wp = 1
    while phase < WAYPOINTS:
        if law is not None:
            projected = t + (WAYPOINTS - phase) * seg / override
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            f = law.force(projected)
            want = clamp(OVERRIDE_MID + (OVERRIDE_MAX - OVERRIDE_MID)
    ↪ * f, OVERRIDE_MIN, OVERRIDE_MAX)
            if projected >= line:
                want = OVERRIDE_MAX                                 # a cy
    ↪ cle that would miss its line runs full speed

            override += clamp(want - override, -SLEW, SLEW)
        phase = min(WAYPOINTS, phase + CTRL_DT * override / seg)
        targets = arm.target(phase)
        tau, vel, pos = arm.tick(targets)
        t += CTRL_DT; n += 1
        for a, v in zip(tau, vel):

            mech += abs(a * v) * CTRL_DT
            copper += a * a * params["copper_w_per_nm2"] * CTRL_DT
            peak = max(peak, abs(a)); tau_abs += abs(a)
        err2 += sum((p - q) ** 2 for p, q in zip(pos, targets)) / len(
    ↪ targets)
        overrides.append(override)

        if phase >= next_wp and next_wp <= WAYPOINTS:
            w = arm.waypoints[next_wp - 1]
            wp_err.append(math.sqrt(sum((p - q) ** 2 for p, q in zip(p
    ↪ os, w)) / len(w))); next_wp += 1
        if t > line * 3:
            break                                                   # neve

    ↪ r spin forever
    standing = params["idle_w"] * line                            # drawn
    ↪ for the whole takt, moving or waiting, in both arms
    override_at_end = override
    override = 1.0                                                 # the h
    ↪ and-back: the knob goes back to native's value, read back

    return {"cycle_s": t, "over_line": t > line, "override_at_end_of_m
    ↪ otion": override_at_end, "energy_j": mech + copper + standing, "
    ↪ mechanical_j": mech, "copper_j": copper,
            "standing_j": standing, "peak_torque_nm": peak, "mean_abs_
    ↪ torque_nm": tau_abs / max(1, n * len(arm.act)),
            "tracking_rms_rad": math.sqrt(err2 / max(1, n)), "waypoint
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ _err_rad": sum(wp_err) / max(1, len(wp_err)),
           "override_mean": sum(overrides) / max(1, len(overrides)),
    ↪ "override_after_handback": override}


def feasible_segment(arm: Arm, params: dict):

    """The planned seconds per segment for this robot: the fastest, in
    ↪  steps of 0.5 s from SEG_S, at which the robot's own
    servo tracks the task within TRACK_OK (RMS) in native mode; the ta
    ↪ sk is then one the native servo can do. None when
    no speed up to 4 x SEG_S does: the robot's own servo cannot do the
    ↪  task, and it is listed as such."""

    g = None
    for seg in [SEG_S + 0.5 * k for k in range(int(6 * SEG_S) + 1)]:
        arm.reset(); g = cycle(arm, params, None, seg, LINE_FACTOR * W
    ↪ AYPOINTS * seg, 1.0)
        if g["tracking_rms_rad"] <= TRACK_OK:
            return seg, g["tracking_rms_rad"]

    return None, g["tracking_rms_rad"] if g else None


def run_robot(menagerie: Path, robot: str, cycles: int, out: Path):
    arm = Arm(menagerie, robot)
    params = MODEL.get(robot, DEFAULT_MODEL)

    seg, track0 = feasible_segment(arm, params)
    if seg is None:
        raise RuntimeError(f"the robot's own servo cannot track the ta
    ↪ sk at any planned speed up to {4 * SEG_S:.0f} s per segment "
                           f"(RMS tracking {track0:.3f} rad at the slo
    ↪ west); the task is not feasible for it as modelled")

    planned = WAYPOINTS * seg; line = LINE_FACTOR * planned
    # the paired physics trial, in native mode, neither cycle counted:
    ↪  is a slower cycle cheaper at all, standing draw in?
    arm.reset(); full = cycle(arm, params, None, seg, line, 1.0)
    arm.reset(); slow = cycle(arm, params, None, seg, line, TRIAL_OVER
    ↪ RIDE)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    slower_is_cheaper = slow["energy_j"] < full["energy_j"]
    rec = {"robot": robot, "joints": len(arm.act), "planned_s": planne
    ↪ d, "line_s": line, "segment_s": seg, "native_tracking_at_planned
    ↪ _speed": track0,
           "energy_model": params,
           "physics_trial": {"full_speed": full, f"override_{TRIAL_OVE
    ↪ RRIDE}": slow, "slower_is_cheaper": slower_is_cheaper},
           "omni_moves": slower_is_cheaper}
    rows = []
    for name in ("native", "omni"):
        law = None
        if name == "omni" and slower_is_cheaper:
            law = CompassLaw(Band(planned, line), dt=CTRL_DT, tau=0.5,
    ↪  kp=1.0, smooth=0.3)
        per = []
        t0 = time.time()
        for c in range(cycles):
            arm.reset()
            if law is not None:
                law.p = None; law.v = 0.0
            g = cycle(arm, params, law, seg, line)
            g["cycle"] = c; g["arm"] = name; per.append(g); rows.appen
    ↪ d(g)
        n = len(per)
        agg = {k: sum(p[k] for p in per) / n for k in ("cycle_s", "ene
    ↪ rgy_j", "mechanical_j", "copper_j", "standing_j", "peak_torque_n
    ↪ m",
                                                       "mean_abs_t
    ↪ orque_nm", "tracking_rms_rad", "waypoint_err_rad", "override_mea
    ↪ n")}
        agg["over_line_share"] = sum(1 for p in per if p["over_line"])
    ↪  / n
        agg["handed_back"] = all(abs(p["override_after_handback"] - 1.
    ↪ 0) < 1e-9 for p in per)
        agg["override_at_end_of_motion"] = sum(p["override_at_end_of_m
    ↪ otion"] for p in per) / n
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        agg["cycles"] = n; agg["seconds"] = round(time.time() - t0, 1)
        rec[name] = agg
        print(f"== {robot} {name}: {n} cycles, {agg['cycle_s']:.2f} s
    ↪ each, energy {agg['energy_j']:.1f} J, tracking {agg['tracking_rm
    ↪ s_rad']:.4f} rad", flush=True)
    if not slower_is_cheaper:

        print(f"== {robot}: the paired trial says a slower cycle is no
    ↪ t cheaper (standing draw dominates); omni leaves the override na
    ↪ tive", flush=True)
    out.mkdir(parents=True, exist_ok=True)
    (out / f"{robot}.json").write_text(json.dumps(rec, indent=1) + "\n
    ↪ ")

    with (out / f"{robot}_cycles.csv").open("w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0].keys())); w.writ
    ↪ eheader(); w.writerows(rows)
    return rec

GAUGES = [("energy_j", "energy per takt (J, declared model: motion plu
    ↪ s the standing draw for the whole takt)", "spent"), ("copper_j",
    ↪  "copper heating per cycle (J, I²R, declared model)", "spent"),
          ("mechanical_j", "mechanical work per cycle (J)", "spent"),
    ↪ ("standing_j", "standing draw per takt (J, declared model; the s
    ↪ ame in both arms)", "spent"),

          ("peak_torque_nm", "peak joint torque (N m)", "spent"), ("me
    ↪ an_abs_torque_nm", "mean |torque| (N m)", "spent"),
          ("cycle_s", "cycle time (s)", "shown"), ("over_line_share",
    ↪ "cycles over the time line (share)", "never more"),
          ("tracking_rms_rad", "tracking error, RMS (rad)", "never mor
    ↪ e"), ("waypoint_err_rad", "end-point error at the waypoints (rad

    ↪ )", "never more"),
          ("override_mean", "speed override, mean", "shown")]


def report(res: dict, out: Path):
    L = ["# Omni-Compass on top of a robot arm's own servos (MuJoCo, M
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ uJoCo Menagerie)", "",
         "MuJoCo (Google DeepMind) integrates each arm; MuJoCo Menager
    ↪ ie supplies the robot as its maker describes it, with the "
         "position servos the model ships with. Native: that servo tra
    ↪ cks a pick-and-place cycle at its planned speed. Omni: the "
         "same servo and trajectory with the compass law on the speed

    ↪ override, inside the cycle's time line "
         "(`docs/ROBOTICS_PREREGISTRATION.md`). Energy is a declared m
    ↪ odel from MuJoCo's own torques and velocities, the same in "
         "both arms. Evidence class: S, our model on an independent si
    ↪ mulator. Each reading is GAIN, EVEN or LOSS by what the "
         "gauge is for, never by its sign; energy, force and copper he

    ↪ ating are resources spent, so a rise reads power given "
         "(the key: `docs/HOW_TO_READ_THE_RESULTS.md`). Every row is s
    ↪ hown, LOSS included.", ""]
    for robot, r in sorted(res.items()):
        if "error" in r:
            L += [f"## {robot}", "", f"could not run: `{r['error']}`",

    ↪  ""]; continue
        n, o = r["native"], r["omni"]
        L += [f"## {robot}: {r['joints']} joints, {n['cycles']} cycles
    ↪  per arm, planned {r['planned_s']:.0f} s (segment {r.get('segmen
    ↪ t_s', SEG_S):.1f} s, "
              f"the fastest the robot's own servo tracks within {TRACK

    ↪ _OK} rad: {r.get('native_tracking_at_planned_speed', 0):.4f}), t
    ↪ akt {r['line_s']:.0f} s", "",
              f"The task: {WAYPOINTS} segments between joint-space way
    ↪ points {int(SPAN * 100)}% of each joint's own range either side
    ↪ of home (at most {MAX_OFFSET} rad), "
              f"a minimum-jerk trajectory; the same for both arms.", "

    ↪ ",
              f"Paired physics trial (native mode, not counted): a cyc
    ↪ le at full speed cost {r['physics_trial']['full_speed']['energy_
    ↪ j']:.1f} J, at override "
              f"{TRIAL_OVERRIDE} {r['physics_trial'][f'override_{TRIAL
    ↪ _OVERRIDE}']['energy_j']:.1f} J, standing draw {r['energy_model'
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ]['idle_w']} W included: "
              + ("a slower cycle is cheaper, so omni moves the overrid
    ↪ e." if r["omni_moves"] else
                 "**a slower cycle is not cheaper, so omni leaves the
    ↪ override native: nothing for Omni to move.**"), "",
              f"Omni handed the override back at the end of every cycl

    ↪ e: {'yes' if o.get('handed_back') else 'NO'}.", "",
              "| Gauge | native | omni | Change | Reading |", "|---|--
    ↪ -:|---:|---:|---|"]
        for k, name, d in GAUGES:
            a, b = n[k], o[k]
            ch = (b - a) / abs(a) if a else None

            if d == "shown":
                rd = SHOWN
            elif ch is None:
                rd = EVEN_SAME if b == a else gel_word(b < a, gel_aim(
    ↪ d, k))
            elif abs(ch) < 1e-6:

                rd = EVEN_SAME
            else:
                rd = gel_word(ch < 0, gel_aim(d, k))            # the
    ↪ word by what the gauge is for, never by its sign
            L.append(f"| {name} | {a:.4g} | {b:.4g} | {'' if ch is Non
    ↪ e else f'{100 * ch:+.2f}%'} | {rd} |")

        L.append("")
    (out / "MUJOCO.md").write_text("\n".join(_legal_stamp(L)) + "\n")


def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=

    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--menagerie", default="menagerie")
    ap.add_argument("--robots", default="")
    ap.add_argument("--cycles", type=int, default=10)
    ap.add_argument("--out", required=True)
    ap.add_argument("--report-only", default="")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    a = ap.parse_args(argv)
    out = Path(a.out); out.mkdir(parents=True, exist_ok=True)
    res = {}
    if a.report_only:
        for f in sorted(Path(a.report_only).glob("*.json")):
            res[f.stem] = json.loads(f.read_text())

    for robot in [r for r in a.robots.split(",") if r]:
        try:
            res[robot] = run_robot(Path(a.menagerie), robot, a.cycles,
    ↪  out)
        except Exception as e:                                  # a r
    ↪ obot an arm cannot run is reported, never left out

            res[robot] = {"robot": robot, "error": f"{type(e).__name__
    ↪ }: {e}"[:600]}
            print(f"== {robot}: ERROR {res[robot]['error']}", flush=Tr
    ↪ ue)
            (out / f"{robot}.json").write_text(json.dumps(res[robot],
    ↪ indent=1) + "\n")

    report(res, out)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

H.54 `tools/run_pandapower.py`

249 lines.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of a distribution grid's own voltage control, i
    ↪ n pandapower (docs/PANDAPOWER_PREREGISTRATION.md).

An independent, recognized simulator and data set: pandapower (Fraunho
    ↪ fer IEE and the University of Kassel; BSD) solves
the AC power flow; SimBench (the German benchmark grids, with a year o
    ↪ f 15-minute load, solar and wind profiles) gives
the grids and their year. Both are someone else's work; nothing here m
    ↪ odels the grid.

  native  the substation's on-load tap changer holds the medium-voltag
    ↪ e busbar at its setpoint (1.00 per unit) inside a
          deadband of half a tap step, one tap per step at most; solar
    ↪  and wind at unity power factor (as SimBench has
          them). This is how a utility runs it on its own.

  omni    the same tap changer, with the compass law (omnicompass/comp
    ↪ ass_law.py) on top of its setpoint: it reads the
          worst voltage margin anywhere in the grid (the bus nearest e
    ↪ ither limit, 0.95 and 1.05 per unit) and moves the
          setpoint inside [0.97, 1.03]: down while every bus has room
    ↪ (conservation voltage reduction), away from the

          nearer limit as soon as one bus nears it, and straight back
    ↪ to native at a violation. At KILL_AT of the year
          the setpoint is handed back. Every move is one whole tap; a
    ↪ move toward saving needs a full week seen under the
          present setpoint whose worst moment still leaves every bus M
    ↪ ARGIN after one more tap; a move away from a limit

          is at once, once the utility's tap changer has reached the p
    ↪ resent setpoint.

Scored by pandapower itself, the same grid and the same year in both a
    ↪ rms:
  energy drawn from the upstream grid (MWh), line and transformer loss
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ es (MWh), bus-hours outside [0.95, 1.05],
  tap operations (wear), and the lowest and highest voltage seen.

Loads: SimBench's loads are constant power. Lowering the voltage saves
    ↪  energy only where a load draws less at a lower
voltage, so the run is made twice: with a declared ZIP mix (40% consta
    ↪ nt impedance, 30% constant current, 30% constant
power, a common residential mix) and with SimBench's own constant-powe
    ↪ r loads, where only losses can change.

  python3 tools/run_pandapower.py --grids 1-MV-rural--0-sw --out out [
    ↪ --step 4]   (step 4: hourly, every 4th quarter hour)

  python3 tools/run_pandapower.py --report-only DIR --out DIR
"""
from __future__ import annotations

import argparse
import json

import math
import sys
import time
from pathlib import Path

try:                                                    # the legal
    ↪  notice every generated report carries
    from tools.legal import stamp as _legal_stamp
except ImportError:
    import sys as _s, pathlib as _p; _s.path.insert(0, str(_p.Path(__f
    ↪ ile__).resolve().parents[1])); from tools.legal import stamp as
    ↪ _legal_stamp

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from omnicompass.compass_law import Band, CompassLaw, clamp  # noqa: E
    ↪ 402
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
V_LO, V_HI = 0.95, 1.05           # the voltage limits (EN 50160 band
    ↪ , per unit)
SET_NATIVE = 1.00                 # the tap changer's own setpoint
SET_LO, SET_HI = 0.97, 1.03       # the cover Omni may move the setpo
    ↪ int in
STEP_V = 0.015                    # the setpoint moves in whole tap s
    ↪ teps (1.5%), so every change is one tap, never a hunt
MARGIN = 0.01                     # the room every bus keeps from a l
    ↪ imit, after any step Omni takes (per unit)
WINDOW_H = 168.0                  # a step toward saving needs a full
    ↪  week seen under the present setpoint (tap wear)
KILL_AT = 0.9

ZIP = (40.0, 30.0)                # constant-impedance and constant-c
    ↪ urrent shares of every load (percent), the rest power


def tap_control(net, vm, setpoint, ops):
    """The utility's own tap changer: one tap toward the setpoint when
    ↪  the busbar is outside half a tap step of it."""
    for t in net.trafo.index:
        v = vm[net.trafo.at[t, "lv_bus"]]
        half = net.trafo.at[t, "tap_step_percent"] / 200.0
        pos = net.trafo.at[t, "tap_pos"]
        if v > setpoint + half and pos < net.trafo.at[t, "tap_max"]:
            net.trafo.at[t, "tap_pos"] = pos + 1; ops[0] += 1      # a
    ↪  higher tap lowers the medium-voltage side
        elif v < setpoint - half and pos > net.trafo.at[t, "tap_min"]:
            net.trafo.at[t, "tap_pos"] = pos - 1; ops[0] += 1

def run_arm(code, arm, zip_loads, step):
    import pandapower as pp
    import simbench as sb
    net = sb.get_simbench_net(code)
    prof = sb.get_absolute_values(net, profiles_instead_of_study_cases
    ↪ =True)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    net.trafo["tap_changer_type"] = "Ratio"
    # ZIP loads are applied by hand, never through pandapower's own co
    ↪ nst_z/const_i columns: pandapower puts a bus's ZIP
    # shares on the bus's whole demand, solar on the same bus included
    ↪ , so the solar would draw less at a lower voltage
    # too (and the solved balance came out backwards). Here each load

    ↪ is set to p0 (z V^2 + i V + p) at its own bus
    # voltage and the flow is solved again until the voltages stop mov
    ↪ ing.
    zz, ii = (ZIP[0] / 100.0, ZIP[1] / 100.0) if zip_loads else (0.0,
    ↪ 0.0)
    lbus = net.load.bus.values

    n = len(prof[("load", "p_mw")])
    ks = list(range(0, n, step))
    kill = int(KILL_AT * len(ks))
    dt_h = 0.25 * step
    law = CompassLaw(Band(0.0, 1.0), dt=1.0, tau=3.0, kp=1.0, smooth=0
    ↪ .3) if arm == "omni" else None

    if law is not None:
        law.kd *= 3.0
    setpoint, ops = SET_NATIVE, [0]
    last_change_h, hist = 0.0, []                          # when the set
    ↪ point last moved; the margins seen since (trailing window)
    e_in = e_loss = e_load = e_gen = 0.0

    viol = steps = 0
    sp_sum, sp_log, worst_bal = 0.0, [], 0.0
    vmin_all, vmax_all = 9.0, 0.0
    t0 = time.time()
    for i, k in enumerate(ks):
        for (elm, col), df in prof.items():

            if elm in net and col in net[elm].columns and df.shape[1]
    ↪ and len(df) > k:
                net[elm].loc[df.columns, col] = df.iloc[k].values
        p0, q0 = net.load.p_mw.values.copy(), net.load.q_mvar.values.c
    ↪ opy()
        pp.runpp(net, numba=False)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if zip_loads:
            for _ in range(8):
                v = net.res_bus.vm_pu.loc[lbus].values
                f = zz * v * v + ii * v + (1.0 - zz - ii)
                net.load["p_mw"], net.load["q_mvar"] = p0 * f, q0 * f
                v_before = net.res_bus.vm_pu.values.copy()

                pp.runpp(net, numba=False, init="results")
                if abs(net.res_bus.vm_pu.values - v_before).max() < 1e
    ↪ -5:
                    break
            net.load["p_mw"], net.load["q_mvar"] = p0, q0        # the
    ↪  nominal draw back, for the next step's profile

        vm = net.res_bus.vm_pu
        if law is not None and i < kill:
            lo_m, hi_m = vm.min() - V_LO, V_HI - vm.max()
            now_h = i * dt_h
            hist.append((now_h, lo_m, hi_m))
            while hist and hist[0][0] < now_h - WINDOW_H:

                hist.pop(0)
            # the compass reads the worst margin now: 0 is a full marg
    ↪ in's room, 1 is at a limit (a violation reads past it)
            p = clamp(1.0 - min(lo_m, hi_m) / (V_HI - SET_NATIVE), 0.0
    ↪ , 1.5)
            F = law.force(p if min(lo_m, hi_m) >= 0 else 1.5)

            step_dir = 0
            # the utility's tap changer first has to reach the present
    ↪  setpoint (busbar within half a tap of it); until it
            # has, a voltage is the tap changer still travelling, not
    ↪ the grid asking Omni for anything
            settled = all(abs(vm[net.trafo.at[t, "lv_bus"]] - setpoint

    ↪ ) <= net.trafo.at[t, "tap_step_percent"] / 200.0 + 1e-9
                          for t in net.trafo.index)
            if not settled:
                pass
            elif lo_m < MARGIN and hi_m > STEP_V + MARGIN and setpoint
    ↪  < SET_HI - 1e-9:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            step_dir = 1                                          # the
↪ lowest bus nears 0.95: one tap up, at once
        elif hi_m < MARGIN and lo_m > STEP_V + MARGIN and setpoint
↪  > SET_LO + 1e-9:
            step_dir = -1                                         # the
↪ highest bus nears 1.05: one tap down, at once

        elif F <= 0 and now_h - last_change_h >= WINDOW_H and now_
↪ h >= WINDOW_H:
            # calm, and the whole trailing window seen under this
↪ setpoint: step only where the worst moment of that
            # window would still keep MARGIN after one more tap (t
↪ he tap moves every bus by about one step)

            lo_w, hi_w = min(h[1] for h in hist), min(h[2] for h i
↪ n hist)
            if setpoint > SET_NATIVE + 1e-9 and hi_w - STEP_V >= M
↪ ARGIN and lo_w - STEP_V >= MARGIN:
                step_dir = -1                                     # an e
↪ arlier raise no longer needed: back toward native

            elif setpoint > SET_LO + 1e-9 and lo_w - STEP_V >= MAR
↪ GIN:
                step_dir = -1                                     # cons
↪ ervation voltage reduction, one tap
        if step_dir:
            setpoint = clamp(setpoint + STEP_V * step_dir, SET_LO,

↪  SET_HI)
            last_change_h = now_h
            hist.clear()                                          # the
↪ window starts again under the new setpoint
    elif law is not None:
        setpoint = SET_NATIVE                                     # hand

↪ ed back
    tap_control(net, vm, setpoint, ops)
    sp_sum += setpoint; sp_log.append((round(i * dt_h), round(setp
↪ oint, 3), round(float(vm.min()), 4), round(float(vm.max()), 4)))
    e_in += float(net.res_ext_grid.p_mw.sum()) * dt_h
    gen = float(net.res_sgen.p_mw.sum())
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        e_gen += gen * dt_h
        e_load += float(net.res_load.p_mw.sum()) * dt_h            # w
    ↪ hat the loads drew at the voltage they saw
        bal = abs(float(net.res_ext_grid.p_mw.sum()) + gen - float(net
    ↪ .res_load.p_mw.sum())
                  - float(net.res_line.pl_mw.sum()) - float(net.res_tr
    ↪ afo.pl_mw.sum()))
        worst_bal = max(worst_bal, bal)                             #
    ↪ the solved flow must close: import + solar = load + losses
        e_loss += (float(net.res_line.pl_mw.sum()) + float(net.res_tra
    ↪ fo.pl_mw.sum())) * dt_h
        bad = int(((vm < V_LO) | (vm > V_HI)).sum())
        viol += bad; steps += len(vm)
        vmin_all, vmax_all = min(vmin_all, float(vm.min())), max(vmax_
    ↪ all, float(vm.max()))
    return {"arm": arm, "zip_loads": zip_loads, "steps": len(ks), "dt_
    ↪ h": dt_h, "energy_in_mwh": e_in,
            "losses_mwh": e_loss, "load_energy_mwh": e_load, "generati
    ↪ on_mwh": e_gen, "violation_bus_share": viol / max(steps, 1), "ta
    ↪ p_operations": ops[0],
            "vmin": vmin_all, "vmax": vmax_all, "seconds": round(time.
    ↪ time() - t0, 1),
            "restored": setpoint == SET_NATIVE, "worst_balance_mw": wo
    ↪ rst_bal, "trafos": len(net.trafo),
            "mean_setpoint": sp_sum / len(ks), "setpoint_changes": [e
    ↪ for j, e in enumerate(sp_log) if j and e[1] != sp_log[j - 1][1]]
    ↪ }


ROWS = [("load_energy_mwh", "energy the loads drew (MWh)"), ("losses_m
    ↪ wh", "line and transformer losses (MWh)"),
        ("energy_in_mwh", "net import from the upstream grid (MWh; neg
    ↪ ative: the grid exports)"),
        ("generation_mwh", "solar and wind fed in (MWh)"),
        ("violation_bus_share", "bus-steps outside 0.95-1.05 per unit
    ↪ (share)"), ("tap_operations", "tap operations (wear)"),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        ("vmin", "lowest voltage seen (per unit)"), ("vmax", "highest
    ↪ voltage seen (per unit)"),
        ("mean_setpoint", "busbar setpoint, mean over the year (per un
    ↪ it)")]
LOWER = {"load_energy_mwh", "energy_in_mwh", "losses_mwh", "violation_
    ↪ bus_share", "tap_operations"}

def reading(k, n, o):
    if k in ("vmin", "vmax", "generation_mwh", "mean_setpoint"):
        return "shown"
    if abs(o - n) <= 1e-6 * max(abs(n), 1e-12):

        return "same"
    toward = (o < n) == (k in LOWER)                        # which way
    ↪  the number went, and whether toward its aim
    return ("lower" if o < n else "higher") + (" (toward its aim)" if
    ↪ toward else " (away from its aim)")

def report(res, out):
    L = ["# Omni-Compass on top of a distribution grid's own voltage c
    ↪ ontrol (pandapower, SimBench)", "",
         "pandapower (Fraunhofer IEE, University of Kassel) solves eve
    ↪ ry AC power flow; SimBench gives the grids and their "

         "year of measured-shape profiles. Native: the substation's ta
    ↪ p changer holding its busbar at 1.00 per unit. Omni: "
         "the compass law on top of that setpoint, inside [0.97, 1.03]
    ↪ , reading the worst voltage margin in the grid. "
         "Evidence class: an independent recognized simulator and data
    ↪  set (not our model).", ""]

    for code, r in res.items():
        for zl in ("zip", "constant_power"):
            if zl not in r:
                continue
            n, o = r[zl]["native"], r[zl]["omni"]
            L += [f"## {code}, loads {'ZIP (40% Z, 30% I, 30% P)' if z
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ l == 'zip' else 'constant power (SimBench as shipped)'}", "",
                  f"{n['steps']} steps of {n['dt_h']:g} h. Omni handed
    ↪  the setpoint back: {'yes' if o['restored'] else 'NO'}. "
                  f"Setpoint changes Omni made (hour, new setpoint, lo
    ↪ west and highest bus): {o.get('setpoint_changes', [])}; "
                  f"each change is one tap on each of the {o.get('traf
    ↪ os', '?')} substation transformers, the hand-back at "
                  f"{KILL_AT:.0%} of the year included. Worst power ba
    ↪ lance of any solved step: "
                  f"{max(n.get('worst_balance_mw', 0), o.get('worst_ba
    ↪ lance_mw', 0)):.1e} MW.", "",
                  "| | native | omni | Change | Reading |", "|---|---:
    ↪ |---:|---:|---|"]
            for k, lab in ROWS:
                ch = (o[k] - n[k]) / abs(n[k]) * 100 if abs(n[k]) > 1e
    ↪ -12 else float("nan")
                chs = f"{ch:+.2f}%" if not math.isnan(ch) else f"{o[k]
    ↪  - n[k]:+.4g}"
                L.append(f"| {lab} | {n[k]:.6g} | {o[k]:.6g} | {chs} |
    ↪  {reading(k, n[k], o[k])} |")
            L.append("")
    (Path(out) / "PANDAPOWER.md").write_text("\n".join(_legal_stamp(L)
    ↪ ) + "\n")
    print("\n".join(L))

def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--grids", default="")
    ap.add_argument("--out", required=True)
    ap.add_argument("--step", type=int, default=4)
    ap.add_argument("--report-only", default="")
    a = ap.parse_args(argv)
    out = Path(a.out); out.mkdir(parents=True, exist_ok=True)
    res = {}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if a.report_only:
        for f in sorted(Path(a.report_only).glob("*.json")):
            res[f.stem] = json.loads(f.read_text())
    for code in [g for g in a.grids.split(",") if g]:
        res[code] = {}
        for zl in (True, False):

            key = "zip" if zl else "constant_power"
            res[code][key] = {}
            for arm in ("native", "omni"):
                try:
                    res[code][key][arm] = run_arm(code, arm, zl, a.ste
    ↪ p)

                except Exception as e:                       # a grid
    ↪ an arm cannot run is reported, never left out
                    res[code][key][arm] = {"arm": arm, "error": f"{typ
    ↪ e(e).__name__}: {e}"[:600]}
                r = res[code][key][arm]
                print(f"== {code} {key} {arm}: " + (f"{r['seconds']} s

    ↪ " if "seconds" in r else f"ERROR {r['error']}"), flush=True)
            if any("error" in res[code][key][x] for x in ("native", "o
    ↪ mni")):
                del res[code][key]
        (out / f"{code}.json").write_text(json.dumps(res[code], indent
    ↪ =1) + "\n")

    report(res, out)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

H.55 `tools/run_pgbench.py`

952 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.

# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of a database's own connection pooler (docs/POS
    ↪ TGRES_PREREGISTRATION.md).

PostgreSQL (the PostgreSQL Global Development Group) runs as the distr

    ↪ ibution ships it; PgBouncer (its own project) sits
in front of it in transaction pooling with the pool size it ships with
    ↪ , 20 server connections; pgbench (PostgreSQL's own
benchmark) offers the load. That is native: the pooler's fixed pool is
    ↪  the DBA's one setting, made once.

omni is the same database, the same pooler and the same load, with the
    ↪  compass law (omnicompass/compass_law.py) on
one knob: the pooler's pool size, written through its own admin consol
    ↪ e (SET default_pool_size), inside the cover
[2, 90]. The reading is the service time the pooler itself reports eve
    ↪ ry second (transaction time plus the time clients

waited for a server, per transaction), or the share of its clients que
    ↪ ued for a server scaled to the line, whichever is
larger (amendment 1), on a band from 0 to the response line. Past the
    ↪ wall the knob is handed back to
the pooler's own setting at once (fail up). At the end it is handed ba
    ↪ ck and read back.

Load: pgbench at a fixed client count, rate-limited, the rate stepping
    ↪  one notch at a time (1 2 3 2 3 4 5 6 5 4 3 2 1 2
1) with the base rate set before the counted runs by one native-only r
    ↪ un with no rate limit (the peak notch offers
native's own unlimited capacity). Both arms run the same steps, paired
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ , the order alternating between repetitions.

Gauges (from pgbench's per-transaction log, PgBouncer's SHOW POOLS and
    ↪  the host's /proc/stat): work inside the
response line (transactions a second whose latency, lag included, was
    ↪ within the line), throughput, p50/p95/p99

latency, failed transactions, server connections alive (the "machines"
    ↪ ), the host's CPU busy share and CPU-seconds.
There is no watt-meter on a GitHub runner: nothing here is an energy c
    ↪ laim.

Amendment 2 (docs/POSTGRES_PREREGISTRATION.md, declared before the sec

    ↪ ond counted set): the console is read over one
connection held for the whole arm (class Console, the server's own wir
    ↪ e protocol) instead of a psql process launched for
every reading, which was about five launches a second in the omni arm
    ↪ at about 50 ms of CPU each and was the CPU the
first counted set charged to Omni; the queue line: a server is taken b

    ↪ ack (the calm give-back and the slow-inside-the-server
shrink alike) only while the pooler's clients waited for a server unde
    ↪ r one percent of its time in the last second and a
transaction was served, and while they waited one percent or more one
    ↪ server a second is added back, up to the pooler's own
setting; the harness's own CPU is recorded beside the host's.

Usage:
  python tools/run_pgbench.py --setup                      # create th
    ↪ e bench role and database (needs the postgres user)
  python tools/run_pgbench.py --workloads tpcb --reps 3 --out out
  python tools/run_pgbench.py --report-only all --out all  # re-read f

    ↪ inished <workload>.json files and write PGBENCH.md
"""
from __future__ import annotations

import argparse
import hashlib
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
import json
import math
import os
import resource
import shutil
import socket

import struct
import subprocess
import sys
import threading
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from omnicompass.compass_law import Band, CompassLaw, Plug, clamp  # n
    ↪ oqa: E402
from tools.legal import stamp as _legal_stamp  # noqa: E402

from tools.knob_verdict import KnobVerdict, sample_cost, OBJECTIVES, R
    ↪ ESOURCE as DEFAULT_OBJECTIVE  # noqa: E402  (amendment 3: the br
    ↪ ain's own verdict)
from tools.readings import word as gel_word, aim as gel_aim, EVEN_SAME
    ↪ , EVEN_NOISE, SHOWN  # noqa: E402  (the words every table uses)

# --- the frozen rule ------------------------------------------------
    ↪ ------------------------------------------------
LINE_MS = 50.0           # the response line: a transaction answered
    ↪ within 50 ms, lag included
CENTER = 0.4             # the compass pulls the service to 40% of th
    ↪ e line (the service profile of the Kubernetes adapter)

DT = 1.0                 # one decision a second
TAU = 2.0                # the pool's response time: a slot added or
    ↪ removed shows in the reading within a couple of seconds
SMOOTH = 0.5
DEAD = 0.05              # a force inside the cushion moves nothing
UP = 0.10                # adding: ceil(force / UP) slots a second (f
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ull force adds ten); taking back: one idle server a second
PEAK = 0.9                # the peak notch offers nine tenths of nativ
    ↪ e's unlimited capacity: native at its knee, not past it
DWELL_S = 30.0            # after adding, no taking back within thirty
    ↪  seconds (no hunting); adding is never held (amendment 1). Store
    ↪  fix of 25

                          # October: five seconds before; near saturat
    ↪ ion the brain gave a server back on a calm second and added it b
    ↪ ack 1 to
                          # 10 seconds later, 21 to 39 writes an arm i
    ↪ n the omni-v16 tpcb_hot runs, each closing a server and opening
    ↪ a new one

FLOOR, CEILING = 2, 90    # the cover: never under two server connecti
    ↪ ons; never within ten of PostgreSQL's 100 connections
NATIVE_POOL = 20          # PgBouncer's shipped default_pool_size: nat
    ↪ ive's setting and Omni's snapshot
WAIT_SHARE = 0.5          # where the time goes decides the direction:
    ↪  waiting for a server (add one) or inside it (take one)

GIVEBACK_WAIT_SHARE = 0.01  # amendment 2: a server is taken back only
    ↪  while clients waited for one under 1% of the pooler's time in t
    ↪ he
                            # last second (the pool holds the demand),
    ↪  the one line the three database tests share (misses under 1%)

def service_reading(latency_s, waiting_share, line_s=LINE_MS / 1000.0)
    ↪ :
    """The reading the compass sees (amendment 1, docs/POSTGRES_PREREG
    ↪ ISTRATION.md): the pooler's service time per
    transaction, or the share of its clients queued for a server scale

    ↪ d to the line, whichever is larger. A pool too small
    for the offered rate shows at the pooler as clients waiting, not a
    ↪ s a long transaction: its own clock cannot see the
    backlog that piles up in the clients' schedules, and the first unt
    ↪ ouched run proved it."""
    return max(latency_s, waiting_share * line_s)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
STEPS = "1 2 3 2 3 4 5 6 5 4 3 2 1 2 1"
WORKLOADS = {             # name: (pgbench script flag, scale factor)
    "tpcb": ("", 20),             # the tuning workload: pgbench's TPC
    ↪ -B-like default script, 20 branches
    "select": ("-S", 20),         # untouched: select-only, read-heavy

    "simple_update": ("-N", 20),  # untouched: the default script with
    ↪ out the branch and teller updates
    "tpcb_hot": ("", 2),          # untouched: two branches, every tra
    ↪ nsaction fights for the same rows
}
GAUGES = [  # key, label, direction

    ("work_inside_line_tps", "work inside the response line (transacti
    ↪ ons a second answered within the line)", "higher"),
    ("tps", "throughput (transactions a second)", "higher"),
    ("p95_ms", "latency, 95th percentile (ms, lag included)", "lower")
    ↪ ,
    ("p99_ms", "latency, 99th percentile (ms)", "lower"),

    ("p50_ms", "latency, median (ms)", "lower"),
    ("mean_ms", "latency, mean (ms)", "lower"),
    ("failed", "failed transactions", "never more"),
    ("servers_alive_mean", "server connections alive, mean (the machin
    ↪ es)", "spent"),
    ("servers_alive_max", "server connections alive, most at once", "s

    ↪ pent"),
    ("cpu_busy_share", "host CPU busy (share of the run)", "spent"),
    ("cpu_seconds", "host CPU-seconds", "spent"),
    ("cpu_s_per_1k_inside", "host CPU-seconds per 1,000 transactions i
    ↪ nside the line", "spent"),
    ("harness_cpu_seconds", "the harness's own CPU-seconds (the compas

    ↪ s's brain in the omni arm, the sampler in both; inside the host'
    ↪ s)", "shown"),
    ("pool_mean", "pool size, mean (the knob)", "shown"),
]
from omnicompass.student_t import T95  # noqa: E402  (the two-sided 95
    ↪ % point of Student's t, any degrees of freedom)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
SAME_REL = 1e-6


# --- pure pieces (tested without a database) ------------------------
    ↪ ------------------------------------------------
def decide(p, force, wait_share, idle, cur, snapshot, last_dir, since_

    ↪ change_s, wall=0.95, queue_share=0.0, served=True):
    """The pool size the brain wants this second, by the frozen rule.
    ↪ Returns (target, direction, reason).

    Amendment 2, the queue line: `queue_share` is the share of the poo
    ↪ ler's time its clients spent waiting for a server in

    the last second and `served` says whether it counted a transaction
    ↪ . A server is taken back (the calm give-back and the
    slow-inside-the-server shrink alike) only while a transaction was
    ↪ served and that share is under GIVEBACK_WAIT_SHARE;
    while it is at the line or above, servers are added back, one per
    ↪ percent of waiting, up to the pooler's own setting

    (the snapshot), so a take-back that put clients in the queue is un
    ↪ done the next second and a rising load gets its
    servers back at once. Above the pooler's own setting only slowness
    ↪  adds, by the force, as before; adding is never held."""
    if p >= wall:
        return snapshot, 0, "fail up: handed back to the pooler's own

    ↪ setting"
    if force > DEAD and wait_share >= WAIT_SHARE:
        d, why = math.ceil(force / UP), f"slow, clients waiting for a
    ↪ server: {math.ceil(force / UP)} more"
    elif not served:
        return cur, 0, "nothing served in the last second: nothing kno

    ↪ wn, nothing moved"
    elif queue_share >= GIVEBACK_WAIT_SHARE:
        if cur < snapshot:
            d = min(snapshot - cur, max(1, math.ceil(queue_share / GIV
    ↪ EBACK_WAIT_SHARE)))   # one server per percent of waiting
            why = f"clients waited for a server {100 * queue_share:.1f
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ }% of the time: {d} added back, up to the pooler's own setting"
        else:
            return cur, 0, "clients waited for a server (1% or more of
    ↪  the time) at the pooler's own setting or above: the pool is in
    ↪ demand, nothing taken"
    elif force > DEAD:

        d, why = -1, "slow inside the server: one fewer"
    elif force < -DEAD:
        if idle < 1:
            return cur, 0, "calm, but no idle server to give back"
        d, why = -1, "calm, an idle server given back"
    else:

        return cur, 0, "inside the cushion"
    sign = 1 if d > 0 else -1
    if sign < 0 and last_dir > 0 and since_change_s < DWELL_S:
        return cur, 0, f"dwell: no taking back within {DWELL_S:.0f} se
    ↪ conds of adding"   # gas is never held; only the brake dwells (a
    ↪ mendment 1)

    target = int(clamp(cur + d, FLOOR, CEILING))
    if target == cur:
        return cur, 0, "at the cover"
    return target, sign, why

def parse_latencies(files):
    """Latencies (microseconds) from pgbench's per-transaction log fil
    ↪ es (-l): client transaction time script epoch us lag."""
    out = []
    for f in files:
        with open(f) as fh:

            for line in fh:
                parts = line.split()
                if len(parts) >= 3:
                    try:
                        out.append(int(parts[2]))
                    except ValueError:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                        continue
    return out


def parse_log_rows(files):
    """(epoch seconds, latency us, schedule lag us) of every transacti
    ↪ on in pgbench's per-transaction log files (-l, with -R):
    client transaction time script epoch us [lag]."""
    out = []
    for f in files:
        with open(f) as fh:
            for line in fh:
                parts = line.split()
                if len(parts) >= 6:
                    try:
                        out.append((int(parts[4]) + int(parts[5]) / 1e
    ↪ 6, int(parts[2]), int(parts[6]) if len(parts) >= 7 else 0))
                    except ValueError:
                        continue
    return out


def aggregate_per_second(rows, interval=1):
    """pgbench's own --aggregate-interval columns, one line a second,
    ↪ from the per-transaction rows (store fix of 25 October):
    interval_start num_transactions sum_latency sum_latency_2 min_late
    ↪ ncy max_latency sum_lag sum_lag_2 min_lag max_lag. pgbench
    writes either this or the per-transaction log, never both, and the
    ↪  gauges are read from the per-transaction log; so the runner
    writes the same aggregate from it before the raw log is removed, a
    ↪ nd every second of a stall stays in the run's files."""
    agg = {}
    for t, lat, lag in rows:
        k = int(t // interval) * interval
        a = agg.get(k)
        if a is None:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            agg[k] = [1, lat, lat * lat, lat, lat, lag, lag * lag, lag
    ↪ , lag]
        else:
            a[0] += 1; a[1] += lat; a[2] += lat * lat; a[3] = min(a[3]
    ↪ , lat); a[4] = max(a[4], lat)
            a[5] += lag; a[6] += lag * lag; a[7] = min(a[7], lag); a[8
    ↪ ] = max(a[8], lag)
    return [f"{k} " + " ".join(str(x) for x in agg[k]) for k in sorted
    ↪ (agg)]


def latency_gauges(lat_us, seconds, line_ms=LINE_MS):
    """The gauges one arm's transactions give over `seconds` of offere
    ↪ d load."""
    n = len(lat_us)
    if n == 0:
        return {"transactions": 0, "tps": 0.0, "work_inside_line_tps":
    ↪  0.0, "inside_line_share": 0.0,
                "p50_ms": float("nan"), "p95_ms": float("nan"), "p99_m
    ↪ s": float("nan"), "mean_ms": float("nan")}
    s = sorted(lat_us)
    q = lambda f: s[min(n - 1, int(math.ceil(f * n)) - 1)] / 1000.0  #
    ↪  noqa: E731
    inside = sum(1 for x in lat_us if x <= line_ms * 1000)
    return {"transactions": n, "tps": n / seconds, "work_inside_line_t
    ↪ ps": inside / seconds,
            "inside_line_share": inside / n, "p50_ms": q(0.5), "p95_ms
    ↪ ": q(0.95), "p99_ms": q(0.99),
            "mean_ms": sum(lat_us) / n / 1000.0}

def paired(reps, key, direction):
    """omni against native on one gauge over the repetitions: means, t
    ↪ he mean paired difference, its 95% interval."""
    d = [r["omni"][key] - r["native"][key] for r in reps if _num(r["om
    ↪ ni"].get(key)) and _num(r["native"].get(key))]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if not d:
        return None
    n = len(d)
    nat = sum(r["native"][key] for r in reps) / n
    om = sum(r["omni"][key] for r in reps) / n
    mean = sum(d) / n

    if n > 1:
        sd = math.sqrt(sum((x - mean) ** 2 for x in d) / (n - 1))
        half = T95.get(n - 1, 1.96) * sd / math.sqrt(n)
    else:
        half = float("nan")
    lo, hi = mean - half, mean + half

    same = abs(mean) <= SAME_REL * max(abs(nat), 1e-12)
    clear = n > 1 and (lo > 0 or hi < 0) and not same
    if direction == "shown":
        reading = SHOWN
    elif direction == "never more" and any(x > 0 for x in d):
        reading = "**LOSS in a repetition**"

    ↪            # any increase, in any repetition, whatever the noise
    elif same:
        reading = EVEN_SAME
    elif not clear:
        reading = EVEN_NOISE
    else:

        reading = gel_word(mean < 0, gel_aim(direction, key))      # t
    ↪ he word by what the gauge is for, never by its sign
    return {"native": nat, "omni": om, "diff": mean, "ci95": [lo, hi],
    ↪  "n": n, "significant": clear, "reading": reading}

def _num(x):
    return isinstance(x, (int, float)) and not (isinstance(x, float) a
    ↪ nd math.isnan(x))


# --- the wires ---------------------------------------------------------
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ----------------------------------------------
class Console:
    """One connection to PgBouncer's admin console, held for the whole
    ↪  arm, speaking the server's own wire protocol (the
    simple query of PostgreSQL's protocol, version 3). No process is s
    ↪ tarted for a reading (amendment 2): the first counted

    set launched a psql process for every reading, about five a second
    ↪  in the omni arm, at about 50 ms of CPU each.
    `query` returns the rows as dicts of strings (none for a command s
    ↪ uch as SET) and reconnects once on a lost connection;
    the protocol parsing is in `parse_messages`, tested without a serv
    ↪ er."""

    PROTOCOL_3 = 196608
    ROW_DESCRIPTION_TAIL = 18                                    # afte
    ↪ r a field's name: table oid, column, type oid, length, modifier,
    ↪  format

    def __init__(self, host, port, user, password="", database="pgboun
    ↪ cer", timeout=5.0):
        self.host, self.port, self.user, self.password, self.database,
    ↪  self.timeout = host, port, user, password, database, timeout
        self.sock = None
        self.logins = 0

        self.lock = threading.Lock()                                 #
    ↪ the sampler and the brain share the one connection, one query at
    ↪  a time

    # the wire
    def _send(self, kind: bytes, payload: bytes):

        self.sock.sendall(kind + struct.pack("!I", len(payload) + 4) +
    ↪  payload)

    def _recv_exact(self, n):
        buf = b""
        while len(buf) < n:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            chunk = self.sock.recv(n - len(buf))
            if not chunk:
                raise ConnectionError("the console closed the connecti
    ↪ on")
            buf += chunk
        return buf

    def _message(self):
        head = self._recv_exact(5)
        return head[:1], self._recv_exact(struct.unpack("!I", head[1:]
    ↪ )[0] - 4)

    def connect(self):
        self.close()
        self.sock = socket.create_connection((self.host, self.port), t
    ↪ imeout=self.timeout)
        params = f"user\0{self.user}\0database\0{self.database}\0\0".e
    ↪ ncode()

        self._send(b"", struct.pack("!I", self.PROTOCOL_3) + params)
        while True:
            kind, body = self._message()
            if kind == b"R":                                    # auth
    ↪ entication: ok, cleartext password, md5 password
                code = struct.unpack("!I", body[:4])[0]

                if code == 0:
                    continue
                if code == 3:
                    self._send(b"p", self.password.encode() + b"\0");
    ↪ continue
                if code == 5:

                    # the digest PostgreSQL's md5 method itself define
    ↪ s (its wire protocol), not a hash chosen for security
                    inner = hashlib.md5(self.password.encode() + self.
    ↪ user.encode(), usedforsecurity=False).hexdigest()
                    self._send(b"p", ("md5" + hashlib.md5(inner.encode
    ↪ () + body[4:8], usedforsecurity=False).hexdigest()).encode()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                                + b"\0"); continue
                raise ConnectionError(f"the console asked for authenti
    ↪ cation method {code}, which this client does not speak")
            if kind == b"E":
                raise ConnectionError(self.error_text(body))
            if kind == b"Z":                                         # read
    ↪ y for query: logged in ('S' parameters, 'K' key, 'N' notices ski
    ↪ pped)
                break
        self.logins += 1

    def close(self):

        if self.sock is not None:
            try:
                self.sock.close()
            except OSError:
                pass
            self.sock = None

    @staticmethod
    def error_text(body: bytes):
        fields = {}
        for part in body.split(b"\0"):
            if part:

                fields[chr(part[0])] = part[1:].decode(errors="replace
    ↪ ")
        return f"{fields.get('S', 'ERROR')}: {fields.get('M', '')}".st
    ↪ rip()

    @classmethod

    def parse_messages(cls, messages):
        """Rows from the messages of one simple query, as the server s
    ↪ ends them: 'T' names the columns, each 'D' is a row,
        'E' is an error (raised once 'Z' arrives), 'Z' ends the result
    ↪ . Messages are (kind, body) pairs."""
        cols, rows, err = [], [], None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        for kind, body in messages:
            if kind == b"T":
                n = struct.unpack("!H", body[:2])[0]; pos = 2; cols =
    ↪ []
                for _ in range(n):
                    end = body.index(b"\0", pos); cols.append(body[pos
    ↪ :end].decode()); pos = end + 1 + cls.ROW_DESCRIPTION_TAIL
            elif kind == b"D":
                n = struct.unpack("!H", body[:2])[0]; pos = 2; vals =
    ↪ []
                for _ in range(n):
                    ln = struct.unpack("!i", body[pos:pos + 4])[0]; po
    ↪ s += 4
                    if ln < 0:
                        vals.append(None)
                    else:
                        vals.append(body[pos:pos + ln].decode()); pos
    ↪ += ln
                rows.append(dict(zip(cols, vals)))
            elif kind == b"E":
                err = cls.error_text(body)
            elif kind == b"Z":
                if err:
                    raise RuntimeError(err)
                return rows
        raise ConnectionError("the console closed the connection befor
    ↪ e the result ended")

    def _messages_until_ready(self):
        while True:
            kind, body = self._message()
            yield kind, body
            if kind == b"Z":
                return

    def query(self, sql, retry=True):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        """The rows of one console command; one reconnect on a lost co
    ↪ nnection, never on a command the console refused."""
        with self.lock:
            return self._query(sql, retry)

    def _query(self, sql, retry):
        try:
            if self.sock is None:
                self.connect()
            self._send(b"Q", sql.encode() + b"\0")
            return self.parse_messages(self._messages_until_ready())
        except (OSError, ConnectionError, ValueError, struct.error, Un
    ↪ icodeDecodeError):   # a lost or garbled connection: once more,
    ↪ fresh
            self.close()
            if not retry:
                raise
            self.connect()
            return self._query(sql, retry=False)


class Bouncer:
    """PgBouncer as a child process with its own ini, its admin consol
    ↪ e read over one held connection (Console)."""
    def __init__(self, workdir: Path, port: int, pg: str, dbname: str,
    ↪  user: str, password: str):
        self.dir, self.port, self.user = workdir, port, user
        self.dir.mkdir(parents=True, exist_ok=True)
        self.as_postgres = os.geteuid() == 0           # PgBouncer ref
    ↪ uses to run as root
        if self.as_postgres:                            # its folder i
    ↪ s postgres's to write (log, pid, socket); never world-writable
            shutil.chown(self.dir, user="postgres")
            os.chmod(self.dir, 0o755)
        (self.dir / "userlist.txt").write_text(f'"{user}" "{password}"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ \n')
        self.ini = self.dir / "pgbouncer.ini"
        self.ini.write_text("\n".join([
            "[databases]", f"{dbname} = {pg} dbname={dbname} user={use
    ↪ r} password={password}", "", "[pgbouncer]",
            "listen_addr = 127.0.0.1", f"listen_port = {port}", f"unix
    ↪ _socket_dir = {self.dir}", "auth_type = trust",
            f"auth_file = {self.dir / 'userlist.txt'}", f"admin_users
    ↪ = {user}", "pool_mode = transaction",
            f"default_pool_size = {NATIVE_POOL}", "max_client_conn = 1
    ↪ 000", f"logfile = {self.dir / 'pgbouncer.log'}",
            f"pidfile = {self.dir / 'pgbouncer.pid'}", ""]))
        self.proc = None
        self.console = Console("127.0.0.1", port, user, password)

    def start(self):
        cmd = ["pgbouncer", "-d", str(self.ini)]
        if self.as_postgres:
            cmd = ["runuser", "-u", "postgres", "--"] + cmd
        subprocess.run(cmd, check=True)
        for _ in range(50):
            time.sleep(0.1)
            try:
                self.console.connect()
                self.console.query("SHOW VERSION", retry=False)
                return
            except (OSError, ConnectionError, RuntimeError):
                continue
        raise RuntimeError("PgBouncer did not come up")

    def admin(self, sql, quiet=False):
        """One console command; the rows, or None if the console refus
    ↪ ed it (said on stderr unless quiet)."""
        try:
            return self.console.query(sql)
        except (OSError, ConnectionError, RuntimeError) as e:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            if not quiet:
                sys.stderr.write(f"{sql}: {e}\n")
            return None

    def rows(self, sql):
        return self.console.query(sql)

    def pool_size(self):
        for r in self.rows("SHOW CONFIG"):
            if r.get("key") == "default_pool_size":
                return int(r["value"])
        raise RuntimeError("SHOW CONFIG gave no default_pool_size")

    def set_pool_size(self, n):
        self.console.query(f"SET default_pool_size = {int(n)}")

    def pools(self, dbname):
        for r in self.rows("SHOW POOLS"):
            if r.get("database") == dbname:
                return {k: int(v) for k, v in r.items() if k.startswit
↪ h(("cl_", "sv_", "maxwait")) and v is not None and v.lstrip("-")
↪ .isdigit()}
        return {}

    def stats(self, dbname):
        for r in self.rows("SHOW STATS"):
            if r.get("database") == dbname:
                return {k: int(v) for k, v in r.items() if k.startswit
↪ h("total_") and v is not None and v.lstrip("-").isdigit()}
        return {}

    def stop(self):
        try:
            self.console.query("SHUTDOWN", retry=False)
        except (OSError, ConnectionError, RuntimeError):
            pass                                                # the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ console closes as it shuts down
        self.console.close()
        pid = self.dir / "pgbouncer.pid"
        for _ in range(30):
            if not pid.exists():
                return

            time.sleep(0.1)


class PoolPlug(Plug):
    """One wire in (the pooler's own service time per transaction), on
    ↪ e wire out (its pool size)."""

    def __init__(self, b: Bouncer, dbname: str, line_ms=LINE_MS):
        super().__init__(FLOOR, CEILING, tolerance=0.5)
        self.b, self.db, self.line_s = b, dbname, line_ms / 1000.0
        self.prev = None
        self.wait_share = 0.0

        self.waiting_share = 0.0
        self.last_latency_s = 0.0
        self.last_pools = {}       # the SHOW POOLS row of the last re
    ↪ ading, so the brain reads it once a second (amendment 2)
        self.queue_share = 0.0     # the share of the pooler's time it
    ↪ s clients spent waiting for a server, last second (amendment 2)

        self.served = False        # whether the pooler counted a tran
    ↪ saction in the last second (amendment 2)
        self.last_xacts = 0        # the transactions the pooler count
    ↪ ed in the last second: the brain's work sample (amendment 3)


    def _read_service(self):

        s = self.b.stats(self.db)
        p = self.b.pools(self.db)
        self.last_pools = p
        clients = p.get("cl_active", 0) + p.get("cl_waiting", 0)
        self.waiting_share = p.get("cl_waiting", 0) / clients if clien
    ↪ ts else 0.0   # clients queued for a server (amendment 1)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if not s:
            self.served, self.last_xacts = False, 0
            return service_reading(self.last_latency_s, self.waiting_s
    ↪ hare, self.line_s)
        if self.prev is None:
            self.prev = s

            self.served, self.last_xacts = False, 0
            return service_reading(0.0, self.waiting_share, self.line_
    ↪ s)
        dx = s["total_xact_count"] - self.prev["total_xact_count"]
        dt = s["total_xact_time"] - self.prev["total_xact_time"]
        dw = s["total_wait_time"] - self.prev["total_wait_time"]

        self.prev = s
        if dx <= 0:
            self.wait_share, self.last_latency_s = self.waiting_share,
    ↪  0.0       # nothing served: calm unless clients are queued
            self.served, self.queue_share, self.last_xacts = False, (1
    ↪ .0 if dw > 0 else 0.0), 0

            return service_reading(0.0, self.waiting_share, self.line_
    ↪ s)
        self.served, self.last_xacts = True, dx
        self.queue_share = dw / (dt + dw) if (dt + dw) > 0 else 0.0
        self.wait_share = max(self.queue_share, self.waiting_share)
        self.last_latency_s = (dt + dw) / dx / 1e6

        return service_reading(self.last_latency_s, self.waiting_share
    ↪ , self.line_s)

    def _read_lever(self):
        return float(self.b.pool_size())

    def _send(self, value):
        self.b.set_pool_size(int(round(value)))


# omni-v22: the max on the pooler's pool (docs/POSTGRES_PREREGISTRATIO
    ↪ N.md, amendment 4): the wire's own rule (decide) with a server
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# added while clients wait never clamped by an allowance, up to the co
    ↪ ver, and the pool never taken below the pooler's own setting
# (native); the hand-back by the queue line (amendment 2) above it. Th
    ↪ e servers it holds are power given, shown beside the waiting
# they cut. `--dial save` keeps omni-v21's loop, the verdict under the
    ↪  resource objective, where giving servers back is the aim.

DIAL = os.environ.get("OMNI_DIAL", "max")


class Brain:
    """The knob's decision, one a second: obs holds reading_s (the poo
    ↪ ler's service time per transaction, wait included),

    wait_share, idle (idle servers), pool (the pool size now), queue_s
    ↪ hare and served (amendment 2's queue line)."""

    def __init__(self, snapshot, objective=DEFAULT_OBJECTIVE, line_ms=
    ↪ LINE_MS):
        self.snapshot, self.objective = int(snapshot), objective

        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE
    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)
        self.last_dir, self.last_change = 0, -1e9
        self.counts = {"spend": 0, "failups": 0, "handed_back": 0, "he
    ↪ ld_native": 0, "hold": 0}

    def step(self, t, obs):
        cur = int(obs["pool"])
        f = self.law.force(obs["reading_s"])
        want, d, why = decide(self.law.p, f, obs["wait_share"], obs["i
    ↪ dle"], cur, self.snapshot, self.last_dir, t - self.last_change,
                              wall=self.law.band.wall_high, queue_shar

    ↪ e=obs.get("queue_share", 0.0), served=obs.get("served", True))
        if want > cur:
            target = want; self.counts["spend"] += 1
    ↪ # clients waiting for a server: never clamped, up to the cover
            if why.startswith("fail up"):
                self.counts["failups"] += 1
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            why += "; power given inside the cover"
        elif want < cur:
            if want >= self.snapshot:
                target = want; self.counts["handed_back"] += 1
            elif cur > self.snapshot:
                target = self.snapshot; self.counts["handed_back"] +=
    ↪ 1
                why = "calm: handed back to the pooler's own setting,
    ↪ never below it"
            else:
                target = cur; self.counts["held_native"] += 1
                why = f"hold: {cur} is the pooler's own setting; the m
    ↪ ax never goes below native"
        else:
            target = cur; self.counts["hold"] += 1
        if target != cur:
    ↪ # the dwell runs from the last change, as the loop's own clock d
    ↪ id
            self.last_dir, self.last_change = (0, t) if why.startswith
    ↪ ("fail up") else ((target > cur) - (target < cur), t)
        info = {"verdict_phase": None, "verdict_direction": None, "all
    ↪ owed_low": None, "allowed_high": None, "dial": "max",
                "p": round(self.law.p, 4), "force": round(f, 4)}
        return target, why, info

    def record(self):
        return {"dial": "max", "objective": self.objective, "native":
    ↪ self.snapshot, "notch": 1, "cover": [FLOOR, CEILING],
                "state": "the max: a server added while clients wait,
    ↪ never clamped; never below the pooler's own setting; no allowanc
    ↪ e",
                "allowed_low": None, "allowed_high": None, "counts": d
    ↪ ict(self.counts)}


class Omni(threading.Thread):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    """The brain, one decision a second, writing the audit. Under the
    ↪ max (omni-v22, the factory tune) the Brain above decides;
    under the tune-down (`--dial save`) amendment 3's loop (the brain'
    ↪ s own verdict, 2026-10-09): the pool starts in
    watch and is written only inside the allowance a paired trial on t
    ↪ he pooler itself has earned under the declared objective

    (tools/knob_verdict.py around the engine's own Verdict): one serve
    ↪ r fewer while calm, one more while clients wait, each step
    judged on the pooler's own second-by-second readings; a trial hold
    ↪ s the pool; the fail-up back to the operator's setting is
    always free, a spend beyond the allowance never."""

    def __init__(self, plug: PoolPlug, audit_path: Path, line_ms=LINE_
    ↪ MS, objective=DEFAULT_OBJECTIVE, dial=DIAL):
        super().__init__(daemon=True)
        self.plug, self.audit, self.objective, self.dial = plug, audit
    ↪ _path, objective, dial
        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE

    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)
        self.stop_flag = threading.Event()
        self.last_dir, self.last_change = 0, -1e9
        self.writes, self.failups = 0, 0
        self.foreign, self.handed_back = False, False
        self.error = None                                           #

    ↪ a fault before the first decision, written into the arm's record
        self.verdict = None; self.brain = None

    def govern_record(self):
        """The record of what governed the knob: the max's counts (omn
    ↪ i-v22) or the verdict's allowance (the tune-down)."""

        if self.brain is not None:
            return self.brain.record()
        return self.verdict.record() if self.verdict else None

    def run(self):
        try:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        snapshot = self.plug.attach()
    except Exception as e:                                          #
↪ the brain never ran: say so in the audit and the record, never s
↪ ilently
        self.error = repr(e)
        self.audit.write_text(json.dumps({"t": 0.0, "error": self.

↪ error, "note": "the brain could not attach; no decision was made
↪ "}) + "\n")
        return
    t0 = time.monotonic(); cpu_prev = cpu_times()
    self.verdict = KnobVerdict(int(snapshot), 1, (FLOOR, CEILING),
↪  objective=self.objective, settle_s=TAU)

    if self.dial == "max":
        self.brain = Brain(int(snapshot), self.objective); self.ve
↪ rdict = None
    with open(self.audit, "w") as fh:
        while not self.stop_flag.is_set():
            tick = time.monotonic()

            try:
                reading = self.plug.read()
                f = self.law.force(reading)
                idle = self.plug.last_pools.get("sv_idle", 0)
↪          # the reading's own SHOW POOLS row (amendment 2)
                cur = int(self.plug.lever())

                cpu_now = cpu_times(); d_busy = cpu_now[0] - cpu_p
↪ rev[0]; d_total = cpu_now[1] - cpu_prev[1]; cpu_prev = cpu_now
                cpu_share = (d_busy / d_total) if d_total > 0 else
↪  None
                cost = sample_cost(self.objective, self.plug.last_
↪ xacts, self.plug.last_latency_s if self.plug.served else None, c

↪ ur, cpu_share)
                if self.brain is not None:
↪                    # the max (omni-v22): the Brain keeps its own
↪  dwell clock
                    target, why, vinfo = self.brain.step(tick - t0
↪ , {"reading_s": reading, "wait_share": self.plug.wait_share, "id
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ le": idle, "pool": cur,

    ↪     "queue_share": self.plug.queue_share, "served": self.plug.se
    ↪ rved})
                        fail_up = why.startswith("fail up")
                    else:

    ↪                    # the tune-down: omni-v21's loop
                        self.verdict.observe(cost, tick - t0)
    ↪               # measured under the pool of the last second
                        target, d, why = decide(self.law.p, f, self.pl
    ↪ ug.wait_share, idle, cur, int(snapshot), self.last_dir,
                                                tick - self.last_chang

    ↪ e, wall=self.law.band.wall_high,
                                                queue_share=self.plug.
    ↪ queue_share, served=self.plug.served)
                        fail_up = why.startswith("fail up")
                        target, why, vinfo = self.verdict.decide(
                            target, spend_ok=(f > DEAD and self.plug.w

    ↪ ait_share >= WAIT_SHARE),
                            give_ok=(f < -DEAD and self.plug.served an
    ↪ d self.plug.queue_share < GIVEBACK_WAIT_SHARE and idle >= 1),
                            t=tick - t0, why=why, stress=f > DEAD, cal
    ↪ m=f < -DEAD, fail_up=fail_up)
                    d = (target > cur) - (target < cur)

                    wrote = None
                    if target != cur:
                        wrote = self.plug.write(target)
                        self.writes += 1
                        if fail_up:
                            self.failups += 1

                            self.last_dir, self.last_change = 0, tick
                        else:
                            self.last_dir, self.last_change = d, tick
                    fh.write(json.dumps({"t": round(tick - t0, 2), "re
    ↪ ading_ms": round(reading * 1000, 3), "latency_ms": round(self.pl
    ↪ ug.last_latency_s * 1000, 3),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                                            "waiting_share": round(self.p
    ↪ lug.waiting_share, 3), "p": round(self.law.p, 4),
                                            "force": round(f, 4), "wait_s
    ↪ hare": round(self.plug.wait_share, 3),
                                            "queue_share": round(self.plu
    ↪ g.queue_share, 4), "served": self.plug.served, "idle": idle,
                                            "xacts_last_s": self.plug.las
    ↪ t_xacts, "cpu_share": None if cpu_share is None else round(cpu_s
    ↪ hare, 4),
                                            "cost": None if cost is None
    ↪ else round(cost, 9), **vinfo,
                                            "pool": cur, "target": target
    ↪ , "wrote": wrote, "why": why}) + "\n")
                except Exception as e:  # a foreign writer or a lost c
    ↪ onsole: say so, stop writing
                    fh.write(json.dumps({"t": round(tick - t0, 2), "er
    ↪ ror": repr(e)}) + "\n")
                    self.foreign = True
                    break
                time.sleep(max(0.0, DT - (time.monotonic() - tick)))
        self.handed_back = self.plug.restore()


class Sampler(threading.Thread):
    """Server connections alive, once a second, in both arms."""

    def __init__(self, b: Bouncer, dbname: str):
        super().__init__(daemon=True)
        self.b, self.db, self.stop_flag, self.alive, self.pool = b, db
    ↪ name, threading.Event(), [], []
    def run(self):
        while not self.stop_flag.is_set():
            p = self.b.pools(self.db)
            if p:
                self.alive.append(sum(p.get(k, 0) for k in ("sv_active
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ", "sv_idle", "sv_used", "sv_tested", "sv_login")))
            try:
                self.pool.append(self.b.pool_size())
            except Exception:
                pass
            time.sleep(DT)

def cpu_times():
    """(busy, total) jiffies of the whole host from /proc/stat."""
    with open("/proc/stat") as fh:
        parts = fh.readline().split()

    v = [int(x) for x in parts[1:]]
    idle = v[3] + (v[4] if len(v) > 4 else 0)
    return sum(v) - idle, sum(v)


def pgbench(args, env, log=None):

    r = subprocess.run(["pgbench"] + args, capture_output=True, text=T
    ↪ rue, env=env)
    if log is not None:
        log.write_text(r.stdout + r.stderr)
    return r

def parse_summary(text):
    out = {"tps": None, "failed": 0}
    for line in text.splitlines():
        if line.startswith("tps = "):
            out["tps"] = float(line.split("=")[1].split("(")[0])

        if line.startswith("number of failed transactions:"):
            out["failed"] = int(line.split(":")[1].split("(")[0])
    return out


# --- one workload ----------------------------------------------------
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ----------------------------------------------
def run_arm(arm, b, wl_dir, rep, dbname, user, port, script, steps, st
    ↪ ep_s, base, clients, jobs, line_ms, objective=DEFAULT_OBJECTIVE,
    ↪  prepare=None):
    d = wl_dir / f"rep-{rep}" / arm
    d.mkdir(parents=True, exist_ok=True)

    env = {**os.environ, "PGPASSWORD": "x"}
    conn = ["-h", "127.0.0.1", "-p", str(port), "-U", user]
    assert b.pool_size() == NATIVE_POOL, "every arm starts from the po
    ↪ oler's own setting"
    prep = prepare(d) if prepare is not None else None            # s
    ↪ tore fix of 25 October: the tables re-initialised and a CHECKPOI

    ↪ NT, both arms alike
    # an uncounted warm-up at the base rate, both arms alike
    pgbench(conn + ["-c", str(clients), "-j", str(jobs), "-T", "5", "-
    ↪ R", f"{base:.1f}", "-n"] + script + [dbname], env)
    sampler = Sampler(b, dbname); sampler.start()
    omni = None

    if arm == "omni":
        omni = Omni(PoolPlug(b, dbname, line_ms), d / "audit.jsonl", l
    ↪ ine_ms, objective); omni.start()
    cpu0, t0 = cpu_times(), time.monotonic()
    own0 = resource.getrusage(resource.RUSAGE_SELF)
    lat, failed, per_step = [], 0, []

    for k, notch in enumerate(steps):
        rate = base * notch
        prefix = d / f"step-{k:02d}"
        r = pgbench(conn + ["-c", str(clients), "-j", str(jobs), "-T",
    ↪  str(step_s), "-R", f"{rate:.1f}", "-n", "-l",
                            f"--log-prefix={prefix}"] + script + [dbnam

    ↪ e], env, d / f"step-{k:02d}.txt")
        s = parse_summary(r.stdout)
        files = sorted(d.glob(f"step-{k:02d}.*[0-9]"))
        l = parse_latencies(files)
        (d / f"step-{k:02d}.agg").write_text("\n".join(aggregate_per_s
    ↪ econd(parse_log_rows(files))) + "\n")   # every second of the st
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ep kept
        g = latency_gauges(l, step_s, line_ms)
        per_step.append({"step": k, "notch": notch, "offered_tps": rat
    ↪ e, **g, "failed": s["failed"], "exit": r.returncode})
        lat += l; failed += s["failed"]
        for f in files:

            f.unlink()                                                  # t
    ↪ he per-step gauges are kept; the raw log is large
    seconds = time.monotonic() - t0
    cpu1 = cpu_times()
    own1 = resource.getrusage(resource.RUSAGE_SELF)
    sampler.stop_flag.set(); sampler.join(timeout=5)

    g = latency_gauges(lat, step_s * len(steps), line_ms)
    g.update({"failed": failed, "seconds": seconds, "prepared": prep,
              "harness_cpu_seconds": (own1.ru_utime + own1.ru_stime) -
    ↪  (own0.ru_utime + own0.ru_stime),
              "console_logins": b.console.logins,
              "servers_alive_mean": sum(sampler.alive) / len(sampler.a

    ↪ live) if sampler.alive else float("nan"),
              "servers_alive_max": max(sampler.alive) if sampler.alive
    ↪  else float("nan"),
              "cpu_busy_share": (cpu1[0] - cpu0[0]) / max(1, cpu1[1] -
    ↪  cpu0[1]),
              "cpu_seconds": (cpu1[0] - cpu0[0]) / os.sysconf("SC_CLK_

    ↪ TCK"),
              "pool_mean": sum(sampler.pool) / len(sampler.pool) if sa
    ↪ mpler.pool else float("nan"),
              "pool_min": min(sampler.pool) if sampler.pool else None,
    ↪  "pool_max": max(sampler.pool) if sampler.pool else None,
              "steps": per_step})

    inside = g["work_inside_line_tps"] * step_s * len(steps)
    g["cpu_s_per_1k_inside"] = g["cpu_seconds"] / inside * 1000 if ins
    ↪ ide else float("nan")
    if omni is not None:
        omni.stop_flag.set(); omni.join(timeout=10)
        g.update({"writes": omni.writes, "fail_ups": omni.failups, "ha
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ nded_back": bool(omni.handed_back),
                  "foreign_writer": omni.foreign, "controller_error":
    ↪ omni.error, "pool_read_back": b.pool_size(),
                  "verdict": omni.govern_record()})
    else:
        g.update({"writes": 0, "handed_back": True, "pool_read_back":

    ↪ b.pool_size()})
    if b.pool_size() != NATIVE_POOL:                              # bel
    ↪ t and braces: the next arm starts native
        b.set_pool_size(NATIVE_POOL)
    (d / "arm.json").write_text(json.dumps(g, indent=1))
    return g

def run_workload(name, out, pg, dbname, user, password, port, reps, st
    ↪ eps, step_s, clients, jobs, calib_s, line_ms, objective=DEFAULT_
    ↪ OBJECTIVE):
    script_flag, scale = WORKLOADS[name]

    script = [script_flag] if script_flag else []
    wl_dir = out / f"pgbench-{name}"
    wl_dir.mkdir(parents=True, exist_ok=True)
    env = {**os.environ, "PGPASSWORD": password}
    direct = ["-h", pg_host(pg), "-p", pg_port(pg), "-U", user]
    r = pgbench(direct + ["-i", "-s", str(scale), "-q", dbname], env,

    ↪ wl_dir / "init.txt")
    if r.returncode != 0:
        raise SystemExit(f"pgbench -i failed: {r.stderr[-400:]}")
    b = Bouncer(wl_dir / "pgb", port, pg, dbname, user, password)
    b.start()
    try:

        snapshot = b.pool_size()
        assert snapshot == NATIVE_POOL
        # the base rate: native's own unlimited capacity, found once b
    ↪ efore the counted runs, over the peak notch
        cal = pgbench(["-h", "127.0.0.1", "-p", str(port), "-U", user,
    ↪  "-c", str(clients), "-j", str(jobs), "-T", str(calib_s), "-n"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                    + script + [dbname], {**os.environ, "PGPASSWORD"
↪ : "x"}, wl_dir / "calibration.txt")
    cap = parse_summary(cal.stdout)["tps"]
    if not cap:
        raise SystemExit(f"calibration gave no tps: {cal.stderr[-3
↪ 00:]}")

    base = PEAK * cap / max(steps)
    print(f"== {name}: native capacity {cap:.0f} tps unlimited; ba
↪ se rate {base:.0f} tps (peak notch {100 * PEAK:.0f}% of it); ste
↪ ps {steps}", flush=True)
    rec = {"workload": name, "script": script_flag or "tpcb (defau
↪ lt)", "scale": scale, "line_ms": line_ms, "steps": steps,

          "step_s": step_s, "clients": clients, "jobs": jobs, "na
↪ tive_capacity_tps": cap, "base_tps": base,
          "snapshot_pool": snapshot, "cover": [FLOOR, CEILING], "
↪ center": CENTER, "objective": objective, "dial": DIAL, "engine":
↪  engine(), "reps": []}
    prepare = lambda d: reset_tables(pg, dbname, user, password, s

↪ cale, d)   # noqa: E731  (the same state before every arm)
    for rep in range(1, reps + 1):
        order = ["native", "omni"] if rep % 2 else ["omni", "nativ
↪ e"]
        one = {"rep": rep, "order": order}
        for arm in order:

            one[arm] = run_arm(arm, b, wl_dir, rep, dbname, user,
↪ port, script, steps, step_s, base, clients, jobs, line_ms, objec
↪ tive, prepare=prepare)
            print(f"== {name} rep {rep} {arm}: inside the line {on
↪ e[arm]['work_inside_line_tps']:.0f} tps, p95 {one[arm]['p95_ms']
↪ :.1f} ms, "

                  f"servers {one[arm]['servers_alive_mean']:.1f},
↪ pool {one[arm]['pool_mean']:.1f}, CPU {100 * one[arm]['cpu_busy_
↪ share']:.0f}%", flush=True)
        rec["reps"].append(one)
        (wl_dir / f"{name}.json").write_text(json.dumps(rec, inden
↪ t=1))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        rec["paired"] = {k: paired(rec["reps"], k, dr) for k, _, dr in
    ↪  GAUGES}
        (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1)
    ↪ )
        return rec
    finally:
        b.stop()


def checkpoint_command(dbname):
    """CHECKPOINT as the postgres superuser (the bench role holds no r
    ↪ ight to it), the way --setup reaches the server."""
    pre = ["runuser", "-u", "postgres", "--"] if os.geteuid() == 0 els
    ↪ e ["sudo", "-n", "-u", "postgres"]
    return pre + ["psql", "-qX", "-d", dbname, "-c", "CHECKPOINT"]


def reset_tables(pg, dbname, user, password, scale, d, run=subprocess.
    ↪ run):
    """Before every arm, both arms alike (store fix of 25 October): pg
    ↪ bench's tables re-initialised at the workload's scale and
    a CHECKPOINT, so each arm starts from the same tables and the same
    ↪  point in the checkpoint cycle (the omni-v16 runs
    initialised once a workload and ran six arms, about 45 minutes, on
    ↪  the growing tables, the checkpoints falling wherever
    they fell). Returns what was done, for the arm's record."""
    env = {**os.environ, "PGPASSWORD": password}
    direct = ["-h", pg_host(pg), "-p", pg_port(pg), "-U", user]
    t0 = time.monotonic()
    r = run(["pgbench"] + direct + ["-i", "-s", str(scale), "-q", dbna
    ↪ me], capture_output=True, text=True, env=env)
    c = run(checkpoint_command(dbname), capture_output=True, text=True
    ↪ )
    (Path(d) / "prepare.txt").write_text((r.stdout or "") + (r.stderr
    ↪ or "") + "\n" + (c.stdout or "") + (c.stderr or ""))
    return {"reinit_exit": r.returncode, "checkpoint_exit": c.returnco
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ de, "seconds": round(time.monotonic() - t0, 2)}


def pg_host(pg):
    return dict(kv.split("=", 1) for kv in pg.split()).get("host", "12
    ↪ 7.0.0.1")

def pg_port(pg):
    return dict(kv.split("=", 1) for kv in pg.split()).get("port", "54
    ↪ 32")

def engine():
    try:
        sha = subprocess.run(["git", "rev-parse", "HEAD"], cwd=ROOT, c
    ↪ apture_output=True, text=True).stdout.strip()
    except Exception:

        sha = ""
    try:
        v = subprocess.run([sys.executable, str(ROOT / "tools" / "omni
    ↪ _version.py")], cwd=ROOT, capture_output=True, text=True)
        ver = v.stdout.strip().splitlines()[0] if v.stdout.strip() els
    ↪ e "unknown"

    except Exception:
        ver = "unknown"
    return {"commit": sha, "version": ver}


# --- the report ----------------------------------------------------

    ↪ ------------------------------------------------
def pct(r, key):
    if r is None:
        return "no value"
    nat, diff, lo, hi = r["native"], r["diff"], r["ci95"][0], r["ci95"
    ↪ ][1]

    if key in ("failed",) or not nat:
        return f"{diff:+.3g} ({lo:+.3g} to {hi:+.3g})"
    return f"{100 * diff / abs(nat):+.1f}% ({100 * lo / abs(nat):+.1f}
    ↪  to {100 * hi / abs(nat):+.1f})"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def fmt(v):
    if v is None or (isinstance(v, float) and math.isnan(v)):
        return "n/a"
    return f"{v:.3g}" if abs(v) < 10 else f"{v:,.1f}" if abs(v) < 1000
    ↪  else f"{v:,.0f}"

def report(recs, out: Path):
    L = ["# PostgreSQL behind PgBouncer: native and omni (Omni-Compass
    ↪  on top of the pooler's own pool size)", "",
         "Native is PostgreSQL as shipped behind PgBouncer's shipped p
    ↪ ool of 20 server connections; omni is the same with the compass

    ↪ law "
         f"writing the pool size through PgBouncer's own console insid
    ↪ e [{FLOOR}, {CEILING}], handed back at the end. The load is pgbe
    ↪ nch, "
         "rate-limited, stepping one notch at a time, the peak notch o
    ↪ ffering native's own unlimited capacity. Every row is shown, los

    ↪ ses "
         "included; a 95% interval of the paired difference (omni minu
    ↪ s native) over the repetitions decides whether a change is beyon
    ↪ d the "
         "noise. There is no watt-meter on the machine: the host's CPU
    ↪ -seconds are measured, no energy is claimed "

         "(`docs/POSTGRES_PREREGISTRATION.md`).", ""]
    for rec in recs:
        e = rec.get("engine", {})
        L += [f"## {rec['workload']} ({rec['script']}, scale {rec['sca
    ↪ le']}): line {rec['line_ms']:.0f} ms, {len(rec['reps'])} paired
    ↪ repetitions", "",

              f"Native capacity, unlimited: {rec['native_capacity_tps'
    ↪ ]:,.0f} tps; base rate {rec['base_tps']:,.0f} tps; steps {rec['s
    ↪ teps']} × {rec['step_s']} s; "
              f"{rec['clients']} clients. Engine {e.get('version', 'un
    ↪ known')} at `{e.get('commit', '')[:12]}`.", "",
              "| Gauge | native | omni | change (95% interval) | readi
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ng |", "|---|---:|---:|---|---|"]
        for k, label, dr in GAUGES:
            r = (rec.get("paired") or {}).get(k)
            if r is None:
                continue
            L.append(f"| {label} | {fmt(r['native'])} | {fmt(r['omni']
    ↪ )} | {pct(r, k)} | {r['reading']} |")
        hb = all(x["omni"].get("handed_back") for x in rec["reps"])
        fw = any(x["omni"].get("foreign_writer") for x in rec["reps"])
        L += ["", f"The knob handed back and read back at the end of e
    ↪ very omni arm: {'yes' if hb else '**NO**'}; "
              f"a foreign writer seen: {'**yes**' if fw else 'no'}; Om
    ↪ ni's writes per arm: "
              + ", ".join(str(x["omni"].get("writes", 0)) for x in rec
    ↪ ["reps"])
              + "; fail-ups per arm: " + ", ".join(str(x["omni"].get("
    ↪ fail_ups", 0)) for x in rec["reps"]) + ".", "",
              "Per repetition (inside the line tps / p95 ms / servers
    ↪ alive / pool mean):", ""]
        for x in rec["reps"]:
            n, o = x["native"], x["omni"]
            L.append(f"- rep {x['rep']} ({' then '.join(x['order'])}):
    ↪  native {n['work_inside_line_tps']:.0f} / {n['p95_ms']:.1f} / "
                     f"{n['servers_alive_mean']:.1f} / {n['pool_mean']
    ↪ :.1f}; omni {o['work_inside_line_tps']:.0f} / {o['p95_ms']:.1f}
    ↪ / "
                     f"{o['servers_alive_mean']:.1f} / {o['pool_mean']
    ↪ :.1f}")
        L.append("")
    (out / "PGBENCH.md").write_text("\n".join(_legal_stamp(L)) + "\n")

def setup(pg, dbname, user, password):
    """The bench role and database, through the postgres superuser."""
    pre = ["runuser", "-u", "postgres", "--"] if os.geteuid() == 0 els
    ↪ e ["sudo", "-u", "postgres"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    sql = (f"DO $$ BEGIN IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE r
    ↪ olname='{user}') THEN CREATE ROLE {user} LOGIN PASSWORD '{passwo
    ↪ rd}'; "
           f"END IF; END $$;")
    subprocess.run(pre + ["psql", "-qc", sql], check=True)
    have = subprocess.run(pre + ["psql", "-tAc", f"SELECT 1 FROM pg_da

    ↪ tabase WHERE datname='{dbname}'"], capture_output=True, text=Tru
    ↪ e).stdout.strip()
    if have != "1":
        subprocess.run(pre + ["createdb", "-O", user, dbname], check=T
    ↪ rue)
    print(f"role {user} and database {dbname} ready")

def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--workloads", default="tpcb")

    ap.add_argument("--reps", type=int, default=3)
    ap.add_argument("--steps", default=STEPS)
    ap.add_argument("--step-s", type=int, default=20)
    ap.add_argument("--clients", type=int, default=64)
    ap.add_argument("--jobs", type=int, default=4)
    ap.add_argument("--calib-s", type=int, default=20)

    ap.add_argument("--line-ms", type=float, default=LINE_MS)
    ap.add_argument("--pg", default="host=127.0.0.1 port=5432")
    ap.add_argument("--dbname", default="bench")
    ap.add_argument("--user", default="bench")
    ap.add_argument("--password", default="bench")
    ap.add_argument("--port", type=int, default=6543, help="the port o

    ↪ ur PgBouncer listens on")
    ap.add_argument("--out", default="out")
    ap.add_argument("--objective", default=DEFAULT_OBJECTIVE, choices=
    ↪ OBJECTIVES,
                    help="what the brain's verdict judges a step by: r
    ↪ esource (the index's reading: work, speed, connections and CPU t
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ogether) or service (work and speed alone)")
    ap.add_argument("--report-only", default="")
    ap.add_argument("--setup", action="store_true")
    ap.add_argument("--smoke", action="store_true", help="a one-minute
    ↪  pass: short steps, one repetition")
    a = ap.parse_args(argv)

    if a.setup:
        setup(a.pg, a.dbname, a.user, a.password)
        return 0
    out = Path(a.out); out.mkdir(parents=True, exist_ok=True)
    if a.report_only:
        recs = [json.loads(f.read_text()) for f in sorted(Path(a.repor

    ↪ t_only).glob("pgbench-*/*.json"))]
        recs = [r for r in recs if "reps" in r]
        for r in recs:
            r["paired"] = {k: paired(r["reps"], k, dr) for k, _, dr in
    ↪  GAUGES}
        report(recs, out)

        return 0
    for tool in ("pgbench", "psql", "pgbouncer"):
        if shutil.which(tool) is None:
            raise SystemExit(f"{tool} not found")
    steps = [int(x) for x in a.steps.split()]
    if a.smoke:

        a.reps, a.step_s, a.calib_s, steps = 1, 4, 4, [1, 2, 3, 2, 1]
    recs = []
    for name in a.workloads.split(","):
        if name not in WORKLOADS:
            raise SystemExit(f"unknown workload {name}; one of {sorted
    ↪ (WORKLOADS)}")

        recs.append(run_workload(name, out, a.pg, a.dbname, a.user, a.
    ↪ password, a.port, a.reps, steps, a.step_s, a.clients, a.jobs,
                                 a.calib_s, a.line_ms, a.objective))
    report(recs, out)
    print(f"report: {out / 'PGBENCH.md'}")
    return 0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
if __name__ == "__main__":
    sys.exit(main())
```

H.56 tools/run_redis.py

653 lines.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES

    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of a Redis cache's operator-set memory ceiling
    ↪ (docs/REDIS_PREREGISTRATION.md).

Redis as shipped, with the two settings an operator sets once for a ca
    ↪ che: a memory ceiling (maxmemory) and the eviction
rule (allkeys-lru). An application in front of it asks for keys drawn
    ↪ from a working set that widens and narrows over the
run; a hit is answered by Redis, a miss costs the application a trip t
    ↪ o the store behind the cache (a declared fixed

delay) and the value is written back. The operator's fixed ceiling is
    ↪ the native controller here.

  native  Redis at the operator's ceiling (64 MB) for the whole run
  omni    the same Redis, with the compass law (omnicompass/compass_la
    ↪ w.py) on one knob, the memory ceiling, through Redis's

          own console (CONFIG SET maxmemory), inside the cover [16 MB,
    ↪  512 MB]: the compass reads the application's own
          request latency (hits fast, misses slow) on a band from 0 to
    ↪  the response line and holds it at 40% of the line;
          misses with the cache full push the ceiling up (a miss with
    ↪ room to spare is a cold miss, which no ceiling can
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
          mend), calm gives memory back 8 MB a second when nothing is
    ↪ being evicted; at 95% of the line with the cache full
          a quarter of the cover is added at once (fail up); the ceili
    ↪ ng is handed back to the operator's at the end and
          read back; a ceiling found at a value Omni did not write sto
    ↪ ps it writing (one writer)

Gauges from the application's own records (every request's latency, hi
    ↪ t or miss), Redis's own INFO (memory used, the
ceiling, evictions, hits and misses) and the host's /proc/stat (CPU se
    ↪ conds, the compass's own cost included). Memory held
is the resource; no energy is claimed beyond the host's CPU seconds. E

    ↪ very row is reported, losses included.

  python3 tools/run_redis.py --setup                      # start a Re
    ↪ dis for the run (the workflow does this)
  python3 tools/run_redis.py --workloads tuning --reps 3 --out DIR
  python3 tools/run_redis.py --report-only DIR            # re-read fi

    ↪ nished <workload>.json files and write REDIS.md
"""
from __future__ import annotations

import argparse
import json

import math
from collections import deque
import os
import random
import subprocess
import sys

import threading
import time
from pathlib import Path

try:                                                     # the legal
    ↪  notice every generated report carries
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    from tools.legal import stamp as _legal_stamp
except ImportError:
    import pathlib as _p; sys.path.insert(0, str(_p.Path(__file__).res
    ↪ olve().parents[1])); from tools.legal import stamp as _legal_sta
    ↪ mp
ROOT = Path(__file__).resolve().parents[1]

sys.path.insert(0, str(ROOT))
from omnicompass.compass_law import Band, CompassLaw  # noqa: E402
from tools.knob_verdict import KnobVerdict, sample_cost, OBJECTIVES, R
    ↪ ESOURCE as DEFAULT_OBJECTIVE  # noqa: E402  (amendment 2: the br
    ↪ ain's own verdict)
from tools.store_brain import MemoryMax, read_meminfo, HOLD_WINDOW_S,

    ↪ MISS_WORTH  # noqa: E402  (omni-v22: the max on the ceiling)
from tools.readings import word as gel_word, aim as gel_aim, EVEN_SAME
    ↪ , EVEN_NOISE, SHOWN  # noqa: E402  (the words every table uses)
from tools.run_ycsb import host_cpu_split, pid_of, proc_cpu_jiffies  #
    ↪  noqa: E402  (the CPU apart, the same reading as the other store
    ↪ s)

HOST = os.environ.get("REDIS_HOST", "127.0.0.1")
PORT = int(os.environ.get("REDIS_PORT", "6390"))
MB = 1024 * 1024
LINE_MS = 2.0             # the response line: a request answered with
    ↪ in 2 ms (a hit is well inside, a miss well outside)

CENTER = 0.4              # the compass holds the application's latenc
    ↪ y at 40% of the line
DT = 1.0                  # one decision a second
TAU = 2.0                 # memory given or taken shows in the misses
    ↪ within a couple of seconds
SMOOTH = 0.5

NATIVE_MB = 64            # the operator's ceiling
NATIVE = "operator"        # --native shipped: Redis exactly as shippe
    ↪ d, no ceiling (maxmemory 0) and noeviction
POLICY = {"operator": "allkeys-lru", "shipped": "noeviction"}
COVER_MB = (16, 512)      # never under 16 MB, never over 512 MB
STEP_MB = 8               # one notch of the knob
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
FAILUP_MB = 128           # past the wall: a quarter of the cover at o
    ↪ nce, and again the next second if still past it
DWELL_S = 5.0             # after growing, no taking back within five
    ↪ seconds (no hunting); giving back is a notch a second
DIAL = os.environ.get("OMNI_DIAL", "max")   # omni-v22: max, the facto
    ↪ ry tune; save, the tune-down

MISS_PENALTY_MS = 5.0     # the declared trip to the store behind the
    ↪ cache, on a miss
WORKERS = 32              # the application's request threads
RATE = 1500.0             # requests a second offered, the same in bot
    ↪ h arms
STEPS = "1 2 3 2 3 4 5 6 5 4 3 2 1 2 1"

BURST = "1 6 1 8 1 6"
ZIPF_S = 0.5              # the working set's key distribution (Zipf e
    ↪ xponent): a hot core and a long tail, as application caches see
FULL = 0.9                # the cache counts as full when memory used
    ↪ is 90% of the ceiling or more
from omnicompass.student_t import T95  # noqa: E402  (the two-sided 95

    ↪ % point of Student's t, any degrees of freedom)
SAME_REL = 1e-6


WORKLOADS = {
    # name: (value bytes, base keyspace (keys at notch 1; about 20 MB
    ↪ of values at notch 1, 120 MB at notch 6), steps, tuning?)

    "tuning": (8192, 2_500, STEPS, True),
    "small": (2048, 10_000, STEPS, False),
    "large": (32768, 625, STEPS, False),
    "burst": (8192, 2_500, BURST, False),
}

GAUGES = [  # key, label, direction
    ("work_inside_line_rps", "work inside the response line (requests
    ↪ a second answered within the line)", "higher"),
    ("rps", "throughput (requests a second)", "higher"),
    ("hit_rate", "cache hit rate", "higher"),
    ("p95_ms", "latency, 95th percentile (ms)", "lower"),
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ("p99_ms", "latency, 99th percentile (ms)", "lower"),
    ("mean_ms", "latency, mean (ms)", "lower"),
    ("failed", "failed requests", "never more"),
    ("maxmemory_mb_mean", "memory ceiling held, mean (MB; the knob, th
    ↪ e resource)", "spent"),
    ("used_mb_mean", "memory used, mean (MB)", "spent"),

    ("evictions", "keys evicted", "shown"),
    ("cpu_busy_share", "host CPU busy (share of the run)", "spent"),
    ("cpu_seconds", "host CPU-seconds", "spent"),
    ("cpu_s_per_1k_inside", "host CPU-seconds per 1,000 requests insid
    ↪ e the line", "spent"),
    ("writes", "ceiling changes written (the knob's moves)", "shown"),

    # store fix of 25 October: the CPU apart, beside the host figure (
    ↪ old runs lack these rows; the readers prefer process_cpu_seconds
    ↪ )
    ("process_cpu_seconds", "the stack's own CPU-seconds (redis-server
    ↪ , the application, the compass and its sampler; steal and the ru
    ↪ nner's other processes excluded)", "spent"),

    ("server_cpu_seconds", "the server's own CPU-seconds (redis-server
    ↪ )", "shown"),
    ("app_cpu_seconds", "the application's own CPU-seconds (its reques
    ↪ t threads)", "shown"),
    ("brain_cpu_seconds", "the compass's own CPU-seconds (the brain's
    ↪ thread; none in the native arm)", "shown"),

    ("sampler_cpu_seconds", "the sampler's own CPU-seconds (both arms)
    ↪ ", "shown"),
    ("host_cpu_seconds_no_steal", "host CPU-seconds without steal", "s
    ↪ hown"),
    ("steal_seconds", "CPU-seconds the hypervisor took (steal), host-w
    ↪ ide", "shown"),

]
OPTIONAL_GAUGES = ("process_cpu_seconds", "server_cpu_seconds", "app_c
    ↪ pu_seconds", "brain_cpu_seconds", "sampler_cpu_seconds",
                   "host_cpu_seconds_no_steal", "steal_seconds")
    ↪ # absent from runs before 25 October
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def zipf_sampler(rng, n, s=ZIPF_S):
    """Keys 0..n-1 with probability proportional to 1/(k+1)^s (the hot
    ↪  set first); a table is built once per keyspace."""
    w = [1.0 / (k + 1) ** s for k in range(n)]
    tot = sum(w); acc = 0.0; cum = []

    for x in w:
        acc += x / tot; cum.append(acc)
    import bisect
    def draw():
        return bisect.bisect_left(cum, rng.random())
    return draw

class App(threading.Thread):
    """The application: requests at the offered rate, drawn from the w
    ↪ orking set of the moment; a hit is Redis's answer, a
    miss is the declared store trip and a write-back; every request's

    ↪ latency and hit/miss is recorded."""

    def __init__(self, r, value_bytes, base_keys, steps, step_s, seed,
    ↪  rec_path):
        super().__init__(daemon=True)
        self.r, self.value, self.steps, self.step_s = r, b"v" * value_

    ↪ bytes, [int(s) for s in steps.split()], step_s
        self.base_keys, self.rng, self.rec_path = base_keys, random.Ra
    ↪ ndom(seed), rec_path
        self.stop_flag = threading.Event(); self.lock = threading.Lock
    ↪ ()
        self.records = []; self.failed = 0; self.hits = 0; self.misses

    ↪  = 0; self.recent = []
        self.sem = threading.Semaphore(WORKERS)

    def one(self, key):
        t0 = time.perf_counter()
        try:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            v = self.r.get(key)
            if v is None:
                time.sleep(MISS_PENALTY_MS / 1000.0)           # the s
    ↪ tore behind the cache
                self.r.set(key, self.value)
                hit = 0

            else:
                hit = 1
            lat = (time.perf_counter() - t0) * 1000.0
            with self.lock:
                self.records.append((time.time(), lat, hit)); self.rec
    ↪ ent.append((time.time(), lat))

                if hit:
                    self.hits += 1
                else:
                    self.misses += 1
        except Exception:
            with self.lock:

                self.failed += 1
        finally:
            self.sem.release()


    def run(self):
        interval = 1.0 / RATE

        for notch in self.steps:
            n = self.base_keys * notch
            draw = zipf_sampler(self.rng, n)
            prefix = f"k{n}:"
            t_end = time.perf_counter() + self.step_s; nxt = time.perf
    ↪ _counter()

            while time.perf_counter() < t_end and not self.stop_flag.i
    ↪ s_set():
                now = time.perf_counter()
                if now >= nxt:
                    self.sem.acquire()
                    threading.Thread(target=self.one, args=(prefix + s
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ tr(draw()),), daemon=True).start()
                    nxt += interval
                else:
                    time.sleep(min(0.0005, nxt - now))
            if self.stop_flag.is_set():
                break

        for _ in range(WORKERS):                                    # wait
    ↪  for the last requests
            self.sem.acquire()

    def reading(self, since_s=1.0):
        """The application's own reading: mean latency (s) of the last
    ↪  second's requests; nothing asked reads calm."""
        now = time.time()
        with self.lock:
            self.recent = [(t, l) for t, l in self.recent if t >= now
    ↪ - since_s]
            lat = [l for _, l in self.recent]
        return (sum(lat) / len(lat) / 1000.0) if lat else 0.0

    def split(self, since_s=1.0):
        """The last second's hits and misses apart: (a hit's mean late
    ↪ ncy ms, a miss's, the miss share), or Nones."""
        cut = time.time() - since_s
        with self.lock:
            recent = [(lat, hit) for t, lat, hit in self.records[-2000
    ↪ 0:] if t >= cut]
        hits = [l for l, h in recent if h]; misses = [l for l, h in re
    ↪ cent if not h]
        if not recent:
            return None, None, None
        return (sum(hits) / len(hits) if hits else None, sum(misses) /
    ↪  len(misses) if misses else None, len(misses) / len(recent))

    def last_second(self, line_ms, since_s=1.0):
        """The brain's sample: (requests answered, of them inside the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ line, mean latency in s) over the last second."""
        now = time.time()
        with self.lock:
            self.recent = [(t, l) for t, l in self.recent if t >= now
    ↪ - since_s]
            lat = [l for _, l in self.recent]

        if not lat:
            return 0, 0, None
        return len(lat), sum(1 for l in lat if l <= line_ms), sum(lat)
    ↪  / len(lat) / 1000.0

class Ceiling:
    """The plug on Redis's memory ceiling: read once before the first
    ↪ write, write through the console, read back,
    yield to any other writer, restore."""

    def __init__(self, r):

        self.r, self.snapshot, self.last_written = r, None, None

    def attach(self):
        self.snapshot = int(self.r.config_get("maxmemory")["maxmemory"
    ↪ ]); self.last_written = self.snapshot
        return self.snapshot

    def lever(self):
        cur = int(self.r.config_get("maxmemory")["maxmemory"])
        if self.last_written is not None and cur != self.last_written:
            raise RuntimeError(f"another writer set maxmemory to {cur}
    ↪  (we wrote {self.last_written}): Omni stops writing")

        return cur

    def write(self, mb):
        self.r.config_set("maxmemory", str(int(mb) * MB))
        back = int(self.r.config_get("maxmemory")["maxmemory"])
        self.last_written = back
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return back // MB

    def restore(self):
        try:
            self.r.config_set("maxmemory", str(self.snapshot))
            return int(self.r.config_get("maxmemory")["maxmemory"]) ==
    ↪  self.snapshot
        except Exception:
            return False

    def evicted(self):
        return int(self.r.info("stats").get("evicted_keys", 0))

    def used_mb(self):
        return int(self.r.info("memory")["used_memory"]) / MB


def decide(p, force, evicted_last_s, cur_mb, last_change_age, full, wa
    ↪ ll=0.95):
    """The knob's move this second, in MB: misses (the service slow) g
    ↪ row the ceiling by ceil(force / 0.1) notches, but
    only while the cache is full (a miss in a cache with room is a col
    ↪ d miss, which no ceiling can mend); calm with nothing
    evicted gives back one notch after the dwell; past the wall, with
    ↪ the cache full, the top of the cover at once."""
    lo, hi = COVER_MB
    if p is not None and p >= wall and full:
        return min(hi, cur_mb + FAILUP_MB), "fail up: the line is at h
    ↪ and and the cache is full, a quarter of the cover at once"
    if force > 0.05:
        if not full:
            return cur_mb, "misses with room to spare: cold misses, no
    ↪ thing for the ceiling to do"
        return min(hi, cur_mb + STEP_MB * max(1, math.ceil(force / 0.1
    ↪ ))), "misses with the cache full: grow"
    if force < -0.05 and evicted_last_s == 0 and cur_mb > lo and last_
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ change_age >= DWELL_S:
        return max(lo, cur_mb - STEP_MB), "calm, nothing evicted: a no
    ↪ tch given back"
    return cur_mb, "hold"

# omni-v22: the max on the cache's ceiling (docs/REDIS_PREREGISTRATION
    ↪ .md, amendment 5; docs/STORE_ROOT_CAUSES_2026-10-30.md).
# On omni-v21 the verdict's allowance clamped every move, the wall's f
    ↪ ail-up included, and the wall abandoned every spend trial,
# so in 4,426 of the 4,581 seconds the service stood over its line the
    ↪  brain asked for memory and was refused; the cache stayed at

# 64 to 80 MB while the working set needed more. Under the max (the fa
    ↪ ctory tune, the founder's order of 12 October) memory the
# cache takes inside the operator's cover is power given, shown beside
    ↪  the hits it bought; the verdict's allowance judges the
# give-back below the operator's ceiling (the tune-down's ground), nev
    ↪ er a spend the service asks for.

METER = "the application's own split of each request into hits and mis
    ↪ ses"


def miss_time(obs):
    """(the misses' time, the service's time) over the last second, in

    ↪  seconds, from the application's own split: the misses'
    count times what a miss cost beyond a hit, and the requests times
    ↪ their mean. (None, None) where the second carries no hit and
    miss latencies (the wire's older records): nothing known, and the
    ↪ max then takes nothing."""
    hit, miss, share, mean, n = obs.get("hit_ms"), obs.get("miss_ms"),

    ↪  obs.get("miss_share"), obs.get("reading_s"), obs.get("n")
    if hit is None or miss is None or share is None or not mean or not
    ↪  n:
        return None, None
    return n * share * max(0.0, miss - hit) / 1000.0, n * mean
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class Brain:
    """The knob's decision, one a second: the code the real run (Omni)
    ↪  and the closed-loop model (tools/redis_model.py) drive.
    obs: reading_s (the application's mean latency over the last secon
    ↪ d), n, inside, evicted_last_s, ceiling_mb, used_mb,

    cpu_share (or None), hit_ms, miss_ms and miss_share (the applicati
    ↪ on's split), host_avail_mb and host_total_mb (None unread).

      the spend     misses with the cache full (the compass asking, by
    ↪  its own law: ceil(force / 0.1) notches a second; at the
                    wall a quarter of the cover at once) go through un

    ↪ clamped, up to the cover's top, where the application's own
                    split says the misses are MISS_WORTH of its time o
    ↪ r more and the host has the notch to spare; the memory is
                    power given (tools/store_brain.py, MemoryMax)
      the hold      calm with nothing evicted for the dwell gives one
    ↪ notch back a second, never below the working set's floor

                    (the most the working set occupied in HOLD_WINDOW_
    ↪ S) nor below the operator's ceiling
      the audit     every decision with the max's reading beside it (t
    ↪ he misses' share of the service's time, the floor, the host)"""

    def __init__(self, native_mb, objective=DEFAULT_OBJECTIVE, line_ms

    ↪ =LINE_MS):
        self.native_mb, self.objective = native_mb, objective
        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE
    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)
        self.max = MemoryMax(native_mb, STEP_MB, COVER_MB, FULL, METER
    ↪ )

        self.last_change = -1e9

    @property
    def failups(self):
        return self.max.failups
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def record(self):
        return self.max.record(self.objective)

    def floor_mb(self, t):
        return self.max.floor.value(t)

    def step(self, t, obs):
        reading = obs["reading_s"]; cur = obs["ceiling_mb"]; ev_last =
    ↪  obs["evicted_last_s"]
        full = obs["used_mb"] >= FULL * cur
        f = self.law.force(reading)
        want, why = decide(self.law.p, f, ev_last, cur, t - self.last_

    ↪ change, full, wall=self.law.band.wall_high)
        miss_s, service_s = miss_time(obs)
        target, why, info = self.max.step(t, want, why, cur, obs["used
    ↪ _mb"], miss_s, service_s, obs.get("host_avail_mb"),
                                          obs.get("host_total_mb"), fa
    ↪ il_up=why.startswith("fail up"))

        if target > cur:
            self.last_change = t
    ↪  # the dwell runs from the last growth
        info.update({"p": round(self.law.p, 4), "force": round(f, 4)})
        return target, why, info

class Omni(threading.Thread):
    """The brain, one decision a second, writing the audit; the knob i
    ↪ s handed back at the end and read back. Amendment 2 (the
    brain's own verdict, 2026-10-09): the ceiling starts in watch and
    ↪ is written only inside the allowance a paired trial on

    the cache itself has earned under the declared objective (tools/kn
    ↪ ob_verdict.py around the engine's own Verdict); a trial
    holds the ceiling; a fail-up never spends beyond the allowance."""

    def __init__(self, plug: Ceiling, app: App, audit_path: Path, line
    ↪ _ms=LINE_MS, objective=DEFAULT_OBJECTIVE, dial=DIAL):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        super().__init__(daemon=True)
        self.plug, self.app, self.audit, self.line_ms, self.objective,
    ↪  self.dial = plug, app, audit_path, line_ms, objective, dial
        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE
    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)
        self.stop_flag = threading.Event(); self.last_change = -1e9; s

    ↪ elf.writes = 0; self.failups = 0
        self.foreign = False; self.handed_back = False; self.verdict =
    ↪  None; self.brain = None
        self.cpu_s = 0.0                                                #
    ↪  the brain thread's own CPU-seconds so far (store fix of 25 Octo
    ↪ ber)

    def govern_record(self):
        """The record of what governed the knob: the max's counts (omn
    ↪ i-v22) or the verdict's allowance (the tune-down)."""
        if self.brain is not None:
            return self.brain.record()

        return self.verdict.record() if self.verdict else None


    def run(self):
        c0 = time.thread_time()
        native_mb = self.plug.attach() // MB; ev0 = self.plug.evicted(
    ↪ ); t0 = time.monotonic(); cpu_prev = cpu_times()

        if native_mb == 0:
            # Redis as shipped holds every key it is given (no ceiling
    ↪ ): the knob is at its limit on native, and any ceiling
            # Omni wrote would sit below native (the max never goes be
    ↪ low native). It reads and writes nothing, and says so.
            with open(self.audit, "w") as fh:

                fh.write(json.dumps({"t": 0.0, "maxmemory_mb": 0, "why
    ↪ ": "native has no ceiling: at its limit on native, nothing writt
    ↪ en"}) + "\n")
            self.stop_flag.wait()
            self.handed_back = self.plug.restore()
            return
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    self.verdict = KnobVerdict(native_mb, STEP_MB, COVER_MB, objec
↪ tive=self.objective, settle_s=TAU)
    if self.dial == "max":
        self.brain = Brain(native_mb, self.objective, self.line_ms
↪ ); self.verdict = None
    with open(self.audit, "w") as fh:

        while not self.stop_flag.is_set():
            tick = time.monotonic()
            try:
                reading = self.app.reading(1.0)
                n_req, inside, mean_s = self.app.last_second(self.
↪ line_ms)

                f = self.law.force(reading)
                ev = self.plug.evicted(); ev_last = ev - ev0; ev0
↪ = ev
                cur = self.plug.lever() // MB
                used = self.plug.used_mb(); full = used >= FULL *
↪ cur

                cpu_now = cpu_times(); d_total = cpu_now[0] - cpu_
↪ prev[0]; d_idle = cpu_now[1] - cpu_prev[1]; cpu_prev = cpu_now
                cpu_share = (1.0 - d_idle / d_total) if d_total >
↪ 0 else None
                cost = sample_cost(self.objective, inside, mean_s,
↪  cur, cpu_share)

                hit_ms, miss_ms, miss_share = self.app.split(1.0)
                if self.brain is not None:
↪                     # the max (omni-v22)
                    h_avail, h_total = read_meminfo()
                    obs = {"reading_s": reading, "n": n_req, "insi
↪ de": inside, "evicted_last_s": ev_last, "ceiling_mb": cur, "used
↪ _mb": used,
                           "cpu_share": cpu_share, "hit_ms": hit_m
↪ s, "miss_ms": miss_ms, "miss_share": miss_share,
                           "host_avail_mb": h_avail, "host_total_m
↪ b": h_total}
                    target, why, vinfo = self.brain.step(tick - t0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ , obs)       # the Brain's decision (above)
                        self.failups = self.brain.failups
                    else:
    ↪                    # the tune-down: omni-v21's loop
                        self.verdict.observe(cost, tick - t0)
    ↪                    # measured under the ceiling of the last seco
    ↪ nd
                        target, why = decide(self.law.p, f, ev_last, c
    ↪ ur, tick - self.last_change, full, wall=self.law.band.wall_high)
                        target, why, vinfo = self.verdict.decide(
                            target, spend_ok=(f > 0.05 and full), give
    ↪ _ok=(f < -0.05 and ev_last == 0),
                            t=tick - t0, why=why, stress=f > 0.05, cal
    ↪ m=f < -0.05, fail_up=why.startswith("fail up"))
                        if target > cur and why.startswith("fail up"):
                            self.failups += 1
                    vinfo.update({"hit_ms": None if hit_ms is None els
    ↪ e round(hit_ms, 3), "miss_ms": None if miss_ms is None else roun
    ↪ d(miss_ms, 3),
                                  "miss_share": None if miss_share is
    ↪ None else round(miss_share, 4)})
                    wrote = None
                    if target != cur:
                        wrote = self.plug.write(target); self.writes +
    ↪ = 1
                        if target > cur:
                            self.last_change = tick
                    fh.write(json.dumps({"t": round(tick - t0, 2), "re
    ↪ ading_ms": round(reading * 1000, 3),
                                         "evicted_last_s": ev_last, "m
    ↪ axmemory_mb": cur, "used_mb": round(used, 1), "full": full, "tar
    ↪ get_mb": target, "wrote_mb": wrote,
                                         "requests_last_s": n_req, "in
    ↪ side_last_s": inside, "cpu_share": None if cpu_share is None els
    ↪ e round(cpu_share, 4),
                                         "cost": None if cost is None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ else round(cost, 9), **vinfo, "why": why}) + "\n")
                except Exception as e:                          # a fore
    ↪ ign writer or a lost console: say so, stop writing
                    fh.write(json.dumps({"t": round(tick - t0, 2), "er
    ↪ ror": repr(e)}) + "\n")
                    self.foreign = True

                    break
                self.cpu_s = time.thread_time() - c0
                time.sleep(max(0.0, DT - (time.monotonic() - tick)))
        self.handed_back = self.plug.restore()

class Sampler(threading.Thread):
    """The ceiling, the memory used and the evictions, once a second,
    ↪ in both arms."""

    def __init__(self, r):
        super().__init__(daemon=True)

        self.r, self.stop_flag, self.rows = r, threading.Event(), []
        self.cpu_s = 0.0                                            #
    ↪  the sampler thread's own CPU-seconds so far

    def run(self):
        c0 = time.thread_time()

        while not self.stop_flag.is_set():
            try:
                m = self.r.info("memory"); s = self.r.info("stats")
                self.rows.append((int(m["maxmemory"]) / MB, int(m["use
    ↪ d_memory"]) / MB, int(s.get("evicted_keys", 0))))
            except Exception:

                pass
            self.cpu_s = time.thread_time() - c0
            time.sleep(DT)


def cpu_apart(server_s, process_s, brain_s, sampler_s, busy_s, steal_s
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ):
    """The arm's CPU apart (store fix of 25 October), all in seconds o
    ↪ ver the arm: the server's from its /proc/<pid>/stat, the
    harness process's from getrusage, the brain's and the sampler's fr
    ↪ om their own threads' clocks; the application is the
    harness process less the two threads. process_cpu_seconds, the sta

    ↪ ck's own (server, application, compass, sampler), is
    what the readers prefer to the host figure, which also counts stea
    ↪ l and every other process on the runner."""
    app = max(0.0, process_s - brain_s - sampler_s)
    return {"server_cpu_seconds": server_s, "app_cpu_seconds": app, "b
    ↪ rain_cpu_seconds": brain_s, "sampler_cpu_seconds": sampler_s,

            "process_cpu_seconds": (server_s + process_s) if server_s
    ↪ is not None else None,
            "host_cpu_seconds_no_steal": busy_s, "steal_seconds": stea
    ↪ l_s}

def cpu_times():
    with open("/proc/stat") as fh:
        f = fh.readline().split()
    vals = list(map(int, f[1:]))
    return sum(vals), vals[3] + vals[4]

def run_arm(arm, wl_dir: Path, rep, value_bytes, base_keys, steps, ste
    ↪ p_s, line_ms, seed, objective=DEFAULT_OBJECTIVE):
    import redis
    d = wl_dir / f"rep-{rep}" / arm; d.mkdir(parents=True, exist_ok=Tr
    ↪ ue)

    r = redis.Redis(host=HOST, port=PORT, socket_timeout=5)
    r.flushall()                                                # a col
    ↪ d cache, both arms alike
    r.config_set("maxmemory", str(NATIVE_MB * MB)); r.config_set("maxm
    ↪ emory-policy", POLICY[NATIVE])   # NATIVE_MB 0: no ceiling
    pipe = r.pipeline(transaction=False)                        # the w
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ arm-up: the notch-1 working set loaded once, both arms alike
    for k in range(base_keys):
        pipe.set(f"k{base_keys}:{k}", b"v" * value_bytes)
        if k % 500 == 499:
            pipe.execute()
    pipe.execute()

    r.config_resetstat()
    app = App(redis.Redis(host=HOST, port=PORT, socket_timeout=5, max_
    ↪ connections=WORKERS + 4), value_bytes, base_keys, steps, step_s,
    ↪  seed, d / "requests.log")
    sampler = Sampler(redis.Redis(host=HOST, port=PORT, socket_timeout
    ↪ =5)); sampler.start()

    omni = None
    if arm == "omni":
        omni = Omni(Ceiling(redis.Redis(host=HOST, port=PORT, socket_t
    ↪ imeout=5)), app, d / "audit.jsonl", line_ms, objective); omni.st
    ↪ art()
    import resource

    tck = os.sysconf("SC_CLK_TCK"); pid = pid_of("redis-server")
    srv0 = proc_cpu_jiffies(pid) if pid else None; own0 = resource.get
    ↪ rusage(resource.RUSAGE_SELF); busy0, steal0 = host_cpu_split()
    brain0 = omni.cpu_s if omni else 0.0; samp0 = sampler.cpu_s
    cpu0 = cpu_times(); t0 = time.time()
    app.start(); app.join()

    t1 = time.time(); cpu1 = cpu_times()
    srv1 = proc_cpu_jiffies(pid) if pid else None; own1 = resource.get
    ↪ rusage(resource.RUSAGE_SELF); busy1, steal1 = host_cpu_split()
    apart = cpu_apart(None if srv0 is None or srv1 is None else (srv1
    ↪ - srv0) / tck,
                      (own1.ru_utime + own1.ru_stime) - (own0.ru_utime

    ↪  + own0.ru_stime),
                      (omni.cpu_s - brain0) if omni else 0.0, sampler.
    ↪ cpu_s - samp0, (busy1 - busy0) / tck, (steal1 - steal0) / tck)
    if omni:
        omni.stop_flag.set(); omni.join(timeout=30)
    sampler.stop_flag.set(); sampler.join(timeout=5)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    seconds = t1 - t0
    with open(d / "requests.log", "w") as fh:
        for t, lat, hit in app.records:
            fh.write(f"{t:.3f} {lat:.3f} {hit}\n")
    lat_sorted = sorted(l for _, l, _ in app.records); n = len(lat_sor
    ↪ ted)

    def q(p):
        return lat_sorted[min(n - 1, int(p * n))] if n else float("nan
    ↪ ")
    inside = sum(1 for l in lat_sorted if l <= line_ms)
    rows = sampler.rows or [(NATIVE_MB, 0.0, 0)]
    total = cpu1[0] - cpu0[0]; idle = cpu1[1] - cpu0[1]; cpu_s = (tota

    ↪ l - idle) / os.sysconf("SC_CLK_TCK")
    g = {"arm": arm, "rep": rep, "seconds": round(seconds, 1), "reques
    ↪ ts": n, "hits": app.hits, "misses": app.misses,
         "work_inside_line_rps": inside / seconds, "rps": n / seconds,
    ↪  "hit_rate": app.hits / max(1, app.hits + app.misses),
         "p95_ms": q(0.95), "p99_ms": q(0.99), "mean_ms": (sum(lat_sor

    ↪ ted) / n) if n else float("nan"), "failed": app.failed,
         "maxmemory_mb_mean": sum(x[0] for x in rows) / len(rows), "us
    ↪ ed_mb_mean": sum(x[1] for x in rows) / len(rows),
         "evictions": rows[-1][2], "cpu_busy_share": (total - idle) /
    ↪ max(1, total), "cpu_seconds": cpu_s,
         "cpu_s_per_1k_inside": 1000.0 * cpu_s / max(1, inside), "writ

    ↪ es": (omni.writes if omni else 0), **apart,
         "handed_back": (omni.handed_back if omni else True), "foreign
    ↪ _writer": (omni.foreign if omni else False), "failups": (omni.fa
    ↪ ilups if omni else 0),
         "verdict": (omni.govern_record() if omni else None)}
    (d / "arm.json").write_text(json.dumps(g, indent=1))

    print(f"   {arm} rep {rep}: {n} requests, hit rate {g['hit_rate']:
    ↪ .1%}, inside the line {g['work_inside_line_rps']:.0f}/s, p95 {g[
    ↪ 'p95_ms']:.2f} ms, "
          f"ceiling {g['maxmemory_mb_mean']:.0f} MB, used {g['used_mb_
    ↪ mean']:.0f} MB, CPU {cpu_s:.1f} s, handed back {g['handed_back']
    ↪ }"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
          + (f", verdict {g['verdict']['state']}" if g.get("verdict")
    ↪ else ""), flush=True)
    return g


def paired(reps, key, direction):
    """omni against native on one gauge over the repetitions: means, t
    ↪ he mean paired difference, its 95% interval."""
    d = [r["omni"][key] - r["native"][key] for r in reps if _num(r["om
    ↪ ni"].get(key)) and _num(r["native"].get(key))]
    if not d:
        return None
    n = len(d)
    nat = sum(r["native"][key] for r in reps) / n
    om = sum(r["omni"][key] for r in reps) / n
    mean = sum(d) / n
    if n > 1:
        sd = math.sqrt(sum((x - mean) ** 2 for x in d) / (n - 1))
        half = T95.get(n - 1, 1.96) * sd / math.sqrt(n)
    else:
        half = float("nan")
    lo, hi = mean - half, mean + half
    same = abs(mean) <= SAME_REL * max(abs(nat), 1e-12)
    clear = n > 1 and (lo > 0 or hi < 0) and not same
    if direction == "shown":
        reading = SHOWN
    elif direction == "never more" and any(x > 0 for x in d):
        reading = "**LOSS in a repetition**"
    elif same:
        reading = EVEN_SAME
    elif not clear:
        reading = EVEN_NOISE
    else:
        reading = gel_word(mean < 0, gel_aim(direction, key))      # t
    ↪ he word by what the gauge is for, never by its sign
    return {"native": nat, "omni": om, "diff": mean, "ci95": [lo, hi],
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  "n": n, "significant": clear, "reading": reading}


def _num(x):
    return isinstance(x, (int, float)) and not (isinstance(x, float) a
    ↪ nd math.isnan(x))

def engine():
    try:
        out = subprocess.run([sys.executable, str(ROOT / "tools" / "om
    ↪ ni_version.py")], cwd=ROOT, capture_output=True, text=True, time
    ↪ out=120).stdout.strip()
        sha = subprocess.run(["git", "rev-parse", "HEAD"], cwd=ROOT, c
    ↪ apture_output=True, text=True).stdout.strip()
        return {"version": out.splitlines()[0] if out else "unknown",
    ↪ "commit": sha[:12]}
    except (OSError, subprocess.SubprocessError):
        return {"version": "unknown", "commit": "?"}


def run_workload(name, out: Path, reps, step_s, line_ms, objective=DEF
    ↪ AULT_OBJECTIVE):
    import redis
    value_bytes, base_keys, steps, tuning = WORKLOADS[name]
    wl_dir = out / f"redis-{name}"; wl_dir.mkdir(parents=True, exist_o
    ↪ k=True)
    ver = redis.Redis(host=HOST, port=PORT).info("server").get("redis_
    ↪ version", "?")
    print(f"== {name}: {value_bytes} B values, working set {base_keys:
    ↪ ,} keys x notch, steps {steps}, {step_s} s a notch, Redis {ver},
    ↪  the verdict's objective {objective}", flush=True)
    rec = {"workload": name, "tuning": tuning, "value_bytes": value_by
    ↪ tes, "base_keys": base_keys, "steps": steps, "step_s": step_s,
           "line_ms": line_ms, "rate_rps": RATE, "miss_penalty_ms": MI
    ↪ SS_PENALTY_MS, "native_mb": NATIVE_MB, "cover_mb": list(COVER_MB
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ),
           "objective": objective, "dial": DIAL, "native": NATIVE, "re
    ↪ dis_version": ver, "engine": engine(), "reps": []}
    for rep in range(1, reps + 1):
        order = ("native", "omni") if rep % 2 else ("omni", "native")
        r = {}

        for arm in order:
            r[arm] = run_arm(arm, wl_dir, rep, value_bytes, base_keys,
    ↪  steps, step_s, line_ms, seed=1000 + rep, objective=objective)
        rec["reps"].append(r)
        (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1)
    ↪ )

    rec["paired"] = {k: paired(rec["reps"], k, dr) for k, _, dr in GAU
    ↪ GES}
    (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1))
    return rec

def fmt(v):
    if v is None or (isinstance(v, float) and math.isnan(v)):
        return "n/a"
    return f"{v:,.0f}" if abs(v) >= 100 else f"{v:,.3g}"

def report(recs, out: Path):
    shipped = bool(recs) and all(r.get("native") == "shipped" for r in
    ↪  recs)
    L = ["# Redis: Omni-Compass on top of a cache's " + ("memory as sh
    ↪ ipped (no ceiling)" if shipped else "operator-set memory ceiling
    ↪ "), "",

         ("Redis exactly as shipped, with no memory ceiling and noevic
    ↪ tion, is native; with no ceiling the knob is at its limit on "
          "native, so omni reads and writes nothing, and every gauge i
    ↪ s native's. Otherwise omni is the compass law on the ceiling "
          if shipped else
          f"Redis as shipped with the operator's ceiling ({recs[0]['na
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ tive_mb'] if recs else NATIVE_MB} MB, allkeys-lru) is native; om
    ↪ ni is the compass law on the ceiling ")
         +
         f"through Redis's own console inside [{COVER_MB[0]}, {COVER_M
    ↪ B[1]}] MB, holding the application's own request latency at "
         f"{CENTER:.0%} of the {LINE_MS:.0f} ms line (a hit is inside,
    ↪  a miss and its {MISS_PENALTY_MS:.0f} ms store trip outside). Th
    ↪ e same "
         "requests in both arms. Memory held is the resource; the host
    ↪ 's CPU seconds include the compass's own cost. Every row is "
         "reported, losses included.", ""]
    for rec in recs:
        L += [f"## {rec['workload']}: {rec['value_bytes']} B values, w
    ↪ orking set {rec['base_keys']:,} keys × notch, steps {rec['steps'
    ↪ ]} × {rec['step_s']} s"
              + (" (the tuning workload)" if rec.get("tuning") else ""
    ↪ ), "",
              f"Redis {rec['redis_version']}; {rec['rate_rps']:.0f} re
    ↪ quests a second offered; {len(rec['reps'])} paired repetitions;
    ↪ engine "
              f"{rec['engine']['version']} at `{rec['engine']['commit'
    ↪ ]}`.", "",
              "| Gauge | native | omni | change | 95% interval of the
    ↪ difference | reading |", "|---|---:|---:|---:|---:|---|"]
        for k, label, direction in GAUGES:
            p = rec.get("paired", {}).get(k)
            if not p:
                continue
            ch = "" if p["native"] == 0 else f"{100 * p['diff'] / abs(
    ↪ p['native']):+.1f}%"
            L.append(f"| {label} | {fmt(p['native'])} | {fmt(p['omni']
    ↪ )} | {ch} | {fmt(p['ci95'][0])} to {fmt(p['ci95'][1])} | {p['rea
    ↪ ding']} |")
        hb = all(r["omni"]["handed_back"] for r in rec["reps"]); fw =
    ↪ any(r["omni"]["foreign_writer"] for r in rec["reps"])
        L += ["", f"Every omni arm handed back to native's ceiling ({r
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ec['native_mb'] or 'none'} MB): {'yes' if hb else 'NO'}; another
    ↪  writer seen: {'yes' if fw else 'no'}; "
              f"fail-ups: {sum(r['omni']['failups'] for r in rec['reps
    ↪ '])}.", ""]
    (out / "REDIS.md").write_text("\n".join(_legal_stamp(L)) + "\n")
    return out / "REDIS.md"

def setup():
    """The workflow's Redis: the shipped binary, the operator's two se
    ↪ ttings (or, --native shipped, none: no ceiling and
    noeviction, as Redis ships), nothing else."""

    log = Path("/tmp/omni-redis.log")
    subprocess.run(["redis-server", "--port", str(PORT), "--daemonize"
    ↪ , "yes", "--save", "", "--appendonly", "no", "--maxmemory", f"{N
    ↪ ATIVE_MB}mb",
                    "--maxmemory-policy", POLICY[NATIVE], "--logfile",
    ↪  str(log), "--dir", "/tmp"], check=True)

    for _ in range(30):
        r = subprocess.run(["redis-cli", "-p", str(PORT), "ping"], cap
    ↪ ture_output=True, text=True)
        if r.stdout.strip() == "PONG":
            print(subprocess.run(["redis-server", "--version"], captur
    ↪ e_output=True, text=True).stdout.strip()); return 0

        time.sleep(1)
    print("redis did not start", file=sys.stderr); return 1


def main(argv=None):
    global NATIVE, NATIVE_MB

    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--setup", action="store_true")
    ap.add_argument("--native", default="operator", choices=("operator
    ↪ ", "shipped"),
                    help="native: the operator's 64 MB ceiling with al
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ lkeys-lru (preregistered) or Redis exactly as shipped, no ceilin
    ↪ g "
                         "and noeviction (the founder's order of 17 Oc
    ↪ tober); with no ceiling the knob is at its limit on native")
    ap.add_argument("--workloads", default="tuning")
    ap.add_argument("--reps", type=int, default=3)

    ap.add_argument("--step-s", type=float, default=20.0)
    ap.add_argument("--line-ms", type=float, default=LINE_MS)
    ap.add_argument("--out", default="redis-out")
    ap.add_argument("--objective", default=DEFAULT_OBJECTIVE, choices=
    ↪ OBJECTIVES,
                    help="what the brain's verdict judges a step by: r

    ↪ esource (the index's reading: work, speed, memory and CPU togeth
    ↪ er) or service (work and speed alone)")
    ap.add_argument("--report-only", default="")
    a = ap.parse_args(argv)
    NATIVE, NATIVE_MB = a.native, (0 if a.native == "shipped" else 64)
    if a.setup:

        return setup()
    out = Path(a.out)
    if a.report_only:
        recs = [json.loads(f.read_text()) for f in sorted(Path(a.repor
    ↪ t_only).rglob("redis-*/*.json"))]
        print(report(recs, Path(a.report_only))); return 0

    names = list(WORKLOADS) if a.workloads == "all" else a.workloads.s
    ↪ plit(",")
    recs = [run_workload(n, out, a.reps, a.step_s, a.line_ms, a.object
    ↪ ive) for n in names]
    print(report(recs, out))
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

H.57 `tools/run_swarm.py`

398 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of a drone swarm's own autopilots (docs/SWARM_P
    ↪ REREGISTRATION.md).

The simulator is gym-pybullet-drones (University of Toronto, DSL; MIT
    ↪ License): PyBullet integrates every quadrotor from
the Crazyflie 2.x model the package ships (mass, inertia, motor consta
    ↪ nts, drag, ground effect and downwash), and its
shipped position controller (DSLPIDControl, one per drone) flies it. N
    ↪ othing in it is ours. A fleet of N drones flies

K delivery missions each from a depot: climb, cruise to a target, hove
    ↪ r, cruise back, descend, land, next mission.


  native  every drone's shipped autopilot at the planner's cruise spee
    ↪ d (1.0 m/s): the mission as the fleet flies it today
  omni    the same autopilots and missions, with the compass law (omni
    ↪ compass/compass_law.py) on one knob, the cruise
          override, inside the autopilot's own limits: the compass rea
    ↪ ds each drone's tracking error (how far it is behind
          the point it was told to be at) on a band from 0 to the decl
    ↪ ared safe error, and spends slack by flying faster
          (fewer seconds airborne, less energy a mission) while the er
    ↪ ror stays in the middle of its band; past the wall the
          cruise is the planner's at once (fail up). The override neve
    ↪ r goes under the planner's cruise or over twice it,
          and never past the autopilot's speed limit.

Energy is a declared model (evidence class S), the same for both arms:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  each motor's mechanical power from the simulator's
own thrust and torque constants (torque = KM rpm^2, power = torque x a
    ↪ ngular speed), divided by a declared motor
efficiency, plus a declared avionics draw for every drone over the who
    ↪ le fleet window, flying or landed. The battery is
the Crazyflie's (250 mAh at 3.7 V); the autopilot's own reserve rule (
    ↪ 20% left at landing) decides a reserve breach.
Collisions are read from the simulator: any two drones closer than two
    ↪  collision radii. A cell where native itself
collides is void and says so; a collision with Omni-Compass on top is
    ↪ a judged row, any increase a LOSS (amendment 2,
from omni-v12's runs on). Every row is reported, LOSS included.

  python3 tools/run_swarm.py --cell tuning --out DIR        # the tuni
    ↪ ng swarm (5 drones, mixed distances)
  python3 tools/run_swarm.py --cell short --out DIR         # an untou
    ↪ ched cell: short | mixed | long (20 drones), and
                                                            # short-31
    ↪ 4 | mixed-315 | long-316 (draws never flown)
  python3 tools/run_swarm.py --cell all --out DIR           # every ce
    ↪ ll, one after the other
  python3 tools/run_swarm.py --report-only DIR              # the repo
    ↪ rt from DIR's swarm-<cell>.json files
"""

from __future__ import annotations

import argparse
import csv
import json
import math

import sys
import time
from pathlib import Path

try:                                                      # the legal
    ↪  notice every generated report carries
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    from tools.legal import stamp as _legal_stamp
except ImportError:
    import pathlib as _p; sys.path.insert(0, str(_p.Path(__file__).res
    ↪ olve().parents[1])); from tools.legal import stamp as _legal_sta
    ↪ mp
ROOT = Path(__file__).resolve().parents[1]

sys.path.insert(0, str(ROOT))
from omnicompass.compass_law import Band, CompassLaw, clamp  # noqa: E
    ↪ 402


# ---------------------------------------------------------------- t
    ↪ he task (the same in both arms, fixed before any run)

PYB_FREQ, CTRL_FREQ = 240, 48
CTRL_DT = 1.0 / CTRL_FREQ
V_PLAN = 1.0            # m/s, the planner's cruise: native's speed
VZ = 0.5                # m/s, climb and descent
HOVER_S = 2.0           # s at the target (the delivery)
SETTLE_S = 1.0          # s allowed for the landing to settle, in the

    ↪ planned time
LINE_FACTOR = 1.5       # a mission's deadline is 1.5 x its planned ti
    ↪ me (the fleet's own service line)
DEPOT_PITCH = 0.6       # m between the five spots of a depot
DEPOT_GAP = 20.0        # m between depots: five drones a depot, a dep
    ↪ ot's drones never reach another's airspace (missions are at most

    ↪  8 m out)
ALT = [0.8, 1.1, 1.4, 1.7, 2.0]     # cruise altitudes, one per spot:
    ↪ two drones of one depot never share a plane, 0.3 m apart
ARRIVE_M = 0.10         # m: the drone is "there"
MISSION_TIMEOUT = 3.0   # x the line: a drone that never arrives is cu
    ↪ t off and its mission counted late

# ---------------------------------------------------------------- t
    ↪ he knob and its guards (frozen on the tuning swarm)
O_MIN, O_MAX = 1.0, 2.0  # the cruise override: never under the planne
    ↪ r's cruise, never over twice it
E_LINE = 0.25           # m: the declared safe tracking error (a quart
    ↪ er of the smallest depot pitch and under the altitude stagger)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
SEP_WALL = 0.5          # m: another drone this close is a wall: the p
    ↪ lanner's cruise at once (omni's proximity reflex)
TAU, KP, SMOOTH = 1.0, 1.0, 0.5
SLEW = 0.01             # per control tick (48 Hz)
TRIAL_OVERRIDE = 1.25   # the paired physics trial: is a faster missio
    ↪ n cheaper at all, and still inside the safe error?

# ---------------------------------------------------------------- t
    ↪ he energy model (declared; the same for both arms)
MOTOR_EFF = 0.5         # electrical to mechanical, small brushed moto
    ↪ rs
AVIONICS_W = 0.5        # W per drone, flying or landed, for the whole
    ↪  fleet window

BATTERY_J = 0.250 * 3.7 * 3600.0   # 250 mAh at 3.7 V
RESERVE = 0.20          # the autopilot's own low-battery reserve

CELLS = {
    # name: (drones, missions per drone, distance range (m), seed, tun
    ↪ ing?)

    "tuning": (5, 4, (3.0, 8.0), 1, True),
    "short": (20, 4, (3.0, 3.0), 11, False),
    "mixed": (20, 4, (3.0, 8.0), 12, False),
    "long": (20, 4, (8.0, 8.0), 13, False),
    # amendment 2 (19 October, before omni-v12's runs): the same three
    ↪  cells on mission draws never flown, the seed plus 303

    "short-314": (20, 4, (3.0, 3.0), 314, False),
    "mixed-315": (20, 4, (3.0, 8.0), 315, False),
    "long-316": (20, 4, (8.0, 8.0), 316, False),
}

GAUGES = [("energy_per_mission_j", "energy per mission (J, declared mo

    ↪ del: motors and avionics over the fleet window)", "spent"),
          ("fleet_energy_j", "fleet energy over the window (J, declare
    ↪ d model)", "spent"),
          ("missions_per_charge", "missions per charge at the autopilo
    ↪ t's reserve", "higher"),
          ("late_share", "missions over their deadline (share)", "neve
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ r more"),
          ("reserve_breaches", "drones that broke the battery reserve"
    ↪ , "never more"),
          ("near_miss_ticks", "control ticks with two drones under hal
    ↪ f the keep-out apart (near misses)", "never more"),
          ("min_separation_m", "closest two drones ever came (m)", "sh
    ↪ own"),
          ("tracking_rms_m", "tracking error, RMS (m; the compass's re
    ↪ ading, held inside its band)", "shown"),
          ("mission_s", "mission time (s)", "shown"),
          ("airborne_s", "seconds airborne per mission", "shown"),
          ("override_mean", "cruise override, mean", "shown")]

# judged from omni-v12 on (amendment 2): a collision with Omni-Compass
    ↪  on top where native has none is a LOSS; only a cell where
# native itself collides is void, its task unsafe before Omni-Compass
    ↪ touches it
COLLISIONS = ("collisions", "control ticks with two drones under two c
    ↪ ollision radii (collisions)", "never more")

def with_collisions(gauges):
    """The gauges with the judged collision row after the near misses
    ↪ (amendment 2)."""
    i = [k for k, _, _ in gauges].index("near_miss_ticks") + 1
    return gauges[:i] + [COLLISIONS] + gauges[i:]


def depot(i):
    """Drone i's spot: depot i // 5 (DEPOT_GAP apart along x), spot i
    ↪ % 5 (DEPOT_PITCH apart along y)."""
    return (DEPOT_GAP * (i // 5), DEPOT_PITCH * (i % 5), 0.1)


KEEPOUT = 0.3           # m: a path keeps this far from the other spot
    ↪ s of its depot, where drones climb and descend
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def _seg_dist(px, py, ax, ay, bx, by):
    """Distance from point p to the segment a-b."""
    vx, vy = bx - ax, by - ay; L2 = vx * vx + vy * vy
    t = 0.0 if L2 == 0 else max(0.0, min(1.0, ((px - ax) * vx + (py -
    ↪ ay) * vy) / L2))

    return math.hypot(px - (ax + t * vx), py - (ay + t * vy))


def missions(n, k, dist, seed):
    """Every drone's K targets, fixed by the seed: a distance in the c
    ↪ ell's range and a bearing from its own depot spot. A

    path that would pass within KEEPOUT of another spot of the same de
    ↪ pot (the column where that drone climbs and lands) is
    drawn again: the task keeps the fleet apart by design, as a planne
    ↪ r would; the autopilots fly it blind."""
    import random
    rng = random.Random(seed)

    out = []
    for i in range(n):
        x0, y0, _ = depot(i)
        others = [depot(j)[:2] for j in range(n) if j != i and j // 5
    ↪ == i // 5]
        ms = []

        while len(ms) < k:
            d = rng.uniform(*dist); th = rng.uniform(0, 2 * math.pi)
            tx, ty = x0 + d * math.cos(th), y0 + d * math.sin(th)
            if all(_seg_dist(ox, oy, x0, y0, tx, ty) >= KEEPOUT for ox
    ↪ , oy in others):
                ms.append((tx, ty, d))

        out.append(ms)
    return out


def planned_time(d, z):
    return 2 * (z - 0.1) / VZ + 2 * d / V_PLAN + HOVER_S + SETTLE_S
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
class Drone:
    """One drone's mission state machine: the carrot it is told to fol
    ↪ low, at the planner's cruise times its override."""

    def __init__(self, i, ms, law):
        self.i = i; self.home = depot(i); self.z = ALT[i % len(ALT)];
    ↪ self.missions = ms; self.law = law
        self.k = 0; self.phase = "idle"; self.t_mission = 0.0; self.ov
    ↪ erride = 1.0; self.carrot = list(self.home)
        self.done = []; self.energy = 0.0; self.airborne = 0.0; self.e

    ↪ rr2 = 0.0; self.n_err = 0; self.ov = []
        self.start_next()

    def start_next(self):
        if self.k >= len(self.missions):
            self.phase = "finished"; self.override = O_MIN; return

    ↪  # the last hand-back: the planner's cruise, read back
        x, y, d = self.missions[self.k]
        self.target = (x, y, self.z); self.d = d
        self.planned = planned_time(d, self.z); self.line = LINE_FACTO
    ↪ R * self.planned
        self.phase = "climb"; self.t_mission = 0.0; self.hover_left =

    ↪ HOVER_S; self.m_energy0 = self.energy
        self.m_airborne0 = self.airborne; self.carrot = list(self.home
    ↪ )
        if self.law is not None:
            self.law.p = None; self.law.v = 0.0
        self.override = 1.0

    def remaining_path(self, pos):
        """Metres of cruise still to fly on this mission, from the car
    ↪ rot."""
        cx, cy, _ = self.carrot
        tx, ty, _ = self.target; hx, hy, _ = self.home
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if self.phase in ("climb", "out"):
            return math.dist((cx, cy), (tx, ty)) + math.dist((tx, ty),
    ↪  (hx, hy))
        if self.phase in ("hover", "back"):
            return math.dist((cx, cy), (hx, hy))
        return 0.0

    def step(self, pos, rpm, nearest=float('inf')):
        """Advance the carrot one control tick; returns (target_pos, t
    ↪ arget_vel). pos: the drone's position now."""
        if self.phase == "finished":
            return self.home, (0.0, 0.0, 0.0)

        self.t_mission += CTRL_DT
        err = math.dist(pos, self.carrot)
        if self.phase != "idle":
            self.err2 += err * err; self.n_err += 1
        # the compass on the cruise override (omni only): the tracking
    ↪  error on its band; slack is spent as speed

        if self.law is not None and self.phase in ("out", "back"):
            f = self.law.force(err)
            want = clamp(O_MIN + (O_MAX - O_MIN) * (0.5 - 0.5 * f), O_
    ↪ MIN, O_MAX)
            # the stopping distance grows with the square of the speed
    ↪ , so the reflex reaches out with it: at the planner's cruise it

    ↪ is SEP_WALL
            if err >= 0.95 * E_LINE or nearest < SEP_WALL * self.overr
    ↪ ide ** 2:
                self.override = O_MIN                           # fail
    ↪ up: the planner's cruise at once, in this tick (error at the wal
    ↪ l, or a drone too close), v12

            else:
                self.override += clamp(want - self.override, -SLEW, SL
    ↪ EW)
        self.ov.append(self.override)
        v = V_PLAN * self.override
        cx, cy, cz = self.carrot
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if self.phase == "climb":
            cz = min(self.z, cz + VZ * CTRL_DT); self.carrot = [cx, cy
    ↪ , cz]; vel = (0, 0, VZ)
            if cz >= self.z and pos[2] >= self.z - ARRIVE_M:
                self.phase = "out"
        elif self.phase in ("out", "back"):

            tx, ty, _ = self.target if self.phase == "out" else self.h
    ↪ ome
            dx, dy = tx - cx, ty - cy; dist = math.hypot(dx, dy)
            step = min(dist, v * CTRL_DT)
            if dist > 1e-9:
                cx += dx / dist * step; cy += dy / dist * step

            self.carrot = [cx, cy, self.z]; vel = (dx / dist * v, dy /
    ↪  dist * v, 0.0) if dist > 1e-9 else (0.0, 0.0, 0.0)
            if dist <= 1e-6 and math.dist(pos[:2], (tx, ty)) <= ARRIVE
    ↪ _M:
                self.phase = "hover" if self.phase == "out" else "desc
    ↪ end"

        elif self.phase == "hover":
            self.hover_left -= CTRL_DT; vel = (0.0, 0.0, 0.0)
            if self.hover_left <= 0:
                self.phase = "back"
        elif self.phase == "descend":
            cz = max(0.1, cz - VZ * CTRL_DT); self.carrot = [cx, cy, c

    ↪ z]; vel = (0, 0, -VZ)
            if cz <= 0.1 and pos[2] <= 0.1 + ARRIVE_M:
                self.phase = "landed"
        if self.phase == "landed" or self.t_mission > MISSION_TIMEOUT
    ↪ * self.line:
            self.done.append({"mission": self.k, "mission_s": self.t_m

    ↪ ission, "late": self.t_mission > self.line,
                              "planned_s": self.planned, "line_s": sel
    ↪ f.line, "distance_m": self.d,
                              "energy_j": self.energy - self.m_energy0
    ↪ , "airborne_s": self.airborne - self.m_airborne0,
                              "cut_off": self.phase != "landed"})
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            self.k += 1; self.start_next()
            return self.home, (0.0, 0.0, 0.0)
        if self.phase != "idle":
            self.airborne += CTRL_DT
        return tuple(self.carrot), vel

    def motor_energy(self, rpm, km, dt):
        """Declared model: torque KM rpm^2, angular speed 2 pi rpm / 6
    ↪ 0, over the motor efficiency."""
        p = sum(km * r * r * (2 * math.pi * r / 60.0) for r in rpm) /
    ↪ MOTOR_EFF
        self.energy += p * dt

def fly(n, k, dist, seed, law_on, fixed_override=None, quiet=False):
    """One arm of one cell. Returns the cell's gauges and the per-miss
    ↪ ion rows."""
    import numpy as np

    from gym_pybullet_drones.control.DSLPIDControl import DSLPIDContro
    ↪ l
    from gym_pybullet_drones.envs.CtrlAviary import CtrlAviary
    from gym_pybullet_drones.utils.enums import DroneModel, Physics
    ms = missions(n, k, dist, seed)
    env = CtrlAviary(drone_model=DroneModel.CF2X, num_drones=n, physic

    ↪ s=Physics.PYB_DRAG, pyb_freq=PYB_FREQ,
                     ctrl_freq=CTRL_FREQ, gui=False, record=False, ini
    ↪ tial_xyzs=np.array([depot(i) for i in range(n)]))
    ctrl = [DSLPIDControl(drone_model=DroneModel.CF2X) for _ in range(
    ↪ n)]
    drones = []

    for i in range(n):
        law = CompassLaw(Band(0.0, E_LINE), dt=CTRL_DT, tau=TAU, kp=KP
    ↪ , smooth=SMOOTH) if law_on else None
        d = Drone(i, ms[i], law)
        if fixed_override is not None:
            d.override = fixed_override
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        drones.append(d)
    window = max(sum(LINE_FACTOR * planned_time(m[2], d.z) for m in d.
    ↪ missions) for d in drones)   # the fleet's own line
    obs, _ = env.reset(); act = np.zeros((n, 4)); t = 0.0; min_sep = f
    ↪ loat("inf"); collisions = 0; near = 0
    coll_d = 2 * getattr(env, "COLLISION_R", 0.06)

    while any(d.phase != "finished" for d in drones) and t < MISSION_T
    ↪ IMEOUT * window:
        obs, _, _, _, _ = env.step(act)
        pos = obs[:, 0:3]
        if n > 1:
            dm = np.linalg.norm(pos[:, None, :] - pos[None, :, :], axi

    ↪ s=2) + np.eye(n) * 1e9
            nearest = dm.min(axis=1)
        else:
            nearest = np.full(n, float("inf"))
        for i, d in enumerate(drones):
            rpm = obs[i, 16:20]

            d.motor_energy(rpm, env.KM, CTRL_DT)
            if fixed_override is not None:
                d.override = fixed_override
            tp, tv = d.step(tuple(pos[i]), rpm, float(nearest[i]))
            act[i], _, _ = ctrl[i].computeControlFromState(control_tim
    ↪ estep=env.CTRL_TIMESTEP, state=obs[i],

                                                           target_pos=
    ↪ np.array(tp), target_vel=np.array(tv))
        if n > 1:
            m = float(nearest.min()); min_sep = min(min_sep, m)
            if m < coll_d:
                collisions += 1

            if m < KEEPOUT / 2:
                near += 1
        t += CTRL_DT
    env.close()
    rows = [dict(r, drone=d.i) for d in drones for r in d.done]
    flown = [r for r in rows]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    nm = max(1, len(flown))
    motors = sum(d.energy for d in drones)
    avionics = AVIONICS_W * window * n                          # every
    ↪ drone, the whole fleet window, both arms alike
    fleet = motors + avionics
    e_mission = fleet / nm

    breaches = sum(1 for d in drones if d.energy + AVIONICS_W * window
    ↪  > (1 - RESERVE) * BATTERY_J)
    overrides = [o for d in drones for o in d.ov]
    for d in drones:                                             # the h
    ↪ and-back: every override to the planner's cruise, read back
        d.override = O_MIN

    g = {"energy_per_mission_j": e_mission, "fleet_energy_j": fleet, "
    ↪ motor_energy_j": motors, "avionics_energy_j": avionics,
         "missions_per_charge": (1 - RESERVE) * BATTERY_J / e_mission
    ↪ if e_mission > 0 else 0.0,
         "late_share": sum(1 for r in flown if r["late"]) / nm, "reser
    ↪ ve_breaches": breaches,

         "min_separation_m": min_sep if min_sep < float("inf") else 0.
    ↪ 0, "collisions": collisions, "near_miss_ticks": near,
         "tracking_rms_m": math.sqrt(sum(d.err2 for d in drones) / max
    ↪ (1, sum(d.n_err for d in drones))),
         "mission_s": sum(r["mission_s"] for r in flown) / nm, "airbor
    ↪ ne_s": sum(r["airborne_s"] for r in flown) / nm,

         "override_mean": sum(overrides) / max(1, len(overrides)), "mi
    ↪ ssions_flown": len(flown),
         "missions_cut_off": sum(1 for r in flown if r["cut_off"]), "w
    ↪ indow_s": window, "sim_s": t,
         "handed_back": all(abs(d.override - O_MIN) < 1e-9 for d in dr
    ↪ ones)}

    if not quiet:
        print(f"   {n} drones x {k} missions: {len(flown)} flown, late
    ↪  {g['late_share']:.0%}, energy {e_mission:.0f} J a mission, "
              f"tracking {g['tracking_rms_m']:.3f} m, min separation {
    ↪ g['min_separation_m']:.2f} m, collisions {collisions}, override
    ↪ {g['override_mean']:.2f}", flush=True)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return g, rows


def run_cell(name, out: Path):
    n, k, dist, seed, tuning = CELLS[name]
    print(f"== {name}: {n} drones, {k} missions each, {dist[0]:.0f} to
    ↪  {dist[1]:.0f} m, seed {seed}", flush=True)
    # the paired physics trial, native mode, neither flight counted: i
    ↪ s a faster mission cheaper at all, inside the safe error?
    t0 = time.time()
    full, _ = fly(n, 1, dist, seed, False, fixed_override=1.0, quiet=T
    ↪ rue)
    fast, _ = fly(n, 1, dist, seed, False, fixed_override=TRIAL_OVERRI
    ↪ DE, quiet=True)
    faster_is_cheaper = fast["energy_per_mission_j"] < full["energy_pe
    ↪ r_mission_j"] and fast["tracking_rms_m"] < E_LINE and fast["coll
    ↪ isions"] == 0
    rec = {"cell": name, "drones": n, "missions_per_drone": k, "distan
    ↪ ce_m": list(dist), "seed": seed, "tuning": tuning,
           "task": {"cruise_mps": V_PLAN, "climb_mps": VZ, "hover_s":
    ↪ HOVER_S, "line_factor": LINE_FACTOR, "altitudes_m": ALT[:min(n,
    ↪ len(ALT))]},
           "knob": {"override_min": O_MIN, "override_max": O_MAX, "saf
    ↪ e_error_m": E_LINE, "separation_wall_m": SEP_WALL, "trial_overri
    ↪ de": TRIAL_OVERRIDE},
           "energy_model": {"motor_efficiency": MOTOR_EFF, "avionics_w
    ↪ ": AVIONICS_W, "battery_j": BATTERY_J, "reserve": RESERVE},
           "physics_trial": {"planner_cruise": full, f"override_{TRIAL
    ↪ _OVERRIDE}": fast, "faster_is_cheaper": faster_is_cheaper},
           "omni_moves": faster_is_cheaper}
    print(f"   physics trial: {full['energy_per_mission_j']:.0f} J a m
    ↪ ission at the planner's cruise, {fast['energy_per_mission_j']:.0
    ↪ f} J at "
          f"{TRIAL_OVERRIDE} x (tracking {fast['tracking_rms_m']:.3f}
    ↪ m): {'a faster mission is cheaper; omni may move' if faster_is_c
    ↪ heaper else 'not cheaper or not safe; omni leaves the cruise nat
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ive'}", flush=True)
    all_rows = []
    for arm in ("native", "omni"):
        g, rows = fly(n, k, dist, seed, arm == "omni" and faster_is_ch
    ↪ eaper)
        for r in rows:

            r["arm"] = arm
        all_rows += rows
        g["seconds"] = round(time.time() - t0, 1); rec[arm] = g
    rec["void"] = rec["native"]["collisions"] > 0          # amendment
    ↪  2: void only when native itself collides
    if rec["void"]:

        print("   COLLISION in the native arm: the cell is void", flus
    ↪ h=True)
    elif rec["omni"]["collisions"] > 0:
        print("   COLLISION with Omni-Compass on top: a judged LOSS ro
    ↪ w", flush=True)
    out.mkdir(parents=True, exist_ok=True)

    (out / f"swarm-{name}.json").write_text(json.dumps(rec, indent=1)
    ↪ + "\n")
    with (out / f"swarm-{name}_missions.csv").open("w", newline="") as
    ↪  f:
        w = csv.DictWriter(f, fieldnames=list(all_rows[0].keys())); w.
    ↪ writeheader(); w.writerows(all_rows)

    return rec


def report(res: dict, out: Path):
    L = ["# Drone swarms: Omni-Compass on top of each drone's own auto
    ↪ pilot (gym-pybullet-drones, Crazyflie 2.x)", "",

         "native: every drone's shipped position controller at the pla
    ↪ nner's cruise; omni: the same, with the compass law on the "
         "cruise override inside the autopilot's limits, spending trac
    ↪ king slack as speed. Energy is a declared model (evidence class
    ↪ "
         "S) computed the same way for both arms. A cell where native
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ itself collides is void; a collision with Omni-Compass on top is
    ↪  "
         "a judged row, any increase a LOSS (amendment 2). Every row i
    ↪ s reported.", ""]
    for name, rec in res.items():
        L += [f"## {name}: {rec['drones']} drones x {rec['missions_per
    ↪ _drone']} missions, {rec['distance_m'][0]:.0f} to {rec['distance
    ↪ _m'][1]:.0f} m"
              + (" (the tuning swarm)" if rec.get("tuning") else ""),
    ↪ ""]
        pt = rec["physics_trial"]
        L += [f"Paired physics trial (native, uncounted): {pt['planner
    ↪ _cruise']['energy_per_mission_j']:.0f} J a mission at the planne
    ↪ r's cruise, "
              f"{pt[f'override_{TRIAL_OVERRIDE}']['energy_per_mission_
    ↪ j']:.0f} J at {TRIAL_OVERRIDE} x; "
              + ("a faster mission is cheaper and inside the safe erro
    ↪ r, so omni may move the cruise." if rec["omni_moves"] else
                 "**not cheaper or not inside the safe error: nothing
    ↪ for Omni to move; the omni arm is native.**"), ""]
        if rec.get("void"):
            L += ["**VOID: native collided.** Collisions: native "
                  f"{rec['native']['collisions']}, omni {rec['omni']['
    ↪ collisions']}.", ""]
        L += ["| Gauge | native | omni | change | direction |", "|---|
    ↪ ---:|---:|---:|---|"]
        for key, label, direction in with_collisions(GAUGES):
            a, b = rec["native"][key], rec["omni"][key]
            ch = "" if a == 0 else f"{100 * (b - a) / abs(a):+.2f}%"
            if key in ("late_share", "reserve_breaches", "near_miss_ti
    ↪ cks", "collisions"):
                ch = f"{b - a:+.3f}"
            L.append(f"| {label} | {a:.4g} | {b:.4g} | {ch} | {directi
    ↪ on} |")
        L += ["", f"Missions flown: native {rec['native']['missions_fl
    ↪ own']}, omni {rec['omni']['missions_flown']}; cut off: "
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            f"{rec['native']['missions_cut_off']} / {rec['omni']['mi
    ↪ ssions_cut_off']}; every override handed back: native "
            f"{rec['native']['handed_back']}, omni {rec['omni']['han
    ↪ ded_back']}; collisions {rec['native']['collisions']} / {rec['om
    ↪ ni']['collisions']}.", ""]
    (out / "SWARM.md").write_text("\n".join(_legal_stamp(L)) + "\n")

    return out / "SWARM.md"


def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)

    ap.add_argument("--cell", default="tuning", help="tuning | short |
    ↪  mixed | long | short-314 | mixed-315 | long-316 | all")
    ap.add_argument("--out", default="")
    ap.add_argument("--report-only", default="")
    a = ap.parse_args(argv)
    if a.report_only:

        d = Path(a.report_only)
        res = {f.stem.replace("swarm-", ""): json.loads(f.read_text())
    ↪  for f in sorted(d.glob("swarm-*.json"))}
        print(report(res, d)); return 0
    out = Path(a.out or "swarm-out")
    names = list(CELLS) if a.cell == "all" else [a.cell]

    res = {}
    for name in names:
        res[name] = run_cell(name, out)
    print(report(res, out))
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

H.58 `tools/run_sysbench.py`

700 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES

    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of a database's operator-set buffer pool: sysbe
    ↪ nch on MySQL (docs/MYSQL_PREREGISTRATION.md).

MySQL 8.0 as Ubuntu ships it, with the one setting an operator sets on
    ↪ ce for InnoDB: the buffer pool size, which the server
resizes online in chunks of 128 MB. sysbench's published OLTP workload
    ↪ s ask for rows from a set of tables that widens and
narrows over the run (one notch = one more table of a million rows), s
    ↪ o the working set steps through the operator's pool

and past it. The operator's fixed pool is the native controller here.

  native  MySQL at the operator's pool (512 MB) for the whole run
  omni    the same MySQL, with the compass law (omnicompass/compass_la
    ↪ w.py) on one knob, the buffer pool size, through the
          server's own console (SET GLOBAL innodb_buffer_pool_size), i

    ↪ nside the cover [128 MB, 2,048 MB]: the compass reads the
          workload's own mean statement latency over the last second (
    ↪ performance_schema, sysbench's account only, its prepared
          statements counted; store fix of 25 October) on a band from
    ↪ 0 to the statement
          line and holds it at 40% of the line; slow statements with t

    ↪ he pool full push the pool up a chunk or more (slow
          statements with room to spare are not the pool's to mend), c
    ↪ alm with the pool's misses under one percent of its read request
    ↪ s gives a chunk back
          (amendment 2: the pool grows only while it is missing, one p
    ↪ ercent or more, the same line the other way; a second
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
          with no read request moves nothing)
          after a dwell; at 95% of the line with the pool full four ch
    ↪ unks are added at once (fail up); the pool is handed back
          to the operator's at the end and read back; a size found at
    ↪ a value Omni did not write stops it writing

Gauges from sysbench's own latency histogram and summary (every transa
    ↪ ction counted into buckets about 2% wide), the server's
own status (the pool configured, the pages in it, the pages read from
    ↪ disk) and the host's /proc/stat (CPU seconds, the
compass's own cost included). Memory held is the resource; no energy i
    ↪ s claimed beyond the host's CPU seconds. Every row is

reported, losses included.

  python3 tools/run_sysbench.py --setup                    # the bench
    ↪ mark user, the operator's pool in the server's own
                                                           # configura
    ↪ tion file, the server restarted

  python3 tools/run_sysbench.py --workloads tuning --reps 3 --out DIR
  python3 tools/run_sysbench.py --report-only DIR          # re-read f
    ↪ inished <workload>.json files and write SYSBENCH.md
"""
from __future__ import annotations

import argparse
import json
import math
import os
import re
import subprocess

import sys
import threading
import time
from pathlib import Path

try:                                                       # the legal
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  notice every generated report carries
    from tools.legal import stamp as _legal_stamp
except ImportError:
    import pathlib as _p; sys.path.insert(0, str(_p.Path(__file__).res
    ↪ olve().parents[1])); from tools.legal import stamp as _legal_sta
    ↪ mp

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from omnicompass.compass_law import Band, CompassLaw  # noqa: E402
from tools.run_ycsb import cpu_times, engine, fmt, paired  # noqa: E40
    ↪ 2  (the same paired reading and host clock as the other stores)
from tools.knob_verdict import KnobVerdict, sample_cost, OBJECTIVES, R

    ↪ ESOURCE as DEFAULT_OBJECTIVE  # noqa: E402  (amendment 3: the br
    ↪ ain's own verdict)
from tools.store_brain import MemoryMax, read_meminfo  # noqa: E402  (
    ↪ omni-v22: the max on the pool)


HOST = os.environ.get("MYSQL_HOST", "127.0.0.1")

USER = os.environ.get("MYSQL_USER", "omni")
PASSWORD = os.environ.get("MYSQL_PWD", "omni-bench")       # a local b
    ↪ enchmark user on a throwaway runner, created by --setup
LOAD_USER = os.environ.get("SYSBENCH_USER", "omni_load")   # the accou
    ↪ nt sysbench connects as, apart from the harness's (store fix of
    ↪ 25 October):

                                                           # the brain
    ↪  reads the statements of this account only, never its own monito
    ↪ ring queries
# The statements the brain reads (store fix of 25 October): sysbench 1
    ↪ .0.20 runs its OLTP scripts as server-side prepared
# statements, which the performance schema counts as statement/com/Exe

    ↪ cute (the binary protocol's execute) or
# statement/sql/execute_sql (an SQL-level EXECUTE), not as statement/s
    ↪ ql/select and the other three. The omni-v16 runs read
# only those four, so the brain counted 4.1 statements a second in eve
    ↪ ry arm (its own and the sampler's monitoring SELECTs)
# while sysbench sent 994 to 4,839 queries a second: its force and the
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  verdict's work sample never saw the workload.
WORKLOAD_EVENTS = ("statement/sql/select", "statement/sql/update", "st
    ↪ atement/sql/insert", "statement/sql/delete",
                   "statement/com/Execute", "statement/sql/execute", "
    ↪ statement/sql/execute_sql")
DB = "sbtest"

MB = 1024 * 1024
PAGE = 16 * 1024          # InnoDB's page
LINE_STMT_MS = 0.6        # the statement line: one SQL statement answ
    ↪ ered within 0.6 ms on the server (written as 1 ms; set on the tu
    ↪ ning
                          # workload's smoke runs: a hit reads 0.14 to

    ↪  0.22 ms on GitHub's runners and a notch missing a third of its
    ↪ reads
                          # 0.05 to 0.06 ms more, so the center, 40% o
    ↪ f the line, sits at 0.24 ms, above the hits and under the misses
    ↪ )
GIVEBACK_MISS_SHARE = 0.01  # a chunk is given back only while the poo

    ↪ l's misses are under 1% of its read requests (the pool holds the
    ↪  working
                          # set); InnoDB keeps stale pages resident, s
    ↪ o "pages free" and "no page read from disk" never say that
CENTER = 0.4              # the compass holds the server's own stateme
    ↪ nt latency at 40% of the line

DT = 1.0                  # one decision a second
TAU = 2.0
SMOOTH = 0.5
NATIVE_MB = 512           # the operator's pool
OPERATOR_MB, SHIPPED_MB = 512, 128   # --native operator (preregistere
    ↪ d) or shipped (MySQL 8.0's own default pool, 128 MB)

NATIVE = "operator"
CHUNK_MB = 128            # innodb_buffer_pool_chunk_size as shipped:
    ↪ the pool is resized in whole chunks
COVER_MB = (128, 2048)    # never under one chunk, never over 2 GB
STEP_MB = CHUNK_MB        # one notch of the knob is one chunk
FAILUP_MB = 512           # past the wall: four chunks at once (about
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ a quarter of the cover), and again the next second if still past
    ↪  it
DWELL_S = 10.0           # after growing, no taking back within ten s
    ↪ econds (a shrink evicts pages and the resize itself takes second
    ↪ s)
DIAL = os.environ.get("OMNI_DIAL", "max")   # omni-v22: max, the facto

    ↪ ry tune (the Brain); save, the tune-down (omni-v21's loop)
FULL = 0.9               # the pool counts as full when the pages hol
    ↪ ding data are 90% of its pages or more
THREADS = 32             # sysbench's client threads
TABLES = 6               # tables prepared; notch n uses the first n
TABLE_ROWS = 1_000_000   # rows a table: about 240 MB of data and ind

    ↪ ex each, so notch 1 fits the operator's pool and notch 6 does no
    ↪ t
STEPS = "1 2 3 2 3 4 5 6 5 4 3 2 1 2 1"
BURST = "1 6 1 8 1 6"
RESIZE_WAIT_S = 90.0     # the server's online resize is asynchronous
    ↪ ; the plug waits this long for it to complete

RAND_TYPE = "uniform"    # rows drawn uniformly over the tables in us
    ↪ e, so the working set is the key space (sysbench's shipped 'spec
    ↪ ial'
                         # draw sends three quarters of the requests
    ↪ to one percent of the rows: the first smoke run showed the pool
                         # filling only at the top notch and misses a

    ↪ t 3.5% of reads, nothing for the knob to answer; see the preregi
    ↪ stration)
SAME_REL = 1e-6
LUA = Path(os.environ.get("SYSBENCH_LUA", "/usr/share/sysbench"))

WORKLOADS = {

    # name: (sysbench script, statements a transaction as the script s
    ↪ hips, transactions a second offered (--rate), steps, tuning?)
    #   the transaction line for the work-inside-the-line gauge is the
    ↪  statement line times the statements a transaction
    "tuning": ("oltp_point_select", 1, 3000, STEPS, True),       # one
    ↪  point select a transaction: the rules were set here
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    "read_only": ("oltp_read_only", 14, 300, STEPS, False),      # 10
    ↪ point selects and 4 range queries a transaction
    "read_write": ("oltp_read_write", 18, 200, STEPS, False),    # the
    ↪  read-only mix plus 2 updates, a delete and an insert
    "update_index": ("oltp_update_index", 1, 1000, STEPS, False),  # o
    ↪ ne indexed update a transaction

    "burst": ("oltp_read_only", 14, 300, BURST, False),          # rea
    ↪ d-only on the burst schedule
}

GAUGES = [  # key, label, direction
    ("work_inside_line_tps", "work inside the response line (transacti

    ↪ ons a second answered within the line)", "higher"),
    ("tps", "throughput (transactions a second)", "higher"),
    ("qps", "queries a second", "higher"),
    ("p95_ms", "latency, 95th percentile (ms)", "lower"),
    ("p99_ms", "latency, 99th percentile (ms)", "lower"),
    ("mean_ms", "latency, mean (ms)", "lower"),

    ("failed", "errors (sysbench's ignored errors: deadlocks and retri
    ↪ es)", "never more"),
    ("pool_mb_mean", "buffer pool held, mean (MB; the knob, the resour
    ↪ ce)", "spent"),
    ("pool_used_mb_mean", "pages holding data, mean (MB)", "spent"),
    ("disk_reads", "pages read from disk into the pool (misses)", "sho

    ↪ wn"),
    ("cpu_busy_share", "host CPU busy (share of the run)", "spent"),
    ("cpu_seconds", "host CPU-seconds", "spent"),
    ("cpu_s_per_1k_inside", "host CPU-seconds per 1,000 transactions i
    ↪ nside the line", "spent"),
    ("writes", "buffer pool size changes written (the knob's moves)",

    ↪ "shown"),
]


def connect():
    import pymysql
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return pymysql.connect(host=HOST, user=USER, password=PASSWORD, au
    ↪ tocommit=True, connect_timeout=5)


def query(conn, sql):
    with conn.cursor() as cur:
        cur.execute(sql)
        return cur.fetchall()


def status(conn):
    """The server's own figures, one read: {name: value} from SHOW GLO
    ↪ BAL STATUS for the pool and the statements' timer."""
    rows = query(conn, "SHOW GLOBAL STATUS WHERE Variable_name IN ('In
    ↪ nodb_buffer_pool_pages_total', 'Innodb_buffer_pool_pages_data',
    ↪ "
                       "'Innodb_buffer_pool_pages_free', 'Innodb_buffe
    ↪ r_pool_reads', 'Innodb_buffer_pool_read_requests', 'Innodb_buffe
    ↪ r_pool_resize_status')")
    st = {k: v for k, v in rows}
    st["pool_bytes"] = int(query(conn, "SELECT @@innodb_buffer_pool_si
    ↪ ze")[0][0])
    t = query(conn, workload_statements_sql())[0]
    st["stmt_timer_ps"], st["stmt_count"] = int(t[0]), int(t[1])
    f = query(conn, FILE_READS_SQL)[0]
    st["file_reads"], st["file_read_ps"] = int(f[0]), int(f[1])
    return st


# omni-v22: the server's own account of what a pool miss costs: the pe
    ↪ rformance schema's timer on the data files' reads (every
# page the pool did not hold and had to read, read-ahead included), in
    ↪  picoseconds, beside the workload's own statement timer.
# The instrument wait/io/file/innodb/innodb_data_file is enabled and t
    ↪ imed as MySQL 8.0 ships (checked on 8.0.46).
FILE_READS_SQL = ("SELECT COALESCE(SUM(COUNT_READ), 0), COALESCE(SUM(S
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ UM_TIMER_READ), 0) FROM performance_schema.file_summary_by_event
    ↪ _name "
                  "WHERE EVENT_NAME = 'wait/io/file/innodb/innodb_data
    ↪ _file'")

def file_reads(st):
    """(data file reads, their time in picoseconds) from one status re
    ↪ ad."""
    return st["file_reads"], st["file_read_ps"]

def workload_statements_sql(user=None):
    """The workload's statement timer and count: sysbench's own accoun
    ↪ t only (the harness's monitoring queries run as another
    account and are never read), its plain and prepared statements ali
    ↪ ke (WORKLOAD_EVENTS)."""
    user = LOAD_USER if user is None else user

    events = ", ".join(f"'{e}'" for e in WORKLOAD_EVENTS)
    return ("SELECT COALESCE(SUM(SUM_TIMER_WAIT), 0), COALESCE(SUM(COU
    ↪ NT_STAR), 0) FROM performance_schema.events_statements_summary_b
    ↪ y_user_by_event_name "
            f"WHERE USER = '{user}' AND EVENT_NAME IN ({events})")

def pool_stats(st):
    """(configured MB, pages holding data MB, pages read from disk, re
    ↪ size status text) from one status read."""
    return st["pool_bytes"] // MB, int(st.get("Innodb_buffer_pool_page
    ↪ s_data", 0)) * PAGE / MB, int(st.get("Innodb_buffer_pool_reads",

    ↪  0)), str(st.get("Innodb_buffer_pool_resize_status", ""))


def read_requests(st):
    """The pool's logical read requests (hits and misses together) fro
    ↪ m one status read."""

    return int(st.get("Innodb_buffer_pool_read_requests", 0))


def miss_share(disk_reads_delta, requests_delta):
    """The pool's misses as a share of its read requests over an inter
    ↪ val; no requests reads as no misses."""
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return (disk_reads_delta / requests_delta) if requests_delta > 0 e
    ↪ lse 0.0


def pool_full(st):
    total = int(st.get("Innodb_buffer_pool_pages_total", 0)) or 1

    return int(st.get("Innodb_buffer_pool_pages_data", 0)) >= FULL * t
    ↪ otal


def stmt_latency(st):
    """(total statement time in picoseconds, statements) from one stat

    ↪ us read (performance_schema)."""
    return st["stmt_timer_ps"], st["stmt_count"]


def resizing(text):
    t = text.lower()

    return bool(t) and "completed" not in t and ("resiz" in t or "prog
    ↪ ress" in t or "withdraw" in t or "start" in t)


class BufferPool:
    """The plug on the server's buffer pool size: read once before the

    ↪  first write, write through the server's own console in
    whole chunks, wait for the server's online resize, read back, yiel
    ↪ d to any other writer, restore."""

    def __init__(self, conn):
        self.c, self.snapshot, self.last_written = conn, None, None

        self.restore_info = {}

    def _status(self):
        return status(self.c)

    def configured_mb(self, st=None):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        st = st or self._status()
        return st["pool_bytes"] // MB

    def attach(self):
        self.snapshot = self.configured_mb(); self.last_written = self
    ↪ .snapshot
        return self.snapshot

    def lever(self, st=None):
        st = st or self._status()
        cur = self.configured_mb(st)
        if resizing(pool_stats(st)[3]):
            return self.last_written if self.last_written is not None
    ↪ else cur     # mid-resize: the server is still carrying out our
    ↪ write
        if self.last_written is not None and cur != self.last_written:
            raise RuntimeError(f"another writer set the pool to {cur}
    ↪ MB (we wrote {self.last_written}): Omni stops writing")
        return cur

    def write(self, mb):
        mb = int(round(mb / CHUNK_MB)) * CHUNK_MB
        query(self.c, f"SET GLOBAL innodb_buffer_pool_size = {mb * MB}
    ↪ ")
        self.last_written = mb
        t0 = time.monotonic()
        while time.monotonic() - t0 < RESIZE_WAIT_S:
            st = self._status()
            if self.configured_mb(st) == mb and not resizing(pool_stat
    ↪ s(st)[3]):
                return mb
            time.sleep(0.5)
        return self.configured_mb()

    def _settle(self, budget_s):
        """Wait, up to budget_s, for a resize the server is still carr
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ying out; the status seen first and the seconds waited."""
        t0 = time.monotonic(); st = self._status(); first = pool_stats
    (st)[3]
        while resizing(pool_stats(st)[3]) and time.monotonic() - t0 <
    budget_s:
            time.sleep(0.5); st = self._status()

        return first, round(time.monotonic() - t0, 1)

    def restore(self):
        """Hand the pool back to the snapshot and read it back. The se
    rver carries out a resize on its own time, and a shrink
        under load withdraws its last blocks only when the load eases;

     while one is in progress the server ignores a new size.
        So: wait for any resize in flight, write the snapshot, and if
    the server did not take it (still resizing), wait once
        more and write once more. The receipt (what was in flight, the
     seconds waited, the writes, the final read-back) is
        restore_info, written into the arm's record. The first counted

     runs restored without this wait and 15 of 45 omni arms
        read not handed back for exactly this reason (docs/MYSQL_PRERE
    GISTRATION.md, the first set's result)."""
        info = {"ok": False, "resize_in_flight_at_end": "", "waited_s"
    : 0.0, "writes": 0, "final_mb": None}
        try:

            first, waited = self._settle(RESIZE_WAIT_S)
            info["resize_in_flight_at_end"], info["waited_s"] = first,
     waited
            got = None
            for _ in range(2):
                got = self.write(self.snapshot); info["writes"] += 1

                if got == self.snapshot:
                    break
                _, w2 = self._settle(RESIZE_WAIT_S); info["waited_s"]
    = round(info["waited_s"] + w2, 1)
            info["final_mb"], info["ok"] = got, (got == self.snapshot)
        except Exception as e:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            info["error"] = repr(e)
        self.restore_info = info
        return info["ok"]


GROW_MISS_SHARE = GIVEBACK_MISS_SHARE  # amendment 2: the pool grows o
    ↪ nly while its misses are 1% or more of its read requests (it is
    ↪ not holding
                                       # the working set); a slow stat
    ↪ ement in a full pool that holds its working set is not the pool'
    ↪ s to mend.
                                       # The same one-percent line gat
    ↪ es the give-back; no new number.


def decide(p, force, misses_last_s, cur_mb, last_change_age, full, wal
    ↪ l=0.95, reads=True):
    """The knob's move this second, in MB: slow statements (the servic
    ↪ e slow) grow the pool by ceil(force / 0.1) chunks, but only
    while the pool is full (a slow statement with room to spare is not
    ↪  the pool's to mend) and missing (amendment 2: misses at one
    percent of its read requests or more; a full pool that holds its w
    ↪ orking set cannot mend a slow statement); calm with the
    pool's misses under one percent of its read requests (the pool hol
    ↪ ds the working set) gives back one chunk after the dwell,
    and only in a second that saw read requests (amendment 2: a second
    ↪  with none says nothing about the working set); past the
    wall, with the pool full, four chunks at once. misses_last_s is th
    ↪ e miss share of the last second."""
    lo, hi = COVER_MB

    if p is not None and p >= wall and full:
        return min(hi, cur_mb + FAILUP_MB), "fail up: the line is at h
    ↪ and and the pool is full, four chunks at once"
    if force > 0.05:
        if not full:
            return cur_mb, "slow with room to spare: not the pool's to
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  mend"
        if misses_last_s < GROW_MISS_SHARE:
            return cur_mb, "slow with the pool full but holding its wo
    ↪ rking set (misses under 1% of reads): not the pool's to mend"
        return min(hi, cur_mb + STEP_MB * max(1, math.ceil(force / 0.1
    ↪ ))), "slow with the pool full and missing: grow"

    if force < -0.05 and not reads:
        return cur_mb, "calm, but no read request in the last second:
    ↪ nothing known, nothing taken"
    if force < -0.05 and misses_last_s < GIVEBACK_MISS_SHARE and cur_m
    ↪ b > lo and last_change_age >= DWELL_S:
        return max(lo, cur_mb - STEP_MB), "calm, the pool holding the

    ↪ working set (misses under 1% of reads): a chunk given back"
    return cur_mb, "hold"


# omni-v22: the max on the buffer pool (docs/MYSQL_PREREGISTRATION.md,
    ↪  amendment 6; docs/STORE_ROOT_CAUSES_2026-10-30.md): the

# wire's own rule (decide) with the spend never clamped by an allowanc
    ↪ e where the server's own account says the misses cost the
# statements (the performance schema's timer on the data files' reads
    ↪ against the workload's statement timer: tools/store_brain.py),
# held where they cost nothing (on a runner whose page cache holds the
    ↪  data files a pool miss is a read from memory: 9% to 12% of

# page reads missing with the latency unmoved), held while the host ha
    ↪ s no chunk to spare, and no give-back below the working set's
# floor nor below the operator's pool. `--dial save` keeps omni-v21's
    ↪ loop, the verdict under the resource objective, where giving
# memory back is the aim.
METER = "the performance schema's timer on the data files' reads again

    ↪ st the workload's statement timer"


class Brain:
    """The knob's decision, one a second: obs holds reading_s (the mea
    ↪ n statement latency), statements, miss_share, pool_mb,
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    used_mb, full, reads (any read request this second), resizing, cpu
    ↪ _share, miss_time_s (the data files' read time this second),
    service_time_s (the statements' own time this second), host_avail_
    ↪ mb and host_total_mb (the host's memory, None unread)."""

    def __init__(self, native, objective=DEFAULT_OBJECTIVE, line_ms=LI
    ↪ NE_STMT_MS):
        self.native, self.objective = native, objective
        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE
    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)
        self.max = MemoryMax(native, STEP_MB, COVER_MB, FULL, METER)
        self.last_change = -1e9

    @property
    def failups(self):
        return self.max.failups

    def record(self):
        return self.max.record(self.objective)

    def step(self, t, obs):
        reading, cur = obs["reading_s"], obs["pool_mb"]
        f = self.law.force(reading)
        want, why = decide(self.law.p, f, obs["miss_share"], cur, t -
    ↪ self.last_change, obs["full"], wall=self.law.band.wall_high, rea
    ↪ ds=obs.get("reads", True))
        target, why, info = self.max.step(t, want, why, cur, obs["used
    ↪ _mb"], obs.get("miss_time_s"), obs.get("service_time_s"),
                                          obs.get("host_avail_mb"), ob
    ↪ s.get("host_total_mb"), fail_up=why.startswith("fail up"))
        info.update({"p": round(self.law.p, 4), "force": round(f, 4)})
        return target, why, info


class Omni(threading.Thread):
    """The brain, one decision a second, reading the server's own stat
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ement latency, writing the audit; the knob is handed back
    at the end and read back. Amendment 3 (the brain's own verdict, 20
    ↪ 26-10-09): the pool starts in watch and is written only
    inside the allowance a paired trial on the server itself has earne
    ↪ d under the declared objective (tools/knob_verdict.py
    around the engine's own Verdict), one chunk a trial; a trial holds

    ↪  the pool, and no sample is taken while the server is
    still resizing; a fail-up never spends beyond the allowance."""

    def __init__(self, plug: BufferPool, audit_path: Path, line_ms=LIN
    ↪ E_STMT_MS, objective=DEFAULT_OBJECTIVE, dial=DIAL):
        super().__init__(daemon=True)

        self.plug, self.audit, self.objective, self.dial, self.line_ms
    ↪  = plug, audit_path, objective, dial, line_ms
        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE
    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)
        self.stop_flag = threading.Event(); self.last_change = -1e9; s
    ↪ elf.writes = 0; self.failups = 0

        self.foreign = False; self.handed_back = False; self.error = N
    ↪ one; self.verdict = None; self.brain = None

    def govern_record(self):
        """The record of what governed the knob: the max's counts (omn
    ↪ i-v22) or the verdict's allowance (the tune-down)."""

        if self.brain is not None:
            return self.brain.record()
        return self.verdict.record() if self.verdict else None

    def run(self):
        try:

            native = self.plug.attach(); st = self.plug._status(); lat
    ↪ 0, n0 = stmt_latency(st); _, _, rd0, _ = pool_stats(st); rq0 = r
    ↪ ead_requests(st)
            fr0, frt0 = file_reads(st)
        except Exception as e:                                      #
    ↪ the brain never ran: say so in the audit and the record, never s
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ilently
            self.error = repr(e)
            self.audit.write_text(json.dumps({"t": 0.0, "error": self.
    ↪ error, "note": "the brain could not attach; no decision was made
    ↪ "}) + "\n")
            return

        t0 = time.monotonic(); cpu_prev = cpu_times()
        self.verdict = KnobVerdict(native, STEP_MB, COVER_MB, objectiv
    ↪ e=self.objective, settle_s=TAU, max_trial=120)   # a resize take
    ↪ s seconds
        if self.dial == "max":
            self.brain = Brain(native, self.objective, self.line_ms);

    ↪ self.verdict = None
        with open(self.audit, "w") as fh:
            while not self.stop_flag.is_set():
                tick = time.monotonic()
                try:
                    st = self.plug._status()

                    lat, n = stmt_latency(st); dl, dn = lat - lat0, n
    ↪ - n0; lat0, n0 = lat, n
                    fr, frt = file_reads(st); fr_last, frt_last = fr -
    ↪  fr0, frt - frt0; fr0, frt0 = fr, frt
                    reading = (dl / dn / 1e12) if dn > 0 else 0.0
    ↪    # the mean statement latency of the last second, in seconds;

    ↪ none reads calm
                    f = self.law.force(reading)
                    cur, used, rd, rs = pool_stats(st); rd_last = rd -
    ↪  rd0; rd0 = rd
                    rq = read_requests(st); rq_last = rq - rq0; rq0 =
    ↪ rq

                    share = miss_share(rd_last, rq_last)
                    cur = self.plug.lever(st); full = pool_full(st)
                    cpu_now = cpu_times(); d_total = cpu_now[0] - cpu_
    ↪ prev[0]; d_idle = cpu_now[1] - cpu_prev[1]; cpu_prev = cpu_now
                    cpu_share = (1.0 - d_idle / d_total) if d_total >
    ↪ 0 else None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                cost = sample_cost(self.objective, dn, reading if
↪ dn > 0 else None, cur, cpu_share)
                if self.brain is not None:
↪                    # the max (omni-v22)
                    h_avail, h_total = read_meminfo()
                    obs = {"reading_s": reading, "statements": dn,
↪  "miss_share": share, "pool_mb": cur, "used_mb": used, "full": f
↪ ull,
                           "reads": rq_last > 0, "resizing": resiz
↪ ing(rs), "cpu_share": cpu_share,
                           "miss_time_s": frt_last / 1e12, "servic
↪ e_time_s": dl / 1e12, "host_avail_mb": h_avail, "host_total_mb":
↪  h_total}
                    self.brain.last_change = self.last_change - t0
                    target, why, vinfo = self.brain.step(tick - t0
↪ , obs)
                    vinfo["file_reads_last_s"] = fr_last
                    self.failups = self.brain.failups
                else:
↪                    # the tune-down: omni-v21's loop
                    self.verdict.observe(cost, tick - t0, settled=
↪ not resizing(rs))  # measured under the pool of the last second,
↪  once resized
                    target, why = decide(self.law.p, f, share, cur
↪ , tick - self.last_change, full, wall=self.law.band.wall_high, r
↪ eads=rq_last > 0)
                    target, why, vinfo = self.verdict.decide(
                        target, spend_ok=(f > 0.05 and full and sh
↪ are >= GROW_MISS_SHARE),
                        give_ok=(f < -0.05 and rq_last > 0 and sha
↪ re < GIVEBACK_MISS_SHARE),
                        t=tick - t0, why=why, stress=f > 0.05, cal
↪ m=f < -0.05, fail_up=why.startswith("fail up"))
                wrote = None
                if target != cur and not resizing(rs):
                    wrote = self.plug.write(target); self.writes +
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ = 1
                        self.last_change = tick
    ↪      # a shrink and a grow both start the dwell: the resize take
    ↪ s seconds
                        if why.startswith("fail up") and self.brain is
    ↪  None:
                            self.failups += 1
                    fh.write(json.dumps({"t": round(tick - t0, 2), "re
    ↪ ading_ms": round(reading * 1000, 3), "statements": dn, "p": roun
    ↪ d(self.law.p, 4), "force": round(f, 4),
                                         "disk_reads_last_s": rd_last,
    ↪  "read_requests_last_s": rq_last, "miss_share": round(share, 4),
    ↪  "pool_mb": cur, "used_mb": round(used, 1),
                                         "full": full, "resize": rs, "
    ↪ target_mb": target, "cpu_share": None if cpu_share is None else
    ↪ round(cpu_share, 4),
                                         "cost": None if cost is None
    ↪ else round(cost, 9), **vinfo,
                                         "wrote_mb": wrote, "why": why
    ↪ }) + "\n")
                except Exception as e:
                    fh.write(json.dumps({"t": round(tick - t0, 2), "er
    ↪ ror": repr(e)}) + "\n")
                    self.foreign = True
                    break
                time.sleep(max(0.0, DT - (time.monotonic() - tick)))
        self.handed_back = self.plug.restore()


class Sampler(threading.Thread):
    """The pool configured, the pages holding data and the pages read
    ↪ from disk, once a second, in both arms."""

    def __init__(self, conn):
        super().__init__(daemon=True)
        self.c, self.stop_flag, self.rows = conn, threading.Event(), [
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ]

    def run(self):
        while not self.stop_flag.is_set():
            try:
                self.rows.append(pool_stats(status(self.c))[:3])
            except Exception:
                pass
            time.sleep(DT)


def parse_histogram(text):
    """sysbench's latency histogram (--histogram=on): one line a bucke
    ↪ t, 'value |bars count', values in ms; returns [(ms, count)]."""
    out = []
    on = False
    for line in text.splitlines():
        if "Latency histogram" in line:
            on = True; continue
        if on:
            m = re.match(r"\s*([0-9.]+)\s*\|[* ]*\s*(\d+)\s*$", line)
            if m:
                out.append((float(m.group(1)), int(m.group(2))))
            elif out and line.strip() and not line.strip().startswith(
    ↪ "value"):
                on = False
    return out


def parse_summary(text):
    """transactions, queries, ignored errors, seconds, and sysbench's
    ↪ own mean and 95th percentile (ms) from its summary."""
    def grab(pat, default=0.0):
        m = re.search(pat, text)
        return float(m.group(1)) if m else default
    return {"transactions": grab(r"transactions:\s+(\d+)"), "queries":
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  grab(r"queries:\s+(\d+)"), "errors": grab(r"ignored errors:\s+(
    ↪ \d+)"),
            "seconds": grab(r"total time:\s+([0-9.]+)s"), "avg_ms": gr
    ↪ ab(r"avg:\s+([0-9.]+)"), "p95_ms_sysbench": grab(r"95th percenti
    ↪ le:\s+([0-9.]+)")}

def quantile(hist, q):
    """A quantile from the histogram's buckets (the bucket's value sta
    ↪ nds for its latency; buckets are about 2% apart)."""
    total = sum(c for _, c in hist)
    if not total:
        return float("nan")
    acc = 0
    for v, c in hist:
        acc += c
        if acc >= q * total:
            return v
    return hist[-1][0]


def sysbench(script, cmd, tables, rate, seconds, out_file: Path | None
    ↪ , extra=()):
    args = ["sysbench", str(LUA / f"{script}.lua"), f"--mysql-host={HO
    ↪ ST}", f"--mysql-user={LOAD_USER}", f"--mysql-password={PASSWORD}
    ↪ ", f"--mysql-db={DB}",
            f"--tables={tables}", f"--table-size={TABLE_ROWS}", f"--th
    ↪ reads={THREADS}", *extra]
    if cmd == "run":
        args += [f"--rate={rate}", f"--time={int(seconds)}", f"--rand-
    ↪ type={RAND_TYPE}", "--histogram=on", "--report-interval=0", "--p
    ↪ ercentile=95"]
    args.append(cmd)
    r = subprocess.run(args, capture_output=True, text=True)
    txt = r.stdout + "\n" + r.stderr
    if out_file is not None:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        out_file.write_text(txt)
    return txt


def prepare_dataset(log: Path):
    """The dataset, the same tables for every workload: dropped and pr
    ↪ epared once per workload (both arms read the same rows)."""
    c = connect(); query(c, f"DROP DATABASE IF EXISTS {DB}"); query(c,
    ↪  f"CREATE DATABASE {DB}"); c.close()
    txt = sysbench("oltp_read_only", "prepare", TABLES, 0, 0, log)
    if "Creating table" not in txt and "Inserting" not in txt:
        raise RuntimeError(f"sysbench prepare failed: {txt[-2000:]}")

def server_journal(unit):
    """The server's own journal and status, for the record when a rest
    ↪ art does not come back."""
    out = []

    for cmd in (["sudo", "systemctl", "status", unit, "--no-pager", "-
    ↪ l"], ["sudo", "journalctl", "-u", unit, "--no-pager", "-n", "80"
    ↪ ]):
        r = subprocess.run(cmd, capture_output=True, text=True)
        out.append(" ".join(cmd) + "\n" + (r.stdout + r.stderr)[-6000:
    ↪ ])

    return "\n".join(out)


def fresh_server(pool_mb):
    """The pool emptied and the operator's size restored before an arm
    ↪ , both arms alike: the server restarted at the operator's

    configured size, the OS page cache dropped. A server that has not
    ↪ answered after its restart is restarted once more, with
    its journal printed; one that has not answered after that stops th
    ↪ e run with the journal in the error (20 October: the
    MongoDB rig had died at the same wait with nothing recorded)."""
    for attempt in (1, 2):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        subprocess.run(["sudo", "systemctl", "restart", "mysql"], chec
    ↪ k=True)
        subprocess.run(["sudo", "sh", "-c", "sync; echo 3 > /proc/sys/
    ↪ vm/drop_caches"], check=False)
        for _ in range(120):
            try:
                c = connect(); st = status(c)
                if st["pool_bytes"] // MB != pool_mb:
                    BufferPool(c).write(pool_mb)
                return c
            except Exception:
                time.sleep(1)
        journal = server_journal("mysql")
        print(f"mysql did not answer after its restart (attempt {attem
    ↪ pt} of 2); its status and journal:\n{journal}", flush=True)
    raise RuntimeError("mysql did not come back after two restarts; it
    ↪ s status and journal are printed above")

def run_arm(arm, wl_dir: Path, rep, script, stmts, rate, steps, step_s
    ↪ , line_stmt_ms, objective=DEFAULT_OBJECTIVE):
    d = wl_dir / f"rep-{rep}" / arm; d.mkdir(parents=True, exist_ok=Tr
    ↪ ue)
    line_ms = line_stmt_ms * stmts                          # the tr
    ↪ ansaction line: the statement line times the statements a transa
    ↪ ction
    c = fresh_server(NATIVE_MB)
    sampler = Sampler(connect()); sampler.start()
    omni = None
    if arm == "omni":
        omni = Omni(BufferPool(connect()), d / "audit.jsonl", line_stm
    ↪ t_ms, objective); omni.start()
    hist_all, failed, tx, qs, summaries = [], 0, 0, 0, []
    cpu0 = cpu_times(); t0 = time.time()
    for i, notch in enumerate(int(s) for s in steps.split()):
        txt = sysbench(script, "run", notch, rate, step_s, d / f"notch
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ -{i + 1}.log")
        h = parse_histogram(txt); s = parse_summary(txt); summaries.ap
    ↪ pend(txt)
        hist_all += h; failed += int(s["errors"]); tx += int(s["transa
    ↪ ctions"]); qs += int(s["queries"])
        inside = sum(cnt for v, cnt in h if v <= line_ms); n = sum(cnt
    ↪  for _, cnt in h)
        row = sampler.rows[-1] if sampler.rows else (NATIVE_MB, 0.0, 0
    ↪ )
        print(f"   {arm} rep {rep} notch {notch}: {int(s['transactions
    ↪ '])} transactions, mean {s['avg_ms']:.3f} ms, p95 {quantile(h, 0
    ↪ .95):.3f} ms, inside the line "
              f"{100 * inside / max(1, n):.1f}%, pool {row[0]:.0f} MB,
    ↪  data pages {row[1]:.0f} MB", flush=True)
    t1 = time.time(); cpu1 = cpu_times()
    if omni:
        omni.stop_flag.set(); omni.join(timeout=4 * RESIZE_WAIT_S + 30
    ↪ )      # the last decision's resize, the settle, two writes
    sampler.stop_flag.set(); sampler.join(timeout=5)
    (d / "sysbench.log").write_text("\n\n".join(summaries))
    seconds = t1 - t0
    hist = sorted(hist_all); n = sum(cnt for _, cnt in hist)
    inside = sum(cnt for v, cnt in hist if v <= line_ms)
    rows = sampler.rows or [(NATIVE_MB, 0.0, 0)]
    total = cpu1[0] - cpu0[0]; idle = cpu1[1] - cpu0[1]; cpu_s = (tota
    ↪ l - idle) / os.sysconf("SC_CLK_TCK")
    g = {"arm": arm, "rep": rep, "seconds": round(seconds, 1), "transa
    ↪ ctions": tx, "queries": qs,
         "work_inside_line_tps": inside / seconds, "tps": tx / seconds
    ↪ , "qps": qs / seconds, "p95_ms": quantile(hist, 0.95), "p99_ms":
    ↪  quantile(hist, 0.99),
         "mean_ms": (sum(v * cnt for v, cnt in hist) / n) if n else fl
    ↪ oat("nan"), "failed": failed,
         "pool_mb_mean": sum(x[0] for x in rows) / len(rows), "pool_us
    ↪ ed_mb_mean": sum(x[1] for x in rows) / len(rows),
         "disk_reads": rows[-1][2] - rows[0][2], "cpu_busy_share": (to
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ tal - idle) / max(1, total), "cpu_seconds": cpu_s,
         "cpu_s_per_1k_inside": 1000.0 * cpu_s / max(1, inside), "writ
    ↪ es": (omni.writes if omni else 0),
         "handed_back": (omni.handed_back if omni else True), "foreign
    ↪ _writer": (omni.foreign if omni else False), "failups": (omni.fa
    ↪ ilups if omni else 0),

         "controller_error": (omni.error if omni else None),
         "restore": (omni.plug.restore_info if omni else {}),
         "verdict": (omni.govern_record() if omni else None)}
    (d / "arm.json").write_text(json.dumps(g, indent=1))
    print(f"   {arm} rep {rep}: {tx} transactions, inside the line {g[
    ↪ 'work_inside_line_tps']:.0f}/s, p95 {g['p95_ms']:.2f} ms, pool {

    ↪ g['pool_mb_mean']:.0f} MB, "
          f"data pages {g['pool_used_mb_mean']:.0f} MB, disk reads {g[
    ↪ 'disk_reads']:,}, CPU {cpu_s:.1f} s, handed back {g['handed_back
    ↪ ']}"
          + (f", verdict {g['verdict']['state']}" if g.get("verdict")
    ↪ else ""), flush=True)

    return g


def server_version():
    try:
        return str(query(connect(), "SELECT VERSION()")[0][0])

    except Exception:
        return "?"


def sysbench_version():
    try:

        return subprocess.run(["sysbench", "--version"], capture_outpu
    ↪ t=True, text=True).stdout.strip()
    except OSError:
        return "?"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def run_workload(name, out: Path, reps, step_s, line_stmt_ms, objectiv
    ↪ e=DEFAULT_OBJECTIVE):
    script, stmts, rate, steps, tuning = WORKLOADS[name]
    wl_dir = out / f"sysbench-{name}"; wl_dir.mkdir(parents=True, exis
    ↪ t_ok=True)
    ver = server_version()

    print(f"== {name}: sysbench {script} ({stmts} statements a transac
    ↪ tion, {rate} a second offered), {TABLES} tables of {TABLE_ROWS:,
    ↪ } rows, steps {steps}, "
          f"{step_s} s a notch, MySQL {ver}, the verdict's objective {
    ↪ objective}", flush=True)
    fresh_server(NATIVE_MB)

    prepare_dataset(wl_dir / "prepare.log")
    rec = {"workload": name, "tuning": tuning, "script": script, "stat
    ↪ ements": stmts, "rate_tps": rate, "steps": steps, "step_s": step
    ↪ _s,
           "line_stmt_ms": line_stmt_ms, "line_ms": line_stmt_ms * stm
    ↪ ts, "threads": THREADS, "tables": TABLES, "table_rows": TABLE_RO

    ↪ WS, "rand_type": RAND_TYPE,
           "native_mb": NATIVE_MB, "chunk_mb": CHUNK_MB, "cover_mb": l
    ↪ ist(COVER_MB), "mysql_version": ver, "sysbench_version": sysbenc
    ↪ h_version(),
           "objective": objective, "dial": DIAL, "native": NATIVE, "en
    ↪ gine": engine(), "reps": []}

    for rep in range(1, reps + 1):
        order = ("native", "omni") if rep % 2 else ("omni", "native")
        r = {}
        for arm in order:
            r[arm] = run_arm(arm, wl_dir, rep, script, stmts, rate, st
    ↪ eps, step_s, line_stmt_ms, objective)

        rec["reps"].append(r)
        (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1)
    ↪ )
    rec["paired"] = {k: paired(rec["reps"], k, dr) for k, _, dr in GAU
    ↪ GES}
    (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return rec


def report(recs, out: Path):
    shipped = bool(recs) and all(r.get("native") == "shipped" for r in
    ↪  recs)
    native_mb = recs[0]["native_mb"] if recs else NATIVE_MB
    L = ["# sysbench on MySQL: Omni-Compass on top of a database's " +
    ↪  ("buffer pool as shipped" if shipped else "operator-set buffer
    ↪ pool"), "",
         (f"MySQL exactly as Ubuntu ships it, its InnoDB buffer pool a
    ↪ t MySQL's own default ({native_mb} MB), is native" if shipped el
    ↪ se
          f"MySQL as Ubuntu ships it with the operator's InnoDB buffer
    ↪  pool ({native_mb} MB) is native")
         + "; omni is the compass law on the pool size "
         f"through the server's own console inside [{COVER_MB[0]}, {CO
    ↪ VER_MB[1]}] MB in chunks of {CHUNK_MB} MB, holding the server's
    ↪ own statement "
         f"latency at {CENTER:.0%} of the {LINE_STMT_MS:g} ms statemen
    ↪ t line. sysbench's published OLTP workloads, the same transactio
    ↪ ns in both arms, "
         "the tables in use stepping through the pool and past it. Mem
    ↪ ory held is the resource; the host's CPU seconds include the com
    ↪ pass's own "
         "cost. Every row is reported, losses included.", ""]
    for rec in recs:
        L += [f"## {rec['workload']}: sysbench {rec['script']}, {rec['
    ↪ statements']} statements a transaction, {rec['rate_tps']} a seco
    ↪ nd offered, steps {rec['steps']} × {rec['step_s']} s"
              + (" (the tuning workload)" if rec.get("tuning") else ""
    ↪ ), "",
              f"MySQL {rec['mysql_version']}, {rec['sysbench_version']
    ↪ }; {rec['threads']} client threads; the transaction line {rec['l
    ↪ ine_ms']:g} ms; "
              f"{len(rec['reps'])} paired repetitions; engine {rec['en
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ gine']['version']} at `{rec['engine']['commit']}`.", "",
              "| Gauge | native | omni | change | 95% interval of the
    ↪ difference | reading |", "|---|---:|---:|---:|---:|---|"]
        for k, label, direction in GAUGES:
            p = rec.get("paired", {}).get(k)
            if not p:

                continue
            ch = "" if p["native"] == 0 else f"{100 * p['diff'] / abs(
    ↪ p['native']):+.1f}%"
            L.append(f"| {label} | {fmt(p['native'])} | {fmt(p['omni']
    ↪ )} | {ch} | {fmt(p['ci95'][0])} to {fmt(p['ci95'][1])} | {p['rea
    ↪ ding']} |")

        hb = all(r["omni"]["handed_back"] for r in rec["reps"]); fw =
    ↪ any(r["omni"]["foreign_writer"] for r in rec["reps"])
        L += ["", f"Every omni arm handed back to native's pool size (
    ↪ {rec['native_mb']} MB): {'yes' if hb else 'NO'}; another writer
    ↪ seen: {'yes' if fw else 'no'}; "
              f"fail-ups: {sum(r['omni']['failups'] for r in rec['reps

    ↪ '])}.", ""]
    (out / "SYSBENCH.md").write_text("\n".join(_legal_stamp(L)) + "\n"
    ↪ )
    return out / "SYSBENCH.md"

def setup():
    """The benchmark user on the local server, the operator's pool and
    ↪  chunk in the server's own configuration file, the
    performance schema on; the server restarted. MySQL and sysbench th
    ↪ emselves are installed by the workflow from Ubuntu's own
    packages."""

    sql = (f"CREATE USER IF NOT EXISTS '{USER}'@'%' IDENTIFIED BY '{PA
    ↪ SSWORD}'; GRANT ALL PRIVILEGES ON *.* TO '{USER}'@'%' WITH GRANT
    ↪  OPTION; "
           f"CREATE USER IF NOT EXISTS '{LOAD_USER}'@'%' IDENTIFIED BY
    ↪  '{PASSWORD}'; GRANT ALL PRIVILEGES ON {DB}.* TO '{LOAD_USER}'@'
    ↪ %'; FLUSH PRIVILEGES;")
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ok = False
    for cmd in (["sudo", "mysql", "-e", sql], ["mysql", "-uroot", "-pr
    ↪ oot", "-h", HOST, "-e", sql]):
        r = subprocess.run(cmd, capture_output=True, text=True)
        if r.returncode == 0:
            ok = True; break

    if not ok:
        print(f"could not create the benchmark user: {r.stderr[-500:]}
    ↪ ", file=sys.stderr); return 1
    conf = Path("/etc/mysql/mysql.conf.d/omni-bench.cnf")
    txt = (f"[mysqld]\ninnodb_buffer_pool_size = {NATIVE_MB}M\ninnodb_
    ↪ buffer_pool_chunk_size = {CHUNK_MB}M\ninnodb_buffer_pool_instanc

    ↪ es = 1\n"
           "performance_schema = ON\n")
    subprocess.run(["sudo", "tee", str(conf)], input=txt, text=True, c
    ↪ heck=True, capture_output=True)
    subprocess.run(["sudo", "systemctl", "restart", "mysql"], check=Tr
    ↪ ue)

    c = fresh_server(NATIVE_MB)
    print(f"MySQL {server_version()} at native's {NATIVE_MB} MB pool (
    ↪ {NATIVE}) (chunk {CHUNK_MB} MB); {sysbench_version()}; user {USE
    ↪ R}, load user {LOAD_USER}; pool reads {status(c)['pool_bytes'] /
    ↪ / MB} MB")
    return 0

def main(argv=None):
    global NATIVE, NATIVE_MB
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)

    ap.add_argument("--setup", action="store_true")
    ap.add_argument("--native", default="operator", choices=("operator
    ↪ ", "shipped"),
                    help="native's pool: the operator's 512 MB (prereg
    ↪ istered) or MySQL as shipped, its own 128 MB (the founder's orde
    ↪ r of "
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
                          "17 October: native as it is shipped); omni s
    ↪ tarts from the same pool and goes beyond it if it can")
    ap.add_argument("--workloads", default="tuning")
    ap.add_argument("--reps", type=int, default=3)
    ap.add_argument("--step-s", type=float, default=20.0)
    ap.add_argument("--line-ms", type=float, default=LINE_STMT_MS, hel
    ↪ p="the statement line (ms)")
    ap.add_argument("--out", default="sysbench-out")
    ap.add_argument("--objective", default=DEFAULT_OBJECTIVE, choices=
    ↪ OBJECTIVES,
                    help="what the brain's verdict judges a step by: r
    ↪ esource (the index's reading: work, speed, memory and CPU togeth
    ↪ er) or service (work and speed alone)")
    ap.add_argument("--report-only", default="")
    a = ap.parse_args(argv)
    NATIVE, NATIVE_MB = a.native, (SHIPPED_MB if a.native == "shipped"
    ↪  else OPERATOR_MB)
    if a.setup:
        return setup()
    out = Path(a.out)
    if a.report_only:
        recs = [json.loads(f.read_text()) for f in sorted(Path(a.repor
    ↪ t_only).rglob("sysbench-*/*.json"))]
        print(report(recs, Path(a.report_only))); return 0
    names = list(WORKLOADS) if a.workloads == "all" else a.workloads.s
    ↪ plit(",")
    recs = [run_workload(n, out, a.reps, a.step_s, a.line_ms, a.object
    ↪ ive) for n in names]
    print(report(recs, out))
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

H.59 `tools/run_ycsb.py`

811 lines.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES

    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""Omni-Compass on top of a database's operator-set cache size: YCSB o
    ↪ n MongoDB (docs/YCSB_PREREGISTRATION.md).

MongoDB as its publisher ships it, with the one setting an operator se
    ↪ ts once for its storage engine's cache: the WiredTiger
cache size. YCSB (Yahoo's Cloud Serving Benchmark, the published core
    ↪ workloads) asks for records drawn from a key space that
widens and narrows over the run, so the working set steps through the
    ↪ operator's cache and past it. The operator's fixed

cache is the native controller here.

  native  MongoDB at the operator's cache size (512 MB) for the whole
    ↪ run
  omni    the same MongoDB, with the compass law (omnicompass/compass_
    ↪ law.py) on one knob, the cache size, through the

          server's own console (setParameter wiredTigerEngineRuntimeCo
    ↪ nfig cache_size), inside the cover [256 MB, 2,048 MB]:
          the compass reads the server's own mean read latency over th
    ↪ e last second (serverStatus opLatencies) on a band from
          0 to the response line and holds it at 40% of the line; slow
    ↪  reads with the cache full and missing push the cache up

          (slow reads with room to spare, or in a cache that holds its
    ↪  working set, are not the cache's to mend), calm gives
          memory back 64 MB a second while the cache's misses are unde
    ↪ r one percent of the pages requested from it (amendment
          1: it holds the working set; the first counted set read "not
    ↪ hing evicted", which a cold cache satisfies while it fills);
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
          at 95% of the line with the cache full a quarter of the cove
    ↪ r is added at once (fail up); the cache size is handed
          back to the operator's at the end and read back; a size foun
    ↪ d at a value Omni did not write stops it writing

Gauges from YCSB's own per-operation records (every operation's latenc

    ↪ y, raw), the server's own serverStatus (the cache
configured, the bytes in it, pages read into it) and the host's /proc/
    ↪ stat (CPU seconds, the compass's own cost included).
Memory held is the resource; no energy is claimed beyond the host's CP
    ↪ U seconds. Every row is reported, losses included.

  python3 tools/run_ycsb.py --setup                        # fetch and
    ↪  check YCSB, point the server at the operator's cache
  python3 tools/run_ycsb.py --workloads tuning --reps 3 --out DIR
  python3 tools/run_ycsb.py --report-only DIR              # re-read f
    ↪ inished <workload>.json files and write YCSB.md
"""

from __future__ import annotations

import argparse
import hashlib
import json
import math

import os
import subprocess
import sys
import threading
import time
import urllib.request

from pathlib import Path

try:                                                       # the legal
    ↪  notice every generated report carries
    from tools.legal import stamp as _legal_stamp
except ImportError:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    import pathlib as _p; sys.path.insert(0, str(_p.Path(__file__).res
    ↪ olve().parents[1])); from tools.legal import stamp as _legal_sta
    ↪ mp
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from omnicompass.compass_law import Band, CompassLaw  # noqa: E402

from tools.knob_verdict import KnobVerdict, sample_cost, OBJECTIVES, R
    ↪ ESOURCE as DEFAULT_OBJECTIVE  # noqa: E402  (amendment 2: the br
    ↪ ain's own verdict)
from tools.store_brain import MemoryMax, read_meminfo  # noqa: E402  (
    ↪ omni-v22: the max on the cache)
from tools.readings import word as gel_word, aim as gel_aim, EVEN_SAME

    ↪ , EVEN_NOISE, SHOWN  # noqa: E402  (the words every table uses)


URL = os.environ.get("MONGO_URL", "mongodb://127.0.0.1:27017")
DB = "ycsb"
MB = 1024 * 1024
LINE_MS = 1.0            # the response line: an operation answered w

    ↪ ithin 1 ms (set on the tuning workload's fourth smoke run, from
    ↪ 2 ms:
                         # on this stack a hit reads 0.15 to 0.24 ms
    ↪ and a miss 0.28 to 0.46 ms, so the 2 ms line kept every reading
    ↪ under the center)
CENTER = 0.4             # the compass holds the server's own read la

    ↪ tency at 40% of the line
DT = 1.0                 # one decision a second
TAU = 2.0                # memory given or taken shows in the misses
    ↪ within a couple of seconds
SMOOTH = 0.5
NATIVE_MB = 512          # the operator's cache (None with --native s

    ↪ hipped: the server's own default, half of RAM less 1 GB)
NATIVE = "operator"
COVER_MB = (256, 2048)   # never under 256 MB (the server's own floor
    ↪ ), never over 2 GB
STEP_MB = 64             # one notch of the knob
FAILUP_MB = 448          # past the wall: a quarter of the cover at o
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ nce, and again the next second if still past it
DWELL_S = 5.0             # after growing, no taking back within five
    ↪ seconds; giving back is a notch a second
DIAL = os.environ.get("OMNI_DIAL", "max")   # omni-v22: max, the facto
    ↪ ry tune (the Brain); save, the tune-down (omni-v21's loop)
FULL = 0.8                # the cache counts as full when the bytes in

    ↪  it are 80% of its size or more (WiredTiger's own eviction_targe
    ↪ t, which
                          # its eviction workers hold the cache at), o
    ↪ r when it evicted a page in the last second (store fix of 25 Oct
    ↪ ober:
                          # at 0.9 the cache read full in only 2 to 61

    ↪  of about 460 seconds of every omni-v16 arm, so the grow path ne
    ↪ ver opened)
RATE = 3000               # operations a second offered by YCSB (-targ
    ↪ et), the same in both arms
THREADS = 32              # YCSB's client threads
STEPS = "1 2 3 2 3 4 5 6 5 4 3 2 1 2 1"

BURST = "1 6 1 8 1 6"
RECORD_BYTES = 1000       # YCSB's default: 10 fields of 100 bytes
DISTRIBUTION = "uniform"  # over the notch's key space, so the working
    ↪  set is the key space (see run_arm)
YCSB_VERSION = "0.17.0"
YCSB_URL = f"https://github.com/brianfrankcooper/YCSB/releases/downloa

    ↪ d/{YCSB_VERSION}/ycsb-mongodb-binding-{YCSB_VERSION}.tar.gz"
YCSB_SHA256 = "6a054a706812269c80bfc6ed1e83457990c1c60b01f5083c873aaed
    ↪ 05577e30d"   # recorded from the release before any run
YCSB_HOME = Path(os.environ.get("YCSB_HOME", str(Path.home() / f"ycsb-
    ↪ mongodb-binding-{YCSB_VERSION}")))
from omnicompass.student_t import T95  # noqa: E402  (the two-sided 95

    ↪ % point of Student's t, any degrees of freedom)
SAME_REL = 1e-6

WORKLOADS = {
    # name: (YCSB core workload file, base record count (keys at notch
    ↪  1), steps, tuning?)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    #   250,000 records of 1 KB are about 275 MB in the cache at notch
    ↪  1; notch 6 is about 1.6 GB, past the operator's 512 MB
    "tuning": ("workloada", 250_000, STEPS, True),      # 50/50 read/u
    ↪ pdate, zipfian: the rules were set here
    "b": ("workloadb", 250_000, STEPS, False),          # 95/5 read/up
    ↪ date, zipfian

    "c": ("workloadc", 250_000, STEPS, False),          # 100% read, z
    ↪ ipfian
    "f": ("workloadf", 250_000, STEPS, False),          # read-modify-
    ↪ write, zipfian
    "burst": ("workloadb", 250_000, BURST, False),      # 95/5, the bu
    ↪ rst schedule

}

GAUGES = [  # key, label, direction
    ("work_inside_line_ops", "work inside the response line (operation
    ↪ s a second answered within the line)", "higher"),
    ("ops", "throughput (operations a second)", "higher"),

    ("p95_ms", "latency, 95th percentile (ms)", "lower"),
    ("p99_ms", "latency, 99th percentile (ms)", "lower"),
    ("mean_ms", "latency, mean (ms)", "lower"),
    ("failed", "failed operations", "never more"),
    ("cache_mb_mean", "cache size held, mean (MB; the knob, the resour
    ↪ ce)", "spent"),

    ("cache_used_mb_mean", "bytes in the cache, mean (MB)", "spent"),
    ("pages_read", "pages read into the cache (misses)", "shown"),
    ("cpu_busy_share", "host CPU busy (share of the run)", "spent"),
    ("cpu_seconds", "host CPU-seconds", "spent"),
    ("cpu_s_per_1k_inside", "host CPU-seconds per 1,000 operations ins
    ↪ ide the line", "spent"),

    ("writes", "cache size changes written (the knob's moves)", "shown
    ↪ "),
    # store fix of 25 October: the CPU apart, beside the host figure (
    ↪ old runs lack these rows; the readers prefer process_cpu_seconds
    ↪ )
    ("process_cpu_seconds", "the store's and the harness's own CPU-sec
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ onds (mongod plus the compass and its sampler; steal excluded)",
    ↪  "spent"),
    ("server_cpu_seconds", "the server's own CPU-seconds (mongod)", "s
    ↪ hown"),
    ("harness_cpu_seconds", "the harness's own CPU-seconds (the compas
    ↪ s's brain in the omni arm, the sampler in both)", "shown"),

    ("host_cpu_seconds_no_steal", "host CPU-seconds without steal (the
    ↪  load generator included)", "shown"),
    ("steal_seconds", "CPU-seconds the hypervisor took (steal), host-w
    ↪ ide", "shown"),
]
OPTIONAL_GAUGES = ("process_cpu_seconds", "server_cpu_seconds", "harne

    ↪ ss_cpu_seconds", "host_cpu_seconds_no_steal", "steal_seconds")
    ↪ # absent from runs before 25 October
# the serverStatus sections the brain and the sampler read (store fix
    ↪ of 25 October: every other section is asked away, so the
# one-a-second reading no longer has the server build, and Python deco
    ↪ de, the whole document)

STATUS_SECTIONS = ("wiredTiger", "opLatencies")


def lean_status_command(full_doc, keep=STATUS_SECTIONS):
    """The serverStatus command that asks only for the sections kept:
    ↪ every other section the server reported in a full

    document (a sub-document at the top level) is asked away with 0, s
    ↪ o only names the server itself gave are sent."""
    cmd = {"serverStatus": 1}
    for k, v in full_doc.items():
        if isinstance(v, dict) and k not in keep:
            cmd[k] = 0

    return cmd


class LeanStatus:
    """serverStatus once in full, then only the sections the brain rea
    ↪ ds (STATUS_SECTIONS)."""
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def __init__(self, client):
        self.c, self.cmd = client, None

    def __call__(self):
        if self.cmd is None:
            full = self.c.admin.command("serverStatus")
            self.cmd = lean_status_command(full)
            return full
        return self.c.admin.command(dict(self.cmd))

class CacheSize:
    """The plug on the server's cache size: read once before the first
    ↪  write, write through the server's own console, read
    back, yield to any other writer, restore."""

    def __init__(self, client):
        self.c, self.snapshot, self.last_written = client, None, None
        self.status = LeanStatus(client)

    def _status(self):
        return self.status()

    def configured_mb(self, st=None):
        st = st or self._status()
        return int(st["wiredTiger"]["cache"]["maximum bytes configured
    ↪ "]) // MB

    def attach(self):
        self.snapshot = self.configured_mb(); self.last_written = self
    ↪ .snapshot
        return self.snapshot

    def lever(self, st=None):
        cur = self.configured_mb(st)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        if self.last_written is not None and cur != self.last_written:
            raise RuntimeError(f"another writer set the cache to {cur}
    ↪  MB (we wrote {self.last_written}): Omni stops writing")
        return cur

    def write(self, mb):

        self.c.admin.command({"setParameter": 1, "wiredTigerEngineRunt
    ↪ imeConfig": f"cache_size={int(mb)}M"})
        back = self.configured_mb(); self.last_written = back
        return back

    def restore(self):

        try:
            self.c.admin.command({"setParameter": 1, "wiredTigerEngine
    ↪ RuntimeConfig": f"cache_size={int(self.snapshot)}M"})
            return self.configured_mb() == self.snapshot
        except Exception:
            return False

def cache_stats(st):
    """(configured MB, bytes in cache MB, pages read into cache, pages
    ↪  evicted, pages requested from the cache) from one serverStatus.
    ↪ """

    c = st["wiredTiger"]["cache"]
    evicted = sum(int(c.get(k, 0)) for k in ("unmodified pages evicted
    ↪ ", "modified pages evicted", "pages evicted by application threa
    ↪ ds"))
    return (int(c["maximum bytes configured"]) // MB, int(c["bytes cur
    ↪ rently in the cache"]) / MB, int(c.get("pages read into cache",

    ↪ 0)), evicted,
            int(c.get("pages requested from the cache", 0)))


def miss_share(pages_read_last_s, requested_last_s):
    """The cache's misses as a share of the pages requested from it in
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  the last second; no request reads as no miss."""
    return pages_read_last_s / requested_last_s if requested_last_s >
    ↪ 0 else 0.0


# omni-v22: the server's own account of what a cache miss costs: Wired
    ↪ Tiger's time reading pages from disk for the application's
# threads (the reads that stood in a request's way; the server's own e
    ↪ viction and read-ahead threads are not counted), in
# microseconds, beside the server's own read latency (opLatencies, mic
    ↪ roseconds).
MISS_TIME_KEY = "application threads page read from disk to cache time
    ↪  (usecs)"
MISS_COUNT_KEY = "application threads page read from disk to cache cou
    ↪ nt"
REQUIRED_CACHE_KEYS = ("maximum bytes configured", "bytes currently in
    ↪  the cache", "pages read into cache", "pages requested from the
    ↪ cache",
                       MISS_TIME_KEY, MISS_COUNT_KEY)


def miss_time(st):
    """(the application threads' page reads from disk, their time in m
    ↪ icroseconds) from one serverStatus; None where the server
    gives no account of them (the attach check stops a run on such a s
    ↪ erver before it starts)."""
    c = st["wiredTiger"]["cache"]
    if MISS_TIME_KEY not in c:
        return None, None
    return int(c.get(MISS_COUNT_KEY, 0)), int(c[MISS_TIME_KEY])

def read_latency(st):
    """(total read latency in microseconds, read operations) from one
    ↪ serverStatus (opLatencies)."""
    r = st["opLatencies"]["reads"]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return int(r["latency"]), int(r["ops"])


GIVEBACK_MISS_SHARE = 0.01  # amendment 1: a notch is given back only
    ↪ while the cache's misses (pages read into it) are under 1% of th
    ↪ e pages
                            # requested from it in the last second (th
    ↪ e cache holds the working set). The first counted set read "no p
    ↪ age
                            # evicted in the last second", which a col
    ↪ d cache satisfies while it fills: every give-back of every burst
    ↪  arm came
                            # in the first seconds, and the cache then
    ↪  sat at the floor of the cover, evicting, for the rest of the ar
    ↪ m.
GROW_MISS_SHARE = GIVEBACK_MISS_SHARE  # and the cache grows only whil
    ↪ e missing (1% or more): a full cache that holds its working set
    ↪ cannot mend a slow read

def cache_full(used_mb, cur_mb, evicted_last_s=0):
    """The cache is full when the bytes in it reach WiredTiger's own e
    ↪ viction target (FULL of its size) or it evicted a page in
    the last second (store fix of 25 October)."""

    return used_mb >= FULL * cur_mb or (evicted_last_s or 0) > 0


def decide(p, force, miss_share_last_s, cur_mb, last_change_age, full,
    ↪  wall=0.95, reads=True, native_mb=None):
    """The knob's move this second, in MB: slow reads (the service slo

    ↪ w) grow the cache by ceil(force / 0.1) notches, but only
    while the cache is full (a slow read with room to spare is not the
    ↪  cache's to mend) and missing (amendment 1: misses at one
    percent of its requests or more); calm with the cache's misses und
    ↪ er one percent of its requests (it holds the working set)
    gives back one notch after the dwell, and only in a second that re
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ quested pages (a second with none says nothing about the
    working set); past the wall, with the cache full, a quarter of the
    ↪  cover at once.
    Store fix of 25 October, the queue line of the database wire (docs
    ↪ /POSTGRES_PREREGISTRATION.md, amendment 2) for the cache:
    below the operator's own size (native_mb), a second that requested

    ↪  pages with misses at one percent of its requests or more
    adds one notch back, up to the operator's size, with no slow or fu
    ↪ ll condition (a size below native's that misses no longer
    holds the working set; the omni-v16 runs held the cache at 448 MB
    ↪ to the end of two arms with misses at 9 to 13%)."""
    target, why = _decide(p, force, miss_share_last_s, cur_mb, last_ch

    ↪ ange_age, full, wall, reads)
    if native_mb is not None and cur_mb < native_mb and reads and miss
    ↪ _share_last_s >= GROW_MISS_SHARE:
        back = min(native_mb, cur_mb + STEP_MB)
        if back > target:
            return back, f"missing below the operator's size ({100 * m

    ↪ iss_share_last_s:.1f}% of requests): a notch added back, up to t
    ↪ he operator's own size"
    return target, why


def _decide(p, force, miss_share_last_s, cur_mb, last_change_age, full

    ↪ , wall=0.95, reads=True):
    """The rule before the store fix (the slow, calm and wall moves);
    ↪ decide() adds the hand-back below native."""
    lo, hi = COVER_MB
    if p is not None and p >= wall and full:
        return min(hi, cur_mb + FAILUP_MB), "fail up: the line is at h

    ↪ and and the cache is full, a quarter of the cover at once"
    if force > 0.05:
        if not full:
            return cur_mb, "slow with room to spare: not the cache's t
    ↪ o mend"
        if miss_share_last_s < GROW_MISS_SHARE:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            return cur_mb, "slow with the cache full but holding its w
    ↪ orking set (misses under 1% of requests): not the cache's to men
    ↪ d"
        return min(hi, cur_mb + STEP_MB * max(1, math.ceil(force / 0.1
    ↪ ))), "slow with the cache full and missing: grow"
    if force < -0.05 and not reads:

        return cur_mb, "calm, but no page requested in the last second
    ↪ : nothing known, nothing taken"
    if force < -0.05 and miss_share_last_s < GIVEBACK_MISS_SHARE and c
    ↪ ur_mb > lo and last_change_age >= DWELL_S:
        return max(lo, cur_mb - STEP_MB), "calm, the cache holding the
    ↪  working set (misses under 1% of requests): a notch given back"

    return cur_mb, "hold"


# omni-v22: the max on the WiredTiger cache (docs/YCSB_PREREGISTRATION
    ↪ .md, amendment 5; docs/STORE_ROOT_CAUSES_2026-10-30.md): the
# wire's own rule (decide) with the spend never clamped by an allowanc

    ↪ e where the server's own account says the misses cost the
# reads (WiredTiger's time reading pages from disk for the application
    ↪ 's threads against the server's own read latency:
# tools/store_brain.py), held where they cost nothing (on a runner who
    ↪ se page cache holds the data files a cache miss is a read from
# memory: 12% to 16% of pages missing with the read latency unmoved),

    ↪ held while the host has no notch to spare, and no give-back
# below the working set's floor nor below the operator's cache. `--dia
    ↪ l save` keeps omni-v21's loop, the verdict under the resource
# objective, where giving memory back is the aim.
METER = "WiredTiger's time reading pages from disk for the application
    ↪ 's threads against the server's own read latency"

class Brain:
    """The knob's decision, one a second: obs holds reading_s (the mea
    ↪ n read latency), reads (operations this second), miss_share,
    cache_mb, used_mb, full, requested (any page requested this second
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ), cpu_share, miss_time_s (the application threads' time
    reading pages from disk this second), service_time_s (the reads' o
    ↪ wn latency summed this second), host_avail_mb and
    host_total_mb (the host's memory, None unread)."""

    def __init__(self, native, objective=DEFAULT_OBJECTIVE, line_ms=LI
    ↪ NE_MS):
        self.native, self.objective = native, objective
        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE
    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)
        self.max = MemoryMax(native, STEP_MB, COVER_MB, FULL, METER)
        self.last_change = -1e9

    @property
    def failups(self):
        return self.max.failups

    def record(self):
        return self.max.record(self.objective)

    def step(self, t, obs):
        reading, cur = obs["reading_s"], obs["cache_mb"]
        f = self.law.force(reading)
        want, why = decide(self.law.p, f, obs["miss_share"], cur, t -
    ↪ self.last_change, obs["full"], wall=self.law.band.wall_high,
                           reads=obs.get("requested", True), native_mb
    ↪ =self.native)
        target, why, info = self.max.step(t, want, why, cur, obs["used
    ↪ _mb"], obs.get("miss_time_s"), obs.get("service_time_s"),
                                          obs.get("host_avail_mb"), ob
    ↪ s.get("host_total_mb"), fail_up=why.startswith("fail up"))
        info.update({"p": round(self.law.p, 4), "force": round(f, 4)})
        return target, why, info


class Omni(threading.Thread):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    """The brain, one decision a second, reading the server's own read
    ↪  latency, writing the audit; the knob is handed back at
    the end and read back. Amendment 2 (the brain's own verdict, 2026-
    ↪ 10-09): the cache starts in watch and is written only
    inside the allowance a paired trial on the server itself has earne
    ↪ d under the declared objective (tools/knob_verdict.py

    around the engine's own Verdict), one notch a trial; a trial holds
    ↪  the cache; a fail-up never spends beyond the allowance."""

    def __init__(self, plug: CacheSize, audit_path: Path, line_ms=LINE
    ↪ _MS, objective=DEFAULT_OBJECTIVE, dial=DIAL):
        super().__init__(daemon=True)

        self.plug, self.audit, self.objective = plug, audit_path, obje
    ↪ ctive
        self.law = CompassLaw(Band(0.0, line_ms / 1000.0, center=CENTE
    ↪ R), dt=DT, tau=TAU, smooth=SMOOTH)
        self.stop_flag = threading.Event(); self.last_change = -1e9; s
    ↪ elf.writes = 0; self.failups = 0

        self.foreign = False; self.handed_back = False; self.error = N
    ↪ one; self.verdict = None; self.brain = None; self.dial = dial; s
    ↪ elf.line_ms = line_ms

    def govern_record(self):
        """The record of what governed the knob: the max's counts (omn

    ↪ i-v22) or the verdict's allowance (the tune-down)."""
        if self.brain is not None:
            return self.brain.record()
        return self.verdict.record() if self.verdict else None

    def run(self):

        try:
            native = self.plug.attach(); st = self.plug._status(); lat
    ↪ 0, ops0 = read_latency(st); _, _, rd0, ev0, rq0 = cache_stats(st
    ↪ )
            mr0, mt0 = miss_time(st)
            if native > COVER_MB[1]:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            # MongoDB as shipped (its default, half of RAM less 1
↪ GB) holds more cache than the most Omni may set: the knob
            # is at its limit on native, and any size Omni wrote w
↪ ould sit below native (the max never goes below native)
            self.audit.write_text(json.dumps({"t": 0.0, "cache_mb"
↪ : native, "why": "native's cache is above the cover: at its "
                                                "limit on native, no
↪ thing written"}) + "\n")
            self.stop_flag.wait()
            self.handed_back = self.plug.restore()
            return
    except Exception as e:                                      #
↪ the brain never ran: say so in the audit and the record, never s
↪ ilently
        self.error = repr(e)
        self.audit.write_text(json.dumps({"t": 0.0, "error": self.
↪ error, "note": "the brain could not attach; no decision was made
↪ "}) + "\n")
        return
    t0 = time.monotonic(); cpu_prev = cpu_times()
    self.verdict = KnobVerdict(native, STEP_MB, COVER_MB, objectiv
↪ e=self.objective, settle_s=TAU)
    if self.dial == "max":
        self.brain = Brain(native, self.objective, self.line_ms);
↪ self.verdict = None
    with open(self.audit, "w") as fh:
        while not self.stop_flag.is_set():
            tick = time.monotonic()
            try:
                st = self.plug._status()
                lat, ops = read_latency(st); dl, do = lat - lat0,
↪ ops - ops0; lat0, ops0 = lat, ops
                mr, mt = miss_time(st)
                mr_last, mt_last = (None, None) if mt is None else
↪  (mr - mr0, mt - mt0); mr0, mt0 = mr, mt
                reading = (dl / do / 1e6) if do > 0 else 0.0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪      # the mean read latency of the last second, in seconds; no
    ↪ reads reads calm
                    f = self.law.force(reading)
                    cur, used, rd, ev, rq = cache_stats(st); ev_last =
    ↪  ev - ev0; ev0 = ev
                    rd_last = rd - rd0; rd0 = rd; rq_last = rq - rq0;
    ↪ rq0 = rq
                    share = miss_share(rd_last, rq_last)
                    cur = self.plug.lever(st); full = cache_full(used,
    ↪  cur, ev_last)
                    cpu_now = cpu_times(); d_total = cpu_now[0] - cpu_
    ↪ prev[0]; d_idle = cpu_now[1] - cpu_prev[1]; cpu_prev = cpu_now
                    cpu_share = (1.0 - d_idle / d_total) if d_total >
    ↪ 0 else None
                    cost = sample_cost(self.objective, do, reading if
    ↪ do > 0 else None, cur, cpu_share)
                    if self.brain is not None:
    ↪                    # the max (omni-v22)
                        h_avail, h_total = read_meminfo()
                        obs = {"reading_s": reading, "reads": do, "mis
    ↪ s_share": share, "cache_mb": cur, "used_mb": used, "full": full,
                               "requested": rq_last > 0, "cpu_share":
    ↪ cpu_share,
                               "miss_time_s": None if mt_last is None
    ↪ else mt_last / 1e6, "service_time_s": dl / 1e6,
                               "host_avail_mb": h_avail, "host_total_m
    ↪ b": h_total}
                        self.brain.last_change = self.last_change - t0
                        target, why, vinfo = self.brain.step(tick - t0
    ↪ , obs)
                        vinfo["disk_reads_last_s"] = mr_last
                        self.failups = self.brain.failups
                    else:
    ↪                    # the tune-down: omni-v21's loop
                        self.verdict.observe(cost, tick - t0)
    ↪                       # measured under the cache of the last sec
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ond
                        target, why = decide(self.law.p, f, share, cur
    ↪ , tick - self.last_change, full, wall=self.law.band.wall_high, r
    ↪ eads=rq_last > 0,
                                                native_mb=native)
                        missing_below = cur < native and rq_last > 0 a
    ↪ nd share >= GROW_MISS_SHARE     # the size below native no longe
    ↪ r holds the working set
                        target, why, vinfo = self.verdict.decide(
                            target, spend_ok=(f > 0.05 and full and sh
    ↪ are >= GROW_MISS_SHARE),
                            give_ok=(f < -0.05 and rq_last > 0 and sha
    ↪ re < GIVEBACK_MISS_SHARE),
                            t=tick - t0, why=why, stress=(f > 0.05 or
    ↪ missing_below), calm=f < -0.05, fail_up=why.startswith("fail up"
    ↪ ))
                    wrote = None
                    if target != cur:
                        wrote = self.plug.write(target); self.writes +
    ↪ = 1
                        if target > cur:
                            self.last_change = tick
                        if why.startswith("fail up") and self.brain is
    ↪  None:
                            self.failups += 1
                    fh.write(json.dumps({"t": round(tick - t0, 2), "re
    ↪ ading_ms": round(reading * 1000, 3), "reads": do, "p": round(sel
    ↪ f.law.p, 4), "force": round(f, 4),
                                         "evicted_last_s": ev_last, "p
    ↪ ages_read_last_s": rd_last, "pages_requested_last_s": rq_last, "
    ↪ miss_share": round(share, 4),
                                         "cache_mb": cur, "used_mb": r
    ↪ ound(used, 1), "full": full, "target_mb": target, "wrote_mb": wr
    ↪ ote,
                                         "cpu_share": None if cpu_shar
    ↪ e is None else round(cpu_share, 4), "cost": None if cost is None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  else round(cost, 9),
                                          **vinfo, "why": why}) + "\n")
                except Exception as e:
                    fh.write(json.dumps({"t": round(tick - t0, 2), "er
    ↪ ror": repr(e)}) + "\n")
                    self.foreign = True
                    break
                time.sleep(max(0.0, DT - (time.monotonic() - tick)))
        self.handed_back = self.plug.restore()


class Sampler(threading.Thread):
    """The cache configured, the bytes in it and the pages read into i
    ↪ t, once a second, in both arms."""

    def __init__(self, client):
        super().__init__(daemon=True)
        self.c, self.stop_flag, self.rows = client, threading.Event(),
    ↪  []
        self.status = LeanStatus(client)

    def run(self):
        while not self.stop_flag.is_set():
            try:
                self.rows.append(cache_stats(self.status()))
            except Exception:
                pass
            time.sleep(DT)

def cpu_times():
    with open("/proc/stat") as fh:
        f = fh.readline().split()
    vals = list(map(int, f[1:]))
    return sum(vals), vals[3] + vals[4]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# --- the CPU apart (store fix of 25 October) ------------------------
    ↪ ------------------------------------------------
# The host figure above (kept, so every older table reads the same) co
    ↪ unts every field but idle and iowait as busy: steal (the
# hypervisor's), and guest twice. Beside it: the host's busy time with
    ↪ out steal, the steal itself, the server's own CPU from
# its /proc/<pid>/stat and the harness's own from getrusage, so the st
    ↪ ore's work and the compass's cost are read apart from
# the load generator, the runner's own processes and the machine's nei
    ↪ ghbours.
def host_cpu_split(line=None):
    """(busy jiffies without steal or guest, steal jiffies) from /proc
    ↪ /stat's cpu line (or the line given)."""
    if line is None:
        with open("/proc/stat") as fh:
            line = fh.readline()
    v = [int(x) for x in line.split()[1:]] + [0] * 10
    user, nice, system, idle, iowait, irq, softirq, steal = v[:8]
    return user + nice + system + irq + softirq, steal        # guest
    ↪ and guest_nice are already inside user and nice


def pid_of(name):
    """The oldest process whose command name is `name` (its /proc/<pid
    ↪ >/comm), or None."""
    best = None
    for d in Path("/proc").iterdir():
        if d.name.isdigit():
            try:
                if (d / "comm").read_text().strip() == name:
                    pid = int(d.name)
                    best = pid if best is None or pid < best else best
            except OSError:
                continue
    return best
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def proc_cpu_jiffies(pid, stat_text=None):
    """utime + stime (jiffies) of one process from /proc/<pid>/stat (o
    ↪ r the text given); None when it is gone."""
    try:
        txt = stat_text if stat_text is not None else Path(f"/proc/{pi
    ↪ d}/stat").read_text()
    except (OSError, TypeError):
        return None
    rest = txt[txt.rindex(")") + 2:].split()                    # afte
    ↪ r the command name, which may hold spaces
    return int(rest[11]) + int(rest[12])                        # fiel
    ↪ ds 14 and 15 of the stat line


class CpuApart:
    """The CPU of one arm apart: start() before the load, stop() after
    ↪ ; figures in seconds."""

    def __init__(self, server_name):
        self.server_name = server_name

    def _read(self):
        import resource
        pid = pid_of(self.server_name)
        own = resource.getrusage(resource.RUSAGE_SELF)
        busy, steal = host_cpu_split()
        return {"pid": pid, "server": proc_cpu_jiffies(pid) if pid els
    ↪ e None, "harness": own.ru_utime + own.ru_stime, "busy": busy, "s
    ↪ teal": steal}

    def start(self):
        self.a = self._read()
        return self
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    def stop(self):
        b, a, tck = self._read(), self.a, os.sysconf("SC_CLK_TCK")
        server = (b["server"] - a["server"]) / tck if (a["server"] is
    ↪ not None and b["server"] is not None and a["pid"] == b["pid"]) e
    ↪ lse None
        harness = b["harness"] - a["harness"]

        return {"server_cpu_seconds": server, "harness_cpu_seconds": h
    ↪ arness,
                "process_cpu_seconds": (server + harness) if server is
    ↪  not None else None,
                "host_cpu_seconds_no_steal": (b["busy"] - a["busy"]) /
    ↪  tck, "steal_seconds": (b["steal"] - a["steal"]) / tck}

def parse_raw(path: Path):
    """YCSB's raw measurements: one line per operation, 'op, timestamp
    ↪ (ms), latency(us)'; returns [(t_s, latency_ms)]."""
    out = []

    if not path.exists():
        return out
    for line in path.read_text().splitlines():
        parts = [x.strip() for x in line.split(",")]
        if len(parts) < 3 or not parts[2].lstrip("-").isdigit():
            continue

        try:
            out.append((float(parts[1]) / 1000.0, float(parts[2]) / 10
    ↪ 00.0))
        except ValueError:
            continue
    return out

def parse_failed(summary_text):
    """Failed operations from YCSB's summary: every '[<OP>-FAILED], Op
    ↪ erations, n' line."""
    n = 0
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    for line in summary_text.splitlines():
        p = [x.strip() for x in line.split(",")]
        if len(p) >= 3 and p[0].endswith("-FAILED]") and p[1] == "Oper
    ↪ ations":
            try:
                n += int(p[2])
            except ValueError:
                pass
    return n


def ycsb(cmd, workload_file, props, out_file: Path):
    """One YCSB invocation (load or run) with raw per-operation measur
    ↪ ements written to out_file; returns its summary text."""
    out_file = Path(out_file).resolve(); out_file.parent.mkdir(parents
    ↪ =True, exist_ok=True)   # YCSB runs from its own home: absolute
    ↪ paths
    args = ["bash", str(YCSB_HOME / "bin" / "ycsb.sh"), cmd, "mongodb"
    ↪ , "-s", "-P", str(YCSB_HOME / "workloads" / workload_file),
            "-p", f"mongodb.url={URL}/{DB}?w=1", "-p", "measurementtyp
    ↪ e=raw", "-p", f"measurement.raw.output_file={out_file}"]
    for k, v in props.items():
        args += ["-p", f"{k}={v}"]
    r = subprocess.run(args, capture_output=True, text=True, cwd=str(Y
    ↪ CSB_HOME))
    return r.stdout + "\n" + r.stderr


def load_dataset(workload_file, records, log: Path):
    """The dataset for a workload, loaded once per workload (both arms
    ↪  read the same records); the collection is dropped first."""
    import pymongo
    c = pymongo.MongoClient(URL); c.drop_database(DB); c.close()
    txt = ycsb("load", workload_file, {"recordcount": records, "thread
    ↪ s": THREADS, "insertorder": "ordered"}, log.with_suffix(".raw"))
    log.write_text(txt)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if "[OVERALL]" not in txt:
        raise RuntimeError(f"YCSB load failed: {txt[-2000:]}")


def server_journal(unit):
    """The server's own journal and status, for the record when a rest
    ↪ art does not come back (20 October: the omni-v12
    MongoDB runs B and C died at this wait with an empty mongod.log an
    ↪ d nothing else recorded)."""
    out = []
    for cmd in (["sudo", "systemctl", "status", unit, "--no-pager", "-
    ↪ l"], ["sudo", "journalctl", "-u", unit, "--no-pager", "-n", "80"
    ↪ ]):
        r = subprocess.run(cmd, capture_output=True, text=True)
        out.append(" ".join(cmd) + "\n" + (r.stdout + r.stderr)[-6000:
    ↪ ])
    return "\n".join(out)

def fresh_server(cache_mb):
    """The cache emptied and the operator's size restored before an ar
    ↪ m, both arms alike: the server restarted at the
    operator's configured size, the OS page cache dropped. A server th
    ↪ at has not answered five minutes after its restart is

    restarted once more, with its journal printed; one that has not an
    ↪ swered after that stops the run with the journal in the
    error, so the cause is in the record (never a bare "did not come b
    ↪ ack")."""
    import pymongo
    for attempt in (1, 2):

        subprocess.run(["sudo", "systemctl", "restart", "mongod"], che
    ↪ ck=True)
        subprocess.run(["sudo", "sh", "-c", "sync; echo 3 > /proc/sys/
    ↪ vm/drop_caches"], check=False)
        for _ in range(100):
    ↪  # about five minutes: 2 s to select a server, 1 s between tries
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            try:
                c = pymongo.MongoClient(URL, serverSelectionTimeoutMS=
    ↪ 2000); st = c.admin.command("serverStatus")
                got = int(st["wiredTiger"]["cache"]["maximum bytes con
    ↪ figured"]) // MB
                if cache_mb is not None and got != cache_mb:

    ↪        # None: MongoDB's own default, as shipped
                    c.admin.command({"setParameter": 1, "wiredTigerEng
    ↪ ineRuntimeConfig": f"cache_size={cache_mb}M"})
                return c
            except Exception:
                time.sleep(1)

        journal = server_journal("mongod")
        print(f"mongod did not answer five minutes after its restart (
    ↪ attempt {attempt} of 2); its status and journal:\n{journal}", fl
    ↪ ush=True)
    raise RuntimeError("mongod did not come back after two restarts; i
    ↪ ts status and journal are printed above")

def run_arm(arm, wl_dir: Path, rep, workload_file, base, steps, step_s
    ↪ , line_ms, seed, objective=DEFAULT_OBJECTIVE):
    import pymongo
    d = wl_dir / f"rep-{rep}" / arm; d.mkdir(parents=True, exist_ok=Tr

    ↪ ue)
    c = fresh_server(NATIVE_MB)
    native_now = int(c.admin.command("serverStatus")["wiredTiger"]["ca
    ↪ che"]["maximum bytes configured"]) // MB   # native's size, read
    sampler = Sampler(pymongo.MongoClient(URL)); sampler.start()
    omni = None

    if arm == "omni":
        omni = Omni(CacheSize(pymongo.MongoClient(URL)), d / "audit.js
    ↪ onl", line_ms, objective); omni.start()
    records, failed, summaries = [], 0, []
    apart = CpuApart("mongod").start()
    cpu0 = cpu_times(); t0 = time.time()
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    for i, notch in enumerate(int(s) for s in steps.split()):
        raw = d / f"notch-{i + 1}.raw"
        # the notch's key space: the first base x notch records of the
    ↪  loaded dataset, drawn uniformly, so the working set IS the
        # key space (YCSB's shipped zipfian keeps the hot set in a few
    ↪  megabytes whatever the key space: the smoke run showed the

        # cache never filling); the workload's read/update mix stays a
    ↪ s shipped
        txt = ycsb("run", workload_file, {"recordcount": base * notch,
    ↪  "operationcount": 10 ** 9, "maxexecutiontime": int(step_s), "ta
    ↪ rget": RATE,
                                          "threads": THREADS, "inserts

    ↪ tart": 0, "insertorder": "ordered", "requestdistribution": DISTR
    ↪ IBUTION,
                                          "seed": seed * 100 + i}, raw
    ↪ )      # ordered keys, as loaded: a hashed key space would read
    ↪ records that are not there
        got = parse_raw(raw); summaries.append(txt); failed += parse_f

    ↪ ailed(txt); records += got
        lat = sorted(l for _, l in got); row = sampler.rows[-1] if sam
    ↪ pler.rows else (native_now, 0.0, 0, 0, 0)
        print(f"   {arm} rep {rep} notch {notch}: {len(lat)} ops, mean
    ↪  {sum(lat) / max(1, len(lat)):.3f} ms, p95 {(lat[int(0.95 * len(
    ↪ lat))] if lat else float('nan')):.3f} ms, "

              f"inside the line {100 * sum(1 for l in lat if l <= line
    ↪ _ms) / max(1, len(lat)):.1f}%, cache {row[0]:.0f} MB, in cache {
    ↪ row[1]:.0f} MB", flush=True)
    t1 = time.time(); cpu1 = cpu_times(); cpu_apart = apart.stop()
    if omni:
        omni.stop_flag.set(); omni.join(timeout=30)

    sampler.stop_flag.set(); sampler.join(timeout=5)
    (d / "ycsb.log").write_text("\n\n".join(summaries))
    seconds = t1 - t0
    lat_sorted = sorted(l for _, l in records); n = len(lat_sorted)
    def q(p):
        return lat_sorted[min(n - 1, int(p * n))] if n else float("nan
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ")
    inside = sum(1 for l in lat_sorted if l <= line_ms)
    rows = sampler.rows or [(native_now, 0.0, 0, 0)]
    total = cpu1[0] - cpu0[0]; idle = cpu1[1] - cpu0[1]; cpu_s = (tota
    ↪ l - idle) / os.sysconf("SC_CLK_TCK")
    g = {"arm": arm, "rep": rep, "seconds": round(seconds, 1), "operat
    ↪ ions": n,
         "work_inside_line_ops": inside / seconds, "ops": n / seconds,
    ↪  "p95_ms": q(0.95), "p99_ms": q(0.99),
         "mean_ms": (sum(lat_sorted) / n) if n else float("nan"), "fai
    ↪ led": failed,
         "cache_mb_mean": sum(x[0] for x in rows) / len(rows), "cache_
    ↪ used_mb_mean": sum(x[1] for x in rows) / len(rows),
         "pages_read": rows[-1][2] - rows[0][2], "cpu_busy_share": (to
    ↪ tal - idle) / max(1, total), "cpu_seconds": cpu_s,
         "cpu_s_per_1k_inside": 1000.0 * cpu_s / max(1, inside), "writ
    ↪ es": (omni.writes if omni else 0),
         "handed_back": (omni.handed_back if omni else True), "foreign
    ↪ _writer": (omni.foreign if omni else False), "failups": (omni.fa
    ↪ ilups if omni else 0),
         "controller_error": (omni.error if omni else None),
         **cpu_apart,
         "verdict": (omni.govern_record() if omni else None)}
    (d / "arm.json").write_text(json.dumps(g, indent=1))
    print(f"   {arm} rep {rep}: {n} operations, inside the line {g['wo
    ↪ rk_inside_line_ops']:.0f}/s, p95 {g['p95_ms']:.2f} ms, cache {g[
    ↪ 'cache_mb_mean']:.0f} MB, "
          f"in cache {g['cache_used_mb_mean']:.0f} MB, pages read {g['
    ↪ pages_read']:,}, CPU {cpu_s:.1f} s, handed back {g['handed_back'
    ↪ ]}"
          + (f", verdict {g['verdict']['state']}" if g.get("verdict")
    ↪ else ""), flush=True)
    return g


def paired(reps, key, direction):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    """omni against native on one gauge over the repetitions: means, t
    ↪ he mean paired difference, its 95% interval."""
    d = [r["omni"][key] - r["native"][key] for r in reps if _num(r["om
    ↪ ni"].get(key)) and _num(r["native"].get(key))]
    if not d:
        return None

    n = len(d)
    nat = sum(r["native"][key] for r in reps) / n
    om = sum(r["omni"][key] for r in reps) / n
    mean = sum(d) / n
    if n > 1:
        sd = math.sqrt(sum((x - mean) ** 2 for x in d) / (n - 1))

        half = T95.get(n - 1, 1.96) * sd / math.sqrt(n)
    else:
        half = float("nan")
    lo, hi = mean - half, mean + half
    same = abs(mean) <= SAME_REL * max(abs(nat), 1e-12)
    clear = n > 1 and (lo > 0 or hi < 0) and not same

    if direction == "shown":
        reading = SHOWN
    elif direction == "never more" and any(x > 0 for x in d):
        reading = "**LOSS in a repetition**"
    elif same:
        reading = EVEN_SAME

    elif not clear:
        reading = EVEN_NOISE
    else:
        reading = gel_word(mean < 0, gel_aim(direction, key))      # t
    ↪ he word by what the gauge is for, never by its sign
    return {"native": nat, "omni": om, "diff": mean, "ci95": [lo, hi],

    ↪  "n": n, "significant": clear, "reading": reading}


def _num(x):
    return isinstance(x, (int, float)) and not (isinstance(x, float) a
    ↪ nd math.isnan(x))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
def engine():
    try:
        out = subprocess.run([sys.executable, str(ROOT / "tools" / "om
    ↪ ni_version.py")], cwd=ROOT, capture_output=True, text=True, time
    ↪ out=120).stdout.strip()
        sha = subprocess.run(["git", "rev-parse", "HEAD"], cwd=ROOT, c
    ↪ apture_output=True, text=True).stdout.strip()
        return {"version": out.splitlines()[0] if out else "unknown",
    ↪ "commit": sha[:12]}
    except (OSError, subprocess.SubprocessError):
        return {"version": "unknown", "commit": "?"}


def server_version():
    try:
        import pymongo
        return pymongo.MongoClient(URL).server_info().get("version", "
    ↪ ?")
    except Exception:
        return "?"

def run_workload(name, out: Path, reps, step_s, line_ms, objective=DEF
    ↪ AULT_OBJECTIVE):
    workload_file, base, steps, tuning = WORKLOADS[name]
    wl_dir = out / f"ycsb-{name}"; wl_dir.mkdir(parents=True, exist_ok
    ↪ =True)
    ver = server_version()
    top = max(int(s) for s in steps.split())
    print(f"== {name}: YCSB {workload_file}, {base:,} records x notch
    ↪ (loaded to notch {top}: {base * top:,}), steps {steps}, {step_s}
    ↪  s a notch, MongoDB {ver}, "
          f"the verdict's objective {objective}", flush=True)
    c0 = fresh_server(NATIVE_MB)
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    have = c0.admin.command("serverStatus")["wiredTiger"]["cache"]
    for key in REQUIRED_CACHE_KEYS:
        if key not in have:                                          # t
    ↪ he brain reads these; a server that lacks one must stop the run,
    ↪  not run native in Omni's name
            raise SystemExit(f"serverStatus().wiredTiger.cache has no

    ↪ '{key}' on this server: the brain's account of a miss cannot be
    ↪ read")
    load_dataset(workload_file, base * top, wl_dir / "load.log")
    rec = {"workload": name, "tuning": tuning, "ycsb_workload": worklo
    ↪ ad_file, "base_records": base, "steps": steps, "step_s": step_s,
           "line_ms": line_ms, "rate_ops": RATE, "threads": THREADS, "

    ↪ record_bytes": RECORD_BYTES, "native": NATIVE,
           "native_mb": NATIVE_MB if NATIVE_MB is not None else int(ha
    ↪ ve["maximum bytes configured"]) // MB, "cover_mb": list(COVER_MB
    ↪ ),
           "mongodb_version": ver, "ycsb_version": YCSB_VERSION, "ycsb
    ↪ _sha256": YCSB_SHA256, "objective": objective, "dial": DIAL, "en

    ↪ gine": engine(), "reps": []}
    for rep in range(1, reps + 1):
        order = ("native", "omni") if rep % 2 else ("omni", "native")
        r = {}
        for arm in order:
            r[arm] = run_arm(arm, wl_dir, rep, workload_file, base, st

    ↪ eps, step_s, line_ms, seed=1000 + rep, objective=objective)
        rec["reps"].append(r)
        (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1)
    ↪ )
    rec["paired"] = {k: paired(rec["reps"], k, dr) for k, _, dr in GAU
    ↪ GES}

    (wl_dir / f"{name}.json").write_text(json.dumps(rec, indent=1))
    return rec


def fmt(v):
    if v is None or (isinstance(v, float) and math.isnan(v)):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return "n/a"
    return f"{v:,.0f}" if abs(v) >= 100 else f"{v:,.3g}"


def report(recs, out: Path):
    shipped = bool(recs) and all(r.get("native") == "shipped" for r in
    ↪  recs)
    native_mb = recs[0]["native_mb"] if recs else NATIVE_MB
    L = ["# YCSB on MongoDB: Omni-Compass on top of a database's " + (
    ↪ "cache as shipped" if shipped else "operator-set cache size"), "
    ↪ ",
         (f"MongoDB exactly as shipped, its WiredTiger cache at the se
    ↪ rver's own default ({native_mb} MB on this machine), is native;
    ↪ "
          "a cache above the cover is at its limit on native, and omni
    ↪  then reads and writes nothing. Otherwise omni is the compass la
    ↪ w on the cache size "
          if shipped else f"MongoDB as shipped with the operator's Wir
    ↪ edTiger cache ({native_mb} MB) is native; omni is the compass la
    ↪ w on the cache size ")
         +
         f"through the server's own console inside [{COVER_MB[0]}, {CO
    ↪ VER_MB[1]}] MB, holding the server's own read latency at {CENTER
    ↪ :.0%} of "
         f"the {LINE_MS:.0f} ms line. YCSB's published core workloads,
    ↪  the same operations in both arms, the key space stepping throug
    ↪ h the "
         "cache and past it. Memory held is the resource; the host's C
    ↪ PU seconds include the compass's own cost. Every row is reported
    ↪ , "
         "losses included.", ""]
    for rec in recs:
        L += [f"## {rec['workload']}: YCSB {rec['ycsb_workload']}, {re
    ↪ c['base_records']:,} records × notch, steps {rec['steps']} × {re
    ↪ c['step_s']} s"
              + (" (the tuning workload)" if rec.get("tuning") else ""
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ), "",
              f"MongoDB {rec['mongodb_version']}, YCSB {rec['ycsb_vers
    ↪ ion']}; {rec['rate_ops']:,} operations a second offered from {re
    ↪ c['threads']} threads; "
              f"{len(rec['reps'])} paired repetitions; engine {rec['en
    ↪ gine']['version']} at `{rec['engine']['commit']}`.", "",

              "| Gauge | native | omni | change | 95% interval of the
    ↪ difference | reading |", "|---|---:|---:|---:|---:|---|"]
        for k, label, direction in GAUGES:
            p = rec.get("paired", {}).get(k)
            if not p:
                continue

            ch = "" if p["native"] == 0 else f"{100 * p['diff'] / abs(
    ↪ p['native']):+.1f}%"
            L.append(f"| {label} | {fmt(p['native'])} | {fmt(p['omni']
    ↪ )} | {ch} | {fmt(p['ci95'][0])} to {fmt(p['ci95'][1])} | {p['rea
    ↪ ding']} |")
        hb = all(r["omni"]["handed_back"] for r in rec["reps"]); fw =

    ↪ any(r["omni"]["foreign_writer"] for r in rec["reps"])
        L += ["", f"Every omni arm handed back to the operator's cache
    ↪  size: {'yes' if hb else 'NO'}; another writer seen: {'yes' if f
    ↪ w else 'no'}; "
              f"fail-ups: {sum(r['omni']['failups'] for r in rec['reps
    ↪ '])}.", ""]

    (out / "YCSB.md").write_text("\n".join(_legal_stamp(L)) + "\n")
    return out / "YCSB.md"


def setup():
    """YCSB fetched from the project's release and checked against the

    ↪  SHA-256 recorded before any run; the server pointed at
    the operator's cache (its configuration file) and restarted. Mongo
    ↪ DB itself is installed by the workflow from its publisher's
    repository, signed by the publisher's key."""
    tgz = YCSB_HOME.parent / f"ycsb-mongodb-binding-{YCSB_VERSION}.tar
    ↪ .gz"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    if not tgz.exists():
        urllib.request.urlretrieve(YCSB_URL, tgz)
    got = hashlib.sha256(tgz.read_bytes()).hexdigest()
    if got != YCSB_SHA256:
        print(f"YCSB checksum mismatch: {got} (expected {YCSB_SHA256})
    ↪ ", file=sys.stderr); return 1

    if not YCSB_HOME.exists():
        subprocess.run(["tar", "xzf", str(tgz), "-C", str(YCSB_HOME.pa
    ↪ rent)], check=True)
    conf = Path("/etc/mongod.conf")
    if conf.exists():
        txt = conf.read_text()

        if NATIVE_MB is not None and "cacheSizeGB" not in txt:
    ↪     # --native shipped: the server's own default stands
            txt = txt.replace("storage:\n", f"storage:\n  wiredTiger:\
    ↪ n    engineConfig:\n      cacheSizeGB: {NATIVE_MB / 1024:.3f}\n"
    ↪ , 1)
            subprocess.run(["sudo", "tee", str(conf)], input=txt, text

    ↪ =True, check=True, capture_output=True)
        subprocess.run(["sudo", "systemctl", "restart", "mongod"], che
    ↪ ck=True)
    ver = server_version()
    print(f"YCSB {YCSB_VERSION} at {YCSB_HOME} (sha256 {got[:16]}...);
    ↪  MongoDB {ver} at native's cache ({NATIVE}: "

          f"{NATIVE_MB if NATIVE_MB is not None else 'the server defau
    ↪ lt'} MB)")
    if ver == "?":                       # the server did not answer a
    ↪ fter its restart: no workload runs on a server that is not up
        print(f"mongod did not answer after the restart at native's ca
    ↪ che (kernel {os.uname().release}); its status and journal:\n"

              f"{server_journal('mongod')}", file=sys.stderr)
        return 1
    return 0


def main(argv=None):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    global NATIVE, NATIVE_MB
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=
    ↪ argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--setup", action="store_true")
    ap.add_argument("--native", default="operator", choices=("operator
    ↪ ", "shipped"),

                    help="native: the operator's 512 MB cache (preregi
    ↪ stered) or MongoDB exactly as shipped, its own default cache "
                         "(the founder's order of 17 October); a nativ
    ↪ e cache above the cover is at its limit on native")
    ap.add_argument("--workloads", default="tuning")
    ap.add_argument("--reps", type=int, default=3)

    ap.add_argument("--step-s", type=float, default=20.0)
    ap.add_argument("--line-ms", type=float, default=LINE_MS)
    ap.add_argument("--out", default="ycsb-out")
    ap.add_argument("--objective", default=DEFAULT_OBJECTIVE, choices=
    ↪ OBJECTIVES,
                    help="what the brain's verdict judges a step by: r

    ↪ esource (the index's reading: work, speed, memory and CPU togeth
    ↪ er) or service (work and speed alone)")
    ap.add_argument("--report-only", default="")
    a = ap.parse_args(argv)
    NATIVE, NATIVE_MB = a.native, (None if a.native == "shipped" else
    ↪ 512)

    if a.setup:
        return setup()
    out = Path(a.out)
    if a.report_only:
        recs = [json.loads(f.read_text()) for f in sorted(Path(a.repor
    ↪ t_only).rglob("ycsb-*/*.json"))]

        print(report(recs, Path(a.report_only))); return 0
    names = list(WORKLOADS) if a.workloads == "all" else a.workloads.s
    ↪ plit(",")
    recs = [run_workload(n, out, a.reps, a.step_s, a.line_ms, a.object
    ↪ ive) for n in names]
    print(report(recs, out))
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

H.60 tools/store_brain.py

187 lines.

```
#!/usr/bin/env python3
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
"""The max on a store's memory knob (omni-v22; docs/STORE_ROOT_CAUSES_
    ↪ 2026-10-30.md): what the three cache wires share.

The store wires were written on 9 and 10 October 2026 with the brain's
    ↪  verdict judging every move under the resource objective,
before the founder's order of 12 October that the max is the default b
    ↪ rain, and were never brought to it. Under the max a knob
runs at its own limit inside the operator's cover, never past it and n
    ↪ ever below native; the memory it takes is power given,

shown beside the misses it bought off; the tune-down (`dial save`) kee
    ↪ ps omni-v21's loop, where giving memory back is the aim.
Where the memory can bring the knob no GAIN, the knob stays native (th
    ↪ e engine's strictest form, the founder's words of 19
October): on a store that is not slow for want of memory, memory is no
    ↪ t spent.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
Four pieces, each read from the store's and the host's own meters, not
    ↪ hing written for a workload:

  the working set's floor   the most memory the store held in the last
    ↪  HOLD_WINDOW_S, with the fullness margin, rounded up to a
                            notch, never under native nor over the cov
    ↪ er: a knob handed back below it evicts what the service
                            still asks for, and the next rise buys it
    ↪ back at the store's price (the sawtooth the Redis model
                            showed on a candidate without it)
  the cost of the misses    the store's own account: the share of the
    ↪ service's own time that its misses are, over
                            WORTH_WINDOW_S. MySQL: the performance sch
    ↪ ema's timer on the data files' reads against the
                            workload's statement timer; MongoDB: Wired
    ↪ Tiger's time reading pages from disk for the application's
                            threads against the server's own read late
    ↪ ncy; Redis: the application's own split of each request
                            into hits and misses. A slope of latency o
    ↪ n miss share was tried first and is not used: on the
                            archived runs it read 0.5 to 1.2 ms a unit
    ↪  of miss share inside the arms, while the controlled
                            comparison across pool sizes (a third of t
    ↪ he misses gone, the latency unmoved) read none; the
                            seconds' noise is shared by both, and the
    ↪ slope reads it as a price
  the gate                  memory is spent only where that share is M
    ↪ ISS_WORTH or more, the one key's own floor (a change of
                            0.5% or more is a change, the founder's or
    ↪ der of 24 October): under it the misses are nothing the
                            key could read, so memory could buy nothin
    ↪ g it could read, and the knob holds and says so; where the
                            store's account is not read this second, n
    ↪ othing is known and nothing is taken. The floor is not a
                            tuning: it is the smallest change the reco
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ rd reads at all
  the host's room           a notch is taken only while the host keeps
    ↪  HOST_CUSHION of its memory available after it (the
                            compass's cushion; the host is a muscle to
    ↪ o, and never past its own limit); unread, nothing is taken
"""

from __future__ import annotations

import math
from collections import deque

HOLD_WINDOW_S = 60.0

WORTH_WINDOW_S = 10.0
MISS_WORTH = 0.005              # the one key's floor, tools/readings.
    ↪ py SIZE_FLOOR (a change under 0.5% reads EVEN, under the floor);
    ↪  written
                                # here as a number because this file i
    ↪ s in the engine's fingerprint and readings.py is not

HOST_CUSHION = 0.05
MEMINFO = "/proc/meminfo"
KINDS = ("spend", "held_unread", "held_worth", "held_host", "handed_ba
    ↪ ck", "handed_to_floor", "held_floor", "failups")

class WorkingSetFloor:
    def __init__(self, native, notch, cover, full=0.9, window_s=HOLD_W
    ↪ INDOW_S):
        self.native, self.notch, self.cover, self.full, self.window_s
    ↪ = native, notch, cover, full, window_s
        self.used = deque()

    def observe(self, t, used):
        if used is None:
            return
        self.used.append((t, float(used)))
        while self.used and t - self.used[0][0] > self.window_s:
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            self.used.popleft()

    def value(self, t=None):
        peak = max((u for _, u in self.used), default=0.0) / self.full
        return min(self.cover[1], max(self.native, int(math.ceil(peak
    ↪ / self.notch)) * self.notch))

class MissCost:
    """The share of the service's own time that its misses are, from t
    ↪ he store's own meter, over a window of seconds."""

    def __init__(self, window_s=WORTH_WINDOW_S):
        self.window_s = window_s
        self.rows = deque()

    def observe(self, t, miss_time_s, service_time_s):
        """One second: the time the store spent on its misses and the

    ↪ time the service spent on its requests, both in seconds.
        A second with no service time, or without the store's account,
    ↪  tells nothing and is not kept."""
        if miss_time_s is None or service_time_s is None or service_ti
    ↪ me_s <= 0:
            return False

        self.rows.append((t, max(0.0, float(miss_time_s)), float(servi
    ↪ ce_time_s)))
        while self.rows and t - self.rows[0][0] > self.window_s:
            self.rows.popleft()
        return True

    def worth(self):
        """Σ miss time / Σ service time over the window, between 0 and
    ↪  1; None until a second has been read."""
        service = sum(s for _, _, s in self.rows)
        if not self.rows or service <= 0:
            return None
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        return max(0.0, min(1.0, sum(m for _, m, _ in self.rows) / ser
    ↪ vice))


def read_meminfo(path=MEMINFO):
    """(available MB, total MB) from the host's own account, or (None,
    ↪  None) where it cannot be read."""
    try:
        avail = total = None
        with open(path) as fh:
            for line in fh:
                if line.startswith("MemAvailable:"):
                    avail = int(line.split()[1]) / 1024.0
                elif line.startswith("MemTotal:"):
                    total = int(line.split()[1]) / 1024.0
        return avail, total
    except (OSError, ValueError, IndexError):
        return None, None

def host_room(available_mb, total_mb, notch_mb, cushion=HOST_CUSHION):
    """Whether the host can give a notch and keep its cushion; None wh
    ↪ ere the host's memory is unread (nothing known)."""
    if available_mb is None or total_mb is None or total_mb <= 0:
        return None
    return available_mb - notch_mb >= cushion * total_mb


def max_target(want, cur, native, floor, worth, room, why, worth_floor
    ↪ =MISS_WORTH):
    """The max's target from the wire's own rule (want): a spend goes
    ↪ through unclamped where the store's own account says the
    misses cost the service and the host has the notch to spare; it ho
    ↪ lds where the misses cost nothing, where the account is not
    read, or where the host has no room; a give-back never goes below
    ↪ the working set's floor nor below native.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    Returns (target, why, kind), kind one of KINDS or "hold"."""
    if want > cur:
        if worth is None:
            return cur, "the store's account of a miss's cost not read
    ↪  this second: nothing known, nothing taken", "held_unread"
        if worth < worth_floor:

            return cur, (f"misses are {100 * worth:.0f}% of the servic
    ↪ e's own time, under {100 * worth_floor:.0f}%: memory would buy "
                         "nothing; held"), "held_worth"
        if room is None:
            return cur, "the host's memory unread: nothing known, noth
    ↪ ing taken", "held_host"

        if not room:
            return cur, f"the host has no notch to spare over its {100
    ↪  * HOST_CUSHION:.0f}% cushion: held", "held_host"
        return want, why + f"; misses {100 * worth:.0f}% of the servic
    ↪ e's time: power given inside the cover", "spend"
    if want < cur:

        if want >= floor:
            return want, why + f"; the working set's last {HOLD_WINDOW
    ↪ _S:.0f} s fit under {floor}", "handed_back"
        if cur > floor:
            return max(want, floor), f"calm: handed back to the workin
    ↪ g set's floor, {floor}", "handed_to_floor"

        return cur, f"hold: {cur} is what the working set's last {HOLD
    ↪ _WINDOW_S:.0f} s occupied", "held_floor"
    return cur, why, "hold"


class MemoryMax:

    """The max on one store's memory knob: the four pieces above aroun
    ↪ d the wire's own rule. Each wire's Brain keeps its own
    compass law and its own `decide`; this governs what that rule want
    ↪ s."""

    def __init__(self, native, notch, cover, full, meter):
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        self.native, self.notch, self.cover, self.meter = native, notc
    ↪ h, cover, meter
        self.floor = WorkingSetFloor(native, notch, cover, full=full)
        self.cost = MissCost()
        self.counts = {k: 0 for k in KINDS}
        self.counts["hold"] = 0

    def step(self, t, want, why, cur, used_mb, miss_time_s, service_ti
    ↪ me_s, host_avail_mb, host_total_mb, fail_up=False):
        self.floor.observe(t, used_mb)
        self.cost.observe(t, miss_time_s, service_time_s)
        worth = self.cost.worth()

        room = host_room(host_avail_mb, host_total_mb, self.notch)
        floor = self.floor.value(t)
        target, why, kind = max_target(want, cur, self.native, floor,
    ↪ worth, room, why)
        self.counts[kind] += 1
        if kind == "spend" and fail_up:

            self.counts["failups"] += 1
        info = {"verdict_phase": None, "verdict_direction": None, "all
    ↪ owed_low": None, "allowed_high": None, "dial": "max",
                "floor_mb": floor, "miss_worth": None if worth is None
    ↪  else round(worth, 3),
                "miss_time_ms": None if miss_time_s is None else round

    ↪ (1000.0 * miss_time_s, 3),
                "host_avail_mb": None if host_avail_mb is None else ro
    ↪ und(host_avail_mb),
                "host_room": room}
        return target, why, info

    @property
    def failups(self):
        return self.counts["failups"]


    def record(self, objective):
        """What the arm's record carries where omni-v21's loop carried
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  the verdict's allowance."""
        return {"dial": "max", "objective": objective, "native": self.
    ↪ native, "notch": self.notch, "cover": list(self.cover),
                "state": f"the max: spends by {self.meter}, never belo
    ↪ w native; no allowance",
                "allowed_low": None, "allowed_high": None, "counts": d
    ↪ ict(self.counts)}


def summarize_max(records):
    """One line for a table from the omni arms' records of one workloa
    ↪ d under the max."""
    parts = []
    for r in records:
        c = r.get("counts", {})
        parts.append(f"the max ({c.get('spend', 0)} spends, {c.get('he
    ↪ ld_worth', 0)} held: misses cheap, {c.get('held_unread', 0)} hel
    ↪ d: "
                     f"account unread, {c.get('held_host', 0)} held: t
    ↪ he host, {c.get('handed_back', 0) + c.get('handed_to_floor', 0)}
    ↪  handed back)")
    return "; ".join(parts)
```

APPENDIX

Appendix I. The License

The license under which this book and the software are made available, as it stands in the file `LICENSE` at the root of the repository.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
Omni-Compass Evaluation License, version 1.0 (revised 11 October 2026)
SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0

Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
All patents, copyrights and trademarks filed in the USA. www.omni-comp
    ↪ ass.com

THIS IS NOT AN OPEN-SOURCE LICENSE. The Software is proprietary. It is
    ↪  published so that it can be evaluated, reproduced
and audited, and for no other purpose. It is licensed, not sold. Evalu
    ↪ ation and simulation are free of charge; every
other use, and in particular every commercialization or monetization o

    ↪ f any kind, requires a signed, paid Omni-Compass
Enterprise License.

ACCEPTANCE. By downloading, copying, forking, installing, running, sca
    ↪ nning or otherwise using any part of the Software,
You accept this license. If You do not accept it, You may not use the

    ↪ Software. If You use the Software on behalf of an
organization, You represent that You have authority to bind it, and "Y
    ↪ ou" includes that organization and its
Affiliates.

1. DEFINITIONS

   1.1 "Licensor" means The Omni-Compass LLC.
   1.2 "Software" means everything Licensor publishes or makes availab
    ↪ le under this license, in any form: the source
       code and any object code; the Omni-Compass engine, the compass
    ↪ law, the governors, controllers, brains, shields,

       plugs and wiring; the harnesses, tools, scripts, workflows, con
    ↪ figurations, presets, catalogs, organisms and realms;
       the documentation, the manual, the book, the specifications, th
    ↪ e preregistrations and the receipts; the benchmarks,
       data, tables, reports and Results published by Licensor; and ev
    ↪ ery update, version and copy of any of them.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
1.3 "Mechanism" means the methods, processes, control laws, algorit
 ↪ hms, architectures and systems embodied in or
    described by the Software, including everything described in th
 ↪ e patent applications Licensor has filed.
1.4 "Evaluation" means internal use by You, free of charge, solely
 ↪ to test, study and assess the Software in

    contemplation of an Enterprise License: reading it; running its
 ↪  tests, verifiers, benchmarks and simulations; and
    running it in observe, watch, dry-run or shadow mode, or in a t
 ↪ est environment, on systems You own or control,
    where it writes to no system that serves production traffic, re
 ↪ al users, customers, patients or passengers, and to

    no power system, vehicle, machine or physical process outside a
 ↪  test bench.
1.5 "Simulation" means running the Software against modelled, recor
 ↪ ded or synthetic systems, including the organisms,
    realms, traces and simulators published with the Software.
1.6 "Production Use" means any use that is not Evaluation or Simula

 ↪ tion, including any use in which the Software, the
    Mechanism or any output of either controls, governs, tunes, siz
 ↪ es, schedules or otherwise influences a system that
    serves production workloads or real users, or any physical proc
 ↪ ess outside a test bench.
1.7 "Commercial Use" means any use for, or in connection with, comm

 ↪ ercial advantage, revenue, cost reduction in
    operations, or monetary or other compensation, including consul
 ↪ ting, integration, managed, outsourcing, training,
    benchmarking-for-hire and advisory services, and any use by or
 ↪ for a for-profit operation beyond Evaluation.
1.8 "Hosted Service" means making the Software, the Mechanism or an

 ↪ y of their functionality available to any third
    party over a network or as a service, including software-as-a-s
 ↪ ervice, platform-as-a-service, a managed or
    outsourced service, a cloud marketplace offering, or a feature
 ↪ of another product or service.
1.9 "Derivative" means any modification, adaptation, translation, p
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ort, extension or re-implementation of the Software
       or of the Mechanism, and any work that incorporates, is based o
    ↪ n, or is built using the Software as a reference.
   1.10 "Results" means the measurements, tables, receipts, logs, repo
    ↪ rts and other outputs produced by running the
       Software.

   1.11 "Enterprise License" means a written agreement for the Softwar
    ↪ e signed by Licensor and a licensee, under which
       the licensee pays the fees it sets out; it may include order fo
    ↪ rms, riders and statements of work.
   1.12 "Marks" means the names "Omni-Compass" and "OmniCompass", the
    ↪ Omni-Compass compass rose with its ring of Greek

       letters, and every related name, logo and trade dress of Licens
    ↪ or.
   1.13 "Affiliate" means any entity that controls, is controlled by,
    ↪ or is under common control with You, where control
       means ownership of more than half of the voting interests or th
    ↪ e power to direct management.

   1.14 "You" means the individual or entity exercising rights under t
    ↪ his license, with its Affiliates where the
       Acceptance paragraph above includes them.

2. WHAT THIS LICENSE ALLOWS

   2.1 Copyright license. Subject to Your compliance with this license
    ↪ , Licensor grants You a personal, worldwide,
       royalty-free, non-exclusive, non-transferable, non-sublicensabl
    ↪ e and revocable license to download, copy, read,
       run and modify the Software solely for Evaluation and Simulatio
    ↪ n, and to reproduce and publish Results as section

       4 allows.
   2.2 Patent license. Subject to the same conditions, Licensor grants
    ↪  You a license of the same scope, under the patent
       rights Licensor holds or will hold in the filed applications an
    ↪ d any patent issuing from them, solely to the
       extent needed to carry out Evaluation and Simulation with the S
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ oftware. No patent license is granted for any
       Production Use, Commercial Use or Hosted Service, for any Deriv
    ↪ ative used outside Evaluation or Simulation, or for
       any implementation of the Mechanism other than the Software its
    ↪ elf.
   2.3 Forks. You may keep a copy or a fork of the Software on a code-

    ↪ hosting service, publicly or privately, only for
       Evaluation and Simulation, with this license and every notice i
    ↪ ntact. A fork confers no right beyond this license.
   2.4 Reservation. Every right not expressly granted is reserved by L
    ↪ icensor. No right, title, license or interest is
       granted by implication, estoppel, exhaustion or otherwise.

3. WHAT THIS LICENSE DOES NOT ALLOW

   Unless an Enterprise License expressly permits it, You may not, and
    ↪  may not let or help anyone else:
   (a) make any Production Use or Commercial Use of the Software, the

    ↪ Mechanism or any Derivative;
   (b) commercialize or monetize the Software, the Mechanism, any Deri
    ↪ vative or any Result in any way, directly or
       indirectly, including by charging for access, services, support
    ↪ , results, savings or outcomes they produce;
   (c) offer the Software, the Mechanism or any Derivative as, or as p

    ↪ art of, a Hosted Service;
   (d) sell, resell, license, sublicense, lease, rent, lend, distribut
    ↪ e, publish as a package or binary, or otherwise
       transfer the Software or any Derivative, except forks under sec
    ↪ tion 2.3;
   (e) incorporate the Software, any part of it or any Derivative into

    ↪  any product, service, platform, appliance, image,
       firmware or offering;
   (f) use the Software, the Mechanism, the documentation or the Resul
    ↪ ts to design, build, train, tune, benchmark for
       sale, or improve any product or service that competes with Omni
    ↪ -Compass, or to re-implement the Mechanism;
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   (g) use the Software or the Results to train, fine-tune or evaluate
    ↪  any machine-learning model intended to reproduce,
       imitate or substitute for the Mechanism;
   (h) remove, alter or obscure any notice, the SPDX identifiers, the
    ↪ patent and trademark notices, this license, the
       NOTICE or DISCLOSURES.md, or circumvent, disable or work around
    ↪  any limit, switch or safeguard in the Software,
       including the kill switch and the hand-back to native settings;
   (i) connect the Software so that it writes to a safety-critical sys
    ↪ tem, as section 10 describes;
   (j) use the Marks, except to refer to Omni-Compass accurately as TR
    ↪ ADEMARKS.md allows; or
   (k) use the Software in violation of any law, or in breach of secti
    ↪ on 11.

4. REPRODUCING AND PUBLISHING RESULTS

   You may reproduce the benchmarks published with the Software and pu
    ↪ blish Your own Results, provided that You: (a)
   name the Software's engine version and the exact commit You ran; (b
    ↪ ) report every measured row, the unfavourable ones
   included, with the evidence class the Software assigns to it; (c) m
    ↪ ake Your complete raw files and configuration
   available to Licensor on request; (d) do not state or imply any end
    ↪ orsement, certification or affiliation by
   Licensor; and (e) carry the notice at the top of README.md. Publish
    ↪ ing Results is not Commercial Use so long as no
   compensation is received for it, and it grants no right beyond this
    ↪  license.

5. THE OMNI-COMPASS ENTERPRISE LICENSE

   5.1 Every Production Use, Commercial Use, Hosted Service, distribut
    ↪ ion, incorporation, OEM or embedded use, and every
       other commercialization or monetization of the Software, the Me
    ↪ chanism or any Derivative requires an Enterprise
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
      License, signed before the use begins and paid as it provides.
    ↪ Moving from Evaluation to Production Use requires
      an Enterprise License before the first write to any production
    ↪ system.
   5.2 An Enterprise License sets its own scope, fees, term, support,
    ↪ warranties, indemnities and limits

      (ENTERPRISE_LICENSE.md summarizes the standard terms; the signe
    ↪ d agreement alone is binding). For its licensee and
      its scope, a signed Enterprise License governs where it differs
    ↪  from this license.
   5.3 Enterprise licensing: The Omni-Compass LLC, main office infrast
    ↪ ructure, toll-free direct 1-888-339-9551, system route asset 1-8

    ↪ 88-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.o
    ↪ mni-compass.com.

6. OWNERSHIP AND INTELLECTUAL PROPERTY

   6.1 Licensor and its licensors own the Software, the Mechanism, the

    ↪  Marks and all intellectual property rights in
      them, including in every Derivative to the extent it embodies t
    ↪ hem. Nothing in this license transfers ownership.
   6.2 All patents, copyrights and trademarks covering the Omni-Compas
    ↪ s engine, its mathematics and its software have
      been filed in the United States by The Omni-Compass LLC. No fil

    ↪ ing number is stated here, and no grant,
      registration or approval is claimed; the status of every filing
    ↪  is held by the office where it was filed. Every
      person who downloads, copies, forks, scans or runs the Software
    ↪  is on notice of these filings and of this
      license (PATENTS.md, TRADEMARKS.md).

   6.3 Any use outside this license, without an Enterprise License, is
    ↪  unlicensed and infringes Licensor's copyright and
      other rights, in addition to breaching this license.

7. PATENT AND RIGHTS DEFENSE
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   If You or any of Your Affiliates bring or join any claim, countercl
    ↪ aim or proceeding alleging that the Software or
   the Mechanism infringes, misappropriates or violates any patent, co
    ↪ pyright or other intellectual property right, or
   challenge the validity or enforceability of any of Licensor's right
    ↪ s in them, every license granted to You under this

   license ends automatically on the date the claim or challenge is fi
    ↪ led.

8. FEEDBACK AND CONTRIBUTIONS

   8.1 If You give Licensor comments, suggestions, ideas, reports or o

    ↪ ther feedback about the Software, You grant
       Licensor a perpetual, irrevocable, worldwide, royalty-free righ
    ↪ t to use, disclose and exploit it for any purpose,
       without obligation of any kind.
   8.2 Contributions are accepted only under the contributor license a
    ↪ greement (.github/CLA.md).

9. CHANGES

   Licensor reserves the right to change the Software, its documentati
    ↪ on, its results and these terms at any time,
   without notice. A changed version of these terms applies to every c

    ↪ opy of the Software obtained, and to every use
   made, after it is published in the Software's repository; continued
    ↪  use after publication is acceptance of the
   changed terms. Licensor may discontinue the Software, or this licen
    ↪ se for future copies, at any time.

10. SAFETY

   The Software is not designed, tested or certified for safety-critic
    ↪ al use. Under this license it may not write to
   any system whose failure could cause death, injury, environmental d
    ↪ amage or significant property damage, including
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   vehicles and aircraft, medical devices, power-grid protection and d
    ↪ ispatch, industrial safety systems and life-safety
   systems; such systems may be studied only in Simulation. Any live c
    ↪ onnection to a physical system requires an
   Enterprise License, a written safety case accepted by the system's
    ↪ owner, and every review that system's regulator or

   owner requires. The machines' own controls and protections stay in
    ↪ place; the kill switch must remain operable at all
   times.

11. EXPORT CONTROL, SANCTIONS AND LAWFUL USE

   You will comply with every export-control, sanctions, anti-corrupti
    ↪ on and other law that applies to the Software and
   to Your use of it, including the U.S. Export Administration Regulat
    ↪ ions and the sanctions administered by the U.S.
   Treasury's Office of Foreign Assets Control. You represent that You
    ↪  are not located in, organized under the laws of,

   or ordinarily resident in a country or region subject to comprehens
    ↪ ive U.S. sanctions, and that You are not on, or
   owned or controlled by a party on, any U.S. restricted-party list.
    ↪ You will not use the Software for any end use
   prohibited by those laws, including the design, development or prod
    ↪ uction of weapons of mass destruction.

12. DATA AND PRIVACY


   The Software contains no telemetry. It sends nothing to Licensor an
    ↪ d makes no network connection of its own except to
   the systems You point it at. Your data and Your Results stay on You

    ↪ r systems and are Yours, subject to section 4. If
   You send Licensor any material (for example in an issue or an e-mai
    ↪ l), You represent that You may lawfully do so and
   that it contains no personal data or confidential information of an
    ↪ y third party.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
13. THIRD-PARTY COMPONENTS

   Files that carry a different license notice (for example tests/thir
    ↪ d_party/) and third-party data sets (for example
   the PlanetLab traces under fleet/traces/ and k8s_controlplane/trace
    ↪ s/) remain under their own terms

   (THIRD_PARTY_NOTICES.md). Third-party software the Software runs on
    ↪  or connects to (Kubernetes, KEDA, Karpenter,
   NVIDIA drivers, databases and others) is licensed by its owners und
    ↪ er their own terms.

14. TERM AND TERMINATION

   14.1 This license lasts until it ends under this section or section
    ↪  7.
   14.2 It ends automatically, without notice, if You breach it. If Yo
    ↪ ur breach is Your first, was inadvertent, and is
        not a breach of sections 3(a) to 3(g) or section 10, Your lice

    ↪ nse is reinstated if You cure the breach within
        thirty days after You learn of it or after Licensor notifies Y
    ↪ ou, whichever comes first.
   14.3 Licensor may end this license for any person at any time by wr
    ↪ itten notice, and for all persons for future
        copies under section 9.

   14.4 When this license ends, You must stop all use of the Software,
    ↪  delete or destroy every copy and Derivative in Your
        possession or control, and, on request, certify in writing tha
    ↪ t You have done so.
   14.5 Sections 1, 3, 4(b) to 4(e), 6, 7, 8, 11, 12, 15 to 18 and 19
    ↪ survive the end of this license.

15. NO WARRANTY

   TO THE FULLEST EXTENT PERMITTED BY LAW, THE SOFTWARE, THE MECHANISM
    ↪  AND THE RESULTS ARE PROVIDED "AS IS" AND "AS
   AVAILABLE", WITH ALL FAULTS AND WITHOUT WARRANTY OR CONDITION OF AN
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ Y KIND, EXPRESS, IMPLIED OR STATUTORY, INCLUDING ANY
   WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITL
    ↪ E, NON-INFRINGEMENT, ACCURACY, QUIET ENJOYMENT, OR
   THAT THE SOFTWARE WILL BE ERROR-FREE OR UNINTERRUPTED. NO RESULT IS
    ↪  A PROMISE OF ANY OUTCOME ON ANY OTHER SYSTEM;
   MODELLED RESULTS (EVIDENCE CLASS S) ARE STATEMENTS ABOUT THE MODEL,

    ↪  NOT MEASUREMENTS (DISCLOSURES.md). YOU USE THE
   SOFTWARE AT YOUR OWN RISK AND ARE RESPONSIBLE FOR ITS USE ON YOUR S
    ↪ YSTEMS.

16. LIMITATION OF LIABILITY

   TO THE FULLEST EXTENT PERMITTED BY LAW, LICENSOR, ITS MEMBERS, MANA
    ↪ GERS, EMPLOYEES, CONTRACTORS AND LICENSORS SHALL NOT
   BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXE
    ↪ MPLARY OR PUNITIVE DAMAGES, OR FOR ANY LOSS OF
   PROFITS, REVENUE, SAVINGS, DATA, USE OR GOODWILL, OR ANY COST OF SU
    ↪ BSTITUTE GOODS OR SERVICES, ARISING OUT OF OR

   RELATING TO THE SOFTWARE, THE MECHANISM, THE RESULTS OR THIS LICENS
    ↪ E, UNDER ANY THEORY OF LIABILITY, EVEN IF ADVISED OF
   THEIR POSSIBILITY. LICENSOR'S TOTAL LIABILITY FOR ALL CLAIMS SHALL
    ↪ NOT EXCEED ONE HUNDRED U.S. DOLLARS (USD 100). THESE
   LIMITS APPLY EVEN IF A REMEDY FAILS OF ITS ESSENTIAL PURPOSE.

17. INDEMNITY

   You will defend, indemnify and hold harmless Licensor and its membe
    ↪ rs, managers, employees and contractors from and
   against every claim, loss, liability, damage, cost and expense (inc
    ↪ luding reasonable attorneys' fees) arising out of

   or relating to (a) Your use of the Software outside this license, (
    ↪ b) any Production Use, Commercial Use or Hosted
   Service without an Enterprise License, (c) Your Derivatives, (d) Yo
    ↪ ur Results as You publish them, or (e) Your
   violation of any law or third-party right.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
18. U.S. GOVERNMENT END USERS

   The Software is "commercial computer software" and "commercial comp
    ↪ uter software documentation" as those terms are
   used in 48 C.F.R. 2.101, 12.212 and 227.7202. A U.S. Government end
    ↪  user acquires only the rights set out in this

   license, consistent with 48 C.F.R. 12.212 and 227.7202-1 through 22
    ↪ 7.7202-4.

19. GENERAL

   19.1 Governing law. This license is governed by the laws of the Sta

    ↪ te of Delaware and the federal laws of the United
        States, without regard to conflict-of-laws rules. The United N
    ↪ ations Convention on Contracts for the
        International Sale of Goods does not apply.
   19.2 Venue. The state courts of the State of Delaware and the Unite
    ↪ d States District Court for the District of

        Delaware have exclusive jurisdiction over any dispute arising
    ↪ out of or relating to this license, and You consent
        to their personal jurisdiction. Licensor may seek
        injunctive or other equitable relief in any court of competent
    ↪  jurisdiction, without posting a bond, to protect
        its intellectual property or confidential information; a breac

    ↪ h of sections 3 or 10 would cause Licensor
        irreparable harm.
   19.3 Enforcement costs. In any action to enforce this license, the
    ↪ prevailing party may recover its reasonable
        attorneys' fees and costs.
   19.4 Assignment. You may not assign or transfer this license or any

    ↪  right under it, by operation of law or otherwise;
        any attempt is void. Licensor may assign this license freely.
   19.5 Notices. Notices to Licensor go to The Omni-Compass LLC by tel
    ↪ ephone at its main office infrastructure (toll-free direct 1-888
    ↪ -339-9551; system route asset 1-888-DEX-YLL-1, landline 321-746-
    ↪ 4411 registered in Florida) or through www.omni-compass.com. Lic
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ensor may give notice to
        You through the Software's repository or at any address You ha
    ↪ ve given it.
   19.6 Severability and waiver. If any provision is held unenforceabl
    ↪ e, it is enforced to the maximum extent permitted and
        the rest of this license remains in effect. No failure or dela

    ↪ y in enforcing any provision is a waiver of it.
   19.7 Entire license. This license, with the NOTICE and DISCLOSURES.
    ↪ md, is the entire agreement between You and
        Licensor about its subject; for a licensee and its scope, a si
    ↪ gned Enterprise License governs where it differs.
   19.8 Interpretation. "Including" means "including without limitatio

    ↪ n". Headings are for convenience only. The
        English text of this license controls over any translation.
   19.9 Copies. Every copy, fork, export, report, archive or printout
    ↪ of any part of the Software must carry this
        LICENSE, the NOTICE and DISCLOSURES.md unchanged.

All rights reserved. Revision history: version 1.0 (evaluation and sim
    ↪ ulation only; first published 2026); revised
10 October 2026 (the notice in one wording everywhere); revised 11 Oct
    ↪ ober 2026 (definitions, patent license for
evaluation, hosted-service and machine-learning restrictions, results
    ↪ publication, patent and rights defense, safety,

export control, data, U.S. Government end users, term, cure, indemnity
    ↪ , Delaware governing law and venue).


Enterprise licensing: The Omni-Compass LLC, main office infrastructure
    ↪ , toll-free direct 1-888-339-9551, system route asset 1-888-DEX-
    ↪ YLL-1, landline 321-746-4411 registered in Florida, www.omni-com

    ↪ pass.com
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

APPENDIX

Appendix J. Reproducing Everything, From a Clean Machine

A referee with a laptop and a GitHub account can rebuild every table in this manual and rerun every benchmark. The steps, in order, with what each costs:

1. **Get the code and prove it is the code.** `git clone` the repository, `pip install -r requirements.txt` under Python 3.12 (the version GitHub runs), then `python3 verify.py`: it checks every sealed file's fingerprint, runs every test in `tests/`, including the rules of every three-run table on fixed cases, and ends `VERIFICATION: PASS`. About ten minutes on a laptop. `python3 tools/omni_version.py` then says which frozen engine the checkout carries.
2. **Rebuild any table from its raw files.** Every three-run table names its three run ids; their files are in `results/live/raw/run-<id>/` once the branch `raw` is opened there (`bash scripts/raw_archive.sh open /run-<A>/ /run-<B>/ /run-<C>/`). `python3 tools/confirm_abc.py "<title>" results/live/raw/run-<A> results/live/raw/run-<B> results/live/raw/run-<C> --out /tmp/check.md` rebuilds a Kubernetes or Azure table; `tools/pgbench_abc.py`, `kafka_abc.py`, `redis_abc.py`, `ycsb_abc.py`, `swarm_abc.py`, `mujoco_abc.py`, `pandapower_abc.py` and `citylearn_abc.py` the others, with the same arguments. Compare the result with the committed table: they are the same bytes below the legal notice. A few seconds each.
3. **Rebuild the index and the dossier.** `python3 tools/omni_index.py` reads every `V1_*.json` and `V3_*.json` table and writes `results/OMNI_INDEX.md`; `python3 tools/dossier.py` writes `docs/DOSSIER.md` and its charts. Seconds.
4. **Rerun a benchmark.** Every benchmark is a GitHub Actions workflow with its inputs documented at its head (`.github/workflows/`): `benchmark-reps` for the Kubernetes tests (`duration_s`, `arms`, `loadgen`, `load_steps`, `faults`, `workload`), `pgbench`, `kafka`, `redis`, `ycsb`, `swarm`, `robustness`, `citylearn`, `pandapower`, `mujoco`, `six`, `six-kube`. Dispatch it three times for A, B and C; each run archives its files when its id is added to `.github/archive_request.txt`. The Kubernetes tests run on GitHub's free runners in about an hour each; the database, broker, cache and swarm runs in about half an hour; the organisms at 1,000 copies need a rented machine (`aks-metered`

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

and `big-organism-detached` need an Azure subscription and its credentials in the repository's secrets).

5. **Run it on your own system.** Chapter 57 is the method; `scripts/kind_paired.sh` and `tools/live_reps.py` are the tools; the preregistrations are the templates for writing your own rules down before you run.
6. **Check a result against its preregistration.** Each `docs/*_PREREGISTRATION.md` was committed before its first counted run; `git log` on the file shows when, and every amendment carries its own date and reason in the text.

Nothing in this chain requires a credential, a licence key or a word from us; the evaluation licence covers all of it.

BACK MATTER

Index

Every term of the glossary where this book defines or discusses it, every muscle family where it is named, every file of the repository and every command-line setting this book names, with the pages they stand on. A plain mention of a common word is not an entry: the index points to where a thing is explained. A term whose pages would run past its line keeps the pages that fit, where it is defined and first used; after those it is throughout the book.

-

--allow 228
--audit 260, 476
--baseline-file 480
--batch 226, 262, 294
--batch-pace 226, 262
--brake-demand 533
--cap-deployments 226, 262, 547
--check 847
--coast-step 532
--coldstart-deployments 226, 262

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

--coldstart-idle-s . . . 804
--commit . . . 730
--compass-center . . . 226
--contain-namespaces . . . 226, 263
--convey-every-s . . . 805
--convey-on . . . 262
--cooling-cmd . . . 205, 228, 257, 265
--cpufreq-policy-root . . . 205, 228, 294
--cruise-after . . . 86, 89, 234, 533, 546, 767, 810
--cruise-after-s . . . 234, 767, 810
--dial . . . 757
--floor-interval . . . 237, 546, 805, 812
--floor-w . . . 490
--give-back-only . . . 127, 237, 553, 764, 767, 768
--gpu-query-cmd . . . 264, 294
--headroom . . . 264
--idle-demand . . . 86
--interval . . . 227, 237, 264, 546, 732, 805, 807, 812
--kill-file . . . 86, 118, 260
--latency-file . . . 218, 226, 262
--law . . . 226, 514
--max-nodes . . . 75, 533, 541, 760
--min-nodes . . . 529
--min-share . . . 264, 480
--mode . . . 219, 226, 262, 263, 293
--n . . . 478
--native . . . 475, 796, 817
--node-restore-cmd . . . 226
--node-scale-cmd . . . 205

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

--own-every-s . . . 817
--own-replicas . . . 2, 122, 817
--own-target . . . 817
--phases . . . 498
--pod-reflex-writes . . . 291
--power-cmd . . . 205, 228, 257, 265, 294, 547, 797
--profile . . . 492
--rapl-cmd . . . 205
--replica-ceiling . . . 524, 541, 761
--rightsize-deployments . . . 226, 262
--rollout-guard . . . 226, 262
--security-configmap . . . 226, 263
--sensed . . . 520, 547
--setup . . . 820
--sim . . . 484
--slo-clear . . . 234, 260, 767, 800
--slo-clear-s . . . 234, 280, 767, 800
--slo-ms . . . 209, 218, 226, 227, 228, 860
--speed-gain . . . 264
--strict-replicas . . . 227, 263, 294
--strict-window . . . 767
--strict-window-s . . . 234, 767
--trust-uid . . . 98, 280
--util-gate . . . 264, 480
--verdict-every . . . 546, 767
--verdict-every-s . . . 234, 767
--verdict-recheck . . . 546, 767
--verdict-recheck-s . . . 234, 767
--verdict-samples . . . 3

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

--verdict-trial-s . . . 234, 767
--workloads . . . 847, 848

.

.github/workflows/aks-metered.yml . . . 357, 519
.github/workflows/azure-inventory.yml . . . 539
.github/workflows/big-organism-detached.yml . . . 242, 267
.github/workflows/kind-addons.yml . . . 556
.github/workflows/orchestrator.yml . . . 814
.github/workflows/verify.yml . . . 748

A

Agriculture & Irrigation . . . 179, 1012
AI Inference Serving . . . 173, 926
AI Training . . . 174, 196, 930
Amendment . . . 75, 240, 486, 488, 489, 490, 491, 492, 493, 494, 495, 496
Authority . . . 143, 736
Automotive EV & Mobile Powertrain . . . 178, 977
Aviation & Autonomous Flight . . . 177, 979

B

Band . . . 7, 258, 260, 290, 506, 515, 516, 530
benchmarks/core_evidence.py . . . 55
benchmarks/stack_benchmark.py . . . 200
Beyond the noise . . . 311, 314, 479, 497
Blockchain & Ledger Nodes . . . 176, 758, 973
Buffer pool . . . 138, 250, 334, 356, 435
Building & Critical Environment HVAC . . . 180, 996

C

Cache & Memory Services . . . 182, 1021

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CI/CD & Build Farms 176, 758, 975
Cloud VM & Capacity 171, 870
Commerce & Payment Systems 183, 1024
Compass . . . . . xiv, 7, 50, 58, 64, 89, 131, 509, 515, 524, 729, 738, 758
Container Resources 170, 875
Content Delivery & Edge Cache 183, 758, 1057
Cooling; Chillers & Thermodynamics 173, 921
Cover 97, 131, 133, 506
cpp/include/omnicompass/core.hpp 116
cpp/include/omnicompass/immune.hpp 823
cpp/src/closure.cpp 270, 749
cpp/src/compass.cpp 270, 749
cpp/src/conveyance.cpp 270, 749
cpp/src/core.cpp 53, 116, 269, 749
cpp/src/governor.cpp 270, 749, 799, 803, 804
cpp/src/gpu_rules.cpp 270, 749
cpp/src/hpa.cpp 270, 749
cpp/src/immune.cpp 823
cpp/src/nervous_system.cpp 270, 749, 823
cpp/src/shield.cpp 270, 746, 749
cpp/tools/engine_runs.cpp . . . . . xiii, 216
Cross-Cluster; Multi-Region & Edge 174, 934
Cruise 6, 90, 245, 533, 535, 810
Cushion 134

D

Data Analytics & ETL 182, 1027
Database & Transactions 181, 1030
deploy/controller.yaml 805, 812

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

deploy/Dockerfile 259
deploy/grafana/omni-compass.json 20, 812
deploy/install/omni-compass-cloud.yaml 764, 767
deploy/install/omni-compass.yaml 259, 805, 812
deploy/kind/batch-jobs.yaml 533
deploy/kind/demo.yaml 292, 523, 524, 559
deploy/kind/loadgen-meter.yaml 554
deploy/kind/loadgen.yaml 768
deploy/kind/noisy.yaml 520
deploy/kind/prepull.yaml 549, 559
deploy/kind/rbac-omni-noisy.yaml 520
deploy/kind/rbac-omni.yaml 95, 100, 115, 219, 226, 263, 286, 293, 746, 765
deploy/prometheus/omni-compass-rules.yaml 20, 805, 812
deploy/rbac-*.yaml 259
deploy/rbac-target.yaml 226, 262, 293
Distributed Cluster Managers 174, 940
District Heating & Cooling 180, 1016
docs/*_PREREGISTRATION.md . . . . . xvii, 311, 361, 364, 852, 1813
docs/AMAZON_SETUP.md 714
docs/AUDIT_2026-10-27.md 715
docs/AUDIT_2026-10-30.md 715
docs/AZURE_SETUP.md 234, 266
docs/BATCH_QUEUE_PREREGISTRATION.md 12, 670, 671, 793
docs/BENEFIT_SHEET.md . . . . . xlv, 168, 310, 318, 367, 715, 719, 851
docs/book/THEORY.md 68
docs/CANONICAL_ENGINE.md 63, 855
docs/CAPACITY_PREREGISTRATION.md 554, 673
docs/CHANGE_CONTROL.md 18, 19, 564, 679, 792, 795, 798, 799, 801, 802, 807, 812
docs/CITYLEARN_PREREGISTRATION.md 241, 454, 672, 774, 836, 861

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

docs/CLAIMS_REGISTER.md 475, 681, 753, 861
docs/COMPARISON.md 20, 259, 829
docs/CONVEYANCE_LAW.md 707, 829, 831, 838
docs/CPU_POWER_PREREGISTRATION.md 215, 254, 267, 473
docs/CRANE_PREREGISTRATION.md 670
docs/CRUISEKUBE_PREREGISTRATION.md 29, 672, 673
docs/DEPENDENCY_LICENSES.md 19
docs/DEVIATIONS.md 18, 676
docs/DOMAIN_MAP.md 202, 265
docs/DOSSIER.md . . . . . xl, xli, 162, 318, 748, 852, 861, 1812
docs/engine/check_live_engine.py . . . . . xliii, 734
docs/engine/export_cpp_twins.py 216
docs/engine/export_harness_trace.py 162
docs/engine/export_whole.py . . . . . xli, xlii, 395
docs/engine/monitors.json xli
docs/engine/OMNI_COMPASS_LIVE_ENGINE.html . . . . . xli, xliii, 52, 564, 797
docs/engine/PAGE_WORD_FOR_WORD.md xli
docs/ENGINE_COVERAGE.md 18, 123
docs/ENGINE_PROOFS.md 13, 16, 18, 21, 62, 63, 123, 128
docs/EVIDENCE_LEDGER.md . . . . . xxxvii, xli, 60, 72, 752, 861
docs/FACTORY_PROFILE.md . . . . . xxxix, 311, 312, 475, 732, 841
docs/FORMAL_STATUS.md 13, 56, 58, 73, 97, 681, 862
docs/FORMAL_V5.md . . . . . xxvii, 10, 23, 152, 305, 759, 760
docs/GOLDILOCKS_PREREGISTRATION.md 29
docs/GOOGLE_SETUP.md 714
docs/GPU_BENCH.md 829
docs/GPU_PREREGISTRATION.md . . . . . xxxvii, xl, xli, 30, 117, 132, 168, 276, 278, 314, 359, 413
docs/GPU_RUN_GUIDE.md 30, 264, 276, 305, 413
docs/GROUP_PREREGISTRATION.md 670

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

docs/HANDOFF.md . . . 861
docs/handoff/mathematics/omni_compass_engine_source_c527df2d.py . . . 117
docs/history . . . 162, 240, 296, 370, 404, 413, 714
docs/HISTORY.md . . . 117, 686, 687, 709, 852
docs/history/AMENDMENT_RISING_STEP_CAP.md . . . 523
docs/history/BENCHMARK_REPORT.md . . . 687, 839
docs/history/CONSENSUS_AUDIT.md . . . 833
docs/history/OMNICOMPASS_BUYER_EDITION.md . . . 833
docs/history/SCENARIOS_30_2026-10-24.md . . . 404
docs/history/STATE_OF_PLAY_2026-10-10.md . . . 716, 833
docs/history/STATE_OF_PLAY_2026-10-27.md . . . 713, 716
docs/history/V3_KAFKA_set1.md . . . 354
docs/history/V3_PGBENCH_set1.md . . . 353
docs/history/V3_REDIS_set1.md . . . 355
docs/history/V3_SYSBENCH_set1.md . . . 252, 357
docs/history/V3_YCSB_set1.md . . . 356
docs/HOW_TO_READ_THE_RESULTS.md . . . 10, 29, 296, 319, 320, 336, 414, 477, 497, 708
docs/INTEGRATION_MANUAL.md . . . 217, 287, 480, 704, 829
docs/K8S_COMPASS_PREREGISTRATION.md . . . xxxvii, xli, 94, 104, 138, 216, 234, 358, 392, 471, 556, 668
docs/KAFKA_PREREGISTRATION.md . . . 243, 266, 354, 426, 427, 669, 673, 674, 827, 861
docs/LAMBDA_SETUP.md . . . 272
docs/LEVELS.md . . . 715
docs/LIMITATIONS.md . . . 11, 12, 18, 22, 398, 401
docs/LOSS_LEDGER.md . . . 714, 719, 824
docs/MECHANISM_OF_ACTION.md . . . 5, 48, 242, 509, 535, 536
docs/METAL_KIT.md . . . 26
docs/METRICS_CATALOG.md . . . 259, 268, 829, 856
docs/MUSCLE_CATALOG.md . . . xli, 79, 164, 166, 168, 169, 760
docs/MYSQL_PREREGISTRATION.md . . . 138, 250, 267, 357, 438, 669, 672, 673, 674, 827, 861

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

docs/OBSERVABILITY.md 20, 792
docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.md . . . . . xli, 715, 812
docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.pdf 715
docs/OMNI_COMPASS_MANUAL.md . . . . . xli, 220, 714, 748
docs/OMNI_COMPASS_MANUAL.pdf . . . . . xli, 714, 748
docs/OMNI_COMPASS_OWNERS_MANUAL.md 319, 715
docs/OMNI_COMPASS_OWNERS_MANUAL.pdf 715
docs/OMNI_ORCHESTRATOR.md 114, 389, 814, 815, 819
docs/OMNI_V1.md 220, 269, 353, 370, 509, 729, 833, 834, 861
docs/OMNI_V10.md 373, 402, 764, 772, 774, 787
docs/OMNI_V11.md 373, 548, 560, 781, 790
docs/OMNI_V12.md 11, 123, 124, 297, 373, 401, 678, 787, 792
docs/OMNI_V13.md 12, 124, 373, 790, 795
docs/OMNI_V14.md 124, 374, 792, 798
docs/OMNI_V15.md 124, 374, 795, 803
docs/OMNI_V16.md 123, 236, 375, 552, 798, 807
docs/OMNI_V17.md 122, 123, 236, 374, 552, 802, 813
docs/OMNI_V18.md 122, 807, 816
docs/OMNI_V19.md 122, 813, 820
docs/OMNI_V2.md 370, 510
docs/OMNI_V20.md 122
docs/OMNI_V21.md 122, 713, 719, 820
docs/OMNI_V22.md 122, 713, 715, 719, 822
docs/OMNI_V23.md 122, 123, 712, 824
docs/OMNI_V3.md 220, 359, 370
docs/OMNI_V4_PLAN.md 140, 231, 411, 472, 729
docs/OMNI_V5.md 152, 371, 473, 541
docs/omni_v5/NEXT_VERSION_PLAN.md 160
docs/OMNI_V6.md 122, 371

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

docs/OMNI_V7.md 3, 236, 372, 762, 772, 774, 781, 850
docs/OMNI_V8.md 372, 402, 762, 764, 775
docs/OMNI_V9.md 372, 764, 772, 782, 785
docs/OPERATOR_MANUAL.md 217, 220
docs/ORCHESTRATORS_PREREGISTRATION.md 669
docs/PACE_PREREGISTRATION.md 121, 237, 375, 553, 668, 669, 670, 802, 807, 813
docs/PANDAPOWER_PREREGISTRATION.md 241, 449, 861
docs/PILOT_KIT.md 475
docs/PILOT_PROTOCOL.md 17, 305, 477, 676, 704
docs/POD_START_WAIT.md . . . . . xiii, 11, 96, 126, 160, 383, 549, 561, 789
docs/POSTGRES_PREREGISTRATION.md . 237, 238, 240, 266, 353, 423, 669, 673, 674, 822, 827, 861
docs/POWER.md 16, 18, 562, 678
docs/PROOF_PROGRAM.md 162, 379, 850, 861
docs/REALMS.md 867
docs/REALMS_PREREGISTRATION.md . . . . . xxxvii, xli, 168, 414, 672, 674, 686, 748, 861
docs/REALM_MUSCLES.md 155, 164, 166, 168, 752, 839
docs/realm_study/TRUE_MUSCLES.csv 510
docs/REDIS_PREREGISTRATION.md 243, 266, 355, 429, 430, 432, 669, 672, 673, 674, 827, 861
docs/REFEREE_SUMMARY.md 22
docs/REGISTER.md 150, 155, 161, 258, 362, 379, 413, 715, 850, 861
docs/RELATED_WORK.md 16, 18
docs/REPORT_CARD.md 715
docs/RERUN_2026-10-10.md 231, 354, 410, 472
docs/ROBOTICS_PREREGISTRATION.md 241, 512, 861
docs/ROBUSTNESS_PREREGISTRATION.md 92, 399, 413, 539, 562, 800, 801, 861, 862
docs/RUN_LEDGER.md 336, 713, 715, 729
docs/SAFETY_CASE.md 10, 21, 97
docs/SECURITY_SCAN.md 18, 796, 797
docs/SOLO_PREREGISTRATION.md 2, 115, 475, 668, 816

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

docs/STATE_OF_PLAY.md . . . . . xl, xli, 166, 198, 305, 318, 781, 829, 851, 861
docs/STORE_LOSS_ROOT_CAUSES_2026-10-25.md 820
docs/STORE_ROOT_CAUSES_2026-10-30.md 824
docs/SWARM_PREREGISTRATION.md 245, 266, 671, 674, 678, 861
docs/TAIL_LATENCY.md 11, 18, 124, 297, 300, 398, 399, 677
docs/THE_CHIPS.md 714, 715
docs/THE_RECEIPTS.md 30, 76, 379
docs/THE_SEAT.md 10, 76, 152, 704, 758
docs/THE_SPINE.md 166, 168, 169, 197, 372, 513, 547, 763, 771, 774, 867, 870
docs/THE_TUNE.md . . . . . xxxviii, 50, 95, 104, 127, 554
docs/THREAT_MODEL.md 19, 96, 97, 260, 747
docs/TRACES_PREREGISTRATION.md 382, 412, 740, 863
docs/TRACKING_THEOREM.md 33, 46, 56, 58, 67, 72, 73, 97, 168, 351, 680, 750, 755, 855
docs/TWO_WAY_NERVOUS_SYSTEM.md 832
docs/WHOLE_SYSTEM_PREREGISTRATION.md . . . . . xli, xlii, 395
docs/WIRING_GUIDE.md 471
docs/wiring_recipes.csv . . . . . xxii, 164, 165, 167, 214, 870
docs/WIRING_VERDICTS.md 3, 217, 231, 258, 318, 320, 409, 528, 715, 851, 861
docs/YCSB_PREREGISTRATION.md 137, 248, 267, 356, 668, 670, 672, 673, 674, 827, 861
DPU SmartNIC & Programmable IO 176, 938
Dwell 135

E

Elevators & Vertical Transport 178, 995
Energy Storage & Microgrid 178, 998

F

Facility & Grid Optimization 179, 1001
Factory profile . . . . . xxviii, xxxix, 52, 312, 732, 733
Fail up 132, 141, 146

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Financial Markets & Trading Systems 184, 758, 1058
fleet/capture/kube_capture.sh 476, 531, 548, 550, 551, 747, 819
fleet/harness.py 270, 749
Full-cache gate 137

G

GAIN . . . . . xii, xv, xix, xxii, xxiii, xxiv, xxvii, 13, 71, 119, 276, 306
Game Servers & Real-Time Interactive 176, 758, 970
Governor . . . . . xiv, 23, 74, 82, 84, 98, 252, 279, 281, 316, 335, 358, 399
GPU Fabric & RDMA 173, 943
GPU Hardware 171, 758, 897
Grid Transmission & Distribution 181, 1003
Gut: Data-plane Intake & Waste 175, 966

H

Handed back 75, 77, 297, 820
hardware/node_exchange.py 71, 228, 265
hardware/plant.py 201
hardware/site_exchange.py 70, 228, 265
Healthcare Critical Environments 179, 1010
Host CPU & Memory 171, 881
HPC & Distributed Compute 174, 947

I

Idle at once 6
Industrial PLC & Process Automation 178, 1005

K

k8s_controlplane/mechanism_harness.py 106
Kill switch 6, 85, 281, 815
Knee 411

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Kubernetes Dynamic Device Allocation 176, 950
Kubernetes Placement & Scheduling 170, 886
Kubernetes Workload Scaling 170, 891

L

Lease 75, 757
Linux Kernel & OS Control 175, 758, 959
LOSS . . . . . xii, xv, xix, xxii, xxiii, xxiv, xxvii, 76, 276, 306, 307, 308

M

Marine Propulsion & Vessel Automation 178, 994
Media Streaming & CDN 175, 758, 968
Medical Imaging & Clinical Systems 183, 1053
Messaging & Streaming 182, 1034
Mining & Mineral Processing 179, 1014
Muscle . . . . . xxv, xxxvii, 2, 8, 23, 78, 83, 84, 89, 119, 138, 152, 169

N

Native . . . . . xvii, xxv, xxviii, 2, 30, 52, 71, 73, 75, 77, 85, 165, 240, 310
Network Routing & Switching 172, 907
Node Fleet & Karpenter-Class Control 171, 902

O

Observability & Telemetry 172, 911
Off the clock 160, 316, 364
Oil & Gas Pipelines 179, 1013
Omni index 379, 527, 556, 834
Omni-Compass direct . . . . . xxv, 310, 389
Omni-Compass on native 310
omnicompass/adapter.Governor 800
omnicompass/adapter.py 48, 49, 129, 270, 480, 698, 749, 750, 799, 803, 806, 854

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

omnicompass/authority_contract.py 129
omnicompass/ce_score.py 130
omnicompass/closure.py 128, 130, 270, 749, 833
omnicompass/compass.py 129, 203, 270, 288, 749
omnicompass/compass_law.py 7, 64, 86, 129, 131, 204, 351, 370, 490, 749
omnicompass/configurations.py 53, 55, 129
omnicompass/conveyance.py 129, 270, 291, 749
omnicompass/core.py 45, 53, 55, 60, 65, 116, 118, 130, 216, 269, 351, 749, 750
omnicompass/disruption_budget.py 129
omnicompass/edge.py 129, 803, 804
omnicompass/immune.py 130, 823
omnicompass/master.py 6, 21, 86, 92, 93, 99, 106, 129, 260, 476
omnicompass/mathdrive.py 128, 130
omnicompass/muscles.py 6
omnicompass/muscle_sdk.py 93, 94, 96, 101, 129, 210
omnicompass/nervous.py 149
omnicompass/nervous_system.py 130, 141, 143, 203, 270, 503, 749, 751, 823, 833
omnicompass/pools.py 216
omnicompass/shield.py 95, 128, 130, 270, 746, 749, 833
omnicompass/speed.py 83, 128, 130
omnicompass/stack_sim.py 128, 130
omnicompass/storage.py 118, 129, 270
omnicompass/student_t.py 129, 311, 372, 497, 564, 775, 777
omnicompass/verdict.py . . . . . xiv, xviii, xxvii, 3, 22, 65, 87, 88, 101, 124, 130, 228, 314
omni_controller/* 68
omni_controller/capacity.py 94, 129, 558, 766, 803, 804, 815
omni_controller/controller.py 1, 50, 83, 86, 93, 94, 106, 107, 114, 129, 204, 259, 270, 475
omni_controller/gpu_compass.py 24, 106, 129, 205, 358, 473, 490, 492, 493, 494, 506, 515
omni_controller/gpu_governor.py 106, 129, 259, 270, 475, 749, 798, 799

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

omni_controller/muscles.py 72, 83, 110, 129, 219, 259, 502, 503, 764, 790, 792, 793
omni_controller/quota.py 94, 129, 764
One pedal . . . . . xv, xxix, 85, 757
OpenShift & Machine API 173, 952
Organism . . . . . xxv, xxxvii, xlii, 156, 162, 163, 164, 170, 173, 176, 178
Own cost 316, 399, 837

P

PDU; UPS & Electrical Distribution 172, 923
Pharmaceutical & Food Manufacturing 180, 1020
pilot/bench_report.py 518
pilot/selfpilot.py 768
Plug 50, 65, 96, 204, 206, 489, 869
Ports & Maritime Logistics 183, 1055
Power Generation & Turbine Control 180, 1017
Power given . . . . . xv, xix, xxiii, xxiv, xxvi, xxxviii, 76, 307, 308, 311, 312

Q

Quantum Computing Control Simulation 174, 954

R

Race 492
Rail Traction & Train Control 177, 992
realms/catalog.csv 150, 167, 359, 499, 758
realms/catalog_provenance.json 510
realms/catalog_v1.csv 167, 510
realms/catalog_v5.csv . . . . . xxv, 29, 150, 163, 166, 167, 169, 197, 758, 865
realms/compass_arm.py 129, 505, 506, 512, 515
realms/compass_arm_v5.py 73, 129, 168, 758
realms/gpu_card.py 130, 358, 492, 493, 494

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

realms/harness.py 2, 73, 129, 168, 404, 501, 505, 508, 509, 757, 758, 775, 816
realms/muscle_cards_v5.csv . . . . . xxxviii, 79, 152, 167, 732, 760, 866
realms/plants.py 73, 129, 167, 499, 508, 816, 817
realms/presets.py 130, 167, 499, 510, 756
realms/wave4_families.csv 510
Receipt 30, 210, 296, 298, 319, 336, 362, 365, 389, 413, 479, 545
Reliability; Security & Recovery 172, 183, 913, 1061
Renewable Generation & Inverter Control 180, 1018
results/BENEFIT_SHEET.json 851
results/cards_v5/CARDS.md 152
results/core_evidence.json 63, 681
results/ENGINE_COVERAGE.json 128
results/ENGINE_PROOFS.json 63
results/external_review/CHATGPT2_REVIEW.md 831
results/gpu/run-*/GPU_REPS.md 203
results/gpu/run-20261002T082232Z/GPU_REPS.md 683, 685, 752, 837
results/gpu/run-20261024T091235Z/GPU_REPS.md 683
results/gpu/sim/after 202, 707, 838
results/gpu/sim/before 275
results/gpu/sim/FINDINGS.md 681, 687
results/gpu/V21_A100_CAP_ABC.md . . . . . xliii, 395
results/gpu/V21_A100_SAVE_ABC.md 713
results/gpu/V21_STACKS_A10_ABC.md 498, 714
results/gpu/V23_A10_CAP_ABC.md 336
results/gpu/V23_A10_CAP_FULL_ABC.md . . . . . xliii, 336, 395
results/gpu/V23_A10_FAST_ABC.md 336
results/grid_v5/GRID.md 160, 167, 169, 198, 317, 850
results/grid_v6 763
results/hardware/NODE_EXCHANGE_*.json 71, 203, 269, 682, 707, 829, 838

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

results/hardware/SITE_EXCHANGE_*.json . . . 203
results/hardware/SITE_EXCHANGE_HELDOUT_*.json . . . 70, 831, 838
results/hardware/SUMMARY_HELDOUT.json . . . 831
results/heldout_seed_* . . . 691, 692, 693, 694, 695, 696, 697
results/hil/run-20261002T082232Z/HIL.md . . . 838
results/live/AKS_*.md . . . xlv, 310
results/live/AKS_BILL.md . . . 13, 95, 103, 312, 350, 525, 534, 733, 834, 835
results/live/AKS_STEADY_100_OMNI_V18.md . . . xliii, 395
results/live/ALL_FOUR.md . . . 529, 835
results/live/AMENDMENT_2_RUNS.md . . . 523
results/live/AMENDMENT_3_RUNS.md . . . 524, 533, 835
results/live/BATCH.md . . . 536
results/live/CAPACITY.md . . . 520
results/live/CAPACITY_2.md . . . 521
results/live/FAIRNESS.md . . . 520, 533, 835
results/live/FAIRNESS_2.md . . . 521
results/live/FAULTS.md . . . 518, 533, 835
results/live/FAULTS_30.md . . . 518
results/live/FAULTS_31.md . . . 518
results/live/FAULTS_32.md . . . 521
results/live/LIVE_LEVERS_2_NERVOUS.txt . . . 219, 831
results/live/LIVE_PAIRED.md . . . 220, 829
results/live/LIVE_REPS_20.md . . . 686
results/live/LIVE_REPS_21.md . . . 684, 686, 687, 829
results/live/LIVE_REPS_22.md . . . 684, 686, 835, 836
results/live/LIVE_REPS_23.md . . . 684, 753, 836
results/live/LIVE_REPS_24.md . . . 683, 836
results/live/LIVE_REPS_25.md . . . 683, 837
results/live/LIVE_REPS_26.md . . . 515, 683, 837

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

results/live/LIVE_REPS_27.md . . . 516, 683, 835, 837
results/live/LIVE_REPS_28.md . . . 516
results/live/LIVE_REPS_29.md . . . 517
results/live/LIVE_REPS_30.md . . . 518
results/live/LIVE_REPS_31.md . . . 518
results/live/LIVE_REPS_4_DIAGNOSIS.md . . . 832
results/live/LIVE_REPS_PROBE_DEFECT.md . . . 686, 832
results/live/LIVE_SHADOW_1.txt . . . 219, 831
results/live/POD_START_PHASES.md . . . 549, 557
results/live/raw/addons-run-38725705826 . . . 765
results/live/REPLICA_ROOM.md . . . 525, 761
results/live/SIX_KUBE.md . . . 350, 526, 530, 835
results/live/V*_*.json . . . 564
results/live/V10_ALL_FOUR.md . . . 302, 405
results/live/V10_BATCH.md . . . 303, 407
results/live/V10_CRUISEKUBE.md . . . 301
results/live/V10_FAIRNESS.md . . . 303, 407
results/live/V10_FAULTS.md . . . 302, 407
results/live/V10_STEADY.md . . . 301, 404
results/live/V10_TRACE_GOOGLE2011.md . . . 302, 407
results/live/V10_WANDERING.md . . . 301, 404
results/live/V11_*.md . . . 123, 366, 787
results/live/V11_CAPACITY.md . . . 300
results/live/V11_CITYLEARN.md . . . 304, 409
results/live/V11_GOLDILOCKS.md . . . 299, 407
results/live/V11_KAFKA.md . . . 300, 408
results/live/V11_KEDA_CPU_REQUESTS.md . . . 299
results/live/V11_KEDA_REQUESTS.md . . . 299, 561
results/live/V11_MUJOCO.md . . . 304, 408

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

results/live/V11_PANDAPOWER.md . . . 303, 408
results/live/V11_PGBENCH*.md . . . 300, 408
results/live/V11_REDIS.md . . . 300, 408
results/live/V11_ROBUST_KILL.md . . . 298, 316, 407, 862
results/live/V11_ROBUST_LONG.md . . . 298, 407, 862
results/live/V11_SWARM.md . . . 304, 409
results/live/V12_*.md . . . 123
results/live/V12_KUBERAY_4W_BARE.md . . . 793
results/live/V1_*.md . . . 220, 353, 833, 850
results/live/V1_AKS_*.md . . . 360
results/live/V1_AKS_BURST.md . . . 358
results/live/V1_AKS_STEADY.md . . . 234, 269, 303, 335, 358, 471
results/live/V1_ALL_FOUR.md . . . 268
results/live/V1_BIG_ORGANISM.md . . . 160
results/live/V1_SIX_KUBE.md . . . 360
results/live/V21_ALL_FOUR.md . . . 325, 339, 417, 723
results/live/V21_BATCH.md . . . 326, 339, 419, 724
results/live/V21_CAPACITY.md . . . 339
results/live/V21_CITYLEARN.md . . . 331, 349, 728
results/live/V21_CRUISEKUBE.md . . . 326, 341, 724
results/live/V21_FAIRNESS.md . . . 327, 339, 418, 725
results/live/V21_FAULTS.md . . . 324, 339, 419, 722
results/live/V21_GOLDILOCKS.md . . . 325, 341, 723
results/live/V21_KAFKA.md . . . 330, 342, 425, 426, 427, 728
results/live/V21_KAFKA_SERVICE.md . . . 342
results/live/V21_KEDA_CPU_REQUESTS.md . . . 328, 341, 421, 726
results/live/V21_KEDA_REQUESTS.md . . . 327, 341, 421, 725
results/live/V21_MUJOCO.md . . . 322, 349, 721
results/live/V21_PANDAPOWER.md . . . 328, 349, 726

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

results/live/V21_PGBENCH.md 329, 342, 422, 423, 424, 726
results/live/V21_REDIS.md 322, 342, 428, 429, 430, 431, 721
results/live/V21_REDIS_SERVICE.md 342
results/live/V21_ROBUST_KILL.md 322, 339, 720
results/live/V21_ROBUST_LONG.md 321, 339, 720
results/live/V21_STEADY.md 323, 416, 722
results/live/V21_SWARM.md 331, 349, 728
results/live/V21_SYSBENCH.md 329, 342, 436, 437, 438, 727
results/live/V21_TRACE_GOOGLE2011.md 326, 339, 420, 724
results/live/V21_WANDERING.md 324, 417, 723
results/live/V21_YCSB.md 330, 342, 433, 434, 435, 727
results/live/V3_*.md 220, 360
results/live/V3_ALL_FOUR.md 352
results/live/V3_BATCH.md 352
results/live/V3_BIG_ORGANISM.md 160, 321, 350, 470, 531, 720
results/live/V3_CITYLEARN.md 359
results/live/V3_FAIRNESS.md 352
results/live/V3_FAULTS.md 352
results/live/V3_KAFKA.md 354, 360
results/live/V3_MUJOCO.md 359
results/live/V3_OWN_COST.md 335, 862
results/live/V3_PANDAPOWER.md 359, 361
results/live/V3_PGBENCH.md 269, 353, 360
results/live/V3_REDIS.md 355, 369
results/live/V3_ROBUST_LONG.md 146
results/live/V3_SIX_KUBE.md 320, 350, 465, 466, 467, 468, 469, 719
results/live/V3_STEADY.md 352
results/live/V3_SWARM.md 361
results/live/V3_SYSBENCH.md 252, 357

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

results/live/V3_TRACE_GOOGLE2011.md 352, 863
results/live/V3_WANDERING.md 352
results/live/V3_YCSB.md 356
results/live/V5_CAPACITY.md 371
results/live/V5_GOLDILOCKS.md 29
results/live/V6_*.md 371, 762
results/live/V6_KEDA_REQUESTS.md 765
results/live/V7_*.md 372, 764, 769
results/live/V7_CITYLEARN.md 449, 450, 451, 452, 453, 772
results/live/V7_FAIRNESS.md 93
results/live/V7_FAULTS.md 93
results/live/V7_MUJOCO.md 441, 442
results/live/V7_PANDAPOWER.md 443, 444, 445, 446, 447, 448
results/live/V7_SWARM.md 454, 455
results/LOCK_AMENDMENTS.json 56, 806
results/MECHANISM_IDENTITY.json 55, 60, 72, 117, 351, 680, 681
results/MECHANISM_OF_ACTION.json 49, 80
results/model_gate/HEARTBEAT_SWEEP.md 733
results/model_gate/SOLO_V19.md 816, 818
results/model_gate/SOLO_V19_VS_V18.md 816, 818
results/model_gate/STORE_GATE.md 820
results/model_gate/V10_VS_V9.md 373, 786
results/model_gate/V11_VS_V10.md 94, 373, 788
results/model_gate/V12_SWARM_REFLEX.md 11, 18, 123, 297, 304, 401, 678, 790, 791
results/model_gate/V12_VS_V11.md 373, 790, 791
results/model_gate/V13_VS_V12.md 126, 374, 792, 794, 797
results/model_gate/V14_VS_V13.md 374, 798
results/model_gate/V15_VS_V14.md 126, 374, 801
results/model_gate/V16_PACE.json 808

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

results/model_gate/V16_PACE.md 125, 375, 803, 806
results/model_gate/V16_VS_V15.md 125, 375, 803, 813
results/model_gate/V17_LADDER.md 121, 237, 374, 733, 807, 808
results/model_gate/V17_VS_V16.md 374, 807
results/model_gate/V18_VS_V17.md 815
results/model_gate/V19_VS_V18.md 816, 817
results/model_gate/V21_VS_V20.md 823
results/model_gate/V22_KAFKA_MODEL.md 828
results/model_gate/V22_REDIS_MODEL.md 828
results/model_gate/V22_STORE_GATE.md 827
results/model_gate/V22_VS_V21.md 828
results/model_gate/V23_LADDER.md 554
results/model_gate/V23_VS_V22.md 115, 713
results/model_gate/V8_VS_V7.md 126, 372, 774
results/model_gate/V9_VS_V8.md 372, 781
results/OMNI_INDEX.md . . . . . xiii, xliii, xlv, 310, 317, 320, 336, 361, 379, 395, 416, 529
results/orchestrator/V18_THREE_ARMS_ABC.md . . . . . xi, 317, 389, 815
results/orchestrator/V21_THREE_ARMS_ABC.md 714
results/PREREGISTRATION.json 53
results/realms/REALMS.md 59, 321, 359, 361, 512, 682, 686, 720, 752, 839
results/realms/RELEASE_MARGIN_SWEEP.md 91, 496, 509
results/realms/stack/STACK.md 682
results/realms_v5 410
results/realms_v5/MUSCLES_V5.csv 339, 415, 456, 770
results/realms_v5/REALMS_V5.md 13, 76, 156, 165, 167, 169, 305, 339, 400, 408, 456, 677
results/REDIS_BRAIN_PROBE.json 563
results/scale/GRID.md 359, 361, 506, 507, 682, 685, 838
results/scale/v1/receipts/round6-1000x.md 508
results/scenarios/SCENARIOS_20.json . . . . . xliii, 402

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

results/scenarios/SCENARIOS_20.md . . . 162, 317, 402
results/SEAL.json . . . xli, 53, 202, 259, 270, 351, 681, 734, 747, 748, 749, 799
results/sim/gpu_two_wire/RESULT.md . . . 359, 682, 838
results/SOAK.json . . . 58, 749, 831, 862
results/STEERING.md . . . 20, 790, 800, 802
results/traces/google2011/schedule.json . . . 543, 863
results/traces/google2011_1to9/schedule.json . . . 543
results/TRACKING_BOUNDS.json . . . 60, 680, 681
results/TWO_BILLS.md . . . xliii, xliv, xlv, 309, 310, 395, 498
results/VERIFY_RECEIPT.txt . . . 350, 361
results/WHOLE_SYSTEM.md . . . xli, xlii, xliii, xlv, 310, 312, 317, 395, 713
results/WIRING_VERDICTS.csv . . . 318, 409, 414, 456, 851
Robotics Fleet & Warehouse Automation . . . 177, 983
Robotics Motion Control . . . 176, 986
Runtime & Application . . . 182, 1038

S

scripts/aks_kubectl.py . . . 107
scripts/aks_paired.sh . . . 360, 519, 537
scripts/amazon_queue.txt . . . 543, 553
scripts/azure_queue.txt . . . 543, 545, 554, 555, 714, 774
scripts/benchmark_campaign.json . . . 763
scripts/dispatch_grid.py . . . 106, 107, 116, 158, 165, 166, 409, 539, 541, 549, 551, 553
scripts/google_queue.txt . . . 543, 553
scripts/gpu_8card.sh . . . 276, 495
scripts/gpu_campaign_profiles.json . . . 763
scripts/gpu_fault_drill.sh . . . 494
scripts/gpu_paired.sh . . . 489, 498, 684, 829
scripts/gpu_rented_run.sh . . . 206, 276, 361, 362, 490, 493, 494, 495, 505, 753

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

scripts/gpu_vllm.sh 494, 495
scripts/kind_addons.sh 548
scripts/kind_addons_cruisekube.sh 29
scripts/kind_addons_goldilocks.sh 29
scripts/kind_bench.sh 93, 284, 286, 294, 515, 517, 519, 531, 540, 541, 542, 547
scripts/kind_faults.sh 517, 549
scripts/kind_install.sh 540
scripts/kind_levers.sh 805
scripts/kind_nodepool.sh 87, 95, 103, 119, 219, 529, 547, 554
scripts/kind_paired.sh 242, 268, 360, 362, 514, 531, 541, 846, 1813
scripts/kind_power.sh 551
scripts/latency_probe.py 108, 220, 221, 261, 292
scripts/organism_detached.sh 815
scripts/pilot_shadow.sh 218, 225, 262, 293, 475
scripts/raw_archive.sh . . . . . xviii, 104, 361, 850
scripts/reruns.txt . . . . . xxxiii, 116, 120, 167, 371, 373, 409, 550, 552, 710, 713
scripts/restart_aks.py 108, 110
Search; Indexing & Vector DB 182, 1042
Semiconductor Fab & Precision Manufacturing 181, 1007
Serverless & Functions (FaaS) 175, 758, 962
Service Mesh & API Reliability 181, 1045
Skin: Perimeter & Admission 175, 967
Slack gate 137, 512
Snapshot 146, 479
Spacecraft & Flight Software 177, 991
Speed floor 492
STEADY . . . . . xi, xv, xix, xxii, xxiii, xxiv, 52, 306, 307, 308, 314, 315
Storage Block/File/Object 172, 916

T

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Telecom RAN & Edge Radio 184, 1048
tests/fixtures/robust_long_39173420146_decisions.json 800
tests/standin_cluster/kubectl 216
tests/test_* 206
tests/test_active_nodes.py 119
tests/test_api_proxy.py 517
tests/test_blind_reading.py 94, 101, 373, 788
tests/test_capacity.py 93
tests/test_capture_in_service.py 551
tests/test_cap_band.py 374, 800
tests/test_cloud_cadence.py 771
tests/test_cloud_check.py 102, 103
tests/test_compass.py 118
tests/test_compass_arm.py 89
tests/test_compass_arm_v5.py 77, 762
tests/test_compass_controller.py 76, 90, 518, 521, 522, 524, 532, 535
tests/test_confirm_abc.py 95, 376
tests/test_convey.py 118, 119, 523, 532
tests/test_conveyance.py 70, 707
tests/test_convey_handback.py 792
tests/test_convey_shared_host.py 771
tests/test_core_parity.py 118
tests/test_cpp_*_parity.py 475
tests/test_cpp_closure_parity.py 270, 749
tests/test_cpp_conveyance_parity.py 270, 749
tests/test_cpp_governor_parity.py 270, 749, 799, 806
tests/test_cpp_hpa_parity.py 119, 270, 749
tests/test_cpp_immune_parity.py 823
tests/test_cpp_shield_adversarial.py 95

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

tests/test_cpp_shield_parity.py 95, 270, 749
tests/test_cpp_twins_parity.py 270, 749
tests/test_cruise_idle.py 533, 534
tests/test_engine_configurations.py 55
tests/test_engine_proofs.py 63
tests/test_engine_properties.py 18, 64, 128
tests/test_failsafe.py 118
tests/test_gpu_bench.py 498
tests/test_gpu_compass.py 497
tests/test_immune_system.py 823
tests/test_kind_capacity.py 93
tests/test_kind_rig_rbac.py 115
tests/test_knob_verdict.py 94, 101
tests/test_living_band.py 290
tests/test_master_switch.py 92, 93, 99
tests/test_muscles.py 76, 118
tests/test_muscle_sdk.py 93, 94, 101, 214
tests/test_native_muscles.py 797
tests/test_nervous_system.py 149
tests/test_node_exchange.py 72
tests/test_node_release_gate.py 89, 93, 149
tests/test_omni_control.py 812
tests/test_omni_controller.py 76
tests/test_omni_exporter.py 812
tests/test_omni_version.py 801
tests/test_operator_cmd.py 797
tests/test_own_replicas.py 116, 817
tests/test_pace_clocks.py 805, 810, 812
tests/test_pod_phases.py 95, 557

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

tests/test_quota_convey.py 93, 771
tests/test_raw_archive.py 104
tests/test_refused_raise.py 791
tests/test_release_campaign.py 95
tests/test_restore_retry.py 826
tests/test_robust.py 92, 800
tests/test_robust_restart.py 92
tests/test_shield_properties.py 57, 95, 351, 831, 862
tests/test_staging_order.py 89
tests/test_store_gate.py 822
tests/test_store_wires.py 822
tests/test_supply_chain.py 102
tests/test_switch_trust.py 96, 98, 99
tests/test_time_correct.py 771
tests/test_tune_study.py 52
tests/test_two_way.py 149
tests/test_verdict.py 94, 771
tests/test_verdict_interval.py 94, 767, 771
tests/test_workflow_permissions.py 103
tests/third_party/hpa_independent.py 739
The best validated factory profile for the declared operating conditions 733
The cloud is a wire . . . . . xiii, xlii, 395
The runs disagree . . . . . xvii, xxiii, xxiv, xxvi, 307, 308, 313, 315, 319, 336, 376
tools/artifact_legal.py 108
tools/beat_ladder.py 808
tools/benefit_sheet.py 320, 367, 719, 851
tools/capacity_report.py 554
tools/citylearn_abc.py 241, 774, 775, 782, 846
tools/cloud_key_check.py 103, 104

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

tools/confirm_abc.py 15, 76, 96, 360, 376, 476, 543, 549, 555, 559, 812, 819, 851
tools/cpu_power_abc.py 254
tools/declare_envelope.py 489
tools/dossier.py 350, 851, 852, 864
tools/engine_runs.py . . . . . xli, 216, 553
tools/function_tests.py 128
tools/governors_page.py 162
tools/gpu_actuator_watchdog.py 106
tools/gpu_baseline.py 480
tools/gpu_reps.py 63, 72, 199, 486, 498, 685, 754
tools/gpu_wire_check.py 361, 491
tools/gpu_workload.py 228, 479, 480, 485
tools/grid.py 361
tools/health_watch.py 108
tools/kafka_abc.py 846
tools/kafka_model.py 827
tools/knob_verdict.py 94, 101, 130, 217, 228, 410, 472, 473, 713, 826, 827, 851
tools/land_engine.py 123, 729
tools/layout_check.py 221, 851
tools/legal.py 286, 851
tools/levels.py 337
tools/live_glossary.py 53
tools/live_reps.py 111, 199, 202, 234, 268, 360, 362, 365, 515, 517, 520, 523
tools/llm_workload.py 108, 494
tools/mechanism_identity.py 55
tools/model_gate.py 115, 119, 554, 560, 730, 768, 774, 781, 786, 788, 828
tools/mujoco_abc.py 241, 846
tools/omni_alert_bridge.py 108
tools/omni_control.py 20, 106, 797, 812

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

tools/omni_exporter.py 20, 791, 812
tools/omni_index.py 361, 379, 528, 556, 821, 851, 863, 864
tools/omni_switch.py 93, 96, 99, 476, 746
tools/omni_version.py 73, 123, 240, 285, 360, 741, 756, 799, 801, 814, 851, 863
tools/orchestrator_show.py 109, 814, 819
tools/own_cost.py 851
tools/pandapower_abc.py 241, 846
tools/pgbench_abc.py 846, 1812
tools/places.py 555
tools/pod_phases.py 557
tools/pod_report.py 522
tools/pool_scale.py 361
tools/prom_feed.py 109
tools/readings.py 76, 296, 312, 376, 477, 499, 766, 775, 782, 825
tools/realms_catalog.py 499, 504, 510, 512
tools/realms_catalog_v2.py 510
tools/realm_muscles.py 169
tools/redis_abc.py 15, 821, 846
tools/redis_brain_probe.py 563, 802
tools/redis_model.py 827
tools/release_manifest.py 361, 750, 851
tools/reproduce.py 18, 123, 676, 713, 719
tools/reread.py 315, 380
tools/run_*.py 128, 713
tools/run_citylearn.py 123, 128, 130, 240, 772, 775, 781, 782
tools/run_cpu_power.py 129, 254
tools/run_gpu_card.py 493, 496
tools/run_hil.py 166, 495, 496, 497, 505, 525
tools/run_kafka.py 107, 128, 130, 243, 360, 820, 827

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

tools/run_kil.py 123, 129, 160, 198, 360, 364, 370, 525, 526, 531, 801
tools/run_mujoco.py 129, 241
tools/run_orchestra.py 162
tools/run_pandapower.py 123, 240, 361
tools/run_pgbench.py 105, 111, 128, 130, 237, 360, 820, 827
tools/run_realms.py 361
tools/run_realms_v5.py 165, 513, 762, 819
tools/run_redis.py 107, 123, 128, 130, 243, 360, 563, 799, 820, 827
tools/run_scale.py 165, 169, 361, 514
tools/run_stack.py 505
tools/run_swarm.py 123, 128, 130, 245, 361, 790
tools/run_sysbench.py 111, 129, 130, 146, 250, 820, 827
tools/run_ycsb.py 109, 129, 130, 248, 820, 827
tools/sbom.py 102
tools/seal.py 216, 734, 823
tools/six_kube_report.py 360, 526, 540
tools/stop_rule.py 550, 553
tools/store_brain.py 129, 713, 719, 826, 827
tools/store_gate.py 820, 827
tools/strip_reference.py 117
tools/swarm_abc.py 846
tools/trace_schedule.py 109
tools/tracking_bounds.py 60
tools/tune_cards.py 78, 152, 760, 866
tools/tune_study.py . . . . . xxxviii, 52
tools/wall_meter.py 109, 110, 200, 275, 796
tools/whole_system.py . . . . . xli, xlii, 162, 395, 713
tools/wiring_verdicts.py 409, 414, 528, 851
tools/ycsb_abc.py 821

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Tuning case 412, 555
tuning/bound.py 832
tuning/B_FIX.json 830
tuning/B_TONE_HELDOUT.json 830
tuning/CONFIRMATORY.json 830
tuning/CONFIRMATORY_COORD.json 830
tuning/PLANETLAB_LEAGUE.json 830
tuning/SITE_LEAGUE.json 830

V

Verdict 2, 50, 65, 137, 228, 311, 312, 314, 409, 410, 472, 487, 516
Virtualization & Hypervisor 175, 758, 964

W

Water Wastewater & Pumping 181, 1008
Wire check . . . . . xxii, 206, 209, 258, 273, 491
Work Admission & Demand Shaping 173, 957
Work per energy 156, 314, 734
Workflow; Logistics & Fulfillment 183, 1050

BACK MATTER

Contact

Licensing, pilots, evaluations and the Omni-Compass Enterprise License:

The Omni-Compass LLC

Owner and developer: AJ Dubra

Main office infrastructure: toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1

Landline, registered in Florida: 321-746-4411

www.omni-compass.com

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Repository: github.com/The-Omni-Compass-LLC/The-Omni-Compass-LLC

Any use beyond evaluation and simulation requires a signed, paid Omni-Compass Enterprise License. Write to The Omni-Compass LLC before any production use.

© 2026 The Omni-Compass LLC. All rights reserved. **Evaluation and Simulation use only.** Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` and `DISCLOSURES.md`. `SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0` All patents, copyrights and trademarks filed in the USA. www.omni-compass.com